Elixir

v1.20.5



xmlns:epub="http://www.idpf.org/2007/ops">

Table of contents

- Changelog for Elixir v1.20

- Getting started

  - Introduction
  - Basic types
  - Lists and tuples
  - Pattern matching
  - case, cond, and if
  - Anonymous functions
  - Binaries, strings, and charlists
  - Keyword lists and maps

- Modules and functions
- alias, require, import, and use
- Module attributes
- Structs
- Recursion
- Enumerables and Streams
- Comprehensions
- Protocols
- Sigils
- try, catch, and rescue
- Processes
- IO and the file system
- Writing documentation
- Optional syntax sheet
- Erlang libraries
- Debugging

- Cheatsheets

- Enum cheatsheet
- Set-theoretic types cheatsheet

- Mix & OTP

  - Introduction to Mix
  - Simple state with agents
  - Registries and supervision trees
  - Supervising dynamic children
  - Task and gen_tcp
  - Doctests, patterns, and with
  - Configuration and distribution
  - Client-server with GenServer
  - Releases

- Anti-patterns
    - What are anti-patterns?
    - Code-related anti-patterns
    - Design-related anti-patterns
    - Process-related anti-patterns
    - Meta-programming anti-patterns
- Meta-programming
    - Quote and unquote
    - Macros
    - Domain-Specific Languages (DSLs)
- References

- Compatibility and deprecations
- Gradual set-theoretic types
- Library guidelines
- Naming conventions
- Operators reference
- Patterns and guards
- Syntax reference
- Software Bill of Materials
- Typespecs reference
- Unicode syntax

- Modules

  - Kernel
  - Kernel.SpecialForms

- Data Types
    - Atom
    - Base
    - Bitwise
    - Date
    - DateTime
    - Duration
    - Exception
    - Float
    - Function
    - Integer
    - JSON
    - Module
    - NaiveDateTime
    - Record
    - Regex
    - String
    - Time
    - Tuple

- URI
- Version
- Version.Requirement

- Collections & Enumerables
  - Access
  - Date.Range
  - Enum
  - Keyword
  - List
  - Map
  - MapSet
  - Range
  - Stream

- IO & System
    - File
    - File.Stat
    - File.Stream
    - IO
    - IO.ANSI
    - IO.Stream
    - OptionParser
    - Path
    - Port
    - StringIO
    - System
- Calendar
    - Calendar
    - Calendar.ISO

- Calendar.TimeZoneDatabase
- Calendar.UTCOnlyTimeZoneDatabase

- Processes & Applications
  - Agent
  - Application
  - Config
  - Config.Provider
  - Config.Reader
  - DynamicSupervisor
  - GenServer
  - Node
  - PartitionSupervisor
  - Process
  - Registry
  - Supervisor
  - Task

- Task.Supervisor

- Protocols
  - Collectable
  - Enumerable
  - Inspect
  - Inspect.Algebra
  - Inspect.Opts
  - JSON.Encoder
  - List.Chars
  - Protocol
  - String.Chars

- Code & Macros

- Code
- Code.Fragment
- Kernel.ParallelCompiler
- Macro
- Macro.Env

- Deprecated

  - Behaviour
  - Dict
  - GenEvent
  - HashDict
  - HashSet
  - Set
  - Supervisor.Spec

- Exceptions

    - ArgumentError
    - ArithmeticError
    - BadArityError
    - BadBooleanError
    - BadFunctionError
    - BadMapError
    - CaseClauseError
    - Code.LoadError
    - CompileError
    - CondClauseError
    - Enum.EmptyError
    - Enum.OutOfBoundsError
    - ErlangError
    - File.CopyError
    - File.Error
    - File.LinkError
    - File.RenameError
    - FunctionClauseError

- IO.StreamError
- Inspect.Error
- JSON.DecodeError
- Kernel.TypespecError
- KeyError
- MatchError
- MismatchedDelimiterError
- MissingApplicationsError
- OptionParser.ParseError
- Protocol.UndefinedError
- Regex.CompileError
- RuntimeError
- SyntaxError
- System.EnvError
- SystemLimitError
- TokenMissingError
- TryClauseError
- URI.Error
- UndefinedFunctionError
- UnicodeConversionError

- Version.InvalidRequirementError
- Version.InvalidVersionError
- WithClauseError

Changelog for Elixir v1.20

This release requires Erlang/OTP 27+ and is compatible with Erlang/OTP 29.

Type system improvements

Elixir's type system now understands all language constructs and can infer types for your function definitions, using typing information from Elixir's standard library and your dependencies, to find verified bugs and dead code.

This has been achieved through a series of improvements, such as type refinement across clauses, occurrence typing, typing of map keys and domains, and more.

Type inference of guards

This release also performs inference of guards! Let's see some examples:

```
def example(x, y) when is_list(x) and is_integer(y)
```

The code above correctly infers `x` is a list and `y` is an integer.

```
def example({:ok, x} = y) when is_binary(x) or is_integer(x)
```

The one above infers x is a binary or an integer, and `y` is a two element tuple with `:ok` as first element and a binary or integer as second.

```
def example(x) when is_map_key(x, :foo)
```

The code above infers `x` is a map which has the `:foo` key, represented as `%{..., foo: dynamic()}`. Remember the leading `...` indicates the map may have other keys.

```
def example(x) when not is_map_key(x, :foo)
```

And the code above infers `x` does not have the `:foo` key (hence `x.foo` will raise a typing violation), which has the type: `%{..., foo: not_set()}`.

You can also have expressions that assert on the size of data structures:

```
def example(x) when tuple_size(x) < 3
```

Elixir will correctly track the tuple has at most two elements, and therefore accessing `elem(x, 3)` will emit a typing violation. In other words, Elixir can look at complex guards, infer types, and use this information to find bugs in our code, without a need to introduce type signatures (yet).

Whole-body type inference

Elixir also performs inference based on the function body itself. Take the following code:

```
def add_foo_and_bar(data) do
  data.foo + data.bar
end
```

Elixir now infers that the function expects a `map` as first argument, and the map must have the keys `.foo` and `.bar` whose values are either `integer()` or `float()`. The return type will be either `integer()` or `float()`.

Here is another example:

```
def sum_to_string(a, b) do
  Integer.to_string(a + b)
end
```

Even though the `+` operator works with both integers and floats, Elixir infers that `a` and `b` must be both integers, as the result of `+` is given to a function that expects an integer. The inferred type information is then used during type checking to find possible typing errors. The typing inferred from your dependencies are also used to help infer more precise types for your own applications.

Typing across clauses

Elixir now infers the type of a given clause based on previous clauses. Let's see an example:

```
case System.get_env("SOME_VAR") do
  nil -> :not_found
  value -> {:ok, String.upcase(value)}
end
```

`System.get_env("SOME_VAR")` returns either `nil` or a `binary()`. Because the first clause matches on `nil`, the type system knows `value` can no longer be `nil`, and therefore it must only be a `binary()`, which allows the second clause to also type check without violations.

This type inference across clauses also helps the type system find redundant clauses and dead code in existing codebases. Elixir v1.20 also implements occurrence typing for `cond`, `case`, and `with`, providing more precise types within each clause.

Typing of atom and domain keys in maps

Maps were one of the first data-structures we implemented within the Elixir type system however, up to this point, they only supported atom keys. If they had additional keys, those keys were simply marked as `dynamic()`.

As of Elixir v1.20, we can track all possible domains as map keys. For example, the map:

```
%{123 => "hello", 456.0 => :ok}
```

will have the type:

```
%{integer() => binary(), float() => :ok}
```

It is also possible to mix domain keys, as above, with atom keys, yielding the following:

```
%{integer() => integer(), root: integer()}
```

This system is an implementation of Typing Records, Maps, and Structs, by Giuseppe Castagna (2023).

Typing of map operations

We have typed the majority of the functions in the `Map` module, allowing the type system to track how keys are added, updated, and removed across

all possible key types.

For example, imagine we are calling the following `Map` functions with a variable `map`, which we don't know the exact shape of, and an atom key:

```
Map.put(map, :key, 123)
#=> returns type %{..., key: integer()}

Map.delete(map, :key)
#=> returns type %{..., key: not_set()}
```

As you can see, we track when keys are set and also when they are removed.

Some operations, like `Map.replace/3`, only replace the key if it exists, and that is also propagated by the type system:

```
Map.replace(map, :key, 123)
#=> returns type %{..., key: if_set(integer())}
```

In other words, if the key exists, it would have been replaced by an integer value. Furthermore, whenever calling a function in the `Map` module and the given key is statically proven to never exist in the map, an error is emitted.

By combining full type inference with bang operations like `Map.fetch!/2`, `Map.pop!/2`, `Map.replace!/3`, and `Map.update!/3`, Elixir is able to propagate information about the desired keys. Take this module:

```
defmodule User do
  def name(map), do: Map.fetch!(map, :name)
end

defmodule CallsUser do
```

```
  def calls_name do
    User.name(%{})
  end
end
```

The code above has a type violation, which is now caught by the type system:

```
warning: incompatible types given to User.name/1:

    User.name(%{})

given types:

    %{name: not_set()}

but expected one of:

    dynamic(%{..., name: term()})

type warning found at:
  │
16 │     User.name(%{})
  │         ~
  │
  └─ lib/calls_user.ex:7:5: CallsUser.calls_name/0
```

Acknowledgements

The type system was made possible thanks to a partnership between CNRS and Remote. The development work is currently sponsored by Fresha and Tidewave.

Compile-time improvements

Elixir's v1.20 improves compilation times once more, especially on applications with many cores.

It also introduces a new compiler option called `:module_definition`, which if the module definition should be `:compiled` (the default) or `:interpreted`. Note this does not affect the `.beam` file written to disk, only how the contents inside `defmodule` are executed. Using the `:interpreted` mode may offer better compilation times for large projects, especially on machines with high core count, however, it comes with some downsides:

- Errors during compilation may have less precise stacktraces
- Anonymous functions within `defmodule` can have only up to 20 arguments. If this is an issue, you can use maps or tuples to group the data. Note the functions themselves inside `defmodule`, such as the ones defined inside `def` and friends, can still have up to 255 arguments

You can enable it by setting `elixirc_options: [module_definition: :interpreted]` in your `mix.exs`.

v1.20.5 (2026-10-29)

1. Security

Elixir

- [URI] Bind port limits during `URI.parse` (CVE-2026-109821)
- [JSON] Raise when decoding integers in JSON beyond float64 ranges (CVE-2026-115274)

Mix

- [mix deps.partition] Validate `deps.partition` configuration with checksum on multi-user systems (CVE-2026-97209)

2. Enhancements

Elixir

- [Calendar.ISO] Optimize `Calendar.ISO.parse_duration/1`
- [Kernel] Improve boot time by parsing the Erlang/OTP release with `list_to_integer/1`

3. Bug fixes

Elixir

- [Date] Fix `Date.from_gregorian_days/2` passing an invalid day fraction to non-ISO calendars
- [Duration] Fix `Duration.from_iso8601/1` accepting a fractional seconds component after whole seconds, such as `PT1S2.5S`
- [Duration] Reject inputs longer than 192 bytes in `Duration.from_iso8601/1`
- [Regex] Raise `SystemLimitError` when a match reaches the regex engine match limit, instead of returning no match
- [Registry] Do not crash on timeouts on `Registry.lock/3`
- [String] Skip U+055F (Armenian abbreviation mark) when applying the Greek final sigma rule in `String.downcase/2` and `String.capitalize/2`
- [Task] Do not leak timeout messages on `Task.async_stream/3`

IEx

- [IEx] Do not halt the remote node when the remote shell receives EOF with `--remsh`
- [IEx] Fix crash on tab completion when shell is not found

Mix

- [mix xref] Fix outgoing compile-connected counts in `mix xref graph --format stats`

v1.20.4 (2026-08-28)

1. Security

- [List] Avoid recursion when invalid charlists are given to `List.to_string/1` or `List.to_charlist/1` (CVE-2026-75758, GHSA-jf5q-v438-665c)

2. Bug fixes

Elixir

- [Base] Fix false positives in Base64 validation
- [Kernel] Fix inferred type of `:erlang.split_binary/2` when given a bitstring
- [Kernel] Fix type checker crash on comprehensions with a non-returning `:into` expression
- [String] Fix final sigma handling in Greek casing
- [Task] Fix `Task.yield_many/2` waiting indefinitely when `:limit` exceeds the task count

v1.20.3 (2026-08-04)

1. Enhancements

Elixir

- [Kernel.ParallelCompiler] Batch type checker cache operations to improve compilation times

2. Bug fixes

Elixir

- [Kernel] Improve type precision of the `++/2` operator
- [Kernel] Improve type precision of the `send/2` return type
- [Kernel] Improve type refinement for `and/2` and `or/2`
- [Kernel] Fix type checking of `Map.put/3` on empty map types
- [Kernel] Fix type checking of tuple insertion on equivalent tuple types
- [Kernel] Fix type checking of tuple insertion on gradual types with empty static parts
- [Kernel] Fix variable refinements leaking from invalid `receive/after` timeout expressions
- [Kernel] Fix rendering inferred types containing unions of static and dynamic functions
- [Kernel] Ensure typing errors in `:maps.values/1` reference the correct function
- [Kernel] Mark `%_{}` patterns as precise so subsequent redundant struct clauses are detected
- [elixirc] Fix parsing of the `--profile time` option

IEx

- [IEx.Helpers] Fix `break!/1` with unknown expressions

v1.20.2 (2026-06-23)

1. Enhancements

Elixir

- [Kernel.ParallelCompiler] Include per-module type checking times when compiler profiling is enabled with `profile: :time`

2. Bug fixes

Elixir

- [Kernel] Fix binary comprehensions with sizes when options such as `:uniq` or `:into` are used
- [Kernel] Improve compiler error messages when `quote` with `unquote` is used inside a pattern or guard
- [Kernel] Restore the compiler optimization of `Kernel.put_elem/3` to emit `:erlang.setelement/3`
- [Module] Fix type checking when applying an empty function type
- [Module] Fix type checking of bitstring patterns that reuse variables
- [Module] Fix type information for `__info__(:struct)` to include the `:required` key
- [Module] Fix type operations on map and optional keys during difference/intersection, including open keys and empty intersections
- [Module] Fix typing of list types involving dynamic or empty lists
- [Module] Include bitstrings as a possible domain key in the type system
- [Module] Preserve file metadata from each clause in type system warnings
- [Module] Fix type warnings for protocol implementations whose protocol module defines additional callbacks
- [Module] Raise clearer type checking errors when an expected struct is removed or redefined during compilation

Mix

- [mix compile] Avoid unnecessary umbrella recompilation when a path dependency's manifest is newer but unchanged
- [mix deps.compile] Recompile fetched dependencies when their compile-time environment changes

v1.20.1 (2026-06-09)

1. Security

Elixir

- [Version] Limit integer components in Version to 14 decimal bytes, to avoid parsing too large integers from untrusted user input. We strongly advise developers parsing versions from user input to limit the data size given to the `Version` module (CVE-2026-49762, GHSA-w2h8-8x3g-278p)

2. Bug fixes

Elixir

- [Calendar] Cap width in `Calendar.strftime/2` to 1024 characters
- [Code] Ensure `Code.require_file` releases the file if compilation fails
- [Kernel] Fix documentation generation to use the correct version in search

Mix

- [mix archive.install] Validate paths and files when extracting archives
- [mix format] Honor `--no-compile` option when loading plugins

v1.20.0 (2026-06-03)

This release requires Erlang/OTP 27+ and is compatible with Erlang/OTP 29.

1. Enhancements

EEx

- [EEx] Optimize compiler by flattening expr list only once

Elixir

- [Base] Optimize Base validation functions by using SWAR techniques
- [Calendar] Optimize `date_from_iso_days` by using the Neri-Schneider algorithm
- [Code] Add `:dbg_callback` option to eval functions
- [Code] Add `module_definition: :interpreted` option to `Code` which allows module definitions to be evaluated instead of compiled. In some applications/architectures, this can lead to drastic improvements to compilation times. Note this does not affect the generated `.beam` file, which will have the same performance/behaviour as before
- [Code] Make module purging opt-in and move temporary module deletion to the background to speed up compilation times
- [Code.Fragment] Allow preserving sigil metadata in `container_cursor_to_quoted`
- [Enum] Add `Enum.min_max` sorter
- [File] Add support for `[:raw]` opts in `File.read/2`
- [File] Skip device, named pipes, etc in `File.cp_r/3` instead of erroring with reason `:eio`
- [Float] Optimize `Float.round/2` by avoiding big integers
- [Inspect] Increase inspect limit to help print deeply nested data structures
- [Inspect] Support printing Erlang records (using Erlang notation)
- [Integer] Add `Integer.ceil_div/2`

- [Integer] Add `Integer.popcount/1`
- [IO] Add `IO.iodata_empty?/1`
- [Kernel] Add type inference across clauses. For example, if one clause says `x when is_integer(x)`, then the next clause may no longer be an integer
- [Kernel] Add occurrence typing on `case`, `cond`, and `with`
- [Kernel] Detect and warn on redundant clauses
- [Kernel] Perform type inference across applications
- [Kernel] Print intermediate results of `dbg` for pipes
- [Kernel] Show undefined function errors even when missing variables (this helps debug errors caused when the developer forgets to require a macro)
- [Kernel] Warn on unused requires
- [List] Add `List.first!/1` and `List.last!/1`
- [Module] Purge and delete modules if `after_compile/2` callback fails
- [PartitionSupervisor] Support via tuples in `count_children/1` and `stop/3`
- [Process] Add `Process.get_label/1`
- [Registry] Switch `keys: {:duplicate, :key}` to `ordered_set` with composite keys
- [Regex] Add `Regex.import/1` to import regexes defined with `/E`
- [String] SWAR-optimize ASCII fast paths in `String.length/1` and `String.slice/3`
- Add Software Bill of Materials guide to the Documentation

ExUnit

- [ExUnit] Show remaining runs when using `--repeat-until-failure`
- [ExUnit.CaptureLog] Add `:formatter` option for custom log formatting

IEx

- [IEx] Optimize autocompleting modules
- [IEx.Helpers] Add `source/1`

Mix

- [mix app.tree] Support `--output` option
- [mix compile] Add `module_definition: :interpreted` option to `Code` which allows module definitions to be evaluated instead of compiled. In some applications/architectures, this can lead to drastic improvements to compilation times. Note this does not affect the generated `.beam` file, which will have the same performance/behaviour as before
- [mix compile] Enforce `:elixirc_paths` to be a list of strings to avoid paths from being discarded (the only documented type was lists of strings)
- [mix deps] Parallelize dep lock status checks during `deps.loadpaths`, improving boot times in projects with many git dependencies
- [mix deps] Support filtering `mix deps` output
- [mix deps.tree] Support `--output` option
- [mix format] Support `--no-compile` option
- [mix help] Support printing docs for types and callbacks
- [mix source] Add `mix source MODULE` to print or open a given module/function location
- [mix test] Add `mix test --dry-run`

2. Potential breaking changes

Elixir

- [Kernel] Disallow raw CR line ending in strings, comments, and after `?` for security reasons
- [Kernel] `require SomeModule` no longer expands to the given module at compile-time, but it still returns the module at runtime. Note Elixir

does not guarantee macros will expand to certain constructs, only what its execution result, but since this can break code relying on the previous behaviour, such as `require(SomeMod).some_macro()`, we are adding this note to the CHANGELOG

3. Bug fixes

Elixir

- [Enum] Fix `Enum.slice/2` for ranges with step > 1 sliced by step > 1
- [File] Allowing preserving directory permissions in `File.cp_r/3`
- [File] Fix `File.cp_r/3` infinite loop with symlink cycles
- [File] Fix `File.cp_r/3` infinite loop when copying into subdirectory of source
- [File] Fix `File.Stream`'s `Enumerable.count` for files without trailing newline
- [File] Warn when defining `@type record()` for Erlang/OTP 29
- [Float] Fix `Float.parse/1` inconsistent error handling for non-scientific notation overflow
- [Integer] Fix `Integer.extended_gcd/2` returning negative GCD for zero base cases
- [Integer] Raise when negative out-of-range digits are given to `Integer.undigits/2`
- [Kernel] Fix a compiler crash when importing a module with `only: :sigils` option when the imported module exports non-sigil symbols with `sigil_` prefix
- [Kernel] Protocols should not add compile-time dependencies on `Any` implementation
- [Kernel] Preserve evaluation order when rewriting function calls from Elixir modules into Erlang ones
- [Kernel] Reject negative Duration in `to_timeout/1`
- [Keyword] Raise `ArgumentError` in `Keyword.from_keys/2` for non-atom keys

- [Macro] Fix generation of heredocs in `Macro.to_string/1` with escaped trailing newline
- [Path] Consistently return path as binary in `Path.relative_to_cwd/2`
- [Stream] Raise in `Stream.cycle/1` when enumerable reduce call yields no elements
- [String] Support empty pattern list in `String.count/2`
- [URI] Fix `URI.merge` leaking `:+` marker when base path is empty string

ExUnit

- [ExUnit.Diff] Avoid false positives when diffing bitstrings

IEx

- [IEx] Ensure pry works across remote nodes
- [IEx] Ensure warnings emitted during IEx parsing are properly displayed/printed

Logger

- [Logger] Persist log level to app env in `Logger.configure/1`

Mix

- [Mix] Use `non_executable_binary_to_term` on loopback pubsub
- [mix compile] Add a build lock around protocol consolidation in umbrellas
- [mix compile] Ensure compilation of sibling deps do not mark path deps as changed
- [mix compile] Fix compile env change triggering full recompilation of path dependencies
- [mix compile.elixir] Fix scenario where Elixir would tag mtimes in the future

- [mix compile.erlang] Topsort Erlang modules before compilation for proper dependency resolution
- [mix deps] Use config files to pass project state to avoid argv limits on Windows when using `MIX_OS_DEPS_COMPILE_PARTITION_COUNT`
- [mix test] Fix `--warnings-as-errors` not catching misnamed test file warnings
- [mix test] Respect `--raise` when `mix test --warnings-as-errors` passes with warnings

4. Hard deprecations

Elixir

- [File] `File.stream!(path, modes, lines_or_bytes)` is deprecated in favor of `File.stream!(path, lines_or_bytes, modes)`
- [Kernel] Matching on the size inside a bit pattern now requires the pin operator for consistency, such as `<<x::size(^existing_var)>>`
- [Kernel.ParallelCompiler] `Kernel.ParallelCompiler.async/1` is deprecated in favor of `Kernel.ParallelCompiler.pmap/2`, which is more performant and addresses known limitations

Logger

- [Logger] `Logger.*_backend` functions are deprecated in favor of handlers. If you really want to keep on using backends, see the `:logger_backends` package
- [Logger] `Logger.enable/1` and `Logger.disable/1` have been deprecated in favor of `Logger.put_process_level/2` and `Logger.delete_process_level/1`

Mix

- [mix compile.elixir] `xref: [exclude: ...]` in your `mix.exs` is deprecated in favor of `elixirc_options: [no_warn_undefined: ...]`

v1.19

The CHANGELOG for v1.19 releases can be found in the v1.19 branch.

Introduction

Welcome!

This guide will teach you about Elixir fundamentals - the language syntax, how to define modules, the common data structures in the language, and more. This chapter will focus on ensuring that Elixir is installed and that you can successfully run Elixir's Interactive Shell, called IEx.

Let's get started.

Installation

If you haven't yet installed Elixir, visit our installation page. Once you are done, you can run `elixir --version` to get the current Elixir version. The requirements for this guide are:

- Elixir 1.18.0 onwards
- Erlang/OTP 27 onwards

If you are looking for other resources for learning Elixir, you can also consult the learning page of the official website.

Interactive mode

When you install Elixir, you will have three new command line executables: `iex`, `elixir` and `elixirc`.

For now, let's start by running `iex` (or `iex.bat` if you are on Windows PowerShell, where `iex` is a PowerShell command) which stands for Interactive Elixir. In interactive mode, we can type any Elixir expression and get its result. Let's warm up with some basic expressions.

Open up `iex` and type the following expressions:

```
Erlang/OTP 26 [64-bit] [smp:2:2] [...]

Interactive Elixir - press Ctrl+C to exit
iex(1)> 40 + 2
42
iex(2)> "hello" <> " world"
"hello world"
```

Please note that some details like version numbers may differ a bit in your session, that's not important. By executing the code above, you should evaluate expressions and see their results. To exit `iex` press `Ctrl+C` twice.

It seems we are ready to go! We will use the interactive shell quite a lot in the next chapters to get a bit more familiar with the language constructs and basic types, starting in the next chapter.

Running scripts

After getting familiar with the basics of the language you may want to try writing simple programs. This can be accomplished by putting the following Elixir code into a file:

```
IO.puts("Hello world from Elixir")
```

Save it as `simple.exs` and execute it with `elixir`:

```
$ elixir simple.exs
Hello world from Elixir
```

`iex` and `elixir` are all we need to learn the main language concepts. There is a separate guide named "Mix and OTP guide" that explores how to actually create, manage, and test full-blown Elixir projects. For now, let's move on to learn the basic data types in the language.

Basic types

In this chapter we will learn more about Elixir basic types: integers, floats, booleans, atoms, and strings. Other data types, such as lists and tuples, will be explored in the next chapter.

```
iex> 1          # integer
iex> 0x1F       # integer
iex> 1.0        # float
iex> true       # boolean
iex> :atom      # atom / symbol
iex> "elixir"   # string
iex> [1, 2, 3]  # list
iex> {1, 2, 3}  # tuple
```

Basic arithmetic

Open up `iex` and type the following expressions:

```
iex> 1 + 2
3
iex> 5 * 5
25
iex> 10 / 2
5.0
```

Notice that `10 / 2` returned a float `5.0` instead of an integer `5`. This is expected. In Elixir, the operator `/` always returns a float. If you want to do integer division or get the division remainder, you can invoke the `div` and `rem` functions:

```
iex> div(10, 2)
5
iex> div 10, 2
5
iex> rem 10, 3
1
```

Notice that Elixir allows you to drop the parentheses when invoking functions that expect one or more arguments. This feature gives a cleaner syntax when writing declarations and control-flow constructs. However, Elixir developers generally prefer to use parentheses.

Elixir also supports shortcut notations for entering binary, octal, and hexadecimal numbers:

```
iex> 0b1010
10
iex> 0o777
511
iex> 0x1F
31
```

Float numbers require a dot followed by at least one digit and also support `e` for scientific notation:

```
iex> 1.0
1.0
iex> 1.0e-10
1.0e-10
```

Floats in Elixir are 64-bit precision.

You can invoke the `round` function to get the closest integer to a given float, or the `trunc` function to get the integer part of a float.

```
iex> round(3.58)
4
iex> trunc(3.58)
3
```

Finally, as we work with different data types, we will learn that Elixir provides several predicate functions to check for the type of a value. For example, `is_integer` can be used to check if a value is an integer or not:

```
iex> is_integer(1)
true
iex> is_integer(2.0)
false
```

You can also use `is_float` or `is_number` to check, respectively, if an argument is a float, or either an integer or float.

Booleans and `nil`

Elixir supports `true` and `false` as booleans:

```
iex> true
true
iex> true == false
false
```

Elixir also provides three boolean operators: `or`, `and`, and `not`. These operators are strict in the sense that they expect something that evaluates to

a boolean (`true` or `false`) as their first argument:

```
iex> true and true
true
iex> false or is_boolean(true)
true
```

Providing a non-boolean will raise an exception:

```
iex> 1 and true
** (BadBooleanError) expected a boolean on left-side of
"and", got: 1
```

`or` and `and` are short-circuit operators. They only execute the right side if the left side is not enough to determine the result:

```
iex> false and raise("This error will never be raised")
false
iex> true or raise("This error will never be raised")
true
```

Elixir also provides the concept of `nil`, to indicate the absence of a value, and a set of logical operators that also manipulate `nil`: `||/2`, `&&/2`, and `!/1`. For these operators, `false` and `nil` are considered "falsy", all other values are considered "truthy":

```
# or
iex> 1 || true
1
iex> false || 11
11
```

```
# and
iex> nil && 13
nil
iex> true && 17
17

# not
iex> !true
false
iex> !1
false
iex> !nil
true
```

Similarly, values like `0` and `""`, which some other programming languages consider to be "falsy", are also "truthy" in Elixir.

As a rule of thumb, use `and`, `or` and `not` when you are expecting booleans. If any of the arguments are non-boolean, use `&&`, `||` and `!`.

Atoms

An atom is a constant whose value is its own name. Some other languages call these symbols. They are often useful to enumerate over distinct values, such as:

```
iex> :apple
:apple
iex> :orange
:orange
```

```
iex> :watermelon
:watermelon
```

Atoms are equal if their names are equal.

```
iex> :apple == :apple
true
iex> :apple == :orange
false
```

Often they are used to express the state of an operation, by using values such as `:ok` and `:error`.

The booleans `true` and `false` are also atoms:

```
iex> true == :true
true
iex> is_atom(false)
true
iex> is_boolean(:false)
true
```

Elixir allows you to skip the leading `:` for the atoms `false`, `true` and `nil`.

Strings

Strings in Elixir are delimited by double quotes, and they are encoded in UTF-8:

```
iex> "hellö"
"hellö"
```

> Note: if you are running on Windows, there is a chance your terminal does not use UTF-8 by default. You can change the encoding of your current session by running `chcp 65001` before entering IEx.

You can concatenate two strings with the `<>` operator:

```
iex> "hello " <> "world!"
"hello world!"
```

Elixir also supports string interpolation:

```
iex> string = "world"
iex> "hello #{string}!"
"hello world!"
```

String concatenation requires both sides to be strings but interpolation supports any data type that may be converted to a string:

```
iex> number = 42
iex> "i am #{number} years old!"
"i am 42 years old!"
```

Strings can have line breaks in them. You can introduce them using escape sequences:

```
iex> "hello
...> world"
"hello\nworld"
```

```
iex> "hello\nworld"
"hello\nworld"
```

You can print a string using the `IO.puts` function from the `IO` module:

```
iex> IO.puts("hello\nworld")
hello
world
:ok
```

Notice that the `IO.puts` function returns the atom `:ok` after printing.

Strings in Elixir are represented internally by contiguous sequences of bytes known as binaries:

```
iex> is_binary("hellö")
true
```

We can also get the number of bytes in a string:

```
iex> byte_size("hellö")
6
```

Notice that the number of bytes in that string is 6, even though it has 5 graphemes. That's because the grapheme "ö" takes 2 bytes to be represented in UTF-8. We can get the actual length of the string, based on the number of graphemes, by using the `String.length` function:

```
iex> String.length("hellö")
5
```

The `String` module contains a bunch of functions that operate on strings as defined in the Unicode standard:

```
iex> String.upcase("hellö")
"HELLÖ"
```

Structural comparison

Elixir also provides `==`, `!=`, `<=`, `>=`, `<` and `>` as comparison operators. We can compare numbers:

```
iex> 1 == 1
true
iex> 1 != 2
true
iex> 1 < 2
true
```

But also atoms, strings, booleans, etc:

```
iex> "foo" == "foo"
true
iex> "foo" == "bar"
false
```

Integers and floats compare the same if they have the same value:

```
iex> 1 == 1.0
true
```

```
iex> 1 == 2.0
false
```

However, you can use the strict comparison operator `===` and `!==` if you want to distinguish between integers and floats:

```
iex> 1 === 1.0
false
```

The comparison operators in Elixir can compare across any data type. We say these operators perform *structural comparison*. For more information, you can read our documentation on Structural vs Semantic comparisons.

Elixir also provides data-types for expressing collections, such as lists and tuples, which we'll learn next. When we talk about concurrency and fault-tolerance via processes, we will also discuss ports, pids, and references, but that will come in later chapters. Let's move forward.

Lists and tuples

In this chapter we will learn two of the most used collection data-types in Elixir: lists and tuples.

(Linked) Lists

Elixir uses square brackets to specify a list of values. Values can be of any type:

```
iex> [1, 2, true, 3]
[1, 2, true, 3]
iex> length([1, 2, 3])
3
```

Two lists can be concatenated or subtracted using the `++` and `--` operators respectively:

```
iex> [1, 2, 3] ++ [4, 5, 6]
[1, 2, 3, 4, 5, 6]
iex> [1, true, 2, false, 3, true] -- [true, false]
[1, 2, 3, true]
```

List operators never modify the existing list. Concatenating to or removing elements from a list returns a new list. We say that Elixir data structures are *immutable*. One advantage of immutability is that it leads to clearer code. You can freely pass the data around with the guarantee no one will mutate it in memory - only transform it.

Throughout the tutorial, we will talk a lot about the head and tail of a list. The head is the first element of a list and the tail is the remainder of the list.

They can be retrieved with the functions `hd` and `tl`. Let's assign a list to a variable and retrieve its head and tail:

```
iex> list = [1, 2, 3]
iex> hd(list)
1
iex> tl(list)
[2, 3]
```

Getting the head or the tail of an empty list throws an error:

```
iex> hd([])
** (ArgumentError) argument error
```

Sometimes you will create a list and it will return a quoted value preceded by `~c`. For example:

```
iex> [11, 12, 13]
~c"\v\f\r"
iex> [104, 101, 108, 108, 111]
~c"hello"
```

When Elixir sees a list of printable ASCII numbers, Elixir will print that as a charlist (literally a list of characters). Charlists are quite common when interfacing with existing Erlang code. Whenever you see a value in IEx and you are not quite sure what it is, you can use `i` to retrieve information about it:

```
iex> i ~c"hello"
Term
  i ~c"hello"
Data type
```

```
  List
Description
  ...
Raw representation
  [104, 101, 108, 108, 111]
Reference modules
  List
Implemented protocols
  ...
```

We will talk more about charlists in the "Binaries, strings, and charlists" chapter.

> **Single-quoted strings**
>
> In Elixir, you can also use `'hello'` to build charlists, but this notation has been soft-deprecated in Elixir v1.15 and will emit warnings in future versions. Prefer to write `~c"hello"` instead.

Tuples

Elixir uses curly brackets to define tuples. Like lists, tuples can hold any value:

```
iex> {:ok, "hello"}
{:ok, "hello"}
iex> tuple_size({:ok, "hello"})
2
```

Tuples store elements contiguously in memory. This means accessing a tuple element by index or getting the tuple size is a fast operation. Indexes start from zero:

```
iex> tuple = {:ok, "hello"}
{:ok, "hello"}
iex> elem(tuple, 1)
"hello"
iex> tuple_size(tuple)
2
```

It is also possible to put an element at a particular index in a tuple with `put_elem`:

```
iex> tuple = {:ok, "hello"}
{:ok, "hello"}
iex> put_elem(tuple, 1, "world")
{:ok, "world"}
iex> tuple
{:ok, "hello"}
```

Notice that `put_elem` returned a new tuple. The original tuple stored in the `tuple` variable was not modified. Like lists, tuples are also immutable. Every operation on a tuple returns a new tuple, it never changes the given one.

Lists or tuples?

What is the difference between lists and tuples?

Lists are stored in memory as linked lists, meaning that each element in a list holds its value and points to the following element until the end of the list is reached. This means accessing the length of a list is a linear operation: we need to traverse the whole list in order to figure out its size.

Similarly, the performance of list concatenation depends on the length of the left-hand list:

```
iex> list = [1, 2, 3]
[1, 2, 3]

# This is fast as we only need to traverse `[0]` to prepend
to `list`
iex> [0] ++ list
[0, 1, 2, 3]

# This is slow as we need to traverse `list` to append 4
iex> list ++ [4]
[1, 2, 3, 4]
```

Tuples, on the other hand, are stored contiguously in memory. This means getting the tuple size or accessing an element by index is fast. On the other hand, updating or adding elements to tuples is expensive because it requires creating a new tuple in memory:

```
iex> tuple = {:a, :b, :c, :d}
{:a, :b, :c, :d}
iex> put_elem(tuple, 2, :e)
{:a, :b, :e, :d}
```

Note, however, the elements themselves are not copied. When you update a tuple, all entries are shared between the old and the new tuple, except for the entry that has been replaced. This rule applies to most data structures in Elixir. This reduces the amount of memory allocation the language needs to perform and is only possible thanks to the immutable semantics of the language.

Those performance characteristics dictate the usage of those data structures. In a nutshell, lists are used when the number of elements returned may vary. Tuples have a fixed size. Let's see two examples from the `String` module:

```
iex> String.split("hello world")
["hello", "world"]
iex> String.split("hello beautiful world")
["hello", "beautiful", "world"]
```

The `String.split` function breaks a string into a list of strings on every whitespace character. Since the amount of elements returned depends on the input, we use a list.

On the other hand, `String.split_at` splits a string in two parts at a given position. Since it always returns two entries, regardless of the input size, it returns tuples:

```
iex> String.split_at("hello world", 3)
{"hel", "lo world"}
iex> String.split_at("hello world", -4)
{"hello w", "orld"}
```

It is also very common to use tuples and atoms to create "tagged tuples", which is a handy return value when an operation may succeed or fail. For example, `File.read` reads the contents of a file at a given path, which may or may not exist. It returns tagged tuples:

```
iex> File.read("path/to/existing/file")
{:ok, "... contents ..."}
iex> File.read("path/to/unknown/file")
{:error, :enoent}
```

If the path given to `File.read` exists, it returns a tuple with the atom `:ok` as the first element and the file contents as the second. Otherwise, it returns a tuple with `:error` and the error description. As we will soon learn, Elixir

allows us to *pattern match* on tagged tuples and effortlessly handle both success and failure cases.

Given Elixir consistently follows those rules, the choice between lists and tuples get clearer as you learn and use the language. Elixir often guides you to do the right thing. For example, there is an `elem` function to access a tuple item:

```
iex> tuple = {:ok, "hello"}
{:ok, "hello"}
iex> elem(tuple, 1)
"hello"
```

However, given you often don't know the number of elements in a list, there is no built-in equivalent for accessing arbitrary entries in a lists, apart from its head.

Size or length?

When counting the elements in a data structure, Elixir also abides by a simple rule: the function is named `size` if the operation is in constant time (the value is pre-calculated) or `length` if the operation is linear (calculating the length gets slower as the input grows). As a mnemonic, both "length" and "linear" start with "l".

For example, we have used 4 counting functions so far: `byte_size` (for the number of bytes in a string), `tuple_size` (for tuple size), `length` (for list length) and `String.length` (for the number of graphemes in a string). We use `byte_size` to get the number of bytes in a string, which is a cheap operation. Retrieving the number of Unicode graphemes, on the other hand, uses `String.length`, and may be expensive as it relies on a traversal of the entire string.

Now that we are familiar with the basic data-types in the language, let's learn important constructs for writing code, before we discuss more complex data structures.

Pattern matching

In this chapter, we will learn why the `=` operator in Elixir is called the match operator and how to use it to pattern match inside data structures. We will learn about the pin operator `^` used to access previously bound values.

The match operator

We have used the `=` operator a couple times to assign variables in Elixir:

```
iex> x = 1
1
iex> x
1
```

In Elixir, the `=` operator is actually called *the match operator*. Let's see why:

```
iex> x = 1
1
iex> 1 = x
1
iex> 2 = x
** (MatchError) no match of right hand side value: 1
```

Notice that `1 = x` is a valid expression, and it matched because both the left and right side are equal to `1`. When the sides do not match, a `MatchError` is raised.

A variable can only be assigned on the left side of `=`:

```
iex> 1 = unknown
** (CompileError) iex:1: undefined variable "unknown"
```

Pattern matching

The match operator is not only used to match against simple values, but it is also useful for destructuring more complex data types. For example, we can pattern match on tuples:

```
iex> {a, b, c} = {:hello, "world", 42}
{:hello, "world", 42}
iex> a
:hello
iex> b
"world"
```

A pattern match error will occur if the sides can't be matched, for example if the tuples have different sizes:

```
iex> {a, b, c} = {:hello, "world"}
** (MatchError) no match of right hand side value: {:hello,
"world"}
```

And also when comparing different types, for example if matching a tuple on the left side with a list on the right side:

```
iex> {a, b, c} = [:hello, "world", 42]
** (MatchError) no match of right hand side value: [:hello,
"world", 42]
```

More interestingly, we can match on specific values. The example below asserts that the left side will only match the right side when the right side is a tuple that starts with the atom `:ok`:

```
iex> {:ok, result} = {:ok, 13}
{:ok, 13}
iex> result
13

iex> {:ok, result} = {:error, :oops}
** (MatchError) no match of right hand side value: {:error,
:oops}
```

We can pattern match on lists:

```
iex> [a, b, c] = [1, 2, 3]
[1, 2, 3]
iex> a
1
```

A list also supports matching on its own head and tail:

```
iex> [head | tail] = [1, 2, 3]
[1, 2, 3]
iex> head
1
iex> tail
[2, 3]
```

Similar to the `hd` and `tl` functions, we can't match an empty list with a head and tail pattern:

```
iex> [head | tail] = []
** (MatchError) no match of right hand side value: []
```

The `[head | tail]` format is not only used on pattern matching but also for prepending items to a list:

```
iex> list = [1, 2, 3]
[1, 2, 3]
iex> [0 | list]
[0, 1, 2, 3]
```

In some cases, you don't care about a particular value in a pattern. It is a common practice to bind those values to the underscore, `_`. For example, if only the head of the list matters to us, we can assign the tail to underscore:

```
iex> [head | _] = [1, 2, 3]
[1, 2, 3]
iex> head
1
```

The variable `_` is special in that it can never be read from. Trying to read from it gives a compile error:

```
iex> _
** (CompileError) iex:1: invalid use of _. "_" represents a
value to be ignored in a pattern and cannot be used in
expressions
```

If a variable is mentioned more than once in a pattern, all references must bind to the same value:

```
iex> {x, x} = {1, 1}
{1, 1}
iex> {x, x} = {1, 2}
** (MatchError) no match of right hand side value: {1, 2}
```

Although pattern matching allows us to build powerful constructs, its usage is limited. For instance, you cannot make function calls on the left side of a match. The following example is invalid:

```
iex> length([1, [2], 3]) = 3
** (CompileError) iex:1: cannot invoke remote function
:erlang.length/1 inside match
```

Pattern matching allows developers to easily destructure data types such as tuples and lists. As we will see in the following chapters, it is one of the foundations of recursion in Elixir and applies to other types as well, like maps and binaries.

The pin operator

Variables in Elixir can be rebound:

```
iex> x = 1
1
iex> x = 2
2
```

However, there are times when we don't want variables to be rebound.

Use the pin operator `^` when you want to pattern match against a variable's *existing value* rather than rebinding the variable.

```
iex> x = 1
1
iex> ^x = 2
** (MatchError) no match of right hand side value: 2
```

Because we have pinned `x` when it was bound to the value of `1`, it is equivalent to the following:

```
iex> 1 = 2
** (MatchError) no match of right hand side value: 2
```

Notice that we even see the exact same error message.

We can use the pin operator inside other pattern matches, such as tuples or lists:

```
iex> x = 1
1
iex> [^x, 2, 3] = [1, 2, 3]
[1, 2, 3]
iex> {y, ^x} = {2, 1}
{2, 1}
iex> y
2
iex> {y, ^x} = {2, 2}
** (MatchError) no match of right hand side value: {2, 2}
```

Because `x` was bound to the value of `1` when it was pinned, this last example could have been written as:

```
iex> {y, 1} = {2, 2}
```

```
** (MatchError) no match of right hand side value: {2, 2}
```

This finishes our introduction to pattern matching. As we will see in the next chapter, pattern matching is very common in many language constructs and they can be further augmented with guards.

case, cond, and if

In this chapter, we will learn about the `case`, `cond`, and `if` control flow structures.

case

`case` allows us to compare a value against many patterns until we find a matching one:

```
iex> case {1, 2, 3} do
...>   {4, 5, 6} ->
...>     "This clause won't match"
...>   {1, x, 3} ->
...>     "This clause will match and bind x to 2 in this
clause"
...>   _ ->
...>     "This clause would match any value"
...> end
"This clause will match and bind x to 2 in this clause"
```

If you want to pattern match against an existing variable, you need to use the `^` operator:

```
iex> x = 1
1
iex> case 10 do
...>   ^x -> "Won't match"
...>   _ -> "Will match"
```

```
...> end
"Will match"
```

Clauses also allow extra conditions to be specified via guards:

```
iex> case {1, 2, 3} do
...>   {1, x, 3} when x > 0 ->
...>     "Will match"
...>   _ ->
...>     "Would match, if guard condition were not
satisfied"
...> end
"Will match"
```

The first clause above will only match when `x` is positive.

Keep in mind errors in guards do not leak but simply make the guard fail:

```
iex> hd(1)
** (ArgumentError) argument error
iex> case 1 do
...>   x when hd(x) -> "Won't match"
...>   x -> "Got #{x}"
...> end
"Got 1"
```

If none of the clauses match, an error is raised:

```
iex> case :ok do
...>   :error -> "Won't match"
```

```
...> end
** (CaseClauseError) no case clause matching: :ok
```

The documentation for the `Kernel` module lists all available guards in its sidebar. You can also consult the complete Patterns and Guards reference for in-depth documentation.

if

`case` builds on pattern matching and guards to destructure and match on certain conditions. However, patterns and guards are limited only to certain expressions which are optimized by the compiler. In many situations, you need to write conditions that go beyond what can be expressed with `case`. For those, `if` is a useful alternative:

```
iex> if true do
...>   "This works!"
...> end
"This works!"
iex> if false do
...>   "This will never be seen"
...> end
nil
```

If the condition given to `if` returns `false` or `nil`, the body given between `do-end` is not executed and instead it returns `nil`.

`if` also supports `else` blocks:

```
iex> if nil do
...>   "This won't be seen"
...> else
```

```
...>   "This will"
...> end
"This will"
```

Expressions

Some programming languages make a distinction about expressions (code that returns a value) and statements (code that returns no value). In Elixir, there are only expressions, no statements. Everything you write in Elixir language returns some value.

This property allows variables to be scoped to individual blocks of code such as `if`, `case`, where declarations or changes are only visible inside the block. A change can't leak to outer blocks, which makes code easier to follow and understand. For example:

```
iex> x = 1
1
iex> if true do
...>   x = x + 1
...> end
2
iex> x
1
```

You see the return value of the `if` expression as the resulting `2` here. To retain changes made within the `if` expression on the outer block you need to assign the returned value to a variable in the outer block.

```
iex> x = 1
1
iex> x =
...>   if true do
```

```
...>     x + 1
...>   else
...>     x
...>   end
2
```

With all expressions returning a value there's also no need for alternative constructs, such as ternary operators posing as an alternative to `if`. Elixir does include an inline notation for `if` and, as we will learn later, it is a syntactic variation on `if`'s arguments.

> **`if` is a macro**
>
> An interesting note regarding `if` is that it is implemented as a macro in the language: it isn't a special language construct as it would be in many languages. You can check the documentation and its source for more information.

If you find yourself nesting several `if` blocks, you may want to consider using `cond` instead. Let's check it out.

cond

We have used `case` to find a matching clause from many patterns. We have used `if` to check for a single condition. If you need to check across several conditions and find the first one that does not evaluate to `nil` or `false`, `cond` is a useful construct:

```
iex> cond do
...>   2 + 2 == 5 ->
...>     "This will not be true"
...>   2 * 2 == 3 ->
...>     "Nor this"
```

```
...>   1 + 1 == 2 ->
...>     "But this will"
...> end
"But this will"
```

This is equivalent to `else if` clauses in many imperative languages - although used less frequently in Elixir.

If all of the conditions return `nil` or `false`, an error (`CondClauseError`) is raised. For this reason, it may be necessary to add a final condition, equal to `true`, which will always match:

```
iex> cond do
...>   2 + 2 == 5 ->
...>     "This is never true"
...>   2 * 2 == 3 ->
...>     "Nor this"
...>   true ->
...>     "This is always true (equivalent to else)"
...> end
"This is always true (equivalent to else)"
```

Similar to `if`, `cond` considers any value besides `nil` and `false` to be true:

```
iex> cond do
...>   hd([1, 2, 3]) ->
...>     "1 is considered as true"
...> end
"1 is considered as true"
```

Summing up

We have concluded the introduction to the most fundamental control-flow constructs in Elixir. Generally speaking, Elixir developers prefer pattern matching and guards, using `case` and function definitions (which we will explore in future chapters), as they are succinct and precise. When your logic cannot be outlined within patterns and guards, you may consider `if`, falling back to `cond` when there are several conditions to check.

Anonymous functions

Anonymous functions allow us to store and pass executable code around as if it was an integer or a string. Let's learn more.

Identifying functions and documentation

Before we move on to discuss anonymous functions, let's talk about how Elixir identifies named functions – the functions defined in modules.

Functions in Elixir are identified by both their name and their arity. The arity of a function describes the number of arguments that the function takes. From this point on we will use both the function name and its arity to describe functions throughout the documentation. `trunc/1` identifies the function which is named `trunc` and takes `1` argument, whereas `trunc/2` identifies a different (nonexistent) function with the same name but with an arity of `2`.

We can also use this syntax to access documentation. The Elixir shell defines the `h` function, which you can use to access documentation for any function. For example, typing `h trunc/1` is going to print the documentation for the `trunc/1` function:

```
iex> h trunc/1

                                def trunc(number)

Returns the integer part of number.
```

`h trunc/1` works because it is defined in the `Kernel` module. All functions in the `Kernel` module are automatically imported into our

namespace. Most often you will also include the module name when looking up the documentation for a given function:

```
iex> h Kernel.trunc/1
                                def trunc(number)

Returns the integer part of number.
```

You can use the module+function identifiers to lookup documentation for anything, including operators (try `h Kernel.+/2`). Invoking `h` without arguments displays the documentation for `IEx.Helpers`, which is where `h` and other functionalities are defined.

Defining anonymous functions

Anonymous functions in Elixir are delimited by the keywords `fn` and `end`:

```
iex> add = fn a, b -> a + b end
#Function<12.71889879/2 in :erl_eval.expr/5>
```

In the example above, we defined an anonymous function that receives two arguments, `a` and `b`, and returns the result of `a + b`. The arguments are always on the left-hand side of `->` and the code to be executed on the right-hand side. The anonymous function is stored in the variable `add`. You can see it returns a value represented by `#Function<...>`. While its representation is opaque, the `:erl_eval.expr` bit tells us the function was defined in the shell (during evaluation).

We can invoke anonymous functions by passing arguments to it, using a dot (`.`) between the variable and the opening parenthesis:

```
iex> add.(1, 2)
3
```

The dot makes it clear when you are calling an anonymous function, stored in the variable `add`, opposed to a function named `add/2`. For example, if you have an anonymous function stored in the variable `is_atom`, there is no ambiguity between `is_atom.(:foo)` and `is_atom(:foo)`. If both used the same `is_atom(:foo)` syntax, the only way to know the actual behavior of `is_atom(:foo)` would be by scanning all code thus far for a possible definition of the `is_atom` variable. This scanning hurts maintainability as it requires developers to track additional context in their head when reading and writing code.

Anonymous functions in Elixir are also identified by the number of arguments they receive. We can check if a value is a function using `is_function/1` and also check its arity by using `is_function/2`:

```
iex> is_function(add)
true
# check if add is a function that expects exactly 2
arguments
iex> is_function(add, 2)
true
# check if add is a function that expects exactly 1 argument
iex> is_function(add, 1)
false
```

Closures

Anonymous functions can also access variables that are in scope when the function is defined. This is typically referred to as closures, as they close

over their scope. Let's define a new anonymous function that uses the `add` anonymous function we have previously defined:

```
iex> double = fn a -> add.(a, a) end
#Function<6.71889879/1 in :erl_eval.expr/5>
iex> double.(2)
4
```

A variable assigned inside a function does not affect its surrounding environment:

```
iex> x = 42
42
iex> (fn -> x = 0 end).()
0
iex> x
42
```

Clauses and guards

Similar to `case/2`, we can pattern match on the arguments of anonymous functions as well as define multiple clauses and guards:

```
iex> f = fn
...>   x, y when x > 0 -> x + y
...>   x, y -> x * y
...> end
#Function<12.71889879/2 in :erl_eval.expr/5>
iex> f.(1, 3)
4
```

```
iex> f.(-1, 3)
-3
```

The number of arguments in each anonymous function clause needs to be the same, otherwise an error is raised.

```
iex> f2 = fn
...>   x, y when x > 0 -> x + y
...>   x, y, z -> x * y + z
...> end
** (CompileError) iex:1: cannot mix clauses with different
arities in anonymous functions
```

The capture operator

Throughout this guide, we have been using the notation `name/arity` to refer to functions. It happens that this notation can actually be used to capture an existing function into a data-type we can pass around, similar to how anonymous functions behave.

```
iex> fun = &is_atom/1
&:erlang.is_atom/1
iex> is_function(fun)
true
iex> fun.(:hello)
true
iex> fun.(123)
false
```

As you can see, once a function is captured, we can pass it as argument or invoke it using the anonymous function notation. The returned value above also hints we can capture functions defined in modules:

```
iex> fun = &String.length/1
&String.length/1
iex> fun.("hello")
5
```

Since operators are functions in Elixir, you can also capture operators:

```
iex> add = &+/2
&:erlang.+/2
iex> add.(1, 2)
3
```

The capture syntax can also be used as a shortcut for creating functions that wrap existing functions. For example, imagine you want to create an anonymous function that checks if a given function has arity 2. You could write it as:

```
iex> is_arity_2 = fn fun -> is_function(fun, 2) end
#Function<8.71889879/1 in :erl_eval.expr/5>
iex> is_arity_2.(add)
true
```

But using the capture syntax, you can write it as:

```
iex> is_arity_2 = &is_function(&1, 2)
#Function<8.71889879/1 in :erl_eval.expr/5>
iex> is_arity_2.(add)
true
```

The `&1` represents the first argument passed into the function. Therefore both `is_arity_2` anonymous functions defined above are equivalent.

Once again, given operators are function calls, the capture syntax shorthand also works with operators, or even string interpolation:

```
iex> fun = &(&1 + 1)
#Function<6.71889879/1 in :erl_eval.expr/5>
iex> fun.(1)
2

iex> fun2 = &"Good #{&1}"
#Function<6.127694169/1 in :erl_eval.expr/5>
iex> fun2.("morning")
"Good morning"
```

`&(&1 + 1)` above is exactly the same as `fn x -> x + 1 end`. You can read more about the capture operator `&` in its documentation.

Next let's revisit some of the data-types we learned in the past and dig deeper into how they work.

Binaries, strings, and charlists

In "Basic types", we learned a bit about strings and we used the `is_binary/1` function for checks:

```
iex> string = "hello"
"hello"
iex> is_binary(string)
true
```

In this chapter, we will gain clarity on what exactly binaries are and how they relate to strings. We will also learn about charlists, `~c"like this"`, which are often used for interoperability with Erlang.

Although strings are one of the most common data types in computer languages, they are subtly complex and are often misunderstood. To understand strings in Elixir, let's first discuss Unicode and character encodings, specifically the UTF-8 encoding.

Unicode and Code Points

In order to facilitate meaningful communication between computers across multiple languages, a standard is required so that the ones and zeros on one machine mean the same thing when they are transmitted to another. The Unicode Standard acts as an official registry of virtually all the characters we know: this includes characters from classical and historical texts, emoji, and formatting and control characters as well.

Unicode organizes all of the characters in its repertoire into code charts, and each character is given a unique numerical index. This numerical index is known as a Code Point.

In Elixir you can use a `?` in front of a character literal to reveal its code point:

```
iex> ?a
97
iex> ?ł
322
```

Note that most Unicode code charts will refer to a code point by its hexadecimal (hex) representation, e.g. `97` translates to `0061` in hex, and we can represent any Unicode character in an Elixir string by using the `\uXXXX` notation and the hex representation of its code point number:

```
iex> "\u0061" == "a"
true
iex> 0x0061 = 97 = ?a
97
```

The hex representation will also help you look up information about a code point, e.g. https://codepoints.net/U+0061 has a data sheet all about the lower case `a`, a.k.a. code point 97.

UTF-8 and Encodings

Now that we understand what the Unicode standard is and what code points are, we can finally talk about encodings. Whereas the code point is **what** we store, an encoding deals with **how** we store it: encoding is an implementation. In other words, we need a mechanism to convert the code point numbers into bytes so they can be stored in memory, written to disk, etc.

Elixir uses UTF-8 to encode its strings, which means that code points are encoded as a series of 8-bit bytes. UTF-8 is a **variable width** character

encoding that uses one to four bytes to store each code point. It is capable of encoding all valid Unicode code points. Let's see an example:

```
iex> string = "héllo"
"héllo"
iex> String.length(string)
5
iex> byte_size(string)
6
```

Although the string above has 5 characters, it uses 6 bytes, as two bytes are used to represent the character `é`.

> Note: if you are running on Windows, there is a chance your terminal does not use UTF-8 by default. You can change the encoding of your current session by running `chcp 65001` before entering `iex` (`iex.bat`).

Besides defining characters, UTF-8 also provides a notion of graphemes. Graphemes may consist of multiple characters that are often perceived as one. For example, the [woman firefighter emoji] is represented as the combination of three characters: the woman emoji ( ), a hidden zero-width joiner, and the fire engine emoji ( ):

```
iex> String.codepoints("▫▫")
["▫", "", "▫"]
iex> String.graphemes("▫▫")
["▫▫"]
```

However, Elixir is smart enough to know they are seen as a single character, and therefore the length is still one:

```
iex> String.length("▫▫")
1
```

> Note: if you can't see the emoji above in your terminal, you need to make sure your terminal supports emoji and that you are using a font that can render them.

Although these rules may sound complicated, UTF-8 encoded documents are everywhere. This page itself is encoded in UTF-8. The encoding information is given to your browser which then knows how to render all of the bytes, characters, and graphemes accordingly.

If you want to see the exact bytes that a string would be stored in a file, a common trick is to concatenate the null byte `<<0>>` to it:

```
iex> "hełło" <> <<0>>
<<104, 101, 197, 130, 197, 130, 111, 0>>
```

Alternatively, you can view a string's binary representation by using `IO.inspect/2`:

```
iex> IO.inspect("hełło", binaries: :as_binaries)
<<104, 101, 197, 130, 197, 130, 111>>
```

We are getting a little bit ahead of ourselves. Let's talk about bitstrings to learn about what exactly the `<<>>` constructor means.

Bitstrings

Although we have covered code points and UTF-8 encoding, we still need to go a bit deeper into how exactly we store the encoded bytes, and this is where we introduce the **bitstring**. A bitstring is a fundamental data type in

Elixir, denoted with the `<<>>` syntax. **A bitstring is a contiguous sequence of bits in memory.**

By default, 8 bits (i.e. 1 byte) is used to store each number in a bitstring, but you can manually specify the number of bits via a `::n` modifier to denote the size in `n` bits, or you can use the more verbose declaration `::size(n)`:

```
iex> <<42>> == <<42::8>>
true
iex> <<3::4>>
<<3::size(4)>>
```

For example, the decimal number `3` when represented with 4 bits in base 2 would be `0011`, which is equivalent to the values `0`, `0`, `1`, `1`, each stored using 1 bit:

```
iex> <<0::1, 0::1, 1::1, 1::1>> == <<3::4>>
true
```

Any value that exceeds what can be stored by the number of bits provisioned is truncated:

```
iex> <<1>> == <<257>>
true
```

Here, 257 in base 2 would be represented as `100000001`, but since we have reserved only 8 bits for its representation (by default), the left-most bit is ignored and the value becomes truncated to `00000001`, or simply `1` in decimal.

A complete reference for the bitstring constructor can be found in `<<>>`'s documentation.

Binaries

A binary is a bitstring where the number of bits is divisible by 8. That means that every binary is a bitstring, but not every bitstring is a binary. We can use the `is_bitstring/1` and `is_binary/1` functions to demonstrate this.

```
iex> is_bitstring(<<3::4>>)
true
iex> is_binary(<<3::4>>)
false
iex> is_bitstring(<<0, 255, 42>>)
true
iex> is_binary(<<0, 255, 42>>)
true
iex> is_binary(<<42::16>>)
true
```

We can pattern match on binaries / bitstrings:

```
iex> <<0, 1, x>> = <<0, 1, 2>>
<<0, 1, 2>>
iex> x
2
iex> <<0, 1, x>> = <<0, 1, 2, 3>>
** (MatchError) no match of right hand side value: <<0, 1,
2, 3>>
```

Note that unless you explicitly use `::` modifiers, each entry in the binary pattern is expected to match a single byte (exactly 8 bits). If we want to match on a binary of unknown size, we can use the `binary` modifier at the end of the pattern:

```
iex> <<0, 1, x::binary>> = <<0, 1, 2, 3>>
<<0, 1, 2, 3>>
iex> x
<<2, 3>>
```

There are a couple other modifiers that can be useful when doing pattern matches on binaries. The `binary-size(n)` modifier will match `n` bytes in a binary:

```
iex> <<head::binary-size(2), rest::binary>> = <<0, 1, 2, 3>>
<<0, 1, 2, 3>>
iex> head
<<0, 1>>
iex> rest
<<2, 3>>
```

A string is a UTF-8 encoded binary, where the code point for each character is encoded using 1 to 4 bytes. Thus every string is a binary, but due to the UTF-8 standard encoding rules, not every binary is a valid string.

```
iex> is_binary("hello")
true
iex> is_binary(<<239, 191, 19>>)
true
iex> String.valid?(<<239, 191, 19>>)
false
```

The string concatenation operator `<>` is actually a binary concatenation operator:

```
iex> "a" <> "ha"
"aha"
```

```
iex> <<0, 1>> <> <<2, 3>>
<<0, 1, 2, 3>>
```

Given that strings are binaries, we can also pattern match on strings:

```
iex> <<head, rest::binary>> = "banana"
"banana"
iex> head == ?b
true
iex> rest
"anana"
```

However, remember that binary pattern matching works on *bytes*, so matching on the string like "über" with multibyte characters won't match on the *character*, it will match on the *first byte of that character*:

```
iex> "ü" <> <<0>>
<<195, 188, 0>>
iex> <<x, rest::binary>> = "über"
"über"
iex> x == ?ü
false
iex> rest
<<188, 98, 101, 114>>
```

Above, `x` matched on only the first byte of the multibyte `ü` character.

Therefore, when pattern matching on strings, it is important to use the `utf8` modifier:

```
iex> <<x::utf8, rest::binary>> = "über"
"über"
```

```
iex> x == ?ü
true
iex> rest
"ber"
```

Charlists

Our tour of our bitstrings, binaries, and strings is nearly complete, but we have one more data type to explain: the charlist.

A charlist is a list of integers where all the integers are valid code points. In practice, you will not come across them often, only in specific scenarios such as interfacing with older Erlang libraries that do not accept binaries as arguments.

```
iex> ~c"hello"
~c"hello"
iex> [?h, ?e, ?l, ?l, ?o]
~c"hello"
```

The `~c` sigil (we'll cover sigils later in the "Sigils" chapter) indicates the fact that we are dealing with a charlist and not a regular string.

Instead of containing bytes, a charlist contains integer code points. However, the list is only printed as a sigil if all code points are within the ASCII range:

```
iex> ~c"hełło"
[104, 101, 322, 322, 111]
iex> is_list(~c"hełło")
true
```

This is done to ease interoperability with Erlang, even though it may lead to some surprising behavior. For example, if you are storing a list of integers that happen to range between 0 and 127, by default IEx will interpret this as a charlist and it will display the corresponding ASCII characters.

```
iex> heartbeats_per_minute = [99, 97, 116]
~c"cat"
```

You can always force charlists to be printed in their list representation by calling the `inspect/2` function:

```
iex> inspect(heartbeats_per_minute, charlists: :as_list)
"[99, 97, 116]"
```

Furthermore, you can convert a charlist to a string and back by using the `to_string/1` and `to_charlist/1`:

```
iex> to_charlist("hełło")
[104, 101, 322, 322, 111]
iex> to_string(~c"hełło")
"hełło"
iex> to_string(:hello)
"hello"
iex> to_string(1)
"1"
```

The functions above are polymorphic, in other words, they accept many shapes: not only do they convert charlists to strings (and vice-versa), they can also convert integers, atoms, and so on.

String (binary) concatenation uses the `<>` operator but charlists, being lists, use the list concatenation operator `++`:

```
iex> ~c"this " <> ~c"fails"
** (ArgumentError) expected binary argument in <> operator
but got: ~c"this "
    (elixir) lib/kernel.ex:1821: Kernel.wrap_concatenation/3
    (elixir) lib/kernel.ex:1808:
Kernel.extract_concatenations/2
    (elixir) expanding macro: Kernel.<>/2
    iex:1: (file)
iex> ~c"this " ++ ~c"works"
~c"this works"
iex> "he" ++ "llo"
** (ArgumentError) argument error
    :erlang.++("he", "llo")
iex> "he" <> "llo"
"hello"
```

With binaries, strings, and charlists out of the way, it is time to talk about key-value data structures.

Keyword lists and maps

Now let's talk about associative data structures. Associative data structures are able to associate a key to a certain value. Different languages call these different names like dictionaries, hashes, associative arrays, etc.

In Elixir, we have two main associative data structures: keyword lists and maps.

Keyword lists

Keyword lists are a data-structure used to pass options to functions. Let's see a scenario where they may be useful.

Imagine you want to split a string of numbers. Initially, we can invoke `String.split/2` passing two strings as arguments:

```
iex> String.split("1 2 3 4", " ")
["1", "2", "3", "4"]
```

What if you only want to split at most 2 times? The `String.split/3` function allows the `parts` option to be set to the maximum number of entries in the result:

```
iex> String.split("1 2 3 4", " ", [parts: 3])
["1", "2", "3 4"]
```

As you can see, we got 3 parts, the last one containing the remaining of the input without splitting it.

Now imagine that some of the inputs you must split on contains additional spaces between the numbers:

```
iex> String.split("1  2  3  4", " ", [parts: 3])
["1", "", "2  3  4"]
```

As you can see, the additional spaces lead to empty entries in the output. Luckily, we can also set the `trim` option to `true` to remove them:

```
iex> String.split("1  2  3  4", " ", [parts: 3, trim: true])
["1", "2", " 3  4"]
```

Once again we got 3 parts, with the last one containing the leftovers.

`[parts: 3]` and `[parts: 3, trim: true]` are keyword lists. When a keyword list is the last argument of a function, we can skip the brackets and write:

```
iex> String.split("1  2  3  4", " ", parts: 3, trim: true)
["1", "2", " 3  4"]
```

As shown in the example above, keyword lists are mostly used as optional arguments to functions.

As the name implies, keyword lists are simply lists. In particular, they are lists consisting of 2-item tuples where the first element (the key) is an atom and the second element can be any value. Both representations are the same:

```
iex> [{:parts, 3}, {:trim, true}] == [parts: 3, trim: true]
true
```

Keyword lists are important because they have three special characteristics:

- Keys must be atoms.
- Keys are ordered, as specified by the developer.
- Keys can be given more than once.

For example, we use the fact that keys can be repeated when importing functions in Elixir:

```
iex> import String, only: [split: 1, split: 2]
String
iex> split("hello world")
["hello", "world"]
```

In the example above, we imported both `split/1` and `split/2` from the `String` module, allowing us to invoke them without typing the module name. We used a keyword list to list the functions to import.

Since keyword lists are lists, we can use all operations available to lists. For example, we can use `++` to add new values to a keyword list:

```
iex> list = [a: 1, b: 2]
[a: 1, b: 2]
iex> list ++ [c: 3]
[a: 1, b: 2, c: 3]
iex> [a: 0] ++ list
[a: 0, a: 1, b: 2]
```

You can read the value of a keyword list using the brackets syntax, which will return the value of the first matching key. This is also known as the access syntax, as it is defined by the `Access` module:

```
iex> list[:a]
1
iex> list[:b]
2
```

Although we can pattern match on keyword lists, it is not done in practice since pattern matching on lists requires the number of items and their order to match:

```
iex> [a: a] = [a: 1]
[a: 1]
iex> a
1
iex> [a: a] = [a: 1, b: 2]
** (MatchError) no match of right hand side value: [a: 1, b:
2]
iex> [b: b, a: a] = [a: 1, b: 2]
** (MatchError) no match of right hand side value: [a: 1, b:
2]
```

Furthermore, given keyword lists are often used as optional arguments, they are used in situations where not all keys may be present, which would make it impossible to match on them. In a nutshell, do not pattern match on keyword lists.

In order to manipulate keyword lists, Elixir provides the `Keyword` module. Remember, though, keyword lists are simply lists, and as such they provide the same linear performance characteristics: the longer the list, the longer it will take to find a key, to count the number of items, and so on. If you need to store a large amount of keys in a key-value data structure, Elixir offers maps, which we will soon learn.

`do`-blocks and keywords

As we have seen, keywords are mostly used in the language to pass optional values. In fact, we have used keywords in earlier chapters. Let's look at the `if/2` macro:

```
iex> if true do
...>   "This will be seen"
...> else
...>   "This won't"
...> end
"This will be seen"
```

In the example above, the `do` and `else` blocks make up a keyword list. They are nothing more than a syntax convenience on top of keyword lists. We can rewrite the above to:

```
iex> if(true, do: "This will be seen", else: "This won't")
"This will be seen"
```

Pay close attention to both syntaxes. The second example uses keyword lists, exactly as in the `String.split/3` example, so we separate each key-value pair with commas and each key is followed by `:`. In the `do`-blocks, we use bare words, such as `do`, `else`, and `end`, and separate them by a newline. They are useful precisely when writing blocks of code. Most of the time, you will use the block syntax, but it is good to know they are equivalent.

The fact the block syntax is equivalent to keywords means we only need few data structures to represent the language, keeping it simple overall. We will come back to this topic when discussing optional syntax and meta-programming.

With this out of the way, let's talk about maps.

Maps as key-value pairs

Whenever you need to store key-value pairs, maps are the "go to" data structure in Elixir. A map is created using the `%{}` syntax:

```
iex> map = %{:a => 1, 2 => :b}
%{2 => :b, :a => 1}
iex> map[:a]
1
iex> map[2]
:b
iex> map[:c]
nil
```

Compared to keyword lists, we can already see two differences:

- Maps allow any value as a key.
- Maps have their own internal ordering, which is not guaranteed to be the same across different maps, even if they have the same keys

In contrast to keyword lists, maps are very useful with pattern matching. When a map is used in a pattern, it will always match on a subset of the given value:

```
iex> %{} = %{:a => 1, 2 => :b}
%{2 => :b, :a => 1}
iex> %{:a => a} = %{:a => 1, 2 => :b}
%{2 => :b, :a => 1}
iex> a
1
iex> %{:c => c} = %{:a => 1, 2 => :b}
** (MatchError) no match of right hand side value: %{2 =>
:b, :a => 1}
```

As shown above, a map matches as long as the keys in the pattern exist in the given map. Therefore, an empty map matches all maps.

The `Map` module provides a very similar API to the `Keyword` module with convenience functions to add, remove, and update maps keys:

```
iex> Map.get(%{:a => 1, 2 => :b}, :a)
1
iex> Map.put(%{:a => 1, 2 => :b}, :c, 3)
%{2 => :b, :a => 1, :c => 3}
iex> Map.to_list(%{:a => 1, 2 => :b})
[{2, :b}, {:a, 1}]
```

Maps of predefined keys

In the previous section, we have used maps as a key-value data structure where keys can be added or removed at any time. However, it is also common to create maps with a predefined set of keys. Their values may be updated, but new keys are never added nor removed. This is useful when we know the shape of the data we are working with and, if we get a different key, it likely means a mistake was done elsewhere. In such cases, the keys are most often atoms:

```
iex> map = %{:name => "John", :age => 23}
%{name: "John", age: 23}
```

As you can see from the printed result above, Elixir also allows you to write maps of atom keys using the same `key: value` syntax as keyword lists:

```
iex> map = %{name: "John", age: 23}
%{name: "John", age: 23}
```

When a key is an atom, we can also access them using the `map.key` syntax:

```
iex> map.name
"John"
iex> map.agee
** (KeyError) key :agee not found in: %{name: "John", age:
23}
```

There is also syntax for updating keys, which also raises if the key has not yet been defined:

```
iex> %{map | name: "Mary"}
%{name: "Mary", age: 23}
iex> %{map | agee: 27}
** (KeyError) key :agee not found in: %{name: "John", age:
23}
```

These operations have one large benefit in that they raise if the key does not exist in the map and the compiler may even detect and warn when possible. This makes them useful to get quick feedback and spot bugs and typos early on. This is also the syntax used to power another Elixir feature called "Structs", which we will learn later on.

Elixir developers typically prefer to use the `map.key` syntax and pattern matching instead of the functions in the `Map` module when working with maps because they lead to an assertive style of programming. This blog post by José Valim provides insight and examples on how you get more concise and faster software by writing assertive code in Elixir.

In a further chapter you'll learn about "Structs", which further enforce the idea of a map with predefined keys.

Nested data structures

Often we will have maps inside maps, or even keywords lists inside maps, and so forth. Elixir provides conveniences for manipulating nested data structures via the `get_in/1`, `put_in/2`, `update_in/2`, and other macros giving the same conveniences you would find in imperative languages while keeping the immutable properties of the language.

Imagine you have the following structure:

```
iex> users = [
  john: %{name: "John", age: 27, languages: ["Erlang",
"Ruby", "Elixir"]},
  mary: %{name: "Mary", age: 29, languages: ["Elixir", "F#",
"Clojure"]}
]
[
  john: %{age: 27, languages: ["Erlang", "Ruby", "Elixir"],
name: "John"},
  mary: %{age: 29, languages: ["Elixir", "F#", "Clojure"],
name: "Mary"}
]
```

We have a keyword list of users where each value is a map containing the name, age and a list of programming languages each user likes. If we wanted to access the age for john, we could write:

```
iex> users[:john].age
27
```

It happens we can also use this same syntax for updating the value:

```
iex> users = put_in(users[:john].age, 31)
[
  john: %{age: 31, languages: ["Erlang", "Ruby", "Elixir"],
```

```
name: "John"},
  mary: %{age: 29, languages: ["Elixir", "F#", "Clojure"],
name: "Mary"}
]
```

The `update_in/2` macro is similar but allows us to pass a function that controls how the value changes. For example, let's remove "Clojure" from Mary's list of languages:

```
iex> users = update_in(users[:mary].languages, fn languages
-> List.delete(languages, "Clojure") end)
[
  john: %{age: 31, languages: ["Erlang", "Ruby", "Elixir"],
name: "John"},
  mary: %{age: 29, languages: ["Elixir", "F#"], name:
"Mary"}
]
```

Summary

There are two different data structures for working with key-value stores in Elixir. Alongside the `Access` module and pattern matching, they provide a rich set of tools for manipulating complex, potentially nested, data structures.

As we conclude this chapter, remember that you should:

- Use keyword lists for passing optional values to functions
- Use maps for general key-value data structures
- Use maps when working with data that has a predefined set of keys

Now let's talk about modules and functions.

Modules and functions

In Elixir we group several functions into modules. We've already used many different modules in the previous chapters, such as the `String` module:

```
iex> String.length("hello")
5
```

In order to create our own modules in Elixir, we use the `defmodule` macro. The first letter of a module name (an alias, as described further down) must be in uppercase. We use the `def` macro to define functions in that module. The first letter of every function must be in lowercase (or underscore):

```
iex> defmodule Math do
...>   def sum(a, b) do
...>     a + b
...>   end
...> end

iex> Math.sum(1, 2)
3
```

In this chapter we will define our own modules, with different levels of complexity. As our examples get longer in size, it can be tricky to type them all in the shell, so we will resort more frequently to scripting.

Scripting

Elixir has two file extensions `.ex` (Elixir) and `.exs` (Elixir scripts). Elixir treats both files exactly the same way, the only difference is in intention.

`.ex` files are meant to be compiled while `.exs` files are used for scripting.

Let's create a file named `math.exs`:

```
defmodule Math do
  def sum(a, b) do
    a + b
  end
end

IO.puts Math.sum(1, 2)
```

And execute it as:

```
$ elixir math.exs
```

You can also load the file within `iex` by running:

```
$ iex math.exs
```

And then have direct access to the `Math` module.

Function definition

Inside a module, we can define functions with `def/2` and private functions with `defp/2`. A function defined with `def/2` can be invoked from other modules while a private function can only be invoked locally.

```
defmodule Math do
  def sum(a, b) do
    do_sum(a, b)
```

```
  end

  defp do_sum(a, b) do
    a + b
  end
end

IO.puts Math.sum(1, 2)    #=> 3
IO.puts Math.do_sum(1, 2) #=> ** (UndefinedFunctionError)
```

Function declarations also support guards and multiple clauses. If a function has several clauses, Elixir will try each clause until it finds one that matches. Here is an implementation of a function that checks if the given number is zero or not:

```
defmodule Math do
  def zero?(0) do
    true
  end

  def zero?(x) when is_integer(x) do
    false
  end
end

IO.puts Math.zero?(0)         #=> true
IO.puts Math.zero?(1)         #=> false
IO.puts Math.zero?([1, 2, 3]) #=> ** (FunctionClauseError)
IO.puts Math.zero?(0.0)       #=> ** (FunctionClauseError)
```

The trailing question mark in `zero?` means that this function returns a boolean. To learn more about the naming conventions for modules, function names, variables and more in Elixir, see Naming Conventions.

Giving an argument that does not match any of the clauses raises an error.

Similar to constructs like `if`, function definitions support both `do:` and `do`-block syntax, as we learned in the previous chapter. For example, we can edit `math.exs` to look like this:

```
defmodule Math do
  def zero?(0), do: true
  def zero?(x) when is_integer(x), do: false
end
```

And it will provide the same behavior. You may use `do:` for one-liners but always use `do`-blocks for functions spanning multiple lines. If you prefer to be consistent, you can use `do`-blocks throughout your codebase.

Default arguments

Function definitions in Elixir also support default arguments:

```
defmodule Concat do
  def join(a, b, sep \\ " ") do
    a <> sep <> b
  end
end

IO.puts(Concat.join("Hello", "world"))      #=> Hello world
IO.puts(Concat.join("Hello", "world", "_")) #=> Hello_world
```

Any expression is allowed to serve as a default value, but it won't be evaluated during the function definition. Every time the function is invoked and any of its default values have to be used, the expression for that default value will be evaluated:

```
defmodule DefaultTest do
  def dowork(x \\ "hello") do
    x
  end
end


iex> DefaultTest.dowork()
"hello"
iex> DefaultTest.dowork(123)
123
iex> DefaultTest.dowork()
"hello"
```

If a function with default values has multiple clauses, it is required to create a function head (a function definition without a body) for declaring defaults:

```
defmodule Concat do
  # A function head declaring defaults
  def join(a, b, sep \\ " ")

  # The separator is unused in this case, so we prefix it
with underscore
  def join(a, b, _sep) when b == "" do
    a
  end

  def join(a, b, sep) do
    a <> sep <> b
  end
end

IO.puts(Concat.join("Hello", ""))           #=> Hello
```

```
IO.puts(Concat.join("Hello", "world"))      #=> Hello world
IO.puts(Concat.join("Hello", "world", "_")) #=> Hello_world
```

Function heads cannot have patterns nor guards. They may only define the argument names and their default values.

Understanding Aliases

An alias in Elixir is a capitalized identifier (like `String`, `Keyword`, etc) which is converted to an atom during compilation. For instance, the `String` alias translates by default to the atom `:"Elixir.String"`:

```
iex> is_atom(String)
true
iex> to_string(String)
"Elixir.String"
iex> :"Elixir.String" == String
true
```

By using the `alias/2` directive, we are changing the atom the alias expands to.

Aliases expand to atoms because in the Erlang Virtual Machine (and consequently Elixir) modules are always represented by atoms. By namespacing those atoms, Elixir modules avoid conflicting with existing Erlang modules.

```
iex> List.flatten([1, [2], 3])
[1, 2, 3]
iex> :"Elixir.List".flatten([1, [2], 3])
[1, 2, 3]
```

That's the mechanism we use to call Erlang modules:

```
iex> :lists.flatten([1, [2], 3])
[1, 2, 3]
```

Module nesting

Now that we have talked about aliases, we can talk about nesting and how it works in Elixir. Consider the following example:

```
defmodule Foo do
  defmodule Bar do
  end
end
```

The example above will define two modules: `Foo` and `Foo.Bar`. The second can be accessed as `Bar` inside `Foo` as long as they are in the same lexical scope.

> **Module names are isolated**
>
> You don't have to define the `Foo` module before defining the `Foo.Bar` module, as they are effectively independent.

If, later, the `Bar` module is moved outside the `Foo` module definition, it must be referenced by its full name (`Foo.Bar`) or an alias must be set using the `alias` directive, which we will learn in the next chapter:

```
defmodule Foo.Bar do
end
```

```elixir
defmodule Foo do
  alias Foo.Bar
  # Can still access it as `Bar`
end
```

alias, require, import, and use

In order to facilitate software reuse, Elixir provides three directives (`alias` , `require` , and `import`) plus a macro called `use` summarized below:

```
# Alias the module so it can be called as Bar instead of
Foo.Bar
alias Foo.Bar, as: Bar

# Require the module in order to use its macros
require Foo

# Import functions from Foo so they can be called without
the `Foo.` prefix
import Foo

# Invokes the custom code defined in Foo as an extension
point
use Foo
```

We are going to explore them in detail now. Keep in mind the first three are called directives because they have *lexical scope*, while `use` is a common extension point that allows the used module to inject code.

alias

`alias` allows you to set up aliases for any given module name.

Imagine a module uses a specialized list implemented in `Math.List` . The `alias` directive allows referring to `Math.List` just as `List` within the module definition:

```
defmodule Stats do
  alias Math.List, as: List
  # In the remaining module definition List expands to
Math.List.
end
```

The original `List` can still be accessed within `Stats` by the fully-qualified name `Elixir.List`.

> All modules defined in Elixir are defined inside the main `Elixir` namespace, such as `Elixir.String`. However, for convenience, you can omit "Elixir." when referencing them.

Aliases are frequently used to define shortcuts. In fact, calling `alias` without an `:as` option sets the alias automatically to the last part of the module name, for example:

```
alias Math.List
```

Is the same as:

```
alias Math.List, as: List
```

Note that `alias` is *lexically scoped*, which allows you to set aliases inside specific functions:

```
defmodule Math do
  def plus(a, b) do
    alias Math.List
    # ...
  end
```

```
  def minus(a, b) do
    # ...
  end
end
```

In the example above, since we are invoking `alias` inside the function `plus/2`, the alias will be valid only inside the function `plus/2`. `minus/2` won't be affected at all.

require

Elixir provides macros as a mechanism for meta-programming (writing code that generates code). Macros are expanded at compile time.

Public functions in modules are globally available, but in order to use macros, you need to opt-in by requiring the module they are defined in.

```
iex> Integer.is_odd(3)
** (UndefinedFunctionError) function Integer.is_odd/1 is
undefined or private. However, there is a macro with the
same name and arity. Be sure to require Integer if you
intend to invoke this macro
    (elixir) Integer.is_odd(3)
iex> require Integer
Integer
iex> Integer.is_odd(3)
true
```

In Elixir, `Integer.is_odd/1` is defined as a macro so that it can be used as a guard. This means that, in order to invoke `Integer.is_odd/1`, we need to first require the `Integer` module.

Note that like the `alias` directive, `require` is also lexically scoped. We will talk more about macros in a later chapter.

import

We use `import` whenever we want to access functions or macros from other modules without using the fully-qualified name. Note we can only import public functions, as private functions are never accessible externally.

For example, if we want to use the `duplicate/2` function from the `List` module several times, we can import it:

```
iex> import List, only: [duplicate: 2]
List
iex> duplicate(:ok, 3)
[:ok, :ok, :ok]
```

We imported only the function `duplicate` (with arity 2) from `List`. Although `:only` is optional, its usage is recommended in order to avoid importing all the functions of a given module inside the current scope. `:except` could also be given as an option in order to import everything in a module except a list of functions.

Note that `import` is *lexically scoped* too. This means that we can import specific macros or functions inside function definitions:

```
defmodule Math do
  def some_function do
    import List, only: [duplicate: 2]
    duplicate(:ok, 10)
  end
end
```

In the example above, the imported `List.duplicate/2` is only visible within that specific function. `duplicate/2` won't be available in any other function in that module (or any other module for that matter).

While `import`s can be useful for frameworks and libraries to build abstractions, developers should generally prefer `alias` to `import` on their own codebases, as aliases make the origin of the function being invoked clearer.

use

The `use` macro is frequently used as an extension point. This means that, when you `use` a module `FooBar`, you allow that module to inject *any* code in the current module, such as importing itself or other modules, defining new functions, setting a module state, etc.

For example, in order to write tests using the ExUnit framework, a developer should use the `ExUnit.Case` module:

```
defmodule AssertionTest do
  use ExUnit.Case, async: true

  test "always pass" do
    assert true
  end
end
```

Behind the scenes, `use` requires the given module and then calls the `__using__/1` callback on it allowing the module to inject some code into the current context. Some modules (for example, the above `ExUnit.Case`, but also `Supervisor` and `GenServer`) use this mechanism to populate your module with some basic behaviour, which your module is intended to override or complete.

Generally speaking, the following module:

```
defmodule Example do
  use Feature, option: :value
end
```

is compiled into

```
defmodule Example do
  require Feature
  Feature.__using__(option: :value)
end
```

Since `use` allows any code to run, we can't really know the side-effects of using a module without reading its documentation. Therefore use this function with care and only if strictly required. Don't use `use` where an `import` or `alias` would do.

Multi alias/import/require/use

It is possible to `alias`, `import`, `require`, or `use` multiple modules at once. This is particularly useful once we start nesting modules, which is very common when building Elixir applications. For example, imagine you have an application where all modules are nested under `MyApp`, you can alias the modules `MyApp.Foo`, `MyApp.Bar` and `MyApp.Baz` at once as follows:

```
alias MyApp.{Foo, Bar, Baz}
```

With this, we have finished our tour of Elixir modules.

Module attributes

Module attributes in Elixir serve three purposes:

1. as module and function annotations
2. as temporary module storage to be used during compilation
3. as compile-time constants

Let's check these examples.

As annotations

Elixir brings the concept of module attributes from Erlang. For example:

```
defmodule MyServer do
  @moduledoc "My server code."
end
```

In the example above, we are defining the module documentation by using the module attribute syntax. Elixir has a handful of reserved attributes. Here are a few of them, the most commonly used ones:

- `@moduledoc` — provides documentation for the current module.
- `@doc` — provides documentation for the function or macro that follows the attribute.
- `@spec` — provides a typespec for the function that follows the attribute.
- `@behaviour` — (notice the British spelling) used for specifying an OTP or user-defined behaviour.

`@moduledoc` and `@doc` are by far the most used attributes, and we expect you to use them a lot. Elixir treats documentation as first-class and provides

many functions to access documentation. We will cover them in their own chapter.

Documentation is only accessible from compiled modules. So in order to give it a try, let's once again define the `Math` module, but this time within a file named `math.ex`:

```
defmodule Math do
  @moduledoc """
  Provides math-related functions.

  ## Examples

      iex> Math.sum(1, 2)
      3

  """

  @doc """
  Calculates the sum of two numbers.
  """
  def sum(a, b), do: a + b
end
```

Elixir promotes the use of Markdown with heredocs to write readable documentation. Heredocs are multi-line strings, they start and end with triple double-quotes, keeping the formatting of the inner text.

Now let's compile it. Start `iex` and then invoke the `c/2` helper:

```
iex> c("math.ex", ".")
[Math]
```

And now we can access them:

```
iex> h Math # Docs for module Math
...
iex> h Math.sum # Docs for the sum function
...
```

When we compiled the module, you may have noticed Elixir created a `Elixir.Math.beam` file. That's the bytecode for the module and that's where the documentation is stored.

In our day to day, Elixir developers use the `Mix` build tool to compile code and projects like ExDoc to generate HTML and EPUB pages from the documentation.

Take a look at the docs for `Module` for a complete list of supported attributes.

As temporary storage

So far, we have seen how to define attributes, but how can we read them? Let's see an example:

```
defmodule MyServer do
  @service URI.parse("https://example.com")
  IO.inspect(@service)
end
```

> **Newlines**
>
> Do not add a newline between the attribute and its value, otherwise Elixir will assume you are reading the value, rather than setting it.

Trying to access an attribute that was not defined will print a warning:

```
defmodule MyServer do
  @unknown
end
warning: undefined module attribute @unknown, please remove
access to @unknown or explicitly set it before access
```

Attributes can also be read inside functions:

```
defmodule MyApp.Status do
  @service URI.parse("https://example.com")
  def status(email) do
    SomeHttpClient.get(@service)
  end
end
```

The module attribute is defined at compilation time and its *return value*, not the function call itself, is what will be substituted in for the attribute. So the above will effectively compile to this:

```
defmodule MyApp.Status do
  def status(email) do
    SomeHttpClient.get(%URI{
      authority: "example.com",
      host: "example.com",
      port: 443,
      scheme: "https"
    })
  end
end
```

This can be useful for pre-computing values and then injecting its results into the module. This is what we mean by temporary storage: after the module is compiled, the module attribute is discarded, except for the functions that have read the attribute. Note you cannot invoke functions defined in the same module as part of the attribute itself, as those functions have not yet been defined.

Every time we read an attribute inside a function, Elixir takes a snapshot of its current value. Therefore if you read the same attribute multiple times inside multiple functions, you end-up increasing compilation times as Elixir now has to compile every snapshot. Generally speaking, you want to avoid reading the same attribute multiple times and instead move it to function. For example, instead of this:

```
def some_function, do: do_something_with(@example)
def another_function, do: do_something_else_with(@example)
```

Prefer this:

```
def some_function, do: do_something_with(example())
def another_function, do: do_something_else_with(example())
defp example, do: @example
```

As compile-time constants

Module attributes may also be useful as compile-time constants. Generally speaking, functions themselves are sufficient for the role of constants in a codebase. For example, instead of defining:

```
@hours_in_a_day 24
```

You should prefer:

```
defp hours_in_a_day(), do: 24
```

You may even define a public function if it needs to be shared across modules. It is common in many projects to have a module called `MyApp.Constants` that defines all constants used throughout the codebase.

You can even have composite data structures as constants, as long as they are made exclusively of other data types (no function calls, no operators, and no other expressions). For example, you may specify a system configuration constant as follows:

```
defp system_config(), do: %{timezone: "Etc/UTC", locale:
"pt-BR"}
```

Given data structures in Elixir are immutable, only a single instance of the data structure above is allocated and shared across all functions calls, as long as it doesn't have any executable expression.

The use case for module attributes arise when you need to do some work at compile-time and then inject its results inside a function. A common scenario is module attributes inside patterns and guards (as an alternative to `defguard/1`), since they only support a limited set of expressions:

```
# Inside pattern
@default_timezone "Etc/UTC"
def shift(@default_timezone), do: ...

# Inside guards
@time_periods [:am, :pm]
def shift(time, period) when period in @time_periods, do:
...
```

Module attributes as constants and as temporary storage are most often used together: the module attribute is used to compute and store an expensive value, and then exposed as constant from that module.

Going further

Libraries and frameworks can leverage module attributes to provide custom annotations. To see an example, look no further than Elixir's unit test framework called `ExUnit`. `ExUnit` uses module attributes for multiple different purposes:

```
defmodule MyTest do
  use ExUnit.Case, async: true

  @tag :external
  @tag os: :unix
  test "contacts external service" do
    # ...
  end
end
```

In the example above, `ExUnit` stores the value of `async: true` in a module attribute to change how the module is compiled. Tags also work as annotations and they can be supplied multiple times, thanks to Elixir's ability to accumulate attributes. Then you can use tags to setup and filter tests, such as avoiding executing Unix specific tests while running your test suite on Windows.

To fully understand how `ExUnit` works, we'd need macros, so we will revisit this pattern in the Meta-programming guide and learn how to use module attributes as storage for custom annotations.

In the next chapters, we'll explore structs and protocols before moving to exception handling and other constructs like sigils and comprehensions.

Structs

We learned about maps in earlier chapters:

```
iex> map = %{a: 1, b: 2}
%{a: 1, b: 2}
iex> map[:a]
1
iex> %{map | a: 3}
%{a: 3, b: 2}
```

Structs are extensions built on top of maps that provide compile-time checks and default values.

Defining structs

To define a struct, the `defstruct/1` construct is used:

```
iex> defmodule User do
...>   defstruct name: "John", age: 27
...> end
```

The keyword list used with `defstruct` defines what fields the struct will have along with their default values. Structs take the name of the module they're defined in. In the example above, we defined a struct named `User`.

We can now create `User` structs by using a syntax similar to the one used to create maps:

```
iex> %User{}
%User{age: 27, name: "John"}
iex> %User{name: "Jane"}
%User{age: 27, name: "Jane"}
```

Structs provide *compile-time* guarantees that only the fields defined through `defstruct` will be allowed to exist in a struct:

```
iex> %User{oops: :field}
** (KeyError) key :oops not found expanding struct:
User.__struct__/1
```

Accessing and updating structs

Structs share the same syntax for accessing and updating fields as maps of fixed keys:

```
iex> john = %User{}
%User{age: 27, name: "John"}
iex> john.name
"John"
iex> jane = %{john | name: "Jane"}
%User{age: 27, name: "Jane"}
iex> %{jane | oops: :field}
** (KeyError) key :oops not found in: %User{age: 27, name:
"Jane"}
```

When using the update syntax (`|`), Elixir is aware that no new keys will be added to the struct, allowing the maps underneath to share their structure in memory. In the example above, both `john` and `jane` share the same key structure in memory.

Structs can also be used in pattern matching, both for matching on the value of specific keys as well as for ensuring that the matching value is a struct of the same type as the matched value.

```
iex> %User{name: name} = john
%User{age: 27, name: "John"}
iex> name
"John"
iex> %User{} = %{}
** (MatchError) no match of right hand side value: %{}
```

For more details on creating, updating, and pattern matching structs, see the documentation for `%/2`.

Dynamic struct updates

When you need to update structs with data from keyword lists or maps, use `Kernel.struct!/2`:

```
iex> john = %User{name: "John", age: 27}
%User{age: 27, name: "John"}
iex> updates = [name: "Jane", age: 30]
[name: "Jane", age: 30]
iex> struct!(john, updates)
%User{age: 30, name: "Jane"}
```

`struct!/2` will raise an error if you try to set invalid fields:

```
iex> struct!(john, invalid: "field")
** (KeyError) key :invalid not found in: %User{age: 27,
name: "John"}
```

Use the map update syntax (`%{john | name: "Jane"}`) when you know the exact fields at compile time. Always use `struct!/2` instead of `Map` functions to preserve struct integrity.

Structs are bare maps underneath

Structs are simply maps with a "special" field named `__struct__` that holds the name of the struct:

```
iex> is_map(john)
true
iex> john.__struct__
User
```

However, structs do not inherit any of the built-in features that maps do. For example, you can neither enumerate nor access a struct:

```
iex> john = %User{}
%User{age: 27, name: "John"}
iex> john[:name]
** (UndefinedFunctionError) function User.fetch/2 is
undefined (User does not implement the Access behaviour)
iex> Enum.each(john, fn {field, value} -> IO.puts(value)
end)
** (Protocol.UndefinedError) protocol Enumerable not
implemented for %User{age: 27, name: "John"} of type User (a
struct)
```

As we will learn in future chapters, Elixir does allow developers to attach custom behaviours to structs based on their names, enabling custom data enumeration, pretty printing, and more. This dispatch mechanism is called protocols and enables extensible data polymorphism in Elixir.

Default values and required keys

If you don't specify a default key value when defining a struct, `nil` will be assumed:

```
iex> defmodule Product do
...>   defstruct [:name]
...> end
iex> %Product{}
%Product{name: nil}
```

You can define a structure combining both fields with explicit default values, and implicit `nil` values. In this case you must first specify the fields which implicitly default to `nil`:

```
iex> defmodule User do
...>   defstruct [:email, name: "John", age: 27]
...> end
iex> %User{}
%User{age: 27, email: nil, name: "John"}
```

Doing it in reverse order will raise a syntax error:

```
iex> defmodule User do
...>   defstruct [name: "John", age: 27, :email]
...> end
** (SyntaxError) iex:107: unexpected expression after
keyword list. Keyword lists must always come last in lists
and maps.
```

You can also enforce that certain keys have to be specified when creating the struct via the `@enforce_keys` module attribute:

```
iex> defmodule Car do
...>   @enforce_keys [:make]
...>   defstruct [:model, :make]
...> end
iex> %Car{}
** (ArgumentError) the following keys must also be given
when building struct Car: [:make]
    expanding struct: Car.__struct__/1
```

Enforcing keys provides a simple compile-time guarantee to aid developers when building structs. It is not enforced on updates and it does not provide any sort of value-validation.

Recursion

Elixir does not provide loop constructs. Instead we leverage recursion and high-level functions for working with collections. This chapter will explore the former.

Loops through recursion

Due to immutability, loops in Elixir (as in any functional programming language) are written differently from imperative languages. For example, in an imperative language like C, one would write:

```
for(i = 0; i < sizeof(array); i++) {
  array[i] = array[i] * 2;
}
```

In the example above, we are mutating both the array and the variable `i`. However, data structures in Elixir are immutable. For this reason, functional languages rely on recursion: a function is called recursively until a condition is reached that stops the recursive action from continuing. No data is mutated in this process. Consider the example below that prints a string an arbitrary number of times:

```
defmodule Recursion do
  def print_multiple_times(msg, n) when n > 0 do
    IO.puts(msg)
    print_multiple_times(msg, n - 1)
  end

  def print_multiple_times(_msg, 0) do
    :ok
```

```
  end
end

Recursion.print_multiple_times("Hello!", 3)
# Hello!
# Hello!
# Hello!
:ok
```

Similar to `case`, a function may have many clauses. A particular clause is executed when the arguments passed to the function match the clause's argument patterns and its guards evaluate to `true`.

When `print_multiple_times/2` is initially called in the example above, the argument `n` is equal to `3`.

The first clause has a guard which says "use this definition if and only if `n` is more than `0`". Since this is the case, it prints the `msg` and then calls itself passing `n - 1 (2)` as the second argument.

Now we execute the same function again, starting from the first clause. Given the second argument, `n`, is still more than 0, we print the message and call ourselves once more, now with the second argument set to `1`. Then we print the message one last time and call `print_multiple_times("Hello!", 0)`, starting from the top once again.

When the second argument is zero, the guard `n > 0` evaluates to false, and the first function clause won't execute. Elixir then proceeds to try the next function clause, which explicitly matches on the case where `n` is `0`. This clause, also known as the termination clause, ignores the message argument by assigning it to the `_msg` variable and returns the atom `:ok`.

Finally, if you pass an argument that does not match any clause, Elixir raises a `FunctionClauseError`:

```
iex> Recursion.print_multiple_times("Hello!", -1)
** (FunctionClauseError) no function clause matching in
Recursion.print_multiple_times/2

    The following arguments were given to
Recursion.print_multiple_times/2:

        # 1
        "Hello!"

        # 2
        -1

    iex:1: Recursion.print_multiple_times/2
```

Reduce and map algorithms

Let's now see how we can use the power of recursion to sum a list of numbers:

```
defmodule Math do
  def sum_list([head | tail], accumulator) do
    sum_list(tail, head + accumulator)
  end

  def sum_list([], accumulator) do
    accumulator
  end
end

IO.puts Math.sum_list([1, 2, 3], 0) #=> 6
```

We invoke `sum_list` with the list `[1, 2, 3]` and the initial value `0` as arguments. We will try each clause until we find one that matches according to the pattern matching rules. In this case, the list `[1, 2, 3]` matches against `[head | tail]` which binds `head` to `1` and `tail` to `[2, 3]`; `accumulator` is set to `0`.

Then, we add the head of the list to the accumulator `head + accumulator` and call `sum_list` again, recursively, passing the tail of the list as its first argument. The tail will once again match `[head | tail]` until the list is empty, as seen below:

```
sum_list([1, 2, 3], 0)
sum_list([2, 3], 1)
sum_list([3], 3)
sum_list([], 6)
```

When the list is empty, it will match the final clause which returns the final result of `6`.

The process of taking a list and *reducing* it down to one value is known as a *reduce algorithm* and is central to functional programming.

What if we instead want to double all of the values in our list?

```
defmodule Math do
  def double_each([head | tail]) do
    [head * 2 | double_each(tail)]
  end

  def double_each([]) do
    []
  end
end
```

```
Math.double_each([1, 2, 3]) #=> [2, 4, 6]
```

Here we have used recursion to traverse a list, doubling each element and returning a new list. The process of taking a list and *mapping* over it is known as a *map algorithm*.

Recursion and tail call optimization are an important part of Elixir and are commonly used to create loops. However, when programming in Elixir you will rarely use recursion as above to manipulate lists.

The `Enum` module, which we're going to see in the next chapter already provides many conveniences for working with lists. For instance, the examples above could be written as:

```
iex> Enum.reduce([1, 2, 3], 0, fn x, acc -> x + acc end)
6
iex> Enum.map([1, 2, 3], fn x -> x * 2 end)
[2, 4, 6]
```

Or, using the capture syntax:

```
iex> Enum.reduce([1, 2, 3], 0, &+/2)
6
iex> Enum.map([1, 2, 3], &(&1 * 2))
[2, 4, 6]
```

Let's take a deeper look at `Enumerable` and, while we're at it, its lazy counterpart, `Stream`.

Enumerables and Streams

While Elixir allows us to write recursive code, most operations we perform on collections is done with the help of the `Enum` and `Stream` modules. Let's learn how.

Enumerables

Elixir provides the concept of enumerables and the `Enum` module to work with them. We have already learned two enumerables: lists and maps.

```
iex> Enum.map([1, 2, 3], fn x -> x * 2 end)
[2, 4, 6]
iex> Enum.map(%{1 => 2, 3 => 4}, fn {k, v} -> k * v end)
[2, 12]
```

The `Enum` module provides a huge range of functions to transform, sort, group, filter and retrieve items from enumerables. It is one of the modules developers use frequently in their Elixir code. For a general overview of all functions in the `Enum` module, see the `Enum` cheatsheet.

Elixir also provides ranges (see `Range`), which are also enumerable:

```
iex> Enum.map(1..3, fn x -> x * 2 end)
[2, 4, 6]
iex> Enum.reduce(1..3, 0, &+/2)
6
```

The functions in the `Enum` module are limited to, as the name says, enumerating values in data structures. For specific operations, like inserting

and updating particular elements, you may need to reach for modules specific to the data type. For example, if you want to insert an element at a given position in a list, you should use the `List.insert_at/3` function, as it would make little sense to insert a value into, for example, a range.

We say the functions in the `Enum` module are polymorphic because they can work with diverse data types. In particular, the functions in the `Enum` module can work with any data type that implements the `Enumerable` protocol. We are going to discuss Protocols in a later chapter, for now we are going to move on to a specific kind of enumerable called a stream.

Eager vs Lazy

All the functions in the `Enum` module are eager. Many functions expect an enumerable and return a list back:

```
iex> odd? = fn x -> rem(x, 2) != 0 end
#Function<6.80484245/1 in :erl_eval.expr/5>
iex> Enum.filter(1..3, odd?)
[1, 3]
```

This means that when performing multiple operations with `Enum`, each operation is going to generate an intermediate list until we reach the result:

```
iex> 1..100_000 |> Enum.map(&(&1 * 3)) |> Enum.filter(odd?)
|> Enum.sum()
7500000000
```

The example above has a pipeline of operations. We start with a range and then multiply each element in the range by 3. This first operation will now create and return a list with `100_000` items. Then we keep all odd elements

from the list, generating a new list, now with `50_000` items, and then we sum all entries.

The pipe operator

The `|>` symbol used in the snippet above is the **pipe operator**: it takes the output from the expression on its left side and passes it as the first argument to the function call on its right side. Its purpose is to highlight the data being transformed by a series of functions. To see how it can make the code cleaner, have a look at the example above rewritten without using the `|>` operator:

```
iex> Enum.sum(Enum.filter(Enum.map(1..100_000, &(&1 * 3)),
odd?))
7500000000
```

Find more about the pipe operator by reading its documentation.

Streams

As an alternative to `Enum`, Elixir provides the `Stream` module which supports lazy operations:

```
iex> 1..100_000 |> Stream.map(&(&1 * 3)) |>
Stream.filter(odd?) |> Enum.sum()
7500000000
```

Streams are lazy, composable enumerables.

In the example above, `1..100_000 |> Stream.map(&(&1 * 3))` returns a data type, an actual stream, that represents the `map` computation over the range `1..100_000`:

```
iex> 1..100_000 |> Stream.map(&(&1 * 3))
#Stream<[enum: 1..100000, funs: [#Function<34.16982430/1 in
Stream.map/2>]]>
```

Furthermore, they are composable because we can pipe many stream operations:

```
iex> 1..100_000 |> Stream.map(&(&1 * 3)) |>
Stream.filter(odd?)
#Stream<[enum: 1..100000, funs: [...]]>
```

Instead of generating intermediate lists, streams build a series of computations that are invoked only when we pass the underlying stream to the `Enum` module. Streams are useful when working with large, *possibly infinite*, collections.

Many functions in the `Stream` module accept any enumerable as an argument and return a stream as a result. It also provides functions for creating streams. For example, `Stream.cycle/1` can be used to create a stream that cycles a given enumerable infinitely. Be careful to not call a function like `Enum.map/2` on such streams, as they would cycle forever:

```
iex> stream = Stream.cycle([1, 2, 3])
#Function<15.16982430/2 in Stream.unfold/2>
iex> Enum.take(stream, 10)
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1]
```

Another interesting function is `Stream.resource/3` which can be used to wrap around resources, guaranteeing they are opened right before enumeration and closed afterwards, even in the case of failures. For example, `File.stream!/1` builds on top of `Stream.resource/3` to stream files:

```
iex> "path/to/file" |> File.stream!() |> Enum.take(10)
```

The example above will fetch the first 10 lines of the file you have selected. This means streams can be very useful for handling large files or even slow resources like network resources.

The `Enum` and `Stream` modules provide a wide range of functions, but you don't have to know all of them by heart. Familiarize yourself with `Enum.map/2`, `Enum.reduce/3` and other functions with either `map` or `reduce` in their names, and you will naturally build an intuition around the most important use cases. You may also focus on the `Enum` module first and only move to `Stream` for the particular scenarios where laziness is required, to either deal with slow resources or large, possibly infinite, collections.

Next, we will look at an alternate approach of operating on collections: Comprehensions.

Comprehensions

In Elixir, it is common to loop over an `Enumerable`, often filtering out some results and mapping values into another list. Comprehensions are syntactic sugar for such constructs: they group those common tasks into the `for` special form.

For example, we can map a list of integers into their squared values:

```
iex> for n <- [1, 2, 3, 4], do: n * n
[1, 4, 9, 16]
```

A comprehension is made of three parts: generators, filters, and collectables.

Generators and filters

In the expression above, `n <- [1, 2, 3, 4]` is the **generator**. It is literally generating values to be used in the comprehension. Any enumerable can be passed on the right-hand side of the generator expression:

```
iex> for n <- 1..4, do: n * n
[1, 4, 9, 16]
```

Generator expressions also support pattern matching on their left-hand side; all non-matching patterns are *ignored*. Imagine that, instead of a range, we have a keyword list where the key is the atom `:good` or `:bad` and we only want to compute the square of the `:good` values:

```
iex> values = [good: 1, good: 2, bad: 3, good: 4]
iex> for {:good, n} <- values, do: n * n
[1, 4, 16]
```

Alternatively to pattern matching, filters can be used to select some particular elements. For example, we can select the multiples of 3 and discard all others:

```
iex> for n <- 0..5, rem(n, 3) == 0, do: n * n
[0, 9]
```

Comprehensions discard all elements for which the filter expression returns `false` or `nil`; all other values are selected.

Comprehensions generally provide a much more concise representation than using the equivalent functions from the `Enum` and `Stream` modules. Furthermore, comprehensions also allow multiple generators and filters to be given. Here is an example that receives a list of directories and gets the size of each file in those directories:

```
dirs = ["/home/mikey", "/home/james"]

for dir <- dirs,
    file <- File.ls!(dir),
    path = Path.join(dir, file),
    File.regular?(path) do
  File.stat!(path).size
end
```

Multiple generators can also be used to calculate the Cartesian product of two lists:

```
iex> for i <- [:a, :b, :c], j <- [1, 2], do:  {i, j}
[a: 1, a: 2, b: 1, b: 2, c: 1, c: 2]
```

Finally, keep in mind that variable assignments inside the comprehension, be it in generators, filters or inside the block, are not reflected outside of the comprehension.

Bitstring generators

Bitstring generators are also supported and are very useful when you need to comprehend over bitstring streams. The example below receives a list of pixels from a binary with their respective red, green and blue values and converts them into tuples of three elements each:

```
iex> pixels = <<213, 45, 132, 64, 76, 32, 76, 0, 0, 234, 32,
15>>
iex> for <<r::8, g::8, b::8 <- pixels>>, do: {r, g, b}
[{213, 45, 132}, {64, 76, 32}, {76, 0, 0}, {234, 32, 15}]
```

A bitstring generator can be mixed with "regular" enumerable generators, and supports filters as well.

The `:into` option

In the examples above, all the comprehensions returned lists as their result. However, the result of a comprehension can be inserted into different data structures by passing the `:into` option to the comprehension.

For example, a bitstring generator can be used with the `:into` option in order to easily remove all spaces in a string:

```
iex> for <<c <- " hello world ">>, c != ?\s, into: "", do:
<<c>>
"helloworld"
```

Sets, maps, and other dictionaries can also be given to the `:into` option. In general, `:into` accepts any structure that implements the `Collectable` protocol.

A common use case of `:into` can be transforming values in a map:

```
iex> for {key, val} <- %{"a" => 1, "b" => 2}, into: %{}, do:
{key, val * val}
%{"a" => 1, "b" => 4}
```

Let's make another example using streams. Since the `IO` module provides streams (that are both `Enumerable`s and `Collectable`s), an echo terminal that echoes back the upcased version of whatever is typed can be implemented using comprehensions:

```
iex> stream = IO.stream(:stdio, :line)
iex> for line <- stream, into: stream do
...>   String.upcase(line) <> "\n"
...> end
```

Now type any string into the terminal and you will see that the same value will be printed in upper-case. Unfortunately, this example also got your IEx shell stuck in the comprehension, so you will need to hit `Ctrl+C` twice to get out of it. :)

Other options

Comprehensions support other options, such as `:reduce` and `:uniq`. Here are additional resources to learn more about comprehensions:

- `for` official reference in Elixir documentation
- Mitchell Hanberg's comprehensive guide to Elixir's comprehensions

Protocols

Protocols are a mechanism to achieve polymorphism in Elixir where you want the behavior to vary depending on the data type. We are already familiar with one way of solving this type of problem: via pattern matching and guard clauses. Consider a simple utility module that would tell us the type of input variable:

```
defmodule Utility do
  def type(value) when is_binary(value), do: "string"
  def type(value) when is_integer(value), do: "integer"
  # ... other implementations ...
end
```

If the use of this module were confined to your own project, you would be able to keep defining new `type/1` functions for each new data type. However, this code could be problematic if it was shared as a dependency by multiple apps because there would be no easy way to extend its functionality.

This is where protocols can help us: protocols allow us to extend the original behavior for as many data types as we need. That's because **dispatching on a protocol is available to any data type that has implemented the protocol** and a protocol can be implemented by anyone, at any time.

Here's how we could write the same `Utility.type/1` functionality as a protocol:

```
defprotocol Utility do
  @spec type(t) :: String.t()
  def type(value)
```

```
end

defimpl Utility, for: BitString do
  def type(_value), do: "string"
end

defimpl Utility, for: Integer do
  def type(_value), do: "integer"
end
```

We define the protocol using `defprotocol/2` - its functions and specs may look similar to interfaces or abstract base classes in other languages. We can add as many implementations as we like using `defimpl/2`. The output is exactly the same as if we had a single module with multiple functions:

```
iex> Utility.type("foo")
"string"
iex> Utility.type(123)
"integer"
```

With protocols, however, we are no longer stuck having to continuously modify the same module to support more and more data types. For example, we could spread the `defimpl` calls above over multiple files and Elixir will dispatch the execution to the appropriate implementation based on the data type. Functions defined in a protocol may have more than one input, but the **dispatching will always be based on the data type of the first input**.

One of the most common protocols you may encounter is the `String.Chars` protocol: implementing its `to_string/1` function for your custom structs will tell the Elixir kernel how to represent them as strings. We will explore all the built-in protocols later. For now, let's implement our own.

Example

Now that you have seen an example of the type of problem protocols help solve and how they solve them, let's look at a more in-depth example.

In Elixir, we have two idioms for checking how many items there are in a data structure: `length` and `size`. `length` means the information must be computed. For example, `length(list)` needs to traverse the whole list to calculate its length. On the other hand, `tuple_size(tuple)` and `byte_size(binary)` do not depend on the tuple and binary size as the size information is pre-computed in the data structure.

Even if we have type-specific functions for getting the size built into Elixir (such as `tuple_size/1`), we could implement a generic `Size` protocol that all data structures for which size is pre-computed would implement.

The protocol definition would look like this:

```
defprotocol Size do
  @doc "Calculates the size (and not the length!) of a data
structure"
  def size(data)
end
```

The `Size` protocol expects a function called `size` that receives one argument (the data structure we want to know the size of) to be implemented. We can now implement this protocol for the data structures that would have a compliant implementation:

```
defimpl Size, for: BitString do
  def size(string), do: byte_size(string)
end
```

```elixir
defimpl Size, for: Map do
  def size(map), do: map_size(map)
end

defimpl Size, for: Tuple do
  def size(tuple), do: tuple_size(tuple)
end
```

We didn't implement the `Size` protocol for lists as there is no "size" information pre-computed for lists, and the length of a list has to be computed (with `length/1`).

Now with the protocol defined and implementations in hand, we can start using it:

```elixir
iex> Size.size("foo")
3
iex> Size.size({:ok, "hello"})
2
iex> Size.size(%{label: "some label"})
1
```

Passing a data type that doesn't implement the protocol raises an error:

```elixir
iex> Size.size([1, 2, 3])
** (Protocol.UndefinedError) protocol Size not implemented
for [1, 2, 3] of type List
```

It's possible to implement protocols for all Elixir data types:

- `Atom`
- `BitString`

- Float
- Function
- Integer
- List
- Map
- PID
- Port
- Reference
- Tuple

Protocols and structs

The power of Elixir's extensibility comes when protocols and structs are used together.

In the previous chapter, we have learned that although structs are maps, they do not share protocol implementations with maps. For example, MapSets (sets based on maps) are implemented as structs. Let's try to use the Size protocol with a MapSet:

```
iex> Size.size(%{})
0
iex> set = %MapSet{} = MapSet.new
MapSet.new([])
iex> Size.size(set)
** (Protocol.UndefinedError) protocol Size not implemented
for MapSet.new([]) of type MapSet (a struct)
```

Instead of sharing protocol implementation with maps, structs require their own protocol implementation. Since a MapSet has its size precomputed and accessible through MapSet.size/1, we can define a Size implementation for it:

```
defimpl Size, for: MapSet do
  def size(set), do: MapSet.size(set)
end
```

If desired, you could come up with your own semantics for the size of your struct. Not only that, you could use structs to build more robust data types, like queues, and implement all relevant protocols, such as `Enumerable` and possibly `Size`, for this data type.

```
defmodule User do
  defstruct [:name, :age]
end

defimpl Size, for: User do
  def size(_user), do: 2
end
```

Implementing `Any`

Manually implementing protocols for all types can quickly become repetitive and tedious. In such cases, Elixir provides two options: we can explicitly derive the protocol implementation for our types or automatically implement the protocol for all types. In both cases, we need to implement the protocol for `Any`.

Deriving

Elixir allows us to derive a protocol implementation based on the `Any` implementation. Let's first implement `Any` as follows:

```
defimpl Size, for: Any do
  def size(_), do: 0
```

```
end
```

The implementation above is arguably not a reasonable one. For example, it makes no sense to say that the size of a `PID` or an `Integer` is `0`.

However, should we be fine with the implementation for `Any`, in order to use such implementation we would need to tell our struct to explicitly derive the `Size` protocol:

```
defmodule OtherUser do
  @derive [Size]
  defstruct [:name, :age]
end
```

When deriving, Elixir will implement the `Size` protocol for `OtherUser` based on the implementation provided for `Any`.

Fallback to `Any`

Another alternative to `@derive` is to explicitly tell the protocol to fallback to `Any` when an implementation cannot be found. This can be achieved by setting `@fallback_to_any` to `true` in the protocol definition:

```
defprotocol Size do
  @fallback_to_any true
  def size(data)
end
```

As we said in the previous section, the implementation of `Size` for `Any` is not one that can apply to any data type. That's one of the reasons why `@fallback_to_any` is an opt-in behavior. For the majority of protocols,

raising an error when a protocol is not implemented is the proper behavior. That said, assuming we have implemented `Any` as in the previous section:

```
defimpl Size, for: Any do
  def size(_), do: 0
end
```

Now all data types (including structs) that have not implemented the `Size` protocol will be considered to have a size of `0`.

Which technique is best between deriving and falling back to `Any` depends on the use case but, given Elixir developers prefer explicit over implicit, you may see many libraries pushing towards the `@derive` approach.

Built-in protocols

Elixir ships with some built-in protocols. In previous chapters, we have discussed the `Enum` module which provides many functions that work with any data structure that implements the `Enumerable` protocol:

```
iex> Enum.map([1, 2, 3], fn x -> x * 2 end)
[2, 4, 6]
iex> Enum.reduce(1..3, 0, fn x, acc -> x + acc end)
6
```

Another useful example is the `String.Chars` protocol, which specifies how to convert a data structure to its human representation as a string. It's exposed via the `to_string` function:

```
iex> to_string(:hello)
"hello"
```

Notice that string interpolation in Elixir calls the `to_string` function:

```
iex> "age: #{25}"
"age: 25"
```

The snippet above only works because numbers implement the `String.Chars` protocol. Passing a tuple, for example, will lead to an error:

```
iex> tuple = {1, 2, 3}
{1, 2, 3}
iex> "tuple: #{tuple}"
** (Protocol.UndefinedError) protocol String.Chars not
implemented for {1, 2, 3} of type Tuple
```

When there is a need to "print" a more complex data structure, one can use the `inspect` function, based on the `Inspect` protocol:

```
iex> "tuple: #{inspect(tuple)}"
"tuple: {1, 2, 3}"
```

The `Inspect` protocol is the protocol used to transform any data structure into a readable textual representation. This is what tools like IEx use to print results:

```
iex> {1, 2, 3}
{1, 2, 3}
iex> %User{}
%User{name: "john", age: 27}
```

Keep in mind that, by convention, whenever the inspected value starts with `#`, it is representing a data structure in non-valid Elixir syntax. This means

the inspect protocol is not reversible as information may be lost along the way:

```
iex> inspect &(&1+2)
"#Function<6.71889879/1 in :erl_eval.expr/5>"
```

There are other protocols in Elixir, but this covers the most common ones. You can learn more about protocols and implementations in the `Protocol` module.

Sigils

Elixir provides double-quoted strings as well as a concept called charlists, which are defined using the `~c"hello world"` sigil syntax. In this chapter, we will learn more about sigils and how to define our own.

One of Elixir's goals is extensibility: developers should be able to extend the language to fit any particular domain. Sigils provide the foundation for extending the language with custom textual representations. Sigils start with the tilde (~) character which is followed by either a single lower-case letter or one or more upper-case letters, and then a delimiter. Optional modifiers are added after the final delimiter.

Regular expressions

The most common sigil in Elixir is `~r` , which is used to create [regular expressions]:

```
# A regular expression that matches strings which contain
"foo" or "bar":
iex> regex = ~r/foo|bar/
~r/foo|bar/
iex> "foo" =~ regex
true
iex> "bat" =~ regex
false
```

Elixir provides Perl-compatible regular expressions (regexes), as implemented by the [PCRE] library. Regexes also support modifiers. For example, the `i` modifier makes a regular expression case insensitive:

```
iex> "HELLO" =~ ~r/hello/
false
iex> "HELLO" =~ ~r/hello/i
true
```

Check out the `Regex` module for more information on other modifiers and the supported operations with regular expressions.

So far, all examples have used `/` to delimit a regular expression. However, sigils support 8 different delimiters:

```
~r/hello/
~r|hello|
~r"hello"
~r'hello'
~r(hello)
~r[hello]
~r{hello}
~r<hello>
```

The reason behind supporting different delimiters is to provide a way to write literals without escaped delimiters. For example, a regular expression with forward slashes like `~r(^https?://)` reads arguably better than `~r/^https?:\/\//`. Similarly, if the regular expression has forward slashes and capturing groups (that use `()`), you may then choose double quotes instead of parentheses.

Strings, charlists, and word lists sigils

Elixir ships with three sigils for building textual data structures. These allow you to choose an appropriate delimiter for your literal text such that you do not have to worry about escaping.

Strings

The `~s` sigil is used to generate strings, like double quotes are. The `~s` sigil is useful when a string contains double quotes:

```
iex> ~s(this is a string with "double" quotes, not 'single'
ones)
"this is a string with \"double\" quotes, not 'single' ones"
```

Charlists

The `~c` sigil is the regular way to represent charlists.

```
iex> [?c, ?a, ?t]
~c"cat"
iex> ~c(this is a char list containing "double quotes")
~c"this is a char list containing \"double quotes\""
```

Word lists

The `~w` sigil is used to generate lists of words (*words* are just regular strings). Inside the `~w` sigil, words are separated by whitespace.

```
iex> ~w(foo bar bat)
["foo", "bar", "bat"]
```

The `~w` sigil also accepts the `c`, `s` and `a` modifiers (for charlists, strings, and atoms, respectively), which specify the data type of the elements of the resulting list:

```
iex> ~w(foo bar bat)a
[:foo, :bar, :bat]
```

Interpolation and escaping in textual sigils

Sigils also help deal with escaping characters and interpolation. In particular, uppercase-letter textual sigils do not perform interpolation nor escaping, and most lowercase sigils have an uppercase variant. For example, although both `~s` and `~S` will return strings, the former allows escape codes and interpolation while the latter does not:

```
iex> ~s(String with escape codes \x26 #{"inter" <>
"polation"})
"String with escape codes & interpolation"
iex> ~S(String without escape codes \x26 without #
{interpolation})
"String without escape codes \\x26 without \#
{interpolation}"
```

The following escape codes can be used in textual sigils:

- `\\` – single backslash
- `\a` – bell/alert
- `\b` – backspace
- `\d` - delete
- `\e` - escape
- `\f` - form feed
- `\n` – newline
- `\r` – carriage return
- `\s` – space
- `\t` – tab

- `\v` – vertical tab
- `\0` - null byte
- `\xDD` - represents a single byte in hexadecimal (such as `\x13`)
- `\uDDDD` and `\u{D...}` - represents a Unicode codepoint in hexadecimal (such as `\u{1F600}`)

Sigils also support heredocs, that is, three double-quotes or single-quotes as separators:

```
iex> ~s"""
...> this is
...> a heredoc string
...> """
```

The most common use case for heredoc sigils is when writing documentation. For example, writing escape characters in the documentation would soon become error prone because of the need to double-escape some characters:

```
@doc """
Converts double-quotes to single-quotes.

## Examples

    iex> convert("\\\"foo\\\"")
    "'foo'"

"""
def convert(...)
```

By using `~S`, this problem can be avoided altogether:

```
@doc ~S"""
Converts double-quotes to single-quotes.

## Examples

    iex> convert("\"foo\"")
    "'foo'"

"""
def convert(...)
```

Calendar sigils

Elixir offers sigils to deal with various flavors of times and dates.

Date

A %Date{} struct contains the fields `year`, `month`, `day`, and `calendar`. You can create one using the `~D` sigil:

```
iex> d = ~D[2019-10-31]
~D[2019-10-31]
iex> d.day
31
```

Time

The %Time{} struct contains the fields `hour`, `minute`, `second`, `microsecond`, and `calendar`. You can create one using the `~T` sigil:

```
iex> t = ~T[23:00:07.0]
~T[23:00:07.0]
```

```
iex> t.second
7
```

NaiveDateTime

The %NaiveDateTime{} struct contains fields from both `Date` and `Time`. You can create one using the `~N` sigil:

```
iex> ndt = ~N[2019-10-31 23:00:07]
~N[2019-10-31 23:00:07]
```

Why is it called naive? Because it does not contain timezone information. Therefore, the given datetime may not exist at all or it may exist twice in certain timezones - for example, when we move the clock back and forward for daylight saving time.

UTC DateTime

A %DateTime{} struct contains the same fields as a `NaiveDateTime` with the addition of fields to track timezones. The `~U` sigil allows developers to create a DateTime in the UTC timezone:

```
iex> dt = ~U[2019-10-31 19:59:03Z]
~U[2019-10-31 19:59:03Z]
iex> %DateTime{minute: minute, time_zone: time_zone} = dt
~U[2019-10-31 19:59:03Z]
iex> minute
59
iex> time_zone
"Etc/UTC"
```

Custom sigils

As hinted at the beginning of this chapter, sigils in Elixir are extensible. In fact, using the sigil `~r/foo/i` is equivalent to calling `sigil_r` with a binary and a char list as the argument:

```
iex> sigil_r(<<"foo">>, [?i])
~r"foo"i
```

We can access the documentation for the `~r` sigil via `sigil_r`:

```
iex> h sigil_r
...
```

We can also provide our own sigils by implementing functions that follow the `sigil_{character}` pattern. For example, let's implement the `~i` sigil that returns an integer (with the optional `n` modifier to make it negative):

```
iex> defmodule MySigils do
...>   def sigil_i(string, []), do:
String.to_integer(string)
...>   def sigil_i(string, [?n]), do: -
String.to_integer(string)
...> end
iex> import MySigils
iex> ~i(13)
13
iex> ~i(42)n
-42
```

Custom sigils may be either a single lowercase character, or an uppercase character followed by more uppercase characters and digits. In practice,

they are often used to embed templating languages or even represent regular languages within Elixir itself. If you're interested in learning more, check out how sigils are implemented in the `Kernel` module (where the `sigil_*` functions/macros are defined) for a starting point.

try, catch, and rescue

Elixir has three error mechanisms: errors, throws, and exits. In this chapter, we will explore each of them and include remarks about when each should be used.

Errors

Errors (or *exceptions*) are used when exceptional things happen in the code. A sample error can be retrieved by trying to add a number to an atom:

```
iex> :foo + 1
** (ArithmeticError) bad argument in arithmetic expression
     :erlang.+(:foo, 1)
```

A runtime error can be raised any time by using `raise/1`:

```
iex> raise "oops"
** (RuntimeError) oops
```

Other errors can be raised with `raise/2` passing the error name and a list of keyword arguments:

```
iex> raise ArgumentError, message: "invalid argument foo"
** (ArgumentError) invalid argument foo
```

You can also define your own errors by creating a module and using the `defexception/1` construct inside it. This way, you'll create an error with the same name as the module it's defined in. The most common case is to define a custom exception with a message field:

```
iex> defmodule MyError do
iex>   defexception message: "default message"
iex> end
iex> raise MyError
** (MyError) default message
iex> raise MyError, message: "custom message"
** (MyError) custom message
```

Errors can be **rescued** using the `try/rescue` construct:

```
iex> try do
...>   raise "oops"
...> rescue
...>   e in RuntimeError -> e
...> end
%RuntimeError{message: "oops"}
```

The example above rescues the runtime error and returns the exception itself, which is then printed in the `iex` session.

If you don't have any use for the exception, you don't have to pass a variable to `rescue`:

```
iex> try do
...>   raise "oops"
...> rescue
...>   RuntimeError -> "Error!"
...> end
"Error!"
```

In practice, Elixir developers rarely use the `try/rescue` construct. For example, many languages would force you to rescue an error when a file

cannot be opened successfully. Elixir instead provides a `File.read/1` function which returns a tuple containing information about whether the file was opened successfully:

```
iex> File.read("hello")
{:error, :enoent}
iex> File.write("hello", "world")
:ok
iex> File.read("hello")
{:ok, "world"}
```

There is no `try/rescue` here. In case you want to handle multiple outcomes of opening a file, you can use pattern matching using the `case` construct:

```
iex> case File.read("hello") do
...>   {:ok, body} -> IO.puts("Success: #{body}")
...>   {:error, reason} -> IO.puts("Error: #{reason}")
...> end
```

For the cases where you do expect a file to exist (and the lack of that file is truly an *error*) you may use `File.read!/1`:

```
iex> File.read!("unknown")
** (File.Error) could not read file "unknown": no such file
or directory
    (elixir) lib/file.ex:272: File.read!/1
```

At the end of the day, it's up to your application to decide if an error while opening a file is exceptional or not. That's why Elixir doesn't impose exceptions on `File.read/1` and many other functions. Instead, it leaves it up to the developer to choose the best way to proceed.

Many functions in the standard library follow the pattern of having a counterpart that raises an exception instead of returning tuples to match against. The convention is to create a function (`foo`) which returns `{:ok, result}` or `{:error, reason}` tuples and another function (`foo!` , same name but with a trailing `!`) that takes the same arguments as `foo` but which raises an exception if there's an error. `foo!` should return the result (not wrapped in a tuple) if everything goes fine. The `File` module is a good example of this convention.

Fail fast / Let it crash

One saying that is common in the Erlang community, as well as Elixir's, is "fail fast" / "let it crash". The idea behind let it crash is that, in case something *unexpected* happens, it is best to let the exception happen, without rescuing it.

It is important to emphasize the word *unexpected*. For example, imagine you are building a script to process files. Your script receives filenames as inputs. It is expected that users may make mistakes and provide unknown filenames. In this scenario, while you could use `File.read!/1` to read files and let it crash in case of invalid filenames, it probably makes more sense to use `File.read/1` and provide users of your script with a clear and precise feedback of what went wrong.

Other times, you may fully expect a certain file to exist, and in case it does not, it means something terribly wrong has happened elsewhere. In such cases, `File.read!/1` is all you need.

The second approach also works because, as discussed in the Processes chapter, all Elixir code runs inside processes that are isolated and don't share anything by default. Therefore, an unhandled exception in a process will never crash or corrupt the state of another process. This allows us to define supervisor processes, which are meant to observe when a process terminates unexpectedly, and start a new one in its place.

At the end of the day, "fail fast" / "let it crash" is a way of saying that, when *something unexpected* happens, it is best to start from scratch within a new process, freshly started by a supervisor, rather than blindly trying to rescue all possible error cases without the full context of when and how they can happen.

Reraise

While we generally avoid using `try/rescue` in Elixir, one situation where we may want to use such constructs is for observability/monitoring. Imagine you want to log that something went wrong, you could do:

```
try do
  ... some code ...
rescue
  e ->
    Logger.error(Exception.format(:error, e,
__STACKTRACE__))
    reraise e, __STACKTRACE__
end
```

In the example above, we rescued the exception, logged it, and then re-raised it. We use the `__STACKTRACE__` construct both when formatting the exception and when re-raising. This ensures we reraise the exception as is, without changing value or its origin.

Generally speaking, we take errors in Elixir literally: they are reserved for unexpected and/or exceptional situations, never for controlling the flow of our code. In case you actually need flow control constructs, *throws* should be used. That's what we are going to see next.

Throws

In Elixir, a value can be thrown and later be caught. `throw` and `catch` are reserved for situations where it is not possible to retrieve a value unless by using `throw` and `catch`.

Those situations are quite uncommon in practice except when interfacing with libraries that do not provide a proper API. For example, let's imagine the `Enum` module did not provide any API for finding a value and that we needed to find the first multiple of 13 in a list of numbers:

```
iex> try do
...>   Enum.each(-50..50, fn x ->
...>     if rem(x, 13) == 0, do: throw(x)
...>   end)
...>   "Got nothing"
...> catch
...>   x -> "Got #{x}"
...> end
"Got -39"
```

Since `Enum` *does* provide a proper API, in practice `Enum.find/2` is the way to go:

```
iex> Enum.find(-50..50, &(rem(&1, 13) == 0))
-39
```

Exits

All Elixir code runs inside processes that communicate with each other. When a process dies of "natural causes" (e.g., unhandled exceptions), it sends an `exit` signal. A process can also die by explicitly sending an `exit` signal:

```
iex> spawn_link(fn -> exit(1) end)
** (EXIT from #PID<0.56.0>) shell process exited with
reason: 1
```

In the example above, the linked process died by sending an `exit` signal with a value of 1. The Elixir shell automatically handles those messages and prints them to the terminal.

`exit` can also be "caught" using `try/catch`:

```
iex> try do
...>   exit("I am exiting")
...> catch
...>   :exit, _ -> "not really"
...> end
"not really"
```

`catch` can also be used within a function body without a matching `try`.

```
defmodule Example do
  def matched_catch do
    exit(:timeout)
  catch
    :exit, :timeout ->
      {:error, :timeout}
  end

  def mismatched_catch do
    exit(:timeout)
  catch
    # Since no clause matches, this catch will have no
effect
    :exit, :explosion ->
```

```
      {:error, :explosion}
  end
end
```

However, using `try/catch` is already uncommon and using it to catch exits is even rarer.

`exit` signals are an important part of the fault tolerant system provided by the Erlang VM. Processes usually run under supervision trees which are themselves processes that listen to `exit` signals from the supervised processes. Once an `exit` signal is received, the supervision strategy kicks in and the supervised process is restarted.

It is exactly this supervision system that makes constructs like `try/catch` and `try/rescue` so uncommon in Elixir. Instead of rescuing an error, we'd rather "fail fast" since the supervision tree will guarantee our application will go back to a known initial state after the error.

After

Sometimes it's necessary to ensure that a resource is cleaned up after some action that could potentially raise an error. The `try/after` construct allows you to do that. For example, we can open a file and use an `after` clause to close it—even if something goes wrong:

```
iex> {:ok, file} = File.open("sample", [:utf8, :write])
iex> try do
...>   IO.write(file, "olá")
...>   raise "oops, something went wrong"
...> after
...>   File.close(file)
```

```
...> end
** (RuntimeError) oops, something went wrong
```

The `after` clause will be executed regardless of whether or not the tried block succeeds. Note, however, that if a linked process exits, this process will exit and the `after` clause will not get run. Thus `after` provides only a soft guarantee. Luckily, files in Elixir are also linked to the current processes and therefore they will always get closed if the current process crashes, independent of the `after` clause. You will find the same to be true for other resources like ETS tables, sockets, ports and more.

Sometimes you may want to wrap the entire body of a function in a `try` construct, often to guarantee some code will be executed afterwards. In such cases, Elixir allows you to omit the `try` line:

```
iex> defmodule RunAfter do
...>   def without_even_trying do
...>     raise "oops"
...>   after
...>     IO.puts("cleaning up!")
...>   end
...> end
iex> RunAfter.without_even_trying
cleaning up!
** (RuntimeError) oops
```

Elixir will automatically wrap the function body in a `try` whenever one of `after`, `rescue` or `catch` is specified. The `after` block handles side effects and does not change the return value from the clauses above it.

Else

If an `else` block is present, it will match on the results of the `try` block whenever the `try` block finishes without a throw or an error.

```
iex> x = 2
2
iex> try do
...>   1 / x
...> rescue
...>   ArithmeticError ->
...>     :infinity
...> else
...>   y when y < 1 and y > -1 ->
...>     :small
...>   _ ->
...>     :large
...> end
:small
```

Exceptions in the `else` block are not caught. If no pattern inside the `else` block matches, an exception will be raised; this exception is not caught by the current `try/catch/rescue/after` block.

Variables scope

Similar to `case`, `cond`, `if` and other constructs in Elixir, variables defined inside `try/catch/rescue/after` blocks do not leak to the outer context. In other words, this code is invalid:

```
iex> try do
...>   raise "fail"
...>   what_happened = :did_not_raise
...> rescue
...>   _ -> what_happened = :rescued
```

```
...> end
iex> what_happened
** (CompileError) undefined variable "what_happened"
```

Instead, you should return the value of the `try` expression:

```
iex> what_happened =
...>   try do
...>     raise "fail"
...>     :did_not_raise
...>   rescue
...>     _ -> :rescued
...>   end
iex> what_happened
:rescued
```

Furthermore, variables defined in the do-block of `try` are not available inside `rescue/after/else` either. This is because the `try` block may fail at any moment and therefore the variables may have never been bound in the first place. So this also isn't valid:

```
iex> try do
...>   raise "fail"
...>   another_what_happened = :did_not_raise
...> rescue
...>   _ -> another_what_happened
...> end
** (CompileError) undefined variable "another_what_happened"
```

This finishes our introduction on `try`, `catch`, and `rescue`. You will find they are used less frequently in Elixir than in other languages.

Next, we'll look at a feature central to Elixir, Processes, which allows us to write concurrent, parallel and distributed programs in an easy and understandable way.

Processes

In Elixir, all code runs inside processes. Processes are isolated from each other, run concurrent to one another and communicate via message passing. Processes are not only the basis for concurrency in Elixir, but they also provide the means for building distributed and fault-tolerant programs.

Elixir's processes should not be confused with operating system processes. Processes in Elixir are extremely lightweight in terms of memory and CPU (even compared to threads as used in many other programming languages). Because of this, it is not uncommon to have tens or even hundreds of thousands of processes running simultaneously.

In this chapter, we will learn about the basic constructs for spawning new processes, as well as sending and receiving messages between processes.

Spawning processes

The basic mechanism for spawning new processes is the auto-imported `spawn/1` function:

```
iex> spawn(fn -> 1 + 2 end)
#PID<0.43.0>
```

`spawn/1` takes a function which it will execute in another process.

Notice `spawn/1` returns a PID (process identifier). At this point, the process you spawned is very likely dead. The spawned process will execute the given function and exit after the function is done:

```
iex> pid = spawn(fn -> 1 + 2 end)
#PID<0.44.0>
```

```
iex> Process.alive?(pid)
false
```

> Note: you will likely get different process identifiers than the ones we are showing in our snippets.

We can retrieve the PID of the current process by calling `self/0`:

```
iex> self()
#PID<0.41.0>
iex> Process.alive?(self())
true
```

Processes get much more interesting when we are able to send and receive messages.

Sending and receiving messages

We can send messages to a process with `send/2` and receive them with `receive/1`:

```
iex> send(self(), {:hello, "world"})
{:hello, "world"}
iex> receive do
...>   {:hello, msg} -> msg
...>   {:world, _msg} -> "won't match"
...> end
"world"
```

When a message is sent to a process, the message is stored in the process mailbox. The `receive/1` block goes through the current process mailbox

searching for a message that matches any of the given patterns. `receive/1` supports guards and many clauses, exactly as `case/2`.

The process that sends the message does not block on `send/2`, it puts the message in the recipient's mailbox and continues. In particular, a process can send messages to itself.

If there is no message in the mailbox matching any of the patterns, the current process will wait until a matching message arrives. A timeout can also be specified:

```
iex> receive do
...>   {:hello, msg}  -> msg
...> after
...>   1_000 -> "nothing after 1s"
...> end
"nothing after 1s"
```

A timeout of 0 can be given when you already expect the message to be in the mailbox.

Let's put it all together and send messages between processes:

```
iex> parent = self()
#PID<0.41.0>
iex> spawn(fn -> send(parent, {:hello, self()}) end)
#PID<0.48.0>
iex> receive do
...>   {:hello, pid} -> "Got hello from #{inspect pid}"
...> end
"Got hello from #PID<0.48.0>"
```

The `inspect/1` function is used to convert a data structure's internal representation into a string, typically for printing. Notice that when the

`receive` block gets executed the sender process we have spawned may already be dead, as its only instruction was to send a message.

While in the shell, you may find the helper `flush/0` quite useful. It flushes and prints all the messages in the mailbox.

```
iex> send(self(), :hello)
:hello
iex> flush()
:hello
:ok
```

Links

The majority of times we spawn processes in Elixir, we spawn them as linked processes. Before we show an example with `spawn_link/1`, let's see what happens when a process started with `spawn/1` fails:

```
iex> spawn(fn -> raise "oops" end)
#PID<0.58.0>

[error] Process #PID<0.58.00> raised an exception
** (RuntimeError) oops
    (stdlib) erl_eval.erl:668: :erl_eval.do_apply/6
```

It merely logged an error but the parent process is still running. That's because processes are isolated. If we want the failure in one process to propagate to another one, we should link them. This can be done with `spawn_link/1`:

```
iex> self()
#PID<0.41.0>
```

```
iex> spawn_link(fn -> raise "oops" end)

** (EXIT from #PID<0.41.0>) evaluator process exited with
reason: an exception was raised:
    ** (RuntimeError) oops
        (stdlib) erl_eval.erl:668: :erl_eval.do_apply/6

[error] Process #PID<0.289.0> raised an exception
** (RuntimeError) oops
    (stdlib) erl_eval.erl:668: :erl_eval.do_apply/6
```

Because processes are linked, we now see a message saying the parent process, which is the shell process, has received an EXIT signal from another process causing the shell to terminate. IEx detects this situation and starts a new shell session.

Linking can also be done manually by calling `Process.link/1`. We recommend that you take a look at the `Process` module for other functionality provided by processes.

Processes and links play an important role when building fault-tolerant systems. Elixir processes are isolated and don't share anything by default. Therefore, a failure in a process will never crash or corrupt the state of another process. Links, however, allow processes to establish a relationship in case of failure. We often link our processes to supervisors which will detect when a process dies and start a new process in its place.

While other languages would require us to catch/handle exceptions, in Elixir we are actually fine with letting processes fail because we expect supervisors to properly restart our systems. "Failing fast" (sometimes referred as "let it crash") is a common philosophy when writing Elixir software!

`spawn/1` and `spawn_link/1` are the basic primitives for creating processes in Elixir. Although we have used them exclusively so far, most of the time

we are going to use abstractions that build on top of them. Let's see the most common one, called tasks.

Tasks

Tasks build on top of the spawn functions to provide better error reports and introspection:

```
iex> Task.start(fn -> raise "oops" end)
{:ok, #PID<0.55.0>}

15:22:33.046 [error] Task #PID<0.55.0> started from
#PID<0.53.0> terminating
** (RuntimeError) oops
    (stdlib) erl_eval.erl:668: :erl_eval.do_apply/6
    (elixir) lib/task/supervised.ex:85:
Task.Supervised.do_apply/2
    (stdlib) proc_lib.erl:247: :proc_lib.init_p_do_apply/3
Function: #Function<20.99386804/0 in :erl_eval.expr/5>
    Args: []
```

Instead of `spawn/1` and `spawn_link/1`, we use `Task.start/1` and `Task.start_link/1` which return `{:ok, pid}` rather than just the PID. This is what enables tasks to be used in supervision trees. Furthermore, `Task` provides convenience functions, like `Task.async/1` and `Task.await/1`, and functionality to ease distribution.

We will explore tasks and other abstractions around processes in the "Mix and OTP guide".

State

We haven't talked about state so far. If you are building an application that requires state, for example, to keep your application configuration, or you need to parse a file and keep it in memory, where would you store it?

Processes are the most common answer to this question. We can write processes that loop infinitely, maintain state, and send and receive messages. As an example, let's write a module that starts new processes that work as a key-value store in a file named `kv.exs`:

```
defmodule KV do
  def start_link do
    Task.start_link(fn -> loop(%{}) end)
  end

  defp loop(map) do
    receive do
      {:get, key, caller} ->
        send(caller, Map.get(map, key))
        loop(map)
      {:put, key, value} ->
        loop(Map.put(map, key, value))
    end
  end
end
```

Note that the `start_link` function starts a new process that runs the `loop/1` function, starting with an empty map. The `loop/1` (private) function then waits for messages and performs the appropriate action for each message. We made `loop/1` private by using `defp` instead of `def`. In the case of a `:get` message, it sends a message back to the caller and calls `loop/1` again, to wait for a new message. While the `:put` message actually invokes `loop/1` with a new version of the map, with the given `key` and `value` stored.

Let's give it a try by running `iex kv.exs`:

```
iex> {:ok, pid} = KV.start_link()
{:ok, #PID<0.62.0>}
iex> send(pid, {:get, :hello, self()})
{:get, :hello, #PID<0.41.0>}
iex> flush()
nil
:ok
```

At first, the process map has no keys, so sending a `:get` message and then flushing the current process inbox returns `nil`. Let's send a `:put` message and try it again:

```
iex> send(pid, {:put, :hello, :world})
{:put, :hello, :world}
iex> send(pid, {:get, :hello, self()})
{:get, :hello, #PID<0.41.0>}
iex> flush()
:world
:ok
```

Notice how the process is keeping a state and we can get and update this state by sending the process messages. In fact, any process that knows the `pid` above will be able to send it messages and manipulate the state.

It is also possible to register the `pid`, giving it a name, and allowing everyone that knows the name to send it messages:

```
iex> Process.register(pid, :kv)
true
iex> send(:kv, {:get, :hello, self()})
{:get, :hello, #PID<0.41.0>}
```

```
iex> flush()
:world
:ok
```

Using processes to maintain state and name registration are very common patterns in Elixir applications. However, most of the time, we won't implement those patterns manually as above, but by using one of the many abstractions that ship with Elixir. For example, Elixir provides `Agent`s, which are simple abstractions around state. Our code above could be directly written as:

```
iex> {:ok, pid} = Agent.start_link(fn -> %{} end)
{:ok, #PID<0.72.0>}
iex> Agent.update(pid, fn map -> Map.put(map, :hello,
:world) end)
:ok
iex> Agent.get(pid, fn map -> Map.get(map, :hello) end)
:world
```

A `:name` option could also be given to `Agent.start_link/2` and it would be automatically registered. Besides agents, Elixir provides an API for building generic servers (called `GenServer`), registries, and more, all powered by processes underneath. Those, along with supervision trees, will be explored with more detail in the "Mix and OTP guide", which will build a complete Elixir application from start to finish.

For now, let's move on and explore the world of I/O in Elixir.

IO and the file system

This chapter introduces the input/output mechanisms, file-system-related tasks, and related modules such as `IO`, `File`, and `Path`. The IO system provides a great opportunity to shed some light on some philosophies and curiosities of Elixir and the Erlang VM.

The `IO` module

The `IO` module is the main mechanism in Elixir for reading and writing to standard input/output (`:stdio`), standard error (`:stderr`), files, and other IO devices. Usage of the module is pretty straightforward:

```
iex> IO.puts("hello world")
hello world
:ok
iex> IO.gets("yes or no? ")
yes or no? yes
"yes\n"
```

By default, functions in the `IO` module read from the standard input and write to the standard output. We can change that by passing, for example, `:stderr` as an argument (in order to write to the standard error device):

```
iex> IO.puts(:stderr, "hello world")
hello world
:ok
```

The `File` module

The `File` module contains functions that allow us to open files as IO devices. By default, files are opened in binary mode, which requires developers to use the specific `IO.binread/2` and `IO.binwrite/2` functions from the `IO` module:

> **Potential data loss warning**
>
> The following code opens a file for writing. If an existing file is available at the given path, its contents will be deleted.

```
iex> {:ok, file} = File.open("path/to/file/hello", [:write])
{:ok, #PID<0.47.0>}
iex> IO.binwrite(file, "world")
:ok
iex> File.close(file)
:ok
iex> File.read("path/to/file/hello")
{:ok, "world"}
```

The file could be opened with the `:append` option, instead of `:write`, to preserve its contents. You may also pass the `:utf8` option, which tells the `File` module to interpret the bytes read from the file as UTF-8-encoded bytes.

Besides functions for opening, reading and writing files, the `File` module has many functions to work with the file system. Those functions are named after their UNIX equivalents. For example, `File.rm/1` can be used to remove files, `File.mkdir/1` to create directories, `File.mkdir_p/1` to create directories and all their parent chain. There are even `File.cp_r/2` and `File.rm_rf/1` to respectively copy and remove files and directories recursively (i.e., copying and removing the contents of the directories too).

You will also notice that functions in the `File` module have two variants: one "regular" variant and another variant with a trailing bang (`!`). For example, when we read the `"hello"` file in the example above, we use `File.read/1`. Alternatively, we can use `File.read!/1`:

```
iex> File.read("path/to/file/hello")
{:ok, "world"}
iex> File.read!("path/to/file/hello")
"world"
iex> File.read("path/to/file/unknown")
{:error, :enoent}
iex> File.read!("path/to/file/unknown")
** (File.Error) could not read file "path/to/file/unknown":
no such file or directory
```

Notice that the version with `!` returns the contents of the file instead of a tuple, and if anything goes wrong the function raises an error.

The version without `!` is preferred when you want to handle different outcomes using pattern matching:

```
case File.read("path/to/file/hello") do
  {:ok, body} -> # do something with the `body`
  {:error, reason} -> # handle the error caused by `reason`
end
```

However, if you expect the file to be there, the bang variation is more useful as it raises a meaningful error message. Avoid writing:

```
{:ok, body} = File.read("path/to/file/unknown")
```

as, in case of an error, `File.read/1` will return `{:error, reason}` and the pattern matching will fail. You will still get the desired result (a raised error), but the message will be about the pattern which doesn't match (thus being cryptic in respect to what the error actually is about).

Therefore, if you don't want to handle the error outcomes, prefer to use the functions ending with an exclamation mark, such as `File.read!/1`.

The `Path` module

The majority of the functions in the `File` module expect paths as arguments. Most commonly, those paths will be regular binaries. The `Path` module provides facilities for working with such paths:

```
iex> Path.join("foo", "bar")
"foo/bar"
iex> Path.expand("~/hello")
"/Users/jose/hello"
```

Using functions from the `Path` module as opposed to directly manipulating strings is preferred since the `Path` module takes care of different operating systems transparently. Finally, keep in mind that Elixir will automatically convert slashes (`/`) into backslashes (`\`) on Windows when performing file operations.

With this, we have covered the main modules that Elixir provides for dealing with IO and interacting with the file system. In the next section, we will peek a bit under the covers and learn how the IO system is implemented in the VM.

Processes

You may have noticed that `File.open/2` returns a tuple like `{:ok, pid}`:

```
iex> {:ok, file} = File.open("hello")
{:ok, #PID<0.47.0>}
```

This happens because the `IO` module actually works with processes (see the previous chapter). Given a file is a process, when you write to a file that has been closed, you are actually sending a message to a process which has been terminated:

```
iex> File.close(file)
:ok
iex> IO.write(file, "is anybody out there")
** (ErlangError) Erlang error: :terminated:

  * 1st argument: the device has terminated

    (stdlib 5.0) io.erl:94: :io.put_chars(#PID<0.114.0>, "is
anybody out there")
    iex:4: (file)
```

Let's see in more detail what happens when you request `IO.write(pid, binary)`. The `IO` module sends a message to the process identified by `pid` with the desired operation. A small ad-hoc process can help us see it:

```
iex> pid = spawn(fn ->
...>   receive do
...>     msg -> IO.inspect(msg)
...>   end
...> end)
#PID<0.57.0>
iex> IO.write(pid, "hello")
{:io_request, #PID<0.41.0>, #Reference<0.0.8.91>,
```

```
 {:put_chars, :unicode, "hello"}}
** (ErlangError) erlang error: :terminated
```

After `IO.write/2`, we can see the request sent by the `IO` module printed out (a four-elements tuple). Soon after that, we see that it fails since the `IO` module expected some kind of result, which we did not supply.

By modeling IO devices with processes, the Erlang VM allows us to even read and write to files across nodes. Neat!

`iodata` and `chardata`

In all of the examples above, we used binaries when writing to files. However, most of the IO functions in Elixir also accept either "iodata" or "chardata".

One of the main reasons for using "iodata" and "chardata" is for performance. For example, imagine you need to greet someone in your application:

```
name = "Mary"
IO.puts("Hello " <> name <> "!")
```

Given strings in Elixir are immutable, as most data structures, the example above will copy the string "Mary" into the new "Hello Mary!" string. While this is unlikely to matter for the short string as above, copying can be quite expensive for large strings! For this reason, the IO functions in Elixir allow you to pass instead a list of strings:

```
name = "Mary"
IO.puts(["Hello ", name, "!"])
```

In the example above, there is no copying. Instead we create a list that contains the original name. We call such lists either "iodata" or "chardata" and we will learn the precise difference between them soon.

Those lists are very useful because it can actually simplify the processing strings in several scenarios. For example, imagine you have a list of values, such as `["apple", "banana", "lemon"]` that you want to write to disk separated by commas. How can you achieve this?

One option is to use `Enum.join/2` and convert the values to a string:

```
iex> Enum.join(["apple", "banana", "lemon"], ",")
"apple,banana,lemon"
```

The above returns a new string by copying each value into the new string. However, with the knowledge in this section, we know that we can pass a list of strings to the IO/File functions. So instead we can do:

```
iex> Enum.intersperse(["apple", "banana", "lemon"], ",")
["apple", ",", "banana", ",", "lemon"]
```

"iodata" and "chardata" do not only contain strings, but they may contain arbitrary nested lists of strings too:

```
iex> IO.puts(["apple", [",", "banana", [",", "lemon"]]])
```

"iodata" and "chardata" may also contain integers. For example, we could print our comma separated list of values by using `?,` as separator, which is the integer representing a comma (`44`):

```
iex> IO.puts(["apple", ?,, "banana", ?,, "lemon"])
```

The difference between "iodata" and "chardata" is precisely what said integer represents. For iodata, the integers represent bytes. For chardata, the integers represent Unicode codepoints. For ASCII characters, the byte representation is the same as the codepoint representation, so it fits both classifications. However, the default IO device works with chardata, which means we can do:

```
iex> IO.puts([?O, ?l, ?á, ?\s, "Mary", ?!])
```

Charlists, such as `~c"hello world"`, are lists of integers, and therefore are chardata.

We packed a lot into this small section, so let's break it down:

- iodata and chardata are lists of binaries and integers. Those binaries and integers can be arbitrarily nested inside lists. Their goal is to give flexibility and performance when working with IO devices and files;
- the choice between iodata and chardata depends on the encoding of the IO device. If the file is opened without encoding, the file expects iodata, and the functions in the `IO` module starting with `bin*` must be used. The default IO device (`:stdio`) and files opened with `:utf8` encoding expect chardata and work with the remaining functions in the `IO` module;

This finishes our tour of IO devices and IO related functionality. We have learned about three Elixir modules - `IO`, `File`, and `Path` - as well as how the VM uses processes for the underlying IO mechanisms and how to use `chardata` and `iodata` for IO operations.

Next we will talk about a very important subject to Elixir developers: writing documentation.

Writing documentation

Elixir treats documentation as a first-class citizen. Documentation must be easy to write and easy to read. In this guide you will learn how to write documentation in Elixir, covering constructs like module attributes, style practices, and doctests.

Markdown

Elixir documentation is written using Markdown. There are plenty of guides on Markdown online, we recommend the one from GitHub as a getting started point:

- Basic writing and formatting syntax

Module Attributes

Documentation in Elixir is usually attached to module attributes. Let's see an example:

```
defmodule MyApp.Hello do
  @moduledoc """
  This is the Hello module.
  """
  @moduledoc since: "1.0.0"

  @doc """
  Says hello to the given `name`.

  Returns `:ok`.

  ## Examples
```

```
      iex> MyApp.Hello.world(:john)
      :ok

  """
  @doc since: "1.3.0"
  def world(name) do
    IO.puts("hello #{name}")
  end
end
```

The `@moduledoc` attribute is used to add documentation to the module. `@doc` is used before a function to provide documentation for it. Besides the attributes above, `@typedoc` can also be used to attach documentation to types defined as part of typespecs.

Function arguments

When documenting a function, argument names are inferred by the compiler. For example:

```
def size(%{size: size}) do
  size
end
```

The compiler will infer this argument as `map`. Sometimes the inference will be suboptimal, especially if the function contains multiple clauses with the argument matching on different values each time. You can specify the proper names for documentation by declaring only the function head at any moment before the implementation:

```elixir
def size(map_with_size)

def size(%{size: size}) do
  size
end
```

Documentation metadata

Elixir allows developers to attach arbitrary metadata to the documentation. This is done by passing a keyword list to the relevant attribute (such as `@moduledoc`, `@typedoc`, and `@doc`).

Metadata can have any key. Documentation tools often use metadata to provide more data to readers and to enrich the user experience. The following keys already have a predefined meaning used by tooling:

`:deprecated`

Another common metadata is `:deprecated`, which emits a warning in the documentation, explaining that its usage is discouraged:

```elixir
@doc deprecated: "Use Foo.bar/2 instead"
```

Note that the `:deprecated` key does not warn when a developer invokes the functions. If you want the code to also emit a warning, you can use the `@deprecated` attribute:

```elixir
@deprecated "Use Foo.bar/2 instead"
```

`:group`

The group a function, callback or type belongs to. This is used in `iex` for autocompleting and also to automatically by ExDoc to group items in the sidebar:

```
@doc group: "Query"
def all(query)

@doc group: "Schema"
def insert(schema)
```

`:since`

It annotates in which version that particular module, function, type, or callback was added:

```
@doc since: "1.3.0"
def world(name) do
  IO.puts("hello #{name}")
end
```

Recommendations

When writing documentation:

- Keep the first paragraph of the documentation concise and simple, typically one-line. Tools like ExDoc use the first line to generate a summary.
- Reference modules by their full name. Markdown uses backticks (`) to quote code. Elixir builds on top of that to automatically generate links when module or function names are referenced. For this reason, always use full module names. If you have a module called

`MyApp.Hello`, always reference it as `` `MyApp.Hello` `` and never as `` `Hello` ``.

- Reference functions by name and arity if they are local, as in `` `world/1` ``, or by module, name and arity if pointing to an external module: `` `MyApp.Hello.world/1` ``.
- Reference a `@callback` by prepending `c:`, as in `` `c:world/1` ``.
- Reference a `@type` by prepending `t:`, as in `` `t:values/0` ``.
- Start new sections with second level Markdown headers `##`. First level headers are reserved for module and function names.
- Place documentation before the first clause of multi-clause functions. Documentation is always per function and arity and not per clause.
- Use the `:since` key in the documentation metadata to annotate whenever new functions or modules are added to your API.

Doctests

We advise developers to include examples in their documentation, often under their own `## Examples` heading. To ensure examples do not get out of date, Elixir's test framework (ExUnit) provides a feature called doctests that allows developers to test the examples in their documentation. Doctests work by parsing out code samples starting with `iex>` from the documentation. You can read more about them at `ExUnit.DocTest`.

Documentation != Code comments

Elixir treats documentation and code comments as different concepts. Documentation is an explicit contract between you and users of your Application Programming Interface (API), be they third-party developers,

co-workers, or your future self. Modules and functions must always be documented if they are part of your API.

Code comments are aimed at developers reading the code. They are useful for marking improvements, leaving notes (for example, why you had to resort to a workaround due to a bug in a library), and so forth. They are tied to the source code: you can completely rewrite a function and remove all existing code comments, and it will continue to behave the same, with no change to either its behavior or its documentation.

Because private functions cannot be accessed externally, Elixir will warn if a private function has a `@doc` attribute and will discard its content. However, you can add code comments to private functions, as with any other piece of code, and we recommend developers to do so whenever they believe it will add relevant information to the readers and maintainers of such code.

In summary, documentation is a contract with users of your API, who may not necessarily have access to the source code, whereas code comments are for those who interact directly with the source. You can learn and express different guarantees about your software by separating those two concepts.

Hiding internal modules and functions

Besides the modules and functions libraries provide as part of their public interface, libraries may also implement important functionality that is not part of their API. While these modules and functions can be accessed, they are meant to be internal to the library and thus should not have documentation for end users.

Conveniently, Elixir allows developers to hide modules and functions from the documentation, by setting `@doc false` to hide a particular function, or `@moduledoc false` to hide the whole module. If a module is hidden, you may even document the functions in the module, but the module itself won't be listed in the documentation:

```elixir
defmodule MyApp.Hidden do
  @moduledoc false

  @doc """
  This function won't be listed in docs.
  """
  def function_that_wont_be_listed_in_docs do
    # ...
  end
end
```

In case you don't want to hide a whole module, you can hide functions individually:

```elixir
defmodule MyApp.Sample do
  @doc false
  def add(a, b), do: a + b
end
```

However, keep in mind `@moduledoc false` or `@doc false` do not make a function private. The function above can still be invoked as `MyApp.Sample.add(1, 2)`. Not only that, if `MyApp.Sample` is imported, the `add/2` function will also be imported into the caller. For those reasons, be cautious when adding `@doc false` to functions, instead use one of these two options:

- Move the undocumented function to a module with `@moduledoc false`, like `MyApp.Hidden`, ensuring the function won't be accidentally exposed or imported. Remember that you can use `@moduledoc false` to hide a whole module and still document each function with `@doc`. Tools will still ignore the module.

- Start the function name with one or two underscores, for example, `__add__/2`. Functions starting with underscore are automatically treated as hidden, although you can also be explicit and add `@doc false`. The compiler does not import functions with leading underscores and they hint to anyone reading the code of their intended private usage.

`Code.fetch_docs/1`

Elixir stores documentation inside pre-defined chunks in the bytecode. Documentation is not loaded into memory when modules are loaded, instead, it can be read from the bytecode in disk using the `Code.fetch_docs/1` function. The downside is that modules defined in-memory, like the ones defined in IEx, cannot have their documentation accessed as they do not write their bytecode to disk.

Optional syntax sheet

In the previous chapters, we learned that the Elixir syntax allows developers to omit delimiters in a few occasions to make code more readable. For example, we learned that parentheses are optional:

```
iex> length([1, 2, 3]) == length [1, 2, 3]
true
```

and that `do-end` blocks are equivalent to keyword lists:

```
# do-end blocks
iex> if true do
...>   :this
...> else
...>   :that
...> end
:this

# keyword lists
iex> if true, do: :this, else: :that
:this
```

Keyword lists use Elixir's regular notation for separating arguments, where we separate each key-value pair with commas, and each key is followed by `:`. In the `do`-blocks, we get rid of the colons, the commas, and separate each keyword by a newline. They are useful exactly because they remove the verbosity when writing blocks of code. Most of the time, we use the block syntax, but it is good to know they are equivalent.

Those conveniences, which we call here "optional syntax", allow the language syntax core to be small, without sacrificing the readability and expressiveness of your code. In this brief chapter, we will review the four rules provided by the language, using a short snippet as playground.

Walk-through

Take the following code:

```
if variable? do
  Call.this()
else
  Call.that()
end
```

Now let's remove the conveniences one by one:

1. `do-end` blocks are equivalent to keywords:

    ```
    if variable?, do: Call.this(), else: Call.that()
    ```

2. Keyword lists as last argument do not require square brackets, but let's add them:

    ```
    if variable?, [do: Call.this(), else: Call.that()]
    ```

3. Keyword lists are the same as lists of two-element tuples:

    ```
    if variable?, [{:do, Call.this()}, {:else, Call.that()}]
    ```

4. Finally, parentheses are optional on function calls, but let's add them:

```
if(variable?, [{:do, Call.this()}, {:else,
Call.that()}])
```

That's it! Those four rules outline the optional syntax available in Elixir.

To understand why these rules matter, we can briefly compare Elixir with many other programming languages. Most programming languages have several keywords for defining methods, functions, conditionals, loops, and so forth. Each of those keywords have their own syntax rules attached to them.

However, in Elixir, none of these language features require special "keywords", instead they all build from this small set of rules. The other benefit is that developers can also extend the language in a way that is consistent with the language itself, since the constructs for designing and extending the language are the same. We further explore this topic in the "Meta-programming" guide.

At the end of the day, those rules are what enables us to write:

```
defmodule Math do
  def add(a, b) do
    a + b
  end
end
```

instead of:

```
defmodule(Math, [
  {:do, def(add(a, b), [{:do, a + b}])}
])
```

Whenever you have any questions, this quick walk-through has you covered.

Finally, if you are concerned about when to apply these rules, it's worth noting that the Elixir formatter handles those concerns for you. Most Elixir developers use the `mix format` task to format their codebases according to a well-defined set of rules defined by the Elixir team and the community. For instance, `mix format` will always add parentheses to function calls unless explicitly configured not to do so. This helps to maintain consistency across all codebases within organizations and the wider community.

Erlang libraries

Elixir provides excellent interoperability with Erlang libraries. In fact, Elixir discourages simply wrapping Erlang libraries in favor of directly interfacing with Erlang code. In this section, we will present some of the most common and useful Erlang functionality that is not found in Elixir.

Erlang modules have a different naming convention than in Elixir and start in lowercase. In both cases, module names are atoms and we invoke functions by dispatching to the module name:

```
iex> is_atom(String)
true
iex> String.first("hello")
"h"
iex> is_atom(:binary)
true
iex> :binary.first("hello")
104
```

As you grow more proficient in Elixir, you may want to explore the Erlang STDLIB Reference Manual in more detail.

The binary module

The built-in Elixir String module handles binaries that are UTF-8 encoded. The `:binary` module is useful when you are dealing with binary data that is not necessarily UTF-8 encoded.

```
iex> String.to_charlist("Ø")
[216]
```

```
iex> :binary.bin_to_list("Ø")
[195, 152]
```

The above example shows the difference; the `String` module returns Unicode codepoints, while `:binary` deals with raw data bytes.

Formatted text output

Elixir does not contain a function similar to `printf` found in C and other languages. Luckily, the Erlang standard library functions `:io.format/2` and `:io_lib.format/2` may be used. The first formats to terminal output, while the second formats to an iolist. The format specifiers differ from `printf`, refer to the Erlang documentation for details.

```
iex> :io.format("Pi is approximately given by:~10.3f~n",
[:math.pi])
Pi is approximately given by:     3.142
:ok
iex> to_string(:io_lib.format("Pi is approximately given
by:~10.3f~n", [:math.pi]))
"Pi is approximately given by:     3.142\n"
```

The crypto module

The `:crypto` module contains hashing functions, digital signatures, encryption and more:

```
iex> Base.encode16(:crypto.hash(:sha256, "Elixir"))
"3315715A7A3AD57428298676C5AE465DADA38D951BDFAC9348A8A31E9C7
401CB"
```

The `:crypto` module is part of the `:crypto` application that ships with Erlang. This means you must list the `:crypto` application as an additional application in your project configuration. To do this, edit your `mix.exs` file to include:

```
def application do
  [extra_applications: [:crypto]]
end
```

Any module that is not part of the `:kernel` or `:stdlib` Erlang applications must have their application explicitly listed in your `mix.exs`. You can find the application name of any Erlang module in the Erlang documentation, immediately below the Erlang logo in the sidebar.

The digraph module

The `:digraph` and `:digraph_utils` modules contain functions for dealing with directed graphs built of vertices and edges. After constructing the graph, the algorithms in there will help find, for instance, the shortest path between two vertices, or loops in the graph.

Given three vertices, find the shortest path from the first to the last.

```
iex> digraph = :digraph.new()
iex> coords = [{0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}]
iex> [v0, v1, v2] = (for c <- coords, do:
:digraph.add_vertex(digraph, c))
iex> :digraph.add_edge(digraph, v0, v1)
iex> :digraph.add_edge(digraph, v1, v2)
iex> :digraph.get_short_path(digraph, v0, v2)
[{0.0, 0.0}, {1.0, 0.0}, {1.0, 1.0}]
```

Note that the functions in `:digraph` alter the graph structure in-place, this is possible because they are implemented as ETS tables, explained next.

Erlang Term Storage (ETS)

The modules `:ets` and `:dets` handle storage of large data structures in memory or on disk respectively.

ETS lets you create a table containing tuples. By default, ETS tables are protected, which means only the owner process may write to the table but any other process can read. ETS has some functionality to allow a table to be used as a simple database, a key-value store or as a cache mechanism.

The functions in the `ets` module will modify the state of the table as a side-effect.

```
iex> table = :ets.new(:ets_test, [])
# Store as tuples with {name, population}
iex> :ets.insert(table, {"China", 1_374_000_000})
iex> :ets.insert(table, {"India", 1_284_000_000})
iex> :ets.insert(table, {"USA", 322_000_000})
iex> :ets.i(table)
<1   > {<<"India">>,1284000000}
<2   > {<<"USA">>,322000000}
<3   > {<<"China">>,1374000000}
```

The math module

The `:math` module contains common mathematical operations covering trigonometry, exponential, and logarithmic functions.

```
iex> angle_45_deg = :math.pi() * 45.0 / 180.0
iex> :math.sin(angle_45_deg)
```

```
0.7071067811865475
iex> :math.exp(55.0)
7.694785265142018e23
iex> :math.log(7.694785265142018e23)
55.0
```

The queue module

The `:queue` module provides a data structure that implements (double-ended) FIFO (first-in first-out) queues efficiently:

```
iex> q = :queue.new
iex> q = :queue.in("A", q)
iex> q = :queue.in("B", q)
iex> {value, q} = :queue.out(q)
iex> value
{:value, "A"}
iex> {value, q} = :queue.out(q)
iex> value
{:value, "B"}
iex> {value, q} = :queue.out(q)
iex> value
:empty
```

The rand module

The `:rand` has functions for returning random values and setting the random seed.

```
iex> :rand.uniform()
0.8175669086010815
iex> _ = :rand.seed(:exs1024, {123, 123_534, 345_345})
```

```
iex> :rand.uniform()
0.5820506340260994
iex> :rand.uniform(6)
6
```

The zip and zlib modules

The `:zip` module lets you read and write ZIP files to and from disk or memory, as well as extracting file information.

This code counts the number of files in a ZIP file:

```
iex> :zip.foldl(fn _, _, _, acc -> acc + 1 end, 0,
:binary.bin_to_list("file.zip"))
{:ok, 633}
```

The `:zlib` module deals with data compression in zlib format, as found in the `gzip` command line utility found in Unix systems.

```
iex> song = "
...> Mary had a little lamb,
...> His fleece was white as snow,
...> And everywhere that Mary went,
...> The lamb was sure to go."
iex> compressed = :zlib.compress(song)
iex> byte_size(song)
110
iex> byte_size(compressed)
99
iex> :zlib.uncompress(compressed)
"\nMary had a little lamb,\nHis fleece was white as
```

```
snow,\nAnd everywhere that Mary went,\nThe lamb was sure to
go."
```

Learning Erlang

If you want to get deeper into Erlang, here's a list of online resources that cover Erlang's fundamentals and its more advanced features:

- This Erlang Syntax: A Crash Course provides a concise intro to Erlang's syntax. Each code snippet is accompanied by equivalent code in Elixir. This is an opportunity for you to not only get some exposure to Erlang's syntax but also review what you learned about Elixir.

- Erlang's official website has a short tutorial. There is a chapter with pictures briefly describing Erlang's primitives for concurrent programming.

- Learn You Some Erlang for Great Good! is an excellent introduction to Erlang, its design principles, standard library, best practices, and much more. Once you have read through the crash course mentioned above, you'll be able to safely skip the first couple of chapters in the book that mostly deal with the syntax. When you reach The Hitchhiker's Guide to Concurrency chapter, that's where the real fun starts.

Our last step is to take a look at existing Elixir (and Erlang) libraries you might use while debugging.

Debugging

There are a number of ways to debug code in Elixir. In this chapter we will cover some of the more common ways of doing so.

IO.inspect/2

What makes `IO.inspect(item, opts \\ [])` really useful in debugging is that it returns the `item` argument passed to it without affecting the behavior of the original code. Let's see an example.

```
(1..10)
|> IO.inspect()
|> Enum.map(fn x -> x * 2 end)
|> IO.inspect()
|> Enum.sum()
|> IO.inspect()
```

Prints:

```
1..10
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
110
```

As you can see `IO.inspect/2` makes it possible to "spy" on values almost anywhere in your code without altering the result, making it very helpful inside of a pipeline like in the above case.

`IO.inspect/2` also provides the ability to decorate the output with a `label` option. The label will be printed before the inspected `item`:

```
[1, 2, 3]
|> IO.inspect(label: "before")
|> Enum.map(&(&1 * 2))
|> IO.inspect(label: "after")
|> Enum.sum
```

Prints:

```
before: [1, 2, 3]
after: [2, 4, 6]
```

It is also very common to use `IO.inspect/2` with `binding/0`, which returns all variable names and their values:

```
def some_function(a, b, c) do
  IO.inspect(binding())
  ...
end
```

When `some_function/3` is invoked with `:foo`, `"bar"`, `:baz` it prints:

```
[a: :foo, b: "bar", c: :baz]
```

See `IO.inspect/2` and `Inspect.Opts` respectively to learn more about the function and read about all supported options.

dbg/2

Elixir v1.14 introduced `dbg/2`. `dbg` is similar to `IO.inspect/2` but specifically tailored for debugging. It prints the value passed to it and

returns it (just like `IO.inspect/2`), but it also prints the code and location.

```
# In my_file.exs
feature = %{name: :dbg, inspiration: "Rust"}
dbg(feature)
dbg(Map.put(feature, :in_version, "1.14.0"))
```

The code above prints this:

```
[my_file.exs:2: (file)]
feature #=> %{inspiration: "Rust", name: :dbg}
[my_file.exs:3: (file)]
Map.put(feature, :in_version, "1.14.0") #=> %{in_version:
"1.14.0", inspiration: "Rust", name: :dbg}
```

When talking about `IO.inspect/2`, we mentioned its usefulness when placed between steps of `|>` pipelines. `dbg` does it better: it understands Elixir code, so it will print values at *every step of the pipeline*.

```
# In dbg_pipes.exs
__ENV__.file
|> String.split("/", trim: true)
|> List.last()
|> File.exists?()
|> dbg()
```

This code prints:

```
[dbg_pipes.exs:5: (file)]
__ENV__.file #=> "/home/myuser/dbg_pipes.exs"
|> String.split("/", trim: true) #=> ["home", "myuser",
"dbg_pipes.exs"]
```

```
|> List.last() #=> "dbg_pipes.exs"
|> File.exists?() #=> true
```

While `dbg` provides conveniences around Elixir constructs, you will need `IEx` if you want to execute code and set breakpoints while debugging.

Pry

When using `IEx`, you may pass `--dbg pry` as an option to "stop" the code execution where the `dbg` call is:

```
$ iex --dbg pry
```

Or to debug inside of a project:

```
$ iex --dbg pry -S mix
```

Now any call to `dbg` will ask if you want to pry the existing code. If you accept, you'll be able to access all variables, as well as imports and aliases from the code, directly from IEx. This is called "prying". While the pry session is running, the code execution stops, until `continue` (or `c`) or `next` (or `n`) are called. Remember you can always run `iex` in the context of a project with `iex -S mix TASK`.

Breakpoints

`dbg` calls require us to change the code we intend to debug and has limited stepping functionality. Luckily IEx also provides a `IEx.break!/2` function which allows you to set and manage breakpoints on any Elixir code without modifying its source:

Similar to `dbg`, once a breakpoint is reached, code execution stops until `continue` (or `c`) or `next` (or `n`) are invoked. Breakpoints can navigate line-by-line by default, however, they do not have access to aliases and imports when breakpoints are set on compiled modules.

The `mix test` task direct integration with breakpoints via the `-b`/`--breakpoints` flag. When the flag is used, a breakpoint is set at the beginning of every test that will run:

Here are some commands you can use in practice:

```
# Debug all failed tests
$ iex -S mix test --breakpoints --failed
# Debug the test at the given file:line
$ iex -S mix test -b path/to/file:line
```

Observer

For debugging complex systems, jumping at the code is not enough. It is necessary to have an understanding of the whole virtual machine, processes, applications, as well as set up tracing mechanisms. Luckily this can be achieved in Erlang with `:observer`. In your application:

```
$ iex
iex> :observer.start()
```

> **Missing dependencies**
>
> When running `iex` inside a project with `iex -S mix`, `observer` won't be available as a dependency. To do so, you will need to call the following functions before:
>
> ```
> iex> Mix.ensure_application!(:observer)
> iex> :observer.start()
> ```
>
> If any of the calls above fail, here is what may have happened: some package managers default to installing a minimized Erlang without WX bindings for GUI support. In some package managers, you may be able to replace the headless Erlang with a more complete package (look for packages named `erlang` vs `erlang-nox` on Debian/Ubuntu/Arch). In others managers, you may need to install a separate `erlang-wx` (or similarly named) package.

The above will open another Graphical User Interface that provides many panes to fully understand and navigate the runtime and your project.

We explore the Observer in the context of an actual project in the Dynamic Supervisor chapter of the Mix & OTP guide. This is one of the debugging techniques the Phoenix framework used to achieve 2 million connections on a single machine.

If you are using the Phoenix web framework, it ships with the Phoenix LiveDashboard, a web dashboard for production nodes which provides similar features to Observer.

Finally, remember you can also get a mini-overview of the runtime info by calling `runtime_info/0` directly in IEx.

Other tools and community

We have just scratched the surface of what the Erlang VM has to offer, for example:

- Alongside the observer application, Erlang also includes a `:crashdump_viewer` to view crash dumps

- Integration with OS level tracers, such as Linux Trace Toolkit, DTRACE, and SystemTap

- Microstate accounting measures how much time the runtime spends in several low-level tasks in a short time interval

- Mix ships with many tasks under the `profile` namespace, such as `mix profile.cprof` and `mix profile.fprof`

- For more advanced use cases, we recommend the excellent Erlang in Anger, which is available as a free ebook

Happy debugging!

<code class="makeup elixir" translate="no"><samp class="n">cart</samp>
<samp class="w"> </samp><samp class="o">=</samp><samp class="w">
</samp><samp class="p" data-group-id="group-1">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-2">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-2">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-1">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">any?</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp

class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">==</samp><samp class="w"> </samp><samp
class="s">"orange"</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w">

</samp><samp class="no">true</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">any?</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="w">
</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">==
</samp><samp class="w"> </samp><samp class="s">"pear"</samp>
<samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="no">false</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">any?</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="p" data-group-id="group-2">
[</samp><samp class="p" data-group-id="group-2">]</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">==</samp><samp class="w"> </samp><samp
class="s">"orange"</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w">

</samp><samp class="no">false</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">all?</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&

</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w">
</samp><samp class="o">></samp><samp class="w"> </samp><samp class="mi">0</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="no">true</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">all?</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="w">
</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">>>
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="no">false</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">all?</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-2">
[</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w">
</samp><samp class="o">></samp><samp class="w"> </samp><samp class="mi">0</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="no">true</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">member?</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp

class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-2">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple" </samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-2">}</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w">

</samp><samp class="no">true</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">member?</samp><samp class="p" data-group-id="group-3">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">:something_else</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="no">false</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="p" data-group-id="group-1">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w"> </samp><samp class="ow">in</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="w">

</samp><samp class="no">true</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="ss">:something_else</samp><samp class="w"> </samp><samp class="ow">in</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="w">

</samp><samp class="no">false</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">empty?</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="no">false</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">empty?</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-3">[</samp><samp class="p" data-group-id="group-3">]</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="no">true</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">filter</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~</samp><samp class="w"> </samp><samp class="s">"o"</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp

class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p" data-group-id="group-3">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">filter</samp><samp class="p" data-group-id="group-5">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="p" data-
group-id="group-6">(</samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">=~</samp><samp class="w"> </samp><samp
class="s">"e"</samp><samp class="p" data-group-id="group-6">)</samp>
<samp class="p" data-group-id="group-5">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-8">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-9">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-9">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-7">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">reject</samp><samp class="p" data-

group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="p" data-group-id="group-2">(</samp><samp
class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"o"</samp><samp
class="p" data-group-id="group-2">)</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">flat_map</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="k" data-
group-id="group-2">fn</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k">if</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">>>
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="ss">do</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p" data-group-id="group-3">]
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="ss">else</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="p" data-group-id="group-4">[</samp><samp
class="p" data-group-id="group-4">]</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-
id="group-5">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="k">for</samp><samp
class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp
class="w"> </samp><samp class="k" data-group-id="group-1">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">item</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">for</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">%{</samp><samp class="ss">count</samp><samp class="p">:</samp>
<samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w"> </samp><samp class="o"><-
</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">fruit</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="s">"banana"</samp><samp class="p" data-group-id="group-3">]
</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">map</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-
group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p" data-group-id="group-2">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">map</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="k" data-group-id="group-4">fn</samp>
<samp class="w"> </samp><samp class="n">item</samp><samp
class="w"> </samp><samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="n">item</samp><samp class="w"> </samp><samp class="o">|
</samp><samp class="w"> </samp><samp class="ss">count</samp>
<samp class="p">:</samp><samp class="w"> </samp><samp

class="n">item</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">+</samp><samp class="w"> </samp><samp class="mi">10</samp><samp class="p" data-group-id="group-5">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-4">end</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">13</samp><samp class="p" data-group-id="group-7">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-8">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">11</samp><samp class="p" data-group-id="group-8">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-9">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">16</samp><samp class="p" data-group-id="group-9">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">]</samp></code>

```
<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">map_every</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="k" data-group-id="group-2">fn</samp><samp
class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="n">item</samp><samp class="w"> </samp><samp class="o">|
</samp><samp class="w"> </samp><samp class="ss">count</samp>
<samp class="p">:</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="mi">10</samp><samp
class="p" data-group-id="group-3">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">13</samp><samp class="p" data-group-id="group-5">}
</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-6">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
```

<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-6">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">16</samp><samp class="p" data-group-id="group-7">}
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">for</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="k" data-group-id="group-1">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">for</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="w">

</samp><samp class="o"><-</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp class="w"> </samp><samp class="k" data-group-id="group-1">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">each</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="nc">IO</samp><samp class="o">.</samp><samp class="n">puts</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="n">apple</samp><samp class="w">

</samp><samp class="n">banana</samp><samp class="w">

</samp><samp class="n">orange</samp><samp class="w">

</samp><samp class="ss">:ok</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">reduce</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">0</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">acc</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">+</samp><samp class="w"> </samp><samp class="n">acc</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-2">end</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">10</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">map_reduce</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">0</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">acc</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w">
</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">+</samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-2">end</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">10</samp></code>

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">{</samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp
class="p" data-group-id="group-3">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">{</samp><samp
class="p" data-group-id="group-5">[</samp><samp class="s">"apple"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"banana"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-
id="group-5">]</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">10</samp><samp class="p" data-group-
id="group-4">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">scan</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">0</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="k" data-group-id="group-2">fn</samp><samp
class="w"> </samp><samp class="n">item</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp
class="w"> </samp><samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp
class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-2">end</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="mi">3</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">4</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">10</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">reduce_while</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">0</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp>
<samp class="w"> </samp><samp class="n">item</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">acc</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k">if</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">==</samp><samp class="w"> </samp><samp class="s">"orange"</samp>
<samp class="w"> </samp><samp class="k" data-group-id="group-3">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-4">{</samp><samp class="ss">:halt</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="p" data-group-id="group-4">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k" data-group-id="group-3">else</samp><samp

class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-5">{</samp><samp
class="ss">:cont</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp
class="p" data-group-id="group-5">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k" data-group-id="group-3">end</samp><samp
class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="mi">4</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="k">for</samp><samp
class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">reduce</samp><samp class="p">:</samp>
<samp class="w"> </samp><samp class="mi">0</samp><samp
class="w"> </samp><samp class="k" data-group-id="group-1">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="w"> </samp><samp
class="o">-></samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp
class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="mi">10</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">for</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">reduce</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="mi">0</samp><samp class="w"> </samp><samp class="k" data-group-id="group-1">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="n">item</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">acc</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="mi">9</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">3</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">frequencies</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-2">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">%{</samp><samp class="s">"apple"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">frequencies_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="nc">String</samp><samp class="o">.</samp><samp class="n">last</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">%{</samp><samp class="s">"a"</samp><samp class="w"> </samp><samp class="p">=>

</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"e" </samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">& </samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~ </samp><samp class="w"> </samp><samp class="s">"e"</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">2</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-3">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="p" data-group-id="group-4">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~</samp><samp class="w"> </samp><samp class="s">"y"</samp><samp class="p" data-group-id="group-4">)</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="mi">0</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">map</samp><samp class="p" data-group-id="group-1">

(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">. </samp><samp class="n">sum</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">) </samp><samp class="w">

</samp><samp class="mi">10</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">sum_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">& </samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">10</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.</samp> <samp class="n">map</samp><samp class="p" data-group-id="group-1">(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">. </samp><samp class="n">product</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">) </samp><samp class="w">

</samp><samp class="mi">18</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">product_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">18</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w">
</samp><samp class="nc">Enum</samp><samp class="o">.</samp>
<samp class="n">map</samp><samp class="p" data-group-id="group-1">
(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.
</samp><samp class="n">sort</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-3">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|>
</samp><samp class="w"> </samp><samp class="nc">Enum</samp>
<samp class="o">.</samp><samp class="n">map</samp><samp class="p" data-group-id="group-4">(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.
</samp><samp class="n">fruit</samp><samp class="p" data-group-

id="group-4">)</samp><samp class="w"> </samp><samp class="o">|>
</samp><samp class="w"> </samp><samp class="nc">Enum</samp>
<samp class="o">.</samp><samp class="n">sort</samp><samp class="p"
data-group-id="group-5">(</samp><samp class="ss">:desc</samp><samp
class="p" data-group-id="group-5">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp
class="s">"orange"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"apple"</samp><samp
class="p" data-group-id="group-6">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">sort_by</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">count</samp><samp class="p" data-
group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp

class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp><samp
class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">sort_by</samp><samp class="p" data-group-id="group-6">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="w">
</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">:desc</samp><samp class="p" data-group-
id="group-6">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">[</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-8">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-9">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>

<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-9">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-10">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-10">}
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w">
</samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">map</samp><samp class="p" data-group-id="group-1">(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.
</samp><samp class="n">min</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="w">

</samp><samp class="mi">1</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">min_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp

class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-2">}</samp>
</code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">cart</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w">
</samp><samp class="nc">Enum</samp><samp class="o">.</samp>
<samp class="n">map</samp><samp class="p" data-group-id="group-1">(</samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w"> </samp><samp class="o">|></samp><samp class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.
</samp><samp class="n">max</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="w">

</samp><samp class="mi">6</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">max_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-2">}</samp>
</code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">concat</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="p" data-group-id="group-
2">[</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">3</samp><samp class="p" data-
group-id="group-3">]</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="p" data-group-id="group-4">[</samp>
<samp class="mi">4</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">5</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">6</samp><samp class="p" data-
group-id="group-4">]</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="p" data-group-id="group-5">[</samp>
<samp class="mi">7</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">8</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">9</samp><samp class="p" data-
group-id="group-5">]</samp><samp class="p" data-group-id="group-2">]
</samp><samp class="p" data-group-id="group-1">)</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp
class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">4</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">5</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">6</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">7</samp><samp class="p">,

</samp><samp class="w"> </samp><samp class="mi">8</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">9</samp><samp class="p" data-group-id="group-6">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">concat</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-2">[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-3">[</samp><samp class="mi">4</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">5</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-3">]</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">4</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">5</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">flat_map</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp><samp class="w"> </samp><samp

class="n">item</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="nc">List</samp><samp class="o">.</samp><samp
class="n">duplicate</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"apple"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"orange"</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"orange"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-
id="group-4">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">flat_map_reduce</samp><samp
class="p" data-group-id="group-1">(</samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">0</samp><samp class="p">,</samp><samp

class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp>
<samp class="w"> </samp><samp class="n">item</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="n">acc</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">list</samp><samp class="w"> </samp><samp
class="o">=</samp><samp class="w"> </samp><samp
class="nc">List</samp><samp class="o">.</samp><samp
class="n">duplicate</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="w"> </samp><samp
class="o">=</samp><samp class="w"> </samp><samp
class="n">acc</samp><samp class="w"> </samp><samp class="o">+
</samp><samp class="w"> </samp><samp class="n">item</samp><samp
class="o">.</samp><samp class="n">count</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-4">{</samp><samp
class="n">list</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="p" data-group-
id="group-4">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">{</samp><samp
class="p" data-group-id="group-6">[</samp><samp class="s">"apple"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"banana"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"orange"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-
id="group-6">]</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">10</samp><samp class="p" data-group-
id="group-5">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="k">for</samp><samp
class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp
class="o"><-</samp><samp class="w"> </samp><samp
class="nc">List</samp><samp class="o">.</samp><samp
class="n">duplicate</samp><samp class="p" data-group-id="group-1">
(</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w"> </samp><samp class="k" data-group-
id="group-2">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">fruit</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">pairs</samp><samp class="w"> </samp><samp class="o">=</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">[</samp><samp class="p" data-group-id="group-2">{</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-2">}</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-3">{</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-4">{</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp><samp class="p" data-group-id="group-1">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">into</samp><samp class="p" data-group-id="group-5">(</samp><samp class="n">pairs</samp><samp class="p">,</samp><samp

class="w"> </samp><samp class="p" data-group-id="group-6">%
{</samp><samp class="p" data-group-id="group-6">}</samp><samp
class="p" data-group-id="group-5">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">%{</samp><samp
class="s">"apple"</samp><samp class="w"> </samp><samp class="p">=>
</samp><samp class="w"> </samp><samp class="mi">3</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"
</samp><samp class="w"> </samp><samp class="p">=></samp><samp
class="w"> </samp><samp class="mi">1</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="s">"orange"</samp>
<samp class="w"> </samp><samp class="p">=></samp><samp
class="w"> </samp><samp class="mi">6</samp><samp class="p" data-
group-id="group-7">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">into</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="p" data-
group-id="group-2">%{</samp><samp class="p" data-group-id="group-
2">}</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="k" data-group-id="group-3">fn</samp><samp class="w"> </samp>
<samp class="n">item</samp><samp class="w"> </samp><samp
class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-4">{</samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p" data-group-id="group-4">}
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-3">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="s">"apple"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-5">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">to_list</samp><samp class="p" data-group-id="group-1">(</samp><samp class="mi">1</samp><samp class="o">..</samp><samp class="mi">5</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">4</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">5</samp><samp class="p" data-group-id="group-2">]</samp>
</code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">for</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="w">
</samp><samp class="o"><-</samp><samp class="w"> </samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">into</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">%
{</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">{</samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="p" data-group-id="group-3">}
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="s">"apple"</samp><samp class="w"> </samp><samp class="p">=>
</samp><samp class="w"> </samp><samp class="mi">3</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"
</samp><samp class="w"> </samp><samp class="p">=></samp><samp
class="w"> </samp><samp class="mi">1</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="s">"orange"</samp>
<samp class="w"> </samp><samp class="p">=></samp><samp
class="w"> </samp><samp class="mi">6</samp><samp class="p" data-
group-id="group-4">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">dedup</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="p" data-group-id="group-2">
[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">2</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">2</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p" data-group-

id="group-2">]</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p" data-group-
id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">dedup_by</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">=~</samp><samp class="w"> </samp><samp
class="s">"a"</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p" data-group-id="group-2">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">dedup_by</samp><samp class="p" data-group-id="group-4">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="w">

</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">count</samp><samp class="w"> </samp><samp class="o">
<</samp><samp class="w"> </samp><samp class="mi">5</samp><samp class="p" data-group-id="group-4">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-6">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-6">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-5">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">uniq</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-2">
[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp

class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">uniq_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">& </samp><samp class="nc">String</samp><samp class="o">.</samp><samp class="n">last</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">) </samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp> <samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp>

<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">at</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">0</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-2">}</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">at</samp><samp class="p" data-group-id="group-3">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">10</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="no">nil</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">at</samp><samp class="p" data-group-id="group-4">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">10</samp><samp class="p">,

</samp><samp class="w"> </samp><samp class="ss">:none</samp>
<samp class="p" data-group-id="group-4">)</samp><samp class="w">

</samp><samp class="ss">:none</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">fetch</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">0</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">{</samp><samp
class="ss">:ok</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p" data-group-id="group-2">}</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">fetch</samp><samp class="p" data-group-id="group-4">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">10</samp><samp class="p" data-
group-id="group-4">)</samp><samp class="w">

</samp><samp class="ss">:error</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">fetch!</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp

class="mi">0</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-2">}</samp>
<samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">fetch!</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">10</samp><samp class="p" data-
group-id="group-3">)</samp><samp class="w">

</samp><samp class="gt">** (Enum.OutOfBoundsError) out of bounds
error</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">with_index</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-3">{</samp><samp
class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w"> </samp><samp

class="mi">0</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">{</samp><samp
class="p" data-group-id="group-6">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-6">}</samp>
<samp class="p">,</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">{</samp><samp
class="p" data-group-id="group-8">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">with_index</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="k" data-
group-id="group-2">fn</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">index</samp><samp class="w"> </samp>
<samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-3">{</samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">index</samp><samp class="p" data-group-
id="group-3">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-5">{</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">0</samp><samp class="p" data-group-
id="group-5">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-6">{</samp><samp
class="s">"banana"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">1</samp><samp class="p" data-group-
id="group-6">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">{</samp><samp
class="s">"orange"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p" data-group-
id="group-7">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">find</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="p" data-group-id="group-2">(</samp><samp
class="ni">&1</samp><samp class="o">.</samp><samp

class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"o"</samp><samp
class="p" data-group-id="group-2">)</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">find</samp><samp class="p" data-group-id="group-4">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="p" data-
group-id="group-5">(</samp><samp class="ni">&1</samp><samp
class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">=~</samp><samp class="w"> </samp><samp
class="s">"y"</samp><samp class="p" data-group-id="group-5">)</samp>
<samp class="p" data-group-id="group-4">)</samp><samp class="w">

</samp><samp class="no">nil</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">find</samp><samp class="p" data-group-id="group-6">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="ss">:none</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="o">&</samp><samp
class="p" data-group-id="group-7">(</samp><samp
class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"y"</samp><samp
class="p" data-group-id="group-7">)</samp><samp class="p" data-group-
id="group-6">)</samp><samp class="w">

</samp><samp class="ss">:none</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">find_index</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~</samp><samp class="w"> </samp><samp class="s">"o"</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="mi">2</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">find_index</samp><samp class="p" data-group-id="group-3">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="p" data-group-id="group-4">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w">
</samp><samp class="o">=~</samp><samp class="w"> </samp><samp class="s">"y"</samp><samp class="p" data-group-id="group-4">)</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="no">nil</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">find_value</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">fn</samp><samp class="w"> </samp><samp class="n">item</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w">
</samp><samp class="w">
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k">if</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">==
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="ss">do</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">else</samp><samp class="p">:</samp><samp
class="w"> </samp><samp class="no">nil</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-2">end</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="s">"banana"</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">find_value</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="ss">:none</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="k" data-group-
id="group-4">fn</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="k">if</samp><samp class="w"> </samp><samp
class="n">item</samp><samp class="o">.</samp><samp
class="n">count</samp><samp class="w"> </samp><samp class="o">==
</samp><samp class="w"> </samp><samp class="mi">100</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="ss">do</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">item</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="ss">else</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="no">nil</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-4">end</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="ss">:none</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">group_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="nc">String</samp><samp class="o">.</samp><samp class="n">last</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">%{</samp><samp class="w">

  </samp><samp class="s">"a"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="p" data-group-id="group-4">[</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp><samp class="p" data-group-id="group-4">]</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="s">"e"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="p" data-group-id="group-6">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-7">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-8">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp> <samp class="w">

</samp><samp class="p" data-group-id="group-6">]</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">group_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="nc">String</samp><samp class="o">.</samp><samp class="n">last</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">) </samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">%{</samp><samp class="w">

</samp><samp class="s">"a"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="p" data-group-id="group-4">[</samp><samp class="s">"banana"</samp><samp class="p" data-group-id="group-4">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">"e"</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="p" data-group-id="group-5">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-5">]</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">join</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-2">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">", "</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="s">"apple, banana, orange"</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">map_join</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">", "</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="w"> </samp><samp class="ni">&1</samp><samp class="o">.</samp><samp

class="n">fruit</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w">

</samp><samp class="s">"apple, banana, orange"</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">intersperse</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="p" data-group-id="group-
2">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"banana"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p" data-group-id="group-2">]</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">", "
</samp><samp class="p" data-group-id="group-1">)</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">", "</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="s">", "</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">map_intersperse</samp><samp
class="p" data-group-id="group-1">(</samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">", "</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="o">&</samp><samp class="w">
</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="s">", "</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">, </samp><samp class="w"> </samp><samp class="s">", "</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange" </samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">slice</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">0</samp><samp class="o">..</samp><samp class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-4">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp

class="o">.</samp><samp class="n">slice</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">-</samp><samp class="mi">2</samp><samp class="o">..</samp><samp class="o">-</samp><samp class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">slice</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

```
</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="w">

   </samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

   </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp
class="n">fruits</samp><samp class="w"> </samp><samp class="o">=
</samp><samp class="w"> </samp><samp class="p" data-group-
id="group-1">[</samp><samp class="s">"apple"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"grape"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"pear"</samp><samp
class="p" data-group-id="group-1">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">slide</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="n">fruits</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="mi">2</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
```

class="mi">0</samp><samp class="p" data-group-id="group-2">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="s">"grape"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"banana"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"pear"</samp><samp class="p" data-group-id="group-3">]
</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">slide</samp><samp class="p" data-group-id="group-4">
(</samp><samp class="n">fruits</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="mi">2</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">4</samp><samp class="p" data-group-id="group-4">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-5">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="s">"pear"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="s">"grape"</samp><samp class="p" data-group-id="group-5">]
</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">slide</samp><samp class="p" data-group-id="group-6">
(</samp><samp class="n">fruits</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="mi">1</samp><samp
class="o">..</samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">0</samp><samp
class="p" data-group-id="group-6">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">[</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"grape"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"apple" </samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"pear"</samp><samp class="p" data-group-id="group-7">] </samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">slide</samp><samp class="p" data-group-id="group-8">(</samp><samp class="n">fruits</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="mi">1</samp><samp class="o">..</samp><samp class="mi">3</samp><samp class="p">, </samp><samp class="w"> </samp><samp class="mi">4</samp><samp class="p" data-group-id="group-8">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-9">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"pear"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">, </samp><samp class="w"> </samp><samp class="s">"grape"</samp> <samp class="p">,</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-9">] </samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">reverse</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">reverse</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="p" data-
group-id="group-2">[</samp><samp class="ss">:this_will_be</samp>
<samp class="p">,</samp><samp class="w"> </samp><samp
class="ss">:the_tail</samp><samp class="p" data-group-id="group-2">]
</samp><samp class="p" data-group-id="group-1">)</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-6">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="ss">:this_will_be</samp><samp class="p">, </samp><samp class="w">

</samp><samp class="ss">:the_tail</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">reverse_slice</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp

class="w"> </samp><samp class="mi">2</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">split</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp

class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">{</samp><samp
class="p" data-group-id="group-3">[</samp><samp class="p" data-group-
id="group-4">%{</samp><samp class="ss">fruit</samp><samp
class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="ss">count</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p" data-group-
id="group-4">}</samp><samp class="p" data-group-id="group-3">]
</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-6">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-6">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-5">]</samp><samp
class="p" data-group-id="group-2">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">split</samp><samp class="p" data-

group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">-</samp><samp class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">{</samp><samp class="p" data-group-id="group-3">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="p" data-group-id="group-6">]</samp><samp class="p" data-group-id="group-2">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">split_while</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">{</samp><samp class="p" data-group-id="group-4">[</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp><samp class="p" data-group-id="group-4">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-7">}</samp><samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-8">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp

class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-6">]</samp><samp
class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">split_with</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="p" data-group-id="group-2">(</samp><samp
class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp
class="p" data-group-id="group-2">)</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">{</samp><samp
class="p" data-group-id="group-4">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-6">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-6">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-4">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">[</samp><samp class="p" data-group-id="group-8">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-8">}</samp> <samp class="p" data-group-id="group-7">]</samp><samp class="p" data-group-id="group-3">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">drop</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp> <samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp> <samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp

class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">drop</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">-
</samp><samp class="mi">1</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">drop_every</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp

class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">drop_while</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&
</samp><samp class="p" data-group-id="group-2">(</samp><samp
class="ni">&1</samp><samp class="o">.</samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp
class="p" data-group-id="group-2">)</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">

</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">take</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">take</samp><samp class="p" data-
group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">-
</samp><samp class="mi">1</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp

class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">take_every</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p" data-group-id="group-1">)</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">take_while</samp><samp class="p"
data-group-id="group-1">(</samp><samp class="n">cart</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="o">&

</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~ </samp><samp class="w"> </samp><samp class="s">"e"</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp> <samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">random</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-2">}</samp> </code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">take_random</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">shuffle</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-3">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-3">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp> <samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">chunk_by</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="o">&</samp><samp class="nc">String</samp><samp class="o">.</samp><samp class="n">length</samp><samp class="p" data-group-id="group-2">(</samp><samp class="ni">&1</samp><samp class="o">.</samp><samp class="n">fruit</samp><samp class="p" data-group-id="group-2">)</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp

class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="p" data-group-id="group-4">]</samp><samp class="p">,
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-7">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-8">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-6">]</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-3">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="nc">Enum</samp><samp
class="o">.</samp><samp class="n">chunk_every</samp><samp
class="p" data-group-id="group-1">(</samp><samp
class="n">cart</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">2</samp><samp class="p" data-group-
id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"apple"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-4">}</samp> <samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-5">}</samp> <samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp class="p" data-group-id="group-7">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-7">}</samp> <samp class="p" data-group-id="group-6">]</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp

class="o">.</samp><samp class="n">chunk_every</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">cart</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-2">[</samp><samp class="ss">:elements</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">:to_complete</samp><samp class="p" data-group-id="group-2">]</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-4">[</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-5">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-6">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-6">}</samp>
<samp class="w">

  </samp><samp class="p" data-group-id="group-4">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-8">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="p">,</samp><samp class="w">

</samp><samp class="ss">:elements</samp><samp class="w">

</samp><samp class="p" data-group-id="group-7">]</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">chunk_every</samp><samp class="p" data-group-id="group-9">(</samp><samp class="n">cart</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="mi">2</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">:discard</samp><samp class="p" data-group-id="group-9">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-10">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-11">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-12">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>

<samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-12">}</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-13">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-13">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-11">]</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-14">[</samp><samp class="w">

</samp><samp class="p" data-group-id="group-15">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-15">}</samp><samp class="p">,</samp><samp class="w">

</samp><samp class="p" data-group-id="group-16">%{</samp><samp class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">count</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-16">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-14">]</samp><samp class="w">

</samp><samp class="p" data-group-id="group-10">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">fruits</samp><samp class="w"> </samp><samp class="o">=</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">[</samp><samp class="s">"apple"</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="s">"banana"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="s">"orange"</samp><samp class="p" data-group-id="group-1">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="n">counts</samp><samp class="w"> </samp><samp class="o">= </samp><samp class="w"> </samp><samp class="p" data-group-id="group-2">[</samp><samp class="mi">3</samp><samp class="p">, </samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">6</samp><samp class="p" data-group-id="group-2">]</samp> <samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Enum</samp><samp class="o">.</samp><samp class="n">zip</samp><samp class="p" data-group-id="group-3">(</samp> <samp class="n">fruits</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">counts</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">[</samp><samp class="p" data-group-id="group-5">{</samp><samp class="s">"apple" </samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="p" data-group-id="group-5">}</samp> <samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-6">{</samp><samp class="s">"banana"</samp> <samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-6">}</samp> <samp class="p">,</samp><samp class="w"> </samp><samp class="p" data-group-id="group-7">{</samp><samp class="s">"orange"</samp> <samp class="p">,</samp><samp class="w"> </samp><samp

class="mi">6</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="p" data-group-id="group-4">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="n">fruits</samp><samp
class="w"> </samp><samp class="o">=</samp><samp class="w">
</samp><samp class="p" data-group-id="group-1">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p" data-group-id="group-1">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="n">counts</samp><samp class="w"> </samp><samp class="o">=
</samp><samp class="w"> </samp><samp class="p" data-group-
id="group-2">[</samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-2">]</samp>
<samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">zip_with</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">fruits</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="n">counts</samp><samp
class="p">,</samp><samp class="w"> </samp><samp class="k" data-
group-id="group-4">fn</samp><samp class="w"> </samp><samp
class="n">fruit</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">count</samp><samp class="w"> </samp>
<samp class="o">-></samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-5">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">fruit</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="ss">count</samp><samp class="p">:

</samp><samp class="w"> </samp><samp class="n">count</samp>
<samp class="p" data-group-id="group-5">}</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-4">end</samp><samp class="p" data-group-
id="group-3">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">[</samp><samp
class="w">

  </samp><samp class="p" data-group-id="group-7">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"apple"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">3</samp><samp class="p" data-group-id="group-7">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-8">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">1</samp><samp class="p" data-group-id="group-8">}</samp>
<samp class="p">,</samp><samp class="w">

  </samp><samp class="p" data-group-id="group-9">%{</samp><samp
class="ss">fruit</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="ss">count</samp><samp
class="p">:</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-9">}</samp>
<samp class="w">

</samp><samp class="p" data-group-id="group-6">]</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="n">fruits</samp><samp

class="w"> </samp><samp class="o">=</samp><samp class="w">
</samp><samp class="p" data-group-id="group-1">[</samp><samp
class="s">"apple"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"banana"</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="s">"orange"</samp><samp
class="p" data-group-id="group-1">]</samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="n">counts</samp><samp class="w"> </samp><samp class="o">=
</samp><samp class="w"> </samp><samp class="p" data-group-
id="group-2">[</samp><samp class="mi">3</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="mi">1</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">6</samp><samp class="p" data-group-id="group-2">]</samp>
<samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp
class="nc">Enum</samp><samp class="o">.</samp><samp
class="n">zip_reduce</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="n">fruits</samp><samp class="p">,</samp>
<samp class="w"> </samp><samp class="n">counts</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="mi">0</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="k" data-group-id="group-4">fn</samp><samp
class="w"> </samp><samp class="n">fruit</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="n">count</samp>
<samp class="p">,</samp><samp class="w"> </samp><samp
class="n">acc</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">price</samp><samp class="w"> </samp><samp
class="o">=</samp><samp class="w"> </samp><samp
class="k">if</samp><samp class="w"> </samp><samp
class="n">fruit</samp><samp class="w"> </samp><samp class="o">=~
</samp><samp class="w"> </samp><samp class="s">"e"</samp><samp
class="p">,</samp><samp class="w"> </samp><samp

class="ss">do</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">count</samp><samp class="w"> </samp>
<samp class="o">*</samp><samp class="w"> </samp><samp
class="mi">2</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">else</samp><samp class="p">:</samp><samp
class="w"> </samp><samp class="n">count</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">acc</samp><samp class="w"> </samp><samp
class="o">+</samp><samp class="w"> </samp><samp
class="n">price</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k"
data-group-id="group-4">end</samp><samp class="p" data-group-
id="group-3">)</samp><samp class="w">

</samp><samp class="mi">19</samp></code>

<code class="makeup elixir" translate="no"><samp class="gp
unselectable">iex> </samp><samp class="n">cart</samp><samp
class="w"> </samp><samp class="o">|></samp><samp class="w">
</samp><samp class="nc">Enum</samp><samp class="o">.</samp>
<samp class="n">map</samp><samp class="p" data-group-id="group-1">
(</samp><samp class="o">&</samp><samp class="p" data-group-
id="group-2">{</samp><samp class="ni">&1</samp><samp class="o">.
</samp><samp class="n">fruit</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="ni">&1</samp><samp class="o">.
</samp><samp class="n">count</samp><samp class="p" data-group-
id="group-2">}</samp><samp class="p" data-group-id="group-1">)
</samp><samp class="w"> </samp><samp class="o">|></samp><samp
class="w"> </samp><samp class="nc">Enum</samp><samp class="o">.
</samp><samp class="n">unzip</samp><samp class="p" data-group-
id="group-3">(</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">{</samp><samp
class="p" data-group-id="group-5">[</samp><samp class="s">"apple"
</samp><samp class="p">,</samp><samp class="w"> </samp><samp

class="s">"banana"</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="s">"orange"</samp><samp class="p" data-group-
id="group-5">]</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="p" data-group-id="group-6">[</samp><samp
class="mi">3</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="mi">1</samp><samp class="p">,</samp><samp
class="w"> </samp><samp class="mi">6</samp><samp class="p" data-
group-id="group-6">]</samp><samp class="p" data-group-id="group-4">}
</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">type1</samp><samp class="w"> </samp><samp class="ow">or</samp><samp class="w"> </samp><samp class="n">type2</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">type1</samp><samp class="w"> </samp><samp class="ow">and</samp><samp class="w"> </samp><samp class="n">type2</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">type1</samp><samp class="w"> </samp><samp class="ow">and</samp><samp class="w"> </samp><samp class="ow">not</samp><samp class="w"> </samp><samp class="n">type2</samp></code>

<code class="makeup elixir" translate="no"><samp class="ow">not</samp><samp class="w"> </samp><samp class="n">type</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">bitstring</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="n">binary</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="n">empty_list</samp><samp class="p" data-group-id="group-3">(</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w">

</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-4">(</samp><samp class="p" data-group-id="group-4">)</samp><samp class="w">

</samp><samp class="n">float</samp><samp class="p" data-group-id="group-5">(</samp><samp class="p" data-group-id="group-5">)</samp><samp class="w">

</samp><samp class="n">pid</samp><samp class="p" data-group-id="group-6">(</samp><samp class="p" data-group-id="group-6">)</samp><samp class="w">

</samp><samp class="n">port</samp><samp class="p" data-group-id="group-7">(</samp><samp class="p" data-group-id="group-7">)</samp><samp class="w">

</samp><samp class="n">reference</samp><samp class="p" data-group-id="group-8">(</samp><samp class="p" data-group-id="group-8">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">atom</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="ss">:ok</samp><samp class="w">

</samp><samp class="ss">:error</samp><samp class="w">

</samp><samp class="nc">SomeModule</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">function</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="p" data-group-id="group-1">(</samp><samp class="o">-></samp><samp class="w">
</samp><samp class="ss">:ok</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-2">(</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-3">

(</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="n">boolean</samp><samp class="p" data-group-id="group-4">(</samp><samp class="p" data-group-id="group-4">) </samp><samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">(</samp><samp class="n">binary</samp><samp class="p" data-group-id="group-6"> (</samp><samp class="p" data-group-id="group-6">)</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-7"> (</samp><samp class="p" data-group-id="group-7">)</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-8">(</samp><samp class="p" data-group-id="group-8">) </samp><samp class="p" data-group-id="group-5">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="p" data-group-id="group-1">(</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w"> </samp><samp class="o">-></samp> <samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-3">(</samp><samp class="p" data-group-id="group-3">)</samp><samp class="p" data-group-id="group-1">) </samp><samp class="w"> </samp><samp class="ow">and</samp><samp class="w"> </samp><samp class="p" data-group-id="group-4">(</samp> <samp class="n">binary</samp><samp class="p" data-group-id="group-5">(</samp><samp class="p" data-group-id="group-5">)</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="n">atom</samp><samp class="p" data-group-id="group-6">(</samp><samp class="p" data-group-id="group-6">) </samp><samp class="p" data-group-id="group-4">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">map</samp><samp class="p" data-group-id="group-1"> (</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">empty_map</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="c1"># Only has the keys name and age</samp><samp class="w">

</samp><samp class="p" data-group-id="group-1">%{</samp><samp class="ss">name</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-id="group-2">(</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">age</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-3">(</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w">

</samp><samp class="c1"># Has the name key and age is optional</samp><samp class="w">

</samp><samp class="p" data-group-id="group-4">%{</samp><samp class="ss">name</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-id="group-5">(</samp><samp class="p" data-group-id="group-5">)
</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">age</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">if_set</samp><samp class="p" data-group-id="group-6">(</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-7">(</samp><samp class="p" data-group-id="group-7">)</samp><samp class="p" data-group-id="group-6">)</samp><samp class="p" data-group-id="group-4">}</samp><samp class="w">

</samp><samp class="c1"># Has the keys name and age and may have other keys (open map)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-8">%{</samp><samp class="n">...</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">name</samp><samp class="p">:</samp>
<samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-9">(</samp><samp class="p" data-group-id="group-9">)</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">age</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="n">integer</samp><samp class="p" data-group-id="group-10">(</samp><samp class="p" data-group-id="group-10">)</samp><samp class="p" data-group-id="group-8">}
</samp><samp class="w">

</samp><samp class="c1"># Has the key name, may have other keys, but age is not set</samp><samp class="w">

</samp><samp class="p" data-group-id="group-11">%{</samp><samp class="n">...</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="ss">name</samp><samp class="p">:</samp>
<samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-12">(</samp><samp class="p" data-group-id="group-12">)</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">age</samp><samp class="p">:
</samp><samp class="w"> </samp><samp class="n">not_set</samp>
<samp class="p" data-group-id="group-13">(</samp><samp class="p" data-group-id="group-13">)</samp><samp class="p" data-group-id="group-11">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="c1"># Has atom and binary keys</samp><samp class="w">

</samp><samp class="p" data-group-id="group-1">%{</samp><samp class="n">atom</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="p" data-group-id="group-2">)</samp><samp

class="w"> </samp><samp class="p">=></samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-
id="group-3">(</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="n">binary</samp><samp class="p" data-group-id="group-4">
(</samp><samp class="p" data-group-id="group-4">)</samp><samp
class="w"> </samp><samp class="p">=></samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-
id="group-5">(</samp><samp class="p" data-group-id="group-5">)
</samp><samp class="p" data-group-id="group-1">}</samp><samp
class="w">

</samp><samp class="c1"># Has atom and binary keys and may have other keys (open map)</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">%{</samp><samp
class="n">...</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">atom</samp><samp class="p" data-group-
id="group-7">(</samp><samp class="p" data-group-id="group-7">)
</samp><samp class="w"> </samp><samp class="p">=></samp><samp
class="w"> </samp><samp class="n">binary</samp><samp class="p"
data-group-id="group-8">(</samp><samp class="p" data-group-id="group-
8">)</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="n">binary</samp><samp class="p" data-group-id="group-9">
(</samp><samp class="p" data-group-id="group-9">)</samp><samp
class="w"> </samp><samp class="p">=></samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-
id="group-10">(</samp><samp class="p" data-group-id="group-10">)
</samp><samp class="p" data-group-id="group-6">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="c1"># Has atom keys with binary values but a `:root` key of type integer</samp><samp
class="w">

</samp><samp class="p" data-group-id="group-1">%{</samp><samp
class="n">atom</samp><samp class="p" data-group-id="group-2">

(</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-id="group-3">(</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">root</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-4">(</samp><samp class="p" data-group-id="group-4">)
</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w">

</samp><samp class="c1"># Has atom keys with binary values but a `:root` key of type integer, and may have other keys</samp><samp class="w">

</samp><samp class="p" data-group-id="group-5">%{</samp><samp class="n">...</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">atom</samp><samp class="p" data-group-id="group-6">(</samp><samp class="p" data-group-id="group-6">)
</samp><samp class="w"> </samp><samp class="p">=></samp><samp class="w"> </samp><samp class="n">binary</samp><samp class="p" data-group-id="group-7">(</samp><samp class="p" data-group-id="group-7">)</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">root</samp><samp class="p">:</samp><samp class="w">
</samp><samp class="n">integer</samp><samp class="p" data-group-id="group-8">(</samp><samp class="p" data-group-id="group-8">)
</samp><samp class="p" data-group-id="group-5">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">non_empty_list</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">elem_type</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">non_empty_list</samp><samp class="p" data-group-id="group-1">(</samp><samp class="n">elem_type</samp><samp class="p">,

</samp><samp class="w"> </samp><samp class="n">tail_type</samp>
<samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp
class="n">tuple</samp><samp class="p" data-group-id="group-1">
(</samp><samp class="p" data-group-id="group-1">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="p" data-group-
id="group-1">{</samp><samp class="ss">:ok</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="n">binary</samp>
<samp class="p" data-group-id="group-2">(</samp><samp class="p" data-
group-id="group-2">)</samp><samp class="p" data-group-id="group-1">}
</samp><samp class="w">

</samp><samp class="p" data-group-id="group-3">{</samp><samp
class="ss">:error</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="n">binary</samp><samp class="p" data-group-
id="group-4">(</samp><samp class="p" data-group-id="group-4">)
</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="n">term</samp><samp class="p" data-group-id="group-5">
(</samp><samp class="p" data-group-id="group-5">)</samp><samp
class="p" data-group-id="group-3">}</samp><samp class="w">

</samp><samp class="p" data-group-id="group-6">{</samp><samp
class="n">pid</samp><samp class="p" data-group-id="group-7">
(</samp><samp class="p" data-group-id="group-7">)</samp><samp
class="p">,</samp><samp class="w"> </samp><samp
class="n">reference</samp><samp class="p" data-group-id="group-8">
(</samp><samp class="p" data-group-id="group-8">)</samp><samp
class="p" data-group-id="group-6">}</samp></code>

<code class="makeup elixir" translate="no"><samp class="p" data-group-
id="group-1">{</samp><samp class="n">binary</samp><samp class="p"
data-group-id="group-2">(</samp><samp class="p" data-group-id="group-
2">)</samp><samp class="p">,</samp><samp class="w"> </samp><samp
class="n">binary</samp><samp class="p" data-group-id="group-3">
(</samp><samp class="p" data-group-id="group-3">)</samp><samp

class="p">,</samp><samp class="w"> </samp><samp class="n">...
</samp><samp class="p" data-group-id="group-1">}</samp></code>

<code class="makeup elixir" translate="no"><samp
class="n">boolean</samp><samp class="p" data-group-id="group-1">
(</samp><samp class="p" data-group-id="group-1">)</samp><samp
class="w"> </samp><samp class="o">=</samp><samp class="w">
</samp><samp class="no">true</samp><samp class="w"> </samp><samp
class="ow">or</samp><samp class="w"> </samp><samp
class="no">false</samp><samp class="w">

</samp><samp class="n">number</samp><samp class="p" data-group-
id="group-2">(</samp><samp class="p" data-group-id="group-2">)
</samp><samp class="w"> </samp><samp class="o">=</samp><samp
class="w"> </samp><samp class="n">integer</samp><samp class="p"
data-group-id="group-3">(</samp><samp class="p" data-group-id="group-
3">)</samp><samp class="w"> </samp><samp class="ow">or</samp>
<samp class="w"> </samp><samp class="n">float</samp><samp
class="p" data-group-id="group-4">(</samp><samp class="p" data-group-
id="group-4">)</samp></code>

<code class="makeup elixir" translate="no"><samp class="n">list</samp>
<samp class="p" data-group-id="group-1">(</samp><samp class="p" data-
group-id="group-1">)</samp><samp class="w"> </samp><samp
class="o">=</samp><samp class="w"> </samp><samp
class="n">empty_list</samp><samp class="p" data-group-id="group-2">
(</samp><samp class="p" data-group-id="group-2">)</samp><samp
class="w"> </samp><samp class="ow">or</samp><samp class="w">
</samp><samp class="n">non_empty_list</samp><samp class="p" data-
group-id="group-3">(</samp><samp class="n">term</samp><samp
class="p" data-group-id="group-4">(</samp><samp class="p" data-group-
id="group-4">)</samp><samp class="p" data-group-id="group-3">)
</samp><samp class="w">

</samp><samp class="n">list</samp><samp class="p" data-group-
id="group-5">(</samp><samp class="n">a</samp><samp class="p" data-
group-id="group-5">)</samp><samp class="w"> </samp><samp
class="o">=</samp><samp class="w"> </samp><samp

class="n">empty_list</samp><samp class="p" data-group-id="group-6">(</samp><samp class="p" data-group-id="group-6">)</samp><samp class="w"> </samp><samp class="ow">or</samp><samp class="w">
</samp><samp class="n">non_empty_list</samp><samp class="p" data-group-id="group-7">(</samp><samp class="n">a</samp><samp class="p" data-group-id="group-7">)</samp><samp class="w">

</samp><samp class="n">list</samp><samp class="p" data-group-id="group-8">(</samp><samp class="n">a</samp><samp class="p">,
</samp><samp class="w"> </samp><samp class="n">b</samp><samp class="p" data-group-id="group-8">)</samp><samp class="w"> </samp>
<samp class="o">=</samp><samp class="w"> </samp><samp class="n">empty_list</samp><samp class="p" data-group-id="group-9">(</samp><samp class="p" data-group-id="group-9">)</samp><samp class="w"> </samp><samp class="ow">or</samp><samp class="w">
</samp><samp class="n">non_empty_list</samp><samp class="p" data-group-id="group-10">(</samp><samp class="n">a</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="n">b</samp><samp class="p" data-group-id="group-10">)</samp>
</code>

Introduction to Mix

In this guide, we will build a complete Elixir application, with its own supervision tree, configuration, tests, and more.

The requirements for this guide are (see `elixir -v`):

- Elixir 1.18.0 onwards
- Erlang/OTP 27 onwards

The application works as a distributed key-value store. We are going to organize key-value pairs into buckets and distribute those buckets across multiple nodes. We will also build a simple client that allows us to connect to any of those nodes and send requests such as:

```
CREATE shopping
OK

PUT shopping milk 1
OK

PUT shopping eggs 3
OK

GET shopping milk
1
OK

DELETE shopping eggs
OK
```

In order to build our key-value application, we are going to use three main tools:

- ***OTP*** *(Open Telecom Platform)* is a set of libraries that ships with Erlang. Erlang developers use OTP to build robust, fault-tolerant applications. In this chapter we will explore how many aspects from OTP integrate with Elixir, including supervision trees, event managers and more;

- ***Mix*** is a build tool that ships with Elixir that provides tasks for creating, compiling, testing your application, managing its dependencies and much more;

- ***ExUnit*** is a unit-test based framework that ships with Elixir.

In this chapter, we will create our first project using Mix and explore different features in OTP, Mix, and ExUnit as we go.

> **Source code**
>
> The final code for the application built in this guide is in this repository and can be used as a reference.

> **Is this guide required reading?**
>
> This guide is not required reading in your Elixir journey. We'll explain.
>
> As an Elixir developer, you will most likely use one of the many existing frameworks when writing your Elixir code. Phoenix covers web applications, Ecto communicates with databases, you can craft embedded software with Nerves, Nx powers machine learning and AI projects, Membrane assembles audio/video processing pipelines, Broadway handles data ingestion and processing, and many more. These frameworks handle the lower level details of concurrency, distribution, and fault-tolerance, so you, as a user, can focus on your own needs and demands.
>
> On the other hand, if you want to learn the foundations these frameworks are built upon, and the abstractions that power the Elixir ecosystem, this guide will give you a tour through several important concepts.

Our first project

When you install Elixir, besides getting the `elixir`, `elixirc`, and `iex` executables, you also get an executable Elixir script named `mix`.

Let's create our first project by invoking `mix new` from the command line. We'll pass the project path as the argument (`kv`, in this case). By default, the application name and module name will be retrieved from the path. So we tell Mix that our main module should be the all-uppercase `KV`, instead of the default, which would have been `Kv`:

```
$ mix new kv --module KV
```

Mix will create a directory named `kv` with a few files in it:

```
* creating README.md
* creating .formatter.exs
* creating .gitignore
* creating mix.exs
* creating lib
* creating lib/kv.ex
* creating test
* creating test/test_helper.exs
* creating test/kv_test.exs
```

Let's take a brief look at those generated files.

> **Executables in the `PATH`**
>
> Mix is an Elixir executable. This means that in order to run `mix`, you need to have both `mix` and `elixir` executables in your `PATH`. That's what happens when you install Elixir.

Project compilation

A file named `mix.exs` was generated inside our new project folder (`kv`) and its main responsibility is to configure our project. Let's take a look at it:

```
defmodule KV.MixProject do
  use Mix.Project

  def project do
    [
      app: :kv,
      version: "0.1.0",
      elixir: "~> 1.11",
      start_permanent: Mix.env() == :prod,
      deps: deps()
    ]
  end

  # Run "mix help compile.app" to learn about applications
  def application do
    [
      extra_applications: [:logger]
    ]
  end

  # Run "mix help deps" to learn about dependencies
  defp deps do
    [
      # {:dep_from_hexpm, "~> 0.3.0"},
      # {:dep_from_git, git: "https://github.com/elixir-
lang/my_dep.git", tag: "0.1.0"},
    ]
  end
end
```

Our `mix.exs` defines two public functions: `project`, which returns project configuration like the project name and version, and `application`, which is used to generate an application file.

There is also a private function named `deps`, which is invoked from the `project` function, that defines our project dependencies. Defining `deps` as a separate function is not required, but it helps keep the project configuration tidy.

Mix also generates a file at `lib/kv.ex` with a module containing exactly one function, called `hello`:

```
defmodule KV do
  @moduledoc """
  Documentation for KV.
  """

  @doc """
  Hello world.

  ## Examples

      iex> KV.hello()
      :world

  """
  def hello do
    :world
  end
end
```

This structure is enough to compile our project:

```
$ cd kv
$ mix compile
```

Will output:

```
Compiling 1 file (.ex)
Generated kv app
```

The `lib/kv.ex` file was compiled and an application manifest named `kv.app` was generated. All compilation artifacts are placed inside the `_build` directory using the options defined in the `mix.exs` file.

Once the project is compiled, you can start a `iex` session inside the project by running the command below. The `-S mix` is necessary to load the project in the interactive shell:

```
$ iex -S mix
```

We are going to work on this `kv` project, making modifications and trying out the latest changes from a `iex` session. While you may start a new session whenever there are changes to the project source code, you can also recompile the project from within `iex` with the `recompile` helper, like this:

```
iex> recompile()
Compiling 1 file (.ex)
:ok
iex> recompile()
:noop
```

If anything had to be compiled, you see some informative text, and get the `:ok` atom back, otherwise the function is silent, and returns `:noop`.

Running tests

Mix also generated the appropriate structure for running our project tests. Mix projects usually follow the convention of having a `<filename>_test.exs` file in the `test` directory for each file in the `lib` directory. For this reason, we can already find a `test/kv_test.exs` corresponding to our `lib/kv.ex` file. It doesn't do much at this point:

```
defmodule KVTest do
  use ExUnit.Case
  doctest KV

  test "greets the world" do
    assert KV.hello() == :world
  end
end
```

It is important to note a couple of things:

1. the test file is an Elixir script file (`.exs`). This is convenient because we don't need to compile test files before running them;

2. we define a test module named `KVTest`, in which we `use ExUnit.Case` to inject the testing API;

3. we use one of the imported macros, `ExUnit.DocTest.doctest/1`, to indicate that the `KV` module contains doctests (we will discuss those in a later chapter);

4. we use the `ExUnit.Case.test/2` macro to define a simple test;

Mix also generated a file named `test/test_helper.exs` which is responsible for setting up the test framework:

```
ExUnit.start()
```

This file will be required by Mix every time before we run our tests. We can run tests with:

```
$ mix test
Compiled lib/kv.ex
Generated kv app
Running ExUnit with seed: 540224, max_cases: 16
..

Finished in 0.04 seconds
1 doctest, 1 test, 0 failures
```

Notice that by running `mix test`, Mix has compiled the source files and generated the application manifest once again. This happens because Mix supports multiple environments, which we will discuss later in this chapter.

Furthermore, you can see that ExUnit prints a dot for each successful test and automatically randomizes tests too. Let's make the test fail on purpose and see what happens.

Change the assertion in `test/kv_test.exs` to the following:

```
assert KV.hello() == :oops
```

Now run `mix test` again (notice this time there will be no compilation):

```
  1) test greets the world (KVTest)
     test/kv_test.exs:5
     Assertion with == failed
     code:  assert KV.hello() == :oops
     left:  :world
     right: :oops
     stacktrace:
       test/kv_test.exs:6: (test)

.

Finished in 0.05 seconds
1 doctest, 1 test, 1 failure
```

For each failure, ExUnit prints a detailed report, containing the test name with the test case, the code that failed and the values for the left side and right side (RHS) of the `==` operator.

In the second line of the failure, right below the test name, there is the location where the test was defined. If you copy the test location in full, including the file and line number, and append it to `mix test`, Mix will load and run just that particular test:

```
$ mix test test/kv_test.exs:5
```

This shortcut will be extremely useful as we build our project, allowing us to quickly iterate by running a single test.

Finally, the stacktrace relates to the failure itself, giving information about the test and often the place the failure was generated from within the source files.

Automatic code formatting

One of the files generated by `mix new` is the `.formatter.exs`. Elixir ships with a code formatter that is capable of automatically formatting our codebase according to a consistent style. The formatter is triggered with the `mix format` task. The generated `.formatter.exs` file configures which files should be formatted when `mix format` runs.

To give the formatter a try, change a file in the `lib` or `test` directories to include extra spaces or extra newlines, such as `def hello do`, and then run `mix format`.

Most editors provide built-in integration with the formatter, allowing a file to be formatted on save or via a chosen keybinding. If you are learning Elixir, editor integration gives you useful and quick feedback when learning the Elixir syntax.

For companies and teams, we recommend developers to run `mix format --check-formatted` on their continuous integration servers, ensuring all current and future code follows the standard.

You can learn more about the code formatter by checking the format task documentation or by reading the release announcement for Elixir v1.6, the first version to include the formatter.

Environments

Mix provides the concept of "environments". They allow a developer to customize compilation and other options for specific scenarios. By default, Mix understands three environments:

- `:dev` — the one in which Mix tasks (like `compile`) run by default
- `:test` — used by `mix test`
- `:prod` — the one you will use to run your project in production

The environment applies only to the current project. As we will see in future chapters, any dependency you add to your project will by default run

in the `:prod` environment.

Customization per environment can be done by accessing the `Mix.env/0` in your `mix.exs` file, which returns the current environment as an atom. That's what we have used in the `:start_permanent` options:

```
def project do
  [
    ...,
    start_permanent: Mix.env() == :prod,
    ...
  ]
end
```

When true, the `:start_permanent` option starts your application in permanent mode, which means the Erlang VM will crash if your application's supervision tree shuts down. Notice we don't want this behavior in dev and test because it is useful to keep the VM instance running in those environments for troubleshooting purposes.

Mix will default to the `:dev` environment, except for the `test` task that will default to the `:test` environment. The environment can be changed via the `MIX_ENV` environment variable:

```
$ MIX_ENV=prod mix compile
```

Or on Windows:

```
> set "MIX_ENV=prod" && mix compile
```

> **Mix in production**
>
> Mix is a **build tool** and, as such, it is not expected to be available in production. Therefore, it is recommended to access `Mix.env/0` only in configuration files and inside `mix.exs`, never in your application code (`lib`).

Exploring

There is much more to Mix, and we will continue to explore it as we build our project. A general overview is available on the Mix documentation and you can always invoke the help task to list all available tasks:

```
$ mix help
$ mix help compile
```

Now let's move forward and add the first modules and functions to our application.

Simple state with agents

In this chapter, we will learn how to keep and share state between multiple entities. If you have previous programming experience, you may think of globally shared variables, but the model we will learn here is quite different. The next chapters will generalize the concepts introduced here.

If you have skipped the *Getting Started* guide or read it long ago, be sure to re-read the Processes chapter. We will use it as a starting point.

The trouble with (mutable) state

Elixir is an immutable language where nothing is shared by default. If we want to share information, this is typically done by sending messages between processes.

When it comes to processes though, we rarely hand-roll our own, instead we use the abstractions available in Elixir and OTP:

- `Agent` — Simple wrappers around state.
- `GenServer` — "Generic servers" (processes) that encapsulate state, provide sync and async calls, support code reloading, and more.
- `Task` — Asynchronous units of computation that allow spawning a process and potentially retrieving its result at a later time.

Here, we will use agents, and create a module named `KV.Bucket`, responsible for storing our key-value entries in a way that allows them to be read and modified by other processes.

Agents 101

`Agent`s are simple wrappers around state. If all you want from a process is to keep state, agents are a great fit. Let's start a `iex` session inside the project with:

```
$ iex -S mix
```

And play a bit with agents:

```
iex> {:ok, agent} = Agent.start_link(fn -> [] end)
{:ok, #PID<0.57.0>}
iex> Agent.update(agent, fn list -> ["eggs" | list] end)
:ok
iex> Agent.get(agent, fn list -> list end)
["eggs"]
iex> Agent.stop(agent)
:ok
```

We started an agent with an initial state of an empty list. The `start_link/1` function returned the `:ok` tuple with a process identifier (PID) of the agent. We will use this PID for all further interactions. We then updated the agent's state, adding our new item to the head of the list. The second argument of `Agent.update/3` is a function that takes the agent's current state as input and returns its desired new state. Finally, we retrieved the whole list. The second argument of `Agent.get/3` is a function that takes the state as input and returns the value that `Agent.get/3` itself will return. Once we are done with the agent, we can call `Agent.stop/3` to terminate the agent process.

The `Agent.update/3` function accepts as a second argument any function that receives one argument and returns a value:

```
iex> {:ok, agent} = Agent.start_link(fn -> [] end)
{:ok, #PID<0.338.0>}
iex> Agent.update(agent, fn _list -> 123 end)
:ok
iex> Agent.update(agent, fn content -> %{a: content} end)
:ok
iex> Agent.update(agent, fn content -> [12 | [content]] end)
:ok
iex> Agent.update(agent, fn list -> [:nop | list] end)
:ok
iex> Agent.get(agent, fn content -> content end)
[:nop, 12, %{a: 123}]
```

As you can see, we can modify the agent state in any way we want. Therefore, we most likely don't want to access the Agent API throughout many different places in our code. Instead, we want to encapsulate all Agent-related functionality in a single module, which we will call `KV.Bucket`. Before we implement it, let's write some tests which will outline the API exposed by our module.

Create a file at `test/kv/bucket_test.exs` (remember the `.exs` extension) with the following:

```
defmodule KV.BucketTest do
  use ExUnit.Case, async: true

  test "stores values by key" do
    {:ok, bucket} = KV.Bucket.start_link([])
    assert KV.Bucket.get(bucket, "milk") == nil

    KV.Bucket.put(bucket, "milk", 3)
    assert KV.Bucket.get(bucket, "milk") == 3
```

```
  end
end
```

`use ExUnit.Case` is responsible for setting up our module for testing and imports many test-related functionality, such as the `test/2` macro.

Our first test starts a new `KV.Bucket` by calling the `start_link/1` and passing an empty list of options. Then we perform some `get/2` and `put/3` operations on it, asserting the result.

Also note the `async: true` option passed to `ExUnit.Case`. This option makes the test case run in parallel with other `:async` test cases by using multiple cores in our machine. This is extremely useful to speed up our test suite. However, `:async` must *only* be set if the test case does not rely on or change any global values. For example, if the test requires writing to the file system or access a database, keep it synchronous (omit the `:async` option) to avoid race conditions between tests.

Async or not, our new test should obviously fail, as none of the functionality is implemented in the module being tested:

```
1) test stores values by key (KV.BucketTest)
   test/kv/bucket_test.exs:4
   ** (UndefinedFunctionError) function
KV.Bucket.start_link/1 is undefined (module KV.Bucket is not
available)
```

In order to fix the failing test, let's create a file at `lib/kv/bucket.ex` with the contents below. Feel free to give a try at implementing the `KV.Bucket` module yourself using agents before peeking at the implementation below.

```elixir
defmodule KV.Bucket do
  use Agent

  @doc """
  Starts a new bucket.

  All options are forwarded to `Agent.start_link/2`.
  """
  def start_link(opts) do
    Agent.start_link(fn -> %{} end, opts)
  end

  @doc """
  Gets a value from the `bucket` by `key`.
  """
  def get(bucket, key) do
    Agent.get(bucket, &Map.get(&1, key))
  end

  @doc """
  Puts the `value` for the given `key` in the `bucket`.
  """
  def put(bucket, key, value) do
    Agent.update(bucket, &Map.put(&1, key, value))
  end
end
```

The first step in our implementation is to call `use Agent`. This is a pattern we will see throughout the guides and understand in depth in the next chapter.

Then we define a `start_link/1` function, which will effectively start the agent. It is a convention to define a `start_link/1` function that always accepts a list of options. We then call `Agent.start_link/2` passing an

anonymous function that returns the Agent's initial state and the same list of options we received.

We are keeping a map inside the agent to store our keys and values. Getting and putting values on the map is done with the Agent API and the capture operator `&`, introduced in the Getting Started guide. The agent passes its state to the anonymous function via the `&1` argument when `Agent.get/2` and `Agent.update/2` are called.

Now that the `KV.Bucket` module has been defined, our test should pass! You can try it yourself by running: `mix test`.

Naming processes

When starting `KV.Bucket`, we pass a list of options which we forward to `Agent.start_link/2`. One of the options accepted by `Agent.start_link/2` is a name option which allows us to name a process, so we can interact with it using its name instead of its PID.

Let's write a test as an example. Back on `KV.BucketTest`, add this:

```
test "stores values by key on a named process" do
  {:ok, _} = KV.Bucket.start_link(name: :shopping_list)
  assert KV.Bucket.get(:shopping_list, "milk") == nil

  KV.Bucket.put(:shopping_list, "milk", 3)
  assert KV.Bucket.get(:shopping_list, "milk") == 3
end
```

However, keep in mind that names are shared in the current node. If two tests attempt to create two processes named `:shopping_list` at the same time, one would succeed and the other would fail. For this reason, it is a

common practice in Elixir to name processes started during tests after the test itself, like this:

```
  test "stores values by key on a named process", config do
    {:ok, _} = KV.Bucket.start_link(name: config.test)
    assert KV.Bucket.get(config.test, "milk") == nil

    KV.Bucket.put(config.test, "milk", 3)
    assert KV.Bucket.get(config.test, "milk") == 3
  end
```

The `config` argument, passed after the test name, is the *test context* and it includes configuration and metadata about the current test, which is useful in scenarios like these.

Other agent actions

Besides getting a value and updating the agent state, agents allow us to get a value and update the agent state in one function call via `Agent.get_and_update/2`. Let's implement a `KV.Bucket.delete/2` function that deletes a key from the bucket, returning its current value:

```
@doc """
Deletes `key` from `bucket`.

Returns the current value of `key`, if `key` exists.
"""
def delete(bucket, key) do
  Agent.get_and_update(bucket, &Map.pop(&1, key))
end
```

Now it is your turn to write a test for the functionality above! Also, be sure to explore the documentation for the `Agent` module to learn more about

them.

Client/server in agents

Before we move on to the next chapter, let's discuss the client/server dichotomy in agents. Let's expand the `delete/2` function we have just implemented:

```
def delete(bucket, key) do
  Agent.get_and_update(bucket, fn map ->
    Map.pop(map, key)
  end)
end
```

Everything that is inside the function we passed to the agent happens in the agent process. In this case, since the agent process is the one receiving and responding to our messages, we say the agent process is the server. Everything outside the function is happening in the client.

This distinction is important. If there are expensive actions to be done, you must consider if it will be better to perform these actions on the client or on the server. For example:

```
def delete(bucket, key) do
  Process.sleep(1000) # puts client to sleep
  Agent.get_and_update(bucket, fn map ->
    Process.sleep(1000) # puts server to sleep
    Map.pop(map, key)
  end)
end
```

When a long action is performed on the server, all other requests to that particular server will wait until the action is done, which may cause some

clients to timeout.

Some APIs, such as GenServers, make a clearer distinction between client and server, and we will explore them in future chapters. Next let's talk about naming things, applications, and supervisors.

Registries and supervision trees

In the previous chapter, we used agents to represent our buckets. In the introduction to mix, we specified we would like to name each bucket so we can do the following:

```
CREATE shopping
OK

PUT shopping milk 1
OK

GET shopping milk
1
OK
```

In the example session above we interacted with the "shopping" bucket by referencing its name. Therefore, an important feature in our key-value store is to give names to processes.

We have also learned in the previous chapter we can already name our buckets. For example:

```
iex> KV.Bucket.start_link(name: :shopping)
{:ok, #PID<0.43.0>}
iex> KV.Bucket.put(:shopping, "milk", 1)
:ok
iex> KV.Bucket.get(:shopping, "milk")
1
```

However, naming dynamic processes with atoms is a terrible idea! If we use atoms, we would need to convert the bucket name (often received from an external client) to atoms, and **we should never convert user input to atoms**. This is because atoms are not garbage collected. Once an atom is created, it is never reclaimed. Generating atoms from user input would mean the user can inject enough different names to exhaust our system memory!

In practice, it is more likely you will reach the Erlang VM limit for the maximum number of atoms before you run out of memory, which will bring your system down regardless.

Luckily, Elixir (and Erlang) comes with built-in abstractions for naming processes, called name registries, each with different trade-offs which we will explore throughout these guides.

Local, decentralized, and scalable registry

Elixir ships with a single-node process registry module aptly called `Registry`. Its main feature is that you can use any Elixir value to name a process, not only atoms. Let's take it for a spin in `iex`:

```
iex> Registry.start_link(name: KV, keys: :unique)
iex> name = {:via, Registry, {KV, "shopping"}}
iex> KV.Bucket.start_link(name: name)
{:ok, #PID<0.43.0>}
iex> KV.Bucket.put(name, "milk", 1)
:ok
iex> KV.Bucket.get(name, "milk")
1
```

As you can see, instead of passing an atom to the `:name` option, we pass a tuple of shape `{:via, registry_module, {registry_name, process_name}}`, and everything just worked. You could have used

anything as the `process_name`, even an integer or a map! That's because all of Elixir built-in behaviours, agents, supervisors, tasks, etc, are compatible with naming registries, as long as you pass them using the "via" tuple format.

Therefore, all we need to do to name our buckets is to start a `Registry`, using `Registry.start_link/1`. But you may be wondering, where exactly should we place that?

Understanding applications

Every Elixir project is an application. Elixir itself is defined in an application named `:elixir`. The `ExUnit.Case` module is part of the `:ex_unit` application. And so forth.

In fact, we have been working inside an application this entire time. Every time we changed a file and ran `mix compile`, we could see a `Generated kv app` message in the compilation output.

We can find the generated `.app` file at `_build/dev/lib/kv/ebin/kv.app`. Let's have a look at its contents:

```
{application,kv,
             [{applications,[kernel,stdlib,elixir,logger]},
              {description,"kv"},
              {modules,['Elixir.KV','Elixir.KV.Bucket']},
              {registered,[]},
              {vsn,"0.1.0"}]}.
```

This file contains Erlang terms (written using Erlang syntax). Even though we are not familiar with Erlang, it is easy to guess this file holds our application definition. It contains our application `version`, all the modules

defined by it, as well as a list of applications we depend on, like Erlang's `kernel`, `elixir` itself, and `logger`.

> The `logger` application ships as part of Elixir. We stated that our application needs it by specifying it in the `:extra_applications` list in `mix.exs`. See the official documentation for more information.

In a nutshell, an application consists of all the modules defined in the `.app` file, including the `.app` file itself. The application itself is located at the `_build/dev/lib/kv` folder and typically has only two directories: `ebin`, for Elixir artifacts, such as `.beam` and `.app` files, and `priv`, with any other artifact or asset you may need in your application.

Although Mix generates and maintains the `.app` file for us, we can customize its contents by adding new entries to the `application/0` function inside the `mix.exs` project file. We are going to do our first customization soon.

Starting applications

Each application in our system can be started and stopped. The rules for starting and stopping an application are also defined in the `.app` file. When we invoke `iex -S mix`, Mix compiles our application and then starts it.

Let's see this in practice. Start a console with `iex -S mix` and try:

```
iex> Application.start(:kv)
{:error, {:already_started, :kv}}
```

Oops, it's already started. Mix starts the current application and all of its dependencies automatically. This is also true for `mix test` and many other Mix commands.

We can, however, stop our `:kv` application, as well as the `:logger` application:

```
iex> Application.stop(:kv)
:ok
iex> Application.stop(:logger)
:ok
```

And let's try to start our application again:

```
iex> Application.start(:kv)
{:error, {:not_started, :logger}}
```

Now we get an error because an application that `:kv` depends on (`:logger` in this case) isn't started. We need to either start each application manually in the correct order or call `Application.ensure_all_started/1` as follows:

```
iex> Application.ensure_all_started(:kv)
{:ok, [:logger, :kv]}
```

In practice, our tools always start our applications for us, and you don't have to worry about the above, but it is good to know how it all works behind the scenes.

The application callback

Whenever we invoke `iex -S mix`, Mix automatically starts our application by calling `Application.start(:kv)`. But can we customize what happens when our application starts? As a matter of fact, we can! To do so, we define an application callback.

The first step is to tell our application definition (for example, our `.app` file) which module is going to implement the application callback. Let's do so by opening `mix.exs` and changing `def application` to the following:

```
  def application do
    [
      extra_applications: [:logger],
      mod: {KV, []}
    ]
  end
```

The `:mod` option specifies the "application callback module", followed by the arguments to be passed on application start. The application callback module can be any module that invokes `use Application`. Since we have specified `KV` as the module callback, let's change the `KV` module defined in `lib/kv.ex` to the following:

```
defmodule KV do
  use Application
end
```

Now run `mix test` and you will see a couple things happening. First of all, you will get a compilation warning:

```
Compiling 1 file (.ex)
    warning: function start/2 required by behaviour
Application is not implemented (in module KV)
    |
  1 | defmodule KV do
    | ~~~~~~~~~~~~~~
```

```
    │
    └─ lib/kv.ex:1: KV (module)
```

This warning is telling us that `use Application` actually defines a behaviour, which expects us to implement to a `start/2` function in our `KV` module.

Then our application does not even boot because the `start/2` function is not actually implemented:

```
18:29:39.109 [notice] Application kv exited: exited in:
KV.start(:normal, [])
    ** (EXIT) an exception was raised:
        ** (UndefinedFunctionError) function KV.start/2 is
undefined or private
```

Implementing the `start/2` callback is relatively straight-forward, all we need to do is to start a supervision tree, and return `{:ok, root_supervisor_pid}`. The `Supervisor.start_link/2` function does precisely that, it only expects a list of children and the supervision strategy. Let's just pass an empty list of children for now:

```elixir
defmodule KV do
  use Application

  # The @impl true annotation says we are implementing a
callback
  @impl true
  def start(_type, _args) do
    Supervisor.start_link([], strategy: :one_for_one)
```

```
  end
end
```

Now run `mix test` again and our app should boot but we should see one failure. When we changed the `KV` module, we broke the boilerplate test case which tested the `KV.hello/0` function. You can simply remove that test case and we are back to a green suite.

We wrote very little code but we did something incredibly powerful. We now have a function, `KV.start/2` that is invoked whenever your application starts. This gives us the perfect place to start our key-value registry. The `Application` module also allows us to define a `stop/1` callback and other functionality. You can check the `Application` and `Supervisor` modules for extensive documentation on their uses.

Let's finally start our registry.

Supervision trees

Now that we have the `start/2` callback, we can finally go ahead and start our registry. You may be tempted to do it like this:

```
def start(_type, _args) do
  Registry.start_link(name: KV, keys: :unique)
  Supervisor.start_link([], strategy: :one_for_one)
end
```

However, this would not be a good idea. In Elixir, we typically start processes inside supervision trees. In fact, we rarely use the `start_link` functions to start processes (except at the root of the supervision tree itself). Instead, do this:

```
  def start(_type, _args) do
    children = [
      {Registry, name: KV, keys: :unique}
    ]

    Supervisor.start_link(children, strategy: :one_for_one)
  end
```

A supervisor receives one or more child specifications that tell it exactly how to start each child. A child specification is typically represented by a `{module, options}` pair, as shown above, and often as simply the module name. Sometimes, these children are supervisors themselves, giving us supervision trees.

Let's take it for a spin and see if we can indeed name our buckets using our new registry. Let's make sure to start a new `iex -S mix` (`recompile()` is not enough, as it does not reload your supervision tree) and then:

```
iex> name = {:via, Registry, {KV, "shopping"}}
iex> KV.Bucket.start_link(name: name)
{:ok, #PID<0.43.0>}
iex> KV.Bucket.put(name, "milk", 1)
:ok
iex> KV.Bucket.get(name, "milk")
1
```

Perfect, this time we didn't need to start the registry inside `iex`, as it was started as part of the application itself.

By starting processes inside supervisors, we gain important properties such as:

- **Introspection**: for each application, you can fully introspect and visualize each process in its supervision tree, its memory usage, message queue, etc

- **Resilience**: when a process fails for an unexpected reason, its supervisor controls if and how those processes should be restarted, leading to self-healing systems

- **Graceful shutdown**: when your application is shutting down, the children of a supervision tree are terminated in the opposite order they were started, leading to graceful shutdowns

Projects or applications?

Mix makes a distinction between projects and applications. Based on the contents of our `mix.exs` file, we would say we have a Mix project that defines the `:kv` application.

When we say "project" you should think about Mix. Mix is the tool that manages your project. It knows how to compile your project, test your project and more. It also knows how to compile and start the application relevant to your project.

When we talk about applications, we talk about OTP. Applications are the entities that are started and stopped as a whole by the runtime. You can learn more about applications and how they relate to booting and shutting down of your system as a whole in the documentation for the `Application` module.

Summing up

We learned important concepts in this chapter:

- Naming registries allow us to find processes in a given machine (or, as we will see in the future, even in a cluster)

- Applications bundle our modules, its dependencies, and how code starts and stops

- Processes are started as part of supervisors for introspection and fault-tolerance

In the next chapter, we will tie it all up by making sure all our buckets are named and supervised. To do so, we will learn a new tool called dynamic supervisors.

Supervising dynamic children

We have successfully learned how our supervision tree is automatically started (and stopped) as part of our application's life cycle. We can also name our buckets via the `:name` option. We also learned that, in practice, we should always start new processes inside supervisors. Let's apply these insights by ensuring our buckets are named and supervised.

Child specs

Supervisors know how to start processes because they are given "child specifications". In our `lib/kv.ex` file, we defined a list of children with a single child spec:

```
children = [
  {Registry, name: KV, keys: :unique}
]
```

When the child specification is a tuple (as above) or module, then it is equivalent to calling the `child_spec/1` function on said module, which then returns the full specification. The pair above is equivalent to:

```
iex> Registry.child_spec(name: KV, keys: :unique)
%{
  id: KV,
  start: {Registry, :start_link, [[name: KV, keys:
:unique]]},
  type: :supervisor
}
```

The underlying map returns the `:id` (required), the module-function-args triplet to invoke to start the process (required), the type of the process (optional), among other optional keys. In other words, the `child_spec/1` function allows us to compose and encapsulate specifications in modules.

Therefore, if we want to supervise `KV.Bucket`, we only need to define a `child_spec/1` function. Luckily for us, whenever we invoke `use Agent` (or `use GenServer` or `use Supervisor` and so forth), an implementation with reasonable defaults is provided. So let's take it for a spin. Back on `iex -S mix`, try this:

```
iex> KV.Bucket.child_spec([])
%{id: KV.Bucket, start: {KV.Bucket, :start_link, [[]]}}
iex> KV.Bucket.child_spec([name: :shopping])
%{id: KV.Bucket, start: {KV.Bucket, :start_link, [[name:
:shopping]]}}
```

Let's try to start it as part of a supervisor then, using the `{module, options}` format to pass the bucket name (let's also use an atom as the name for convenience):

```
iex> children = [{KV.Bucket, name: :shopping}]
iex> Supervisor.start_link(children, strategy: :one_for_one)
iex> KV.Bucket.put(:shopping, "milk", 1)
:ok
iex> KV.Bucket.get(:shopping, "milk")
1
```

What happens now if we explicitly kill the bucket process?

```
# Find the pid for the given name
iex> pid = Process.whereis(:shopping)
```

```
#PID<0.48.0>
# Send it a kill exit signal
iex> Process.exit(pid, :kill)
true
# But a new process is alive in its place
iex> Process.whereis(:shopping)
#PID<0.50.0>
```

Given our buckets can already be supervised, it is time to hook them into our supervision tree.

Dynamic supervisors

Given our buckets can already be supervised, you may be thinking to start them as part of our application `start/2` callback, such as:

```
children = [
  {Registry, name: KV, keys: :unique}
  {KV.Bucket, name: {:via, Registry, {KV, "shopping"}}}
]
```

And while the above would definitely work, it comes with a huge caveat: it only starts a single bucket. In practice, we want the user to be able to create new buckets at any time. In other words, we need to start and supervise processes dynamically.

While the `Supervisor` module has APIs for starting children after its initialization, it was not designed or optimized for the use case of having potentially millions of children. For this purpose, Elixir instead provides the `DynamicSupervisor` module. Using it is quite similar to `Supervisor` except that, instead of specifying the children during start, you do it afterwards. Let's take it for a spin:

```
iex> {:ok, sup_pid} = DynamicSupervisor.start_link(strategy:
:one_for_one)
iex> DynamicSupervisor.start_child(sup_pid, {KV.Bucket,
name: :another_list})
iex> KV.Bucket.put(:another_list, "milk", 1)
:ok
iex> KV.Bucket.get(:another_list, "milk")
1
```

And it all works as expected. In fact, we can even give names to `DynamicSupervisor` themselves, instead of passing PIDs around and also use it to start buckets named using the registry:

```
iex> DynamicSupervisor.start_link(strategy: :one_for_one,
name: :dyn_sup)
iex> name = {:via, Registry, {KV, "yet_another_list"}}
iex> DynamicSupervisor.start_child(:dyn_sup, {KV.Bucket,
name: name})
iex> KV.Bucket.put(name, "milk", 1)
:ok
iex> KV.Bucket.get(name, "milk")
1
```

Overall, processes can be named and supervised, regardless if they are supervisors, agents, etc, since all of Elixir standard library was designed around those capabilities.

With all ingredients in place to supervise and name buckets, open up the `lib/kv.ex` module and let's add a new function called `KV.lookup_bucket/1`, which receives a name and either create or returns a bucket for the given name:

```elixir
defmodule KV do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      {Registry, name: KV, keys: :unique},
      {DynamicSupervisor, name: KV.BucketSupervisor,
strategy: :one_for_one}
    ]

    Supervisor.start_link(children, strategy: :one_for_one)
  end

  @doc """
  Creates a bucket with the given name.
  """
  def create_bucket(name) do
    DynamicSupervisor.start_child(KV.BucketSupervisor,
{KV.Bucket, name: via(name)})
  end

  @doc """
  Looks up the given bucket.
  """
  def lookup_bucket(name) do
    GenServer.whereis(via(name))
  end

  defp via(name), do: {:via, Registry, {KV, name}}
end
```

The code is relatively simple. First we changed `start/2` to also start a dynamic supervisor named `KV.BucketSupervisor`. Then, when

implemented `KV.create_bucket/1` which receives a bucket and starts with using our registry and dynamic supervisor. And we also added `KV.lookup_bucket/1` that receives the same name and attempts to find its PID.

To make sure it all works as expected, let's write a test. Open up `test/kv_test.exs` and add this:

```
defmodule KVTest do
  use ExUnit.Case, async: true

  test "creates and looks up buckets by any name" do
    name = "a unique name that won't be shared"
    assert is_nil(KV.lookup_bucket(name))

    assert {:ok, bucket} = KV.create_bucket(name)
    assert KV.lookup_bucket(name) == bucket

    assert KV.create_bucket(name) == {:error,
{:already_started, bucket}}
  end
end
```

The test shows we are creating and locating buckets with any name, making sure we use a unique name to avoid conflicts between tests.

The `start_supervised` test helper

Before we move on, let's do some clean up.

In `test/kv/bucket_test.exs`, we explicitly invoked `KV.Bucket.start_link/1` to start our buckets. However, we now know that we should avoid calling `start_link/1` directly and instead start processes as part of supervision trees.

In order to aid testing, `ExUnit` already starts a supervision tree per test and provides the `start_supervised` function to start processes within test-specific supervision tree. One advantage of this approach is that `ExUnit` guarantees any started process is shut down at the end of the test too. Let's rewrite our tests to use it instead:

```
defmodule KV.BucketTest do
  use ExUnit.Case, async: true

  test "stores values by key" do
    {:ok, bucket} = start_supervised(KV.Bucket)
    assert KV.Bucket.get(bucket, "milk") == nil

    KV.Bucket.put(bucket, "milk", 3)
    assert KV.Bucket.get(bucket, "milk") == 3
  end

  test "stores values by key on a named process", config do
    {:ok, _} = start_supervised({KV.Bucket, name:
config.test})
    assert KV.Bucket.get(config.test, "milk") == nil

    KV.Bucket.put(config.test, "milk", 3)
    assert KV.Bucket.get(config.test, "milk") == 3
  end
end
```

It is a small change, but our tests are now using all of the relevant best practices. Excellent!

Observer

Now that we have defined our supervision tree, it is a great opportunity to introduce the Observer tool that ships with Erlang. Start your application

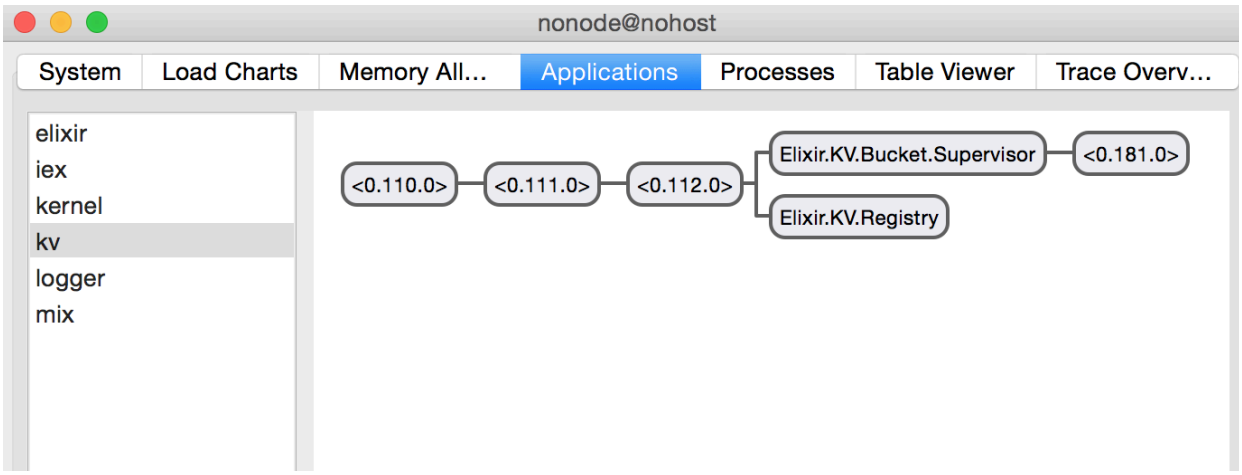
with `iex -S mix` and key this in:

```
iex> :observer.start()
```

> **Missing dependencies**
>
> When running `iex` inside a project with `iex -S mix, observer` won't be available as a dependency. To do so, you will need to call the following functions:
>
> ```
> iex> Mix.ensure_application!(:observer)
> iex> :observer.start()
> ```
>
> If the call above fails, here is what may have happened: some package managers default to installing a minimized Erlang without WX bindings for GUI support. In some package managers, you may be able to replace the headless Erlang with a more complete package (look for packages named `erlang` vs `erlang-nox` on Debian/Ubuntu/Arch). In others managers, you may need to install a separate `erlang-wx` (or similarly named) package.
>
> There are conversations to improve this experience in future releases.

A GUI should pop up containing all sorts of information about our system, from general statistics to load charts as well as a list of all running processes and applications.

In the Applications tab, you will see all applications currently running in your system alongside their supervision tree. You can select the `kv` application to explore it further:



Not only that, as you create new buckets on the terminal, you should see new processes spawned in the supervision tree shown in Observer:

```
iex> KV.create_bucket("shopping")
#PID<0.89.0>
```

We will leave it up to you to further explore what Observer provides. Note you can double-click any process in the supervision tree to retrieve more information about it, as well as right-click a process to send "a kill signal", a perfect way to emulate failures and see if your supervisor reacts as expected.

At the end of the day, tools like Observer are one of the reasons you want to always start processes inside supervision trees, even if they are temporary, to ensure they are always reachable and introspectable.

Now that our buckets are named and supervised, we are ready to start our server and start receiving requests.

Task and gen_tcp

In this chapter, we are going to learn how to use Erlang's `:gen_tcp` module to serve requests. This provides a great opportunity to explore Elixir's `Task` module. In future chapters, we will expand our server so that it can actually interact with buckets.

Echo server

We will start our TCP server by first implementing an echo server. It will send a response with the text it received in the request. We will slowly improve our server until it is supervised and ready to handle multiple connections.

A TCP server, in broad strokes, performs the following steps:

1. Listens to a port until the port is available and it gets hold of the socket
2. Waits for a client connection on that port and accepts it
3. Reads the client request and writes a response back

Let's implement those steps. Create a new `lib/kv/server.ex` and add the following functions:

```
defmodule KV.Server do
  require Logger

  def accept(port) do
    # The options below mean:
    #
    # 1. `:binary` - receives data as binaries (instead of
lists)
    # 2. `packet: :line` - receives data line by line
    # 3. `active: false` - blocks on `:gen_tcp.recv/2` until
```

```
data is available
    # 4. `reuseaddr: true` - allows us to reuse the address
if the listener crashes
    #
    {:ok, socket} =
      :gen_tcp.listen(port, [:binary, packet: :line, active:
false, reuseaddr: true])
    Logger.info("Accepting connections on port #{port}")
    loop_acceptor(socket)
  end

  defp loop_acceptor(socket) do
    {:ok, client} = :gen_tcp.accept(socket)
    serve(client)
    loop_acceptor(socket)
  end

  defp serve(socket) do
    socket
    |> read_line()
    |> write_line(socket)

    serve(socket)
  end

  defp read_line(socket) do
    {:ok, data} = :gen_tcp.recv(socket, 0)
    data
  end

  defp write_line(line, socket) do
    :gen_tcp.send(socket, line)
  end
end
```

We are going to start our server by calling `KV.Server.accept(4040)`, where 4040 is the port. The first step in `accept/1` is to listen to the port until the socket becomes available and then call `loop_acceptor/1`. `loop_acceptor/1` is a loop accepting client connections. For each accepted connection, we call `serve/1`.

`serve/1` is another loop that reads a line from the socket and writes those lines back to the socket. Note that the `serve/1` function uses the pipe operator `|>/2` to express this flow of operations. The pipe operator evaluates the left side and passes its result as the first argument to the function on the right side. The example above:

```
socket |> read_line() |> write_line(socket)
```

is equivalent to:

```
write_line(read_line(socket), socket)
```

The `read_line/1` implementation receives data from the socket using `:gen_tcp.recv/2` and `write_line/2` writes to the socket using `:gen_tcp.send/2`.

Note that `serve/1` is an infinite loop called sequentially inside `loop_acceptor/1`, so the tail call to `loop_acceptor/1` is never reached and could be avoided. However, as we shall see, we will need to execute `serve/1` in a separate process, so we will need that tail call soon.

This is pretty much all we need to implement our echo server. Let's give it a try!

Start an IEx session inside the `kv_server` application with `iex -S mix`. Inside IEx, run:

```
iex> KV.Server.accept(4040)
```

The server is now running, and you will even notice the console is blocked. Let's use a `telnet` client to access our server. There are clients available on most operating systems, and their command lines are generally similar:

```
$ telnet 127.0.0.1 4040
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
hello
hello
is it me
is it me
you are looking for?
you are looking for?
```

Type "hello", press enter, and you will get "hello" back. Excellent!

My particular telnet client can be exited by typing `ctrl + ]`, typing `quit`, and pressing `<Enter>`, but your client may require different steps.

Once you exit the telnet client, you will likely see an error in the IEx session:

```
** (MatchError) no match of right hand side value: {:error,
:closed}
    (kv) lib/kv/server.ex:45: KV.Server.read_line/1
    (kv) lib/kv/server.ex:37: KV.Server.serve/1
    (kv) lib/kv/server.ex:30: KV.Server.loop_acceptor/1
```

That's because we were expecting data from `:gen_tcp.recv/2` but the client closed the connection. We need to handle such cases better in future revisions of our server.

For now, there is a more important bug we need to fix: what happens if our TCP acceptor crashes? Since there is no supervision, the server dies and we won't be able to serve more requests, because it won't be restarted. That's why we must move our server to a supervision tree.

Tasks

Whenever you have an existing function and you simply want to execute it when your application starts, the `Task` module is exactly what you need. For example, it has a `Task.start_link/1` function that receives an anonymous function and executes it inside a new process that will be part of a supervision tree.

Let's give it a try. Open up `lib/kv.ex` and let's add a new child:

```
    def start(_type, _args) do
      children = [
        {Registry, name: KV, keys: :unique},
        {DynamicSupervisor, name: KV.BucketSupervisor,
  strategy: :one_for_one},
        {Task, fn -> KV.Server.accept(4040) end}
      ]

      Supervisor.start_link(children, strategy: :one_for_one)
    end
```

With this change, we are saying that we want to run `KV.Server.accept(4040)` as a task. We are hardcoding the port for now but we will make this a configuration in later chapters. As usual, we've

passed a two-element tuple as a child specification, which in turn will invoke `Task.start_link/1`.

Now that the server is part of the supervision tree, it should start automatically when we run the application. Run `iex -S mix` to boot the app and use the `telnet` client to make sure that everything still works:

```
$ telnet 127.0.0.1 4040
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
say you
say you
say me
say me
```

Yes, it works! However, can it handle more than one client?

Try to connect two telnet clients at the same time. When you do so, you will notice that the second client doesn't echo:

```
$ telnet 127.0.0.1 4040
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
hello
hello?
HELLOOOOOO?
```

It doesn't seem to work at all. That's because we are serving requests in the same process that are accepting connections. When one client is connected, we can't accept another client.

Adding (flawed) concurrency

In order to make our server handle simultaneous connections, we need to have one process working as an acceptor that spawns other processes to serve requests. One solution would be to change:

```
defp loop_acceptor(socket) do
  {:ok, client} = :gen_tcp.accept(socket)
  serve(client)
  loop_acceptor(socket)
end
```

to also use `Task.start_link/1`:

```
defp loop_acceptor(socket) do
  {:ok, client} = :gen_tcp.accept(socket)
  {:ok, pid} = Task.start_link(fn -> serve(client) end)
  :ok = :gen_tcp.controlling_process(client, pid)
  loop_acceptor(socket)
end
```

In the new acceptor loop, we are starting a new task every time there is a new client. Now, if you attempt to connect two clients at the same time, it should work!

Or does it? For example, what happens when you exit one telnet session? The other session should crash! The reason of this crash is two fold:

1. We have a bug in our server where we don't expect `:gen_tcp.recv/2` to return an `{:error, :closed}` tuple

2. Because each server task is linked to the acceptor process, if one task crashes, the acceptor process will also crash, taking down all other

tasks and clients

An important rule of thumb throughout this guide is to always start processes as children of supervisors. The code above is an excellent example of what happens when we don't. If we don't isolate the different parts of our systems, failures can now cascade through our system, as it would happen in other languages.

To fix this, we could use a `DynamicSupervisor`, but tasks also provide a specialized `Task.Supervisor` which has better ergonomics and is optimized for supervising tasks themselves. Let's give it a try.

Adding a task supervisor

Let's change `start/2` in `lib/kv.ex` once more, to add the task supervisor to our tree:

```
  def start(_type, _args) do
    children = [
      {Registry, name: KV, keys: :unique},
      {DynamicSupervisor, name: KV.BucketSupervisor,
strategy: :one_for_one},
      {Task.Supervisor, name: KV.ServerSupervisor},
      {Task, fn -> KV.Server.accept(4040) end}
    ]

    Supervisor.start_link(children, strategy: :one_for_one)
  end
```

We'll now start a `Task.Supervisor` process with name `KV.TaskSupervisor`. Keep in mind that the order children are started matters. For example, the acceptor must come last because, if it comes first, it means our application can start accepting requests before the `Task.Supervisor` is running or before we can locate buckets. Shutting

down an application will also stop the children in reverse order, guaranteeing a clean termination.

Now we need to change `loop_acceptor/1` to use `Task.Supervisor` to serve each request:

```
defp loop_acceptor(socket) do
  {:ok, client} = :gen_tcp.accept(socket)
  {:ok, pid} =
Task.Supervisor.start_child(KV.ServerSupervisor, fn ->
serve(client) end)
  :ok = :gen_tcp.controlling_process(client, pid)
  loop_acceptor(socket)
end
```

You might notice that we added a line, `:ok = :gen_tcp.controlling_process(client, pid)`. This makes the child process the "controlling process" of the `client` socket. If we didn't do this, the acceptor would bring down all the clients if it crashed because sockets would be tied to the process that accepted them (which is the default behavior).

Now start a new server with `iex -S mix` and try to open up many concurrent telnet clients. You will notice that quitting a client does not bring the acceptor down, even though we haven't fixed the bug in `:gen_tcp.recv/2` yet (which we will address in the next chapter). Excellent!

Restart strategies

There is one important topic we haven't explored yet with the necessary depth. What happens when a supervised process crashes?

In the previous chapter, when we started a bucket and killed it, the supervisor automatically started one in its place:

```
iex> children = [{KV.Bucket, name: :shopping}]
iex> Supervisor.start_link(children, strategy: :one_for_one)
iex> KV.Bucket.put(:shopping, "milk", 1)
iex> pid = Process.whereis(:shopping)
#PID<0.48.0>
iex> Process.exit(pid, :kill)
true
iex> Process.whereis(:shopping)
#PID<0.50.0>
```

What exactly happens when a process terminates is part of its child specification. For `KV.Bucket`, we have this:

```
iex> KV.Bucket.child_spec([])
%{id: KV.Bucket, start: {KV.Bucket, :start_link, [[]]}}
```

However, for tasks, we have this:

```
iex> Task.child_spec(fn -> :ok end)
%{
  id: Task,
  restart: :temporary,
  start: {Task, :start_link, [#Function<43.39164016/0 in
:erl_eval.expr/6>]}
}
```

Notice that a task says `:restart` is `:temporary`. `KV.Bucket` says nothing, which means it defaults to `:permanent`. `:temporary` means that a process is never restarted, regardless of why it crashed. `:permanent` means a

process is always restarted, regardless of the exit reason. There is also `:transient`, which means it won't be restarted as long as it terminates successfully.

Now we must ask ourselves, are those the correct settings?

For `KV.Bucket`, using `:permanent` seems logical, as we should not require the user to recreate a bucket they have previously created. Although currently we would lose the bucket data, in an actual system we would add mechanisms to recover it on initialization. However, for tasks, we have used them in two opposing ways in this chapter, which means at least one of them is wrong.

We use a task to start the acceptor. The acceptor is a critical component of our infrastructure. If it crashes, it means we won't accept further requests, and our server would then be useless as no one can connect to it. On the other hand, we also use `Task.Supervisor` to start tasks that deal with each connection. In this case, restarting may not be useful at all, given the reason we crashed could just as well be a connection issue, and attempting to restart over the same connection would lead to further failures.

Therefore, we want the acceptor to actually run in `:permanent` mode, while we preserve the `Task.Supervisor` as `:temporary`. Luckily Elixir has an API that allows us to change an existing child specification, which we use below.

Let's change `start/2` in `lib/kv.ex` once more to the following:

```
    def start(_type, _args) do
      children = [
        {Registry, name: KV, keys: :unique},
        {DynamicSupervisor, name: KV.BucketSupervisor,
  strategy: :one_for_one},
        {Task.Supervisor, name: KV.ServerSupervisor},
        Supervisor.child_spec({Task, fn ->
```

```
    KV.Server.accept(4040) end}, restart: :permanent)
    ]

    Supervisor.start_link(children, strategy: :one_for_one)
  end
```

Now we have an always running acceptor that starts temporary task processes under an always running task supervisor.

Leveraging the ecosystem

In this chapter, we implemented a basic TCP acceptor while exploring concurrency and fault-tolerance. Our acceptor can manage concurrent connections, but it is still not ready for production. Production-ready TCP servers run a pool of acceptors, each with their own supervisor. Elixir's `PartitionSupervisor` might be used to partition and scale the acceptor, but it is out of scope for this guide. In practice, you will use existing packages tailored for this use-case, such as Ranch (in Erlang) or Thousand Island (in Elixir).

In the next chapter, we will start parsing the client requests and sending responses, finishing our server.

Doctests, patterns, and with

In this chapter, we will implement the code that parses the commands we described in the first chapter:

```
CREATE shopping
OK

PUT shopping milk 1
OK

PUT shopping eggs 3
OK

GET shopping milk
1
OK

DELETE shopping eggs
OK
```

After the parsing is done, we will update our server to dispatch the parsed commands to the relevant buckets.

Doctests

On the language homepage, we mention that Elixir makes documentation a first-class citizen in the language. We have explored this concept many times throughout this guide, be it via `mix help` or by typing `h Enum` or another module in an IEx console.

In this section, we will implement the parsing functionality, document it and make sure our documentation is up to date with doctests. This helps us provide documentation with accurate code samples.

Let's create our command parser at `lib/kv/command.ex` and start with the doctest:

```
defmodule KV.Command do
  @doc ~S"""
  Parses the given `line` into a command.

  ## Examples

      iex> KV.Command.parse("CREATE shopping\r\n")
      {:ok, {:create, "shopping"}}

  """
  def parse(_line) do
    :not_implemented
  end
end
```

Doctests are specified by an indentation of four spaces followed by the `iex>` prompt in a documentation string. If a command spans multiple lines, you can use `...>`, as in IEx. The expected result should start at the next line after `iex>` or `...>` line(s) and is terminated either by a newline or a new `iex>` prefix.

Also, note that we started the documentation string using `@doc ~S"""`. The `~S` prevents the `\r\n` characters from being converted to a carriage return and line feed until they are evaluated in the test.

To run our doctests, we'll create a file at `test/kv/command_test.exs` and call `doctest KV.Command` in the test case:

```
defmodule KV.CommandTest do
  use ExUnit.Case, async: true
  doctest KV.Command
end
```

Run the test suite and the doctest should fail:

```
  1) doctest KV.Command.parse/1 (1) (KV.CommandTest)
     test/kv/command_test.exs:3
     Doctest failed
     doctest:
       iex> KV.Command.parse("CREATE shopping\r\n")
       {:ok, {:create, "shopping"}}
     code: KV.Command.parse "CREATE shopping\r\n" === {:ok,
{:create, "shopping"}}
     left:  :not_implemented
     right: {:ok, {:create, "shopping"}}
     stacktrace:
       lib/kv/command.ex:7: KV.Command (module)
```

Excellent!

Now let's make the doctest pass. Let's implement the `parse/1` function:

```
def parse(line) do
  case String.split(line) do
    ["CREATE", bucket] -> {:ok, {:create, bucket}}
  end
end
```

Our implementation splits the line on whitespace and then matches the command against a list. Using `String.split/1` means our commands will

be whitespace-insensitive. Leading and trailing whitespace won't matter, nor will consecutive spaces between words. Let's add some new doctests to test this behavior along with the other commands:

```
    @doc ~S"""
    Parses the given `line` into a command.

    ## Examples

        iex> KV.Command.parse "CREATE shopping\r\n"
        {:ok, {:create, "shopping"}}

        iex> KV.Command.parse "CREATE  shopping  \r\n"
        {:ok, {:create, "shopping"}}

        iex> KV.Command.parse "PUT shopping milk 1\r\n"
        {:ok, {:put, "shopping", "milk", "1"}}

        iex> KV.Command.parse "GET shopping milk\r\n"
        {:ok, {:get, "shopping", "milk"}}

        iex> KV.Command.parse "DELETE shopping eggs\r\n"
        {:ok, {:delete, "shopping", "eggs"}}

    Unknown commands or commands with the wrong number of
    arguments return an error:

        iex> KV.Command.parse "UNKNOWN shopping eggs\r\n"
        {:error, :unknown_command}

        iex> KV.Command.parse "GET shopping\r\n"
        {:error, :unknown_command}

    """
```

With doctests at hand, it is your turn to make tests pass! Once you're ready, you can compare your work with our solution below:

```
  def parse(line) do
    case String.split(line) do
      ["CREATE", bucket] -> {:ok, {:create, bucket}}
      ["GET", bucket, key] -> {:ok, {:get, bucket, key}}
      ["PUT", bucket, key, value] -> {:ok, {:put, bucket,
key, value}}
      ["DELETE", bucket, key] -> {:ok, {:delete, bucket,
key}}
      _ -> {:error, :unknown_command}
    end
  end
```

Notice how we were able to elegantly parse the commands without adding a bunch of `if/else` clauses that check the command name and number of arguments!

Finally, you may have observed that each doctest corresponds to a different test in our suite, which now reports a total of 7 doctests. That is because ExUnit considers the following to define two different doctests:

```
iex> KV.Command.parse("UNKNOWN shopping eggs\r\n")
{:error, :unknown_command}

iex> KV.Command.parse("GET shopping\r\n")
{:error, :unknown_command}
```

Without new lines, as seen below, ExUnit compiles it into a single doctest:

```
iex> KV.Command.parse("UNKNOWN shopping eggs\r\n")
{:error, :unknown_command}
```

```
iex> KV.Command.parse("GET shopping\r\n")
{:error, :unknown_command}
```

As the name says, doctest is documentation first and a test later. Their goal is not to replace tests but to provide up-to-date documentation. You can read more about doctests in the `ExUnit.DocTest` documentation.

Using `with`

As we are now able to parse commands, we can finally start implementing the logic that runs the commands. Let's add a stub definition for this function for now:

```
defmodule KV.Command do
  @doc """
  Runs the given command.
  """
  def run(command, socket) do
    :gen_tcp.send(socket, "OK\r\n")
    :ok
  end
end
```

Before we implement this function, let's change our server to start using our new `parse/1` and `run/1` functions. Remember, our `read_line/1` function was also crashing when the client closed the socket, so let's take the opportunity to fix it, too. Open up `lib/kv/server.ex` and replace the existing server definition:

```
  defp serve(socket) do
    socket
    |> read_line()
```

```
    |> write_line(socket)

    serve(socket)
  end

  defp read_line(socket) do
    {:ok, data} = :gen_tcp.recv(socket, 0)
    data
  end

  defp write_line(line, socket) do
    :gen_tcp.send(socket, line)
  end
```

by the following:

```
  defp serve(socket) do
    msg =
      case read_line(socket) do
        {:ok, data} ->
          case KV.Command.parse(data) do
            {:ok, command} ->
              KV.Command.run(command, socket)

            {:error, _} = err ->
              err
          end

        {:error, _} = err ->
          err
      end

    write_line(socket, msg)
    serve(socket)
```

```
  end

  defp read_line(socket) do
    :gen_tcp.recv(socket, 0)
  end

  defp write_line(_socket, :ok) do
    :ok
  end

  defp write_line(socket, {:error, :unknown_command}) do
    # Known error; write to the client
    :gen_tcp.send(socket, "UNKNOWN COMMAND\r\n")
  end

  defp write_line(_socket, {:error, :closed}) do
    # The connection was closed, exit politely
    exit(:shutdown)
  end

  defp write_line(socket, {:error, error}) do
    # Unknown error; write to the client and exit
    :gen_tcp.send(socket, "ERROR\r\n")
    exit(error)
  end
```

If we start our server, we can now send commands to it. For now, we will get two different responses: "OK" when the command is known and "UNKNOWN COMMAND" otherwise:

```
$ telnet 127.0.0.1 4040
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
```

```
CREATE shopping
OK
HELLO
UNKNOWN COMMAND
```

This means our implementation is going in the correct direction, but it doesn't look very elegant, does it?

The previous implementation used pipelines which made the logic straightforward to follow. However, now that we need to handle different error codes along the way, our server logic is nested inside many `case` calls.

Thankfully, Elixir has the `with` construct, which allows you to simplify code like the above, replacing nested `case` calls with a chain of matching clauses. Let's rewrite the `serve/1` function to use `with`:

```
defp serve(socket) do
  msg =
    with {:ok, data} <- read_line(socket),
         {:ok, command} <- KV.Command.parse(data),
         do: KV.Command.run(command, socket)

  write_line(socket, msg)
  serve(socket)
end
```

Much better! `with` will retrieve the value returned by the right-side of `<-` and match it against the pattern on the left side. If the value matches the pattern, `with` moves on to the next expression. In case there is no match, the non-matching value is returned.

In other words, we converted each expression given to `case/2` as a step in `with`. As soon as any of the steps return something that does not match `{:ok, x}`, `with` aborts, and returns the non-matching value.

You can read more about `with/1` in our documentation.

Running commands

The last step is to implement `KV.Command.run/1` to run the parsed commands on top of buckets. Its implementation is shown below:

```
@doc """
Runs the given command.
"""
def run(command, socket)

def run({:create, bucket}, socket) do
  KV.create_bucket(bucket)
  :gen_tcp.send(socket, "OK\r\n")
  :ok
end

def run({:get, bucket, key}, socket) do
  lookup(bucket, fn pid ->
    value = KV.Bucket.get(pid, key)
    :gen_tcp.send(socket, "#{value}\r\nOK\r\n")
    :ok
  end)
end

def run({:put, bucket, key, value}, socket) do
  lookup(bucket, fn pid ->
    KV.Bucket.put(pid, key, value)
    :gen_tcp.send(socket, "OK\r\n")
```

```
      :ok
    end)
  end

  def run({:delete, bucket, key}, socket) do
    lookup(bucket, fn pid ->
      KV.Bucket.delete(pid, key)
      :gen_tcp.send(socket, "OK\r\n")
      :ok
    end)
  end

  defp lookup(bucket, callback) do
    if bucket = KV.lookup_bucket(bucket) do
      callback.(bucket)
    else
      {:error, :not_found}
    end
  end
```

Each function clause dispatches the appropriate command to the appropriate bucket.

You might have noticed we have a function head, `def run(command, socket)`, without a body. In the Modules and Functions chapter, we learned that a bodiless function can be used to declare default arguments for a multi-clause function. Here is another use case where we use a function without a body to document what the arguments are.

We have also defined a private function named `lookup/2` to help with the common functionality of looking up a bucket and returning its `pid` if it exists, `{:error, :not_found}` otherwise.

By the way, since we are now returning `{:error, :not_found}`, we should amend the `write_line/2` function in `KV.Server` to print such error as well:

```
defp write_line(socket, {:error, :not_found}) do
  :gen_tcp.send(socket, "NOT FOUND\r\n")
end
```

Our server functionality is almost complete. Only tests are missing.

Integration tests

`KV.Command.run/1`'s implementation is sending commands directly to the `KV` module, which is using a local registry to name processes. This means if we have two tests sending messages to the same bucket, our tests will conflict with each other (and likely fail). One might think this would be a reason to use mocks and other strategies to keep our tests isolated, but such techniques often make our testing environment too distant from how our code actually runs in production, and you may end-up with bugs lurking.

Luckily, there is a technique that we have been using throughout this guide that would be equally applicable here: it is ok to rely on the local registry as long as each test uses unique names. Using a combination of the test module and test name is more than enough to guarantee that.

So let's write integration tests that rely on unique names to exercise the whole stack from the TCP server to the bucket.

Create a new file at `test/kv/server_test.exs` as shown below:

```
defmodule KV.ServerTest do
  use ExUnit.Case, async: true
```

```
  @socket_options [:binary, packet: :line, active: false]

  setup config do
    {:ok, socket} = :gen_tcp.connect(~c"localhost", 4040,
@socket_options)
    test_name = config.test |> Atom.to_string() |>
String.replace(" ", "-")
    %{socket: socket, name: "#{config.module}-#{test_name}"}
  end

  test "server interaction", %{socket: socket, name: name}
do
    # CREATE
    assert send_and_recv(socket, "CREATE #{name}\r\n") ==
"OK\r\n"

    # PUT
    assert send_and_recv(socket, "PUT #{name} eggs 3\r\n")
== "OK\r\n"

    # GET
    assert send_and_recv(socket, "GET #{name} eggs\r\n") ==
"3\r\n"
    assert send_and_recv(socket, "") == "OK\r\n"

    # DELETE
    assert send_and_recv(socket, "DELETE #{name} eggs\r\n")
== "OK\r\n"

    # GET
    assert send_and_recv(socket, "GET #{name} eggs\r\n") ==
"\r\n"
    assert send_and_recv(socket, "") == "OK\r\n"
  end

  test "unknown command", %{socket: socket} do
```

```
    assert send_and_recv(socket, "WHATEVER\r\n") ==
             "UNKNOWN COMMAND\r\n"
  end

  test "unknown bucket", %{socket: socket} do
    assert send_and_recv(socket, "GET whatever eggs\r\n") ==
             "NOT FOUND\r\n"
  end

  defp send_and_recv(socket, command) do
    :ok = :gen_tcp.send(socket, command)
    {:ok, data} = :gen_tcp.recv(socket, 0, 1000)
    data
  end
end
```

Run `mix test` and the tests should all pass. However, make sure to terminate any `iex -S mix` session you may have running, as currently tests and development environment are running on the same port (4040). We will address it in the next chapter.

We added three tests, the first one tests most bucket actions, while the other two deal with error cases. Given there is a lot of shared setup across these tests, we used the `setup/2` macro to deal with common boilerplate. The macro receives the same *test context* as tests and starts a client TCP connection per test. It also defines a unique bucket name using the module name and the test name, making sure any space in the test name is replaced by `-` as to not interfere with our command parsing logic.

Then, in each test, we pattern matched on the *test context*, extracting the socket or name as necessary. This is similar to the code we wrote in `test/kv/bucket_test.exs`:

```
    test "stores values by key on a named process", config do
```

Except back then we matched on all config and, this time around, we matched only on the data we needed.

Let's move to the next chapter. We will finally make our system distributed by adding a tiny bit of configuration and, *spoiler alert*, changing one line of code.

Configuration and distribution

So far we have hardcoded our applications to run a web server on port 4040. This has been somewhat problematic since we can't, for example, run our development server and tests at the same time. In this chapter, we will learn how to use the application environment for configuration, paving the way for us to enable distribution by running multiple development servers on the same machine (on different ports).

In this last guide, we will make the routing table for our distributed key-value store configurable, and then finally package the software for production.

Let's do this.

Application environment

In the chapter Registries, applications, and supervisors, we have learned that our project is backed by an application, which bundles our modules and specifies how your supervision tree starts and shuts down. Each application can also have its own configuration, which in Erlang/OTP (and therefore Elixir) is called "application environment".

We can use the application environment to configure our own application, as well as others. Let's see the application environment in practice. Create a file `config/runtime.exs` with the following:

```
import Config

port =
  cond do
    port_env = System.get_env("PORT") ->
      String.to_integer(port_env)
```

```
    config_env() == :test ->
      4040

    true ->
      4050
  end

config :kv, :port, port
```

The above is attempting to read the "PORT" environment variable and use it as the port if defined. Otherwise, we default to port `4040` for tests and port `4050` for other environments, eliminating the conflict between environments we have seen in the past. Then we store its value under the `:port` key of our `:kv` application.

Now we just need to read this configuration. Open up `lib/kv.ex` and the `start/2` function to the following:

```
  def start(_type, _args) do
    port = Application.fetch_env!(:kv, :port)

    children = [
      {Registry, name: KV, keys: :unique},
      {DynamicSupervisor, name: KV.BucketSupervisor,
strategy: :one_for_one},
      {Task.Supervisor, name: KV.ServerSupervisor},
      Supervisor.child_spec({Task, fn ->
KV.Server.accept(port) end}, restart: :permanent)
    ]

    Supervisor.start_link(children, strategy: :one_for_one)
  end
```

Run `iex -S mix` and you will see the following message printed:

```
[info] Accepting connections on port 4050
```

Run tests, without killing the development server, and you will see it running on port 4040.

Our change was straight-forward. We used `Application.fetch_env!/2` to read the entry for `port` in `:kv`'s environment. We explicitly used `fetch_env!/2` (instead of `get_env/2` or `fetch_env`) because it will raise if the port was not configured (preventing the app from booting).

Compile vs runtime configuration

Configuration files provide a mechanism for us to configure the environment of any application. Elixir provides two configuration entry points:

- `config/config.exs` — this file is read at build time, before we compile our application and before we even load our dependencies. This means we can't access the code in our application nor in our dependencies. However, it means we can control how they are compiled

- `config/runtime.exs` — this file is read after our application and dependencies are compiled and therefore it can configure how our application works at runtime. If you want to read system environment variables (via `System.get_env/1`) or access external configuration, this is the appropriate place to do so

You can learn more about configuration in the `Config` and `Config.Provider` modules.

Generally speaking, we use `Application.fetch_env!/2` (and friends) to read runtime configuration. `Application.compile_env/2` is available for reading compile-time configuration. This allows Elixir to track which modules to recompile when the compilation environment changes.

Now that we can start multiple servers, let's explore distribution.

Our first distributed code

Elixir ships with facilities to connect nodes and exchange information between them. In fact, we use the same concepts of processes, message passing and receiving messages when working in a distributed environment because Elixir processes are *location transparent*. This means that when sending a message, it doesn't matter if the recipient process is on the same node or on another node, the VM will be able to deliver the message in both cases.

In order to run distributed code, we need to start the VM with a name. The name can be short (when in the same network) or long (requires the full computer address). Let's start a new IEx session:

```
$ iex --sname foo
```

You can see now the prompt is slightly different and shows the node name followed by the computer name:

```
Interactive Elixir - press Ctrl+C to exit (type h() ENTER
for help)
iex(foo@jv)1>
```

My computer is named `jv`, so I see `foo@jv` in the example above, but you will get a different result. We will use `foo@computer-name` in the following

examples and you should update them accordingly when trying out the code.

Let's define a module named `Hello` in this shell:

```
iex> defmodule Hello do
...>   def world, do: IO.puts("hello world")
...> end
```

If you have another computer on the same network with both Erlang and Elixir installed, you can start another shell on it. If you don't, you can start another IEx session in another terminal. In either case, give it the short name of `bar`:

```
$ iex --sname bar
```

Note that inside this new IEx session, we cannot access `Hello.world/0`:

```
iex> Hello.world
** (UndefinedFunctionError) function Hello.world/0 is
undefined (module Hello is not available)
    Hello.world()
```

However, we can spawn a new process on `foo@computer-name` from `bar@computer-name`! Let's give it a try (where `@computer-name` is the one you see locally):

```
iex> Node.spawn_link(:"foo@computer-name", fn ->
Hello.world() end)
```

```
#PID<9014.59.0>
hello world
```

Elixir spawned a process on another node and returned its PID. You can see the PID number no longer starts with zero, showing it belongs to another node. The code then executed on the other node where the `Hello.world/0` function exists and invoked that function. Note that the result of "hello world" was printed on the current node `bar` and not on `foo`. In other words, the message to be printed was sent back from `foo` to `bar`. This happens because the process spawned on the other node (`foo`) knows all the output should be sent back to the original node!

We can send and receive messages from the PID returned by `Node.spawn_link/2` as usual. Let's try a quick ping-pong example:

```
iex> pid = Node.spawn_link(:"foo@computer-name", fn ->
...>   receive do
...>     {:ping, client} -> send(client, :pong)
...>   end
...> end)
#PID<9014.59.0>
iex> send(pid, {:ping, self()})
{:ping, #PID<0.73.0>}
iex> flush()
:pong
:ok
```

In other words, we can spawn processes in other nodes, hold onto their PIDs, and then send messages to them as if they were running on the same machine. That's the *location transparency* principle. And because everything we have built so far was built on top of messaging passing, we should be able to adjust our key-value store to become a distributed one with little work.

Distributed naming registry with `:global`

First, let's check that our code is not currently distributed. Start a new node like this:

```
$ PORT=4100 iex --sname foo -S mix
```

And the other like this:

```
$ PORT=4101 iex --sname bar -S mix
```

Now, within `foo@computer-name`, do this:

```
iex> :erpc.call(:"bar@computer-name", KV, :create_bucket,
["shopping"])
{:ok, #PID<22121.164.0>}
```

Instead of using `Node.spawn_link/2`, we used Erlang's builtin RPC module to call the function `create_bucket` in the `KV` module passing a one element list with the string "shopping" as the argument list. We could have used `Node.spawn_link/2`, but `:erpc.call/4` conveniently returns the result of the invocation.

Still in `foo@computer-name`, let's try to access the bucket:

```
iex> KV.lookup_bucket("shopping")
nil
```

It returns `nil`. However, if you run `KV.lookup_bucket("shopping")` in `bar@computer-name`, it will return the proper bucket. In other words, the

nodes can communicate with each other, but buckets spawned in one node are not visible to the other.

This is because we are using Elixir's Registry to name our buckets, which is a **local** process registry. In other words, it is designed for processes running on a single node and not for distribution.

Luckily, Erlang ships with a distributed registry called `:global`, which is directly supported by the `:name` option by passing a `{:global, name}` tuple. All we need to do is update the `via/1` function in `lib/kv.ex` from this:

```
defp via(name), do: {:via, Registry, {KV, name}}
```

to this:

```
defp via(name), do: {:global, name}
```

Do the change above and restart both `foo@computer-name` and `bar@computer-name`. Now, back on `foo@computer-name`, let's give it another try:

```
iex> :erpc.call(:"bar@computer-name", KV, :create_bucket,
["shopping"])
{:ok, #PID<21821.179.0>}
iex> KV.lookup_bucket("shopping")
#PID<21821.179.0>
```

And there you go! By simply changing which naming registry we used, we now have a distributed key value store. You can even try using `telnet` to connect to the servers on different ports and validate that changes in one session are visible in the other one. Exciting!

Node discovery and dependencies

There is one essential ingredient to wrap up our distributed key-value store. In order for the `:global` registry to work, we need to make sure the nodes are connected to each other. When we run `:erpc` call passing the node name:

```
:erpc.call(:"bar@computer-name", KV, :create_bucket,
["shopping"])
```

Elixir automatically connected the nodes together. This is easy to do in an IEx session when both instances are running on the same machine but it requires more work in a production environment, where instances are on different machines which may be started at any time and running on different IP addresses.

Luckily for us, this is also a well-solved problem. For example, if you are using the Phoenix web framework in production, it ships with the `dns_cluster` package, which automatically runs DNS queries to find new nodes and connect them. If you are using Kubernetes or cloud providers, packages like `libcluster` ship with different strategies to discover and connect nodes.

Installing dependencies in Elixir is simple. Most commonly, we use the Hex Package Manager, by listing the dependency inside the deps function in our `mix.exs` file:

```
def deps do
  [{:dns_cluster, "~> 0.2"}]
end
```

This dependency refers to the latest version of `dns_cluster` in the 0.x version series that has been pushed to Hex. This is indicated by the `~>`

preceding the version number. For more information on specifying version requirements, see the documentation for the `Version` module.

Typically, stable releases are pushed to Hex. If you want to depend on an external dependency still in development, Mix is able to manage Git dependencies too:

```
def deps do
  [{:dns_cluster, git:
"https://github.com/phoenixframework/dns_cluster.git"}]
end
```

You will notice that when you add a dependency to your project, Mix generates a `mix.lock` file that guarantees *repeatable builds*. The lock file must be checked in to your version control system, to guarantee that everyone who uses the project will use the same dependency versions as you.

Mix provides many tasks for working with dependencies, which can be seen in `mix help`:

```
$ mix help
mix deps              # Lists dependencies and their status
mix deps.clean        # Deletes the given dependencies'
files
mix deps.compile      # Compiles dependencies
mix deps.get          # Fetches unavailable and out of date
dependencies
mix deps.tree         # Prints the dependency tree
mix deps.unlock       # Unlocks the given dependencies
mix deps.update       # Updates the given dependencies
```

The most common tasks are `mix deps.get` and `mix deps.update`. Once fetched, dependencies are automatically compiled for you. You can read more about deps by running `mix help deps`.

To wrap up this chapter, we will build a very simple node discovery mechanism, where the name of the nodes we should connect to are given on boot, using the lessons we learned in this chapter.

`Node.connect/1`

We will change our application to support a "NODES" environment variable with the name of all nodes each instance should connect to.

Open up `config/runtime.exs` and add this to the bottom:

```
nodes =
  System.get_env("NODES", "")
  |> String.split(",", trim: true)
  |> Enum.map(&String.to_atom/1)

config :kv, :nodes, nodes
```

We fetch the environment variable, split it on "," while discarding all empty strings, and then convert each entry to an atom, as node names are atoms.

Now, in your `start/2` callback, we will add this to of the `start/2` function:

```
  def start(_type, _args) do
    for node <- Application.fetch_env!(:kv, :nodes) do
      Node.connect(node)
    end
```

Now we can start our nodes as:

```
$ NODES="foo@computer-name,bar@computer-name" PORT=4040 iex
--sname foo -S mix
$ NODES="foo@computer-name,bar@computer-name" PORT=4041 iex
--sname bar -S mix
```

And they should connect to each other. Give it a try!

In an actual production system, there is some additional care we must take. For example, we often use `--name` instead of `--sname` and give fully qualified node names.

Furthermore, when connecting two instances, we must guarantee they have the same cookie, which is a secret Erlang uses to authorize the connection. When they run on the same machine, they share the same cookie by default, but it must be either explicitly set or shared in other ways when deploying in a cluster.

We will revisit these topics in the last chapter when we talk about releases.

Distributed system trade-offs

In this chapter, we made our key-value store distributed by using the `:global` naming registry. However, it is important to keep in mind that every distributed system, be it a library or a full-blown database, is designed with a series of trade-offs in mind.

In particular, `:global` requires consistency across all known nodes whenever a new bucket is created. For example, if your cluster has three nodes, creating a new bucket will require all three nodes to agree on its name. This means if one node is unresponsive, perhaps due to a network partition, the node will have to either reconnect or be kicked out before registration succeeds. This also means that, as your cluster grows in size,

registration becomes more expensive, although lookups are always cheap and immediate. Within the ecosystem, there are other named registries, which explore different trade-offs, such as Syn.

Further complications arise when we consider storage. Today, when our nodes terminate, we lose all data stored in the buckets. In our current design, since we allow each node to store their own buckets, it means we would need to backup each node. And, if we don't want data losses, we would also need to replicate the data.

For those reasons, it is still very common to use a database (or any storage system) when writing production applications in Elixir, and use Elixir to implement the realtime and collaborative aspects of your applications that extend beyond storage. For example, we can use Elixir to track which clients are connected to the cluster at any given moment or implement a feed where users are notified in realtime whenever items are added or removed from a bucket.

In fact, that's exactly what we will build in the next chapter. Allowing us to wrap up everything we have learned so far and also talk about one of the essential building blocks in Elixir software: GenServers.

Client-server with GenServer

To wrap up our distributed key-value store, we will implement a feature where a client can subscribe to a bucket and receive realtime notifications of any modification happening in the bucket, regardless of where in the cluster the bucket is located.

We will do by adding a new command, called SUBSCRIBE, to be used like this:

```
SUBSCRIBE shopping
milk SET TO 1
eggs SET TO 10
milk DELETED
```

To make this work, we must change our `KV.Bucket` implementation to track subscriptions and emit broadcasts. However, as we will see, we cannot implement such on top of agents, and we will need to rewrite our bucket implementation to a `GenServer`.

Links and monitors

Processes in Elixir are isolated. When they need to communicate, they do so by sending messages. However, how do you know when a process terminates, either because it has completed or due to a crash?

We have two options: links and monitors.

We have used links extensively. Whenever we started a process, we typically did so by using `start_link` or similar. The idea behind links is that, if any of the processes crash, the other will crash due to the link. We

talked about them in the Process chapter of the Getting Started guide. Here is a refresher:

```
iex> self()
#PID<0.115.0>
iex> spawn_link(fn -> :nothing_bad_will_happen end)
#PID<0.116.0>
iex> self()
#PID<0.115.0>


iex> spawn_link(fn -> raise "oops" end)
#PID<0.117.0>

12:37:33.229 [error] Process #PID<0.117.0> raised an
exception
Interactive Elixir (1.18.4) - press Ctrl+C to exit (type h()
ENTER for help)
iex> self()
#PID<0.118.0>
```

The reason why we links are so pervasive is because when we start a process inside a supervisor, we want our process to crash if the supervisor terminates. On the other hand, we don't want the supervisor to crash when a child terminates, and therefore supervisors trap exits from links by calling `Process.flag(:trap_exit, true)`.

In other words, links create an intrinsic relationship between the processes. If we simply want to track when a process dies, without tying their exit signals to each other, a better solution is to use monitors. When a monitored process terminates, we receive a message in our inbox, regardless of the reason:

```
iex> pid = spawn(fn -> Process.sleep(5000) end)
#PID<0.119.0>
iex> Process.monitor(pid)
#Reference<0.1076459149.2159017989.118674>
iex> flush()
:ok
# Wait five seconds
iex> flush()
{:DOWN, #Reference<0.1076459149.2159017989.118674>,
:process, #PID<0.119.0>, :normal}
:ok
```

Once the process terminates, we receive a "DOWN message", represented in a five-element tuple. The last element is the reason why it crashed (`:normal` means it terminated successfully).

Monitors will play a very important role in our subscribe feature. When a client subscribes to a bucket, the bucket will store the client PID and send messages to it on every change. However, if the client terminates (for example because it was disconnected), the bucket must remove the client from its list of subscribers (otherwise the list would keep on growing forever as clients connect and disconnect).

We chose the `Agent` module to implement our `KV.Bucket` and, unfortunately, agents cannot receive messages. So the first step is to rewrite our `KV.Bucket` to a `GenServer`. The `GenServer` module documentation has a good overview on what they are and how to implement them. Give it a read and then we are ready to proceed.

GenServer callbacks

A GenServer is a process that invokes a limited set of functions under specific conditions. When we used an `Agent`, we would keep both the client code and the server code side by side, like this:

```elixir
def put(bucket, key, value) do
  Agent.update(bucket, &Map.put(&1, key, value))
end
```

Let's break that code apart a bit:

```elixir
def put(bucket, key, value) do
  # Here is the client code
  Agent.update(bucket, fn state ->
    # Here is the server code
    Map.put(state, key, value)
  end)
  # Back to the client code
end
```

In the code above, we have a process, which we call "the client" sending a request to an agent, "the server". The request contains an anonymous function, which must be executed by the server.

In a GenServer, the code above would be two separate functions, roughly like this:

```elixir
def put(bucket, key, value) do
  # Send the server a :put "instruction"
  GenServer.call(bucket, {:put, key, value})
end

# Server callback

def handle_call({:put, key, value}, _from, state) do
  {:reply, :ok, Map.put(state, key, value)}
end
```

Let's go ahead and rewrite `KV.Bucket` at once. Open up `lib/kv/bucket.ex` and replace its contents with this new version:

```
defmodule KV.Bucket do
  use GenServer

  @doc """
  Starts a new bucket.
  """
  def start_link(opts) do
    GenServer.start_link(__MODULE__, %{}, opts)
  end

  @doc """
  Gets a value from the `bucket` by `key`.
  """
  def get(bucket, key) do
    GenServer.call(bucket, {:get, key})
  end

  @doc """
  Puts the `value` for the given `key` in the `bucket`.
  """
  def put(bucket, key, value) do
    GenServer.call(bucket, {:put, key, value})
  end

  @doc """
  Deletes `key` from `bucket`.

  Returns the current value of `key`, if `key` exists.
  """
  def delete(bucket, key) do
    GenServer.call(bucket, {:delete, key})
  end
```

```
  ### Callbacks

  @impl true
  def init(bucket) do
    state = %{
      bucket: bucket
    }

    {:ok, state}
  end

  @impl true
  def handle_call({:get, key}, _from, state) do
    value = get_in(state.bucket[key])
    {:reply, value, state}
  end

  def handle_call({:put, key, value}, _from, state) do
    state = put_in(state.bucket[key], value)
    {:reply, :ok, state}
  end

  def handle_call({:delete, key}, _from, state) do
    {value, state} = pop_in(state.bucket[key])
    {:reply, value, state}
  end
end
```

The first function is `start_link/1`, which starts a new GenServer passing a list of options. `GenServer.start_link/3`, which takes three arguments:

1. The module where the server callbacks are implemented, in this case `__MODULE__` (meaning the current module)

2. The initialization arguments, in this case the empty bucket `%{}`

3. A list of options which can be used to specify things like the name of the server. Once again, we forward the list of options that we receive on `start_link/1` to `GenServer.start_link/3`, as we did for agents

Once started, the GenServer will invoke the `init/1` callback, that receives the second argument given to `GenServer.start_link/3` and returns `{:ok, state}`, where state is a new map. We can already notice how the `GenServer` API makes the client/server segregation more apparent. `start_link/3` happens in the client, while `init/1` is the respective callback that runs on the server.

There are two types of requests you can send to a GenServer: calls and casts. Calls are synchronous and the server **must** send a response back to such requests. While the server computes the response, the client is **waiting**. Casts are asynchronous: the server won't send a response back and therefore the client won't wait for one. Both requests are messages sent to the server, and will be handled in sequence. So far we have only used `GenServer.call/2`, to keep the same semantics as the Agent, but we will give `cast` a try when implementing subscriptions. Given we kept the same behaviour, all tests will still pass.

Each request must be implemented as a specific callback. For `call/2` requests, we implement a `handle_call/3` callback that receives the `request`, the process from which we received the request (`_from`), and the current server state (`state`). The `handle_call/3` callback returns a tuple in the format `{:reply, reply, updated_state}`. The first element of the tuple, `:reply`, indicates that the server should send a reply back to the client. The second element, `reply`, is what will be sent to the client while the third, `updated_state` is the new server state.

Another Elixir feature we used in the implementation above are the nested traversal functions: `get_in/1`, `put_in/2`, and `pop_in/1`. Instead of

keeping the `bucket` as our GenServer state, we defined a state map with a `bucket` key inside. This will be important as we also need to track subscribers as part of the GenServer state. These new functions make it straight-forward to manipulate data structures nested in other data structures.

With our GenServer in place, let's work on subscription, starting with the tests.

Implementing subscriptions

Our new test will subscribe to a bucket and then assert that, as operations are performed against the bucket, we receive messages of said events.

Open up `test/kv/bucket_test.exs` and key this in:

```
test "subscribes to puts and deletes" do
  {:ok, bucket} = start_supervised(KV.Bucket)
  KV.Bucket.subscribe(bucket)

  KV.Bucket.put(bucket, "milk", 3)
  assert_receive {:put, "milk", 3}

  # Also check it works even from another process
  spawn(fn -> KV.Bucket.delete(bucket, "milk") end)
  assert_receive {:delete, "milk"}
end
```

In order to make the test pass, we need to implement the `KV.Bucket.subscribe/1`. So let's add these three new functions to `KV.Bucket`:

```
  @doc """
  Subscribes the current process to the bucket.
  """
  def subscribe(bucket) do
    GenServer.cast(bucket, {:subscribe, self()})
  end

  @impl true
  def handle_cast({:subscribe, pid}, state) do
    Process.monitor(pid)
    state = update_in(state.subscribers, &MapSet.put(&1,
pid))
    {:noreply, state}
  end

  @impl true
  def handle_info({:DOWN, _ref, _type, pid, _reason}, state)
do
    state = update_in(state.subscribers, &MapSet.delete(&1,
pid))
    {:noreply, state}
  end
```

On subscription, we send a `cast/2` request with the current process identifier and implement its `handle_cast/2` callback that receives the `request` and the current server state. We then proceed to monitor the given `pid` and add it to the list of subscribers, which we are implementing using `MapSet`. The `handle_cast/2` callback returns a tuple in the format `{:noreply, updated_state}`. Note that in a real application we would have probably implemented it with a synchronous call, as it provides back pressure, instead of an asynchronous cast. We are doing it this way to illustrate how to implement a cast callback.

Then, because we have monitored a process, once that process terminates, we will receive a "DOWN message". GenServers handle regular messages using the `handle_info/2` callback, which also typically return `{:noreply, updated_state}`. In this callback, we remove the PID that terminated from our list of subscribers.

We are almost there. We can see both `handle_cast/2` and `handle_info/2` callbacks assume there is a subscribers key in our state with a `MapSet`. So let's add it by updating the existing `init/1` to the following:

```
@impl true
def init(bucket) do
  state = %{
    bucket: bucket,
    subscribers: MapSet.new()
  }

  {:ok, state}
end
```

And finally let's update the callbacks for `put/3` and `delete/2` to broadcast messages whenever they are invoked, like this:

```
def handle_call({:put, key, value}, _from, state) do
  state = put_in(state.bucket[key], value)
  broadcast(state, {:put, key, value})
  {:reply, :ok, state}
end

def handle_call({:delete, key}, _from, state) do
  {value, state} = pop_in(state.bucket[key])
  broadcast(state, {:delete, key})
  {:reply, value, state}
```

```
  end

  defp broadcast(state, message) do
    for pid <- state.subscribers do
      send(pid, message)
    end
  end
```

There is no need to modify the callback for `get/2`. And that's it, run the tests again, and our new test should pass!

Wiring it all up

Now that our bucket deals with subscriptions, we need to expose this new functionality in our server. Let's once again start with the test.

Open up `test/kv/server_test.exs` and add this new test:

```
  test "subscribes to buckets", %{socket: socket, name:
name} do
    assert send_and_recv(socket, "CREATE #{name}\r\n") ==
"OK\r\n"
    :gen_tcp.send(socket, "SUBSCRIBE #{name}\r\n")

    {:ok, other} = :gen_tcp.connect(~c"localhost", 4040,
@socket_options)

    assert send_and_recv(other, "PUT #{name} milk 3\r\n") ==
"OK\r\n"
    assert :gen_tcp.recv(socket, 0, 1000) == {:ok, "milk SET
TO 3\r\n"}

    assert send_and_recv(other, "DELETE #{name} milk\r\n")
== "OK\r\n"
```

```
    assert :gen_tcp.recv(socket, 0, 1000) == {:ok, "milk
DELETED\r\n"}
  end
```

The test creates a bucket and subscribes to it. Then it opens up another TCP connection to send commands. For each command sent, we expect the subscribed socket to receive a message.

To make the test pass, we need to change `KV.Command` to parse the new `SUBSCRIBE` command and then run it. Open up `lib/kv/commands.ex` and then first change the `parse/1` definition to the following:

```
  def parse(line) do
    case String.split(line) do
      ["SUBSCRIBE", bucket] -> {:ok, {:subscribe, bucket}}
      ["CREATE", bucket] -> {:ok, {:create, bucket}}
      ["GET", bucket, key] -> {:ok, {:get, bucket, key}}
      ["PUT", bucket, key, value] -> {:ok, {:put, bucket,
key, value}}
      ["DELETE", bucket, key] -> {:ok, {:delete, bucket,
key}}
      _ -> {:error, :unknown_command}
    end
  end
```

We added a new clause that converts "SUBSCRIBE" into a tuple. Now we need to match on this tuple within `run/1`. We can do so by adding a new clause at the bottom of `run/1`, with the following code:

```
  def run({:subscribe, bucket}, socket) do
    lookup(bucket, fn pid ->
      KV.Bucket.subscribe(pid)
      :inet.setopts(socket, active: true)
```

```
      receive_messages(socket)
    end)
  end

  defp receive_messages(socket) do
    receive do
      {:put, key, value} ->
        :gen_tcp.send(socket, "#{key} SET TO #{value}\r\n")
        receive_messages(socket)

      {:delete, key} ->
        :gen_tcp.send(socket, "#{key} DELETED\r\n")
        receive_messages(socket)

      {:tcp_closed, ^socket} ->
        {:error, :closed}

      # If we receive any message, including socket writes,
we discard them
      _ ->
        receive_messages(socket)
    end
  end
```

Let's go over it by parts. We use the existing `lookup/2` private function to lookup for a bucket. If one is found, we subscribe the current process to the bucket. Then we call `:inet.setopts(socket, active: true)` (which we will explain soon) and `receive_messages/1`.

`receive_messages/1` awaits for messages from the bucket and then calls itself again, becoming a loop. We match on `{:put, key, value}` and `{:delete, key}` and write to those events to the socket. We also match on `{:tcp_closed, ^socket}`, which is a message that will be delivered if the

TCP socket closes, and use it to abort the loop. We discard any other message.

At this point you may be wondering: where does `{:tcp_closed, ^socket}` come from?

So far, when receiving messages from the socket, we used `:gen_tcp.recv/3` to perform calls that will block the current process until content is available. This is known as "passive mode". However, we can also ask `:gen_tcp` to stream messages to the current process inbox as they arrive, which is known as "active mode", which is exactly what we configured when we called `:inet.setopts(socket, active: true)`. Those messages have the shape `{:tcp, socket, data}`. When the socket is in active mode and it is closed, it delivers a `{:tcp_closed, socket}` message. Once we receive this message, we exit the loop, which will exit the connection process. Since the bucket is monitoring the process, it will automatically remove the subscription too. You could verify this in practice by adding a `COUNT SUBSCRIPTIONS` command that returns the number of subscribers for a given bucket.

In practice, many systems would prefer to call `:inet.setopts(socket, active: :once)` to specify only a single TCP message should be delivered to avoid overflowing message queues. Once the message is received, they call `:inet.setopts/2` again. In our case, we are simply discarding anything that arrives over the socket, so setting `active: true` is equally fine. In all scenarios, the benefit of using active mode is that the process can receive TCP messages as well as messages from other processes at the same time, instead of blocking on `:gen_tcp.recv/3`.

To wrap it all up, you should give our new feature a try in a distributed setting too. Start two `NODES=... PORT=... iex --sname ... -S mix` instances. In one of them, create a bucket. In the other, subscribe to the same bucket. Once you go back to the first shell, you will see that, even as you send commands to the bucket in one machine, the messages will be

streamed to the other one. In other words, our subscription system is also distributed, and all we had to do is to send messages!

`call`, `cast` or `info`?

So far we have used three callbacks: `handle_call/3`, `handle_cast/2` and `handle_info/2`. Here is what we should consider when deciding when to use each:

1. `handle_call/3` must be used for synchronous requests. This should be the default choice as waiting for the server reply is a useful back-pressure mechanism.

2. `handle_cast/2` must be used for asynchronous requests, when you don't care about a reply. A cast does not guarantee the server has received the message and, for this reason, should be used sparingly. For example, the `subscribe/1` function we have defined in this chapter should have used `call/2`. We have used `cast/2` for educational purposes.

3. `handle_info/2` must be used for all other messages a server may receive that are not sent via `GenServer.call/2` or `GenServer.cast/2`, including regular messages sent with `send/2`. The monitoring `:DOWN` messages are an example of this.

To help developers remember the differences between call, cast and info, the supported return values and more, we have a tiny GenServer cheat sheet.

Agents or GenServers?

Before moving forward to the last chapter, you may be wondering: in the future, should you use an `Agent` or a `GenServer`?

As we saw throughout this guide, agents are straight-forward to get started but they are limited in what they can do. Agents are effectively a subset of GenServers. In fact, agents are implemented on top of GenServers. As well as supervisors, the `Registry` module, and many other features you will find in both Erlang and Elixir.

In other words, GenServers are the most essential component for building concurrent and fault-tolerant systems in Elixir. They provide a robust and flexible framework for managing state and coordinating interactions between processes.

For those reasons, many adopt a rule of thumb to never use Agents and jump straight into GenServers instead. On the other hand, others are more than fine with using agents to store a bit of state here and there. Either way, you will be fine!

This is the last feature we have implemented for our distributed key-value store. In the next chapter, we will learn how to package our application before shipping it to production.

Releases

Now that our application is ready, you may be wondering how we can package our application to run in production. After all, all of our code so far depends on Erlang and Elixir versions that are installed in your current system. To achieve this goal, Elixir provides releases.

A release is a self-contained directory that consists of your application code, all of its dependencies, plus the whole Erlang Virtual Machine (VM) and runtime. Once a release is assembled, it can be packaged and deployed to a target as long as the target runs on the same operating system (OS) distribution and version as the machine that assembled the release.

To get started, simply run `mix release` while setting `MIX_ENV=prod`:

```
$ MIX_ENV=prod mix release
Compiling 4 files (.ex)
Generated kv app
* assembling kv-0.1.0 on MIX_ENV=prod
* using config/runtime.exs to configure the release at
runtime

Release created at _build/prod/rel/kv

    # To start your system
    _build/prod/rel/kv/bin/kv start

Once the release is running:

    # To connect to it remotely
    _build/prod/rel/kv/bin/kv remote

    # To stop it gracefully (you may also send
```

```
SIGINT/SIGTERM)
    _build/prod/rel/kv/bin/kv stop

To list all commands:

    _build/prod/rel/kv/bin/kv
```

Excellent! A release was assembled in `_build/prod/rel/kv`. Everything you need to run your application is inside that directory. In particular, there is a `bin/kv` file which is the entry point to your system. It supports multiple commands, such as:

- `bin/kv start`, `bin/kv start_iex`, `bin/kv restart`, and `bin/kv stop` — for general management of the release
- `bin/kv rpc COMMAND` and `bin/kv remote` — for running commands on the running system or to connect to the running system
- `bin/kv eval COMMAND` — to start a fresh system that runs a single command and then shuts down
- `bin/kv daemon` and `bin/kv daemon_iex` — to start the system as a daemon on Unix-like systems
- `bin/kv install` — to install the system as a service on Windows machines

If you run `bin/kv start_iex` inside the release directory, it will start the system using a short name (`--sname`) equal to the release name, which in this case is `kv`. The next step is to start two instances, on different ports and different names, as we did earlier on. But before we do this, let's talk a bit about the benefits of releases.

Why releases?

Releases allow developers to precompile and package all of their code and the runtime into a single unit. The benefits of releases are:

- Code preloading. The VM has two mechanisms for loading code: interactive and embedded. By default, it runs in the interactive mode which dynamically loads modules when they are used for the first time. The first time your application calls `Enum.map/2`, the VM will find the `Enum` module and load it. There's a downside. When you start a new server in production, it may need to load many other modules, causing the first requests to have an unusual spike in response time. Releases run in embedded mode, which loads all available modules upfront, guaranteeing your system is ready to handle requests after booting.

- Configuration and customization. Releases give developers fine grained control over system configuration and the VM flags used to start the system.

- Self-contained. A release does not require the source code to be included in your production artifacts. All of the code is precompiled and packaged. Releases do not even require Erlang or Elixir on your servers, as they include the Erlang VM and its runtime by default. Furthermore, both Erlang and Elixir standard libraries are stripped to bring only the parts you are actually using.

- Multiple releases. You can assemble different releases with different configuration per application or even with different applications altogether.

We have written extensive documentation on releases, so please check the official documentation for more information. For now, we will continue exploring some of the features outlined above.

Configuring releases

Releases also provide built-in hooks for configuring almost every need of the production system:

- `config/config.exs` — provides build-time application configuration, which is executed before our application compiles. This file often imports configuration files based on the environment, such as `config/dev.exs` and `config/prod.exs`.

- `config/runtime.exs` — provides runtime application configuration. It is executed every time the release boots and is further extensible via config providers.

- `rel/env.sh.eex` and `rel/env.bat.eex` — template files that are copied into every release and executed on every command to set up environment variables, including ones specific to the VM, and the general environment.

- `rel/vm.args.eex` — a template file that is copied into every release and provides static configuration of the Erlang Virtual Machine and other runtime flags.

In this case, we already have specified a `config/runtime.exs` that deals with both `PORT` and `NODES` environment variables. Furthermore, while releases don't accept a `--sname` parameter, they do allow us to set the name via the `RELEASE_NODE` env var. Therefore, we can start two copies of the system by jumping into `_build/prod/rel/kv` and typing this (remember to adjust `@computer-name` to your actual computer name):

```
$ NODES="foo@computer-name,bar@computer-name" PORT=4040
RELEASE_NODE="foo" bin/kv start_iex
```

```
$ NODES="foo@computer-name,bar@computer-name" PORT=4041
```

```
RELEASE_NODE="bar" bin/kv start_iex
```

To verify it all worked out, you can type `Node.list` in the IEx section and see if it returns the other node. If it doesn't, you can start diagnosing, first by comparing the node names within each `iex>` prompt and calling `Node.connect/1` directly. With applications running, you can `telnet` into them as usual too.

While the above is enough to get started, you may want to perform advanced configuration based on the environment you are replying to. Releases provide scripts for that, which are great to automate based on host, network, or cloud settings.

Operating System scripts

Every release contains an environment file, named `env.sh` on Unix-like systems and `env.bat` on Windows machines, that executes before the Elixir system starts. In this file, you can execute any OS-level code, such as invoke other applications, set environment variables and so on. Some of those environment variables can even configure how the release itself runs.

For instance, releases run using short-names (`--sname`). However, if you want to actually run a distributed key-value store in production, you will need multiple nodes and start the release with the `--name` option. We can achieve this by setting the `RELEASE_DISTRIBUTION` environment variable inside the `env.sh` and `env.bat` files. Mix already has a template for said files which we can customize, so let's ask Mix to copy them to our application:

```
$ mix release.init
* creating rel/vm.args.eex
* creating rel/remote.vm.args.eex
```

```
* creating rel/env.sh.eex
* creating rel/env.bat.eex
```

If you open up `rel/env.sh.eex`, you will see:

```
#!/bin/sh

# # Sets and enables heart (recommended only in daemon mode)
# case $RELEASE_COMMAND in
#   daemon*)
#     HEART_COMMAND="$RELEASE_ROOT/bin/$RELEASE_NAME
$RELEASE_COMMAND"
#     export HEART_COMMAND
#     export ELIXIR_ERL_OPTIONS="-heart"
#     ;;
#   *)
#     ;;
# esac

# # Set the release to load code on demand (interactive)
instead of preloading (embedded).
# export RELEASE_MODE=interactive

# # Set the release to work across nodes.
# # RELEASE_DISTRIBUTION must be "sname" (local), "name"
(distributed) or "none".
# export RELEASE_DISTRIBUTION=name
# export RELEASE_NODE=<%= @release.name %>
```

The steps necessary to work across nodes is already commented out as an example. You can enable full distribution by setting the `RELEASE_DISTRIBUTION` variable to `name`.

If you are on Windows, you will have to open up `rel/env.bat.eex`, where you will find this:

```
@echo off
rem Set the release to load code on demand (interactive)
instead of preloading (embedded).
rem set RELEASE_MODE=interactive

rem Set the release to work across nodes.
rem RELEASE_DISTRIBUTION must be "sname" (local), "name"
(distributed) or "none".
rem set RELEASE_DISTRIBUTION=name
rem set RELEASE_NODE=<%= @release.name %>
```

Once again, set the `RELEASE_DISTRIBUTION` variable to `name` and you are good to go!

VM arguments

The `rel/vm.args.eex` allows you to specify low-level flags that control how the Erlang VM and its runtime operate. You specify entries as if you were specifying arguments in the command line with code comments also supported. Here is the default generated file:

```
## Customize flags given to the VM:
https://www.erlang.org/doc/man/erl.html
## -mode/-name/-sname/-setcookie are configured via env
vars, do not set them here

## Increase number of concurrent ports/sockets
##+Q 65536
```

```
## Tweak GC to run more often
##-env ERL_FULLSWEEP_AFTER 10
```

You can see a complete list of VM arguments and flags in the Erlang documentation.

Summing up

Throughout the guide, we have built a very simple distributed key-value store as an opportunity to explore many constructs like generic servers, supervisors, tasks, agents, applications and more. Not only that, we have written tests for the whole application, got familiar with ExUnit, and learned how to use the Mix build tool to accomplish a wide range of tasks.

If you are looking for a distributed key-value store to use in production, you should definitely look into Riak, which also runs in the Erlang VM. In Riak, the buckets are replicated and stored across several nodes to avoid data loss.

Of course, Elixir can be used for much more than distributed key-value stores. Embedded systems, data-processing and data-ingestion, web applications, audio/video streaming systems, machine learning, and others are many of the different domains Elixir excels at. We hope this guide has prepared you to explore any of those domains or any future domain you may desire to bring Elixir into.

Happy coding!

What are anti-patterns?

Anti-patterns describe common mistakes or indicators of problems in code. They are also known as "code smells".

The goal of these guides is to document potential anti-patterns found in Elixir software and teach developers how to identify them and their pitfalls. If an existing piece of code matches an anti-pattern, it does not mean your code must be rewritten. Sometimes, even if a snippet matches a potential anti-pattern and its limitations, it may be the best approach to the problem at hand. No codebase is free of anti-patterns and one should not aim to remove all of them.

The anti-patterns in these guides are broken into 4 main categories:

- **Code-related anti-patterns:** related to your code and particular language idioms and features;
- **Design-related anti-patterns:** related to your modules, functions, and the role they play within a codebase;
- **Process-related anti-patterns:** related to processes and process-based abstractions;
- **Meta-programming anti-patterns:** related to meta-programming.

Each anti-pattern is documented using the following structure:

- **Name:** Unique identifier of the anti-pattern. This name is important to facilitate communication between developers;
- **Problem:** How the anti-pattern can harm code quality and what impacts this can have for developers;

- **Example:** Code and textual descriptions to illustrate the occurrence of the anti-pattern;

- **Refactoring:** Ways to change your code to improve its qualities. Examples of refactored code are presented to illustrate these changes.

An additional section with "Additional Remarks" may be provided. Those may include known scenarios where the anti-pattern does not apply.

The initial catalog of anti-patterns was proposed by Lucas Vegi and Marco Tulio Valente, from ASERG/DCC/UFMG. For more info, see Understanding Code Smells in Elixir Functional Language and the associated code repository.

Additionally, the Security Working Group of the Erlang Ecosystem Foundation publishes documents with security resources and best-practices of both Erlang and Elixir, including detailed guides for web applications.

Code-related anti-patterns

This document outlines potential anti-patterns related to your code and particular Elixir idioms and features.

Comments overuse

Problem

When you overuse comments or comment self-explanatory code, it can have the effect of making code *less readable*.

Example

```elixir
# Returns the Unix timestamp of 5 minutes from the current
time
defp unix_five_min_from_now do
  # Get the current time
  now = DateTime.utc_now()

  # Convert it to a Unix timestamp
  unix_now = DateTime.to_unix(now, :second)

  # Add five minutes in seconds
  unix_now + (60 * 5)
end
```

Refactoring

Prefer clear and self-explanatory function names, module names, and variable names when possible. In the example above, the function name

explains well what the function does, so you likely won't need the comment before it. The code also explains the operations well through variable names and clear function calls.

You could refactor the code above like this:

```
@five_min_in_seconds 60 * 5

defp unix_five_min_from_now do
  now = DateTime.utc_now()
  unix_now = DateTime.to_unix(now, :second)
  unix_now + @five_min_in_seconds
end
```

We removed the unnecessary comments. We also added a `@five_min_in_seconds` module attribute, which serves the additional purpose of giving a name to the "magic" number `60 * 5`, making the code clearer and more expressive.

Additional remarks

Elixir makes a clear distinction between **documentation** and code comments. The language has built-in first-class support for documentation through `@doc`, `@moduledoc`, and more. See the "Writing documentation" guide for more information.

Complex `else` clauses in `with`

Problem

This anti-pattern refers to `with` expressions that flatten all its error clauses into a single complex `else` block. This situation is harmful to the code

readability and maintainability because it's difficult to know from which clause the error value came.

Example

An example of this anti-pattern, as shown below, is a function `open_decoded_file/1` that reads a Base64-encoded string content from a file and returns a decoded binary string. This function uses a `with` expression that needs to handle two possible errors, all of which are concentrated in a single complex `else` block.

```
def open_decoded_file(path) do
  with {:ok, encoded} <- File.read(path),
       {:ok, decoded} <- Base.decode64(encoded) do
    {:ok, String.trim(decoded)}
  else
    {:error, _} -> {:error, :badfile}
    :error -> {:error, :badencoding}
  end
end
```

In the code above, it is unclear how each pattern on the left side of `<-` relates to their error at the end. The more patterns in a `with`, the less clear the code gets, and the more likely it is that unrelated failures will overlap each other.

Refactoring

In this situation, instead of concentrating all error handling within a single complex `else` block, it is better to normalize the return types in specific private functions. In this way, `with` can focus on the success case and the errors are normalized closer to where they happen, leading to better organized and maintainable code.

```elixir
def open_decoded_file(path) do
  with {:ok, encoded} <- file_read(path),
       {:ok, decoded} <- base_decode64(encoded) do
    {:ok, String.trim(decoded)}
  end
end

defp file_read(path) do
  case File.read(path) do
    {:ok, contents} -> {:ok, contents}
    {:error, _} -> {:error, :badfile}
  end
end

defp base_decode64(contents) do
  case Base.decode64(contents) do
    {:ok, decoded} -> {:ok, decoded}
    :error -> {:error, :badencoding}
  end
end
```

Complex extractions in clauses

Problem

When we use multi-clause functions, it is possible to extract values in the clauses for further usage and for pattern matching/guard checking. This extraction itself does not represent an anti-pattern, but when you have *extractions made across several clauses and several arguments of the same function*, it becomes hard to know which extracted parts are used for pattern/guards and what is used only inside the function body. This anti-pattern is related to Unrelated multi-clause function, but with implications of its own. It impairs the code readability in a different way.

Example

The multi-clause function `drive/1` is extracting fields of an `%User{}` struct for usage in the clause expression (`age`) and for usage in the function body (`name`):

```
def drive(%User{name: name, age: age}) when age >= 18 do
  "#{name} can drive"
end

def drive(%User{name: name, age: age}) when age < 18 do
  "#{name} cannot drive"
end
```

While the example above is small and does not constitute an anti-pattern, it is an example of mixed extraction and pattern matching. A situation where `drive/1` was more complex, having many more clauses, arguments, and extractions, would make it hard to know at a glance which variables are used for pattern/guards and which ones are not.

Refactoring

As shown below, a possible solution to this anti-pattern is to extract only pattern/guard related variables in the signature once you have many arguments or multiple clauses:

```
def drive(%User{age: age} = user) when age >= 18 do
  %User{name: name} = user
  "#{name} can drive"
end

def drive(%User{age: age} = user) when age < 18 do
  %User{name: name} = user
```

```
    "#{name} cannot drive"
  end
```

Dynamic atom creation

Problem

An `Atom` is an Elixir basic type whose value is its own name. Atoms are often useful to identify resources or express the state, or result, of an operation. Creating atoms dynamically is not an anti-pattern by itself. However, atoms are not garbage collected by the Erlang Virtual Machine, so values of this type live in memory during a software's entire execution lifetime. The Erlang VM limits the number of atoms that can exist in an application by default to *1 048 576*, which is more than enough to cover all atoms defined in a program, but attempts to serve as an early limit for applications which are "leaking atoms" through dynamic creation.

For these reasons, creating atoms dynamically can be considered an anti-pattern when the developer has no control over how many atoms will be created during the software execution. This unpredictable scenario can expose the software to unexpected behavior caused by excessive memory usage, or even by reaching the maximum number of *atoms* possible.

Example

Picture yourself implementing code that converts string values into atoms. These strings could have been received from an external system, either as part of a request into our application, or as part of a response to your application. This dynamic and unpredictable scenario poses a security risk, as these uncontrolled conversions can potentially trigger out-of-memory errors.

```
defmodule MyRequestHandler do
  def parse(%{"status" => status, "message" => message} =
_payload) do
    %{status: String.to_atom(status), message: message}
  end
end
```

```
iex> MyRequestHandler.parse(%{"status" => "ok", "message" =>
"all good"})
%{status: :ok, message: "all good"}
```

When we use the `String.to_atom/1` function to dynamically create an atom, it essentially gains potential access to create arbitrary atoms in our system, causing us to lose control over adhering to the limits established by the BEAM. This issue could be exploited by someone to create enough atoms to shut down a system.

Refactoring

To eliminate this anti-pattern, developers must either perform explicit conversions by mapping strings to atoms or replace the use of `String.to_atom/1` with `String.to_existing_atom/1`. An explicit conversion could be done as follows:

```
defmodule MyRequestHandler do
  def parse(%{"status" => status, "message" => message} =
_payload) do
    %{status: convert_status(status), message: message}
  end

  defp convert_status("ok"), do: :ok
  defp convert_status("error"), do: :error
```

```
  defp convert_status("redirect"), do: :redirect
end
```

```
iex> MyRequestHandler.parse(%{"status" =>
"status_not_seen_anywhere", "message" => "all good"})
** (FunctionClauseError) no function clause matching in
MyRequestHandler.convert_status/1
```

By explicitly listing all supported statuses, you guarantee only a limited number of conversions may happen. Passing an invalid status will lead to a function clause error.

An alternative is to use `String.to_existing_atom/1`, which will only convert a string to atom if the atom already exists in the system:

```
defmodule MyRequestHandler do
  def parse(%{"status" => status, "message" => message} =
_payload) do
    %{status: String.to_existing_atom(status), message:
message}
  end
end
```

```
iex> MyRequestHandler.parse(%{"status" =>
"status_not_seen_anywhere", "message" => "all good"})
** (ArgumentError) errors were found at the given arguments:

  * 1st argument: not an already existing atom
```

In such cases, passing an unknown status will raise as long as the status was not defined anywhere as an atom in the system. However, assuming `status` can be either `:ok`, `:error`, or `:redirect`, how can you guarantee

those atoms exist? You must ensure those atoms exist somewhere **in the same module** where `String.to_existing_atom/1` is called. For example, if you had this code:

```
defmodule MyRequestHandler do
  def parse(%{"status" => status, "message" => message} =
_payload) do
    %{status: String.to_existing_atom(status), message:
message}
  end

  def handle(%{status: status}) do
    case status do
      :ok -> ...
      :error -> ...
      :redirect -> ...
    end
  end
end
```

All valid statuses are defined as atoms within the same module, and that's enough. If you want to be explicit, you could also have a function that lists them:

```
def valid_statuses do
  [:ok, :error, :redirect]
end
```

However, keep in mind using a module attribute or defining the atoms in the module body, outside of a function, are not sufficient, as the module body is only executed during compilation and it is not necessarily part of the compiled module loaded at runtime.

Long parameter list

Problem

In a functional language like Elixir, functions tend to explicitly receive all inputs and return all relevant outputs, instead of relying on mutations or side-effects. As functions grow in complexity, the amount of arguments (parameters) they need to work with may grow, to a point where the function's interface becomes confusing and prone to errors during use.

Example

In the following example, the `loan/6` functions takes too many arguments, causing its interface to be confusing and potentially leading developers to introduce errors during calls to this function.

```
defmodule Library do
  # Too many parameters that can be grouped!
  def loan(user_name, email, password, user_alias,
book_title, book_ed) do
    ...
  end
end
```

Refactoring

To address this anti-pattern, related arguments can be grouped using key-value data structures, such as maps, structs, or even keyword lists in the case of optional arguments. This effectively reduces the number of arguments and the key-value data structures adds clarity to the caller.

For this particular example, the arguments to `loan/6` can be grouped into two different maps, thereby reducing its arity to `loan/2`:

```
defmodule Library do
  def loan(%{name: name, email: email, password: password,
alias: alias} = user, %{title: title, ed: ed} = book) do
    ...
  end
end
```

In some cases, the function with too many arguments may be a private function, which gives us more flexibility over how to separate the function arguments. One possible suggestion for such scenarios is to split the arguments in two maps (or tuples): one map keeps the data that may change, and the other keeps the data that won't change (read-only). This gives us a mechanical option to refactor the code.

Other times, a function may legitimately take half a dozen or more completely unrelated arguments. This may suggest the function is trying to do too much and would be better broken into multiple functions, each responsible for a smaller piece of the overall responsibility.

Namespace trespassing

Problem

This anti-pattern manifests when a package author or a library defines modules outside of its "namespace". A library should use its name as a "prefix" for all of its modules. For example, a package named `:my_lib` should define all of its modules within the `MyLib` namespace, such as `MyLib.User`, `MyLib.SubModule`, `MyLib.Application`, and `MyLib` itself.

This is important because the Erlang VM can only load one instance of a module at a time. So if there are multiple libraries that define the same module, then they are incompatible with each other due to this limitation. By always using the library name as a prefix, it avoids module name clashes due to the unique prefix.

Example

This problem commonly manifests when writing an extension of another library. For example, imagine you are writing a package that adds authentication to Plug called `:plug_auth`. You must avoid defining modules within the `Plug` namespace:

```
defmodule Plug.Auth do
  # ...
end
```

Even if `Plug` does not currently define a `Plug.Auth` module, it may add such a module in the future, which would ultimately conflict with `plug_auth`'s definition.

Refactoring

Given the package is named `:plug_auth`, it must define modules inside the `PlugAuth` namespace:

```
defmodule PlugAuth do
  # ...
end
```

Additional remarks

There are few known exceptions to this anti-pattern:

- Protocol implementations are, by design, defined under the protocol namespace

- In some scenarios, the namespace owner may allow exceptions to this rule. For example, in Elixir itself, you defined custom Mix tasks by placing them under the `Mix.Tasks` namespace, such as `Mix.Tasks.PlugAuth`

- If you are the maintainer for both `plug` and `plug_auth`, then you may allow `plug_auth` to define modules with the `Plug` namespace, such as `Plug.Auth`. However, you are responsible for avoiding or managing any conflicts that may arise in the future

Non-assertive map access

Problem

In Elixir, it is possible to access values from `Map`s, which are key-value data structures, either statically or dynamically.

When a key is expected to exist in a map, it must be accessed using the `map.key` notation, making it clear to developers (and the compiler) that the key must exist. If the key does not exist, an exception is raised (and in some cases also compiler warnings). This is also known as the static notation, as the key is known at the time of writing the code.

When a key is optional, the `map[:key]` notation must be used instead. This way, if the informed key does not exist, `nil` is returned. This is the dynamic notation, as it also supports dynamic key access, such as `map[some_var]`.

When you use `map[:key]` to access a key that always exists in the map, you are making the code less clear for developers and for the compiler, as they now need to work with the assumption the key may not be there. This mismatch may also make it harder to track certain bugs. If the key is unexpectedly missing, you will have a `nil` value propagate through the system, instead of raising on map access.

Table: Comparison of map access notations

Access notation	Key exists	Key doesn't exist	Use case
`map.key`	Returns the value	Raises `KeyError`	Structs and maps with known atom keys
`map[:key]`	Returns the value	Returns `nil`	Any `Access`-based data structure, optional keys

Example

The function `plot/1` tries to draw a graphic to represent the position of a point in a Cartesian plane. This function receives a parameter of `Map` type with the point attributes, which can be a point of a 2D or 3D Cartesian coordinate system. This function uses dynamic access to retrieve values for the map keys:

```
defmodule Graphics do
  def plot(point) do
    # Some other code...
    {point[:x], point[:y], point[:z]}
  end
end
```

```
iex> point_2d = %{x: 2, y: 3}
%{x: 2, y: 3}
iex> point_3d = %{x: 5, y: 6, z: 7}
%{x: 5, y: 6, z: 7}
iex> Graphics.plot(point_2d)
{2, 3, nil}
iex> Graphics.plot(point_3d)
{5, 6, 7}
```

Given we want to plot both 2D and 3D points, the behavior above is expected. But what happens if we forget to pass a point with either `:x` or `:y`?

```
iex> bad_point = %{y: 3, z: 4}
%{y: 3, z: 4}
iex> Graphics.plot(bad_point)
{nil, 3, 4}
```

The behavior above is unexpected because our function should not work with points without a `:x` key. This leads to subtle bugs, as we may now pass `nil` to another function, instead of raising early on, as shown next:

```
iex> point_without_x = %{y: 10}
%{y: 10}
iex> {x, y, _} = Graphics.plot(point_without_x)
{nil, 10, nil}
iex> distance_from_origin = :math.sqrt(x * x + y * y)
** (ArithmeticError) bad argument in arithmetic expression
    :erlang.*(nil, nil)
```

The error above occurs later in the code because `nil` (from missing `:x`) is invalid for arithmetic operations, making it harder to identify the original issue.

Refactoring

To remove this anti-pattern, we must use the dynamic `map[:key]` syntax and the static `map.key` notation according to our requirements. We expect `:x` and `:y` to always exist, but not `:z`. The next code illustrates the refactoring of `plot/1`, removing this anti-pattern:

```elixir
defmodule Graphics do
  def plot(point) do
    # Some other code...
    {point.x, point.y, point[:z]}
  end
end
```

```
iex> Graphics.plot(point_2d)
{2, 3, nil}
iex> Graphics.plot(bad_point)
** (KeyError) key :x not found in: %{y: 3, z: 4}
  graphic.ex:4: Graphics.plot/1
```

This is beneficial because:

1. It makes your expectations clear to others reading the code
2. It fails fast when required data is missing
3. It allows the compiler to provide warnings when accessing non-existent fields, particularly in compile-time structures like structs

Overall, the usage of `map.key` and `map[:key]` encode important information about your data structure, allowing developers to be clear about their intent. The `Access` module documentation also provides useful reference on this topic. You can also consider the `Map` module when working with maps of any keys, which contains functions for fetching keys (with or without default values), updating and removing keys, traversals, and more.

An alternative to refactor this anti-pattern is to use pattern matching, defining explicit clauses for 2D vs 3D points:

```elixir
defmodule Graphics do
  # 3d
```

```
  def plot(%{x: x, y: y, z: z}) do
    # Some other code...
    {x, y, z}
  end

  # 2d
  def plot(%{x: x, y: y}) do
    # Some other code...
    {x, y}
  end
end
```

Pattern-matching is specially useful when matching over multiple keys as well as on the values themselves at once. In the example above, the code will not only extract the values but also verify that the required keys exist. If we try to call `plot/1` with a map that doesn't have the required keys, we'll get a `FunctionClauseError`:

```
iex> incomplete_point = %{x: 5}
%{x: 5}
iex> Graphics.plot(incomplete_point)
** (FunctionClauseError) no function clause matching in
Graphics.plot/1

    The following arguments were given to Graphics.plot/1:

        # 1
        %{x: 5}
```

Another option is to use structs. By default, structs only support static access to its fields. In such scenarios, you may consider defining structs for both 2D and 3D points:

```elixir
defmodule Point2D do
  @enforce_keys [:x, :y]
  defstruct [x: nil, y: nil]
end
```

Generally speaking, structs are useful when sharing data structures across modules, at the cost of adding a compile time dependency between these modules. If module `A` uses a struct defined in module `B`, `A` must be recompiled if the fields in the struct `B` change.

In summary, Elixir provides several ways to access map values, each with different behaviors:

1. **Static access** (`map.key`): Fails fast when keys are missing, ideal for structs and maps with known atom keys
2. **Dynamic access** (`map[:key]`): Works with any `Access` data structure, suitable for optional fields, returns nil for missing keys
3. **Pattern matching**: Provides a powerful way to both extract values and ensure required map/struct keys exist in one operation

Choosing the right approach depends if the keys are known upfront or not. Static access and pattern matching are mostly equivalent (although pattern matching allows you to match on multiple keys at once, including matching on the struct name).

Additional remarks

This anti-pattern was formerly known as Accessing non-existent map/struct fields.

Non-assertive pattern matching

Problem

Overall, Elixir systems are composed of many supervised processes, so the effects of an error are localized to a single process, and don't propagate to the entire application. A supervisor detects the failing process, reports it, and possibly restarts it. This anti-pattern arises when developers write defensive or imprecise code, capable of returning incorrect values which were not planned for, instead of programming in an assertive style through pattern matching and guards.

Example

The function `get_value/2` tries to extract a value from a specific key of a URL query string. As it is not implemented using pattern matching, `get_value/2` always returns a value, regardless of the format of the URL query string passed as a parameter in the call. Sometimes the returned value will be valid. However, if a URL query string with an unexpected format is used in the call, `get_value/2` will extract incorrect values from it:

```
defmodule Extract do
  def get_value(string, desired_key) do
    parts = String.split(string, "&")

    Enum.find_value(parts, fn pair ->
      key_value = String.split(pair, "=")
      Enum.at(key_value, 0) == desired_key &&
Enum.at(key_value, 1)
    end)
  end
end


# URL query string with the planned format - OK!
iex>
Extract.get_value("name=Lucas&university=UFMG&lab=ASERG",
"lab")
```

```
"ASERG"
iex>
Extract.get_value("name=Lucas&university=UFMG&lab=ASERG",
"university")
"UFMG"
# Unplanned URL query string format - Unplanned value
extraction!
iex>
Extract.get_value("name=Lucas&university=institution=UFMG&la
b=ASERG", "university")
"institution"   # <= why not "institution=UFMG"? or only
"UFMG"?
```

Refactoring

To remove this anti-pattern, `get_value/2` can be refactored through the use of pattern matching. So, if an unexpected URL query string format is used, the function will crash instead of returning an invalid value. This behavior, shown below, allows clients to decide how to handle these errors and doesn't give a false impression that the code is working correctly when unexpected values are extracted:

```
defmodule Extract do
  def get_value(string, desired_key) do
    parts = String.split(string, "&")

    Enum.find_value(parts, fn pair ->
      [key, value] = String.split(pair, "=") # <= pattern
matching
      key == desired_key && value
    end)
```

```
  end
end
```

```
# URL query string with the planned format - OK!
iex>
Extract.get_value("name=Lucas&university=UFMG&lab=ASERG",
"name")
"Lucas"
# Unplanned URL query string format - Crash explaining the
problem to the client!
iex>
Extract.get_value("name=Lucas&university=institution=UFMG&la
b=ASERG", "university")
** (MatchError) no match of right hand side value:
["university", "institution", "UFMG"]
  extract.ex:7: anonymous fn/2 in Extract.get_value/2 # <=
left hand: [key, value] pair
iex> Extract.get_value("name=Lucas&university&lab=ASERG",
"university")
** (MatchError) no match of right hand side value:
["university"]
  extract.ex:7: anonymous fn/2 in Extract.get_value/2 # <=
left hand: [key, value] pair
```

Elixir and pattern matching promote an assertive style of programming where you handle the known cases. Once an unexpected scenario arises, you can decide to address it accordingly based on practical examples, or conclude the scenario is indeed invalid and the exception is the desired choice.

`case/2` is another important construct in Elixir that help us write assertive code, by matching on specific patterns. For example, if a function returns `{:ok, ...}` or `{:error, ...}`, prefer to explicitly match on both patterns:

```
case some_function(arg) do
  {:ok, value} -> # ...
  {:error, _} -> # ...
end
```

In particular, avoid matching solely on `_`, as shown below:

```
case some_function(arg) do
  {:ok, value} -> # ...
  _ -> # ...
end
```

Matching on `_` is less clear in intent and it may hide bugs if `some_function/1` adds new return values in the future.

Additional remarks

This anti-pattern was formerly known as Speculative assumptions.

Non-assertive truthiness

Problem

Elixir provides the concept of truthiness: `nil` and `false` are considered "falsy" and all other values are "truthy". Many constructs in the language, such as `&&/2`, `||/2`, and `!/1` handle truthy and falsy values. Using those operators is not an anti-pattern. However, using those operators when all operands are expected to be booleans, may be an anti-pattern.

Example

The simplest scenario where this anti-pattern manifests is in conditionals, such as:

```
if is_binary(name) && is_integer(age) do
  # ...
else
  # ...
end
```

Given both operands of `&&/2` are booleans, the code is more generic than necessary, and potentially unclear.

Refactoring

To remove this anti-pattern, we can replace `&&/2`, `||/2`, and `!/1` by `and/2`, `or/2`, and `not/1` respectively. These operators assert at least their first argument is a boolean:

```
if is_binary(name) and is_integer(age) do
  # ...
else
  # ...
end
```

This technique may be particularly important when working with Erlang code. Erlang does not have the concept of truthiness. It never returns `nil`, instead its functions may return `:error` or `:undefined` in places an Elixir developer would return `nil`. Therefore, to avoid accidentally interpreting `:undefined` or `:error` as a truthy value, you may prefer to use `and/2`, `or/2`, and `not/1` exclusively when interfacing with Erlang APIs.

Structs with 32 fields or more

Problem

Structs in Elixir are implemented as compile-time maps, which have a predefined amount of fields. When structs have 32 or more fields, their internal representation in the Erlang Virtual Machines changes, potentially leading to bloating and higher memory usage.

Example

Any struct with 32 or more fields will be problematic:

```
defmodule MyExample do
  defstruct [
    :field1,
    :field2,
    ...,
    :field35
  ]
end
```

The Erlang VM has two internal representations for maps: a flat map and a hash map. A flat map is represented internally as two tuples: one tuple containing the keys and another tuple holding the values. Whenever you update a flat map, the tuple keys are shared, reducing the amount of memory used by the update. A hash map has a more complex structure, which is efficient for a large amount of keys, but it does not share the key space.

Maps of up to 32 keys are represented as flat maps. All others are hash map. Structs *are* maps (with a metadata field called `__struct__`) and so any struct with fewer than 32 fields is represented as a flat map. This allows us to optimize several struct operations, as we never add or remove fields to structs, we simply update them.

Furthermore, structs of the same name "instantiated" in the same module will share the same "tuple keys" at compilation times, as long as they have fewer than 32 fields. For example, in the following code:

```
defmodule Example do
  def users do
    [%User{name: "John"}, %User{name: "Meg"}, ...]
  end
end
```

All user structs will point to the same tuple keys at compile-time, also reducing the memory cost of instantiating structs with `%MyStruct{...}` notation. This optimization is also not available if the struct has 32 keys or more.

Refactoring

Removing this anti-pattern, in a nutshell, requires ensuring your struct has fewer than 32 fields. There are a few techniques you could apply:

- If the struct has "optional" fields, for example, fields which are initialized with nil, you could nest all optional fields into other field, called `:metadata`, `:optionals`, or similar. This could lead to benefits such as being able to use pattern matching to check if a field exists or not, instead of relying on `nil` values

- You could nest structs, by storing structs within other fields. Fields that are rarely read or written to are good candidates to be moved to a nested struct

- You could nest fields as tuples. For example, if two fields are always read or updated together, they could be moved to a tuple (or another composite data structure)

The challenge is to balance the changes above with API ergonomics, in particular, when fields may be frequently read and written to.

Design-related anti-patterns

This document outlines potential anti-patterns related to your modules, functions, and the role they play within a codebase.

Alternative return types

Problem

This anti-pattern refers to functions that receive options (typically as a *keyword list* parameter) that drastically change their return type. Because options are optional and sometimes set dynamically, if they also change the return type, it may be hard to understand what the function actually returns.

Example

An example of this anti-pattern, as shown below, is when a function has many alternative return types, depending on the options received as a parameter.

```
defmodule AlternativeInteger do
  @spec parse(String.t(), keyword()) :: integer() |
{integer(), String.t()} | :error
  def parse(string, options \\ []) when is_list(options) do
    if Keyword.get(options, :discard_rest, false) do
      case Integer.parse(string) do
        {int, _rest} -> int
        :error -> :error
      end
    else
      Integer.parse(string)
    end
```

```
  end
end
```

```
iex> AlternativeInteger.parse("13")
{13, ""}
iex> AlternativeInteger.parse("13", discard_rest: false)
{13, ""}
iex> AlternativeInteger.parse("13", discard_rest: true)
13
```

Refactoring

To refactor this anti-pattern, as shown next, add a specific function for each return type (for example, `parse_discard_rest/1`), no longer delegating this to options passed as arguments.

```
defmodule AlternativeInteger do
  @spec parse(String.t()) :: {integer(), String.t()} |
:error
  def parse(string) do
    Integer.parse(string)
  end

  @spec parse_discard_rest(String.t()) :: integer() | :error
  def parse_discard_rest(string) do
    case Integer.parse(string) do
      {int, _rest} -> int
      :error -> :error
    end
  end
end
```

```
iex> AlternativeInteger.parse("13")
{13, ""}
iex> AlternativeInteger.parse_discard_rest("13")
13
```

Boolean obsession

Problem

This anti-pattern happens when booleans are used instead of atoms to encode information. The usage of booleans themselves is not an anti-pattern, but whenever multiple booleans are used with overlapping states, replacing the booleans by atoms (or composite data types such as *tuples*) may lead to clearer code.

This is a special case of *Primitive obsession*, specific to boolean values.

Example

An example of this anti-pattern is a function that receives two or more options, such as `editor: true` and `admin: true`, to configure its behavior in overlapping ways. In the code below, the `:editor` option has no effect if `:admin` is set, meaning that the `:admin` option has higher priority than `:editor`, and they are ultimately related.

```
defmodule MyApp do
  def process(invoice, options \\ []) do
    cond do
      options[:admin] ->  # Is an admin
      options[:editor] -> # Is an editor
      true ->             # Is none
    end
```

```
  end
end
```

Refactoring

Instead of using multiple options, the code above could be refactored to receive a single option, called `:role`, that can be either `:admin`, `:editor`, or `:default`:

```
defmodule MyApp do
  def process(invoice, options \\ []) do
    case Keyword.get(options, :role, :default) do
      :admin ->   # Is an admin
      :editor ->  # Is an editor
      :default -> # Is none
    end
  end
end
```

This anti-pattern may also happen in our own data structures. For example, we may define a `User` struct with two boolean fields, `:editor` and `:admin`, while a single field named `:role` may be preferred.

Finally, it is worth noting that using atoms may be preferred even when we have a single boolean argument/option. For example, consider an invoice which may be set as approved/unapproved. One option is to provide a function that expects a boolean:

```
MyApp.update(invoice, approved: true)
```

However, using atoms may read better and make it simpler to add further states (such as pending) in the future:

```
MyApp.update(invoice, status: :approved)
```

Remember booleans are internally represented as atoms. Therefore there is no performance penalty in one approach over the other.

Exceptions for control-flow

Problem

This anti-pattern refers to code that uses `Exception`s for control flow. Exception handling itself does not represent an anti-pattern, but developers must prefer to use `case` and pattern matching to change the flow of their code, instead of `try/rescue`. In turn, library authors should provide developers with APIs to handle errors without relying on exception handling. When developers have no freedom to decide if an error is exceptional or not, this is considered an anti-pattern.

Example

An example of this anti-pattern, as shown below, is using `try/rescue` to deal with file operations:

```
defmodule MyModule do
  def print_file(file) do
    try do
      IO.puts(File.read!(file))
    rescue
      e -> IO.puts(:stderr, Exception.message(e))
    end
  end
end
```

```
iex> MyModule.print_file("valid_file")
This is a valid file!
:ok
iex> MyModule.print_file("invalid_file")
could not read file "invalid_file": no such file or
directory
:ok
```

Refactoring

To refactor this anti-pattern, as shown next, use `File.read/1`, which returns tuples instead of raising when a file cannot be read:

```
defmodule MyModule do
  def print_file(file) do
    case File.read(file) do
      {:ok, binary} -> IO.puts(binary)
      {:error, reason} -> IO.puts(:stderr, "could not read
file #{file}: #{reason}")
    end
  end
end
```

This is only possible because the `File` module provides APIs for reading files with tuples as results (`File.read/1`), as well as a version that raises an exception (`File.read!/1`). The bang (exclamation point) is effectively part of Elixir's naming conventions.

Library authors are encouraged to follow the same practices. In practice, the bang variant is implemented on top of the non-raising version of the code. For example, `File.read!/1` is implemented as:

```elixir
def read!(path) do
  case read(path) do
    {:ok, binary} ->
      binary

    {:error, reason} ->
      raise File.Error, reason: reason, action: "read file",
path: IO.chardata_to_string(path)
  end
end
```

A common practice followed by the community is to make the non-raising version return `{:ok, result}` or `{:error, Exception.t}`. For example, an HTTP client may return `{:ok, %HTTP.Response{}}` on success cases and `{:error, %HTTP.Error{}}` for failures, where `HTTP.Error` is implemented as an exception. This makes it convenient for anyone to raise an exception by simply calling `Kernel.raise/1`.

Additional remarks

This anti-pattern is of special importance to library authors and whenever writing functions that will be invoked by other developers and third-party code. Nevertheless, there are still scenarios where developers can afford to raise exceptions directly, for example:

- invalid arguments: it is expected that functions will raise for invalid arguments, as those are structural error and not semantic errors. For example, `File.read(123)` will always raise, because `123` is never a valid filename

- during tests, scripts, etc: those are common scenarios where you want your code to fail as soon as possible in case of errors. Using `!` functions, such as `File.read!/1`, allows you to do so quickly and with clear error messages

- some frameworks, such as Phoenix, allow developers to raise exceptions in their code and uses a protocol to convert these exceptions into semantic HTTP responses

This anti-pattern was formerly known as Using exceptions for control-flow.

Primitive obsession

Problem

This anti-pattern happens when Elixir basic types (for example, *integer*, *float*, and *string*) are excessively used to carry structured information, rather than creating specific composite data types (for example, *tuples*, *maps*, and *structs*) that can better represent a domain.

Example

An example of this anti-pattern is the use of a single *string* to represent an `Address`. An `Address` is a more complex structure than a simple basic (aka, primitive) value.

```
defmodule MyApp do
  def extract_postal_code(address) when is_binary(address)
do
    # Extract postal code with address...
  end

  def fill_in_country(address) when is_binary(address) do
    # Fill in missing country...
  end
end
```

While you may receive the `address` as a string from a database, web request, or a third-party, if you find yourself frequently manipulating or

extracting information from the string, it is a good indicator you should convert the address into structured data:

Another example of this anti-pattern is using floating numbers to model money and currency, when richer data structures should be preferred.

Refactoring

Possible solutions to this anti-pattern is to use maps or structs to model our address. The example below creates an `Address` struct, better representing this domain through a composite type. Additionally, we introduce a `parse/1` function, that converts the string into an `Address`, which will simplify the logic of remaining functions. With this modification, we can extract each field of this composite type individually when needed.

```
defmodule Address do
  defstruct [:street, :city, :state, :postal_code, :country]
end


defmodule MyApp do
  def parse(address) when is_binary(address) do
    # Returns %Address{}
  end

  def extract_postal_code(%Address{} = address) do
    # Extract postal code with address...
  end

  def fill_in_country(%Address{} = address) do
    # Fill in missing country...
  end
end
```

Unrelated multi-clause function

Problem

Using multi-clause functions is a powerful Elixir feature. However, some developers may abuse this feature to group *unrelated* functionality, which is an anti-pattern.

Example

A frequent example of this usage of multi-clause functions occurs when developers mix unrelated business logic into the same function definition, in a way that the behavior of each clause becomes completely distinct from the others. Such functions often have too broad specifications, making it difficult for other developers to understand and maintain them.

Some developers may use documentation mechanisms such as `@doc` annotations to compensate for poor code readability, however the documentation itself may end-up full of conditionals to describe how the function behaves for each different argument combination. This is a good indicator that the clauses are ultimately unrelated.

```
@doc """
Updates a struct.

If given a product, it will...

If given an animal, it will...
"""
def update(%Product{count: count, material: material})  do
  # ...
end

def update(%Animal{count: count, skin: skin})  do
```

```
  # ...
end
```

If updating an animal is completely different from updating a product and requires a different set of rules, it may be worth splitting those over different functions or even different modules.

Refactoring

As shown below, a possible solution to this anti-pattern is to break the business rules that are mixed up in a single unrelated multi-clause function in simple functions. Each function can have a specific name and `@doc`, describing its behavior and parameters received. While this refactoring sounds simple, it can impact the function's callers, so be careful!

```
@doc """
Updates a product.

It will...
"""
def update_product(%Product{count: count, material:
material}) do
  # ...
end

@doc """
Updates an animal.

It will...
"""
def update_animal(%Animal{count: count, skin: skin}) do
  # ...
end
```

These functions may still be implemented with multiple clauses, as long as the clauses group related functionality. For example, `update_product` could be in practice implemented as follows:

```
def update_product(%Product{count: 0}) do
  # ...
end

def update_product(%Product{material: material})
    when material in ["metal", "glass"] do
  # ...
end

def update_product(%Product{material: material})
    when material not in ["metal", "glass"] do
  # ...
end
```

You can see this pattern in practice within Elixir itself. The `+/2` operator can add `Integer`s and `Float`s together, but not `String`s, which instead use the `<>/2` operator. In this sense, it is reasonable to handle integers and floats in the same operation, but strings are unrelated enough to deserve their own function.

You will also find examples in Elixir of functions that work with any struct, which would seemingly be an occurrence of this anti-pattern, such as `struct/2`:

```
iex> struct(URI.parse("/foo/bar"), path: "/bar/baz")
%URI{
  scheme: nil,
  userinfo: nil,
  host: nil,
  port: nil,
```

```
  path: "/bar/baz",
  query: nil,
  fragment: nil
}
```

The difference here is that the `struct/2` function behaves precisely the same for any struct given, therefore there is no question of how the function handles different inputs. If the behavior is clear and consistent for all inputs, then the anti-pattern does not take place.

Using application configuration for libraries

Problem

The *application environment* can be used to parameterize global values that can be used in an Elixir system. This mechanism can be very useful and therefore is not considered an anti-pattern by itself. However, library authors should avoid using the application environment to configure their library. The reason is exactly that the application environment is a **global** state, so there can only be a single value for each key in the environment for an application. This makes it impossible for multiple applications depending on the same library to configure the same aspect of the library in different ways.

Example

The `DashSplitter` module represents a library that configures the behavior of its functions through the global application environment. These configurations are concentrated in the *config/config.exs* file, shown below:

```
import Config

config :app_config,
```

```
  parts: 3

import_config "#{config_env()}.exs"
```

One of the functions implemented by the `DashSplitter` library is `split/1`. This function aims to separate a string received via a parameter into a certain number of parts. The character used as a separator in `split/1` is always `"-"` and the number of parts the string is split into is defined globally by the application environment. This value is retrieved by the `split/1` function by calling `Application.fetch_env!/2`, as shown next:

```
defmodule DashSplitter do
  def split(string) when is_binary(string) do
    parts = Application.fetch_env!(:app_config, :parts) # <=
retrieve parameterized value
    String.split(string, "-", parts: parts)             # <=
parts: 3
  end
end
```

Due to this parameterized value used by the `DashSplitter` library, all applications dependent on it can only use the `split/1` function with identical behavior about the number of parts generated by string separation. Currently, this value is equal to 3, as we can see in the use examples shown below:

```
iex> DashSplitter.split("Lucas-Francisco-Vegi")
["Lucas", "Francisco", "Vegi"]
iex> DashSplitter.split("Lucas-Francisco-da-Matta-Vegi")
["Lucas", "Francisco", "da-Matta-Vegi"]
```

Refactoring

To remove this anti-pattern, this type of configuration should be performed using a parameter passed to the function. The code shown below performs the refactoring of the `split/1` function by accepting keyword lists as a new optional parameter. With this new parameter, it is possible to modify the default behavior of the function at the time of its call, allowing multiple different ways of using `split/2` within the same application:

```
defmodule DashSplitter do
  def split(string, opts \\ []) when is_binary(string) and
is_list(opts) do
    parts = Keyword.get(opts, :parts, 2) # <= default config
of parts == 2
    String.split(string, "-", parts: parts)
  end
end
```

```
iex> DashSplitter.split("Lucas-Francisco-da-Matta-Vegi",
[parts: 5])
["Lucas", "Francisco", "da", "Matta", "Vegi"]
iex> DashSplitter.split("Lucas-Francisco-da-Matta-Vegi") #<=
default config is used!
["Lucas", "Francisco-da-Matta-Vegi"]
```

Of course, not all uses of the application environment by libraries are incorrect. One example is using configuration to replace a component (or dependency) of a library by another that must behave the exact same. Consider a library that needs to parse CSV files. The library author may pick one package to use as default parser but allow its users to swap to different implementations via the application environment. At the end of the day, choosing a different CSV parser should not change the outcome, and

library authors can even enforce this by defining behaviours with the exact semantics they expect.

Additional remarks: Supervision trees

In practice, libraries may require additional configuration beyond keyword lists. For example, if a library needs to start a supervision tree, how can the user of said library customize its supervision tree? Given the supervision tree itself is global (as it belongs to the library), library authors may be tempted to use the application configuration once more.

One solution is for the library to provide its own child specification, instead of starting the supervision tree itself. This allows the user to start all necessary processes under its own supervision tree, potentially passing custom configuration options during initialization.

You can see this pattern in practice in projects like Nx and DNS Cluster. These libraries require that you list processes under your own supervision tree:

```
children = [
  {DNSCluster, query: "my.subdomain"}
]
```

In such cases, if the users of `DNSCluster` need to configure DNSCluster per environment, they can be the ones reading from the application environment, without the library forcing them to:

```
children = [
  {DNSCluster, query: Application.get_env(:my_app,
:dns_cluster_query) || :ignore}
]
```

Some libraries, such as Ecto, allow you to pass your application name as an option (called `:otp_app` or similar) and then automatically read the environment from *your* application. While this addresses the issue with the application environment being global, as they read from each individual application, it comes at the cost of some indirection, compared to the example above where users explicitly read their application environment from their own code, whenever desired.

Additional remarks: Compile-time configuration

A similar discussion entails compile-time configuration. What if a library author requires some configuration to be provided at compilation time?

Once again, instead of forcing users of your library to provide compile-time configuration, you may want to allow users of your library to generate the code themselves. That's the approach taken by libraries such as Ecto:

```
defmodule MyApp.Repo do
  use Ecto.Repo, adapter: Ecto.Adapters.Postgres
end
```

Instead of forcing developers to share a single repository, Ecto allows its users to define as many repositories as they want. Given the `:adapter` configuration is required at compile-time, it is a required value on `use Ecto.Repo`. If developers want to configure the adapter per environment, then it is their choice:

```
defmodule MyApp.Repo do
  use Ecto.Repo, adapter: Application.compile_env(:my_app,
:repo_adapter)
end
```

On the other hand, code generation comes with its own anti-patterns, and must be considered carefully. That's to say: while using the application environment for libraries is discouraged, especially compile-time configuration, in some cases they may be the best option. For example, consider a library needs to parse CSV or JSON files to generate code based on data files. In such cases, it is best to provide reasonable defaults and make them customizable via the application environment, instead of asking each user of your library to generate the exact same code.

Additional remarks: Mix tasks

For Mix tasks and related tools, it may be necessary to provide per-project configuration. For example, imagine you have a `:linter` project, which supports setting the output file and the verbosity level. You may choose to configure it through application environment:

```
config :linter,
  output_file: "/path/to/output.json",
  verbosity: 3
```

However, `Mix` allows tasks to read per-project configuration via `Mix.Project.config/0`. In this case, you can configure the `:linter` directly in the `mix.exs` file:

```
def project do
  [
    app: :my_app,
    version: "1.0.0",
    linter: [
      output_file: "/path/to/output.json",
      verbosity: 3
    ],
    ...
```

```
    ]
  end
```

Additionally, if a Mix task is available, you can also accept these options as command line arguments (see `OptionParser`):

```
mix linter --output-file /path/to/output.json --verbosity 3
```

Process-related anti-patterns

This document outlines potential anti-patterns related to processes and process-based abstractions.

Code organization by process

Problem

This anti-pattern refers to code that is unnecessarily organized by processes. A process itself does not represent an anti-pattern, but it should only be used to model runtime properties (such as concurrency, access to shared resources, error isolation, etc). When you use a process for code organization, it can create bottlenecks in the system.

Example

An example of this anti-pattern, as shown below, is a module that implements arithmetic operations (like `add` and `subtract`) by means of a `GenServer` process. If the number of calls to this single process grows, this code organization can compromise the system performance, therefore becoming a bottleneck.

```
defmodule Calculator do
  @moduledoc """
  Calculator that performs basic arithmetic operations.

  This code is unnecessarily organized in a GenServer
process.
  """

  use GenServer
```

```
  def add(a, b, pid) do
    GenServer.call(pid, {:add, a, b})
  end

  def subtract(a, b, pid) do
    GenServer.call(pid, {:subtract, a, b})
  end

  @impl GenServer
  def init(init_arg) do
    {:ok, init_arg}
  end

  @impl GenServer
  def handle_call({:add, a, b}, _from, state) do
    {:reply, a + b, state}
  end

  def handle_call({:subtract, a, b}, _from, state) do
    {:reply, a - b, state}
  end
end


iex> {:ok, pid} = GenServer.start_link(Calculator, :init)
{:ok, #PID<0.132.0>}
iex> Calculator.add(1, 5, pid)
6
iex> Calculator.subtract(2, 3, pid)
-1
```

Refactoring

In Elixir, as shown next, code organization must be done only through modules and functions. Whenever possible, a library should not impose specific behavior (such as parallelization) on its users. It is better to delegate this behavioral decision to the developers of clients, thus increasing the potential for code reuse of a library.

```
defmodule Calculator do
  def add(a, b) do
    a + b
  end

  def subtract(a, b) do
    a - b
  end
end
```

```
iex> Calculator.add(1, 5)
6
iex> Calculator.subtract(2, 3)
-1
```

Scattered process interfaces

Problem

In Elixir, the use of an `Agent`, a `GenServer`, or any other process abstraction is not an anti-pattern in itself. However, when the responsibility for direct interaction with a process is spread throughout the entire system, it can become problematic. This bad practice can increase the difficulty of code maintenance and make the code more prone to bugs.

Example

The following code seeks to illustrate this anti-pattern. The responsibility for interacting directly with the `Agent` is spread across four different modules (`A`, `B`, `C`, and `D`).

```
defmodule A do
  def update(process) do
    # Some other code...
    Agent.update(process, fn _list -> 123 end)
  end
end


defmodule B do
  def update(process) do
    # Some other code...
    Agent.update(process, fn content -> %{a: content} end)
  end
end


defmodule C do
  def update(process) do
    # Some other code...
    Agent.update(process, fn content -> [:atom_value |
content] end)
  end
end


defmodule D do
  def get(process) do
    # Some other code...
    Agent.get(process, fn content -> content end)
  end
end
```

This spreading of responsibility can generate duplicated code and make code maintenance more difficult. Also, due to the lack of control over the format of the shared data, complex composed data can be shared. This freedom to use any format of data is dangerous and can induce developers to introduce bugs.

```
# start an agent with initial state of an empty list
iex> {:ok, agent} = Agent.start_link(fn -> [] end)
{:ok, #PID<0.135.0>}

# many data formats (for example, List, Map, Integer, Atom)
are
# combined through direct access spread across the entire
system
iex> A.update(agent)
iex> B.update(agent)
iex> C.update(agent)

# state of shared information
iex> D.get(agent)
[:atom_value, %{a: 123}]
```

For a `GenServer` and other behaviours, this anti-pattern will manifest when scattering calls to `GenServer.call/3` and `GenServer.cast/2` throughout multiple modules, instead of encapsulating all the interaction with the `GenServer` in a single place.

Refactoring

Instead of spreading direct access to a process abstraction, such as `Agent`, over many places in the code, it is better to refactor this code by centralizing the responsibility for interacting with a process in a single module. This refactoring improves maintainability by removing duplicated code; it also allows you to limit the accepted format for shared data, reducing bug-

proneness. As shown below, the module `Foo.Bucket` is centralizing the responsibility for interacting with the `Agent`. Any other place in the code that needs to access shared data must now delegate this action to `Foo.Bucket`. Also, `Foo.Bucket` now only allows data to be shared in `Map` format.

```
defmodule Foo.Bucket do
  use Agent

  def start_link(_opts) do
    Agent.start_link(fn -> %{} end)
  end

  def get(bucket, key) do
    Agent.get(bucket, &Map.get(&1, key))
  end

  def put(bucket, key, value) do
    Agent.update(bucket, &Map.put(&1, key, value))
  end
end
```

The following are examples of how to delegate access to shared data (provided by an `Agent`) to `Foo.Bucket`.

```
# start an agent through `Foo.Bucket`
iex> {:ok, bucket} = Foo.Bucket.start_link(%{})
{:ok, #PID<0.114.0>}

# add shared values to the keys `milk` and `beer`
iex> Foo.Bucket.put(bucket, "milk", 3)
iex> Foo.Bucket.put(bucket, "beer", 7)

# access shared data of specific keys
```

```
iex> Foo.Bucket.get(bucket, "beer")
7
iex> Foo.Bucket.get(bucket, "milk")
3
```

Additional remarks

This anti-pattern was formerly known as Agent obsession.

Sending unnecessary data

Problem

Sending a message to a process can be an expensive operation if the message is big enough. That's because that message will be fully copied to the receiving process, which may be CPU and memory intensive. This is due to Erlang's "share nothing" architecture, where each process has its own memory, which simplifies and speeds up garbage collection.

This is more obvious when using `send/2`, `GenServer.call/3`, or the initial data in `GenServer.start_link/3`. Notably this also happens when using `spawn/1`, `Task.async/1`, `Task.async_stream/3`, and so on. It is more subtle here as the anonymous function passed to these functions captures the variables it references, and all captured variables will be copied over. By doing this, you can accidentally send way more data to a process than you actually need.

Example

Imagine you were to implement some simple reporting of IP addresses that made requests against your application. You want to do this asynchronously and not block processing, so you decide to use `spawn/1`. It may seem like a good idea to hand over the whole connection because we might need more

data later. However passing the connection results in copying a lot of unnecessary data like the request body, params, etc.

```
# log_request_ip send the ip to some external service
spawn(fn -> log_request_ip(conn) end)
```

This problem also occurs when accessing only the relevant parts:

```
spawn(fn -> log_request_ip(conn.remote_ip) end)
```

This will still copy over all of `conn`, because the `conn` variable is being captured inside the spawned function. The function then extracts the `remote_ip` field, but only after the whole `conn` has been copied over.

`send/2` and the `GenServer` APIs also rely on message passing. In the example below, the `conn` is once again copied to the underlying `GenServer`:

```
GenServer.cast(pid, {:report_ip_address, conn})
```

Refactoring

This anti-pattern has many potential remedies:

- Limit the data you send to the absolute necessary minimum instead of sending an entire struct. For example, don't send an entire `conn` struct if all you need is a couple of fields.
- If the only process that needs data is the one you are sending to, consider making the process fetch that data instead of passing it.
- Some abstractions, such as `:persistent_term`, allows you to share data between processes, as long as such data changes infrequently.

In our case, limiting the input data is a reasonable strategy. If all we need *right now* is the IP address, then let's only work with that and make sure we're only passing the IP address into the closure, like so:

```
ip_address = conn.remote_ip
spawn(fn -> log_request_ip(ip_address) end)
```

Or in the `GenServer` case:

```
GenServer.cast(pid, {:report_ip_address, conn.remote_ip})
```

Unsupervised processes

Problem

In Elixir, creating a process outside a supervision tree is not an anti-pattern in itself. However, when you spawn many long-running processes outside of supervision trees, this can make visibility and monitoring of these processes difficult, preventing developers from fully controlling their lifecycle.

Example

The following code example seeks to illustrate a library responsible for maintaining a numerical `Counter` through a `Agent` process *outside a supervision tree*.

```
defmodule Counter do
  @moduledoc """
  Global counter implemented as an Agent.
  """

  use Agent
```

```
  @doc "Starts a counter process."
  def start_link(opts \\ []) do
    initial_state = Keyword.get(opts, :initial_value, 0)
    name = Keyword.get(opts, :name, __MODULE__)
    Agent.start_link(fn -> initial_state end, name: name)
  end

  @doc "Gets the current value of the given counter."
  def get(name \\ __MODULE__) do
    Agent.get(name, fn state -> state end)
  end

  @doc "Bumps the value of the given counter."
  def bump(name \\ __MODULE__, value) do
    Agent.get_and_update(fn state -> {state, value + state}
end)
  end
end
```

While it is possible to start the process outside of a supervision tree:

```
iex> Counter.start_link()
{:ok, #PID<0.115.0>}
iex> Counter.bump(13)
0
iex> Counter.get()
13
```

Such processes are harder to observe and control their lifecycle. For example, if you have other processes that depend on the `Counter` above, you will need ad-hoc mechanisms to make sure they are initialized in order. Furthermore, when your application is shutting down, there is no guarantee when they are terminated.

Refactoring

To ensure that clients of a library have full control over their systems, regardless of the number of processes used and the lifetime of each one, all processes must be started inside a supervision tree. As shown below, this code uses a `Supervisor` as a supervision tree.

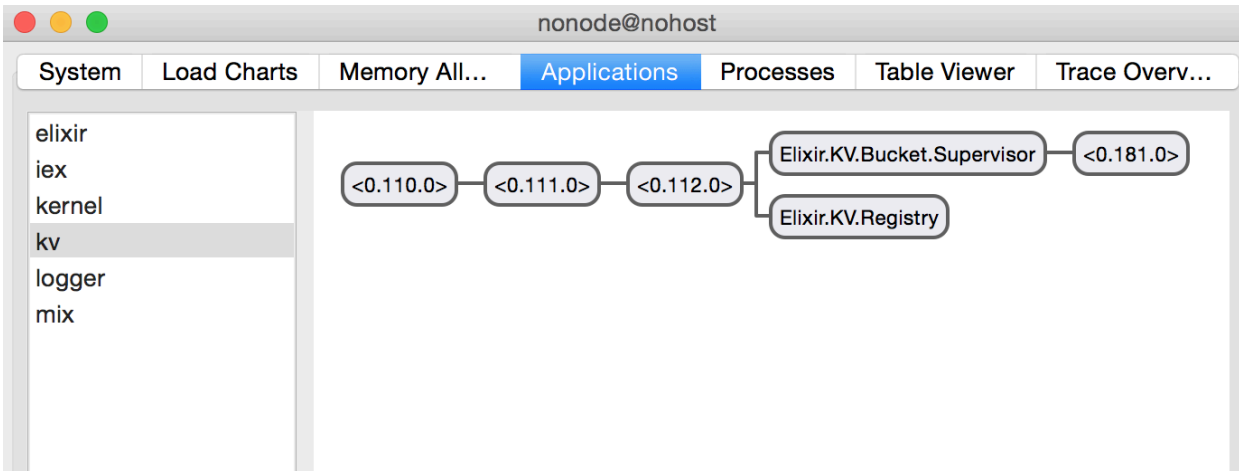
```
defmodule SupervisedProcess.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # With the default values for counter and name
      Counter,
      # With custom values for counter, name, and a custom ID
      Supervisor.child_spec(
        {Counter, name: :other_counter, initial_value: 15},
        id: :other_counter
      )
    ]

    Supervisor.start_link(children, strategy: :one_for_one,
name: App.Supervisor)
  end
end
```

Besides having a deterministic order in which processes are started, supervision trees also guarantee they are terminated in reverse order, allowing you to perform any necessary clean up during shut down. Furthermore, supervision strategies allows us to configure exactly how process should act in case of unexpected failures.

Finally, applications and supervision trees can be introspected through applications like the Phoenix.LiveDashboard and Erlang's built-in observer:


nonode@nohost
System
Load Charts
Memory All...
Applications
Processes
Table Viewer
Trace Overv...
elixir
iex
kernel
kv
logger
mix
<0.110.0>
<0.111.0>
<0.112.0>
Elixir.KV.Bucket.Supervisor
<0.181.0>
Elixir.KV.Registry

Meta-programming anti-patterns

This document outlines potential anti-patterns related to meta-programming.

Compile-time dependencies

Problem

This anti-pattern is related to dependencies between files in Elixir. Because macros are used at compile-time, the use of any macro in Elixir adds a compile-time dependency to the module that defines the macro.

However, when macros are used in the body of a module, the arguments to the macro themselves may become compile-time dependencies. These dependencies may lead to dependency graphs where changing a single file causes several files to be recompiled.

Example

Let's take the `Plug` library as an example. The `Plug` project allows you to specify several modules, also known as plugs, which will be invoked whenever there is a request. As a user of `Plug`, you would use it as follows:

```
defmodule MyApp do
  use Plug.Builder

  plug MyApp.Authentication
end
```

And imagine `Plug` has the following definitions of the macros above (simplified):

```
defmodule Plug.Builder do
  defmacro __using__(_opts) do
    quote do
      Module.register_attribute(__MODULE__, :plugs,
accumulate: true)
      @before_compile Plug.Builder
    end
  end

  defmacro plug(mod) do
    quote do
      @plugs unquote(mod)
    end
  end

  ...
end
```

The implementation accumulates all modules inside the `@plugs` module attribute. Right before the module is compiled, `Plug.Builder` will reads all modules stored in `@plugs` and compile them into a function, like this:

```
def call(conn, _opts) do
  MyApp.Authentication.call(conn)
end
```

The trouble with the code above is that, because the `plug MyApp.Authentication` was invoked at compile-time, the module `MyApp.Authentication` is now a compile-time dependency of `MyApp`, even though `MyApp.Authentication` is never used at compile-time. If

`MyApp.Authentication` depends on other modules, even at runtime, this can now lead to a large recompilation graph in case of changes.

Refactoring

To address this anti-pattern, a macro can expand literals within the context they are meant to be used, as follows:

```
  defmacro plug(mod) do
    mod = Macro.expand_literals(mod, %{__CALLER__ |
function: {:call, 2}})

    quote do
      @plugs unquote(mod)
    end
  end
```

In the example above, since `mod` is used only within the `call/2` function, we prematurely expand module reference as if it was inside the `call/2` function. Now `MyApp.Authentication` is only a runtime dependency of `MyApp`, no longer a compile-time one.

Note, however, the above must only be done if your macros do not attempt to invoke any function, access any struct, or any other metadata of the module at compile-time. If you interact with the module given to a macro anywhere outside of definition of a function, then you effectively have a compile-time dependency. And, even though you generally want to avoid them, it is not always possible.

In actual projects, developers may use `mix xref trace path/to/file.ex` to execute a file and have it print information about which modules it depends on, and if those modules are compile-time, runtime, or export dependencies. See `mix xref` for more information.

Large code generation

Problem

This anti-pattern is related to macros that generate too much code. When a macro generates a large amount of code, it impacts how the compiler and/or the runtime work. The reason for this is that Elixir may have to expand, compile, and execute the code multiple times, which will make compilation slower and the resulting compiled artifacts larger.

Example

Imagine you are defining a router for a web application, where you could have macros like `get/2`. On every invocation of the macro (which could be hundreds), the code inside `get/2` will be expanded and compiled, which can generate a large volume of code overall.

```
defmodule Routes do
  defmacro get(route, handler) do
    quote do
      route = unquote(route)
      handler = unquote(handler)

      if not is_binary(route) do
        raise ArgumentError, "route must be a binary"
      end

      if not is_atom(handler) do
        raise ArgumentError, "handler must be a module"
      end

      @store_route_for_compilation {route, handler}
    end
```

```
  end
end
```

Refactoring

To remove this anti-pattern, the developer should simplify the macro, delegating part of its work to other functions. As shown below, by encapsulating the code inside `quote/1` inside the function `__define__/3` instead, we reduce the code that is expanded and compiled on every invocation of the macro, and instead we dispatch to a function to do the bulk of the work.

```
defmodule Routes do
  defmacro get(route, handler) do
    quote do
      Routes.__define__(__MODULE__, unquote(route),
unquote(handler))
    end
  end

  def __define__(module, route, handler) do
    if not is_binary(route) do
      raise ArgumentError, "route must be a binary"
    end

    if not is_atom(handler) do
      raise ArgumentError, "handler must be a module"
    end

    Module.put_attribute(module,
:store_route_for_compilation, {route, handler})
```

```
  end
end
```

Unnecessary macros

Problem

Macros are powerful meta-programming mechanisms that can be used in Elixir to extend the language. While using macros is not an anti-pattern in itself, this meta-programming mechanism should only be used when absolutely necessary. Whenever a macro is used, but it would have been possible to solve the same problem using functions or other existing Elixir structures, the code becomes unnecessarily more complex and less readable. Because macros are more difficult to implement and reason about, their indiscriminate use can compromise the evolution of a system, reducing its maintainability.

Example

The `MyMath` module implements the `sum/2` macro to perform the sum of two numbers received as parameters. While this code has no syntax errors and can be executed correctly to get the desired result, it is unnecessarily more complex. By implementing this functionality as a macro rather than a conventional function, the code became less clear:

```
defmodule MyMath do
  defmacro sum(v1, v2) do
    quote do
      unquote(v1) + unquote(v2)
    end
  end
end
```

```
iex> require MyMath
MyMath
iex> MyMath.sum(3, 5)
8
iex> MyMath.sum(3 + 1, 5 + 6)
15
```

Refactoring

To remove this anti-pattern, the developer must replace the unnecessary macro with structures that are simpler to write and understand, such as named functions. The code shown below is the result of the refactoring of the previous example. Basically, the `sum/2` macro has been transformed into a conventional named function. Note that the `require/2` call is no longer needed:

```
defmodule MyMath do
  def sum(v1, v2) do # <= The macro became a named function
    v1 + v2
  end
end


iex> MyMath.sum(3, 5)
8
iex> MyMath.sum(3+1, 5+6)
15
```

`use` instead of `import`

Problem

Elixir has mechanisms such as `import/1`, `alias/1`, and `use/1` to establish dependencies between modules. Code implemented with these mechanisms does not characterize a smell by itself. However, while the `import/1` and `alias/1` directives have lexical scope and only facilitate a module calling functions of another, the `use/1` directive has a *broader scope*, which can be problematic.

The `use/1` directive allows a module to inject any type of code into another, including propagating dependencies. In this way, using the `use/1` directive makes code harder to read, because to understand exactly what will happen when it references a module, it is necessary to have knowledge of the internal details of the referenced module.

Example

The code shown below is an example of this anti-pattern. It defines three modules -- `ModuleA`, `Library`, and `ClientApp`. `ClientApp` is reusing code from the `Library` via the `use/1` directive, but is unaware of its internal details. This makes it harder for the author of `ClientApp` to visualize which modules and functionality are now available within its module. To make matters worse, `Library` also imports `ModuleA`, which defines a `foo/0` function that conflicts with a local function defined in `ClientApp`:

```
defmodule ModuleA do
  def foo do
    "From Module A"
  end
end


defmodule Library do
  defmacro __using__(_opts) do
```

```
    quote do
      import Library
      import ModuleA  # <= propagating dependencies!
    end
  end

  def from_lib do
    "From Library"
  end
end



defmodule ClientApp do
  use Library

  def foo do
    "Local function from client app"
  end

  def from_client_app do
    from_lib() <> " - " <> foo()
  end
end
```

When we try to compile `ClientApp`, Elixir detects the conflict and throws the following error:

```
error: imported ModuleA.foo/0 conflicts with local function
  └ client_app.ex:4:
```

Refactoring

To remove this anti-pattern, we recommend library authors avoid providing `__using__/1` callbacks whenever it can be replaced by `alias/1` or `import/1` directives. In the following code, we assume `use Library` is no longer available and `ClientApp` was refactored in this way, and with that, the code is clearer and the conflict as previously shown no longer exists:

```
defmodule ClientApp do
  import Library

  def foo do
    "Local function from client app"
  end

  def from_client_app do
    from_lib() <> " - " <> foo()
  end
end


iex> ClientApp.from_client_app()
"From Library - Local function from client app"
```

Additional remarks

In situations where you need to do more than importing and aliasing modules, providing `use MyModule` may be necessary, as it provides a common extension point within the Elixir ecosystem.

Therefore, to provide guidance and clarity, we recommend library authors to include an admonition block in their `@moduledoc` that explains how `use MyModule` impacts the developer's code. As an example, the `GenServer` documentation outlines:

> use GenServer
>
> When you `use GenServer`, the `GenServer` module will set `@behaviour GenServer` and define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

Think of this summary as a "Nutrition facts label" for code generation. Make sure to only list changes made to the public API of the module. For example, if `use Library` sets an internal attribute called `@_some_module_info` and this attribute is never meant to be public, avoid documenting it in the nutrition facts.

For convenience, the markup notation to generate the admonition block above is this:

```
> #### `use GenServer` {: .info}
>
> When you `use GenServer`, the `GenServer` module will
> set `@behaviour GenServer` and define a `child_spec/1`
> function, so your module can be used as a child
> in a supervision tree.
```

Untracked compile-time dependencies

Problem

This anti-pattern is the opposite of "Compile-time dependencies" and it happens when a compile-time dependency is accidentally bypassed, making the Elixir compiler unable to track dependencies and recompile files correctly. This happens when building aliases (in other words, module names) dynamically, either within a module or within a macro.

Example

For example, imagine you invoke a module at compile-time, you could write it as such:

```
defmodule MyModule do
  SomeOtherModule.example()
end
```

In this case, Elixir knows `MyModule` is invoked `SomeOtherModule.example/0` outside of a function, and therefore at compile-time.

Elixir can also track module names even during dynamic calls:

```
defmodule MyModule do
  mods = [OtherModule.Foo, OtherModule.Bar]

  for mod <- mods do
    mod.example()
  end
end
```

In the previous example, even though Elixir does not know which modules the function `example/0` was invoked on, it knows the modules `OtherModule.Foo` and `OtherModule.Bar` are referred outside of a function and therefore they become compile-time dependencies. If any of them change, Elixir will recompile `MyModule` itself.

However, you should not programmatically generate the module names themselves, as that would make it impossible for Elixir to track them. More precisely, do not do this:

```
defmodule MyModule do
  parts = [:Foo, :Bar]
```

```
  for part <- parts do
    Module.concat(OtherModule, part).example()
  end
end
```

In this case, because the whole module was generated, Elixir sees a dependency only to `OtherModule`, never to `OtherModule.Foo` and `OtherModule.Bar`, potentially leading to inconsistencies when recompiling projects.

A similar bug can happen when abusing the property that aliases are simply atoms, defining the atoms directly. In the case below, Elixir never sees the aliases, leading to untracked compile-time dependencies:

```
defmodule MyModule do
  mods = [:"Elixir.OtherModule.Foo",
:"Elixir.OtherModule.Bar"]

  for mod <- mods do
    mod.example()
  end
end
```

Refactoring

To address this anti-pattern, you should avoid defining module names programmatically. For example, if you need to dispatch to multiple modules, do so by using full module names.

Instead of:

```
defmodule MyModule do
  parts = [:Foo, :Bar]

  for part <- parts do
    Module.concat(OtherModule, part).example()
  end
end
```

Do:

```
defmodule MyModule do
  mods = [OtherModule.Foo, OtherModule.Bar]

  for mod <- mods do
    mod.example()
  end
end
```

If you really need to define modules dynamically, you can do so via meta-programming, building the whole module name at compile-time:

```
defmodule MyMacro do
  defmacro call_examples(parts) do
    for part <- parts do
      quote do
        # This builds OtherModule.Foo at compile-time
        OtherModule.unquote(part).example()
      end
    end
  end
end

defmodule MyModule do
```

```
  import MyMacro
  call_examples [:Foo, :Bar]
end
```

In actual projects, developers may use `mix xref trace path/to/file.ex` to execute a file and have it print information about which modules it depends on, and if those modules are compile-time, runtime, or export dependencies. This can help you debug if the dependencies are being properly tracked in relation to external modules. See `mix xref` for more information.

Quote and unquote

This guide aims to introduce the meta-programming techniques available in Elixir. The ability to represent an Elixir program by its own data structures is at the heart of meta-programming. This chapter starts by exploring those structures and the associated `quote/2` and `unquote/1` constructs, so we can take a look at macros in the next guide, and finally build our own domain specific language.

Quoting

The building block of an Elixir program is a tuple with three elements. For example, the function call `sum(1, 2, 3)` is represented internally as:

```
{:sum, [], [1, 2, 3]}
```

You can get the representation of any expression by using the `quote/2` macro:

```
iex> quote do: sum(1, 2, 3)
{:sum, [], [1, 2, 3]}
```

The first element is the function name, the second is a keyword list containing metadata, and the third is the arguments list.

Operators are also represented as such tuples:

```
iex> quote do: 1 + 2
{:+, [context: Elixir, import: Kernel], [1, 2]}
```

Even a map is represented as a call to `%{}`:

```
iex> quote do: %{1 => 2}
{:%{}, [], [{1, 2}]}
```

Variables are represented using such triplets, with the difference that the last element is an atom, instead of a list:

```
iex> quote do: x
{:x, [], Elixir}
```

When quoting more complex expressions, we can see that the code is represented in such tuples, which are often nested inside each other in a structure resembling a tree. Many languages would call such representations an *Abstract Syntax Tree* (AST). Elixir calls them *quoted expressions*:

```
iex> quote do: sum(1, 2 + 3, 4)
{:sum, [], [1, {:+, [context: Elixir, import: Kernel], [2,
3]}, 4]}
```

Sometimes, when working with quoted expressions, it may be useful to get the textual code representation back. This can be done with `Macro.to_string/1`:

```
iex> Macro.to_string(quote do: sum(1, 2 + 3, 4))
"sum(1, 2 + 3, 4)"
```

In general, the tuples above are structured according to the following format:

```
{atom | tuple, list, list | atom}
```

- The first element is an atom or another tuple in the same representation;
- The second element is a keyword list containing metadata, like numbers and contexts;
- The third element is either a list of arguments for the function call or an atom. When this element is an atom, it means the tuple represents a variable.

Besides the tuple defined above, there are five Elixir literals that, when quoted, return themselves (and not a tuple). They are:

```
:sum          #=> Atoms
1.0           #=> Numbers
[1, 2]        #=> Lists
"strings"     #=> Strings
{key, value} #=> Tuples with two elements
```

Most Elixir code has a straight-forward translation to its underlying quoted expression. We recommend you try out different code samples and see what the results are. For example, what does `String.upcase("foo")` expand to? We have also learned that `if(true, do: :this, else: :that)` is the same as `if true do :this else :that end`. How does this affirmation hold with quoted expressions?

Unquoting

Quoting is about retrieving the inner representation of some particular chunk of code. However, sometimes it may be necessary to inject some other particular chunk of code inside the representation we want to retrieve.

For example, imagine you have a variable called `number` which contains the number you want to inject inside a quoted expression.

```
iex> number = 13
iex> Macro.to_string(quote do: 11 + number)
"11 + number"
```

That's not what we wanted, since the value of the `number` variable has not been injected and `number` has been quoted in the expression. In order to inject the *value* of the `number` variable, `unquote/1` has to be used inside the quoted representation:

```
iex> number = 13
iex> Macro.to_string(quote do: 11 + unquote(number))
"11 + 13"
```

`unquote/1` can even be used to inject function names:

```
iex> fun = :hello
iex> Macro.to_string(quote do: unquote(fun)(:world))
"hello(:world)"
```

In some cases, it may be necessary to inject many values inside a list. For example, imagine you have a list containing `[1, 2, 6]`, and we want to inject `[3, 4, 5]` into it. Using `unquote/1` won't yield the desired result:

```
iex> inner = [3, 4, 5]
iex> Macro.to_string(quote do: [1, 2, unquote(inner), 6])
"[1, 2, [3, 4, 5], 6]"
```

That's when `unquote_splicing/1` comes in handy:

```
iex> inner = [3, 4, 5]
iex> Macro.to_string(quote do: [1, 2,
unquote_splicing(inner), 6])
"[1, 2, 3, 4, 5, 6]"
```

Unquoting is very useful when working with macros. When writing macros, developers are able to receive code chunks and inject them inside other code chunks, which can be used to transform code or write code that generates code during compilation.

Escaping

As we saw at the beginning of this chapter, only some values are valid quoted expressions in Elixir. For example, a map is not a valid quoted expression. Neither is a tuple with four elements. However, such values *can* be expressed as a quoted expression:

```
iex> quote do: %{1 => 2}
{:%{}, [], [{1, 2}]}
```

In some cases, you may need to inject such *values* into *quoted expressions*. To do that, we need to first escape those values into quoted expressions with the help of `Macro.escape/1`:

```
iex> map = %{hello: :world}
iex> Macro.escape(map)
{:%{}, [], [hello: :world]}
```

Macros receive quoted expressions and must return quoted expressions. However, sometimes during the execution of a macro, you may need to work with values and making a distinction between values and quoted expressions will be required.

In other words, it is important to make a distinction between a regular Elixir value (like a list, a map, a process, a reference, and so on) and a quoted expression. Some values, such as integers, atoms, and strings, have a quoted expression equal to the value itself. Other values, like maps, need to be explicitly converted. Finally, values like functions and references cannot be converted to a quoted expression at all.

When working with macros and code that generates code, check out the documentation for the `Macro` module, which contains many functions to work with Elixir's AST.

In this introduction, we have laid the groundwork to finally write our first macro. You can check that out in the next guide.

Macros

Even though Elixir attempts its best to provide a safe environment for macros, most of the responsibility of writing clean code with macros falls on developers. Macros are harder to write than ordinary Elixir functions, and it's considered to be bad style to use them when they're not necessary. Write macros responsibly.

Elixir already provides mechanisms to write your everyday code in a simple and readable fashion by using its data structures and functions. Macros should only be used as a last resort. Remember that **explicit is better than implicit**. **Clear code is better than concise code.**

Our first macro

Macros in Elixir are defined via `defmacro/2`.

> For this guide, we will be using files instead of running code samples in IEx. That's because the code samples will span multiple lines of code and typing them all in IEx can be counter-productive. You should be able to run the code samples by saving them into a `macros.exs` file and running it with `elixir macros.exs` or `iex macros.exs`.

In order to better understand how macros work, let's create a new module where we are going to implement `unless` (which does the opposite of `if/2`), as a macro and as a function:

```
defmodule Unless do
  def fun_unless(clause, do: expression) do
    if(!clause, do: expression)
  end
```

```
  defmacro macro_unless(clause, do: expression) do
    quote do
      if(!unquote(clause), do: unquote(expression))
    end
  end
end
```

The function receives the arguments and passes them to `if/2`. However, as we learned in the previous guide, the macro will receive quoted expressions, inject them into the quote, and finally return another quoted expression.

Let's start `iex` with the module above:

```
$ iex macros.exs
```

and play with those definitions:

```
iex> require Unless
iex> Unless.macro_unless(true, do: IO.puts("this should
never be printed"))
nil
iex> Unless.fun_unless(true, do: IO.puts("this should never
be printed"))
"this should never be printed"
nil
```

In our *macro* implementation, the sentence was not printed, although it was printed in our *function* implementation. That's because the arguments to a function call are evaluated before calling the function. However, macros do not evaluate their arguments. Instead, they receive the arguments as quoted expressions which are then transformed into other quoted expressions. In

this case, we have rewritten our `unless` macro to become an `if/2` behind the scenes.

In other words, when invoked as:

```
Unless.macro_unless(true, do: IO.puts("this should never be
printed"))
```

Our `macro_unless` macro received the following:

```
macro_unless(true, [do: {{:., [], [{:__aliases__, [],
[:IO]}, :puts]}, [], ["this should never be printed"]}])
```

and it then returned a quoted expression as follows:

```
{:if, [],
 [{:!, [], [true]},
  [do: {{:., [],
     [{:__aliases__,
       [], [:IO]},
      :puts]}, [], ["this should never be printed"]}]]}
```

We can actually verify that this is the case by using `Macro.expand_once/2`:

```
iex> expr = quote do: Unless.macro_unless(true, do:
IO.puts("this should never be printed"))
iex> res  = Macro.expand_once(expr, __ENV__)
iex> IO.puts(Macro.to_string(res))
if(!true) do
  IO.puts("this should never be printed")
```

```
end
:ok
```

`Macro.expand_once/2` receives a quoted expression and expands it according to the current environment. In this case, it expanded/invoked the `Unless.macro_unless/2` macro and returned its result. We then proceeded to convert the returned quoted expression to a string and print it (we will talk about `__ENV__` later in this chapter).

That's what macros are all about. They are about receiving quoted expressions and transforming them into something else. In fact, `if/2` in Elixir is implemented as a macro:

```
defmacro if(clause, do: expression) do
  quote do
    case unquote(clause) do
      x when x in [false, nil] -> nil
      _ -> unquote(expression)
  end
end
```

Constructs such as `if/2`, `defmacro/2`, `def/2`, `defprotocol/2`, and many others used throughout the Elixir standard library are written in pure Elixir, often as a macro. This means that the constructs being used to build the language can be used by developers to extend the language to the domains they are working on.

We can define any function and macro we want, including ones that override the built-in definitions provided by Elixir. The only exceptions are Elixir special forms which are not implemented in Elixir and therefore cannot be overridden. The full list of special forms is available in `Kernel.SpecialForms`.

Macro hygiene

Elixir macros have "late resolution". This guarantees that a variable defined inside a quote won't conflict with a variable defined in the context where that macro is expanded. For example:

```
defmodule Hygiene do
  defmacro no_interference do
    quote do: a = 1
  end
end

defmodule HygieneTest do
  def go do
    require Hygiene
    a = 13
    Hygiene.no_interference()
    a
  end
end

HygieneTest.go()
# => 13
```

In the example above, even though the macro injects `a = 1`, it does not affect the variable `a` defined by the `go/0` function. If a macro wants to explicitly affect the context, it can use `var!/1`:

```
defmodule Hygiene do
  defmacro interference do
    quote do: var!(a) = 1
  end
end
```

```
defmodule HygieneTest do
  def go do
    require Hygiene
    a = 13
    Hygiene.interference()
    a
  end
end

HygieneTest.go()
# => 1
```

The code above will work but issue a warning: `variable "a" is unused.` The macro is overriding the original value and the original value is never used.

Variable hygiene only works because Elixir annotates variables with their **context**. For example, a variable `x` defined on line 3 of a module would be represented as:

```
{:x, [line: 3], nil}
```

However, a quoted variable would be represented as:

```
defmodule Sample do
  def quoted do
    quote do: x
  end
end

Sample.quoted() #=> {:x, [line: 3], Sample}
```

Notice that the *third element* in the quoted variable is the atom `Sample`, instead of `nil`, which marks the variable as coming from the `Sample` module. Therefore, Elixir considers these two variables as coming from different contexts and handles them accordingly.

Elixir provides similar mechanisms for imports and aliases too. This guarantees that a macro will behave as specified by its source module rather than conflicting with the target module where the macro is expanded. Hygiene can be bypassed under specific situations by using macros like `var!/2` and `alias!/1`, although one must be careful when using those as they directly change the user environment.

Sometimes variable names might be dynamically created. In such cases, `Macro.var/2` can be used to define new variables:

```
defmodule Sample do
  defmacro initialize_to_char_count(variables) do
    Enum.map(variables, fn name ->
      var = Macro.var(name, nil)
      length = name |> Atom.to_string() |> String.length()

      quote do
        unquote(var) = unquote(length)
      end
    end)
  end

  def run do
    initialize_to_char_count([:red, :green, :yellow])
    [red, green, yellow]
  end
end

> Sample.run() #=> [3, 5, 6]
```

Take note of the second argument to `Macro.var/2`. This is the **context** being used and will determine hygiene as described in the next section. Check out also `Macro.unique_var/2`, for cases when you need to generate variables with unique names.

The environment

When calling `Macro.expand_once/2` earlier in this chapter, we used the special form `__ENV__/0`.

`__ENV__/0` returns a `Macro.Env` struct which contains useful information about the compilation environment, including the current module, file, and line, all variables defined in the current scope, as well as imports, requires, and more:

```
iex> __ENV__.module
nil
iex> __ENV__.file
"iex"
iex> __ENV__.requires
[IEx.Helpers, Kernel, Kernel.Typespec]
iex> require Integer
nil
iex> __ENV__.requires
[IEx.Helpers, Integer, Kernel, Kernel.Typespec]
```

Many of the functions in the `Macro` module expect a `Macro.Env` environment. You can read more about these functions in `Macro` and learn more about the compilation environment in the `Macro.Env`.

Private macros

Elixir also supports **private macros** via `defmacrop`. Like private functions, these macros are only available inside the module that defines them, and only at compilation time.

It is important that a macro is defined before its usage. Failing to define a macro before its invocation will raise an error at runtime, since the macro won't be expanded and will be translated to a function call:

```
iex> defmodule Sample do
...>  def four, do: two() + two()
...>  defmacrop two, do: 2
...> end
** (CompileError) iex:2: function two/0 undefined
```

Write macros responsibly

Macros are a powerful construct and Elixir provides many mechanisms to ensure they are used responsibly.

- Macros are **hygienic**: by default, variables defined inside a macro are not going to affect the user code. Furthermore, function calls and aliases available in the macro context are not going to leak into the user context.

- Macros are **lexical**: it is impossible to inject code or macros globally. In order to use a macro, you need to explicitly `require` or `import` the module that defines the macro.

- Macros are **explicit**: it is impossible to run a macro without explicitly invoking it. For example, some languages allow developers to completely rewrite functions behind the scenes, often via parse transforms or via some reflection mechanisms. In Elixir, a macro must be explicitly invoked in the caller during compilation time.

- Macros' language is clear: many languages provide syntax shortcuts for `quote` and `unquote`. In Elixir, we preferred to have them explicitly spelled out, in order to clearly delimit the boundaries of a macro definition and its quoted expressions.

Even with such guarantees, the developer plays a big role when writing macros responsibly. If you are confident you need to resort to macros, remember that macros are not your API. Keep your macro definitions short, including their quoted contents. For example, instead of writing a macro like this:

```
defmodule MyModule do
  defmacro my_macro(a, b, c) do
    quote do
      do_this(unquote(a))
      # ...
      do_that(unquote(b))
      # ...
      and_that(unquote(c))
    end
  end
end
```

write:

```
defmodule MyModule do
  defmacro my_macro(a, b, c) do
    quote do
      # Keep what you need to do here to a minimum
      # and move everything else to a function
      MyModule.do_this_that_and_that(unquote(a), unquote(b),
unquote(c))
    end
  end
```

```
  def do_this_that_and_that(a, b, c) do
    do_this(a)
    ...
    do_that(b)
    ...
    and_that(c)
  end
end
```

This makes your code clearer and easier to test and maintain, as you can invoke and test `do_this_that_and_that/3` directly. It also helps you design an actual API for developers that do not want to rely on macros.

With this guide, we finish our introduction to macros. The next guide is a brief discussion on **DSLs** that shows how we can mix macros and module attributes to annotate and extend modules and functions.

Domain-Specific Languages (DSLs)

Domain-specific Languages (DSLs) are languages tailored to a specific application domain. You don't need macros in order to have a DSL: every data structure and every function you define in your module is part of your domain-specific language.

For example, imagine we want to implement a `Validator` module which provides a data validation domain-specific language. We could implement it using data structures, functions, or macros. Let's see what those different DSLs would look like:

```
# 1. Data structures
import Validator
validate user, name: [length: 1..100], email: [matches:
~r/@/]

# 2. Functions
import Validator
user
|> validate_length(:name, 1..100)
|> validate_matches(:email, ~r/@/)

# 3. Macros + modules
defmodule MyValidator do
  use Validator
  validate_length :name, 1..100
  validate_matches :email, ~r/@/
end

MyValidator.validate(user)
```

Of all the approaches above, the first is definitely the most flexible. If our domain rules can be encoded with data structures, they are by far the easiest to compose and implement, as Elixir's standard library is filled with functions for manipulating different data types.

The second approach uses function calls which better suits more complex APIs (for example, if you need to pass many options) and reads nicely in Elixir thanks to the pipe operator.

The third approach uses macros, and is by far the most complex. It will take more lines of code to implement, it is hard and expensive to test (compared to testing simple functions), and it limits how the user may use the library since all validations need to be defined inside a module.

To drive the point home, imagine you want to validate a certain attribute only if a given condition is met. We could easily achieve it with the first solution, by manipulating the data structure accordingly, or with the second solution by using conditionals (if/else) before invoking the function. However, it is impossible to do so with the macros approach unless its DSL is augmented.

In other words:

```
data > functions > macros
```

That said, there are still cases where using macros and modules to build domain-specific languages is useful. Since we have explored data structures and function definitions in the Getting Started guide, this chapter will explore how to use macros and module attributes to tackle more complex DSLs.

Building our own test case

The goal in this chapter is to build a module named `TestCase` that allows us to write the following:

```
defmodule MyTest do
  use TestCase

  test "arithmetic operations" do
    4 = 2 + 2
  end

  test "list operations" do
    [1, 2, 3] = [1, 2] ++ [3]
  end
end

MyTest.run()
```

In the example above, by using `TestCase`, we can write tests using the `test` macro, which defines a function named `run` to automatically run all tests for us. Our prototype will rely on the match operator (`=`) as a mechanism to do assertions.

The `test` macro

Let's start by creating a module that defines and imports the `test` macro when used:

```
defmodule TestCase do
  # Callback invoked by `use`.
  #
  # For now it returns a quoted expression that
  # imports the module itself into the user code.
  @doc false
  defmacro __using__(_opts) do
    quote do
      import TestCase
```

```
    end
  end

  @doc """
  Defines a test case with the given description.

  ## Examples

      test "arithmetic operations" do
        4 = 2 + 2
      end

  """
  defmacro test(description, do: block) do
    function_name = String.to_atom("test " <> description)
    quote do
      def unquote(function_name)(), do: unquote(block)
    end
  end
end
```

Assuming we defined `TestCase` in a file named `tests.exs`, we can open it up by running `iex tests.exs` and define our first tests:

```
iex> defmodule MyTest do
...>   use TestCase
...>
...>   test "hello" do
...>     "hello" = "world"
...>   end
...> end
```

For now, we don't have a mechanism to run tests, but we know that a function named `test hello` was defined behind the scenes. When we invoke it, it should fail:

```
iex> MyTest."test hello"()
** (MatchError) no match of right hand side value: "world"
```

Storing information with attributes

In order to finish our `TestCase` implementation, we need to be able to access all defined test cases. One way of doing this is by retrieving the tests at runtime via `__MODULE__.__info__(:functions)`, which returns a list of all functions in a given module. However, considering that we may want to store more information about each test besides the test name, a more flexible approach is required.

When discussing module attributes in earlier chapters, we mentioned how they can be used as temporary storage. That's exactly the property we will apply in this section.

In the `__using__/1` implementation, we will initialize a module attribute named `@tests` to an empty list, then store the name of each defined test in this attribute so the tests can be invoked from the `run` function.

Here is the updated code for the `TestCase` module:

```
defmodule TestCase do
  @doc false
  defmacro __using__(_opts) do
    quote do
      import TestCase

      # Initialize @tests to an empty list
```

```
      @tests []

      # Invoke TestCase.__before_compile__/1 before the
module is compiled
      @before_compile TestCase
    end
  end

  @doc """
  Defines a test case with the given description.

  ## Examples

      test "arithmetic operations" do
        4 = 2 + 2
      end

  """
  defmacro test(description, do: block) do
    function_name = String.to_atom("test " <> description)
    quote do
      # Prepend the newly defined test to the list of tests
      @tests [unquote(function_name) | @tests]
      def unquote(function_name)(), do: unquote(block)
    end
  end

  # This will be invoked right before the target module is
compiled
  # giving us the perfect opportunity to inject the `run/0`
function
  @doc false
  defmacro __before_compile__(_env) do
    quote do
      def run do
        Enum.each(@tests, fn name ->
```

```
          IO.puts("Running #{name}")
          apply(__MODULE__, name, [])
        end)
      end
    end
  end
end
```

By starting a new IEx session, we can now define our tests and run them:

```
iex> defmodule MyTest do
...>   use TestCase
...>
...>   test "hello" do
...>     "hello" = "world"
...>   end
...> end
iex> MyTest.run()
Running test hello
** (MatchError) no match of right hand side value: "world"
```

Although we have overlooked some details, this is the main idea behind creating domain-specific languages in Elixir via modules and macros. Macros enable us to return quoted expressions that are executed in the caller, which we can then use to transform code and store relevant information in the target module via module attributes. Finally, callbacks such as `@before_compile` allow us to inject code into the module when its definition is complete.

Besides `@before_compile`, there are other useful module attributes like `@on_definition` and `@after_compile`, which you can read more about in the docs for `Module`. You can also find useful information about macros

and the compilation environment in the documentation for the `Macro` and `Macro.Env`.

Compatibility and deprecations

Elixir is versioned according to a vMAJOR.MINOR.PATCH schema.

Elixir is currently at major version v1. A new backwards compatible minor release happens every 6 months. Patch releases are not scheduled and are made whenever there are bug fixes or security patches.

Elixir applies bug fixes only to the latest minor branch. Security patches are available for the last 5 minor branches:

Elixir version	Support
1.20	Bug fixes and security patches
1.19	Security patches only
1.18	Security patches only
1.17	Security patches only
1.16	Security patches only

New releases are announced in the read-only announcements mailing list. All security releases will be tagged with `[security]`.

There are currently no plans for a major v2 release.

Between non-major Elixir versions

Elixir minor and patch releases are backwards compatible: well-defined behaviors and documented APIs in a given version will continue working on future versions.

Although we expect the vast majority of programs to remain compatible over time, it is impossible to guarantee that no future change will break any program. Under some unlikely circumstances, we may introduce changes that break existing code:

- Security: a security issue in the implementation may arise whose resolution requires backwards incompatible changes. We reserve the right to address such security issues.
- Bugs: if an API has undesired behavior, a program that depends on the buggy behavior may break if the bug is fixed. We reserve the right to fix such bugs.
- Compiler front-end: improvements may be done to the compiler, introducing new warnings for ambiguous modes and providing more detailed error messages. Those can lead to compilation errors (when running with `--warnings-as-errors`) or tooling failures when asserting on specific error messages (although one should avoid such). We reserve the right to do such improvements.
- Imports: new functions may be added to the `Kernel` module, which is auto-imported. They may collide with local functions defined in your modules. Collisions can be resolved in a backwards compatible fashion using `import Kernel, except: [...]` with a list of all functions you don't want to be imported from `Kernel`. We reserve the right to do such additions.

In order to continue evolving the language without introducing breaking changes, Elixir will rely on deprecations to demote certain practices and promote new ones. Our deprecation policy is outlined in the "Deprecations" section.

The only exception to the compatibility guarantees above are experimental features, which will be explicitly marked as such, and do not provide any compatibility guarantee until they are stabilized.

Between Elixir and Erlang/OTP

Erlang/OTP versioning is independent from the versioning of Elixir. Erlang releases a new major version yearly. Our goal is to support the last three Erlang major versions by the time Elixir is released. The compatibility table is shown below.

Elixir version	Supported Erlang/OTP versions
1.20	27 - 29
1.19	26 - 28
1.18	25 - 27
1.17	25 - 27
1.16	24 - 26
1.15	24 - 26
1.14	23 - 25 (and Erlang/OTP 26 from v1.14.5)
1.13	22 - 24 (and Erlang/OTP 25 from v1.13.4)
1.12	22 - 24
1.11	21 - 23 (and Erlang/OTP 24 from v1.11.4)
1.10	21 - 22 (and Erlang/OTP 23 from v1.10.3)
1.9	20 - 22
1.8	20 - 22
1.7	19 - 22
1.6	19 - 20 (and Erlang/OTP 21 from v1.6.6)
1.5	18 - 20
1.4	18 - 19 (and Erlang/OTP 20 from v1.4.5)
1.3	18 - 19
1.2	18 - 18 (and Erlang/OTP 19 from v1.2.6)
1.1	17 - 18
1.0	17 - 17 (and Erlang/OTP 18 from v1.0.5)

Elixir may add compatibility to new Erlang/OTP versions on patch releases, such as support for Erlang/OTP 20 in v1.4.5. Those releases are made for convenience and typically contain the minimum changes for Elixir to run without errors, if any changes are necessary. Only the next minor release, in this case v1.5.0, effectively leverages the new features provided by the latest Erlang/OTP release.

Deprecations

Policy

Elixir deprecations happen in 3 steps:

1. The feature is soft-deprecated. It means both CHANGELOG and documentation must list the feature as deprecated but no warning is effectively emitted by running the code. There is no requirement to soft-deprecate a feature.

2. The feature is effectively deprecated by emitting warnings on usage. This is also known as hard-deprecation. In order to deprecate a feature, the proposed alternative MUST exist for AT LEAST THREE minor versions. For example, `Enum.uniq/2` was soft-deprecated in favor of `Enum.uniq_by/2` in Elixir v1.1. This means a deprecation warning may only be emitted by Elixir v1.4 or later.

3. The feature is removed. This can only happen on major releases. This means deprecated features in Elixir v1.x shall only be removed by Elixir v2.x.

Table of deprecations

The first column is the version the feature was hard deprecated. The second column shortly describes the deprecated feature and the third column explains the replacement and from which the version the replacement is available from.

Version	Deprecated feature	Replaced by (available since)
v1.20	`<<x::size(y)>>` in patterns without `^`	`<<x::size(^y)>>` (v1.15)
v1.20	`File.stream!(path, modes, lines_or_bytes)`	`File.stream!(path, lines_or_bytes, modes)` (v1.16)
v1.20	`Kernel.ParallelCompiler.async/1`	`Kernel.ParallelCompiler.pmap/2` (v1.16)
v1.20	`Logger.*_backend` functions	The `LoggerBackends` module from `:logger_backends` package
v1.20	`Logger.enable/1` and `Logger.disable/1`	`Logger.put_process_level/2` and `Logger.delete_process_level/1` respectively (v1.15)
v1.19	CLI configuration in `def project` inside `mix.exs`	Moving it to `def cli` (v1.14)
v1.19	Using `,` to separate tasks in `mix do`	Using `+` (v1.14)
v1.19	`Logger`'s `:backends` configuration	`Logger`'s `:default_handler` configuration (v1.15)
v1.19	Passing a callback to `File.cp/3`, `File.cp!/3`, `File.cp_r/3`, and `File.cp_r!/3`	The `:on_conflict` option (v1.14)
v1.18	`<%#` in EEx	`<%!--` (v1.14) or `<% #` (v1.0)
v1.18	`EEx.Engine.handle_text/2` callback in EEx	`EEx.Engine.handle_text/3` (v1.14)
v1.18	Returning a 2-arity function from `Enumerable.slice/1`	Returning a 3-arity function (v1.14)
v1.18	Ranges with negative steps in `Range.new/2`	Explicit steps in ranges (v1.11)
v1.18	`Tuple.append/2`	`Tuple.insert_at/3` (v1.0)

Version	Deprecated feature	Replaced by (available since)
v1.18	`mix cmd --app APP`	`mix do --app APP` (v1.14)
v1.18	`List.zip/1`	`Enum.zip/1` (v1.0)
v1.18	`Module.eval_quoted/3`	`Code.eval_quoted/3` (v1.0)
v1.17	Single-quoted charlists (`'foo'`)	`~c"foo"` (v1.0)
v1.17	`left..right` in patterns and guards	`left..right//step` (v1.11)
v1.17	`ExUnit.Case.register_test/4`	`register_test/6` (v1.10)
v1.17	`:all` in `IO.read/2` and `IO.binread/2`	`:eof` (v1.13)
v1.16	`~R/.../`	`~r/.../` (v1.0)
v1.16	Ranges with negative steps in `Enum.slice/2`	Explicit steps in ranges (v1.11)
v1.16	Ranges with negative steps in `String.slice/2`	Explicit steps in ranges (v1.11)
v1.15	`Calendar.ISO.day_of_week/3`	`Calendar.ISO.day_of_week/4` (v1.11)
v1.15	`Exception.exception?/1`	`Kernel.is_exception/1` (v1.11)
v1.15	`Regex.regex?/1`	`Kernel.is_struct/2` (`Kernel.is_struct(term, Regex)`) (v1.11)
v1.15	`Logger.warn/2`	`Logger.warning/2` (v1.11)
v1.14	`use Bitwise`	`import Bitwise` (v1.0)
v1.14	`~~~/1`	`bnot/2` (v1.0)
v1.14	`Application.get_env/3` and similar in module body	`Application.compile_env/3` (v1.10)
v1.14	Compiled patterns in `String.starts_with?/2`	Pass a list of strings instead (v1.0)
v1.14	`Mix.Tasks.Xref.calls/1`	Compilation tracers (outlined in `Code`) (v1.10)
v1.14	`$levelpad` in Logger	*None*
v1.14	`<|>` as a custom operator	Another custom operator (v1.0)
v1.13	`!` and `!=` in Version requirements	`~>` or `>=` (v1.0)
v1.13	`Mix.Config`	`Config` (v1.9)
v1.13	`:strip_beam` config to `mix escript.build`	`:strip_beams` (v1.9)
v1.13	`Macro.to_string/2`	`Macro.to_string/1` (v1.0)
v1.13	`System.get_pid/0`	`System.pid/0` (v1.9)
v1.12	`^^^/2`	`bxor/2` (v1.0)
v1.12	`@foo()` to read module attributes	Remove the parenthesis (v1.0)
v1.12	`use EEx.Engine`	Explicitly delegate to EEx.Engine instead (v1.0)
v1.12	`:xref` compiler in Mix	Nothing (it always runs as part of the compiler now)
v1.11	`Mix.Project.compile/2`	`Mix.Task.run("compile", args)` (v1.0)
v1.11	`Supervisor.Spec.worker/3` and `Supervisor.Spec.supervisor/3`	The new child specs outlined in `Supervisor` (v1.5)
v1.11	`Supervisor.start_child/2` and `Supervisor.terminate_child/2`	`DynamicSupervisor` (v1.6)

Version	Deprecated feature	Replaced by (available since)
v1.11	`System.stacktrace/1`	`__STACKTRACE__` in `try/catch/rescue` (v1.7)
v1.10	`Code.ensure_compiled?/1`	`Code.ensure_compiled/1` (v1.0)
v1.10	`Code.load_file/2`	`Code.require_file/2` (v1.0) or `Code.compile_file/2` (v1.7)
v1.10	`Code.loaded_files/0`	`Code.required_files/0` (v1.7)
v1.10	`Code.unload_file/1`	`Code.unrequire_files/1` (v1.7)
v1.10	Passing non-chardata to `Logger.log/2`	Explicitly convert to string with `to_string/1` (v1.0)
v1.10	`:compile_time_purge_level` in `Logger` app environment	`:compile_time_purge_matching` in `Logger` app environment (v1.7)
v1.10	`Supervisor.Spec.supervise/2`	The new child specs outlined in `Supervisor` (v1.5)
v1.10	`:simple_one_for_one` strategy in `Supervisor`	`DynamicSupervisor` (v1.6)
v1.10	`:restart` and `:shutdown` in `Task.Supervisor.start_link/1`	`:restart` and `:shutdown` in `Task.Supervisor.start_child/3` (v1.6)
v1.9	Enumerable keys in `Map.drop/2`, `Map.split/2`, and `Map.take/2`	Call `Enum.to_list/1` on the second argument before hand (v1.0)
v1.9	`Mix.Project.load_paths/1`	`Mix.Project.compile_path/1` (v1.0)
v1.9	Passing `:insert_replaced` to `String.replace/4`	Use `:binary.replace/4` (v1.0)
v1.8	Passing a non-empty list to `Collectable.into/1`	`++/2` or `Keyword.merge/2` (v1.0)
v1.8	Passing a non-empty list to `:into` in `for/1`	`++/2` or `Keyword.merge/2` (v1.0)
v1.8	Passing a non-empty list to `Enum.into/2`	`++/2` or `Keyword.merge/2` (v1.0)
v1.8	Time units in its plural form, such as: `:seconds`, `:milliseconds`, and the like	Use the singular form, such as: `:second`, `:millisecond`, and so on (v1.4)
v1.8	`Inspect.Algebra.surround/3`	`Inspect.Algebra.concat/2` and `Inspect.Algebra.nest/2` (v1.0)
v1.8	`Inspect.Algebra.surround_many/6`	`Inspect.Algebra.container_doc/6` (v1.6)
v1.9	`--detached` in `Kernel.CLI`	`--erl "-detached"` (v1.0)
v1.8	`Kernel.ParallelCompiler.files/2`	`Kernel.ParallelCompiler.compile/2` (v1.6)
v1.8	`Kernel.ParallelCompiler.files_to_path/2`	`Kernel.ParallelCompiler.compile_to_path/2` (v1.6)
v1.8	`Kernel.ParallelRequire.files/2`	`Kernel.ParallelCompiler.require/2` (v1.6)
v1.8	Returning `{:ok, contents}` or `:error` from `Mix.Compilers.Erlang.compile/6`'s callback	Return `{:ok, contents, warnings}` or `{:error, errors, warnings}` (v1.6)
v1.8	`System.cwd/0` and `System.cwd!/0`	`File.cwd/0` and `File.cwd!/0` respectively (v1.0)
v1.7	`Code.get_docs/2`	`Code.fetch_docs/1` (v1.7)

Version	Deprecated feature	Replaced by (available since)
v1.7	`Enum.chunk/2,3,4`	`Enum.chunk_every/2` and `Enum.chunk_every/3,4` (v1.5)
v1.7	Calling `super/1` in `GenServer` callbacks	Implementing the behaviour explicitly without calling `super/1` (v1.0)
v1.7	`not left in right`	`left not in right` (v1.5)
v1.7	`Registry.start_link/3`	`Registry.start_link/1` (v1.5)
v1.7	`Stream.chunk/2,3,4`	`Stream.chunk_every/2` and `Stream.chunk_every/3,4` (v1.5)
v1.6	`Enum.partition/2`	`Enum.split_with/2` (v1.4)
v1.6	`Macro.unescape_tokens/1,2`	Use `Enum.map/2` to traverse over the arguments (v1.0)
v1.6	`Module.add_doc/6`	`@doc` module attribute (v1.0)
v1.6	`Range.range?/1`	Pattern match on `_..._` (v1.0)
v1.5	`()` to mean `nil`	`nil` (v1.0)
v1.5	`char_list/0` type	`charlist/0` type (v1.3)
v1.5	`Atom.to_char_list/1`	`Atom.to_charlist/1` (v1.3)
v1.5	`Enum.filter_map/3`	`Enum.filter/2` + `Enum.map/2` or `for/1` comprehensions (v1.0)
v1.5	`Float.to_char_list/1`	`Float.to_charlist/1` (v1.3)
v1.5	`GenEvent` module	`Supervisor` and `GenServer` (v1.0)
v1.5	`<%=` in middle and end expressions in `EEx`	Use `<%` (`<%=` is allowed only in start expressions) (v1.0)
v1.5	`:as_char_lists` value in `Inspect.Opts.t/0` type	`:as_charlists` value (v1.3)
v1.5	`:char_lists` key in `Inspect.Opts.t/0` type	`:charlists` key (v1.3)
v1.5	`Integer.to_char_list/1,2`	`Integer.to_charlist/1` and `Integer.to_charlist/2` (v1.3)
v1.5	`to_char_list/1`	`to_charlist/1` (v1.3)
v1.5	`List.Chars.to_char_list/1`	`List.Chars.to_charlist/1` (v1.3)
v1.5	`@compile {:parse_transform, _}` in `Module`	*None*
v1.5	`Stream.filter_map/3`	`Stream.filter/2` + `Stream.map/2` (v1.0)
v1.5	`String.ljust/3` and `String.rjust/3`	Use `String.pad_leading/3` and `String.pad_trailing/3` with a binary padding (v1.3)
v1.5	`String.lstrip/1` and `String.rstrip/1`	`String.trim_leading/1` and `String.trim_trailing/1` (v1.3)
v1.5	`String.lstrip/2` and `String.rstrip/2`	Use `String.trim_leading/2` and `String.trim_trailing/2` with a binary as second argument (v1.3)

Version	Deprecated feature	Replaced by (available since)
v1.5	`String.strip/1` and `String.strip/2`	`String.trim/1` and `String.trim/2` (v1.3)
v1.5	`String.to_char_list/1`	`String.to_charlist/1` (v1.3)
v1.4	Anonymous functions with no expression after `->`	Use an expression or explicitly return `nil` (v1.0)
v1.4	Support for making private functions overridable	Use public functions (v1.0)
v1.4	Variable used as function call	Use parentheses (v1.0)
v1.4	`Access.key/1`	`Access.key/2` (v1.3)
v1.4	`Behaviour` module	`@callback` module attribute (v1.0)
v1.4	`Enum.uniq/2`	`Enum.uniq_by/2` (v1.2)
v1.4	`Float.to_char_list/2`	`:erlang.float_to_list/2` (Erlang/OTP 17)
v1.4	`Float.to_string/2`	`:erlang.float_to_binary/2` (Erlang/OTP 17)
v1.4	`HashDict` module	`Map` (v1.2)
v1.4	`HashSet` module	`MapSet` (v1.1)
v1.4	`IEx.Helpers.import_file/2`	`IEx.Helpers.import_file_if_available/1` (v1.3)
v1.4	`Mix.Utils.camelize/1`	`Macro.camelize/1` (v1.2)
v1.4	`Mix.Utils.underscore/1`	`Macro.underscore/1` (v1.2)
v1.4	Multi-letter aliases in `OptionParser`	Use single-letter aliases (v1.0)
v1.4	`Set` module	`MapSet` (v1.1)
v1.4	`Stream.uniq/2`	`Stream.uniq_by/2` (v1.2)
v1.3	`\x{X*}` inside strings/sigils/charlists	`\uXXXX` or `\u{X*}` (v1.1)
v1.3	`Dict` module	`Keyword` (v1.0) or `Map` (v1.2)
v1.3	`:append_first` option in `defdelegate/2`	Define the function explicitly (v1.0)
v1.3	Map/dictionary as 2nd argument in `Enum.group_by/3`	`Enum.reduce/3` (v1.0)
v1.3	`Keyword.size/1`	`length/1` (v1.0)
v1.3	`Map.size/1`	`map_size/1` (v1.0)
v1.3	`/r` option in `Regex`	`/U` (v1.1)
v1.3	`Set` behaviour	`MapSet` data structure (v1.1)
v1.3	`String.valid_character?/1`	`String.valid?/1` (v1.0)
v1.3	`Task.find/2`	Use direct message matching (v1.0)
v1.3	Non-map as 2nd argument in `URI.decode_query/2`	Use a map (v1.0)
v1.2	`Dict` behaviour	`Map` and `Keyword` (v1.0)
v1.1	`?\xHEX`	`0xHEX` (v1.0)
v1.1	`Access` protocol	`Access` behaviour (v1.1)
v1.1	`as: true | false` in `alias/2` and `require/2`	*None*

Gradual set-theoretic types

Elixir is in the process of incorporating set-theoretic types into the compiler. This document outlines the current stage of our implementation for this Elixir version. Elixir's type system is:

- **sound** - the inferred and assigned by the type system align with the behaviour of the program
- **gradual** - Elixir's type system includes the `dynamic()` type, which can be used when the type of a variable or expression is checked at runtime. However, instead of simply discarding all typing information, Elixir's `dynamic()` type works as a range. For example, if you write `dynamic(integer() or binary())`, Elixir's type system will still emit violations if none of those types are accepted. Furthermore, in the absence of `dynamic()`, Elixir's type system behaves as a static one
- **developer friendly** - the types are described, implemented, and composed using basic set operations: unions, intersections, and negation (hence it is a set-theoretic type system)

The current milestone aims to infer types from existing programs and use them for type checking, enabling the Elixir compiler to find faults and bugs in codebases without requiring changes to existing software. User provided type signatures are planned for future releases. The underlying principles, theory, and roadmap of our work have been outlined in "The Design Principles of the Elixir Type System" by Giuseppe Castagna, Guillaume Duboc, José Valim.

A gentle introduction

Types in Elixir are written using the type named followed by parentheses, such as `integer()` or `list(integer())`.

The basic types are:

```
atom()
binary()
bitstring()
empty_list()
integer()
float()
function()
map()
non_empty_list(elem_type, tail_type)
pid()
port()
reference()
tuple()
```

Many of the types above can also be written more precisely. We will discuss their syntax in the next sections, but here are two examples:

- While `atom()` represents all atoms, the atom `:ok` can also be represented in the type system as `:ok`
- While `tuple()` represents all tuples, you can specify the type of a two-element tuple where the first element is the atom `:ok` and the second is an integer as `{:ok, integer()}`

There are also three special types: `none()` (represents an empty set), `term()` (represents all types), `dynamic()` (represents a range of the given types).

Given the types are set-theoretic, we can compose them using unions (`or`), intersections (`and`), and negations (`not`). For example, to say a function returns either atoms or integers, one could write: `atom() or integer()`.

Intersections will find the elements in common between the operands. For example, `atom() and integer()`, which in this case is the empty set `none()`. You can combine intersections and negations to perform difference, for example, to say that a function expects all atoms, except `nil` (which is an atom), you could write: `atom() and not nil`.

You can find a complete reference in the set-theoretic types cheatsheet.

The syntax of data types

In this section we will cover the syntax of all data types. At the moment, developers will interact with those types mostly through compiler warnings and diagnostics.

Broad types

These types are broad in that they cannot represent individual elements, only the whole set. For example, the numbers `1` and `42` are both represented by the type `integer()`.

They are: `binary()`, `bitstring()`, `integer()`, `float()`, `pid()`, `port()`, `reference()`.

The `binary()` type is a subtype of the less frequently used `bitstring()` type, as binaries are bitstrings where the number of bits is divisible by 8.

Atoms

You can represent all atoms as `atom()`. You can also represent each individual atom using their literal syntax. For instance, the atom `:foo` and `:hello_world` are also valid (distinct) types.

`nil`, `true`, and `false` are also atoms and can be written as is. `boolean()` is a convenience type alias that represents `true or false`.

Tuples

You can represent all tuples as `tuple()`. Tuples may also be written using the curly brackets syntax, such as `{:ok, binary()}`.

You may use `...` at the end of the tuple to imply the overall size of the tuple is unknown. For example, the following tuple has at least two elements: `{:ok, binary(), ...}`.

Lists

You can represent all *proper* lists as `list()`, which also includes the empty list.

You can also specify the type of the list element as argument. For example, `list(integer())` represents the values `[]` and `[1, 2, 3]`, but not `[1, "two", 3]`.

Internally, Elixir represents the type `list(a)` as the union two distinct types, `empty_list()` and `not_empty_list(a)`. In other words, `list(integer())` is equivalent to `empty_list() or non_empty_list(integer())`.

Improper lists

While most developers will simply use `list(a)`, the type system can express all different representations of lists in Elixirby passing a second argument to `non_empty_list`, which represents the type of the tail.

A proper list is one where the tail is the empty list itself. The type `non_empty_list(integer())` is equivalent to `non_empty_list(integer(), empty_list())`.

If the `tail_type` is anything but a list, then we have an improper list. For example, the value `[1, 2 | 3]` would have the type `non_empty_list(integer(), integer())`.

If you pass a list type as the tail, then the list type is merged into the element type. For example, `non_empty_list(integer(), list(binary()))` is the same as `non_empty_list(integer() or binary(), empty_list())`.

Maps

You can represent all maps as `map()`.

Maps may also be written using their literal syntax:

```
%{name: binary(), age: integer()}
```

which outlines a map with exactly two keys, `:name` and `:age`, and values of type `binary()` and `integer()` respectively. We say the map above is "closed": it only supports the keys explicitly defined. We can also mark a map as "open", by including `...` as its first element:

```
%{..., name: binary(), age: integer()}
```

The type above says the keys `:name` and `:age` must exist, with their respective types, but other keys may be present. The `map()` type is the same as `%{...}`. For the empty map, you may write `%{}`, although we recommend using `empty_map()` for clarity.

Optional keys

A key may be marked as optional using the `if_set/1` operation on its value type:

```
%{name: binary(), age: if_set(integer())}
```

is a map that certainly has the `:name` key but it may have the `:age` key (and if it has such key, its value type is `integer()`).

You can also use `not_set()` to denote a key cannot be present:

```
%{..., age: not_set()}
```

The type above says the map may have any key, except the `:age` one. This is, for instance, the type returned by `Map.delete(map, :age)`.

Domain types

In the examples above, all map keys were atoms, but we can also use other types as map keys. For example:

```
# Closed map
%{binary() or atom() => integer()}

# Open map
%{..., binary() or atom() => integer()}
```

Currently, the type system only tracks the top of each individual type as the domain keys. For example, if you say:

```
%{list(integer()) => integer(), list(binary()) => binary()}
```

That's the same as specifying all lists:

```
%{list() => integer() or binary()}
```

The supported domain keys are `atom()`, `bitstring()`, `binary()`, `integer()`, `float()`, `fun()`, `list()`, `map()`, `pid()`, `port()`, `reference()`, and `tuple()`. In the case of maps, the `bitstring()` domain stores exclusively keys which are not binary. The ones which are `binary()` are stored under the `binary()` domain.

Furthermore, it is important to note that domain keys are, by definition, optional. Whenever you have a `%{integer() => integer()}` and you try to fetch a key, we must assume the key may not exist (after all, it is not possible to store all integers as map keys as they are infinite).

Mixed keys

It is also possible to mix domain and atom keys. For example, the following map says that all atom keys are of type `binary()`, except the `:root` key, which has type `integer()`:

```
# Closed map
%{atom() => binary(), root: integer()}

# Open map
%{..., atom() => binary(), root: integer()}
```

The order of the keys is of increasing precision. `:root` is more precise than `atom()`, therefore it comes later. This mirrors the runtime semantics of maps, where duplicate keys override the value of earlier ones.

Functions

You can represent all functions as `function()`. However, in practice, most functions are represented as arrows. For example, a function that receives an integer and return boolean would be written as `(integer() -> boolean())`. A function that receives two integers and return a string (i.e. a binary) would be written as `(integer(), integer() -> binary())`.

When representing functions with multiple clauses, which may take different input types, we use intersections. For example, imagine the following function:

```
def negate(x) when is_integer(x), do: -x
def negate(x) when is_boolean(x), do: not x
```

If you give it an integer, it negates it. If you give it a boolean, it negates it.

We can say this function has the type `(integer() -> integer())` because it is capable of receiving an integer and returning an integer. In this case, `(integer() -> integer())` is a set that represents all functions that can receive an integer and return an integer. Even though this function can receive other arguments and return other values, it is still part of the `(integer() -> integer())` set.

This function also has the type `(boolean() -> boolean())`, because it also receives booleans and returns booleans. If you pass the function above to another function that expects `(boolean() -> boolean())`, type checking will succeed. Therefore, we can say the overall type of the function is `(integer() -> integer()) and (boolean() -> boolean())`. The intersection means the function belongs to both sets.

At this point, you may ask, why not a union? As a real-world example, take a t-shirt with green and yellow stripes. We can say the t-shirt belongs to the set of "t-shirts with green color". We can also say the t-shirt belongs to the set of "t-shirts with yellow color". Let's see the difference between unions and intersections:

- `(t_shirts_with_green() or t_shirts_with_yellow())` - contains t-shirts with either green or yellow, such as green, green and red, green and yellow, but also only yellow, yellow and red, etc.

- `(t_shirts_with_green() and t_shirts_with_yellow())` - contains t-shirts with both green and yellow (and maybe other colors)

Since the t-shirt has both colors, we could say it belongs to the union of green and yellow t-shirts, but doing so would not capture the fact it is both green and yellow. Therefore it is more precise to say it belongs to the intersection of both sets. The same way that a function that goes from `(integer() -> integer())` and `(boolean() -> boolean())` is also an intersection. In practice, it is not useful to define the union of two functions in Elixir, so the compiler will point you to the right direction if you specify the wrong one.

The `dynamic()` type

Existing Elixir programs do not have type declarations, but we still want to be able to type check them. This is done with the introduction of the `dynamic()` type.

When Elixir sees the following function:

```
def negate(x) when is_integer(x), do: -x
def negate(x) when is_boolean(x), do: not x
```

Elixir type checks it as if the function had the type `(dynamic() -> dynamic())`. Then, based on patterns and guards, we can refine the value of the variable `x` to be `dynamic() and integer()` and `dynamic() and boolean()` for each clause respectively. We say `dynamic()` is a gradual type, which leads us to *gradual set-theoretic types*.

The simplest way to reason about `dynamic()` in Elixir is that it is a range of types. If you have a type `atom() or integer()`, the underlying code needs to work with both `atom() or integer()`. For example, if you call `Integer.to_string(var)`, and `var` has type `atom() or integer()`, the type system will emit a warning, because `Integer.to_string/1` does not accept atoms.

However, by intersecting a type with `dynamic()`, we make the type gradual and therefore only a subset of the type needs to be valid. For instance, if you call `Integer.to_string(var)`, and `var` has type `dynamic() and (atom() or integer())`, the type system will not emit a warning, because `Integer.to_string/1` works with at least one of the types. For convenience, most programs will write `dynamic(atom() or integer())` instead of the intersection. They are equivalent.

Compared to other gradually typed languages, the `dynamic()` type in Elixir is quite powerful: it restricts our program to certain types, via intersections, while still emitting warnings once it is certain the code will fail. This makes `dynamic()` an excellent tool for typing existing Elixir code with meaningful warnings.

If the user provides their own types, and those types are not `dynamic()`, then Elixir's type system behaves as a statically typed one. This brings us to one last property of dynamic types in Elixir: dynamic types are always at the root. For example, when you write a tuple of type `{:ok, dynamic()}`, Elixir will rewrite it to `dynamic({:ok, term()})`. While this has the downside that you cannot make part of a tuple/map/list gradual, only the whole tuple/map/list, it comes with the upside that dynamic is always explicitly at the root, making it harder to accidentally sneak `dynamic()` in a statically typed program.

Type inference

Type inference (or reconstruction) is the ability of a type system to automatically deduce, either partially or fully, the type of an expression at compile time. Type inference may occur at different levels. For example, many programming languages can automatically infer the types of variables, also known "local type inference", but not all can infer type signatures of functions.

Inferring type signatures comes with a series of trade-offs:

- Speed - type inference algorithms are often more computationally intensive than type checking algorithms.

- Expressiveness - in any given type system, the constructs that support inference are always a subset of those that can be type-checked. Therefore, if a programming language is restricted to fully reconstructed types, it is less expressive than a solely type checked counterpart.

- Incremental compilation - type inference complicates incremental compilation. If module A depends on module B, which depends on module C, a change to C may require the type signature in B to be reconstructed, which may then require A to be recomputed (and so on). This dependency chain may require large projects to explicitly add type signatures for stability and compilation efficiency.

- Cascading errors - when a user accidentally makes type errors or the code has conflicting assumptions, type inference may lead to less clear error messages as the type system tries to reconcile diverging type assumptions across code paths.

On the other hand, type inference offers the benefit of enabling type checking for functions and codebases without requiring the user to add type annotations. To balance these trade-offs, Elixir aims to provide type inference across dependencies: our goal is to infer the types of functions considering the current module, Elixir's standard library and your dependencies, while calls to modules within the same project are assumed

to be `dynamic()`. Once types are inferred, then the whole project is type checked considering all modules and all types (inferred or otherwise).

Type inference in Elixir is best-effort: it doesn't guarantee it will find all possible type incompatibilities, only that it may find bugs where all combinations of a type *will* fail, even in the absence of explicit type annotations. It is meant to be an efficient routine that brings developers some benefits of static typing, without requiring any effort from them and keeping the expressiveness of the language.

In the long term, Elixir developers who want static typing guarantees may explicitly add type signatures to their functions (see "Roadmap"). Any function with an explicit type signature will be typed checked against the user-provided annotations, as in other statically typed languages.

False positives

Elixir's type inference generally avoids emitting false positive type violations: which are warnings emitted by the type checker when there are no runtime errors. However, in some situations, those may happen and are documented below.

`for`-comprehensions assume they are executed at least once

For comprehensions in Elixir assume they are executed at least once. Take this code:

```
def example(x, list) do
  for _i <- list do
    Atom.to_string(x)
  end

  x + 1
end
```

`x + 1` will fail because it assumes `x` is an atom from the `Atom.to_string(x)` call, even though the function may raise no runtime error if `list` is an empty list. This is intentional, as it helps find discrepancies inside and outside comprehensions. You can address this by explicitly wrapping the comprehension in a `if list != [] do` block (or similar condition).

Struct update syntax must be statically proven

Elixir will warn if you use the struct update syntax and it is not statically proven that the given value does not have said struct type. For example:

```
user = find_user_by_id(42)
%User{user | name: "John Doe"}
```

Even though it is guaranteed at runtime that user is always a `User` struct. If the type system cannot prove it, it will emit a typing violation. This is how stuct updates work by design. In such cases, you can address it by matching on the struct when the user variable is defined:

```
%User{} = user = find_user_by_id(42)
%User{user | name: "John Doe"}
```

Roadmap

At this moment, Elixir implements type inference of all language constructs. The goal is to assess performance and collect feedback on the quality of error messages, before introducing user facing types.

If the results are satisfactory, the next milestone will include a mechanism for defining typed structs. Elixir programs frequently pattern match on structs, which reveals information about the struct fields, but it knows

nothing about their respective types. By propagating types from structs and their fields throughout the program, we will increase the type system's ability to find errors while further straining our type system implementation. Proposals including the required changes to the language surface will be sent to the community once we reach this stage.

The third milestone is to introduce set-theoretic type signatures for functions. Unfortunately, the existing Erlang Typespecs are not precise enough for set-theoretic types and they will be phased out of the language and have their postprocessing moved into a separate library once this stage concludes.

Resources

- (Paper) "The Design Principles of the Elixir Type System" by Giuseppe Castagna, Guillaume Duboc, José Valim
- (Video) "The foundations of the Elixir type system" by José Valim
- (Video) "Precision in type system design" by José Valim

Acknowledgements

The type system was made possible thanks to a partnership between CNRS and Remote. The development work is currently sponsored by Fresha, and Tidewave.

Library guidelines

This document outlines general guidelines for those writing and publishing Elixir libraries meant to be consumed by other developers.

Getting started

You can create a new Elixir library by running the `mix new` command:

```
$ mix new my_library
```

The project name is given in the `snake_case` convention where all letters are lowercase and words are separate with underscores. This is the same convention used by variables, function names and atoms in Elixir. See the Naming Conventions document for more information.

Every project has a `mix.exs` file, with instructions on how to build, compile, run tests, and so on. Libraries commonly have a `lib` directory, which includes Elixir source code, and a `test` directory. A `src` directory may also exist for Erlang sources.

The `mix new` command also allows the `--sup` option to scaffold a new project with a supervision tree out of the box. For more information on running your project, see the official Mix & OTP guide or Mix documentation.

Publishing

Writing code is only the first of many steps to publish a package. We strongly recommend developers to:

- Choose a versioning schema. Elixir requires versions to be in the format `MAJOR.MINOR.PATCH` but the meaning of those numbers is up to you. Most projects choose Semantic Versioning.

- Choose a license. The most common licenses in the Elixir community are the MIT License and the Apache License 2.0. The latter is also the one used by Elixir itself.

- Run the code formatter. The code formatter formats your code according to a consistent style shared by your library and the whole community, making it easier for other developers to understand your code and contribute.

- Write tests. Elixir ships with a test-framework named ExUnit. The project generated by `mix new` includes sample tests and doctests.

- Write documentation. The Elixir community is proud of treating documentation as a first-class citizen and making documentation easily accessible. Libraries contribute to the status quo by providing complete API documentation with examples for their modules, types and functions. See the Writing documentation chapter of the Getting Started guide for more information. Projects like ExDoc can be used to generate HTML and EPUB documents from the documentation. ExDoc also supports "extra pages", like this one that you are reading. Such pages augment the documentation with tutorials, guides, references, and even cheat-sheets.

- Follow best practices. The Elixir project documents a series of anti-patterns that you may want to avoid in your code. The process-related anti-patterns and meta-programming anti-patterns are of special attention to library authors.

Projects are often made available to other developers by publishing a Hex package. Hex also supports private packages for organizations. If ExDoc is configured for the Mix project, publishing a package on Hex will also automatically publish the generated documentation to HexDocs.

Dependency handling

When your library is used as a dependency, it runs by default in the `:prod` environment. Therefore, if your library has dependencies that are only useful in development or testing, you want to specify those dependencies with the `:only` option. You can also specify `:optional` dependencies in your library, which are not enforced upon users of your library. In such cases, you should also consider compiling your projects with the `mix compile --no-optional-deps --warnings-as-errors` in your test environments, to ensure your library compiles without warnings even if optional dependencies are missing. See `mix deps` for all available options.

Keep in mind your library's lockfile (usually named `mix.lock`) is *ignored by the host project*. Running `mix deps.get` in the host project attempts to get the latest possible versions of your library's dependencies, as specified by the requirements in the `deps` section of your `mix.exs`. These versions might be greater than those stored in your `mix.lock` (and hence used in your tests / CI).

On the other hand, contributors of your library, need a deterministic build, which implies the presence of `mix.lock` in your Version Control System (VCS), such as `git`.

If you want to validate both scenarios, you should check the `mix.lock` into version control and run two different Continuous Integration (CI) workflows: one that relies on the `mix.lock` for deterministic builds, and another one, that starts with `mix deps.unlock --all` and always compiles your library and runs tests against latest versions of dependencies. The latter one might be even run nightly or otherwise recurrently to stay notified about any possible issue in regard to dependencies updates.

Dependency Version Requirements

When depending on other libraries, the dependency version requirements are ultimately up to you. However, you should consider the effects that an overly strict dependency requirement can have on users of your library. Most dependencies adopt Semantic Versioning, and therefore provide reasonable guarantees about what each release contains. For instance, if you use `{:some_dep, “== 0.2.3”}`, this prevents users from using any other version but the one that you specified, which means that they cannot receive bug fix upgrades to that package. When in doubt, use a dependency in the format of `"~> x.y"`. This prevents the user from using a higher major version of the library, but allows them to upgrade to newer minor and patch versions, which should only include bug fixes and non-breaking improvements.

The exception to this is pre 1.0 libraries using Semantic Versioning, which provide no guarantees about what might change from one version to the next. In this scenario, depending on the full patch version, i.e `"~> 0.1.2"` is a better default.

A common mistake is to use a dependency in the format of `"~> x.y.z"` to express "a version greater than `x.y.z`". For example, if you are depending on `"~> 1.2"`, and the dependency publishes a fix in version `1.2.1` that you need for the next version of your library. If you use `"~> 1.2.1"` to express that dependency, you are preventing users from upgrading to `"1.3.0"` or higher! Instead of `"~> 1.2.1"`, you should use `"~> 1.2 and >= 1.2.1"` as the version requirement. This allows users to use any version less than `2.0`, and greater than `1.2.1`.

Naming conventions

This document is a reference of the naming conventions in Elixir, from casing to punctuation characters.

The naming convention is, by definition, a subset of the Elixir syntax. A convention aims to follow and set best practices for language and the community. If instead you want a complete reference into the Elixir syntax, beyond its conventions, see the Syntax reference.

Casing

Elixir developers must use `snake_case` when defining variables, function names, module attributes, and the like:

```
some_map = %{this_is_a_key: "and a value"}
is_map(some_map)
```

Aliases, commonly used as module names, are an exception as they must be capitalized and written in `CamelCase`, like `OptionParser`. For aliases, capital letters are kept in acronyms, like `ExUnit.CaptureIO` or `Mix.SCM`.

Atoms can be written either in `:snake_case` or `:CamelCase`, although the convention is to use the snake case version throughout Elixir.

Generally speaking, filenames follow the `snake_case` convention of the module they define. For example, `MyApp` should be defined inside the `my_app.ex` file. However, this is only a convention. At the end of the day any filename can be used as they do not affect the compiled code in any way.

Underscore (`_foo`)

Elixir relies on underscores in different situations.

For example, a value that is not meant to be used must be assigned to `_` or to a variable starting with underscore:

```
iex> {:ok, _contents} = File.read("README.md")
```

Function names may also start with an underscore. Such functions are never imported by default:

```
iex> defmodule Example do
...>   def _wont_be_imported do
...>     :oops
...>   end
...> end

iex> import Example
iex> _wont_be_imported()
** (CompileError) iex:1: undefined function
_wont_be_imported/0
```

Due to this property, Elixir relies on functions starting with underscore to attach compile-time metadata to modules. Such functions are most often in the `__foo__` format. For example, every module in Elixir has an `__info__/1` function:

```
iex> String.__info__(:functions)
[at: 2, capitalize: 1, chunk: 2, ...]
```

Elixir also includes five special forms that follow the double underscore format: `__CALLER__/0`, `__DIR__/0`, `__ENV__/0` and `__MODULE__/0` retrieve compile-time information about the current environment, while `__STACKTRACE__/0` retrieves the stacktrace for the current exception.

Trailing bang (`foo!`)

A trailing bang (exclamation mark) signifies a function or macro where failure cases raise an exception. They most often exist as a "raising variant" of a function that returns `:ok`/`:error` tuples (or `nil`).

One example is `File.read/1` and `File.read!/1`. `File.read/1` will return a success or failure tuple, whereas `File.read!/1` will return a plain value or else raise an exception:

```
iex> File.read("file.txt")
{:ok, "file contents"}
iex> File.read("no_such_file.txt")
{:error, :enoent}

iex> File.read!("file.txt")
"file contents"
iex> File.read!("no_such_file.txt")
** (File.Error) could not read file no_such_file.txt: no
such file or directory
```

The version without `!` is preferred when you want to handle different outcomes using pattern matching:

```
case File.read(file) do
  {:ok, body} -> # do something with the `body`
```

```
  {:error, reason} -> # handle the error caused by `reason`
end
```

However, if you expect the outcome to always be successful (for instance, if you expect the file always to exist), the bang variation can be more convenient and will raise a more helpful error message (than a failed pattern match) on failure.

When thinking about failure cases, we are often thinking about semantic errors related to the operation being performed, such as failing to open a file or trying to fetch key from a map. Errors that come from invalid argument types, or similar, must always raise regardless if the function has a bang or not. In such cases, the exception is often an `ArgumentError` or a detailed `FunctionClauseError`:

```
iex(1)> File.read(123)
** (FunctionClauseError) no function clause matching in
IO.chardata_to_string/1

    The following arguments were given to
IO.chardata_to_string/1:

        # 1
        123

    Attempted function clauses (showing 2 out of 2):

        def chardata_to_string(string) when
is_binary(string)
        def chardata_to_string(list) when is_list(list)
```

More examples of paired functions: `Base.decode16/2` and `Base.decode16!/2`, `File.cwd/0` and `File.cwd!/0`. In some situations,

you may have bang functions without a non-bang counterpart. They also imply the possibility of errors, such as: `Protocol.assert_protocol!/1` and `PartitionSupervisor.resize!/2`. This can be useful if you foresee the possibility of adding a non-raising variant in the future.

Trailing question mark (`foo?`)

Functions that return a boolean are named with a trailing question mark.

Examples: `Keyword.keyword?/1`, `Mix.debug?/0`, `String.contains?/2`

However, functions that return booleans and are valid in guards follow another convention, described next.

`is_` prefix (`is_foo`)

Type checks and other boolean checks that are allowed in guard clauses are named with an `is_` prefix.

Examples: `Integer.is_even/1`, `is_list/1`

These functions and macros follow the Erlang convention of an `is_` prefix, instead of a trailing question mark, precisely to indicate that they are allowed in guard clauses. Type checks that are not valid in guard clauses do not follow this convention, such as `Keyword.keyword?/1`.

A trailing question mark should not be used in combination with the `is_` prefix.

Special names

Some names have specific meaning in Elixir. We detail those cases below.

length and size

When you see `size` in a function name, it means the operation runs in constant time (also written as "O(1) time") because the size is stored alongside the data structure.

Examples: `map_size/1`, `tuple_size/1`

When you see `length`, the operation runs in linear time ("O(n) time") because the entire data structure has to be traversed.

Examples: `length/1`, `String.length/1`

In other words, functions using the word "size" in its name will take the same amount of time whether the data structure is tiny or huge. Conversely, functions having "length" in its name will take more time as the data structure grows in size.

get, fetch, fetch!

When you see the functions `get`, `fetch`, and `fetch!` for key-value data structures, you can expect the following behaviours:

- `get` returns a default value (which itself defaults to `nil`) if the key is not present, or returns the requested value.
- `fetch` returns `:error` if the key is not present, or returns `{:ok, value}` if it is.
- `fetch!` *raises* if the key is not present, or returns the requested value.

Examples: `Map.get/2`, `Map.fetch/2`, `Map.fetch!/2`, `Keyword.get/2`, `Keyword.fetch/2`, `Keyword.fetch!/2`

compare

The function `compare/2` should return `:lt` if the first term is less than the second, `:eq` if the two terms compare as equivalent, or `:gt` if the first term is greater than the second.

Examples: `DateTime.compare/2`

Note that this specific convention is important due to the expectations of `Enum.sort/2`

Operators reference

This document is a complete reference of operators in Elixir, how they are parsed, how they can be defined, and how they can be overridden.

General operators

Elixir provides the following built-in operators:

- `+` and `-` - unary positive/negative
- `+`, `-`, `*`, and `/` - basic arithmetic operations
- `++` and `--` - list concatenation and subtraction
- `and` and `&&` - strict and relaxed boolean "and"
- `or` and `||` - strict and relaxed boolean "or"
- `not` and `!` - strict and relaxed boolean "not"
- `in` and `not in` - membership
- `@` - module attribute
- `..`, `..`, and `..//` - range creation
- `<>` - binary concatenation
- `|>` - pipeline
- `=~` - text-based match

Many of those can be used in guards. Consult the list of allowed guard functions and operators.

Additionally, there are a few other operators that Elixir parses but doesn't actually use. See Custom and overridden operators below for a list and for guidelines about their use.

Some other operators are special forms and cannot be overridden:

- `^` - pin operator

- `.` - dot operator
- `=` - match operator
- `&` - capture operator
- `::` - type operator

Finally, these operators appear in the precedence table below but are only meaningful within certain constructs:

- `=>` - see `%{}`
- `when` - see Guards
- `<-` - see `for` and `with`
- `\\` - see Default arguments

Comparison operators

Elixir provides the following built-in comparison operators (all of which can be used in guards):

- `==` - equal to
- `===` - strictly equal to
- `!=` - not equal to
- `!==` - strictly not equal to
- `<` - less-than
- `>` - greater-than
- `<=` - less-than or equal to
- `>=` - greater-than or equal to

The only difference between `==` and `===` is that `===` is strict when it comes to comparing integers and floats:

```
iex> 1 == 1.0
true
```

```
iex> 1 === 1.0
false
```

`!=` and `!==` act as the negation of `==` and `===`, respectively.

Operator precedence and associativity

The following is a list of all operators that Elixir is capable of parsing, ordered from higher to lower precedence, alongside their associativity:

Operator	Associativity
`@`	Unary
`.`	Left
`+ - ! ^ not`	Unary
`**`	Left
`* /`	Left
`+ -`	Left
`++ -- +++ --- .. <>`	Right
`//` (valid only inside `../2`)	Right
`in not in`	Left
`|> <<< >>> <<~ ~>> <~ ~> <~>`	Left
`< > <= >=`	Left
`== != =~ === !==`	Left
`&& &&& and`	Left
`|| ||| or`	Left
`=`	Right
`&, ...`	Unary
`|`	Right
`::`	Right

Operator	Associativity
`when`	Right
`<- \\`	Left
`=>` (valid only inside `%{}`)	None

Elixir also has two ternary operators:

Operator	Associativity
`first..last//step`	Right
`%{map | key => value, ...}`	None

> **Deprecated operator precedence**
>
> Elixir parses `not left in right` as `not(left in right)` and `!left in right` as `!(left in right)`, which mismatches the precedence table above, but such behaviour is deprecated and emits a warning. Both constructs must be written as `left not in right` instead. In future major versions, the parser will match the table above.

Custom and overridden operators

Elixir is capable of parsing a predefined set of operators. It's not possible to define new operators (as supported by some languages). However, not all operators that Elixir can parse are *used* by Elixir: for example, `+` and `||` are used by Elixir for addition and boolean *or*, but `<~>` is not used (but valid).

To define an operator, you can use the usual `def*` constructs (`def`, `defp`, `defmacro`, and so on) but with a syntax similar to how the operator is used:

```
defmodule MyOperators do
  # We define ~> to return the maximum of the given two
numbers,
  # and <~ to return the minimum.
```

```
  def a ~> b, do: max(a, b)
  def a <~ b, do: min(a, b)
end
```

To use the newly defined operators, you **have to** import the module that defines them:

```
iex> import MyOperators
iex> 1 ~> 2
2
iex> 1 <~ 2
1
```

The following is a table of all the operators that Elixir is capable of parsing, but that are not used by default:

- `|||`
- `&&&`
- `<<<`
- `>>>`
- `<<~`
- `~>>`
- `<~`
- `~>`
- `<~>`
- `+++`
- `---`
- `...`

The following operators are used by the `Bitwise` module when imported: `&&&`, `<<<`, `>>>`, and `|||`. See the `Bitwise` documentation for more

information.

Note that the Elixir community generally discourages custom operators. They can be hard to read and even more to understand, as they don't have a descriptive name like functions do. That said, some specific cases or custom domain specific languages (DSLs) may justify these practices.

It is also possible to replace predefined operators, such as `+`, but doing so is extremely discouraged.

Patterns and guards

Elixir provides pattern matching, which allows us to assert on the shape or extract values from data structures. Patterns are often augmented with guards, which give developers the ability to perform more complex checks, albeit limited.

This document provides a complete reference on patterns and guards, their semantics, where they are allowed, and how to extend them.

Patterns

Patterns in Elixir are made of variables, literals, and data structure specific syntax. One of the most used constructs to perform pattern matching is the match operator (`=`):

```
iex> x = 1
1
iex> 1 = x
1
```

In the example above, `x` starts without a value and has `1` assigned to it. Then, we compare the value of `x` to the literal `1`, which succeeds as they are both `1`.

Matching `x` against 2 would raise:

```
iex> 2 = x
** (MatchError) no match of right hand side value: 1
```

Patterns are not bidirectional. If you have a variable `y` that was never assigned to (often called an unbound variable) and you write `1 = y`, an error will be raised:

```
iex> 1 = y
** (CompileError) iex:2: undefined variable "y"
```

In other words, patterns are allowed only on the left side of `=`. The right side of `=` follows the regular evaluation semantics of the language.

Now let's cover the pattern matching rules for each construct and then for each relevant data types.

Variables

Variables in patterns are always assigned to:

```
iex> x = 1
1
iex> x = 2
2
iex> x
2
```

In other words, Elixir supports rebinding. In case you don't want the value of a variable to change, you can use the pin operator (`^`):

```
iex> x = 1
1
iex> ^x = 2
** (MatchError) no match of right hand side value: 2
```

If the same variable appears multiple times in the same pattern, then all of them must be bound to the same value:

```
iex> {x, x} = {1, 1}
{1, 1}
iex> {x, x} = {1, 2}
** (MatchError) no match of right hand side value: {1, 2}
```

However, a variable cannot be defined through itself in the same pattern. This includes cases where the variable appears on both sides of a nested match operator:

```
{:ok, x} = {x, :ok}
%{a: value} = %{b: value}
```

Such patterns are rejected as cyclic variable definitions. You can rewrite them by matching once and then using guards or separate comparisons for the additional constraints.

The underscore variable (_) has a special meaning as it can never be bound to any value. It is especially useful when you don't care about certain value in a pattern:

```
iex> {_, integer} = {:not_important, 1}
{:not_important, 1}
iex> integer
1
iex> _
** (CompileError) iex:3: invalid use of _
```

A pinned value represents the value itself and not its – even if syntactically equal – pattern. The right hand side is compared to be equal to the pinned value:

```
iex> x = %{}
%{}
iex> {:ok, %{}} = {:ok, %{a: 13}}
{:ok, %{a: 13}}
iex> {:ok, ^x} = {:ok, %{a: 13}}
** (MatchError) no match of right hand side value: {:ok, %
{a: 13}}
    (stdlib 6.2) erl_eval.erl:667: :erl_eval.expr/6
    iex:2: (file)
```

Literals (numbers and atoms)

Atoms and numbers (integers and floats) can appear in patterns and they are always represented as is. For example, an atom will only match an atom if they are the same atom:

```
iex> :atom = :atom
:atom
iex> :atom = :another_atom
** (MatchError) no match of right hand side value:
:another_atom
```

Similar rule applies to numbers. Finally, note that numbers in patterns perform strict comparison. In other words, integers to do not match floats:

```
iex> 1 = 1.0
** (MatchError) no match of right hand side value: 1.0
```

Tuples

Tuples may appear in patterns using the curly brackets syntax (`{}`). A tuple in a pattern will match only tuples of the same size, where each individual

tuple element must also match:

```
iex> {:ok, integer} = {:ok, 13}
{:ok, 13}

# won't match due to different size
iex> {:ok, integer} = {:ok, 11, 13}
** (MatchError) no match of right hand side value: {:ok, 11,
13}

# won't match due to mismatch on first element
iex> {:ok, binary} = {:error, :enoent}
** (MatchError) no match of right hand side value: {:error,
:enoent}
```

Lists

Lists may appear in patterns using the square brackets syntax (`[]`). A list in a pattern will match only lists of the same size, where each individual list element must also match:

```
iex> [:ok, integer] = [:ok, 13]
[:ok, 13]

# won't match due to different size
iex> [:ok, integer] = [:ok, 11, 13]
** (MatchError) no match of right hand side value: [:ok, 11,
13]

# won't match due to mismatch on first element
iex> [:ok, binary] = [:error, :enoent]
** (MatchError) no match of right hand side value: [:error,
:enoent]
```

Opposite to tuples, lists also allow matching on non-empty lists by using the `[head | tail]` notation, which matches on the `head` and `tail` of a list:

```
iex> [head | tail] = [1, 2, 3]
[1, 2, 3]
iex> head
1
iex> tail
[2, 3]
```

Multiple elements may prefix the `| tail` construct:

```
iex> [first, second | tail] = [1, 2, 3]
[1, 2, 3]
iex> tail
[3]
```

Note `[head | tail]` does not match empty lists:

```
iex> [head | tail] = []
** (MatchError) no match of right hand side value: []
```

Given charlists are represented as a list of integers, one can also perform prefix matches on charlists using the list concatenation operator (`++`):

```
iex> ~c"hello " ++ world = ~c"hello world"
~c"hello world"
iex> world
~c"world"
```

Which is equivalent to matching on `[?h, ?e, ?l, ?l, ?o, ?\s | world]`. Suffix matches (`hello ++ ~c" world"`) are not valid patterns.

Maps

Maps may appear in patterns using the percentage sign followed by the curly brackets syntax (`%{}`). Opposite to lists and tuples, maps perform a subset match. This means a map pattern will match any other map that has at least all of the keys in the pattern.

Here is an example where all keys match:

```
iex> %{name: name} = %{name: "meg"}
%{name: "meg"}
iex> name
"meg"
```

Here is when a subset of the keys match:

```
iex> %{name: name} = %{name: "meg", age: 23}
%{age: 23, name: "meg"}
iex> name
"meg"
```

If a key in the pattern is not available in the map, then they won't match:

```
iex> %{name: name, age: age} = %{name: "meg"}
** (MatchError) no match of right hand side value: %{name:
"meg"}
```

Note that the empty map will match all maps, which is a contrast to tuples and lists, where an empty tuple or an empty list will only match empty

tuples and empty lists respectively:

```
iex> %{} = %{name: "meg"}
%{name: "meg"}
```

Finally, note map keys in patterns must always be literals or previously bound variables matched with the pin operator.

Structs

Structs may appear in patterns using the percentage sign, the struct module name or a variable followed by the curly brackets syntax (`%{}`).

Given the following struct:

```
defmodule User do
  defstruct [:name]
end
```

Here is an example where all keys match:

```
iex> %User{name: name} = %User{name: "meg"}
%User{name: "meg"}
iex> name
"meg"
```

If an unknown key is given, the compiler will raise an error:

```
iex> %User{type: type} = %User{name: "meg"}
** (CompileError) iex: unknown key :type for struct User
```

The struct name can be extracted when putting a variable instead of a module name:

```
iex> %struct_name{} = %User{name: "meg"}
%User{name: "meg"}
iex> struct_name
User
```

Binaries

Binaries may appear in patterns using the double less-than/greater-than syntax (`<<>>`). A binary in a pattern can match multiple segments at the same time, each with different type, size, and unit:

```
iex> <<val::unit(8)-size(2)-integer>> = <<123, 56>>
"{8"
iex> val
31544
```

See the documentation for `<<>>` for a complete definition of pattern matching for binaries.

Finally, remember that strings in Elixir are UTF-8 encoded binaries. This means that, similar to charlists, prefix matches on strings are also possible with the binary concatenation operator (`<>`):

```
iex> "hello " <> world = "hello world"
"hello world"
iex> world
"world"
```

Suffix matches (`hello <> " world"`) are not valid patterns.

Guards

Guards are a way to augment pattern matching with more complex checks. They are allowed in a predefined set of constructs where pattern matching is allowed, such as function definitions, case clauses, and others.

Not all expressions are allowed in guard clauses, but only a handful of them. This is a deliberate choice. This way, Elixir (through Erlang) ensures that all guards are predictable (no mutations or other side-effects) and they can be optimized and performed efficiently.

List of allowed functions and operators

You can find the built-in list of guards in the `Kernel` module. Here is an overview:

- comparison operators (`==`, `!=`, `===`, `!==`, `<`, `<=`, `>`, `>=`), `max`, `min`
- strictly boolean operators (`and`, `or`, `not`). Note `&&`, `||`, and `!` sibling operators are **not allowed** as they're not *strictly* boolean - meaning they don't require arguments to be booleans
- arithmetic unary operators (`+`, `-`)
- arithmetic binary operators (`+`, `-`, `*`, `/`)
- `in` and `not in` operators (as long as the right-hand side is a list or a range)
- "type-check" functions (`is_list/1`, `is_number/1`, and the like)
- functions that work on built-in data types (`abs/1`, `hd/1`, `map_size/1`, and others)
- the `map.field` syntax

The module `Bitwise` also includes a handful of Erlang bitwise operations as guards.

Macros constructed out of any combination of the above guards are also valid guards - for example, `Integer.is_even/1`. For more information, see

the "Custom patterns and guards expressions" section shown below.

Why guards

Let's see an example of a guard used in a function clause:

```
def empty_map?(map) when map_size(map) == 0, do: true
def empty_map?(map) when is_map(map), do: false
```

Guards start with the `when` operator, followed by a guard expression. The clause will be executed if and only if the guard expression returns `true`. Multiple boolean conditions can be combined with the `and` and `or` operators.

Writing the `empty_map?/1` function by only using pattern matching would not be possible (as pattern matching on `%{}` would match *any* map, not only the empty ones).

Non-passing guards

A function clause will be executed if and only if its guard expression evaluates to `true`. If any other value is returned, the function clause will be skipped. In particular, guards have no concept of "truthy" or "falsy".

For example, imagine a function that checks that the head of a list is not `nil`:

```
def not_nil_head?([head | _]) when head, do: true
def not_nil_head?(_), do: false

not_nil_head?(["some_value", "another_value"])
#=> false
```

Even though the head of the list is not `nil`, the first clause for `not_nil_head?/1` fails because the expression does not evaluate to `true`, but to `"some_value"`, therefore triggering the second clause which returns `false`. To make the guard behave correctly, you must ensure that the guard evaluates to `true`, like so:

```
def not_nil_head?([head | _]) when head != nil, do: true
def not_nil_head?(_), do: false

not_nil_head?(["some_value", "another_value"])
#=> true
```

Errors in guards

In guards, when functions would normally raise exceptions, they cause the guard to fail instead.

For example, the `tuple_size/1` function only works with tuples. If we use it with anything else, an argument error is raised:

```
iex> tuple_size("hello")
** (ArgumentError) argument error
```

However, when used in guards, the corresponding clause will fail to match instead of raising an error:

```
iex> case "hello" do
...>   something when tuple_size(something) == 2 ->
...>     :worked
...>   _anything_else ->
...>     :failed
```

```
...> end
:failed
```

In many cases, we can take advantage of this. In the code above, we used `tuple_size/1` to both check that the given value is a tuple *and* check its size (instead of using `is_tuple(something) and tuple_size(something) == 2`).

However, if your guard has multiple conditions, such as checking for tuples or maps, it is best to call type-check functions like `is_tuple/1` before `tuple_size/1`, otherwise the whole guard will fail if a tuple is not given. Alternatively, your function clause can use multiple guards as shown in the following section.

Multiple guards in the same clause

There exists an additional way to simplify a chain of `or` expressions in guards: Elixir supports writing "multiple guards" in the same clause. The following code:

```
def categorize_number(term) when is_integer(term) or
is_float(term) or is_nil(term),
  do: :maybe_number
def categorize_number(_other),
  do: :something_else
```

can be alternatively written as:

```
def categorize_number(term)
    when is_integer(term)
    when is_float(term)
    when is_nil(term) do
  :maybe_number
```

```
  end

  def categorize_number(_other) do
    :something_else
  end
```

If each guard expression always returns a boolean, the two forms are equivalent. However, recall that if any function call in a guard raises an exception, the entire guard fails. To illustrate this, the following function will not detect empty tuples:

```
  defmodule Check do
    # If given a tuple, map_size/1 will raise, and
  tuple_size/1 will not be evaluated
    def empty?(val) when map_size(val) == 0 or tuple_size(val)
  == 0, do: true
    def empty?(_val), do: false
  end

  Check.empty?(%{})
  #=> true

  Check.empty?({})
  #=> false # true was expected!
```

This could be corrected by ensuring that no exception is raised, either via type checks like `is_map(val) and map_size(val) == 0`, or by using multiple guards, so that if an exception causes one guard to fail, the next one is evaluated.

```
  defmodule Check do
    # If given a tuple, map_size/1 will raise, and the second
  guard will be evaluated
```

```
  def empty?(val)
      when map_size(val) == 0
      when tuple_size(val) == 0,
      do: true

  def empty?(_val), do: false
end

Check.empty?(%{})
#=> true

Check.empty?({})
#=> true
```

Where patterns and guards can be used

In the examples above, we have used the match operator (`=`) and function clauses to showcase patterns and guards respectively. Here is the list of the built-in constructs in Elixir that support patterns and guards.

- `match?/2`:

  ```
  match?({:ok, value} when value > 0, {:ok, 13})
  ```

- function clauses:

  ```
  def type(term) when is_integer(term), do: :integer
  def type(term) when is_float(term), do: :float
  ```

- `case` expressions:

```
case x do
  1 -> :one
  2 -> :two
  n when is_integer(n) and n > 2 -> :larger_than_two
end
```

- anonymous functions (`fn/1`):

```
larger_than_two? = fn
  n when is_integer(n) and n > 2 -> true
  n when is_integer(n) -> false
end
```

- `for` and `with` support patterns and guards on the left side of `<-` :

```
for x when x >= 0 <- [1, -2, 3, -4], do: x
```

  `with` also supports the `else` keyword, which supports patterns matching and guards.

- `try` supports patterns and guards on `catch` and `else`
- `receive` supports patterns and guards to match on the received messages.
- custom guards can also be defined with `defguard/1` and `defguardp/1` . A custom guard can only be defined based on existing guards.

Note that the match operator (`=`) does *not* support guards:

```
{:ok, binary} = File.read("some/file")
```

Custom patterns and guards expressions

Only the constructs listed in this page are allowed in patterns and guards. However, we can take advantage of macros to write custom patterns guards that can simplify our programs or make them more domain-specific. At the end of the day, what matters is that the *output* of the macros boils down to a combination of the constructs above.

For example, the `Record` module in Elixir provides a series of macros to be used in patterns and guards that allows tuples to have named fields during compilation.

For defining your own guards, Elixir even provides conveniences in `defguard` and `defguardp`. Let's look at a quick case study: we want to check whether an argument is an even or an odd integer. With pattern matching this is impossible because there is an infinite number of integers, and therefore we can't pattern match on every single one of them. Therefore we must use guards. We will just focus on checking for even numbers since checking for the odd ones is almost identical.

Such a guard would look like this:

```
def my_function(number) when is_integer(number) and
rem(number, 2) == 0 do
  # do stuff
end
```

It would be repetitive to write every time we need this check. Instead, you can use `defguard/1` and `defguardp/1` to create guard macros. Here's an example how:

```
defmodule MyInteger do
  defguard is_even(term) when is_integer(term) and rem(term,
2) == 0
end
```

and then:

```
import MyInteger, only: [is_even: 1]

def my_function(number) when is_even(number) do
  # do stuff
end
```

While it's possible to create custom guards with macros, it's recommended to define them using `defguard/1` and `defguardp/1` which perform additional compile-time checks.

Syntax reference

Elixir syntax was designed to have a straightforward conversion to an abstract syntax tree (AST). This means the Elixir syntax is mostly uniform with a handful of "syntax sugar" constructs to reduce the noise in common Elixir idioms.

This document covers all of Elixir syntax constructs as a reference and then discuss their exact AST representation.

Reserved words

These are the reserved words in the Elixir language. They are detailed throughout this guide but summed up here for convenience:

- `true`, `false`, `nil` - used as atoms
- `when`, `and`, `or`, `not`, `in` - used as operators
- `fn` - used for anonymous function definitions
- `do`, `end`, `catch`, `rescue`, `after`, `else` - used in do-end blocks

Data types

Numbers

Integers (`1234`) and floats (`123.4`) in Elixir are represented as a sequence of digits that may be separated by underscore for readability purposes, such as `1_000_000`. Integers never contain a dot (`.`) in their representation. Floats contain a dot and at least one other digit after the dot. Floats also support the scientific notation, such as `123.4e10` or `123.4E10`.

Atoms

Unquoted atoms start with a colon (`:`) which must be immediately followed by a Unicode letter or an underscore. The atom may continue using a sequence of Unicode letters, numbers, underscores, and `@`. Atoms may end in `!` or `?`. Valid unquoted atoms are: `:ok`, `:ISO8601`, and `:integer?`.

If the colon is immediately followed by a pair of double- or single-quotes surrounding the atom name, the atom is considered quoted. In contrast with an unquoted atom, this one can be made of any Unicode character (not only letters), such as `:'💧 Elixir'`, `:"++olá++"`, and `:"123"`.

Quoted and unquoted atoms with the same name are considered equivalent, so `:atom`, `:"atom"`, and `:'atom'` represent the same atom. The only catch is that the compiler will warn when quotes are used in atoms that do not need to be quoted.

All operators in Elixir are also valid atoms. Valid examples are `:foo`, `:FOO`, `:foo_42`, `:foo@bar`, and `:++`. Invalid examples are `:@foo` (`@` is not allowed at start), `:123` (numbers are not allowed at start), and `:(*)` (not a valid operator).

`true`, `false`, and `nil` are reserved words that are represented by the atoms `:true`, `:false` and `:nil` respectively.

To learn more about all Unicode characters allowed in atom, see the Unicode syntax document.

Strings

Single-line strings in Elixir are written between double-quotes, such as `"foo"`. Any double-quote inside the string must be escaped with `\`. Strings support Unicode characters and are stored as UTF-8 encoded binaries.

Multi-line strings in Elixir are called heredocs. They are written with three double-quotes, and can have unescaped quotes within them. The resulting

string will end with a newline. The indentation of the last `"""` is used to strip indentation from the inner string. For example:

```
iex> test = """
...>     this
...>     is
...>     a
...>     test
...> """
"    this\n    is\n    a\n    test\n"
iex> test = """
...>     This
...>     Is
...>     A
...>     Test
...>     """
"This\nIs\nA\nTest\n"
```

Strings are always represented as themselves in the AST.

Charlists

Charlists are lists of non-negative integers where each integer represents a Unicode code point.

```
iex(6)> 'abc' === [97, 98, 99]
true
```

Charlists are written in single-quotes, such as `'foo'`. Any single-quote inside the string must be escaped with `\`. Multi-line charlists are written with three single-quotes (`'''`), the same way multi-line strings are. However, this syntax is deprecated in favor of the charlist sigil `~c`.

Charlists are always represented as themselves in the AST.

For more in-depth information, please read the "Charlists" section in the `List` module.

Lists, tuples and binaries

Data structures such as lists, tuples, and binaries are marked respectively by the delimiters `[...]`, `{...}`, and `<<...>>`. Each element is separated by comma. A trailing comma is also allowed, such as in `[1, 2, 3,]`.

Maps and keyword lists

Maps use the `%{...}` notation and each key-value is given by pairs marked with `=>`, such as `%{"hello" => 1, 2 => "world"}`.

Both keyword lists (list of two-element tuples where the first element is an atom) and maps with atom keys support a keyword notation where the colon character `:` is moved to the end of the atom. `%{hello: "world"}` is equivalent to `%{:hello => "world"}` and `[foo: :bar]` is equivalent to `[{:foo, :bar}]`. We discuss keywords in later sections.

Structs

Structs built on the map syntax by passing the struct name between `%` and `{`. For example, `%User{...}`.

Expressions

Variables

Variables in Elixir must start with an underscore or a Unicode letter that is not in uppercase or titlecase. The variable may continue using a sequence of Unicode letters, numbers, and underscores. Variables may end in `?` or `!`.

To learn more about all Unicode characters allowed in variables, see the Unicode syntax document.

Elixir's naming conventions recommend variables to be in `snake_case` format.

Non-qualified calls (local calls)

Non-qualified calls, such as `add(1, 2)`, must start with characters and then follow the same rules as variables, which are optionally followed by parentheses, and then arguments.

Parentheses are required for zero-arity calls (i.e. calls without arguments), to avoid ambiguity with variables. If parentheses are used, they must immediately follow the function name *without spaces*. For example, `add (1, 2)` is a syntax error, since `(1, 2)` is treated as an invalid block which is attempted to be given as a single argument to `add`.

Elixir's naming conventions recommend calls to be in `snake_case` format.

Operators

As many programming languages, Elixir also support operators as non-qualified calls with their precedence and associativity rules. Constructs such as `=`, `when`, `&` and `@` are simply treated as operators. See the Operators page for a full reference.

Qualified calls (remote calls)

Qualified calls, such as `Math.add(1, 2)`, must start with characters and then follow the same rules as variables, which are optionally followed by parentheses, and then arguments. Qualified calls also support operators, such as `Kernel.+(1, 2)`. Elixir also allows the function name to be written between double- or single-quotes, allowing any character in between the quotes, such as `Math."++add++"(1, 2)`.

Similar to non-qualified calls, parentheses have different meaning for zero-arity calls (i.e. calls without arguments). If parentheses are used, such as `mod.fun()`, it means a function call. If parenthesis are skipped, such as `map.field`, it means accessing a field of a map.

Elixir's naming conventions recommend calls to be in `snake_case` format.

Aliases

Aliases are constructs that expand to atoms at compile-time. The alias `String` expands to the atom `:"Elixir.String"`. Aliases must start with an ASCII uppercase character which may be followed by any ASCII letter, number, or underscore. Non-ASCII characters are not supported in aliases.

Multiple aliases can be joined with `.`, such as `MyApp.String`, and it expands to the atom `:"Elixir.MyApp.String"`. The dot is effectively part of the name but it can also be used for composition. If you define `alias MyApp.Example, as: Example` in your code, then `Example` will always expand to `:"Elixir.MyApp.Example"` and `Example.String` will expand to `:"Elixir.MyApp.Example.String"`.

Elixir's naming conventions recommend aliases to be in `CamelCase` format.

Module attributes

Module attributes are module-specific storage and are written as the composition of the unary operator `@` with variables and local calls. For example, to write to a module attribute named `foo`, use `@foo "value"`, and use `@foo` to read from it. Given module attributes are written using existing constructs, they follow the same rules above defined for operators, variables, and local calls.

Blocks

Blocks are multiple Elixir expressions separated by newlines or semi-colons. A new block may be created at any moment by using parentheses.

Left to right arrow

The left to right arrow (`->`) is used to establish a relationship between left and right, commonly referred as clauses. The left side may have zero, one, or more arguments; the right side is zero, one, or more expressions separated by new line. The `->` may appear one or more times between one of the following terminators: `do-end`, `fn-end` or `(-)`. When `->` is used, only other clauses are allowed between those terminators. Mixing clauses and regular expressions is invalid syntax.

It is seen on `case` and `cond` constructs between `do` and `end`:

```
case 1 do
  2 -> 3
  4 -> 5
end

cond do
  true -> false
end
```

Seen in typespecs between `(` and `)`:

```
(integer(), boolean() -> integer())
```

It is also used between `fn` and `end` for building anonymous functions:

```
fn
  x, y -> x + y
```

```
end
```

Sigils

Sigils start with `~` and are followed by one lowercase letter or by one or more uppercase letters, immediately followed by one of the following pairs:

- `(` and `)`
- `{` and `}`
- `[` and `]`
- `<` and `>`
- `"` and `"`
- `'` and `'`
- `|` and `|`
- `/` and `/`

After closing the pair, zero or more ASCII letters and digits can be given as a modifier. Sigils are expressed as non-qualified calls prefixed with `sigil_` where the first argument is the sigil contents as a string and the second argument is a list of integers as modifiers:

If the sigil letter is in uppercase, no interpolation is allowed in the sigil, otherwise its contents may be dynamic. Compare the results of the sigils below for more information:

```
~s/f#{"o"}o/
~S/f#{"o"}o/
```

Sigils are useful to encode text with their own escaping rules, such as regular expressions, datetimes, and others.

The Elixir AST

Elixir syntax was designed to have a straightforward conversion to an abstract syntax tree (AST). Elixir's AST is a regular Elixir data structure composed of the following elements:

- atoms - such as `:foo`
- integers - such as `42`
- floats - such as `13.1`
- strings - such as `"hello"`
- lists - such as `[1, 2, 3]`
- tuples with two elements - such as `{"hello", :world}`
- tuples with three elements, representing calls or variables, as explained next

The building block of Elixir's AST is a call, such as:

```
sum(1, 2, 3)
```

which is represented as a tuple with three elements:

```
{:sum, meta, [1, 2, 3]}
```

the first element is an atom (or another tuple), the second element is a list of two-element tuples with metadata (such as line numbers) and the third is a list of arguments.

We can retrieve the AST for any Elixir expression by calling `quote`:

```
quote do
  sum()
end
#=> {:sum, [], []}
```

Variables are also represented using a tuple with three elements and a combination of lists and atoms, for example:

```
quote do
  sum
end
#=> {:sum, [], Elixir}
```

You can see that variables are also represented with a tuple, except the third element is an atom expressing the variable context.

Over the course of this section, we will explore many Elixir syntax constructs alongside their AST representations.

Operators

Operators are treated as non-qualified calls:

```
quote do
  1 + 2
end
#=> {:+, [], [1, 2]}
```

Note that `.` is also an operator. Remote calls use the dot in the AST with two arguments, where the second argument is always an atom:

```
quote do
  foo.bar(1, 2, 3)
end
#=> {{:., [], [{:foo, [], Elixir}, :bar]}, [], [1, 2, 3]}
```

Calling anonymous functions uses the dot in the AST with a single argument, mirroring the fact the function name is "missing" from right side

of the dot:

```
quote do
  foo.(1, 2, 3)
end
#=> {{:., [], [{:foo, [], Elixir}]}, [], [1, 2, 3]}
```

Aliases

Aliases are represented by an `__aliases__` call with each segment separated by a dot as an argument:

```
quote do
  Foo.Bar.Baz
end
#=> {:__aliases__, [], [:Foo, :Bar, :Baz]}

quote do
  __MODULE__.Bar.Baz
end
#=> {:__aliases__, [], [{:__MODULE__, [], Elixir}, :Bar,
:Baz]}
```

All arguments, except the first, are guaranteed to be atoms.

Data structures

Remember that lists are literals, so they are represented as themselves in the AST:

```
quote do
  [1, 2, 3]
```

```
end
#=> [1, 2, 3]
```

Tuples have their own representation, except for two-element tuples, which are represented as themselves:

```
quote do
  {1, 2}
end
#=> {1, 2}

quote do
  {1, 2, 3}
end
#=> {:{}, [], [1, 2, 3]}
```

Binaries have a representation similar to tuples, except they are tagged with `:<<>>` instead of `:{}`:

```
quote do
  <<1, 2, 3>>
end
#=> {:<<>>, [], [1, 2, 3]}
```

The same applies to maps, where pairs are treated as a list of tuples with two elements:

```
quote do
  %{1 => 2, 3 => 4}
end
#=> {:%{}, [], [{1, 2}, {3, 4}]}
```

Blocks

Blocks are represented as a `__block__` call with each line as a separate argument:

```
quote do
  1
  2
  3
end
#=> {:__block__, [], [1, 2, 3]}

quote do 1; 2; 3; end
#=> {:__block__, [], [1, 2, 3]}
```

Left to right arrow

The left to right arrow (`->`) is represented similar to operators except that they are always part of a list, its left side represents a list of arguments and the right side is an expression.

For example, in `case` and `cond`:

```
quote do
  case 1 do
    2 -> 3
    4 -> 5
  end
end
#=> {:case, [], [1, [do: [{:->, [], [[2], 3]}, {:->, [],
[[4], 5]}]]]}

quote do
  cond do
```

```
    true -> false
  end
end
#=> {:cond, [], [[do: [{:->, [], [[true], false]}]]]}
```

Between `(` and `)`:

```
quote do
  (1, 2 -> 3
   4, 5 -> 6)
end
#=> [{:->, [], [[1, 2], 3]}, {:->, [], [[4, 5], 6]}]
```

Between `fn` and `end`:

```
quote do
  fn
    1, 2 -> 3
    4, 5 -> 6
  end
end
#=> {:fn, [], [{:->, [], [[1, 2], 3]}, {:->, [], [[4, 5],
6]}]}
```

Qualified tuples

Qualified tuples (`foo.{bar, baz}`) are represented by a `{:., [], [expr, :{}]}` call, where the `expr` represents the left hand side of the dot, and the arguments represent the elements inside the curly braces. This is used in Elixir to provide multi aliases:

```
quote do
  Foo.{Bar, Baz}
end
#=> {{:., [], [{:__aliases__, [], [:Foo]}, :{}]}, [],
[{:__aliases__, [], [:Bar]}, {:__aliases__, [], [:Baz]}]}
```

`do-end` blocks

Elixir's `do-end` blocks are equivalent to keywords as the last argument of a function call, where the block contents are wrapped in parentheses. For example:

```
if true do
  this
else
  that
end
```

is the same as:

```
if(true, do: (this), else: (that))
```

While the construct above does not require custom nodes in Elixir's AST, they are restricted only to certain keywords, listed next:

- `after`
- `catch`
- `else`
- `rescue`

You will find them in constructs such as `receive`, `try`, and others. You can also find more examples in the Optional Syntax chapter.

Software Bill of Materials

A Software Bill of Materials (SBoM) is a structured inventory of the components that make up a software system. This guide explains what SBoMs are, why they matter, and how to generate them for Elixir projects.

What is an SBoM?

An SBoM is a formal, machine-readable record of all components in a piece of software. Think of it as a detailed ingredient list for your application. A typical SBoM includes:

- **Dependencies**: all libraries and packages your project uses
- **Versions**: the exact version of each component
- **Source locations**: where each component was obtained (Hex, GitHub, etc.)
- **Checksums**: cryptographic hashes to verify integrity
- **Licensing information**: the license under which each component is distributed

Two widely adopted standards exist for SBoM formats:

- CycloneDX: a lightweight standard designed for security contexts
- SPDX: a more comprehensive standard originally focused on licensing

Both formats are machine-readable (JSON, XML) and designed to be consumed by automated tooling.

> **SBoM is an inventory, not a certification**
>
> An SBoM does not claim that software is secure, compliant, or free of vulnerabilities. It simply provides a detailed inventory that enables further analysis. Security and compliance assessments are performed by separate tools that consume the SBoM.

Why generate SBoMs?

There are three main reasons to generate SBoMs for your projects.

Vulnerability analysis

When a security vulnerability (CVE) is discovered in a library, you need to quickly determine if your projects are affected. SBoMs make this possible by providing a complete inventory of your dependencies.

Tools like OWASP Dependency-Track continuously monitor your SBoMs against vulnerability databases. When a new CVE is published, you get notified if any of your projects use the affected component.

Without an SBoM, answering "are we affected by CVE-2024-XXXXX?" means manually checking each project, which is time-consuming and error-prone.

Regulatory requirements

SBoMs are required by some regulations and procurement policies:

- **US Executive Order 14028** (2021) requires SBoMs for certain software supplied to the U.S. federal government, particularly for designated critical software
- **EU Cyber Resilience Act** introduces requirements for component inventories (commonly fulfilled using SBoMs) for products with digital elements placed on the EU market
- **Safety-critical industries** (medical devices, automotive, aerospace) often require detailed component inventories as part of certification

Customers and partners may also request SBoMs as part of their own compliance efforts.

License compliance

Every dependency in your project comes with a license. An SBoM provides a starting point for license review by listing the declared license for each package. This helps you:

- Get an overview of licenses in your dependency tree
- Flag packages that may need closer review
- Support due diligence for acquisitions, audits, or legal review

Note that package-level license information (as provided by mix_sbom) reflects what the package declares, not necessarily all licenses present in its source files. For thorough license compliance, file-level scanning tools like ORT provide deeper analysis.

Generating SBoMs with mix_sbom

mix_sbom is an EEF project that generates CycloneDX SBoMs for Elixir projects.

Installation

There are several ways to install mix_sbom:

- **As a project dependency**: add `sbom` to your `mix.exs`
- **As a global escript**: run `mix escript.install hex sbom` (requires Elixir 1.19.4+)
- **As a standalone binary**: download from the releases page

The standalone binary is useful for CI environments or when you don't want to modify your project's dependencies. It requires no local Elixir or Erlang installation.

Basic usage

To generate an SBoM for your project using the standalone binary:

```
$ mix_sbom cyclonedx /path/to/your/project
```

This creates a `bom.cdx.json` file in CycloneDX format containing your project's complete dependency tree.

Common options

The most useful options are:

- `-o, --output PATH`: specify the output file (default: `bom.cdx.json`)
- `-t, --format FORMAT`: output format (`json`, `xml`, or `protobuf`)
- `-s, --schema VERSION`: CycloneDX schema version

For example, to generate an XML format SBoM:

```
$ mix_sbom cyclonedx --format xml --output sbom.xml
/path/to/project
```

CI integration

For automated SBoM generation, mix_sbom provides a GitHub Action:

```
name: Generate SBoM
on:
  release:
    types: [published]

jobs:
  sbom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- uses: erlef/mix_sbom@v0
  with:
    path: "."
    format: "json"
- uses: actions/upload-artifact@v4
  with:
    name: sbom
    path: bom.cdx.json
```

This workflow generates an SBoM whenever you publish a release and uploads it as a build artifact. See the action's documentation for additional options.

Deeper analysis with ORT

mix_sbom provides package-level license information based on what each dependency declares. However, some compliance workflows require file-level scanning. A package might declare an MIT license but contain individual files under different licenses, or include vendored code with its own licensing terms.

The OSS Review Toolkit (ORT) addresses this by scanning actual source files for license headers and copyright notices. ORT supports Mix projects and provides:

- **File-level license detection**: scans source code for license texts and SPDX identifiers
- **Copyright holder identification**: extracts copyright notices from files
- **Policy enforcement**: define rules for allowed and denied licenses
- **Multi-ecosystem support**: analyze projects that span multiple package managers

For organizations with strict compliance requirements, ORT complements mix_sbom by providing the deeper analysis needed for thorough license audits.

See the ORT Mix plugin documentation for details.

Next steps

- CycloneDX Specification: learn more about the SBoM format
- OWASP Dependency-Track: continuous SBoM analysis platform
- mix sbom documentation: full documentation and advanced options

Typespecs reference

> **Typespecs are not set-theoretic types**
>
> Elixir is in the process of implementing its own type system based on set-theoretic types. Typespecs, which are described in the following document, are a distinct notation for declaring types and specifications based on Erlang. Typespecs may be phased out as the set-theoretic type effort moves forward.

Elixir is a dynamically typed language, and as such, type specifications are never used by the compiler to optimize or modify code. Still, using type specifications is useful because:

- they provide documentation (for example, tools such as `ExDoc` show type specifications in the documentation)
- they're used by tools such as Dialyzer, that can analyze code with typespecs to find type inconsistencies and possible bugs

Type specifications (most often referred to as *typespecs*) are defined in different contexts using the following attributes:

- `@type`
- `@opaque`
- `@typep`
- `@spec`
- `@callback`
- `@macrocallback`

In addition, you can use `@typedoc` to document a custom `@type` definition.

See the "User-defined types" and "Defining a specification" sub-sections below for more information on defining types and typespecs.

A simple example

```elixir
defmodule StringHelpers do
  @typedoc "A word from the dictionary"
  @type word() :: String.t()

  @spec long_word?(word()) :: boolean()
  def long_word?(word) when is_binary(word) do
    String.length(word) > 8
  end
end
```

In the example above:

- We declare a new type (`word()`) that is equivalent to the string type (`String.t()`).
- We describe the type using a `@typedoc`, which will be included in the generated documentation.
- We specify that the `long_word?/1` function takes an argument of type `word()` and returns a boolean (`boolean()`), that is, either `true` or `false`.

Types and their syntax

The syntax Elixir provides for type specifications is similar to the one in Erlang. Most of the built-in types provided in Erlang (for example, `pid()`) are expressed in the same way: `pid()` (or simply `pid`). Parameterized types (such as `list(integer)`) are supported as well and so are remote types (such as `Enum.t()`). Integers and atom literals are allowed as types (for example, `1`, `:atom`, or `false`). All other types are built out of unions of predefined types. Some types can also be declared using their syntactical notation, such as `[type]` for lists, `{type1, type2, ...}` for tuples and `<<_ * _>>` for binaries.

The notation to represent the union of types is the pipe `|`. For example, the typespec `type :: atom() | pid() | tuple()` creates a type `type` that can be either an `atom`, a `pid`, or a `tuple`. This is usually called a sum type in other languages

> **Differences with set-theoretic types**
>
> While they do share some similarities, the types below do not map one-to-one to the new types from the set-theoretic type system.
>
> For example, there is no plan to support subsets of the `integer()` type such as positive, ranges or literals.
>
> Furthermore, set-theoretic types support the full range of set operations, including intersections and negations.

Basic types

```
type ::
      any()                    # the top type, the set of all
terms
      | none()                 # the bottom type, contains no
terms
      | atom()
      | map()                  # any map
      | pid()                  # process identifier
      | port()                 # port identifier
      | reference()
      | tuple()                # tuple of any size

                               ## Numbers
      | float()
      | integer()
      | neg_integer()          # ..., -3, -2, -1
      | non_neg_integer()      # 0, 1, 2, 3, ...
      | pos_integer()          # 1, 2, 3, ...
```

```
## Lists
      | list(type)
# proper list ([]-terminated)
      | nonempty_list(type)
# non-empty proper list
      | maybe_improper_list(content_type, termination_type)
# proper or improper list
      | nonempty_improper_list(content_type, termination_type)
# improper list
      | nonempty_maybe_improper_list(content_type,
termination_type)  # non-empty proper or improper list

      | Literals               # Described in section
"Literals"
      | BuiltIn                # Described in section "Built-
in types"
      | Remotes                # Described in section "Remote
types"
      | UserDefined            # Described in section "User-
defined types"
```

Literals

The following literals are also supported in typespecs:

```
type ::                                ## Atoms
      :atom                            # atoms: :foo, :bar, ...
      | true | false | nil             # special atom literals

                                       ## Bitstrings
      | <<>>                           # empty bitstring
      | <<_::size>>                    # size is 0 or a positive
integer
      | <<_::_*unit>>                  # unit is an integer from
1 to 256
```

```
      | <<_::size, _::_*unit>>

                                    ## (Anonymous) Functions
      | (-> type)                    # zero-arity, returns
type
      | (type1, type2 -> type)      # two-arity, returns type
      | (... -> type)               # any arity, returns type

                                    ## Integers
      | 1                           # integer
      | 1..10                       # integer from 1 to 10

                                    ## Lists
      | [type]                      # list with any number of
type elements
      | []                          # empty list
      | [...]                       # shorthand for
nonempty_list(any())
      | [type, ...]                 # shorthand for
nonempty_list(type)
      | [key: value_type]           # keyword list with
optional key :key of value_type

                                              ## Maps
      | %{}                                   # empty map
      | %{key: value_type}                    # map with
required key :key of value_type
      | %{key_type => value_type}             # map with
required pairs of key_type and value_type
      | %{required(key_type) => value_type}   # map with
required pairs of key_type and value_type
      | %{optional(key_type) => value_type}   # map with
optional pairs of key_type and value_type
      | %SomeStruct{}                         # struct with all
fields of any type
      | %SomeStruct{key: value_type}          # struct with
required key :key of value_type
```

```
      | {}
      | {:ok, type}
an atom and any type
```

```
## Tuples
# empty tuple
# two-element tuple with
```

Built-in types

The following types are also provided by Elixir as shortcuts on top of the basic and literal types described above.

Built-in type	Defined as
`term()`	`any()`
`arity()`	`0..255`
`as_boolean(t)`	`t`
`binary()`	`<<_::_*8>>`
`nonempty_binary()`	`<<_::8, _::_*8>>`
`bitstring()`	`<<_::_*1>>`
`nonempty_bitstring()`	`<<_::1, _::_*1>>`
`boolean()`	`true | false`
`byte()`	`0..255`
`char()`	`0..0x10FFFF`
`charlist()`	`[char()]`
`nonempty_charlist()`	`[char(), ...]`
`fun()`	`(... -> any)`
`function()`	`fun()`
`identifier()`	`pid() | port() | reference()`
`iodata()`	`iolist() | binary()`
`iolist()`	`maybe_improper_list(byte() | binary() | iolist(), binary() | [])`
`keyword()`	`[{atom(), any()}]`

Built-in type	Defined as
`keyword(t)`	`[{atom(), t}]`
`list()`	`[any()]`
`nonempty_list()`	`nonempty_list(any())`
`maybe_improper_list()`	`maybe_improper_list(any(), any())`
`nonempty_maybe_improper_list()`	`nonempty_maybe_improper_list(any(), any())`
`mfa()`	`{module(), atom(), arity()}`
`module()`	`atom()`
`no_return()`	`none()`
`node()`	`atom()`
`number()`	`integer() | float()`
`struct()`	`%{:__struct__ => atom(), optional(atom()) => any()}`
`timeout()`	`:infinity | non_neg_integer()`

`as_boolean(t)` exists to signal users that the given value will be treated as a boolean, where `nil` and `false` will be evaluated as `false` and everything else is `true`. For example, `Enum.filter/2` has the following specification: `filter(t, (element -> as_boolean(term))) :: list`.

Remote types

Any module is also able to define its own types and the modules in Elixir are no exception. For example, the `Range` module defines a `t/0` type that represents a range: this type can be referred to as `Range.t/0`. In a similar fashion, a string is `String.t/0`, and so on.

Maps

The key types in maps are allowed to overlap, and if they do, the leftmost key takes precedence. A map value does not belong to this type if it contains a key

that is not in the allowed map keys.

If you want to denote that keys that were not previously defined in the map are allowed, it is common to end a map type with `optional(any) => any`.

Note that the syntactic representation of `map()` is `%{optional(any) => any}`, not `%{}`. The notation `%{}` specifies the singleton type for the empty map.

Keyword Lists

Beyond `keyword()` and `keyword(t)`, it can be helpful to compose a spec for an expected keyword list. For example:

```
@type option :: {:name, String.t} | {:max, pos_integer} |
{:min, pos_integer}
@type options :: [option()]
```

This makes it clear that only these options are allowed, none are required, and order does not matter.

It also allows composition with existing types. For example:

```
@type option :: {:my_option, String.t()} | GenServer.option()

@spec start_link([option()]) :: GenServer.on_start()
def start_link(opts) do
  {my_opts, gen_server_opts} = Keyword.split(opts,
[:my_option])
  GenServer.start_link(__MODULE__, my_opts, gen_server_opts)
end
```

The following spec syntaxes are equivalent:

```
@type options [{:name, String.t} | {:max, pos_integer} | {:min,
pos_integer}]
```

```
@type options [name: String.t, max: pos_integer, min:
pos_integer]
```

User-defined types

The `@type`, `@typep`, and `@opaque` module attributes can be used to define new types:

```
@type type_name :: type
@typep type_name :: type
@opaque type_name :: type
```

A type defined with `@typep` is private. An opaque type, defined with `@opaque` is a type where the internal structure of the type will not be visible, but the type is still public.

Types can be parameterized by defining variables as parameters; these variables can then be used to define the type.

```
@type dict(key, value) :: [{key, value}]
```

Defining a specification

A specification for a function can be defined as follows:

```
@spec function_name(type1, type2) :: return_type
```

Guards can be used to restrict type variables given as arguments to the function.

```
@spec function(arg) :: [arg] when arg: atom
```

If you want to specify more than one variable, you separate them by a comma.

```
@spec function(arg1, arg2) :: {arg1, arg2} when arg1: atom,
arg2: integer
```

Type variables with no restriction can also be defined using `var`.

```
@spec function(arg) :: [arg] when arg: var
```

This guard notation only works with `@spec`, `@callback`, and `@macrocallback`.

You can also name your arguments in a typespec using `arg_name :: arg_type` syntax. This is particularly useful in documentation as a way to differentiate multiple arguments of the same type (or multiple elements of the same type in a type definition):

```
@spec days_since_epoch(year :: integer, month :: integer, day
:: integer) :: integer
@type color :: {red :: integer, green :: integer, blue ::
integer}
```

Specifications can be overloaded, just like ordinary functions.

```
@spec function(integer) :: atom
@spec function(atom) :: integer
```

Behaviours

Behaviours in Elixir (and Erlang) are a way to separate and abstract the generic part of a component (which becomes the *behaviour module*) from the specific part (which becomes the *callback module*).

A behaviour module defines a set of functions and macros (referred to as *callbacks*) that callback modules implementing that behaviour must export. This "interface" identifies the specific part of the component. For example, the `GenServer` behaviour and functions abstract away all the message-passing (sending and receiving) and error reporting that a "server" process will likely want to implement from the specific parts such as the actions that this server process has to perform.

Say we want to implement a bunch of parsers, each parsing structured data: for example, a JSON parser and a MessagePack parser. Each of these two parsers will *behave* the same way: both will provide a `parse/1` function and an `extensions/0` function. The `parse/1` function will return an Elixir representation of the structured data, while the `extensions/0` function will return a list of file extensions that can be used for each type of data (e.g., `.json` for JSON files).

We can create a `Parser` behaviour:

```
defmodule Parser do
  @doc """
  Parses a string.
  """
  @callback parse(String.t) :: {:ok, term} | {:error, atom}

  @doc """
  Lists all supported file extensions.
  """
  @callback extensions() :: [String.t]
end
```

As seen in the example above, defining a callback is a matter of defining a specification for that callback, made of:

- the callback name (`parse` or `extensions` in the example)
- the arguments that the callback must accept (`String.t`)
- the *expected* type of the callback return value

Modules adopting the `Parser` behaviour will have to implement all the functions defined with the `@callback` attribute. As you can see, `@callback` expects a function name but also a function specification like the ones used with the `@spec` attribute we saw above.

Implementing behaviours

Implementing a behaviour is straightforward:

```
defmodule JSONParser do
  @behaviour Parser

  @impl Parser
  def parse(str), do: {:ok, "some json " <> str} # ... parse
JSON

  @impl Parser
  def extensions, do: [".json"]
end


defmodule CSVParser do
  @behaviour Parser

  @impl Parser
  def parse(str), do: {:ok, "some csv " <> str} # ... parse CSV

  @impl Parser
  def extensions, do: [".csv"]
end
```

If a module adopting a given behaviour doesn't implement one of the callbacks required by that behaviour, a compile-time warning will be generated.

Furthermore, with `@impl` you can also make sure that you are implementing the **correct** callbacks from the given behaviour in an explicit manner. For

example, the following parser implements both `parse` and `extensions`. However, thanks to a typo, `BADParser` is implementing `parse/0` instead of `parse/1`.

```
defmodule BADParser do
  @behaviour Parser

  @impl Parser
  def parse, do: {:ok, "something bad"}

  @impl Parser
  def extensions, do: ["bad"]
end
```

This code generates a warning letting you know that you are mistakenly implementing `parse/0` instead of `parse/1`. You can read more about `@impl` in the module documentation.

Using behaviours

Behaviours are useful because you can pass modules around as arguments and you can then *call back* to any of the functions specified in the behaviour. For example, we can have a function that receives a filename, several parsers, and parses the file based on its extension:

```
@spec parse_path(Path.t(), [module()]) :: {:ok, term} |
{:error, atom}
def parse_path(filename, parsers) do
  with {:ok, ext} <- parse_extension(filename),
       {:ok, parser} <- find_parser(ext, parsers),
       {:ok, contents} <- File.read(filename) do
    parser.parse(contents)
  end
end
```

```
defp parse_extension(filename) do
  if ext = Path.extname(filename) do
    {:ok, ext}
  else
    {:error, :no_extension}
  end
end

defp find_parser(ext, parsers) do
  if parser = Enum.find(parsers, fn parser -> ext in
parser.extensions() end) do
    {:ok, parser}
  else
    {:error, :no_matching_parser}
  end
end
```

You could also invoke any parser directly: `CSVParser.parse(...)`.

Note you don't need to define a behaviour in order to dynamically dispatch on a module, but those features often go hand in hand.

Optional callbacks

Optional callbacks are callbacks that callback modules may implement if they want to, but are not required to. Usually, behaviour modules know if they should call those callbacks based on configuration, or they check if the callbacks are defined with `function_exported?/3` or `macro_exported?/3`.

> ### Unloaded modules
>
> `function_exported?/3` (and `macro_exported?/3`) do *not* load the module in case it is not loaded and Elixir lazily loads modules by default (except on releases). So in practice you will want to invoke `Code.ensure_loaded?/1` before checking if the function/macro is exported. See the documentation for `function_exported?/3` for examples.

Optional callbacks can be defined through the `@optional_callbacks` module attribute, which has to be a keyword list with function or macro name as key and arity as value. For example:

```
defmodule MyBehaviour do
  @callback vital_fun() :: any
  @callback non_vital_fun() :: any
  @macrocallback non_vital_macro(arg :: any) :: Macro.t
  @optional_callbacks non_vital_fun: 0, non_vital_macro: 1
end
```

One example of optional callback in Elixir's standard library is `GenServer.format_status/1`.

Inspecting behaviours

The `@callback` and `@optional_callbacks` attributes are used to create a `behaviour_info/1` function available on the defining module. This function can be used to retrieve the callbacks and optional callbacks defined by that module.

For example, for the `MyBehaviour` module defined in "Optional callbacks" above:

```
MyBehaviour.behaviour_info(:callbacks)
#=> [vital_fun: 0, "MACRO-non_vital_macro": 2, non_vital_fun:
0]
MyBehaviour.behaviour_info(:optional_callbacks)
#=> ["MACRO-non_vital_macro": 2, non_vital_fun: 0]
```

When using `iex`, the `IEx.Helpers.b/1` helper is also available.

Pitfalls

There are some known pitfalls when using typespecs, they are documented next.

The `string()` type

Elixir discourages the use of the `string()` type. The `string()` type refers to Erlang strings, which are known as "charlists" in Elixir. They do not refer to Elixir strings, which are UTF-8 encoded binaries. To avoid confusion, if you attempt to use the type `string()`, Elixir will emit a warning. You should use `charlist()`, `nonempty_charlist()`, `binary()` or `String.t()` accordingly, or any of the several literal representations for these types.

Note that `String.t()` and `binary()` are equivalent to analysis tools. Although, for those reading the documentation, `String.t()` implies it is a UTF-8 encoded binary.

Functions which raise an error

Typespecs do not need to indicate that a function can raise an error; any function can fail any time if given invalid input. In the past, the Elixir standard library sometimes used `no_return()` to indicate this, but these usages have been removed.

The `no_return()` type also should not be used for functions which do return but whose purpose is a "side effect", such as `IO.puts/1`. In these cases, the expected return type is `:ok`.

Instead, `no_return()` should be used as the return type for functions which can never return a value. This includes functions which loop forever calling `receive`, or which exist specifically to raise an error, or which shut down the VM.

Unicode syntax

Elixir supports Unicode throughout the language. This document is a complete reference of how Elixir supports Unicode in its syntax.

Strings (`"olá"`) and charlists (`'olá'`) support Unicode since Elixir v1.0. Strings are UTF-8 encoded. Charlists are lists of Unicode code points. In such cases, the contents are kept as written by developers, without any transformation.

Elixir also supports Unicode in variables, atoms, and calls since Elixir v1.5. The focus of this document is to provide a high-level introduction to how Elixir allows Unicode in its syntax. We also provide technical documentation describing how Elixir complies with the Unicode specification.

To check the Unicode version of your current Elixir installation, run `String.Unicode.version()`.

Introduction

Elixir allows Unicode characters in its variables, atoms, and calls. However, the Unicode characters must still obey the rules of the language syntax. In particular, variables and calls cannot start with an uppercase letter. From now on, we will refer to those terms as identifiers.

The characters allowed in identifiers are the ones specified by Unicode. Generally speaking, it is restricted to characters typically used by the writing system of human languages still in activity. In particular, it excludes symbols such as emojis, alternate numeric representations, musical notes, and the like.

Elixir imposes many restrictions on identifiers for security purposes. For example, the word "josé" can be written in two ways in Unicode: as the combination of the characters `j o s é` and as a combination of the characters `j o s e ´`, where the accent is its own character. The former is called NFC form and the latter is the NFD form. Elixir normalizes all characters to be the in the NFC form.

Elixir also disallows mixed-scripts which are not explicitly separated by `_`. For example, it is not possible to name a variable `admin`, where `a` is in Cyrillic and the remaining characters are in Latin. Doing so will raise the following error:

```
** (SyntaxError) invalid mixed-script identifier found:
admin

Mixed-script identifiers are not supported for security
reasons. 'admin' is made of the following scripts:

  \u0430 a {Cyrillic}
  \u0064 d {Latin}
  \u006D m {Latin}
  \u0069 i {Latin}
  \u006E n {Latin}

Make sure all characters in the identifier resolve to a
single script or a highly
restrictive script. See https://hexdocs.pm/elixir/unicode-
syntax.html for more information.
```

Finally, Elixir will also warn of confusable identifiers in the same file. For example, Elixir will emit a warning if you use both variables `a` (Cyrillic) and `a` (Latin) in your code.

That's the overall introduction of how Unicode is used in Elixir identifiers. In a nutshell, its goal is to support different writing systems in use today while keeping the Elixir language itself clear and secure.

For the technical details, see the next sections that cover the technical Unicode requirements.

Unicode Standard Annex #31

Elixir conforms to the standards outlined in the Unicode Standard Annex #31: Unicode Identifiers and Syntax, version 17.0.

R1. Default Identifiers

The general Elixir identifier rule is specified as:

```
<Identifier> := <Start> <Continue>* <Ending>?
```

where `<Start>` uses the same categories as the spec but normalizes them to the NFC form (see R4):

> characters derived from the Unicode General Category of uppercase letters, lowercase letters, titlecase letters, modifier letters, other letters, letter numbers, plus `Other_ID_Start`, minus `Pattern_Syntax` and `Pattern_White_Space` code points
>
> In set notation: `[\p{L}\p{Nl}\p{Other_ID_Start}-\p{Pattern_Syntax}-\p{Pattern_White_Space}]`.

and `<Continue>` uses the same categories as the spec but normalizes them to the NFC form (see R4):

> ID_Start characters, plus characters having the Unicode General Category of nonspacing marks, spacing combining marks, decimal

number, connector punctuation, plus `Other_ID_Continue`, minus `Pattern_Syntax` and `Pattern_White_Space` code points.

In set notation: `[\p{ID_Start}\p{Mn}\p{Mc}\p{Nd}\p{Pc}\p{Other_ID_Continue}-\p{Pattern_Syntax}-\p{Pattern_White_Space}]`.

`<Ending>` is an addition specific to Elixir that includes only the code points `?` (003F) and `!` (0021).

The spec also provides a `<Medial>` set, but Elixir does not include any character on this set. Therefore, the identifier rule has been simplified to consider this.

Elixir does not allow the use of ZWJ or ZWNJ in identifiers and therefore does not implement R1a. Bidirectional control characters are also not supported. R1b is guaranteed for backwards compatibility purposes.

Atoms

Unicode atoms in Elixir follow the identifier rule above with the following modifications:

- `<Start>` additionally includes the code point `_` (005F)
- `<Continue>` additionally includes the code point `@` (0040)

Note atoms can also be quoted, which allows any characters, such as `:"hello elixir"`. All Elixir operators are also valid atoms, such as `:+`, `:@`, `:|>`, and others. The full description of valid atoms is available in the "Atoms" section in the syntax reference.

Variables, local calls, and remote calls

Variables in Elixir follow the identifier rule above with the following modifications:

- `<Start>` additionally includes the code point `_` (005F)
- `<Start>` additionally excludes Lu (letter uppercase) and Lt (letter titlecase) characters

In set notation:
`[\u{005F}\p{Ll}\p{Lm}\p{Lo}\p{Nl}\p{Other_ID_Start}-\p{Pattern_Syntax}-\p{Pattern_White_Space}].`

Aliases

Aliases in Elixir only allow ASCII characters, starting in uppercase, and no punctuation characters.

R3. Pattern_White_Space and Pattern_Syntax Characters

Elixir supports only code points `\t` (0009), `\n` (000A), `\r` (000D) and `\s` (0020) as whitespace and therefore does not follow requirement R3. R3 requires a wider variety of whitespace and syntax characters to be supported.

R4. Equivalent Normalized Identifiers

Identifiers in Elixir are case sensitive.

Elixir normalizes all atoms and variables to NFC form. Quoted-atoms and strings can, however, be in any form and are not verified by the parser.

In other words, the atom `:josé` can only be written with the code points `006A 006F 0073 00E9` or `006A 006F 0073 0065 0301`, but Elixir will rewrite it to the former (from Elixir 1.14). On the other hand, `:"josé"` may be written as `006A 006F 0073 00E9` or `006A 006F 0073 0065 0301` and its form will be retained, since it is written between quotes.

Choosing requirement R4 automatically excludes requirements R5, R6, and R7.

Unicode Technical Standard #39

Elixir conforms to the clauses outlined in the Unicode Technical Standard #39 on Security, version 17.0.

C1. General Security Profile for Identifiers

Elixir will not allow tokenization of identifiers with codepoints in `\p{Identifier_Status=Restricted}`, except for the outlined 'Additional normalizations' section below.

> An implementation following the General Security Profile does not permit any characters in \p{Identifier_Status=Restricted}, ...

For instance, the 'HANGUL FILLER' ( ) character, which is often invisible, is an uncommon codepoint and will trigger a warning.

C2. Confusable detection

Elixir will warn of identifiers that look the same, but aren't. Examples: in `а = a = 1`, the two 'a' characters are Cyrillic and Latin, and could be confused for each other; in `カ = カ = 1`, both are Japanese, but different codepoints, in different scripts of that writing system. Confusable identifiers can lead to hard-to-catch bugs (say, due to copy-pasted code) and can be unsafe, so we will warn of identifiers within a single file that could be confused with each other.

We use the means described in Section 4, 'Confusable Detection', with one noted modification:

> Alternatively, it shall declare that it uses a modification, and provide a precise list of character mappings that are added to or removed from the provided ones.

Elixir will not warn about confusability for identifiers made up exclusively of characters in a-z, A-Z, 0-9, and _. This is because ASCII identifiers have existed for so long that the programming community has had their own means of dealing with confusability between identifiers like `l,1` or `O,0` (for instance, fonts designed for programming usually make it easy to differentiate between those characters).

C3. Mixed Script Detection

Elixir will not allow tokenization of mixed-script identifiers unless it is via chunks separated by an underscore, like `http_сервер`. We use the means described in Section 5.1, Mixed-Script Detection, to determine if script mixing is occurring, with the 'Additional Normalizations' documented in.

Examples: Elixir allows an identifiers like `幻한`, even though it includes characters from multiple 'scripts', as Han characters may be mixed with Japanese and Korean, according to the rules from UTS 39 5.1. When mixing Latin and Japanese scripts, underscores are necessary, as in `:T_シャツ` (the Japanese word for 't-shirt' with an additional underscore separating the letter T).

Elixir does not allow code like `if admin, do: :ok, else: :err`, where the scriptset for the 'a' character is {Cyrillic} but all other characters have scriptsets of {Latin}. The scriptsets fail to resolve and a descriptive error is shown.

C4, C5 (inapplicable)

'C4 - Restriction Level detection' conformance is not claimed and does not apply to identifiers in code; rather, it applies to classifying the level of safety of a given arbitrary string into one of 5 restriction levels.

'C5 - Mixed number detection' conformance is inapplicable as Elixir does not support Unicode numbers.

Addition Normalizations

As of Elixir 1.14, some codepoints in `\p{Identifier_Status=Restricted}` are *normalized* to other, unrestricted codepoints.

This is currently only applied to translate MICRO SIGN (µ) to Greek lowercase mu (μ).

The normalization avoids confusability and the mixed-script detection is modified to the extent that the normalized codepoint is given the union of scriptsets from both characters.

- For instance, in the example of MICRO => MU, MICRO was a 'Common'-script character - the same script given to the '_' underscore codepoint - and thus the normalized character's scriptset will be {Greek, Common}. 'Common' intersects with all non-empty scriptsets, and thus the normalized character can be used in tokens written in any script without causing script mixing.

- The code points normalized in this fashion are those that are in use in the community, and judged not likely to cause issues with unsafe script mixing. For instance, the MICRO or MU codepoint may be used in an atom or variable dealing with microseconds.

Kernel

`Kernel` is Elixir's default environment.

It mainly consists of:

- basic language primitives, such as arithmetic operators, spawning of processes, data type handling, and others
- macros for control-flow and defining new functionality (modules, functions, and the like)
- guard checks for augmenting pattern matching

You can invoke `Kernel` functions and macros anywhere in Elixir code without the use of the `Kernel.` prefix since they have all been automatically imported. For example, in IEx, you can call:

```
iex> is_number(13)
true
```

If you don't want to import a function or macro from `Kernel`, use the `:except` option and then list the function/macro by arity:

```
import Kernel, except: [if: 2, is_number: 1]
```

See `import/2` for more information on importing.

Elixir also has special forms that are always imported and cannot be skipped. These are described in `Kernel.SpecialForms`.

The standard library

`Kernel` provides the basic capabilities the Elixir standard library is built on top of. It is recommended to explore the standard library for advanced functionality. Here are the main groups of modules in the standard library (this list is not a complete reference, see the documentation sidebar for all entries).

Built-in types

The following modules handle Elixir built-in data types:

- `Atom` - literal constants with a name (`true`, `false`, and `nil` are atoms)
- `Float` - numbers with floating point precision
- `Function` - a reference to code chunk, created with the `fn/1` special form
- `Integer` - whole numbers (not fractions)
- `List` - collections of a variable number of elements (linked lists)
- `Map` - collections of key-value pairs
- `Process` - light-weight threads of execution
- `Port` - mechanisms to interact with the external world
- `Tuple` - collections of a fixed number of elements

There are two data types without an accompanying module:

- Bitstring - a sequence of bits, created with `<<>>/1`. When the number of bits is divisible by 8, they are called binaries and can be manipulated with Erlang's `:binary` module
- Reference - a unique value in the runtime system, created with `make_ref/0`

Data types

Elixir also provides other data types that are built on top of the types listed above. Some of them are:

- `Date` - `year-month-day` structs in a given calendar
- `DateTime` - date and time with time zone in a given calendar
- `Exception` - data raised from errors and unexpected scenarios
- `MapSet` - unordered collections of unique elements
- `NaiveDateTime` - date and time without time zone in a given calendar
- `Keyword` - lists of two-element tuples, often representing optional values
- `Range` - inclusive ranges between two integers
- `Regex` - regular expressions
- `String` - UTF-8 encoded binaries representing characters
- `Time` - `hour:minute:second` structs in a given calendar
- `URI` - representation of URIs that identify resources
- `Version` - representation of versions and requirements

System modules

Modules that interface with the underlying system, such as:

- `IO` - handles input and output
- `File` - interacts with the underlying file system
- `Path` - manipulates file system paths
- `System` - reads and writes system information

Protocols

Protocols add polymorphic dispatch to Elixir. They are contracts implementable by data types. See `Protocol` for more information on protocols. Elixir provides the following protocols in the standard library:

- `Collectable` - collects data into a data type
- `Enumerable` - handles collections in Elixir. The `Enum` module provides eager functions for working with collections, the `Stream`

module provides lazy functions
- `Inspect` - converts data types into their programming language representation
- `List.Chars` - converts data types to their outside world representation as charlists (non-programming based)
- `String.Chars` - converts data types to their outside world representation as strings (non-programming based)

Process-based and application-centric functionality

The following modules build on top of processes to provide concurrency, fault-tolerance, and more.

- `Agent` - a process that encapsulates mutable state
- `Application` - functions for starting, stopping and configuring applications
- `GenServer` - a generic client-server API
- `Registry` - a key-value process-based storage
- `Supervisor` - a process that is responsible for starting, supervising and shutting down other processes
- `Task` - a process that performs computations
- `Task.Supervisor` - a supervisor for managing tasks exclusively

Supporting documents

Under the "Pages" section in sidebar you will find tutorials, guides, and reference documents that outline Elixir semantics and behaviors in more detail. Those are:

- Compatibility and deprecations - lists compatibility between every Elixir version and Erlang/OTP, release schema; lists all deprecated functions, when they were deprecated and alternatives
- Library guidelines - general guidelines, anti-patterns, and rules for those writing libraries
- Naming conventions - naming conventions for Elixir code

- Operators reference - lists all Elixir operators and their precedences
- Patterns and guards - an introduction to patterns, guards, and extensions
- Syntax reference - the language syntax reference
- Typespecs reference- types and function specifications, including list of types
- Unicode syntax - outlines Elixir support for Unicode

Guards

This module includes the built-in guards used by Elixir developers. They are a predefined set of functions and macros that augment pattern matching, typically invoked after the `when` operator. For example:

```
def drive(%User{age: age}) when age >= 16 do
  ...
end
```

The clause above will only be invoked if the user's age is more than or equal to 16. Guards also support joining multiple conditions with `and` and `or`. The whole guard is true if all guard expressions will evaluate to `true`. A more complete introduction to guards is available in the Patterns and guards page.

Truthy and falsy values

Besides the booleans `true` and `false`, Elixir has the concept of a "truthy" or "falsy" value.

- a value is truthy when it is neither `false` nor `nil`
- a value is falsy when it is either `false` or `nil`

Elixir has functions, like `and/2`, that *only* work with booleans, but also functions that work with these truthy/falsy values, like `&&/2` and `!/1`.

Structural comparison

The functions in this module perform structural comparison. This allows different data types to be compared using comparison operators:

```
1 < :an_atom
```

This is possible so Elixir developers can create collections, such as dictionaries and ordered sets, that store a mixture of data types in them. To understand why this matters, let's discuss the two types of comparisons we find in software: *structural* and *semantic*.

Structural means we are comparing the underlying data structures and we often want those operations to be as fast as possible, because it is used to power several algorithms and data structures in the language. A semantic comparison worries about what each data type represents. For example, semantically speaking, it doesn't make sense to compare `Time` with `Date`.

One example that shows the differences between structural and semantic comparisons are strings: "alien" sorts less than "office" (`"alien" < "office"`) but "álien" is greater than "office". This happens because `<` compares the underlying bytes that form the string. If you were doing alphabetical listing, you may want "álien" to also appear before "office".

This means **comparisons in Elixir are structural**, as it has the goal of comparing data types as efficiently as possible to create flexible and performant data structures. This distinction is specially important for functions that provide ordering, such as `>/2`, `</2`, `>=/2`, `<=/2`, `min/2`, and `max/2`. For example:

```
~D[2017-03-31] > ~D[2017-04-01]
```

will return `true` because structural comparison compares the `:day` field before `:month` or `:year`. Luckily, the Elixir compiler will detect whenever comparing structs or whenever comparing code that is either always true or false, and emit a warning accordingly.

In order to perform semantic comparisons, the relevant data-types provide a `compare/2` function, such as `Date.compare/2`:

```
iex> Date.compare(~D[2017-03-31], ~D[2017-04-01])
:lt
```

Alternatively, you can use the functions in the `Enum` module to sort or compute a maximum/minimum:

```
iex> Enum.sort([~D[2017-03-31], ~D[2017-04-01]], Date)
[~D[2017-03-31], ~D[2017-04-01]]
iex> Enum.max([~D[2017-03-31], ~D[2017-04-01]], Date)
~D[2017-04-01]
```

The second argument is precisely the module to be used for semantic comparison. Keeping this distinction is important, because if semantic comparison was used by default for implementing data structures and algorithms, they could become orders of magnitude slower!

Finally, note there is an overall structural sorting order, called "Term Ordering", defined below. This order is provided for reference purposes, it is not required for Elixir developers to know it by heart.

Term ordering

```
number < atom < reference < function < port < pid < tuple
< map < list < bitstring
```

When comparing two numbers of different types (a number being either an integer or a float), a conversion to the type with greater precision will always occur, unless the comparison operator used is either `===/2` or `!==/2`. A float will be considered more precise than an integer, unless the float is greater/less than +/-9007199254740992.0 respectively, at which point all the significant figures of the float are to the left of the decimal point. This behavior exists so that the comparison of large numbers remains transitive.

The collection types are compared using the following rules:

- Tuples are compared by size, then element by element.
- Maps are compared by size, then by key-value pairs.
- Lists are compared element by element.
- Bitstrings are compared byte by byte, incomplete bytes are compared bit by bit.
- Atoms are compared using their string value, codepoint by codepoint.

Examples

We can check the truthiness of a value by using the `!/1` function twice.

Truthy values:

```
iex> !!true
true
iex> !!5
true
iex> !![1,2]
true
```

```
iex> !!"foo"
true
```

Falsy values (of which there are exactly two):

```
iex> !!false
false
iex> !!nil
false
```

Inlining

Some of the functions described in this module are inlined by the Elixir compiler into their Erlang counterparts in the `:erlang` module. Those functions are called BIFs (built-in internal functions) in Erlang-land and they exhibit interesting properties, as some of them are allowed in guards and others are used for compiler optimizations.

Most of the inlined functions can be seen in effect when capturing the function:

```
iex> &Kernel.is_atom/1
&:erlang.is_atom/1
```

Those functions will be explicitly marked in their docs as "inlined by the compiler".

Summary

Guards

left * right

Arithmetic multiplication operator.

+value

Arithmetic positive unary operator.

left + right

Arithmetic addition operator.

-value

Arithmetic negative unary operator.

left - right

Arithmetic subtraction operator.

left / right

Arithmetic division operator.

left != right

Not equal to operator.

left !== right

Strictly not equal to operator.

left < right

Less-than operator.

left <= right

Less-than or equal to operator.

left == right

Equal to operator. Returns `true` if the two terms are equal.

left === right

Strictly equal to operator.

left > right

Greater-than operator.

left >= right

Greater-than or equal to operator.

abs(number)

Returns an integer or float which is the arithmetical absolute value of `number`.

left and right

Strictly boolean "and" operator.

binary_part(binary, start, size)

Extracts the part of the binary at `start` with `size`.

bit_size(bitstring)

Returns an integer which is the size in bits of `bitstring`.

byte_size(bitstring)

Returns the number of bytes needed to contain `bitstring`.

ceil(number)

Returns the smallest integer greater than or equal to `number`.

div(dividend, divisor)

Performs an integer division.

elem(tuple, index)

Gets the element at the zero-based `index` in `tuple`.

floor(number)

Returns the largest integer smaller than or equal to `number`.

hd(list)

Returns the head of a list. Raises `ArgumentError` if the list is empty.

left in right

Membership operator.

is_atom(term)

Returns `true` if `term` is an atom, otherwise returns `false`.

is_binary(term)

Returns `true` if `term` is a binary, otherwise returns `false`.

is_bitstring(term)

Returns `true` if `term` is a bitstring (including a binary), otherwise returns `false`.

is_boolean(term)

Returns `true` if `term` is either the atom `true` or the atom `false` (i.e., a boolean), otherwise returns `false`.

is_exception(term)

Returns `true` if `term` is an exception, otherwise returns `false`.

is_exception(term, name)

Returns `true` if `term` is an exception of `name`, otherwise returns `false`.

is_float(term)

Returns `true` if `term` is a floating-point number, otherwise returns `false`.

is_function(term)

Returns `true` if `term` is a function, otherwise returns `false`.

is_function(term, arity)

Returns `true` if `term` is a function that can be applied with `arity` number of arguments; otherwise returns `false`.

is_integer(term)

Returns `true` if `term` is an integer, otherwise returns `false`.

is_list(term)

Returns `true` if `term` is a list with zero or more elements, otherwise returns `false`.

is_map(term)

Returns `true` if `term` is a map, otherwise returns `false`.

is_map_key(map, key)

Returns `true` if `key` is a key in `map`, otherwise returns `false`.

is_nil(term)

Returns `true` if `term` is `nil`, `false` otherwise.

is_non_struct_map(term)

Returns `true` if `term` is a map that is not a struct, otherwise returns `false`.

is_number(term)

Returns `true` if `term` is either an integer or a floating-point number; otherwise returns `false`.

is_pid(term)

Returns `true` if `term` is a PID (process identifier), otherwise returns `false`.

is_port(term)

Returns `true` if `term` is a port identifier, otherwise returns `false`.

is_reference(term)

Returns `true` if `term` is a reference, otherwise returns `false`.

is_struct(term)

Returns `true` if `term` is a struct, otherwise returns `false`.

is_struct(term, name)

Returns `true` if `term` is a struct of `name`, otherwise returns `false`.

is_tuple(term)

Returns `true` if `term` is a tuple, otherwise returns `false`.

length(list)

Returns the length of `list`.

map_size(map)

Returns the size of a map.

max(first, second)

Returns the biggest of the two given terms according to their structural comparison.

min(first, second)

Returns the smallest of the two given terms according to their structural comparison.

node()

Returns an atom representing the name of the local node. If the node is not alive, `:nonode@nohost` is returned instead.

node(arg)

Returns the node where the given argument is located. The argument can be a PID, a reference, or a port. If the local node is not alive, `:nonode@nohost` is returned.

not value

Strictly boolean "not" operator.

left or right

Strictly boolean "or" operator.

rem(dividend, divisor)

Computes the remainder of an integer division.

round(number)

Rounds a number to the nearest integer.

self()

Returns the PID (process identifier) of the calling process.

tl(list)

Returns the tail of a list. Raises `ArgumentError` if the list is empty.

trunc(number)

Returns the integer part of `number`.

tuple_size(tuple)

Returns the size of a tuple.

Functions

left && right

Boolean "and" operator.

base ** exponent

Power operator.

left ++ right

List concatenation operator. Concatenates a proper list and a term, returning a list.

left -- right

List subtraction operator. Removes the first occurrence of an element on the left list for each element on the right.

..

Creates the full-slice range `0..-1//1`.

first..last

Creates a range from `first` to `last`.

first..last//step

Creates a range from `first` to `last` with `step`.

!value

Boolean "not" operator.

left <> right

Binary concatenation operator. Concatenates two binaries.

left =~ right

Text-based match operator. Matches the string on the `left` against the regular expression or string on the `right`.

@expr

Module attribute unary operator.

alias!(alias)

When used inside quoting, marks that the given alias should not be hygienized. This means the alias will be expanded when the macro is

expanded.

apply(fun, args)

Invokes the given anonymous function `fun` with the list of arguments `args`.

apply(module, function_name, args)

Invokes the given function from `module` with the list of arguments `args`.

binary_slice(binary, range)

Returns a binary from the offset given by the start of the range to the offset given by the end of the range.

binary_slice(binary, start, size)

Returns a binary starting at the offset `start` and of the given `size`.

binding(context \\ nil)

Returns the binding for the given context as a keyword list.

dbg(code \\ quote do binding() end, options \\ [])

Debugs the given `code`.

def(call, expr \\ nil)

Defines a public function with the given name and body.

defdelegate(funs, opts)

Defines a function that delegates to another module.

defexception(fields)

Defines an exception.

defguard(guard)

Defines a custom guard with the given name.

defguardp(guard)

Defines a private macro suitable for use in guard expressions.

defimpl(name, opts, do_block \\ [])

Defines an implementation for the given protocol.

defmacro(call, expr \\ nil)

Defines a public macro with the given name and body.

defmacrop(call, expr \\ nil)

Defines a private macro with the given name and body.

defmodule(alias, do_block)

Defines a module given by name with the given contents.

defoverridable(keywords_or_behaviour)

Makes the given definitions in the current module overridable.

defp(call, expr \\ nil)

Defines a private function with the given name and body.

defprotocol(name, do_block)

Defines a protocol.

defstruct(fields)

Defines a struct.

destructure(left, right)

Destructures two lists, assigning each term in the right one to the matching term in the left one.

exit(reason)

Stops the execution of the calling process with the given reason.

function_exported?(module, function, arity)

Returns `true` if `module` is loaded and contains a public `function` with the given `arity`, otherwise `false`.

get_and_update_in(path, fun)

Gets a value and updates a nested data structure via the given `path`.

get_and_update_in(data, keys, fun)

Gets a value and updates a nested structure.

get_in(path)

Gets a key from the nested structure via the given `path`, with nil-safe handling.

get_in(data, keys)

Gets a value from a nested structure with nil-safe handling.

if(condition, clauses)

Provides an `if/2` macro.

inspect(term, opts \\ [])

Inspects the given argument according to the `Inspect` protocol. The second argument is a keyword list with options to control inspection.

macro_exported?(module, macro, arity)

Returns `true` if `module` is loaded and contains a public `macro` with the given `arity`, otherwise `false`.

make_ref()

Returns an almost unique reference.

match?(pattern, expression)

A convenience macro that checks if the result of `expression` matches `pattern`.

pop_in(path)

Pops a key from the nested structure via the given `path`.

pop_in(data, keys)

Pops a key from the given nested structure.

put_elem(tuple, index, value)

Puts `value` at the given zero-based `index` in `tuple`.

put_in(path, value)

Puts a value in a nested structure via the given `path`.

put_in(data, keys, value)

Puts a value in a nested structure.

raise(message)

Raises an exception.
raise(exception, attributes)

Raises an exception.

reraise(message, stacktrace)

Raises an exception preserving a previous stacktrace.

reraise(exception, attributes, stacktrace)

Raises an exception preserving a previous stacktrace.

send(dest, message)

Sends a message to the given `dest` and returns the message.

sigil_C(term, modifiers)

Handles the sigil `~C` for charlists.

sigil_c(term, modifiers)

Handles the sigil `~c` for charlists.

sigil_D(date_string, modifiers)

Handles the sigil `~D` for dates.

sigil_N(naive_datetime_string, modifiers)

Handles the sigil `~N` for naive date times.

sigil_r(term, modifiers)

Handles the sigil `~r` for regular expressions.

sigil_S(term, modifiers)

Handles the sigil `~S` for strings.

sigil_s(term, modifiers)

Handles the sigil `~s` for strings.

sigil_T(time_string, modifiers)

Handles the sigil `~T` for times.

sigil_U(datetime_string, modifiers)

Handles the sigil `~U` to create a UTC `DateTime`.

sigil_W(term, modifiers)

Handles the sigil `~W` for list of words.

sigil_w(term, modifiers)

Handles the sigil `~w` for list of words.

spawn(fun)

Spawns the given function and returns its PID.

spawn(module, fun, args)

Spawns the given function `fun` from the given `module` passing it the given `args` and returns its PID.

spawn_link(fun)

Spawns the given function, links it to the current process, and returns its PID.

spawn_link(module, fun, args)

Spawns the given function `fun` from the given `module` passing it the given `args`, links it to the current process, and returns its PID.

spawn_monitor(fun)

Spawns the given function, monitors it and returns its PID and monitoring reference.

spawn_monitor(module, fun, args)

Spawns the given module and function passing the given args, monitors it and returns its PID and monitoring reference.

struct(struct, fields \\ [])

Creates and updates a struct.

struct!(struct, fields \\ [])

Similar to `struct/2` but checks for key validity.

tap(value, fun)

Pipes the first argument, `value`, into the second argument, a function `fun`, and returns `value` itself.

then(value, fun)

Pipes the first argument, `value`, into the second argument, a function `fun`, and returns the result of calling `fun`.

throw(term)

A non-local return from a function.

to_charlist(term)

Converts the given term to a charlist according to the `List.Chars` protocol.

to_string(term)

Converts the argument to a string according to the `String.Chars` protocol.

to_timeout(duration)

Constructs a millisecond timeout from the given components, duration, or timeout.

unless(condition, clauses) deprecated

Provides an `unless` macro.

update_in(path, fun)

Updates a nested structure via the given `path`.

update_in(data, keys, fun)

Updates a key in a nested structure.

use(module, opts \\ [])

Uses the given module in the current context.

var!(var, context \\ nil)

Marks that the given variable should not be hygienized.

left |> right

Pipe operator.

left || right

Boolean "or" operator.

Guards

left * right

```
@spec integer() * integer() :: integer()
@spec float() * float() :: float()
@spec integer() * float() :: float()
@spec float() * integer() :: float()
```

Arithmetic multiplication operator.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 * 2
2
```

+value

```
@spec +integer() :: integer()
@spec +float() :: float()
```

Arithmetic positive unary operator.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> +1
1
```

left + right

```
@spec integer() + integer() :: integer()
@spec float() + float() :: float()
@spec integer() + float() :: float()
@spec float() + integer() :: float()
```

Arithmetic addition operator.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 + 2
3
```

-value

```
@spec -0 :: 0
@spec -pos_integer() :: neg_integer()
@spec -neg_integer() :: pos_integer()
@spec -float() :: float()
```

Arithmetic negative unary operator.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> -2
-2
```

left - right

```
@spec integer() - integer() :: integer()
@spec float() - float() :: float()
@spec integer() - float() :: float()
@spec float() - integer() :: float()
```

Arithmetic subtraction operator.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 - 2
-1
```

left / right

```
@spec number() / number() :: float()
```

Arithmetic division operator.

The result is always a float. Use `div/2` and `rem/2` if you want an integer division or the remainder.

Raises `ArithmeticError` if `right` is 0 or 0.0.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 / 2
0.5

iex> -3.0 / 2.0
-1.5

iex> 5 / 1
5.0

7 / 0
** (ArithmeticError) bad argument in arithmetic
expression
```

left != right

```
@spec term() != term() :: boolean()
```

Not equal to operator.

Returns `true` if the two terms are not equal.

This operator considers 1 and 1.0 to be equal. For match comparison, use `!==/2` instead.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 != 2
true

iex> 1 != 1.0
false
```

left !== right

```
@spec term() !== term() :: boolean()
```

Strictly not equal to operator.

Returns `true` if the two terms are not exactly equal. See `===/2` for a definition of what is considered "exactly equal".

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 !== 2
true

iex> 1 !== 1.0
true
```

left < right

```
@spec term() < term() :: boolean()
```

Less-than operator.

Returns `true` if `left` is less than `right`.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 < 2
true
```

left <= right

```
@spec term() <= term() :: boolean()
```

Less-than or equal to operator.

Returns `true` if `left` is less than or equal to `right`.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 <= 2
true
```

left == right

```
@spec term() == term() :: boolean()
```

Equal to operator. Returns `true` if the two terms are equal.

This operator considers 1 and 1.0 to be equal. For stricter semantics, use `===/2` instead.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 == 2
false

iex> 1 == 1.0
true
```

left === right

```
@spec term() === term() :: boolean()
```

Strictly equal to operator.

Returns `true` if the two terms are exactly equal.

The terms are only considered to be exactly equal if they have the same value and are of the same type. For example, `1 == 1.0` returns `true`, but since they are of different types, `1 === 1.0` returns `false`.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 === 2
false

iex> 1 === 1.0
false
```

left > right

```
@spec term() > term() :: boolean()
```

Greater-than operator.

Returns `true` if `left` is more than `right`.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 > 2
false
```

left >= right

```
@spec term() >= term() :: boolean()
```

Greater-than or equal to operator.

Returns `true` if `left` is more than or equal to `right`.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 >= 2
false
```

abs(number)

```
@spec abs(number()) :: number()
```

Returns an integer or float which is the arithmetical absolute value of `number`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> abs(-3.33)
3.33

iex> abs(-3)
3
```

left and right

(macro)

Strictly boolean "and" operator.

If `left` is `false`, returns `false`, otherwise returns `right`.

Requires only the `left` operand to be a boolean since it short-circuits. If the `left` operand is not a boolean, a `BadBooleanError` exception is raised.

Allowed in guard tests.

Examples

```
iex> true and false
false

iex> true and "yay!"
"yay!"

iex> "yay!" and true
** (BadBooleanError) expected a boolean on left-side of
"and", got: "yay!"
```

binary_part(binary, start, size)

```
@spec binary_part(binary(), non neg integer(), integer()) ::
binary()
```

Extracts the part of the binary at `start` with `size`.

If `start` or `size` reference in any way outside the binary, an `ArgumentError` exception is raised.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> binary_part("foo", 1, 2)
"oo"
```

A negative `size` can be used to extract bytes that come *before* the byte at `start`:

```
iex> binary_part("Hello", 5, -3)
"llo"
```

An `ArgumentError` is raised when the `size` is outside of the binary:

```
binary_part("Hello", 0, 10)
** (ArgumentError) argument error
```

bit_size(bitstring)

```
@spec bit_size(bitstring()) :: non_neg_integer()
```

Returns an integer which is the size in bits of `bitstring`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bit_size(<<433::16, 3::3>>)
19

iex> bit_size(<<1, 2, 3>>)
24
```

byte_size(bitstring)

```
@spec byte_size(bitstring()) :: non_neg_integer()
```

Returns the number of bytes needed to contain `bitstring`.

That is, if the number of bits in `bitstring` is not divisible by 8, the resulting number of bytes will be rounded up (by excess). This operation happens in constant time.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> byte_size(<<433::16, 3::3>>)
3

iex> byte_size(<<1, 2, 3>>)
3
```

ceil(number)

(since 1.8.0)

```
@spec ceil(number()) :: integer()
```

Returns the smallest integer greater than or equal to `number`.

If you want to perform ceil operation on other decimal places, use `Float.ceil/2` instead.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> ceil(10)
10
```

```
iex> ceil(10.1)
11

iex> ceil(-10.1)
-10
```

div(dividend, divisor)

```
@spec div(integer(), neg_integer() | pos_integer()) ::
integer()
```

Performs an integer division.

Raises an `ArithmeticError` exception if one of the arguments is not an integer, or when the `divisor` is `0`.

`div/2` performs *truncated* integer division. This means that the result is always rounded towards zero.

If you want to perform floored integer division (rounding towards negative infinity), use `Integer.floor_div/2` instead.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> div(5, 2)
2

iex> div(6, -4)
-1
```

```
iex> div(-99, 2)
-49

div(100, 0)
** (ArithmeticError) bad argument in arithmetic
expression
```

elem(tuple, index)

```
@spec elem(tuple(), non_neg_integer()) :: term()
```

Gets the element at the zero-based `index` in `tuple`.

It raises `ArgumentError` when index is negative or it is out of range of the tuple elements.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> tuple = {:foo, :bar, 3}
iex> elem(tuple, 1)
:bar

elem({}, 0)
** (ArgumentError) argument error

elem({:foo, :bar}, 2)
** (ArgumentError) argument error
```

floor(number)

(since 1.8.0)

```
@spec floor(number()) :: integer()
```

Returns the largest integer smaller than or equal to `number`.

If you want to perform floor operation on other decimal places, use `Float.floor/2` instead.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> floor(10)
10

iex> floor(9.7)
9

iex> floor(-9.7)
-10
```

hd(list)

```
@spec hd(nonempty_maybe_improper_list(elem, term())) :: elem
when elem: term()
```

Returns the head of a list. Raises `ArgumentError` if the list is empty.

The head of a list is its first element.

It works with improper lists.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> hd([1, 2, 3, 4])
1

iex> hd([1 | 2])
1
```

Giving it an empty list raises:

```
hd([])
** (ArgumentError) argument error
```

left in right

(macro)

Membership operator.

Checks if the element on the left-hand side is a member of the collection on the right-hand side.

Examples

```
iex> x = 1
iex> x in [1, 2, 3]
true
```

This operator (which is a macro) simply translates to a call to `Enum.member?/2`. The example above would translate to:

```
Enum.member?([1, 2, 3], x)
```

Elixir also supports `left not in right`, which evaluates to `not(left in right)`:

```
iex> x = 1
iex> x not in [1, 2, 3]
false
```

Guards

The `in/2` operator (as well as `not in`) can be used in guard clauses as long as the right-hand side is a range or a list.

If the right-hand side is a list, Elixir will expand the operator to a valid guard expression which needs to check each value. For example:

```
when x in [1, 2, 3]
```

translates to:

```
when x === 1 or x === 2 or x === 3
```

However, this construct will be inefficient for large lists. In such cases, it is best to stop using guards and use a more appropriate data structure, such as `MapSet`.

If the right-hand side is a range, a more efficient comparison check will be done. For example:

```
when x in 1..1000
```

translates roughly to:

```
when x >= 1 and x <= 1000
```

AST considerations

`left not in right` is parsed by the compiler into the AST:

```
{:not, _, [{:in, _, [left, right]}]}
```

This is the same AST as `not(left in right)`.

Additionally, `Macro.to_string/2` and `Code.format_string!/2` will translate all occurrences of this AST to `left not in right`.

is_atom(term)

```
@spec is_atom(term()) :: boolean()
```

Returns `true` if `term` is an atom, otherwise returns `false`.

Note `true`, `false`, and `nil` are atoms in Elixir, as well as module names. Therefore this function will return `true` to all of those values.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_atom(:name)
true

iex> is_atom(false)
true

iex> is_atom(AnAtom)
true

iex> is_atom("string")
false
```

is_binary(term)

```
@spec is_binary(term()) :: boolean()
```

Returns `true` if `term` is a binary, otherwise returns `false`.

A binary always contains a complete number of bytes.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_binary("foo")
true
iex> is_binary(<<1::3>>)
false
```

is_bitstring(term)

```
@spec is_bitstring(term()) :: boolean()
```

Returns `true` if `term` is a bitstring (including a binary), otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_bitstring("foo")
true
iex> is_bitstring(<<1::3>>)
true
```

is_boolean(term)

```
@spec is_boolean(term()) :: boolean()
```

Returns `true` if `term` is either the atom `true` or the atom `false` (i.e., a boolean), otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_boolean(false)
true
```

```
iex> is_boolean(true)
true

iex> is_boolean(:test)
false
```

is_exception(term)

(since 1.11.0) (macro)

Returns `true` if `term` is an exception, otherwise returns `false`.

Allowed in guard tests.

Examples

```
iex> is_exception(%RuntimeError{})
true

iex> is_exception(%{})
false
```

is_exception(term, name)

(since 1.11.0) (macro)

Returns `true` if `term` is an exception of `name`, otherwise returns `false`.

Allowed in guard tests.

Examples

```
iex> is_exception(%RuntimeError{}, RuntimeError)
true

iex> is_exception(%RuntimeError{}, Macro.Env)
false
```

is_float(term)

```
@spec is_float(term()) :: boolean()
```

Returns `true` if `term` is a floating-point number, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_float(2.15)
true

iex> is_float(3.45e5)
true

iex> is_float(5)
false
```

is_function(term)

```
@spec is_function(term()) :: boolean()
```

Returns `true` if `term` is a function, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_function(fn x -> x + x end)
true

iex> is_function("not a function")
false
```

is_function(term, arity)

```
@spec is_function(term(), non_neg_integer()) :: boolean()
```

Returns `true` if `term` is a function that can be applied with `arity` number of arguments; otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_function(fn x -> x * 2 end, 1)
true
iex> is_function(fn x -> x * 2 end, 2)
false
```

is_integer(term)

```
@spec is_integer(term()) :: boolean()
```

Returns `true` if `term` is an integer, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_integer(5)
true

iex> is_integer(5.0)
false
```

is_list(term)

```
@spec is_list(term()) :: boolean()
```

Returns `true` if `term` is a list with zero or more elements, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_list([1, 2, 3])
true
```

```
iex> is_list(key: :sum, value: 3)
true

iex> is_list({1, 2, 3})
false
```

is_map(term)

```
@spec is_map(term()) :: boolean()
```

Returns `true` if `term` is a map, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

> **Structs are maps**
>
> Structs are also maps, and many of Elixir data structures are implemented using structs: `Range`s, `Regex`es, `Date`s...
>
> ```
> iex> is_map(1..10)
> true
> iex> is_map(~D[2024-04-18])
> true
> ```
>
> If you mean to specifically check for non-struct maps, use `is_non_struct_map/1` instead.
>
> ```
> iex> is_non_struct_map(1..10)
> false
> ```

is_map_key(map, key)

(since 1.10.0)

```
@spec is_map_key(map(), term()) :: boolean()
```

Returns `true` if `key` is a key in `map`, otherwise returns `false`.

It raises `BadMapError` if the first element is not a map.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_map_key(%{a: "foo", b: "bar"}, :a)
true

iex> is_map_key(%{a: "foo", b: "bar"}, :c)
false
```

is_nil(term)

(macro)

Returns `true` if `term` is `nil`, `false` otherwise.

Allowed in guard clauses.

Examples

```
iex> is_nil(1 + 2)
false

iex> is_nil(nil)
true
```

is_non_struct_map(term)

(since 1.17.0) (macro)

Returns `true` if `term` is a map that is not a struct, otherwise returns `false`.

Allowed in guard tests.

Examples

```
iex> is_non_struct_map(%{})
true

iex> is_non_struct_map(URI.parse("/"))
false

iex> is_non_struct_map(nil)
false
```

is_number(term)

```
@spec is_number(term()) :: boolean()
```

Returns `true` if `term` is either an integer or a floating-point number; otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_number(2.15)
true

iex> is_number(5)
true

iex> is_number(:one)
false
```

is_pid(term)

```
@spec is_pid(term()) :: boolean()
```

Returns `true` if `term` is a PID (process identifier), otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> {:ok, agent_pid} = Agent.start_link(fn -> 0 end)
iex> is_pid(agent_pid)
true

iex> is_pid(self())
```

```
true

iex> is_pid(:pid)
false
```

is_port(term)

```
@spec is_port(term()) :: boolean()
```

Returns `true` if `term` is a port identifier, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> [port | _] = Port.list()
iex> is_port(port)
true

iex> is_port(:port)
false
```

is_reference(term)

```
@spec is_reference(term()) :: boolean()
```

Returns `true` if `term` is a reference, otherwise returns `false`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> ref = make_ref()
iex> is_reference(ref)
true

iex> is_reference(:ref)
false
```

is_struct(term)

(since 1.10.0) (macro)

Returns `true` if `term` is a struct, otherwise returns `false`.

Allowed in guard tests.

Examples

```
iex> is_struct(URI.parse("/"))
true

iex> is_struct(%{})
false
```

is_struct(term, name)

(since 1.11.0) (macro)

Returns `true` if `term` is a struct of `name`, otherwise returns `false`.

is_struct/2 does not check that name exists and is a valid struct. If you want such validations, you must pattern match on the struct instead, such as match?(%URI{}, arg).

Allowed in guard tests.

Examples

```
iex> is_struct(URI.parse("/"), URI)
true

iex> is_struct(URI.parse("/"), Macro.Env)
false
```

is_tuple(term)

```
@spec is_tuple(term()) :: boolean()
```

Returns true if term is a tuple, otherwise returns false.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> is_tuple({1, 2, 3})
true

iex> is_tuple({})
true
```

```
iex> is_tuple(true)
false
```

length(list)

```
@spec length(list()) :: non_neg_integer()
```

Returns the length of `list`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> length([1, 2, 3, 4, 5, 6, 7, 8, 9])
9
```

map_size(map)

```
@spec map_size(map()) :: non_neg_integer()
```

Returns the size of a map.

The size of a map is the number of key-value pairs that the map contains.

This operation happens in constant time.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> map_size(%{a: "foo", b: "bar"})
2
```

max(first, second)

```
@spec max(first, second) :: first | second when first: term(), second: term()
```

Returns the biggest of the two given terms according to their structural comparison.

If the terms compare equal, the first one is returned.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> max(1, 2)
2
iex> max("a", "b")
"b"
```

min(first, second)

```
@spec min(first, second) :: first | second when first: term(), second: term()
```

Returns the smallest of the two given terms according to their structural comparison.

If the terms compare equal, the first one is returned.

This performs a structural comparison where all Elixir terms can be compared with each other. See the "Structural comparison" section for more information.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> min(1, 2)
1
iex> min("foo", "bar")
"bar"
```

node()

```
@spec node() :: node()
```

Returns an atom representing the name of the local node. If the node is not alive, `:nonode@nohost` is returned instead.

Allowed in guard tests. Inlined by the compiler.

node(arg)

```
@spec node(pid() | reference() | port()) :: node()
```

Returns the node where the given argument is located. The argument can be a PID, a reference, or a port. If the local node is not alive, `:nonode@nohost` is returned.

Allowed in guard tests. Inlined by the compiler.

not value

```
@spec not true :: false
@spec not false :: true
```

Strictly boolean "not" operator.

`value` must be a boolean; if it's not, an `ArgumentError` exception is raised.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> not false
true
```

left or right

(macro)

Strictly boolean "or" operator.

If `left` is `true`, returns `true`, otherwise returns `right`.

Requires only the `left` operand to be a boolean since it short-circuits. If the `left` operand is not a boolean, a `BadBooleanError` exception is raised.

Allowed in guard tests.

Examples

```
iex> true or false
true

iex> false or 42
42

iex> 42 or false
** (BadBooleanError) expected a boolean on left-side of
"or", got: 42
```

rem(dividend, divisor)

```
@spec rem(integer(), neg_integer() | pos_integer()) ::
integer()
```

Computes the remainder of an integer division.

`rem/2` uses truncated division, which means that the result will always have the sign of the `dividend`.

Raises an `ArithmeticError` exception if one of the arguments is not an integer, or when the `divisor` is `0`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> rem(5, 2)
1
iex> rem(6, -4)
2
```

round(number)

```
@spec round(number()) :: integer()
```

Rounds a number to the nearest integer.

If the number is equidistant to the two nearest integers, rounds away from zero.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> round(5.6)
6

iex> round(5.2)
5

iex> round(-9.9)
-10

iex> round(-9)
-9
```

```
iex> round(2.5)
3

iex> round(-2.5)
-3
```

self()

```
@spec self() :: pid()
```

Returns the PID (process identifier) of the calling process.

Allowed in guard clauses. Inlined by the compiler.

tl(list)

```
@spec tl(nonempty maybe improper list(elem, last)) ::
  maybe improper list(elem, last) | last
when elem: term(), last: term()
```

Returns the tail of a list. Raises `ArgumentError` if the list is empty.

The tail of a list is the list without its first element.

It works with improper lists.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> tl([1, 2, 3, :go])
[2, 3, :go]

iex> tl([:one])
[]

iex> tl([:a, :b | :improper_end])
[:b | :improper_end]

iex> tl([:a | %{b: 1}])
%{b: 1}
```

Giving it an empty list raises:

```
tl([])
** (ArgumentError) argument error
```

trunc(number)

```
@spec trunc(number()) :: integer()
```

Returns the integer part of `number`.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> trunc(5.4)
5

iex> trunc(-5.99)
```

```
-5

iex> trunc(-5)
-5
```

tuple_size(tuple)

```
@spec tuple_size(tuple()) :: non_neg_integer()
```

Returns the size of a tuple.

This operation happens in constant time.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> tuple_size({:a, :b, :c})
3
```

Functions

left && right

(macro)

Boolean "and" operator.

Provides a short-circuit operator that evaluates and returns the second expression only if the first one evaluates to a truthy value (neither `false` nor `nil`). Returns the first expression otherwise.

Not allowed in guard clauses.

Examples

```
iex> Enum.empty?([]) && Enum.empty?([])
true

iex> List.first([]) && true
nil

iex> Enum.empty?([]) && List.first([1])
1

iex> false && throw(:bad)
false
```

Note that, unlike `and/2`, this operator accepts any expression as the first argument, not only booleans.

base ** exponent

(since 1.13.0)

```
@spec integer() ** non_neg_integer() :: integer()
@spec integer() ** neg_integer() :: float()
@spec float() ** float() :: float()
@spec integer() ** float() :: float()
@spec float() ** integer() :: float()
```

Power operator.

It takes two numbers for input. If both are integers and the right-hand side (the `exponent`) is also greater than or equal to 0, then the result will also be an integer. Otherwise it returns a float.

Examples

```
iex> 2 ** 2
4
iex> 2 ** -4
0.0625

iex> 2.0 ** 2
4.0
iex> 2 ** 2.0
4.0
```

left ++ right

```
@spec [] ++ a :: a when a: term()
@spec [...] ++ term() :: maybe improper list()
```

List concatenation operator. Concatenates a proper list and a term, returning a list.

The complexity of `a ++ b` is proportional to `length(a)`, so avoid repeatedly appending to lists of arbitrary length, for example, `list ++ [element]`. Instead, consider prepending via `[element | rest]` and then reversing.

If the `right` operand is not a proper list, it returns an improper list. If the `left` operand is not a proper list, it raises `ArgumentError`. If the `left` operand is an empty list, it returns the `right` operand.

Inlined by the compiler.

Examples

```
iex> [1] ++ [2, 3]
[1, 2, 3]

iex> ~c"foo" ++ ~c"bar"
~c"foobar"

# a non-list on the right will return an improper list
# with said element at the end
iex> [1, 2] ++ 3
[1, 2 | 3]
iex> [1, 2] ++ {3, 4}
[1, 2 | {3, 4}]

# improper list on the right will return an improper list
iex> [1] ++ [2 | 3]
[1, 2 | 3]

# empty list on the left will return the right operand
iex> [] ++ 1
1
```

The `++/2` operator is right associative, meaning:

```
iex> [1, 2, 3] -- [1] ++ [2]
[3]
```

As it is equivalent to:

```
iex> [1, 2, 3] -- ([1] ++ [2])
[3]
```

left -- right

```
@spec list() -- list() :: list()
```

List subtraction operator. Removes the first occurrence of an element on the left list for each element on the right.

This function is optimized so the complexity of `a -- b` is proportional to `length(a) * log(length(b))`. See also the Erlang efficiency guide.

Inlined by the compiler.

Examples

```
iex> [1, 2, 3] -- [1, 2]
[3]

iex> [1, 2, 3, 2, 1] -- [1, 2, 2]
[3, 1]
```

The `--/2` operator is right associative, meaning:

```
iex> [1, 2, 3] -- [2] -- [3]
[1, 3]
```

As it is equivalent to:

```
iex> [1, 2, 3] -- ([2] -- [3])
[1, 3]
```

..

(since 1.14.0) (macro)

Creates the full-slice range `0..-1//1`.

This macro returns a range with the following properties:

- When enumerated, it is empty
- When used as a `slice`, it returns the sliced element as is

See `..///3` and the `Range` module for more information.

Examples

```
iex> Enum.to_list(..)
[]

iex> String.slice("Hello world!", ..)
"Hello world!"
```

first..last

(macro)

Creates a range from `first` to `last`.

If first is less than last, the range will be increasing from first to last. If first is equal to last, the range will contain one element, which is the number itself.

If first is more than last, the range will be decreasing from first to last, albeit this behavior is deprecated. Instead prefer to explicitly list the step with `first..last//-1`.

See the `Range` module for more information.

Examples

```
iex> 0 in 1..3
false
iex> 2 in 1..3
true

iex> Enum.to_list(1..3)
[1, 2, 3]
```

first..last//step

(since 1.12.0) (macro)

Creates a range from `first` to `last` with `step`.

See the `Range` module for more information.

Examples

```
iex> 0 in 1..3//1
false
iex> 2 in 1..3//1
true
iex> 2 in 1..3//2
false

iex> Enum.to_list(1..3//1)
[1, 2, 3]
iex> Enum.to_list(1..3//2)
[1, 3]
iex> Enum.to_list(3..1//-1)
[3, 2, 1]
```

```
iex> Enum.to_list(1..0//1)
[]
```

!value

(macro)

Boolean "not" operator.

Receives any value (not just booleans) and returns `true` if `value` is `false` or `nil`; returns `false` otherwise.

Not allowed in guard clauses.

Examples

```
iex> !Enum.empty?([])
false

iex> !List.first([])
true
```

left <> right

(macro)

Binary concatenation operator. Concatenates two binaries.

Raises an `ArgumentError` if one of the sides aren't binaries.

Examples

```
iex> "foo" <> "bar"
"foobar"
```

The `<>/2` operator can also be used in pattern matching (and guard clauses) as long as the left argument is a literal binary:

```
iex> "foo" <> x = "foobar"
iex> x
"bar"
```

`x <> "bar" = "foobar"` would result in an `ArgumentError` exception.

left =~ right

```
@spec String.t() =~ (String.t() | Regex.t()) :: boolean()
```

Text-based match operator. Matches the string on the `left` against the regular expression or string on the `right`.

If `right` is a regular expression, returns `true` if `left` matches right.

If `right` is a string, returns `true` if `left` contains `right`.

Examples

```
iex> "abcd" =~ ~r/c(d)/
true

iex> "abcd" =~ ~r/e/
false
```

```
iex> "abcd" =~ ~r//
true

iex> "abcd" =~ "bc"
true

iex> "abcd" =~ "ad"
false

iex> "abcd" =~ "abcd"
true

iex> "abcd" =~ ""
true
```

For more information about regular expressions, please check the `Regex` module.

@expr

(macro)

Module attribute unary operator.

Reads and writes attributes in the current module.

The canonical example for attributes is annotating that a module implements an OTP behaviour, such as `GenServer`:

```
defmodule MyServer do
  @behaviour GenServer
  # ... callbacks ...
end
```

By default Elixir supports all the module attributes supported by Erlang, but custom attributes can be used as well:

```
defmodule MyServer do
  @my_data 13
  IO.inspect(@my_data)
  #=> 13
end
```

Unlike Erlang, such attributes are not stored in the module by default since it is common in Elixir to use custom attributes to store temporary data that will be available at compile-time. Custom attributes may be configured to behave closer to Erlang by using `Module.register_attribute/3`.

> **Prefixing module attributes**
>
> Libraries and frameworks should consider prefixing any module attributes that are private by underscore, such as `@_my_data`, so code completion tools do not show them on suggestions and prompts.

Finally, note that attributes can also be read inside functions:

```
defmodule MyServer do
  @my_data 11
  def first_data, do: @my_data
  @my_data 13
  def second_data, do: @my_data
end

MyServer.first_data()
#=> 11
```

```
MyServer.second_data()
#=> 13
```

It is important to note that reading an attribute takes a snapshot of its current value. In other words, the value is read at compilation time and not at runtime. Check the `Module` module for other functions to manipulate module attributes.

Attention! Multiple references of the same attribute

As mentioned above, every time you read a module attribute, a snapshot of its current value is taken. Therefore, if you are storing large values inside module attributes (for example, embedding external files in module attributes), you should avoid referencing the same attribute multiple times. For example, don't do this:

```
@files %{
  example1: File.read!("lib/example1.data"),
  example2: File.read!("lib/example2.data")
}

def example1, do: @files[:example1]
def example2, do: @files[:example2]
```

In the above, each reference to `@files` may end-up with a complete and individual copy of the whole `@files` module attribute. Instead, reference the module attribute once in a private function:

```
@files %{
  example1: File.read!("lib/example1.data"),
  example2: File.read!("lib/example2.data")
```

```
}

defp files(), do: @files
def example1, do: files()[:example1]
def example2, do: files()[:example2]
```

alias!(alias)

(macro)

When used inside quoting, marks that the given alias should not be hygienized. This means the alias will be expanded when the macro is expanded.

Check `quote/2` for more information.

apply(fun, args)

```
@spec apply(fun(), [any()]) :: any()
```

Invokes the given anonymous function `fun` with the list of arguments `args`.

If the number of arguments is known at compile time, prefer `fun.(arg_1, arg_2, ..., arg_n)` as it is clearer than `apply(fun, [arg_1, arg_2, ..., arg_n])`.

Inlined by the compiler.

Examples

```
iex> apply(fn x -> x * 2 end, [2])
4
```

apply(module, function_name, args)

```
@spec apply(module(), function_name :: atom(), [any()]) ::
any()
```

Invokes the given function from `module` with the list of arguments `args`.

`apply/3` is used to invoke functions where the module, function name or arguments are defined dynamically at runtime. For this reason, you can't invoke macros using `apply/3`, only functions.

If the number of arguments and the function name are known at compile time, prefer `module.function(arg_1, arg_2, ..., arg_n)` as it is clearer than `apply(module, :function, [arg_1, arg_2, ..., arg_n])`.

`apply/3` cannot be used to call private functions.

Inlined by the compiler.

Examples

```
iex> apply(Enum, :reverse, [[1, 2, 3]])
[3, 2, 1]
```

binary_slice(binary, range)

(since 1.14.0)

Returns a binary from the offset given by the start of the range to the offset given by the end of the range.

If the start or end of the range are negative, they are converted into positive indices based on the binary size. For example, `-1` means the last byte of the binary.

This is similar to `binary_part/3` except that it works with ranges and it is not allowed in guards.

This function works with bytes. For a slicing operation that considers characters, see `String.slice/2`.

Examples

```
iex> binary_slice("elixir", 0..5)
"elixir"
iex> binary_slice("elixir", 1..3)
"lix"
iex> binary_slice("elixir", 1..10)
"lixir"

iex> binary_slice("elixir", -4..-1)
"ixir"
iex> binary_slice("elixir", -4..6)
"ixir"
iex> binary_slice("elixir", -10..10)
"elixir"
```

For ranges where `start > stop`, you need to explicitly mark them as increasing:

```
iex> binary_slice("elixir", 2..-1//1)
"ixir"
iex> binary_slice("elixir", 1..-2//1)
"lixi"
```

You can use `../0` as a shortcut for `0..-1//1`, which returns the whole binary as is:

```
iex> binary_slice("elixir", ..)
"elixir"
```

The step can be any positive number. For example, to get every 2 characters of the binary:

```
iex> binary_slice("elixir", 0..-1//2)
"eii"
```

If the first position is after the string ends or after the last position of the range, it returns an empty string:

```
iex> binary_slice("elixir", 10..3//1)
""
iex> binary_slice("elixir", -10..-7)
""
iex> binary_slice("a", 1..1500)
""
```

binary_slice(binary, start, size)

(since 1.14.0)

Returns a binary starting at the offset `start` and of the given `size`.

This is similar to `binary_part/3` except that if `start + size` is greater than the binary size, it automatically clips it to the binary size instead of raising. Opposite to `binary_part/3`, this function is not allowed in guards.

This function works with bytes. For a slicing operation that considers characters, see `String.slice/3`.

Examples

```
iex> binary_slice("elixir", 0, 6)
"elixir"
iex> binary_slice("elixir", 0, 5)
"elixi"
iex> binary_slice("elixir", 1, 4)
"lixi"
iex> binary_slice("elixir", 0, 10)
"elixir"
```

If `start` is negative, it is normalized against the binary size and clamped to 0:

```
iex> binary_slice("elixir", -3, 10)
"xir"
iex> binary_slice("elixir", -10, 10)
"elixir"
```

If the `size` is zero, an empty binary is returned:

```
iex> binary_slice("elixir", 1, 0)
""
```

If `start` is greater than or equal to the binary size, an empty binary is returned:

```
iex> binary_slice("elixir", 10, 10)
""
```

binding(context \\ nil)

(macro)

Returns the binding for the given context as a keyword list.

In the returned result, keys are variable names and values are the corresponding variable values.

If the given `context` is `nil` (by default it is), the binding for the current context is returned.

Examples

```
iex> x = 1
iex> binding()
[x: 1]
iex> x = 2
iex> binding()
[x: 2]

iex> binding(:foo)
[]
```

```
iex> var!(x, :foo) = 1
1
iex> binding(:foo)
[x: 1]
```

dbg(code \\ quote do binding() end, options \\ [])

(since 1.14.0) (macro)

Debugs the given `code`.

`dbg/2` can be used to debug the given `code` through a configurable debug function. It returns the result of the given code.

Examples

Let's take this call to `dbg/2`:

```
dbg(Atom.to_string(:debugging))
#=> "debugging"
```

It returns the string `"debugging"`, which is the result of the `Atom.to_string/1` call. Additionally, the call above prints:

```
[my_file.ex:10: MyMod.my_fun/0]
Atom.to_string(:debugging) #=> "debugging"
```

The default debugging function prints additional debugging info when dealing with pipelines. It prints the values at every "step" of the pipeline.

```
"Elixir is cool!"
|> String.trim_trailing("!")
|> String.split()
|> List.first()
|> dbg()
#=> "Elixir"
```

The code above prints:

```
[my_file.ex:10: MyMod.my_fun/0]
"Elixir is cool!" #=> "Elixir is cool!"
|> String.trim_trailing("!") #=> "Elixir is cool"
|> String.split() #=> ["Elixir", "is", "cool"]
|> List.first() #=> "Elixir"
```

With no arguments, `dbg()` debugs information about the current binding. See `binding/1`.

`dbg` inside IEx

You can enable IEx to replace `dbg` with its `IEx.pry/0` backend by calling:

```
$ iex --dbg pry
```

In such cases, `dbg` will start a `pry` session where you can interact with the imports, aliases, and variables of the current environment at the location of the `dbg` call.

If you call `dbg` at the end of a pipeline (using `|>`) within IEx, you are able to go through each step of the pipeline one by one by entering

"next" (or "n").

Note `dbg` only supports stepping for pipelines (in other words, it can only step through the code it sees). For general stepping, you can set breakpoints using `IEx.break!/4`.

For more information, see IEx documentation.

Configuring the debug function

One of the benefits of `dbg/2` is that its debugging logic is configurable, allowing tools to extend `dbg` with enhanced behaviour. This is done, for example, by `IEx` which extends `dbg` with an interactive shell where you can directly inspect and access values.

The debug function can be configured at compile time through the `:dbg_callback` key of the `:elixir` application. The debug function must be a `{module, function, args}` tuple. The `function` function in `module` will be invoked with three arguments *prepended* to `args`:

1. The AST of `code`
2. The AST of `options`
3. The `Macro.Env` environment of where `dbg/2` is invoked

The debug function is invoked at compile time and it must also return an AST. The AST is expected to ultimately return the result of evaluating the debugged expression.

Here's a simple example:

```
defmodule MyMod do
  def debug_fun(code, options, caller, device) do
    quote do
      result = unquote(code)
```

```
      IO.inspect(unquote(device), result, label:
unquote(Macro.to_string(code)))
    end
  end
end
```

To configure the debug function:

```
# In config/config.exs
config :elixir, :dbg_callback, {MyMod, :debug_fun,
[:stdio]}
```

Default debug function

By default, the debug function we use is `Macro.dbg/3`. It just prints information about the code to standard output and returns the value returned by evaluating `code`. `options` are used to control how terms are inspected. They are the same options accepted by `inspect/2`.

def(call, expr \\ nil)

(macro)

Defines a public function with the given name and body.

Examples

```
defmodule Foo do
  def bar, do: :baz
end
```

```
Foo.bar()
#=> :baz
```

A function that expects arguments can be defined as follows:

```
defmodule Foo do
  def sum(a, b) do
    a + b
  end
end
```

In the example above, a `sum/2` function is defined; this function receives two arguments and returns their sum.

Default arguments

`\\` is used to specify a default value for a parameter of a function. For example:

```
defmodule MyMath do
  def multiply_by(number, factor \\ 2) do
    number * factor
  end
end

MyMath.multiply_by(4, 3)
#=> 12

MyMath.multiply_by(4)
#=> 8
```

The compiler translates this into multiple functions with different arities, here `MyMath.multiply_by/1` and `MyMath.multiply_by/2`, that represent cases when arguments for parameters with default values are passed or not passed.

When defining a function with default arguments as well as multiple explicitly declared clauses, you must write a function head that declares the defaults. For example:

```
defmodule MyString do
  def join(string1, string2 \\ nil, separator \\ " ")

  def join(string1, nil, _separator) do
    string1
  end

  def join(string1, string2, separator) do
    string1 <> separator <> string2
  end
end
```

Note that `\\` can't be used with anonymous functions because they can only have a sole arity.

Keyword lists with default arguments

Functions containing many arguments can benefit from using `Keyword` lists to group and pass attributes as a single value.

```
defmodule MyConfiguration do
  @default_opts [storage: "local"]

  def configure(resource, opts \\ []) do
    opts = Keyword.merge(@default_opts, opts)
```

```
    storage = opts[:storage]
    # ...
  end
end
```

The difference between using `Map` and `Keyword` to store many arguments is `Keyword`'s keys:

- must be atoms
- can be given more than once
- ordered, as specified by the developer

Function names

Function and variable names in Elixir must start with an underscore or a Unicode letter that is not in uppercase or titlecase. They may continue using a sequence of Unicode letters, numbers, and underscores. They may end in `?` or `!`. Elixir's Naming Conventions suggest for function and variable names to be written in the `snake_case` format.

`rescue/catch/after/else`

Function bodies support `rescue`, `catch`, `after`, and `else` as `try/1` does (known as "implicit try"). For example, the following two functions are equivalent:

```
def convert(number) do
  try do
    String.to_integer(number)
  rescue
    e in ArgumentError -> {:error, e.message}
  end
end
```

```
def convert(number) do
  String.to_integer(number)
rescue
  e in ArgumentError -> {:error, e.message}
end
```

defdelegate(funs, opts)

(macro)

Defines a function that delegates to another module.

Functions defined with `defdelegate/2` are public and can be invoked from outside the module they're defined in, as if they were defined using `def/2`. Therefore, `defdelegate/2` is about extending the current module's public API. If what you want is to invoke a function defined in another module without using its full module name, then use `alias/2` to shorten the module name or use `import/2` to be able to invoke the function without the module name altogether.

Delegation only works with functions; delegating macros is not supported.

Check `def/2` for rules on naming and default arguments.

Options

- `:to` - the module to dispatch to.
- `:as` - the function to call on the target given in `:to`. This parameter is optional and defaults to the name being delegated (`funs`).

Examples

```
defmodule MyList do
  defdelegate reverse(list), to: Enum
  defdelegate other_reverse(list), to: Enum, as: :reverse
end

MyList.reverse([1, 2, 3])
#=> [3, 2, 1]

MyList.other_reverse([1, 2, 3])
#=> [3, 2, 1]
```

defexception(fields)

(macro)

Defines an exception.

Exceptions are structs backed by a module that implements the `Exception` behaviour. The `Exception` behaviour requires two functions to be implemented:

- `exception/1` - receives the arguments given to `raise/2` and returns the exception struct. The default implementation accepts either a set of keyword arguments that is merged into the struct or a string to be used as the exception's message.
- `message/1` - receives the exception struct and must return its message. Most commonly exceptions have a message field which by default is accessed by this function. However, if an exception does not have a message field, this function must be explicitly implemented.

Since exceptions are structs, the API supported by `defstruct/1` is also available in `defexception/1`.

Raising exceptions

The most common way to raise an exception is via `raise/2`:

```
defmodule MyAppError do
  defexception [:message]
end

value = [:hello]

raise MyAppError,
  message: "did not get what was expected, got: #
{inspect(value)}"
```

In many cases it is more convenient to pass the expected value to `raise/2` and generate the message in the `Exception.exception/1` callback:

```
defmodule MyAppError do
  defexception [:message]

  @impl true
  def exception(value) do
    msg = "did not get what was expected, got: #
{inspect(value)}"
    %MyAppError{message: msg}
  end
end
```

```
  raise MyAppError, value
```

The example above shows the preferred strategy for customizing exception messages.

defguard(guard)

(since 1.6.0) (macro)

```
@spec defguard(Macro.t()) :: Macro.t()
```

Defines a custom guard with the given name.

Once defined, custom guards can be invoked within regular code or in guards. The module that contains the custom guard must be required before usage.

Custom guards are defined by providing a valid guard expression to the right-hand side of `when`. `defguard` will then expand and validate the expressions as guards. `defguard` will raise at compile time if the guard uses expressions that aren't allowed in guard clauses.

When defining your own guards, consider the naming conventions around boolean-returning guards.

Example

For example, to define a guard similar to `Integer.is_even/1`, you can write:

```
defmodule Integer.Guards do
  defguard is_even(value) when is_integer(value) and
```

```
  rem(value, 2) == 0
  end
```

which can then be used as:

```
  require Integer.Guards
  Integer.Guards.is_even(3)
  #=> false
```

Implementation details

Behind the scenes, `defguard` will generate a macro which can be used inside and outside of guards, preserving their respective semantics.

When invoked inside a guard, it behaves as if the right-hand side of `when` is injected as part of the guard, replacing the custom guard arguments by the expressions given as inputs.

When invoked outside of a guard, it preserves regular function calling semantics with one caveat: all arguments are evaluated before invocation, except arguments which are unused, which are then never evaluated.

defguardp(guard)

(since 1.6.0) (macro)

```
@spec defguardp(Macro.t()) :: Macro.t()
```

Defines a private macro suitable for use in guard expressions.

It raises at compile time if the `guard` uses expressions that aren't allowed in guard clauses, and otherwise creates a private macro that can

be used both inside or outside guards in the current module.

When defining your own guards, consider the naming conventions around boolean-returning guards.

Similar to `defmacrop/2`, `defguardp/1` must be defined before its use in the current module.

defimpl(name, opts, do_block \\ [])

(macro)

Defines an implementation for the given protocol.

See the `Protocol` module for more information.

defmacro(call, expr \\ nil)

(macro)

Defines a public macro with the given name and body.

Macros must be defined before its usage.

Check `def/2` for rules on naming and default arguments.

Examples

```
defmodule MyLogic do
  defmacro unless(expr, opts) do
    quote do
      if !unquote(expr), unquote(opts)
    end
```

```
  end
end

require MyLogic

MyLogic.unless false do
  IO.puts("It works")
end
```

defmacrop(call, expr \\ nil)

(macro)

Defines a private macro with the given name and body.

Private macros are only accessible from the same module in which they are defined.

Private macros must be defined before its usage.

Check `defmacro/2` for more information, and check `def/2` for rules on naming and default arguments.

defmodule(alias, do_block)

(macro)

Defines a module given by name with the given contents.

This macro defines a module with the given `alias` as its name and with the given contents. It returns a tuple with four elements:

- `:module`

- the module name
- the binary contents of the module
- the result of evaluating the contents block

Examples

```
defmodule Number do
  def one, do: 1
  def two, do: 2
end
#=> {:module, Number, <<70, 79, 82, ...>>, {:two, 0}}

Number.one()
#=> 1

Number.two()
#=> 2
```

Module names and aliases

Module names (and aliases) must start with an ASCII uppercase character which may be followed by any ASCII letter, number, or underscore. Elixir's Naming Conventions suggest for module names and aliases to be written in the `CamelCase` format.

You can also use atoms as the module name, although they must only contain ASCII characters.

Nesting

Nesting a module inside another module affects the name of the nested module:

```elixir
defmodule Foo do
  defmodule Bar do
  end
end
```

In the example above, two modules - `Foo` and `Foo.Bar` - are created. When nesting, Elixir automatically creates an alias to the inner module, allowing the second module `Foo.Bar` to be accessed as `Bar` in the same lexical scope where it's defined (the `Foo` module). This only happens if the nested module is defined via an alias.

If the `Foo.Bar` module is moved somewhere else, the references to `Bar` in the `Foo` module need to be updated to the fully-qualified name (`Foo.Bar`) or an alias has to be explicitly set in the `Foo` module with the help of `alias/2`.

```elixir
defmodule Foo.Bar do
  # code
end

defmodule Foo do
  alias Foo.Bar
  # code here can refer to "Foo.Bar" as just "Bar"
end
```

Dynamic names

Elixir module names can be dynamically generated. This is very useful when working with macros. For instance, one could write:

```elixir
defmodule Module.concat(["Foo", "Bar"]) do
  # contents ...
```

```
end
```

Elixir will accept any module name as long as the expression passed as the first argument to `defmodule/2` evaluates to an atom. Note that, when a dynamic name is used, Elixir won't nest the name under the current module nor automatically set up an alias.

Reserved module names

If you attempt to define a module that already exists, you will get a warning saying that a module has been redefined.

There are some modules that Elixir does not currently implement but it may be implement in the future. Those modules are reserved and defining them will result in a compilation error:

```
defmodule Any do
  # code
end
** (CompileError) iex:1: module Any is reserved and
cannot be defined
```

Elixir reserves the following module names: `Elixir`, `Any`, `BitString`, `PID`, and `Reference`.

defoverridable(keywords_or_behaviour)

(macro)

Makes the given definitions in the current module overridable.

If the user defines a new function or macro with the same name and arity, then the overridable ones are discarded. Otherwise, the original

definitions are used.

It is possible for the overridden definition to have a different visibility than the original: a public function can be overridden by a private function and vice-versa. Macros cannot be overridden as functions and vice-versa.

Example

```
defmodule DefaultMod do
  defmacro __using__(_opts) do
    quote do
      def test(x, y) do
        x + y
      end

      defoverridable test: 2
    end
  end
end

defmodule ChildMod do
  use DefaultMod

  def test(x, y) do
    x * y + super(x, y)
  end
end
```

As seen as in the example above, `super` can be used to call the default implementation.

Example with behaviour

`defoverridable` is commonly used with behaviours. The behaviours use `@callback` definitions to define the general module API and the `__using__` callback is used to define default implementations of functions, which can then be overridable.

For convenience, you can pass a behaviour to `defoverridable` and it will mark all of the callbacks in the behaviour as overridable:

```
defmodule Behaviour do
  @callback test(number(), number()) :: number()
end

defmodule DefaultMod do
  defmacro __using__(_opts) do
    quote do
      @behaviour Behaviour

      def test(x, y) do
        x + y
      end

      defoverridable Behaviour
    end
  end
end

defmodule ChildMod do
  use DefaultMod

  def test(x, y) do
    x * y + super(x, y)
  end
end
```

Narrow behaviours and entry points

When defining behaviours, a general rule of thumb is to define narrow behaviours, with the minimum amount of callbacks, to facilitate maintenance over time. Fewer callbacks minimize the points of contact between different parts of the system and reduces the risk of breaking changes and of different implementations having inconsistent behaviour. However, when using `defoverridable` with behaviours, you may accidentally define broad interfaces as all default behaviour is provided via `defoverridable`. Furthermore, `defoverridable` necessarily relies on meta-programming, which complicates debugging. `super` is also hard to troubleshoot, as it by definition relies on calling an implicitly defined function.

A possible alternative to `defoverridable` is to use optional callbacks and move the default implementation to the caller. Then you can check if a callback exists via `Code.ensure_loaded?/1` and `function_exported?/3`. For instance, in the example above, imagine there is a module that calls the `test/2` function. This module could be defined as such:

```
defmodule CallsTest do
  def receives_module_and_calls_test(module, x, y) do
    if Code.ensure_loaded?(module) and function_exported?(module, :test,
2) do
      module.test(x, y)
    else
      x + y
    end
  end
end
```

The downside of the above code is that it must call `Code.ensure_loaded?/1` and `function_exported?/3` on every invocation of the behaviour, which may impact runtime performance. For this reason, this approach works best when the behaviour has an entry point, such as a `init` callback (as seen in `GenServer`), which you invoke once to guarantee the module is loaded, and from that moment, you only need to perform `function_exported?/3` checks.

To recap:

> - Prefer narrow behaviours
> - If your behaviour has an entry point, consider using optional callbacks followed by `Code.ensure_loaded?/1` and `function_exported?/3` checks
> - If using `defoverridable`, avoid relying on `super` to trigger the default behaviour, suggesting users to invoke well-defined APIs instead.

defp(call, expr \\ nil)

(macro)

Defines a private function with the given name and body.

Private functions are only accessible from within the module in which they are defined. Trying to access a private function from outside the module it's defined in results in an `UndefinedFunctionError` exception.

Check `def/2` for more information.

Examples

```
defmodule Foo do
  def bar do
    sum(1, 2)
  end

  defp sum(a, b), do: a + b
end

Foo.bar()
#=> 3
```

```
Foo.sum(1, 2)
** (UndefinedFunctionError) undefined function Foo.sum/2
```

defprotocol(name, do_block)

(macro)

Defines a protocol.

See the `Protocol` module for more information.

defstruct(fields)

(macro)

Defines a struct.

A struct is a tagged map that allows developers to provide default values for keys, tags to be used in polymorphic dispatches and compile time assertions.

It is only possible to define a struct per module, as the struct is tied to the module itself.

Examples

```
defmodule User do
  defstruct name: nil, age: nil
end
```

Struct fields are evaluated at compile-time, which allows them to be dynamic. In the example below, `10 + 11` is evaluated at compile-time

and the age field is stored with value `21`:

```
defmodule User do
  defstruct name: nil, age: 10 + 11
end
```

The `fields` argument is usually a keyword list with field names as atom keys and default values as corresponding values. `defstruct/1` also supports a list of atoms as its argument: in that case, the atoms in the list will be used as the struct's field names and they will all default to `nil`.

```
defmodule Post do
  defstruct [:title, :content, :author]
end
```

Add documentation to a struct with the `@doc` attribute, like a function.

```
defmodule Post do
  @doc "A post. The content should be valid Markdown."
  defstruct [:title, :content, :author]
end
```

Once a struct is defined, it is possible to create them as follows:

```
%Post{title: "Hello world!"}
```

For more information on creating, updating, and pattern matching on structs, please check `%/2`.

Deriving

Although structs are maps, by default structs do not implement any of the protocols implemented for maps. For example, attempting to use a protocol with the `User` struct leads to an error:

```
john = %User{name: "John"}
MyProtocol.call(john)
** (Protocol.UndefinedError) protocol MyProtocol not
implemented for User (a struct)
```

`defstruct/1`, however, allows protocol implementations to be *derived*. This can be done by defining a `@derive` attribute as a list before invoking `defstruct/1`:

```
defmodule User do
  @derive MyProtocol
  defstruct name: nil, age: nil
end

MyProtocol.call(john) # it works!
```

A common example is to `@derive` the `Inspect` protocol to hide certain fields when the struct is printed:

```
defmodule User do
  @derive {Inspect, only: :name}
  defstruct name: nil, age: nil
end
```

For each protocol in `@derive`, Elixir will verify if the protocol has implemented the `Protocol.__deriving__/2` callback. If so, the callback will be invoked and it should define the implementation module. Otherwise an implementation that simply points to the `Any`

implementation is automatically derived. For more information, see `Protocol.derive/3`.

Enforcing keys

When building a struct, Elixir will automatically guarantee all keys belong to the struct:

```
%User{name: "john", unknown: :key}
** (KeyError) key :unknown not found in: %User{age: 21,
name: nil}
```

Elixir also allows developers to enforce that certain keys must always be given when building the struct:

```
defmodule User do
  @enforce_keys [:name]
  defstruct name: nil, age: 10 + 11
end
```

Now trying to build a struct without the name key will fail:

```
%User{age: 21}
** (ArgumentError) the following keys must also be given
when building struct User: [:name]
```

Keep in mind `@enforce_keys` is a simple compile-time guarantee to aid developers when building structs. It is not enforced on updates and it does not provide any sort of value-validation.

Types

It is recommended to define types for structs. By convention, such a type is called `t`. To define a type for a struct, the struct literal syntax is used:

```
defmodule User do
  defstruct name: "John", age: 25
  @type t :: %__MODULE__{name: String.t(), age:
non_neg_integer}
end
```

It is recommended to only use the struct syntax when defining the struct's type. When referring to another struct, it's better to use `User.t()` instead of `%User{}`.

The types of the struct fields that are not included in `%User{}` default to `term()` (see `term/0`).

Structs whose internal structure is private to the local module (pattern matching them or directly accessing their fields should not be allowed) should use the `@opaque` attribute. Structs whose internal structure is public should use `@type`.

destructure(left, right)

(macro)

Destructures two lists, assigning each term in the right one to the matching term in the left one.

Unlike pattern matching via `=`, if the sizes of the left and right lists don't match, destructuring simply stops instead of raising an error.

Examples

```
iex> destructure([x, y, z], [1, 2, 3, 4, 5])
iex> {x, y, z}
{1, 2, 3}
```

In the example above, even though the right list has more entries than the left one, destructuring works fine. If the right list is smaller, the remaining elements are simply set to `nil`:

```
iex> destructure([x, y, z], [1])
iex> {x, y, z}
{1, nil, nil}
```

The left-hand side supports any expression you would use on the left-hand side of a match:

```
iex> x = 1
iex> destructure([^x, y, z], [1, 2, 3])
iex> {x, y, z}
{1, 2, 3}
```

The example above will only work if `x` matches the first value in the right list. Otherwise, it will raise a `MatchError` (like the `=` operator would do).

exit(reason)

```
@spec exit(term()) :: no return()
```

Stops the execution of the calling process with the given reason.

Since evaluating this function causes the process to terminate, it has no return value.

Inlined by the compiler.

Examples

When a process reaches its end, by default it exits with reason `:normal`. You can also call `exit/1` explicitly if you want to terminate a process but not signal any failure:

```
exit(:normal)
```

In case something goes wrong, you can also use `exit/1` with a different reason:

```
exit(:seems_bad)
```

If the exit reason is not `:normal`, all the processes linked to the process that exited will crash (unless they are trapping exits).

OTP exits

Exits are used by the OTP to determine if a process exited abnormally or not. The following exits are considered "normal":

- `exit(:normal)`
- `exit(:shutdown)`
- `exit({:shutdown, term})`

Exiting with any other reason is considered abnormal and treated as a crash. This means the default supervisor behavior kicks in, error reports

are emitted, and so forth.

This behavior is relied on in many different places. For example, `ExUnit` uses `exit(:shutdown)` when exiting the test process to signal linked processes, supervision trees and so on to politely shut down too.

CLI exits

Building on top of the exit signals mentioned above, if the process started by the command line exits with any of the three reasons above, its exit is considered normal and the Operating System process will exit with status 0.

It is, however, possible to customize the operating system exit signal by invoking:

```
exit({:shutdown, integer})
```

This will cause the operating system process to exit with the status given by `integer` while signaling all linked Erlang processes to politely shut down.

Any other exit reason will cause the operating system process to exit with status `1` and linked Erlang processes to crash.

function_exported?(module, function, arity)

```
@spec function_exported?(module(), atom(), arity()) ::
boolean()
```

Returns `true` if `module` is loaded and contains a public `function` with the given `arity`, otherwise `false`.

> ### Unloaded modules
>
> This function does *not* load the module in case it is not loaded and Elixir lazily loads modules by default (except on releases).
>
> This may lead to unexpected behaviour as the result of this function may depend if another code has happened to load the given module as argument beforehand. For example, this could manifest in `mix test` by having tests that fail when running in isolation or depending on the test seed. For those reasons, it is recommended to always check for `Code.ensure_loaded?/1` before `function_exported?/3`, unless you are certain the module has been loaded before.
>
> See `Code.ensure_loaded/1` for more information.

Inlined by the compiler.

Examples

```
iex> Code.ensure_loaded?(Enum) and function_exported?
(Enum, :map, 2)
true

iex> Code.ensure_loaded?(Enum) and function_exported?
(Enum, :map, 10)
false

iex> Code.ensure_loaded?(List) and function_exported?
(List, :to_string, 1)
true
```

get_and_update_in(path, fun)

(macro)

Gets a value and updates a nested data structure via the given `path`.

This is similar to `get_and_update_in/3`, except the path is extracted via a macro rather than passing a list. For example:

```
get_and_update_in(opts[:foo][:bar], &{&1, &1 + 1})
```

Is equivalent to:

```
get_and_update_in(opts, [:foo, :bar], &{&1, &1 + 1})
```

This also works with nested structs and the `struct.path.to.value` way to specify paths:

```
get_and_update_in(struct.foo.bar, &{&1, &1 + 1})
```

Note that in order for this macro to work, the complete path must always be visible by this macro. See the "Paths" section below.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_and_update_in(users["john"].age, &{&1, &1 + 1})
{27, %{"john" => %{age: 28}, "meg" => %{age: 23}}}
```

Paths

A path may start with a variable, local or remote call, and must be followed by one or more:

- `foo[bar]` - accesses the key `bar` in `foo`; in case `foo` is nil, `nil` is returned
- `foo.bar` - accesses a map/struct field; in case the field is not present, an error is raised

Here are some valid paths:

```
users["john"][:age]
users["john"].age
User.all()["john"].age
all_users()["john"].age
```

Here are some invalid ones:

```
# Does a remote call after the initial value
users["john"].do_something(arg1, arg2)

# Does not access any key or field
users
```

get_and_update_in(data, keys, fun)

```
@spec get_and_update_in(
  structure,
  keys,
  (term() | nil -> {current_value, new_value} | :pop)
) :: {current_value, new_structure :: structure}
when structure: Access.t(),
     keys: [term(), ...],
     current_value: Access.value(),
     new_value: Access.value()
```

Gets a value and updates a nested structure.

`data` is a nested structure (that is, a map, keyword list, or struct that implements the `Access` behaviour).

The `fun` argument receives the value of `key` (or `nil` if `key` is not present) and must return one of the following values:

- a two-element tuple `{current_value, new_value}`. In this case, `current_value` is the retrieved value which can possibly be operated on before being returned. `new_value` is the new value to be stored under `key`.

- `:pop`, which implies that the current value under `key` should be removed from the structure and returned.

This function uses the `Access` module to traverse the structures according to the given `keys`, unless the `key` is a function, which is detailed in a later section.

Examples

This function is useful when there is a need to retrieve the current value (or something calculated in function of the current value) and update it at the same time. For example, it could be used to read the current age of a user while increasing it by one in one pass:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_and_update_in(users, ["john", :age], &{&1, &1 +
1})
{27, %{"john" => %{age: 28}, "meg" => %{age: 23}}}
```

Note the current value given to the anonymous function may be `nil`. If any of the intermediate values are nil, it will raise:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_and_update_in(users, ["jane", :age], &{&1, &1 + 1})
** (ArgumentError) could not put/update key :age on a nil value
```

Functions as keys

If a key is a function, the function will be invoked passing three arguments:

- the operation (`:get_and_update`)
- the data to be accessed
- a function to be invoked next

This means `get_and_update_in/3` can be extended to provide custom lookups. The downside is that functions cannot be stored as keys in the accessed data structures.

When one of the keys is a function, the function is invoked. In the example below, we use a function to get and increment all ages inside a list:

```
iex> users = [%{name: "john", age: 27}, %{name: "meg", age: 23}]
iex> all = fn :get_and_update, data, next ->
...>   data |> Enum.map(next) |> Enum.unzip()
...> end
iex> get_and_update_in(users, [all, :age], &{&1, &1 + 1})
{[27, 23], [%{name: "john", age: 28}, %{name: "meg", age: 24}]}
```

If the previous value before invoking the function is `nil`, the function *will* receive `nil` as a value and must handle it accordingly (be it by failing or providing a sane default).

The `Access` module ships with many convenience accessor functions, like the `all` anonymous function defined above. See `Access.all/0`, `Access.key/2`, and others as examples.

get_in(path)

(since 1.17.0) (macro)

Gets a key from the nested structure via the given `path`, with nil-safe handling.

This is similar to `get_in/2`, except the path is extracted via a macro rather than passing a list. For example:

```
get_in(opts[:foo][:bar])
```

Is equivalent to:

```
get_in(opts, [:foo, :bar])
```

Additionally, this macro can traverse structs:

```
get_in(struct.foo.bar)
```

In case any of the keys returns `nil`, then `nil` will be returned and `get_in/1` won't traverse any further.

Note that in order for this macro to work, the complete path must always be visible by this macro. For more information about the supported path expressions, please check `get_and_update_in/2` docs.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_in(users["john"].age)
27
iex> get_in(users["unknown"].age)
nil
```

get_in(data, keys)

```
@spec get_in(Access.t(), [term(), ...]) :: term()
```

Gets a value from a nested structure with nil-safe handling.

Uses the `Access` module to traverse the structures according to the given `keys`, unless the `key` is a function, which is detailed in a later section.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_in(users, ["john", :age])
27
iex> # Equivalent to:
iex> users["john"][:age]
27
```

`get_in/2` can also use the accessors in the `Access` module to traverse more complex data structures. For example, here we use `Access.all/0` to traverse a list:

```
iex> users = [%{name: "john", age: 27}, %{name: "meg",
age: 23}]
iex> get_in(users, [Access.all(), :age])
[27, 23]
```

In case any of the components returns `nil`, `nil` will be returned and `get_in/2` won't traverse any further:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_in(users, ["unknown", :age])
nil
iex> # Equivalent to:
iex> users["unknown"][:age]
nil
```

Functions as keys

If a key given to `get_in/2` is a function, the function will be invoked passing three arguments:

- the operation (`:get`)
- the data to be accessed
- a function to be invoked next

This means `get_in/2` can be extended to provide custom lookups. That's precisely how the `Access.all/0` key in the previous section behaves. For example, we can manually implement such traversal as follows:

```
iex> users = [%{name: "john", age: 27}, %{name: "meg",
age: 23}]
iex> all = fn :get, data, next -> Enum.map(data, next)
end
iex> get_in(users, [all, :age])
[27, 23]
```

The `Access` module ships with many convenience accessor functions. See `Access.all/0`, `Access.key/2`, and others as examples.

Working with structs

By default, structs do not implement the `Access` behaviour required by this function. Therefore, you can't do this:

```
get_in(some_struct, [:some_key, :nested_key])
```

There are two alternatives. Given structs have predefined keys, we can use the `struct.field` notation:

```
some_struct.some_key.nested_key
```

However, the code above will fail if any of the values return `nil`. If you also want to handle nil values, you can use `get_in/1`:

```
get_in(some_struct.some_key.nested_key)
```

Pattern-matching is another option for handling such cases, which can be especially useful if you want to match on several fields at once or provide custom return values:

```
case some_struct do
  %{some_key: %{nested_key: value}} -> value
  %{} -> nil
end
```

if(condition, clauses)

(macro)

Provides an `if/2` macro.

This macro expects the first argument to be a condition and the second argument to be a keyword list. Generally speaking, Elixir developers prefer to use pattern matching and guards in function definitions and `case/2`, as they are succinct and precise. However, not all conditions can be expressed through patterns and guards, which makes `if/2` a viable alternative.

Similar to `case/2`, any assignment in the condition will be available on both clauses, as well as after the `if` expression.

One-liner examples

```
iex> if 7 > 5, do: "yes"
"yes"

iex> if "truthy value", do: "yes"
"yes"
```

In the examples above, the `do` clause is evaluated and `"yes"` will be returned because the condition evaluates to a truthy value (neither `false`

nor `nil`). Otherwise, the clause is not evaluated and `nil` will be returned:

```
iex> if 3 > 5, do: "yes"
nil

iex> if nil, do: IO.puts("this won't be printed")
nil
```

An `else` option can be given to specify the opposite:

```
iex> if 3 > 5, do: "yes", else: "no"
"no"
```

Blocks examples

It's also possible to pass a block to the `if/2` macro. The first example above would be translated to:

```
iex> if 7 > 5 do
...>   "yes"
...> end
"yes"
```

Note that `do-end` become delimiters. The third example would translate to:

```
iex> if 3 > 5 do
...>   "yes"
...> else
...>   "no"
```

```
...> end
"no"
```

If you find yourself nesting conditionals inside conditionals, consider using `cond/1`.

Variables scope

Variables set within `do`/`else` blocks do not leak to the outer context:

```
x = 1
if x > 0 do
  x = x + 1
  IO.puts(x)  # prints 2
end
x  # 1
```

Variables set in the condition are available in the outer context:

```
fruits = %{oranges: 3}
if count = fruits[:apples] do
  # won't be evaluated
  IO.puts(count + 1)
end
count  # nil
```

inspect(term, opts \\ [])

```
@spec inspect(Inspect.t(), [Inspect.Opts.new opt()]) ::
String.t()
```

Inspects the given argument according to the `Inspect` protocol. The second argument is a keyword list with options to control inspection.

Options

`inspect/2` accepts a list of options that are internally translated to an `Inspect.Opts` struct. Check the docs for `Inspect.Opts` to see the supported options.

Examples

```
iex> inspect(:foo)
":foo"

iex> inspect([1, 2, 3, 4, 5], limit: 3)
"[1, 2, 3, ...]"

iex> inspect([1, 2, 3], pretty: true, width: 0)
"[1,\n 2,\n 3]"

iex> inspect("olá" <> <<0>>)
"<<111, 108, 195, 161, 0>>"

iex> inspect("olá" <> <<0>>, binaries: :as_strings)
"\"olá\\0\""

iex> inspect("olá", binaries: :as_binaries)
"<<111, 108, 195, 161>>"

iex> inspect(~c"bar")
"~c\"bar\""

iex> inspect([0 | ~c"bar"])
"[0, 98, 97, 114]"
```

```
iex> inspect(100, base: :octal)
"0o144"

iex> inspect(100, base: :hex)
"0x64"
```

Note that the `Inspect` protocol does not necessarily return a valid representation of an Elixir term. In such cases, the inspected result must start with `#`. For example, inspecting a function will return:

```
inspect(fn a, b -> a + b end)
#=> #Function<...>
```

The `Inspect` protocol can be derived to hide certain fields from structs, so they don't show up in logs, inspects and similar. See the "Deriving" section of the documentation of the `Inspect` protocol for more information.

macro_exported?(module, macro, arity)

```
@spec macro_exported?(module(), atom(), arity()) :: boolean()
```

Returns `true` if `module` is loaded and contains a public `macro` with the given `arity`, otherwise `false`.

Note that this function does not load the module in case it is not loaded. See the notes under `function_exported?/3` for more information.

If `module` is an Erlang module (as opposed to an Elixir module), this function always returns `false`.

Examples

```
iex> Code.ensure_loaded?(Kernel) and macro_exported?
(Kernel, :use, 2)
true

iex> Code.ensure_loaded?(:erlang) and macro_exported?
(:erlang, :abs, 1)
false
```

make_ref()

```
@spec make_ref() :: reference()
```

Returns an almost unique reference.

The returned reference will re-occur after approximately 2^82 calls; therefore it is unique enough for practical purposes.

Inlined by the compiler.

Examples

```
make_ref()
#=> #Reference<0.0.0.135>
```

match?(pattern, expression)

(macro)

A convenience macro that checks if the result of `expression` matches `pattern`.

Examples

```
iex> match?(1, 1)
true

iex> match?({1, _}, {1, 2})
true

iex> map = %{a: 1, b: 2}
iex> match?(%{a: _}, map)
true

iex> a = 1
iex> match?(^a, 1)
true
```

`match?/2` is very useful when filtering or finding a value in an enumerable:

```
iex> list = [a: 1, b: 2, a: 3]
iex> Enum.filter(list, &match?({:a, _}, &1))
[a: 1, a: 3]
```

Guard clauses can also be given to the match:

```
iex> list = [a: 1, b: 2, a: 3]
iex> Enum.filter(list, &match?({:a, x} when x < 2, &1))
[a: 1]
```

Variables assigned in the match will not be available outside of the function call (unlike regular pattern matching with the `=` operator):

```
iex> match?(_x, 1)
true
iex> binding()
[]
```

Values vs patterns

Remember the pin operator matches *values*, not *patterns*. Passing a variable as the pattern will always return `true` and will result in a warning that the variable is unused. Don't do this:

```
pattern = %{a: :a}
match?(pattern, %{b: :b})
#=> true
```

Similarly, moving an expression out the pattern may no longer preserve its semantics. For example:

```
iex> match?([_ | _], [1, 2, 3])
true

pattern = [_ | _]
match?(pattern, [1, 2, 3])
** (CompileError) invalid use of _. _ can only be used inside patterns to ignore values and cannot be used in expressions. Make sure you are inside a pattern or change it accordingly
```

Another example is that a map as a pattern performs a subset match, but not once assigned to a variable:

```
iex> match?(%{x: 1}, %{x: 1, y: 2})
true

iex> attrs = %{x: 1}
iex> match?(^attrs, %{x: 1, y: 2})
false
```

The pin operator will check if the values are equal, using `===/2`, while patterns have their own rules when matching maps, lists, and so forth. Such behavior is not specific to `match?/2`. The following code also throws an exception:

```
attrs = %{x: 1}
^attrs = %{x: 1, y: 2}
#=> (MatchError) no match of right hand side value: %{x:
1, y: 2}
```

pop_in(path)

(macro)

Pops a key from the nested structure via the given `path`.

This is similar to `pop_in/2`, except the path is extracted via a macro rather than passing a list. For example:

```
pop_in(opts[:foo][:bar])
```

Is equivalent to:

```
pop_in(opts, [:foo, :bar])
```

Note that in order for this macro to work, the complete path must always be visible by this macro. For more information about the supported path expressions, please check `get_and_update_in/2` docs.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> pop_in(users["john"][:age])
{27, %{"john" => %{}, "meg" => %{age: 23}}}

iex> users = %{john: %{age: 27}, meg: %{age: 23}}
iex> pop_in(users.john[:age])
{27, %{john: %{}, meg: %{age: 23}}}
```

In case any entry returns `nil`, its key will be removed and the deletion will be considered a success.

pop_in(data, keys)

```
@spec pop_in(data, [Access.get_and_update_fun(term(), data) |
term(), ...]) ::
  {term(), data}
when data: Access.container()
```

Pops a key from the given nested structure.

Uses the `Access` protocol to traverse the structures according to the given `keys`, unless the `key` is a function. If the key is a function, it will be invoked as specified in `get_and_update_in/3`.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> pop_in(users, ["john", :age])
{27, %{"john" => %{}, "meg" => %{age: 23}}}
```

In case any entry returns `nil`, its key will be removed and the deletion will be considered a success.

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> pop_in(users, ["jane", :age])
{nil, %{"john" => %{age: 27}, "meg" => %{age: 23}}}
```

put_elem(tuple, index, value)

```
@spec put_elem(tuple(), non_neg_integer(), term()) :: tuple()
```

Puts `value` at the given zero-based `index` in `tuple`.

Inlined by the compiler.

Examples

```
iex> tuple = {:foo, :bar, 3}
iex> put_elem(tuple, 0, :baz)
{:baz, :bar, 3}
```

put_in(path, value)

(macro)

Puts a value in a nested structure via the given `path`.

This is similar to `put_in/3`, except the path is extracted via a macro rather than passing a list. For example:

```
put_in(opts[:foo][:bar], :baz)
```

Is equivalent to:

```
put_in(opts, [:foo, :bar], :baz)
```

This also works with nested structs and the `struct.path.to.value` way to specify paths:

```
put_in(struct.foo.bar, :baz)
```

Note that in order for this macro to work, the complete path must always be visible by this macro. For more information about the supported path expressions, please check `get_and_update_in/2` docs.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> put_in(users["john"][:age], 28)
%{"john" => %{age: 28}, "meg" => %{age: 23}}

iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
```

```
iex> put_in(users["john"].age, 28)
%{"john" => %{age: 28}, "meg" => %{age: 23}}
```

put_in(data, keys, value)

```
@spec put_in(Access.t(), [term(), ...], term()) :: Access.t()
```

Puts a value in a nested structure.

Uses the `Access` module to traverse the structures according to the given `keys`, unless the `key` is a function. If the key is a function, it will be invoked as specified in `get_and_update_in/3`.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> put_in(users, ["john", :age], 28)
%{"john" => %{age: 28}, "meg" => %{age: 23}}
```

If any of the intermediate values are nil, it will raise:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> put_in(users, ["jane", :age], "oops")
** (ArgumentError) could not put/update key :age on a nil
value
```

raise(message)

(macro)

Raises an exception.

If `message` is a string, it raises a `RuntimeError` exception with it.

If `message` is an atom, it just calls `raise/2` with the atom as the first argument and `[]` as the second one.

If `message` is an exception struct, it is raised as is.

If `message` is anything else, `raise` will fail with an `ArgumentError` exception.

Examples

```
iex> raise "oops"
** (RuntimeError) oops

try do
  1 + :foo
rescue
  x in [ArithmeticError] ->
    IO.puts("that was expected")
    raise x
end
```

raise(exception, attributes)

(macro)

Raises an exception.

Calls the `exception/1` function on the given argument (which has to be a module name like `ArgumentError` or `RuntimeError`) passing `attributes` in order to retrieve the exception struct.

Any module that contains a call to the `defexception/1` macro automatically implements the `Exception.exception/1` callback expected by `raise/2`. For more information, see `defexception/1`.

Examples

```
iex> raise(ArgumentError, "Sample")
** (ArgumentError) Sample
```

reraise(message, stacktrace)

(macro)

Raises an exception preserving a previous stacktrace.

Works like `raise/1` but does not generate a new stacktrace.

Note that `__STACKTRACE__` can be used inside catch/rescue to retrieve the current stacktrace.

Examples

```
iex> try do
...>  raise "oops"
...> rescue
...>  exception ->
...>    reraise exception, __STACKTRACE__
...> end
** (RuntimeError) oops
```

reraise(exception, attributes, stacktrace)

(macro)

Raises an exception preserving a previous stacktrace.

`reraise/3` works like `reraise/2`, except it passes arguments to the `exception/1` function as explained in `raise/2`.

Examples

```
try do
  raise "oops"
rescue
  exception ->
    reraise WrapperError, [exception: exception],
__STACKTRACE__
end
```

send(dest, message)

```
@spec send(dest :: Process.dest(), message) :: message when
message: any()
```

Sends a message to the given `dest` and returns the message.

`dest` may be a remote or local PID, a local port, a locally registered name, or a tuple in the form of `{registered_name, node}` for a registered name at another node.

For additional documentation, see the `!` operator Erlang documentation.

Inlined by the compiler.

Examples

```
iex> send(self(), :hello)
:hello
```

sigil_C(term, modifiers)

(macro)

Handles the sigil `~c` for charlists.

It returns a charlist without interpolations and without escape characters.

A charlist is a list of integers where all the integers are valid code points. The three expressions below are equivalent:

```
~C"foo\n"
[?f, ?o, ?o, ?\\, ?n]
[102, 111, 111, 92, 110]
```

In practice, charlists are mostly used in specific scenarios such as interfacing with older Erlang libraries that do not accept binaries as arguments.

Examples

```
iex> ~C(foo)
~c"foo"

iex> ~C(f#{o}o)
```

```
~c"f\#{o}o"

iex> ~C(foo\n)
~c"foo\\n"
```

sigil_c(term, modifiers)

(macro)

Handles the sigil `~c` for charlists.

It returns a charlist, unescaping characters and replacing interpolations.

A charlist is a list of integers where all the integers are valid code points. The three expressions below are equivalent:

```
~c"foo"
[?f, ?o, ?o]
[102, 111, 111]
```

In practice, charlists are mostly used in specific scenarios such as interfacing with older Erlang libraries that do not accept binaries as arguments.

Examples

```
iex> ~c(foo)
~c"foo"

iex> ~c(f#{:o}o)
~c"foo"
```

```
iex> ~c(f\#{:o}o)
~c"f\#{:o}o"
```

The list is only printed as a `~c` sigil if all code points are within the ASCII range:

```
iex> ~c"hełło"
[104, 101, 322, 322, 111]

iex> [104, 101, 108, 108, 111]
~c"hello"
```

See `Inspect.Opts` for more information.

sigil_D(date_string, modifiers)

(macro)

Handles the sigil `~D` for dates.

By default, this sigil uses the built-in `Calendar.ISO`, which requires dates to be written in the ISO8601 format:

```
~D[yyyy-mm-dd]
```

such as:

```
~D[2015-01-13]
```

If you are using alternative calendars, any representation can be used as long as you follow the representation by a single space and the calendar

name:

```
~D[SOME-REPRESENTATION My.Alternative.Calendar]
```

The lower case `~d` variant does not exist as interpolation and escape characters are not useful for date sigils.

More information on dates can be found in the `Date` module.

Examples

```
iex> ~D[2015-01-13]
~D[2015-01-13]
```

sigil_N(naive_datetime_string, modifiers)

(macro)

Handles the sigil `~N` for naive date times.

By default, this sigil uses the built-in `Calendar.ISO`, which requires naive date times to be written in the ISO8601 format:

```
~N[yyyy-mm-dd hh:mm:ss]
~N[yyyy-mm-dd hh:mm:ss.ssssss]
~N[yyyy-mm-ddThh:mm:ss.ssssss]
```

such as:

```
~N[2015-01-13 13:00:07]
```

```
~N[2015-01-13T13:00:07.123]
```

If you are using alternative calendars, any representation can be used as long as you follow the representation by a single space and the calendar name:

```
~N[SOME-REPRESENTATION My.Alternative.Calendar]
```

The lower case `~n` variant does not exist as interpolation and escape characters are not useful for date time sigils.

More information on naive date times can be found in the `NaiveDateTime` module.

Examples

```
iex> ~N[2015-01-13 13:00:07]
~N[2015-01-13 13:00:07]
iex> ~N[2015-01-13T13:00:07.001]
~N[2015-01-13 13:00:07.001]
```

sigil_r(term, modifiers)

(macro)

Handles the sigil `~r` for regular expressions.

It returns a regular expression pattern, unescaping characters and replacing interpolations.

More information on regular expressions can be found in the `Regex` module.

Examples

```
iex> Regex.match?(~r/foo/, "foo")
true

iex> Regex.match?(~r/a#{:b}c/, "abc")
true
```

While the `~r` sigil allows parens and brackets to be used as delimiters, it is preferred to use `"` or `/` to avoid escaping conflicts with reserved regex characters.

sigil_S(term, modifiers)

(macro)

Handles the sigil `~S` for strings.

It returns a string without interpolations and without escape characters.

Examples

```
iex> ~S(foo)
"foo"
iex> ~S(f#{o}o)
"f\#{o}o"
iex> ~S(\o/)
"\\o/"
```

sigil_s(term, modifiers)

(macro)

Handles the sigil `~s` for strings.

It returns a string as if it was a double quoted string, unescaping characters and replacing interpolations.

Examples

```
iex> ~s(foo)
"foo"

iex> ~s(f#{:o}o)
"foo"

iex> ~s(f\#{:o}o)
"f\#{:o}o"
```

sigil_T(time_string, modifiers)

(macro)

Handles the sigil `~T` for times.

By default, this sigil uses the built-in `Calendar.ISO`, which requires times to be written in the ISO8601 format:

```
~T[hh:mm:ss]
~T[hh:mm:ss.ssssss]
```

such as:

```
~T[13:00:07]
~T[13:00:07.123]
```

If you are using alternative calendars, any representation can be used as long as you follow the representation by a single space and the calendar name:

```
~T[SOME-REPRESENTATION My.Alternative.Calendar]
```

The lower case `~t` variant does not exist as interpolation and escape characters are not useful for time sigils.

More information on times can be found in the `Time` module.

Examples

```
iex> ~T[13:00:07]
~T[13:00:07]
iex> ~T[13:00:07.001]
~T[13:00:07.001]
```

sigil_U(datetime_string, modifiers)

(since 1.9.0) (macro)

Handles the sigil `~U` to create a UTC `DateTime`.

By default, this sigil uses the built-in `Calendar.ISO`, which requires UTC date times to be written in the ISO8601 format:

```
~U[yyyy-mm-dd hh:mm:ssZ]
~U[yyyy-mm-dd hh:mm:ss.ssssssZ]
~U[yyyy-mm-ddThh:mm:ss.ssssss+00:00]
```

such as:

```
~U[2015-01-13 13:00:07Z]
~U[2015-01-13T13:00:07.123+00:00]
```

If you are using alternative calendars, any representation can be used as long as you follow the representation by a single space and the calendar name:

```
~U[SOME-REPRESENTATION My.Alternative.Calendar]
```

The given `datetime_string` must include "Z" or "00:00" offset which marks it as UTC, otherwise an error is raised.

The lower case `~u` variant does not exist as interpolation and escape characters are not useful for date time sigils.

More information on date times can be found in the `DateTime` module.

Examples

```
iex> ~U[2015-01-13 13:00:07Z]
~U[2015-01-13 13:00:07Z]
iex> ~U[2015-01-13T13:00:07.001+00:00]
~U[2015-01-13 13:00:07.001Z]
```

sigil_W(term, modifiers)

(macro)

Handles the sigil `~W` for list of words.

It returns a list of "words" split by whitespace without interpolations and without escape characters.

Modifiers

- `s` : words in the list are strings (default)
- `a` : words in the list are atoms
- `c` : words in the list are charlists

Examples

```
iex> ~W(foo #{bar} baz)
["foo", "\#{bar}", "baz"]
```

sigil_w(term, modifiers)

(macro)

Handles the sigil `~w` for list of words.

It returns a list of "words" split by whitespace. Character unescaping and interpolation happens for each word.

Modifiers

- `s` : words in the list are strings (default)
- `a` : words in the list are atoms
- `c` : words in the list are charlists

Examples

```
iex> ~w(foo #{:bar} baz)
["foo", "bar", "baz"]

iex> ~w(foo #{" bar baz "})
["foo", "bar", "baz"]

iex> ~w(--source test/enum_test.exs)
["--source", "test/enum_test.exs"]

iex> ~w(foo bar baz)a
[:foo, :bar, :baz]

iex> ~w(foo bar baz)c
[~c"foo", ~c"bar", ~c"baz"]
```

spawn(fun)

```
@spec spawn((-> any())) :: pid()
```

Spawns the given function and returns its PID.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions.

The anonymous function receives 0 arguments, and may return any value.

Inlined by the compiler.

Examples

```
iex> current = self()
iex> child = spawn(fn -> send(current, {self(), 1 + 2})
end)
iex> receive do
...>   {^child, 3} -> :ok
...> end
:ok
```

spawn(module, fun, args)

```
@spec spawn(module(), atom(), list()) :: pid()
```

Spawns the given function `fun` from the given `module` passing it the given `args` and returns its PID.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions.

Inlined by the compiler.

Examples

```
spawn(SomeModule, :function, [1, 2, 3])
```

spawn_link(fun)

```
@spec spawn_link((-> any())) :: pid()
```

Spawns the given function, links it to the current process, and returns its PID.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions. For more information on linking, check `Process.link/1`.

The anonymous function receives 0 arguments, and may return any value.

Inlined by the compiler.

Examples

```
iex> current = self()
iex> child = spawn_link(fn -> send(current, {self(), 1 +
2}) end)
iex> receive do
...>   {^child, 3} -> :ok
```

```
...> end
:ok
```

spawn_link(module, fun, args)

```
@spec spawn_link(module(), atom(), list()) :: pid()
```

Spawns the given function `fun` from the given `module` passing it the given `args`, links it to the current process, and returns its PID.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions. For more information on linking, check `Process.link/1`.

Inlined by the compiler.

Examples

```
spawn_link(SomeModule, :function, [1, 2, 3])
```

spawn_monitor(fun)

```
@spec spawn_monitor((-> any())) :: {pid(), reference()}
```

Spawns the given function, monitors it and returns its PID and monitoring reference.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions.

The anonymous function receives 0 arguments, and may return any value.

Inlined by the compiler.

Examples

```
iex> current = self()
iex> {child, _ref} = spawn_monitor(fn -> send(current,
{self(), 1 + 2}) end)
iex> receive do
...>   {^child, 3} -> :ok
...> end
:ok
```

spawn_monitor(module, fun, args)

```
@spec spawn_monitor(module(), atom(), list()) :: {pid(),
reference()}
```

Spawns the given module and function passing the given args, monitors it and returns its PID and monitoring reference.

Typically developers do not use the `spawn` functions, instead they use abstractions such as `Task`, `GenServer` and `Agent`, built on top of `spawn`, that spawns processes with more conveniences in terms of introspection and debugging.

Check the `Process` module for more process-related functions.

Inlined by the compiler.

Examples

```
spawn_monitor(SomeModule, :function, [1, 2, 3])
```

struct(struct, fields \\ [])

```
@spec struct(module() | struct(), Enumerable.t()) :: struct()
```

Creates and updates a struct.

The `struct` argument may be an atom (which defines `defstruct`) or a `struct` itself. The second argument is any `Enumerable` that emits two-element tuples (key-value pairs) during enumeration.

Keys in the `Enumerable` that don't exist in the struct are automatically discarded. Note that keys must be atoms, as only atoms are allowed when defining a struct. If there are duplicate keys in the `Enumerable`, the last entry will be taken (same behavior as `Map.new/1`).

This function is useful for dynamically creating and updating structs, as well as for converting maps to structs; in the latter case, just inserting the appropriate `:__struct__` field into the map may not be enough and `struct/2` should be used instead.

Examples

```
defmodule User do
  defstruct name: "john"
end

struct(User)
#=> %User{name: "john"}

opts = [name: "meg"]
user = struct(User, opts)
#=> %User{name: "meg"}

struct(user, unknown: "value")
#=> %User{name: "meg"}

struct(User, %{name: "meg"})
#=> %User{name: "meg"}

# String keys are ignored
struct(User, %{"name" => "meg"})
#=> %User{name: "john"}
```

struct!(struct, fields \\ [])

```
@spec struct!(module() | struct(), Enumerable.t()) ::
struct()
```

Similar to `struct/2` but checks for key validity.

The function `struct!/2` emulates the compile time behavior of structs. This means that:

- when building a struct, as in `struct!(SomeStruct, key: :value)`, it is equivalent to `%SomeStruct{key: :value}` and therefore this function will check if every given key-value belongs to the struct. If the struct is enforcing any key via `@enforce_keys`, those will be enforced as well;
- when updating a struct, as in `struct!(%SomeStruct{}, key: :value)`, it is equivalent to `%SomeStruct{struct | key: :value}` and therefore this function will check if every given key-value belongs to the struct.

tap(value, fun)

(since 1.12.0) (macro)

Pipes the first argument, `value`, into the second argument, a function `fun`, and returns `value` itself.

Useful for running synchronous side effects in a pipeline, using the `|>/2` operator.

Examples

```
iex> tap(1, fn x -> x + 1 end)
1
```

Most commonly, this is used in pipelines, using the `|>/2` operator. For example, let's suppose you want to inspect part of a data structure. You could write:

```
%{a: 1}
|> Map.update!(:a, & &1 + 2)
```

```
|> tap(&IO.inspect(&1.a))
|> Map.update!(:a, & &1 * 2)
```

then(value, fun)

(since 1.12.0) (macro)

Pipes the first argument, `value`, into the second argument, a function `fun`, and returns the result of calling `fun`.

In other words, it invokes the function `fun` with `value` as argument, and returns its result.

This is most commonly used in pipelines, using the `|>/2` operator, allowing you to pipe a value to a function outside of its first argument.

Examples

```
iex> 1 |> then(fn x -> x * 2 end)
2

iex> 1 |> then(fn x -> Enum.drop(["a", "b", "c"], x) end)
["b", "c"]
```

throw(term)

```
@spec throw(term()) :: no_return()
```

A non-local return from a function.

Using `throw/1` is generally discouraged, as it allows a function to escape from its regular execution flow, which can make the code harder to read. Furthermore, all thrown values must be caught by `try/catch`. See `try/1` for more information.

Inlined by the compiler.

to_charlist(term)

(macro)

Converts the given term to a charlist according to the `List.Chars` protocol.

Examples

```
iex> to_charlist(:foo)
~c"foo"
```

to_string(term)

(macro)

Converts the argument to a string according to the `String.Chars` protocol.

This is invoked when there is string interpolation.

Examples

```
iex> to_string(:foo)
"foo"
```

to_timeout(duration)

(since 1.17.0)

```
@spec to_timeout([{unit, non_neg_integer()}] | timeout() |
Duration.t()) :: timeout()
when unit: :week | :day | :hour | :minute | :second |
:millisecond
```

Constructs a millisecond timeout from the given components, duration, or timeout.

This function is useful for constructing timeouts to use in functions that expect `timeout/0` values (such as `Process.send_after/4` and many others).

Argument

The `duration` argument can be one of a `Duration`, a `timeout/0`, or a list of components. Each of these is described below.

Passing `Duration`s

`Duration.t/0` structs can be converted to timeouts. The given duration must have `year` and `month` fields set to `0`, since those cannot be reliably converted to milliseconds (due to the varying number of days in a month and year).

Microseconds in durations are converted to milliseconds (through `System.convert_time_unit/3`).

Passing components

The `duration` argument can also be keyword list which can contain the following keys, each appearing at most once with a non-negative integer value:

- `:week` - the number of weeks (a week is always 7 days)
- `:day` - the number of days (a day is always 24 hours)
- `:hour` - the number of hours
- `:minute` - the number of minutes
- `:second` - the number of seconds
- `:millisecond` - the number of milliseconds

The timeout is calculated as the sum of the components, each multiplied by the corresponding factor.

Passing timeouts

You can also pass timeouts directly to this functions, that is, milliseconds or the atom `:infinity`. In this case, this function just returns the given argument.

Examples

With a keyword list:

```
iex> to_timeout(hour: 1, minute: 30)
5400000
```

With a duration:

```
iex> to_timeout(%Duration{hour: 1, minute: 30})
```

```
5400000
```

With a timeout:

```
iex> to_timeout(5_400_000)
5400000
iex> to_timeout(:infinity)
:infinity
```

unless(condition, clauses)

(macro)

This macro is deprecated. Use if/2 instead.

Provides an `unless` macro.

This macro evaluates and returns the `do` block passed in as the second argument if `condition` evaluates to a falsy value (`false` or `nil`). Otherwise, it returns the value of the `else` block if present or `nil` if not.

See also `if/2`.

Examples

```
iex> unless(Enum.empty?([]), do: "Hello")
nil

iex> unless(Enum.empty?([1, 2, 3]), do: "Hello")
"Hello"
```

```
iex> unless Enum.sum([2, 2]) == 5 do
...>   "Math still works"
...> else
...>   "Math is broken"
...> end
"Math still works"
```

update_in(path, fun)

(macro)

Updates a nested structure via the given `path`.

This is similar to `update_in/3`, except the path is extracted via a macro rather than passing a list. For example:

```
update_in(opts[:foo][:bar], &(&1 + 1))
```

Is equivalent to:

```
update_in(opts, [:foo, :bar], &(&1 + 1))
```

This also works with nested structs and the `struct.path.to.value` way to specify paths:

```
update_in(struct.foo.bar, &(&1 + 1))
```

Note that in order for this macro to work, the complete path must always be visible by this macro. For more information about the supported path expressions, please check `get_and_update_in/2` docs.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> update_in(users["john"][:age], &(&1 + 1))
%{"john" => %{age: 28}, "meg" => %{age: 23}}

iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> update_in(users["john"].age, &(&1 + 1))
%{"john" => %{age: 28}, "meg" => %{age: 23}}
```

update_in(data, keys, fun)

```
@spec update_in(Access.t(), [term(), ...], (term() ->
term())) :: Access.t()
```

Updates a key in a nested structure.

Uses the `Access` module to traverse the structures according to the given `keys`, unless the `key` is a function. If the key is a function, it will be invoked as specified in `get_and_update_in/3`.

`data` is a nested structure (that is, a map, keyword list, or struct that implements the `Access` behaviour). The `fun` argument receives the value of `key` (or `nil` if `key` is not present) and the result replaces the value in the structure.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> update_in(users, ["john", :age], &(&1 + 1))
```

```
%{"john" => %{age: 28}, "meg" => %{age: 23}}
```

Note the current value given to the anonymous function may be `nil`. If any of the intermediate values are nil, it will raise:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> update_in(users, ["jane", :age], & &1 + 1)
** (ArgumentError) could not put/update key :age on a nil value
```

use(module, opts \\ [])

(macro)

Uses the given module in the current context.

When calling:

```
use MyModule, some: :options
```

Elixir will invoke `MyModule.__using__/1` passing the second argument of `use` as its argument. Since `__using__/1` is typically a macro, all the usual macro rules apply, and its return value should be quoted code that is then inserted where `use/2` is called.

> **Code injection**
>
> `use MyModule` works as a **code-injection point** in the caller. Given the caller of `use MyModule` has little control over how the code is injected, `use/2` should be used with care. If you can, avoid use in favor of `import/2` or `alias/2` whenever possible.

Examples

For example, to write test cases using the `ExUnit` framework provided with Elixir, a developer should `use` the `ExUnit.Case` module:

```
defmodule AssertionTest do
  use ExUnit.Case, async: true

  test "always pass" do
    assert true
  end
end
```

In this example, Elixir will call the `__using__/1` macro in the `ExUnit.Case` module with the keyword list `[async: true]` as its argument.

In other words, `use/2` translates to:

```
defmodule AssertionTest do
  require ExUnit.Case
  ExUnit.Case.__using__(async: true)

  test "always pass" do
    assert true
  end
end
```

where `ExUnit.Case` defines the `__using__/1` macro:

```
defmodule ExUnit.Case do
  defmacro __using__(opts) do
```

```
    # do something with opts
    quote do
      # return some code to inject in the caller
    end
  end
end
```

Best practices

`__using__/1` is typically used when there is a need to set some state (via module attributes) or callbacks (like `@before_compile`, see the documentation for `Module` for more information) into the caller.

`__using__/1` may also be used to alias, require, or import functionality from different modules:

```
defmodule MyModule do
  defmacro __using__(_opts) do
    quote do
      import MyModule.Foo
      import MyModule.Bar
      import MyModule.Baz

      alias MyModule.Repo
    end
  end
end
```

However, do not provide `__using__/1` if all it does is to import, alias or require the module itself. For example, avoid this:

```
defmodule MyModule do
  defmacro __using__(_opts) do
    quote do
      import MyModule
    end
  end
end
```

In such cases, developers should instead import or alias the module directly, so that they can customize those as they wish, without the indirection behind `use/2`. Developers must also avoid defining functions inside `__using__/1`.

Given `use MyModule` can generate any code, it may not be easy for developers to understand the impact of `use MyModule`.

For this reason, to provide guidance and clarity, we recommend developers to include an admonition block in their `@moduledoc` that explains how `use MyModule` impacts their code. As an example, the `GenServer` documentation outlines:

> `use GenServer`
>
> When you `use GenServer`, the `GenServer` module will set `@behaviour GenServer` and define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

This provides a quick summary of how using a module impacts the user code. Keep in mind to only list changes made to the public API of the module. For example, if `use MyModule` sets an internal attribute called `@_my_module_info` and this attribute is never meant to be public, it must not be listed.

For convenience, the markup notation to generate the admonition block above is:

```
> #### `use GenServer` {: .info}
>
> When you `use GenServer`, the GenServer module will
> set `@behaviour GenServer` and define a `child_spec/1`
> function, so your module can be used as a child
> in a supervision tree.
```

var!(var, context \\ nil)

(macro)

Marks that the given variable should not be hygienized.

This macro expects a variable and it is typically invoked inside `quote/2` to mark that a variable should not be hygienized. See `quote/2` for more information.

Examples

```
iex> Kernel.var!(example) = 1
1
iex> Kernel.var!(example)
1
```

left |> right

(macro)

Pipe operator.

This operator introduces the expression on the left-hand side as the first argument to the function call on the right-hand side.

Examples

```
iex> [1, [2], 3] |> List.flatten()
[1, 2, 3]
```

The example above is the same as calling `List.flatten([1, [2], 3])`.

The `|>/2` operator is mostly useful when there is a desire to execute a series of operations resembling a pipeline:

```
iex> [1, [2], 3] |> List.flatten() |> Enum.map(fn x -> x
* 2 end)
[2, 4, 6]
```

In the example above, the list `[1, [2], 3]` is passed as the first argument to the `List.flatten/1` function, then the flattened list is passed as the first argument to the `Enum.map/2` function which doubles each element of the list.

In other words, the expression above simply translates to:

```
Enum.map(List.flatten([1, [2], 3]), fn x -> x * 2 end)
```

Pitfalls

There are two common pitfalls when using the pipe operator.

The first one is related to operator precedence. For example, the following expression:

```
String.graphemes "Hello" |> Enum.reverse
```

Translates to:

```
String.graphemes("Hello" |> Enum.reverse())
```

which results in an error as the `Enumerable` protocol is not defined for binaries. Adding explicit parentheses resolves the ambiguity:

```
String.graphemes("Hello") |> Enum.reverse()
```

Or, even better:

```
"Hello" |> String.graphemes() |> Enum.reverse()
```

The second limitation is that Elixir always pipes to a function call. Therefore, to pipe into an anonymous function, you need to invoke it:

```
iex> some_fun = &Regex.replace(~r/l/, &1, "L")
iex> "Hello" |> some_fun.()
"HeLLo"
```

Alternatively, you can use `then/2` for the same effect:

```
iex> some_fun = &Regex.replace(~r/l/, &1, "L")
iex> "Hello" |> then(some_fun)
"HeLLo"
```

`then/2` is most commonly used when you want to pipe to a function but the value is expected outside of the first argument, such as above. By replacing `some_fun` by its value, we get:

```
iex> "Hello" |> then(&Regex.replace(~r/l/, &1, "L"))
"HeLLo"
```

left || right

(macro)

Boolean "or" operator.

Provides a short-circuit operator that evaluates and returns the second expression only if the first one does not evaluate to a truthy value (that is, it is either `nil` or `false`). Returns the first expression otherwise.

Not allowed in guard clauses.

Examples

```
iex> Enum.empty?([1]) || Enum.empty?([1])
false

iex> List.first([]) || true
true

iex> Enum.empty?([1]) || 1
```

```
1

iex> Enum.empty?([]) || throw(:bad)
true
```

Note that, unlike `or/2`, this operator accepts any expression as the first argument, not only booleans.

Kernel.SpecialForms

Special forms are the basic building blocks of Elixir, and therefore cannot be overridden by the developer.

The `Kernel.SpecialForms` module consists solely of macros that can be invoked anywhere in Elixir code without the use of the `Kernel.SpecialForms.` prefix. This is possible because they all have been automatically imported, in the same fashion as the functions and macros from the `Kernel` module.

These building blocks are defined in this module. Some of these special forms are lexical (such as `alias/2` and `case/2`). The macros `{}/1` and `<<>>/1` are also special forms used to define tuple and binary data structures respectively.

This module also documents macros that return information about Elixir's compilation environment, such as (`__ENV__/0`, `__MODULE__/0`, `__DIR__/0`, `__STACKTRACE__/0`, and `__CALLER__/0`).

Additionally, it documents two special forms, `__block__/1` and `__aliases__/1`, which are not intended to be called directly by the developer but they appear in quoted contents since they are essential in Elixir's constructs.

Summary

Functions

%struct{}

Matches on or builds a struct.

%{}

Creates a map.

&expr

Capture operator. Captures or creates an anonymous function.

left . right

Dot operator. Defines a remote call, a call to an anonymous function, or an alias.

__aliases__(args)

Internal special form to hold aliases information.

__block__(args)

Internal special form for block expressions.

__CALLER__

Returns the current calling environment as a `Macro.Env` struct.

__cursor__(args)

Internal special form for cursor position.

__DIR__

Returns the absolute path of the directory of the current file as a binary.

__ENV__

Returns the current environment information as a `Macro.Env` struct.

__MODULE__

Returns the current module name as an atom or `nil` otherwise.

__STACKTRACE__

Returns the stacktrace for the currently handled exception.

left :: right

Type operator. Used by types and bitstrings to specify types.

<<args>>

Defines a new bitstring.

left = right

Match operator. Matches the value on the right against the pattern on the left.

alias(module, opts)

`alias/2` is used to set up aliases, often useful with modules' names.

case(condition, clauses)

Matches the given expression against the given clauses.

cond(clauses)

Evaluates the expression corresponding to the first clause that evaluates to a truthy value.

fn(clauses)

Defines an anonymous function.

for(args)

Comprehensions allow you to quickly build a data structure from an enumerable or a bitstring.

import(module, opts)

Imports functions and macros from other modules.

quote(opts, block)

Gets the representation of any expression.

receive(args)

Consumes the first message matching any of the given clauses in the current process mailbox.

require(module, opts)

Requires a module in order to use its macros.

super(args)

Calls the overridden function when overriding it with `Kernel.defoverridable/1`.

try(args)

Evaluates the given expressions and handles any error, exit, or throw that may have happened.

unquote(expr)

Unquotes the given expression inside a quoted expression.

unquote_splicing(expr)

Unquotes the given list expanding its arguments.
with(args)

Combine matching clauses.

^var

Pin operator. Accesses an already bound variable in match clauses.

{args}

Creates a tuple.

Functions

%struct{}

(macro)

Matches on or builds a struct.

A struct is a tagged map that allows developers to provide default values for keys, tags to be used in polymorphic dispatches and compile time assertions.

Structs are usually defined with the `Kernel.defstruct/1` macro:

```
defmodule User do
  defstruct name: "john", age: 27
end
```

Now a struct can be created as follows:

```
%User{}
```

Underneath a struct is a map with a `:__struct__` key pointing to the `User` module, where the keys are validated at compile-time:

```
%User{} == %{__struct__: User, name: "john", age: 27}
```

The struct fields can be given when building the struct:

```
%User{age: 31}
#=> %{__struct__: User, name: "john", age: 31}
```

Or also on pattern matching to extract values out:

```
%User{age: age} = user
```

The advantage of structs is that they validate that the given keys are part of the defined struct. The example below will fail because there is no key `:full_name` in the `User` struct:

```
%User{full_name: "john doe"}
```

The map update syntax can also be used for updating structs:

```
%{user | age: 28}
```

Pattern matching on struct names

Besides allowing pattern matching on struct fields, such as:

```
%User{age: age} = user
```

Structs also allow pattern matching on the struct name:

```
%struct_name{} = user
struct_name #=> User
```

You can also assign the struct name to `_` when you want to check if something is a struct but you are not interested in its name:

```
%_{} = user
```

%{}

(macro)

Creates a map.

See the `Map` module for more information about maps, their syntax, and ways to access and manipulate them.

AST representation

Regardless of whether `=>` or the keyword syntax is used, key-value pairs in maps are always represented internally as a list of two-element tuples for simplicity:

```
iex> quote do
...>   %{"a" => :b, c: :d}
```

```
...> end
{:%{}, [], [{"a", :b}, {:c, :d}]}
```

&expr

(macro)

Capture operator. Captures or creates an anonymous function.

Capture

The capture operator is most commonly used to capture a function with given name and arity from a module:

```
iex> fun = &Kernel.is_atom/1
iex> fun.(:atom)
true
iex> fun.("string")
false
```

In the example above, we captured `Kernel.is_atom/1` as an anonymous function and then invoked it.

The capture operator can also be used to capture local functions, including private ones, and imported functions by omitting the module name:

```
&local_function/1
```

Note that `&local_function/1` creates a local capture, but `&__MODULE__.local_function/1` or `&imported_function/1` create a

remote capture. For more information, refer to the "Functions" section in the Erlang Reference Manual.

Whether a capture is local or remote has implications when using hot code reloading: local captures dispatch to the version of the module that existed at the time they were created, while remote captures dispatch to the current version of the module.

See also `Function.capture/3`.

Anonymous functions

The capture operator can also be used to partially apply functions, where `&1`, `&2` and so on can be used as value placeholders. For example:

```
iex> double = &(&1 * 2)
iex> double.(2)
4
```

In other words, `&(&1 * 2)` is equivalent to `fn x -> x * 2 end`.

We can partially apply a remote function with placeholder:

```
iex> take_five = &Enum.take(&1, 5)
iex> take_five.(1..10)
[1, 2, 3, 4, 5]
```

Another example while using an imported or local function:

```
iex> first_elem = &elem(&1, 0)
iex> first_elem.({0, 1})
0
```

The `&` operator can be used with more complex expressions:

```
iex> fun = &(&1 + &2 + &3)
iex> fun.(1, 2, 3)
6
```

As well as with lists and tuples:

```
iex> fun = &{&1, &2}
iex> fun.(1, 2)
{1, 2}

iex> fun = &[&1 | &2]
iex> fun.(1, [2, 3])
[1, 2, 3]
```

The only restrictions when creating anonymous functions is that at least one placeholder must be present, i.e. it must contain at least `&1`, and that block expressions are not supported:

```
# No placeholder, fails to compile.
&(:foo)

# Block expression, fails to compile.
&(&1; &2)
```

left . right

(macro)

Dot operator. Defines a remote call, a call to an anonymous function, or an alias.

The dot (`.`) in Elixir can be used for remote calls:

```
iex> String.downcase("FOO")
"foo"
```

In this example above, we have used `.` to invoke `downcase` in the `String` module, passing `"FOO"` as argument.

The dot may be used to invoke anonymous functions too:

```
iex> (fn n -> n end).(7)
7
```

in which case there is a function on the left hand side.

We can also use the dot for creating aliases:

```
iex> Hello.World
Hello.World
```

This time, we have joined two aliases, defining the final alias `Hello.World`.

Syntax

The right side of `.` may be a word starting with an uppercase letter, which represents an alias, a word starting with lowercase or underscore, any valid language operator or any name wrapped in single- or double-quotes. Those are all valid examples:

```
iex> Kernel.Sample
Kernel.Sample

iex> Kernel.length([1, 2, 3])
3

iex> Kernel.+(1, 2)
3

iex> Kernel."+"(1, 2)
3
```

Wrapping the function name in single- or double-quotes is always a remote call. Therefore `Kernel."Foo"` will attempt to call the function "Foo" and not return the alias `Kernel.Foo`. This is done by design as module names are more strict than function names.

When the dot is used to invoke an anonymous function there is only one operand, but it is still written using a postfix notation:

```
iex> negate = fn n -> -n end
iex> negate.(7)
-7
```

Quoted expression

When `.` is used, the quoted expression may take two distinct forms. When the right side starts with a lowercase letter (or underscore):

```
iex> quote do
...>   String.downcase("FOO")
...> end
```

```
{{:., [], [{:__aliases__, [alias: false], [:String]},
:downcase]}, [], ["FOO"]}
```

Note that we have an inner tuple, containing the atom `:.` representing the dot as first element:

```
{:., [], [{:__aliases__, [alias: false], [:String]},
:downcase]}
```

This tuple follows the general quoted expression structure in Elixir, with the name as first argument, some keyword list as metadata as second, and the list of arguments as third. In this case, the arguments are the alias `String` and the atom `:downcase`. The second argument in a remote call is **always** an atom.

In the case of calls to anonymous functions, the inner tuple with the dot special form has only one argument, reflecting the fact that the operator is unary:

```
iex> quote do
...>   negate.(0)
...> end
{{:., [], [{:negate, [], __MODULE__}]}, [], [0]}
```

When the right side is an alias (i.e. starts with uppercase), we get instead:

```
iex> quote do
...>   Hello.World
...> end
{:__aliases__, [alias: false], [:Hello, :World]}
```

We go into more details about aliases in the `__aliases__/1` special form documentation.

Unquoting

We can also use unquote to generate a remote call in a quoted expression:

```
iex> x = :downcase
iex> quote do
...>   String.unquote(x)("FOO")
...> end
{{:., [], [{:__aliases__, [alias: false], [:String]},
:downcase]}, [], ["FOO"]}
```

Similar to `Kernel."FUNCTION_NAME"`, `unquote(x)` will always generate a remote call, independent of the value of `x`. To generate an alias via the quoted expression, one needs to rely on `Module.concat/2`:

```
iex> x = Sample
iex> quote do
...>   Module.concat(String, unquote(x))
...> end
{{:., [], [{:__aliases__, [alias: false], [:Module]},
:concat]}, [],
 [{:__aliases__, [alias: false], [:String]}, Sample]}
```

__aliases__(args)

(macro)

Internal special form to hold aliases information.

It is usually compiled to an atom:

```
iex> quote do
...>   Foo.Bar
...> end
{:__aliases__, [alias: false], [:Foo, :Bar]}
```

Elixir represents `Foo.Bar` as `__aliases__` so calls can be unambiguously identified by the operator `:.`. For example:

```
iex> quote do
...>   Foo.bar()
...> end
{{:., [], [{:__aliases__, [alias: false], [:Foo]},
:bar]}, [], []}
```

Whenever an expression iterator sees a `:.` as the tuple key, it can be sure that it represents a call and the second argument in the list is an atom.

On the other hand, aliases hold some properties:

1. The head element of aliases can be any term that must expand to an atom at compilation time.
2. The tail elements of aliases are guaranteed to always be atoms.
3. When the head element of aliases is the atom `:Elixir`, no expansion happens.

__block__(args)

(macro)

Internal special form for block expressions.

This is the special form used whenever we have a block of expressions in Elixir. This special form is private and should not be invoked directly:

```
iex> quote do
...>   1
...>   2
...>   3
...> end
{:__block__, [], [1, 2, 3]}
```

__CALLER__

(macro)

Returns the current calling environment as a `Macro.Env` struct.

In the environment you can access the filename, line numbers, set up aliases, the function and others.

__cursor__(args)

(macro)

Internal special form for cursor position.

This is the special form used whenever we need to represent the cursor position in Elixir's AST. See `Code.Fragment` for more information.

__DIR__

(macro)

Returns the absolute path of the directory of the current file as a binary.

Although the directory can be accessed as `Path.dirname(__ENV__.file)`, this macro is a convenient shortcut.

__ENV__

(macro)

Returns the current environment information as a `Macro.Env` struct.

In the environment you can access the current filename, line numbers, set up aliases, the current function and others.

__MODULE__

(macro)

Returns the current module name as an atom or `nil` otherwise.

Although the module can be accessed in the `__ENV__/0`, this macro is a convenient shortcut.

__STACKTRACE__

(since 1.7.0) (macro)

Returns the stacktrace for the currently handled exception.

It is available only in the `catch` and `rescue` clauses of `try/1` expressions and function definitions.

To retrieve the stacktrace of the current process, use `Process.info(self(), :current_stacktrace)` instead.

left :: right

(macro)

Type operator. Used by types and bitstrings to specify types.

This operator is used in two distinct occasions in Elixir. It is used in typespecs to specify the type of a variable, function or of a type itself:

```
@type number :: integer | float
@spec add(number, number) :: number
```

It may also be used in bit strings to specify the type of a given bit segment:

```
<<int::integer-little, rest::bits>> = bits
```

Read the documentation on the Typespecs page and `<<>>/1` for more information on typespecs and bitstrings respectively.

<<**args**>>

(macro)

Defines a new bitstring.

Examples

```
iex> <<1, 2, 3>>
<<1, 2, 3>>
```

Types

A bitstring is made of many segments and each segment has a type. There are 9 types used in bitstrings:

- `integer`
- `float`
- `bits` (alias for `bitstring`)
- `bitstring`
- `binary`
- `bytes` (alias for `binary`)
- `utf8`
- `utf16`
- `utf32`

When no type is specified, the default is `integer`:

```
iex> <<1, 2, 3>>
<<1, 2, 3>>
```

Elixir also accepts by default the segment to be a literal string which expands to integers:

```
iex> <<0, "foo">>
<<0, 102, 111, 111>>
```

You can use one of `utf8` (the default), `utf16`, and `utf32` to control how the string is encoded:

```
iex> <<"foo"::utf16>>
<<0, 102, 0, 111, 0, 111>>
```

Which is equivalent to writing:

```
iex> <<?f::utf16, ?o::utf16, ?o::utf16>>
<<0, 102, 0, 111, 0, 111>>
```

At runtime, binaries need to be explicitly tagged as `binary`:

```
iex> rest = "oo"
iex> <<102, rest::binary>>
"foo"
```

Otherwise we get an `ArgumentError` when constructing the binary:

```
rest = "oo"
<<102, rest>>
** (ArgumentError) argument error
```

Options

Many options can be given by using `-` as separator. Order is arbitrary, so the following are all equivalent:

```
<<102::integer-native, rest::binary>>
<<102::native-integer, rest::binary>>
```

```
<<102::unsigned-big-integer, rest::binary>>
<<102::unsigned-big-integer-size(8), rest::binary>>
<<102::unsigned-big-integer-8, rest::binary>>
<<102::8-integer-big-unsigned, rest::binary>>
<<102, rest::binary>>
```

Unit and Size

The length of the match is equal to the `unit` (a number of bits) times the `size` (the number of repeated segments of length `unit`).

Type	Default Unit
`integer`	1 bit
`float`	1 bit
`binary`	8 bits

Sizes for types are a bit more nuanced. The default size for integers is 8.

For floats, it is 64. For floats, `size * unit` must result in 16, 32, or 64, corresponding to IEEE 754 binary16, binary32, and binary64, respectively.

For binaries, the default is the size of the binary. Only the last binary in a match can use the default size. All others must have their size specified explicitly, even if the match is unambiguous. For example:

```
iex> <<name::binary-size(5), " the ", species::binary>> =
<<"Frank the Walrus">>
"Frank the Walrus"
iex> {name, species}
{"Frank", "Walrus"}
```

The size can be a variable or any valid guard expression:

```
iex> name_size = 5
iex> <<name::binary-size(^name_size), " the ",
species::binary>> = <<"Frank the Walrus">>
iex> {name, species}
{"Frank", "Walrus"}
```

The size can access prior variables defined in the binary itself:

```
iex> <<name_size::size(8), name::binary-size(name_size),
" the ", species::binary>> = <<5, "Frank the Walrus">>
iex> {name, species}
{"Frank", "Walrus"}
```

However, it cannot access variables defined in the match outside of the binary/bitstring:

```
{name_size, <<name::binary-size(name_size),
_rest::binary>>} = {5, <<"Frank the Walrus">>}
** (CompileError): undefined variable "name_size" in
bitstring segment
```

Failing to specify the size for the non-last causes compilation to fail:

```
<<name::binary, " the ", species::binary>> = <<"Frank the
Walrus">>
** (CompileError): a binary field without size is only
allowed at the end of a binary pattern
```

Shortcut Syntax

Size and unit can also be specified using a syntax shortcut when passing integer values:

```
iex> x = 1
iex> <<x::8>> == <<x::size(8)>>
true
iex> <<x::8*4>> == <<x::size(8)-unit(4)>>
true
```

This syntax reflects the fact the effective size is given by multiplying the size by the unit.

Modifiers

Some types have associated modifiers to clear up ambiguity in byte representation.

Modifier	Relevant Type(s)
`signed`	`integer`
`unsigned` (default)	`integer`
`little`	`integer`, `float`, `utf16`, `utf32`
`big` (default)	`integer`, `float`, `utf16`, `utf32`
`native`	`integer`, `float`, `utf16`, `utf32`

Sign

Integers can be `signed` or `unsigned`, defaulting to `unsigned`.

```
iex> <<int::integer>> = <<-100>>
<<156>>
iex> int
156
```

```
iex> <<int::integer-signed>> = <<-100>>
<<156>>
iex> int
-100
```

`signed` and `unsigned` are only used for matching binaries (see below) and are only used for integers.

```
iex> <<-100::signed, _rest::binary>> = <<-100, "foo">>
<<156, 102, 111, 111>>
```

Endianness

Elixir has three options for endianness: `big`, `little`, and `native`. The default is `big`:

```
iex> <<number::little-integer-size(16)>> = <<0, 1>>
<<0, 1>>
iex> number
256
iex> <<number::big-integer-size(16)>> = <<0, 1>>
<<0, 1>>
iex> number
1
```

`native` is determined by the VM at startup and will depend on the host operating system.

Binary/Bitstring Matching

Binary matching is a powerful feature in Elixir that is useful for extracting information from binaries as well as pattern matching.

Binary matching can be used by itself to extract information from binaries:

```
iex> <<"Hello, ", place::binary>> = "Hello, World"
"Hello, World"
iex> place
"World"
```

Or as a part of function definitions to pattern match:

```
defmodule ImageType do
  @png_signature <<137::size(8), 80::size(8),
78::size(8), 71::size(8),
                   13::size(8), 10::size(8), 26::size(8),
10::size(8)>>
  @jpg_signature <<255::size(8), 216::size(8)>>

  def type(<<@png_signature, _rest::binary>>), do: :png
  def type(<<@jpg_signature, _rest::binary>>), do: :jpg
  def type(_), do: :unknown
end
```

Performance & Optimizations

The Erlang compiler can provide a number of optimizations on binary creation and matching. To see optimization output, set the `bin_opt_info` compiler option:

```
ERL_COMPILER_OPTIONS=bin_opt_info mix compile
```

To learn more about specific optimizations and performance considerations, check out the "Constructing and matching binaries"

chapter of the Erlang's Efficiency Guide.

left = right

(macro)

Match operator. Matches the value on the right against the pattern on the left.

alias(module, opts)

(macro)

`alias/2` is used to set up aliases, often useful with modules' names.

Examples

`alias/2` can be used to set up an alias for any module:

```
defmodule Math do
  alias MyKeyword, as: Keyword
end
```

In the example above, we have set up `MyKeyword` to be aliased as `Keyword`. So now, any reference to `Keyword` will be automatically replaced by `MyKeyword`.

In case one wants to access the original `Keyword`, it can be done by accessing `Elixir`:

```
Keyword.values #=> uses MyKeyword.values
Elixir.Keyword.values #=> uses Keyword.values
```

Note that calling `alias` without the `:as` option automatically sets an alias based on the last part of the module. For example:

```
alias Foo.Bar.Baz
```

Is the same as:

```
alias Foo.Bar.Baz, as: Baz
```

We can also alias multiple modules in one line:

```
alias Foo.{Bar, Baz, Biz}
```

Is the same as:

```
alias Foo.Bar
alias Foo.Baz
alias Foo.Biz
```

Lexical scope

`import/2`, `require/2` and `alias/2` are called directives and all have lexical scope. This means you can set up aliases inside specific functions and it won't affect the overall scope.

Warnings

If you alias a module and you don't use the alias, Elixir is going to issue a warning implying the alias is not being used.

In case the alias is generated automatically by a macro, Elixir won't emit any warnings though, since the alias was not explicitly defined.

Both warning behaviors could be changed by explicitly setting the `:warn` option to `true` or `false`.

case(condition, clauses)

(macro)

Matches the given expression against the given clauses.

`case/2` relies on pattern matching and guards to choose which clause to execute. If your logic cannot be expressed within patterns and guards, consider using `if/2` or `cond/1` instead.

Examples

```
iex> string_date = "2015-01-23"
iex> case Date.from_iso8601(string_date) do
...>   {:ok, date} -> date
...>   {:error, _reason} -> Date.utc_today()
...> end
~D[2015-01-23]
```

In the example above, we match the result of `Date.from_iso8601/1` against each clause "head" and execute the clause "body" corresponding to the first clause that matches. In our case `string_date` contains a string with a valid ISO 8601 representation of date. The function returns `{:ok, ~D[2015-01-23]}`, so the `{:ok, date}` clause is matched.

If no clause matches, an error is raised. For this reason, it may be necessary to add a final catch-all clause (like `_`) which will always match.

```
iex> x = 10
iex> case x do
...>   0 -> "This clause won't match"
...>   _ -> "This clause would match any value (x = #
{x})"
...> end
"This clause would match any value (x = 10)"
```

If you find yourself nesting `case` expressions inside `case` expressions, consider using `with/1`.

Variable handling

Note that variables bound in a clause do not leak to the outer context:

```
iex> case {:ok, 7} do
...>   {:ok, value} -> value
...>   :error -> nil
...> end

...> value
** (CompileError) undefined variable "value"
```

Variables in the outer context cannot be overridden either:

```
iex> value = 7
iex> case 3 > 5 do
...>   false ->
```

```
...>     value = 3
...>     value + 2
...>   true ->
...>     3
...> end
iex> value
7
```

In the example above, `value` is going to be `7` regardless of which clause matched. The variable `value` bound in the clause and the variable `value` bound in the outer context are two entirely separate variables.

If you want to pattern match against an existing variable, you need to use the `^/1` operator:

```
iex> x = 1
iex> case 10 do
...>   ^x -> "Won't match"
...>   _ -> "Will match"
...> end
"Will match"
```

Using guards to match against multiple values

While it is not possible to match against multiple patterns in a single clause, it's possible to match against multiple values by using guards:

```
iex> case 2 do
...>   value when value in [1, 2] ->
...>     "#{value} has been matched"
...>   3 ->
...>     "3 has been matched"
```

```
...> end
"2 has been matched"
```

cond(clauses)

(macro)

Evaluates the expression corresponding to the first clause that evaluates to a truthy value.

Examples

The following example has a single clause that always evaluates to true:

```
iex> cond do
...>   hd([1, 2, 3]) -> "1 is considered as true"
...> end
"1 is considered as true"
```

If all clauses evaluate to `nil` or `false`, `cond` raises an error. For this reason, it may be necessary to add a final always-truthy condition (anything non-`false` and non-`nil`), which will always match:

```
iex> cond do
...>   1 + 1 == 1 -> "This will never match"
...>   2 * 2 != 4 -> "Nor this"
...>   true -> "This will"
...> end
"This will"
```

If your `cond` has two clauses, and the last one falls back to `true`, you may consider using `if/2` instead.

fn(clauses)

(macro)

Defines an anonymous function.

See `Function` for more information.

Examples

```
iex> add = fn a, b -> a + b end
iex> add.(1, 2)
3
```

Anonymous functions can also have multiple clauses. All clauses should expect the same number of arguments:

```
iex> negate = fn
...>   true -> false
...>   false -> true
...> end
iex> negate.(false)
true
```

for(args)

(macro)

Comprehensions allow you to quickly build a data structure from an enumerable or a bitstring.

Let's start with an example:

```
iex> for n <- [1, 2, 3, 4], do: n * 2
[2, 4, 6, 8]
```

A comprehension accepts many generators and filters. `for` uses the `<-` operator to extract values from the enumerable on its right side and match them against the pattern on the left. We call them generators:

```
# A list generator:
iex> for n <- [1, 2, 3, 4], do: n * 2
[2, 4, 6, 8]

# A comprehension with two generators
iex> for x <- [1, 2], y <- [2, 3], do: x * y
[2, 3, 4, 6]
```

Filters can also be given:

```
# A comprehension with a generator and a filter
iex> for n <- [1, 2, 3, 4, 5, 6], rem(n, 2) == 0, do: n
[2, 4, 6]
```

Filters must evaluate to truthy values (everything but `nil` and `false`). If a filter is falsy, then the current value is discarded.

Generators can also be used to filter as it removes any value that doesn't match the pattern on the left side of `<-`:

```
iex> users = [user: "john", admin: "meg", guest:
"barbara"]
iex> for {type, name} when type != :guest <- users do
```

```
...>   String.upcase(name)
...> end
["JOHN", "MEG"]
```

Bitstring generators are also supported and are very useful when you need to organize bitstring streams:

```
iex> pixels = <<213, 45, 132, 64, 76, 32, 76, 0, 0, 234,
32, 15>>
iex> for <<r::8, g::8, b::8 <- pixels>>, do: {r, g, b}
[{213, 45, 132}, {64, 76, 32}, {76, 0, 0}, {234, 32, 15}]
```

Variable assignments inside the comprehension, be it in generators, filters or inside the block, are not reflected outside of the comprehension.

Variable assignments inside filters must still return a truthy value, otherwise values are discarded. Let's see an example. Imagine you have a keyword list where the key is a programming language and the value is its direct parent. Then let's try to compute the grandparent of each language. You could try this:

```
iex> languages = [elixir: :erlang, erlang: :prolog,
prolog: nil]
iex> for {language, parent} <- languages, grandparent =
languages[parent], do: {language, grandparent}
[elixir: :prolog]
```

Given the grandparents of Erlang and Prolog were nil, those values were filtered out. If you don't want this behavior, a simple option is to move the filter inside the do-block:

```
iex> languages = [elixir: :erlang, erlang: :prolog,
prolog: nil]
iex> for {language, parent} <- languages do
...>   grandparent = languages[parent]
...>   {language, grandparent}
...> end
[elixir: :prolog, erlang: nil, prolog: nil]
```

However, such option is not always available, as you may have further filters. An alternative is to convert the filter into a generator by wrapping the right side of `=` in a list:

```
iex> languages = [elixir: :erlang, erlang: :prolog,
prolog: nil]
iex> for {language, parent} <- languages, grandparent <-
[languages[parent]], do: {language, grandparent}
[elixir: :prolog, erlang: nil, prolog: nil]
```

The `:into` and `:uniq` options

In the examples above, the result returned by the comprehension was always a list. The returned result can be configured by passing an `:into` option, that accepts any structure as long as it implements the `Collectable` protocol.

For example, we can use bitstring generators with the `:into` option to easily remove all spaces in a string:

```
iex> for <<c <- " hello world ">>, c != ?\s, into: "",
do: <<c>>
"helloworld"
```

The `IO` module provides streams, that are both `Enumerable` and `Collectable`, here is an upcase echo server using comprehensions:

```
for line <- IO.stream(), into: IO.stream() do
  String.upcase(line)
end
```

Similarly, `uniq: true` can also be given to comprehensions to guarantee the results are only added to the collection if they were not returned before. For example:

```
iex> for x <- [1, 1, 2, 3], uniq: true, do: x * 2
[2, 4, 6]

iex> for <<x <- "abcabc">>, uniq: true, into: "", do: <<x
- 32>>
"ABC"
```

The `:reduce` option

Available since Elixir v1.8.

While the `:into` option allows us to customize the comprehension behavior to a given data type, such as putting all of the values inside a map or inside a binary, it is not always enough.

For example, imagine that you have a binary with letters where you want to count how many times each lowercase letter happens, ignoring all uppercase ones. For instance, for the string `"AbCabCABc"`, we want to return the map `%{"a" => 1, "b" => 2, "c" => 1}`.

If we were to use `:into`, we would need a data type that computes the frequency of each element it holds. While there is no such data type in

Elixir, you could implement one yourself.

A simpler option would be to use comprehensions for the mapping and filtering of letters, and then we invoke `Enum.reduce/3` to build a map, for example:

```
iex> letters = for <<x <- "AbCabCABc">>, x in ?a..?z, do:
<<x>>
iex> Enum.reduce(letters, %{}, fn x, acc ->
Map.update(acc, x, 1, & &1 + 1) end)
%{"a" => 1, "b" => 2, "c" => 1}
```

While the above is straight-forward, it has the downside of traversing the data at least twice. If you are expecting long strings as inputs, this can be quite expensive.

Luckily, comprehensions also support the `:reduce` option, which would allow us to fuse both steps above into a single step:

```
iex> for <<x <- "AbCabCABc">>, x in ?a..?z, reduce: %{}
do
...>   acc -> Map.update(acc, <<x>>, 1, & &1 + 1)
...> end
%{"a" => 1, "b" => 2, "c" => 1}
```

When the `:reduce` key is given, its value is used as the initial accumulator and the `do` block must be changed to use `->` clauses, where the left side of `->` receives the accumulated value of the previous iteration and the expression on the right side must return the new accumulator value. Once there are no more elements, the final accumulated value is returned. If there are no elements at all, then the initial accumulator value is returned.

import(module, opts)

(macro)

Imports functions and macros from other modules.

`import/2` allows one to easily access functions or macros from other modules without using the qualified name.

Examples

If you are using several functions from a given module, you can import those functions and reference them as local functions, for example:

```
iex> import List
iex> flatten([1, [2], 3])
[1, 2, 3]
```

Selector

By default, Elixir imports functions and macros from the given module, except the ones starting with an underscore (which are usually callbacks):

```
import List
```

A developer can filter to import only functions, macros, or sigils (which can be functions or macros) via the `:only` option:

```
import List, only: :functions
import List, only: :macros
```

```
import Kernel, only: :sigils
```

Alternatively, Elixir allows a developer to pass pairs of name/arities to `:only` or `:except` as a fine grained control on what to import (or not):

```
import List, only: [flatten: 1]
import String, except: [split: 2]
```

Importing the same module again will erase the previous imports, except when the `except` option is used, which is always exclusive on a previously declared `import/2`. If there is no previous import, then it applies to all functions and macros in the module. For example:

```
import List, only: [flatten: 1, keyfind: 4]
import List, except: [flatten: 1]
```

After the two import calls above, only `List.keyfind/4` will be imported.

Underscore functions

By default functions starting with `_` are not imported. If you really want to import a function starting with `_` you must explicitly include it in the `:only` selector.

```
import File.Stream, only: [__build__: 3]
```

Lexical scope

It is important to note that `import/2` is lexical. This means you can import specific macros inside specific functions:

```
defmodule Math do
  def some_function do
    # 1) Disable "if/2" from Kernel
    import Kernel, except: [if: 2]

    # 2) Require the new "if/2" macro from MyMacros
    import MyMacros

    # 3) Use the new macro
    if do_something, it_works
  end
end
```

In the example above, we imported macros from `MyMacros`, replacing the original `if/2` implementation by our own within that specific function. All other functions in that module will still be able to use the original one.

Warnings

If you import a module and you don't use any of the imported functions or macros from this module, Elixir is going to issue a warning implying the import is not being used.

In case the import is generated automatically by a macro, Elixir won't emit any warnings though, since the import was not explicitly defined.

Both warning behaviors could be changed by explicitly setting the `:warn` option to `true` or `false`.

Ambiguous function/macro names

If two modules `A` and `B` are imported and they both contain a `foo` function with an arity of `1`, an error is only emitted if an ambiguous call to `foo/1` is actually made; that is, the errors are emitted lazily, not eagerly.

quote(opts, block)

(macro)

Gets the representation of any expression.

Examples

```
iex> quote do
...>   sum(1, 2, 3)
...> end
{:sum, [], [1, 2, 3]}
```

Elixir's AST (Abstract Syntax Tree)

Any Elixir code can be represented using Elixir data structures. The building block of Elixir macros is a tuple with three elements, for example:

```
{:sum, [], [1, 2, 3]}
```

The tuple above represents a function call to `sum` passing 1, 2 and 3 as arguments. The tuple elements are:

- The first element of the tuple is always an atom or another tuple in the same representation.

- The second element of the tuple represents metadata.
- The third element of the tuple are the arguments for the function call. The third argument may be an atom, which is usually a variable (or a local call).

Besides the tuple described above, Elixir has a few literals that are also part of its AST. Those literals return themselves when quoted. They are:

```
:sum          #=> Atoms
1             #=> Integers
2.0           #=> Floats
[1, 2]        #=> Lists
"strings"     #=> Strings
{key, value} #=> Tuples with two elements
```

Any other value, such as a map or a four-element tuple, must be escaped (`Macro.escape/1`) before being introduced into an AST.

Options

- `:bind_quoted` - passes a binding to the macro. Whenever a binding is given, `unquote/1` is automatically disabled.
- `:context` - sets the resolution context.
- `:generated` - marks the given chunk as generated so it does not emit warnings. It is also useful to prevent the type system or dialyzer from reporting errors when macros generate unused clauses.
- `:file` - sets the quoted expressions to have the given file.
- `:line` - sets the quoted expressions to have the given line.

- `:location` - when set to `:keep`, keeps the current line and file from quote. Read the "Stacktrace information" section below for more information.

- `:unquote` - when `false`, disables unquoting. This means any `unquote` call will be kept as is in the AST, instead of replaced by the `unquote` arguments. For example:

```
iex> quote do
...>   unquote("hello")
...> end
"hello"

iex> quote unquote: false do
...>   unquote("hello")
...> end
{:unquote, [], ["hello"]}
```

Quote and macros

`quote/2` is commonly used with macros for code generation. As an exercise, let's define a macro that multiplies a number by itself (squared). In practice, there is no reason to define such a macro (and it would actually be seen as a bad practice), but it is simple enough that it allows us to focus on the important aspects of quotes and macros:

```
defmodule Math do
  defmacro squared(x) do
    quote do
      unquote(x) * unquote(x)
    end
```

```
  end
end
```

We can invoke it as:

```
import Math
IO.puts("Got #{squared(5)}")
```

At first, there is nothing in this example that actually reveals it is a macro. But what is happening is that, at compilation time, `squared(5)` becomes `5 * 5`. The argument `5` is duplicated in the produced code, we can see this behavior in practice though because our macro actually has a bug:

```
import Math
my_number = fn ->
  IO.puts("Returning 5")
  5
end
IO.puts("Got #{squared(my_number.())}")
```

The example above will print:

```
Returning 5
Returning 5
Got 25
```

Notice how "Returning 5" was printed twice, instead of just once. This is because a macro receives an expression and not a value (which is what we would expect in a regular function). This means that:

```
squared(my_number.())
```

Actually expands to:

```
my_number.() * my_number.()
```

Which invokes the function twice, explaining why we get the printed value twice! In the majority of the cases, this is actually unexpected behavior, and that's why one of the first things you need to keep in mind when it comes to macros is to **not unquote the same value more than once**.

Let's fix our macro:

```
defmodule Math do
  defmacro squared(x) do
    quote do
      x = unquote(x)
      x * x
    end
  end
end
```

Now invoking `squared(my_number.())` as before will print the value just once.

In fact, this pattern is so common that most of the times you will want to use the `bind_quoted` option with `quote/2`:

```
defmodule Math do
  defmacro squared(x) do
    quote bind_quoted: [x: x] do
```

```
      x * x
    end
  end
end
```

`:bind_quoted` will translate to the same code as the example above. `:bind_quoted` can be used in many cases and is seen as good practice, not only because it helps prevent us from running into common mistakes, but also because it allows us to leverage other tools exposed by macros, such as unquote fragments discussed in some sections below.

Before we finish this brief introduction, you will notice that, even though we defined a variable `x` inside our quote:

```
quote do
  x = unquote(x)
  x * x
end
```

When we call:

```
import Math
squared(5)
x
** (CompileError) undefined variable "x"
```

We can see that `x` did not leak to the user context. This happens because Elixir macros are hygienic, a topic we will discuss at length in the next sections as well.

Hygiene in variables

Consider the following example:

```
defmodule Hygiene do
  defmacro no_interference do
    quote do
      a = 1
    end
  end
end

require Hygiene

a = 10
Hygiene.no_interference()
a
#=> 10
```

In the example above, `a` returns 10 even if the macro is apparently setting it to 1 because variables defined in the macro do not affect the context the macro is executed in. If you want to set or get a variable in the caller's context, you can do it with the help of the `var!` macro:

```
defmodule NoHygiene do
  defmacro interference do
    quote do
      var!(a) = 1
    end
  end
end

require NoHygiene

a = 10
NoHygiene.interference()
```

```
a
#=> 1
```

You cannot even access variables defined in the same module unless you explicitly give it a context:

```
defmodule Hygiene do
  defmacro write do
    quote do
      a = 1
    end
  end

  defmacro read do
    quote do
      a
    end
  end
end

require Hygiene
Hygiene.write()
Hygiene.read()
** (CompileError) undefined variable "a" (context
Hygiene)
```

For such, you can explicitly pass the current module scope as argument:

```
defmodule ContextHygiene do
  defmacro write do
    quote do
      var!(a, ContextHygiene) = 1
    end
```

```
  end

  defmacro read do
    quote do
      var!(a, ContextHygiene)
    end
  end
end

require ContextHygiene
ContextHygiene.write()
ContextHygiene.read()
#=> 1
```

The contexts of a variable is identified by the third element of the tuple. The default context is `nil` and `quote` assigns another context to all variables within:

```
quote(do: var)
#=> {:var, [], Elixir}
```

In case of variables returned by macros, there may also be a `:counter` key in the metadata, which is used to further refine its contexts and guarantee isolation between macro invocations as seen in the previous example.

Hygiene in aliases

Aliases inside quote are hygienic by default. Consider the following example:

```elixir
defmodule Hygiene do
  alias Map, as: M

  defmacro no_interference do
    quote do
      M.new()
    end
  end
end

require Hygiene
Hygiene.no_interference()
#=> %{}
```

Note that, even though the alias `M` is not available in the context the macro is expanded, the code above works because `M` still expands to `Map`.

Similarly, even if we defined an alias with the same name before invoking a macro, it won't affect the macro's result:

```elixir
defmodule Hygiene do
  alias Map, as: M

  defmacro no_interference do
    quote do
      M.new()
    end
  end
end

require Hygiene
alias SomethingElse, as: M
```

```
Hygiene.no_interference()
#=> %{}
```

In some cases, you want to access an alias or a module defined in the caller. For such, you can use the `alias!` macro:

```
defmodule Hygiene do
  # This will expand to Elixir.Nested.hello()
  defmacro no_interference do
    quote do
      Nested.hello()
    end
  end

  # This will expand to Nested.hello() for
  # whatever is Nested in the caller
  defmacro interference do
    quote do
      alias!(Nested).hello()
    end
  end
end

defmodule Parent do
  defmodule Nested do
    def hello, do: "world"
  end

  require Hygiene
  Hygiene.no_interference()
  ** (UndefinedFunctionError) ...

  Hygiene.interference()
```

```
  #=> "world"
end
```

Hygiene in imports

Similar to aliases, imports in Elixir are hygienic. Consider the following code:

```
defmodule Hygiene do
  defmacrop get_length do
    quote do
      length([1, 2, 3])
    end
  end

  def return_length do
    import Kernel, except: [length: 1]
    get_length
  end
end

Hygiene.return_length()
#=> 3
```

Notice how `Hygiene.return_length/0` returns `3` even though the `Kernel.length/1` function is not imported. In fact, even if `return_length/0` imported a function with the same name and arity from another module, it wouldn't affect the function result:

```
def return_length do
  import String, only: [length: 1]
```

```
  get_length
end
```

Calling this new `return_length/0` will still return `3` as result.

Elixir is smart enough to delay the resolution to the latest possible moment. So, if you call `length([1, 2, 3])` inside quote, but no `length/1` function is available, it is then expanded in the caller:

```
defmodule Lazy do
  defmacrop get_length do
    import Kernel, except: [length: 1]

    quote do
      length("hello")
    end
  end

  def return_length do
    import Kernel, except: [length: 1]
    import String, only: [length: 1]
    get_length
  end
end

Lazy.return_length()
#=> 5
```

Stacktrace information

When defining functions via macros, developers have the option of choosing if runtime errors will be reported from the caller or from inside the quote. Let's see an example:

```
# adder.ex
defmodule Adder do
  @doc "Defines a function that adds two numbers"
  defmacro defadd do
    quote location: :keep do
      def add(a, b), do: a + b
    end
  end
end

# sample.ex
defmodule Sample do
  import Adder
  defadd
end

require Sample
Sample.add(:one, :two)
** (ArithmeticError) bad argument in arithmetic
expression
    adder.ex:5: Sample.add/2
```

When using `location: :keep` and invalid arguments are given to `Sample.add/2`, the stacktrace information will point to the file and line inside the quote. Without `location: :keep`, the error is reported to where `defadd` was invoked. `location: :keep` affects only definitions inside the quote.

> **`location: :keep` and unquote**
>
> Do not use `location: :keep` if the function definition also `unquote`s some of the macro arguments. If you do so, Elixir will store the file definition of the current location but the unquoted arguments may contain line information of the macro caller, leading to erroneous stacktraces.

Binding and unquote fragments

Elixir quote/unquote mechanisms provide a functionality called unquote fragments. Unquote fragments provide an easy way to generate functions on the fly. Consider this example:

```
kv = [foo: 1, bar: 2]
Enum.each(kv, fn {k, v} ->
  def unquote(k)(), do: unquote(v)
end)
```

In the example above, we have generated the functions `foo/0` and `bar/0` dynamically. Note the parentheses in `unquote(k)()` are important, otherwise we would try to define a function as `def :foo` instead of `def foo()`.

Now, imagine that we want to convert this functionality into a macro:

```
defmacro defkv(kv) do
  Enum.map(kv, fn {k, v} ->
    quote do
      def unquote(k)(), do: unquote(v)
    end
  end)
end
```

We can invoke this macro as:

```
defkv [foo: 1, bar: 2]
```

However, we can't invoke it as follows:

```
kv = [foo: 1, bar: 2]
defkv kv
```

This is because the macro is expecting its arguments to be a keyword list at **compilation** time. Since in the example above we are passing the representation of the variable `kv`, our code fails.

This is actually a common pitfall when developing macros. We are assuming a particular shape at compilation time, within the macro implementation. One may try to work around it by unquoting the variable inside the quoted expression:

```
defmacro defkv(kv) do
  quote do
    Enum.each(unquote(kv), fn {k, v} ->
      def unquote(k)(), do: unquote(v)
    end)
  end
end
```

If you try to run our new macro, you will notice it won't even compile, complaining that the variables `k` and `v` do not exist. This is because the two `unquote`s in the call above are meant to run at distinct moments: `unquote(kv)` applies to the immediate quote, `unquote(k)` is an unquote fragment.

One solution to this problem is to disable unquoting in the macro, however, doing that would make it impossible to inject the `kv` representation into the tree. That's when the `:bind_quoted` option comes to the rescue (again!). By using `:bind_quoted`, we can automatically disable unquoting while still injecting the desired variables into the tree:

```
defmacro defkv(kv) do
  quote bind_quoted: [kv: kv] do
    Enum.each(kv, fn {k, v} ->
      def unquote(k)(), do: unquote(v)
    end)
  end
end
```

In fact, the `:bind_quoted` option is recommended every time one desires to inject a value into the quote.

receive(args)

(macro)

Consumes the first message matching any of the given clauses in the current process mailbox.

If there is no matching message, the current process waits until a matching message arrives or until after a given timeout value.

Any new and existing messages that do not match will remain in the mailbox.

Examples

```
iex> send(self(), {:selector, 5, :quantity})
iex> receive do
...>   {:selector, number, name} when is_integer(number)
->
...>     name
...>   name when is_atom(name) ->
...>     name
...>   _ ->
```

```
...>     IO.puts(:stderr, "Unexpected message received")
...> end
:quantity
```

An optional `after` clause can be given in case no matching message is received during the given timeout period, specified in milliseconds:

```
iex> receive do
...>   {:selector, number, name} when is_integer(number)
->
...>     name
...>   name when is_atom(name) ->
...>     name
...>   _ ->
...>     IO.puts(:stderr, "Unexpected message received")
...> after
...>   10 ->
...>     "No message in 10 milliseconds"
...> end
"No message in 10 milliseconds"
```

The `after` clause can be specified even if there are no match clauses. The timeout value given to `after` can be any expression evaluating to one of the allowed values:

- `:infinity` - the process should wait indefinitely for a matching message, this is the same as not using the after clause
- `0` - if there is no matching message in the mailbox, the timeout will occur immediately
- positive integer smaller than or equal to `4_294_967_295` (`0xFFFFFFFF` in hexadecimal notation) - it should be possible to

represent the timeout value as an unsigned 32-bit integer.

Variable handling

The `receive/1` special form handles variables exactly as the `case/2` special macro. For more information, check the docs for `case/2`.

require(module, opts)

(macro)

Requires a module in order to use its macros.

Examples

Public functions in modules are globally available, but in order to use macros, you need to opt-in by requiring the module they are defined in.

Let's suppose you created your own `if/2` implementation in the module `MyMacros`. If you want to invoke it, you need to first explicitly require the `MyMacros`:

```
defmodule Math do
  require MyMacros
  MyMacros.if do_something, it_works
end
```

An attempt to call a macro that was not loaded will raise an error.

Alias shortcut

`require/2` also accepts `:as` as an option so it automatically sets up an alias. Please check `alias/2` for more information.

super(args)

(macro)

Calls the overridden function when overriding it with `Kernel.defoverridable/1`.

See `Kernel.defoverridable/1` for more information and documentation.

try(args)

(macro)

Evaluates the given expressions and handles any error, exit, or throw that may have happened.

Examples

```
try do
  do_something_that_may_fail(some_arg)
rescue
  ArgumentError ->
    IO.puts("Invalid argument given")
catch
  value ->
    IO.puts("Caught #{inspect(value)}")
else
  value ->
    IO.puts("Success! The result was #{inspect(value)}")
after
  IO.puts("This is printed regardless if it failed or
```

```
succeeded")
end
```

The `rescue` clause is used to handle exceptions while the `catch` clause can be used to catch thrown values and exits. The `else` clause can be used to control flow based on the result of the expression. `catch`, `rescue`, and `else` clauses work based on pattern matching (similar to the `case` special form).

Calls inside `try/1` are not tail recursive since the VM needs to keep the stacktrace in case an exception happens. To retrieve the stacktrace, access `__STACKTRACE__/0` inside the `rescue` or `catch` clause.

`rescue` clauses

Besides relying on pattern matching, `rescue` clauses provide some conveniences around exceptions that allow one to rescue an exception by its name. All the following formats are valid patterns in `rescue` clauses:

Rescue a single exception without binding the exception to a variable:

```
iex> try do
...>   1 / 0
...> rescue
...>   ArithmeticError -> :rescued
...> end
:rescued
```

Rescue any of the given exception without binding:

```
iex> try do
...>   1 / 0
```

```
...> rescue
...>   [ArithmeticError, ArgumentError] -> :rescued
...> end
:rescued
```

Rescue and bind the exception to the variable `x`:

```
iex> try do
...>   1 / 0
...> rescue
...>   x in [ArithmeticError] -> [:rescued,
is_exception(x)]
...> end
[:rescued, true]
```

Rescue different errors with separate clauses:

```
iex> try do
...>   1 / 0
...> rescue
...>   ArgumentError -> :rescued_argument_error
...>   ArithmeticError -> :rescued_arithmetic_error
...> end
:rescued_arithmetic_error
```

Rescue all kinds of exceptions and bind the rescued exception to the variable `x`:

```
iex> try do
...>   1 / 0
...> rescue
...>   x -> [:rescued, is_exception(x)]
```

```
...> end
[:rescued, true]
```

Erlang errors

Erlang errors are transformed into Elixir ones when rescuing:

```
iex> try do
...>   :erlang.error(:badarg)
...> rescue
...>   ArgumentError -> :rescued
...> end
:rescued
```

The most common Erlang errors will be transformed into their Elixir counterpart. Those which are not will be transformed into the more generic `ErlangError`:

```
iex> try do
...>   :erlang.error(:unknown)
...> rescue
...>   ErlangError -> :rescued
...> end
:rescued
```

In fact, `ErlangError` can be used to rescue any error that is not a proper Elixir error. For example, it can be used to rescue the earlier `:badarg` error too, prior to transformation:

```
iex> try do
...>   :erlang.error(:badarg)
...> rescue
```

```
...>   ErlangError -> :rescued
...> end
:rescued
```

`catch` clauses

The `catch` clause can be used to catch thrown values, exits, and errors.

Catching thrown values

`catch` can be used to catch values thrown by `Kernel.throw/1`:

```
iex> try do
...>   throw(:some_value)
...> catch
...>   thrown_value ->
...>     "Thrown value: #{inspect(thrown_value)}"
...> end
"Thrown value: :some_value"
```

Catching values of any kind

The `catch` clause also supports catching exits and errors. To do that, it allows matching on both the *kind* of the caught value as well as the value itself:

```
iex> try do
...>   exit(:shutdown)
...> catch
...>   :exit, value ->
...>     "Exited with value #{inspect(value)}"
...> end
```

```
"Exited with value :shutdown"

iex> try do
...>   exit(:shutdown)
...> catch
...>   kind, value when kind in [:exit, :throw] ->
...>     "Caught exit or throw with value #
{inspect(value)}"
...> end
"Caught exit or throw with value :shutdown"
```

The `catch` clause also supports `:error` alongside `:exit` and `:throw` as in Erlang, although this is commonly avoided in favor of `raise`/`rescue` control mechanisms. One reason for this is that when catching `:error`, the error is not automatically transformed into an Elixir error:

```
iex> try do
...>   :erlang.error(:badarg)
...> catch
...>   :error, :badarg -> :rescued
...> end
:rescued
```

`after` clauses

An `after` clause allows you to define cleanup logic that will be invoked both when the block of code passed to `try/1` succeeds and also when an error is raised. Note that the process will exit as usual when receiving an exit signal that causes it to exit abruptly and so the `after` clause is not guaranteed to be executed. Luckily, most resources in Elixir (such as open files, ETS tables, ports, sockets, and so on) are linked to or monitor

the owning process and will automatically clean themselves up if that process exits.

```
File.write!("tmp/story.txt", "Hello, World")
try do
  do_something_with("tmp/story.txt")
after
  File.rm("tmp/story.txt")
end
```

Although `after` clauses are invoked whether or not there was an error, they do not modify the return value. Both of the following examples print a message to STDOUT and return `:returned`:

```
try do
  :returned
after
  IO.puts("This message will be printed")
  :not_returned
end
#=> :returned

try do
  raise "boom"
rescue
  _ -> :returned
after
  IO.puts("This message will be printed")
  :not_returned
end
#=> :returned
```

`else` clauses

`else` clauses allow the result of the body passed to `try/1` to be pattern matched on:

```
iex> x = 2
...> try do
...>   1 / x
...> rescue
...>   ArithmeticError -> :infinity
...> else
...>   y when y < 1 and y > -1 -> :small
...>   _ -> :large
...> end
:small
```

If an `else` clause is not present and no exceptions are raised, the result of the expression will be returned:

```
iex> x = 5
iex> try do
...>   1 / x
...> rescue
...>   ArithmeticError -> :infinity
...> end
0.2
```

However, when an `else` clause is present but the result of the expression does not match any of the patterns then an exception will be raised. This exception will not be caught by a `catch` or `rescue` in the same `try`:

```
iex> x = 1
iex> try do
...>   try do
...>     1 / x
```

```
...>   rescue
...>     # The TryClauseError cannot be rescued here:
...>     TryClauseError -> :error_a
...>   else
...>      0.5 -> :small
...>   end
...> rescue
...>   # The TryClauseError is rescued here:
...>   TryClauseError -> :error_b
...> end
:error_b
```

Similarly, an exception inside an `else` clause is not caught or rescued inside the same `try`:

```
iex> x = 1
iex> try do
...>   try do
...>     1 / x
...>   catch
...>     # The exit(1) call below can not be caught here:
...>     :exit, _ -> :exit_a
...>   else
...>     _ -> exit(1)
...>   end
...> catch
...>   # The exit is caught here:
...>   :exit, _ -> :exit_b
...> end
:exit_b
```

This means the VM no longer needs to keep the stacktrace once inside an `else` clause and so tail recursion is possible when using a `try` with a

tail call as the final call inside an `else` clause. The same is true for `rescue` and `catch` clauses.

Only the result of the tried expression falls down to the `else` clause. If the `try` ends up in the `rescue` or `catch` clauses, their result will not fall down to `else`:

```
iex> try do
...>   throw(:catch_this)
...> catch
...>   :throw, :catch_this -> :it_was_caught
...> else
...>   # :it_was_caught will not fall down to this "else"
clause.
...>   other -> {:else, other}
...> end
:it_was_caught
```

Variable handling

Since an expression inside `try` may not have been evaluated due to an exception, any variable created inside `try` cannot be accessed externally. For instance:

```
try do
  x = 1
  do_something_that_may_fail(same_arg)
  :ok
catch
  _, _ -> :failed
end
```

```
x
#=> unbound variable "x"
```

In the example above, `x` cannot be accessed since it was defined inside the `try` clause. A common practice to address this issue is to return the variables defined inside `try`:

```
x =
  try do
    x = 1
    do_something_that_may_fail(same_arg)
    x
  catch
    _, _ -> :failed
  end
```

unquote(expr)

(macro)

Unquotes the given expression inside a quoted expression.

This function expects a valid Elixir AST, also known as quoted expression, as argument. If you would like to `unquote` any value, such as a map or a four-element tuple, you should call `Macro.escape/1` before unquoting.

Examples

Imagine the situation you have a quoted expression and you want to inject it inside some quote. The first attempt would be:

```
value =
  quote do
    13
  end

quote do
  sum(1, value, 3)
end
```

Which the argument for the `:sum` function call is not the expected result:

```
{:sum, [], [1, {:value, [], Elixir}, 3]}
```

For this, we use `unquote`:

```
iex> value =
...>   quote do
...>     13
...>   end
iex> quote do
...>   sum(1, unquote(value), 3)
...> end
{:sum, [], [1, 13, 3]}
```

If you want to unquote a value that is not a quoted expression, such as a map, you need to call `Macro.escape/1` before:

```
iex> value = %{foo: :bar}
iex> quote do
...>   process_map(unquote(Macro.escape(value)))
```

```
...> end
{:process_map, [], [{:%{}, [], [foo: :bar]}]}
```

If you forget to escape it, Elixir will raise an error when compiling the code.

unquote_splicing(expr)

(macro)

Unquotes the given list expanding its arguments.

Similar to `unquote/1`.

Examples

```
iex> values = [2, 3, 4]
iex> quote do
...>   sum(1, unquote_splicing(values), 5)
...> end
{:sum, [], [1, 2, 3, 4, 5]}
```

Also can be used in block context, outside of function arguments. Though, it is still required to be wrapped into parentheses.

```
iex> requires = for module <- [Integer, Logger] do
...>   quote do
...>     require unquote(module)
...>   end
...> end
iex> block = quote do: (unquote_splicing(requires))
```

```
iex> Macro.to_string(block)
"require Integer\nrequire Logger"
```

with(args)

(macro)

Combine matching clauses.

One of the ways to understand `with` is to show which code patterns it improves. Imagine you have a map where the fields `width` and `height` are optional and you want to compute its area, as `{:ok, area}` or return `:error`. We could implement this function as:

```
def area(opts) do
  case Map.fetch(opts, :width) do
    {:ok, width} ->
      case Map.fetch(opts, :height) do
        {:ok, height} -> {:ok, width * height}
        :error -> :error
      end

    :error ->
      :error
  end
end
```

when called as `area(%{width: 10, height: 15})`, it should return `{:ok, 150}`. If any of the fields are missing, it returns `:error`.

While the code above works, it is quite verbose. Using `with`, we could rewrite it as:

```
def area(opts) do
  with {:ok, width} <- Map.fetch(opts, :width),
       {:ok, height} <- Map.fetch(opts, :height) do
    {:ok, width * height}
  end
end
```

Instead of defining nested `case`s with clauses, we use `with` alongside the `PATTERN <- EXPRESSION` operator to match expressions on its right side against the pattern on the left. Consider `<-` as a sibling to `=`, except that, while `=` raises in case of not matches, `<-` will simply abort the `with` chain and return the non-matched value.

Let's give it a try on IEx:

```
iex> opts = %{"width" => 10, "height" => 15}
iex> with {:ok, width} <- Map.fetch(opts, "width"),
...>      {:ok, height} <- Map.fetch(opts, "height") do
...>   {:ok, width * height}
...> end
{:ok, 150}
```

If all clauses match, the `do` block is executed, returning its result. Otherwise the chain is aborted and the non-matched value is returned:

```
iex> opts = %{"width" => 10}
iex> with {:ok, width} <- Map.fetch(opts, "width"),
...>      {:ok, height} <- Map.fetch(opts, "height") do
...>   {:ok, width * height}
...> end
:error
```

As in `for/1`, variables bound inside `with/1` won't be accessible outside of `with/1`.

Expressions without `<-` may also be used in clauses. For instance, you can perform regular matches with the `=` operator:

```
iex> width = nil
iex> opts = %{width: 10, height: 15}
iex> with {:ok, width} <- Map.fetch(opts, :width),
...>      double_width = width * 2,
...>      {:ok, height} <- Map.fetch(opts, :height) do
...>   {:ok, double_width * height}
...> end
{:ok, 300}
iex> width
nil
```

The behavior of any expression in a clause is the same as if it was written outside of `with`. For example, `=` will raise a `MatchError` instead of returning the non-matched value:

```
with :foo = :bar, do: :ok
** (MatchError) no match of right hand side value: :bar
```

As with any other function or macro call in Elixir, explicit parens can also be used around the arguments before the `do-end` block:

```
iex> opts = %{width: 10, height: 15}
iex> with(
...>   {:ok, width} <- Map.fetch(opts, :width),
...>   {:ok, height} <- Map.fetch(opts, :height)
...> ) do
```

```
...>   {:ok, width * height}
...> end
{:ok, 150}
```

The choice between parens and no parens is a matter of preference.

Else clauses

An `else` option can be given to modify what is being returned from `with` in the case of a failed match:

```
with {:ok, content} <- File.read(path),
     :ok <- File.write(path, [content, "!"]) do
  :ok
else
  {:error, reason} ->
    Logger.error("could not append ! to #{path} with
reason: #{reason}")
    :error
end
```

The `else` block works like a `case`: it can have multiple clauses, and the first match will be used. Variables bound inside `with` (such as `content` in this example) are not available in the `else` block.

If an `else` block is used and there are no matching clauses, a `WithClauseError` exception is raised.

Beware!

Keep in mind that, one of potential drawback of `with` is that all failure clauses are flattened into a single `else` block. For example, take this

code that checks if a given path points to an Elixir file and that it exists before creating a backup copy:

```
with ".ex" <- Path.extname(path),
     true <- File.exists?(path) do
  backup_path = path <> ".backup"
  File.cp!(path, backup_path)
  {:ok, backup_path}
else
  binary when is_binary(binary) ->
    {:error, :invalid_extension}

  false ->
    {:error, :missing_file}
end
```

Note how we are having to reconstruct the result types of `Path.extname/1` and `File.exists?/1` to build error messages. In this case, it is better to refactor the code so each `<-` already return the desired format in case of errors, like this:

```
with :ok <- validate_extension(path),
     :ok <- validate_exists(path) do
  backup_path = path <> ".backup"
  File.cp!(path, backup_path)
  {:ok, backup_path}
end

defp validate_extension(path) do
  if Path.extname(path) == ".ex", do: :ok, else: {:error,
:invalid_extension}
end

defp validate_exists(path) do
```

```
  if File.exists?(path), do: :ok, else: {:error,
:missing_file}
end
```

Note how the code above is better organized and clearer once we make sure each `<-` in `with` returns a normalized format.

^var

(macro)

Pin operator. Accesses an already bound variable in match clauses.

Examples

Elixir allows variables to be rebound via static single assignment:

```
iex> x = 1
iex> x = x + 1
iex> x
2
```

However, in some situations, it is useful to match against an existing value, instead of rebinding. This can be done with the `^` special form, colloquially known as the pin operator:

```
iex> x = 1
iex> ^x = List.first([1])
iex> ^x = List.first([2])
** (MatchError) no match of right hand side value:
...
```

Note that `^x` always refers to the value of `x` prior to the match. The following example will match:

```
iex> x = 0
iex> {x, ^x} = {1, 0}
iex> x
1
```

{args}

(macro)

Creates a tuple.

More information about the tuple data type and about functions to manipulate tuples can be found in the `Tuple` module; some functions for working with tuples are also available in `Kernel` (such as `Kernel.elem/2` or `Kernel.tuple_size/1`).

AST representation

Only two-element tuples are considered literals in Elixir and return themselves when quoted. Therefore, all other tuples are represented in the AST as calls to the `:{}` special form.

```
iex> quote do
...>   {1, 2}
...> end
{1, 2}

iex> quote do
...>   {1, 2, 3}
```

```
...> end
{:{}, [], [1, 2, 3]}
```

Atom

Atoms are constants whose values are their own name.

They are often useful to enumerate over distinct values, such as:

```
iex> :apple
:apple
iex> :orange
:orange
iex> :watermelon
:watermelon
```

Atoms are equal if their names are equal.

```
iex> :apple == :apple
true
iex> :apple == :orange
false
```

Often they are used to express the state of an operation, by using values such as `:ok` and `:error`.

The booleans `true` and `false` are also atoms:

```
iex> true == :true
true
iex> is_atom(false)
true
```

```
iex> is_boolean(:false)
true
```

Elixir allows you to skip the leading `:` for the atoms `false`, `true`, and `nil`.

Atoms must be composed of Unicode characters such as letters, numbers, underscore, and `@`. If the keyword has a character that does not belong to the category above, such as spaces, you can wrap it in quotes:

```
iex> :"this is an atom with spaces"
:"this is an atom with spaces"
```

Summary

Functions

to_charlist(atom)

Converts an atom to a charlist.

to_string(atom)

Converts an atom to a string.

Functions

to_charlist(atom)

```
@spec to_charlist(atom()) :: charlist()
```

Converts an atom to a charlist.

Inlined by the compiler.

Examples

```
iex> Atom.to_charlist(:"An atom")
~c"An atom"
```

to_string(atom)

```
@spec to_string(atom()) :: String.t()
```

Converts an atom to a string.

Inlined by the compiler.

Examples

```
iex> Atom.to_string(:foo)
"foo"
```

Base

This module provides data encoding and decoding functions according to RFC 4648.

This document defines the commonly used base 16, base 32, and base 64 encoding schemes.

Base 16 alphabet

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	0	4	4	8	8	12	C
1	1	5	5	9	9	13	D
2	2	6	6	10	A	14	E
3	3	7	7	11	B	15	F

Base 32 alphabet

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	A	9	J	18	S	27	3
1	B	10	K	19	T	28	4
2	C	11	L	20	U	29	5
3	D	12	M	21	V	30	6
4	E	13	N	22	W	31	7
5	F	14	O	23	X		
6	G	15	P	24	Y	(pad)	=
7	H	16	Q	25	Z		
8	I	17	R	26	2		

Base 32 (extended hex) alphabet

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	0	9	9	18	I	27	R
1	1	10	A	19	J	28	S
2	2	11	B	20	K	29	T
3	3	12	C	21	L	30	U
4	4	13	D	22	M	31	V
5	5	14	E	23	N		
6	6	15	F	24	O	(pad)	=
7	7	16	G	25	P		
8	8	17	H	26	Q		

Base 64 alphabet

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	+
12	M	29	d	46	u	63	/
13	N	30	e	47	v		
14	O	31	f	48	w	(pad)	=
15	P	32	g	49	x		

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
16	Q	33	h	50	y		

Base 64 (URL and filename safe) alphabet

Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	-
12	M	29	d	46	u	63	_
13	N	30	e	47	v		
14	O	31	f	48	w	(pad)	=
15	P	32	g	49	x		
16	Q	33	h	50	y		

Summary

Types

decode_case()
encode_case()

Functions

decode16(string, opts \\ [])

Decodes a base 16 encoded string into a binary string.

decode16!(string, opts \\ [])

Decodes a base 16 encoded string into a binary string.

decode32(string, opts \\ [])

Decodes a base 32 encoded string into a binary string.

decode32!(string, opts \\ [])

Decodes a base 32 encoded string into a binary string.

decode64(string, opts \\ [])

Decodes a base 64 encoded string into a binary string.

decode64!(string, opts \\ [])

Decodes a base 64 encoded string into a binary string.

encode16(data, opts \\ [])

Encodes a binary string into a base 16 encoded string.

encode32(data, opts \\ [])

Encodes a binary string into a base 32 encoded string.

encode64(data, opts \\ [])

Encodes a binary string into a base 64 encoded string.

hex_decode32(string, opts \\ [])

Decodes a base 32 encoded string with extended hexadecimal alphabet into a binary string.

hex_decode32!(string, opts \\ [])

Decodes a base 32 encoded string with extended hexadecimal alphabet into a binary string.

hex_encode32(data, opts \\ [])

Encodes a binary string into a base 32 encoded string with an extended hexadecimal alphabet.

hex_valid32?(string, opts \\ [])

Checks if a base 32 encoded string with extended hexadecimal alphabet is valid.

url_decode64(string, opts \\ [])

Decodes a base 64 encoded string with URL and filename safe alphabet into a binary string.

url_decode64!(string, opts \\ [])

Decodes a base 64 encoded string with URL and filename safe alphabet into a binary string.

url_encode64(data, opts \\ [])

Encodes a binary string into a base 64 encoded string with URL and filename safe alphabet.

url_valid64?(string, opts \\ [])

Validates a base 64 encoded string with URL and filename safe alphabet.

valid16?(string, opts \\ [])

Checks if a string is a valid base 16 encoded string.

valid32?(string, opts \\ [])

Checks if a base 32 encoded string is valid.

valid64?(string, opts \\ [])

Validates a base 64 encoded string.

Types

decode_case()

```
@type decode_case() :: :upper | :lower | :mixed
```

encode_case()

```
@type encode_case() :: :upper | :lower
```

Functions

decode16(string, opts \\ [])

```
@spec decode16(binary(), [{:case, decode_case()}]) :: {:ok,
binary()} | :error
```

Decodes a base 16 encoded string into a binary string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

Examples

```
iex> Base.decode16("666F6F626172")
{:ok, "foobar"}

iex> Base.decode16("666f6f626172", case: :lower)
{:ok, "foobar"}

iex> Base.decode16("666f6F626172", case: :mixed)
{:ok, "foobar"}
```

decode16!(string, opts \\ [])

```
@spec decode16!(binary(), [{:case, decode_case()}]) ::
binary()
```

Decodes a base 16 encoded string into a binary string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

An `ArgumentError` exception is raised if the padding is incorrect or a non-alphabet character is present in the string.

Examples

```
iex> Base.decode16!("666F6F626172")
"foobar"

iex> Base.decode16!("666f6f626172", case: :lower)
"foobar"

iex> Base.decode16!("666f6F626172", case: :mixed)
"foobar"
```

decode32(string, opts \\ [])

```
@spec decode32(binary(), case: decode_case(), padding:
boolean()) ::
  {:ok, binary()} | :error
```

Decodes a base 32 encoded string into a binary string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding
- `:padding` - specifies whether to require padding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

The values for `:padding` can be:

- `true` - requires the input string to be padded to the nearest multiple of 8 (default)
- `false` - ignores padding from the input string

Examples

```
iex> Base.decode32("MZXW6YTBOI======")
{:ok, "foobar"}

iex> Base.decode32("mzxw6ytboi======", case: :lower)
{:ok, "foobar"}

iex> Base.decode32("mzXW6ytBOi======", case: :mixed)
{:ok, "foobar"}

iex> Base.decode32("MZXW6YTBOI", padding: false)
{:ok, "foobar"}
```

decode32!(string, opts \\ [])

```
@spec decode32!(binary(), case: decode_case(), padding: boolean()) :: binary()
```

Decodes a base 32 encoded string into a binary string.

An `ArgumentError` exception is raised if the padding is incorrect or a non-alphabet character is present in the string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding
- `:padding` - specifies whether to require padding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

The values for `:padding` can be:

- `true` - requires the input string to be padded to the nearest multiple of 8 (default)
- `false` - ignores padding from the input string

Examples

```
iex> Base.decode32!("MZXW6YTBOI======")
"foobar"
```

```
iex> Base.decode32!("mzxw6ytboi======", case: :lower)
"foobar"

iex> Base.decode32!("mzXW6ytBOi======", case: :mixed)
"foobar"

iex> Base.decode32!("MZXW6YTBOI", padding: false)
"foobar"
```

decode64(string, opts \\ [])

```
@spec decode64(binary(), ignore: :whitespace, padding:
boolean()) ::
  {:ok, binary()} | :error
```

Decodes a base 64 encoded string into a binary string.

Accepts `ignore: :whitespace` option which will ignore all the whitespace characters in the input string.

Accepts `padding: false` option which will ignore padding from the input string.

Examples

```
iex> Base.decode64("Zm9vYmFy")
{:ok, "foobar"}

iex> Base.decode64("Zm9vYmFy\n", ignore: :whitespace)
{:ok, "foobar"}

iex> Base.decode64("Zm9vYg==")
```

```
{:ok, "foob"}

iex> Base.decode64("Zm9vYg", padding: false)
{:ok, "foob"}
```

decode64!(string, opts \\ [])

```
@spec decode64!(binary(), ignore: :whitespace, padding:
boolean()) :: binary()
```

Decodes a base 64 encoded string into a binary string.

Accepts `ignore: :whitespace` option which will ignore all the whitespace characters in the input string.

Accepts `padding: false` option which will ignore padding from the input string.

An `ArgumentError` exception is raised if the padding is incorrect or a non-alphabet character is present in the string.

Examples

```
iex> Base.decode64!("Zm9vYmFy")
"foobar"

iex> Base.decode64!("Zm9vYmFy\n", ignore: :whitespace)
"foobar"

iex> Base.decode64!("Zm9vYg==")
"foob"
```

```
iex> Base.decode64!("Zm9vYg", padding: false)
"foob"
```

encode16(data, opts \\ [])

```
@spec encode16(binary(), [{:case, encode_case()}]) ::
binary()
```

Encodes a binary string into a base 16 encoded string.

Options

The accepted options are:

- `:case` - specifies the character case to use when encoding

The values for `:case` can be:

- `:upper` - uses upper case characters (default)
- `:lower` - uses lower case characters

Examples

```
iex> Base.encode16("foobar")
"666F6F626172"

iex> Base.encode16("foobar", case: :lower)
"666f6f626172"
```

encode32(data, opts \\ [])

```
@spec encode32(binary(), case: encode_case(), padding:
boolean()) :: binary()
```

Encodes a binary string into a base 32 encoded string.

Options

The accepted options are:

- `:case` - specifies the character case to use when encoding
- `:padding` - specifies whether to apply padding

The values for `:case` can be:

- `:upper` - uses upper case characters (default)
- `:lower` - uses lower case characters

The values for `:padding` can be:

- `true` - pad the output string to the nearest multiple of 8 (default)
- `false` - omit padding from the output string

Examples

```
iex> Base.encode32("foobar")
"MZXW6YTBOI======"

iex> Base.encode32("foobar", case: :lower)
"mzxw6ytboi======"
```

```
iex> Base.encode32("foobar", padding: false)
"MZXW6YTBOI"
```

encode64(data, opts \\ [])

```
@spec encode64(binary(), [{:padding, boolean()}]) :: binary()
```

Encodes a binary string into a base 64 encoded string.

Accepts `padding: false` option which will omit padding from the output string.

Examples

```
iex> Base.encode64("foobar")
"Zm9vYmFy"

iex> Base.encode64("foob")
"Zm9vYg=="

iex> Base.encode64("foob", padding: false)
"Zm9vYg"
```

hex_decode32(string, opts \\ [])

```
@spec hex_decode32(binary(), case: decode_case(), padding:
boolean()) ::
  {:ok, binary()} | :error
```

Decodes a base 32 encoded string with extended hexadecimal alphabet into a binary string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding
- `:padding` - specifies whether to require padding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

The values for `:padding` can be:

- `true` - requires the input string to be padded to the nearest multiple of 8 (default)
- `false` - ignores padding from the input string

Examples

```
iex> Base.hex_decode32("CPNMUOJ1E8======")
{:ok, "foobar"}

iex> Base.hex_decode32("cpnmuoj1e8======", case: :lower)
{:ok, "foobar"}

iex> Base.hex_decode32("cpnMuOJ1E8======", case: :mixed)
{:ok, "foobar"}
```

```
iex> Base.hex_decode32("CPNMUOJ1E8", padding: false)
{:ok, "foobar"}
```

hex_decode32!(string, opts \\ [])

```
@spec hex_decode32!(binary(), case: decode_case(), padding: boolean()) :: binary()
```

Decodes a base 32 encoded string with extended hexadecimal alphabet into a binary string.

An `ArgumentError` exception is raised if the padding is incorrect or a non-alphabet character is present in the string.

Options

The accepted options are:

- `:case` - specifies the character case to accept when decoding
- `:padding` - specifies whether to require padding

The values for `:case` can be:

- `:upper` - only allows upper case characters (default)
- `:lower` - only allows lower case characters
- `:mixed` - allows mixed case characters

The values for `:padding` can be:

- `true` - requires the input string to be padded to the nearest multiple of 8 (default)
- `false` - ignores padding from the input string

Examples

```
iex> Base.hex_decode32!("CPNMUOJ1E8======")
"foobar"

iex> Base.hex_decode32!("cpnmuoj1e8======", case: :lower)
"foobar"

iex> Base.hex_decode32!("cpnMuOJ1E8======", case: :mixed)
"foobar"

iex> Base.hex_decode32!("CPNMUOJ1E8", padding: false)
"foobar"
```

hex_encode32(data, opts \\ [])

```
@spec hex_encode32(binary(), case: encode_case(), padding: boolean()) :: binary()
```

Encodes a binary string into a base 32 encoded string with an extended hexadecimal alphabet.

Options

The accepted options are:

- `:case` - specifies the character case to use when encoding
- `:padding` - specifies whether to apply padding

The values for `:case` can be:

- `:upper` - uses upper case characters (default)
- `:lower` - uses lower case characters

The values for `:padding` can be:

- `true` - pad the output string to the nearest multiple of 8 (default)
- `false` - omit padding from the output string

Examples

```
iex> Base.hex_encode32("foobar")
"CPNMUOJ1E8======"

iex> Base.hex_encode32("foobar", case: :lower)
"cpnmuoj1e8======"

iex> Base.hex_encode32("foobar", padding: false)
"CPNMUOJ1E8"
```

hex_valid32?(string, opts \\ [])

(since 1.19.0)

```
@spec hex_valid32?(binary(), case: decode_case(), padding: boolean()) :: boolean()
```

Checks if a base 32 encoded string with extended hexadecimal alphabet is valid.

> **When to use this**
>
> Use this function when you just need to *validate* that a string is valid (extended hexadecimal) base 32 data, without actually producing a decoded output string. This function is both more performant and memory efficient than using `hex_decode32/2`, checking that the result is `{:ok, ...}`, and then discarding the decoded binary.

Options

Accepts the same options as `hex_decode32/2`.

Examples

```
iex> Base.hex_valid32?("CPNMUOJ1E8======")
true

iex> Base.hex_valid32?("cpnmuoj1e8======", case: :lower)
true

iex> Base.hex_valid32?("zzz", padding: false)
false
```

url_decode64(string, opts \\ [])

```
@spec url_decode64(binary(), ignore: :whitespace, padding:
boolean()) ::
  {:ok, binary()} | :error
```

Decodes a base 64 encoded string with URL and filename safe alphabet into a binary string.

Accepts `ignore: :whitespace` option which will ignore all the whitespace characters in the input string.

Accepts `padding: false` option which will ignore padding from the input string.

Examples

```
iex> Base.url_decode64("_3_-_A==")
{:ok, <<255, 127, 254, 252>>}

iex> Base.url_decode64("_3_-_A==\n", ignore: :whitespace)
{:ok, <<255, 127, 254, 252>>}

iex> Base.url_decode64("_3_-_A", padding: false)
{:ok, <<255, 127, 254, 252>>}
```

url_decode64!(string, opts \\ [])

```
@spec url_decode64!(binary(), ignore: :whitespace, padding:
boolean()) :: binary()
```

Decodes a base 64 encoded string with URL and filename safe alphabet into a binary string.

Accepts `ignore: :whitespace` option which will ignore all the whitespace characters in the input string.

Accepts `padding: false` option which will ignore padding from the input string.

An `ArgumentError` exception is raised if the padding is incorrect or a non-alphabet character is present in the string.

Examples

```
iex> Base.url_decode64!("_3_-_A==")
<<255, 127, 254, 252>>

iex> Base.url_decode64!("_3_-_A==\n", ignore:
```

```
:whitespace)
<<255, 127, 254, 252>>

iex> Base.url_decode64!("_3_-_A", padding: false)
<<255, 127, 254, 252>>
```

url_encode64(data, opts \\ [])

```
@spec url_encode64(binary(), [{:padding, boolean()}]) ::
binary()
```

Encodes a binary string into a base 64 encoded string with URL and filename safe alphabet.

Accepts `padding: false` option which will omit padding from the output string.

Examples

```
iex> Base.url_encode64(<<255, 127, 254, 252>>)
"_3_-_A=="

iex> Base.url_encode64(<<255, 127, 254, 252>>, padding:
false)
"_3_-_A"
```

url_valid64?(string, opts \\ [])

(since 1.19.0)

```
@spec url_valid64?(binary(), ignore: :whitespace, padding:
boolean()) :: boolean()
```

Validates a base 64 encoded string with URL and filename safe alphabet.

> **When to use this**
>
> Use this function when you just need to *validate* that a string is valid (URL-safe) base 64 data, without actually producing a decoded output string. This function is both more performant and memory efficient than using `url_decode64/2`, checking that the result is `{:ok, ...}`, and then discarding the decoded binary.

Options

Accepts the same options as `url_decode64/2`.

Examples

```
iex> Base.url_valid64?("_3_-_A==")
true

iex> Base.url_valid64?("_3_-_A==\n", ignore: :whitespace)
true

iex> Base.url_valid64?("_3_-_A", padding: false)
true
```

valid16?(string, opts \\ [])

(since 1.19.0)

```
@spec valid16?(binary(), [{:case, decode_case()}]) ::
boolean()
```

Checks if a string is a valid base 16 encoded string.

> **When to use this**
>
> Use this function when you just need to *validate* that a string is valid base 16 data, without actually producing a decoded output string. This function is both more performant and memory efficient than using `decode16/2`, checking that the result is `{:ok, ...}`, and then discarding the decoded binary.

Options

Accepts the same options as `decode16/2`.

Examples

```
iex> Base.valid16?("666F6F626172")
true

iex> Base.valid16?("666f6f626172", case: :lower)
true

iex> Base.valid16?("666f6F626172", case: :mixed)
true

iex> Base.valid16?("ff", case: :upper)
false
```

valid32?(string, opts \\ [])

(since 1.19.0)

```
@spec valid32?(binary(), case: decode_case(), padding:
boolean()) :: boolean()
```

Checks if a base 32 encoded string is valid.

> **When to use this**
>
> Use this function when you just need to *validate* that a string is valid base 32 data, without actually producing a decoded output string. This function is both more performant and memory efficient than using `decode32/2`, checking that the result is `{:ok, ...}`, and then discarding the decoded binary.

Options

Accepts the same options as `decode32/2`.

Examples

```
iex> Base.valid32?("MZXW6YTBOI======")
true

iex> Base.valid32?("mzxw6ytboi======", case: :lower)
true

iex> Base.valid32?("zzz")
false
```

valid64?(string, opts \\ [])

(since 1.19.0)

```
@spec valid64?(binary(), ignore: :whitespace, padding:
boolean()) :: boolean()
```

Validates a base 64 encoded string.

> **When to use this**
>
> Use this function when you just need to *validate* that a string is valid base 64 data, without actually producing a decoded output string. This function is both more performant and memory efficient than using `decode64/2`, checking that the result is `{:ok, ...}`, and then discarding the decoded binary.

Options

Accepts the same options as `decode64/2`.

Examples

```
iex> Base.valid64?("Zm9vYmFy")
true

iex> Base.valid64?("Zm9vYmFy\n", ignore: :whitespace)
true

iex> Base.valid64?("Zm9vYg==")
true
```

Bitwise

A set of functions that perform calculations on bits.

All bitwise functions work only on integers, otherwise an `ArithmeticError` is raised. The functions `band/2`, `bor/2`, `bsl/2`, and `bsr/2` also have operators, respectively: `&&&/2`, `|||/2`, `<<</2`, and `>>>/2`.

Guards

All bitwise functions can be used in guards: `iex> odd? = fn ...> int when Bitwise.band(int, 1) == 1 -> true ...> _ -> false ...> end iex> odd?.(1) true`

All functions in this module are inlined by the compiler.

Summary

Guards

left &&& right

Bitwise AND operator.

left <<< right

Arithmetic left bitshift operator.

left >>> right

Arithmetic right bitshift operator.

band(left, right)

Calculates the bitwise AND of its arguments.

bnot(expr)

Calculates the bitwise NOT of the argument.

bor(left, right)

Calculates the bitwise OR of its arguments.

bsl(left, right)

Calculates the result of an arithmetic left bitshift.

bsr(left, right)

Calculates the result of an arithmetic right bitshift.

bxor(left, right)

Calculates the bitwise XOR of its arguments.

left ||| right

Bitwise OR operator.

Guards

left &&& right

```
@spec integer() &&& integer() :: integer()
```

Bitwise AND operator.

Calculates the bitwise AND of its arguments.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 9 &&& 3
1
```

left <<< right

```
@spec integer() <<< integer() :: integer()
```

Arithmetic left bitshift operator.

Calculates the result of an arithmetic left bitshift.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 <<< 2
4

iex> 1 <<< -2
0

iex> -1 <<< 2
-4
```

```
iex> -1 <<< -2
-1
```

left >>> right

```
@spec integer() >>> integer() :: integer()
```

Arithmetic right bitshift operator.

Calculates the result of an arithmetic right bitshift.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 1 >>> 2
0

iex> 1 >>> -2
4

iex> -1 >>> 2
-1

iex> -1 >>> -2
-4
```

band(left, right)

```
@spec band(integer(), integer()) :: integer()
```

Calculates the bitwise AND of its arguments.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> band(9, 3)
1
```

bnot(expr)

```
@spec bnot(integer()) :: integer()
```

Calculates the bitwise NOT of the argument.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bnot(2)
-3

iex> bnot(2) &&& 3
1
```

bor(left, right)

```
@spec bor(integer(), integer()) :: integer()
```

Calculates the bitwise OR of its arguments.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bor(9, 3)
11
```

bsl(left, right)

```
@spec bsl(integer(), integer()) :: integer()
```

Calculates the result of an arithmetic left bitshift.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bsl(1, 2)
4

iex> bsl(1, -2)
0

iex> bsl(-1, 2)
-4

iex> bsl(-1, -2)
-1
```

bsr(left, right)

```
@spec bsr(integer(), integer()) :: integer()
```

Calculates the result of an arithmetic right bitshift.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bsr(1, 2)
0

iex> bsr(1, -2)
4

iex> bsr(-1, 2)
-1

iex> bsr(-1, -2)
-4
```

bxor(left, right)

```
@spec bxor(integer(), integer()) :: integer()
```

Calculates the bitwise XOR of its arguments.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> bxor(9, 3)
10
```

left ||| right

```
@spec integer() ||| integer() :: integer()
```

Bitwise OR operator.

Calculates the bitwise OR of its arguments.

Allowed in guard tests. Inlined by the compiler.

Examples

```
iex> 9 ||| 3
11
```

Date

A Date struct and functions.

The Date struct contains the fields year, month, day and calendar. New dates can be built with the `new/3` function or using the `~D` (see `sigil_D/2`) sigil:

```
iex> ~D[2000-01-01]
~D[2000-01-01]
```

Both `new/3` and sigil return a struct where the date fields can be accessed directly:

```
iex> date = ~D[2000-01-01]
iex> date.year
2000
iex> date.month
1
```

The functions on this module work with the `Date` struct as well as any struct that contains the same fields as the `Date` struct, such as `NaiveDateTime` and `DateTime`. Such functions expect `Calendar.date/0` in their typespecs (instead of `t/0`).

Developers should avoid creating the Date structs directly and instead rely on the functions provided by this module as well as the ones in third-party calendar libraries.

Comparing dates

Comparisons in Elixir using `==/2`, `>/2`, `</2` and similar are structural and based on the `Date` struct fields. For proper comparison between dates, use the `compare/2`, `after?/2` and `before?/2` functions. The existence of the `compare/2` function in this module also allows using `Enum.min/2` and `Enum.max/2` functions to get the minimum and maximum date of an `Enum`. For example:

```
iex> Enum.min([~D[2017-03-31], ~D[2017-04-01]], Date)
~D[2017-03-31]
```

Using epochs

The `add/2`, `diff/2` and `shift/2` functions can be used for computing dates or retrieving the number of days between instants. For example, if there is an interest in computing the number of days from the Unix epoch (1970-01-01):

```
iex> Date.diff(~D[2010-04-17], ~D[1970-01-01])
14716

iex> Date.add(~D[1970-01-01], 14_716)
~D[2010-04-17]

iex> Date.shift(~D[1970-01-01], year: 40, month: 3, week:
2, day: 2)
~D[2010-04-17]
```

Those functions are optimized to deal with common epochs, such as the Unix Epoch above or the Gregorian Epoch (0000-01-01).

Summary

Types

t()

Functions

add(date, days)

Adds the number of days to the given `date`.

after?(date1, date2)

Returns `true` if the first date is strictly later than the second.

before?(date1, date2)

Returns `true` if the first date is strictly earlier than the second.

beginning_of_month(date)

Calculates a date that is the first day of the month for the given `date`.

beginning_of_week(date, starting_on \\ :default)

Calculates a date that is the first day of the week for the given `date`.

compare(date1, date2)

Compares two date structs.

convert(date, calendar)

Converts the given `date` from its calendar to the given `calendar`.

convert!(date, calendar)

Similar to `Date.convert/2`, but raises an `ArgumentError` if the conversion between the two calendars is not possible.

day_of_era(date)

Calculates the day-of-era and era for a given calendar `date`.

day_of_week(date, starting_on \\ :default)

Calculates the ordinal day of the week of a given `date`.

day_of_year(date)

Calculates the day of the year of a given `date`.

days_in_month(date)

Returns the number of days in the given `date` month.

diff(date1, date2)

Calculates the difference between two dates, in a full number of days.

end_of_month(date)

Calculates a date that is the last day of the month for the given `date`.

end_of_week(date, starting_on \\ :default)

Calculates a date that is the last day of the week for the given `date`.

from_erl(tuple, calendar \\ Calendar.ISO)

Converts an Erlang date tuple to a `Date` struct.

from_erl!(tuple, calendar \\ Calendar.ISO)

Converts an Erlang date tuple but raises for invalid dates.

from_gregorian_days(days, calendar \\ Calendar.ISO)

Converts a number of gregorian days to a `Date` struct.

from_iso8601(string, calendar \\ Calendar.ISO)

Parses the extended "Dates" format described by ISO 8601:2019.

from_iso8601!(string, calendar \\ Calendar.ISO)

Parses the extended "Dates" format described by ISO 8601:2019.

leap_year?(date)

Returns `true` if the year in the given `date` is a leap year.

months_in_year(date)

Returns the number of months in the given `date` year.

new(year, month, day, calendar \\ Calendar.ISO)

Builds a new ISO date.

new!(year, month, day, calendar \\ Calendar.ISO)

Builds a new ISO date.

quarter_of_year(date)

Calculates the quarter of the year of a given `date`.

range(first, last)

Returns a range of dates.

range(first, last, step)

Returns a range of dates with a step.

shift(date, duration)

Shifts given `date` by `duration` according to its calendar.

to_erl(date)

Converts the given `date` to an Erlang date tuple.

to_gregorian_days(date)

Converts a `date` struct to a number of gregorian days.

to_iso8601(date, format \\ :extended)

Converts the given `date` to ISO 8601:2019.

to_string(date)

Converts the given date to a string according to its calendar.

utc_today(calendar \\ Calendar.ISO)

Returns the current date in UTC.

year_of_era(date)

Calculates the year-of-era and era for a given calendar year.

Types

t()

```
@type t() :: %Date{
  calendar: Calendar.calendar(),
  day: Calendar.day(),
```

```
  month: Calendar.month(),
  year: Calendar.year()
}
```

Functions

add(date, days)

(since 1.5.0)

```
@spec add(Calendar.date(), integer()) :: t()
```

Adds the number of days to the given `date`.

> **Prefer** `shift/2`
>
> Prefer `shift/2` over `add/2`, as it offers a more ergonomic API.
>
> `add/2` always considers a day to be measured according to the `Calendar.ISO`.

The days are counted as Gregorian days, independent of the underlying calendar. The date is returned in the same calendar as it was given in.

Examples

```
iex> Date.add(~D[2000-01-03], -2)
~D[2000-01-01]
iex> Date.add(~D[2000-01-01], 2)
~D[2000-01-03]
iex> Date.add(~N[2000-01-01 09:00:00], 2)
~D[2000-01-03]
iex> Date.add(~D[-0010-01-01], -2)
~D[-0011-12-30]
```

after?(date1, date2)

(since 1.15.0)

```
@spec after?(Calendar.date(), Calendar.date()) :: boolean()
```

Returns `true` if the first date is strictly later than the second.

Examples

```
iex> Date.after?(~D[2022-02-02], ~D[2021-01-01])
true
iex> Date.after?(~D[2021-01-01], ~D[2021-01-01])
false
iex> Date.after?(~D[2021-01-01], ~D[2022-02-02])
false
```

before?(date1, date2)

(since 1.15.0)

```
@spec before?(Calendar.date(), Calendar.date()) :: boolean()
```

Returns `true` if the first date is strictly earlier than the second.

Examples

```
iex> Date.before?(~D[2021-01-01], ~D[2022-02-02])
true
iex> Date.before?(~D[2021-01-01], ~D[2021-01-01])
false
```

```
iex> Date.before?(~D[2022-02-02], ~D[2021-01-01])
false
```

beginning_of_month(date)

(since 1.11.0)

```
@spec beginning_of_month(Calendar.date()) :: t()
```

Calculates a date that is the first day of the month for the given `date`.

Examples

```
iex> Date.beginning_of_month(~D[2000-01-31])
~D[2000-01-01]
iex> Date.beginning_of_month(~D[2000-01-01])
~D[2000-01-01]
iex> Date.beginning_of_month(~N[2000-01-31 01:23:45])
~D[2000-01-01]
```

beginning_of_week(date, starting_on \\ :default)

(since 1.11.0)

```
@spec beginning_of_week(Calendar.date(), starting_on ::
:default | atom()) :: t()
```

Calculates a date that is the first day of the week for the given `date`.

If the day is already the first day of the week, it returns the day itself. For the built-in ISO calendar, the week starts on Monday. A weekday rather

than `:default` can be given as `starting_on`.

Examples

```
iex> Date.beginning_of_week(~D[2020-07-11])
~D[2020-07-06]
iex> Date.beginning_of_week(~D[2020-07-06])
~D[2020-07-06]
iex> Date.beginning_of_week(~D[2020-07-11], :sunday)
~D[2020-07-05]
iex> Date.beginning_of_week(~D[2020-07-11], :saturday)
~D[2020-07-11]
iex> Date.beginning_of_week(~N[2020-07-11 01:23:45])
~D[2020-07-06]
```

compare(date1, date2)

(since 1.4.0)

```
@spec compare(Calendar.date(), Calendar.date()) :: :lt | :eq
| :gt
```

Compares two date structs.

Returns `:gt` if first date is later than the second and `:lt` for vice versa. If the two dates are equal `:eq` is returned.

Examples

```
iex> Date.compare(~D[2016-04-16], ~D[2016-04-28])
:lt
```

This function can also be used to compare across more complex calendar types by considering only the date fields:

```
iex> Date.compare(~D[2016-04-16], ~N[2016-04-28
01:23:45])
:lt
iex> Date.compare(~D[2016-04-16], ~N[2016-04-16
01:23:45])
:eq
iex> Date.compare(~N[2016-04-16 12:34:56], ~N[2016-04-16
01:23:45])
:eq
```

convert(date, calendar)

(since 1.5.0)

```
@spec convert(Calendar.date(), Calendar.calendar()) ::
  {:ok, t()} | {:error, :incompatible_calendars}
```

Converts the given `date` from its calendar to the given `calendar`.

Returns `{:ok, date}` if the calendars are compatible, or `{:error, :incompatible_calendars}` if they are not.

See also `Calendar.compatible_calendars?/2`.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> Date.convert(~D[2000-01-01], Calendar.Holocene)
{:ok, %Date{calendar: Calendar.Holocene, year: 12000,
month: 1, day: 1}}
```

convert!(date, calendar)

(since 1.5.0)

```
@spec convert!(Calendar.date(), Calendar.calendar()) :: t()
```

Similar to `Date.convert/2`, but raises an `ArgumentError` if the conversion between the two calendars is not possible.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> Date.convert!(~D[2000-01-01], Calendar.Holocene)
%Date{calendar: Calendar.Holocene, year: 12000, month: 1,
day: 1}
```

day_of_era(date)

(since 1.8.0)

```
@spec day_of_era(Calendar.date()) :: {Calendar.day(),
non_neg_integer()}
```

Calculates the day-of-era and era for a given calendar `date`.

Returns a tuple `{day, era}` representing the day within the era and the era number.

Examples

```
iex> Date.day_of_era(~D[0001-01-01])
{1, 1}

iex> Date.day_of_era(~D[0000-12-31])
{1, 0}
```

day_of_week(date, starting_on \\ :default)

(since 1.4.0)

```
@spec day_of_week(Calendar.date(), starting_on :: :default |
atom()) ::
  Calendar.day_of_week()
```

Calculates the ordinal day of the week of a given `date`.

Returns the day of the week as an integer. For the ISO 8601 calendar (the default), it is an integer from 1 to 7, where 1 is Monday and 7 is Sunday.

An optional `starting_on` value may be supplied, which configures the weekday the week starts on. The default value for it is `:default`, which translates to `:monday` for the built-in ISO 8601 calendar. Any other weekday may be used for `starting_on`, in such cases, that weekday will be considered the first day of the week, and therefore it will be assigned the ordinal number 1.

The other calendars, the value returned is an ordinal day of week. For example, `1` may mean "first day of the week" and `7` is defined to mean

"seventh day of the week". Custom calendars may also accept their own variations of the `starting_on` parameter with their own meaning.

Examples

```
# 2016-10-31 is a Monday and by default Monday is the
first day of the week
iex> Date.day_of_week(~D[2016-10-31])
1
iex> Date.day_of_week(~D[2016-11-01])
2
iex> Date.day_of_week(~N[2016-11-01 01:23:45])
2
iex> Date.day_of_week(~D[-0015-10-30])
3

# 2016-10-31 is a Monday but, as we start the week on
Sunday, now it returns 2
iex> Date.day_of_week(~D[2016-10-31], :sunday)
2
iex> Date.day_of_week(~D[2016-11-01], :sunday)
3
iex> Date.day_of_week(~N[2016-11-01 01:23:45], :sunday)
3
iex> Date.day_of_week(~D[-0015-10-30], :sunday)
4
```

day_of_year(date)

(since 1.8.0)

```
@spec day_of_year(Calendar.date()) :: Calendar.day()
```

Calculates the day of the year of a given `date`.

Returns the day of the year as an integer. For the ISO 8601 calendar (the default), it is an integer from 1 to 366.

Examples

```
iex> Date.day_of_year(~D[2016-01-01])
1
iex> Date.day_of_year(~D[2016-11-01])
306
iex> Date.day_of_year(~D[-0015-10-30])
303
iex> Date.day_of_year(~D[2004-12-31])
366
```

days_in_month(date)

(since 1.4.0)

```
@spec days_in_month(Calendar.date()) :: Calendar.day()
```

Returns the number of days in the given `date` month.

Examples

```
iex> Date.days_in_month(~D[1900-01-13])
31
iex> Date.days_in_month(~D[1900-02-09])
28
iex> Date.days_in_month(~N[2000-02-20 01:23:45])
29
```

diff(date1, date2)

(since 1.5.0)

```
@spec diff(Calendar.date(), Calendar.date()) :: integer()
```

Calculates the difference between two dates, in a full number of days.

It returns the number of Gregorian days between the dates. Only `Date` structs that follow the same or compatible calendars can be compared this way. If two calendars are not compatible, it will raise.

Examples

```
iex> Date.diff(~D[2000-01-03], ~D[2000-01-01])
2
iex> Date.diff(~D[2000-01-01], ~D[2000-01-03])
-2
iex> Date.diff(~D[0000-01-02], ~D[-0001-12-30])
3
iex> Date.diff(~D[2000-01-01], ~N[2000-01-03 09:00:00])
-2
```

end_of_month(date)

(since 1.11.0)

```
@spec end_of_month(Calendar.date()) :: t()
```

Calculates a date that is the last day of the month for the given `date`.

Examples

```
iex> Date.end_of_month(~D[2000-01-01])
~D[2000-01-31]
iex> Date.end_of_month(~D[2000-01-31])
~D[2000-01-31]
iex> Date.end_of_month(~N[2000-01-01 01:23:45])
~D[2000-01-31]
```

end_of_week(date, starting_on \\ :default)

(since 1.11.0)

```
@spec end_of_week(Calendar.date(), starting_on :: :default | atom()) :: t()
```

Calculates a date that is the last day of the week for the given `date`.

If the day is already the last day of the week, it returns the day itself. For the built-in ISO calendar, the week ends on Sunday. A weekday rather than `:default` can be given as `starting_on`.

Examples

```
iex> Date.end_of_week(~D[2020-07-11])
~D[2020-07-12]
iex> Date.end_of_week(~D[2020-07-05])
~D[2020-07-05]
iex> Date.end_of_week(~D[2020-07-06], :sunday)
~D[2020-07-11]
iex> Date.end_of_week(~D[2020-07-06], :saturday)
~D[2020-07-10]
iex> Date.end_of_week(~N[2020-07-11 01:23:45])
~D[2020-07-12]
```

from_erl(tuple, calendar \\ Calendar.ISO)

```
@spec from_erl(:calendar.date(), Calendar.calendar()) ::
{:ok, t()} | {:error, atom()}
```

Converts an Erlang date tuple to a `Date` struct.

Only supports converting dates which are in the ISO calendar, or other calendars in which the days also start at midnight. Attempting to convert dates from other calendars will return an error tuple.

Examples

```
iex> Date.from_erl({2000, 1, 1})
{:ok, ~D[2000-01-01]}
iex> Date.from_erl({2000, 13, 1})
{:error, :invalid_date}
```

from_erl!(tuple, calendar \\ Calendar.ISO)

```
@spec from_erl!(:calendar.date(), Calendar.calendar()) :: t()
```

Converts an Erlang date tuple but raises for invalid dates.

Examples

```
iex> Date.from_erl!({2000, 1, 1})
~D[2000-01-01]
iex> Date.from_erl!({2000, 13, 1})
```

```
** (ArgumentError) cannot convert {2000, 13, 1} to date,
reason: :invalid_date
```

from_gregorian_days(days, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec from_gregorian_days(integer(), Calendar.calendar()) ::
t()
```

Converts a number of gregorian days to a `Date` struct.

Examples

```
iex> Date.from_gregorian_days(1)
~D[0000-01-02]
iex> Date.from_gregorian_days(730_485)
~D[2000-01-01]
iex> Date.from_gregorian_days(-1)
~D[-0001-12-31]
```

from_iso8601(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601(String.t(), Calendar.calendar()) :: {:ok,
t()} | {:error, atom()}
```

Parses the extended "Dates" format described by ISO 8601:2019.

The year parsed by this function is limited to four digits.

Examples

```
iex> Date.from_iso8601("2015-01-23")
{:ok, ~D[2015-01-23]}

iex> Date.from_iso8601("2015:01:23")
{:error, :invalid_format}

iex> Date.from_iso8601("2015-01-32")
{:error, :invalid_date}
```

from_iso8601!(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601!(String.t(), Calendar.calendar()) :: t()
```

Parses the extended "Dates" format described by ISO 8601:2019.

Raises if the format is invalid.

Examples

```
iex> Date.from_iso8601!("2015-01-23")
~D[2015-01-23]
iex> Date.from_iso8601!("2015:01:23")
** (ArgumentError) cannot parse "2015:01:23" as date,
reason: :invalid_format
```

leap_year?(date)

(since 1.4.0)

```
@spec leap_year?(Calendar.date()) :: boolean()
```

Returns `true` if the year in the given `date` is a leap year.

Examples

```
iex> Date.leap_year?(~D[2000-01-01])
true
iex> Date.leap_year?(~D[2001-01-01])
false
iex> Date.leap_year?(~D[2004-01-01])
true
iex> Date.leap_year?(~D[1900-01-01])
false
iex> Date.leap_year?(~N[2004-01-01 01:23:45])
true
```

months_in_year(date)

(since 1.7.0)

```
@spec months_in_year(Calendar.date()) :: Calendar.month()
```

Returns the number of months in the given `date` year.

Example

```
iex> Date.months_in_year(~D[1900-01-13])
12
```

new(year, month, day, calendar \\ Calendar.ISO)

```
@spec new(Calendar.year(), Calendar.month(), Calendar.day(),
Calendar.calendar()) ::
  {:ok, t()} | {:error, atom()}
```

Builds a new ISO date.

Expects all values to be integers. Returns `{:ok, date}` if each entry fits its appropriate range, returns `{:error, reason}` otherwise.

Examples

```
iex> Date.new(2000, 1, 1)
{:ok, ~D[2000-01-01]}
iex> Date.new(2000, 13, 1)
{:error, :invalid_date}
iex> Date.new(2000, 2, 29)
{:ok, ~D[2000-02-29]}

iex> Date.new(2000, 2, 30)
{:error, :invalid_date}
iex> Date.new(2001, 2, 29)
{:error, :invalid_date}
```

new!(year, month, day, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec new!(Calendar.year(), Calendar.month(), Calendar.day(),
Calendar.calendar()) ::
```

```
t()
```

Builds a new ISO date.

Expects all values to be integers. Returns `date` if each entry fits its appropriate range, raises if the date is invalid.

Examples

```
iex> Date.new!(2000, 1, 1)
~D[2000-01-01]
iex> Date.new!(2000, 13, 1)
** (ArgumentError) cannot build date, reason:
:invalid_date
iex> Date.new!(2000, 2, 29)
~D[2000-02-29]
```

quarter_of_year(date)

(since 1.8.0)

```
@spec quarter_of_year(Calendar.date()) :: non_neg_integer()
```

Calculates the quarter of the year of a given `date`.

Returns the day of the year as an integer. For the ISO 8601 calendar (the default), it is an integer from 1 to 4.

Examples

```
iex> Date.quarter_of_year(~D[2016-10-31])
4
iex> Date.quarter_of_year(~D[2016-01-01])
```

```
1
iex> Date.quarter_of_year(~N[2016-04-01 01:23:45])
2
iex> Date.quarter_of_year(~D[-0015-09-30])
3
```

range(first, last)

(since 1.5.0)

```
@spec range(Calendar.date(), Calendar.date()) ::
Date.Range.t()
```

Returns a range of dates.

A range of dates represents a discrete number of dates where the first and last values are dates with matching calendars.

Ranges of dates can be increasing (`first <= last`) and are always inclusive. For a decreasing range, use `range/3` with a step of -1 as first argument.

Examples

```
iex> Date.range(~D[1999-01-01], ~D[2000-01-01])
Date.range(~D[1999-01-01], ~D[2000-01-01])
```

A range of dates implements the `Enumerable` protocol, which means functions in the `Enum` module can be used to work with ranges:

```
iex> range = Date.range(~D[2001-01-01], ~D[2002-01-01])
iex> range
```

```
Date.range(~D[2001-01-01], ~D[2002-01-01])
iex> Enum.count(range)
366
iex> ~D[2001-02-01] in range
true
iex> Enum.take(range, 3)
[~D[2001-01-01], ~D[2001-01-02], ~D[2001-01-03]]
```

range(first, last, step)

(since 1.12.0)

```
@spec range(Calendar.date(), Calendar.date(), step ::
pos_integer() | neg_integer()) ::
  Date.Range.t()
```

Returns a range of dates with a step.

Examples

```
iex> range = Date.range(~D[2001-01-01], ~D[2002-01-01],
2)
iex> range
Date.range(~D[2001-01-01], ~D[2002-01-01], 2)
iex> Enum.count(range)
183
iex> ~D[2001-01-03] in range
true
iex> Enum.take(range, 3)
[~D[2001-01-01], ~D[2001-01-03], ~D[2001-01-05]]
```

shift(date, duration)

(since 1.17.0)

```
@spec shift(Calendar.date(), Duration.t() | [unit_pair]) ::
t()
when unit_pair:
       {:year, integer()}
       | {:month, integer()}
       | {:week, integer()}
       | {:day, integer()}
```

Shifts given `date` by `duration` according to its calendar.

Allowed units are: `:year`, `:month`, `:week`, `:day`.

When using the default ISO calendar, durations are collapsed and applied in the order of months and then days:

- when shifting by 1 year and 2 months the date is actually shifted by 14 months
- when shifting by 2 weeks and 3 days the date is shifted by 17 days

When shifting by month, days are rounded down to the nearest valid date.

Raises an `ArgumentError` when called with time scale units.

Examples

```
iex> Date.shift(~D[2016-01-03], month: 2)
~D[2016-03-03]
iex> Date.shift(~D[2016-01-30], month: -1)
~D[2015-12-30]
iex> Date.shift(~D[2016-01-31], year: 4, day: 1)
```

```
~D[2020-02-01]
iex> Date.shift(~D[2016-01-03], Duration.new!(month: 2))
~D[2016-03-03]

# leap years
iex> Date.shift(~D[2024-02-29], year: 1)
~D[2025-02-28]
iex> Date.shift(~D[2024-02-29], year: 4)
~D[2028-02-29]

# rounding down
iex> Date.shift(~D[2015-01-31], month: 1)
~D[2015-02-28]
```

to_erl(date)

```
@spec to_erl(Calendar.date()) :: :calendar.date()
```

Converts the given `date` to an Erlang date tuple.

Only supports converting dates which are in the ISO calendar, or other calendars in which the days also start at midnight. Attempting to convert dates from other calendars will raise.

Examples

```
iex> Date.to_erl(~D[2000-01-01])
{2000, 1, 1}

iex> Date.to_erl(~N[2000-01-01 00:00:00])
{2000, 1, 1}
```

to_gregorian_days(date)

(since 1.11.0)

```
@spec to_gregorian_days(Calendar.date()) :: integer()
```

Converts a `date` struct to a number of gregorian days.

Examples

```
iex> Date.to_gregorian_days(~D[0000-01-02])
1
iex> Date.to_gregorian_days(~D[2000-01-01])
730_485
iex> Date.to_gregorian_days(~N[2000-01-01 00:00:00])
730_485
```

to_iso8601(date, format \\ :extended)

```
@spec to_iso8601(Calendar.date(), :extended | :basic) ::
String.t()
```

Converts the given `date` to ISO 8601:2019.

By default, `Date.to_iso8601/2` returns dates formatted in the "extended" format, for human readability. It also supports the "basic" format through passing the `:basic` option.

Only supports converting dates which are in the ISO calendar, or other calendars in which the days also start at midnight. Attempting to convert dates from other calendars will raise an `ArgumentError`.

Examples

```
iex> Date.to_iso8601(~D[2000-02-28])
"2000-02-28"

iex> Date.to_iso8601(~D[2000-02-28], :basic)
"20000228"

iex> Date.to_iso8601(~N[2000-02-28 00:00:00])
"2000-02-28"
```

to_string(date)

```
@spec to_string(Calendar.date()) :: String.t()
```

Converts the given date to a string according to its calendar.

Examples

```
iex> Date.to_string(~D[2000-02-28])
"2000-02-28"
iex> Date.to_string(~N[2000-02-28 01:23:45])
"2000-02-28"
iex> Date.to_string(~D[-0100-12-15])
"-0100-12-15"
```

utc_today(calendar \\ Calendar.ISO)

(since 1.4.0)

```
@spec utc_today(Calendar.calendar()) :: t()
```

Returns the current date in UTC.

Examples

```
iex> date = Date.utc_today()
iex> date.year >= 2016
true
```

year_of_era(date)

(since 1.8.0)

```
@spec year_of_era(Calendar.date()) :: {Calendar.year(), non_neg_integer()}
```

Calculates the year-of-era and era for a given calendar year.

Returns a tuple `{year, era}` representing the year within the era and the era number.

Examples

```
iex> Date.year_of_era(~D[0001-01-01])
{1, 1}
iex> Date.year_of_era(~D[0000-12-31])
{1, 0}
iex> Date.year_of_era(~D[-0001-01-01])
{2, 0}
```

DateTime

A datetime implementation with a time zone.

This datetime can be seen as a snapshot of a date and time at a given time zone. For such purposes, it also includes both UTC and Standard offsets, as well as the zone abbreviation field used exclusively for formatting purposes. Note future datetimes are not necessarily guaranteed to exist, as time zones may change any time in the future due to geopolitical reasons. See the "Datetimes as snapshots" section for more information.

Remember, comparisons in Elixir using `==/2`, `>/2`, `</2` and friends are structural and based on the DateTime struct fields. For proper comparison between datetimes, use the `compare/2`, `after?/2` and `before?/2` functions. The existence of the `compare/2` function in this module also allows using `Enum.min/2` and `Enum.max/2` functions to get the minimum and maximum datetime of an `Enum`. For example:

```
iex> Enum.min([~U[2022-01-12 00:01:00.00Z], ~U[2021-01-12
00:01:00.00Z]], DateTime)
~U[2021-01-12 00:01:00.00Z]
```

Developers should avoid creating the `DateTime` struct directly and instead rely on the functions provided by this module as well as the ones in third-party calendar libraries.

Time zone database

Many functions in this module require a time zone database. A time zone database is a record of the UTC offsets that its locales have used at

various times in the past, are using, and are expected to use in the future. Because those plans can change, it needs to be periodically updated.

By default, `DateTime` uses the default time zone database returned by `Calendar.get_time_zone_database/0`, which defaults to `Calendar.UTCOnlyTimeZoneDatabase` which only handles "Etc/UTC" datetimes and returns `{:error, :utc_only_time_zone_database}` for any other time zone.

Other time zone databases can also be configured. Here are some available options and libraries:

- `time_zone_info`
- `tz`
- `tzdata`
- `zoneinfo` - recommended for embedded devices

To use one of them, first make sure it is added as a dependency in `mix.exs`. It can then be configured either via configuration:

```
config :elixir, :time_zone_database, Tz.TimeZoneDatabase
```

or by calling `Calendar.put_time_zone_database/1`:

```
Calendar.put_time_zone_database(Tz.TimeZoneDatabase)
```

See the proper names in the library installation instructions.

Datetimes as snapshots

In the first section, we described datetimes as a "snapshot of a date and time at a given time zone". To understand precisely what we mean, let's see an example.

Imagine someone in Poland who wants to schedule a meeting with someone in Brazil in the next year. The meeting will happen at 2:30 AM in the Polish time zone. At what time will the meeting happen in Brazil?

You can consult the time zone database today, one year before, using the API in this module and it will give you an answer that is valid right now. However, this answer may not be valid in the future. Why? Because both Brazil and Poland may change their timezone rules, ultimately affecting the result. For example, a country may choose to enter or abandon "Daylight Saving Time", which is a process where we adjust the clock one hour forward or one hour back once per year. Whenever the rules change, the exact instant that 2:30 AM in Polish time will be in Brazil may change.

In other words, whenever working with future DateTimes, there is no guarantee the results you get will always be correct, until the event actually happens. Therefore, when you ask for a future time, the answers you get are a snapshot that reflects the current state of the time zone rules. For datetimes in the past, this is not a problem, because time zone rules do not change for past events.

To make matters worse, it may be that 2:30 AM in Polish time does not actually even exist or it is ambiguous. If a certain time zone observes "Daylight Saving Time", they will move their clock forward once a year. When this happens, there is a whole hour that does not exist. Then, when they move the clock back, there is a certain hour that will happen twice. So if you want to schedule a meeting when this shift back happens, you would need to explicitly say which occurrence of 2:30 AM you mean: the one in "Summer Time", which occurs before the shift, or the one in "Standard Time", which occurs after it. Applications that are date and time sensitive need to take these scenarios into account and correctly communicate them to users.

The good news is: Elixir contains all of the building blocks necessary to tackle those problems. The default timezone database used by Elixir, `Calendar.UTCOnlyTimeZoneDatabase`, only works with UTC, which

does not observe those issues. Once you bring a proper time zone database, the functions in this module will query the database and return the relevant information. For example, look at how `DateTime.new/4` returns different results based on the scenarios described in this section.

Converting between timezones

Bearing in mind the cautions above, and assuming you've brought in a full timezone database, here are some examples of common shifts between time zones.

```
# Local time to UTC
new_york = DateTime.from_naive!(~N[2023-06-26T09:30:00],
"America/New_York")
#=> #DateTime<2023-06-26 09:30:00-04:00 EDT
America/New_York>

utc = DateTime.shift_zone!(new_york, "Etc/UTC")
#=> ~U[2023-06-26 13:30:00Z]

# UTC to local time
DateTime.shift_zone!(utc, "Europe/Paris")
#=> #DateTime<2023-06-26 15:30:00+02:00 CEST
Europe/Paris>
```

Summary

Types

t()

Functions

add(datetime, amount_to_add, unit \\ :second, time_zone_database \\ Calendar.get_time_zone_database())

Adds a specified amount of time to a `DateTime`.

after?(datetime1, datetime2)

Returns `true` if the first datetime is strictly later than the second.

before?(datetime1, datetime2)

Returns `true` if the first datetime is strictly earlier than the second.

compare(datetime1, datetime2)

Compares two datetime structs.

convert(datetime, calendar)

Converts a given `datetime` from one calendar to another.

convert!(datetime, calendar)

Converts a given `datetime` from one calendar to another.

diff(datetime1, datetime2, unit \\ :second)

Subtracts `datetime2` from `datetime1`.

from_gregorian_seconds(seconds, arg \\ {0, 0}, calendar \\ Calendar.ISO)

Converts a number of gregorian seconds to a `DateTime` struct.

from_iso8601(string, format_or_calendar \\ Calendar.ISO)

Parses the extended "Date and time of day" format described by ISO 8601:2019.

from_iso8601(string, calendar, format)

Converts from ISO8601 specifying both a calendar and a mode.

from_naive(naive_datetime, time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Converts the given `NaiveDateTime` to `DateTime`.

from_naive!(naive_datetime, time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Converts the given `NaiveDateTime` to `DateTime`.

from_unix(integer, unit \\ :second, calendar \\ Calendar.ISO)

Converts the given Unix time to `DateTime`.

from_unix!(integer, unit \\ :second, calendar \\ Calendar.ISO)

Converts the given Unix time to `DateTime`.

new(date, time, time_zone \\ "Etc/UTC", time_zone_database \\ Calendar.get_time_zone_database())

Builds a datetime from date and time structs.

new!(date, time, time_zone \\ "Etc/UTC", time_zone_database \\ Calendar.get_time_zone_database())

Builds a datetime from date and time structs, raising on errors.

now(time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Returns the current datetime in the provided time zone.

now!(time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Returns the current datetime in the provided time zone or raises on errors

shift(datetime, duration, time_zone_database \\ Calendar.get_time_zone_database())

Shifts given `datetime` by `duration` according to its calendar.

shift_zone(datetime, time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Changes the time zone of a `DateTime`.

shift_zone!(datetime, time_zone, time_zone_database \\ Calendar.get_time_zone_database())

Changes the time zone of a `DateTime` or raises on errors.

to_date(datetime)

Converts a `DateTime` into a `Date`.

to_gregorian_seconds(datetime)

Converts a `DateTime` struct to a number of gregorian seconds and microseconds.

to_iso8601(datetime, format \\ :extended, offset \\ nil)

Converts the given datetime to ISO 8601:2019 format.

to_naive(datetime)

Converts the given `datetime` into a `NaiveDateTime`.

to_string(datetime)

Converts the given `datetime` to a string according to its calendar.

to_time(datetime)

Converts a `DateTime` into `Time`.

to_unix(datetime, unit \\ :second)

Converts the given `datetime` to Unix time.

truncate(datetime, precision)

Returns the given datetime with the microsecond field truncated to the given precision (`:microsecond`, `:millisecond` or `:second`).

utc_now(calendar_or_time_unit \\ Calendar.ISO)

Returns the current datetime in UTC.

utc_now(time_unit, calendar)

Returns the current datetime in UTC, supporting a specific calendar and precision.

Types

t()

```
@type t() :: %DateTime{
  calendar: Calendar.calendar(),
  day: Calendar.day(),
  hour: Calendar.hour(),
  microsecond: Calendar.microsecond(),
  minute: Calendar.minute(),
  month: Calendar.month(),
  second: Calendar.second(),
  std_offset: Calendar.std_offset(),
```

```
  time_zone: Calendar.time_zone(),
  utc_offset: Calendar.utc_offset(),
  year: Calendar.year(),
  zone_abbr: Calendar.zone_abbr()
}
```

Functions

add(datetime, amount_to_add, unit \\ :second, time_zone_database \\ Calendar.get_time_zone_database())

(since 1.8.0)

```
@spec add(
  Calendar.datetime(),
  integer(),
  :day | :hour | :minute | System.time_unit(),
  Calendar.time_zone_database()
) :: t()
```

Adds a specified amount of time to a `DateTime`.

> **Prefer** `shift/2`
>
> Prefer `shift/2` over `add/3`, as it offers a more ergonomic API.
>
> `add/3` provides a lower-level API which only supports fixed units such as `:hour` and `:second`, but not `:month` (as the exact length of a month depends on the current month). `add/3` always considers the unit to be computed according to the `Calendar.ISO`.

Accepts an `amount_to_add` in any `unit`. `unit` can be `:day`, `:hour`, `:minute`, `:second` or any subsecond precision from `System.time_unit/0` for convenience but ultimately they are all

converted to microseconds. Negative values will move backwards in time and the default precision is `:second`.

This function relies on a contiguous representation of time, ignoring timezone changes. For example, if you add one day when there are summer time/daylight saving time changes, it will also change the time forward or backward by one hour, so the elapsed time is precisely 24 hours. Similarly, adding just a few seconds to a datetime just before "spring forward" can cause wall time to increase by more than an hour.

While this means this function is precise in terms of elapsed time, its result may be confusing in certain use cases. For example, if a user requests a meeting to happen every day at 15:00 and you use this function to compute all future meetings by adding day after day, this function may change the meeting time to 14:00 or 16:00 if there are changes to the current timezone.

In case you don't want these changes to happen automatically or you want to surface time zone conflicts to the user, you can add to the datetime as a naive datetime and then use `from_naive/2`:

```
dt |> NaiveDateTime.add(1, :day) |>
DateTime.from_naive(dt.time_zone)
```

The above will surface time jumps and ambiguous datetimes, allowing you to deal with them accordingly.

Examples

```
iex> dt = DateTime.from_naive!(~N[2018-11-15 10:00:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> dt |> DateTime.add(3600, :second,
FakeTimeZoneDatabase)
#DateTime<2018-11-15 11:00:00+01:00 CET
```

```
Europe/Copenhagen>

iex> DateTime.add(~U[2018-11-15 10:00:00Z], 3600,
:second)
~U[2018-11-15 11:00:00Z]
```

When adding 3 seconds just before "spring forward" we go from 1:59:59 to 3:00:02:

```
iex> dt = DateTime.from_naive!(~N[2019-03-31
01:59:59.123], "Europe/Copenhagen", FakeTimeZoneDatabase)
iex> dt |> DateTime.add(3, :second, FakeTimeZoneDatabase)
#DateTime<2019-03-31 03:00:02.123+02:00 CEST
Europe/Copenhagen>
```

When adding 1 day during "spring forward", the hour also changes:

```
iex> dt = DateTime.from_naive!(~N[2019-03-31 01:00:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> dt |> DateTime.add(1, :day, FakeTimeZoneDatabase)
#DateTime<2019-04-01 02:00:00+02:00 CEST
Europe/Copenhagen>
```

This operation merges the precision of the naive date time with the given unit:

```
iex> result = DateTime.add(~U[2014-10-02 00:29:10Z], 21,
:millisecond)
~U[2014-10-02 00:29:10.021Z]
iex> result.microsecond
{21000, 3}
```

after?(datetime1, datetime2)

(since 1.15.0)

```
@spec after?(Calendar.datetime(), Calendar.datetime()) ::
boolean()
```

Returns `true` if the first datetime is strictly later than the second.

Examples

```
iex> DateTime.after?(~U[2022-02-02 11:00:00Z], ~U[2021-
01-01 11:00:00Z])
true
iex> DateTime.after?(~U[2021-01-01 11:00:00Z], ~U[2021-
01-01 11:00:00Z])
false
iex> DateTime.after?(~U[2021-01-01 11:00:00Z], ~U[2022-
02-02 11:00:00Z])
false
```

before?(datetime1, datetime2)

(since 1.15.0)

```
@spec before?(Calendar.datetime(), Calendar.datetime()) ::
boolean()
```

Returns `true` if the first datetime is strictly earlier than the second.

Examples

```
iex> DateTime.before?(~U[2021-01-01 11:00:00Z], ~U[2022-
02-02 11:00:00Z])
true
iex> DateTime.before?(~U[2021-01-01 11:00:00Z], ~U[2021-
01-01 11:00:00Z])
false
iex> DateTime.before?(~U[2022-02-02 11:00:00Z], ~U[2021-
01-01 11:00:00Z])
false
```

compare(datetime1, datetime2)

(since 1.4.0)

```
@spec compare(Calendar.datetime(), Calendar.datetime()) ::
:lt | :eq | :gt
```

Compares two datetime structs.

Returns `:gt` if the first datetime is later than the second and `:lt` for vice versa. If the two datetimes are equal `:eq` is returned.

Note that both UTC and Standard offsets will be taken into account when comparison is done.

Examples

```
iex> dt1 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
```

```
iex> dt2 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.compare(dt1, dt2)
:gt
```

convert(datetime, calendar)

(since 1.5.0)

```
@spec convert(Calendar.datetime(), Calendar.calendar()) ::
  {:ok, t()} | {:error, :incompatible_calendars}
```

Converts a given `datetime` from one calendar to another.

If it is not possible to convert unambiguously between the calendars (see `Calendar.compatible_calendars?/2`), an `{:error, :incompatible_calendars}` tuple is returned.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> dt1 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: -14400, std_offset: 0,
```

```
time_zone: "America/Manaus"}
iex> DateTime.convert(dt1, Calendar.Holocene)
{:ok, %DateTime{calendar: Calendar.Holocene, day: 29,
hour: 23,
                microsecond: {0, 0}, minute: 0, month: 2,
second: 7, std_offset: 0,
                time_zone: "America/Manaus", utc_offset:
-14400, year: 12000,
                zone_abbr: "AMT"}}
```

convert!(datetime, calendar)

(since 1.5.0)

```
@spec convert!(Calendar.datetime(), Calendar.calendar()) ::
t()
```

Converts a given `datetime` from one calendar to another.

If it is not possible to convert unambiguously between the calendars (see `Calendar.compatible_calendars?/2`), an ArgumentError is raised.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> dt1 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
```

```
iex> DateTime.convert!(dt1, Calendar.Holocene)
%DateTime{calendar: Calendar.Holocene, day: 29, hour: 23,
          microsecond: {0, 0}, minute: 0, month: 2,
second: 7, std_offset: 0,
          time_zone: "America/Manaus", utc_offset:
-14400, year: 12000,
          zone_abbr: "AMT"}
```

diff(datetime1, datetime2, unit \\ :second)

(since 1.5.0)

```
@spec diff(
  Calendar.datetime(),
  Calendar.datetime(),
  :day | :hour | :minute | System.time_unit()
) :: integer()
```

Subtracts `datetime2` from `datetime1`.

The answer can be returned in any `:day`, `:hour`, `:minute`, or any `unit` available from `System.time_unit/0`. The unit is measured according to `Calendar.ISO` and defaults to `:second`.

Fractional results are not supported and are truncated.

Examples

```
iex> DateTime.diff(~U[2024-01-15 10:00:10Z], ~U[2024-01-
15 10:00:00Z])
10
```

This function also considers timezone offsets:

```
iex> dt1 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> dt2 = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                 hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                 utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.diff(dt1, dt2)
18000
iex> DateTime.diff(dt2, dt1)
-18000
iex> DateTime.diff(dt1, dt2, :hour)
5
iex> DateTime.diff(dt2, dt1, :hour)
-5
```

from_gregorian_seconds(seconds, arg \\ {0, 0}, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec from_gregorian_seconds(integer(),
Calendar.microsecond(), Calendar.calendar()) ::
  t()
```

Converts a number of gregorian seconds to a `DateTime` struct.

The returned `DateTime` will have `UTC` timezone, if you want other timezone, please use `DateTime.shift_zone/3`.

Examples

```
iex> DateTime.from_gregorian_seconds(1)
~U[0000-01-01 00:00:01Z]
iex> DateTime.from_gregorian_seconds(63_755_511_991,
{5000, 3})
~U[2020-05-01 00:26:31.005Z]
iex> DateTime.from_gregorian_seconds(-1)
~U[-0001-12-31 23:59:59Z]
```

from_iso8601(string, format_or_calendar \\ Calendar.ISO)

(since 1.4.0)

```
@spec from_iso8601(String.t(), Calendar.calendar() |
:extended | :basic) ::
  {:ok, t(), Calendar.utc_offset()} | {:error, atom()}
```

Parses the extended "Date and time of day" format described by ISO 8601:2019.

Since ISO 8601 does not include the proper time zone, the given string will be converted to UTC and its offset in seconds will be returned as part of this function. Therefore offset information must be present in the string.

As specified in the standard, the separator "T" may be omitted if desired as there is no ambiguity within this function.

Note leap seconds are not supported by the built-in Calendar.ISO.

Examples

```
iex> {:ok, datetime, 0} = DateTime.from_iso8601("2015-01-23T23:50:07Z")
iex> datetime
~U[2015-01-23 23:50:07Z]

iex> {:ok, datetime, 9000} = DateTime.from_iso8601("2015-01-23T23:50:07.123+02:30")
iex> datetime
~U[2015-01-23 21:20:07.123Z]

iex> {:ok, datetime, 9000} = DateTime.from_iso8601("2015-01-23T23:50:07,123+02:30")
iex> datetime
~U[2015-01-23 21:20:07.123Z]

iex> {:ok, datetime, 0} = DateTime.from_iso8601("-2015-01-23T23:50:07Z")
iex> datetime
~U[-2015-01-23 23:50:07Z]

iex> {:ok, datetime, 9000} =
DateTime.from_iso8601("-2015-01-23T23:50:07,123+02:30")
iex> datetime
~U[-2015-01-23 21:20:07.123Z]

iex> {:ok, datetime, 9000} =
DateTime.from_iso8601("20150123T235007.123+0230", :basic)
iex> datetime
~U[2015-01-23 21:20:07.123Z]

iex> DateTime.from_iso8601("2015-01-23P23:50:07")
{:error, :invalid_format}
iex> DateTime.from_iso8601("2015-01-23T23:50:07")
{:error, :missing_offset}
iex> DateTime.from_iso8601("2015-01-23 23:50:61")
```

```
{:error, :invalid_time}
iex> DateTime.from_iso8601("2015-01-32 23:50:07")
{:error, :invalid_date}
iex> DateTime.from_iso8601("2015-01-23T23:50:07.123-
00:00")
{:error, :invalid_format}
```

from_iso8601(string, calendar, format)

```
@spec from_iso8601(String.t(), Calendar.calendar(), :extended
| :basic) ::
  {:ok, t(), Calendar.utc_offset()} | {:error, atom()}
```

Converts from ISO8601 specifying both a calendar and a mode.

See `from_iso8601/2` for more information.

Examples

```
iex> {:ok, datetime, 9000} = DateTime.from_iso8601("2015-
01-23T23:50:07,123+02:30", Calendar.ISO, :extended)
iex> datetime
~U[2015-01-23 21:20:07.123Z]

iex> {:ok, datetime, 9000} =
DateTime.from_iso8601("20150123T235007.123+0230",
Calendar.ISO, :basic)
iex> datetime
~U[2015-01-23 21:20:07.123Z]
```

**from_naive(naive_datetime, time_zone, time_zone_database **

Calendar.get_time_zone_database())

(since 1.4.0)

```
@spec from_naive(
  Calendar.naive_datetime(),
  Calendar.time_zone(),
  Calendar.time_zone_database()
) ::
  {:ok, t()}
  | {:ambiguous, first_datetime :: t(), second_datetime ::
t()}
  | {:gap, t(), t()}
  | {:error,
     :incompatible_calendars
     | :time_zone_not_found
     | :utc_only_time_zone_database}
```

Converts the given `NaiveDateTime` to `DateTime`.

It expects a time zone to put the `NaiveDateTime` in. If the time zone is "Etc/UTC", it always succeeds. Otherwise, the NaiveDateTime is checked against the time zone database given as `time_zone_database`. See the "Time zone database" section in the module documentation.

Examples

```
iex> DateTime.from_naive(~N[2016-05-24 13:26:08.003],
"Etc/UTC")
{:ok, ~U[2016-05-24 13:26:08.003Z]}
```

When the datetime is ambiguous - for instance during changing from summer to winter time - the two possible valid datetimes are returned in

a tuple. The first datetime is also the one which comes first chronologically, while the second one comes last.

```
iex> {:ambiguous, first_dt, second_dt} =
DateTime.from_naive(~N[2018-10-28 02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> first_dt
#DateTime<2018-10-28 02:30:00+02:00 CEST
Europe/Copenhagen>
iex> second_dt
#DateTime<2018-10-28 02:30:00+01:00 CET
Europe/Copenhagen>
```

When there is a gap in wall time - for instance in spring when the clocks are turned forward - the latest valid datetime just before the gap and the first valid datetime just after the gap.

```
iex> {:gap, just_before, just_after} =
DateTime.from_naive(~N[2019-03-31 02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> just_before
#DateTime<2019-03-31 01:59:59.999999+01:00 CET
Europe/Copenhagen>
iex> just_after
#DateTime<2019-03-31 03:00:00+02:00 CEST
Europe/Copenhagen>
```

Most of the time there is one, and just one, valid datetime for a certain date and time in a certain time zone.

```
iex> {:ok, datetime} = DateTime.from_naive(~N[2018-07-28
12:30:00], "Europe/Copenhagen", FakeTimeZoneDatabase)
iex> datetime
```

```
#DateTime<2018-07-28 12:30:00+02:00 CEST
Europe/Copenhagen>
```

This function accepts any map or struct that contains at least the same fields as a `NaiveDateTime` struct. The most common example of that is a `DateTime`. In this case the information about the time zone of that `DateTime` is completely ignored. This is the same principle as passing a `DateTime` to `Date.to_iso8601/2`. `Date.to_iso8601/2` extracts only the date-specific fields (calendar, year, month and day) of the given structure and ignores all others.

This way if you have a `DateTime` in one time zone, you can get the same wall time in another time zone. For instance if you have 2018-08-24 10:00:00 in Copenhagen and want a `DateTime` for 2018-08-24 10:00:00 in UTC you can do:

```
iex> cph_datetime = DateTime.from_naive!(~N[2018-08-24
10:00:00], "Europe/Copenhagen", FakeTimeZoneDatabase)
iex> {:ok, utc_datetime} =
DateTime.from_naive(cph_datetime, "Etc/UTC",
FakeTimeZoneDatabase)
iex> utc_datetime
~U[2018-08-24 10:00:00Z]
```

If instead you want a `DateTime` for the same point time in a different time zone see the `DateTime.shift_zone/3` function which would convert 2018-08-24 10:00:00 in Copenhagen to 2018-08-24 08:00:00 in UTC.

**from_naive!(naive_datetime, time_zone, time_zone_database **

Calendar.get_time_zone_database())

(since 1.4.0)

```
@spec from_naive!(
  NaiveDateTime.t(),
  Calendar.time_zone(),
  Calendar.time_zone_database()
) :: t()
```

Converts the given `NaiveDateTime` to `DateTime`.

It expects a time zone to put the NaiveDateTime in. If the time zone is "Etc/UTC", it always succeeds. Otherwise, the NaiveDateTime is checked against the time zone database given as `time_zone_database`. See the "Time zone database" section in the module documentation.

Examples

```
iex> DateTime.from_naive!(~N[2016-05-24 13:26:08.003],
"Etc/UTC")
~U[2016-05-24 13:26:08.003Z]

iex> DateTime.from_naive!(~N[2018-05-24 13:26:08.003],
"Europe/Copenhagen", FakeTimeZoneDatabase)
#DateTime<2018-05-24 13:26:08.003+02:00 CEST
Europe/Copenhagen>
```

from_unix(integer, unit \\ :second, calendar \\ Calendar.ISO)

```
@spec from_unix(integer(), :native | System.time_unit(),
Calendar.calendar()) ::
  {:ok, t()} | {:error, atom()}
```

Converts the given Unix time to `DateTime`.

The integer can be given in different unit, according to `System.convert_time_unit/3`, and it will be converted to microseconds internally, which is the maximum precision supported by `DateTime`. In other words, any precision higher than microseconds will lead to truncation.

Unix times are always in UTC. Therefore the DateTime will be returned in UTC.

Examples

```
iex> {:ok, datetime} = DateTime.from_unix(1_464_096_368)
iex> datetime
~U[2016-05-24 13:26:08Z]

iex> {:ok, datetime} =
DateTime.from_unix(1_432_560_368_868_569, :microsecond)
iex> datetime
~U[2015-05-25 13:26:08.868569Z]

iex> {:ok, datetime} =
DateTime.from_unix(253_402_300_799)
iex> datetime
~U[9999-12-31 23:59:59Z]

iex> {:error, :invalid_unix_time} =
DateTime.from_unix(253_402_300_800)
```

The unit can also be an integer as in `System.time_unit/0`:

```
iex> {:ok, datetime} =
DateTime.from_unix(143_256_036_886_856, 1024)
```

```
iex> datetime
~U[6403-03-17 07:05:22.320312Z]
```

Negative Unix times are supported up to -377705116800 seconds:

```
iex> {:ok, datetime} =
DateTime.from_unix(-377_705_116_800)
iex> datetime
~U[-9999-01-01 00:00:00Z]

iex> {:error, :invalid_unix_time} =
DateTime.from_unix(-377_705_116_801)
```

from_unix!(integer, unit \\ :second, calendar \\ Calendar.ISO)

```
@spec from_unix!(integer(), :native | System.time_unit(),
Calendar.calendar()) :: t()
```

Converts the given Unix time to `DateTime`.

The integer can be given in different unit according to `System.convert_time_unit/3` and it will be converted to microseconds internally.

Unix times are always in UTC and therefore the DateTime will be returned in UTC.

Examples

```
# An easy way to get the Unix epoch is passing 0 to this
function
iex> DateTime.from_unix!(0)
```

```
~U[1970-01-01 00:00:00Z]

iex> DateTime.from_unix!(1_464_096_368)
~U[2016-05-24 13:26:08Z]

iex> DateTime.from_unix!(1_432_560_368_868_569,
:microsecond)
~U[2015-05-25 13:26:08.868569Z]

iex> DateTime.from_unix!(143_256_036_886_856, 1024)
~U[6403-03-17 07:05:22.320312Z]
```

new(date, time, time_zone \\ "Etc/UTC", time_zone_database \\ Calendar.get_time_zone_database())

(since 1.11.0)

```
@spec new(Date.t(), Time.t(), Calendar.time_zone(),
Calendar.time_zone_database()) ::
  {:ok, t()}
  | {:ambiguous, first_datetime :: t(), second_datetime ::
t()}
  | {:gap, t(), t()}
  | {:error,
     :incompatible_calendars
     | :time_zone_not_found
     | :utc_only_time_zone_database}
```

Builds a datetime from date and time structs.

It expects a time zone to put the `DateTime` in. If the time zone is not passed it will default to `"Etc/UTC"`, which always succeeds. Otherwise, the `DateTime` is checked against the time zone database given as

`time_zone_database`. See the "Time zone database" section in the module documentation.

Examples

```
iex> DateTime.new(~D[2016-05-24], ~T[13:26:08.003],
"Etc/UTC")
{:ok, ~U[2016-05-24 13:26:08.003Z]}
```

When the datetime is ambiguous - for instance during changing from summer to winter time - the two possible valid datetimes are returned in a tuple. The first datetime is also the one which comes first chronologically, while the second one comes last.

```
iex> {:ambiguous, first_dt, second_dt} =
DateTime.new(~D[2018-10-28], ~T[02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> first_dt
#DateTime<2018-10-28 02:30:00+02:00 CEST
Europe/Copenhagen>
iex> second_dt
#DateTime<2018-10-28 02:30:00+01:00 CET
Europe/Copenhagen>
```

When there is a gap in wall time - for instance in spring when the clocks are turned forward - the latest valid datetime just before the gap and the first valid datetime just after the gap.

```
iex> {:gap, just_before, just_after} =
DateTime.new(~D[2019-03-31], ~T[02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
iex> just_before
```

```
#DateTime<2019-03-31 01:59:59.999999+01:00 CET
Europe/Copenhagen>
iex> just_after
#DateTime<2019-03-31 03:00:00+02:00 CEST
Europe/Copenhagen>
```

Most of the time there is one, and just one, valid datetime for a certain date and time in a certain time zone.

```
iex> {:ok, datetime} = DateTime.new(~D[2018-07-28],
~T[12:30:00], "Europe/Copenhagen", FakeTimeZoneDatabase)
iex> datetime
#DateTime<2018-07-28 12:30:00+02:00 CEST
Europe/Copenhagen>
```

new!(date, time, time_zone \\ "Etc/UTC", time_zone_database \\ Calendar.get_time_zone_database())

(since 1.11.0)

```
@spec new!(Date.t(), Time.t(), Calendar.time_zone(),
Calendar.time_zone_database()) ::
  t()
```

Builds a datetime from date and time structs, raising on errors.

It expects a time zone to put the `DateTime` in. If the time zone is not passed it will default to `"Etc/UTC"`, which always succeeds. Otherwise, the DateTime is checked against the time zone database given as `time_zone_database`. See the "Time zone database" section in the module documentation.

Examples

```
iex> DateTime.new!(~D[2016-05-24], ~T[13:26:08.003],
"Etc/UTC")
~U[2016-05-24 13:26:08.003Z]
```

When the datetime is ambiguous - for instance during changing from summer to winter time - an error will be raised.

```
iex> DateTime.new!(~D[2018-10-28], ~T[02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
** (ArgumentError) cannot build datetime with ~D[2018-10-
28] and ~T[02:30:00] because such instant is ambiguous in
time zone Europe/Copenhagen as there is an overlap
between #DateTime<2018-10-28 02:30:00+02:00 CEST
Europe/Copenhagen> and #DateTime<2018-10-28
02:30:00+01:00 CET Europe/Copenhagen>
```

When there is a gap in wall time - for instance in spring when the clocks are turned forward - an error will be raised.

```
iex> DateTime.new!(~D[2019-03-31], ~T[02:30:00],
"Europe/Copenhagen", FakeTimeZoneDatabase)
** (ArgumentError) cannot build datetime with ~D[2019-03-
31] and ~T[02:30:00] because such instant does not exist
in time zone Europe/Copenhagen as there is a gap between
#DateTime<2019-03-31 01:59:59.999999+01:00 CET
Europe/Copenhagen> and #DateTime<2019-03-31
03:00:00+02:00 CEST Europe/Copenhagen>
```

Most of the time there is one, and just one, valid datetime for a certain date and time in a certain time zone.

```
iex> datetime = DateTime.new!(~D[2018-07-28],
~T[12:30:00], "Europe/Copenhagen", FakeTimeZoneDatabase)
iex> datetime
#DateTime<2018-07-28 12:30:00+02:00 CEST
Europe/Copenhagen>
```

now(time_zone, time_zone_database \\ Calendar.get_time_zone_database())

(since 1.8.0)

```
@spec now(Calendar.time_zone(),
Calendar.time_zone_database()) ::
  {:ok, t()} | {:error, :time_zone_not_found |
:utc_only_time_zone_database}
```

Returns the current datetime in the provided time zone.

By default, it uses the default time_zone returned by `Calendar.get_time_zone_database/0`, which defaults to `Calendar.UTCOnlyTimeZoneDatabase` which only handles "Etc/UTC" datetimes. Other time zone databases can be passed as argument or set globally. See the "Time zone database" section in the module docs.

Examples

```
iex> {:ok, datetime} = DateTime.now("Etc/UTC")
iex> datetime.time_zone
"Etc/UTC"

iex> DateTime.now("Europe/Copenhagen")
{:error, :utc_only_time_zone_database}
```

```
iex> DateTime.now("bad timezone", FakeTimeZoneDatabase)
{:error, :time_zone_not_found}
```

now!(time_zone, time_zone_database \\ Calendar.get_time_zone_database())

(since 1.10.0)

```
@spec now!(Calendar.time_zone(),
Calendar.time_zone_database()) :: t()
```

Returns the current datetime in the provided time zone or raises on errors

See `now/2` for more information.

Examples

```
iex> datetime = DateTime.now!("Etc/UTC")
iex> datetime.time_zone
"Etc/UTC"

iex> DateTime.now!("Europe/Copenhagen")
** (ArgumentError) cannot get current datetime in
"Europe/Copenhagen" time zone, reason:
:utc_only_time_zone_database

iex> DateTime.now!("bad timezone", FakeTimeZoneDatabase)
** (ArgumentError) cannot get current datetime in "bad
timezone" time zone, reason: :time_zone_not_found
```

shift(datetime, duration, time_zone_database \\ Calendar.get_time_zone_database())

(since 1.17.0)

```
@spec shift(Calendar.datetime(), Duration.duration(),
Calendar.time_zone_database()) ::
  t()
```

Shifts given `datetime` by `duration` according to its calendar.

Allowed units are: `:year`, `:month`, `:week`, `:day`, `:hour`, `:minute`, `:second`, `:microsecond`.

This operation is equivalent to shifting the datetime wall clock (in other words, the value as someone in that timezone would see on their watch), then applying the time zone offset to convert it to UTC, and finally computing the new timezone in case of shifts. This ensures `shift/3` always returns a valid datetime.

Consequently, time zones that observe "Daylight Saving Time" or other changes, across summer/winter time will add/remove hours from the resulting datetime:

```
dt = DateTime.new!(~D[2019-03-31], ~T[01:00:00],
"Europe/Copenhagen")
DateTime.shift(dt, hour: 1)
#=> #DateTime<2019-03-31 03:00:00+02:00 CEST
Europe/Copenhagen>

dt = DateTime.new!(~D[2018-11-04], ~T[00:00:00],
"America/Los_Angeles")
DateTime.shift(dt, hour: 2)
#=> #DateTime<2018-11-04 01:00:00-08:00 PST
America/Los_Angeles>
```

Although the first example shows a difference of 2 hours when comparing the wall clocks of the given datetime with the returned one, due to the "spring forward" time jump, the actual elapsed time is still exactly of 1 hour.

In case you don't want these changes to happen automatically or you want to surface time zone conflicts to the user, you can shift the datetime as a naive datetime and then use `from_naive/2`:

```
dt |> NaiveDateTime.shift(duration) |>
DateTime.from_naive(dt.time_zone)
```

The above will surface time jumps and ambiguous datetimes, allowing you to deal with them accordingly.

ISO calendar considerations

When using the default ISO calendar, durations are collapsed and applied in the order of months, then seconds and microseconds:

- when shifting by 1 year and 2 months the date is actually shifted by 14 months
- weeks, days and smaller units are collapsed into seconds and microseconds

When shifting by month, days are rounded down to the nearest valid date.

Examples

```
iex> DateTime.shift(~U[2016-01-01 00:00:00Z], month: 2)
~U[2016-03-01 00:00:00Z]
iex> DateTime.shift(~U[2016-01-01 00:00:00Z], year: 1,
```

```
  week: 4)
  ~U[2017-01-29 00:00:00Z]
  iex> DateTime.shift(~U[2016-01-01 00:00:00Z], minute:
  -25)
  ~U[2015-12-31 23:35:00Z]
  iex> DateTime.shift(~U[2016-01-01 00:00:00Z], minute: 5,
  microsecond: {500, 4})
  ~U[2016-01-01 00:05:00.0005Z]

  # leap years
  iex> DateTime.shift(~U[2024-02-29 00:00:00Z], year: 1)
  ~U[2025-02-28 00:00:00Z]
  iex> DateTime.shift(~U[2024-02-29 00:00:00Z], year: 4)
  ~U[2028-02-29 00:00:00Z]

  # rounding down
  iex> DateTime.shift(~U[2015-01-31 00:00:00Z], month: 1)
  ~U[2015-02-28 00:00:00Z]
```

**shift_zone(datetime, time_zone, time_zone_database **

Calendar.get_time_zone_database())

(since 1.8.0)

```
@spec shift_zone(t(), Calendar.time_zone(),
Calendar.time_zone_database()) ::
  {:ok, t()} | {:error, :time_zone_not_found |
:utc_only_time_zone_database}
```

Changes the time zone of a `DateTime`.

Returns a `DateTime` for the same point in time, but instead at the time zone provided. It assumes that `DateTime` is valid and exists in the given time zone and calendar.

By default, it uses the default time zone database returned by `Calendar.get_time_zone_database/0`, which defaults to `Calendar.UTCOnlyTimeZoneDatabase` which only handles "Etc/UTC" datetimes. Other time zone databases can be passed as argument or set globally. See the "Time zone database" section in the module docs.

Examples

```
iex> {:ok, pacific_datetime} =
DateTime.shift_zone(~U[2018-07-16 10:00:00Z],
"America/Los_Angeles", FakeTimeZoneDatabase)
iex> pacific_datetime
#DateTime<2018-07-16 03:00:00-07:00 PDT
America/Los_Angeles>

iex> DateTime.shift_zone(~U[2018-07-16 10:00:00Z], "bad
timezone", FakeTimeZoneDatabase)
{:error, :time_zone_not_found}
```

shift_zone!(datetime, time_zone, time_zone_database \\ Calendar.get_time_zone_database())

(since 1.10.0)

```
@spec shift_zone!(t(), Calendar.time_zone(),
Calendar.time_zone_database()) :: t()
```

Changes the time zone of a `DateTime` or raises on errors.

See `shift_zone/3` for more information.

Examples

```
iex> DateTime.shift_zone!(~U[2018-07-16 10:00:00Z],
"America/Los_Angeles", FakeTimeZoneDatabase)
#DateTime<2018-07-16 03:00:00-07:00 PDT
America/Los_Angeles>

iex> DateTime.shift_zone!(~U[2018-07-16 10:00:00Z], "bad
timezone", FakeTimeZoneDatabase)
** (ArgumentError) cannot shift ~U[2018-07-16 10:00:00Z]
to "bad timezone" time zone, reason: :time_zone_not_found
```

to_date(datetime)

```
@spec to_date(Calendar.datetime()) :: Date.t()
```

Converts a `DateTime` into a `Date`.

Because `Date` does not hold time nor time zone information, data will be lost during the conversion.

Examples

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.to_date(dt)
~D[2000-02-29]
```

to_gregorian_seconds(datetime)

(since 1.11.0)

```
@spec to_gregorian_seconds(Calendar.datetime()) ::
{integer(), non neg integer()}
```

Converts a `DateTime` struct to a number of gregorian seconds and microseconds.

Examples

```
iex> dt = %DateTime{year: 0000, month: 1, day: 1,
zone_abbr: "UTC",
...>                hour: 0, minute: 0, second: 1,
microsecond: {0, 0},
...>                utc_offset: 0, std_offset: 0,
time_zone: "Etc/UTC"}
iex> DateTime.to_gregorian_seconds(dt)
{1, 0}

iex> dt = %DateTime{year: 2020, month: 5, day: 1,
zone_abbr: "UTC",
...>                hour: 0, minute: 26, second: 31,
microsecond: {5000, 0},
...>                utc_offset: 0, std_offset: 0,
time_zone: "Etc/UTC"}
iex> DateTime.to_gregorian_seconds(dt)
{63_755_511_991, 5000}

iex> dt = %DateTime{year: 2020, month: 5, day: 1,
zone_abbr: "CET",
...>                hour: 1, minute: 26, second: 31,
microsecond: {5000, 0},
```

```
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.to_gregorian_seconds(dt)
{63_755_511_991, 5000}
```

to_iso8601(datetime, format \\ :extended, offset \\ nil)

```
@spec to_iso8601(Calendar.datetime(), :basic | :extended, nil
| integer()) ::
  String.t()
```

Converts the given datetime to ISO 8601:2019 format.

By default, `DateTime.to_iso8601/2` returns datetimes formatted in the "extended" format, for human readability. It also supports the "basic" format through passing the `:basic` option.

You can also optionally specify an offset for the formatted string. If none is given, the one in the given `datetime` is used.

Only supports converting datetimes which are in the ISO calendar. If another calendar is given, it is automatically converted to ISO. It raises if not possible.

WARNING: the ISO 8601 datetime format does not contain the time zone nor its abbreviation, which means information is lost when converting to such format.

Examples

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
```

```
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.to_iso8601(dt)
"2000-02-29T23:00:07+01:00"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "UTC",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 0, std_offset: 0,
time_zone: "Etc/UTC"}
iex> DateTime.to_iso8601(dt)
"2000-02-29T23:00:07Z"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> DateTime.to_iso8601(dt, :extended)
"2000-02-29T23:00:07-04:00"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> DateTime.to_iso8601(dt, :basic)
"20000229T230007-0400"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                hour: 23, minute: 0, second: 7,
```

```
microsecond: {0, 0},
...>                utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> DateTime.to_iso8601(dt, :extended, 3600)
"2000-03-01T04:00:07+01:00"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> DateTime.to_iso8601(dt, :extended, 0)
"2000-03-01T03:00:07+00:00"

iex> dt = %DateTime{year: 2000, month: 3, day: 01,
zone_abbr: "UTC",
...>                hour: 03, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 0, std_offset: 0,
time_zone: "Etc/UTC"}
iex> DateTime.to_iso8601(dt, :extended, 0)
"2000-03-01T03:00:07Z"

iex> {:ok, dt, offset} = DateTime.from_iso8601("2000-03-
01T03:00:07Z")
iex> "2000-03-01T03:00:07Z" = DateTime.to_iso8601(dt,
:extended, offset)
```

to_naive(datetime)

```
@spec to_naive(Calendar.datetime()) :: NaiveDateTime.t()
```

Converts the given `datetime` into a `NaiveDateTime`.

Because `NaiveDateTime` does not hold time zone information, any time zone related data will be lost during the conversion.

Examples

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 1},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.to_naive(dt)
~N[2000-02-29 23:00:07.0]
```

to_string(datetime)

```
@spec to_string(Calendar.datetime()) :: String.t()
```

Converts the given `datetime` to a string according to its calendar.

Unfortunately, there is no standard that specifies rendering of a datetime with its complete time zone information, so Elixir uses a custom (but relatively common) representation which appends the time zone abbreviation and full name to the datetime.

Examples

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
```

```
time_zone: "Europe/Warsaw"}
iex> DateTime.to_string(dt)
"2000-02-29 23:00:07+01:00 CET Europe/Warsaw"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "UTC",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 0, std_offset: 0,
time_zone: "Etc/UTC"}
iex> DateTime.to_string(dt)
"2000-02-29 23:00:07Z"

iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "AMT",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: -14400, std_offset: 0,
time_zone: "America/Manaus"}
iex> DateTime.to_string(dt)
"2000-02-29 23:00:07-04:00 AMT America/Manaus"

iex> dt = %DateTime{year: -100, month: 12, day: 19,
zone_abbr: "CET",
...>                hour: 3, minute: 20, second: 31,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Stockholm"}
iex> DateTime.to_string(dt)
"-0100-12-19 03:20:31+01:00 CET Europe/Stockholm"
```

to_time(datetime)

```
@spec to_time(Calendar.datetime()) :: Time.t()
```

Converts a `DateTime` into `Time`.

Because `Time` does not hold date nor time zone information, data will be lost during the conversion.

Examples

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 1},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> DateTime.to_time(dt)
~T[23:00:07.0]
```

to_unix(datetime, unit \\ :second)

```
@spec to_unix(Calendar.datetime(), :native |
System.time_unit()) :: integer()
```

Converts the given `datetime` to Unix time.

The `datetime` is expected to be using the ISO calendar with a year greater than or equal to 0.

It will return the integer with the given unit, according to `System.convert_time_unit/3`. If the given unit is different than microseconds, the returned value will be either truncated or padded accordingly.

Examples

```
iex> 1_464_096_368 |> DateTime.from_unix!() |>
DateTime.to_unix()
1464096368

iex> dt = %DateTime{calendar: Calendar.ISO, day: 20,
hour: 18, microsecond: {273806, 6},
...>                minute: 58, month: 11, second: 19,
time_zone: "America/Montevideo",
...>                utc_offset: -10800, std_offset: 3600,
year: 2014, zone_abbr: "UYST"}
iex> DateTime.to_unix(dt)
1416517099

iex> flamel = %DateTime{calendar: Calendar.ISO, day: 22,
hour: 8, microsecond: {527771, 6},
...>                minute: 2, month: 3, second: 25,
std_offset: 0, time_zone: "Etc/UTC",
...>                utc_offset: 0, year: 1418, zone_abbr:
"UTC"}
iex> DateTime.to_unix(flamel)
-17412508655
```

truncate(datetime, precision)

(since 1.6.0)

```
@spec truncate(Calendar.datetime(), :microsecond |
:millisecond | :second) :: t()
```

Returns the given datetime with the microsecond field truncated to the given precision (`:microsecond`, `:millisecond` or `:second`).

The given datetime is returned unchanged if it already has lower precision than the given precision.

Examples

```
iex> dt1 = %DateTime{year: 2017, month: 11, day: 7,
zone_abbr: "CET",
...>                 hour: 11, minute: 45, second: 18,
microsecond: {123456, 6},
...>                 utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Paris"}
iex> DateTime.truncate(dt1, :microsecond)
#DateTime<2017-11-07 11:45:18.123456+01:00 CET
Europe/Paris>

iex> dt2 = %DateTime{year: 2017, month: 11, day: 7,
zone_abbr: "CET",
...>                 hour: 11, minute: 45, second: 18,
microsecond: {123456, 6},
...>                 utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Paris"}
iex> DateTime.truncate(dt2, :millisecond)
#DateTime<2017-11-07 11:45:18.123+01:00 CET Europe/Paris>

iex> dt3 = %DateTime{year: 2017, month: 11, day: 7,
zone_abbr: "CET",
...>                 hour: 11, minute: 45, second: 18,
microsecond: {123456, 6},
...>                 utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Paris"}
iex> DateTime.truncate(dt3, :second)
#DateTime<2017-11-07 11:45:18+01:00 CET Europe/Paris>
```

utc_now(calendar_or_time_unit \\ Calendar.ISO)

```
@spec utc_now(Calendar.calendar() | :native | :microsecond |
:millisecond | :second) ::
  t()
```

Returns the current datetime in UTC.

If you want the current time in Unix seconds, use `System.os_time/1` instead.

You can also pass a time unit to automatically truncate the resulting datetime. This is available since v1.15.0.

The default unit if none gets passed is `:native`, which results on a default resolution of microseconds.

Examples

```
iex> datetime = DateTime.utc_now()
iex> datetime.time_zone
"Etc/UTC"

iex> datetime = DateTime.utc_now(:second)
iex> datetime.microsecond
{0, 0}
```

utc_now(time_unit, calendar)

(since 1.15.0)

```
@spec utc_now(:native | :microsecond | :millisecond |
:second, Calendar.calendar()) ::
  t()
```

Returns the current datetime in UTC, supporting a specific calendar and precision.

If you want the current time in Unix seconds, use `System.os_time/1` instead.

Examples

```
iex> datetime = DateTime.utc_now(:microsecond,
Calendar.ISO)
iex> datetime.time_zone
"Etc/UTC"

iex> datetime = DateTime.utc_now(:second, Calendar.ISO)
iex> datetime.microsecond
{0, 0}
```

Duration

Struct and functions for handling durations.

A `Duration` struct represents a collection of time scale units, allowing for manipulation and calculation of durations.

Date and time scale units are represented as integers, allowing for both positive and negative values.

Microseconds are represented using a tuple `{microsecond, precision}`. This ensures compatibility with other calendar types implementing time, such as `Time`, `DateTime`, and `NaiveDateTime`.

Shifting

The most common use of durations in Elixir's standard library is to "shift" the calendar types.

```
iex> Date.shift(~D[2016-01-03], month: 2)
~D[2016-03-03]
```

In the example above, `Date.shift/2` automatically converts the units into a `Duration` struct, although one can also be given directly:

```
iex> Date.shift(~D[2016-01-03], Duration.new!(month: 2))
~D[2016-03-03]
```

It is important to note that shifting is not an arithmetic operation. For example, adding `date + 1 month + 1 month` does not yield the same

result as `date + 2 months`. Let's see an example:

```
iex> ~D[2016-01-31] |> Date.shift(month: 1) |>
Date.shift(month: 1)
~D[2016-03-29]

iex> ~D[2016-01-31] |> Date.shift(month: 2)
~D[2016-03-31]
```

As you can see above, the results differ, which explains why operations with durations are called "shift" rather than "add". This happens because, once we add one month to `2016-01-31`, we get `2016-02-29`. Then adding one extra month gives us `2016-03-29` instead of `2016-03-31`.

In particular, when applying durations to `Calendar.ISO` types:

- larger units (such as years and months) are applied before smaller ones (such as weeks, hours, days, and so on)
- units are collapsed into months (`:year` and `:month`), seconds (`:week`, `:day`, `:hour`, `:minute`, `:second`) and microseconds (`:microsecond`) before they are applied
- 1 year is equivalent to 12 months, 1 week is equivalent to 7 days. Therefore, 4 weeks *are not* equivalent to 1 month
- in case of non-existing dates, the results are rounded down to the nearest valid date

As the `shift/2` functions are calendar aware, they are guaranteed to return valid date/times, considering leap years as well as DST in applicable time zones.

Intervals

Durations in Elixir can be combined with stream operations to build intervals. For example, to retrieve the next three Wednesdays starting from 17th April, 2024:

```
iex> ~D[2024-04-17] |> Stream.iterate(&Date.shift(&1,
week: 1)) |> Enum.take(3)
[~D[2024-04-17], ~D[2024-04-24], ~D[2024-05-01]]
```

However, once again, it is important to remember that shifting a duration is not arithmetic, so you may want to use the functions in this module depending on what you to achieve. Compare the results of both examples below:

```
# Adding one month after the other
iex> date = ~D[2016-01-31]
iex> duration = Duration.new!(month: 1)
iex> stream = Stream.iterate(date, fn prev_date ->
Date.shift(prev_date, duration) end)
iex> Enum.take(stream, 3)
[~D[2016-01-31], ~D[2016-02-29], ~D[2016-03-29]]

# Multiplying durations by an index
iex> date = ~D[2016-01-31]
iex> duration = Duration.new!(month: 1)
iex> stream = Stream.from_index(fn i -> Date.shift(date,
Duration.multiply(duration, i)) end)
iex> Enum.take(stream, 3)
[~D[2016-01-31], ~D[2016-02-29], ~D[2016-03-31]]
```

The second example consistently points to the last day of the month, as it performs operations on the duration, rather than shifting date after date.

Comparing durations

In order to accurately compare durations, you need to either compare only certain fields or use a reference time instant. This is because some fields are relative to others. For example, you may say that 1 month is the same as 30 days, but if you add both of these durations to `~D[2015-02-01]`, you would get different results, as that month has only 28 days.

Therefore, if you wish to compare durations, one option is to use `Date.shift/2` (or `DateTime.shift/2` or similar), and then compare the dates:

```
iex> date = ~D[2015-02-01]
iex> Date.compare(Date.shift(date, month: 1),
Date.shift(date, day: 30))
:lt
```

Or alternatively convert the durations to a fixed unit by using `to_timeout/1`, which supports durations only up to weeks, raising if it has the month or year fields set.

```
iex> to_timeout(hour: 24) == to_timeout(day: 1)
true
```

Summary

Types

duration()

The duration type specifies a `%Duration{}` struct or a keyword list of valid duration unit pairs.

t()

The duration struct type.

to_string_opts()

Options for `Duration.to_string/2`.

unit_pair()

The unit pair type specifies a pair of a valid duration unit key and value.

Functions

add(d1, d2)

Adds units of given durations `d1` and `d2`.

from_iso8601(string)

Parses an ISO 8601 formatted duration string to a `Duration` struct.

from_iso8601!(string)

Same as `from_iso8601/1` but raises an `ArgumentError`.

multiply(duration, integer)

Multiplies `duration` units by given `integer`.

negate(duration)

Negates `duration` units.

new!(duration)

Creates a new `Duration` struct from given `unit_pairs`.

subtract(d1, d2)

Subtracts units of given durations `d1` and `d2`.

to_iso8601(duration)

Converts the given `duration` to an ISO 8601-2:2019 formatted string.

to_string(duration, opts \\ [])

Converts the given `duration` to a human readable representation.

Types

duration()

(since 1.17.0)

```
@type duration() :: t() | [unit_pair()]
```

The duration type specifies a `%Duration{}` struct or a keyword list of valid duration unit pairs.

t()

(since 1.17.0)

```
@type t() :: %Duration{
  day: integer(),
  hour: integer(),
  microsecond: Calendar.microsecond(),
  minute: integer(),
  month: integer(),
  second: integer(),
```

```
  week: integer(),
  year: integer()
}
```

The duration struct type.

to_string_opts()

(since 1.17.0)

```
@type to_string_opts() :: [
  units: [
    year: String.t(),
    month: String.t(),
    week: String.t(),
    day: String.t(),
    hour: String.t(),
    minute: String.t(),
    second: String.t()
  ],
  separator: String.t()
]
```

Options for `Duration.to_string/2`.

unit_pair()

(since 1.17.0)

```
@type unit_pair() ::
  {:year, integer()}
  | {:month, integer()}
  | {:week, integer()}
  | {:day, integer()}
```

```
| {:hour, integer()}
| {:minute, integer()}
| {:second, integer()}
| {:microsecond, Calendar.microsecond()}
```

The unit pair type specifies a pair of a valid duration unit key and value.

Functions

add(d1, d2)

(since 1.17.0)

```
@spec add(t(), t()) :: t()
```

Adds units of given durations `d1` and `d2`.

Respects the highest microsecond precision of the two.

Examples

```
iex> Duration.add(Duration.new!(week: 2, day: 1),
Duration.new!(day: 2))
%Duration{week: 2, day: 3}
iex> Duration.add(Duration.new!(microsecond: {400, 3}),
Duration.new!(microsecond: {600, 6}))
%Duration{microsecond: {1000, 6}}
```

from_iso8601(string)

(since 1.17.0)

```
@spec from_iso8601(String.t()) :: {:ok, t()} | {:error,
atom()}
```

Parses an ISO 8601 formatted duration string to a `Duration` struct.

Duration strings, as well as individual units, may be prefixed with plus/minus signs so that:

- `-PT6H3M` parses as `%Duration{hour: -6, minute: -3}`
- `-PT6H-3M` parses as `%Duration{hour: -6, minute: 3}`
- `+PT6H3M` parses as `%Duration{hour: 6, minute: 3}`
- `+PT6H-3M` parses as `%Duration{hour: 6, minute: -3}`

Duration designators must be provided in order of magnitude: `P[n]Y[n]M[n]W[n]DT[n]H[n]M[n]S`.

Only seconds may be specified with a decimal fraction, using either a comma or a full stop: `P1DT4,5S`.

Inputs longer than 192 bytes are rejected, which is enough to parse each unit as a maximum signed 64-bit integer (although it also allows parsing a single unit with 190 digits).

Examples

```
iex> Duration.from_iso8601("P1Y2M3DT4H5M6S")
{:ok, %Duration{year: 1, month: 2, day: 3, hour: 4,
minute: 5, second: 6}}
iex> Duration.from_iso8601("P3Y-2MT3H")
{:ok, %Duration{year: 3, month: -2, hour: 3}}
iex> Duration.from_iso8601("-PT10H-30M")
{:ok, %Duration{hour: -10, minute: 30}}
```

```
iex> Duration.from_iso8601("PT4.650S")
{:ok, %Duration{second: 4, microsecond: {650000, 3}}}
```

from_iso8601!(string)

(since 1.17.0)

```
@spec from_iso8601!(String.t()) :: t()
```

Same as `from_iso8601/1` but raises an `ArgumentError`.

Examples

```
iex> Duration.from_iso8601!("P1Y2M3DT4H5M6S")
%Duration{year: 1, month: 2, day: 3, hour: 4, minute: 5,
second: 6}
iex> Duration.from_iso8601!("P10D")
%Duration{day: 10}
```

multiply(duration, integer)

(since 1.17.0)

```
@spec multiply(t(), integer()) :: t()
```

Multiplies `duration` units by given `integer`.

Examples

```
iex> Duration.multiply(Duration.new!(day: 1, minute: 15,
second: -10), 3)
%Duration{day: 3, minute: 45, second: -30}
iex> Duration.multiply(Duration.new!(microsecond: {200,
4}), 3)
%Duration{microsecond: {600, 4}}
```

negate(duration)

(since 1.17.0)

```
@spec negate(t()) :: t()
```

Negates `duration` units.

Examples

```
iex> Duration.negate(Duration.new!(day: 1, minute: 15,
second: -10))
%Duration{day: -1, minute: -15, second: 10}
iex> Duration.negate(Duration.new!(microsecond: {500000,
4}))
%Duration{microsecond: {-500000, 4}}
```

new!(duration)

(since 1.17.0)

```
@spec new!(duration()) :: t()
```

Creates a new `Duration` struct from given `unit_pairs`.

Raises an `ArgumentError` when called with invalid unit pairs.

Examples

```
iex> Duration.new!(year: 1, week: 3, hour: 4, second: 1)
%Duration{year: 1, week: 3, hour: 4, second: 1}
iex> Duration.new!(second: 1, microsecond: {1000, 6})
%Duration{second: 1, microsecond: {1000, 6}}
iex> Duration.new!(month: 2)
%Duration{month: 2}
```

subtract(d1, d2)

(since 1.17.0)

```
@spec subtract(t(), t()) :: t()
```

Subtracts units of given durations `d1` and `d2`.

Respects the highest microsecond precision of the two.

Examples

```
iex> Duration.subtract(Duration.new!(week: 2, day: 1),
Duration.new!(day: 2))
%Duration{week: 2, day: -1}
iex> Duration.subtract(Duration.new!(microsecond: {400,
6}), Duration.new!(microsecond: {600, 3}))
%Duration{microsecond: {-200, 6}}
```

to_iso8601(duration)

(since 1.17.0)

```
@spec to_iso8601(t()) :: String.t()
```

Converts the given `duration` to an ISO 8601-2:2019 formatted string.

This function implements the extension of ISO 8601:2019, allowing weeks to appear between months and days: `P3M3W3D`.

Examples

```
iex> Duration.to_iso8601(Duration.new!(year: 3))
"P3Y"
iex> Duration.to_iso8601(Duration.new!(day: 40, hour: 12,
minute: 42, second: 12))
"P40DT12H42M12S"
iex> Duration.to_iso8601(Duration.new!(second: 30))
"PT30S"

iex> Duration.to_iso8601(Duration.new!([]))
"PT0S"

iex> Duration.to_iso8601(Duration.new!(second: 1,
microsecond: {2_200, 3}))
"PT1.002S"
iex> Duration.to_iso8601(Duration.new!(second: 1,
microsecond: {-1_200_000, 4}))
"PT-0.2000S"
```

to_string(duration, opts \\ [])

(since 1.18.0)

```
@spec to_string(t(), to_string_opts()) :: String.t()
```

Converts the given `duration` to a human readable representation.

Options

- `:units` - the units to be used alongside each duration component. The default units follow the ISO 80000-3 standard:

```
[
  year: "a",
  month: "mo",
  week: "wk",
  day: "d",
  hour: "h",
  minute: "min",
  second: "s"
]
```

- `:separator` - a string used to separate the distinct components. Defaults to `" "`.

Examples

```
iex> Duration.to_string(Duration.new!(second: 30))
"30s"
iex> Duration.to_string(Duration.new!(day: 40, hour: 12,
minute: 42, second: 12))
"40d 12h 42min 12s"
```

By default, this function uses ISO 80000-3 units, which uses "a" for years. But you can customize all units via the units option:

```
iex> Duration.to_string(Duration.new!(year: 3))
"3a"
iex> Duration.to_string(Duration.new!(year: 3), units:
[year: "y"])
"3y"
```

You may also choose the separator:

```
iex> Duration.to_string(Duration.new!(day: 40, hour: 12,
minute: 42, second: 12), separator: ", ")
"40d, 12h, 42min, 12s"
```

A duration without components is rendered as "0s":

```
iex> Duration.to_string(Duration.new!([]))
"0s"
```

Microseconds are rendered as part of seconds with the appropriate precision:

```
iex> Duration.to_string(Duration.new!(second: 1,
microsecond: {2_200, 3}))
"1.002s"
iex> Duration.to_string(Duration.new!(second: 1,
microsecond: {-1_200_000, 4}))
"-0.2000s"
```

Exception behaviour

Functions for dealing with throw/catch/exit and exceptions.

This module also defines the behaviour required by custom exceptions. To define your own, see `defexception/1`.

Formatting functions

Several functions in this module help format exceptions. Some of these functions expect the stacktrace as argument. The stacktrace is typically available inside catch and rescue by using the `__STACKTRACE__/0` variable.

Do not rely on the particular format returned by the functions in this module. They may be changed in future releases in order to better suit Elixir's tool chain. In other words, by using the functions in this module it is guaranteed you will format exceptions as in the current Elixir version being used.

Summary

Types

arity_or_args()
kind()

The kind handled by formatting functions

location()
non_error_kind()

stacktrace()
stacktrace_entry()
t()

The exception type

Callbacks

blame(t, stacktrace)

Called from `Exception.blame/3` to augment the exception struct.

exception(term)

Receives the arguments given to `raise/2` and returns the exception struct.

message(t)

Receives the exception struct and must return its message.

Functions

blame(kind, error, stacktrace)

Attaches information to throws/errors/exits for extra debugging.

blame_mfa(module, function, args)

Blames the invocation of the given module, function and arguments.

format(kind, payload, stacktrace \\ [])

Normalizes and formats throws/errors/exits and stacktraces.

format_banner(kind, exception, stacktrace \\ [])

Normalizes and formats any throw/error/exit.

format_exit(reason)

Formats an exit. It returns a string.

format_fa(fun, arity)

Receives an anonymous function and arity and formats it as shown in stacktraces. The arity may also be a list of arguments.

format_file_line(file, line, suffix \\ "")

Formats the given `file` and `line` as shown in stacktraces.

format_file_line_column(file, line, column, suffix \\ "")

Formats the given `file`, `line`, and `column` as shown in stacktraces.

format_mfa(module, fun, arity)

Receives a module, fun and arity and formats it as shown in stacktraces. The arity may also be a list of arguments.

format_stacktrace(trace \\ nil)

Formats the stacktrace.

format_stacktrace_entry(entry)

Receives a stacktrace entry and formats it into a string.

message(exception)

Gets the message for an `exception`.

normalize(kind, payload, stacktrace \\ [])

Normalizes an exception, converting Erlang exceptions to Elixir exceptions.

Types

arity_or_args()

```
@type arity_or_args() :: non_neg_integer() | list()
```

kind()

```
@type kind() :: :error | non_error_kind()
```

The kind handled by formatting functions

location()

```
@type location() :: keyword()
```

non_error_kind()

```
@type non_error_kind() :: :exit | :throw | {:EXIT, pid()}
```

stacktrace()

```
@type stacktrace() :: [stacktrace_entry()]
```

stacktrace_entry()

```
@type stacktrace_entry() ::
  {module(), atom(), arity_or_args(), location()}
  | {(... -> any()), arity_or_args(), location()}
```

t()

```
@type t() :: %{
  :__struct__ => module(),
  :__exception__ => term(),
  optional(atom()) => any()
}
```

The exception type

Callbacks

blame(t, stacktrace)

(optional)

```
@callback blame(t(), stacktrace()) :: {t(), stacktrace()}
```

Called from `Exception.blame/3` to augment the exception struct.

Can be used to collect additional information about the exception or do some additional expensive computation.

exception(term)

```
@callback exception(term()) :: t()
```

Receives the arguments given to `raise/2` and returns the exception struct.

The default implementation accepts either a set of keyword arguments that is merged into the struct or a string to be used as the exception's message.

message(t)

```
@callback message(t()) :: String.t()
```

Receives the exception struct and must return its message.

Many exceptions have a message field which by default is accessed by this function. However, if an exception does not have a message field, this function must be explicitly implemented.

Functions

blame(kind, error, stacktrace)

(since 1.5.0)

```
@spec blame(:error, any(), stacktrace()) :: {t(),
stacktrace()}
@spec blame(non_error_kind(), payload, stacktrace()) ::
{payload, stacktrace()}
when payload: var
```

Attaches information to throws/errors/exits for extra debugging.

This operation is potentially expensive, as it reads data from the file system, parses beam files, evaluates code and so on.

If `kind` argument is `:error` and the `error` is an Erlang exception, this function will normalize it. If the `error` argument is an Elixir exception, this function will invoke the optional `blame/2` callback on the exception module if it is implemented. Unlike `message/1`, this function will not rescue errors - if the callback raises an exception, the error will propagate to the caller. It is your choice if you want to rescue and return the original exception, return a different exception, or let it cascade.

blame_mfa(module, function, args)

(since 1.5.0)

```
@spec blame_mfa(module(), function :: atom(), args ::
[term()]) ::
  {:ok, :def | :defp | :defmacro | :defmacrop,
   [{args :: [term()], guards :: [term()]}]}
  | :error
```

Blames the invocation of the given module, function and arguments.

This function will retrieve the available clauses from bytecode and evaluate them against the given arguments. The clauses are returned as a list of `{args, guards}` pairs where each argument and each top-level condition in a guard separated by `and/or` is wrapped in a tuple with blame metadata.

This function returns either `{:ok, definition, clauses}` or `:error`. Where `definition` is `:def`, `:defp`, `:defmacro` or `:defmacrop`.

format(kind, payload, stacktrace \\ [])

```
@spec format(kind(), any(), stacktrace()) :: String.t()
```

Normalizes and formats throws/errors/exits and stacktraces.

It relies on `format_banner/3` and `format_stacktrace/1` to generate the final format.

If `kind` is `{:EXIT, pid}`, it does not generate a stacktrace, as such exits are retrieved as messages without stacktraces.

format_banner(kind, exception, stacktrace \\ [])

```
@spec format_banner(kind(), any(), stacktrace()) ::
String.t()
```

Normalizes and formats any throw/error/exit.

The message is formatted and displayed in the same format as used by Elixir's CLI.

The third argument is the stacktrace which is used to enrich a normalized error with more information. It is only used when the kind is an error.

format_exit(reason)

```
@spec format_exit(any()) :: String.t()
```

Formats an exit. It returns a string.

Often there are errors/exceptions inside exits. Exits are often wrapped by the caller and provide stacktraces too. This function formats exits in a way to nicely show the exit reason, caller and stacktrace.

format_fa(fun, arity)

```
@spec format_fa(fun(), arity()) :: String.t()
```

Receives an anonymous function and arity and formats it as shown in stacktraces. The arity may also be a list of arguments.

Examples

```
Exception.format_fa(fn -> nil end, 1)
#=> "#Function<...>/1"
```

format_file_line(file, line, suffix \\ "")

```
@spec format_file_line(String.t() | nil, non_neg_integer() | nil, String.t()) ::
  String.t()
```

Formats the given `file` and `line` as shown in stacktraces.

If any of the values are `nil`, they are omitted.

Examples

```
iex> Exception.format_file_line("foo", 1)
"foo:1:"

iex> Exception.format_file_line("foo", nil)
"foo:"
```

```
iex> Exception.format_file_line(nil, nil)
""
```

format_file_line_column(file, line, column, suffix \\ "")

```
@spec format_file_line_column(
  String.t() | nil,
  non_neg_integer() | nil,
  non_neg_integer() | nil,
  String.t()
) :: String.t()
```

Formats the given `file`, `line`, and `column` as shown in stacktraces.

If any of the values are `nil`, they are omitted.

Examples

```
iex> Exception.format_file_line_column("foo", 1, 2)
"foo:1:2:"

iex> Exception.format_file_line_column("foo", 1, nil)
"foo:1:"

iex> Exception.format_file_line_column("foo", nil, nil)
"foo:"

iex> Exception.format_file_line_column("foo", nil, 2)
"foo:"
```

```
iex> Exception.format_file_line_column(nil, nil, nil)
""
```

format_mfa(module, fun, arity)

```
@spec format_mfa(module(), atom(), arity or args()) ::
String.t()
```

Receives a module, fun and arity and formats it as shown in stacktraces. The arity may also be a list of arguments.

Examples

```
iex> Exception.format_mfa(Foo, :bar, 1)
"Foo.bar/1"

iex> Exception.format_mfa(Foo, :bar, [])
"Foo.bar()"

iex> Exception.format_mfa(nil, :bar, [])
"nil.bar()"
```

Anonymous functions are reported as -func/arity-anonfn-count-, where func is the name of the enclosing function. Convert to "anonymous fn in func/arity"

format_stacktrace(trace \\ nil)

```
@spec format_stacktrace(stacktrace() | nil) :: String.t()
```

Formats the stacktrace.

A stacktrace must be given as an argument. If not, the stacktrace is retrieved from `Process.info/2`.

format_stacktrace_entry(entry)

```
@spec format_stacktrace_entry(stacktrace_entry()) ::
String.t()
```

Receives a stacktrace entry and formats it into a string.

message(exception)

```
@spec message(t()) :: String.t()
```

Gets the message for an `exception`.

This function will invoke the `message/1` callback on the exception module to retrieve the message. If the callback raises an exception or returns a non-binary value, this function will rescue the error and return a descriptive error message instead.

normalize(kind, payload, stacktrace \\ [])

```
@spec normalize(:error, any(), stacktrace()) :: t()
@spec normalize(non_error_kind(), payload, stacktrace()) ::
payload when payload: var
```

Normalizes an exception, converting Erlang exceptions to Elixir exceptions.

It takes the `kind` spilled by `catch` as an argument and normalizes only `:error`, returning the untouched payload for others.

The third argument is the stacktrace which is used to enrich a normalized error with more information. It is only used when the kind is an error.

Float

Functions for working with floating-point numbers.

For mathematical operations on top of floating-points, see Erlang's `:math` module.

Kernel functions

There are functions related to floating-point numbers on the `Kernel` module too. Here is a list of them:

- `Kernel.round/1`: rounds a number to the nearest integer.
- `Kernel.trunc/1`: returns the integer part of a number.

Known issues

There are some very well known problems with floating-point numbers and arithmetic due to the fact most decimal fractions cannot be represented by a floating-point binary and most operations are not exact, but operate on approximations. Those issues are not specific to Elixir, they are a property of floating point representation itself.

For example, the numbers 0.1 and 0.01 are two of them, what means the result of squaring 0.1 does not give 0.01 neither the closest representable. Here is what happens in this case:

- The closest representable number to 0.1 is 0.1000000014
- The closest representable number to 0.01 is 0.0099999997
- Doing 0.1 * 0.1 should return 0.01, but because 0.1 is actually 0.1000000014, the result is 0.010000000000000002, and because this is not the closest representable number to 0.01, you'll get the wrong result for this operation

There are also other known problems like flooring or rounding numbers. See `round/2` and `floor/2` for more details about them.

To learn more about floating-point arithmetic visit:

- 0.30000000000000004.com
- What Every Programmer Should Know About Floating-Point Arithmetic

Summary

Types

precision_range()

Functions

ceil(number, precision \\ 0)

Rounds a float to the smallest float greater than or equal to `number`.

floor(number, precision \\ 0)

Rounds a float to the largest float less than or equal to `number`.

max_finite()

Returns the maximum finite value for a float.

min_finite()

Returns the minimum finite value for a float.

parse(binary)

Parses a binary into a float.

pow(base, exponent)

Computes `base` raised to power of `exponent`.

ratio(float)

Returns a pair of integers whose ratio is exactly equal to the original float and with a positive denominator.

round(float, precision \\ 0)

Rounds a floating-point value to an arbitrary number of fractional digits (between 0 and 15).

to_charlist(float)

Returns a charlist which corresponds to the shortest text representation of the given float.

to_string(float)

Returns a binary which corresponds to the shortest text representation of the given float.

Types

precision_range()

```
@type precision_range() :: 0..15
```

Functions

ceil(number, precision \\ 0)

```
@spec ceil(float(), precision_range()) :: float()
```

Rounds a float to the smallest float greater than or equal to `number`.

`ceil/2` also accepts a precision to round a floating-point value down to an arbitrary number of fractional digits (between 0 and 15).

The operation is performed on the binary floating point, without a conversion to decimal.

The behavior of `ceil/2` for floats can be surprising. For example:

```
iex> Float.ceil(-12.52, 2)
-12.51
```

One may have expected it to ceil to -12.52. This is not a bug. Most decimal fractions cannot be represented as a binary floating point and therefore the number above is internally represented as -12.51999999, which explains the behavior above.

This function always returns floats. `Kernel.trunc/1` may be used instead to truncate the result to an integer afterwards.

Examples

```
iex> Float.ceil(34.25)
35.0
iex> Float.ceil(-56.5)
-56.0
iex> Float.ceil(34.251, 2)
34.26
```

```
iex> Float.ceil(-0.01)
-0.0
```

floor(number, precision \\ 0)

```
@spec floor(float(), precision_range()) :: float()
```

Rounds a float to the largest float less than or equal to `number`.

`floor/2` also accepts a precision to round a floating-point value down to an arbitrary number of fractional digits (between 0 and 15). The operation is performed on the binary floating point, without a conversion to decimal.

This function always returns a float. `Kernel.trunc/1` may be used instead to truncate the result to an integer afterwards.

Known issues

The behavior of `floor/2` for floats can be surprising. For example:

```
iex> Float.floor(12.52, 2)
12.51
```

One may have expected it to floor to 12.52. This is not a bug. Most decimal fractions cannot be represented as a binary floating point and therefore the number above is internally represented as 12.51999999, which explains the behavior above.

Examples

```
iex> Float.floor(34.25)
34.0
iex> Float.floor(-56.5)
-57.0
iex> Float.floor(34.259, 2)
34.25
```

max_finite()

```
@spec max_finite() :: float()
```

Returns the maximum finite value for a float.

Examples

```
iex> Float.max_finite()
1.7976931348623157e308
```

min_finite()

```
@spec min_finite() :: float()
```

Returns the minimum finite value for a float.

Examples

```
iex> Float.min_finite()
```

```
-1.7976931348623157e308
```

parse(binary)

```
@spec parse(binary()) :: {float(), binary()} | :error
```

Parses a binary into a float.

If successful, returns a tuple in the form of `{float, remainder_of_binary}`; when the binary cannot be coerced into a valid float, the atom `:error` is returned.

If the size of float exceeds the maximum size of `1.7976931348623157e+308`, `:error` is returned even though the textual representation itself might be well formed.

If you want to convert a string-formatted float directly to a float, `String.to_float/1` can be used instead.

Examples

```
iex> Float.parse("34")
{34.0, ""}
iex> Float.parse("34.25")
{34.25, ""}
iex> Float.parse("56.5xyz")
{56.5, "xyz"}

iex> Float.parse(".12")
:error
iex> Float.parse("pi")
:error
```

```
iex> Float.parse("1.7976931348623159e+308")
:error
```

pow(base, exponent)

(since 1.12.0)

```
@spec pow(float(), number()) :: float()
```

Computes `base` raised to power of `exponent`.

`base` must be a float and `exponent` can be any number. However, if a negative base and a fractional exponent are given, it raises `ArithmeticError`.

It always returns a float. See `Integer.pow/2` for exponentiation that returns integers.

Examples

```
iex> Float.pow(2.0, 0)
1.0
iex> Float.pow(2.0, 1)
2.0
iex> Float.pow(2.0, 10)
1024.0
iex> Float.pow(2.0, -1)
0.5
iex> Float.pow(2.0, -3)
0.125

iex> Float.pow(3.0, 1.5)
5.196152422706632
```

```
iex> Float.pow(-2.0, 3)
-8.0
iex> Float.pow(-2.0, 4)
16.0

iex> Float.pow(-1.0, 0.5)
** (ArithmeticError) bad argument in arithmetic
expression
```

ratio(float)

(since 1.4.0)

```
@spec ratio(float()) :: {integer(), pos_integer()}
```

Returns a pair of integers whose ratio is exactly equal to the original float and with a positive denominator.

Examples

```
iex> Float.ratio(0.0)
{0, 1}
iex> Float.ratio(3.14)
{7070651414971679, 2251799813685248}
iex> Float.ratio(-3.14)
{-7070651414971679, 2251799813685248}
iex> Float.ratio(1.5)
{3, 2}
iex> Float.ratio(-1.5)
{-3, 2}
iex> Float.ratio(16.0)
{16, 1}
```

```
iex> Float.ratio(-16.0)
{-16, 1}
```

round(float, precision \\ 0)

```
@spec round(float(), precision_range()) :: float()
```

Rounds a floating-point value to an arbitrary number of fractional digits (between 0 and 15).

The rounding direction always ties to half up. The operation is performed on the binary floating point, without a conversion to decimal.

This function only accepts floats and always returns a float. Use `Kernel.round/1` if you want a function that accepts both floats and integers and always returns an integer.

Known issues

The behavior of `round/2` for floats can be surprising. For example:

```
iex> Float.round(5.5675, 3)
5.567
```

One may have expected it to round to the half up 5.568. This is not a bug. Most decimal fractions cannot be represented as a binary floating point and therefore the number above is internally represented as 5.567499999, which explains the behavior above. If you want exact rounding for decimals, you must use a decimal library. The behavior above is also in accordance to reference implementations, such as "Correctly Rounded Binary-Decimal and Decimal-Binary Conversions" by David M. Gay.

Examples

```
iex> Float.round(12.5)
13.0
iex> Float.round(5.5674, 3)
5.567
iex> Float.round(5.5675, 3)
5.567
iex> Float.round(-5.5674, 3)
-5.567
iex> Float.round(-5.5675)
-6.0
iex> Float.round(12.341444444444441, 15)
12.341444444444441
iex> Float.round(-0.01)
-0.0
```

to_charlist(float)

```
@spec to_charlist(float()) :: charlist()
```

Returns a charlist which corresponds to the shortest text representation of the given float.

It uses the algorithm presented in "Ryū: fast float-to-string conversion" in Proceedings of the SIGPLAN '2018 Conference on Programming Language Design and Implementation.

For a configurable representation, use `:erlang.float_to_list/2`.

Inlined by the compiler.

Examples

```
iex> Float.to_charlist(7.0)
~c"7.0"
```

to_string(float)

```
@spec to_string(float()) :: String.t()
```

Returns a binary which corresponds to the shortest text representation of the given float.

The underlying algorithm changes depending on the Erlang/OTP version:

- For OTP >= 24, it uses the algorithm presented in "Ryū: fast float-to-string conversion" in Proceedings of the SIGPLAN '2018 Conference on Programming Language Design and Implementation.
- For OTP < 24, it uses the algorithm presented in "Printing Floating-Point Numbers Quickly and Accurately" in Proceedings of the SIGPLAN '1996 Conference on Programming Language Design and Implementation.

For a configurable representation, use `:erlang.float_to_binary/2`.

Inlined by the compiler.

Examples

```
iex> Float.to_string(7.0)
"7.0"
```

Function

A set of functions for working with functions.

Anonymous functions are typically created by using `fn`:

```
iex> add = fn a, b -> a + b end
iex> add.(1, 2)
3
```

Anonymous functions can also have multiple clauses. All clauses should expect the same number of arguments:

```
iex> negate = fn
...>   true -> false
...>   false -> true
...> end
iex> negate.(false)
true
```

The capture operator

It is also possible to capture public module functions and pass them around as if they were anonymous functions by using the capture operator `&/1`:

```
iex> add = &Kernel.+/2
iex> add.(1, 2)
3
```

```
iex> length = &String.length/1
iex> length.("hello")
5
```

To capture a definition within the current module, you can skip the module prefix, such as `&my_fun/2`. In those cases, the captured function can be public (`def`) or private (`defp`).

The capture operator can also be used to create anonymous functions that expect at least one argument:

```
iex> add = &(&1 + &2)
iex> add.(1, 2)
3
```

In such cases, using the capture operator is no different than using `fn`.

Internal and external functions

We say that functions that point to definitions residing in modules, such as `&String.length/1`, are **external** functions. All other functions are **local** and they are always bound to the file or module that defined them.

Besides the functions in this module to work with functions, `Kernel` also has an `apply/2` function that invokes a function with a dynamic number of arguments, as well as `is_function/1` and `is_function/2`, to check respectively if a given value is a function or a function of a given arity.

Summary

Types

information()

Functions

capture(module, function_name, arity)

Captures the given function.

identity(value)

Returns its input `value`. This function can be passed as an anonymous function to transformation functions.

info(fun)

Returns a keyword list with information about a function.

info(fun, item)

Returns a specific information about the function.

Types

information()

```
@type information() ::
  :arity
  | :env
  | :index
  | :module
  | :name
  | :new_index
  | :new_uniq
```

```
  | :pid
  | :type
  | :uniq
```

Functions

capture(module, function_name, arity)

(since 1.7.0)

```
@spec capture(module(), atom(), arity()) :: fun()
```

Captures the given function.

Inlined by the compiler.

Examples

```
iex> Function.capture(String, :length, 1)
&String.length/1
```

identity(value)

(since 1.10.0)

```
@spec identity(value) :: value when value: var
```

Returns its input `value`. This function can be passed as an anonymous function to transformation functions.

Examples

```
iex> Function.identity("Hello world!")
"Hello world!"

iex> ~c"abcdaabccc" |> Enum.sort() |>
Enum.chunk_by(&Function.identity/1)
[~c"aaa", ~c"bb", ~c"cccc", ~c"d"]

iex> Enum.group_by(~c"abracadabra", &Function.identity/1)
%{97 => ~c"aaaaa", 98 => ~c"bb", 99 => ~c"c", 100 =>
~c"d", 114 => ~c"rr"}

iex> Enum.map([1, 2, 3, 4], &Function.identity/1)
[1, 2, 3, 4]
```

info(fun)

(since 1.7.0)

```
@spec info(fun()) :: [{information(), term()}]
```

Returns a keyword list with information about a function.

The returned keys (with the corresponding possible values) for all types of functions (local and external) are the following:

- `:type` - `:local` (for anonymous functions) or `:external` (for named functions).
- `:module` - an atom which is the module where the function is defined when anonymous or the module which the function refers to when it's a named function.
- `:arity` - (integer) the number of arguments the function is to be called with.

- `:name` - (atom) the name of the function.
- `:env` - a list of the environment or free variables. For named functions, the returned list is always empty.

When `fun` is an anonymous function (that is, the type is `:local`), the following additional keys are returned:

- `:pid` - PID of the process that originally created the function.
- `:index` - (integer) an index into the module function table.
- `:new_index` - (integer) an index into the module function table.
- `:new_uniq` - (binary) a unique value for this function. It's calculated from the compiled code for the entire module.
- `:uniq` - (integer) a unique value for this function. This integer is calculated from the compiled code for the entire module.

Note: this function must be used only for debugging purposes.

Inlined by the compiler.

Examples

```
iex> fun = fn x -> x end
iex> info = Function.info(fun)
iex> Keyword.get(info, :arity)
1
iex> Keyword.get(info, :type)
:local

iex> fun = &String.length/1
iex> info = Function.info(fun)
```

```
iex> Keyword.get(info, :type)
:external
iex> Keyword.get(info, :name)
:length
```

info(fun, item)

(since 1.7.0)

```
@spec info(fun(), item) :: {item, term()} when item:
information()
```

Returns a specific information about the function.

The returned information is a two-element tuple in the shape of `{info, value}`.

For any function, the information asked for can be any of the atoms `:module`, `:name`, `:arity`, `:env`, or `:type`.

For anonymous functions, there is also information about any of the atoms `:index`, `:new_index`, `:new_uniq`, `:uniq`, and `:pid`. For a named function, the value of any of these items is always the atom `:undefined`.

For more information on each of the possible returned values, see `info/1`.

Inlined by the compiler.

Examples

```
iex> f = fn x -> x end
iex> Function.info(f, :arity)
```

```
{:arity, 1}
iex> Function.info(f, :type)
{:type, :local}

iex> fun = &String.length/1
iex> Function.info(fun, :name)
{:name, :length}
iex> Function.info(fun, :pid)
{:pid, :undefined}
```

Integer

Functions for working with integers.

Some functions that work on integers are found in `Kernel`:

- `Kernel.abs/1`
- `Kernel.div/2`
- `Kernel.max/2`
- `Kernel.min/2`
- `Kernel.rem/2`

Summary

Guards

is_even(integer)

Determines if an `integer` is even.

is_odd(integer)

Determines if `integer` is odd.

Functions

ceil_div(dividend, divisor)

Performs a ceiled integer division.

digits(integer, base \\ 10)

Returns the ordered digits for the given `integer`.

extended_gcd(a, b)

Returns the extended greatest common divisor of the two given integers.

floor_div(dividend, divisor)

Performs a floored integer division.

gcd(integer1, integer2)

Returns the greatest common divisor of the two given integers.

mod(dividend, divisor)

Computes the modulo remainder of an integer division.

parse(binary, base \\ 10)

Parses a text representation of an integer.

popcount(integer)

Counts the number of set bits (1) in the binary representation of a non-negative `integer`.

pow(base, exponent)

Computes `base` raised to power of `exponent`.

to_charlist(integer, base \\ 10)

Returns a charlist which corresponds to the text representation of `integer` in the given `base`.

to_string(integer, base \\ 10)

Returns a binary which corresponds to the text representation of `integer` in the given `base`.
undigits(digits, base \\ 10)

Returns the integer represented by the ordered `digits`.

Guards

is_even(integer)

(macro)

Determines if an `integer` is even.

Returns `true` if the given `integer` is an even number, otherwise it returns `false`.

Allowed in guard clauses.

Examples

```
iex> Integer.is_even(10)
true

iex> Integer.is_even(5)
false

iex> Integer.is_even(-10)
true

iex> Integer.is_even(0)
true
```

is_odd(integer)

(macro)

Determines if `integer` is odd.

Returns `true` if the given `integer` is an odd number, otherwise it returns `false`.

Allowed in guard clauses.

Examples

```
iex> Integer.is_odd(5)
true

iex> Integer.is_odd(6)
false

iex> Integer.is_odd(-5)
true

iex> Integer.is_odd(0)
false
```

Functions

ceil_div(dividend, divisor)

(since 1.20.0)

```
@spec ceil_div(integer(), neg_integer() | pos_integer()) ::
integer()
```

Performs a ceiled integer division.

Raises an `ArithmeticError` exception if one of the arguments is not an integer, or when the `divisor` is `0`.

This function performs a *ceiled* integer division, which means that the result will always be rounded towards positive infinity.

Examples

```
iex> Integer.ceil_div(5, 2)
3
iex> Integer.ceil_div(6, -4)
-1
iex> Integer.ceil_div(-99, 2)
-49
```

digits(integer, base \\ 10)

```
@spec digits(integer(), pos_integer()) :: [integer(), ...]
```

Returns the ordered digits for the given `integer`.

An optional `base` value may be provided representing the radix for the returned digits. This one must be an integer >= 2.

Examples

```
iex> Integer.digits(123)
[1, 2, 3]

iex> Integer.digits(170, 2)
```

```
[1, 0, 1, 0, 1, 0, 1, 0]

iex> Integer.digits(-170, 2)
[-1, 0, -1, 0, -1, 0, -1, 0]
```

extended_gcd(a, b)

(since 1.12.0)

```
@spec extended_gcd(integer(), integer()) ::
{non_neg_integer(), integer(), integer()}
```

Returns the extended greatest common divisor of the two given integers.

This function uses the extended Euclidean algorithm to return a three-element tuple with the `gcd` and the coefficients `m` and `n` of Bézout's identity such that:

```
gcd(a, b) = m*a + n*b
```

By convention, `extended_gcd(0, 0)` returns `{0, 0, 0}`.

Examples

```
iex> Integer.extended_gcd(240, 46)
{2, -9, 47}
iex> Integer.extended_gcd(46, 240)
{2, 47, -9}
iex> Integer.extended_gcd(-46, 240)
{2, -47, -9}
iex> Integer.extended_gcd(-46, -240)
{2, -47, 9}
```

```
iex> Integer.extended_gcd(14, 21)
{7, -1, 1}

iex> Integer.extended_gcd(10, 0)
{10, 1, 0}
iex> Integer.extended_gcd(-10, 0)
{10, -1, 0}
iex> Integer.extended_gcd(0, 10)
{10, 0, 1}
iex> Integer.extended_gcd(0, -10)
{10, 0, -1}
iex> Integer.extended_gcd(0, 0)
{0, 0, 0}
```

floor_div(dividend, divisor)

(since 1.4.0)

```
@spec floor_div(integer(), neg_integer() | pos_integer()) ::
integer()
```

Performs a floored integer division.

Raises an `ArithmeticError` exception if one of the arguments is not an integer, or when the `divisor` is `0`.

This function performs a *floored* integer division, which means that the result will always be rounded towards negative infinity.

If you want to perform truncated integer division (rounding towards zero), use `Kernel.div/2` instead.

Examples

```
iex> Integer.floor_div(5, 2)
2
iex> Integer.floor_div(6, -4)
-2
iex> Integer.floor_div(-99, 2)
-50
```

gcd(integer1, integer2)

(since 1.5.0)

```
@spec gcd(integer(), integer()) :: non_neg_integer()
```

Returns the greatest common divisor of the two given integers.

The greatest common divisor (GCD) of `integer1` and `integer2` is the largest positive integer that divides both `integer1` and `integer2` without leaving a remainder.

By convention, `gcd(0, 0)` returns `0`.

Examples

```
iex> Integer.gcd(2, 3)
1

iex> Integer.gcd(8, 12)
4

iex> Integer.gcd(8, -12)
4

iex> Integer.gcd(10, 0)
```

```
10

iex> Integer.gcd(7, 7)
7

iex> Integer.gcd(0, 0)
0
```

mod(dividend, divisor)

(since 1.4.0)

```
@spec mod(integer(), neg integer() | pos integer()) ::
integer()
```

Computes the modulo remainder of an integer division.

This function performs a floored division, which means that the result will always have the sign of the `divisor`.

Raises an `ArithmeticError` exception if one of the arguments is not an integer, or when the `divisor` is `0`.

Examples

```
iex> Integer.mod(5, 2)
1
iex> Integer.mod(6, -4)
-2
```

parse(binary, base \\ 10)

```
@spec parse(binary(), 2..36) :: {integer(),
remainder_of_binary :: binary()} | :error
```

Parses a text representation of an integer.

An optional `base` to the corresponding integer can be provided. If `base` is not given, 10 will be used.

If successful, returns a tuple in the form of `{integer, remaining_string}`. Otherwise `:error`.

Raises an error if `base` is less than 2 or more than 36.

If you want to convert a string-formatted integer directly to an integer, `String.to_integer/1` or `String.to_integer/2` can be used instead.

Examples

```
iex> Integer.parse("34")
{34, ""}

iex> Integer.parse("34.5")
{34, ".5"}

iex> Integer.parse("three")
:error

iex> Integer.parse("404 not found")
{404, " not found"}

iex> Integer.parse("34", 10)
{34, ""}
```

```
iex> Integer.parse("f4", 16)
{244, ""}

iex> Integer.parse("Awww++", 36)
{509216, "++"}

iex> Integer.parse("fab", 10)
:error

iex> Integer.parse("a2", 38)
** (ArgumentError) invalid base 38
```

popcount(integer)

(since 1.20.0)

```
@spec popcount(non_neg_integer()) :: non_neg_integer()
```

Counts the number of set bits (1) in the binary representation of a non-negative `integer`.

This operation is known as the Hamming weight or population count.

Raises an `ArithmeticError` if `integer` is negative.

Examples

```
iex> Integer.popcount(0)
0

iex> Integer.popcount(1)
1

iex> Integer.popcount(0b10110101)
```

```
5

iex> Integer.popcount(255)
8

iex> Integer.popcount(0b1111111111111111)
16

iex> Integer.popcount(-1)
** (ArithmeticError) bad argument in arithmetic
expression
```

pow(base, exponent)

(since 1.12.0)

```
@spec pow(integer(), non_neg_integer()) :: integer()
```

Computes `base` raised to power of `exponent`.

Both `base` and `exponent` must be integers. The exponent must be zero or positive.

See `Float.pow/2` for exponentiation of negative exponents as well as floats.

Examples

```
iex> Integer.pow(2, 0)
1
iex> Integer.pow(2, 1)
2
iex> Integer.pow(2, 10)
1024
```

```
iex> Integer.pow(2, 11)
2048
iex> Integer.pow(2, 64)
0x10000000000000000

iex> Integer.pow(3, 4)
81
iex> Integer.pow(4, 3)
64

iex> Integer.pow(-2, 3)
-8
iex> Integer.pow(-2, 4)
16

iex> Integer.pow(2, -2)
** (ArithmeticError) bad argument in arithmetic
expression
```

to_charlist(integer, base \\ 10)

```
@spec to_charlist(integer(), 2..36) :: charlist()
```

Returns a charlist which corresponds to the text representation of `integer` in the given `base`.

`base` can be an integer between 2 and 36. If no `base` is given, it defaults to `10`.

Inlined by the compiler.

Examples

```
iex> Integer.to_charlist(123)
~c"123"

iex> Integer.to_charlist(+456)
~c"456"

iex> Integer.to_charlist(-789)
~c"-789"

iex> Integer.to_charlist(0123)
~c"123"

iex> Integer.to_charlist(100, 16)
~c"64"

iex> Integer.to_charlist(-100, 16)
~c"-64"

iex> Integer.to_charlist(882_681_651, 36)
~c"ELIXIR"
```

to_string(integer, base \\ 10)

```
@spec to_string(integer(), 2..36) :: String.t()
```

Returns a binary which corresponds to the text representation of `integer` in the given `base`.

`base` can be an integer between 2 and 36. If no `base` is given, it defaults to `10`.

Inlined by the compiler.

Examples

```
iex> Integer.to_string(123)
"123"

iex> Integer.to_string(+456)
"456"

iex> Integer.to_string(-789)
"-789"

iex> Integer.to_string(0123)
"123"

iex> Integer.to_string(100, 16)
"64"

iex> Integer.to_string(-100, 16)
"-64"

iex> Integer.to_string(882_681_651, 36)
"ELIXIR"
```

undigits(digits, base \\ 10)

```
@spec undigits([integer()], pos_integer()) :: integer()
```

Returns the integer represented by the ordered `digits`.

An optional `base` value may be provided representing the radix for the `digits`. Base has to be an integer greater than or equal to `2`.

Examples

```
iex> Integer.undigits([1, 2, 3])
123

iex> Integer.undigits([1, 4], 16)
20

iex> Integer.undigits([])
0
```

JSON

JSON encoding and decoding.

Both encoder and decoder fully conform to RFC 8259 and ECMA 404 standards.

Encoding

Elixir primitive types are encoded to JSON as follows:

Elixir	JSON
`integer() | float()`	Number
`true | false`	Boolean
`nil`	Null
`binary()`	String
`atom()`	String
`list()`	Array
`%{String.Chars.t() => _}`	Object

You may also implement the `JSON.Encoder` protocol for custom data structures. Some built-in data-structures already derive the `JSON.Encoder` protocol:

Elixir	JSON
`Date.t()`	ISO 8601 string
`Time.t()`	ISO 8601 string
`DateTime.t()`	ISO 8601 string
`NaiveDateTime.t()`	ISO 8601 string

Elixir	JSON
`Duration.t()`	ISO 8601 string

Decoding

Elixir types are decoded from JSON as follows:

JSON	Elixir
Number	`integer() | float()`
Boolean	`true | false`
Null	`nil`
String	`binary()`
Object	`%{binary() => _}`

Summary

Types

decode_error_reason()
decoders()

Decoders for customizing JSON decoding behavior.

encoder()

Functions

decode(binary)

Decodes the given JSON.

decode(binary, acc, decoders)

Decodes the given JSON with the given decoders.
decode!(binary)

Decodes the given JSON but raises an exception in case of errors.

encode!(term, encoder \\ &protocol_encode/2)

Encodes the given term to JSON as a binary.

encode_to_iodata!(term, encoder \\ &protocol_encode/2)

Encodes the given term to JSON as an iodata.

protocol_encode(value, encoder)

This is the default encode implementation passed to `encode!/1`.

Types

decode_error_reason()

(since 1.18.0)

```
@type decode_error_reason() ::
  {:unexpected_end, non_neg_integer()}
  | {:invalid_byte, non_neg_integer(), byte()}
  | {:unexpected_sequence, non_neg_integer(), binary()}
  | {:integer_too_large, pos_integer()}
```

decoders()

(since 1.18.0)

```
@type decoders() :: [
  array_start: (term() -> term()),
  array_push: (term(), term() -> term()),
  array_finish: (term(), term() -> {term(), term()}),
  object_start: (term() -> term()),
  object_push: (term(), term(), term() -> term()),
  object_finish: (term(), term() -> {term(), term()}),
  float: (String.t() -> term()),
  integer: (String.t() -> term()),
  string: (String.t() -> term()),
  null: term()
]
```

Decoders for customizing JSON decoding behavior.

encoder()

(since 1.18.0)

```
@type encoder() :: (term(), encoder() -> iodata())
```

Functions

decode(binary)

(since 1.18.0)

```
@spec decode(binary()) :: {:ok, term()} | {:error,
decode_error_reason()}
```

Decodes the given JSON.

Returns `{:ok, decoded}` or `{:error, reason}`.

Examples

```
iex> JSON.decode("[null,123,\"string\",
{\"key\":\"value\"}]")
{:ok, [nil, 123, "string", %{"key" => "value"}]}
```

Error reasons

The error tuple will have one of the following reasons.

- `{:unexpected_end, offset}` if `binary` contains incomplete JSON value
- `{:invalid_byte, offset, byte}` if `binary` contains unexpected byte or invalid UTF-8 byte
- `{:unexpected_sequence, offset, bytes}` if `binary` contains invalid UTF-8 escape
- `{:integer_too_large, digits}` if `binary` contains an integer with more than 310 digits (enough to cover all 64-bit floats)

decode(binary, acc, decoders)

(since 1.18.0)

```
@spec decode(binary(), term(), decoders()) ::
  {term(), term(), binary()} | {:error,
decode_error_reason()}
```

Decodes the given JSON with the given decoders.

Returns `{decoded, acc, rest}` or `{:error, reason}`. See `decode/1` for the error reasons.

Decoders

All decoders are optional. If not provided, they will fall back to implementations used by the `decode/1` function:

- for `array_start`: `fn _ -> [] end`
- for `array_push`: `fn elem, acc -> [elem | acc] end`
- for `array_finish`: `fn acc, old_acc -> {Enum.reverse(acc), old_acc} end`
- for `object_start`: `fn _ -> [] end`
- for `object_push`: `fn key, value, acc -> [{key, value} | acc] end`
- for `object_finish`: `fn acc, old_acc -> {Map.new(acc), old_acc} end`
- for `float`: `&String.to_float/1`
- for `integer`: `&String.to_integer/1`, except integers with more than 310 digits (enough to cover all 64-bit floats) are rejected with `{:integer_too_large, digits}`
- for `string`: `&Function.identity/1`
- for `null`: the atom `nil`

For streaming decoding, see Erlang's `:json` module.

decode!(binary)

(since 1.18.0)

```
@spec decode!(binary()) :: term()
```

Decodes the given JSON but raises an exception in case of errors.

Returns the decoded content. See `decode/1` for possible errors.

Examples

```
iex> JSON.decode!("[null,123,\"string\",
{\"key\":\"value\"}]")
[nil, 123, "string", %{"key" => "value"}]
```

encode!(term, encoder \\ &protocol_encode/2)

(since 1.18.0)

```
@spec encode!(term(), encoder()) :: binary()
```

Encodes the given term to JSON as a binary.

The second argument is a function that is recursively invoked to encode a term.

> **IO and performance**
>
> If you need to encode data to be sent over the network or written to the filesystem, consider using the more efficient `encode_to_iodata!/2`.

Examples

```
iex> JSON.encode!([123, "string", %{key: "value"}])
"[123,\"string\",{\"key\":\"value\"}]"
```

encode_to_iodata!(term, encoder \\ &protocol_encode/2)

(since 1.18.0)

```
@spec encode_to_iodata!(term(), encoder()) :: iodata()
```

Encodes the given term to JSON as an iodata.

This is the most efficient format if the JSON is going to be used for IO purposes.

The second argument is a function that is recursively invoked to encode a term.

Examples

```
iex> data = JSON.encode_to_iodata!([123, "string", %{key:
"value"}])
iex> IO.iodata_to_binary(data)
"[123,\"string\",{\"key\":\"value\"}]"
```

protocol_encode(value, encoder)

(since 1.18.0)

```
@spec protocol_encode(term(), encoder()) :: iodata()
```

This is the default encode implementation passed to `encode!/1`.

This function is most typically passed as second argument to `encode!/2` and `encode_to_iodata!/2`. The default implementation is an optimized dispatch to the `JSON.Encoder` protocol.

Module behaviour

Provides functions to deal with modules during compilation time.

It allows a developer to dynamically add, delete and register attributes, attach documentation and so forth.

After a module is compiled, using many of the functions in this module will raise errors, since it is out of their scope to inspect runtime data. Most of the runtime data can be inspected via the `__info__/1` function attached to each compiled module.

Module attributes

Each module can be decorated with one or more attributes. The following ones are currently defined by Elixir:

`@after_compile`

A hook that will be invoked right after the current module is compiled. Accepts a module or a `{module, function_name}`. See the "Compile callbacks" section below.

`@after_verify` **(since v1.14.0)**

A hook that will be invoked right after the current module is verified for undefined functions, deprecations, etc. Accepts a module or a `{module, function_name}`. See the "Compile callbacks" section below.

`@before_compile`

A hook that will be invoked before the module is compiled. Accepts a module or a `{module, function_or_macro_name}` tuple. See the "Compile callbacks" section below.

`@behaviour`

Note the British spelling!

Behaviours can be referenced by modules to ensure they implement required specific function signatures defined by `@callback`.

For example, you could specify a `URI.Parser` behaviour as follows:

```
defmodule URI.Parser do
  @doc "Defines a default port"
  @callback default_port() :: integer

  @doc "Parses the given URL"
  @callback parse(uri_info :: URI.t()) :: URI.t()
end
```

And then a module may use it as:

```
defmodule URI.HTTP do
  @behaviour URI.Parser
  def default_port(), do: 80
  def parse(info), do: info
end
```

If the behaviour changes or `URI.HTTP` does not implement one of the callbacks, a warning will be raised.

For detailed documentation, see the behaviour typespec documentation.

`@impl` (since v1.5.0)

To aid in the correct implementation of behaviours, you may optionally declare `@impl` for implemented callbacks of a behaviour. This makes callbacks explicit and can help you to catch errors in your code. The compiler will warn in these cases:

- if you mark a function with `@impl` when that function is not a callback.
- if you don't mark a function with `@impl` when other functions are marked with `@impl`. If you mark one function with `@impl`, you must mark all other callbacks for that behaviour as `@impl`.

`@impl` works on a per-context basis. If you generate a function through a macro and mark it with `@impl`, that won't affect the module where that function is generated in.

`@impl` also helps with maintainability by making it clear to other developers that the function is implementing a callback.

Using `@impl`, the example above can be rewritten as:

```
defmodule URI.HTTP do
  @behaviour URI.Parser

  @impl true
  def default_port(), do: 80

  @impl true
  def parse(info), do: info
end
```

You may pass either `false`, `true`, or a specific behaviour to `@impl`.

```
defmodule Foo do
  @behaviour Bar
  @behaviour Baz

  # Will warn if neither Bar nor Baz specify a callback
named bar/0.
  @impl true
  def bar(), do: :ok

  # Will warn if Baz does not specify a callback named
baz/0.
  @impl Baz
  def baz(), do: :ok
end
```

The code is now more readable, as it is now clear which functions are part of your API and which ones are callback implementations. To reinforce this idea, `@impl true` automatically marks the function as `@doc false`, disabling documentation unless `@doc` is explicitly set.

`@compile`

Defines options for module compilation. This is used to configure both Elixir and Erlang compilers, as well as any other compilation pass added by external tools. For example:

```
defmodule MyModule do
  @compile {:inline, my_fun: 1}

  def my_fun(arg) do
    to_string(arg)
```

```
  end
end
```

Multiple uses of `@compile` will accumulate instead of overriding previous ones. See the "Compile options" section below.

`@deprecated` (since v1.6.0)

Provides the deprecation reason for a function. For example:

```
defmodule Keyword do
  @deprecated "Use Kernel.length/1 instead"
  def size(keyword) do
    length(keyword)
  end
end
```

The Mix compiler automatically looks for calls to deprecated modules and emit warnings during compilation.

Using the `@deprecated` attribute will also be reflected in the documentation of the given function and macro. You can choose between the `@deprecated` attribute and the documentation metadata to provide hard-deprecations (with warnings) and soft-deprecations (without warnings):

This is a soft-deprecation as it simply annotates the documentation as deprecated:

```
@doc deprecated: "Use Kernel.length/1 instead"
def size(keyword)
```

This is a hard-deprecation as it emits warnings and annotates the documentation as deprecated:

```
@deprecated "Use Kernel.length/1 instead"
def size(keyword)
```

Currently `@deprecated` only supports functions and macros. However you can use the `:deprecated` key in the annotation metadata to annotate the docs of modules, types and callbacks too.

We recommend using this feature with care, especially library authors. Deprecating code always pushes the burden towards library users. We also recommend for deprecated functionality to be maintained for long periods of time, even after deprecation, giving developers plenty of time to update (except for cases where keeping the deprecated API is undesired, such as in the presence of security issues).

`@doc` and `@typedoc`

Provides documentation for the entity that follows the attribute. `@doc` is to be used with a function, macro, callback, or macrocallback, while `@typedoc` with a type (public or opaque).

Accepts one of these:

- a string (often a heredoc)
- `false`, which will make the entity invisible to documentation-extraction tools like `ExDoc`
- a keyword list, since Elixir 1.7.0

For example:

```
defmodule MyModule do
  @typedoc "This type"
```

```
  @typedoc since: "1.1.0"
  @type t :: term

  @doc "Hello world"
  @doc since: "1.1.0"
  def hello do
    "world"
  end

  @doc """
  Sums `a` to `b`.
  """
  def sum(a, b) do
    a + b
  end
end
```

As can be seen in the example above, since Elixir 1.7.0 `@doc` and `@typedoc` also accept a keyword list that serves as a way to provide arbitrary metadata about the entity. Tools like `ExDoc` and `IEx` may use this information to display annotations. A common use case is the `:since` key, which may be used to annotate in which version the function was introduced.

As illustrated in the example, it is possible to use these attributes more than once before an entity. However, the compiler will warn if used twice with binaries as that replaces the documentation text from the preceding use. Multiple uses with keyword lists will merge the lists into one.

Note that since the compiler also defines some additional metadata, there are a few reserved keys that will be ignored and warned if used. Currently these are: `:opaque` and `:defaults`.

Once this module is compiled, this information becomes available via the `Code.fetch_docs/1` function.

`@dialyzer`

Defines warnings to request or suppress when using `:dialyzer`.

Accepts an atom, a tuple, or a list of atoms and tuples. For example:

```
defmodule MyModule do
  @dialyzer {:nowarn_function, [my_fun: 1]}

  def my_fun(arg) do
    M.not_a_function(arg)
  end
end
```

For the list of supported warnings, see `:dialyzer` module.

Multiple uses of `@dialyzer` will accumulate instead of overriding previous ones.

`@external_resource`

Specifies an external resource for the current module.

Sometimes a module embeds information from an external file. This attribute allows the module to annotate which external resources have been used.

Tools may use this information to ensure the module is recompiled in case any of the external resources change, see for example: `mix compile.elixir`.

The specified file path provided is interpreted as relative to the folder containing the project's `mix.exs`, which is the current working directory, not the file where `@external_resource` is declared.

If the external resource does not exist, the module still has a dependency on it, causing the module to be recompiled as soon as the file is added.

For more control over when a module is recompiled, see `__mix_recompile__?/0`.

`@file`

Changes the filename used in stacktraces for the function or macro that follows the attribute, such as:

```
defmodule MyModule do
  @doc "Hello world"
  @file "hello.ex"
  def hello do
    "world"
  end
end
```

Note that this is only valid for exceptions/diagnostics that come from the definition inner scope (which includes its patterns and guards). For example:

```
defmodule MyModule do # <---- module definition
  @file "hello.ex"
  defp unused(a) do # <---- function definition
    "world" # <---- function scope
  end

  @file "bye.ex"
```

```
  def unused(_), do: true
end
```

If you run this code with the second "unused" definition commented, you will see that `hello.ex` is used as the stacktrace when reporting warnings, but if you uncomment it you'll see that the error will not mention `bye.ex`, because it's a module-level error rather than an expression-level error.

`@moduledoc`

Provides documentation for the current module.

```
defmodule MyModule do
  @moduledoc """
  A very useful module.
  """
  @moduledoc authors: ["Alice", "Bob"]
end
```

Accepts a string (often a heredoc) or `false` where `@moduledoc false` will make the module invisible to documentation extraction tools like ExDoc.

Similarly to `@doc` also accepts a keyword list to provide metadata about the module. For more details, see the documentation of `@doc` above.

Once this module is compiled, this information becomes available via the `Code.fetch_docs/1` function.

`@nifs` (since v1.16.0)

A list of functions and their arities which will be overridden by a native implementation (NIF).

```
defmodule MyLibrary.MyModule do
  @nifs [foo: 1, bar: 2]

  def foo(arg1), do: :erlang.nif_error(:not_loaded)
  def bar(arg1, arg2), do: :erlang.nif_error(:not_loaded)
end
```

See the Erlang documentation for more information: https://www.erlang.org/doc/man/erl_nif

`@on_definition`

A hook that will be invoked when each function or macro in the current module is defined. Useful when annotating functions.

Accepts a module or a `{module, function_name}` tuple. The function must take 6 arguments:

- the module environment
- the kind of the function/macro: `:def`, `:defp`, `:defmacro`, or `:defmacrop`
- the function/macro name
- the list of quoted arguments
- the list of quoted guards
- the quoted function body

If the function/macro being defined has multiple clauses, the hook will be called for each clause.

Unlike other hooks, `@on_definition` will only invoke functions and never macros. This is to avoid `@on_definition` callbacks from

redefining functions that have just been defined in favor of more explicit approaches.

When just a module is provided, the function is assumed to be `__on_definition__/6`.

Example

```elixir
defmodule Hooks do
  def on_def(_env, kind, name, args, guards, body) do
    IO.puts("Defining #{kind} named #{name} with args:")
    IO.inspect(args)
    IO.puts("and guards")
    IO.inspect(guards)
    IO.puts("and body")
    IO.puts(Macro.to_string(body))
  end
end

defmodule MyModule do
  @on_definition {Hooks, :on_def}

  def hello(arg) when is_binary(arg) or is_list(arg) do
    "Hello" <> to_string(arg)
  end

  def hello(_) do
    :ok
  end
end
```

`@on_load`

A hook that will be invoked whenever the module is loaded.

Accepts the function name (as an atom) of a function in the current module. The function must have an arity of 0 (no arguments). If the function does not return `:ok`, the loading of the module will be aborted. Its primary use case is to load NIFs:

```
defmodule MyModule do
  @on_load :load_external_code

  def load_external_code do
    :erlang.load_nif(~c"path/to/extension.so_or_dll")
  end
end
```

The function given to `on_load` should avoid calling functions from other modules. This is because, when running a `mix release`, `on_load` runs extremely early, before any application starts running, and therefore even systems like the `Logger` and `IO` are not yet available.

`@vsn`

Specify the module version. Accepts any valid Elixir value, for example:

```
defmodule MyModule do
  @vsn "1.0"
end
```

Struct attributes

- `@derive` - derives an implementation for the given protocol for the struct defined in the current module
- `@enforce_keys` - ensures the given keys are always set when building the struct defined in the current module

See `defstruct/1` for more information on building and using structs.

Typespec attributes

The following attributes are part of typespecs and are also built-in in Elixir:

- `@type` - defines a type to be used in `@spec`
- `@typep` - defines a private type to be used in `@spec`
- `@opaque` - defines an opaque type to be used in `@spec`
- `@spec` - provides a specification for a function
- `@callback` - provides a specification for a behaviour callback (and generates a `behaviour_info/1` function in the module, see below)
- `@macrocallback` - provides a specification for a macro behaviour callback
- `@optional_callbacks` - specifies which behaviour callbacks and macro behaviour callbacks are optional
- `@impl` - declares an implementation of a callback function or macro

For detailed documentation, see the typespec documentation.

Custom attributes

In addition to the built-in attributes outlined above, custom attributes may also be added. Custom attributes are expressed using the `@/1` operator followed by a valid variable name. The value given to the custom attribute must be a valid Elixir value:

```
defmodule MyModule do
  @custom_attr [some: "stuff"]
end
```

For more advanced options available when defining custom attributes, see `register_attribute/3`.

Compile callbacks

There are three compilation callbacks, invoked in this order: `@before_compile`, `@after_compile`, and `@after_verify`. They are described next.

`@before_compile`

A hook that will be invoked before the module is compiled. This is often used to change how the current module is being compiled.

Accepts a module or a `{module, function_or_macro_name}` tuple. The function/macro must take one argument: the module environment. If it's a macro, its returned value will be injected at the end of the module definition before the compilation starts.

When just a module is provided, the function/macro is assumed to be `__before_compile__/1`.

Callbacks will run in the order they are registered. Any overridable definition will be made concrete before the first callback runs. A definition may be made overridable again in another before compile callback and it will be made concrete one last time after all callbacks run.

Note: the callback function/macro must be placed in a separate module (because when the callback is invoked, the current module does not yet exist).

Example

```elixir
defmodule A do
  defmacro __before_compile__(_env) do
    quote do
      def hello, do: "world"
    end
  end
end

defmodule B do
  @before_compile A
end

B.hello()
#=> "world"
```

`@after_compile`

A hook that will be invoked with the bytecode of the current module. Although the module has already been compiled, its bytecode may not have been loaded to memory nor written to disk. For those reasons, prefer to use `@after_verify` callbacks or use `Code.ensure_compiled!/1` to wait until the module is fully available for introspection/invocation.

Accepts a module or a `{module, function_name}` tuple. The function must take two arguments: the module environment and its bytecode. When just a module is provided, the function is assumed to be `__after_compile__/2`.

Callbacks will run in the order they are registered.

`Module` functions expecting not yet compiled modules (such as `definitions_in/1`) are still available at the time `@after_compile` is invoked.

Example

```
defmodule MyModule do
  @after_compile __MODULE__

  def __after_compile__(env, _bytecode) do
    IO.inspect(env)
  end
end
```

`@after_verify`

A hook that will be invoked right after the current module is verified for undefined functions, deprecations, etc. A module is always verified after it is compiled. In Mix projects, a module is also verified when any of its runtime dependencies change. Therefore this is useful to perform verification of the current module while avoiding compile-time dependencies. Given the callback is invoked under different scenarios, Elixir provides no guarantees of when in the compilation cycle nor in which process the callback runs.

Furthermore, after verification callbacks are not expected to raise. Given they run after the code is compiled, artifacts have already been written to disk, and therefore raising does not effectively halt compilation and may leave unused artifacts on disk. If you must raise, use `@after_compile` or other callback. Given modules have already been compiled, functions in this module, such as `get_attribute/2`, which expect modules to not have been yet compiled, do not work on `@after_verify` callback.

Accepts a module or a `{module, function_name}` tuple. The function must take one argument: the module name. When just a module is provided, the function is assumed to be `__after_verify__/1`.

Callbacks will run in the order they are registered.

Example

```
defmodule MyModule do
  @after_verify __MODULE__

  def __after_verify__(module) do
    IO.inspect(module)
    :ok
  end
end
```

Compile options

The `@compile` attribute accepts different options that are used by both Elixir and Erlang compilers. Some of the common use cases are documented below:

- `@compile :debug_info` - includes `:debug_info` regardless of the corresponding setting in `Code.get_compiler_option/1`
- `@compile {:debug_info, false}` - disables `:debug_info` regardless of the corresponding setting in `Code.get_compiler_option/1`. Note disabling `:debug_info` is not recommended as it removes the ability of the Elixir compiler and other tools to static analyse the code. If you want to remove the `:debug_info` while deploying, tools like `mix release` already do such by default.
- `@compile {:inline, some_fun: 2, other_fun: 3}` - inlines the given name/arity pairs. Inlining is applied locally, calls from another module are not affected by this option

- `@compile {:autoload, true}` - configures if modules are automatically loaded after definition. It defaults to `false` when compiling modules to `.beam` files in disk (as the modules are then lazily loaded from disk). If modules are not compiled to disk, then they are always loaded, regardless of this flag

- `@compile {:no_warn_undefined, Mod}` or `@compile {:no_warn_undefined, {Mod, fun, arity}}` - does not warn if the given module or the given `Mod.fun/arity` are not defined

Generated functions

Sometimes the compiler will generate public functions within modules. These are documented below.

`behaviour_info/1`

This function is generated for modules that define a behaviour, that is, that have one or more `@callback` definitions. The signature for this function, expressed as a spec, is:

```
@spec behaviour_info(:callbacks) :: [function_info]
  when function_info: {function_name :: atom(), arity ::
non_neg_integer()}

@spec behaviour_info(:optional_callbacks) ::
[function_info]
  when function_info: {function_name :: atom(), arity ::
non_neg_integer()}
```

`behaviour_info(:callbacks)` includes optional callbacks.

For example:

```
iex> Enum.sort(GenServer.behaviour_info(:callbacks))
[
  code_change: 3,
  format_status: 1,
  format_status: 2,
  handle_call: 3,
  handle_cast: 2,
  handle_continue: 2,
  handle_info: 2,
  init: 1,
  terminate: 2
]
```

`module_info/0`

This function is generated for all modules. It returns all the attributes returned by `module_info/1` (see below), but as a single keyword list. See also the Erlang documentation.

`module_info/1`

This function is generated for all modules and returns information about the module. The signature for this function, expressed as a spec, is:

```
@spec module_info(:module) :: module() # Returns the
module itself
@spec module_info(:attributes) :: keyword()
@spec module_info(:compile) :: keyword()
@spec module_info(:md5) :: binary()
@spec module_info(:nifs) :: [function_info]
  when function_info: {function_name :: atom(), arity ::
non_neg_integer()}
@spec module_info(:exports) :: [function_info]
```

```
  when function_info: {function_name :: atom(), arity ::
non_neg_integer()}
@spec module_info(:functions) :: [function_info]
  when function_info: {function_name :: atom(), arity ::
non_neg_integer()}
```

For example:

```
iex> URI.module_info(:module)
URI
iex> {:decode_www_form, 1} in URI.module_info(:exports)
true
```

For more information about `module_info/1`, also check out the Erlang documentation.

`__info__/1`

This function is generated for all modules. It's similar to `module_info/1` but includes some additional Elixir-specific information, such as struct and macro information. For documentation, see `Module.__info__/1`.

Summary

Types

create_opts()
def_kind()
definition()
get_definition_opts()

Callbacks

__info__(atom)

Provides runtime information about functions, macros, and other information defined by the module.

Functions

attributes_in(module)

Returns all module attributes names defined in `module`.

concat(list)

Concatenates a list of aliases and returns a new alias.

concat(left, right)

Concatenates two aliases and returns a new alias.

create(module, quoted, opts)

Creates a module with the given name and defined by the given quoted expressions.

defines?(module, definition)

Checks if the module defines the given function or macro.

defines?(module, definition, def_kind)

Checks if the module defines a function or macro of the given kind.

defines_type?(module, definition)

Checks if the current module defines the given type (private, opaque or not).

definitions_in(module)

Returns all functions and macros defined in `module`.

definitions_in(module, def_kind)

Returns all functions defined in `module`, according to its kind.

delete_attribute(module, key)

Deletes the entry (or entries) for the given module attribute.

delete_definition(module, definition)

Deletes a definition from a module.

eval_quoted(module_or_env, quoted, binding \\ [], opts \\ []) deprecated
get_attribute(module, key, default \\ nil)

Gets the given attribute from a module.

get_definition(module, definition, options \\ [])

Returns the definition for the given name-arity pair.

get_last_attribute(module, key, default \\ nil)

Gets the last set value of a given attribute from a module.

has_attribute?(module, key)

Checks if the given attribute has been defined.

make_overridable(module, definitions)

Makes the given functions in `module` overridable.

open?(module)

Checks if a module is open.

overridable?(module, definition)

Returns `true` if `tuple` in `module` was marked as overridable at some point.

overridables_in(module)

Returns all overridable definitions in `module`.

put_attribute(module, key, value)

Puts a module attribute with `key` and `value` in the given `module`.

register_attribute(module, attribute, options)

Registers an attribute.

reserved_attributes()

Returns information about module attributes used by Elixir.

safe_concat(list)

Concatenates a list of aliases and returns a new alias only if the alias was already referenced.

safe_concat(left, right)

Concatenates two aliases and returns a new alias only if the alias was already referenced.

spec_to_callback(module, definition)

Copies the given spec as a callback.

split(module)

Splits the given module name into binary parts.

Types

create_opts()

```
@type create_opts() :: [file: binary(), line: pos_integer(),
generated: boolean()]
```

def_kind()

```
@type def_kind() :: :def | :defp | :defmacro | :defmacrop
```

definition()

```
@type definition() :: {function_name :: atom(), arity()}
```

get_definition_opts()

```
@type get_definition_opts() :: [{:skip_clauses, boolean()}]
```

Callbacks

__info__(atom)

```
@callback __info__(:attributes) :: keyword()
@callback __info__(:compile) :: [term()]
@callback __info__(:functions) :: keyword()
@callback __info__(:macros) :: keyword()
@callback __info__(:md5) :: binary()
@callback __info__(:module) :: module()
@callback __info__(:struct) ::
  [
    %{
      :field => atom(),
      optional(:required) => boolean(),
      optional(:default) => term()
    }
  ]
  | nil
```

Provides runtime information about functions, macros, and other information defined by the module.

Each module gets an `__info__/1` function when it's compiled. The function takes one of the following items:

- `:attributes` - a keyword list with all persisted attributes
- `:compile` - a list with compiler metadata
- `:functions` - a keyword list of public functions and their arities
- `:macros` - a keyword list of public macros and their arities
- `:md5` - the MD5 of the module
- `:module` - the module atom name

- `:struct` - (since v1.14.0) if the module defines a struct and if so each field in order. See `Macro.struct_info!/2` for more information

Functions

attributes_in(module)

(since 1.13.0)

```
@spec attributes_in(module()) :: [atom()]
```

Returns all module attributes names defined in `module`.

This function can only be used on modules that have not yet been compiled.

Examples

```
defmodule Example do
  @foo 1
  Module.register_attribute(__MODULE__, :bar, accumulate:
true)

  :foo in Module.attributes_in(__MODULE__)
  #=> true

  :bar in Module.attributes_in(__MODULE__)
  #=> true
end
```

concat(list)

```
@spec concat([binary() | atom()]) :: atom()
```

Concatenates a list of aliases and returns a new alias.

It handles binaries and atoms.

> **Untracked compile-time dependencies**
>
> Use this function with care, as dynamically defining module names at compilation time may lead to untracked compile-time dependencies.

Examples

```
iex> Module.concat([Foo, Bar])
Foo.Bar

iex> Module.concat([Foo, "Bar"])
Foo.Bar
```

concat(left, right)

```
@spec concat(binary() | atom(), binary() | atom()) :: atom()
```

Concatenates two aliases and returns a new alias.

It handles binaries and atoms. If one of the aliases is nil, it is discarded.

> **Untracked compile-time dependencies**
>
> Use this function with care, as dynamically defining module names at compilation time may lead to untracked compile-time dependencies.

Examples

```
iex> Module.concat(Foo, Bar)
Foo.Bar

iex> Module.concat(Foo, "Bar")
Foo.Bar

iex> Module.concat(Foo, nil)
Foo
```

create(module, quoted, opts)

```
@spec create(module(), Macro.t(), Macro.Env.t() |
create_opts()) ::
  {:module, module(), binary(), term()}
```

Creates a module with the given name and defined by the given quoted expressions.

The line where the module is defined and its file **must** be passed as options. See `Code.env_for_eval/1` for a complete list of options.

It returns a tuple of shape `{:module, module, binary, term}` where `module` is the module name, `binary` is the module bytecode and `term` is the result of the last expression in `quoted`.

Similar to `Kernel.defmodule/2`, the binary will only be written to disk as a `.beam` file if `Module.create/3` is invoked in a file that is currently being compiled.

Examples

```
contents =
  quote do
    def world, do: true
  end

Module.create(Hello, contents,
Macro.Env.location(__ENV__))

Hello.world()
#=> true
```

Differences from `defmodule`

`Module.create/3` works similarly to `Kernel.defmodule/2` and return the same results. While one could also use `Kernel.defmodule/2` to define modules dynamically, this function is preferred when the module body is given by a quoted expression.

Another important distinction is that `Module.create/3` allows you to control the environment variables used when defining the module, while `Kernel.defmodule/2` automatically uses the environment it is invoked at.

defines?(module, definition)

```
@spec defines?(module(), definition()) :: boolean()
```

Checks if the module defines the given function or macro.

Use `defines?/3` to assert for a specific type.

This function can only be used on modules that have not yet been compiled. Use `Kernel.function_exported?/3` and `Kernel.macro_exported?/3` to check for public functions and macros respectively in compiled modules.

Note that `defines?` returns `false` for functions and macros that have been defined but then marked as overridable and no other implementation has been provided. You can check the overridable status by calling `overridable?/2`.

Examples

```
defmodule Example do
  Module.defines?(__MODULE__, {:version, 0}) #=> false
  def version, do: 1
  Module.defines?(__MODULE__, {:version, 0}) #=> true
end
```

defines?(module, definition, def_kind)

```
@spec defines?(module(), definition(), def_kind()) ::
boolean()
```

Checks if the module defines a function or macro of the given kind.

`def_kind` can be any of `:def`, `:defp`, `:defmacro`, or `:defmacrop`.

This function can only be used on modules that have not yet been compiled. Use `Kernel.function_exported?/3` and

Kernel.macro_exported?/3 to check for public functions and macros respectively in compiled modules.

Examples

```
defmodule Example do
  Module.defines?(__MODULE__, {:version, 0}, :def) #=>
false
  def version, do: 1
  Module.defines?(__MODULE__, {:version, 0}, :def) #=>
true
end
```

defines_type?(module, definition)

(since 1.7.0)

```
@spec defines_type?(module(), definition()) :: boolean()
```

Checks if the current module defines the given type (private, opaque or not).

This function is only available for modules being compiled.

definitions_in(module)

```
@spec definitions_in(module()) :: [definition()]
```

Returns all functions and macros defined in `module`.

It returns a list with all defined functions and macros, public and private, in the shape of `[{name, arity}, ...]`.

This function can only be used on modules that have not yet been compiled. Use the `Module.__info__/1` callback to get the public functions and macros in compiled modules.

Examples

```
defmodule Example do
  def version, do: 1
  defmacrop test(arg), do: arg
  Module.definitions_in(__MODULE__) #=> [{:version, 0},
{:test, 1}]
end
```

definitions_in(module, def_kind)

```
@spec definitions_in(module(), def_kind()) :: [definition()]
```

Returns all functions defined in `module`, according to its kind.

This function can only be used on modules that have not yet been compiled. Use the `Module.__info__/1` callback to get the public functions and macros in compiled modules.

Examples

```
defmodule Example do
  def version, do: 1
  Module.definitions_in(__MODULE__, :def)  #=>
[{:version, 0}]
  Module.definitions_in(__MODULE__, :defp) #=> []
end
```

delete_attribute(module, key)

```
@spec delete_attribute(module(), atom()) :: term()
```

Deletes the entry (or entries) for the given module attribute.

It returns the deleted attribute value. If the attribute has not been set nor configured to accumulate, it returns `nil`.

If the attribute is set to accumulate, then this function always returns a list. Deleting the attribute removes existing entries but the attribute will still accumulate.

Examples

```
defmodule MyModule do
  Module.put_attribute(__MODULE__,
:custom_threshold_for_lib, 10)
  Module.delete_attribute(__MODULE__,
:custom_threshold_for_lib)
end
```

delete_definition(module, definition)

(since 1.12.0)

```
@spec delete_definition(module(), definition()) :: boolean()
```

Deletes a definition from a module.

It returns `true` if the definition exists and it was removed, otherwise it returns `false`.

eval_quoted(module_or_env, quoted, binding \\ [], opts \\ [])

This function is deprecated. Use Code.eval_quoted/3 instead.

get_attribute(module, key, default \\ nil)

```
@spec get_attribute(module(), atom(), term()) :: term()
```

Gets the given attribute from a module.

If the attribute was marked with `accumulate` with `Module.register_attribute/3`, a list is always returned. `nil` is returned if the attribute has not been marked with `accumulate` and has not been set to any value.

The `@` macro compiles to a call to this function. For example, the following code:

```
@foo
```

Expands to something akin to:

```
Module.get_attribute(__MODULE__, :foo)
```

This function can only be used on modules that have not yet been compiled. Use the `Module.__info__/1` callback to get all persisted attributes, or `Code.fetch_docs/1` to retrieve all documentation related attributes in compiled modules.

Examples

```
defmodule Foo do
  Module.put_attribute(__MODULE__, :value, 1)
  Module.get_attribute(__MODULE__, :value) #=> 1

  Module.get_attribute(__MODULE__, :value, :default) #=>
1
  Module.get_attribute(__MODULE__, :not_found, :default)
#=> :default

  Module.register_attribute(__MODULE__, :value,
accumulate: true)
  Module.put_attribute(__MODULE__, :value, 1)
  Module.get_attribute(__MODULE__, :value) #=> [1]
end
```

get_definition(module, definition, options \\ [])

(since 1.12.0)

```
@spec get_definition(module(), definition(),
get_definition_opts()) ::
  {:v1, def_kind(), meta :: keyword(),
   [
     {meta :: keyword(), arguments :: [Macro.t()], guards ::
[Macro.t()],
      Macro.t()}
   ]}
  | nil
```

Returns the definition for the given name-arity pair.

It returns a tuple with the `version`, the `kind`, the definition `metadata`, and a list with each clause. Each clause is a four-element tuple with metadata, the arguments, the guards, and the clause AST.

The clauses are returned in the Elixir AST but a subset that has already been expanded and normalized. This makes it useful for analyzing code but it cannot be reinjected into the module as it will have lost some of its original context. Given this AST representation is mostly internal, it is versioned and it may change at any time. Therefore, **use this API with caution**.

Options

- `:skip_clauses` (since v1.14.0) - returns `[]` instead of returning the clauses. This is useful when there is only an interest in fetching the kind and the metadata

get_last_attribute(module, key, default \\ nil)

(since 1.15.0)

```
@spec get_last_attribute(module(), atom(), term()) :: term()
```

Gets the last set value of a given attribute from a module.

If the attribute was marked with `accumulate` with `Module.register_attribute/3`, the previous value to have been set will be returned. If the attribute does not accumulate, this call is the same as calling `Module.get_attribute/3`.

This function can only be used on modules that have not yet been compiled. Use the `Module.__info__/1` callback to get all persisted attributes, or `Code.fetch_docs/1` to retrieve all documentation related attributes in compiled modules.

Examples

```
defmodule Foo do
  Module.put_attribute(__MODULE__, :value, 1)
  Module.get_last_attribute(__MODULE__, :value) #=> 1

  Module.get_last_attribute(__MODULE__, :not_found,
:default) #=> :default

  Module.register_attribute(__MODULE__, :acc, accumulate:
true)
  Module.put_attribute(__MODULE__, :acc, 1)
  Module.get_last_attribute(__MODULE__, :acc) #=> 1
  Module.put_attribute(__MODULE__, :acc, 2)
  Module.get_last_attribute(__MODULE__, :acc) #=> 2
end
```

has_attribute?(module, key)

(since 1.10.0)

```
@spec has_attribute?(module(), atom()) :: boolean()
```

Checks if the given attribute has been defined.

An attribute is defined if it has been registered with `register_attribute/3` or assigned a value. If an attribute has been deleted with `delete_attribute/2` it is no longer considered defined.

This function can only be used on modules that have not yet been compiled.

Examples

```
defmodule MyModule do
  @value 1
  Module.register_attribute(__MODULE__, :other_value)
  Module.put_attribute(__MODULE__, :another_value, 1)

  Module.has_attribute?(__MODULE__, :value) #=> true
  Module.has_attribute?(__MODULE__, :other_value) #=>
true
  Module.has_attribute?(__MODULE__, :another_value) #=>
true

  Module.has_attribute?(__MODULE__, :undefined) #=> false

  Module.delete_attribute(__MODULE__, :value)
  Module.has_attribute?(__MODULE__, :value) #=> false
end
```

make_overridable(module, definitions)

```
@spec make_overridable(module(), [definition()]) :: :ok
@spec make_overridable(module(), module()) :: :ok
```

Makes the given functions in `module` overridable.

An overridable function is lazily defined, allowing a developer to customize it. See `Kernel.defoverridable/1` for more information and documentation.

Once a function or a macro is marked as overridable, it will no longer be listed under `definitions_in/1` or return true when given to `defines?/2` until another implementation is given.

open?(module)

```
@spec open?(module()) :: boolean()
```

Checks if a module is open.

A module is "open" if it is currently being defined and its attributes and functions can be modified.

overridable?(module, definition)

```
@spec overridable?(module(), definition()) :: boolean()
```

Returns `true` if `tuple` in `module` was marked as overridable at some point.

Note `overridable?/2` returns `true` even if the definition was already overridden. You can use `defines?/2` to see if a definition exists or one is pending.

overridables_in(module)

(since 1.13.0)

```
@spec overridables_in(module()) :: [definition()]
```

Returns all overridable definitions in `module`.

Note a definition is included even if it was already overridden. You can use `defines?/2` to see if a definition exists or one is pending.

This function can only be used on modules that have not yet been compiled.

Examples

```
defmodule Example do
  def foo, do: 1
  def bar, do: 2

  defoverridable foo: 0, bar: 0
  def foo, do: 3

  [bar: 0, foo: 0] = Module.overridables_in(__MODULE__)
|> Enum.sort()
end
```

put_attribute(module, key, value)

```
@spec put_attribute(module(), atom(), term()) :: :ok
```

Puts a module attribute with `key` and `value` in the given `module`.

Examples

```
defmodule MyModule do
  Module.put_attribute(__MODULE__,
:custom_threshold_for_lib, 10)
end
```

register_attribute(module, attribute, options)

```
@spec register_attribute(module(), atom(), accumulate:
boolean(), persist: boolean()) ::
  :ok
```

Registers an attribute.

By registering an attribute, a developer is able to customize how Elixir will store and accumulate the attribute values.

Options

When registering an attribute, two options can be given:

- `:accumulate` - several calls to the same attribute will accumulate instead of overriding the previous one. New attributes are always added to the top of the accumulated list.
- `:persist` - the attribute will be persisted in the Erlang Abstract Format. Useful when interfacing with Erlang libraries.

By default, both options are `false`. Once an attribute has been set to accumulate or persist, the behaviour cannot be reverted.

Examples

```
defmodule MyModule do
  Module.register_attribute(__MODULE__,
:custom_threshold_for_lib, accumulate: true)

  @custom_threshold_for_lib 10
  @custom_threshold_for_lib 20
```

```
  @custom_threshold_for_lib #=> [20, 10]
end
```

reserved_attributes()

(since 1.12.0)

```
@spec reserved_attributes() :: map()
```

Returns information about module attributes used by Elixir.

See the "Module attributes" section in the module documentation for more information on each attribute.

Examples

```
iex> map = Module.reserved_attributes()
iex> Map.has_key?(map, :moduledoc)
true
iex> Map.has_key?(map, :doc)
true
```

safe_concat(list)

```
@spec safe_concat([binary() | atom()]) :: atom()
```

Concatenates a list of aliases and returns a new alias only if the alias was already referenced.

If the alias was not referenced yet, fails with `ArgumentError`. It handles binaries and atoms.

> **Untracked compile-time dependencies**
>
> Use this function with care, as dynamically defining module names at compilation time may lead to untracked compile-time dependencies.

Examples

```
iex> Module.safe_concat([List, Chars])
List.Chars
```

safe_concat(left, right)

```
@spec safe_concat(binary() | atom(), binary() | atom()) ::
atom()
```

Concatenates two aliases and returns a new alias only if the alias was already referenced.

If the alias was not referenced yet, fails with `ArgumentError`. It handles binaries and atoms.

> **Untracked compile-time dependencies**
>
> Use this function with care, as dynamically defining module names at compilation time may lead to untracked compile-time dependencies.

Examples

```
iex> Module.safe_concat(List, Chars)
List.Chars
```

spec_to_callback(module, definition)

(since 1.7.0)

```
@spec spec_to_callback(module(), definition()) :: boolean()
```

Copies the given spec as a callback.

Returns `true` if there is such a spec and it was copied as a callback. If the function associated to the spec has documentation defined prior to invoking this function, the docs are copied too.

split(module)

```
@spec split(module() | String.t()) :: [String.t(), ...]
```

Splits the given module name into binary parts.

`module` has to be an Elixir module, as `split/1` won't work with Erlang-style modules (for example, `split(:lists)` raises an error).

`split/1` also supports splitting the string representation of Elixir modules (that is, the result of calling `Atom.to_string/1` with the module name).

Examples

```
iex> Module.split(Very.Long.Module.Name.And.Even.Longer)
["Very", "Long", "Module", "Name", "And", "Even",
"Longer"]
```

```
iex> Module.split("Elixir.String.Chars")
["String", "Chars"]
```

NaiveDateTime

A NaiveDateTime struct (without a time zone) and functions.

The NaiveDateTime struct contains the fields year, month, day, hour, minute, second, microsecond and calendar. New naive datetimes can be built with the `new/2` and `new/8` functions or using the `~N` (see `sigil_N/2`) sigil:

```
iex> ~N[2000-01-01 23:00:07]
~N[2000-01-01 23:00:07]
```

The date and time fields in the struct can be accessed directly:

```
iex> naive = ~N[2000-01-01 23:00:07]
iex> naive.year
2000
iex> naive.second
7
```

We call them "naive" because this datetime representation does not have a time zone. This means the datetime may not actually exist in certain areas in the world even though it is valid.

For example, when daylight saving changes are applied by a region, the clock typically moves forward or backward by one hour. This means certain datetimes never occur or may occur more than once. Since `NaiveDateTime` is not validated against a time zone, such errors would go unnoticed.

Developers should avoid creating the NaiveDateTime structs directly and instead, rely on the functions provided by this module as well as the ones in third-party calendar libraries.

Comparing naive date times

Comparisons in Elixir using `==/2`, `>/2`, `</2` and similar are structural and based on the `NaiveDateTime` struct fields. For proper comparison between naive datetimes, use the `compare/2`, `after?/2` and `before?/2` functions. The existence of the `compare/2` function in this module also allows using `Enum.min/2` and `Enum.max/2` functions to get the minimum and maximum naive datetime of an `Enum`. For example:

```
iex> Enum.min([~N[2020-01-01 23:00:07], ~N[2000-01-01
23:00:07]], NaiveDateTime)
~N[2000-01-01 23:00:07]
```

Using epochs

The `add/3` and `diff/3` functions can be used for computing date times or retrieving the number of seconds between instants. For example, if there is an interest in computing the number of seconds from the Unix epoch (1970-01-01 00:00:00):

```
iex> NaiveDateTime.diff(~N[2010-04-17 14:00:00], ~N[1970-
01-01 00:00:00])
1271512800

iex> NaiveDateTime.add(~N[1970-01-01 00:00:00],
1_271_512_800)
~N[2010-04-17 14:00:00]
```

Those functions are optimized to deal with common epochs, such as the Unix Epoch above or the Gregorian Epoch (0000-01-01 00:00:00).

Summary

Types

t()

Functions

add(naive_datetime, amount_to_add, unit \\ :second)

Adds a specified amount of time to a `NaiveDateTime`.

after?(naive_datetime1, naive_datetime2)

Returns `true` if the first `NaiveDateTime` is strictly later than the second.

before?(naive_datetime1, naive_datetime2)

Returns `true` if the first `NaiveDateTime` is strictly earlier than the second.

beginning_of_day(naive_datetime)

Calculates a `NaiveDateTime` that is the first moment for the given `NaiveDateTime`.

compare(naive_datetime1, naive_datetime2)

Compares two `NaiveDateTime` structs.

convert(ndt, calendar)

Converts the given `naive_datetime` from one calendar to another.

convert!(naive_datetime, calendar)

Converts the given `naive_datetime` from one calendar to another.

diff(naive_datetime1, naive_datetime2, unit \\ :second)

Subtracts `naive_datetime2` from `naive_datetime1`.

end_of_day(naive_datetime)

Calculates a `NaiveDateTime` that is the last moment for the given `NaiveDateTime`.

from_erl(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Converts an Erlang datetime tuple to a `NaiveDateTime` struct.

from_erl!(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Converts an Erlang datetime tuple to a `NaiveDateTime` struct.

from_gregorian_seconds(seconds, microsecond_precision \\ {0, 0}, calendar \\ Calendar.ISO)

Converts a number of gregorian seconds to a `NaiveDateTime` struct.

from_iso8601(string, calendar \\ Calendar.ISO)

Parses the extended "Date and time of day" format described by ISO 8601:2019.

from_iso8601!(string, calendar \\ Calendar.ISO)

Parses the extended "Date and time of day" format described by ISO 8601:2019.

local_now(calendar \\ Calendar.ISO)

Returns the "local time" for the machine the Elixir program is running on.
new(date, time)

Builds a naive datetime from date and time structs.

new(year, month, day, hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Builds a new ISO naive datetime.

new!(date, time)

Builds a naive datetime from date and time structs.

new!(year, month, day, hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Builds a new ISO naive datetime.

shift(naive_datetime, duration)

Shifts given `naive_datetime` by `duration` according to its calendar.

to_date(naive_datetime)

Converts a `NaiveDateTime` into a `Date`.

to_erl(naive_datetime)

Converts a `NaiveDateTime` struct to an Erlang datetime tuple.

to_gregorian_seconds(naive_datetime)

Converts a `NaiveDateTime` struct to a number of gregorian seconds and microseconds.

to_iso8601(naive_datetime, format \\ :extended)

Converts the given naive datetime to ISO 8601:2019.
to_string(naive_datetime)

Converts the given naive datetime to a string according to its calendar.

to_time(naive_datetime)

Converts a `NaiveDateTime` into `Time`.

truncate(naive_datetime, precision)

Returns the given naive datetime with the microsecond field truncated to the given precision (`:microsecond`, `:millisecond` or `:second`).

utc_now(calendar_or_time_unit \\ Calendar.ISO)

Returns the current naive datetime in UTC.

utc_now(time_unit, calendar)

Returns the current naive datetime in UTC, supporting a specific calendar and precision.

Types

t()

```
@type t() :: %NaiveDateTime{
  calendar: Calendar.calendar(),
  day: Calendar.day(),
  hour: Calendar.hour(),
  microsecond: Calendar.microsecond(),
  minute: Calendar.minute(),
  month: Calendar.month(),
  second: Calendar.second(),
```

```
  year: Calendar.year()
}
```

Functions

add(naive_datetime, amount_to_add, unit \\ :second)

(since 1.4.0)

```
@spec add(
  Calendar.naive_datetime(),
  integer(),
  :day | :hour | :minute | System.time_unit()
) :: t()
```

Adds a specified amount of time to a `NaiveDateTime`.

> **Prefer** `shift/2`
>
> Prefer `shift/2` over `add/3`, as it offers a more ergonomic API.
>
> `add/3` provides a lower-level API which only supports fixed units such as `:hour` and `:second`, but not `:month` (as the exact length of a month depends on the current month). `add/3` always considers the unit to be computed according to the `Calendar.ISO`.

Accepts an `amount_to_add` in any `unit`. `unit` can be `:day`, `:hour`, `:minute`, `:second` or any subsecond precision from `System.time_unit/0` for convenience but ultimately they are all converted to microseconds. Negative values will move backwards in time and the default precision is `:second`.

Examples

It uses seconds by default:

```
# adds seconds by default
iex> NaiveDateTime.add(~N[2014-10-02 00:29:10], 2)
~N[2014-10-02 00:29:12]

# accepts negative offsets
iex> NaiveDateTime.add(~N[2014-10-02 00:29:10], -2)
~N[2014-10-02 00:29:08]
```

It can also work with subsecond precisions:

```
iex> NaiveDateTime.add(~N[2014-10-02 00:29:10], 2_000,
:millisecond)
~N[2014-10-02 00:29:12.000]
```

As well as days/hours/minutes:

```
iex> NaiveDateTime.add(~N[2015-02-28 00:29:10], 2, :day)
~N[2015-03-02 00:29:10]
iex> NaiveDateTime.add(~N[2015-02-28 00:29:10], 36,
:hour)
~N[2015-03-01 12:29:10]
iex> NaiveDateTime.add(~N[2015-02-28 00:29:10], 60,
:minute)
~N[2015-02-28 01:29:10]
```

This operation merges the precision of the naive date time with the given unit:

```
iex> result = NaiveDateTime.add(~N[2014-10-02 00:29:10],
21, :millisecond)
~N[2014-10-02 00:29:10.021]
```

```
iex> result.microsecond
{21000, 3}
```

Operations on top of gregorian seconds or the Unix epoch are optimized:

```
# from Gregorian seconds
iex> NaiveDateTime.add(~N[0000-01-01 00:00:00],
63_579_428_950)
~N[2014-10-02 00:29:10]
```

Passing a `DateTime` automatically converts it to `NaiveDateTime`, discarding the time zone information:

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> NaiveDateTime.add(dt, 21, :second)
~N[2000-02-29 23:00:28]
```

after?(naive_datetime1, naive_datetime2)

(since 1.15.0)

```
@spec after?(Calendar.naive_datetime(),
Calendar.naive_datetime()) :: boolean()
```

Returns `true` if the first `NaiveDateTime` is strictly later than the second.

Examples

```
iex> NaiveDateTime.after?(~N[2022-02-02 11:00:00],
~N[2021-01-01 11:00:00])
true
iex> NaiveDateTime.after?(~N[2021-01-01 11:00:00],
~N[2021-01-01 11:00:00])
false
iex> NaiveDateTime.after?(~N[2021-01-01 11:00:00],
~N[2022-02-02 11:00:00])
false
```

before?(naive_datetime1, naive_datetime2)

(since 1.15.0)

```
@spec before?(Calendar.naive_datetime(),
Calendar.naive_datetime()) :: boolean()
```

Returns `true` if the first `NaiveDateTime` is strictly earlier than the second.

Examples

```
iex> NaiveDateTime.before?(~N[2021-01-01 11:00:00],
~N[2022-02-02 11:00:00])
true
iex> NaiveDateTime.before?(~N[2021-01-01 11:00:00],
~N[2021-01-01 11:00:00])
false
iex> NaiveDateTime.before?(~N[2022-02-02 11:00:00],
~N[2021-01-01 11:00:00])
false
```

beginning_of_day(naive_datetime)

(since 1.15.0)

```
@spec beginning_of_day(Calendar.naive_datetime()) :: t()
```

Calculates a `NaiveDateTime` that is the first moment for the given `NaiveDateTime`.

To calculate the beginning of day of a `DateTime`, call this function, then convert back to a `DateTime`:

```
datetime
|> NaiveDateTime.beginning_of_day()
|> DateTime.from_naive(datetime.time_zone)
```

Note that the beginning of the day may not exist or be ambiguous in a given timezone, so you must handle those cases accordingly.

Examples

```
iex> NaiveDateTime.beginning_of_day(~N[2000-01-01
23:00:07.123456])
~N[2000-01-01 00:00:00.000000]
```

compare(naive_datetime1, naive_datetime2)

(since 1.4.0)

```
@spec compare(Calendar.naive_datetime(),
Calendar.naive_datetime()) :: :lt | :eq | :gt
```

Compares two `NaiveDateTime` structs.

Returns `:gt` if first is later than the second and `:lt` for vice versa. If the two NaiveDateTime are equal `:eq` is returned.

Examples

```
iex> NaiveDateTime.compare(~N[2016-04-16 13:30:15],
~N[2016-04-28 16:19:25])
:lt
iex> NaiveDateTime.compare(~N[2016-04-16 13:30:15.1],
~N[2016-04-16 13:30:15.01])
:gt
```

This function can also be used to compare a DateTime without the time zone information:

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> NaiveDateTime.compare(dt, ~N[2000-02-29 23:00:07])
:eq
iex> NaiveDateTime.compare(dt, ~N[2000-01-29 23:00:07])
:gt
iex> NaiveDateTime.compare(dt, ~N[2000-03-29 23:00:07])
:lt
```

convert(ndt, calendar)

(since 1.5.0)

```
@spec convert(Calendar.naive_datetime(), Calendar.calendar())
::
  {:ok, t()} | {:error, :incompatible_calendars}
```

Converts the given `naive_datetime` from one calendar to another.

If it is not possible to convert unambiguously between the calendars (see `Calendar.compatible_calendars?/2`), an `{:error, :incompatible_calendars}` tuple is returned.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> NaiveDateTime.convert(~N[2000-01-01 13:30:15],
Calendar.Holocene)
{:ok, %NaiveDateTime{calendar: Calendar.Holocene, year:
12000, month: 1, day: 1,
                     hour: 13, minute: 30, second: 15,
microsecond: {0, 0}}}
```

convert!(naive_datetime, calendar)

(since 1.5.0)

```
@spec convert!(Calendar.naive_datetime(),
Calendar.calendar()) :: t()
```

Converts the given `naive_datetime` from one calendar to another.

If it is not possible to convert unambiguously between the calendars (see `Calendar.compatible_calendars?/2`), an ArgumentError is raised.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> NaiveDateTime.convert!(~N[2000-01-01 13:30:15],
Calendar.Holocene)
%NaiveDateTime{calendar: Calendar.Holocene, year: 12000,
month: 1, day: 1,
               hour: 13, minute: 30, second: 15,
microsecond: {0, 0}}
```

diff(naive_datetime1, naive_datetime2, unit \\ :second)

(since 1.4.0)

```
@spec diff(
  Calendar.naive_datetime(),
  Calendar.naive_datetime(),
  :day | :hour | :minute | System.time_unit()
) :: integer()
```

Subtracts `naive_datetime2` from `naive_datetime1`.

The answer can be returned in any `:day`, `:hour`, `:minute`, or any `unit` available from `System.time_unit/0`. The unit is measured according to `Calendar.ISO` and defaults to `:second`.

Fractional results are not supported and are truncated.

Examples

```
iex> NaiveDateTime.diff(~N[2014-10-02 00:29:12], ~N[2014-
10-02 00:29:10])
2
iex> NaiveDateTime.diff(~N[2014-10-02 00:29:12], ~N[2014-
10-02 00:29:10], :microsecond)
2_000_000

iex> NaiveDateTime.diff(~N[2014-10-02 00:29:10.042],
~N[2014-10-02 00:29:10.021])
0
iex> NaiveDateTime.diff(~N[2014-10-02 00:29:10.042],
~N[2014-10-02 00:29:10.021], :millisecond)
21

iex> NaiveDateTime.diff(~N[2014-10-02 00:29:10], ~N[2014-
10-02 00:29:12])
-2
iex> NaiveDateTime.diff(~N[-0001-10-02 00:29:10],
~N[-0001-10-02 00:29:12])
-2
```

It can also compute the difference in days, hours, or minutes:

```
iex> NaiveDateTime.diff(~N[2014-10-10 00:29:10], ~N[2014-
10-02 00:29:10], :day)
8
iex> NaiveDateTime.diff(~N[2014-10-02 12:29:10], ~N[2014-
10-02 00:29:10], :hour)
12
iex> NaiveDateTime.diff(~N[2014-10-02 00:39:10], ~N[2014-
10-02 00:29:10], :minute)
10
```

But it also rounds incomplete days to zero:

```
iex> NaiveDateTime.diff(~N[2014-10-10 00:29:09], ~N[2014-
10-02 00:29:10], :day)
7
```

end_of_day(naive_datetime)

(since 1.15.0)

```
@spec end_of_day(Calendar.naive_datetime()) :: t()
```

Calculates a `NaiveDateTime` that is the last moment for the given `NaiveDateTime`.

To calculate the end of day of a `DateTime`, call this function, then convert back to a `DateTime`:

```
datetime
|> NaiveDateTime.end_of_day()
|> DateTime.from_naive(datetime.time_zone)
```

Note that the end of the day may not exist or be ambiguous in a given timezone, so you must handle those cases accordingly.

Examples

```
iex> NaiveDateTime.end_of_day(~N[2000-01-01
23:00:07.123456])
~N[2000-01-01 23:59:59.999999]
```

from_erl(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec from_erl(
  :calendar.datetime(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: {:ok, t()} | {:error, atom()}
```

Converts an Erlang datetime tuple to a `NaiveDateTime` struct.

Attempting to convert an invalid ISO calendar date will produce an error tuple.

Examples

```
iex> NaiveDateTime.from_erl({{2000, 1, 1}, {13, 30, 15}})
{:ok, ~N[2000-01-01 13:30:15]}
iex> NaiveDateTime.from_erl({{2000, 1, 1}, {13, 30, 15}},
5000)
{:ok, ~N[2000-01-01 13:30:15.005000]}
iex> NaiveDateTime.from_erl({{2000, 1, 1}, {13, 30, 15}},
{5000, 3})
{:ok, ~N[2000-01-01 13:30:15.005]}
iex> NaiveDateTime.from_erl({{2000, 13, 1}, {13, 30,
15}})
{:error, :invalid_date}
iex> NaiveDateTime.from_erl({{2000, 13, 1}, {13, 30,
15}})
{:error, :invalid_date}
```

from_erl!(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec from_erl!(
  :calendar.datetime(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: t()
```

Converts an Erlang datetime tuple to a `NaiveDateTime` struct.

Raises if the datetime is invalid. Attempting to convert an invalid ISO calendar date will produce an error tuple.

Examples

```
iex> NaiveDateTime.from_erl!({{2000, 1, 1}, {13, 30, 15}})
~N[2000-01-01 13:30:15]
iex> NaiveDateTime.from_erl!({{2000, 1, 1}, {13, 30, 15}}, 5000)
~N[2000-01-01 13:30:15.005000]
iex> NaiveDateTime.from_erl!({{2000, 1, 1}, {13, 30, 15}}, {5000, 3})
~N[2000-01-01 13:30:15.005]
iex> NaiveDateTime.from_erl!({{2000, 13, 1}, {13, 30, 15}})
** (ArgumentError) cannot convert {{2000, 13, 1}, {13, 30, 15}} to naive datetime, reason: :invalid_date
```

from_gregorian_seconds(seconds, microsecond_precision \\ {0, 0}, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec from_gregorian_seconds(integer(),
Calendar.microsecond(), Calendar.calendar()) ::
  t()
```

Converts a number of gregorian seconds to a `NaiveDateTime` struct.

Examples

```
iex> NaiveDateTime.from_gregorian_seconds(1)
~N[0000-01-01 00:00:01]
iex> NaiveDateTime.from_gregorian_seconds(63_755_511_991,
{5000, 3})
~N[2020-05-01 00:26:31.005]
iex> NaiveDateTime.from_gregorian_seconds(-1)
~N[-0001-12-31 23:59:59]
```

from_iso8601(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601(String.t(), Calendar.calendar()) :: {:ok,
t()} | {:error, atom()}
```

Parses the extended "Date and time of day" format described by ISO 8601:2019.

Time zone offset may be included in the string but they will be simply discarded as such information is not included in naive date times.

As specified in the standard, the separator "T" may be omitted if desired as there is no ambiguity within this function.

Note leap seconds are not supported by the built-in Calendar.ISO.

Examples

```
iex> NaiveDateTime.from_iso8601("2015-01-23 23:50:07")
{:ok, ~N[2015-01-23 23:50:07]}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07")
{:ok, ~N[2015-01-23 23:50:07]}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07Z")
{:ok, ~N[2015-01-23 23:50:07]}

iex> NaiveDateTime.from_iso8601("2015-01-23 23:50:07.0")
{:ok, ~N[2015-01-23 23:50:07.0]}
iex> NaiveDateTime.from_iso8601("2015-01-23
23:50:07,0123456")
{:ok, ~N[2015-01-23 23:50:07.012345]}
iex> NaiveDateTime.from_iso8601("2015-01-23
23:50:07.0123456")
{:ok, ~N[2015-01-23 23:50:07.012345]}
iex> NaiveDateTime.from_iso8601("2015-01-
23T23:50:07.123Z")
{:ok, ~N[2015-01-23 23:50:07.123]}

iex> NaiveDateTime.from_iso8601("2015-01-23P23:50:07")
{:error, :invalid_format}
iex> NaiveDateTime.from_iso8601("2015:01:23 23-50-07")
{:error, :invalid_format}
iex> NaiveDateTime.from_iso8601("2015-01-23 23:50:07A")
{:error, :invalid_format}
iex> NaiveDateTime.from_iso8601("2015-01-23 23:50:61")
{:error, :invalid_time}
iex> NaiveDateTime.from_iso8601("2015-01-32 23:50:07")
{:error, :invalid_date}

iex> NaiveDateTime.from_iso8601("2015-01-
23T23:50:07.123+02:30")
{:ok, ~N[2015-01-23 23:50:07.123]}
iex> NaiveDateTime.from_iso8601("2015-01-
23T23:50:07.123+00:00")
```

```
{:ok, ~N[2015-01-23 23:50:07.123]}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07.123-
02:30")
{:ok, ~N[2015-01-23 23:50:07.123]}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07.123-
00:00")
{:error, :invalid_format}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07.123-
00:60")
{:error, :invalid_format}
iex> NaiveDateTime.from_iso8601("2015-01-23T23:50:07.123-
24:00")
{:error, :invalid_format}
```

from_iso8601!(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601!(String.t(), Calendar.calendar()) :: t()
```

Parses the extended "Date and time of day" format described by ISO 8601:2019.

Raises if the format is invalid.

Examples

```
iex> NaiveDateTime.from_iso8601!("2015-01-
23T23:50:07.123Z")
~N[2015-01-23 23:50:07.123]
iex> NaiveDateTime.from_iso8601!("2015-01-
23T23:50:07,123Z")
~N[2015-01-23 23:50:07.123]
iex> NaiveDateTime.from_iso8601!("2015-01-23P23:50:07")
```

```
** (ArgumentError) cannot parse "2015-01-23P23:50:07" as
naive datetime, reason: :invalid_format
```

local_now(calendar \\ Calendar.ISO)

(since 1.10.0)

```
@spec local_now(Calendar.calendar()) :: t()
```

Returns the "local time" for the machine the Elixir program is running on.

WARNING: This function can cause insidious bugs. It depends on the time zone configuration at run time. This can changed and be set to a time zone that has daylight saving jumps (spring forward or fall back).

This function can be used to display what the time is right now for the time zone configuration that the machine happens to have. An example would be a desktop program displaying a clock to the user. For any other uses it is probably a bad idea to use this function.

For most cases, use `DateTime.now/2` or `DateTime.utc_now/1` instead.

Does not include fractional seconds.

Examples

```
iex> naive_datetime = NaiveDateTime.local_now()
iex> naive_datetime.year >= 2019
true
```

new(date, time)

```
@spec new(Date.t(), Time.t()) :: {:ok, t()}
```

Builds a naive datetime from date and time structs.

Examples

```
iex> NaiveDateTime.new(~D[2010-01-13], ~T[23:00:07.005])
{:ok, ~N[2010-01-13 23:00:07.005]}
```

new(year, month, day, hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec new(
  Calendar.year(),
  Calendar.month(),
  Calendar.day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: {:ok, t()} | {:error, atom()}
```

Builds a new ISO naive datetime.

Expects all values to be integers. Returns `{:ok, naive_datetime}` if each entry fits its appropriate range, returns `{:error, reason}` otherwise.

Examples

```
iex> NaiveDateTime.new(2000, 1, 1, 0, 0, 0)
{:ok, ~N[2000-01-01 00:00:00]}
iex> NaiveDateTime.new(2000, 13, 1, 0, 0, 0)
{:error, :invalid_date}
iex> NaiveDateTime.new(2000, 2, 29, 0, 0, 0)
{:ok, ~N[2000-02-29 00:00:00]}
iex> NaiveDateTime.new(2000, 2, 30, 0, 0, 0)
{:error, :invalid_date}
iex> NaiveDateTime.new(2001, 2, 29, 0, 0, 0)
{:error, :invalid_date}

iex> NaiveDateTime.new(2000, 1, 1, 23, 59, 59, {0, 1})
{:ok, ~N[2000-01-01 23:59:59.0]}
iex> NaiveDateTime.new(2000, 1, 1, 23, 59, 59, 999_999)
{:ok, ~N[2000-01-01 23:59:59.999999]}
iex> NaiveDateTime.new(2000, 1, 1, 24, 59, 59, 999_999)
{:error, :invalid_time}
iex> NaiveDateTime.new(2000, 1, 1, 23, 60, 59, 999_999)
{:error, :invalid_time}
iex> NaiveDateTime.new(2000, 1, 1, 23, 59, 60, 999_999)
{:error, :invalid_time}
iex> NaiveDateTime.new(2000, 1, 1, 23, 59, 59, 1_000_000)
{:error, :invalid_time}

iex> NaiveDateTime.new(2000, 1, 1, 23, 59, 59, {0, 1},
Calendar.ISO)
{:ok, ~N[2000-01-01 23:59:59.0]}
```

new!(date, time)

(since 1.11.0)

```
@spec new!(Date.t(), Time.t()) :: t()
```

Builds a naive datetime from date and time structs.

Examples

```
iex> NaiveDateTime.new!(~D[2010-01-13], ~T[23:00:07.005])
~N[2010-01-13 23:00:07.005]
```

new!(year, month, day, hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec new!(
  Calendar.year(),
  Calendar.month(),
  Calendar.day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: t()
```

Builds a new ISO naive datetime.

Expects all values to be integers. Returns `naive_datetime` if each entry fits its appropriate range, raises if time or date is invalid.

Examples

```
iex> NaiveDateTime.new!(2000, 1, 1, 0, 0, 0)
~N[2000-01-01 00:00:00]
iex> NaiveDateTime.new!(2000, 2, 29, 0, 0, 0)
~N[2000-02-29 00:00:00]
iex> NaiveDateTime.new!(2000, 1, 1, 23, 59, 59, {0, 1})
~N[2000-01-01 23:59:59.0]
iex> NaiveDateTime.new!(2000, 1, 1, 23, 59, 59, 999_999)
~N[2000-01-01 23:59:59.999999]
iex> NaiveDateTime.new!(2000, 1, 1, 23, 59, 59, {0, 1},
Calendar.ISO)
~N[2000-01-01 23:59:59.0]
iex> NaiveDateTime.new!(2000, 1, 1, 24, 59, 59, 999_999)
** (ArgumentError) cannot build naive datetime, reason:
:invalid_time
```

shift(naive_datetime, duration)

(since 1.17.0)

```
@spec shift(Calendar.naive_datetime(), Duration.duration())
:: t()
```

Shifts given `naive_datetime` by `duration` according to its calendar.

Allowed units are: `:year`, `:month`, `:week`, `:day`, `:hour`, `:minute`, `:second`, `:microsecond`.

When using the default ISO calendar, durations are collapsed and applied in the order of months, then seconds and microseconds:

- when shifting by 1 year and 2 months the date is actually shifted by 14 months
- weeks, days and smaller units are collapsed into seconds and microseconds

When shifting by month, days are rounded down to the nearest valid date.

Examples

```
iex> NaiveDateTime.shift(~N[2016-01-31 00:00:00], month:
1)
~N[2016-02-29 00:00:00]
iex> NaiveDateTime.shift(~N[2016-01-31 00:00:00], year:
4, day: 1)
~N[2020-02-01 00:00:00]
iex> NaiveDateTime.shift(~N[2016-01-31 00:00:00], year:
-2, day: 1)
~N[2014-02-01 00:00:00]
iex> NaiveDateTime.shift(~N[2016-01-31 00:00:00], second:
45)
~N[2016-01-31 00:00:45]
iex> NaiveDateTime.shift(~N[2016-01-31 00:00:00],
microsecond: {100, 6})
~N[2016-01-31 00:00:00.000100]

# leap years
iex> NaiveDateTime.shift(~N[2024-02-29 00:00:00], year:
1)
~N[2025-02-28 00:00:00]
iex> NaiveDateTime.shift(~N[2024-02-29 00:00:00], year:
4)
~N[2028-02-29 00:00:00]

# rounding down
iex> NaiveDateTime.shift(~N[2015-01-31 00:00:00], month:
1)
~N[2015-02-28 00:00:00]
```

to_date(naive_datetime)

```
@spec to_date(Calendar.naive_datetime()) :: Date.t()
```

Converts a `NaiveDateTime` into a `Date`.

Because `Date` does not hold time information, data will be lost during the conversion.

Examples

```
iex> NaiveDateTime.to_date(~N[2002-01-13 23:00:07])
~D[2002-01-13]
```

to_erl(naive_datetime)

```
@spec to_erl(Calendar.naive_datetime()) ::
:calendar.datetime()
```

Converts a `NaiveDateTime` struct to an Erlang datetime tuple.

Only supports converting naive datetimes which are in the ISO calendar, attempting to convert naive datetimes from other calendars will raise.

WARNING: Loss of precision may occur, as Erlang time tuples only store hour/minute/second.

Examples

```
iex> NaiveDateTime.to_erl(~N[2000-01-01 13:30:15])
{{2000, 1, 1}, {13, 30, 15}}
```

This function can also be used to convert a DateTime to an Erlang datetime tuple without the time zone information:

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> NaiveDateTime.to_erl(dt)
{{2000, 2, 29}, {23, 00, 07}}
```

to_gregorian_seconds(naive_datetime)

(since 1.11.0)

```
@spec to_gregorian_seconds(Calendar.naive_datetime()) ::
  {integer(), non_neg_integer()}
```

Converts a `NaiveDateTime` struct to a number of gregorian seconds and microseconds.

Examples

```
iex> NaiveDateTime.to_gregorian_seconds(~N[0000-01-01
00:00:01])
{1, 0}
iex> NaiveDateTime.to_gregorian_seconds(~N[2020-05-01
```

```
00:26:31.005])
{63_755_511_991, 5000}
```

to_iso8601(naive_datetime, format \\ :extended)

```
@spec to_iso8601(Calendar.naive_datetime(), :basic |
:extended) :: String.t()
```

Converts the given naive datetime to ISO 8601:2019.

By default, `NaiveDateTime.to_iso8601/2` returns naive datetimes formatted in the "extended" format, for human readability. It also supports the "basic" format through passing the `:basic` option.

Only supports converting naive datetimes which are in the ISO calendar, attempting to convert naive datetimes from other calendars will raise.

Examples

```
iex> NaiveDateTime.to_iso8601(~N[2000-02-28 23:00:13])
"2000-02-28T23:00:13"

iex> NaiveDateTime.to_iso8601(~N[2000-02-28
23:00:13.001])
"2000-02-28T23:00:13.001"

iex> NaiveDateTime.to_iso8601(~N[2000-02-28
23:00:13.001], :basic)
"20000228T230013.001"
```

This function can also be used to convert a DateTime to ISO 8601 without the time zone information:

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> NaiveDateTime.to_iso8601(dt)
"2000-02-29T23:00:07"
```

to_string(naive_datetime)

```
@spec to_string(Calendar.naive_datetime()) :: String.t()
```

Converts the given naive datetime to a string according to its calendar.

For readability, this function follows the RFC3339 suggestion of removing the "T" separator between the date and time components.

Examples

```
iex> NaiveDateTime.to_string(~N[2000-02-28 23:00:13])
"2000-02-28 23:00:13"
iex> NaiveDateTime.to_string(~N[2000-02-28 23:00:13.001])
"2000-02-28 23:00:13.001"
iex> NaiveDateTime.to_string(~N[-0100-12-15 03:20:31])
"-0100-12-15 03:20:31"
```

This function can also be used to convert a DateTime to a string without the time zone information:

```
iex> dt = %DateTime{year: 2000, month: 2, day: 29,
zone_abbr: "CET",
```

```
...>                hour: 23, minute: 0, second: 7,
microsecond: {0, 0},
...>                utc_offset: 3600, std_offset: 0,
time_zone: "Europe/Warsaw"}
iex> NaiveDateTime.to_string(dt)
"2000-02-29 23:00:07"
```

to_time(naive_datetime)

```
@spec to_time(Calendar.naive_datetime()) :: Time.t()
```

Converts a `NaiveDateTime` into `Time`.

Because `Time` does not hold date information, data will be lost during the conversion.

Examples

```
iex> NaiveDateTime.to_time(~N[2002-01-13 23:00:07])
~T[23:00:07]
```

truncate(naive_datetime, precision)

(since 1.6.0)

```
@spec truncate(t(), :microsecond | :millisecond | :second) ::
t()
```

Returns the given naive datetime with the microsecond field truncated to the given precision (`:microsecond`, `:millisecond` or `:second`).

The given naive datetime is returned unchanged if it already has lower precision than the given precision.

Examples

```
iex> NaiveDateTime.truncate(~N[2017-11-06
00:23:51.123456], :microsecond)
~N[2017-11-06 00:23:51.123456]

iex> NaiveDateTime.truncate(~N[2017-11-06
00:23:51.123456], :millisecond)
~N[2017-11-06 00:23:51.123]

iex> NaiveDateTime.truncate(~N[2017-11-06
00:23:51.123456], :second)
~N[2017-11-06 00:23:51]
```

utc_now(calendar_or_time_unit \\ Calendar.ISO)

(since 1.4.0)

```
@spec utc_now(Calendar.calendar() | :native | :microsecond |
:millisecond | :second) ::
  t()
```

Returns the current naive datetime in UTC.

Prefer using `DateTime.utc_now/0` when possible as, opposite to `NaiveDateTime`, it will keep the time zone information.

You can also provide a time unit to automatically truncate the naive datetime. This is available since v1.15.0.

Examples

```
iex> naive_datetime = NaiveDateTime.utc_now()
iex> naive_datetime.year >= 2016
true

iex> naive_datetime = NaiveDateTime.utc_now(:second)
iex> naive_datetime.microsecond
{0, 0}
```

utc_now(time_unit, calendar)

(since 1.15.0)

```
@spec utc_now(:native | :microsecond | :millisecond |
:second, Calendar.calendar()) ::
  t()
```

Returns the current naive datetime in UTC, supporting a specific calendar and precision.

Prefer using `DateTime.utc_now/2` when possible as, opposite to `NaiveDateTime`, it will keep the time zone information.

Examples

```
iex> naive_datetime = NaiveDateTime.utc_now(:second,
Calendar.ISO)
iex> naive_datetime.year >= 2016
true

iex> naive_datetime = NaiveDateTime.utc_now(:second,
Calendar.ISO)
```

```
iex> naive_datetime.microsecond
{0, 0}
```

Record

Module to work with, define, and import records.

Records are simply tuples where the first element is an atom:

```
iex> Record.is_record({User, "john", 27})
true
```

This module provides conveniences for working with records at compilation time, where compile-time field names are used to manipulate the tuples, providing fast operations on top of the tuples' compact structure.

In Elixir, records are used mostly in two situations:

1. to work with short, internal data
2. to interface with Erlang records

The macros `defrecord/3` and `defrecordp/3` can be used to create records while `extract/2` and `extract_all/1` can be used to extract records from Erlang files.

Types

Types can be defined for tuples with the `record/2` construct (which is only available in typespecs), with the record name as an atom and a keyword list of fields and their types as argument:

```
defmodule MyModule do
  require Record
  Record.defrecord(:user, name: "john", age: 25)
```

```
  @type user :: record(:user, name: String.t(), age:
integer)
  # expands to: "@type user :: {:user, String.t(),
integer}"
end
```

Reflection

The record tag and its fields are stored as metadata in the "Docs" chunk of the record definition macro. You can retrieve the documentation for a module by calling `Code.fetch_docs/1`.

Summary

Guards

is_record(data)

Checks if the given `data` is a record.

is_record(data, kind)

Checks if the given `data` is a record of kind `kind`.

Types

extract_opts()

Functions

defrecord(name, tag \\ nil, kv)

Defines a set of macros to create, access, and pattern match on a record.
defrecordp(name, tag \\ nil, kv)

Same as `defrecord/3` but generates private macros.

extract(name, opts)

Extracts record information from an Erlang file.

extract_all(opts)

Extracts all records information from an Erlang file.

Guards

is_record(data)

(macro)

Checks if the given `data` is a record.

This is implemented as a macro so it can be used in guard clauses.

Examples

```
Record.is_record({User, "john", 27})
#=> true

Record.is_record({})
#=> false
```

is_record(data, kind)

(macro)

Checks if the given `data` is a record of kind `kind`.

This is implemented as a macro so it can be used in guard clauses.

Examples

```
iex> record = {User, "john", 27}
iex> Record.is_record(record, User)
true
```

Types

extract_opts()

```
@type extract_opts() :: [
  from: binary(),
  from_lib: binary(),
  includes: [binary()],
  macros: keyword()
]
```

Functions

defrecord(name, tag \\ nil, kv)

(macro)

Defines a set of macros to create, access, and pattern match on a record.

The name of the generated macros will be `name` (which has to be an atom). `tag` is also an atom and is used as the "tag" for the record (i.e., the first element of the record tuple); by default (if `nil`), it's the same as `name`. `kv` is a keyword list of `name: default_value` fields for the new record.

The following macros are generated:

- `name/0` to create a new record with default values for all fields
- `name/1` to create a new record with the given fields and values, to get the zero-based index of the given field in a record or to convert the given record to a keyword list
- `name/2` to update an existing record with the given fields and values or to access a given field in a given record

All these macros are public macros (as defined by `defmacro`).

See the "Examples" section for examples on how to use these macros.

Examples

```
defmodule User do
  require Record
  Record.defrecord(:user, name: "meg", age: "25")
end
```

In the example above, a set of macros named `user` but with different arities will be defined to manipulate the underlying record.

```
# Import the module to make the user macros locally
available
import User
```

```
# To create records
record = user()        #=> {:user, "meg", 25}
record = user(age: 26) #=> {:user, "meg", 26}

# To get a field from the record
user(record, :name) #=> "meg"

# To update the record
user(record, age: 26) #=> {:user, "meg", 26}

# To get the zero-based index of the field in record
tuple
# (index 0 is occupied by the record "tag")
user(:name) #=> 1

# Convert a record to a keyword list
user(record) #=> [name: "meg", age: 26]
```

The generated macros can also be used in order to pattern match on records and to bind variables during the match:

```
record = user() #=> {:user, "meg", 25}

user(name: name) = record
name #=> "meg"
```

By default, Elixir uses the record name as the first element of the tuple (the "tag"). However, a different tag can be specified when defining a record, as in the following example, in which we use `Customer` as the second argument of `defrecord/3`:

```
defmodule User do
  require Record
```

```
  Record.defrecord(:user, Customer, name: nil)
end

require User
User.user() #=> {Customer, nil}
```

Defining extracted records with anonymous functions in the values

If a record defines an anonymous function in the default values, an `ArgumentError` will be raised. This can happen unintentionally when defining a record after extracting it from an Erlang library that uses anonymous functions for defaults.

```
Record.defrecord(:my_rec, Record.extract(...))
** (ArgumentError) invalid value for record field
fun_field,
    cannot escape #Function<12.90072148/2 in
:erl_eval.expr/5>.
```

To work around this error, redefine the field with your own &M.f/a function, like so:

```
defmodule MyRec do
  require Record
  Record.defrecord(:my_rec, Record.extract(...) |>
Keyword.merge(fun_field: &__MODULE__.foo/2))
  def foo(bar, baz), do: IO.inspect({bar, baz})
end
```

defrecordp(name, tag \\ nil, kv)

(macro)

Same as `defrecord/3` but generates private macros.

extract(name, opts)

```
@spec extract(name :: atom(), extract_opts()) :: keyword()
```

Extracts record information from an Erlang file.

Returns a quoted expression containing the fields as a list of tuples.

`name`, which is the name of the extracted record, is expected to be an atom *at compile time*.

Options

This function requires one of the following options, which are exclusive to each other (i.e., only one of them can be used in the same call):

- `:from` - (binary representing a path to a file) path to the Erlang file that contains the record definition to extract; with this option, this function uses the same path lookup used by the `-include` attribute used in Erlang modules.
- `:from_lib` - (binary representing a path to a file) path to the Erlang file that contains the record definition to extract; with this option, this function uses the same path lookup used by the `-include_lib` attribute used in Erlang modules.

It additionally accepts the following optional, non-exclusive options:

- `:includes` - (a list of directories as binaries) if the record being extracted depends on relative includes, this option allows

developers to specify the directory where those relative includes exist.

- `:macros` - (keyword list of macro names and values) if the record being extracted depends on the values of macros, this option allows the value of those macros to be set.

These options are expected to be literals (including the binary values) at compile time.

Examples

```
iex> Record.extract(:file_info, from_lib:
"kernel/include/file.hrl")
[
  size: :undefined,
  type: :undefined,
  access: :undefined,
  atime: :undefined,
  mtime: :undefined,
  ctime: :undefined,
  mode: :undefined,
  links: :undefined,
  major_device: :undefined,
  minor_device: :undefined,
  inode: :undefined,
  uid: :undefined,
  gid: :undefined
]
```

extract_all(opts)

```
@spec extract_all(extract opts()) :: [{name :: atom(), keyword()}]
```

Extracts all records information from an Erlang file.

Returns a keyword list of `{record_name, fields}` tuples where `record_name` is the name of an extracted record and `fields` is a list of `{field, value}` tuples representing the fields for that record.

Options

Accepts the same options as listed for `Record.extract/2`.

Regex

Provides regular expressions for Elixir.

Regex is based on PCRE (Perl Compatible Regular Expressions) and built on top of Erlang's `:re` module. More information can be found in the `:re` module documentation.

Regular expressions in Elixir can be created using the sigils `~r` (see `sigil_r/2`):

```
# A simple regular expression that matches foo anywhere
in the string
~r/foo/

# A regular expression with case insensitive and Unicode
options
~r/foo/iu
```

A Regex is represented internally as the `Regex` struct. Therefore, `%Regex{}` can be used whenever there is a need to match on them. Keep in mind that all of the structs fields are private. And since regexes are compiled, there is no guarantee two regular expressions from the same source are equal, for example:

```
~r/(?<foo>.)(?<bar>.)/ == ~r/(?<foo>.)(?<bar>.)/
```

may return `true` or `false` depending on your machine, endianness, available optimizations and others. You can, however, retrieve the source of a compiled regular expression by accessing the `source` field, and then compare those directly:

```
~r/(?<foo>.)(?<bar>.)/.source == ~r/(?<foo>.)(?
<bar>.)/.source
```

Escapes

Escape sequences are split into two categories.

Non-printing characters

- `\a` - Alarm, that is, the BEL character (hex 07)
- `\e` - Escape (hex 1B)
- `\f` - Form feed (hex 0C)
- `\n` - Line feed (hex 0A)
- `\r` - Carriage return (hex 0D)
- `\t` - Tab (hex 09)
- `\xhh` - Character with hex code hh
- `\x{hhh..}` - Character with hex code hhh..

`\u` and `\U` are not supported. Other escape sequences, such as `\ddd` for octals, are supported but discouraged.

Generic character types

- `\d` - Any decimal digit
- `\D` - Any character that is not a decimal digit
- `\h` - Any horizontal whitespace character
- `\H` - Any character that is not a horizontal whitespace character
- `\s` - Any whitespace character
- `\S` - Any character that is not a whitespace character
- `\v` - Any vertical whitespace character
- `\V` - Any character that is not a vertical whitespace character
- `\w` - Any "word" character

- `\W` - Any "non-word" character

Modifiers

The modifiers available when creating a Regex are:

- `:unicode` (u) - enables Unicode specific patterns like `\p` and causes character classes like `\w`, `\W`, `\s`, and the like to also match on Unicode (see examples below in "Character classes"). It expects valid Unicode strings to be given on match
- `:caseless` (i) - adds case insensitivity
- `:dotall` (s) - causes dot to match newlines and also sets newline to `(*ANYCRLF)`. The new line setting, as described in the `:re` documentation, can be overridden by starting the regular expression pattern with:
  - `(*CR)` - carriage return
  - `(*LF)` - line feed
  - `(*CRLF)` - carriage return, followed by line feed
  - `(*ANYCRLF)` - any of the three above
  - `(*ANY)` - all Unicode newline sequences
  - `(*NUL)` - the NUL character (binary zero) (from Erlang/OTP 28)
- `:multiline` (m) - causes `^` and `$` to mark the beginning and end of each line; use `\A` and `\z` to match the end or beginning of the string
- `:extended` (x) - whitespace characters are ignored except when escaped or within `[..]`, and allow `#` to delimit comments

- `:firstline` (f) - forces the unanchored pattern to match before or at the first newline, though the matched text may continue over the newline

- `:ungreedy` (U) - inverts the "greediness" of the regexp (the previous `r` option is deprecated in favor of `U`)

- `:export` (E) (since Elixir 1.19.3) - uses an exported pattern which can be shared across nodes or passed through config, at the cost of a runtime overhead to re-import it every time it is executed. This modifier only has an effect starting on Erlang/OTP 28, and it is ignored on older versions (i.e. `~r/foo/E == ~r/foo/`). This is because patterns cannot and do not need to be exported in order to be shared in these versions.

Captures

Many functions in this module handle what to capture in a regex match via the `:capture` option. The supported values are:

- `:all` - all captured subpatterns including the complete matching string (this is the default)

- `:first` - only the first captured subpattern, which is always the complete matching part of the string; all explicitly captured subpatterns are discarded

- `:all_but_first` - all but the first matching subpattern, i.e. all explicitly captured subpatterns, but not the complete matching part of the string

- `:none` - does not return matching subpatterns at all

- `:all_names` - captures all named subpattern matches in the Regex as a list ordered **alphabetically** by the names of the subpatterns

- `list(binary | atom)` - a list of named captures to capture

Character classes

Regex supports several built in named character classes. These are used by enclosing the class name in `[: :]` inside a group. For example:

```
iex> String.match?("123", ~r/^[[:alnum:]]+$/)
true
iex> String.match?("123 456", ~r/^[[:alnum:]
[:blank:]]+$/)
true
```

The supported class names are:

- alnum - Letters and digits
- alpha - Letters
- blank - Space or tab only
- cntrl - Control characters
- digit - Decimal digits (same as \d)
- graph - Printing characters, excluding space
- lower - Lowercase letters
- print - Printing characters, including space
- punct - Printing characters, excluding letters, digits, and space
- space - Whitespace (the same as \s from PCRE 8.34)
- upper - Uppercase letters
- word - "Word" characters (same as \w)
- xdigit - Hexadecimal digits

There is another character class, `ascii`, that erroneously matches Latin-1 characters instead of the 0-127 range specified by POSIX. This cannot be fixed without altering the behavior of other classes, so we recommend matching the range with `[\\0-\x7f]` instead.

Note the behavior of those classes may change according to the Unicode and other modifiers:

```
iex> String.match?("josé", ~r/^[[:lower:]]+$/)
false
iex> String.match?("josé", ~r/^[[:lower:]]+$/u)
true
iex> Regex.replace(~r/\s/, "Unicode\u00A0spaces", "-")
"Unicode spaces"
iex> Regex.replace(~r/\s/u, "Unicode\u00A0spaces", "-")
"Unicode-spaces"
```

Match limits

The underlying regex engine limits how much backtracking a single match may perform. Patterns with nested quantifiers, such as `~r/(a+)+$/`, may reach this limit on certain inputs (known as catastrophic backtracking). When that happens, the functions in this module raise `SystemLimitError` (since Elixir v1.21) instead of returning no match, since the outcome of the match is unknown.

Summary

Types

capture_opts()

Options for regex functions that capture matches.

named_captures_opts()
split_opts()

Options for `split/3`.

t()

Functions

compile(source, opts \\ "")

Compiles the regular expression.

compile!(source, options \\ "")

Compiles the regular expression and raises `Regex.CompileError` in case of errors.

escape(string)

Escapes a string to be literally matched in a regex.

import(regex)

Imports a `regex` that has been exported, otherwise returns the `regex` unchanged.

match?(regex, string)

Returns a boolean indicating whether there was a match or not.

named_captures(regex, string, options \\ [])

Returns the given captures as a map or `nil` if no captures are found.

names(regex)

Returns a list of names in the regex.

opts(regex)

Returns the regex options.

re_pattern(regex)

Returns the underlying `re_pattern` in the regular expression.

recompile(regex) deprecated

Recompiles the existing regular expression if necessary.

recompile!(regex) deprecated

Recompiles the existing regular expression and raises `Regex.CompileError` in case of errors.

replace(regex, string, replacement, options \\ [])

Receives a regex, a binary and a replacement, returns a new binary where all matches are replaced by the replacement.

run(regex, string, options \\ [])

Runs the regular expression against the given string until the first match. It returns a list with all captures or `nil` if no match occurred.

scan(regex, string, options \\ [])

Same as `run/3` but returns all non-overlapping matches of the regular expression.

source(regex)

Returns the regex source as a binary.

split(regex, string, options \\ [])

Splits the given target based on the given pattern and in the given number of parts.

to_embed(regex, embed_opts \\ [])

Returns the pattern as an embeddable string.
version() deprecated

Returns the version of the underlying Regex engine.

Types

capture_opts()

```
@type capture_opts() :: [
  return: :binary | :index,
  capture:
    :all | :first | :all_but_first | :none | :all_names |
[binary() | atom()],
  offset: non_neg_integer()
]
```

Options for regex functions that capture matches.

named_captures_opts()

```
@type named_captures_opts() :: [return: :binary | :index,
offset: non_neg_integer()]
```

split_opts()

```
@type split_opts() :: [
  parts: pos_integer() | :infinity,
  trim: boolean(),
```

```
  on:
    :first | :all | :all_but_first | :none | :all_names |
[atom() | integer()],
  include_captures: boolean()
]
```

Options for `split/3`.

t()

```
@type t() :: %Regex{opts: [term()], re_pattern: term(),
source: binary()}
```

Functions

compile(source, opts \\ "")

```
@spec compile(binary(), binary() | [term()]) :: {:ok, t()} |
{:error, term()}
```

Compiles the regular expression.

The given options can either be a binary with the characters representing the same regex options given to the `~r` (see `sigil_r/2`) sigil, or a list of options, as expected by the Erlang's `:re` module.

It returns `{:ok, regex}` in case of success, `{:error, reason}` otherwise.

Examples

```
Regex.compile("foo")
#=> {:ok, ~r/foo/}

Regex.compile("foo", "i")
#=> {:ok, ~r/foo/i}

Regex.compile("*foo")
#=> {:error, {~c"quantifier does not follow a repeatable
item", 0}}
```

compile!(source, options \\ "")

```
@spec compile!(binary(), binary() | [term()]) :: t()
```

Compiles the regular expression and raises `Regex.CompileError` in case of errors.

escape(string)

```
@spec escape(String.t()) :: String.t()
```

Escapes a string to be literally matched in a regex.

Examples

```
iex> Regex.escape(".")
"\\."
```

```
iex> Regex.escape("\\what if")
"\\\\what\\ if"
```

import(regex)

(since 1.20.0)

```
@spec import(t()) :: t()
```

Imports a `regex` that has been exported, otherwise returns the `regex` unchanged.

This means it will lose the ability to be sent across nodes or passed through config, but will be faster since it won't need to be imported on the fly every time it is executed.

Exported regexes only exist on OTP 28, so this has no effect on older versions.

Examples

```
Regex.import(~r/foo/E)
~r/foo/

Regex.import(~r/foo/)
~r/foo/
```

match?(regex, string)

```
@spec match?(t(), String.t()) :: boolean()
```

Returns a boolean indicating whether there was a match or not.

Examples

```
iex> Regex.match?(~r/foo/, "foo")
true

iex> Regex.match?(~r/foo/, "bar")
false
```

Elixir also provides text-based match operator `=~/2` and function `String.match?/2` as an alternative to test strings against regular expressions and strings.

named_captures(regex, string, options \\ [])

```
@spec named_captures(t(), String.t(), named_captures_opts())
:: map() | nil
```

Returns the given captures as a map or `nil` if no captures are found.

Options

- `:return` - when set to `:index`, returns byte index and match length. Defaults to `:binary`.
- `:offset` - (since v1.12.0) specifies the starting offset to match in the given string. Defaults to `0`.

Examples

```
iex> Regex.named_captures(~r/c(?<foo>d)/, "abcd")
%{"foo" => "d"}

iex> Regex.named_captures(~r/a(?<foo>b)c(?<bar>d)/,
"abcd")
%{"bar" => "d", "foo" => "b"}

iex> Regex.named_captures(~r/a(?<foo>b)c(?<bar>d)/,
"efgh")
nil
```

You can also retrieve indexes from the named captures. This is particularly useful if you want to know if a named capture matched or not:

```
iex> Regex.named_captures(~r/a(?<foo>b)c(?<bar>d)?/,
"abc", return: :index)
%{"bar" => {-1, 0}, "foo" => {1, 1}}
```

You can then use `binary_part/3` to fetch the relevant part from the given string.

names(regex)

```
@spec names(t()) :: [String.t()]
```

Returns a list of names in the regex.

Examples

```
iex> Regex.names(~r/(?<foo>bar)/)
["foo"]
```

opts(regex)

```
@spec opts(t()) :: [term()]
```

Returns the regex options.

See the documentation of `Regex.compile/2` for more information.

Examples

```
iex> Regex.opts(~r/foo/m)
[:multiline]

iex> Regex.opts(Regex.compile!("foo", [:caseless]))
[:caseless]
```

re_pattern(regex)

```
@spec re_pattern(t()) :: term()
```

Returns the underlying `re_pattern` in the regular expression.

recompile(regex)

(since 1.4.0)

This function is deprecated. It can be removed and it has no effect.

Recompiles the existing regular expression if necessary.

This checks the version stored in the regular expression and recompiles the regex in case of version mismatch.

recompile!(regex)

(since 1.4.0)

This function is deprecated. It can be removed and it has no effect.

Recompiles the existing regular expression and raises `Regex.CompileError` in case of errors.

replace(regex, string, replacement, options \\ [])

```
@spec replace(t(), String.t(), String.t() | (... ->
String.t()), [
  {:global, boolean()}
]) :: String.t()
```

Receives a regex, a binary and a replacement, returns a new binary where all matches are replaced by the replacement.

The replacement can be either a string or a function that returns a string. The resulting string is used as a replacement for every match.

When the replacement is a string, it allows specific captures of the match using brackets at the regex expression and accessing them in the replacement via `\N` or `\g{N}`, where `N` is the number of the capture. In case `\0` is used, the whole match is inserted. Note that in regexes the backslash needs to be escaped, hence in practice you'll need to use `\\N` and `\\g{N}`.

When the replacement is a function, it allows specific captures too. The function may have arity N where each argument maps to a capture, with the first argument being the whole match. If the function expects more arguments than captures found, the remaining arguments will receive `""`.

Options

- `:global` - when `false`, replaces only the first occurrence (defaults to `true`)

Examples

```
iex> Regex.replace(~r/d/, "abc", "d")
"abc"

iex> Regex.replace(~r/b/, "abc", "d")
"adc"

iex> Regex.replace(~r/b/, "abc", "[\\0]")
"a[b]c"

iex> Regex.replace(~r/a(b|d)c/, "abcadc", "[\\1]")
"[b][d]"

iex> Regex.replace(~r/\.(\d)$/, "500.5", ".\\g{1}0")
```

```
"500.50"

iex> Regex.replace(~r/a(b|d)c/, "abcadc", fn _, x -> "[#
{x}]" end)
"[b][d]"

iex> Regex.replace(~r/(\w+)@(\w+).(\w+)/, "abc@def.com",
fn _full, _c1, _c2, c3 -> "TLD: #{c3}" end)
"TLD: com"

iex> Regex.replace(~r/a/, "abcadc", "A", global: false)
"Abcadc"
```

run(regex, string, options \\ [])

```
@spec run(t(), binary(), capture opts()) ::
  nil | [binary()] | [{integer(), integer()}]
```

Runs the regular expression against the given string until the first match. It returns a list with all captures or `nil` if no match occurred.

Options

- `:return` - when set to `:index`, returns byte index and match length. Defaults to `:binary`.
- `:capture` - what to capture in the result. See the "Captures" section to see the possible capture values.
- `:offset` - (since v1.12.0) specifies the starting offset to match in the given string. Defaults to `0`.

Examples

```
iex> Regex.run(~r/c(d)/, "abcd")
["cd", "d"]

iex> Regex.run(~r/e/, "abcd")
nil

iex> Regex.run(~r/c(d)/, "abcd", return: :index)
[{2, 2}, {3, 1}]

iex> Regex.run(~r/c(d)/, "abcd", capture: :first)
["cd"]

iex> Regex.run(~r/c(?<foo>d)/, "abcd", capture: ["foo",
"bar"])
["d", ""]
```

scan(regex, string, options \\ [])

```
@spec scan(t(), String.t(), capture_opts()) ::
  [[String.t()]] | [[{integer(), integer()}]]
```

Same as `run/3` but returns all non-overlapping matches of the regular expression.

A list of lists is returned, where each entry in the primary list represents a match and each entry in the secondary list represents the captured contents.

Options

- `:return` - when set to `:index`, returns byte index and match length. Defaults to `:binary`.

- `:capture` - what to capture in the result. See the "Captures" section to see the possible capture values.
- `:offset` - (since v1.12.0) specifies the starting offset to match in the given string. Defaults to `0`.

Examples

```
iex> Regex.scan(~r/c(d|e)/, "abcd abce")
[["cd", "d"], ["ce", "e"]]

iex> Regex.scan(~r/c(?:d|e)/, "abcd abce")
[["cd"], ["ce"]]

iex> Regex.scan(~r/e/, "abcd")
[]

iex> Regex.scan(~r/ab|bc|cd/, "abcd")
[["ab"], ["cd"]]

iex> Regex.scan(~r/ab|bc|cd/, "abbccd")
[["ab"], ["bc"], ["cd"]]

iex> Regex.scan(~r/\p{Sc}/u, "$, £, and €")
[["$"], ["£"], ["€"]]

iex> Regex.scan(~r/=+/, "=ü†ƒ8===", return: :index)
[[{0, 1}], [{9, 3}]]

iex> Regex.scan(~r/c(d|e)/, "abcd abce", capture: :first)
[["cd"], ["ce"]]
```

source(regex)

```
@spec source(t()) :: String.t()
```

Returns the regex source as a binary.

Examples

```
iex> Regex.source(~r/foo/)
"foo"
```

split(regex, string, options \\ [])

```
@spec split(t(), String.t(), split_opts()) :: [String.t()]
```

Splits the given target based on the given pattern and in the given number of parts.

Options

- `:parts` - when specified, splits the string into the given number of parts. If not specified, `:parts` defaults to `:infinity`, which will split the string into the maximum number of parts possible based on the given pattern.
- `:trim` - when `true`, removes empty strings (`""`) from the result. Defaults to `false`.
- `:on` - specifies which captures to split the string on, and in what order. Defaults to `:first` which means captures inside the regex do not affect the splitting process. See the "Captures" section to see the possible capture values.

- `:include_captures` - when `true`, includes in the result the matches of the regular expression. The matches are not counted towards the maximum number of parts if combined with the `:parts` option. Defaults to `false`.

Examples

```
iex> Regex.split(~r/-/, "a-b-c")
["a", "b", "c"]

iex> Regex.split(~r/-/, "a-b-c", parts: 2)
["a", "b-c"]

iex> Regex.split(~r/-/, "abc")
["abc"]

iex> Regex.split(~r//, "abc")
["", "a", "b", "c", ""]

iex> Regex.split(~r/a(?<second>b)c/, "abc")
["", ""]

iex> Regex.split(~r/a(?<second>b)c/, "abc", on:
[:second])
["a", "c"]

iex> Regex.split(~r/(x)/, "Elixir", include_captures:
true)
["Eli", "x", "ir"]

iex> Regex.split(~r/a(?<second>b)c/, "abc", on:
[:second], include_captures: true)
["a", "b", "c"]
```

```
iex> Regex.split(~r/-/, "-a-b--c", trim: true)
["a", "b", "c"]
```

to_embed(regex, embed_opts \\ [])

(since 1.19.0)

```
@spec to_embed(t(), [{:strict, boolean()}]) :: String.t()
```

Returns the pattern as an embeddable string.

If the pattern was compiled with an option which cannot be represented as an embeddable modifier in the current version of PCRE and strict is true (the default) then an ArgumentError exception will be raised.

When the `:strict` option is false the pattern will be returned as though any offending options had not be used and the function will not raise any exceptions.

Embeddable modifiers/options are currently:

- 'i' - `:caseless`
- 'm' - `:multiline`
- 's' - `:dotall, {:newline, :anycrlf}`
- 'x' - `:extended`

Unembeddable modifiers are:

- 'f' - `:firstline`
- 'U' - `:ungreedy`
- 'u' - `:unicode, :ucp`

Any other regex compilation option not listed here is considered unembeddable and will raise an exception unless the `:strict` option is false.

Examples

```
iex> Regex.to_embed(~r/foo/)
"(?-imsx:foo)"

iex> Regex.to_embed(~r/^foo/m)
"(?m-isx:^foo)"

iex> Regex.to_embed(~r/foo # comment/ix)
"(?ix-ms:foo # comment\n)"

iex> Regex.to_embed(~r/foo/iu)
** (ArgumentError) regex compiled with options [:ucp,
:unicode] which cannot be represented as an embedded
pattern in this version of PCRE

iex> Regex.to_embed(~r/foo/imsxu, strict: false)
"(?imsx:foo\n)"
```

version()

(since 1.4.0)

This function is deprecated. Use :re.version() instead.

Returns the version of the underlying Regex engine.

String

Strings in Elixir are UTF-8 encoded binaries.

Strings in Elixir are a sequence of Unicode characters, typically written between double quoted strings, such as `"hello"` and `"héllò"`.

In case a string must have a double-quote in itself, the double quotes must be escaped with a backslash, for example: `"this is a string with \"double quotes\""`.

You can concatenate two strings with the `<>/2` operator:

```
iex> "hello" <> " " <> "world"
"hello world"
```

The functions in this module act according to The Unicode Standard, Version 17.0.0.

Interpolation

Strings in Elixir also support interpolation. This allows you to place some value in the middle of a string by using the `#{}` syntax:

```
iex> name = "joe"
iex> "hello #{name}"
"hello joe"
```

Any Elixir expression is valid inside the interpolation. If a string is given, the string is interpolated as is. If any other value is given, Elixir

will attempt to convert it to a string using the `String.Chars` protocol. This allows, for example, to output an integer from the interpolation:

```
iex> "2 + 2 = #{2 + 2}"
"2 + 2 = 4"
```

In case the value you want to interpolate cannot be converted to a string, because it doesn't have a human textual representation, a protocol error will be raised.

Escape characters

Besides allowing double-quotes to be escaped with a backslash, strings also support the following escape characters:

- `\0` - Null byte
- `\a` - Bell
- `\b` - Backspace
- `\t` - Horizontal tab
- `\n` - Line feed (New lines)
- `\v` - Vertical tab
- `\f` - Form feed
- `\r` - Carriage return
- `\e` - Command Escape
- `\s` - Space
- `\#` - Returns the `#` character itself, skipping interpolation
- `\\` - Single backslash
- `\xNN` - A byte represented by the hexadecimal `NN`
- `\uNNNN` - A Unicode code point represented by `NNNN`
- `\u{NNNNNN}` - A Unicode code point represented by `NNNNNN`

Note it is generally not advised to use `\xNN` in Elixir strings, as introducing an invalid byte sequence would make the string invalid. If you have to introduce a character by its hexadecimal representation, it is best to work with Unicode code points, such as `\uNNNN`. In fact, understanding Unicode code points can be essential when doing low-level manipulations of string, so let's explore them in detail next.

Unicode and code points

In order to facilitate meaningful communication between computers across multiple languages, a standard is required so that the ones and zeros on one machine mean the same thing when they are transmitted to another. The Unicode Standard acts as an official registry of virtually all the characters we know: this includes characters from classical and historical texts, emoji, and formatting and control characters as well.

Unicode organizes all of the characters in its repertoire into code charts, and each character is given a unique numerical index. This numerical index is known as a Code Point.

In Elixir you can use a `?` in front of a character literal to reveal its code point:

```
iex> ?a
97
iex> ?ł
322
```

Note that most Unicode code charts will refer to a code point by its hexadecimal (hex) representation, e.g. `97` translates to `0061` in hex, and we can represent any Unicode character in an Elixir string by using the `\u` escape character followed by its code point number:

```
iex> "\u0061" === "a"
true
iex> 0x0061 = 97 = ?a
97
```

The hex representation will also help you look up information about a code point, e.g. https://codepoints.net/U+0061 has a data sheet all about the lower case `a`, a.k.a. code point 97. Remember you can get the hex presentation of a number by calling `Integer.to_string/2`:

```
iex> Integer.to_string(?a, 16)
"61"
```

UTF-8 encoded and encodings

Now that we understand what the Unicode standard is and what code points are, we can finally talk about encodings. Whereas the code point is **what** we store, an encoding deals with **how** we store it: encoding is an implementation. In other words, we need a mechanism to convert the code point numbers into bytes so they can be stored in memory, written to disk, and such.

Elixir uses UTF-8 to encode its strings, which means that code points are encoded as a series of 8-bit bytes. UTF-8 is a **variable width** character encoding that uses one to four bytes to store each code point. It is capable of encoding all valid Unicode code points. Let's see an example:

```
iex> string = "héllo"
"héllo"
iex> String.length(string)
5
```

```
iex> byte_size(string)
6
```

Although the string above has 5 characters, it uses 6 bytes, as two bytes are used to represent the character `é`.

Grapheme clusters

This module also works with the concept of grapheme cluster (from now on referenced as graphemes). Graphemes can consist of multiple code points that may be perceived as a single character by readers. For example, "é" can be represented either as a single "e with acute" code point, as seen above in the string `"héllo"`, or as the letter "e" followed by a "combining acute accent" (two code points):

```
iex> string = "\u0065\u0301"
"é"
iex> byte_size(string)
3
iex> String.length(string)
1
iex> String.codepoints(string)
["e", "́"]
iex> String.graphemes(string)
["é"]
```

Although it looks visually the same as before, the example above is made of two characters, it is perceived by users as one.

Graphemes can also be two characters that are interpreted as one by some languages. For example, some languages may consider "ch" as a single character. However, since this information depends on the locale, it is not taken into account by this module.

In general, the functions in this module rely on the Unicode Standard, but do not contain any of the locale specific behavior. More information about graphemes can be found in the Unicode Standard Annex #29.

For converting a binary to a different encoding and for Unicode normalization mechanisms, see Erlang's `:unicode` module.

String and binary operations

To act according to the Unicode Standard, many functions in this module run in linear time, as they need to traverse the whole string considering the proper Unicode code points.

For example, `String.length/1` will take longer as the input grows. On the other hand, `Kernel.byte_size/1` always runs in constant time (i.e. regardless of the input size).

This means often there are performance costs in using the functions in this module, compared to the more low-level operations that work directly with binaries:

- `Kernel.binary_part/3` - retrieves part of the binary
- `Kernel.bit_size/1` and `Kernel.byte_size/1` - size related functions
- `Kernel.is_bitstring/1` and `Kernel.is_binary/1` - type-check function
- Plus a number of functions for working with binaries (bytes) in the `:binary` module

A `utf8` modifier is also available inside the binary syntax `<<>>`. It can be used to match code points out of a binary/string:

```
iex> <<eacute::utf8>> = "é"
iex> eacute
```

```
233
```

See the *Patterns and Guards* guide and the documentation for `<<>>` for more information on binary pattern matching.

You can also fully convert a string into a list of integer code points, known as "charlists" in Elixir, by calling `String.to_charlist/1`:

```
iex> String.to_charlist("héllo")
[104, 233, 108, 108, 111]
```

If you would rather see the underlying bytes of a string, instead of its codepoints, a common trick is to concatenate the null byte `<<0>>` to it:

```
iex> "héllo" <> <<0>>
<<104, 195, 169, 108, 108, 111, 0>>
```

Alternatively, you can view a string's binary representation by passing an option to `IO.inspect/2`:

```
IO.inspect("héllo", binaries: :as_binaries)
#=> <<104, 195, 169, 108, 108, 111>>
```

Self-synchronization

The UTF-8 encoding is self-synchronizing. This means that if malformed data (i.e., data that is not possible according to the definition of the encoding) is encountered, only one code point needs to be rejected.

This module relies on this behavior to ignore such invalid characters. For example, `length/1` will return a correct result even if an invalid code point is fed into it.

In other words, this module expects invalid data to be detected elsewhere, usually when retrieving data from the external source. For example, a driver that reads strings from a database will be responsible to check the validity of the encoding. `String.chunk/2` can be used for breaking a string into valid and invalid parts.

Compile binary patterns

Many functions in this module work with patterns. For example, `String.split/3` can split a string into multiple strings given a pattern. This pattern can be a string, a list of strings or a compiled pattern:

```
iex> String.split("foo bar", " ")
["foo", "bar"]

iex> String.split("foo bar!", [" ", "!"])
["foo", "bar", ""]

iex> pattern = :binary.compile_pattern([" ", "!"])
iex> String.split("foo bar!", pattern)
["foo", "bar", ""]
```

The compiled pattern is useful when the same match will be done over and over again. Note though that the compiled pattern cannot be stored in a module attribute as the pattern is generated at runtime and does not survive compile time.

Summary

Types

codepoint()

A single Unicode code point encoded in UTF-8. It may be one or more bytes.

grapheme()

Multiple code points that may be perceived as a single character by readers

pattern()

Pattern used in functions like `replace/4` and `split/3`.

replace_opts()
split_opts()
splitter_opts()
t()

A UTF-8 encoded binary.

Functions

at(string, position)

Returns the grapheme at the `position` of the given UTF-8 `string`. If `position` is greater than `string` length, then it returns `nil`.

bag_distance(string1, string2)

Computes the bag distance between two strings.

byte_slice(string, start_bytes, size_bytes)

Returns a substring starting at (or after) `start_bytes` and of at most the given `size_bytes`.

capitalize(string, mode \\ :default)

Converts the first character in the given string to uppercase and the remainder to lowercase according to `mode`.

chunk(string, trait)

Splits the string into chunks of characters that share a common trait.

codepoints(string)

Returns a list of code points encoded as strings.

contains?(string, contents)

Searches if `string` contains any of the given `contents`.

count(string, pattern)

Counts the number of non-overlapping occurrences of a `pattern` in a `string`.

downcase(string, mode \\ :default)

Converts all characters in the given string to lowercase according to `mode`.

duplicate(subject, n)

Returns a string `subject` repeated `n` times.

ends_with?(string, suffix)

Returns `true` if `string` ends with any of the suffixes given.

equivalent?(string1, string2)

Returns `true` if `string1` is canonically equivalent to `string2`.

first(string)

Returns the first grapheme from a UTF-8 string, `nil` if the string is empty.

graphemes(string)

Returns Unicode graphemes in the string as per Extended Grapheme Cluster algorithm.

jaro_distance(string1, string2)

Computes the Jaro distance (similarity) between two strings.

last(string)

Returns the last grapheme from a UTF-8 string, `nil` if the string is empty.

length(string)

Returns the number of Unicode graphemes in a UTF-8 string.

match?(string, regex)

Checks if `string` matches the given regular expression.

myers_difference(string1, string2)

Returns a keyword list that represents an edit script.

next_codepoint(arg)

Returns the next code point in a string.

next_grapheme(string)

Returns the next grapheme in a string.

next_grapheme_size(string)

Returns the size (in bytes) of the next grapheme.

normalize(string, form)

Converts all characters in `string` to Unicode normalization form identified by `form`.

pad_leading(string, count, padding \\ [" "])

Returns a new string padded with a leading filler which is made of elements from the `padding`.

pad_trailing(string, count, padding \\ [" "])

Returns a new string padded with a trailing filler which is made of elements from the `padding`.

printable?(string, character_limit \\ :infinity)

Checks if a string contains only printable characters up to `character_limit`.

replace(subject, pattern, replacement, options \\ [])

Returns a new string created by replacing occurrences of `pattern` in `subject` with `replacement`.

replace_invalid(bytes, replacement \\ "\uFFFD")

Returns a new string created by replacing all invalid bytes with `replacement` (`"\uFFFD"` by default).

replace_leading(string, match, replacement)

Replaces all leading occurrences of `match` by `replacement` of `match` in `string`.

replace_prefix(string, match, replacement)

Replaces prefix in `string` by `replacement` if it matches `match`.

replace_suffix(string, match, replacement)

Replaces suffix in `string` by `replacement` if it matches `match`.

replace_trailing(string, match, replacement)

Replaces all trailing occurrences of `match` by `replacement` in `string`.

reverse(string)

Reverses the graphemes in given string.

slice(string, range)

Returns a substring from the offset given by the start of the range to the offset given by the end of the range.

slice(string, start, length)

Returns a substring starting at the offset `start`, and of the given `length`.

split(binary)

Divides a string into substrings at each Unicode whitespace occurrence with leading and trailing whitespace ignored.

split(string, pattern, options \\ [])

Divides a string into parts based on a pattern.

split_at(string, position)

Splits a string into two at the specified offset. When the offset given is negative, location is counted from the end of the string.

splitter(string, pattern, options \\ [])

Returns an enumerable that splits a string on demand.

starts_with?(string, prefix)

Returns `true` if `string` starts with any of the prefixes given.

to_atom(string)

Converts a string to an existing atom or creates a new one.

to_charlist(string)

Converts a string into a charlist.

to_existing_atom(string)

Converts a string to an existing atom or raises if the atom does not exist.

to_float(string)

Returns a float whose text representation is `string`.

to_integer(string)

Returns an integer whose text representation is `string`.

to_integer(string, base)

Returns an integer whose text representation is `string` in base `base`.

trim(string)

Returns a string where all leading and trailing Unicode whitespaces have been removed.

trim(string, to_trim)

Returns a string where all leading and trailing `to_trim` characters have been removed.

trim_leading(string)

Returns a string where all leading Unicode whitespaces have been removed.

trim_leading(string, to_trim)

Returns a string where all leading `to_trim` characters have been removed.

trim_trailing(string)

Returns a string where all trailing Unicode whitespaces have been removed.

trim_trailing(string, to_trim)

Returns a string where all trailing `to_trim` characters have been removed.

upcase(string, mode \\ :default)

Converts all characters in the given string to uppercase according to `mode`.

valid?(string, algorithm \\ :default)

Checks whether `string` contains only valid characters.

Types

codepoint()

```
@type codepoint() :: t()
```

A single Unicode code point encoded in UTF-8. It may be one or more bytes.

grapheme()

```
@type grapheme() :: t()
```

Multiple code points that may be perceived as a single character by readers

pattern()

```
@type pattern() ::
  t() | [nonempty_binary()] | (compiled_search_pattern ::
:binary.cp())
```

Pattern used in functions like `replace/4` and `split/3`.

It must be one of:

- a string
- an empty list
- a list containing non-empty strings
- a compiled search pattern created by `:binary.compile_pattern/1`

replace_opts()

```
@type replace_opts() :: [{:global, boolean()}]
```

split_opts()

```
@type split_opts() :: [parts: pos integer() | :infinity,
trim: boolean()]
```

splitter_opts()

```
@type splitter_opts() :: [{:trim, boolean()}]
```

t()

```
@type t() :: binary()
```

A UTF-8 encoded binary.

The types `String.t()` and `binary()` are equivalent to analysis tools. Although, for those reading the documentation, `String.t()` implies it is a UTF-8 encoded binary.

Functions

at(string, position)

```
@spec at(t(), integer()) :: grapheme() | nil
```

Returns the grapheme at the `position` of the given UTF-8 `string`. If `position` is greater than `string` length, then it returns `nil`.

> **Linear Access**
>
> This function has to linearly traverse the string. If you want to access a string or a binary in constant time based on the number of bytes, use `Kernel.binary_slice/3` or `:binary.at/2` instead.

Examples

```
iex> String.at("elixir", 0)
"e"

iex> String.at("elixir", 1)
"l"

iex> String.at("elixir", 10)
nil

iex> String.at("elixir", -1)
"r"

iex> String.at("elixir", -10)
nil
```

bag_distance(string1, string2)

(since 1.8.0)

```
@spec bag_distance(t(), t()) :: float()
```

Computes the bag distance between two strings.

Returns a float value between 0 and 1 representing the bag distance between `string1` and `string2`.

The bag distance is meant to be an efficient approximation of the distance between two strings to quickly rule out strings that are largely different.

The algorithm is outlined in the "String Matching with Metric Trees Using an Approximate Distance" paper by Ilaria Bartolini, Paolo Ciaccia, and Marco Patella.

Examples

```
iex> String.bag_distance("abc", "")
0.0
iex> String.bag_distance("abcd", "a")
0.25
iex> String.bag_distance("abcd", "ab")
0.5
iex> String.bag_distance("abcd", "abc")
0.75
iex> String.bag_distance("abcd", "abcd")
1.0
```

byte_slice(string, start_bytes, size_bytes)

(since 1.17.0)

```
@spec byte_slice(t(), integer(), non_neg_integer()) :: t()
```

Returns a substring starting at (or after) `start_bytes` and of at most the given `size_bytes`.

This function works on bytes and then adjusts the string to eliminate truncated codepoints. This is useful when you have a string and you need to guarantee it does not exceed a certain amount of bytes.

If the offset is greater than the number of bytes in the string, then it returns `""`. Similar to `String.slice/2`, a negative `start_bytes` will be adjusted to the end of the string (but in bytes).

This function does not guarantee the string won't have invalid codepoints, it only guarantees to remove truncated codepoints immediately at the beginning or the end of the slice.

Examples

Consider the string "héllo". Let's see its representation:

```
iex> inspect("héllo", binaries: :as_binaries)
"<<104, 195, 169, 108, 108, 111>>"
```

Although the string has 5 characters, it is made of 6 bytes. Now imagine we want to get only the first two bytes. To do so, let's use `binary_slice/3`, which is unaware of codepoints:

```
iex> binary_slice("héllo", 0, 2)
<<104, 195>>
```

As you can see, this operation is unsafe and returns an invalid string. That's because we cut the string in the middle of the bytes representing "é". On the other hand, we could use `String.slice/3`:

```
iex> String.slice("héllo", 0, 2)
"hé"
```

While the above is correct, it has 3 bytes. If you have a requirement where you need *at most* 2 bytes, the result would also be invalid. In such

scenarios, you can use this function, which will slice the given bytes, but clean up the truncated codepoints:

```
iex> String.byte_slice("héllo", 0, 2)
"h"
```

Truncated codepoints at the beginning are also cleaned up:

```
iex> String.byte_slice("héllo", 2, 3)
"llo"
```

Note that, if you want to work on raw bytes, then you must use `binary_slice/3` instead.

capitalize(string, mode \\ :default)

```
@spec capitalize(t(), :default | :ascii | :greek | :turkic)
:: t()
```

Converts the first character in the given string to uppercase and the remainder to lowercase according to `mode`.

`mode` may be `:default`, `:ascii`, `:greek` or `:turkic`. The `:default` mode considers all non-conditional transformations outlined in the Unicode standard. `:ascii` capitalizes only the letters A to Z. `:greek` includes the context sensitive mappings found in Greek. `:turkic` properly handles the letter `i` with the dotless variant.

Also see `upcase/2` and `capitalize/2` for other conversions. If you want a variation of this function that does not lowercase the rest of string, see Erlang's `:string.titlecase/1`.

Examples

```
iex> String.capitalize("abcd")
"Abcd"
iex> String.capitalize("ABCD")
"Abcd"

iex> String.capitalize("fin")
"Fin"
iex> String.capitalize("olá")
"Olá"
```

chunk(string, trait)

```
@spec chunk(t(), :valid | :printable) :: [t()]
```

Splits the string into chunks of characters that share a common trait.

The trait can be one of two options:

- `:valid` - the string is split into chunks of valid and invalid character sequences
- `:printable` - the string is split into chunks of printable and non-printable character sequences

Returns a list of binaries each of which contains only one kind of characters.

If the given string is empty, an empty list is returned.

Examples

```
iex> String.chunk(<<?a, ?b, ?c, 0>>, :valid)
[<<97, 98, 99, 0>>]

iex> String.chunk(<<?a, ?b, ?c, 0, 0xFFFF::utf16>>,
:valid)
[<<97, 98, 99, 0>>, <<255, 255>>]

iex> String.chunk(<<?a, ?b, ?c, 0, 0x0FFFF::utf8>>,
:printable)
["abc", <<0, 239, 191, 191>>]
```

codepoints(string)

```
@spec codepoints(t()) :: [codepoint()]
```

Returns a list of code points encoded as strings.

To retrieve code points in their natural integer representation, see `to_charlist/1`. For details about code points and graphemes, see the `String` module documentation.

Examples

```
iex> String.codepoints("olá")
["o", "l", "á"]

iex> String.codepoints("оптимі зації")
["о", "п", "т", "и", "м", "і", " ", "з", "а", "ц", "і",
"ї"]

iex> String.codepoints("a̋HQ̣")
["a̋", "H̋", "Q̣"]
```

```
iex> String.codepoints("\u00e9")
["é"]

iex> String.codepoints("\u0065\u0301")
["e", "́"]
```

contains?(string, contents)

```
@spec contains?(t(), [t()] | pattern()) :: boolean()
```

Searches if `string` contains any of the given `contents`.

`contents` can be either a string, a list of strings, or a compiled pattern. If `contents` is a list, this function will search if any of the strings in `contents` are part of `string`.

> **Searching for a string in a list**
>
> If you want to check if `string` is listed in `contents`, where `contents` is a list, use `Enum.member?(contents, string)` instead.

Examples

```
iex> String.contains?("elixir of life", "of")
true
iex> String.contains?("elixir of life", ["life",
"death"])
true
iex> String.contains?("elixir of life", ["death",
```

```
"mercury"])
false
```

The argument can also be a compiled pattern:

```
iex> pattern = :binary.compile_pattern(["life", "death"])
iex> String.contains?("elixir of life", pattern)
true
```

An empty string will always match:

```
iex> String.contains?("elixir of life", "")
true
iex> String.contains?("elixir of life", ["", "other"])
true
```

An empty list will never match:

```
iex> String.contains?("elixir of life", [])
false

iex> String.contains?("", [])
false
```

Be aware that this function can match within or across grapheme boundaries. For example, take the grapheme "é" which is made of the characters "e" and the acute accent. The following returns `true`:

```
iex> String.contains?(String.normalize("é", :nfd), "e")
true
```

However, if "é" is represented by the single character "e with acute" accent, then it will return `false`:

```
iex> String.contains?(String.normalize("é", :nfc), "e")
false
```

count(string, pattern)

(since 1.19.0)

```
@spec count(t(), pattern() | Regex.t()) :: non_neg_integer()
```

Counts the number of non-overlapping occurrences of a `pattern` in a `string`.

In case the pattern is an empty string, the function returns 1 + the number of graphemes in the string.

Examples

```
iex> String.count("hello world", "o")
2

iex> String.count("hello world", "l")
3

iex> String.count("hello world", "x")
0

iex> String.count("hello world", ~r/o/)
2

iex> String.count("Hellooo", "oo")
```

```
1

iex> String.count("hello world", "")
12
```

The `pattern` can also be a list:

```
iex> String.count("hello world", ["e", "o"])
3

iex> String.count("hello world", [])
0
```

Or a compiled pattern:

```
iex> pattern = :binary.compile_pattern([" ", "!"])
iex> String.count("foo bar baz!!", pattern)
4
```

downcase(string, mode \\ :default)

```
@spec downcase(t(), :default | :ascii | :greek | :turkic) ::
t()
```

Converts all characters in the given string to lowercase according to `mode`.

`mode` may be `:default`, `:ascii`, `:greek` or `:turkic`. The `:default` mode considers all non-conditional transformations outlined in the Unicode standard. `:ascii` lowercases only the letters A to Z. `:greek`

includes the context sensitive mappings found in Greek. `:turkic` properly handles the letter i with the dotless variant.

Also see `upcase/2` and `capitalize/2` for other conversions.

Examples

```
iex> String.downcase("ABCD")
"abcd"

iex> String.downcase("AB 123 XPTO")
"ab 123 xpto"

iex> String.downcase("OLÁ")
"olá"
```

The `:ascii` mode ignores Unicode characters and provides a more performant implementation when you know the string contains only ASCII characters:

```
iex> String.downcase("OLÁ", :ascii)
"olÁ"
```

The `:greek` mode properly handles the context sensitive sigma in Greek:

```
iex> String.downcase("ΣΣ")
"σσ"

iex> String.downcase("ΣΣ", :greek)
"σς"
```

And `:turkic` properly handles the letter i with the dotless variant:

```
iex> String.downcase("Iİ")
"ii̇"

iex> String.downcase("Iİ", :turkic)
"ıi"
```

duplicate(subject, n)

```
@spec duplicate(t(), non_neg_integer()) :: t()
```

Returns a string `subject` repeated `n` times.

Inlined by the compiler.

Examples

```
iex> String.duplicate("abc", 0)
""

iex> String.duplicate("abc", 1)
"abc"

iex> String.duplicate("abc", 2)
"abcabc"
```

ends_with?(string, suffix)

```
@spec ends_with?(t(), t() | [t()]) :: boolean()
```

Returns `true` if `string` ends with any of the suffixes given.

`suffixes` can be either a single suffix or a list of suffixes.

Examples

```
iex> String.ends_with?("language", "age")
true
iex> String.ends_with?("language", ["youth", "age"])
true
iex> String.ends_with?("language", ["youth", "elixir"])
false
```

An empty suffix will always match:

```
iex> String.ends_with?("language", "")
true
iex> String.ends_with?("language", ["", "other"])
true
```

equivalent?(string1, string2)

```
@spec equivalent?(t(), t()) :: boolean()
```

Returns `true` if `string1` is canonically equivalent to `string2`.

It performs Normalization Form Canonical Decomposition (NFD) on the strings before comparing them. This function is equivalent to:

```
String.normalize(string1, :nfd) ==
String.normalize(string2, :nfd)
```

If you plan to compare multiple strings, multiple times in a row, you may normalize them upfront and compare them directly to avoid multiple normalization passes.

Examples

```
iex> String.equivalent?("abc", "abc")
true

iex> String.equivalent?("man\u0303ana", "mañana")
true

iex> String.equivalent?("abc", "ABC")
false

iex> String.equivalent?("nø", "nó")
false
```

first(string)

```
@spec first(t()) :: grapheme() | nil
```

Returns the first grapheme from a UTF-8 string, `nil` if the string is empty.

Examples

```
iex> String.first("elixir")
"e"

iex> String.first("ɛnqɬh")
"ɛ"

iex> String.first("")
nil
```

graphemes(string)

```
@spec graphemes(t()) :: [grapheme()]
```

Returns Unicode graphemes in the string as per Extended Grapheme Cluster algorithm.

The algorithm is outlined in the Unicode Standard Annex #29, Unicode Text Segmentation.

For details about code points and graphemes, see the `String` module documentation.

Examples

```
iex> String.graphemes("Ńaïve")
["Ń", "a", "ï", "v", "e"]

iex> String.graphemes("\u00e9")
["é"]
```

```
iex> String.graphemes("\u0065\u0301")
["é"]
```

jaro_distance(string1, string2)

```
@spec jaro_distance(t(), t()) :: float()
```

Computes the Jaro distance (similarity) between two strings.

Returns a float value between `0.0` (equates to no similarity) and `1.0` (is an exact match) representing Jaro distance between `string1` and `string2`.

The Jaro distance metric is designed and best suited for short strings such as person names. Elixir itself uses this function to provide the "did you mean?" functionality. For instance, when you are calling a function in a module and you have a typo in the function name, we attempt to suggest the most similar function name available, if any, based on the `jaro_distance/2` score.

Examples

```
iex> String.jaro_distance("Dwayne", "Duane")
0.8222222222222223
iex> String.jaro_distance("even", "odd")
0.0
iex> String.jaro_distance("same", "same")
1.0
```

last(string)

```
@spec last(t()) :: grapheme() | nil
```

Returns the last grapheme from a UTF-8 string, `nil` if the string is empty.

It traverses the whole string to find its last grapheme.

Examples

```
iex> String.last("")
nil

iex> String.last("elixir")
"r"

iex> String.last("ɛnqɬh")
"h"
```

length(string)

```
@spec length(t()) :: non_neg_integer()
```

Returns the number of Unicode graphemes in a UTF-8 string.

Examples

```
iex> String.length("elixir")
6
```

```
iex> String.length("ɓnqɻh")
5
```

match?(string, regex)

```
@spec match?(t(), Regex.t()) :: boolean()
```

Checks if `string` matches the given regular expression.

Examples

```
iex> String.match?("foo", ~r/foo/)
true

iex> String.match?("bar", ~r/foo/)
false
```

Elixir also provides text-based match operator `=~/2` and function `Regex.match?/2` as alternatives to test strings against regular expressions.

myers_difference(string1, string2)

(since 1.3.0)

```
@spec myers_difference(t(), t()) :: [{:eq | :ins | :del,
t()}]
```

Returns a keyword list that represents an edit script.

Check `List.myers_difference/2` for more information.

Examples

```
iex> string1 = "fox hops over the dog"
iex> string2 = "fox jumps over the lazy cat"
iex> String.myers_difference(string1, string2)
[eq: "fox ", del: "ho", ins: "jum", eq: "ps over the ",
del: "dog", ins: "lazy cat"]
```

next_codepoint(arg)

```
@spec next_codepoint(t()) :: {codepoint(), t()} | nil
```

Returns the next code point in a string.

The result is a tuple with the code point and the remainder of the string or `nil` in case the string reached its end.

As with other functions in the `String` module, `next_codepoint/1` works with binaries that are invalid UTF-8. If the string starts with a sequence of bytes that is not valid in UTF-8 encoding, the first element of the returned tuple is a binary with the first byte.

Examples

```
iex> String.next_codepoint("olá")
{"o", "lá"}

iex> invalid = "\x80\x80OK" # first two bytes are invalid
in UTF-8
iex> {_, rest} = String.next_codepoint(invalid)
```

```
{<<128>>, <<128, 79, 75>>}
iex> String.next_codepoint(rest)
{<<128>>, "OK"}
```

Comparison with binary pattern matching

Binary pattern matching provides a similar way to decompose a string:

```
iex> <<codepoint::utf8, rest::binary>> = "Elixir"
"Elixir"
iex> codepoint
69
iex> rest
"lixir"
```

though not entirely equivalent because `codepoint` comes as an integer, and the pattern won't match invalid UTF-8.

Binary pattern matching, however, is simpler and more efficient, so pick the option that better suits your use case.

next_grapheme(string)

```
@spec next_grapheme(t()) :: {grapheme(), t()} | nil
```

Returns the next grapheme in a string.

The result is a tuple with the grapheme and the remainder of the string or `nil` in case the String reached its end.

Examples

```
iex> String.next_grapheme("olá")
{"o", "lá"}

iex> String.next_grapheme("")
nil
```

next_grapheme_size(string)

```
@spec next_grapheme_size(t()) :: {pos_integer(), t()} | nil
```

Returns the size (in bytes) of the next grapheme.

The result is a tuple with the next grapheme size in bytes and the remainder of the string or `nil` in case the string reached its end.

Examples

```
iex> String.next_grapheme_size("olá")
{1, "lá"}

iex> String.next_grapheme_size("")
nil
```

normalize(string, form)

```
@spec normalize(t(), :nfd | :nfc | :nfkd | :nfkc) :: t()
```

Converts all characters in `string` to Unicode normalization form identified by `form`.

Invalid Unicode codepoints are skipped and the remaining of the string is converted. If you want the algorithm to stop and return on invalid codepoint, use `:unicode.characters_to_nfd_binary/1`, `:unicode.characters_to_nfc_binary/1`, `:unicode.characters_to_nfkd_binary/1`, and `:unicode.characters_to_nfkc_binary/1` instead.

Normalization forms `:nfkc` and `:nfkd` should not be blindly applied to arbitrary text. Because they erase many formatting distinctions, they will prevent round-trip conversion to and from many legacy character sets.

Forms

The supported forms are:

- `:nfd` - Normalization Form Canonical Decomposition. Characters are decomposed by canonical equivalence, and multiple combining characters are arranged in a specific order.
- `:nfc` - Normalization Form Canonical Composition. Characters are decomposed and then recomposed by canonical equivalence.
- `:nfkd` - Normalization Form Compatibility Decomposition. Characters are decomposed by compatibility equivalence, and multiple combining characters are arranged in a specific order.
- `:nfkc` - Normalization Form Compatibility Composition. Characters are decomposed and then recomposed by compatibility equivalence.

Examples

```
iex> String.normalize("yêṩ", :nfd)
"yêṩ"
```

```
iex> String.normalize("leña", :nfc)
"leña"

iex> String.normalize("fi", :nfkd)
"fi"

iex> String.normalize("fi", :nfkc)
"fi"
```

pad_leading(string, count, padding \\ [" "])

```
@spec pad_leading(t(), non_neg_integer(), t() | [t()]) :: t()
```

Returns a new string padded with a leading filler which is made of elements from the `padding`.

Passing a list of strings as `padding` will take one element of the list for every missing entry. If the list is shorter than the number of inserts, the filling will start again from the beginning of the list. Passing a string `padding` is equivalent to passing the list of graphemes in it. If no `padding` is given, it defaults to whitespace.

When `count` is less than or equal to the length of `string`, given `string` is returned.

Raises `ArgumentError` if the given `padding` contains a non-string element.

Examples

```
iex> String.pad_leading("abc", 5)
"  abc"

iex> String.pad_leading("abc", 4, "12")
"1abc"

iex> String.pad_leading("abc", 6, "12")
"121abc"

iex> String.pad_leading("abc", 5, ["1", "23"])
"123abc"
```

pad_trailing(string, count, padding \\ [" "])

```
@spec pad_trailing(t(), non_neg_integer(), t() | [t()]) ::
t()
```

Returns a new string padded with a trailing filler which is made of elements from the `padding`.

Passing a list of strings as `padding` will take one element of the list for every missing entry. If the list is shorter than the number of inserts, the filling will start again from the beginning of the list. Passing a string `padding` is equivalent to passing the list of graphemes in it. If no `padding` is given, it defaults to whitespace.

When `count` is less than or equal to the length of `string`, given `string` is returned.

Raises `ArgumentError` if the given `padding` contains a non-string element.

Examples

```
iex> String.pad_trailing("abc", 5)
"abc  "

iex> String.pad_trailing("abc", 4, "12")
"abc1"

iex> String.pad_trailing("abc", 6, "12")
"abc121"

iex> String.pad_trailing("abc", 5, ["1", "23"])
"abc123"
```

printable?(string, character_limit \\ :infinity)

```
@spec printable?(t(), 0) :: true
@spec printable?(t(), pos_integer() | :infinity) :: boolean()
```

Checks if a string contains only printable characters up to `character_limit`.

Takes an optional `character_limit` as a second argument. If `character_limit` is `0`, this function will return `true`.

Examples

```
iex> String.printable?("abc")
true

iex> String.printable?("abc" <> <<0>>)
false

iex> String.printable?("abc" <> <<0>>, 2)
```

```
  true

  iex> String.printable?("abc" <> <<0>>, 0)
  true
```

replace(subject, pattern, replacement, options \\ [])

```
@spec replace(
  t(),
  pattern() | Regex.t(),
  t() | (t() -> t() | iodata()),
  replace_opts()
) :: t()
```

Returns a new string created by replacing occurrences of `pattern` in `subject` with `replacement`.

The `subject` is always a string.

The `pattern` may be a string, a list of strings, a regular expression, or a compiled pattern.

The `replacement` may be a string or a function that receives the matched pattern and must return the replacement as a string or iodata.

By default it replaces all occurrences but this behavior can be controlled through the `:global` option; see the "Options" section below.

Options

- `:global` - (boolean) if `true`, all occurrences of `pattern` are replaced with `replacement`, otherwise only the first occurrence is replaced. Defaults to `true`

Examples

```
iex> String.replace("a,b,c", ",", "-")
"a-b-c"

iex> String.replace("a,b,c", ",", "-", global: false)
"a-b,c"
```

The pattern may also be a list of strings and the replacement may also be a function that receives the matches:

```
iex> String.replace("a,b,c", ["a", "c"], fn <<char>> ->
<<char + 1>> end)
"b,b,d"
```

When the pattern is a regular expression, one can give `\N` or `\g{N}` in the `replacement` string to access a specific capture in the regular expression:

```
iex> String.replace("a,b,c", ~r/,(.)/, ",\\1\\g{1}")
"a,bb,cc"
```

Note that we had to escape the backslash escape character (i.e., we used `\\N` instead of just `\N` to escape the backslash; same thing for `\\g{N}`). By giving `\0`, one can inject the whole match in the replacement string.

A compiled pattern can also be given:

```
iex> pattern = :binary.compile_pattern(",")
iex> String.replace("a,b,c", pattern, "[]")
```

```
"a[]b[]c"
```

When an empty string is provided as a `pattern`, the function will treat it as an implicit empty string between each grapheme and the string will be interspersed. If an empty string is provided as `replacement` the `subject` will be returned:

```
iex> String.replace("ELIXIR", "", ".")
".E.L.I.X.I.R."

iex> String.replace("ELIXIR", "", "")
"ELIXIR"
```

Be aware that this function can replace within or across grapheme boundaries. For example, take the grapheme "é" which is made of the characters "e" and the acute accent. The following will replace only the letter "e", moving the accent to the letter "o":

```
iex> String.replace(String.normalize("é", :nfd), "e",
"o")
"ó"
```

However, if "é" is represented by the single character "e with acute" accent, then it won't be replaced at all:

```
iex> String.replace(String.normalize("é", :nfc), "e",
"o")
"é"
```

replace_invalid(bytes, replacement \\ "")

(since 1.16.0)

```
@spec replace_invalid(binary(), t()) :: t()
```

Returns a new string created by replacing all invalid bytes with `replacement` (`"&#65533;"` by default).

Examples

```
iex> String.replace_invalid("asd" <> <<0xFF::8>>)
"asd&#65533;"

iex> String.replace_invalid("nem rán bề bề")
"nem rán bề bề"

iex> String.replace_invalid("nem rán b" <> <<225, 187>>
<> " bề")
"nem rán b&#65533; bề"

iex> String.replace_invalid("nem rán b" <> <<225, 187>>
<> " bề", "ERROR!")
"nem rán bERROR! bề"
```

replace_leading(string, match, replacement)

```
@spec replace_leading(t(), t(), t()) :: t()
```

Replaces all leading occurrences of `match` by `replacement` of `match` in `string`.

Returns the string untouched if there are no occurrences.

If `match` is `""`, this function raises an `ArgumentError` exception: this happens because this function replaces **all** the occurrences of `match` at the beginning of `string`, and it's impossible to replace "multiple" occurrences of `""`.

Examples

```
iex> String.replace_leading("hello world", "hello ", "")
"world"
iex> String.replace_leading("hello hello world", "hello
", "")
"world"

iex> String.replace_leading("hello world", "hello ", "ola
")
"ola world"
iex> String.replace_leading("hello hello world", "hello
", "ola ")
"ola ola world"
```

This function can replace across grapheme boundaries. See `replace/3` for more information and examples.

replace_prefix(string, match, replacement)

```
@spec replace_prefix(t(), t(), t()) :: t()
```

Replaces prefix in `string` by `replacement` if it matches `match`.

Returns the string untouched if there is no match. If `match` is an empty string (`""`), `replacement` is just prepended to `string`.

Examples

```
iex> String.replace_prefix("world", "hello ", "")
"world"
iex> String.replace_prefix("hello world", "hello ", "")
"world"
iex> String.replace_prefix("hello hello world", "hello ",
"")
"hello world"

iex> String.replace_prefix("world", "hello ", "ola ")
"world"
iex> String.replace_prefix("hello world", "hello ", "ola
")
"ola world"
iex> String.replace_prefix("hello hello world", "hello ",
"ola ")
"ola hello world"

iex> String.replace_prefix("world", "", "hello ")
"hello world"
```

This function can replace across grapheme boundaries. See `replace/3` for more information and examples.

replace_suffix(string, match, replacement)

```
@spec replace_suffix(t(), t(), t()) :: t()
```

Replaces suffix in `string` by `replacement` if it matches `match`.

Returns the string untouched if there is no match. If `match` is an empty string (`""`), `replacement` is just appended to `string`.

Examples

```
iex> String.replace_suffix("hello", " world", "")
"hello"
iex> String.replace_suffix("hello world", " world", "")
"hello"
iex> String.replace_suffix("hello world world", " world",
"")
"hello world"

iex> String.replace_suffix("hello", " world", " mundo")
"hello"
iex> String.replace_suffix("hello world", " world", "
mundo")
"hello mundo"
iex> String.replace_suffix("hello world world", " world",
" mundo")
"hello world mundo"

iex> String.replace_suffix("hello", "", " world")
"hello world"
```

This function can replace across grapheme boundaries. See `replace/3` for more information and examples.

replace_trailing(string, match, replacement)

```
@spec replace_trailing(t(), t(), t()) :: t()
```

Replaces all trailing occurrences of `match` by `replacement` in `string`.

Returns the string untouched if there are no occurrences.

If `match` is `""`, this function raises an `ArgumentError` exception: this happens because this function replaces **all** the occurrences of `match` at the end of `string`, and it's impossible to replace "multiple" occurrences of `""`.

Examples

```
iex> String.replace_trailing("hello world", " world", "")
"hello"
iex> String.replace_trailing("hello world world", "
world", "")
"hello"

iex> String.replace_trailing("hello world", " world", "
mundo")
"hello mundo"
iex> String.replace_trailing("hello world world", "
world", " mundo")
"hello mundo mundo"
```

This function can replace across grapheme boundaries. See `replace/3` for more information and examples.

reverse(string)

```
@spec reverse(t()) :: t()
```

Reverses the graphemes in given string.

Examples

```
iex> String.reverse("abcd")
"dcba"

iex> String.reverse("hello world")
"dlrow olleh"

iex> String.reverse("hello ∂og")
"go∂ olleh"
```

Keep in mind reversing the same string twice does not necessarily yield the original string:

```
iex> "̀e"
"̀e"
iex> String.reverse("̀e")
"è"
iex> String.reverse(String.reverse("̀e"))
"è"
```

In the first example the accent is before the vowel, so it is considered two graphemes. However, when you reverse it once, you have the vowel followed by the accent, which becomes one grapheme. Reversing it again will keep it as one single grapheme.

slice(string, range)

```
@spec slice(t(), Range.t()) :: t()
```

Returns a substring from the offset given by the start of the range to the offset given by the end of the range.

This function works on Unicode graphemes. For example, slicing the first three characters of the string "héllo" will return "hél", which

internally is represented by more than three bytes. Use `String.byte_slice/3` if you want to slice by a given number of bytes, while respecting the codepoint boundaries. If you want to work on raw bytes, check `Kernel.binary_part/3` or `Kernel.binary_slice/3` instead.

If the start of the range is not a valid offset for the given string or if the range is in reverse order, returns `""`.

If the start or end of the range is negative, the whole string is traversed first in order to convert the negative indices into positive ones.

Examples

```
iex> String.slice("elixir", 1..3)
"lix"
iex> String.slice("elixir", 1..10)
"lixir"

iex> String.slice("elixir", -4..-1)
"ixir"
iex> String.slice("elixir", -4..6)
"ixir"
iex> String.slice("elixir", -100..100)
"elixir"
```

For ranges where `start > stop`, you need to explicitly mark them as increasing:

```
iex> String.slice("elixir", 2..-1//1)
"ixir"
```

```
iex> String.slice("elixir", 1..-2//1)
"lixi"
```

You can use `../0` as a shortcut for `0..-1//1`, which returns the whole string as is:

```
iex> String.slice("elixir", ..)
"elixir"
```

The step can be any positive number. For example, to get every 2 characters of the string:

```
iex> String.slice("elixir", 0..-1//2)
"eii"
```

If the first position is after the string ends or after the last position of the range, it returns an empty string:

```
iex> String.slice("elixir", 10..3//1)
""
iex> String.slice("a", 1..1500)
""
```

slice(string, start, length)

```
@spec slice(t(), integer(), non_neg_integer()) :: grapheme()
```

Returns a substring starting at the offset `start`, and of the given `length`.

This function works on Unicode graphemes. For example, slicing the first three characters of the string "héllo" will return "hél", which internally is represented by more than three bytes. Use `String.byte_slice/3` if you want to slice by a given number of bytes, while respecting the codepoint boundaries. If you want to work on raw bytes, check `Kernel.binary_part/3` or `Kernel.binary_slice/3` instead.

If the offset is greater than string length, then it returns `""`.

Examples

```
iex> String.slice("elixir", 1, 3)
"lix"

iex> String.slice("elixir", 1, 10)
"lixir"

iex> String.slice("elixir", 10, 3)
""
```

If the start position is negative, it is normalized against the string length and clamped to 0:

```
iex> String.slice("elixir", -4, 4)
"ixir"

iex> String.slice("elixir", -10, 3)
"eli"
```

If start is more than the string length, an empty string is returned:

```
iex> String.slice("elixir", 10, 1500)
""
```

split(binary)

```
@spec split(t()) :: [t()]
```

Divides a string into substrings at each Unicode whitespace occurrence with leading and trailing whitespace ignored.

Groups of whitespace are treated as a single occurrence. Divisions do not occur on non-breaking whitespace.

Examples

```
iex> String.split("foo bar")
["foo", "bar"]

iex> String.split("foo" <> <<194, 133>> <> "bar")
["foo", "bar"]

iex> String.split(" foo   bar ")
["foo", "bar"]

iex> String.split("no\u00a0break")
["no\u00a0break"]
```

Removes empty strings, like when using `trim: true` in `String.split/3`.

```
iex> String.split(" ")
[]
```

split(string, pattern, options \\ [])

```
@spec split(t(), pattern(), split_opts()) :: [t()]
@spec split(t(), Regex.t(), Regex.split_opts()) :: [t()]
```

Divides a string into parts based on a pattern.

Returns a list of these parts.

The `pattern` may be a string, a list of strings, a regular expression, or a compiled pattern.

The string is split into as many parts as possible by default, but can be controlled via the `:parts` option.

Empty strings are only removed from the result if the `:trim` option is set to `true`.

When the pattern used is a regular expression, the string is split using `Regex.split/3`.

If the pattern cannot be found, a list containing the original string will be returned.

Options

- `:parts` (positive integer or `:infinity`) - the string is split into at most as many parts as this option specifies. If `:infinity`, the string will be split into all possible parts. Defaults to `:infinity`.

- `:trim` (boolean) - if `true`, empty strings are removed from the resulting list.

This function also accepts all options accepted by `Regex.split/3` if `pattern` is a regular expression.

Examples

Splitting with a string pattern:

```
iex> String.split("a,b,c", ",")
["a", "b", "c"]

iex> String.split("a,b,c", ",", parts: 2)
["a", "b,c"]

iex> String.split(" a b c ", " ", trim: true)
["a", "b", "c"]
```

A list of patterns:

```
iex> String.split("1,2 3,4", [" ", ","])
["1", "2", "3", "4"]
```

A regular expression:

```
iex> String.split("a,b,c", ~r{,})
["a", "b", "c"]

iex> String.split("a,b,c", ~r{,}, parts: 2)
["a", "b,c"]
```

```
iex> String.split(" a b c ", ~r{\s}, trim: true)
["a", "b", "c"]

iex> String.split("abc", ~r{b}, include_captures: true)
["a", "b", "c"]
```

A compiled pattern:

```
iex> pattern = :binary.compile_pattern([" ", ","])
iex> String.split("1,2 3,4", pattern)
["1", "2", "3", "4"]
```

Splitting on empty string returns graphemes:

```
iex> String.split("abc", "")
["", "a", "b", "c", ""]

iex> String.split("abc", "", trim: true)
["a", "b", "c"]

iex> String.split("abc", "", parts: 1)
["abc"]

iex> String.split("abc", "", parts: 3)
["", "a", "bc"]
```

Splitting on a non-existing pattern returns the original string:

```
iex> String.split("abc", ",")
["abc"]
```

Be aware that this function can split within or across grapheme boundaries. For example, take the grapheme "é" which is made of the characters "e" and the acute accent. The following will split the string into two parts:

```
iex> String.split(String.normalize("é", :nfd), "e")
["", "́"]
```

However, if "é" is represented by the single character "e with acute" accent, then it will split the string into just one part:

```
iex> String.split(String.normalize("é", :nfc), "e")
["é"]
```

When using both the `:trim` and the `:parts` option, the empty values are removed as the parts are computed (if any). No trimming happens after all parts are computed:

```
iex> String.split(" a  b  c  ", " ", trim: true, parts:
2)
["a", " b  c  "]
iex> String.split(" a  b  c  ", " ", trim: true, parts:
3)
["a", "b", " c  "]
```

split_at(string, position)

```
@spec split_at(t(), integer()) :: {t(), t()}
```

Splits a string into two at the specified offset. When the offset given is negative, location is counted from the end of the string.

The offset is capped to the length of the string. Returns a tuple with two elements.

> **Linear Access**
>
> This function splits on graphemes and for such it has to linearly traverse the string. If you want to split a string or a binary based on the number of bytes, use `Kernel.binary_part/3` instead.

Examples

```
iex> String.split_at("sweetelixir", 5)
{"sweet", "elixir"}

iex> String.split_at("sweetelixir", -6)
{"sweet", "elixir"}

iex> String.split_at("abc", 0)
{"", "abc"}

iex> String.split_at("abc", 1000)
{"abc", ""}

iex> String.split_at("abc", -1000)
{"", "abc"}
```

splitter(string, pattern, options \\ [])

```
@spec splitter(t(), pattern(), splitter_opts()) ::
Enumerable.t()
```

Returns an enumerable that splits a string on demand.

This is in contrast to `split/3` which splits the entire string upfront.

This function does not support regular expressions by design. When using regular expressions, it is often more efficient to have the regular expressions traverse the string at once than in parts, like this function does.

Options

- :trim - when `true`, does not emit empty patterns

Examples

```
iex> String.splitter("1,2 3,4 5,6 7,8,...,99999", [" ",
","]) |> Enum.take(4)
["1", "2", "3", "4"]

iex> String.splitter("abcd", "") |> Enum.take(10)
["", "a", "b", "c", "d", ""]

iex> String.splitter("abcd", "", trim: true) |>
Enum.take(10)
["a", "b", "c", "d"]
```

A compiled pattern can also be given:

```
iex> pattern = :binary.compile_pattern([" ", ","])
iex> String.splitter("1,2 3,4 5,6 7,8,...,99999",
pattern) |> Enum.take(4)
["1", "2", "3", "4"]
```

starts_with?(string, prefix)

```
@spec starts_with?(t(), t() | [t()]) :: boolean()
```

Returns `true` if `string` starts with any of the prefixes given.

`prefix` can be either a string, a list of strings, or a compiled pattern.

Examples

```
iex> String.starts_with?("elixir", "eli")
true
iex> String.starts_with?("elixir", ["erlang", "elixir"])
true
iex> String.starts_with?("elixir", ["erlang", "ruby"])
false
```

An empty string will always match:

```
iex> String.starts_with?("elixir", "")
true
iex> String.starts_with?("elixir", ["", "other"])
true
```

An empty list will never match:

```
iex> String.starts_with?("elixir", [])
false

iex> String.starts_with?("", [])
false
```

to_atom(string)

```
@spec to_atom(t()) :: atom()
```

Converts a string to an existing atom or creates a new one.

Warning: this function creates atoms dynamically and atoms are not garbage-collected. Therefore, `string` should not be an untrusted value, such as input received from a socket or during a web request. Consider using `to_existing_atom/1` instead.

By default, the maximum number of atoms is `1_048_576`. This limit can be raised or lowered using the VM option `+t`.

The maximum atom size is of 255 Unicode code points.

Inlined by the compiler.

Examples

```
iex> String.to_atom("my_atom")
:my_atom
```

to_charlist(string)

```
@spec to_charlist(t()) :: charlist()
```

Converts a string into a charlist.

Specifically, this function takes a UTF-8 encoded binary and returns a list of its integer code points. It is similar to `codepoints/1` except that

the latter returns a list of code points as strings.

In case you need to work with bytes, take a look at the `:binary` module.

Examples

```
iex> String.to_charlist("foo")
~c"foo"
```

to_existing_atom(string)

```
@spec to_existing_atom(t()) :: atom()
```

Converts a string to an existing atom or raises if the atom does not exist.

The maximum atom size is of 255 Unicode code points. Raises an `ArgumentError` if the atom does not exist.

Inlined by the compiler.

> **Atoms and modules**
>
> Since Elixir is a compiled language, the atoms defined in a module will only exist after said module is loaded, which typically happens whenever a function in the module is executed. Therefore, it is generally recommended to call `String.to_existing_atom/1` only to convert atoms defined within the module making the function call to `to_existing_atom/1`.
>
> To create a module name itself from a string safely, it is recommended to use `Module.safe_concat/1`.

Examples

```
iex> _ = :my_atom
iex> String.to_existing_atom("my_atom")
:my_atom
```

to_float(string)

```
@spec to_float(t()) :: float()
```

Returns a float whose text representation is `string`.

`string` must be the string representation of a float including leading digits and a decimal point. To parse a string without decimal point as a float, refer to `Float.parse/1`. Otherwise, an `ArgumentError` will be raised.

Inlined by the compiler.

Examples

```
iex> String.to_float("2.2017764e+0")
2.2017764

iex> String.to_float("3.0")
3.0

String.to_float("3")
** (ArgumentError) argument error

String.to_float(".3")
** (ArgumentError) argument error
```

to_integer(string)

```
@spec to_integer(t()) :: integer()
```

Returns an integer whose text representation is `string`.

`string` must be the string representation of an integer. Otherwise, an `ArgumentError` will be raised. If you want to parse a string that may contain an ill-formatted integer, use `Integer.parse/1`.

Inlined by the compiler.

Examples

```
iex> String.to_integer("123")
123
```

Passing a string that does not represent an integer leads to an error:

```
String.to_integer("invalid data")
** (ArgumentError) argument error
```

to_integer(string, base)

```
@spec to_integer(t(), 2..36) :: integer()
```

Returns an integer whose text representation is `string` in base `base`.

Inlined by the compiler.

Examples

```
iex> String.to_integer("3FF", 16)
1023
```

trim(string)

```
@spec trim(t()) :: t()
```

Returns a string where all leading and trailing Unicode whitespaces have been removed.

Examples

```
iex> String.trim("\n  abc\n  ")
"abc"
```

trim(string, to_trim)

```
@spec trim(t(), t()) :: t()
```

Returns a string where all leading and trailing `to_trim` characters have been removed.

Examples

```
iex> String.trim("a  abc  a", "a")
```

```
"  abc  "
```

trim_leading(string)

```
@spec trim_leading(t()) :: t()
```

Returns a string where all leading Unicode whitespaces have been removed.

Examples

```
iex> String.trim_leading("\n  abc   ")
"abc   "
```

trim_leading(string, to_trim)

```
@spec trim_leading(t(), t()) :: t()
```

Returns a string where all leading `to_trim` characters have been removed.

Examples

```
iex> String.trim_leading("__ abc _", "_")
" abc _"
```

```
iex> String.trim_leading("1 abc", "11")
"1 abc"
```

trim_trailing(string)

```
@spec trim_trailing(t()) :: t()
```

Returns a string where all trailing Unicode whitespaces have been removed.

Examples

```
iex> String.trim_trailing("   abc\n  ")
"   abc"
```

trim_trailing(string, to_trim)

```
@spec trim_trailing(t(), t()) :: t()
```

Returns a string where all trailing `to_trim` characters have been removed.

Examples

```
iex> String.trim_trailing("_ abc __", "_")
"_ abc "
```

```
iex> String.trim_trailing("abc 1", "11")
"abc 1"
```

upcase(string, mode \\ :default)

```
@spec upcase(t(), :default | :ascii | :greek | :turkic) ::
t()
```

Converts all characters in the given string to uppercase according to `mode`.

`mode` may be `:default`, `:ascii`, `:greek` or `:turkic`. The `:default` mode considers all non-conditional transformations outlined in the Unicode standard. `:ascii` uppercases only the letters a to z. `:greek` includes the context sensitive mappings found in Greek. `:turkic` properly handles the letter i with the dotless variant.

Examples

```
iex> String.upcase("abcd")
"ABCD"

iex> String.upcase("ab 123 xpto")
"AB 123 XPTO"

iex> String.upcase("olá")
"OLÁ"
```

The `:ascii` mode ignores Unicode characters and provides a more performant implementation when you know the string contains only ASCII characters:

```
iex> String.upcase("olá", :ascii)
"OLá"
```

And `:turkic` properly handles the letter i with the dotless variant:

```
iex> String.upcase("ıi")
"II"

iex> String.upcase("ıi", :turkic)
"Iİ"
```

Also see `downcase/2` and `capitalize/2` for other conversions.

valid?(string, algorithm \\ :default)

```
@spec valid?(t(), :default | :fast_ascii) :: boolean()
```

Checks whether `string` contains only valid characters.

`algorithm` may be `:default` or `:fast_ascii`. Both algorithms are equivalent from a validation perspective (they will always produce the same output), but `:fast_ascii` can yield significant performance benefits in specific scenarios.

If anything else but a string is given as argument, it raises.

Fast ASCII

If all of the following conditions are true, you may want to experiment with the `:fast_ascii` algorithm to see if it yields performance benefits in your specific scenario:

- You expect most of your strings to be longer than ~64 bytes
- You expect most of your strings to contain mostly ASCII codepoints

Note that the `:fast_ascii` algorithm does not affect correctness, you can expect the output of `String.valid?/2` to be the same regardless of algorithm. The only difference to be expected is one of performance, which can be expected to improve roughly linearly in string length compared to the `:default` algorithm.

Examples

```
iex> String.valid?("a")
true

iex> String.valid?("ø")
true

iex> String.valid?(<<0xFFFF::16>>)
false

iex> String.valid?(<<0xEF, 0xB7, 0x90>>)
true

iex> String.valid?("asd" <> <<0xFFFF::16>>)
false

iex> String.valid?("a", :fast_ascii)
true
```

Time

A Time struct and functions.

The Time struct contains the fields hour, minute, second and microseconds. New times can be built with the `new/4` function or using the `~T` (see `sigil_T/2`) sigil:

```
iex> ~T[23:00:07.001]
~T[23:00:07.001]
```

Both `new/4` and sigil return a struct where the time fields can be accessed directly:

```
iex> time = ~T[23:00:07.001]
iex> time.hour
23
iex> time.microsecond
{1000, 3}
```

The functions on this module work with the `Time` struct as well as any struct that contains the same fields as the `Time` struct, such as `NaiveDateTime` and `DateTime`. Such functions expect `Calendar.time/0` in their typespecs (instead of `t/0`).

Developers should avoid creating the Time structs directly and instead rely on the functions provided by this module as well as the ones in third-party calendar libraries.

Comparing times

Comparisons in Elixir using `==/2`, `>/2`, `</2` and similar are structural and based on the `Time` struct fields. For proper comparison between times, use the `compare/2`, `after?/2` and `before?/2` functions. The existence of the `compare/2` function in this module also allows using `Enum.min/2` and `Enum.max/2` functions to get the minimum and maximum time of an `Enum`. For example:

```
iex> Enum.min([~T[23:00:07.001], ~T[10:00:07.001]], Time)
~T[10:00:07.001]
```

Summary

Types

t()

Functions

add(time, amount_to_add, unit \\ :second)

Adds the `amount_to_add` of `unit`s to the given `time`.

after?(time1, time2)

Returns `true` if the first time is strictly later than the second.

before?(time1, time2)

Returns `true` if the first time is strictly earlier than the second.

compare(time1, time2)

Compares two time structs.
convert(time, calendar)

Converts given `time` to a different calendar.

convert!(time, calendar)

Similar to `Time.convert/2`, but raises an `ArgumentError` if the conversion between the two calendars is not possible.

diff(time1, time2, unit \\ :second)

Returns the difference between two times, considering only the hour, minute, second and microsecond.

from_erl(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Converts an Erlang time tuple to a `Time` struct.

from_erl!(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Converts an Erlang time tuple to a `Time` struct.

from_iso8601(string, calendar \\ Calendar.ISO)

Parses the extended "Local time" format described by ISO 8601:2019.

from_iso8601!(string, calendar \\ Calendar.ISO)

Parses the extended "Local time" format described by ISO 8601:2019.

from_seconds_after_midnight(seconds, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Converts a number of seconds after midnight to a `Time` struct.

new(hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Builds a new time.
new!(hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

Builds a new time.

shift(time, duration)

Shifts given `time` by `duration` according to its calendar.

to_erl(time)

Converts given `time` to an Erlang time tuple.

to_iso8601(time, format \\ :extended)

Converts the given time to ISO 8601:2019.

to_seconds_after_midnight(time)

Converts a `Time` struct to a number of seconds after midnight.

to_string(time)

Converts the given `time` to a string.

truncate(time, precision)

Returns the given time with the microsecond field truncated to the given precision (`:microsecond`, `millisecond` or `:second`).

utc_now(calendar_or_time_unit \\ Calendar.ISO)

Returns the current time in UTC.

utc_now(time_unit, calendar)

Returns the current time in UTC, supporting a precision and a specific calendar.

Types

t()

```
@type t() :: %Time{
  calendar: Calendar.calendar(),
  hour: Calendar.hour(),
  microsecond: Calendar.microsecond(),
  minute: Calendar.minute(),
  second: Calendar.second()
}
```

Functions

add(time, amount_to_add, unit \\ :second)

(since 1.6.0)

```
@spec add(Calendar.time(), integer(), :hour | :minute |
System.time_unit()) :: t()
```

Adds the `amount_to_add` of `unit`s to the given `time`.

> **Prefer** `shift/2`
>
> Prefer `shift/2` over `add/3`, as it offers a more ergonomic API.
>
> `add/3` always considers the unit to be computed according to the `Calendar.ISO`.

Accepts an `amount_to_add` in any `unit`. `unit` can be `:hour`, `:minute`, `:second` or any subsecond precision from

`System.time_unit/0` for convenience but ultimately they are all converted to microseconds. Negative values will move backwards in time and the default precision is `:second`.

Note the result value represents the time of day, meaning that it is cyclic, for instance, it will never go over 24 hours for the ISO calendar.

Examples

```
iex> Time.add(~T[10:00:00], 27000)
~T[17:30:00]
iex> Time.add(~T[11:00:00.005], 2400)
~T[11:40:00.005]
iex> Time.add(~T[00:00:00.000], 86_399_999, :millisecond)
~T[23:59:59.999]
```

Negative values are allowed:

```
iex> Time.add(~T[23:00:00], -60)
~T[22:59:00]
```

Note that the time is cyclic:

```
iex> Time.add(~T[17:10:05], 86400)
~T[17:10:05]
```

Hours and minutes are also supported:

```
iex> Time.add(~T[17:10:05], 2, :hour)
~T[19:10:05]
```

```
iex> Time.add(~T[17:10:05], 30, :minute)
~T[17:40:05]
```

This operation merges the precision of the time with the given unit:

```
iex> result = Time.add(~T[00:29:10], 21, :millisecond)
~T[00:29:10.021]
iex> result.microsecond
{21000, 3}
```

after?(time1, time2)

(since 1.15.0)

```
@spec after?(Calendar.time(), Calendar.time()) :: boolean()
```

Returns `true` if the first time is strictly later than the second.

Examples

```
iex> Time.after?(~T[16:04:28], ~T[16:04:16])
true
iex> Time.after?(~T[16:04:16], ~T[16:04:16])
false
iex> Time.after?(~T[16:04:16.001], ~T[16:04:16.01])
false
```

before?(time1, time2)

(since 1.15.0)

```
@spec before?(Calendar.time(), Calendar.time()) :: boolean()
```

Returns `true` if the first time is strictly earlier than the second.

Examples

```
iex> Time.before?(~T[16:04:16], ~T[16:04:28])
true
iex> Time.before?(~T[16:04:16], ~T[16:04:16])
false
iex> Time.before?(~T[16:04:16.01], ~T[16:04:16.001])
false
```

compare(time1, time2)

(since 1.4.0)

```
@spec compare(Calendar.time(), Calendar.time()) :: :lt | :eq
| :gt
```

Compares two time structs.

Returns `:gt` if first time is later than the second and `:lt` for vice versa. If the two times are equal `:eq` is returned.

Examples

```
iex> Time.compare(~T[16:04:16], ~T[16:04:28])
:lt
iex> Time.compare(~T[16:04:16], ~T[16:04:16])
:eq
```

```
iex> Time.compare(~T[16:04:16.01], ~T[16:04:16.001])
:gt
```

This function can also be used to compare across more complex calendar types by considering only the time fields:

```
iex> Time.compare(~N[1900-01-01 16:04:16], ~N[2015-01-01
16:04:16])
:eq
iex> Time.compare(~N[2015-01-01 16:04:16], ~N[2015-01-01
16:04:28])
:lt
iex> Time.compare(~N[2015-01-01 16:04:16.01], ~N[2000-01-
01 16:04:16.001])
:gt
```

convert(time, calendar)

(since 1.5.0)

```
@spec convert(Calendar.time(), Calendar.calendar()) :: {:ok,
t()} | {:error, atom()}
```

Converts given `time` to a different calendar.

Returns `{:ok, time}` if the conversion was successful, or `{:error, reason}` if it was not, for some reason.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> Time.convert(~T[13:30:15], Calendar.Holocene)
{:ok, %Time{calendar: Calendar.Holocene, hour: 13,
minute: 30, second: 15, microsecond: {0, 0}}}
```

convert!(time, calendar)

(since 1.5.0)

```
@spec convert!(Calendar.time(), Calendar.calendar()) :: t()
```

Similar to `Time.convert/2`, but raises an `ArgumentError` if the conversion between the two calendars is not possible.

Examples

Imagine someone implements `Calendar.Holocene`, a calendar based on the Gregorian calendar that adds exactly 10 000 years to the current Gregorian year:

```
iex> Time.convert!(~T[13:30:15], Calendar.Holocene)
%Time{calendar: Calendar.Holocene, hour: 13, minute: 30,
second: 15, microsecond: {0, 0}}
```

diff(time1, time2, unit \\ :second)

(since 1.5.0)

```
@spec diff(Calendar.time(), Calendar.time(), :hour | :minute
| System.time_unit()) ::
  integer()
```

Returns the difference between two times, considering only the hour, minute, second and microsecond.

As with the `compare/2` function both `Time` structs and other structures containing time can be used. If for instance a `NaiveDateTime` or `DateTime` is passed, only the hour, minute, second, and microsecond is considered. Any additional information about a date or time zone is ignored when calculating the difference.

The answer can be returned in any `:hour`, `:minute`, `:second` or any subsecond `unit` available from `System.time_unit/0`. If the first time value is earlier than the second, a negative number is returned.

The unit is measured according to `Calendar.ISO` and defaults to `:second`. Fractional results are not supported and are truncated.

Examples

```
iex> Time.diff(~T[00:29:12], ~T[00:29:10])
2

# When passing a `NaiveDateTime` the date part is
ignored.
iex> Time.diff(~N[2017-01-01 00:29:12], ~T[00:29:10])
2

# Two `NaiveDateTime` structs could have big differences
in the date
# but only the time part is considered.
iex> Time.diff(~N[2017-01-01 00:29:12], ~N[1900-02-03
00:29:10])
2

iex> Time.diff(~T[00:29:12], ~T[00:29:10], :microsecond)
```

```
2_000_000
iex> Time.diff(~T[00:29:10], ~T[00:29:12], :microsecond)
-2_000_000

iex> Time.diff(~T[02:29:10], ~T[00:29:10], :hour)
2
iex> Time.diff(~T[02:29:10], ~T[00:29:11], :hour)
1
```

from_erl(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec from_erl(
  :calendar.time(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: {:ok, t()} | {:error, atom()}
```

Converts an Erlang time tuple to a `Time` struct.

Examples

```
iex> Time.from_erl({23, 30, 15})
{:ok, ~T[23:30:15]}
iex> Time.from_erl({23, 30, 15}, 5000)
{:ok, ~T[23:30:15.005000]}
iex> Time.from_erl({23, 30, 15}, {5000, 3})
{:ok, ~T[23:30:15.005]}
iex> Time.from_erl({24, 30, 15})
{:error, :invalid_time}
```

from_erl!(tuple, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec from_erl!(:calendar.time(), Calendar.microsecond(),
Calendar.calendar()) :: t()
```

Converts an Erlang time tuple to a `Time` struct.

Examples

```
iex> Time.from_erl!({23, 30, 15})
~T[23:30:15]
iex> Time.from_erl!({23, 30, 15}, 5000)
~T[23:30:15.005000]
iex> Time.from_erl!({23, 30, 15}, {5000, 3})
~T[23:30:15.005]
iex> Time.from_erl!({24, 30, 15})
** (ArgumentError) cannot convert {24, 30, 15} to time,
reason: :invalid_time
```

from_iso8601(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601(String.t(), Calendar.calendar()) :: {:ok,
t()} | {:error, atom()}
```

Parses the extended "Local time" format described by ISO 8601:2019.

Time zone offset may be included in the string but they will be simply discarded as such information is not included in times.

As specified in the standard, the separator "T" may be omitted if desired as there is no ambiguity within this function.

Examples

```
iex> Time.from_iso8601("23:50:07")
{:ok, ~T[23:50:07]}
iex> Time.from_iso8601("23:50:07Z")
{:ok, ~T[23:50:07]}
iex> Time.from_iso8601("T23:50:07Z")
{:ok, ~T[23:50:07]}

iex> Time.from_iso8601("23:50:07,0123456")
{:ok, ~T[23:50:07.012345]}
iex> Time.from_iso8601("23:50:07.0123456")
{:ok, ~T[23:50:07.012345]}
iex> Time.from_iso8601("23:50:07.123Z")
{:ok, ~T[23:50:07.123]}

iex> Time.from_iso8601("2015:01:23 23-50-07")
{:error, :invalid_format}
iex> Time.from_iso8601("23:50:07A")
{:error, :invalid_format}
iex> Time.from_iso8601("23:50:07.")
{:error, :invalid_format}
iex> Time.from_iso8601("23:50:61")
{:error, :invalid_time}
```

from_iso8601!(string, calendar \\ Calendar.ISO)

```
@spec from_iso8601!(String.t(), Calendar.calendar()) :: t()
```

Parses the extended "Local time" format described by ISO 8601:2019.

Raises if the format is invalid.

Examples

```
iex> Time.from_iso8601!("23:50:07,123Z")
~T[23:50:07.123]
iex> Time.from_iso8601!("23:50:07.123Z")
~T[23:50:07.123]
iex> Time.from_iso8601!("2015:01:23 23-50-07")
** (ArgumentError) cannot parse "2015:01:23 23-50-07" as
time, reason: :invalid_format
```

from_seconds_after_midnight(seconds, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec from_seconds_after_midnight(
  integer(),
  Calendar.microsecond(),
  Calendar.calendar()
) :: t()
```

Converts a number of seconds after midnight to a `Time` struct.

Examples

```
iex> Time.from_seconds_after_midnight(10_000)
~T[02:46:40]
iex> Time.from_seconds_after_midnight(30_000, {5000, 3})
~T[08:20:00.005]
iex> Time.from_seconds_after_midnight(-1)
~T[23:59:59]
iex> Time.from_seconds_after_midnight(100_000)
~T[03:46:40]
```

new(hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

```
@spec new(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: {:ok, t()} | {:error, atom()}
```

Builds a new time.

Expects all values to be integers. Returns `{:ok, time}` if each entry fits its appropriate range, returns `{:error, reason}` otherwise.

Microseconds can also be given with a precision, which must be an integer between 0 and 6.

The built-in calendar does not support leap seconds.

Examples

```
iex> Time.new(0, 0, 0, 0)
{:ok, ~T[00:00:00.000000]}
iex> Time.new(23, 59, 59, 999_999)
{:ok, ~T[23:59:59.999999]}

iex> Time.new(24, 59, 59, 999_999)
{:error, :invalid_time}
iex> Time.new(23, 60, 59, 999_999)
{:error, :invalid_time}
iex> Time.new(23, 59, 60, 999_999)
{:error, :invalid_time}
iex> Time.new(23, 59, 59, 1_000_000)
```

```
{:error, :invalid_time}

# Invalid precision
Time.new(23, 59, 59, {999_999, 10})
{:error, :invalid_time}
```

new!(hour, minute, second, microsecond \\ {0, 0}, calendar \\ Calendar.ISO)

(since 1.11.0)

```
@spec new!(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond() | non_neg_integer(),
  Calendar.calendar()
) :: t()
```

Builds a new time.

Expects all values to be integers. Returns `time` if each entry fits its appropriate range, raises if the time is invalid.

Microseconds can also be given with a precision, which must be an integer between 0 and 6.

The built-in calendar does not support leap seconds.

Examples

```
iex> Time.new!(0, 0, 0, 0)
~T[00:00:00.000000]
iex> Time.new!(23, 59, 59, 999_999)
~T[23:59:59.999999]
```

```
iex> Time.new!(24, 59, 59, 999_999)
** (ArgumentError) cannot build time, reason:
:invalid_time
```

shift(time, duration)

(since 1.17.0)

```
@spec shift(Calendar.time(), Duration.t() | [unit_pair]) ::
t()
when unit_pair:
       {:hour, integer()}
       | {:minute, integer()}
       | {:second, integer()}
       | {:microsecond, {integer(), 0..6}}
```

Shifts given `time` by `duration` according to its calendar.

Available duration units are: `:hour`, `:minute`, `:second`, `:microsecond`.

When using the default ISO calendar, durations are collapsed to seconds and microseconds before they are applied.

Raises an `ArgumentError` when called with date scale units.

Examples

```
iex> Time.shift(~T[01:00:15], hour: 12)
~T[13:00:15]
iex> Time.shift(~T[01:35:00], hour: 6, minute: -15)
~T[07:20:00]
iex> Time.shift(~T[01:15:00], second: 125)
~T[01:17:05]
```

```
iex> Time.shift(~T[01:00:15], microsecond: {100, 6})
~T[01:00:15.000100]
iex> Time.shift(~T[01:15:00], Duration.new!(second: 65))
~T[01:16:05]
```

to_erl(time)

```
@spec to_erl(Calendar.time()) :: :calendar.time()
```

Converts given `time` to an Erlang time tuple.

WARNING: Loss of precision may occur, as Erlang time tuples only contain hours/minutes/seconds.

Examples

```
iex> Time.to_erl(~T[23:30:15.999])
{23, 30, 15}

iex> Time.to_erl(~N[2010-04-17 23:30:15.999])
{23, 30, 15}
```

to_iso8601(time, format \\ :extended)

```
@spec to_iso8601(Calendar.time(), :extended | :basic) ::
String.t()
```

Converts the given time to ISO 8601:2019.

By default, `Time.to_iso8601/2` returns times formatted in the "extended" format, for human readability. It also supports the "basic" format through passing the `:basic` option.

Examples

```
iex> Time.to_iso8601(~T[23:00:13])
"23:00:13"

iex> Time.to_iso8601(~T[23:00:13.001])
"23:00:13.001"

iex> Time.to_iso8601(~T[23:00:13.001], :basic)
"230013.001"

iex> Time.to_iso8601(~N[2010-04-17 23:00:13])
"23:00:13"
```

to_seconds_after_midnight(time)

(since 1.11.0)

```
@spec to_seconds_after_midnight(Calendar.time()) ::
{integer(), non_neg_integer()}
```

Converts a `Time` struct to a number of seconds after midnight.

The returned value is a two-element tuple with the number of seconds and microseconds.

Examples

```
iex> Time.to_seconds_after_midnight(~T[23:30:15])
{84615, 0}
iex> Time.to_seconds_after_midnight(~N[2010-04-17
23:30:15.999])
{84615, 999000}
```

to_string(time)

```
@spec to_string(Calendar.time()) :: String.t()
```

Converts the given `time` to a string.

Examples

```
iex> Time.to_string(~T[23:00:00])
"23:00:00"
iex> Time.to_string(~T[23:00:00.001])
"23:00:00.001"
iex> Time.to_string(~T[23:00:00.123456])
"23:00:00.123456"

iex> Time.to_string(~N[2015-01-01 23:00:00.001])
"23:00:00.001"
iex> Time.to_string(~N[2015-01-01 23:00:00.123456])
"23:00:00.123456"
```

truncate(time, precision)

(since 1.6.0)

```
@spec truncate(t(), :microsecond | :millisecond | :second) ::
t()
```

Returns the given time with the microsecond field truncated to the given precision (`:microsecond` , `millisecond` or `:second`).

The given time is returned unchanged if it already has lower precision than the given precision.

Examples

```
iex> Time.truncate(~T[01:01:01.123456], :microsecond)
~T[01:01:01.123456]

iex> Time.truncate(~T[01:01:01.123456], :millisecond)
~T[01:01:01.123]

iex> Time.truncate(~T[01:01:01.123456], :second)
~T[01:01:01]
```

utc_now(calendar_or_time_unit \\ Calendar.ISO)

(since 1.4.0)

```
@spec utc_now(Calendar.calendar() | :native | :microsecond |
:millisecond | :second) ::
  t()
```

Returns the current time in UTC.

You can pass a time unit to automatically truncate the resulting time.

The default unit if none gets passed is `:native` which results on a default resolution of microseconds.

Examples

```
iex> time = Time.utc_now()
iex> time.hour >= 0
true

iex> time = Time.utc_now(:second)
iex> time.microsecond
{0, 0}
```

utc_now(time_unit, calendar)

(since 1.19.0)

```
@spec utc_now(:native | :microsecond | :millisecond |
:second, Calendar.calendar()) ::
  t()
```

Returns the current time in UTC, supporting a precision and a specific calendar.

Examples

```
iex> time = Time.utc_now(:microsecond, Calendar.ISO)
iex> time.hour >= 0
true

iex> time = Time.utc_now(:second, Calendar.ISO)
iex> time.microsecond
{0, 0}
```

Tuple

Functions for working with tuples.

Please note the following functions for tuples are found in `Kernel`:

- `elem/2` - accesses a tuple by index
- `put_elem/3` - inserts a value into a tuple by index
- `tuple_size/1` - gets the number of elements in a tuple

Tuples are intended as fixed-size containers for multiple elements. To manipulate a collection of elements, use a list instead. `Enum` functions do not work on tuples.

Tuples are denoted with curly braces:

```
iex> {}
{}
iex> {1, :two, "three"}
{1, :two, "three"}
```

A tuple may contain elements of different types, which are stored contiguously in memory. Accessing any element takes constant time, but modifying a tuple, which produces a shallow copy, takes linear time. Tuples are good for reading data while lists are better for traversals.

Tuples are typically used either when a function has multiple return values or for error handling. `File.read/1` returns `{:ok, contents}` if reading the given file is successful, or else `{:error, reason}` such as when the file does not exist.

The functions in this module that add and remove elements from tuples are rarely used in practice, as they typically imply tuples are being used as collections. To append to a tuple, it is preferable to extract the elements from the old tuple with pattern matching, and then create a new tuple:

```
tuple = {:ok, :example}

# Avoid
result = Tuple.insert_at(tuple, 2, %{})

# Prefer
{:ok, atom} = tuple
result = {:ok, atom, %{}}
```

Summary

Functions

delete_at(tuple, index)

Removes an element from a tuple.

duplicate(data, size)

Creates a new tuple.

insert_at(tuple, index, value)

Inserts an element into a tuple.

product(tuple)

Computes a product of tuple elements.

sum(tuple)

Computes a sum of tuple elements.

to_list(tuple)

Converts a tuple to a list.

Functions

delete_at(tuple, index)

```
@spec delete_at(tuple(), non_neg_integer()) :: tuple()
```

Removes an element from a tuple.

Deletes the element at the given `index` from `tuple`. Raises an `ArgumentError` if `index` is negative or greater than or equal to the length of `tuple`. Index is zero-based.

Inlined by the compiler.

Examples

```
iex> tuple = {:foo, :bar, :baz}
iex> Tuple.delete_at(tuple, 0)
{:bar, :baz}
```

duplicate(data, size)

```
@spec duplicate(term(), non_neg_integer()) :: tuple()
```

Creates a new tuple.

Creates a tuple of `size` containing the given `data` at every position.

Inlined by the compiler.

Examples

```
iex> Tuple.duplicate(:hello, 3)
{:hello, :hello, :hello}
```

insert_at(tuple, index, value)

```
@spec insert_at(tuple(), non_neg_integer(), term()) ::
tuple()
```

Inserts an element into a tuple.

Inserts `value` into `tuple` at the given `index`. Raises an `ArgumentError` if `index` is negative or greater than the length of `tuple`. Index is zero-based.

Inlined by the compiler.

Examples

```
iex> tuple = {:bar, :baz}
iex> Tuple.insert_at(tuple, 0, :foo)
{:foo, :bar, :baz}
```

```
iex> Tuple.insert_at(tuple, 2, :bong)
{:bar, :baz, :bong}
```

product(tuple)

(since 1.12.0)

```
@spec product(tuple()) :: number()
```

Computes a product of tuple elements.

Examples

```
iex> Tuple.product({255, 255})
65025
iex> Tuple.product({255, 1.0})
255.0
iex> Tuple.product({})
1
```

sum(tuple)

(since 1.12.0)

```
@spec sum(tuple()) :: number()
```

Computes a sum of tuple elements.

Examples

```
iex> Tuple.sum({255, 255})
510
iex> Tuple.sum({255, 0.0})
255.0
iex> Tuple.sum({})
0
```

to_list(tuple)

```
@spec to_list(tuple()) :: list()
```

Converts a tuple to a list.

Returns a new list with all the tuple elements.

Inlined by the compiler.

Examples

```
iex> tuple = {:foo, :bar, :baz}
iex> Tuple.to_list(tuple)
[:foo, :bar, :baz]
```

URI

Utilities for working with URIs.

This module provides functions for working with URIs (for example, parsing URIs or encoding query strings). The functions in this module are implemented according to RFC 3986 and it also provides additional functionality for handling "application/x-www-form-urlencoded" segments.

Additionally, the Erlang `:uri_string` module provides additional functionality such as RFC 3986 compliant URI normalization.

Summary

Types

authority() deprecated
t()

Functions

%URI{}

The URI struct.

append_path(uri, path)

Appends `path` to the given `uri`.

append_query(uri, query)

Appends `query` to the given `uri`.

char_reserved?(character)

Checks if `character` is a reserved one in a URI.

char_unescaped?(character)

Checks if `character` is allowed unescaped in a URI.

char_unreserved?(character)

Checks if `character` is an unreserved one in a URI.

decode(uri)

Percent-unescapes a URI.

decode_query(query, map \\ %{}, encoding \\ :www_form)

Decodes `query` into a map.

decode_www_form(string)

Decodes `string` as "x-www-form-urlencoded".

default_port(scheme)

Returns the default port for a given `scheme`.

default_port(scheme, port)

Registers the default `port` for the given `scheme`.

encode(string, predicate \\ &char_unescaped?/1)

Percent-encodes all characters that require escaping in `string`.

encode_query(enumerable, encoding \\ :www_form)

Encodes `enumerable` into a query string using `encoding`.

encode_www_form(string)

Encodes `string` as "x-www-form-urlencoded".

merge(uri, rel)

Merges two URIs.

new(uri)

Creates a new URI struct by parsing and validating a string or from an existing URI.

new!(uri)

Similar to `new/1` but raises `URI.Error` if an invalid string is given.

parse(uri)

Parses a URI into its components, without further validation.

query_decoder(query, encoding \\ :www_form)

Returns a stream of two-element tuples representing key-value pairs in the given `query`.

to_string(uri)

Returns the string representation of the given URI struct.

Types

authority()

This type is deprecated. The authority field is deprecated.

```
@type authority() :: nil | binary()
```

t()

```
@type t() :: %URI{
  authority: authority(),
  fragment: nil | binary(),
  host: nil | binary(),
  path: nil | binary(),
  port: nil | :inet.port_number(),
  query: nil | binary(),
  scheme: nil | binary(),
  userinfo: nil | binary()
}
```

Functions

%URI{}

(struct)

The URI struct.

The fields are defined to match the following URI representation (with field names between brackets):

```
[scheme]://[userinfo]@[host]:[port][path]?[query]#
[fragment]
```

The fields contain the encoded URI components as they appear in the URI itself. For example, a slash inside the userinfo must be stored as `%2F`, not as `/`. Functions such as `parse/1` and `new/1` preserve existing percent-encoded sequences in those fields, and functions such as `to_string/1` expects those fields to already be encoded as needed. Whenever setting or modifying the fields directly, you must encode them accordingly.

Note the `authority` field is deprecated. `parse/1` will still populate it for backwards compatibility but you should generally avoid setting or getting it.

append_path(uri, path)

(since 1.15.0)

```
@spec append_path(t(), String.t()) :: t()
```

Appends `path` to the given `uri`.

Path must start with `/` and cannot contain additional URL components like fragments or query strings. This function further assumes the path is valid and it does not contain a query string or fragment parts.

Examples

```
iex> URI.append_path(URI.parse("http://example.com/foo/?
x=1"), "/my-path") |> URI.to_string()
"http://example.com/foo/my-path?x=1"
```

```
iex> URI.append_path(URI.parse("http://example.com"),
"my-path")
** (ArgumentError) path must start with "/", got: "my-
path"
```

append_query(uri, query)

(since 1.14.0)

```
@spec append_query(t(), binary()) :: t()
```

Appends `query` to the given `uri`.

The given `query` is not automatically encoded, use `encode/2` or `encode_www_form/1`.

Examples

```
iex> URI.append_query(URI.parse("http://example.com/"),
"x=1") |> URI.to_string()
"http://example.com/?x=1"

iex> URI.append_query(URI.parse("http://example.com/?
x=1"), "y=2") |> URI.to_string()
"http://example.com/?x=1&y=2"

iex> URI.append_query(URI.parse("http://example.com/?
x=1"), "x=2") |> URI.to_string()
"http://example.com/?x=1&x=2"
```

char_reserved?(character)

```
@spec char_reserved?(byte()) :: boolean()
```

Checks if `character` is a reserved one in a URI.

As specified in RFC 3986, section 2.2, the following characters are reserved: `:`, `/`, `?`, `#`, `[`, `]`, `@`, `!`, `$`, `&`, `'`, `(`, `)`, `*`, `+`, `,`, `;`, `=`

Examples

```
iex> URI.char_reserved?(?+)
true
```

char_unescaped?(character)

```
@spec char_unescaped?(byte()) :: boolean()
```

Checks if `character` is allowed unescaped in a URI.

This is the default used by `URI.encode/2` where both reserved and unreserved characters are kept unescaped.

Examples

```
iex> URI.char_unescaped?(?{)
false
```

char_unreserved?(character)

```
@spec char_unreserved?(byte()) :: boolean()
```

Checks if `character` is an unreserved one in a URI.

As specified in RFC 3986, section 2.3, the following characters are unreserved:

- Alphanumeric characters: `A-Z, a-z, 0-9`
- `~, _, -, .`

Examples

```
iex> URI.char_unreserved?(?_)
true
```

decode(uri)

```
@spec decode(binary()) :: binary()
```

Percent-unescapes a URI.

Examples

```
iex> URI.decode("https%3A%2F%2Felixir-lang.org")
"https://elixir-lang.org"
```

decode_query(query, map \\ %{}, encoding \\ :www_form)

```
@spec decode_query(binary(), %{optional(binary()) =>
binary()}, :rfc3986 | :www_form) ::
  %{
    optional(binary()) => binary()
  }
```

Decodes `query` into a map.

Given a query string in the form of `key1=value1&key2=value2...`, this function inserts each key-value pair in the query string as one entry in the given `map`. Keys and values in the resulting map will be binaries. Keys and values will be percent-unescaped.

You can specify one of the following `encoding` options:

- `:www_form` - (default, since v1.12.0) keys and values are decoded as per `decode_www_form/1`. This is the format typically used by browsers on query strings and form data. It decodes "+" as " ".
- `:rfc3986` - (since v1.12.0) keys and values are decoded as per `decode/1`. The result is the same as `:www_form` except for leaving "+" as is in line with RFC 3986.

Encoding defaults to `:www_form` for backward compatibility.

Use `query_decoder/1` if you want to iterate over each value manually.

Examples

```
iex> URI.decode_query("foo=1&bar=2")
%{"bar" => "2", "foo" => "1"}

iex> URI.decode_query("percent=oh+yes%21", %{"starting"
=> "map"})
```

```
%{"percent" => "oh yes!", "starting" => "map"}

iex> URI.decode_query("percent=oh+yes%21", %{}, :rfc3986)
%{"percent" => "oh+yes!"}
```

decode_www_form(string)

```
@spec decode_www_form(binary()) :: binary()
```

Decodes `string` as "x-www-form-urlencoded".

Note "x-www-form-urlencoded" is not specified as part of RFC 3986. However, it is a commonly used format to encode query strings and form data by browsers.

Examples

```
iex> URI.decode_www_form("%3Call+in%2F")
"<all in/"
```

default_port(scheme)

```
@spec default_port(binary()) :: nil | non_neg_integer()
```

Returns the default port for a given `scheme`.

If the scheme is unknown to the `URI` module, this function returns `nil`. The default port for any scheme can be configured globally via `default_port/2`.

Examples

```
iex> URI.default_port("ftp")
21

iex> URI.default_port("ponzi")
nil
```

default_port(scheme, port)

```
@spec default_port(binary(), non_neg_integer()) :: :ok
```

Registers the default `port` for the given `scheme`.

After this function is called, `port` will be returned by `default_port/1` for the given scheme `scheme`. Note that this function changes the default port for the given `scheme` *globally*, meaning for every application.

It is recommended for this function to be invoked in your application's start callback in case you want to register new URIs.

encode(string, predicate \\ &char_unescaped?/1)

```
@spec encode(binary(), (byte() -> as_boolean(term()))) ::
binary()
```

Percent-encodes all characters that require escaping in `string`.

The optional `predicate` argument specifies a function used to detect whether a byte in the `string` should be escaped:

- if the function returns a truthy value, the byte should be kept as-is.
- if the function returns a falsy value, the byte should be escaped.

The `predicate` argument can use some built-in functions:

- `URI.char_unescaped?/1` (default) - reserved characters (such as `:` and `/`) or unreserved (such as letters and numbers) are kept as-is. It's typically used to encode the whole URI.
- `URI.char_unreserved?/1` - unreserved characters (such as letters and numbers) are kept as-is. It's typically used to encode components in a URI, such as query or fragment.
- `URI.char_reserved?/1` - Reserved characters (such as `:` and `/`) are kept as-is.

And, you can also use custom functions.

See `encode_www_form/1` if you are interested in encoding `string` as "x-www-form-urlencoded".

Examples

```
iex> URI.encode("ftp://s-ite.tld/?value=put it+й")
"ftp://s-ite.tld/?value=put%20it+%D0%B9"

iex> URI.encode("a string", &(&1 != ?i))
"a str%69ng"
```

encode_query(enumerable, encoding \\ :www_form)

```
@spec encode_query(Enumerable.t(), :rfc3986 | :www_form) ::
binary()
```

Encodes `enumerable` into a query string using `encoding`.

Takes an enumerable that enumerates as a list of two-element tuples (for instance, a map or a keyword list) and returns a string in the form of `key1=value1&key2=value2...`.

Keys and values can be any term that implements the `String.Chars` protocol with the exception of lists, which are explicitly forbidden.

You can specify one of the following `encoding` strategies:

- `:www_form` - (default, since v1.12.0) keys and values are URL encoded as per `encode_www_form/1`. This is the format typically used by browsers on query strings and form data. It encodes " " as "+".

- `:rfc3986` - (since v1.12.0) the same as `:www_form` except it encodes " " as "%20" according RFC 3986. This is the best option if you are encoding in a non-browser situation, since encoding spaces as "+" can be ambiguous to URI parsers. This can inadvertently lead to spaces being interpreted as literal plus signs.

Encoding defaults to `:www_form` for backward compatibility.

Examples

```
iex> query = %{"foo" => 1, "bar" => 2}
iex> URI.encode_query(query)
"bar=2&foo=1"

iex> query = %{"key" => "value with spaces"}
iex> URI.encode_query(query)
"key=value+with+spaces"

iex> query = %{"key" => "value with spaces"}
iex> URI.encode_query(query, :rfc3986)
```

```
"key=value%20with%20spaces"

iex> URI.encode_query(%{key: [:a, :list]})
** (ArgumentError) encode_query/2 values cannot be lists,
got: [:a, :list]
```

encode_www_form(string)

```
@spec encode_www_form(binary()) :: binary()
```

Encodes `string` as "x-www-form-urlencoded".

Note "x-www-form-urlencoded" is not specified as part of RFC 3986. However, it is a commonly used format to encode query strings and form data by browsers.

Example

```
iex> URI.encode_www_form("put: it+й")
"put%3A+it%2B%D0%B9"
```

merge(uri, rel)

```
@spec merge(t() | binary(), t() | binary()) :: t()
```

Merges two URIs.

This function merges two URIs as per RFC 3986, section 5.2.

Examples

```
iex> URI.merge(URI.parse("http://google.com"), "/query")
|> to_string()
"http://google.com/query"

iex> URI.merge("http://example.com", "http://google.com")
|> to_string()
"http://google.com"
```

new(uri)

(since 1.13.0)

```
@spec new(t() | String.t()) :: {:ok, t()} | {:error,
String.t()}
```

Creates a new URI struct by parsing and validating a string or from an existing URI.

If a `%URI{}` struct is given, it returns `{:ok, uri}` as is. If a string is given, it will parse and validate it. If the string is valid, it returns `{:ok, uri}`, otherwise it returns `{:error, part}` with the invalid part of the URI. For parsing URIs without further validation, see `parse/1`.

This function can parse both absolute and relative URLs. You can check if a URI is absolute or relative by checking if the `scheme` field is `nil` or not.

When a URI is given without a port, the value returned by `URI.default_port/1` for the URI's scheme is used for the `:port` field. The scheme is also normalized to lowercase.

> **Browser compatibility**
>
> This function does not follow the same parsing rules as browsers. Browsers adhere to the WHATWG URL standard, while this function implements RFC 3986. You should expect different behaviours between them and this function will often reject URLs that are accepted by browsers (as they tend to be permissive). Common examples include unencoded square brackets in query strings (such as `foo[bar]=baz`) and backslashes used as path separators.

Examples

```
iex> URI.new("https://elixir-lang.org/")
{:ok, %URI{
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: 443,
  query: nil,
  scheme: "https",
  userinfo: nil
}}

iex> URI.new("//elixir-lang.org/")
{:ok, %URI{
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}}

iex> URI.new("/foo/bar")
```

```
{:ok, %URI{
  fragment: nil,
  host: nil,
  path: "/foo/bar",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}}

iex> URI.new("foo/bar")
{:ok, %URI{
  fragment: nil,
  host: nil,
  path: "foo/bar",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}}

iex> URI.new("//[fe80::]/")
{:ok, %URI{
  fragment: nil,
  host: "fe80::",
  path: "/",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}}

iex> URI.new("https:?query")
{:ok, %URI{
  fragment: nil,
  host: nil,
```

```
  path: nil,
  port: 443,
  query: "query",
  scheme: "https",
  userinfo: nil
}}

iex> URI.new("/invalid_greater_than_in_path/>")
{:error, ">"}
```

Giving an existing URI simply returns it wrapped in a tuple:

```
iex> {:ok, uri} = URI.new("https://elixir-lang.org/")
iex> URI.new(uri)
{:ok, %URI{
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: 443,
  query: nil,
  scheme: "https",
  userinfo: nil
}}
```

new!(uri)

(since 1.13.0)

```
@spec new!(t() | String.t()) :: t()
```

Similar to `new/1` but raises `URI.Error` if an invalid string is given.

Examples

```
iex> URI.new!("https://elixir-lang.org/")
%URI{
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: 443,
  query: nil,
  scheme: "https",
  userinfo: nil
}

iex> URI.new!("/invalid_greater_than_in_path/>")
** (URI.Error) cannot parse due to reason invalid_uri:
">"
```

Giving an existing URI simply returns it:

```
iex> uri = URI.new!("https://elixir-lang.org/")
iex> URI.new!(uri)
%URI{
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: 443,
  query: nil,
  scheme: "https",
  userinfo: nil
}
```

parse(uri)

```
@spec parse(t() | binary()) :: t()
```

Parses a URI into its components, without further validation.

This function can parse both absolute and relative URLs. You can check if a URI is absolute or relative by checking if the `scheme` field is nil or not.

This function expects both absolute and relative URIs to be well-formed and does not perform any validation, it simply breaks the URL into parts. Use `new/1` if you want to validate the URI fields after parsing.

When a URI is given without a port, the value returned by `URI.default_port/1` for the URI's scheme is used for the `:port` field. The scheme is also normalized to lowercase.

If a `URI` struct is given to this function, this function returns it unmodified.

> **Browser compatibility**
>
> This function does not follow the same parsing rules as browsers. Browsers adhere to the WHATWG URL standard, while this function implements RFC 3986. You should expect different behaviours between them, especially around corner cases.

> `:authority` **field**
>
> This function sets the deprecated field `:authority` for backwards-compatibility reasons.

Examples

```
iex> URI.parse("https://elixir-lang.org/")
%URI{
  authority: "elixir-lang.org",
  fragment: nil,
  host: "elixir-lang.org",
```

```
  path: "/",
  port: 443,
  query: nil,
  scheme: "https",
  userinfo: nil
}

iex> URI.parse("//elixir-lang.org/")
%URI{
  authority: "elixir-lang.org",
  fragment: nil,
  host: "elixir-lang.org",
  path: "/",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}

iex> URI.parse("/foo/bar")
%URI{
  fragment: nil,
  host: nil,
  path: "/foo/bar",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}

iex> URI.parse("foo/bar")
%URI{
  fragment: nil,
  host: nil,
  path: "foo/bar",
  port: nil,
```

```
  query: nil,
  scheme: nil,
  userinfo: nil
}
```

In contrast to `URI.new/1`, this function will accept poorly-formed URIs and break them into components, as it does not perform validation. For example:

```
iex> URI.parse("/invalid_greater_than_in_path/>")
%URI{
  fragment: nil,
  host: nil,
  path: "/invalid_greater_than_in_path/>",
  port: nil,
  query: nil,
  scheme: nil,
  userinfo: nil
}
```

Another example is a URI with brackets in query strings. It is accepted by `parse/1` for the same reasons as above (no validation), but it will be refused by `new/1`:

```
iex> URI.parse("/?foo[bar]=baz")
%URI{
  fragment: nil,
  host: nil,
  path: "/",
  port: nil,
  query: "foo[bar]=baz",
  scheme: nil,
```

```
  userinfo: nil
}
```

Generally speaking, square brackets are strictly invalid even in the WHATWG URL standard used by browsers, but browsers tend to be permissive and often allow invalid characters to pass through.

query_decoder(query, encoding \\ :www_form)

```
@spec query_decoder(binary(), :rfc3986 | :www_form) ::
Enumerable.t()
```

Returns a stream of two-element tuples representing key-value pairs in the given `query`.

Key and value in each tuple will be binaries and will be percent-unescaped.

You can specify one of the following `encoding` options:

- `:www_form` - (default, since v1.12.0) keys and values are decoded as per `decode_www_form/1`. This is the format typically used by browsers on query strings and form data. It decodes "+" as " ".
- `:rfc3986` - (since v1.12.0) keys and values are decoded as per `decode/1`. The result is the same as `:www_form` except for leaving "+" as is in line with RFC 3986.

Encoding defaults to `:www_form` for backward compatibility.

Examples

```
iex> URI.query_decoder("foo=1&bar=2") |> Enum.to_list()
[{"foo", "1"}, {"bar", "2"}]

iex>
URI.query_decoder("food=bread%26butter&drinks=tap%20water
+please") |> Enum.to_list()
[{"food", "bread&butter"}, {"drinks", "tap water
please"}]

iex>
URI.query_decoder("food=bread%26butter&drinks=tap%20water
+please", :rfc3986) |> Enum.to_list()
[{"food", "bread&butter"}, {"drinks", "tap
water+please"}]
```

to_string(uri)

```
@spec to_string(t()) :: binary()
```

Returns the string representation of the given URI struct.

This function assembles the URI components into a string, assuming each field is valid and escaped as done by `parse/1` and `new/1`.

Examples

```
iex> uri = URI.parse("http://google.com")
iex> URI.to_string(uri)
"http://google.com"

iex> uri = URI.parse("foo://bar.baz")
```

```
iex> URI.to_string(uri)
"foo://bar.baz"
```

Version

Functions for parsing and matching versions against requirements.

A version is a string in a specific format or a `Version` generated after parsing via `Version.parse/1`.

Although Elixir projects are not required to follow SemVer, they must follow the format outlined on SemVer 2.0 schema.

Versions

In a nutshell, a version is represented by three numbers:

```
MAJOR.MINOR.PATCH
```

Each numeric component is limited to at most 14 digits.

Pre-releases are supported by optionally appending a hyphen and a series of period-separated identifiers immediately following the patch version. Identifiers consist of only ASCII alphanumeric characters and hyphens (`[0-9A-Za-z-]`):

```
"1.0.0-alpha.3"
```

Numeric pre-release identifiers are also limited to at most 14 digits.

Build information can be added by appending a plus sign and a series of dot-separated identifiers immediately following the patch or pre-release version. Identifiers consist of only ASCII alphanumeric characters and hyphens (`[0-9A-Za-z-]`):

```
"1.0.0-alpha.3+20130417140000.amd64"
```

Requirements

Requirements allow you to specify which versions of a given dependency you are willing to work against. Requirements support the common comparison operators such as `>`, `>=`, `<`, `<=`, and `==` that work as one would expect, and additionally the special operator `~>` described in detail further below.

```
# Only version 2.0.0
"== 2.0.0"

# Anything later than 2.0.0
"> 2.0.0"
```

You can skip the operator, which is equivalent to `==`:

```
# Only version 2.0.0
"2.0.0"
```

Requirements also support `and` and `or` for complex conditions:

```
# 2.0.0 and later until 2.1.0
">= 2.0.0 and < 2.1.0"
```

Since the example above is such a common requirement, it can be expressed as:

```
"~> 2.0.0"
```

`~>` will never include pre-release versions of its upper bound, regardless of the usage of the `:allow_pre` option, or whether the operand is a pre-release version. It can also be used to set an upper bound on only the major version part. See the table below for `~>` requirements and their corresponding translations.

`~>`	Translation
`~> 2.0.0`	`>= 2.0.0 and < 2.1.0`
`~> 2.1.2`	`>= 2.1.2 and < 2.2.0`
`~> 2.1.3-dev`	`>= 2.1.3-dev and < 2.2.0`
`~> 2.0`	`>= 2.0.0 and < 3.0.0`
`~> 2.1`	`>= 2.1.0 and < 3.0.0`

The requirement operand after the `~>` is allowed to omit the patch version, allowing us to express `~> 2.1` or `~> 2.1-dev`, something that wouldn't be allowed when using the common comparison operators.

When the `:allow_pre` option is set `false` in `Version.match?/3`, the requirement will not match a pre-release version unless the operand is a pre-release version. The default is to always allow pre-releases but note that in Hex `:allow_pre` is set to `false`. See the table below for examples.

Requirement	Version	`:allow_pre`	Matches
`~> 2.0`	`2.1.0`	`true` or `false`	`true`
`~> 2.0`	`3.0.0`	`true` or `false`	`false`
`~> 2.0.0`	`2.0.5`	`true` or `false`	`true`
`~> 2.0.0`	`2.1.0`	`true` or `false`	`false`
`~> 2.1.2`	`2.1.6-dev`	`true`	`true`

Requirement	Version	:allow_pre	Matches
~> 2.1.2	2.1.6-dev	false	false
~> 2.1-dev	2.2.0-dev	true or false	true
~> 2.1.2-dev	2.1.6-dev	true or false	true
>= 2.1.0	2.2.0-dev	true	true
>= 2.1.0	2.2.0-dev	false	false
>= 2.1.0-dev	2.2.6-dev	true or false	true

Summary

Types

build()
major()
match_opts()
minor()
patch()
pre()
requirement()
t()
version()

Functions

%Version{}

The Version struct.

compare(version1, version2)

Compares two versions.

compile_requirement(requirement)

Compiles a requirement to an internal representation that may optimize matching.

match?(version, requirement, opts \\ [])

Checks if the given version matches the specification.

parse(string)

Parses a version string into a `Version` struct.

parse!(string)

Parses a version string into a `Version`.

parse_requirement(string)

Parses a version requirement string into a `Version.Requirement` struct.

parse_requirement!(string)

Parses a version requirement string into a `Version.Requirement` struct.

to_string(version)

Converts the given version to a string.

Types

build()

```
@type build() :: String.t() | nil
```

major()

```
@type major() :: non_neg_integer()
```

match_opts()

```
@type match_opts() :: [{:allow_pre, boolean()}]
```

minor()

```
@type minor() :: non_neg_integer()
```

patch()

```
@type patch() :: non_neg_integer()
```

pre()

```
@type pre() :: [String.t() | non_neg_integer()]
```

requirement()

```
@type requirement() :: String.t() | Version.Requirement.t()
```

t()

```
@type t() :: %Version{
  build: build(),
  major: major(),
  minor: minor(),
  patch: patch(),
  pre: pre()
}
```

version()

```
@type version() :: String.t() | t()
```

Functions

%Version{}

(struct)

The Version struct.

It contains the fields `:major`, `:minor`, `:patch`, `:pre`, and `:build` according to SemVer 2.0, where `:pre` is a list.

You can read those fields but you should not create a new `Version` directly via the struct syntax. Instead use the functions in this module.

compare(version1, version2)

```
@spec compare(version(), version()) :: :gt | :eq | :lt
```

Compares two versions.

Returns `:gt` if the first version is greater than the second one, and `:lt` for vice versa. If the two versions are equal, `:eq` is returned.

Pre-releases are strictly less than their corresponding release versions.

Patch segments are compared lexicographically if they are alphanumeric, and numerically otherwise.

Build segments are ignored: if two versions differ only in their build segment they are considered to be equal.

Raises a `Version.InvalidVersionError` exception if any of the two given versions are not parsable. If given an already parsed version this function won't raise.

Examples

```
iex> Version.compare("2.0.1-alpha1", "2.0.0")
:gt

iex> Version.compare("1.0.0-beta", "1.0.0-rc1")
:lt

iex> Version.compare("1.0.0-10", "1.0.0-2")
:gt

iex> Version.compare("2.0.1+build0", "2.0.1")
:eq
```

```
iex> Version.compare("invalid", "2.0.1")
** (Version.InvalidVersionError) invalid version:
"invalid"
```

compile_requirement(requirement)

```
@spec compile_requirement(Version.Requirement.t()) ::
Version.Requirement.t()
```

Compiles a requirement to an internal representation that may optimize matching.

The internal representation is opaque.

match?(version, requirement, opts \\ [])

```
@spec match?(version(), requirement(), match opts()) ::
boolean()
```

Checks if the given version matches the specification.

Returns `true` if `version` satisfies `requirement`, `false` otherwise. Raises a `Version.InvalidRequirementError` exception if `requirement` is not parsable, or a `Version.InvalidVersionError` exception if `version` is not parsable. If given an already parsed version and requirement this function won't raise.

Options

- `:allow_pre` (boolean) - when `false`, pre-release versions will not match unless the operand is a pre-release version. Defaults to `true`. For examples, please refer to the table above under the "Requirements" section.

Examples

```
iex> Version.match?("2.0.0", "> 1.0.0")
true

iex> Version.match?("2.0.0", "== 1.0.0")
false

iex> Version.match?("2.1.6-dev", "~> 2.1.2")
true

iex> Version.match?("2.1.6-dev", "~> 2.1.2", allow_pre:
false)
false

iex> Version.match?("foo", "== 1.0.0")
** (Version.InvalidVersionError) invalid version: "foo"

iex> Version.match?("2.0.0", "== == 1.0.0")
** (Version.InvalidRequirementError) invalid requirement:
"== == 1.0.0"
```

parse(string)

```
@spec parse(String.t()) :: {:ok, t()} | :error
```

Parses a version string into a `Version` struct.

Examples

```
iex> Version.parse("2.0.1-alpha1")
{:ok, %Version{major: 2, minor: 0, patch: 1, pre:
["alpha1"]}}

iex> Version.parse("2.0-alpha1")
:error
```

parse!(string)

```
@spec parse!(String.t()) :: t()
```

Parses a version string into a `Version`.

If `string` is an invalid version, a `Version.InvalidVersionError` is raised.

Examples

```
iex> Version.parse!("2.0.1-alpha1")
%Version{major: 2, minor: 0, patch: 1, pre: ["alpha1"]}

iex> Version.parse!("2.0-alpha1")
** (Version.InvalidVersionError) invalid version: "2.0-
alpha1"
```

parse_requirement(string)

```
@spec parse_requirement(String.t()) :: {:ok,
Version.Requirement.t()} | :error
```

Parses a version requirement string into a `Version.Requirement` struct.

Examples

```
iex> {:ok, requirement} = Version.parse_requirement("==
2.0.1")
iex> requirement
Version.parse_requirement!("== 2.0.1")

iex> Version.parse_requirement("== == 2.0.1")
:error
```

parse_requirement!(string)

(since 1.8.0)

```
@spec parse_requirement!(String.t()) ::
Version.Requirement.t()
```

Parses a version requirement string into a `Version.Requirement` struct.

If `string` is an invalid requirement, a `Version.InvalidRequirementError` is raised.

Examples

```
iex> Version.parse_requirement!("== 2.0.1")
Version.parse_requirement!("== 2.0.1")

iex> Version.parse_requirement!("== == 2.0.1")
```

```
** (Version.InvalidRequirementError) invalid requirement:
"== == 2.0.1"
```

to_string(version)

(since 1.14.0)

```
@spec to_string(t()) :: String.t()
```

Converts the given version to a string.

Examples

```
iex> Version.to_string(%Version{major: 1, minor: 2,
patch: 3})
"1.2.3"
iex> Version.to_string(Version.parse!("1.14.0-
rc.0+build0"))
"1.14.0-rc.0+build0"
```

Version.Requirement

A struct that holds version requirement information.

The struct fields are private and should not be accessed.

See the "Requirements" section in the `Version` module for more information.

Summary

Types

t()

Types

t()

```
@opaque t()
```

Access behaviour

Key-based access to data structures.

The `Access` module defines a behaviour for dynamically accessing keys of any type in a data structure via the `data[key]` syntax.

`Access` supports keyword lists (`Keyword`) and maps (`Map`) out of the box. Keywords supports only atoms keys, keys for maps can be of any type. Both return `nil` if the key does not exist:

```
iex> keywords = [a: 1, b: 2]
iex> keywords[:a]
1
iex> keywords[:c]
nil

iex> map = %{a: 1, b: 2}
iex> map[:a]
1

iex> star_ratings = %{1.0 => "★", 1.5 => "★☆", 2.0 =>
"★★"}
iex> star_ratings[1.5]
"★☆"
```

This syntax is very convenient as it can be nested arbitrarily:

```
iex> keywords = [a: 1, b: 2]
iex> keywords[:c][:unknown]
nil
```

This works because accessing anything on a `nil` value, returns `nil` itself:

```
iex> nil[:a]
nil
```

Maps and structs

While the access syntax is allowed in maps via `map[key]`, if your map is made of predefined atom keys, you should prefer to access those atom keys with `map.key` instead of `map[key]`, as `map.key` will raise if the key is missing (which is not supposed to happen if the keys are predefined) or if `map` is `nil`.

Similarly, since structs are maps and structs have predefined keys, they only allow the `struct.key` syntax and they do not allow the `struct[key]` access syntax.

In other words, the `map[key]` syntax is loose, returning `nil` for missing keys, while the `map.key` syntax is strict, raising for both nil values and missing keys.

To bridge this gap, Elixir provides the `get_in/1` and `get_in/2` functions, which are capable of traversing nested data structures, even in the presence of `nil`s:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_in(users["john"].age)
27
iex> get_in(users["unknown"].age)
nil
```

Notice how, even if no user was found, `get_in/1` returned `nil`. Outside of `get_in/1`, trying to access the field `.age` on `nil` would raise.

The `get_in/2` function takes one step further by allowing different accessors to be mixed in. For example, given a user map with the `:name` and `:languages` keys, here is how to access the name of all programming languages:

```
iex> languages = [
...>   %{name: "elixir", type: :functional},
...>   %{name: "c", type: :procedural}
...> ]
iex> user = %{name: "john", languages: languages}
iex> get_in(user, [:languages, Access.all(), :name])
["elixir", "c"]
```

This module provides convenience functions for traversing other structures, like tuples and lists. As we will see next, they can even be used to update nested data structures.

If you want to learn more about the dual nature of maps in Elixir, as they can be either for structured data or as a key-value store, see the `Map` module.

Updating nested data structures

The access syntax can also be used with the `Kernel.put_in/2`, `Kernel.update_in/2`, `Kernel.get_and_update_in/2`, and `Kernel.pop_in/1` macros to further manipulate values in nested data structures:

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> put_in(users["john"].age, 28)
```

```
%{"john" => %{age: 28}, "meg" => %{age: 23}}
```

As shown in the previous section, you can also use the `Kernel.put_in/3`, `Kernel.update_in/3`, `Kernel.pop_in/2`, and `Kernel.get_and_update_in/3` functions to provide nested custom accessors. For instance, given a user map with the `:name` and `:languages` keys, here is how to deeply traverse the map and convert all language names to uppercase:

```
iex> languages = [
...>   %{name: "elixir", type: :functional},
...>   %{name: "c", type: :procedural}
...> ]
iex> user = %{name: "john", languages: languages}
iex> update_in(user, [:languages, Access.all(), :name],
&String.upcase/1)
%{
  name: "john",
  languages: [
    %{name: "ELIXIR", type: :functional},
    %{name: "C", type: :procedural}
  ]
}
```

See the functions `key/1`, `key!/1`, `elem/1`, and `all/0` for some of the available accessors.

Summary

Types

access_fun(data, current_value)
container()
get_and_update_fun(data, current_value)
get_fun(data)
key()
nil_container()
t()
value()

Callbacks

fetch(term, key)

Invoked in order to access the value stored under `key` in the given term `term`.

get_and_update(data, key, function)

Invoked in order to access the value under `key` and update it at the same time.

pop(data, key)

Invoked to "pop" the value under `key` out of the given data structure.

Functions

all()

Returns a function that accesses all the elements in a list.

at(index)

Returns a function that accesses the element at `index` (zero based) of a list.

at!(index)

Same as `at/1` except that it raises `Enum.OutOfBoundsError` if the given index is out of bounds.

elem(index)

Returns a function that accesses the element at the given index in a tuple.

fetch(container, key)

Fetches the value for the given key in a container (a map, keyword list, or struct that implements the `Access` behaviour).

fetch!(container, key)

Same as `fetch/2` but returns the value directly, or raises a `KeyError` exception if `key` is not found.

filter(func)

Returns a function that accesses all elements of a list that match the provided predicate.

find(predicate)

Returns a function that accesses the first element of a list that matches the provided predicate.

get(container, key, default \\ nil)

Gets the value for the given key in a container (a map, keyword list, or struct that implements the `Access` behaviour).

get_and_update(container, key, fun)

Gets and updates the given key in a `container` (a map, a keyword list, a struct that implements the `Access` behaviour).

key(key, default \\ nil)

Returns a function that accesses the given key in a map/struct.
key!(key)

Returns a function that accesses the given key in a map/struct.

pop(container, key)

Removes the entry with a given key from a container (a map, keyword list, or struct that implements the `Access` behaviour).

slice(range)

Returns a function that accesses all items of a list that are within the provided range.

values()

Returns a function that accesses all values in a map or a keyword list.

Types

access_fun(data, current_value)

```
@type access_fun(data, current_value) ::
  get_fun(data) | get_and_update_fun(data, current_value)
```

container()

```
@type container() :: keyword() | struct() | map()
```

get_and_update_fun(data, current_value)

```
@type get_and_update_fun(data, current_value) ::
(:get_and_update,
                                                data,
                                                (term() ->
term()) ->
                                                  {current_value,
                                                   new_data ::
container()}
                                                  | :pop)
```

get_fun(data)

```
@type get_fun(data) :: (:get, data, (term() -> term()) ->
new_data :: container())
```

key()

```
@type key() :: any()
```

nil_container()

```
@type nil_container() :: nil
```

t()

```
@type t() :: container() | nil container() | any()
```

value()

```
@type value() :: any()
```

Callbacks

fetch(term, key)

```
@callback fetch(term :: t(), key()) :: {:ok, value()} |
:error
```

Invoked in order to access the value stored under `key` in the given term `term`.

This function should return `{:ok, value}` where `value` is the value under `key` if the key exists in the term, or `:error` if the key does not exist in the term.

Many of the functions defined in the `Access` module internally call this function. This function is also used when the square-brackets access syntax (`structure[key]`) is used: the `fetch/2` callback implemented by the module that defines the `structure` struct is invoked and if it returns `{:ok, value}` then `value` is returned, or if it returns `:error` then `nil` is returned.

See the `Map.fetch/2` and `Keyword.fetch/2` implementations for examples of how to implement this callback.

get_and_update(data, key, function)

```
@callback get_and_update(data, key(), (value() | nil ->
                                 {current_value, new_value ::
value()} | :pop)) ::
  {current_value, new_data :: data}
when current_value: value(), data: container()
```

Invoked in order to access the value under `key` and update it at the same time.

The implementation of this callback should invoke `fun` with the value under `key` in the passed structure `data`, or with `nil` if `key` is not present in it. This function must return either `{current_value, new_value}` or `:pop`.

If the passed function returns `{current_value, new_value}`, the return value of this callback should be `{current_value, new_data}`, where:

- `current_value` is the retrieved value (which can be operated on before being returned)
- `new_value` is the new value to be stored under `key`
- `new_data` is `data` after updating the value of `key` with `new_value`.

If the passed function returns `:pop`, the return value of this callback must be `{value, new_data}` where `value` is the value under `key` (or `nil` if not present) and `new_data` is `data` without `key`.

See the implementations of `Map.get_and_update/3` or `Keyword.get_and_update/3` for more examples.

pop(data, key)

```
@callback pop(data, key()) :: {value(), data} when data: container()
```

Invoked to "pop" the value under `key` out of the given data structure.

When `key` exists in the given structure `data`, the implementation should return a `{value, new_data}` tuple where `value` is the value that was under `key` and `new_data` is `term` without `key`.

When `key` is not present in the given structure, a tuple `{value, data}` should be returned, where `value` is implementation-defined.

See the implementations for `Map.pop/3` or `Keyword.pop/3` for more examples.

Functions

all()

```
@spec all() :: access_fun(data :: list(), current_value :: list())
```

Returns a function that accesses all the elements in a list.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Examples

```
iex> list = [%{name: "john"}, %{name: "mary"}]
iex> get_in(list, [Access.all(), :name])
["john", "mary"]
iex> get_and_update_in(list, [Access.all(), :name], fn
prev ->
...>   {prev, String.upcase(prev)}
...> end)
{["john", "mary"], [%{name: "JOHN"}, %{name: "MARY"}]}
iex> pop_in(list, [Access.all(), :name])
{["john", "mary"], [%{}, %{}]}
```

Here is an example that traverses the list dropping even numbers and multiplying odd numbers by 2:

```
iex> require Integer
iex> get_and_update_in([1, 2, 3, 4, 5], [Access.all()],
fn num ->
...>   if Integer.is_even(num), do: :pop, else: {num, num
* 2}
...> end)
{[1, 2, 3, 4, 5], [2, 6, 10]}
```

An error is raised if the accessed structure is not a list:

```
iex> get_in(%{}, [Access.all()])
** (RuntimeError) Access.all/0 expected a list, got: %{}
```

at(index)

```
@spec at(integer()) :: access_fun(data :: list(),
current_value :: term())
```

Returns a function that accesses the element at `index` (zero based) of a list.

Keep in mind that index lookups in lists take linear time: the larger the list, the longer it will take to access its index. Therefore index-based operations are generally avoided in favor of other functions in the `Enum` module.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Examples

```
iex> list = [%{name: "john"}, %{name: "mary"}]
iex> get_in(list, [Access.at(1), :name])
"mary"
iex> get_in(list, [Access.at(-1), :name])
"mary"
iex> get_and_update_in(list, [Access.at(0), :name], fn
prev ->
...>   {prev, String.upcase(prev)}
...> end)
{"john", [%{name: "JOHN"}, %{name: "mary"}]}
iex> get_and_update_in(list, [Access.at(-1), :name], fn
prev ->
...>   {prev, String.upcase(prev)}
...> end)
{"mary", [%{name: "john"}, %{name: "MARY"}]}
```

`at/1` can also be used to pop elements out of a list or a key inside of a list:

```
iex> list = [%{name: "john"}, %{name: "mary"}]
iex> pop_in(list, [Access.at(0)])
{%{name: "john"}, [%{name: "mary"}]}
iex> pop_in(list, [Access.at(0), :name])
{"john", [%{}, %{name: "mary"}]}
```

When the index is out of bounds, `nil` is returned and the update function is never called:

```
iex> list = [%{name: "john"}, %{name: "mary"}]
iex> get_in(list, [Access.at(10), :name])
nil
iex> get_and_update_in(list, [Access.at(10), :name], fn
prev ->
...>   {prev, String.upcase(prev)}
...> end)
{nil, [%{name: "john"}, %{name: "mary"}]}
```

An error is raised if the accessed structure is not a list:

```
iex> get_in(%{}, [Access.at(1)])
** (RuntimeError) Access.at/1 expected a list, got: %{}
```

at!(index)

(since 1.11.0)

```
@spec at!(integer()) :: access_fun(data :: list(),
current_value :: term())
```

Same as `at/1` except that it raises `Enum.OutOfBoundsError` if the given index is out of bounds.

Examples

```
iex> get_in([:a, :b, :c], [Access.at!(2)])
:c
iex> get_in([:a, :b, :c], [Access.at!(3)])
** (Enum.OutOfBoundsError) out of bounds error at
position 3 when traversing enumerable [:a, :b, :c]
```

elem(index)

```
@spec elem(non_neg_integer()) :: access_fun(data :: tuple(),
current_value :: term())
```

Returns a function that accesses the element at the given index in a tuple.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

The returned function raises if `index` is out of bounds.

Note that popping elements out of tuples is not possible and raises an error.

Examples

```
iex> map = %{user: {"john", 27}}
iex> get_in(map, [:user, Access.elem(0)])
"john"
iex> get_and_update_in(map, [:user, Access.elem(0)], fn
prev ->
...>   {prev, String.upcase(prev)}
...> end)
```

```
{"john", %{user: {"JOHN", 27}}}
iex> pop_in(map, [:user, Access.elem(0)])
** (RuntimeError) cannot pop data from a tuple
```

An error is raised if the accessed structure is not a tuple:

```
iex> get_in(%{}, [Access.elem(0)])
** (RuntimeError) Access.elem/1 expected a tuple, got: %
{}
```

fetch(container, key)

```
@spec fetch(container(), term()) :: {:ok, term()} | :error
@spec fetch(nil container(), any()) :: :error
```

Fetches the value for the given key in a container (a map, keyword list, or struct that implements the `Access` behaviour).

Returns `{:ok, value}` where `value` is the value under `key` if there is such a key, or `:error` if `key` is not found.

Examples

```
iex> Access.fetch(%{name: "meg", age: 26}, :name)
{:ok, "meg"}

iex> Access.fetch([ordered: true, on_timeout: :exit],
:timeout)
:error
```

fetch!(container, key)

(since 1.10.0)

```
@spec fetch!(container(), term()) :: term()
```

Same as `fetch/2` but returns the value directly, or raises a `KeyError` exception if `key` is not found.

Examples

```
iex> Access.fetch!(%{name: "meg", age: 26}, :name)
"meg"
```

filter(func)

(since 1.6.0)

```
@spec filter((term() -> boolean())) ::
  access_fun(data :: list(), current_value :: list())
```

Returns a function that accesses all elements of a list that match the provided predicate.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Examples

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
```

```
iex> get_in(list, [Access.filter(&(&1.salary > 20)),
:name])
["francine"]
iex> get_and_update_in(list, [Access.filter(&(&1.salary
<= 20)), :name], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{["john"], [%{name: "JOHN", salary: 10}, %{name:
"francine", salary: 30}]}
```

`filter/1` can also be used to pop elements out of a list or a key inside of a list:

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
iex> pop_in(list, [Access.filter(&(&1.salary >= 20))])
{[%{name: "francine", salary: 30}], [%{name: "john",
salary: 10}]}
iex> pop_in(list, [Access.filter(&(&1.salary >= 20)),
:name])
{["francine"], [%{name: "john", salary: 10}, %{salary:
30}]}
```

When no match is found, an empty list is returned and the update function is never called

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
iex> get_in(list, [Access.filter(&(&1.salary >= 50)),
:name])
[]
iex> get_and_update_in(list, [Access.filter(&(&1.salary
>= 50)), :name], fn prev ->
```

```
...>   {prev, String.upcase(prev)}
...> end)
{[], [%{name: "john", salary: 10}, %{name: "francine",
salary: 30}]}
```

An error is raised if the accessed structure is not a list:

```
iex> get_in(%{}, [Access.filter(fn a -> a == 10 end)])
** (RuntimeError) Access.filter/1 expected a list, got: %
{}
```

find(predicate)

(since 1.17.0)

```
@spec find((term() -> as boolean(term()))) ::
  access fun(data :: list(), current_value :: term())
```

Returns a function that accesses the first element of a list that matches the provided predicate.

The returned function is typically passed as an accessor to Kernel.get_in/2, Kernel.get_and_update_in/3, and friends.

Examples

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
iex> get_in(list, [Access.find(&(&1.salary > 20)),
:name])
"francine"
iex> get_and_update_in(list, [Access.find(&(&1.salary <=
40)), :name], fn prev ->
```

```
...>   {prev, String.upcase(prev)}
...> end)
{"john", [%{name: "JOHN", salary: 10}, %{name:
"francine", salary: 30}]}
```

`find/1` can also be used to pop the first found element out of a list or a key inside of a list:

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
iex> pop_in(list, [Access.find(&(&1.salary <= 40))])
{%{name: "john", salary: 10}, [%{name: "francine",
salary: 30}]}
```

When no match is found, nil is returned and the update function is never called

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}]
iex> get_in(list, [Access.find(&(&1.salary >= 50)),
:name])
nil
iex> get_and_update_in(list, [Access.find(&(&1.salary >=
50)), :name], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{nil, [%{name: "john", salary: 10}, %{name: "francine",
salary: 30}]}
```

An error is raised if the accessed structure is not a list:

```
iex> get_in(%{}, [Access.find(fn a -> a == 10 end)])
** (RuntimeError) Access.find/1 expected a list, got: %{}
```

get(container, key, default \\ nil)

```
@spec get(container(), term(), term()) :: term()
@spec get(nil container(), any(), default) :: default when
default: var
```

Gets the value for the given key in a container (a map, keyword list, or struct that implements the `Access` behaviour).

Returns the value under `key` if there is such a key, or `default` if `key` is not found.

Examples

```
iex> Access.get(%{name: "john"}, :name, "default name")
"john"
iex> Access.get(%{name: "john"}, :age, 25)
25

iex> Access.get([ordered: true], :timeout)
nil
```

get_and_update(container, key, fun)

```
@spec get_and_update(data, key(), (value() | nil ->
                                   {current_value, new_value ::
```

```
value()} | :pop)) ::
  {current_value, new_data :: data}
when data: container(), current_value: var
```

Gets and updates the given key in a `container` (a map, a keyword list, a struct that implements the `Access` behaviour).

The `fun` argument receives the value of `key` (or `nil` if `key` is not present in `container`) and must return a two-element tuple `{current_value, new_value}`: the "get" value `current_value` (the retrieved value, which can be operated on before being returned) and the new value to be stored under `key` (`new_value`). `fun` may also return `:pop`, which means the current value should be removed from the container and returned.

The returned value is a two-element tuple with the "get" value returned by `fun` and a new container with the updated value under `key`.

Examples

```
iex> Access.get_and_update([a: 1], :a, fn current_value -
>
...>   {current_value, current_value + 1}
...> end)
{1, [a: 2]}
```

key(key, default \\ nil)

```
@spec key(key(), term()) ::
  access fun(data :: struct() | map(), current_value ::
term())
```

Returns a function that accesses the given key in a map/struct.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

The returned function uses the default value if the key does not exist. This can be used to specify defaults and safely traverse missing keys:

```
iex> get_in(%{}, [Access.key(:user, %{}),
Access.key(:name, "meg")])
"meg"
```

Such is also useful when using update functions, allowing us to introduce values as we traverse the data structure for updates:

```
iex> put_in(%{}, [Access.key(:user, %{}),
Access.key(:name)], "Mary")
%{user: %{name: "Mary"}}
```

Examples

```
iex> map = %{user: %{name: "john"}}
iex> get_in(map, [Access.key(:unknown, %{}),
Access.key(:name, "john")])
"john"
iex> get_and_update_in(map, [Access.key(:user),
Access.key(:name)], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{"john", %{user: %{name: "JOHN"}}}
iex> pop_in(map, [Access.key(:user), Access.key(:name)])
{"john", %{user: %{}}}
```

An error is raised if the accessed structure is not a map or a struct:

```
iex> get_in([], [Access.key(:foo)])
** (BadMapError) expected a map, got:
...
```

key!(key)

```
@spec key!(key()) :: access_fun(data :: struct() | map(),
current_value :: term())
```

Returns a function that accesses the given key in a map/struct.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Similar to `key/2`, but the returned function raises if the key does not exist.

Examples

```
iex> map = %{user: %{name: "john"}}
iex> get_in(map, [Access.key!(:user), Access.key!
(:name)])
"john"
iex> get_and_update_in(map, [Access.key!(:user),
Access.key!(:name)], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{"john", %{user: %{name: "JOHN"}}}
iex> pop_in(map, [Access.key!(:user), Access.key!
(:name)])
{"john", %{user: %{}}}
iex> get_in(map, [Access.key!(:user), Access.key!
```

```
(:unknown)])
** (KeyError) key :unknown not found in:
...
```

The examples above could be partially written as:

```
iex> map = %{user: %{name: "john"}}
iex> map.user.name
"john"
iex> get_and_update_in(map.user.name, fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{"john", %{user: %{name: "JOHN"}}}
```

However, it is not possible to remove fields using the dot notation, as it is implied those fields must also be present. In any case, `Access.key!/1` is useful when the key is not known in advance and must be accessed dynamically.

An error is raised if the accessed structure is not a map/struct:

```
iex> get_in([], [Access.key!(:foo)])
** (RuntimeError) Access.key!/1 expected a map/struct,
got: []
```

pop(container, key)

```
@spec pop(data, key()) :: {value(), data} when data:
container()
```

Removes the entry with a given key from a container (a map, keyword list, or struct that implements the `Access` behaviour).

Returns a tuple containing the value associated with the key and the updated container. `nil` is returned for the value if the key isn't in the container.

Examples

With a map:

```
iex> Access.pop(%{name: "Elixir", creator: "Valim"},
:name)
{"Elixir", %{creator: "Valim"}}
```

A keyword list:

```
iex> Access.pop([name: "Elixir", creator: "Valim"],
:name)
{"Elixir", [creator: "Valim"]}
```

An unknown key:

```
iex> Access.pop(%{name: "Elixir", creator: "Valim"},
:year)
{nil, %{creator: "Valim", name: "Elixir"}}
```

slice(range)

(since 1.14)

```
@spec slice(Range.t()) :: access fun(data :: list(),
current_value :: list())
```

Returns a function that accesses all items of a list that are within the provided range.

The range will be normalized following the same rules from `Enum.slice/2`.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Examples

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}, %{name: "vitor", salary: 25}]
iex> get_in(list, [Access.slice(1..2), :name])
["francine", "vitor"]
iex> get_and_update_in(list, [Access.slice(1..3//2),
:name], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{["francine"], [%{name: "john", salary: 10}, %{name:
"FRANCINE", salary: 30}, %{name: "vitor", salary: 25}]}
```

`slice/1` can also be used to pop elements out of a list or a key inside of a list:

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}, %{name: "vitor", salary: 25}]
iex> pop_in(list, [Access.slice(-2..-1)])
{[%{name: "francine", salary: 30}, %{name: "vitor",
salary: 25}], [%{name: "john", salary: 10}]}
iex> pop_in(list, [Access.slice(-2..-1), :name])
```

```
{["francine", "vitor"], [%{name: "john", salary: 10}, %
{salary: 30}, %{salary: 25}]}
```

When no match is found, an empty list is returned and the update function is never called

```
iex> list = [%{name: "john", salary: 10}, %{name:
"francine", salary: 30}, %{name: "vitor", salary: 25}]
iex> get_in(list, [Access.slice(5..10//2), :name])
[]
iex> get_and_update_in(list, [Access.slice(5..10//2),
:name], fn prev ->
...>   {prev, String.upcase(prev)}
...> end)
{[], [%{name: "john", salary: 10}, %{name: "francine",
salary: 30}, %{name: "vitor", salary: 25}]}
```

An error is raised if the accessed structure is not a list:

```
iex> get_in(%{}, [Access.slice(2..10//3)])
** (ArgumentError) Access.slice/1 expected a list, got: %
{}
```

An error is raised if the step of the range is negative:

```
iex> get_in([], [Access.slice(2..10//-1)])
** (ArgumentError) Access.slice/1 does not accept ranges
with negative steps, got: 2..10//-1
```

values()

(since 1.19.0)

```
@spec values() :: access_fun(data :: map() | keyword(),
current_value :: list())
```

Returns a function that accesses all values in a map or a keyword list.

The returned function is typically passed as an accessor to `Kernel.get_in/2`, `Kernel.get_and_update_in/3`, and friends.

Examples

```
iex> users = %{"john" => %{age: 27}, "meg" => %{age: 23}}
iex> get_in(users, [Access.values(), :age]) |>
Enum.sort()
[23, 27]
iex> update_in(users, [Access.values(), :age], fn age ->
age + 1 end)
%{"john" => %{age: 28}, "meg" => %{age: 24}}
iex> put_in(users, [Access.values(), :planet], "Earth")
%{"john" => %{age: 27, planet: "Earth"}, "meg" => %{age:
23, planet: "Earth"}}
```

Values in keyword lists can be accessed as well:

```
iex> users = [john: %{age: 27}, meg: %{age: 23}]
iex> get_and_update_in(users, [Access.values(), :age], fn
age -> {age, age + 1} end)
{[27, 23], [john: %{age: 28}, meg: %{age: 24}]}
```

By returning `:pop` from an accessor function, you can remove the accessed key and value from the map or keyword list:

```
iex> require Integer
iex> numbers = [one: 1, two: 2, three: 3, four: 4]
```

```
iex> get_and_update_in(numbers, [Access.values()], fn num
->
...>   if Integer.is_even(num), do: :pop, else: {num,
to_string(num)}
...> end)
{[1, 2, 3, 4], [one: "1", three: "3"]}
```

An error is raised if the accessed structure is not a map nor a keyword list:

```
iex> get_in([1, 2, 3], [Access.values()])
** (RuntimeError) Access.values/0 expected a map or a
keyword list, got: [1, 2, 3]
```

Date.Range

Returns an inclusive range between dates.

Ranges must be created with the `Date.range/2` or `Date.range/3` function.

The following fields are public:

- `:first` - the initial date on the range
- `:last` - the last date on the range
- `:step` - (since v1.12.0) the step

The remaining fields are private and should not be accessed.

Summary

Types

t()

Types

t()

```
@type t() :: %Date.Range{
  first: Date.t(),
  first_in_iso_days: days(),
  last: Date.t(),
  last_in_iso_days: days(),
```

```
  step: pos_integer() | neg_integer()
}
```

Enum

Functions for working with collections (known as enumerables).

In Elixir, an enumerable is any data type that implements the `Enumerable` protocol. `List`s (`[1, 2, 3]`), `Map`s (`%{foo: 1, bar: 2}`) and `Range`s (`1..3`) are common data types used as enumerables:

```
iex> Enum.map([1, 2, 3], fn x -> x * 2 end)
[2, 4, 6]

iex> Enum.sum([1, 2, 3])
6

iex> Enum.map(1..3, fn x -> x * 2 end)
[2, 4, 6]

iex> Enum.sum(1..3)
6

iex> map = %{"a" => 1, "b" => 2}
iex> Enum.map(map, fn {k, v} -> {k, v * 2} end)
[{"a", 2}, {"b", 4}]
```

Many other enumerables exist in the language, such as `MapSet`s and the data type returned by `File.stream!/3` which allows a file to be traversed as if it was an enumerable.

For a general overview of all functions in the `Enum` module, see the `Enum` cheatsheet.

The functions in this module work in linear time. This means that, the time it takes to perform an operation grows at the same rate as the length of the enumerable. This is expected on operations such as `Enum.map/2`. After all, if we want to traverse every element on a list, the longer the list, the more elements we need to traverse, and the longer it will take.

This linear behavior should also be expected on operations like `count/1`, `member?/2`, `at/2` and similar. While Elixir does allow data types to provide performant variants for such operations, you should not expect it to always be available, since the `Enum` module is meant to work with a large variety of data types and not all data types can provide optimized behavior.

Finally, note the functions in the `Enum` module are eager: they will traverse the enumerable as soon as they are invoked. This is particularly dangerous when working with infinite enumerables. In such cases, you should use the `Stream` module, which allows you to lazily express computations, without traversing collections, and work with possibly infinite collections. See the `Stream` module for examples and documentation.

Summary

Types

acc()
default()
element()
index()

Zero-based index. It can also be a negative integer.

t()

Functions

all?(enumerable)

Returns `true` if all elements in `enumerable` are truthy.

all?(enumerable, fun)

Returns `true` if `fun.(element)` is truthy for all elements in `enumerable`.

any?(enumerable)

Returns `true` if at least one element in `enumerable` is truthy.

any?(enumerable, fun)

Returns `true` if `fun.(element)` is truthy for at least one element in `enumerable`.

at(enumerable, index, default \\ nil)

Finds the element at the given `index` (zero-based).

chunk_by(enumerable, fun)

Splits enumerable on every element for which `fun` returns a new value.

chunk_every(enumerable, count)

Shortcut to `chunk_every(enumerable, count, count)`.

chunk_every(enumerable, count, step, leftover \\ [])

Returns list of lists containing `count` elements each, where each new chunk starts `step` elements into the `enumerable`.

chunk_while(enumerable, acc, chunk_fun, after_fun)

Chunks the `enumerable` with fine grained control when every chunk is emitted.

concat(enumerables)

Given an enumerable of enumerables, concatenates the `enumerables` into a single list.

concat(left, right)

Concatenates the enumerable on the `right` with the enumerable on the `left`.

count(enumerable)

Returns the size of the `enumerable`.

count(enumerable, fun)

Returns the count of elements in the `enumerable` for which `fun` returns a truthy value.

count_until(enumerable, limit)

Counts the enumerable stopping at `limit`.

count_until(enumerable, fun, limit)

Counts the elements in the enumerable for which `fun` returns a truthy value, stopping at `limit`.

dedup(enumerable)

Enumerates the `enumerable`, returning a list where all consecutive duplicate elements are collapsed to a single element.

dedup_by(enumerable, fun)

Enumerates the `enumerable`, returning a list where all consecutive duplicate elements are collapsed to a single element.

drop(enumerable, amount)

Drops the `amount` of elements from the `enumerable`.

drop_every(enumerable, nth)

Returns a list of every `nth` element in the `enumerable` dropped, starting with the first element.

drop_while(enumerable, fun)

Drops elements at the beginning of the `enumerable` while `fun` returns a truthy value.

each(enumerable, fun)

Invokes the given `fun` for each element in the `enumerable`.

empty?(enumerable)

Determines if the `enumerable` is empty.

fetch(enumerable, index)

Finds the element at the given `index` (zero-based).

fetch!(enumerable, index)

Finds the element at the given `index` (zero-based).

filter(enumerable, fun)

Filters the `enumerable`, i.e. returns only those elements for which `fun` returns a truthy value.

find(enumerable, default \\ nil, fun)

Returns the first element for which `fun` returns a truthy value. If no such element is found, returns `default`.

find_index(enumerable, fun)

Similar to `find/3`, but returns the index (zero-based) of the element instead of the element itself.

find_value(enumerable, default \\ nil, fun)

Similar to `find/3`, but returns the value of the function invocation instead of the element itself.

flat_map(enumerable, fun)

Maps the given `fun` over `enumerable` and flattens the result only one level deep.

flat_map_reduce(enumerable, acc, fun)

Maps and reduces an `enumerable`, flattening the results only one level deep.

frequencies(enumerable)

Returns a map with keys as unique elements of `enumerable` and values as the count of every element.

frequencies_by(enumerable, key_fun)

Returns a map with keys as unique elements given by `key_fun` and values as the count of every element.

group_by(enumerable, key_fun, value_fun \\ fn x -> x end)

Splits the `enumerable` into groups based on `key_fun`.

intersperse(enumerable, separator)

Intersperses `separator` between each element of the enumeration.

into(enumerable, collectable)

Inserts the given `enumerable` into a `collectable`.

into(enumerable, collectable, transform)

Inserts the given `enumerable` into a `collectable` according to the transformation function.

join(enumerable, joiner \\ "")

Joins the given `enumerable` into a string using `joiner` as a separator.

map(enumerable, fun)

Returns a list where each element is the result of invoking `fun` on each corresponding element of `enumerable`.

map_every(enumerable, nth, fun)

Returns a list of results of invoking `fun` on every `nth` element of `enumerable`, starting with the first element.

map_intersperse(enumerable, separator, mapper)

Maps and intersperses the given enumerable in one pass.

map_join(enumerable, joiner \\ "", mapper)

Maps and joins the given `enumerable` in one pass.

map_reduce(enumerable, acc, fun)

Invokes the given function to each element in the `enumerable` to reduce it to a single element, while keeping an accumulator.
max(enumerable, sorter \\ &>=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns the maximal element in the `enumerable` according to Erlang's term ordering.

max_by(enumerable, fun, sorter \\ &>=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns the maximal element in the `enumerable` as calculated by the given `fun`.

member?(enumerable, element)

Checks if `element` exists within the `enumerable`.

min(enumerable, sorter \\ &<=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns the minimal element in the `enumerable` according to Erlang's term ordering.

min_by(enumerable, fun, sorter \\ &<=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns the minimal element in the `enumerable` as calculated by the given `fun`.

min_max(enumerable, sorter_or_empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns a tuple with the minimal and the maximal elements in the enumerable.

min_max(enumerable, sorter, empty_fallback)

min_max_by(enumerable, fun, sorter_or_empty_fallback \\ &</2, empty_fallback \\ fn -> raise Enum.EmptyError end)

Returns a tuple with the minimal and the maximal elements in the enumerable as calculated by the given function.

product(enumerable)

Returns the product of all elements.

product_by(enumerable, mapper)

Maps and computes the product of the given `enumerable` in one pass.

random(enumerable)

Returns a random element of an `enumerable`.

reduce(enumerable, fun)

Invokes `fun` for each element in the `enumerable` with the accumulator.

reduce(enumerable, acc, fun)

Invokes `fun` for each element in the `enumerable` with the accumulator.

reduce_while(enumerable, acc, fun)

Reduces `enumerable` until `fun` returns `{:halt, term}`.

reject(enumerable, fun)

Returns a list of elements in `enumerable` excluding those for which the function `fun` returns a truthy value.

reverse(enumerable)

Returns a list of elements in `enumerable` in reverse order.

reverse(enumerable, tail)

Reverses the elements in `enumerable`, appends the `tail`, and returns it as a list.

reverse_slice(enumerable, start_index, count)

Reverses the `enumerable` in the range from initial `start_index` through `count` elements.

scan(enumerable, fun)

Passes each element from `enumerable` to the `fun` as the first argument, stores the `fun` result in a list and passes the result as the second argument for the next computation.

scan(enumerable, acc, fun)

Passes each element from `enumerable` to the `fun` as the first argument, stores the `fun` result in a list and passes the result as the second argument for the next computation.

shuffle(enumerable)

Returns a list with the elements of `enumerable` shuffled.

slice(enumerable, index_range)

Returns a subset list of the given `enumerable` by `index_range`.

slice(enumerable, start_index, amount)

Returns a subset list of the given `enumerable`, from `start_index` (zero-based) with `amount` number of elements if available.

slide(enumerable, range_or_single_index, insertion_index)

Slides a single or multiple elements given by `range_or_single_index` from `enumerable` to `insertion_index`.

sort(enumerable)

Sorts the `enumerable` according to Erlang's term ordering.

sort(enumerable, sorter)

Sorts the `enumerable` by the given function.

sort_by(enumerable, mapper, sorter \\ :asc)

Sorts the mapped results of the `enumerable` according to the provided `sorter` function.

split(enumerable, count)

Splits the `enumerable` into two enumerables, leaving `count` elements in the first one.

split_while(enumerable, fun)

Splits enumerable in two at the position of the element for which `fun` returns a falsy value (`false` or `nil`) for the first time.

split_with(enumerable, fun)

Splits the `enumerable` in two lists according to the given function `fun`.

sum(enumerable)

Returns the sum of all elements.

sum_by(enumerable, mapper)

Maps and sums the given `enumerable` in one pass.

take(enumerable, amount)

Takes an `amount` of elements from the beginning or the end of the `enumerable`.

take_every(enumerable, nth)

Returns a list of every `nth` element in the `enumerable`, starting with the first element.

take_random(enumerable, count)

Takes `count` random elements from `enumerable`.

take_while(enumerable, fun)

Takes the elements from the beginning of the `enumerable` while `fun` returns a truthy value.

to_list(enumerable)

Converts `enumerable` to a list.

uniq(enumerable)

Enumerates the `enumerable`, removing all duplicate elements.

uniq_by(enumerable, fun)

Enumerates the `enumerable`, by removing the elements for which function `fun` returned duplicate elements.

unzip(enumerable)

Opposite of `zip/2`. Extracts two-element tuples from the given `enumerable` and groups them together.

with_index(enumerable, fun_or_offset \\ 0)

Returns the `enumerable` with each element wrapped in a tuple alongside its index or according to a given function.

zip(enumerables)

Zips corresponding elements from a finite collection of enumerables into a list of tuples.

zip(enumerable1, enumerable2)

Zips corresponding elements from two enumerables into a list of tuples.

zip_reduce(enumerables, acc, reducer)

Reduces over all of the given enumerables, halting as soon as any enumerable is empty.

zip_reduce(left, right, acc, reducer)

Reduces over two enumerables halting as soon as either enumerable is empty.

zip_with(enumerables, zip_fun)

Zips corresponding elements from a finite collection of enumerables into list, transforming them with the `zip_fun` function as it goes.

zip_with(enumerable1, enumerable2, zip_fun)

Zips corresponding elements from two enumerables into a list, transforming them with the `zip_fun` function as it goes.

Types

acc()

```
@type acc() :: any()
```

default()

```
@type default() :: any()
```

element()

```
@type element() :: any()
```

index()

```
@type index() :: integer()
```

Zero-based index. It can also be a negative integer.

t()

```
@type t() :: Enumerable.t()
```

Functions

all?(enumerable)

```
@spec all?(t()) :: boolean()
```

Returns `true` if all elements in `enumerable` are truthy.

When an element has a falsy value (`false` or `nil`) iteration stops immediately and `false` is returned. In all other cases `true` is returned.

Examples

```
iex> Enum.all?([1, 2, 3])
true

iex> Enum.all?([1, nil, 3])
false

iex> Enum.all?([])
true
```

all?(enumerable, fun)

```
@spec all?(t(), (element() -> as_boolean(term()))) ::
boolean()
```

Returns `true` if `fun.(element)` is truthy for all elements in `enumerable`.

Iterates over `enumerable` and invokes `fun` on each element. If `fun` ever returns a falsy value (`false` or `nil`), iteration stops immediately and

`false` is returned. Otherwise, `true` is returned.

Examples

```
iex> Enum.all?([2, 4, 6], fn x -> rem(x, 2) == 0 end)
true

iex> Enum.all?([2, 3, 4], fn x -> rem(x, 2) == 0 end)
false

iex> Enum.all?([], fn _ -> nil end)
true
```

As the last example shows, `Enum.all?/2` returns `true` if `enumerable` is empty, regardless of `fun`. In an empty enumerable there is no element for which `fun` returns a falsy value, so the result must be `true`. This is a well-defined logical argument for empty collections.

any?(enumerable)

```
@spec any?(t()) :: boolean()
```

Returns `true` if at least one element in `enumerable` is truthy.

When an element has a truthy value (neither `false` nor `nil`) iteration stops immediately and `true` is returned. In all other cases `false` is returned.

Examples

```
iex> Enum.any?([false, false, false])
false

iex> Enum.any?([false, true, false])
true

iex> Enum.any?([])
false
```

any?(enumerable, fun)

```
@spec any?(t(), (element() -> as boolean(term()))) ::
boolean()
```

Returns `true` if `fun.(element)` is truthy for at least one element in `enumerable`.

Iterates over the `enumerable` and invokes `fun` on each element. When an invocation of `fun` returns a truthy value (neither `false` nor `nil`) iteration stops immediately and `true` is returned. In all other cases `false` is returned.

Examples

```
iex> Enum.any?([2, 4, 6], fn x -> rem(x, 2) == 1 end)
false

iex> Enum.any?([2, 3, 4], fn x -> rem(x, 2) == 1 end)
true
```

```
iex> Enum.any?([], fn x -> x > 0 end)
false
```

at(enumerable, index, default \\ nil)

```
@spec at(t(), index(), default()) :: element() | default()
```

Finds the element at the given `index` (zero-based).

Returns `default` if `index` is out of bounds.

A negative `index` can be passed, which means the `enumerable` is enumerated once and the `index` is counted from the end (for example, `-1` finds the last element).

Examples

```
iex> Enum.at([2, 4, 6], 0)
2

iex> Enum.at([2, 4, 6], 2)
6

iex> Enum.at([2, 4, 6], 4)
nil

iex> Enum.at([2, 4, 6], 4, :none)
:none
```

chunk_by(enumerable, fun)

```
@spec chunk_by(t(), (element() -> any())) :: [list()]
```

Splits enumerable on every element for which `fun` returns a new value.

Returns a list of lists.

Examples

```
iex> Enum.chunk_by([1, 2, 2, 3, 4, 4, 6, 7, 7], &(rem(&1,
2) == 1))
[[1], [2, 2], [3], [4, 4, 6], [7, 7]]
```

chunk_every(enumerable, count)

(since 1.5.0)

```
@spec chunk_every(t(), pos_integer()) :: [list()]
```

Shortcut to `chunk_every(enumerable, count, count)`.

chunk_every(enumerable, count, step, leftover \\ [])

(since 1.5.0)

```
@spec chunk_every(t(), pos_integer(), pos_integer(), t() |
:discard) :: [list()]
```

Returns list of lists containing `count` elements each, where each new chunk starts `step` elements into the `enumerable`.

`step` is optional and, if not passed, defaults to `count`, i.e. chunks do not overlap. Chunking will stop as soon as the collection ends or when we emit an incomplete chunk.

If the last chunk does not have `count` elements to fill the chunk, elements are taken from `leftover` to fill in the chunk. If `leftover` does not have enough elements to fill the chunk, then a partial chunk is returned with less than `count` elements.

If `:discard` is given in `leftover`, the last chunk is discarded unless it has exactly `count` elements.

Examples

```
iex> Enum.chunk_every([1, 2, 3, 4, 5, 6], 2)
[[1, 2], [3, 4], [5, 6]]

iex> Enum.chunk_every([1, 2, 3, 4, 5, 6], 3, 2, :discard)
[[1, 2, 3], [3, 4, 5]]

iex> Enum.chunk_every([1, 2, 3, 4, 5, 6], 3, 2, [7])
[[1, 2, 3], [3, 4, 5], [5, 6, 7]]

iex> Enum.chunk_every([1, 2, 3, 4], 3, 3, [])
[[1, 2, 3], [4]]

iex> Enum.chunk_every([1, 2, 3, 4], 10)
[[1, 2, 3, 4]]

iex> Enum.chunk_every([1, 2, 3, 4, 5], 2, 3, [])
[[1, 2], [4, 5]]
```

```
iex> Enum.chunk_every([1, 2, 3, 4], 3, 3,
Stream.cycle([0]))
[[1, 2, 3], [4, 0, 0]]
```

chunk_while(enumerable, acc, chunk_fun, after_fun)

(since 1.5.0)

```
@spec chunk_while(
  t(),
  acc(),
  (element(), acc() -> {:cont, chunk, acc()} | {:cont, acc()}
| {:halt, acc()}),
  (acc() -> {:cont, chunk, acc()} | {:cont, acc()})
) :: Enumerable.t()
when chunk: any()
```

Chunks the `enumerable` with fine grained control when every chunk is emitted.

`chunk_fun` receives the current element and the accumulator and must return:

- `{:cont, chunk, acc}` to emit a chunk and continue with the accumulator
- `{:cont, acc}` to not emit any chunk and continue with the accumulator
- `{:halt, acc}` to halt chunking over the `enumerable`.

`after_fun` is invoked with the final accumulator when iteration is finished (or `halt`ed) to handle any trailing elements that were returned as part of an accumulator, but were not emitted as a chunk by `chunk_fun`. It must return:

- `{:cont, chunk, acc}` to emit a chunk. The chunk will be appended to the list of already emitted chunks.
- `{:cont, acc}` to not emit a chunk

The `acc` in `after_fun` is required in order to mirror the tuple format from `chunk_fun` but it will be discarded since the traversal is complete.

Returns a list of emitted chunks.

Examples

```
iex> chunk_fun = fn element, acc ->
...>   if rem(element, 2) == 0 do
...>     {:cont, Enum.reverse([element | acc]), []}
...>   else
...>     {:cont, [element | acc]}
...>   end
...> end
iex> after_fun = fn
...>   [] -> {:cont, []}
...>   acc -> {:cont, Enum.reverse(acc), []}
...> end
iex> Enum.chunk_while(1..10, [], chunk_fun, after_fun)
[[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]
iex> Enum.chunk_while([1, 2, 3, 5, 7], [], chunk_fun,
after_fun)
[[1, 2], [3, 5, 7]]
```

concat(enumerables)

```
@spec concat(t()) :: t()
```

Given an enumerable of enumerables, concatenates the `enumerables` into a single list.

Examples

```
iex> Enum.concat([1..3, 4..6, 7..9])
[1, 2, 3, 4, 5, 6, 7, 8, 9]

iex> Enum.concat([[1, [2], 3], [4], [5, 6]])
[1, [2], 3, 4, 5, 6]
```

concat(left, right)

```
@spec concat(t(), t()) :: t()
```

Concatenates the enumerable on the `right` with the enumerable on the `left`.

This function produces the same result as the `++/2` operator for lists.

Examples

```
iex> Enum.concat(1..3, 4..6)
[1, 2, 3, 4, 5, 6]

iex> Enum.concat([1, 2, 3], [4, 5, 6])
[1, 2, 3, 4, 5, 6]
```

count(enumerable)

```
@spec count(t()) :: non_neg_integer()
```

Returns the size of the `enumerable`.

Examples

```
iex> Enum.count([1, 2, 3])
3
```

count(enumerable, fun)

```
@spec count(t(), (element() -> as_boolean(term()))) ::
non_neg_integer()
```

Returns the count of elements in the `enumerable` for which `fun` returns a truthy value.

Examples

```
iex> Enum.count([1, 2, 3, 4, 5], fn x -> rem(x, 2) == 0
end)
2
```

count_until(enumerable, limit)

(since 1.12.0)

```
@spec count_until(t(), pos_integer()) :: non_neg_integer()
```

Counts the enumerable stopping at `limit`.

This is useful for checking certain properties of the count of an enumerable without having to actually count the entire enumerable. For example, if you wanted to check that the count was exactly, at least, or more than a value.

If the enumerable implements `Enumerable.count/1`, the enumerable is not traversed and we return the lower of the two numbers. To force enumeration, use `count_until/3` with `fn _ -> true end` as the second argument.

Examples

```
iex> Enum.count_until(1..20, 5)
5
iex> Enum.count_until(1..20, 50)
20
iex> Enum.count_until(1..10, 10) == 10 # At least 10
true
iex> Enum.count_until(1..11, 10 + 1) > 10 # More than 10
true
iex> Enum.count_until(1..5, 10) < 10 # Less than 10
true
iex> Enum.count_until(1..10, 10 + 1) == 10 # Exactly ten
true
```

count_until(enumerable, fun, limit)

(since 1.12.0)

```
@spec count_until(t(), (element() -> as boolean(term())),
pos integer()) ::
  non neg integer()
```

Counts the elements in the enumerable for which `fun` returns a truthy value, stopping at `limit`.

See `count/2` and `count_until/2` for more information.

Examples

```
iex> Enum.count_until(1..20, fn x -> rem(x, 2) == 0 end,
7)
7
iex> Enum.count_until(1..20, fn x -> rem(x, 2) == 0 end,
11)
10
```

dedup(enumerable)

```
@spec dedup(t()) :: list()
```

Enumerates the `enumerable`, returning a list where all consecutive duplicate elements are collapsed to a single element.

Elements are compared using `===/2`.

If you want to remove all duplicate elements, regardless of order, see `uniq/1`.

Examples

```
iex> Enum.dedup([1, 2, 3, 3, 2, 1])
[1, 2, 3, 2, 1]
```

```
iex> Enum.dedup([1, 1, 2, 2.0, :three, :three])
[1, 2, 2.0, :three]
```

dedup_by(enumerable, fun)

```
@spec dedup_by(t(), (element() -> term())) :: list()
```

Enumerates the `enumerable`, returning a list where all consecutive duplicate elements are collapsed to a single element.

The function `fun` maps every element to a term which is used to determine if two elements are duplicates.

Examples

```
iex> Enum.dedup_by([{1, :a}, {2, :b}, {2, :c}, {1, :a}],
fn {x, _} -> x end)
[{1, :a}, {2, :b}, {1, :a}]

iex> Enum.dedup_by([5, 1, 2, 3, 2, 1], fn x -> x > 2 end)
[5, 1, 3, 2]
```

drop(enumerable, amount)

```
@spec drop(t(), integer()) :: list()
```

Drops the `amount` of elements from the `enumerable`.

If a negative `amount` is given, the `amount` of last values will be dropped. The `enumerable` will be enumerated once to retrieve the proper index and the remaining calculation is performed from the end.

Examples

```
iex> Enum.drop([1, 2, 3], 2)
[3]

iex> Enum.drop([1, 2, 3], 10)
[]

iex> Enum.drop([1, 2, 3], 0)
[1, 2, 3]

iex> Enum.drop([1, 2, 3], -1)
[1, 2]
```

drop_every(enumerable, nth)

```
@spec drop_every(t(), non_neg_integer()) :: list()
```

Returns a list of every `nth` element in the `enumerable` dropped, starting with the first element.

The first element is always dropped, unless `nth` is 0.

The second argument specifying every `nth` element must be a non-negative integer.

Examples

```
iex> Enum.drop_every(1..10, 2)
[2, 4, 6, 8, 10]

iex> Enum.drop_every(1..10, 0)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

iex> Enum.drop_every([1, 2, 3], 1)
[]
```

drop_while(enumerable, fun)

```
@spec drop_while(t(), (element() -> as boolean(term()))) ::
list()
```

Drops elements at the beginning of the `enumerable` while `fun` returns a truthy value.

Examples

```
iex> Enum.drop_while([1, 2, 3, 2, 1], fn x -> x < 3 end)
[3, 2, 1]
```

each(enumerable, fun)

```
@spec each(t(), (element() -> any())) :: :ok
```

Invokes the given `fun` for each element in the `enumerable`.

Returns `:ok`.

Examples

```
Enum.each(["some", "example"], fn x -> IO.puts(x) end)
some
example
#=> :ok
```

empty?(enumerable)

```
@spec empty?(t()) :: boolean()
```

Determines if the `enumerable` is empty.

Returns `true` if `enumerable` is empty, otherwise `false`.

Examples

```
iex> Enum.empty?([])
true

iex> Enum.empty?([1, 2, 3])
false
```

fetch(enumerable, index)

```
@spec fetch(t(), index()) :: {:ok, element()} | :error
```

Finds the element at the given `index` (zero-based).

Returns `{:ok, element}` if found, otherwise `:error`.

A negative `index` can be passed, which means the `enumerable` is enumerated once and the `index` is counted from the end (for example, `-1` fetches the last element).

Examples

```
iex> Enum.fetch([2, 4, 6], 0)
{:ok, 2}

iex> Enum.fetch([2, 4, 6], -3)
{:ok, 2}

iex> Enum.fetch([2, 4, 6], 2)
{:ok, 6}

iex> Enum.fetch([2, 4, 6], 4)
:error
```

fetch!(enumerable, index)

```
@spec fetch!(t(), index()) :: element()
```

Finds the element at the given `index` (zero-based).

Raises `OutOfBoundsError` if the given `index` is outside the range of the `enumerable`.

Examples

```
iex> Enum.fetch!([2, 4, 6], 0)
2

iex> Enum.fetch!([2, 4, 6], 2)
6

iex> Enum.fetch!([2, 4, 6], 4)
** (Enum.OutOfBoundsError) out of bounds error at
position 4 when traversing enumerable [2, 4, 6]
```

filter(enumerable, fun)

```
@spec filter(t(), (element() -> as boolean(term()))) ::
list()
```

Filters the `enumerable`, i.e. returns only those elements for which `fun` returns a truthy value.

See also `reject/2` which discards all elements where the function returns a truthy value.

Examples

```
iex> Enum.filter([1, 2, 3], fn x -> rem(x, 2) == 0 end)
[2]
iex> Enum.filter(["apple", "pear", "banana"], fn fruit ->
String.contains?(fruit, "a") end)
["apple", "pear", "banana"]
iex> Enum.filter([4, 21, 24, 904], fn seconds -> seconds
> 1000 end)
[]
```

Keep in mind that `filter` is not capable of filtering and transforming an element at the same time. If you would like to do so, consider using `flat_map/2`. For example, if you want to convert all strings that represent an integer and discard the invalid one in one pass:

```
strings = ["1234", "abc", "12ab"]

Enum.flat_map(strings, fn string ->
  case Integer.parse(string) do
    # transform to integer
    {int, _rest} -> [int]
    # skip the value
    :error -> []
  end
end)
```

find(enumerable, default \\ nil, fun)

```
@spec find(t(), default(), (element() -> any())) :: element()
| default()
```

Returns the first element for which `fun` returns a truthy value. If no such element is found, returns `default`.

Examples

```
iex> Enum.find([2, 3, 4], fn x -> rem(x, 2) == 1 end)
3

iex> Enum.find([2, 4, 6], fn x -> rem(x, 2) == 1 end)
nil
```

```
iex> Enum.find([2, 4, 6], 0, fn x -> rem(x, 2) == 1 end)
0
```

find_index(enumerable, fun)

```
@spec find_index(t(), (element() -> any())) ::
non_neg_integer() | nil
```

Similar to `find/3`, but returns the index (zero-based) of the element instead of the element itself.

Examples

```
iex> Enum.find_index([2, 4, 6], fn x -> rem(x, 2) == 1
end)
nil

iex> Enum.find_index([2, 3, 4], fn x -> rem(x, 2) == 1
end)
1
```

find_value(enumerable, default \\ nil, fun)

```
@spec find_value(t(), default(), (element() -> found_value))
::
  found_value | default()
when found_value: term()
```

Similar to `find/3`, but returns the value of the function invocation instead of the element itself.

The return value is considered to be found when the result is truthy (neither `nil` nor `false`).

Examples

```
iex> Enum.find_value([2, 3, 4], fn x ->
...>   if x > 2, do: x * x
...> end)
9

iex> Enum.find_value([2, 4, 6], fn x -> rem(x, 2) == 1
end)
nil

iex> Enum.find_value([2, 3, 4], fn x -> rem(x, 2) == 1
end)
true

iex> Enum.find_value([1, 2, 3], "no bools!",
&is_boolean/1)
"no bools!"
```

flat_map(enumerable, fun)

```
@spec flat_map(t(), (element() -> t())) :: list()
```

Maps the given `fun` over `enumerable` and flattens the result only one level deep.

This function returns a new enumerable built by appending the result of invoking `fun` on each element of `enumerable` together; conceptually, this is similar to a combination of `map/2` and `concat/1`.

Examples

```
iex> Enum.flat_map([:a, :b, :c], fn x -> [x, x] end)
[:a, :a, :b, :b, :c, :c]

iex> Enum.flat_map([{1, 3}, {4, 6}], fn {x, y} -> x..y
end)
[1, 2, 3, 4, 5, 6]

iex> Enum.flat_map([:a, :b, :c], fn x -> [[x]] end)
[[:a], [:b], [:c]]
```

This is frequently used to transform and filter in one pass, returning empty lists to exclude results:

```
iex> Enum.flat_map([4, 0, 2, 0], fn x ->
...>   if x != 0, do: [1 / x], else: []
...> end)
[0.25, 0.5]
```

flat_map_reduce(enumerable, acc, fun)

```
@spec flat_map_reduce(t(), acc(), fun) :: {[any()], acc()}
when fun: (element(), acc() -> {t(), acc()} | {:halt, acc()})
```

Maps and reduces an `enumerable`, flattening the results only one level deep.

It expects an accumulator and a function that receives each enumerable element, and must return a tuple containing a new enumerable (often a list) with the new accumulator or a tuple with `:halt` as first element and the accumulator as second.

Returns a 2-element tuple where the first element is the results flattened one level deep and the second element is the last accumulator.

Examples

```
iex> enumerable = 1..100
iex> n = 3
iex> Enum.flat_map_reduce(enumerable, 0, fn x, acc ->
...>   if acc < n, do: {[x], acc + 1}, else: {:halt, acc}
...> end)
{[1, 2, 3], 3}

iex> Enum.flat_map_reduce(1..5, 0, fn x, acc -> {[[x]],
acc + x} end)
{[[1], [2], [3], [4], [5]], 15}
```

frequencies(enumerable)

(since 1.10.0)

```
@spec frequencies(t()) :: map()
```

Returns a map with keys as unique elements of `enumerable` and values as the count of every element.

Examples

```
iex> Enum.frequencies(~w{ant buffalo ant ant buffalo
dingo})
%{"ant" => 3, "buffalo" => 2, "dingo" => 1}
```

frequencies_by(enumerable, key_fun)

(since 1.10.0)

```
@spec frequencies_by(t(), (element() -> any())) :: map()
```

Returns a map with keys as unique elements given by `key_fun` and values as the count of every element.

Examples

```
iex> Enum.frequencies_by(~w{aa aA bb cc},
&String.downcase/1)
%{"aa" => 2, "bb" => 1, "cc" => 1}

iex> Enum.frequencies_by(~w{aaa aA bbb cc c},
&String.length/1)
%{3 => 2, 2 => 2, 1 => 1}
```

group_by(enumerable, key_fun, value_fun \\ fn x -> x end)

```
@spec group_by(t(), (element() -> any()), (element() ->
any())) :: map()
```

Splits the `enumerable` into groups based on `key_fun`.

The result is a map where each key is given by `key_fun` and each value is a list of elements given by `value_fun`. The order of elements within each list is preserved from the `enumerable`. However, like all maps, the resulting map is unordered.

Examples

```
iex> Enum.group_by(~w{ant buffalo cat dingo},
&String.length/1)
%{3 => ["ant", "cat"], 5 => ["dingo"], 7 => ["buffalo"]}

iex> Enum.group_by(~w{ant buffalo cat dingo},
&String.length/1, &String.first/1)
%{3 => ["a", "c"], 5 => ["d"], 7 => ["b"]}
```

The key can be any Elixir value. For example, you may use a tuple to group by multiple keys:

```
iex> collection = [
...>   %{id: 1, lang: "Elixir", seq: 1},
...>   %{id: 1, lang: "Java", seq: 1},
...>   %{id: 1, lang: "Ruby", seq: 2},
...>   %{id: 2, lang: "Python", seq: 1},
...>   %{id: 2, lang: "C#", seq: 2},
...>   %{id: 2, lang: "Haskell", seq: 2},
...> ]
iex> Enum.group_by(collection, &{&1.id, &1.seq})
%{
  {1, 1} => [%{id: 1, lang: "Elixir", seq: 1}, %{id: 1,
lang: "Java", seq: 1}],
  {1, 2} => [%{id: 1, lang: "Ruby", seq: 2}],
  {2, 1} => [%{id: 2, lang: "Python", seq: 1}],
  {2, 2} => [%{id: 2, lang: "C#", seq: 2}, %{id: 2, lang:
```

```
"Haskell", seq: 2}]
}
iex> Enum.group_by(collection, &{&1.id, &1.seq}, &{&1.id,
&1.lang})
%{
  {1, 1} => [{1, "Elixir"}, {1, "Java"}],
  {1, 2} => [{1, "Ruby"}],
  {2, 1} => [{2, "Python"}],
  {2, 2} => [{2, "C#"}, {2, "Haskell"}]
}
```

intersperse(enumerable, separator)

```
@spec intersperse(t(), element()) :: list()
```

Intersperses `separator` between each element of the enumeration.

Examples

```
iex> Enum.intersperse([1, 2, 3], 0)
[1, 0, 2, 0, 3]

iex> Enum.intersperse([1], 0)
[1]

iex> Enum.intersperse([], 0)
[]
```

into(enumerable, collectable)

```
@spec into(Enumerable.t(), Collectable.t()) ::
Collectable.t()
```

Inserts the given `enumerable` into a `collectable`.

Note that passing a non-empty list as the `collectable` is deprecated. If you're collecting into a non-empty keyword list, consider using `Keyword.merge(collectable, Enum.to_list(enumerable))`. If you're collecting into a non-empty list, consider something like `Enum.to_list(enumerable) ++ collectable`.

Examples

```
iex> Enum.into([1, 2], [])
[1, 2]

iex> Enum.into([a: 1, b: 2], %{})
%{a: 1, b: 2}

iex> Enum.into(%{a: 1}, %{b: 2})
%{a: 1, b: 2}

iex> Enum.into([a: 1, a: 2], %{})
%{a: 2}

iex> Enum.into([a: 2], %{a: 1, b: 3})
%{a: 2, b: 3}
```

into(enumerable, collectable, transform)

```
@spec into(Enumerable.t(), Collectable.t(), (term() ->
term())) :: Collectable.t()
```

Inserts the given `enumerable` into a `collectable` according to the transformation function.

Examples

```
iex> Enum.into([1, 2, 3], [], fn x -> x * 3 end)
[3, 6, 9]

iex> Enum.into(%{a: 1, b: 2}, %{c: 3}, fn {k, v} -> {k, v
* 2} end)
%{a: 2, b: 4, c: 3}
```

join(enumerable, joiner \\ "")

```
@spec join(t(), binary()) :: binary()
```

Joins the given `enumerable` into a string using `joiner` as a separator.

If `joiner` is not passed at all, it defaults to an empty string.

All elements in the `enumerable` must be convertible to a string or be a binary, otherwise an error is raised.

Examples

```
iex> Enum.join([1, 2, 3])
"123"

iex> Enum.join([1, 2, 3], " = ")
"1 = 2 = 3"

iex> Enum.join([["a", "b"], ["c", "d", "e", ["f", "g"]],
```

```
"h", "i"], " ")
"ab cdefg h i"
```

map(enumerable, fun)

```
@spec map(t(), (element() -> any())) :: list()
```

Returns a list where each element is the result of invoking `fun` on each corresponding element of `enumerable`.

For maps, the function expects a key-value tuple.

Examples

```
iex> Enum.map([1, 2, 3], fn x -> x * 2 end)
[2, 4, 6]

iex> Enum.map([a: 1, b: 2], fn {k, v} -> {k, -v} end)
[a: -1, b: -2]
```

map_every(enumerable, nth, fun)

(since 1.4.0)

```
@spec map_every(t(), non_neg_integer(), (element() -> any()))
:: list()
```

Returns a list of results of invoking `fun` on every `nth` element of `enumerable`, starting with the first element.

The first element is always passed to the given function, unless `nth` is `0`.

The second argument specifying every `nth` element must be a non-negative integer.

If `nth` is `0`, then `enumerable` is directly converted to a list, without `fun` being ever applied.

Examples

```
iex> Enum.map_every(1..10, 2, fn x -> x + 1000 end)
[1001, 2, 1003, 4, 1005, 6, 1007, 8, 1009, 10]

iex> Enum.map_every(1..10, 3, fn x -> x + 1000 end)
[1001, 2, 3, 1004, 5, 6, 1007, 8, 9, 1010]

iex> Enum.map_every(1..5, 0, fn x -> x + 1000 end)
[1, 2, 3, 4, 5]

iex> Enum.map_every([1, 2, 3], 1, fn x -> x + 1000 end)
[1001, 1002, 1003]
```

map_intersperse(enumerable, separator, mapper)

(since 1.10.0)

```
@spec map_intersperse(t(), element(), (element() -> any()))
:: list()
```

Maps and intersperses the given enumerable in one pass.

Examples

```
iex> Enum.map_intersperse([1, 2, 3], :a, &(&1 * 2))
[2, :a, 4, :a, 6]
```

map_join(enumerable, joiner \\ "", mapper)

```
@spec map_join(t(), String.t(), (element() ->
String.Chars.t())) :: String.t()
```

Maps and joins the given `enumerable` in one pass.

If `joiner` is not passed at all, it defaults to an empty string.

All elements returned from invoking the `mapper` must be convertible to a string, otherwise an error is raised.

Examples

```
iex> Enum.map_join([1, 2, 3], &(&1 * 2))
"246"

iex> Enum.map_join([1, 2, 3], " = ", &(&1 * 2))
"2 = 4 = 6"
```

map_reduce(enumerable, acc, fun)

```
@spec map_reduce(t(), acc(), (element(), acc() -> {element(),
acc()})) ::
  {list(), acc()}
```

Invokes the given function to each element in the `enumerable` to reduce it to a single element, while keeping an accumulator.

Returns a tuple where the first element is the mapped enumerable and the second one is the final accumulator.

The function, `fun`, receives two arguments: the first one is the element, and the second one is the accumulator. `fun` must return a tuple with two elements in the form of `{result, accumulator}`.

For maps, the first tuple element must be a `{key, value}` tuple.

Examples

```
iex> Enum.map_reduce([1, 2, 3], 0, fn x, acc -> {x * 2, x
+ acc} end)
{[2, 4, 6], 6}
```

max(enumerable, sorter \\ &>=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec max(t(), (element(), element() -> boolean()) |
module(), (-> empty_result)) ::
  element() | empty_result
when empty_result: any()
```

Returns the maximal element in the `enumerable` according to Erlang's term ordering.

By default, the comparison is done with the `>=` sorter function. If multiple elements are considered maximal, the first one that was found is returned. If you want the last element considered maximal to be returned, the sorter function should not return true for equal elements.

If the enumerable is empty, the provided `empty_fallback` is called. The default `empty_fallback` raises `Enum.EmptyError`.

Examples

```
iex> Enum.max([1, 2, 3])
3
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. For example:

```
iex> Enum.max([~D[2017-03-31], ~D[2017-04-01]])
~D[2017-03-31]
```

In the example above, `max/2` returned March 31st instead of April 1st because the structural comparison compares the day before the year. For this reason, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> Enum.max([~D[2017-03-31], ~D[2017-04-01]], Date)
~D[2017-04-01]
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.max([], &>=/2, fn -> 0 end)
0
```

max_by(enumerable, fun, sorter \\ &>=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec max_by(
  t(),
  (element() -> any()),
  (element(), element() -> boolean()) | module(),
  (-> empty_result)
) :: element() | empty_result
when empty_result: any()
```

Returns the maximal element in the `enumerable` as calculated by the given `fun`.

By default, the comparison is done with the `>=` sorter function. If multiple elements are considered maximal, the first one that was found is returned. If you want the last element considered maximal to be returned, the sorter function should not return true for equal elements.

Calls the provided `empty_fallback` function and returns its value if `enumerable` is empty. The default `empty_fallback` raises `Enum.EmptyError`.

Examples

```
iex> Enum.max_by(["a", "aa", "aaa"], fn x ->
String.length(x) end)
"aaa"

iex> Enum.max_by(["a", "aa", "aaa", "b", "bbb"],
&String.length/1)
"aaa"
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. Therefore, if you want to compare structs, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> users = [
...>   %{name: "Ellis", birthday: ~D[1943-05-11]},
...>   %{name: "Lovelace", birthday: ~D[1815-12-10]},
...>   %{name: "Turing", birthday: ~D[1912-06-23]}
...> ]
iex> Enum.max_by(users, &(&1.birthday), Date)
%{name: "Ellis", birthday: ~D[1943-05-11]}
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.max_by([], &String.length/1, fn -> nil end)
nil
```

member?(enumerable, element)

```
@spec member?(t(), element()) :: boolean()
```

Checks if `element` exists within the `enumerable`.

Membership is tested with the match (`===/2`) operator.

Examples

```
iex> Enum.member?(1..10, 5)
true
iex> Enum.member?(1..10, 5.0)
false

iex> Enum.member?([1.0, 2.0, 3.0], 2)
false
iex> Enum.member?([1.0, 2.0, 3.0], 2.000)
true

iex> Enum.member?([:a, :b, :c], :d)
false
```

When called outside guards, the `in` and `not in` operators work by using this function.

min(enumerable, sorter \\ &<=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec min(t(), (element(), element() -> boolean()) |
module(), (-> empty_result)) ::
  element() | empty_result
when empty_result: any()
```

Returns the minimal element in the `enumerable` according to Erlang's term ordering.

By default, the comparison is done with the `<=` sorter function. If multiple elements are considered minimal, the first one that was found is returned. If you want the last element considered minimal to be returned, the sorter function should not return true for equal elements.

If the enumerable is empty, the provided `empty_fallback` is called. The default `empty_fallback` raises `Enum.EmptyError`.

Examples

```
iex> Enum.min([1, 2, 3])
1
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. For example:

```
iex> Enum.min([~D[2017-03-31], ~D[2017-04-01]])
~D[2017-04-01]
```

In the example above, `min/2` returned April 1st instead of March 31st because the structural comparison compares the day before the year. For this reason, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> Enum.min([~D[2017-03-31], ~D[2017-04-01]], Date)
~D[2017-03-31]
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.min([], fn -> 0 end)
0
```

min_by(enumerable, fun, sorter \\ &<=/2, empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec min_by(
  t(),
  (element() -> any()),
  (element(), element() -> boolean()) | module(),
  (-> empty_result)
) :: element() | empty_result
when empty_result: any()
```

Returns the minimal element in the `enumerable` as calculated by the given `fun`.

By default, the comparison is done with the `<=` sorter function. If multiple elements are considered minimal, the first one that was found is returned. If you want the last element considered minimal to be returned, the sorter function should not return true for equal elements.

Calls the provided `empty_fallback` function and returns its value if `enumerable` is empty. The default `empty_fallback` raises `Enum.EmptyError`.

Examples

```
iex> Enum.min_by(["a", "aa", "aaa"], fn x ->
String.length(x) end)
"a"

iex> Enum.min_by(["a", "aa", "aaa", "b", "bbb"],
&String.length/1)
"a"
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. Therefore, if you want to

compare structs, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> users = [
...>   %{name: "Ellis", birthday: ~D[1943-05-11]},
...>   %{name: "Lovelace", birthday: ~D[1815-12-10]},
...>   %{name: "Turing", birthday: ~D[1912-06-23]}
...> ]
iex> Enum.min_by(users, &(&1.birthday), Date)
%{name: "Lovelace", birthday: ~D[1815-12-10]}
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.min_by([], &String.length/1, fn -> nil end)
nil
```

min_max(enumerable, sorter_or_empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec min_max(t(), (element(), element() -> boolean()) |
module()) ::
  {element(), element()}
@spec min_max(t(), (-> empty_result)) :: {element(),
element()} | empty_result
when empty_result: any()
```

Returns a tuple with the minimal and the maximal elements in the enumerable.

By default, the comparison is done with the `<=` sorter function, as the function must not return true for equal elements.

Examples

```
iex> Enum.min_max([2, 3, 1])
{1, 3}

iex> Enum.min_max(["foo", "bar", "baz"])
{"bar", "foo"}

iex> Enum.min_max([], fn -> {nil, nil} end)
{nil, nil}
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. Therefore, if you want to compare structs, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> dates = [
...>   ~D[2019-01-01],
...>   ~D[2020-01-01],
...>   ~D[2018-01-01]
...> ]
iex> Enum.min_max(dates, Date)
{~D[2018-01-01], ~D[2020-01-01]}
```

You can also pass a custom sorting function:

```
iex> Enum.min_max([2, 3, 1], &>/2)
{3, 1}
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.min_max([], fn -> nil end)
nil
```

min_max(enumerable, sorter, empty_fallback)

```
@spec min_max(t(), (element(), element() -> boolean()) |
module(), (-> empty_result)) ::
  {element(), element()} | empty_result
when empty_result: any()
```

min_max_by(enumerable, fun, sorter_or_empty_fallback \\ &</2, empty_fallback \\ fn -> raise Enum.EmptyError end)

```
@spec min_max_by(
  t(),
  (element() -> any()),
  (element(), element() -> boolean()) | module(),
  (-> empty_result)
) :: {element(), element()} | empty_result
when empty_result: any()
```

Returns a tuple with the minimal and the maximal elements in the enumerable as calculated by the given function.

By default, the comparison is done with the `<=` sorter function, as the function must not return `true` for equal elements.

Examples

```
iex> Enum.min_max_by(["aaa", "bb", "c"], fn x ->
String.length(x) end)
{"c", "aaa"}

iex> Enum.min_max_by(["aaa", "a", "bb", "c", "ccc"],
&String.length/1)
{"a", "aaa"}

iex> Enum.min_max_by([], &String.length/1, fn -> {nil,
nil} end)
{nil, nil}
```

The fact this function uses Erlang's term ordering means that the comparison is structural and not semantic. Therefore, if you want to compare structs, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> users = [
...>   %{name: "Ellis", birthday: ~D[1943-05-11]},
...>   %{name: "Lovelace", birthday: ~D[1815-12-10]},
...>   %{name: "Turing", birthday: ~D[1912-06-23]}
...> ]
iex> Enum.min_max_by(users, &(&1.birthday), Date)
{
  %{name: "Lovelace", birthday: ~D[1815-12-10]},
```

```
  %{name: "Ellis", birthday: ~D[1943-05-11]}
]
```

Finally, if you don't want to raise on empty enumerables, you can pass the empty fallback:

```
iex> Enum.min_max_by([], &String.length/1, fn -> nil end)
nil
```

product(enumerable)

(since 1.12.0)

```
@spec product(t()) :: number()
```

Returns the product of all elements.

Raises `ArithmeticError` if `enumerable` contains a non-numeric value.

If you need to apply a transformation first, consider using `Enum.product_by/2` instead.

Examples

```
iex> Enum.product([])
1
iex> Enum.product([2, 3, 4])
24
iex> Enum.product([2.0, 3.0, 4.0])
24.0
```

product_by(enumerable, mapper)

(since 1.18.0)

```
@spec product_by(t(), (element() -> number())) :: number()
```

Maps and computes the product of the given `enumerable` in one pass.

Raises `ArithmeticError` if `mapper` returns a non-numeric value.

Examples

```
iex> Enum.product_by([%{count: 2}, %{count: 4}, %{count:
3}], fn x -> x.count end)
24

iex> Enum.product_by(1..3, fn x -> x ** 2 end)
36

iex> Enum.product_by([], fn x -> x.count end)
1
```

Filtering can be achieved by returning `1` to ignore elements:

```
iex> Enum.product_by([2, -1, 3], fn x -> if x > 0, do: x,
else: 1 end)
6
```

random(enumerable)

```
@spec random(t()) :: element()
```

Returns a random element of an `enumerable`.

Raises `Enum.EmptyError` if `enumerable` is empty.

This function uses Erlang's `:rand` module to calculate the random value. Check its documentation for setting a different random algorithm or a different seed.

If a range is passed into the function, this function will pick a random value between the range limits, without traversing the whole range (thus executing in constant time and constant memory).

Examples

The examples below use the `:exsss` pseudorandom algorithm since it's the default from Erlang/OTP 22:

```
# Although not necessary, let's seed the random algorithm
iex> :rand.seed(:exsss, {100, 101, 102})
iex> Enum.random([1, 2, 3])
2
iex> Enum.random([1, 2, 3])
1
iex> Enum.random(1..1_000)
309
```

Implementation

The random functions in this module implement reservoir sampling, which allows them to sample infinite collections. In particular, we implement Algorithm L, as described in by Kim-Hung Li in "Reservoir-Sampling Algorithms of Time Complexity O(n(1+log(N/n)))".

reduce(enumerable, fun)

```
@spec reduce(t(), (element(), acc() -> acc())) :: acc()
```

Invokes `fun` for each element in the `enumerable` with the accumulator.

Raises `Enum.EmptyError` if `enumerable` is empty.

The first element of the `enumerable` is used as the initial value of the accumulator. Then, the function is invoked with the next element and the accumulator. The result returned by the function is used as the accumulator for the next iteration, recursively. When the `enumerable` is done, the last accumulator is returned.

Since the first element of the enumerable is used as the initial value of the accumulator, `fun` will only be executed `n - 1` times where `n` is the length of the enumerable. This function won't call the specified function for enumerables that are one-element long.

If you wish to use another value for the accumulator, use `Enum.reduce/3`.

Examples

```
iex> Enum.reduce([1, 2, 3, 4], fn x, acc -> x * acc end)
24
```

reduce(enumerable, acc, fun)

```
@spec reduce(t(), acc(), (element(), acc() -> acc())) ::
acc()
```

Invokes `fun` for each element in the `enumerable` with the accumulator.

The initial value of the accumulator is `acc`. The function is invoked for each element in the enumerable with the accumulator. The result returned by the function is used as the accumulator for the next iteration. The function returns the last accumulator.

Examples

```
iex> Enum.reduce([1, 2, 3], 0, fn x, acc -> x + acc end)
6

iex> Enum.reduce(%{a: 2, b: 3, c: 4}, 0, fn {_key, val},
acc -> acc + val end)
9
```

Reduce as a building block

Reduce (sometimes called `fold`) is a basic building block in functional programming. Almost all of the functions in the `Enum` module can be implemented on top of reduce. Those functions often rely on other operations, such as `Enum.reverse/1`, which are optimized by the runtime.

For example, we could implement `map/2` in terms of `reduce/3` as follows:

```
def my_map(enumerable, fun) do
  enumerable
  |> Enum.reduce([], fn x, acc -> [fun.(x) | acc] end)
```

```
    |> Enum.reverse()
  end
```

In the example above, `Enum.reduce/3` accumulates the result of each call to `fun` into a list in reverse order, which is correctly ordered at the end by calling `Enum.reverse/1`.

Implementing functions like `map/2`, `filter/2` and others are a good exercise for understanding the power behind `Enum.reduce/3`. When an operation cannot be expressed by any of the functions in the `Enum` module, developers will most likely resort to `reduce/3`.

reduce_while(enumerable, acc, fun)

```
@spec reduce_while(t(), any(), (element(), any() -> {:cont,
any()} | {:halt, any()})) ::
  any()
```

Reduces `enumerable` until `fun` returns `{:halt, term}`.

The return value for `fun` is expected to be

- `{:cont, acc}` to continue the reduction with `acc` as the new accumulator or
- `{:halt, acc}` to halt the reduction

If `fun` returns `{:halt, acc}` the reduction is halted and the function returns `acc`. Otherwise, if the enumerable is exhausted, the function returns the accumulator of the last `{:cont, acc}`.

Examples

```
iex> Enum.reduce_while(1..100, 0, fn x, acc ->
...>   if x < 5 do
...>     {:cont, acc + x}
...>   else
...>     {:halt, acc}
...>   end
...> end)
10
iex> Enum.reduce_while(1..100, 0, fn x, acc ->
...>   if x > 0 do
...>     {:cont, acc + x}
...>   else
...>     {:halt, acc}
...>   end
...> end)
5050
```

reject(enumerable, fun)

```
@spec reject(t(), (element() -> as boolean(term()))) ::
list()
```

Returns a list of elements in `enumerable` excluding those for which the function `fun` returns a truthy value.

See also `filter/2`.

Examples

```
iex> Enum.reject([1, 2, 3], fn x -> rem(x, 2) == 0 end)
[1, 3]
```

reverse(enumerable)

```
@spec reverse(t()) :: list()
```

Returns a list of elements in `enumerable` in reverse order.

Examples

```
iex> Enum.reverse([1, 2, 3])
[3, 2, 1]
```

reverse(enumerable, tail)

```
@spec reverse(t(), t()) :: list()
```

Reverses the elements in `enumerable`, appends the `tail`, and returns it as a list.

This is an optimization for `enumerable |> Enum.reverse() |> Enum.concat(tail)`.

Examples

```
iex> Enum.reverse([1, 2, 3], [4, 5, 6])
[3, 2, 1, 4, 5, 6]
```

reverse_slice(enumerable, start_index, count)

```
@spec reverse_slice(t(), non neg integer(),
non neg integer()) :: list()
```

Reverses the `enumerable` in the range from initial `start_index` through `count` elements.

If `count` is greater than the size of the rest of the `enumerable`, then this function will reverse the rest of the enumerable.

Examples

```
iex> Enum.reverse_slice([1, 2, 3, 4, 5, 6], 2, 4)
[1, 2, 6, 5, 4, 3]
```

scan(enumerable, fun)

```
@spec scan(t(), (element(), any() -> any())) :: list()
```

Passes each element from `enumerable` to the `fun` as the first argument, stores the `fun` result in a list and passes the result as the second argument for the next computation.

The `fun` isn't applied for the first element of the `enumerable`, the element is taken as it is.

Examples

```
iex> Enum.scan(["a", "b", "c", "d", "e"], fn element, acc
-> element <> String.first(acc) end)
["a", "ba", "cb", "dc", "ed"]

iex> Enum.scan(1..5, fn element, acc -> element + acc
end)
[1, 3, 6, 10, 15]
```

scan(enumerable, acc, fun)

```
@spec scan(t(), any(), (element(), any() -> any())) :: list()
```

Passes each element from `enumerable` to the `fun` as the first argument, stores the `fun` result in a list and passes the result as the second argument for the next computation.

Passes the given `acc` as the second argument for the `fun` with the first element.

Examples

```
iex> Enum.scan(["a", "b", "c", "d", "e"], "_", fn
element, acc -> element <> String.first(acc) end)
["a_", "ba", "cb", "dc", "ed"]

iex> Enum.scan(1..5, 0, fn element, acc -> element + acc
end)
[1, 3, 6, 10, 15]
```

shuffle(enumerable)

```
@spec shuffle(t()) :: list()
```

Returns a list with the elements of `enumerable` shuffled.

This function uses Erlang's `:rand` module to calculate the random value. Check its documentation for setting a different random algorithm or a different seed.

Examples

The examples below use the `:exsss` pseudorandom algorithm since it's the default from Erlang/OTP 22:

```
# Although not necessary, let's seed the random algorithm
iex> :rand.seed(:exsss, {11, 22, 33})
iex> Enum.shuffle([1, 2, 3])
[2, 1, 3]
iex> Enum.shuffle([1, 2, 3])
[2, 3, 1]
```

slice(enumerable, index_range)

(since 1.6.0)

```
@spec slice(t(), Range.t()) :: list()
```

Returns a subset list of the given `enumerable` by `index_range`.

`index_range` must be a `Range`. Given an `enumerable`, it drops elements before `index_range.first` (zero-base), then it takes elements until element `index_range.last` (inclusively).

Indexes are normalized, meaning that negative indexes will be counted from the end (for example, `-1` means the last element of the `enumerable`).

If `index_range.last` is out of bounds, then it is assigned as the index of the last element.

If the normalized `index_range.first` is out of bounds of the given `enumerable`, or this one is greater than the normalized `index_range.last`, then `[]` is returned.

If a step `n` (other than `1`) is used in `index_range`, then it takes every `n`th element from `index_range.first` to `index_range.last` (according to the same rules described above).

Examples

```
iex> Enum.slice([1, 2, 3, 4, 5], 1..3)
[2, 3, 4]

iex> Enum.slice([1, 2, 3, 4, 5], 3..10)
[4, 5]

# Last three elements (negative indexes)
iex> Enum.slice([1, 2, 3, 4, 5], -3..-1)
[3, 4, 5]
```

For ranges where `start > stop`, you need to explicit mark them as increasing:

```
iex> Enum.slice([1, 2, 3, 4, 5], 1..-2//1)
[2, 3, 4]
```

The step can be any positive number. For example, to get every 2 elements of the collection:

```
iex> Enum.slice([1, 2, 3, 4, 5], 0..-1//2)
[1, 3, 5]
```

To get every third element of the first ten elements:

```
iex> integers = Enum.to_list(1..20)
iex> Enum.slice(integers, 0..9//3)
[1, 4, 7, 10]
```

If the first position is after the end of the enumerable or after the last position of the range, it returns an empty list:

```
iex> Enum.slice([1, 2, 3, 4, 5], 6..10)
[]

# first is greater than last
iex> Enum.slice([1, 2, 3, 4, 5], 6..5//1)
[]
```

slice(enumerable, start_index, amount)

```
@spec slice(t(), index(), non_neg_integer()) :: list()
```

Returns a subset list of the given `enumerable`, from `start_index` (zero-based) with `amount` number of elements if available.

Given an `enumerable`, it drops elements right before element `start_index`; then, it takes `amount` of elements, returning as many elements as possible if there are not enough elements.

A negative `start_index` can be passed, which means the `enumerable` is enumerated once and the index is counted from the end (for example, `-1` starts slicing from the last element).

It returns `[]` if `amount` is `0` or if `start_index` is out of bounds.

Examples

```
iex> Enum.slice(1..100, 5, 10)
[6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

# amount to take is greater than the number of elements
iex> Enum.slice(1..10, 5, 100)
[6, 7, 8, 9, 10]

iex> Enum.slice(1..10, 5, 0)
[]

# using a negative start index
iex> Enum.slice(1..10, -6, 3)
[5, 6, 7]
iex> Enum.slice(1..10, -11, 5)
[1, 2, 3, 4, 5]

# out of bound start index
```

```
iex> Enum.slice(1..10, 10, 5)
[]
```

slide(enumerable, range_or_single_index, insertion_index)

(since 1.13.0)

```
@spec slide(t(), Range.t() | index(), index()) :: list()
```

Slides a single or multiple elements given by `range_or_single_index` from `enumerable` to `insertion_index`.

The semantics of the range to be moved match the semantics of `Enum.slice/2`. Specifically, that means:

- Indices are normalized, meaning that negative indexes will be counted from the end (for example, -1 means the last element of the enumerable). This will result in *two* traversals of your enumerable on types like lists that don't provide a constant-time count.
- If the normalized index range's `last` is out of bounds, the range is truncated to the last element.
- If the normalized index range's `first` is out of bounds, the selected range for sliding will be empty, so you'll get back your input list.
- Decreasing ranges (such as `5..0//1`) also select an empty range to be moved, so you'll get back your input list.
- Ranges with any step but 1 will raise an error.

Examples

```
# Slide a single element
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], 5, 1)
[:a, :f, :b, :c, :d, :e, :g]

# Slide a range of elements towards the head of the list
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], 3..5, 1)
[:a, :d, :e, :f, :b, :c, :g]

# Slide a range of elements towards the tail of the list
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], 1..3, 5)
[:a, :e, :f, :b, :c, :d, :g]

# Slide with negative indices (counting from the end)
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], 3..-1//1,
2)
[:a, :b, :d, :e, :f, :g, :c]
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], -4..-2, 1)
[:a, :d, :e, :f, :b, :c, :g]

# Insert at negative indices (counting from the end)
iex> Enum.slide([:a, :b, :c, :d, :e, :f, :g], 3, -1)
[:a, :b, :c, :e, :f, :g, :d]
```

sort(enumerable)

```
@spec sort(t()) :: list()
```

Sorts the `enumerable` according to Erlang's term ordering.

This function uses the merge sort algorithm. Do not use this function to sort structs, see `sort/2` for more information.

Examples

```
iex> Enum.sort([3, 2, 1])
[1, 2, 3]
```

sort(enumerable, sorter)

```
@spec sort(
  t(),
  (element(), element() -> boolean())
  | :asc
  | :desc
  | module()
  | {:asc | :desc, module()}
) :: list()
```

Sorts the `enumerable` by the given function.

This function uses the merge sort algorithm. The given function should compare two arguments, and return `true` if the first argument precedes or is in the same place as the second one.

Examples

```
iex> Enum.sort([1, 2, 3], &(&1 >= &2))
[3, 2, 1]
```

The sorting algorithm will be stable as long as the given function returns `true` for values considered equal:

```
iex> Enum.sort(["some", "kind", "of", "monster"], &
(byte_size(&1) <= byte_size(&2)))
```

```
["of", "some", "kind", "monster"]
```

If the function does not return `true` for equal values, the sorting is not stable and the order of equal terms may be shuffled. For example:

```
iex> Enum.sort(["some", "kind", "of", "monster"], &
(byte_size(&1) < byte_size(&2)))
["of", "kind", "some", "monster"]
```

Ascending and descending (since v1.10.0)

`sort/2` allows a developer to pass `:asc` or `:desc` as the sorter, which is a convenience for `&<=/2` and `&>=/2` respectively.

```
iex> Enum.sort([2, 3, 1], :asc)
[1, 2, 3]
iex> Enum.sort([2, 3, 1], :desc)
[3, 2, 1]
```

Sorting structs

Do not use `</2`, `<=/2`, `>/2`, `>=/2` and friends when sorting structs. That's because the built-in operators above perform structural comparison and not a semantic one. Imagine we sort the following list of dates:

```
iex> dates = [~D[2019-01-01], ~D[2020-03-02], ~D[2019-06-
06]]
```

```
iex> Enum.sort(dates)
[~D[2019-01-01], ~D[2020-03-02], ~D[2019-06-06]]
```

Note that the returned result is incorrect, because `sort/1` by default uses `<=/2`, which will compare their structure. When comparing structures, the fields are compared in alphabetical order, which means the dates above will be compared by `day`, `month` and then `year`, which is the opposite of what we want.

For this reason, most structs provide a "compare" function, such as `Date.compare/2`, which receives two structs and returns `:lt` (less-than), `:eq` (equal to), and `:gt` (greater-than). If you pass a module as the sorting function, Elixir will automatically use the `compare/2` function of said module:

```
iex> dates = [~D[2019-01-01], ~D[2020-03-02], ~D[2019-06-06]]
iex> Enum.sort(dates, Date)
[~D[2019-01-01], ~D[2019-06-06], ~D[2020-03-02]]
```

To retrieve all dates in descending order, you can wrap the module in a tuple with `:asc` or `:desc` as first element:

```
iex> dates = [~D[2019-01-01], ~D[2020-03-02], ~D[2019-06-06]]
iex> Enum.sort(dates, {:asc, Date})
[~D[2019-01-01], ~D[2019-06-06], ~D[2020-03-02]]
iex> dates = [~D[2019-01-01], ~D[2020-03-02], ~D[2019-06-06]]
iex> Enum.sort(dates, {:desc, Date})
[~D[2020-03-02], ~D[2019-06-06], ~D[2019-01-01]]
```

sort_by(enumerable, mapper, sorter \\ :asc)

```
@spec sort_by(
  t(),
  (element() -> mapped_element),
  (element(), element() -> boolean())
  | :asc
  | :desc
  | module()
  | {:asc | :desc, module()}
) :: list()
when mapped_element: element()
```

Sorts the mapped results of the `enumerable` according to the provided `sorter` function.

This function maps each element of the `enumerable` using the provided `mapper` function. The enumerable is then sorted by the mapped elements using the `sorter`, which defaults to `:asc` and sorts the elements ascendingly.

`sort_by/3` differs from `sort/2` in that it only calculates the comparison value for each element in the enumerable once instead of once for each element in each comparison. If the same function is being called on both elements, it's more efficient to use `sort_by/3`.

Ascending and descending (since v1.10.0)

`sort_by/3` allows a developer to pass `:asc` or `:desc` as the sorter, which is a convenience for `&<=/2` and `&>=/2` respectively:

```
iex> Enum.sort_by([2, 3, 1], &(&1), :asc)
[1, 2, 3]
```

```
iex> Enum.sort_by([2, 3, 1], &(&1), :desc)
[3, 2, 1]
```

Examples

Using the default `sorter` of `:asc` :

```
iex> Enum.sort_by(["some", "kind", "of", "monster"],
&byte_size/1)
["of", "some", "kind", "monster"]
```

Sorting by multiple properties - first by size, then by first letter (this takes advantage of the fact that tuples are compared element-by-element):

```
iex> Enum.sort_by(["some", "kind", "of", "monster"], &
{byte_size(&1), String.first(&1)})
["of", "kind", "some", "monster"]
```

Similar to `sort/2`, you can pass a custom sorter:

```
iex> Enum.sort_by(["some", "kind", "of", "monster"],
&byte_size/1, :desc)
["monster", "some", "kind", "of"]
```

As in `sort/2`, avoid using the default sorting function to sort structs, as by default it performs structural comparison instead of a semantic one. In such cases, you shall pass a sorting function as third element or any module that implements a `compare/2` function. For example, to sort

users by their birthday in both ascending and descending order respectively:

```
iex> users = [
...>   %{name: "Ellis", birthday: ~D[1943-05-11]},
...>   %{name: "Lovelace", birthday: ~D[1815-12-10]},
...>   %{name: "Turing", birthday: ~D[1912-06-23]}
...> ]
iex> Enum.sort_by(users, &(&1.birthday), Date)
[
  %{name: "Lovelace", birthday: ~D[1815-12-10]},
  %{name: "Turing", birthday: ~D[1912-06-23]},
  %{name: "Ellis", birthday: ~D[1943-05-11]}
]
iex> Enum.sort_by(users, &(&1.birthday), {:desc, Date})
[
  %{name: "Ellis", birthday: ~D[1943-05-11]},
  %{name: "Turing", birthday: ~D[1912-06-23]},
  %{name: "Lovelace", birthday: ~D[1815-12-10]}
]
```

Performance characteristics

As detailed in the initial section, `sort_by/3` calculates the comparison value for each element in the enumerable once instead of once for each element in each comparison. This implies `sort_by/3` must do an initial pass on the data to compute those values.

However, if those values are cheap to compute, for example, you have already extracted the field you want to sort by into a tuple, then those extra passes become overhead. In such cases, consider using `List.keysort/3` instead.

Let's see an example. Imagine you have a list of products and you have a list of IDs. You want to keep all products that are in the given IDs and return their names sorted by their price. You could write it like this:

```
for(
  product <- products,
  product.id in ids,
  do: product
)
|> Enum.sort_by(& &1.price)
|> Enum.map(& &1.name)
```

However, you could also write it like this:

```
for(
  product <- products,
  product.id in ids,
  do: {product.name, product.price}
)
|> List.keysort(1)
|> Enum.map(&elem(&1, 0))
```

Using `List.keysort/3` will be a better choice for performance sensitive code as it avoids additional traversals.

split(enumerable, count)

```
@spec split(t(), integer()) :: {list(), list()}
```

Splits the `enumerable` into two enumerables, leaving `count` elements in the first one.

If `count` is a negative number, it starts counting from the back to the beginning of the `enumerable`.

Be aware that a negative `count` implies the `enumerable` will be enumerated twice: once to calculate the position, and a second time to do the actual splitting.

Examples

```
iex> Enum.split([1, 2, 3], 2)
{[1, 2], [3]}

iex> Enum.split([1, 2, 3], 10)
{[1, 2, 3], []}

iex> Enum.split([1, 2, 3], 0)
{[], [1, 2, 3]}

iex> Enum.split([1, 2, 3], -1)
{[1, 2], [3]}

iex> Enum.split([1, 2, 3], -5)
{[], [1, 2, 3]}
```

split_while(enumerable, fun)

```
@spec split_while(t(), (element() -> as boolean(term()))) ::
{list(), list()}
```

Splits enumerable in two at the position of the element for which `fun` returns a falsy value (`false` or `nil`) for the first time.

It returns a two-element tuple with two lists of elements. The element that triggered the split is part of the second list.

Examples

```
iex> Enum.split_while([1, 2, 3, 4], fn x -> x < 3 end)
{[1, 2], [3, 4]}

iex> Enum.split_while([1, 2, 3, 4], fn x -> x < 0 end)
{[], [1, 2, 3, 4]}

iex> Enum.split_while([1, 2, 3, 4], fn x -> x > 0 end)
{[1, 2, 3, 4], []}
```

split_with(enumerable, fun)

(since 1.4.0)

```
@spec split_with(t(), (element() -> as boolean(term()))) ::
{list(), list()}
```

Splits the `enumerable` in two lists according to the given function `fun`.

Splits the given `enumerable` in two lists by calling `fun` with each element in the `enumerable` as its only argument. Returns a tuple with the first list containing all the elements in `enumerable` for which applying `fun` returned a truthy value, and a second list with all the elements for which applying `fun` returned a falsy value (`false` or `nil`).

The elements in both the returned lists are in the same relative order as they were in the original enumerable (if such enumerable was ordered, like a list). See the examples below.

Examples

```
iex> Enum.split_with([5, 4, 3, 2, 1, 0], fn x -> rem(x,
2) == 0 end)
{[4, 2, 0], [5, 3, 1]}

iex> Enum.split_with([a: 1, b: -2, c: 1, d: -3], fn {_k,
v} -> v < 0 end)
{[b: -2, d: -3], [a: 1, c: 1]}

iex> Enum.split_with([a: 1, b: -2, c: 1, d: -3], fn {_k,
v} -> v > 50 end)
{[], [a: 1, b: -2, c: 1, d: -3]}

iex> Enum.split_with([], fn {_k, v} -> v > 50 end)
{[], []}
```

sum(enumerable)

```
@spec sum(t()) :: number()
```

Returns the sum of all elements.

Raises `ArithmeticError` if `enumerable` contains a non-numeric value.

If you need to apply a transformation first, consider using `Enum.sum_by/2` instead.

Examples

```
iex> Enum.sum([1, 2, 3])
6
```

```
iex> Enum.sum(1..10)
55

iex> Enum.sum(1..10//2)
25
```

sum_by(enumerable, mapper)

(since 1.18.0)

```
@spec sum_by(t(), (element() -> number())) :: number()
```

Maps and sums the given `enumerable` in one pass.

Raises `ArithmeticError` if `mapper` returns a non-numeric value.

Examples

```
iex> Enum.sum_by([%{count: 1}, %{count: 2}, %{count: 3}],
fn x -> x.count end)
6

iex> Enum.sum_by(1..3, fn x -> x ** 2 end)
14

iex> Enum.sum_by([], fn x -> x.count end)
0
```

Filtering can be achieved by returning `0` to ignore elements:

```
iex> Enum.sum_by([1, -2, 3], fn x -> if x > 0, do: x,
else: 0 end)
4
```

take(enumerable, amount)

```
@spec take(t(), integer()) :: list()
```

Takes an `amount` of elements from the beginning or the end of the `enumerable`.

If a positive `amount` is given, it takes the `amount` elements from the beginning of the `enumerable`.

If a negative `amount` is given, the `amount` of elements will be taken from the end. The `enumerable` will be enumerated once to retrieve the proper index and the remaining calculation is performed from the end.

If amount is `0`, it returns `[]`.

Examples

```
iex> Enum.take([1, 2, 3], 2)
[1, 2]

iex> Enum.take([1, 2, 3], 10)
[1, 2, 3]

iex> Enum.take([1, 2, 3], 0)
[]
```

```
iex> Enum.take([1, 2, 3], -1)
[3]
```

take_every(enumerable, nth)

```
@spec take_every(t(), non_neg_integer()) :: list()
```

Returns a list of every `nth` element in the `enumerable`, starting with the first element.

The first element is always included, unless `nth` is 0.

The second argument specifying every `nth` element must be a non-negative integer.

Examples

```
iex> Enum.take_every(1..10, 2)
[1, 3, 5, 7, 9]

iex> Enum.take_every(1..10, 0)
[]

iex> Enum.take_every([1, 2, 3], 1)
[1, 2, 3]
```

take_random(enumerable, count)

```
@spec take_random(t(), non_neg_integer()) :: list()
```

Takes `count` random elements from `enumerable`.

Note that this function will traverse the whole `enumerable` to get the random sublist.

See `random/1` for notes on implementation and random seed.

Examples

```
# Although not necessary, let's seed the random algorithm
iex> :rand.seed(:exsss, {1, 2, 3})
iex> Enum.take_random(1..10, 2)
[6, 1]
iex> Enum.take_random(?a..?z, 5)
~c"bkzmt"
```

take_while(enumerable, fun)

```
@spec take_while(t(), (element() -> as boolean(term()))) ::
list()
```

Takes the elements from the beginning of the `enumerable` while `fun` returns a truthy value.

Examples

```
iex> Enum.take_while([1, 2, 3], fn x -> x < 3 end)
[1, 2]
```

to_list(enumerable)

```
@spec to_list(t()) :: [element()]
```

Converts `enumerable` to a list.

Examples

```
iex> Enum.to_list(1..3)
[1, 2, 3]
```

uniq(enumerable)

```
@spec uniq(t()) :: list()
```

Enumerates the `enumerable`, removing all duplicate elements.

The first occurrence of each element is kept and all following duplicates are removed. The overall order is preserved.

Examples

```
iex> Enum.uniq([1, 2, 3, 3, 2, 1])
[1, 2, 3]
```

uniq_by(enumerable, fun)

```
@spec uniq_by(t(), (element() -> term())) :: list()
```

Enumerates the `enumerable`, by removing the elements for which function `fun` returned duplicate elements.

The function `fun` maps every element to a term. Two elements are considered duplicates if the return value of `fun` is equal for both of them.

The first occurrence of each element is kept and all following duplicates are removed. The overall order is preserved.

Example

```
iex> Enum.uniq_by([{1, :x}, {2, :y}, {1, :z}], fn {x, _}
-> x end)
[{1, :x}, {2, :y}]

iex> Enum.uniq_by([a: {:tea, 2}, b: {:tea, 2}, c:
{:coffee, 1}], fn {_, y} -> y end)
[a: {:tea, 2}, c: {:coffee, 1}]
```

unzip(enumerable)

```
@spec unzip(t()) :: {[element()], [element()]}
```

Opposite of `zip/2`. Extracts two-element tuples from the given `enumerable` and groups them together.

It takes an `enumerable` with elements being two-element tuples and returns a tuple with two lists, each of which is formed by the first and second element of each tuple, respectively.

This function fails unless `enumerable` is or can be converted into a list of tuples with *exactly* two elements in each tuple.

Examples

```
iex> Enum.unzip([{:a, 1}, {:b, 2}, {:c, 3}])
{[:a, :b, :c], [1, 2, 3]}
```

with_index(enumerable, fun_or_offset \\ 0)

```
@spec with_index(t(), integer()) :: [{term(), integer()}]
@spec with_index(t(), (element(), index() -> value)) ::
[value] when value: any()
```

Returns the `enumerable` with each element wrapped in a tuple alongside its index or according to a given function.

If an integer offset is given as `fun_or_offset`, it will index from the given offset instead of from zero.

If a 2-arity function is given as `fun_or_offset`, the function will be invoked for each element in `enumerable` as the first argument and with a zero-based index as the second. `with_index/2` returns a list with the result of each invocation.

Examples

```
iex> Enum.with_index([:a, :b, :c])
[a: 0, b: 1, c: 2]

iex> Enum.with_index([:a, :b, :c], 3)
[a: 3, b: 4, c: 5]
```

```
iex> Enum.with_index([:a, :b, :c], fn element, index ->
{index, element} end)
[{0, :a}, {1, :b}, {2, :c}]
```

zip(enumerables)

(since 1.4.0)

```
@spec zip(enumerables) :: [tuple()] when enumerables: [t()] |
t()
```

Zips corresponding elements from a finite collection of enumerables into a list of tuples.

The zipping finishes as soon as any enumerable in the given collection completes.

Examples

```
iex> Enum.zip([[1, 2, 3], [:a, :b, :c], ["foo", "bar",
"baz"]])
[{1, :a, "foo"}, {2, :b, "bar"}, {3, :c, "baz"}]

iex> Enum.zip([[1, 2, 3, 4, 5], [:a, :b, :c]])
[{1, :a}, {2, :b}, {3, :c}]
```

zip(enumerable1, enumerable2)

```
@spec zip(t(), t()) :: [{any(), any()}]
```

Zips corresponding elements from two enumerables into a list of tuples.

Because a list of two-element tuples with atoms as the first tuple element is a keyword list (`Keyword`), zipping a first list of atoms with a second list of any kind creates a keyword list.

The zipping finishes as soon as either enumerable completes.

Examples

```
iex> Enum.zip([1, 2, 3], [:a, :b, :c])
[{1, :a}, {2, :b}, {3, :c}]

iex> Enum.zip([:a, :b, :c], [1, 2, 3])
[a: 1, b: 2, c: 3]

iex> Enum.zip([1, 2, 3, 4, 5], [:a, :b, :c])
[{1, :a}, {2, :b}, {3, :c}]
```

zip_reduce(enumerables, acc, reducer)

(since 1.12.0)

```
@spec zip_reduce(t(), acc, ([term()], acc -> acc)) :: acc
when acc: term()
```

Reduces over all of the given enumerables, halting as soon as any enumerable is empty.

The reducer will receive 2 args: a list of elements (one from each enum) and the accumulator.

In practice, the behavior provided by this function can be achieved with:

```
Enum.reduce(Stream.zip(enums), acc, reducer)
```

But `zip_reduce/3` exists for convenience purposes.

Examples

```
iex> enums = [[1, 1], [2, 2], [3, 3]]
...>  Enum.zip_reduce(enums, [], fn elements, acc ->
...>    [List.to_tuple(elements) | acc]
...> end)
[{1, 2, 3}, {1, 2, 3}]
```

If one of the lists has more entries than the others, those entries are discarded:

```
iex> enums = [[1, 2], [a: 3, b: 4], [5, 6, 7]]
...> Enum.zip_reduce(enums, [], fn elements, acc ->
...>   [List.to_tuple(elements) | acc]
...> end)
[{2, {:b, 4}, 6}, {1, {:a, 3}, 5}]
```

zip_reduce(left, right, acc, reducer)

(since 1.12.0)

```
@spec zip_reduce(t(), t(), acc, (enum1_elem :: term(),
enum2_elem :: term(), acc ->
                                 acc)) :: acc
when acc: term()
```

Reduces over two enumerables halting as soon as either enumerable is empty.

In practice, the behavior provided by this function can be achieved with:

```
Enum.reduce(Stream.zip(left, right), acc, reducer)
```

But `zip_reduce/4` exists for convenience purposes.

Examples

```
iex> Enum.zip_reduce([1, 2], [3, 4], 0, fn x, y, acc -> x
+ y + acc end)
10
```

If one of the lists has more entries than the others, those entries are discarded:

```
iex> Enum.zip_reduce([1, 2, 3], [4, 5], [], fn x, y, acc
-> [x + y | acc] end)
[7, 5]
```

zip_with(enumerables, zip_fun)

(since 1.12.0)

```
@spec zip_with(t(), ([term()] -> term())) :: [term()]
```

Zips corresponding elements from a finite collection of enumerables into list, transforming them with the `zip_fun` function as it goes.

The first element from each of the enums in `enumerables` will be put into a list which is then passed to the one-arity `zip_fun` function. Then, the second elements from each of the enums are put into a list and passed

to `zip_fun`, and so on until any one of the enums in `enumerables` runs out of elements.

Returns a list with all the results of calling `zip_fun`.

Examples

```
iex> Enum.zip_with([[1, 2], [3, 4], [5, 6]], fn [x, y, z]
-> x + y + z end)
[9, 12]

iex> Enum.zip_with([[1, 2], [3, 4]], fn [x, y] -> x + y
end)
[4, 6]
```

`zip_with/2` can be used to transpose lists of lists:

```
iex> Enum.zip_with([[1, 2,], [3, 4]], & &1)
[[1, 3], [2, 4]]
```

zip_with(enumerable1, enumerable2, zip_fun)

(since 1.12.0)

```
@spec zip_with(t(), t(), (enum1_elem :: term(), enum2_elem ::
term() -> term())) :: [
  term()
]
```

Zips corresponding elements from two enumerables into a list, transforming them with the `zip_fun` function as it goes.

The corresponding elements from each collection are passed to the provided two-arity `zip_fun` function in turn. Returns a list that contains the result of calling `zip_fun` for each pair of elements.

The zipping finishes as soon as either enumerable runs out of elements.

Zipping Maps

It's important to remember that zipping inherently relies on order. If you zip two lists you get the element at the index from each list in turn. If we zip two maps together it's tempting to think that you will get the given key in the left map and the matching key in the right map, but there is no such guarantee because map keys are not ordered! Consider the following:

```
left = %{:a => 1, 1 => 3}
right = %{:a => 1, :b => :c}
Enum.zip(left, right)
#=> [{{1, 3}, {:a, 1}}, {{:a, 1}, {:b, :c}}]
```

As you can see `:a` does not get paired with `:a`. If this is what you want, you should use `Map.merge/3`.

Examples

```
iex> Enum.zip_with([1, 2], [3, 4], fn x, y -> x + y end)
[4, 6]

iex> Enum.zip_with([1, 2], [3, 4, 5, 6], fn x, y -> x + y
end)
[4, 6]

iex> Enum.zip_with([1, 2, 5, 6], [3, 4], fn x, y -> x + y
```

```
end)
[4, 6]
```

Keyword

A keyword list is a list that consists exclusively of two-element tuples.

The first element of these tuples is known as the *key*, and it must be an atom. The second element, known as the *value*, can be any term.

Keywords are mostly used to work with optional values. For a general introduction to keywords and how they compare with maps, see our Keyword and Maps guide.

Examples

For example, the following is a keyword list:

```
[{:exit_on_close, true}, {:active, :once}, {:packet_size,
1024}]
```

Elixir provides a special and more concise syntax for keyword lists:

```
[exit_on_close: true, active: :once, packet_size: 1024]
```

The two syntaxes return the exact same value.

A *key* can be any atom, consisting of Unicode letters, numbers, an underscore or the `@` sign. If the *key* should have any other characters, such as spaces, you can wrap it in quotes:

```
iex> ["exit on close": true]
["exit on close": true]
```

Wrapping an atom in quotes does not make it a string. Keyword list *keys* are always atoms. Quotes should only be used when necessary or Elixir will issue a warning.

Duplicate keys and ordering

A keyword may have duplicate keys so it is not strictly a key-value data type. However, most of the functions in this module work on a key-value structure and behave similar to the functions you would find in the `Map` module. For example, `Keyword.get/3` will get the first entry matching the given key, regardless if duplicate entries exist. Similarly, `Keyword.put/3` and `Keyword.delete/2` ensure all duplicate entries for a given key are removed when invoked. Note, however, that keyword list operations need to traverse the whole list in order to find keys, so these operations are slower than their map counterparts.

A handful of functions exist to handle duplicate keys, for example, `get_values/2` returns all values for a given key and `delete_first/2` deletes just the first entry of the existing ones.

Even though lists preserve the existing order, the functions in `Keyword` do not guarantee any ordering. For example, if you invoke `Keyword.put(opts, new_key, new_value)`, there is no guarantee for where `new_key` will be added to (the front, the end or anywhere else).

Given ordering is not guaranteed, it is not recommended to pattern match on keyword lists either. For example, a function such as:

```
def my_function([some_key: value, another_key:
another_value])
```

will match

```
my_function([some_key: :foo, another_key: :bar])
```

but it won't match

```
my_function([another_key: :bar, some_key: :foo])
```

Most of the functions in this module work in linear time. This means that the time it takes to perform an operation grows at the same rate as the length of the list.

Call syntax

When keyword lists are passed as the last argument to a function, the square brackets around the keyword list can be omitted. For example, the keyword list syntax:

```
String.split("1-0", "-", [trim: true, parts: 2])
```

can be written without the enclosing brackets whenever it is the last argument of a function call:

```
String.split("1-0", "-", trim: true, parts: 2)
```

Since tuples, lists and maps are treated similarly to function arguments in Elixir syntax, this property is also available to them:

```
iex> {1, 2, foo: :bar}
{1, 2, [{:foo, :bar}]}

iex> [1, 2, foo: :bar]
```

```
[1, 2, {:foo, :bar}]

iex> %{1 => 2, foo: :bar}
%{1 => 2, :foo => :bar}
```

Summary

Types

default()
key()
t()
t(value)
value()

Functions

delete(keywords, key)

Deletes the entries in the keyword list under a specific `key`.

delete_first(keywords, key)

Deletes the first entry in the keyword list under a specific `key`.

drop(keywords, keys)

Drops the given `keys` from the keyword list.

equal?(left, right)

Checks if two keywords are equal.

fetch(keywords, key)

Fetches the value for a specific `key` and returns it in a tuple.

fetch!(keywords, key)

Fetches the value for specific `key`.

filter(keywords, fun)

Returns a keyword list containing only the entries from `keywords` for which the function `fun` returns a truthy value.

from_keys(keys, value)

Builds a keyword from the given `keys` and the fixed `value`.

get(keywords, key, default \\ nil)

Gets the value under the given `key`.

get_and_update(keywords, key, fun)

Gets the value for `key` and updates it in one pass, deleting duplicate keys.

get_and_update!(keywords, key, fun)

Gets the value for `key` and updates it in one pass, deleting duplicate keys, raising if `key` can't be found in `keywords`.

get_lazy(keywords, key, fun)

Gets the value under the given `key`.

get_values(keywords, key)

Gets all values under a specific `key`.

has_key?(keywords, key)

Returns whether a given `key` exists in the given `keywords`.

intersect(keyword1, keyword2, fun \\ fn _key, v1, v2 -> v2 end)

Intersects two keyword lists, returning a keyword with the common keys.

keys(keywords)

Returns all keys from the keyword list.

keyword?(term)

Returns `true` if `term` is a keyword list, otherwise `false`.

merge(keywords1, keywords2)

Merges two keyword lists into one.

merge(keywords1, keywords2, fun)

Merges two keyword lists into one.

new()

Returns an empty keyword list, i.e. an empty list.

new(pairs)

Creates a keyword list from an enumerable.

new(pairs, transform)

Creates a keyword list from an enumerable via the transformation function.

pop(keywords, key, default \\ nil)

Returns the first value for `key` and removes all associated entries in the keyword list.

pop!(keywords, key)

Returns the first value for `key` and removes all associated entries in the keyword list, raising if `key` is not present.

pop_first(keywords, key, default \\ nil)

Returns and removes the first value associated with `key` in the keyword list.

pop_lazy(keywords, key, fun)

Lazily returns and removes all values associated with `key` in the keyword list.

pop_values(keywords, key)

Returns all values for `key` and removes all associated entries in the keyword list.

put(keywords, key, value)

Puts the given `value` under the specified `key`.

put_new(keywords, key, value)

Puts the given `value` under `key`, unless the entry `key` already exists.

put_new_lazy(keywords, key, fun)

Evaluates `fun` and puts the result under `key` in keyword list unless `key` is already present.

reject(keywords, fun)

Returns a keyword list excluding the entries from `keywords` for which the function `fun` returns a truthy value.

replace(keywords, key, value)

Puts a value under `key` only if the `key` already exists in `keywords`.

replace!(keywords, key, value)

Puts a value under `key` only if the `key` already exists in `keywords`.

replace_lazy(keywords, key, fun)

Replaces the value under `key` using the given function only if `key` already exists in `keywords`.

split(keywords, keys)

Takes all entries corresponding to the given `keys` and extracts them into a separate keyword list.

split_with(keywords, fun)

Splits the `keywords` into two keyword lists according to the given function `fun`.

take(keywords, keys)

Takes all entries corresponding to the given `keys` and returns them as a new keyword list.

to_list(keywords)

Returns the keyword list itself.

update(keywords, key, default, fun)

Updates the value under `key` in `keywords` using the given function.

update!(keywords, key, fun)

Updates the value under `key` using the given function.

validate(keyword, values)

Ensures the given `keyword` has only the keys given in `values`.

validate!(keyword, values)

Similar to `validate/2` but returns the keyword or raises an error.

values(keywords)

Returns all values from the keyword list.

Types

default()

(since 1.17.0)

```
@type default() :: any()
```

key()

```
@type key() :: atom()
```

t()

```
@type t() :: [{key(), value()}]
```

t(value)

```
@type t(value) :: [{key(), value}]
```

value()

```
@type value() :: any()
```

Functions

delete(keywords, key)

```
@spec delete(t(), key()) :: t()
```

Deletes the entries in the keyword list under a specific `key`.

If the `key` does not exist, it returns the keyword list unchanged. Use `delete_first/2` to delete just the first entry in case of duplicate keys.

Examples

```
iex> Keyword.delete([a: 1, b: 2], :a)
[b: 2]
iex> Keyword.delete([a: 1, b: 2, a: 3], :a)
[b: 2]
iex> Keyword.delete([b: 2], :a)
[b: 2]
```

delete_first(keywords, key)

```
@spec delete_first(t(), key()) :: t()
```

Deletes the first entry in the keyword list under a specific `key`.

If the `key` does not exist, it returns the keyword list unchanged.

Examples

```
iex> Keyword.delete_first([a: 1, b: 2, a: 3], :a)
[b: 2, a: 3]

iex> Keyword.delete_first([a: 1, b: 2, b: 3], :b)
[a: 1, b: 3]

iex> Keyword.delete_first([b: 2], :a)
[b: 2]
```

drop(keywords, keys)

```
@spec drop(t(), [key()]) :: t()
```

Drops the given `keys` from the keyword list.

Removes duplicate keys from the new keyword list.

Examples

```
iex> Keyword.drop([a: 1, a: 2], [:a])
[]
iex> Keyword.drop([a: 1, b: 2, c: 3], [:b, :d])
[a: 1, c: 3]
iex> Keyword.drop([a: 1, b: 2, b: 3, c: 3, a: 5], [:b,
:d])
[a: 1, c: 3, a: 5]
```

equal?(left, right)

```
@spec equal?(t(), t()) :: boolean()
```

Checks if two keywords are equal.

Considers two keywords to be equal if they contain the same keys and those keys contain the same values.

Examples

```
iex> Keyword.equal?([a: 1, b: 2], [b: 2, a: 1])
true
iex> Keyword.equal?([a: 1, b: 2], [b: 1, a: 2])
false
iex> Keyword.equal?([a: 1, b: 2, a: 3], [b: 2, a: 3, a:
1])
true
```

Comparison between values is done with `===/2`, which means integers are not equivalent to floats:

```
iex> Keyword.equal?([a: 1.0], [a: 1])
false
```

fetch(keywords, key)

```
@spec fetch(t(), key()) :: {:ok, value()} | :error
```

Fetches the value for a specific `key` and returns it in a tuple.

If the `key` does not exist, it returns `:error`.

Examples

```
iex> Keyword.fetch([a: 1], :a)
{:ok, 1}
iex> Keyword.fetch([a: 1], :b)
:error
```

fetch!(keywords, key)

```
@spec fetch!(t(), key()) :: value()
```

Fetches the value for specific `key`.

If the `key` does not exist, it raises a `KeyError`.

Examples

```
iex> Keyword.fetch!([a: 1], :a)
1
iex> Keyword.fetch!([a: 1], :b)
** (KeyError) key :b not found in:
...
```

filter(keywords, fun)

(since 1.13.0)

```
@spec filter(t(), ({key(), value()} -> as boolean(term())))
:: t()
```

Returns a keyword list containing only the entries from `keywords` for which the function `fun` returns a truthy value.

See also `reject/2` which discards all entries where the function returns a truthy value.

Examples

```
iex> Keyword.filter([one: 1, two: 2, three: 3], fn {_key,
val} -> rem(val, 2) == 1 end)
[one: 1, three: 3]
```

from_keys(keys, value)

(since 1.14.0)

```
@spec from_keys([key()], value()) :: t(value())
```

Builds a keyword from the given `keys` and the fixed `value`.

Examples

```
iex> Keyword.from_keys([:foo, :bar, :baz], :atom)
[foo: :atom, bar: :atom, baz: :atom]

iex> Keyword.from_keys([], :atom)
[]

iex> Keyword.from_keys(["foo"], :bar)
** (ArgumentError) expected a list of atoms as keys, got:
"foo"
```

get(keywords, key, default \\ nil)

```
@spec get(t(), key(), default()) :: value() | default()
```

Gets the value under the given `key`.

Returns the default value if `key` does not exist (`nil` if no default value is provided).

If duplicate entries exist, it returns the first one. Use `get_values/2` to retrieve all entries.

Examples

```
iex> Keyword.get([], :a)
nil
iex> Keyword.get([a: 1], :a)
1
```

```
iex> Keyword.get([a: 1], :b)
nil
iex> Keyword.get([a: 1], :b, 3)
3
```

With duplicate keys:

```
iex> Keyword.get([a: 1, a: 2], :a, 3)
1
iex> Keyword.get([a: 1, a: 2], :b, 3)
3
```

get_and_update(keywords, key, fun)

```
@spec get_and_update(t(), key(), (value() | nil ->
                                    {current_value, new_value ::
value()} | :pop)) ::
  {current_value, new_keywords :: t()}
when current_value: value()
```

Gets the value for `key` and updates it in one pass, deleting duplicate keys.

The `fun` argument receives the value of `key` (or `nil` if `key` is not present) and must return a two-element tuple: the current value (the retrieved value, which can be operated on before being returned) and the new value to be stored under `key`. The `fun` may also return `:pop`, implying the current value shall be removed from the keyword list and returned.

Returns a tuple that contains the current value returned by `fun` and a new keyword list with the updated value under `key`.

Examples

```
iex> Keyword.get_and_update([a: 1], :a, fn current_value
->
...>   {current_value, "new value!"}
...> end)
{1, [a: "new value!"]}

iex> Keyword.get_and_update([a: 1], :b, fn current_value
->
...>   {current_value, "new value!"}
...> end)
{nil, [b: "new value!", a: 1]}

iex> Keyword.get_and_update([a: 2], :a, fn number ->
...>   {2 * number, 3 * number}
...> end)
{4, [a: 6]}

iex> Keyword.get_and_update([a: 1], :a, fn _ -> :pop end)
{1, []}

iex> Keyword.get_and_update([a: 1], :b, fn _ -> :pop end)
{nil, [a: 1]}
```

get_and_update!(keywords, key, fun)

```
@spec get_and_update!(t(), key(), (value() ->
                                     {current_value, new_value ::
value()} | :pop)) ::
  {current_value, new_keywords :: t()}
when current_value: value()
```

Gets the value for `key` and updates it in one pass, deleting duplicate keys, raising if `key` can't be found in `keywords`.

The `fun` argument receives the value under `key` and must return a two-element tuple: the current value (the retrieved value, which can be operated on before being returned) and the new value to be stored under `key`.

Returns a tuple that contains the current value returned by `fun` and a new keyword list with the updated value under `key`.

Examples

```
iex> Keyword.get_and_update!([a: 1], :a, fn current_value
->
...>   {current_value, "new value!"}
...> end)
{1, [a: "new value!"]}

iex> Keyword.get_and_update!([a: 1], :b, fn current_value
->
...>   {current_value, "new value!"}
...> end)
** (KeyError) key :b not found in:
...

iex> Keyword.get_and_update!([a: 1], :a, fn _ ->
...>   :pop
...> end)
{1, []}
```

get_lazy(keywords, key, fun)

```
@spec get_lazy(t(), key(), (-> value())) :: value()
```

Gets the value under the given `key`.

If `key` does not exist, lazily evaluates `fun` and returns its result.

This is useful if the default value is very expensive to calculate or generally difficult to set up and tear down again.

If duplicate entries exist, it returns the first one. Use `get_values/2` to retrieve all entries.

Examples

```
iex> keyword = [a: 1]
iex> fun = fn ->
...>   # some expensive operation here
...>   13
...> end
iex> Keyword.get_lazy(keyword, :a, fun)
1
iex> Keyword.get_lazy(keyword, :b, fun)
13
```

get_values(keywords, key)

```
@spec get_values(t(), key()) :: [value()]
```

Gets all values under a specific `key`.

Examples

```
iex> Keyword.get_values([], :a)
[]
iex> Keyword.get_values([a: 1], :a)
[1]
iex> Keyword.get_values([a: 1, a: 2], :a)
[1, 2]
```

has_key?(keywords, key)

```
@spec has_key?(t(), key()) :: boolean()
```

Returns whether a given `key` exists in the given `keywords`.

Examples

```
iex> Keyword.has_key?([a: 1], :a)
true
iex> Keyword.has_key?([a: 1], :b)
false
```

intersect(keyword1, keyword2, fun \\ fn _key, _v1, v2 -> v2 end)

(since 1.17.0)

```
@spec intersect(keyword(), keyword(), (key(), value(),
value() -> value())) ::
  keyword()
```

Intersects two keyword lists, returning a keyword with the common keys.

By default, it returns the values of the intersected keys in `keyword2`. The keys are returned in the order found in `keyword1`.

Examples

```
iex> Keyword.intersect([a: 1, b: 2], [b: "b", c: "c"])
[b: "b"]

iex> Keyword.intersect([a: 1, b: 2], [b: 2, c: 3], fn _k,
v1, v2 ->
...>   v1 + v2
...> end)
[b: 4]
```

keys(keywords)

```
@spec keys(t()) :: [key()]
```

Returns all keys from the keyword list.

Keeps duplicate keys in the resulting list of keys.

Examples

```
iex> Keyword.keys(a: 1, b: 2)
[:a, :b]

iex> Keyword.keys(a: 1, b: 2, a: 3)
[:a, :b, :a]

iex> Keyword.keys([{:a, 1}, {"b", 2}, {:c, 3}])
** (ArgumentError) expected a keyword list, but an entry
```

```
in the list is not a two-element tuple with an atom as
its first element, got: {"b", 2}
```

keyword?(term)

```
@spec keyword?(term()) :: boolean()
```

Returns `true` if `term` is a keyword list, otherwise `false`.

When `term` is a list it is traversed to the end.

Examples

```
iex> Keyword.keyword?([])
true
iex> Keyword.keyword?(a: 1)
true
iex> Keyword.keyword?([{Foo, 1}])
true
iex> Keyword.keyword?([{}])
false
iex> Keyword.keyword?([:key])
false
iex> Keyword.keyword?(%{})
false
```

merge(keywords1, keywords2)

```
@spec merge(t(), t()) :: t()
```

Merges two keyword lists into one.

Adds all keys, including duplicate keys, given in `keywords2` to `keywords1`, overriding any existing ones.

There are no guarantees about the order of the keys in the returned keyword.

Examples

```
iex> Keyword.merge([a: 1, b: 2], [a: 3, d: 4])
[b: 2, a: 3, d: 4]

iex> Keyword.merge([a: 1, b: 2], [a: 3, d: 4, a: 5])
[b: 2, a: 3, d: 4, a: 5]

iex> Keyword.merge([a: 1], [2, 3])
** (ArgumentError) expected a keyword list as the second
argument, got: [2, 3]
```

merge(keywords1, keywords2, fun)

```
@spec merge(t(), t(), (key(), value(), value() -> value()))
:: t()
```

Merges two keyword lists into one.

Adds all keys, including duplicate keys, given in `keywords2` to `keywords1`. Invokes the given function to solve conflicts.

If `keywords2` has duplicate keys, it invokes the given function for each matching pair in `keywords1`.

There are no guarantees about the order of the keys in the returned keyword.

Examples

```
iex> Keyword.merge([a: 1, b: 2], [a: 3, d: 4], fn _k, v1,
v2 ->
...>   v1 + v2
...> end)
[b: 2, a: 4, d: 4]

iex> Keyword.merge([a: 1, b: 2], [a: 3, d: 4, a: 5], fn
:a, v1, v2 ->
...>   v1 + v2
...> end)
[b: 2, a: 4, d: 4, a: 5]

iex> Keyword.merge([a: 1, b: 2, a: 3], [a: 3, d: 4, a:
5], fn :a, v1, v2 ->
...>   v1 + v2
...> end)
[b: 2, a: 4, d: 4, a: 8]

iex> Keyword.merge([a: 1, b: 2], [:a, :b], fn :a, v1, v2
->
...>   v1 + v2
...> end)
** (ArgumentError) expected a keyword list as the second
argument, got: [:a, :b]
```

new()

```
@spec new() :: []
```

Returns an empty keyword list, i.e. an empty list.

Examples

```
iex> Keyword.new()
[]
```

new(pairs)

```
@spec new(Enumerable.t()) :: t()
```

Creates a keyword list from an enumerable.

Removes duplicate entries and the last one prevails. Unlike `Enum.into(enumerable, [])`, `Keyword.new(enumerable)` guarantees the keys are unique.

Examples

```
iex> Keyword.new([{:b, 1}, {:a, 2}])
[b: 1, a: 2]

iex> Keyword.new([{:a, 1}, {:a, 2}, {:a, 3}])
[a: 3]
```

new(pairs, transform)

```
@spec new(Enumerable.t(), (term() -> {key(), value()})) ::
t()
```

Creates a keyword list from an enumerable via the transformation function.

Removes duplicate entries and the last one prevails. Unlike `Enum.into(enumerable, [], fun)`, `Keyword.new(enumerable, fun)` guarantees the keys are unique.

Examples

```
iex> Keyword.new([:a, :b], fn x -> {x, x} end)
[a: :a, b: :b]
```

pop(keywords, key, default \\ nil)

```
@spec pop(t(), key(), default()) :: {value() | default(), t()}
```

Returns the first value for `key` and removes all associated entries in the keyword list.

It returns a tuple where the first element is the first value for `key` and the second element is a keyword list with all entries associated with `key` removed. If the `key` is not present in the keyword list, it returns `{default, keyword_list}`.

If you don't want to remove all the entries associated with `key` use `pop_first/3` instead, which will remove only the first entry.

Examples

```
iex> Keyword.pop([a: 1], :a)
{1, []}
iex> Keyword.pop([a: 1], :b)
{nil, [a: 1]}
iex> Keyword.pop([a: 1], :b, 3)
{3, [a: 1]}
iex> Keyword.pop([a: 1, a: 2], :a)
{1, []}
```

pop!(keywords, key)

(since 1.10.0)

```
@spec pop!(t(), key()) :: {value(), t()}
```

Returns the first value for `key` and removes all associated entries in the keyword list, raising if `key` is not present.

This function behaves like `pop/3`, but raises in case the `key` is not present in the given `keywords`.

Examples

```
iex> Keyword.pop!([a: 1], :a)
{1, []}
iex> Keyword.pop!([a: 1, a: 2], :a)
{1, []}
iex> Keyword.pop!([a: 1], :b)
** (KeyError) key :b not found in:
...
```

pop_first(keywords, key, default \\ nil)

```
@spec pop_first(t(), key(), default()) :: {value() |
default(), t()}
```

Returns and removes the first value associated with `key` in the keyword list.

Keeps duplicate keys in the resulting keyword list.

Examples

```
iex> Keyword.pop_first([a: 1], :a)
{1, []}
iex> Keyword.pop_first([a: 1], :b)
{nil, [a: 1]}
iex> Keyword.pop_first([a: 1], :b, 3)
{3, [a: 1]}
iex> Keyword.pop_first([a: 1, a: 2], :a)
{1, [a: 2]}
```

pop_lazy(keywords, key, fun)

```
@spec pop_lazy(t(), key(), (-> value())) :: {value(), t()}
```

Lazily returns and removes all values associated with `key` in the keyword list.

This is useful if the default value is very expensive to calculate or generally difficult to set up and tear down again.

Removes all duplicate keys. See `pop_first/3` for removing only the first entry.

Examples

```
iex> keyword = [a: 1]
iex> fun = fn ->
...>   # some expensive operation here
...>   13
...> end
iex> Keyword.pop_lazy(keyword, :a, fun)
{1, []}
iex> Keyword.pop_lazy(keyword, :b, fun)
{13, [a: 1]}
```

pop_values(keywords, key)

(since 1.10.0)

```
@spec pop_values(t(), key()) :: {[value()], t()}
```

Returns all values for `key` and removes all associated entries in the keyword list.

It returns a tuple where the first element is a list of values for `key` and the second element is a keyword list with all entries associated with `key` removed. If the `key` is not present in the keyword list, it returns `{[], keyword_list}`.

If you don't want to remove all the entries associated with `key` use `pop_first/3` instead, which will remove only the first entry.

Examples

```
iex> Keyword.pop_values([a: 1], :a)
{[1], []}
iex> Keyword.pop_values([a: 1], :b)
{[], [a: 1]}
iex> Keyword.pop_values([a: 1, a: 2], :a)
{[1, 2], []}
```

put(keywords, key, value)

```
@spec put(t(), key(), value()) :: t()
```

Puts the given `value` under the specified `key`.

If a value under `key` already exists, it overrides the value and removes all duplicate entries.

Examples

```
iex> Keyword.put([a: 1], :b, 2)
[b: 2, a: 1]
iex> Keyword.put([a: 1, b: 2], :a, 3)
[a: 3, b: 2]
iex> Keyword.put([a: 1, b: 2, a: 4], :a, 3)
[a: 3, b: 2]
```

put_new(keywords, key, value)

```
@spec put_new(t(), key(), value()) :: t()
```

Puts the given `value` under `key`, unless the entry `key` already exists.

Examples

```
iex> Keyword.put_new([a: 1], :b, 2)
[b: 2, a: 1]
iex> Keyword.put_new([a: 1, b: 2], :a, 3)
[a: 1, b: 2]
```

put_new_lazy(keywords, key, fun)

```
@spec put_new_lazy(t(), key(), (-> value())) :: t()
```

Evaluates `fun` and puts the result under `key` in keyword list unless `key` is already present.

This is useful if the value is very expensive to calculate or generally difficult to set up and tear down again.

Examples

```
iex> keyword = [a: 1]
iex> fun = fn ->
...>   # some expensive operation here
...>   13
...> end
iex> Keyword.put_new_lazy(keyword, :a, fun)
[a: 1]
iex> Keyword.put_new_lazy(keyword, :b, fun)
[b: 13, a: 1]
```

reject(keywords, fun)

(since 1.13.0)

```
@spec reject(t(), ({key(), value()} -> as boolean(term()))) :: t()
```

Returns a keyword list excluding the entries from `keywords` for which the function `fun` returns a truthy value.

See also `filter/2`.

Examples

```
iex> Keyword.reject([one: 1, two: 2, three: 3], fn {_key, val} -> rem(val, 2) == 1 end)
[two: 2]
```

replace(keywords, key, value)

(since 1.11.0)

```
@spec replace(t(), key(), value()) :: t()
```

Puts a value under `key` only if the `key` already exists in `keywords`.

In case a key exists multiple times in the keyword list, it removes later occurrences.

Examples

```
iex> Keyword.replace([a: 1, b: 2, a: 4], :a, 3)
[a: 3, b: 2]

iex> Keyword.replace([a: 1], :b, 2)
[a: 1]
```

replace!(keywords, key, value)

(since 1.5.0)

```
@spec replace!(t(), key(), value()) :: t()
```

Puts a value under `key` only if the `key` already exists in `keywords`.

If `key` is not present in `keywords`, it raises a `KeyError`.

Examples

```
iex> Keyword.replace!([a: 1, b: 2, a: 3], :a, :new)
[a: :new, b: 2]
iex> Keyword.replace!([a: 1, b: 2, c: 3, b: 4], :b, :new)
[a: 1, b: :new, c: 3]

iex> Keyword.replace!([a: 1], :b, 2)
** (KeyError) key :b not found in:
...
```

replace_lazy(keywords, key, fun)

(since 1.14.0)

```
@spec replace_lazy(t(), key(), (existing_value :: value() ->
new_value :: value())) ::
  t()
```

Replaces the value under `key` using the given function only if `key` already exists in `keywords`.

In comparison to `replace/3`, this can be useful when it's expensive to calculate the value.

If `key` does not exist, the original keyword list is returned unchanged.

Examples

```
iex> Keyword.replace_lazy([a: 1, b: 2], :a, fn v -> v * 4
end)
[a: 4, b: 2]

iex> Keyword.replace_lazy([a: 2, b: 2, a: 1], :a, fn v ->
v * 4 end)
[a: 8, b: 2]

iex> Keyword.replace_lazy([a: 1, b: 2], :c, fn v -> v * 4
end)
[a: 1, b: 2]
```

split(keywords, keys)

```
@spec split(t(), [key()]) :: {t(), t()}
```

Takes all entries corresponding to the given `keys` and extracts them into a separate keyword list.

Returns a tuple with the new list and the old list with removed keys.

Ignores keys for which there are no entries in the keyword list.

Entries with duplicate keys end up in the same keyword list.

Examples

```
iex> Keyword.split([a: 1, b: 2, c: 3], [:a, :c, :e])
{[a: 1, c: 3], [b: 2]}
iex> Keyword.split([a: 1, b: 2, c: 3, a: 4], [:a, :c,
:e])
{[a: 1, c: 3, a: 4], [b: 2]}
```

split_with(keywords, fun)

(since 1.15.0)

```
@spec split_with(t(), ({key(), value()} ->
as boolean(term()))) :: {t(), t()}
```

Splits the `keywords` into two keyword lists according to the given function `fun`.

The provided `fun` receives each `{key, value}` pair in the `keywords` as its only argument. Returns a tuple with the first keyword list containing all the elements in `keywords` for which applying `fun` returned a truthy value, and a second keyword list with all the elements for which applying `fun` returned a falsy value (`false` or `nil`).

Examples

```
iex> Keyword.split_with([a: 1, b: 2, c: 3], fn {_k, v} ->
rem(v, 2) == 0 end)
{[b: 2], [a: 1, c: 3]}

iex> Keyword.split_with([a: 1, b: 2, c: 3, b: 4], fn {_k,
v} -> rem(v, 2) == 0 end)
{[b: 2, b: 4], [a: 1, c: 3]}

iex> Keyword.split_with([a: 1, b: 2, c: 3, b: 4], fn {k,
v} -> k in [:a, :c] and rem(v, 2) == 0 end)
{[], [a: 1, b: 2, c: 3, b: 4]}

iex> Keyword.split_with([], fn {_k, v} -> rem(v, 2) == 0
end)
{[], []}
```

take(keywords, keys)

```
@spec take(t(), [key()]) :: t()
```

Takes all entries corresponding to the given `keys` and returns them as a new keyword list.

Preserves duplicate keys in the new keyword list.

Examples

```
iex> Keyword.take([a: 1, b: 2, c: 3], [:a, :c, :e])
[a: 1, c: 3]
iex> Keyword.take([a: 1, b: 2, c: 3, a: 5], [:a, :c, :e])
[a: 1, c: 3, a: 5]
```

to_list(keywords)

```
@spec to_list(t()) :: t()
```

Returns the keyword list itself.

Examples

```
iex> Keyword.to_list(a: 1)
[a: 1]
```

update(keywords, key, default, fun)

```
@spec update(t(), key(), default :: value(), (existing_value
:: value() ->
                                                new_value ::
value())) :: t()
```

Updates the value under `key` in `keywords` using the given function.

If the `key` does not exist, it inserts the given `default` value. Does not pass the `default` value through the update function.

Removes all duplicate keys and only updates the first one.

Examples

```
iex> Keyword.update([a: 1], :a, 13, fn existing_value ->
existing_value * 2 end)
```

```
[a: 2]

iex> Keyword.update([a: 1, a: 2], :a, 13, fn
existing_value -> existing_value * 2 end)
[a: 2]

iex> Keyword.update([a: 1], :b, 11, fn existing_value ->
existing_value * 2 end)
[a: 1, b: 11]
```

update!(keywords, key, fun)

```
@spec update!(t(), key(), (current_value :: value() ->
new_value :: value())) :: t()
```

Updates the value under `key` using the given function.

Raises `KeyError` if the `key` does not exist.

Removes all duplicate keys and only updates the first one.

Examples

```
iex> Keyword.update!([a: 1, b: 2, a: 3], :a, &(&1 * 2))
[a: 2, b: 2]
iex> Keyword.update!([a: 1, b: 2, c: 3], :b, &(&1 * 2))
[a: 1, b: 4, c: 3]

iex> Keyword.update!([a: 1], :b, &(&1 * 2))
** (KeyError) key :b not found in:
...
```

validate(keyword, values)

(since 1.13.0)

```
@spec validate(keyword(), values :: [atom() | {atom(), term()}]) ::
  {:ok, keyword()} | {:error, [atom()]}
```

Ensures the given `keyword` has only the keys given in `values`.

The second argument must be a list of atoms, specifying a given key, or tuples specifying a key and a default value.

If the keyword list has only the given keys, it returns `{:ok, keyword}` with default values applied. Otherwise it returns `{:error, invalid_keys}` with invalid keys.

See also: `validate!/2`.

Examples

```
iex> {:ok, result} = Keyword.validate([], [one: 1, two: 2])
iex> Enum.sort(result)
[one: 1, two: 2]

iex> {:ok, result} = Keyword.validate([two: 3], [one: 1, two: 2])
iex> Enum.sort(result)
[one: 1, two: 3]
```

If atoms are given, they are supported as keys but do not provide a default value:

```
iex> {:ok, result} = Keyword.validate([], [:one, two: 2])
iex> Enum.sort(result)
[two: 2]

iex> {:ok, result} = Keyword.validate([one: 1], [:one,
two: 2])
iex> Enum.sort(result)
[one: 1, two: 2]
```

Passing unknown keys returns an error:

```
iex> Keyword.validate([three: 3, four: 4], [one: 1, two:
2])
{:error, [:four, :three]}
```

Passing the same key multiple times also errors:

```
iex> Keyword.validate([one: 1, two: 2, one: 1], [:one,
:two])
{:error, [:one]}
```

validate!(keyword, values)

(since 1.13.0)

```
@spec validate!(keyword(), values :: [atom() | {atom(),
term()}]) :: keyword()
```

Similar to `validate/2` but returns the keyword or raises an error.

Examples

```
iex> Keyword.validate!([], [one: 1, two: 2]) |>
Enum.sort()
[one: 1, two: 2]
iex> Keyword.validate!([two: 3], [one: 1, two: 2]) |>
Enum.sort()
[one: 1, two: 3]
```

If atoms are given, they are supported as keys but do not provide a default value:

```
iex> Keyword.validate!([], [:one, two: 2]) |> Enum.sort()
[two: 2]
iex> Keyword.validate!([one: 1], [:one, two: 2]) |>
Enum.sort()
[one: 1, two: 2]
```

Passing unknown keys raises an error:

```
iex> Keyword.validate!([three: 3], [one: 1, two: 2])
** (ArgumentError) unknown keys [:three] in [three: 3],
the allowed keys are: [:one, :two]
```

Passing the same key multiple times also errors:

```
iex> Keyword.validate!([one: 1, two: 2, one: 1], [:one,
:two])
** (ArgumentError) duplicate keys [:one] in [one: 1, two:
2, one: 1]
```

values(keywords)

```
@spec values(t()) :: [value()]
```

Returns all values from the keyword list.

Keeps values from duplicate keys in the resulting list of values.

Examples

```
iex> Keyword.values(a: 1, b: 2)
[1, 2]
iex> Keyword.values(a: 1, b: 2, a: 3)
[1, 2, 3]
```

List

Linked lists hold zero, one, or more elements in the chosen order.

Lists in Elixir are specified between square brackets:

```
iex> [1, "two", 3, :four]
[1, "two", 3, :four]
```

Two lists can be concatenated and subtracted using the `++/2` and `--/2` operators:

```
iex> [1, 2, 3] ++ [4, 5, 6]
[1, 2, 3, 4, 5, 6]
iex> [1, true, 2, false, 3, true] -- [true, false]
[1, 2, 3, true]
```

An element can be prepended to a list using `|`:

```
iex> new = 0
iex> list = [1, 2, 3]
iex> [new | list]
[0, 1, 2, 3]
```

Lists in Elixir are effectively linked lists, which means they are internally represented in pairs containing the head and the tail of a list:

```
iex> [head | tail] = [1, 2, 3]
iex> head
1
```

```
iex> tail
[2, 3]
```

Similarly, we could write the list `[1, 2, 3]` using only such pairs (called cons cells):

```
iex> [1 | [2 | [3 | []]]]
[1, 2, 3]
```

Some lists, called improper lists, do not have an empty list as the second element in the last cons cell:

```
iex> [1 | [2 | [3 | 4]]]
[1, 2, 3 | 4]
```

Although improper lists are generally avoided, they are used in some special circumstances like iodata and chardata entities (see the `IO` module).

Due to their cons cell based representation, prepending an element to a list is always fast (constant time), while appending becomes slower as the list grows in size (linear time):

```
iex> list = [1, 2, 3]
iex> [0 | list] # fast
[0, 1, 2, 3]
iex> list ++ [4] # slow
[1, 2, 3, 4]
```

Most of the functions in this module work in linear time. This means that the time it takes to perform an operation grows at the same rate as the

length of the list. For example `length/1` and `last/1` will run in linear time because they need to iterate through every element of the list, but `first/1` will run in constant time because it only needs the first element.

Lists also implement the `Enumerable` protocol, so many functions to work with lists are found in the `Enum` module. Additionally, the following functions and operators for lists are found in `Kernel`:

- `++/2`
- `--/2`
- `hd/1`
- `tl/1`
- `in/2`
- `length/1`

Charlists

If a list is made of non-negative integers, where each integer represents a Unicode code point, the list can also be called a charlist. These integers must:

- be within the range `0..0x10FFFF` (`0..1_114_111`);
- and be out of the range `0xD800..0xDFFF` (`55_296..57_343`), which is reserved in Unicode for UTF-16 surrogate pairs.

Elixir uses the `~c` sigil to define charlists:

```
iex> ~c"héllo"
[104, 233, 108, 108, 111]
```

In particular, charlists will be printed back by default with the `~c` sigil if they contain only printable ASCII characters:

```
iex> ~c"abc"
~c"abc"
```

Even though the representation changed, the raw data does remain a list of integers, which can be handled as such:

```
iex> inspect(~c"abc", charlists: :as_list)
"[97, 98, 99]"
iex> Enum.map(~c"abc", fn num -> 1000 + num end)
[1097, 1098, 1099]
```

You can use the `IEx.Helpers.i/1` helper to get a condensed rundown on charlists in IEx when you encounter them, which shows you the type, description and also the raw representation in one single summary.

The rationale behind this behavior is to better support Erlang libraries which may return text as charlists instead of Elixir strings. In Erlang, charlists are the default way of handling strings, while in Elixir it's binaries. One example of such functions is `Application.loaded_applications/0`:

```
Application.loaded_applications()
#=>  [
#=>    {:stdlib, ~c"ERTS  CXC 138 10", ~c"2.6"},
#=>    {:compiler, ~c"ERTS  CXC 138 10", ~c"6.0.1"},
#=>    {:elixir, ~c"elixir", ~c"1.0.0"},
#=>    {:kernel, ~c"ERTS  CXC 138 10", ~c"4.1"},
#=>    {:logger, ~c"logger", ~c"1.0.0"}
#=>  ]
```

A list can be checked if it is made of only printable ASCII characters with `ascii_printable?/2`.

Improper lists are never deemed as charlists.

Summary

Functions

ascii_printable?(list, limit \\ :infinity)

Checks if `list` is a charlist made only of printable ASCII characters.

delete(list, element)

Deletes the given `element` from the `list`. Returns a new list without the element.

delete_at(list, index)

Produces a new list by removing the value at the specified `index`.

duplicate(elem, n)

Duplicates the given element `n` times in a list.

ends_with?(list, suffix)

Returns `true` if `list` ends with the given `suffix` list, otherwise returns `false`.

first(list, default \\ nil)

Returns the first element in `list` or `default` if `list` is empty.

first!(list)

Returns the first element in `list`.

flatten(list)

Flattens the given `list` of nested lists.

flatten(list, tail)

Flattens the given `list` of nested lists. The list `tail` will be added at the end of the flattened list.

foldl(list, acc, fun)

Folds (reduces) the given list from the left with a function. Requires an accumulator, which can be any value.

foldr(list, acc, fun)

Folds (reduces) the given list from the right with a function. Requires an accumulator, which can be any value.

improper?(list)

Returns `true` if `list` is an improper list. Otherwise returns `false`.

insert_at(list, index, value)

Returns a list with `value` inserted at the specified `index`.

keydelete(list, key, position)

Receives a `list` of tuples and deletes the first tuple where the element at `position` matches the given `key`. Returns the new list.

keyfind(list, key, position, default \\ nil)

Receives a list of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`.

keyfind!(list, key, position)

Receives a list of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`.

keymember?(list, key, position)

Receives a list of tuples and returns `true` if there is a tuple where the element at `position` in the tuple matches the given `key`.

keyreplace(list, key, position, new_tuple)

Receives a list of tuples and if the identified element by `key` at `position` exists, it is replaced with `new_tuple`.

keysort(list, position, sorter \\ :asc)

Receives a list of tuples and sorts the elements at `position` of the tuples.

keystore(list, key, position, new_tuple)

Receives a `list` of tuples and replaces the element identified by `key` at `position` with `new_tuple`.

keytake(list, key, position)

Receives a `list` of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`, as well as the `list` without found tuple.

last(list, default \\ nil)

Returns the last element in `list` or `default` if `list` is empty.

last!(list)

Returns the last element in `list`.

myers_difference(list1, list2)

Returns a keyword list that represents an *edit script*.

myers_difference(list1, list2, diff_script)

Returns a keyword list that represents an *edit script* with nested diffs.

pop_at(list, index, default \\ nil)

Returns and removes the value at the specified `index` in the `list`.

replace_at(list, index, value)

Returns a list with a replaced value at the specified `index`.

starts_with?(list, prefix)

Returns `true` if `list` starts with the given `prefix` list, otherwise returns `false`.

to_atom(charlist)

Converts a charlist to an atom.

to_charlist(list)

Converts a list of integers representing Unicode code points, lists or strings into a charlist.

to_existing_atom(charlist)

Converts a charlist to an existing atom.

to_float(charlist)

Returns the float whose text representation is `charlist`.

to_integer(charlist)

Returns an integer whose text representation is `charlist`.

to_integer(charlist, base)

Returns an integer whose text representation is `charlist` in base `base`.

to_string(list)

Converts a list of integers representing code points, lists or strings into a string.

to_tuple(list)

Converts a list to a tuple.

update_at(list, index, fun)

Returns a list with an updated value at the specified `index`.

wrap(term)

Wraps `term` in a list if this is not list.

zip(list_of_lists) deprecated

Functions

ascii_printable?(list, limit \\ :infinity)

(since 1.6.0)

```
@spec ascii_printable?(list(), 0) :: true
@spec ascii_printable?([], limit) :: true when limit:
:infinity | pos_integer()
```

```
@spec ascii_printable?([...], limit) :: boolean()
when limit: :infinity | pos integer()
```

Checks if `list` is a charlist made only of printable ASCII characters.

Takes an optional `limit` as a second argument. `ascii_printable?/2` only checks the printability of the list up to the `limit`.

A printable charlist in Elixir contains only the printable characters in the standard seven-bit ASCII character encoding, which are characters ranging from 32 to 126 in decimal notation, plus the following control characters:

- `?\a` - Bell
- `?\b` - Backspace
- `?\t` - Horizontal tab
- `?\n` - Line feed
- `?\v` - Vertical tab
- `?\f` - Form feed
- `?\r` - Carriage return
- `?\e` - Escape

For more information read the Character groups section in the Wikipedia article of the ASCII standard.

Examples

```
iex> List.ascii_printable?(~c"abc")
true

iex> List.ascii_printable?(~c"abc" ++ [0])
false
```

```
iex> List.ascii_printable?(~c"abc" ++ [0], 2)
true
```

Improper lists are not printable, even if made only of ASCII characters:

```
iex> List.ascii_printable?(~c"abc" ++ ?d)
false
```

delete(list, element)

```
@spec delete([], any()) :: []
@spec delete([...], any()) :: list()
```

Deletes the given `element` from the `list`. Returns a new list without the element.

If the `element` occurs more than once in the `list`, just the first occurrence is removed.

Examples

```
iex> List.delete([:a, :b, :c], :a)
[:b, :c]

iex> List.delete([:a, :b, :c], :d)
[:a, :b, :c]

iex> List.delete([:a, :b, :b, :c], :b)
[:a, :b, :c]
```

```
iex> List.delete([], :b)
[]
```

delete_at(list, index)

```
@spec delete_at(list(), integer()) :: list()
```

Produces a new list by removing the value at the specified `index`.

Negative indices indicate an offset from the end of the `list`. If `index` is out of bounds, the original `list` is returned.

Examples

```
iex> List.delete_at([1, 2, 3], 0)
[2, 3]

iex> List.delete_at([1, 2, 3], 10)
[1, 2, 3]

iex> List.delete_at([1, 2, 3], -1)
[1, 2]
```

duplicate(elem, n)

```
@spec duplicate(any(), 0) :: []
@spec duplicate(elem, pos_integer()) :: [elem, ...] when
elem: var
```

Duplicates the given element `n` times in a list.

`n` is an integer greater than or equal to `0`.

If `n` is `0`, an empty list is returned.

Examples

```
iex> List.duplicate("hello", 0)
[]

iex> List.duplicate("hi", 1)
["hi"]

iex> List.duplicate("bye", 2)
["bye", "bye"]

iex> List.duplicate([1, 2], 3)
[[1, 2], [1, 2], [1, 2]]
```

ends_with?(list, suffix)

(since 1.18.0)

```
@spec ends_with?([...], [...]) :: boolean()
@spec ends_with?(list(), []) :: true
@spec ends_with?([], [...]) :: false
```

Returns `true` if `list` ends with the given `suffix` list, otherwise returns `false`.

If `suffix` is an empty list, it returns `true`.

Examples

```
iex> List.ends_with?([1, 2, 3], [2, 3])
true

iex> List.ends_with?([1, 2], [1, 2, 3])
false

iex> List.ends_with?([:alpha], [])
true

iex> List.ends_with?([], [:alpha])
false
```

first(list, default \\ nil)

```
@spec first([], any()) :: any()
@spec first([elem, ...], any()) :: elem when elem: var
```

Returns the first element in `list` or `default` if `list` is empty.

`first/2` has been introduced in Elixir v1.12.0, while `first/1` has been available since v1.0.0.

Examples

```
iex> List.first([])
nil

iex> List.first([], 1)
1
```

```
iex> List.first([1])
1

iex> List.first([1, 2, 3])
1
```

first!(list)

(since 1.20.0)

```
@spec first!([elem, ...]) :: elem when elem: var
```

Returns the first element in `list`.

If `list` is empty, an error is raised.

Examples

```
iex> List.first!([1])
1

iex> List.first!([1, 2, 3])
1

iex> List.first!([])
** (ArgumentError) attempted to get the first element of
an empty list
```

flatten(list)

```
@spec flatten(deep_list) :: list() when deep_list: [any() | deep_list]
```

Flattens the given `list` of nested lists.

Empty list elements are discarded.

Examples

```
iex> List.flatten([1, [[2], 3]])
[1, 2, 3]

iex> List.flatten([[], [[], []]])
[]
```

flatten(list, tail)

```
@spec flatten(deep_list, [elem]) :: [elem]
when deep_list: [elem | deep_list], elem: var
```

Flattens the given `list` of nested lists. The list `tail` will be added at the end of the flattened list.

Empty list elements from `list` are discarded, but not the ones from `tail`.

Examples

```
iex> List.flatten([1, [[2], 3]], [4, 5])
[1, 2, 3, 4, 5]
```

```
iex> List.flatten([1, [], 2], [3, [], 4])
[1, 2, 3, [], 4]
```

foldl(list, acc, fun)

```
@spec foldl([elem], acc, (elem, acc -> acc)) :: acc when
elem: var, acc: var
```

Folds (reduces) the given list from the left with a function. Requires an accumulator, which can be any value.

Examples

```
iex> List.foldl([5, 5], 10, fn x, acc -> x + acc end)
20

iex> List.foldl([1, 2, 3, 4], 0, fn x, acc -> x - acc
end)
2

iex> List.foldl([1, 2, 3], {0, 0}, fn x, {a1, a2} -> {a1
+ x, a2 - x} end)
{6, -6}
```

foldr(list, acc, fun)

```
@spec foldr([elem], acc, (elem, acc -> acc)) :: acc when
elem: var, acc: var
```

Folds (reduces) the given list from the right with a function. Requires an accumulator, which can be any value.

Examples

```
iex> List.foldr([1, 2, 3, 4], 0, fn x, acc -> x - acc
end)
-2

iex> List.foldr([1, 2, 3, 4], %{sum: 0, product: 1}, fn
x, %{sum: a1, product: a2} -> %{sum: a1 + x, product: a2
* x} end)
%{product: 24, sum: 10}
```

improper?(list)

(since 1.8.0)

```
@spec improper?(maybe improper list()) :: boolean()
```

Returns `true` if `list` is an improper list. Otherwise returns `false`.

Examples

```
iex> List.improper?([1, 2 | 3])
true

iex> List.improper?([1, 2, 3])
false
```

insert_at(list, index, value)

```
@spec insert_at(list(), integer(), any()) :: list()
```

Returns a list with `value` inserted at the specified `index`.

Note that `index` is capped at the list length. Negative indices indicate an offset from the end of the `list`.

Examples

```
iex> List.insert_at([1, 2, 3, 4], 2, 0)
[1, 2, 0, 3, 4]

iex> List.insert_at([1, 2, 3], 10, 0)
[1, 2, 3, 0]

iex> List.insert_at([1, 2, 3], -1, 0)
[1, 2, 3, 0]

iex> List.insert_at([1, 2, 3], -10, 0)
[0, 1, 2, 3]
```

keydelete(list, key, position)

```
@spec keydelete([tuple()], any(), non_neg_integer()) ::
[tuple()]
```

Receives a `list` of tuples and deletes the first tuple where the element at `position` matches the given `key`. Returns the new list.

Examples

```
iex> List.keydelete([a: 1, b: 2], :a, 0)
[b: 2]

iex> List.keydelete([a: 1, b: 2], 2, 1)
[a: 1]

iex> List.keydelete([a: 1, b: 2], :c, 0)
[a: 1, b: 2]
```

This function works for any list of tuples:

```
iex> List.keydelete([{22, "SSH"}, {80, "HTTP"}], 80, 0)
[{22, "SSH"}]
```

keyfind(list, key, position, default \\ nil)

```
@spec keyfind([tuple()], any(), non_neg_integer(), any()) :: any()
```

Receives a list of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`.

If no matching tuple is found, `default` is returned.

Examples

```
iex> List.keyfind([a: 1, b: 2], :a, 0)
{:a, 1}
```

```
iex> List.keyfind([a: 1, b: 2], 2, 1)
{:b, 2}

iex> List.keyfind([a: 1, b: 2], :c, 0)
nil
```

This function works for any list of tuples:

```
iex> List.keyfind([{22, "SSH"}, {80, "HTTP"}], 22, 0)
{22, "SSH"}
```

keyfind!(list, key, position)

(since 1.13.0)

```
@spec keyfind!([tuple()], any(), non_neg_integer()) :: any()
```

Receives a list of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`.

If no matching tuple is found, an error is raised.

Examples

```
iex> List.keyfind!([a: 1, b: 2], :a, 0)
{:a, 1}

iex> List.keyfind!([a: 1, b: 2], 2, 1)
{:b, 2}

iex> List.keyfind!([a: 1, b: 2], :c, 0)
```

```
** (KeyError) key :c at position 0 not found in: [a: 1,
b: 2]
```

This function works for any list of tuples:

```
iex> List.keyfind!([{22, "SSH"}, {80, "HTTP"}], 22, 0)
{22, "SSH"}
```

keymember?(list, key, position)

```
@spec keymember?([tuple()], any(), non_neg_integer()) ::
boolean()
```

Receives a list of tuples and returns `true` if there is a tuple where the element at `position` in the tuple matches the given `key`.

Examples

```
iex> List.keymember?([a: 1, b: 2], :a, 0)
true

iex> List.keymember?([a: 1, b: 2], 2, 1)
true

iex> List.keymember?([a: 1, b: 2], :c, 0)
false
```

This function works for any list of tuples:

```
iex> List.keymember?([{22, "SSH"}, {80, "HTTP"}], 22, 0)
true
```

keyreplace(list, key, position, new_tuple)

```
@spec keyreplace([tuple()], any(), non_neg_integer(),
tuple()) :: [tuple()]
```

Receives a list of tuples and if the identified element by `key` at `position` exists, it is replaced with `new_tuple`.

Examples

```
iex> List.keyreplace([a: 1, b: 2], :a, 0, {:a, 3})
[a: 3, b: 2]

iex> List.keyreplace([a: 1, b: 2], :a, 1, {:a, 3})
[a: 1, b: 2]
```

This function works for any list of tuples:

```
iex> List.keyreplace([{22, "SSH"}, {80, "HTTP"}], 22, 0,
{22, "Secure Shell"})
[{22, "Secure Shell"}, {80, "HTTP"}]
```

keysort(list, position, sorter \\ :asc)

(since 1.14.0)

```
@spec keysort(
  [tuple()],
  non_neg_integer(),
  (any(), any() -> boolean())
  | :asc
  | :desc
  | module()
  | {:asc | :desc, module()}
) :: [tuple()]
```

Receives a list of tuples and sorts the elements at `position` of the tuples.

The sort is stable.

A `sorter` argument is available since Elixir v1.14.0. Similar to `Enum.sort/2`, the sorter can be an anonymous function, the atoms `:asc` or `:desc`, or module that implements a compare function.

Examples

```
iex> List.keysort([a: 5, b: 1, c: 3], 1)
[b: 1, c: 3, a: 5]

iex> List.keysort([a: 5, c: 1, b: 3], 0)
[a: 5, b: 3, c: 1]
```

To sort in descending order:

```
iex> List.keysort([a: 5, c: 1, b: 3], 0, :desc)
[c: 1, b: 3, a: 5]
```

As in `Enum.sort/2`, avoid using the default sorting function to sort structs, as by default it performs structural comparison instead of a semantic one. In such cases, you shall pass a sorting function as third element or any module that implements a `compare/2` function. For example, if you have tuples with user names and their birthday, and you want to sort on their birthday, in both ascending and descending order, you should do:

```
iex> users = [
...>   {"Ellis", ~D[1943-05-11]},
...>   {"Lovelace", ~D[1815-12-10]},
...>   {"Turing", ~D[1912-06-23]}
...> ]
iex> List.keysort(users, 1, Date)
[
  {"Lovelace", ~D[1815-12-10]},
  {"Turing", ~D[1912-06-23]},
  {"Ellis", ~D[1943-05-11]}
]
iex> List.keysort(users, 1, {:desc, Date})
[
  {"Ellis", ~D[1943-05-11]},
  {"Turing", ~D[1912-06-23]},
  {"Lovelace", ~D[1815-12-10]}
]
```

keystore(list, key, position, new_tuple)

```
@spec keystore([tuple()], any(), non_neg_integer(), tuple())
:: [tuple(), ...]
```

Receives a `list` of tuples and replaces the element identified by `key` at `position` with `new_tuple`.

If the element does not exist, it is added to the end of the `list`.

Examples

```
iex> List.keystore([a: 1, b: 2], :a, 0, {:a, 3})
[a: 3, b: 2]

iex> List.keystore([a: 1, b: 2], :c, 0, {:c, 3})
[a: 1, b: 2, c: 3]
```

This function works for any list of tuples:

```
iex> List.keystore([{22, "SSH"}], 80, 0, {80, "HTTP"})
[{22, "SSH"}, {80, "HTTP"}]
```

keytake(list, key, position)

```
@spec keytake([tuple()], any(), non_neg_integer()) ::
{tuple(), [tuple()]} | nil
```

Receives a `list` of tuples and returns the first tuple where the element at `position` in the tuple matches the given `key`, as well as the `list` without found tuple.

If such a tuple is not found, `nil` will be returned.

Examples

```
iex> List.keytake([a: 1, b: 2], :a, 0)
{{:a, 1}, [b: 2]}
```

```
iex> List.keytake([a: 1, b: 2], 2, 1)
{{:b, 2}, [a: 1]}

iex> List.keytake([a: 1, b: 2], :c, 0)
nil
```

This function works for any list of tuples:

```
iex> List.keytake([{22, "SSH"}, {80, "HTTP"}], 80, 0)
{{80, "HTTP"}, [{22, "SSH"}]}
```

last(list, default \\ nil)

```
@spec last([], any()) :: any()
@spec last([elem, ...], any()) :: elem when elem: var
```

Returns the last element in `list` or `default` if `list` is empty.

`last/2` has been introduced in Elixir v1.12.0, while `last/1` has been available since v1.0.0.

Examples

```
iex> List.last([])
nil

iex> List.last([], 1)
1

iex> List.last([1])
1
```

```
iex> List.last([1, 2, 3])
3
```

last!(list)

(since 1.20.0)

```
@spec last!([elem, ...]) :: elem when elem: var
```

Returns the last element in `list`.

If `list` is empty, an error is raised.

Examples

```
iex> List.last!([1])
1

iex> List.last!([1, 2, 3])
3

iex> List.last!([])
** (ArgumentError) attempted to get the last element of
an empty list
```

myers_difference(list1, list2)

(since 1.4.0)

```
@spec myers_difference(list(), list()) :: [{:eq | :ins |
:del, list()}]
```

Returns a keyword list that represents an *edit script*.

The algorithm is outlined in the "An O(ND) Difference Algorithm and Its Variations" paper by E. Myers.

An *edit script* is a keyword list. Each key describes the "editing action" to take in order to bring `list1` closer to being equal to `list2`; a key can be `:eq`, `:ins`, or `:del`. Each value is a sublist of either `list1` or `list2` that should be inserted (if the corresponding key is `:ins`), deleted (if the corresponding key is `:del`), or left alone (if the corresponding key is `:eq`) in `list1` in order to be closer to `list2`.

See `myers_difference/3` if you want to handle nesting in the diff scripts.

Examples

```
iex> List.myers_difference([1, 4, 2, 3], [1, 2, 3, 4])
[eq: [1], del: [4], eq: [2, 3], ins: [4]]
```

myers_difference(list1, list2, diff_script)

(since 1.8.0)

```
@spec myers_difference(list(), list(), (term(), term() ->
script | nil)) :: script
when script: [{:eq | :ins | :del | :diff, list()}]
```

Returns a keyword list that represents an *edit script* with nested diffs.

This is an extension of `myers_difference/2` where a `diff_script` function can be given in case it is desired to compute nested differences. The function may return a list with the inner edit script or `nil` in case

there is no such script. The returned inner edit script will be under the `:diff` key.

Examples

```
iex> List.myers_difference(["a", "db", "c"], ["a", "bc"],
&String.myers_difference/2)
[eq: ["a"], diff: [del: "d", eq: "b", ins: "c"], del:
["c"]]
```

pop_at(list, index, default \\ nil)

(since 1.4.0)

```
@spec pop_at(list(), integer(), any()) :: {any(), list()}
```

Returns and removes the value at the specified `index` in the `list`.

Negative indices indicate an offset from the end of the `list`. If `index` is out of bounds, the original `list` is returned.

Examples

```
iex> List.pop_at([1, 2, 3], 0)
{1, [2, 3]}
iex> List.pop_at([1, 2, 3], 5)
{nil, [1, 2, 3]}
iex> List.pop_at([1, 2, 3], 5, 10)
{10, [1, 2, 3]}
iex> List.pop_at([1, 2, 3], -1)
{3, [1, 2]}
```

replace_at(list, index, value)

```
@spec replace_at(list(), integer(), any()) :: list()
```

Returns a list with a replaced value at the specified `index`.

Negative indices indicate an offset from the end of the `list`. If `index` is out of bounds, the original `list` is returned.

Examples

```
iex> List.replace_at([1, 2, 3], 0, 0)
[0, 2, 3]

iex> List.replace_at([1, 2, 3], 10, 0)
[1, 2, 3]

iex> List.replace_at([1, 2, 3], -1, 0)
[1, 2, 0]

iex> List.replace_at([1, 2, 3], -10, 0)
[1, 2, 3]
```

starts_with?(list, prefix)

(since 1.5.0)

```
@spec starts_with?([...], [...]) :: boolean()
@spec starts_with?(list(), []) :: true
@spec starts_with?([], [...]) :: false
```

Returns `true` if `list` starts with the given `prefix` list, otherwise returns `false`.

If `prefix` is an empty list, it returns `true`.

Examples

```
iex> List.starts_with?([1, 2, 3], [1, 2])
true

iex> List.starts_with?([1, 2], [1, 2, 3])
false

iex> List.starts_with?([:alpha], [])
true

iex> List.starts_with?([], [:alpha])
false
```

to_atom(charlist)

```
@spec to_atom(charlist()) :: atom()
```

Converts a charlist to an atom.

Elixir supports conversions from charlists which contain any Unicode code point.

Inlined by the compiler.

Examples

```
iex> List.to_atom(~c"Elixir")
:Elixir

iex> List.to_atom(~c"💧 Elixir")
:"💧 Elixir"
```

to_charlist(list)

(since 1.8.0)

```
@spec to_charlist(:unicode.charlist()) :: charlist()
```

Converts a list of integers representing Unicode code points, lists or strings into a charlist.

Note that this function expects a list of integers representing Unicode code points. If you have a list of bytes, you must instead use the `:binary` module.

Examples

```
iex> ~c"æß" = List.to_charlist([0x00E6, 0x00DF])
[230, 223]

iex> List.to_charlist([0x0061, "bc"])
~c"abc"

iex> List.to_charlist([0x0064, "ee", [~c"p"]])
~c"deep"
```

to_existing_atom(charlist)

```
@spec to_existing_atom(charlist()) :: atom()
```

Converts a charlist to an existing atom.

Elixir supports conversions from charlists which contain any Unicode code point. Raises an `ArgumentError` if the atom does not exist.

Inlined by the compiler.

> **Atoms and modules**
>
> Since Elixir is a compiled language, the atoms defined in a module will only exist after said module is loaded, which typically happens whenever a function in the module is executed. Therefore, it is generally recommended to call `List.to_existing_atom/1` only to convert atoms defined within the module making the function call to `to_existing_atom/1`.

Examples

```
iex> _ = :my_atom
iex> List.to_existing_atom(~c"my_atom")
:my_atom

iex> _ = :"🔥 Elixir"
iex> List.to_existing_atom(~c"🔥 Elixir")
:"🔥 Elixir"
```

to_float(charlist)

```
@spec to_float(charlist()) :: float()
```

Returns the float whose text representation is `charlist`.

Inlined by the compiler.

Examples

```
iex> List.to_float(~c"2.2017764e+0")
2.2017764
```

to_integer(charlist)

```
@spec to_integer(charlist()) :: integer()
```

Returns an integer whose text representation is `charlist`.

Inlined by the compiler.

Examples

```
iex> List.to_integer(~c"123")
123
```

to_integer(charlist, base)

```
@spec to_integer(charlist(), 2..36) :: integer()
```

Returns an integer whose text representation is `charlist` in base `base`.

Inlined by the compiler.

The base needs to be between `2` and `36`.

Examples

```
iex> List.to_integer(~c"3FF", 16)
1023
```

to_string(list)

```
@spec to_string(:unicode.charlist()) :: String.t()
```

Converts a list of integers representing code points, lists or strings into a string.

To be converted to a string, a list must either be empty or only contain the following elements:

- strings
- integers representing Unicode code points
- a list containing one of these three elements

Note that this function expects a list of integers representing Unicode code points. If you have a list of bytes, you must instead use the `:binary` module.

Examples

```
iex> List.to_string([0x00E6, 0x00DF])
"æß"

iex> List.to_string([0x0061, "bc"])
"abc"

iex> List.to_string([0x0064, "ee", [~c"p"]])
```

```
"deep"

iex> List.to_string([])
""
```

to_tuple(list)

```
@spec to_tuple(list()) :: tuple()
```

Converts a list to a tuple.

Inlined by the compiler.

Examples

```
iex> List.to_tuple([:share, [:elixir, 163]])
{:share, [:elixir, 163]}
```

update_at(list, index, fun)

```
@spec update_at([elem], integer(), (elem -> any())) :: list()
when elem: var
```

Returns a list with an updated value at the specified `index`.

Negative indices indicate an offset from the end of the `list`. If `index` is out of bounds, the original `list` is returned.

Examples

```
iex> List.update_at([1, 2, 3], 0, &(&1 + 10))
[11, 2, 3]

iex> List.update_at([1, 2, 3], 10, &(&1 + 10))
[1, 2, 3]

iex> List.update_at([1, 2, 3], -1, &(&1 + 10))
[1, 2, 13]

iex> List.update_at([1, 2, 3], -10, &(&1 + 10))
[1, 2, 3]
```

wrap(term)

```
@spec wrap(term()) :: maybe_improper_list()
```

Wraps `term` in a list if this is not list.

If `term` is already a list, it returns the list. If `term` is `nil`, it returns an empty list.

Examples

```
iex> List.wrap("hello")
["hello"]

iex> List.wrap([1, 2, 3])
[1, 2, 3]

iex> List.wrap(nil)
[]
```

zip(list_of_lists)

This function is deprecated. Use Enum.zip/1 instead.

Map

Maps are the "go to" key-value data structure in Elixir.

Maps can be created with the `%{}` syntax, and key-value pairs can be expressed as `key => value`:

```
iex> %{}
%{}
iex> %{"one" => :two, 3 => "four"}
%{3 => "four", "one" => :two}
```

Key-value pairs in a map do not follow any order (that's why the printed map in the example above has a different order than the map that was created).

Maps do not impose any restriction on the key type: anything can be a key in a map. As a key-value structure, maps do not allow duplicate keys. Keys are compared using the exact-equality operator (`===/2`). If colliding keys are defined in a map literal, the last one prevails.

When the key in a key-value pair is an atom, the `key: value` shorthand syntax can be used (as in many other special forms):

```
iex> %{a: 1, b: 2}
%{a: 1, b: 2}
```

If you want to mix the shorthand syntax with `=>` , the shorthand syntax must come at the end:

```
iex> %{"hello" => "world", a: 1, b: 2}
%{:a => 1, :b => 2, "hello" => "world"}
```

Keys in maps can be accessed through some of the functions in this module (such as `Map.get/3` or `Map.fetch/2`) or through the `map[]` syntax provided by the `Access` module:

```
iex> map = %{a: 1, b: 2}
iex> Map.fetch(map, :a)
{:ok, 1}
iex> map[:b]
2
iex> map["non_existing_key"]
nil
```

To access atom keys, one may also use the `map.key` notation. Note that `map.key` will raise a `KeyError` if the `map` doesn't contain the key `:key`, compared to `map[:key]`, that would return `nil`.

```
map = %{foo: "bar", baz: "bong"}
map.foo
#=> "bar"
map.non_existing_key
** (KeyError) key :non_existing_key not found in:
...
```

> **Avoid parentheses**
>
> Do not add parentheses when accessing fields, such as in `data.key()`. If parentheses are used, Elixir will expect `data` to be an atom representing a module and attempt to call the *function* `key/0` in it.

The two syntaxes for accessing keys reveal the dual nature of maps. The `map[key]` syntax is used for dynamically created maps that may have any key, of any type. `map.key` is used with maps that hold a predetermined set of atoms keys, which are expected to always be present. Structs, defined via `defstruct/1`, are one example of such "static maps", where the keys can also be checked during compile time.

Maps can be pattern matched on. When a map is on the left-hand side of a pattern match, it will match if the map on the right-hand side contains the keys on the left-hand side and their values match the ones on the left-hand side. This means that an empty map matches every map.

```
iex> %{} = %{foo: "bar"}
%{foo: "bar"}
iex> %{a: a} = %{:a => 1, "b" => 2, [:c, :e, :e] => 3}
iex> a
1
```

But this will raise a `MatchError` exception:

```
%{:c => 3} = %{:a => 1, 2 => :b}
```

Variables can be used as map keys both when writing map literals as well as when matching:

```
iex> n = 1
1
iex> %{n => :one}
%{1 => :one}
iex> %{^n => :one} = %{1 => :one, 2 => :two, 3 => :three}
%{1 => :one, 2 => :two, 3 => :three}
```

Maps also support a specific update syntax to update the value stored under *existing* keys. You can update using the atom keys syntax:

```
iex> map = %{one: 1, two: 2}
iex> %{map | one: "one"}
%{one: "one", two: 2}
```

Or any other keys:

```
iex> other_map = %{"three" => 3, "four" => 4, "five" =>
5}
iex> %{other_map | "three" => "three", "four" => "four"}
%{"five" => 5, "four" => "four", "three" => "three"}
```

When a key that does not exist in the map is updated a `KeyError` exception will be raised:

```
%{map | three: 3}
```

The functions in this module that need to find a specific key work in logarithmic time. This means that the time it takes to find keys grows as the map grows, but it's not directly proportional to the map size. In comparison to finding an element in a list, it performs better because lists have a linear time complexity. Some functions, such as `keys/1` and `values/1`, run in linear time because they need to get to every element in the map.

Maps also implement the `Enumerable` protocol, so many functions to work with maps are found in the `Enum` module. Additionally, the following functions for maps are found in `Kernel`:

- `map_size/1`

Summary

Types

key()
value()

Functions

delete(map, key)

Deletes the entry in `map` for a specific `key`.

drop(map, keys)

Drops the given `keys` from `map`.

equal?(map1, map2)

Checks if two maps are equal.

fetch(map, key)

Fetches the value for a specific `key` in the given `map`.

fetch!(map, key)

Fetches the value for a specific `key` in the given `map`, erroring out if `map` doesn't contain `key`.

filter(map, fun)

Returns a map containing only those pairs from `map` for which `fun` returns a truthy value.

from_keys(keys, value)

Builds a map from the given `keys` and the fixed `value`.

from_struct(struct)

Converts a `struct` to map.

get(map, key, default \\ nil)

Gets the value for a specific `key` in `map`.

get_and_update(map, key, fun)

Gets the value from `key` and updates it, all in one pass.

get_and_update!(map, key, fun)

Gets the value from `key` and updates it, all in one pass. Raises if there is no `key`.

get_lazy(map, key, fun)

Gets the value for a specific `key` in `map`.

has_key?(map, key)

Returns whether the given `key` exists in the given `map`.

intersect(map1, map2)

Intersects two maps, returning a map with the common keys.

intersect(map1, map2, fun)

Intersects two maps, returning a map with the common keys and resolving conflicts through a function.

keys(map)

Returns all keys from `map`.

merge(map1, map2)

Merges two maps into one.

merge(map1, map2, fun)

Merges two maps into one, resolving conflicts through the given `fun`.

new()

Returns a new empty map.

new(enumerable)

Creates a map from an `enumerable`.

new(enumerable, transform)

Creates a map from an `enumerable` via the given transformation function.

pop(map, key, default \\ nil)

Removes the value associated with `key` in `map` and returns the value and the updated map.

pop!(map, key)

Removes and returns the value associated with `key` in `map` alongside the updated map, or raises if `key` is not present.

pop_lazy(map, key, fun)

Lazily returns and removes the value associated with `key` in `map`.

put(map, key, value)

Puts the given `value` under `key` in `map`.

put_new(map, key, value)

Puts the given `value` under `key` unless the entry `key` already exists in `map`.

put_new_lazy(map, key, fun)

Evaluates `fun` and puts the result under `key` in `map` unless `key` is already present.

reject(map, fun)

Returns map excluding the pairs from `map` for which `fun` returns a truthy value.

replace(map, key, value)

Puts a value under `key` only if the `key` already exists in `map`.

replace!(map, key, value)

Puts a value under `key` only if the `key` already exists in `map`.

replace_lazy(map, key, fun)

Replaces the value under `key` using the given function only if `key` already exists in `map`.

split(map, keys)

Takes all entries corresponding to the given `keys` in `map` and extracts them into a separate map.

split_with(map, fun)

Splits the `map` into two maps according to the given function `fun`.

take(map, keys)

Returns a new map with all the key-value pairs in `map` where the key is in `keys`.

to_list(map)

Converts `map` to a list.

update(map, key, default, fun)

Updates the `key` in `map` with the given function.

update!(map, key, fun)

Updates `key` with the given function.

values(map)

Returns all values from `map`.

Types

key()

```
@type key() :: any()
```

value()

```
@type value() :: any()
```

Functions

delete(map, key)

```
@spec delete(map(), key()) :: map()
```

Deletes the entry in `map` for a specific `key`.

If the `key` does not exist, returns `map` unchanged.

Inlined by the compiler.

Examples

```
iex> Map.delete(%{a: 1, b: 2}, :a)
%{b: 2}
iex> Map.delete(%{b: 2}, :a)
%{b: 2}
```

drop(map, keys)

```
@spec drop(map(), [key()]) :: map()
```

Drops the given `keys` from `map`.

If `keys` contains keys that are not in `map`, they're simply ignored.

Examples

```
iex> Map.drop(%{a: 1, b: 2, c: 3}, [:b, :d])
%{a: 1, c: 3}
```

equal?(map1, map2)

```
@spec equal?(map(), map()) :: boolean()
```

Checks if two maps are equal.

Two maps are considered to be equal if they contain the same keys and those keys contain the same values.

Note this function exists for completeness so the `Map` and `Keyword` modules provide similar APIs. In practice, developers often compare maps using `==/2` or `===/2` directly.

Examples

```
iex> Map.equal?(%{a: 1, b: 2}, %{b: 2, a: 1})
true
iex> Map.equal?(%{a: 1, b: 2}, %{b: 1, a: 2})
false
```

Comparison between keys and values is done with `===/2`, which means integers are not equivalent to floats:

```
iex> Map.equal?(%{a: 1.0}, %{a: 1})
false
```

fetch(map, key)

```
@spec fetch(map(), key()) :: {:ok, value()} | :error
```

Fetches the value for a specific `key` in the given `map`.

If `map` contains the given `key` then its value is returned in the shape of `{:ok, value}`. If `map` doesn't contain `key`, `:error` is returned.

If the type system can verify `:error` is always returned (which means key is never available in the map), it will emit an error.

Inlined by the compiler.

Examples

```
iex> Map.fetch(%{a: 1}, :a)
{:ok, 1}
iex> Map.fetch(%{"foo" => "bar"}, "unknown")
:error
```

fetch!(map, key)

```
@spec fetch!(map(), key()) :: value()
```

Fetches the value for a specific `key` in the given `map`, erroring out if `map` doesn't contain `key`.

The exclamation mark (`!`) implies this function can raise a `KeyError` exception at runtime if `map` doesn't contain `key`. If the type system can

verify this function will always raise (which means the key is never available), then it will emit a warning at compile-time. See the "Type checking" section below.

Inlined by the compiler.

Examples

```
iex> Map.fetch!(%{a: 1}, :a)
1
```

When the key is missing, an exception is raised:

```
Map.fetch!(%{a: 1}, :b)
** (KeyError) key :b not found in: %{a: 1}
```

Type checking

The compiler will emit a warning if it can verify that none of the keys given are available in the map.

When the key is an atom, because only single key is given, a warning will be emitted in case the type system proves the key is not present.

However, this behaviour matters when the type of the key represents multiple values. For example:

```
key = returns_foo_or_bar() #=> :foo or :bar
Map.fetch!(%{foo: 123}, key)
```

Although the key can be `:foo` or `:bar`, there is no warning emitted, as `:foo` will succeed. This is by design: the exclamation mark in Elixir

denotes precisely that a runtime exception may be raised.

In case you are looking up multiple keys and you don't know if they may be present, you can use `Map.fetch/2` instead and deal with the error case accordingly:

```
case Map.fetch(%{foo: 123}, key) do
  {:ok, value} -> ...
  :error -> ...
end
```

Both `Map.fetch!/2` and `Map.fetch/2` will emit a warning if it proves that both `:foo` or `:bar` are absent in the map.

Alternatively, if you want to statically prove that all of keys are in the map, you can match on the possible values and access them directly:

```
case returns_foo_or_bar() do
  :foo -> map.foo
  :bar -> map.bar
end
```

filter(map, fun)

(since 1.13.0)

```
@spec filter(map(), ({key(), value()} -> as_boolean(term()))) :: map()
```

Returns a map containing only those pairs from `map` for which `fun` returns a truthy value.

`fun` receives the key and value of each of the elements in the map as a key-value pair.

See also `reject/2` which discards all elements where the function returns a truthy value.

> **Performance considerations**
>
> If you find yourself doing multiple calls to `Map.filter/2` and/or `Map.reject/2` in a pipeline, it is likely more efficient to use `Enum.filter/2` and `Enum.reject/2` instead and convert to a map at the end using `Map.new/1` or `Map.new/2`.

Examples

```
iex> Map.filter(%{one: 1, two: 2, three: 3}, fn {_key,
val} -> rem(val, 2) == 1 end)
%{one: 1, three: 3}
```

from_keys(keys, value)

(since 1.14.0)

```
@spec from_keys([key()], value()) :: map()
```

Builds a map from the given `keys` and the fixed `value`.

Inlined by the compiler.

Examples

```
iex> Map.from_keys([1, 2, 3], :number)
```

```
%{1 => :number, 2 => :number, 3 => :number}
```

from_struct(struct)

```
@spec from_struct(atom() | struct()) :: map()
```

Converts a `struct` to map.

It accepts a struct and simply removes the `__struct__` field from the given struct.

Example

```
defmodule User do
  defstruct [:name]
end

Map.from_struct(%User{name: "john"})
#=> %{name: "john"}
```

get(map, key, default \\ nil)

```
@spec get(map(), key(), value()) :: value()
```

Gets the value for a specific `key` in `map`.

If `key` is present in `map` then its value `value` is returned. Otherwise, `default` is returned.

If `default` is not provided, `nil` is used.

Examples

```
iex> Map.get(%{"a" => 1}, "a")
1
iex> Map.get(%{"a" => 1}, "b")
nil
iex> Map.get(%{"a" => 1}, "b", 3)
3
iex> Map.get(%{"a" => nil}, "a", 1)
nil
```

get_and_update(map, key, fun)

```
@spec get_and_update(map(), key(), (value() | nil ->
                                      {current_value, new_value ::
value()} | :pop)) ::
  {current_value, new_map :: map()}
when current_value: value()
```

Gets the value from `key` and updates it, all in one pass.

`fun` is called with the current value under `key` in `map` (or `nil` if `key` is not present in `map`) and must return a two-element tuple: the current value (the retrieved value, which can be operated on before being returned) and the new value to be stored under `key` in the resulting new map. `fun` may also return `:pop`, which means the current value shall be removed from `map` and returned (making this function behave like `Map.pop(map, key)`).

The returned value is a two-element tuple with the current value returned by `fun` and a new map with the updated value under `key`.

Examples

```
iex> Map.get_and_update(%{a: 1}, :a, fn current_value ->
...>   {current_value, "new value!"}
...> end)
{1, %{a: "new value!"}}

iex> Map.get_and_update(%{a: 1}, :b, fn current_value ->
...>   {current_value, "new value!"}
...> end)
{nil, %{a: 1, b: "new value!"}}

iex> Map.get_and_update(%{a: 1}, :a, fn _ -> :pop end)
{1, %{}}

iex> Map.get_and_update(%{a: 1}, :b, fn _ -> :pop end)
{nil, %{a: 1}}
```

get_and_update!(map, key, fun)

```
@spec get_and_update!(map(), key(), (value() ->
                                       {current_value, new_value ::
value()} | :pop)) ::
  {current_value, map()}
when current_value: value()
```

Gets the value from `key` and updates it, all in one pass. Raises if there is no `key`.

Behaves exactly like `get_and_update/3`, but raises a `KeyError` exception if `key` is not present in `map`.

Examples

```
iex> Map.get_and_update!(%{a: 1}, :a, fn current_value ->
...>   {current_value, "new value!"}
...> end)
{1, %{a: "new value!"}}

iex> Map.get_and_update!(%{a: 1}, :b, fn current_value ->
...>   {current_value, "new value!"}
...> end)
** (KeyError) key :b not found in:
...

iex> Map.get_and_update!(%{a: 1}, :a, fn _ ->
...>   :pop
...> end)
{1, %{}}
```

get_lazy(map, key, fun)

```
@spec get_lazy(map(), key(), (-> value())) :: value()
```

Gets the value for a specific `key` in `map`.

If `key` is present in `map` then its value `value` is returned. Otherwise, `fun` is evaluated and its result is returned.

This is useful if the default value is very expensive to calculate or generally difficult to setup and teardown again.

Examples

```
iex> Map.get_lazy(%{a: 1}, :a, fn -> :expensive_value
end)
1

iex> Map.get_lazy(%{"a" => 1}, "b", fn ->
:expensive_value end)
:expensive_value
```

has_key?(map, key)

```
@spec has_key?(map(), key()) :: boolean()
```

Returns whether the given `key` exists in the given `map`.

Inlined by the compiler.

Examples

```
iex> Map.has_key?(%{a: 1}, :a)
true
iex> Map.has_key?(%{a: 1}, :b)
false
```

intersect(map1, map2)

(since 1.15.0)

```
@spec intersect(map(), map()) :: map()
```

Intersects two maps, returning a map with the common keys.

The values in the returned map are the values of the intersected keys in `map2`.

Inlined by the compiler.

Examples

```
iex> Map.intersect(%{a: 1, b: 2}, %{b: "b", c: "c"})
%{b: "b"}
```

intersect(map1, map2, fun)

(since 1.15.0)

```
@spec intersect(map(), map(), (key(), value(), value() ->
value())) :: map()
```

Intersects two maps, returning a map with the common keys and resolving conflicts through a function.

The given function will be invoked when there are duplicate keys; its arguments are `key` (the duplicate key), `value1` (the value of `key` in `map1`), and `value2` (the value of `key` in `map2`). The value returned by `fun` is used as the value under `key` in the resulting map.

Examples

```
iex> Map.intersect(%{a: 1, b: 2}, %{b: 2, c: 3}, fn _k,
v1, v2 ->
...>   v1 + v2
```

```
...> end)
%{b: 4}
```

keys(map)

```
@spec keys(map()) :: [key()]
```

Returns all keys from `map`.

Inlined by the compiler.

Examples

```
Map.keys(%{a: 1, b: 2})
[:a, :b]
```

merge(map1, map2)

```
@spec merge(map(), map()) :: map()
```

Merges two maps into one.

All keys in `map2` will be added to `map1`, overriding any existing one (i.e., the keys in `map2` "have precedence" over the ones in `map1`).

If you have a struct and you would like to merge a set of keys into the struct, do not use this function, as it would merge all keys on the right side into the struct, even if the key is not part of the struct. Instead, use `struct/2`.

Inlined by the compiler.

Examples

```
iex> Map.merge(%{a: 1, b: 2}, %{a: 3, d: 4})
%{a: 3, b: 2, d: 4}
```

merge(map1, map2, fun)

```
@spec merge(map(), map(), (key(), value(), value() ->
value())) :: map()
```

Merges two maps into one, resolving conflicts through the given `fun`.

All keys in `map2` will be added to `map1`. The given function will be invoked when there are duplicate keys; its arguments are `key` (the duplicate key), `value1` (the value of `key` in `map1`), and `value2` (the value of `key` in `map2`). The value returned by `fun` is used as the value under `key` in the resulting map.

Examples

```
iex> Map.merge(%{a: 1, b: 2}, %{a: 3, d: 4}, fn _k, v1,
v2 ->
...>   v1 + v2
...> end)
%{a: 4, b: 2, d: 4}
```

new()

```
@spec new() :: map()
```

Returns a new empty map.

Examples

```
iex> Map.new()
%{}
```

new(enumerable)

```
@spec new(Enumerable.t()) :: map()
```

Creates a map from an `enumerable`.

Duplicated keys are removed; the latest one prevails.

Examples

```
iex> Map.new([{:b, 1}, {:a, 2}])
%{a: 2, b: 1}
iex> Map.new(a: 1, a: 2, a: 3)
%{a: 3}
```

new(enumerable, transform)

```
@spec new(Enumerable.t(), (term() -> {key(), value()})) ::
map()
```

Creates a map from an `enumerable` via the given transformation function.

Duplicated keys are removed; the latest one prevails.

Examples

```
iex> Map.new([:a, :b], fn x -> {x, x} end)
%{a: :a, b: :b}

iex> Map.new(%{a: 2, b: 3, c: 4}, fn {key, val} -> {key,
val * 2} end)
%{a: 4, b: 6, c: 8}
```

pop(map, key, default \\ nil)

```
@spec pop(map(), key(), default) :: {value(), updated_map ::
map()} | {default, map()}
when default: value()
```

Removes the value associated with `key` in `map` and returns the value and the updated map.

If `key` is present in `map`, it returns `{value, updated_map}` where `value` is the value of the key and `updated_map` is the result of removing `key` from `map`. If `key` is not present in `map`, `{default, map}` is returned.

Examples

```
iex> Map.pop(%{a: 1}, :a)
{1, %{}}
```

```
iex> Map.pop(%{"a" => 1}, "b")
{nil, %{"a" => 1}}
iex> Map.pop(%{"a" => 1}, "b", 3)
{3, %{"a" => 1}}
```

pop!(map, key)

(since 1.10.0)

```
@spec pop!(map(), key()) :: {value(), updated_map :: map()}
```

Removes and returns the value associated with `key` in `map` alongside the updated map, or raises if `key` is not present.

Behaves the same as `pop/3` but raises a `KeyError` exception if `key` is not present in `map`.

Examples

```
iex> Map.pop!(%{a: 1}, :a)
{1, %{}}
iex> Map.pop!(%{a: 1, b: 2}, :a)
{1, %{b: 2}}
iex> Map.pop!(%{"a" => 1}, "b")
** (KeyError) key "b" not found in:
...
```

pop_lazy(map, key, fun)

```
@spec pop_lazy(map(), key(), (-> value())) :: {value(),
map()}
```

Lazily returns and removes the value associated with `key` in `map`.

If `key` is present in `map`, it returns `{value, new_map}` where `value` is the value of the key and `new_map` is the result of removing `key` from `map`. If `key` is not present in `map`, `{fun_result, map}` is returned, where `fun_result` is the result of applying `fun`.

This is useful if the default value is very expensive to calculate or generally difficult to setup and teardown again.

Examples

```
iex> Map.pop_lazy(%{a: 1}, :a, fn -> :expensive_value
end)
{1, %{}}

iex> Map.pop_lazy(%{"a" => 1}, "b", fn ->
:expensive_value end)
{:expensive_value, %{"a" => 1}}
```

put(map, key, value)

```
@spec put(map(), key(), value()) :: map()
```

Puts the given `value` under `key` in `map`.

Inlined by the compiler.

Examples

```
iex> Map.put(%{a: 1}, :b, 2)
%{a: 1, b: 2}
iex> Map.put(%{a: 1, b: 2}, :a, 3)
%{a: 3, b: 2}
```

put_new(map, key, value)

```
@spec put_new(map(), key(), value()) :: map()
```

Puts the given `value` under `key` unless the entry `key` already exists in `map`.

Examples

```
iex> Map.put_new(%{a: 1}, :b, 2)
%{a: 1, b: 2}
iex> Map.put_new(%{a: 1, b: 2}, :a, 3)
%{a: 1, b: 2}
```

put_new_lazy(map, key, fun)

```
@spec put_new_lazy(map(), key(), (-> value())) :: map()
```

Evaluates `fun` and puts the result under `key` in `map` unless `key` is already present.

This function is useful in case you want to compute the value to put under `key` only if `key` is not already present, as for example, when the

value is expensive to calculate or generally difficult to setup and teardown again.

Examples

```
iex> map = %{a: 1}
iex> fun = fn ->
...>   # some expensive operation here
...>   3
...> end
iex> Map.put_new_lazy(map, :a, fun)
%{a: 1}
iex> Map.put_new_lazy(map, :b, fun)
%{a: 1, b: 3}
```

reject(map, fun)

(since 1.13.0)

```
@spec reject(map(), ({key(), value()} -> as_boolean(term()))) :: map()
```

Returns map excluding the pairs from `map` for which `fun` returns a truthy value.

See also `filter/2`.

> **Performance considerations**
>
> If you find yourself doing multiple calls to `Map.filter/2` and/or `Map.reject/2` in a pipeline, it is likely more efficient to use `Enum.filter/2` and `Enum.reject/2` instead and convert to a map at the end using `Map.new/1` or `Map.new/2`.

Examples

```
iex> Map.reject(%{one: 1, two: 2, three: 3}, fn {_key,
val} -> rem(val, 2) == 1 end)
%{two: 2}
```

replace(map, key, value)

(since 1.11.0)

```
@spec replace(map(), key(), value()) :: map()
```

Puts a value under `key` only if the `key` already exists in `map`.

Examples

```
iex> Map.replace(%{a: 1, b: 2}, :a, 3)
%{a: 3, b: 2}

iex> Map.replace(%{"a" => 1}, "b", 2)
%{"a" => 1}
```

replace!(map, key, value)

(since 1.5.0)

```
@spec replace!(map(), key(), value()) :: map()
```

Puts a value under `key` only if the `key` already exists in `map`.

The exclamation mark (`!`) implies this function can raise a `KeyError` exception at runtime if `map` doesn't contain `key`. If the type system can verify this function will always raise (which means the key is never available), then it will emit a warning at compile-time. See the "Type checking" section in `Map.fetch!/2` for more information.

Inlined by the compiler.

Examples

```
iex> Map.replace!(%{a: 1, b: 2}, :a, 3)
%{a: 3, b: 2}

iex> Map.replace!(%{"foo" => "bar"}, "unknown",
"new_bar")
** (KeyError) key "unknown" not found in:
...
```

replace_lazy(map, key, fun)

(since 1.14.0)

```
@spec replace_lazy(map(), key(), (existing_value :: value() -
> new_value :: value())) ::
  map()
```

Replaces the value under `key` using the given function only if `key` already exists in `map`.

In comparison to `replace/3`, this can be useful when it's expensive to calculate the value.

If `key` does not exist, the original map is returned unchanged.

Examples

```
iex> Map.replace_lazy(%{a: 1, b: 2}, :a, fn v -> v * 4
end)
%{a: 4, b: 2}

iex> Map.replace_lazy(%{"a" => 1, "b" => 2}, "c", fn v ->
v * 4 end)
%{"a" => 1, "b" => 2}
```

split(map, keys)

```
@spec split(map(), [key()]) :: {map(), map()}
```

Takes all entries corresponding to the given `keys` in `map` and extracts them into a separate map.

Returns a tuple with the new map and the old map with removed keys.

Keys for which there are no entries in `map` are ignored.

Examples

```
iex> Map.split(%{a: 1, b: 2, c: 3}, [:a, :c, :e])
{%{a: 1, c: 3}, %{b: 2}}
```

split_with(map, fun)

(since 1.15.0)

```
@spec split_with(map(), ({key(), value()} ->
as boolean(term()))) :: {map(), map()}
```

Splits the `map` into two maps according to the given function `fun`.

`fun` receives each `{key, value}` pair in the `map` as its only argument. Returns a tuple with the first map containing all the elements in `map` for which applying `fun` returned a truthy value, and a second map with all the elements for which applying `fun` returned a falsy value (`false` or `nil`).

Examples

```
iex> Map.split_with(%{a: 1, b: 2, c: 3, d: 4}, fn {_k, v}
-> rem(v, 2) == 0 end)
{%{b: 2, d: 4}, %{a: 1, c: 3}}

iex> Map.split_with(%{a: 1, b: -2, c: 1, d: -3}, fn {k,
_v} -> k in [:b, :d] end)
{%{b: -2, d: -3}, %{a: 1, c: 1}}

iex> Map.split_with(%{a: 1, b: -2, c: 1, d: -3}, fn {_k,
v} -> v > 50 end)
{%{}, %{a: 1, b: -2, c: 1, d: -3}}

iex> Map.split_with(%{}, fn {_k, v} -> v > 50 end)
{%{}, %{}}
```

take(map, keys)

```
@spec take(map(), [key()]) :: map()
```

Returns a new map with all the key-value pairs in `map` where the key is in `keys`.

If `keys` contains keys that are not in `map`, they're simply ignored.

Examples

```
iex> Map.take(%{a: 1, b: 2, c: 3}, [:a, :c, :e])
%{a: 1, c: 3}
```

to_list(map)

```
@spec to_list(map()) :: [{term(), term()}]
```

Converts `map` to a list.

Each key-value pair in the map is converted to a two-element tuple `{key, value}` in the resulting list.

Inlined by the compiler.

Examples

```
iex> Map.to_list(%{a: 1})
[a: 1]
iex> Map.to_list(%{1 => 2})
[{1, 2}]
```

update(map, key, default, fun)

```
@spec update(map(), key(), default :: value(),
(existing_value :: value() ->
                                                new_value ::
value())) ::
  map()
```

Updates the `key` in `map` with the given function.

If `key` is present in `map` then the existing value is passed to `fun` and its result is used as the updated value of `key`. If `key` is not present in `map`, `default` is inserted as the value of `key`. The default value will not be passed through the update function.

Examples

```
iex> Map.update(%{a: 1}, :a, 13, fn existing_value ->
existing_value * 2 end)
%{a: 2}
iex> Map.update(%{a: 1}, :b, 11, fn existing_value ->
existing_value * 2 end)
%{a: 1, b: 11}
```

update!(map, key, fun)

```
@spec update!(map(), key(), (existing_value :: value() ->
new_value :: value())) ::
  map()
```

Updates `key` with the given function.

If `key` is present in `map` then the existing value is passed to `fun` and its result is used as the updated value of `key`. If `key` is not present in `map`, a `KeyError` exception is raised.

Examples

```
iex> Map.update!(%{a: 1}, :a, &(&1 * 2))
%{a: 2}

iex> Map.update!(%{"a" => 1}, "b", &(&1 * 2))
** (KeyError) key "b" not found in:
...
```

values(map)

```
@spec values(map()) :: [value()]
```

Returns all values from `map`.

Inlined by the compiler.

Examples

```
Map.values(%{a: 1, b: 2})
[1, 2]
```

MapSet

Functions that work on sets.

A set is a data structure that can contain unique elements of any kind, without any particular order. `MapSet` is the "go to" set data structure in Elixir.

A set can be constructed using `MapSet.new/0`:

```
iex> MapSet.new()
MapSet.new([])
```

Elements in a set don't have to be of the same type and they can be populated from an enumerable using `MapSet.new/1`:

```
iex> MapSet.new([1, :two, {"three"}])
MapSet.new([1, :two, {"three"}])
```

Elements can be inserted using `MapSet.put/2`:

```
iex> MapSet.new([2]) |> MapSet.put(4) |> MapSet.put(0)
MapSet.new([0, 2, 4])
```

By definition, sets can't contain duplicate elements: when inserting an element in a set where it's already present, the insertion is simply a no-op.

```
iex> map_set = MapSet.new()
iex> MapSet.put(map_set, "foo")
```

```
MapSet.new(["foo"])
iex> map_set |> MapSet.put("foo") |> MapSet.put("foo")
MapSet.new(["foo"])
```

A `MapSet` is represented internally using the `%MapSet{}` struct. This struct can be used whenever there's a need to pattern match on something being a `MapSet`:

```
iex> match?(%MapSet{}, MapSet.new())
true
```

Note that, however, the struct fields are private and must not be accessed directly; use the functions in this module to perform operations on sets.

`MapSet`s can also be constructed starting from other collection-type data structures: for example, see `MapSet.new/1` or `Enum.into/2`.

`MapSet` is built on top of Erlang's `:sets` (version 2). This means that they share many properties, including logarithmic time complexity. Erlang `:sets` (version 2) are implemented on top of maps, so see the documentation for `Map` for more information on its execution time complexity.

Summary

Types

t()
t(value)
value()

Functions

delete(map_set, value)

Deletes `value` from `map_set`.

difference(map_set1, map_set2)

Returns a set that is `map_set1` without the members of `map_set2`.

disjoint?(map_set1, map_set2)

Checks if `map_set1` and `map_set2` have no members in common.

equal?(map_set1, map_set2)

Checks if two sets are equal.

filter(map_set, fun)

Filters the set by returning only the elements from `map_set` for which invoking `fun` returns a truthy value.

intersection(map_set1, map_set2)

Returns a set containing only members that `map_set1` and `map_set2` have in common.

member?(map_set, value)

Checks if `map_set` contains `value`.

new()

Returns a new set.

new(enumerable)

Creates a set from an enumerable.

new(enumerable, transform)

Creates a set from an enumerable via the transformation function.

put(map_set, value)

Inserts `value` into `map_set` if `map_set` doesn't already contain it.

reject(map_set, fun)

Returns a set by excluding the elements from `map_set` for which invoking `fun` returns a truthy value.

size(map_set)

Returns the number of elements in `map_set`.

split_with(map_set, fun)

Splits the `map_set` into two `MapSet`s according to the given function `fun`.

subset?(map_set1, map_set2)

Checks if `map_set1`'s members are all contained in `map_set2`.

symmetric_difference(map_set1, map_set2)

Returns a set with elements that are present in only one but not both sets.

to_list(map_set)

Converts `map_set` to a list.

union(map_set1, map_set2)

Returns a set containing all members of `map_set1` and `map_set2`.

Types

t()

```
@type t() :: t(term())
```

t(value)

```
@type t(value) :: %MapSet{map: internal(value)}
```

value()

```
@type value() :: term()
```

Functions

delete(map_set, value)

```
@spec delete(t(val1), val2) :: t(val1) when val1: value(),
val2: value()
```

Deletes `value` from `map_set`.

Returns a new set which is a copy of `map_set` but without `value`.

Examples

```
iex> map_set = MapSet.new([1, 2, 3])
iex> MapSet.delete(map_set, 4)
MapSet.new([1, 2, 3])
iex> MapSet.delete(map_set, 2)
MapSet.new([1, 3])
```

difference(map_set1, map_set2)

```
@spec difference(t(val1), t(val2)) :: t(val1) when val1:
value(), val2: value()
```

Returns a set that is `map_set1` without the members of `map_set2`.

Examples

```
iex> MapSet.difference(MapSet.new([1, 2]), MapSet.new([2,
3, 4]))
MapSet.new([1])
```

disjoint?(map_set1, map_set2)

```
@spec disjoint?(t(), t()) :: boolean()
```

Checks if `map_set1` and `map_set2` have no members in common.

Examples

```
iex> MapSet.disjoint?(MapSet.new([1, 2]), MapSet.new([3, 4]))
true
iex> MapSet.disjoint?(MapSet.new([1, 2]), MapSet.new([2, 3]))
false
```

equal?(map_set1, map_set2)

```
@spec equal?(t(), t()) :: boolean()
```

Checks if two sets are equal.

The comparison between elements is done using `===/2`, which a set with `1` is not equivalent to a set with `1.0`.

Examples

```
iex> MapSet.equal?(MapSet.new([1, 2]), MapSet.new([2, 1, 1]))
true
iex> MapSet.equal?(MapSet.new([1, 2]), MapSet.new([3, 4]))
false
iex> MapSet.equal?(MapSet.new([1]), MapSet.new([1.0]))
false
```

filter(map_set, fun)

(since 1.14.0)

```
@spec filter(t(a), (a -> as_boolean(term()))) :: t(a) when a: value()
```

Filters the set by returning only the elements from `map_set` for which invoking `fun` returns a truthy value.

Also see `reject/2` which discards all elements where the function returns a truthy value.

> **Performance considerations**
>
> If you find yourself doing multiple calls to `MapSet.filter/2` and `MapSet.reject/2` in a pipeline, it is likely more efficient to use `Enum.map/2` and `Enum.filter/2` instead and convert to a map at the end using `MapSet.new/1`.

Examples

```
iex> MapSet.filter(MapSet.new(1..5), fn x -> x > 3 end)
MapSet.new([4, 5])

iex> MapSet.filter(MapSet.new(["a", :b, "c"]),
&is_atom/1)
MapSet.new([:b])
```

intersection(map_set1, map_set2)

```
@spec intersection(t(val), t(val)) :: t(val) when val: value()
```

Returns a set containing only members that `map_set1` and `map_set2` have in common.

Examples

```
iex> MapSet.intersection(MapSet.new([1, 2]),
MapSet.new([2, 3, 4]))
MapSet.new([2])

iex> MapSet.intersection(MapSet.new([1, 2]),
MapSet.new([3, 4]))
MapSet.new([])
```

member?(map_set, value)

```
@spec member?(t(), value()) :: boolean()
```

Checks if `map_set` contains `value`.

Examples

```
iex> MapSet.member?(MapSet.new([1, 2, 3]), 2)
true
iex> MapSet.member?(MapSet.new([1, 2, 3]), 4)
false
```

new()

```
@spec new() :: t()
```

Returns a new set.

Examples

```
iex> MapSet.new()
MapSet.new([])
```

new(enumerable)

```
@spec new(Enumerable.t()) :: t()
```

Creates a set from an enumerable.

Examples

```
iex> MapSet.new([:b, :a, 3])
MapSet.new([3, :a, :b])
iex> MapSet.new([3, 3, 3, 2, 2, 1])
MapSet.new([1, 2, 3])
```

new(enumerable, transform)

```
@spec new(Enumerable.t(), (term() -> val)) :: t(val) when
val: value()
```

Creates a set from an enumerable via the transformation function.

Examples

```
iex> MapSet.new([1, 2, 1], fn x -> 2 * x end)
MapSet.new([2, 4])
```

put(map_set, value)

```
@spec put(t(val), new_val) :: t(val | new_val) when val:
value(), new_val: value()
```

Inserts `value` into `map_set` if `map_set` doesn't already contain it.

Examples

```
iex> MapSet.put(MapSet.new([1, 2, 3]), 3)
MapSet.new([1, 2, 3])
iex> MapSet.put(MapSet.new([1, 2, 3]), 4)
MapSet.new([1, 2, 3, 4])
```

reject(map_set, fun)

(since 1.14.0)

```
@spec reject(t(a), (a -> as boolean(term()))) :: t(a) when a:
value()
```

Returns a set by excluding the elements from `map_set` for which invoking `fun` returns a truthy value.

See also `filter/2`.

Examples

```
iex> MapSet.reject(MapSet.new(1..5), fn x -> rem(x, 2) !=
0 end)
MapSet.new([2, 4])

iex> MapSet.reject(MapSet.new(["a", :b, "c"]),
&is_atom/1)
MapSet.new(["a", "c"])
```

size(map_set)

```
@spec size(t()) :: non_neg_integer()
```

Returns the number of elements in `map_set`.

Examples

```
iex> MapSet.size(MapSet.new([1, 2, 3]))
3
```

split_with(map_set, fun)

(since 1.15.0)

```
@spec split_with(t(), (term() -> as_boolean(term()))) ::
{t(), t()}
```

Splits the `map_set` into two `MapSet`s according to the given function `fun`.

`fun` receives each element in the `map_set` as its only argument. Returns a tuple with the first `MapSet` containing all the elements in `map_set` for which applying `fun` returned a truthy value, and a second `MapSet` with all the elements for which applying `fun` returned a falsy value (`false` or `nil`).

Examples

```
iex> {while_true, while_false} =
MapSet.split_with(MapSet.new([1, 2, 3, 4]), fn v ->
rem(v, 2) == 0 end)
iex> while_true
MapSet.new([2, 4])
iex> while_false
MapSet.new([1, 3])

iex> {while_true, while_false} =
MapSet.split_with(MapSet.new(), fn {_k, v} -> v > 50 end)
iex> while_true
MapSet.new([])
iex> while_false
MapSet.new([])
```

subset?(map_set1, map_set2)

```
@spec subset?(t(), t()) :: boolean()
```

Checks if `map_set1`'s members are all contained in `map_set2`.

This function checks if `map_set1` is a subset of `map_set2`.

Examples

```
iex> MapSet.subset?(MapSet.new([1, 2]), MapSet.new([1, 2, 3]))
true
iex> MapSet.subset?(MapSet.new([1, 2, 3]), MapSet.new([1, 2]))
false
```

symmetric_difference(map_set1, map_set2)

(since 1.14.0)

```
@spec symmetric_difference(t(val1), t(val2)) :: t(val1 | val2)
when val1: value(), val2: value()
```

Returns a set with elements that are present in only one but not both sets.

Examples

```
iex> MapSet.symmetric_difference(MapSet.new([1, 2, 3]), MapSet.new([2, 3, 4]))
MapSet.new([1, 4])
```

to_list(map_set)

```
@spec to_list(t(val)) :: [val] when val: value()
```

Converts `map_set` to a list.

Examples

```
iex> MapSet.to_list(MapSet.new([1, 2, 3]))
[1, 2, 3]
```

union(map_set1, map_set2)

```
@spec union(t(val1), t(val2)) :: t(val1 | val2) when val1: value(), val2: value()
```

Returns a set containing all members of `map_set1` and `map_set2`.

Examples

```
iex> MapSet.union(MapSet.new([1, 2]), MapSet.new([2, 3, 4]))
MapSet.new([1, 2, 3, 4])
```

Range

Ranges represent a sequence of zero, one or many, ascending or descending integers with a common difference called step.

The most common form of creating and matching on ranges is via the `first..last` and `first..last//step` notations, auto-imported from `Kernel`:

```
iex> 1 in 1..10
true
iex> 5 in 1..10
true
iex> 10 in 1..10
true
```

Ranges are always inclusive in Elixir. When a step is defined, integers will only belong to the range if they match the step:

```
iex> 5 in 1..10//2
true
iex> 4 in 1..10//2
false
```

When defining a range without a step, the step will be defined based on the first and last position of the range, If `last >= first`, it will be an increasing range with a step of 1. Otherwise, it is a decreasing range. Note, however, implicit decreasing ranges are deprecated. Therefore, if you need a decreasing range from `3` to `1`, prefer to write `3..1//-1` instead.

`../0` can also be used as a shortcut to create the range `0..-1//1`, also known as the full-slice range:

```
iex> ..
0..-1//1
```

Use cases

Ranges typically have two uses in Elixir: as a collection or to represent a slice of another data structure.

Ranges as collections

Ranges in Elixir are enumerables and therefore can be used with the `Enum` module:

```
iex> Enum.to_list(1..3)
[1, 2, 3]
iex> Enum.to_list(3..1//-1)
[3, 2, 1]
iex> Enum.to_list(1..5//2)
[1, 3, 5]
```

Ranges may also have a single element:

```
iex> Enum.to_list(1..1)
[1]
iex> Enum.to_list(1..1//2)
[1]
```

Or even no elements at all:

```
iex> Enum.to_list(10..0//1)
[]
iex> Enum.to_list(0..10//-1)
[]
```

The full-slice range, returned by `../0`, is an empty collection:

```
iex> Enum.to_list(..)
[]
```

Ranges as slices

Ranges are also frequently used to slice collections. You can slice strings or any enumerable:

```
iex> String.slice("elixir", 1..4)
"lixi"
iex> Enum.slice([0, 1, 2, 3, 4, 5], 1..4)
[1, 2, 3, 4]
```

In those cases, the first and last values of the range are mapped to positions in the collections.

If a negative number is given, it maps to a position from the back:

```
iex> String.slice("elixir", 1..-2//1)
"lixi"
iex> Enum.slice([0, 1, 2, 3, 4, 5], 1..-2//1)
[1, 2, 3, 4]
```

The range `0..-1//1`, returned by `../0`, returns the collection as is, which is why it is called the full-slice range:

```
iex> String.slice("elixir", ..)
"elixir"
iex> Enum.slice([0, 1, 2, 3, 4, 5], ..)
[0, 1, 2, 3, 4, 5]
```

Definition

An increasing range `first..last//step` is a range from `first` to `last` increasing by `step` where `step` must be a positive integer and all values `v` must be `first <= v and v <= last`. Therefore, a range `10..0//1` is an empty range because there is no value `v` that is `10 <= v and v <= 0`.

Similarly, a decreasing range `first..last//step` is a range from `first` to `last` decreasing by `step` where `step` must be a negative integer and values `v` must be `first >= v and v >= last`. Therefore, a range `0..10//-1` is an empty range because there is no value `v` that is `0 >= v and v >= 10`.

Representation

Internally, ranges are represented as structs:

```
iex> range = 1..9//2
1..9//2
iex> first..last//step = range
iex> first
1
iex> last
```

```
9
iex> step
2
iex> range.step
2
```

You can access the range fields (`first`, `last`, and `step`) directly but you should not modify nor create ranges by hand. Instead use the proper operators or `new/2` and `new/3`.

Ranges implement the `Enumerable` protocol with memory efficient versions of all `Enumerable` callbacks:

```
iex> range = 1..10
1..10
iex> Enum.reduce(range, 0, fn i, acc -> i * i + acc end)
385
iex> Enum.count(range)
10
iex> Enum.member?(range, 11)
false
iex> Enum.member?(range, 8)
true
```

Such function calls are efficient memory-wise no matter the size of the range. The implementation of the `Enumerable` protocol uses logic based solely on the endpoints and does not materialize the whole list of integers.

Summary

Types

limit()
step()
t()
t(first, last)

Functions

disjoint?(range1, range2)

Checks if two ranges are disjoint.

new(first, last)

Creates a new range.

new(first, last, step)

Creates a new range with `step`.

shift(range, steps_to_shift)

Shifts a range by the given number of steps.

size(range)

Returns the size of `range`.

split(range, split)

Splits a range in two.

to_list(arg1)

Converts a range to a list.

Types

limit()

```
@type limit() :: integer()
```

step()

```
@type step() :: pos_integer() | neg_integer()
```

t()

```
@type t() :: %Range{first: limit(), last: limit(), step:
step()}
```

t(first, last)

```
@type t(first, last) :: %Range{first: first, last: last,
step: step()}
```

Functions

disjoint?(range1, range2)

(since 1.8.0)

```
@spec disjoint?(t(), t()) :: boolean()
```

Checks if two ranges are disjoint.

Examples

```
iex> Range.disjoint?(1..5, 6..9)
true
iex> Range.disjoint?(5..1//-1, 6..9)
true
iex> Range.disjoint?(1..5, 5..9)
false
iex> Range.disjoint?(1..5, 2..7)
false
```

Steps are also considered when computing the ranges to be disjoint:

```
iex> Range.disjoint?(1..10//2, 2..10//2)
true

# First element in common is 29
iex> Range.disjoint?(1..100//14, 8..100//21)
false
iex> Range.disjoint?(57..-1//-14, 8..100//21)
false
iex> Range.disjoint?(1..100//14, 50..8//-21)
false
iex> Range.disjoint?(1..28//14, 8..28//21)
true

# First element in common is 14
```

```
iex> Range.disjoint?(2..28//3, 9..28//5)
false
iex> Range.disjoint?(26..2//-3, 29..9//-5)
false

# Starting from the back without alignment
iex> Range.disjoint?(27..11//-3, 30..0//-7)
true
```

new(first, last)

```
@spec new(limit(), limit()) :: t()
```

Creates a new range.

If `first` is less than `last`, the range will be increasing from `first` to `last`. If `first` is equal to `last`, the range will contain one element, which is the number itself.

If `first` is greater than `last`, the range will be decreasing from `first` to `last`, albeit this behavior is deprecated. Therefore, it is advised to explicitly list the step with `new/3`.

Examples

```
iex> Range.new(-100, 100)
-100..100
```

new(first, last, step)

(since 1.12.0)

```
@spec new(limit(), limit(), step()) :: t()
```

Creates a new range with `step`.

Examples

```
iex> Range.new(-100, 100, 2)
-100..100//2
```

shift(range, steps_to_shift)

(since 1.14.0)

```
@spec shift(t(), integer()) :: t()
```

Shifts a range by the given number of steps.

Examples

```
iex> Range.shift(0..10, 1)
1..11
iex> Range.shift(0..10, 2)
2..12

iex> Range.shift(0..10//2, 2)
4..14//2
iex> Range.shift(10..0//-2, 2)
6..-4//-2
```

size(range)

(since 1.12.0)

```
@spec size(t()) :: non_neg_integer()
```

Returns the size of `range`.

Examples

```
iex> Range.size(1..10)
10
iex> Range.size(1..10//2)
5
iex> Range.size(1..10//3)
4
iex> Range.size(1..10//-1)
0

iex> Range.size(10..1//-1)
10
iex> Range.size(10..1//-2)
5
iex> Range.size(10..1//-3)
4
iex> Range.size(10..1//1)
0
```

split(range, split)

(since 1.15.0)

```
@spec split(t(), integer()) :: {t(), t()}
```

Splits a range in two.

It returns a tuple of two elements.

If `split` is less than the number of elements in the range, the first element in the range will have `split` entries and the second will have all remaining entries.

If `split` is more than the number of elements in the range, the second range in the tuple will emit zero elements.

Examples

Increasing ranges:

```
iex> Range.split(1..5, 2)
{1..2, 3..5}

iex> Range.split(1..5//2, 2)
{1..3//2, 5..5//2}

iex> Range.split(1..5//2, 0)
{1..-1//2, 1..5//2}

iex> Range.split(1..5//2, 10)
{1..5//2, 7..5//2}
```

Decreasing ranges can also be split:

```
iex> Range.split(5..1//-1, 2)
{5..4//-1, 3..1//-1}
```

```
iex> Range.split(5..1//-2, 2)
{5..3//-2, 1..1//-2}

iex> Range.split(5..1//-2, 0)
{5..7//-2, 5..1//-2}

iex> Range.split(5..1//-2, 10)
{5..1//-2, -1..1//-2}
```

Empty ranges preserve their property but still return empty ranges:

```
iex> Range.split(2..5//-1, 2)
{2..3//-1, 4..5//-1}

iex> Range.split(2..5//-1, 10)
{2..3//-1, 4..5//-1}

iex> Range.split(5..2//1, 2)
{5..4//1, 3..2//1}

iex> Range.split(5..2//1, 10)
{5..4//1, 3..2//1}
```

If the number to split is negative, it splits from the back:

```
iex> Range.split(1..5, -2)
{1..3, 4..5}

iex> Range.split(5..1//-1, -2)
{5..3//-1, 2..1//-1}
```

If it is negative and greater than the elements in the range, the first element of the tuple will be an empty range:

```
iex> Range.split(1..5, -10)
{1..0//1, 1..5}

iex> Range.split(5..1//-1, -10)
{5..6//-1, 5..1//-1}
```

Properties

When a range is split, the following properties are observed. Given `split(input)` returns `{left, right}`, we have:

```
assert input.first == left.first
assert input.last == right.last
assert input.step == left.step
assert input.step == right.step
assert Range.size(input) == Range.size(left) +
Range.size(right)
```

to_list(arg1)

(since 1.15.0)

```
@spec to_list(t()) :: [integer()]
```

Converts a range to a list.

Examples

```
iex> Range.to_list(0..5)
[0, 1, 2, 3, 4, 5]
```

```
iex> Range.to_list(-3..0)
[-3, -2, -1, 0]
```

Stream

Functions for creating and composing streams.

Streams are composable, lazy enumerables (for an introduction on enumerables, see the `Enum` module). Any enumerable that generates elements one by one during enumeration is called a stream. For example, Elixir's `Range` is a stream:

```
iex> range = 1..5
1..5
iex> Enum.map(range, &(&1 * 2))
[2, 4, 6, 8, 10]
```

In the example above, as we mapped over the range, the elements being enumerated were created one by one, during enumeration. The `Stream` module allows us to map the range, without triggering its enumeration:

```
iex> range = 1..3
iex> stream = Stream.map(range, &(&1 * 2))
iex> Enum.map(stream, &(&1 + 1))
[3, 5, 7]
```

Note that we started with a range and then we created a stream that is meant to multiply each element in the range by 2. At this point, no computation was done. Only when `Enum.map/2` is called we actually enumerate over each element in the range, multiplying it by 2 and adding 1. We say the functions in `Stream` are *lazy* and the functions in `Enum` are *eager*.

Due to their laziness, streams are useful when working with large (or even infinite) collections. When chaining many operations with `Enum`, intermediate lists are created, while `Stream` creates a recipe of computations that are executed at a later moment. Then when the stream is consumed later on, most commonly by using a function in the `Enum` module, the stream will emit its elements one by one.

Let's see another example:

```
1..3
|> Enum.map(&IO.inspect(&1))
|> Enum.map(&(&1 * 2))
|> Enum.map(&IO.inspect(&1))
1
2
3
2
4
6
#=> [2, 4, 6]
```

Note that we first printed each element in the list, then multiplied each element by 2 and finally printed each new value. In this example, the list was enumerated three times. Let's see an example with streams:

```
stream = 1..3
|> Stream.map(&IO.inspect(&1))
|> Stream.map(&(&1 * 2))
|> Stream.map(&IO.inspect(&1))
Enum.to_list(stream)
1
2
2
4
```

```
3
6
#=> [2, 4, 6]
```

Although the end result is the same, the order in which the elements were printed changed! With streams, we print the first element and then print its double. In this example, the list was enumerated just once!

That's what we meant when we said earlier that streams are composable, lazy enumerables. Note that we could call `Stream.map/2` multiple times, effectively composing the streams and keeping them lazy. The computations are only performed when you call a function from the `Enum` module.

Like with `Enum`, the functions in this module work in linear time. This means that, the time it takes to perform an operation grows at the same rate as the length of the list. This is expected on operations such as `Stream.map/2`. After all, if we want to traverse every element on a stream, the longer the stream, the more elements we need to traverse, and the longer it will take.

Creating Streams

There are many functions in Elixir's standard library that return streams, some examples are:

- `IO.stream/2` - streams input lines, one by one
- `URI.query_decoder/1` - decodes a query string, pair by pair

This module also provides many convenience functions for creating streams, like `Stream.cycle/1`, `Stream.unfold/2`, `Stream.resource/3` and more.

Do not check for `Stream` structs

While some functions in this module may return the `Stream` struct, you must never explicitly check for the `Stream` struct, as streams may come in several shapes, such as `IO.Stream`, `File.Stream`, or even `Range`s.

The functions in this module only guarantee to return enumerables and their implementation (structs, anonymous functions, etc) may change at any time. For example, a function that returns an anonymous function today may return a struct in future releases.

Instead of checking for a particular type, you must instead write assertive code that assumes you have an enumerable, using the functions in the `Enum` or `Stream` module accordingly.

Summary

Types

acc()
default()
element()
index()

Zero-based index.

timer()

Functions

chunk_by(enum, fun)

Chunks the `enum` by buffering elements for which `fun` returns the same value.

chunk_every(enum, count)

Shortcut to `chunk_every(enum, count, count)`.

chunk_every(enum, count, step, leftover \\ [])

Streams the enumerable in chunks, containing `count` elements each, where each new chunk starts `step` elements into the enumerable.

chunk_while(enum, acc, chunk_fun, after_fun)

Chunks the `enum` with fine grained control when every chunk is emitted.

concat(enumerables)

Creates a stream that enumerates each enumerable in an enumerable.

concat(first, second)

Creates a stream that enumerates the first argument, followed by the second.

cycle(enumerable)

Creates a stream that cycles through the given enumerable, infinitely.

dedup(enum)

Creates a stream that only emits elements if they are different from the last emitted element.

dedup_by(enum, fun)

Creates a stream that only emits elements if the result of calling `fun` on the element is different from the (stored) result of calling `fun` on the last emitted element.

drop(enum, n)

Lazily drops the next `n` elements from the enumerable.

drop_every(enum, nth)

Creates a stream that drops every `nth` element from the enumerable.

drop_while(enum, fun)

Lazily drops elements of the enumerable while the given function returns a truthy value.

duplicate(value, n)

Duplicates the given element `n` times in a stream.

each(enum, fun)

Executes the given function for each element.

filter(enum, fun)

Creates a stream that filters elements according to the given function on enumeration.

flat_map(enum, mapper)

Maps the given `fun` over `enumerable` and flattens the result.

from_index(fun_or_offset \\ 0)

Builds a stream from an index, either starting from offset, or given by function.

intersperse(enumerable, intersperse_element)

Lazily intersperses `intersperse_element` between each element of the enumeration.

interval(n)

Creates a stream that emits a value after the given period `n` in milliseconds.

into(enum, collectable, transform \\ fn x -> x end)

Injects the stream values into the given collectable as a side-effect.

iterate(start_value, next_fun)

Emits a sequence of values, starting with `start_value`.

map(enum, fun)

Creates a stream that will apply the given function on enumeration.

map_every(enum, nth, fun)

Creates a stream that will apply the given function on every `nth` element from the enumerable.

reject(enum, fun)

Creates a stream that will reject elements according to the given function on enumeration.

repeatedly(generator_fun)

Returns a stream generated by calling `generator_fun` repeatedly.

resource(start_fun, next_fun, after_fun)

Emits a sequence of values for the given resource.

run(stream)

Runs the given stream.

scan(enum, fun)

Creates a stream that applies the given function to each element, emits the result and uses the same result as the accumulator for the next computation. Uses the first element in the enumerable as the starting value.

scan(enum, acc, fun)

Creates a stream that applies the given function to each element, emits the result and uses the same result as the accumulator for the next computation. Uses the given `acc` as the starting value.

take(enum, count)

Lazily takes the next `count` elements from the enumerable and stops enumeration.

take_every(enum, nth)

Creates a stream that takes every `nth` element from the enumerable.

take_while(enum, fun)

Lazily takes elements of the enumerable while the given function returns a truthy value.

timer(n)

Creates a stream that emits a single value after `n` milliseconds.

transform(enum, acc, reducer)

Transforms an existing stream.

transform(enum, start_fun, reducer, after_fun)

Similar to `Stream.transform/5`, except `last_fun` is not supplied.

transform(enum, start_fun, reducer, last_fun, after_fun)

Transforms an existing stream with function-based start, last, and after callbacks.

unfold(next_acc, next_fun)

Emits a sequence of values for the given accumulator.

uniq(enum)

Creates a stream that only emits elements if they are unique.

uniq_by(enum, fun)

Creates a stream that only emits elements if they are unique, by removing the elements for which function `fun` returned duplicate elements.

with_index(enum, fun_or_offset \\ 0)

Creates a stream where each element in the enumerable will be wrapped in a tuple alongside its index or according to a given function.

zip(enumerables)

Zips corresponding elements from a finite collection of enumerables into one stream of tuples.

zip(enumerable1, enumerable2)

Zips two enumerables together, lazily.

zip_with(enumerables, zip_fun)

Lazily zips corresponding elements from a finite collection of enumerables into a new enumerable, transforming them with the `zip_fun` function as it goes.

zip_with(enumerable1, enumerable2, zip_fun)

Lazily zips corresponding elements from two enumerables into a new one, transforming them with the `zip_fun` function as it goes.

Types

acc()

```
@type acc() :: any()
```

default()

```
@type default() :: any()
```

element()

```
@type element() :: any()
```

index()

```
@type index() :: non_neg_integer()
```

Zero-based index.

timer()

```
@type timer() :: non_neg_integer() | :infinity
```

Functions

chunk_by(enum, fun)

```
@spec chunk_by(Enumerable.t(), (element() -> any())) ::
Enumerable.t()
```

Chunks the `enum` by buffering elements for which `fun` returns the same value.

Elements are only emitted when `fun` returns a new value or the `enum` finishes.

Examples

```
iex> stream = Stream.chunk_by([1, 2, 2, 3, 4, 4, 6, 7,
7], &(rem(&1, 2) == 1))
iex> Enum.to_list(stream)
[[1], [2, 2], [3], [4, 4, 6], [7, 7]]
```

chunk_every(enum, count)

(since 1.5.0)

```
@spec chunk_every(Enumerable.t(), pos_integer()) ::
Enumerable.t()
```

Shortcut to `chunk_every(enum, count, count)`.

chunk_every(enum, count, step, leftover \\ [])

(since 1.5.0)

```
@spec chunk_every(
  Enumerable.t(),
  pos_integer(),
  pos_integer(),
  Enumerable.t() | :discard
) ::
  Enumerable.t()
```

Streams the enumerable in chunks, containing `count` elements each, where each new chunk starts `step` elements into the enumerable.

`step` is optional and, if not passed, defaults to `count`, i.e. chunks do not overlap. Chunking will stop as soon as the collection ends or when we emit an incomplete chunk.

If the last chunk does not have `count` elements to fill the chunk, elements are taken from `leftover` to fill in the chunk. If `leftover` does not have enough elements to fill the chunk, then a partial chunk is returned with less than `count` elements.

If `:discard` is given in `leftover`, the last chunk is discarded unless it has exactly `count` elements.

Examples

```
iex> Stream.chunk_every([1, 2, 3, 4, 5, 6], 2) |>
Enum.to_list()
[[1, 2], [3, 4], [5, 6]]
```

```
iex> Stream.chunk_every([1, 2, 3, 4, 5, 6], 3, 2,
:discard) |> Enum.to_list()
[[1, 2, 3], [3, 4, 5]]

iex> Stream.chunk_every([1, 2, 3, 4, 5, 6], 3, 2, [7]) |>
Enum.to_list()
[[1, 2, 3], [3, 4, 5], [5, 6, 7]]

iex> Stream.chunk_every([1, 2, 3, 4, 5, 6], 3, 3, []) |>
Enum.to_list()
[[1, 2, 3], [4, 5, 6]]

iex> Stream.chunk_every([1, 2, 3, 4], 3, 3,
Stream.cycle([0])) |> Enum.to_list()
[[1, 2, 3], [4, 0, 0]]
```

chunk_while(enum, acc, chunk_fun, after_fun)

(since 1.5.0)

```
@spec chunk_while(
  Enumerable.t(),
  acc(),
  (element(), acc() -> {:cont, chunk, acc()} | {:cont, acc()}
| {:halt, acc()}),
  (acc() -> {:cont, chunk, acc()} | {:cont, acc()})
) :: Enumerable.t()
when chunk: any()
```

Chunks the `enum` with fine grained control when every chunk is emitted.

`chunk_fun` receives the current element and the accumulator and must return `{:cont, element, acc}` to emit the given chunk and continue

with accumulator or `{:cont, acc}` to not emit any chunk and continue with the return accumulator.

`after_fun` is invoked when iteration is done and must also return `{:cont, element, acc}` or `{:cont, acc}`.

Examples

```
iex> chunk_fun = fn element, acc ->
...>   if rem(element, 2) == 0 do
...>     {:cont, Enum.reverse([element | acc]), []}
...>   else
...>     {:cont, [element | acc]}
...>   end
...> end
iex> after_fun = fn
...>   [] -> {:cont, []}
...>   acc -> {:cont, Enum.reverse(acc), []}
...> end
iex> stream = Stream.chunk_while(1..10, [], chunk_fun,
after_fun)
iex> Enum.to_list(stream)
[[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]
```

concat(enumerables)

```
@spec concat(Enumerable.t()) :: Enumerable.t()
```

Creates a stream that enumerates each enumerable in an enumerable.

Examples

```
iex> stream = Stream.concat([1..3, 4..6, 7..9])
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

concat(first, second)

```
@spec concat(Enumerable.t(), Enumerable.t()) ::
Enumerable.t()
```

Creates a stream that enumerates the first argument, followed by the second.

Examples

```
iex> stream = Stream.concat(1..3, 4..6)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5, 6]

iex> stream1 = Stream.cycle([1, 2, 3])
iex> stream2 = Stream.cycle([4, 5, 6])
iex> stream = Stream.concat(stream1, stream2)
iex> Enum.take(stream, 6)
[1, 2, 3, 1, 2, 3]
```

cycle(enumerable)

```
@spec cycle(Enumerable.t()) :: Enumerable.t()
```

Creates a stream that cycles through the given enumerable, infinitely.

Examples

```
iex> stream = Stream.cycle([1, 2, 3])
iex> Enum.take(stream, 5)
[1, 2, 3, 1, 2]
```

dedup(enum)

```
@spec dedup(Enumerable.t()) :: Enumerable.t()
```

Creates a stream that only emits elements if they are different from the last emitted element.

This function only ever needs to store the last emitted element.

Elements are compared using `===/2`.

Examples

```
iex> Stream.dedup([1, 2, 3, 3, 2, 1]) |> Enum.to_list()
[1, 2, 3, 2, 1]
```

dedup_by(enum, fun)

```
@spec dedup_by(Enumerable.t(), (element() -> term())) ::
Enumerable.t()
```

Creates a stream that only emits elements if the result of calling `fun` on the element is different from the (stored) result of calling `fun` on the last

emitted element.

Examples

```
iex> Stream.dedup_by([{1, :x}, {2, :y}, {2, :z}, {1,
:x}], fn {x, _} -> x end) |> Enum.to_list()
[{1, :x}, {2, :y}, {1, :x}]
```

drop(enum, n)

```
@spec drop(Enumerable.t(), integer()) :: Enumerable.t()
```

Lazily drops the next `n` elements from the enumerable.

If a negative `n` is given, it will drop the last `n` elements from the collection. Note that the mechanism by which this is implemented will delay the emission of any element until `n` additional elements have been emitted by the enum.

Examples

```
iex> stream = Stream.drop(1..10, 5)
iex> Enum.to_list(stream)
[6, 7, 8, 9, 10]

iex> stream = Stream.drop(1..10, -5)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]
```

drop_every(enum, nth)

```
@spec drop_every(Enumerable.t(), non_neg_integer()) ::
Enumerable.t()
```

Creates a stream that drops every `nth` element from the enumerable.

The first element is always dropped, unless `nth` is 0.

`nth` must be a non-negative integer.

Examples

```
iex> stream = Stream.drop_every(1..10, 2)
iex> Enum.to_list(stream)
[2, 4, 6, 8, 10]

iex> stream = Stream.drop_every(1..1000, 1)
iex> Enum.to_list(stream)
[]

iex> stream = Stream.drop_every([1, 2, 3, 4, 5], 0)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]
```

drop_while(enum, fun)

```
@spec drop_while(Enumerable.t(), (element() ->
as_boolean(term()))) :: Enumerable.t()
```

Lazily drops elements of the enumerable while the given function returns a truthy value.

Examples

```
iex> stream = Stream.drop_while(1..10, &(&1 <= 5))
iex> Enum.to_list(stream)
[6, 7, 8, 9, 10]
```

duplicate(value, n)

(since 1.14.0)

```
@spec duplicate(any(), non_neg_integer()) :: Enumerable.t()
```

Duplicates the given element `n` times in a stream.

`n` is an integer greater than or equal to `0`.

If `n` is `0`, an empty stream is returned.

Examples

```
iex> stream = Stream.duplicate("hello", 0)
iex> Enum.to_list(stream)
[]

iex> stream = Stream.duplicate("hi", 1)
iex> Enum.to_list(stream)
["hi"]

iex> stream = Stream.duplicate("bye", 2)
iex> Enum.to_list(stream)
```

```
["bye", "bye"]

iex> stream = Stream.duplicate([1, 2], 3)
iex> Enum.to_list(stream)
[[1, 2], [1, 2], [1, 2]]
```

each(enum, fun)

```
@spec each(Enumerable.t(), (element() -> term())) ::
Enumerable.t()
```

Executes the given function for each element.

The values in the stream do not change, therefore this function is useful for adding side effects (like printing) to a stream. See `map/2` if producing a different stream is desired.

Examples

```
iex> stream = Stream.each([1, 2, 3], fn x -> send(self(),
x) end)
iex> Enum.to_list(stream)
iex> receive do: (x when is_integer(x) -> x)
1
iex> receive do: (x when is_integer(x) -> x)
2
iex> receive do: (x when is_integer(x) -> x)
3
```

filter(enum, fun)

```
@spec filter(Enumerable.t(), (element() ->
as boolean(term()))) :: Enumerable.t()
```

Creates a stream that filters elements according to the given function on enumeration.

Examples

```
iex> stream = Stream.filter([1, 2, 3], fn x -> rem(x, 2)
== 0 end)
iex> Enum.to_list(stream)
[2]
```

flat_map(enum, mapper)

```
@spec flat_map(Enumerable.t(), (element() -> Enumerable.t()))
:: Enumerable.t()
```

Maps the given `fun` over `enumerable` and flattens the result.

This function returns a new stream built by appending the result of invoking `fun` on each element of `enumerable` together.

Examples

```
iex> stream = Stream.flat_map([1, 2, 3], fn x -> [x, x *
2] end)
iex> Enum.to_list(stream)
[1, 2, 2, 4, 3, 6]

iex> stream = Stream.flat_map([1, 2, 3], fn x -> [[x]]
```

```
end)
iex> Enum.to_list(stream)
[[1], [2], [3]]
```

from_index(fun_or_offset \\ 0)

(since 1.17.0)

```
@spec from_index(integer()) :: Enumerable.t(integer())
@spec from_index((integer() -> return_value)) ::
Enumerable.t(return_value)
when return_value: term()
```

Builds a stream from an index, either starting from offset, or given by function.

May receive a function or an integer offset.

If an `offset` is given, it will emit elements from offset.

If a `function` is given, it will invoke the function with elements from offset.

Examples

```
iex> Stream.from_index() |> Enum.take(3)
[0, 1, 2]

iex> Stream.from_index(1) |> Enum.take(3)
[1, 2, 3]

iex> Stream.from_index(fn x -> x * 10 end) |>
```

```
Enum.take(3)
[0, 10, 20]
```

intersperse(enumerable, intersperse_element)

(since 1.6.0)

```
@spec intersperse(Enumerable.t(), any()) :: Enumerable.t()
```

Lazily intersperses `intersperse_element` between each element of the enumeration.

Examples

```
iex> Stream.intersperse([1, 2, 3], 0) |> Enum.to_list()
[1, 0, 2, 0, 3]

iex> Stream.intersperse([1], 0) |> Enum.to_list()
[1]

iex> Stream.intersperse([], 0) |> Enum.to_list()
[]
```

interval(n)

```
@spec interval(timer()) :: Enumerable.t()
```

Creates a stream that emits a value after the given period `n` in milliseconds.

The values emitted are an increasing counter starting at `0`. This operation will block the caller by the given interval every time a new element is streamed.

Do not use this function to generate a sequence of numbers. If blocking the caller process is not necessary, use `Stream.iterate(0, & &1 + 1)` instead.

Examples

```
iex> Stream.interval(10) |> Enum.take(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

into(enum, collectable, transform \\ fn x -> x end)

```
@spec into(Enumerable.t(), Collectable.t(), (term() -> term())) :: Enumerable.t()
```

Injects the stream values into the given collectable as a side-effect.

This function is often used with `run/1` since any evaluation is delayed until the stream is executed. See `run/1` for an example.

iterate(start_value, next_fun)

```
@spec iterate(element(), (element() -> element())) :: Enumerable.t()
```

Emits a sequence of values, starting with `start_value`.

Successive values are generated by calling `next_fun` on the previous value.

Examples

```
iex> Stream.iterate(1, &(&1 * 2)) |> Enum.take(5)
[1, 2, 4, 8, 16]
```

map(enum, fun)

```
@spec map(Enumerable.t(), (element() -> any())) ::
Enumerable.t()
```

Creates a stream that will apply the given function on enumeration.

Examples

```
iex> stream = Stream.map([1, 2, 3], fn x -> x * 2 end)
iex> Enum.to_list(stream)
[2, 4, 6]
```

map_every(enum, nth, fun)

(since 1.4.0)

```
@spec map_every(Enumerable.t(), non_neg_integer(), (element()
-> any())) ::
  Enumerable.t()
```

Creates a stream that will apply the given function on every `nth` element from the enumerable.

The first element is always passed to the given function.

`nth` must be a non-negative integer.

Examples

```
iex> stream = Stream.map_every(1..10, 2, fn x -> x * 2
end)
iex> Enum.to_list(stream)
[2, 2, 6, 4, 10, 6, 14, 8, 18, 10]

iex> stream = Stream.map_every([1, 2, 3, 4, 5], 1, fn x -
> x * 2 end)
iex> Enum.to_list(stream)
[2, 4, 6, 8, 10]

iex> stream = Stream.map_every(1..5, 0, fn x -> x * 2
end)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]
```

reject(enum, fun)

```
@spec reject(Enumerable.t(), (element() ->
as boolean(term()))) :: Enumerable.t()
```

Creates a stream that will reject elements according to the given function on enumeration.

Examples

```
iex> stream = Stream.reject([1, 2, 3], fn x -> rem(x, 2)
== 0 end)
iex> Enum.to_list(stream)
[1, 3]
```

repeatedly(generator_fun)

```
@spec repeatedly((-> element())) :: Enumerable.t()
```

Returns a stream generated by calling `generator_fun` repeatedly.

Examples

```
# Although not necessary, let's seed the random algorithm
iex> :rand.seed(:exsss, {1, 2, 3})
iex> Stream.repeatedly(&:rand.uniform/0) |> Enum.take(3)
[0.5455598952593053, 0.6039309974353404,
0.6684893034823949]
```

resource(start_fun, next_fun, after_fun)

```
@spec resource((-> acc()), (acc() -> {[element()], acc()} |
{:halt, acc()}), (acc() ->

term())) ::
  Enumerable.t()
```

Emits a sequence of values for the given resource.

Similar to `transform/3` but the initial accumulated value is computed lazily via `start_fun` and executes an `after_fun` at the end of enumeration (both in cases of success and failure).

Successive values are generated by calling `next_fun` with the previous accumulator (the initial value being the result returned by `start_fun`) and it must return a tuple containing a list of elements to be emitted and the next accumulator. The enumeration finishes if it returns `{:halt, acc}`.

As the function name suggests, this function is useful to stream values from resources.

Examples

```
Stream.resource(
  fn -> File.open!("sample") end,
  fn file ->
    case IO.read(file, :line) do
      data when is_binary(data) -> {[data], file}
      _ -> {:halt, file}
    end
  end,
  fn file -> File.close(file) end
)

iex> Stream.resource(
...>  fn ->
...>    {:ok, pid} = StringIO.open("string")
...>    pid
...>  end,
...>  fn pid ->
...>    case IO.getn(pid, "", 1) do
...>      :eof -> {:halt, pid}
```

```
...>       char -> {[char], pid}
...>     end
...>   end,
...>   fn pid -> StringIO.close(pid) end
...> ) |> Enum.to_list()
["s", "t", "r", "i", "n", "g"]
```

run(stream)

```
@spec run(Enumerable.t()) :: :ok
```

Runs the given stream.

This is useful when a stream needs to be run, for side effects, and there is no interest in its return result.

Examples

Open up a file, replace all `#` by `%` and stream to another file without loading the whole file in memory:

```
File.stream!("/path/to/file")
|> Stream.map(&String.replace(&1, "#", "%"))
|> Stream.into(File.stream!("/path/to/other/file"))
|> Stream.run()
```

No computation will be done until we call one of the `Enum` functions or `run/1`.

scan(enum, fun)

```
@spec scan(Enumerable.t(), (element(), acc() -> any())) ::
Enumerable.t()
```

Creates a stream that applies the given function to each element, emits the result and uses the same result as the accumulator for the next computation. Uses the first element in the enumerable as the starting value.

Examples

```
iex> stream = Stream.scan(1..5, &(&1 + &2))
iex> Enum.to_list(stream)
[1, 3, 6, 10, 15]
```

scan(enum, acc, fun)

```
@spec scan(Enumerable.t(), acc(), (element(), acc() ->
any())) :: Enumerable.t()
```

Creates a stream that applies the given function to each element, emits the result and uses the same result as the accumulator for the next computation. Uses the given `acc` as the starting value.

Examples

```
iex> stream = Stream.scan(1..5, 0, &(&1 + &2))
iex> Enum.to_list(stream)
[1, 3, 6, 10, 15]
```

take(enum, count)

```
@spec take(Enumerable.t(), integer()) :: Enumerable.t()
```

Lazily takes the next `count` elements from the enumerable and stops enumeration.

If a negative `count` is given, the last `count` values will be taken. For such, the collection is fully enumerated keeping up to `2 * count` elements in memory. Once the end of the collection is reached, the last `count` elements will be executed. Therefore, using a negative `count` on an infinite collection will never return.

Examples

```
iex> stream = Stream.take(1..100, 5)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]

iex> stream = Stream.take(1..100, -5)
iex> Enum.to_list(stream)
[96, 97, 98, 99, 100]

iex> stream = Stream.cycle([1, 2, 3]) |> Stream.take(5)
iex> Enum.to_list(stream)
[1, 2, 3, 1, 2]
```

take_every(enum, nth)

```
@spec take_every(Enumerable.t(), non_neg_integer()) ::
Enumerable.t()
```

Creates a stream that takes every `nth` element from the enumerable.

The first element is always included, unless `nth` is 0.

`nth` must be a non-negative integer.

Examples

```
iex> stream = Stream.take_every(1..10, 2)
iex> Enum.to_list(stream)
[1, 3, 5, 7, 9]

iex> stream = Stream.take_every([1, 2, 3, 4, 5], 1)
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]

iex> stream = Stream.take_every(1..1000, 0)
iex> Enum.to_list(stream)
[]
```

take_while(enum, fun)

```
@spec take_while(Enumerable.t(), (element() ->
as_boolean(term()))) :: Enumerable.t()
```

Lazily takes elements of the enumerable while the given function returns a truthy value.

Examples

```
iex> stream = Stream.take_while(1..100, &(&1 <= 5))
iex> Enum.to_list(stream)
[1, 2, 3, 4, 5]
```

timer(n)

```
@spec timer(timer()) :: Enumerable.t()
```

Creates a stream that emits a single value after `n` milliseconds.

The value emitted is `0`. This operation will block the caller by the given time until the element is streamed.

Examples

```
iex> Stream.timer(10) |> Enum.to_list()
[0]
```

transform(enum, acc, reducer)

```
@spec transform(Enumerable.t(), acc, fun) :: Enumerable.t()
when fun: (element(), acc -> {Enumerable.t(), acc} | {:halt, acc}), acc: term()
```

Transforms an existing stream.

It expects an accumulator and a function that receives two arguments, the stream element and the updated accumulator. It must return a tuple, where the first element is a new stream (often a list) or the atom `:halt`,

and the second element is the accumulator to be used by the next element.

Note: this function is equivalent to `Enum.flat_map_reduce/3`, except this function does not return the accumulator once the stream is processed.

Examples

`Stream.transform/3` is useful as it can be used as the basis to implement many of the functions defined in this module. For example, we can implement `Stream.take(enum, n)` as follows:

```
iex> enum = 1001..9999
iex> n = 3
iex> stream = Stream.transform(enum, 0, fn i, acc ->
...>   if acc < n, do: {[i], acc + 1}, else: {:halt, acc}
...> end)
iex> Enum.to_list(stream)
[1001, 1002, 1003]
```

`Stream.transform/5` further generalizes this function to allow wrapping around resources.

transform(enum, start_fun, reducer, after_fun)

```
@spec transform(Enumerable.t(), start_fun, reducer,
after_fun) :: Enumerable.t()
when start_fun: (-> acc),
     reducer: (element(), acc -> {Enumerable.t(), acc} |
{:halt, acc}),
     after_fun: (acc -> term()),
     acc: term()
```

Similar to `Stream.transform/5`, except `last_fun` is not supplied.

This function can be seen as a combination of `Stream.resource/3` with `Stream.transform/3`.

transform(enum, start_fun, reducer, last_fun, after_fun)

(since 1.14.0)

```
@spec transform(Enumerable.t(), start_fun, reducer, last_fun,
after_fun) ::
  Enumerable.t()
when start_fun: (-> acc),
     reducer: (element(), acc -> {Enumerable.t(), acc} |
{:halt, acc}),
     last_fun: (acc -> {Enumerable.t(), acc} | {:halt, acc}),
     after_fun: (acc -> term()),
     acc: term()
```

Transforms an existing stream with function-based start, last, and after callbacks.

Once transformation starts, `start_fun` is invoked to compute the initial accumulator. Then, for each element in the enumerable, the `reducer` function is invoked with the element and the accumulator, returning new elements and a new accumulator, as in `transform/3`.

Once the collection is done, `last_fun` is invoked with the accumulator to emit any remaining items. Then `after_fun` is invoked, to close any resource, but not emitting any new items. `last_fun` is only invoked if the given enumerable terminates successfully (either because it is done or it halted itself). `after_fun` is always invoked, therefore `after_fun` must be the one used for closing resources.

unfold(next_acc, next_fun)

```
@spec unfold(acc(), (acc() -> {element(), acc()} | nil)) ::
Enumerable.t()
```

Emits a sequence of values for the given accumulator.

Successive values are generated by calling `next_fun` with the previous accumulator and it must return a tuple with the current value and next accumulator. The enumeration finishes if it returns `nil`.

Examples

To create a stream that counts down and stops before zero:

```
iex> Stream.unfold(5, fn
...>   0 -> nil
...>   n -> {n, n - 1}
...> end) |> Enum.to_list()
[5, 4, 3, 2, 1]
```

If `next_fun` never returns `nil`, the returned stream is *infinite*:

```
iex> Stream.unfold(0, fn
...>   n -> {n, n + 1}
...> end) |> Enum.take(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

iex> Stream.unfold(1, fn
...>   n -> {n, n * 2}
```

```
...> end) |> Enum.take(10)
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

uniq(enum)

```
@spec uniq(Enumerable.t()) :: Enumerable.t()
```

Creates a stream that only emits elements if they are unique.

Keep in mind that, in order to know if an element is unique or not, this function needs to store all unique values emitted by the stream. Therefore, if the stream is infinite, the number of elements stored will grow infinitely, never being garbage-collected.

Examples

```
iex> Stream.uniq([1, 2, 3, 3, 2, 1]) |> Enum.to_list()
[1, 2, 3]
```

uniq_by(enum, fun)

```
@spec uniq_by(Enumerable.t(), (element() -> term())) ::
Enumerable.t()
```

Creates a stream that only emits elements if they are unique, by removing the elements for which function `fun` returned duplicate elements.

The function `fun` maps every element to a term which is used to determine if two elements are duplicates.

Keep in mind that, in order to know if an element is unique or not, this function needs to store all unique values emitted by the stream. Therefore, if the stream is infinite, the number of elements stored will grow infinitely, never being garbage-collected.

Example

```
iex> Stream.uniq_by([{1, :x}, {2, :y}, {1, :z}], fn {x,
_} -> x end) |> Enum.to_list()
[{1, :x}, {2, :y}]

iex> Stream.uniq_by([a: {:tea, 2}, b: {:tea, 2}, c:
{:coffee, 1}], fn {_, y} -> y end) |> Enum.to_list()
[a: {:tea, 2}, c: {:coffee, 1}]
```

with_index(enum, fun_or_offset \\ 0)

```
@spec with_index(Enumerable.t(), integer()) ::
Enumerable.t({element(), integer()})
@spec with_index(Enumerable.t(), (element(), index() ->
return_value)) ::
  Enumerable.t(return_value)
when return_value: term()
```

Creates a stream where each element in the enumerable will be wrapped in a tuple alongside its index or according to a given function.

May receive a function or an integer offset.

If an `offset` is given, it will index from the given offset instead of from zero.

If a `function` is given, it will index by invoking the function for each element and index (zero-based) of the enumerable.

Examples

```
iex> stream = Stream.with_index([1, 2, 3])
iex> Enum.to_list(stream)
[{1, 0}, {2, 1}, {3, 2}]

iex> stream = Stream.with_index([1, 2, 3], 3)
iex> Enum.to_list(stream)
[{1, 3}, {2, 4}, {3, 5}]

iex> stream = Stream.with_index([1, 2, 3], fn x, index ->
x + index end)
iex> Enum.to_list(stream)
[1, 3, 5]
```

zip(enumerables)

(since 1.4.0)

```
@spec zip(enumerables) :: Enumerable.t()
when enumerables: [Enumerable.t()] | Enumerable.t()
```

Zips corresponding elements from a finite collection of enumerables into one stream of tuples.

The zipping finishes as soon as any enumerable in the given collection completes.

Examples

```
iex> concat = Stream.concat(1..3, 4..6)
iex> cycle = Stream.cycle(["foo", "bar", "baz"])
iex> Stream.zip([concat, [:a, :b, :c], cycle]) |>
Enum.to_list()
[{1, :a, "foo"}, {2, :b, "bar"}, {3, :c, "baz"}]
```

zip(enumerable1, enumerable2)

```
@spec zip(Enumerable.t(), Enumerable.t()) :: Enumerable.t()
```

Zips two enumerables together, lazily.

Because a list of two-element tuples with atoms as the first tuple element is a keyword list (`Keyword`), zipping a first `Stream` of atoms with a second `Stream` of any kind creates a `Stream` that generates a keyword list.

The zipping finishes as soon as either enumerable completes.

Examples

```
iex> concat = Stream.concat(1..3, 4..6)
iex> cycle = Stream.cycle([:a, :b, :c])
iex> Stream.zip(concat, cycle) |> Enum.to_list()
[{1, :a}, {2, :b}, {3, :c}, {4, :a}, {5, :b}, {6, :c}]
iex> Stream.zip(cycle, concat) |> Enum.to_list()
[a: 1, b: 2, c: 3, a: 4, b: 5, c: 6]
```

zip_with(enumerables, zip_fun)

(since 1.12.0)

```
@spec zip_with(enumerables, (Enumerable.t() -> term())) ::
Enumerable.t()
when enumerables: [Enumerable.t()] | Enumerable.t()
```

Lazily zips corresponding elements from a finite collection of enumerables into a new enumerable, transforming them with the `zip_fun` function as it goes.

The first element from each of the enums in `enumerables` will be put into a list which is then passed to the one-arity `zip_fun` function. Then, the second elements from each of the enums are put into a list and passed to `zip_fun`, and so on until any one of the enums in `enumerables` completes.

Returns a new enumerable with the results of calling `zip_fun`.

Examples

```
iex> concat = Stream.concat(1..3, 4..6)
iex> Stream.zip_with([concat, concat], fn [a, b] -> a + b
end) |> Enum.to_list()
[2, 4, 6, 8, 10, 12]

iex> concat = Stream.concat(1..3, 4..6)
iex> Stream.zip_with([concat, concat, 1..3], fn [a, b, c]
-> a + b + c end) |> Enum.to_list()
[3, 6, 9]
```

zip_with(enumerable1, enumerable2, zip_fun)

(since 1.12.0)

```
@spec zip_with(Enumerable.t(), Enumerable.t(), (term(),
term() -> term())) ::
  Enumerable.t()
```

Lazily zips corresponding elements from two enumerables into a new one, transforming them with the `zip_fun` function as it goes.

The `zip_fun` will be called with the first element from `enumerable1` and the first element from `enumerable2`, then with the second element from each, and so on until either one of the enumerables completes.

Examples

```
iex> concat = Stream.concat(1..3, 4..6)
iex> Stream.zip_with(concat, concat, fn a, b -> a + b
end) |> Enum.to_list()
[2, 4, 6, 8, 10, 12]
```

File

This module contains functions to manipulate files.

Some of those functions are low-level, allowing the user to interact with files or IO devices, like `open/2`, `copy/3` and others. This module also provides higher level functions that work with filenames and have their naming based on Unix variants. For example, one can copy a file via `cp/3` and remove files and directories recursively via `rm_rf/1`.

Paths given to functions in this module can be either relative to the current working directory (as returned by `File.cwd/0`), or absolute paths. Shell conventions like `~` are not expanded automatically. To use paths like `~/Downloads`, you can use `Path.expand/1` or `Path.expand/2` to expand your path to an absolute path.

Encoding

In order to write and read files, one must use the functions in the `IO` module. By default, a file is opened in binary mode, which requires the functions `IO.binread/2` and `IO.binwrite/2` to interact with the file. A developer may pass `:utf8` as an option when opening the file, then the slower `IO.read/2` and `IO.write/2` functions must be used as they are responsible for doing the proper conversions and providing the proper data guarantees.

Note that filenames when given as charlists in Elixir are always treated as UTF-8. In particular, we expect that the shell and the operating system are configured to use UTF-8 encoding. Binary filenames are considered raw and passed to the operating system as is.

API

Most of the functions in this module return `:ok` or `{:ok, result}` in case of success, `{:error, reason}` otherwise. Those functions also have a variant that ends with `!` which returns the result (instead of the `{:ok, result}` tuple) in case of success or raises an exception in case it fails. For example:

```
File.read("hello.txt")
#=> {:ok, "World"}

File.read("invalid.txt")
#=> {:error, :enoent}

File.read!("hello.txt")
#=> "World"

File.read!("invalid.txt")
#=> raises File.Error
```

In general, a developer should use the former in case they want to react if the file does not exist. The latter should be used when the developer expects their software to fail in case the file cannot be read (i.e. it is literally an exception).

Processes and raw files

Every time a file is opened, Elixir spawns a new process. Writing to a file is equivalent to sending messages to the process that writes to the file descriptor.

This means files can be passed between nodes and message passing guarantees they can write to the same file in a network.

However, you may not always want to pay the price for this abstraction. In such cases, a file can be opened in `:raw` mode. The options

`:read_ahead` and `:delayed_write` are also useful when operating on large files or working with files in tight loops.

Check `:file.open/2` for more information about such options and other performance considerations.

Seeking within a file

You may also use any of the functions from the `:file` module to interact with files returned by Elixir. For example, to read from a specific position in a file, use `:file.pread/3`:

```
File.write!("example.txt", "Eats, Shoots & Leaves")
file = File.open!("example.txt")
:file.pread(file, 15, 6)
#=> {:ok, "Leaves"}
```

Alternatively, if you need to keep track of the current position, use `:file.position/2` and `:file.read/2`:

```
:file.position(file, 6)
#=> {:ok, 6}
:file.read(file, 6)
#=> {:ok, "Shoots"}
:file.position(file, {:cur, -12})
#=> {:ok, 0}
:file.read(file, 4)
#=> {:ok, "Eats"}
```

Summary

Types

encoding_mode()
erlang_time()
file_descriptor()
io_device()
mode()
on_conflict_callback()
posix()
posix_time()
read_offset_mode()
stat_options()
stream_mode()

Functions

cd(path)

Sets the current working directory.

cd!(path)

The same as `cd/1`, but raises a `File.Error` exception if it fails.

cd!(path, function)

Changes the current directory to the given `path`, executes the given function and then reverts back to the previous path regardless of whether there is an exception.

chgrp(path, gid)

Changes the group given by the group ID `gid` for a given `file`. Returns `:ok` on success, or `{:error, reason}` on failure.

chgrp!(path, gid)

Same as `chgrp/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

chmod(path, mode)

Changes the `mode` for a given `file`.

chmod!(path, mode)

Same as `chmod/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

chown(path, uid)

Changes the owner given by the user ID `uid` for a given `file`. Returns `:ok` on success, or `{:error, reason}` on failure.

chown!(path, uid)

Same as `chown/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

close(io_device)

Closes the file referenced by `io_device`. It mostly returns `:ok`, except for some severe errors such as out of memory.

copy(source, destination, bytes_count \\ :infinity)

Copies the contents of `source` to `destination`.

copy!(source, destination, bytes_count \\ :infinity)

The same as `copy/3` but raises a `File.CopyError` exception if it fails. Returns the `bytes_copied` otherwise.

cp(source_file, destination_file, options \\ [])

Copies the contents of `source_file` to `destination_file` preserving its modes.

cp!(source_file, destination_file, options \\ [])

The same as `cp/3`, but raises a `File.CopyError` exception if it fails. Returns `:ok` otherwise.

cp_r(source, destination, options \\ [])

Copies the contents in `source` to `destination` recursively, maintaining the source directory structure and regular file modes.

cp_r!(source, destination, options \\ [])

The same as `cp_r/3`, but raises a `File.CopyError` exception if it fails. Returns the list of copied files otherwise.

cwd()

Gets the current working directory.

cwd!()

The same as `cwd/0`, but raises a `File.Error` exception if it fails.

dir?(path, opts \\ [])

Returns `true` if the given path is a directory.

exists?(path, opts \\ [])

Returns `true` if the given path exists.

ln(existing, new)

Creates a hard link `new` to the file `existing`.

ln!(existing, new)

Same as `ln/2` but raises a `File.LinkError` exception if it fails. Returns `:ok` otherwise.

ln_s(existing, new)

Creates a symbolic link `new` to the file or directory `existing`.

ln_s!(existing, new)

Same as `ln_s/2` but raises a `File.LinkError` exception if it fails. Returns `:ok` otherwise.

ls(path \\ ".")

Returns the list of files in the given directory.

ls!(path \\ ".")

The same as `ls/1` but raises a `File.Error` exception in case of an error.

lstat(path, opts \\ [])

Returns information about the `path`. If the file is a symlink, sets the `type` to `:symlink` and returns a `File.Stat` struct for the link. For any other file, returns exactly the same values as `stat/2`.

lstat!(path, opts \\ [])

Same as `lstat/2` but returns the `File.Stat` struct directly, or raises a `File.Error` exception if an error is returned.

mkdir(path)

Tries to create the directory `path`.

mkdir!(path)

Same as `mkdir/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

mkdir_p(path)

Tries to create the directory `path`.

mkdir_p!(path)

Same as `mkdir_p/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

open(path, modes_or_function \\ [])

Opens the given `path`.

open(path, modes, function)

Similar to `open/2` but expects a function as its last argument.

open!(path, modes_or_function \\ [])

Similar to `open/2` but raises a `File.Error` exception if the file could not be opened. Returns the IO device otherwise.

open!(path, modes, function)

Similar to `open/3` but raises a `File.Error` exception if the file could not be opened.

read(path, opts \\ [])

Returns `{:ok, binary}`, where `binary` is a binary data object that contains the contents of `path`, or `{:error, reason}` if an error occurs.

read!(path, opts \\ [])

Returns a binary with the contents of the given filename, or raises a `File.Error` exception if an error occurs.

read_link(path)

Reads the symbolic link at `path`.

read_link!(path)

Same as `read_link/1` but returns the target directly, or raises a `File.Error` exception if an error is returned.

regular?(path, opts \\ [])

Returns `true` if the path is a regular file.

rename(source, destination)

Renames the `source` file to `destination` file. It can be used to move files (and directories) between directories. If moving a file, you must fully specify the `destination` filename, it is not sufficient to simply specify its directory.

rename!(source, destination)

The same as `rename/2` but raises a `File.RenameError` exception if it fails. Returns `:ok` otherwise.

rm(path)

Tries to delete the file `path`.

rm!(path)

Same as `rm/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

rm_rf(path)

Removes files and directories recursively at the given `path`. Symlinks are not followed but simply removed, non-existing files are simply ignored (i.e. doesn't make this function fail).

rm_rf!(path)

Same as `rm_rf/1` but raises a `File.Error` exception in case of failures, otherwise returns the list of files or directories removed.

rmdir(path)

Tries to delete the dir at `path`.

rmdir!(path)

Same as `rmdir/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

stat(path, opts \\ [])

Returns information about the `path`. If it exists, it returns a `{:ok, info}` tuple, where info is a `File.Stat` struct. Returns `{:error, reason}` with the same reasons as `read/1` if a failure occurs.

stat!(path, opts \\ [])

Same as `stat/2` but returns the `File.Stat` directly, or raises a `File.Error` exception if an error is returned.

stream!(path, line_or_bytes_modes \\ [])

Shortcut for `File.stream!/3`.

stream!(path, line_or_bytes, modes)

Returns a `File.Stream` for the given `path` with the given `modes`.

touch(path, time \\ System.os_time(:second))

Updates modification time (mtime) and access time (atime) of the given file.

touch!(path, time \\ System.os_time(:second))

Same as `touch/2` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

write(path, content, modes \\ [])

Writes `content` to the file `path`.

write!(path, content, modes \\ [])

Same as `write/3` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

write_stat(path, stat, opts \\ [])

Writes the given `File.Stat` back to the file system at the given path. Returns `:ok` or `{:error, reason}`.

write_stat!(path, stat, opts \\ [])

Same as `write_stat/3` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

Types

encoding_mode()

```
@type encoding_mode() ::
  :utf8
  | {:encoding,
     :latin1
     | :unicode
     | :utf8
     | :utf16
     | :utf32
     | {:utf16, :big | :little}
     | {:utf32, :big | :little}}
```

erlang_time()

```
@type erlang_time() ::
  {{year :: non neg integer(), month :: 1..12, day :: 1..31},
   {hour :: 0..23, minute :: 0..59, second :: 0..59}}
```

file_descriptor()

```
@type file_descriptor() :: :file.fd()
```

io_device()

```
@type io_device() :: :file.io device()
```

mode()

```
@type mode() ::
  :append
  | :binary
  | :charlist
  | :compressed
  | :delayed_write
  | :exclusive
  | :raw
  | :read
  | :read_ahead
  | :sync
  | :write
  | {:read_ahead, pos_integer()}
  | {:delayed_write, non_neg_integer(), non_neg_integer()}
  | encoding_mode()
```

on_conflict_callback()

```
@type on_conflict_callback() :: (Path.t(), Path.t() ->
boolean())
```

posix()

```
@type posix() :: :file.posix()
```

posix_time()

```
@type posix_time() :: integer()
```

read_offset_mode()

```
@type read_offset_mode() :: {:read_offset, non_neg_integer()}
```

stat_options()

```
@type stat_options() :: [{:time, :local | :universal |
:posix}]
```

stream_mode()

```
@type stream_mode() ::
  encoding_mode()
  | read_offset_mode()
  | :append
  | :compressed
  | :delayed_write
  | :trim_bom
  | {:read_ahead, pos_integer() | false}
  | {:delayed_write, non_neg_integer(), non_neg_integer()}
```

Functions

cd(path)

```
@spec cd(Path.t()) :: :ok | {:error, posix() | :badarg |
:no_translation}
```

Sets the current working directory.

The current working directory is set for the BEAM globally. This can lead to race conditions if multiple processes are changing the current working directory concurrently. To run an external command in a given directory without changing the global current working directory, use the `:cd` option of `System.cmd/3` and `Port.open/2`.

Returns `:ok` if successful, `{:error, reason}` otherwise.

Examples

```
File.cd("bin")
#=> :ok

File.cd("non_existing_dir")
#=> {:error, :enoent}
```

cd!(path)

```
@spec cd!(Path.t()) :: :ok
```

The same as `cd/1`, but raises a `File.Error` exception if it fails.

Examples

```
File.cd!("bin")
#=> :ok

File.cd!("non_existing_dir")
** (File.Error) could not set current working directory
to "non_existing_dir": no such file or directory
```

cd!(path, function)

```
@spec cd!(Path.t(), (-> res)) :: res when res: var
```

Changes the current directory to the given `path`, executes the given function and then reverts back to the previous path regardless of whether there is an exception.

The current working directory is temporarily set for the BEAM globally. This can lead to race conditions if multiple processes are changing the current working directory concurrently. To run an external command in a given directory without changing the global current working directory, use the `:cd` option of `System.cmd/3` and `Port.open/2`.

Raises an error if retrieving or changing the current directory fails.

Examples

```
File.cd!("bin", fn -> do_something() end)
#=> :result_of_do_something

File.cd!("non_existing_dir", fn -> do_something() end)
** (File.Error) could not set current working directory
to "non_existing_dir": no such file or directory
```

chgrp(path, gid)

```
@spec chgrp(Path.t(), non_neg_integer()) :: :ok | {:error,
posix() | :badarg}
```

Changes the group given by the group ID `gid` for a given `file`. Returns `:ok` on success, or `{:error, reason}` on failure.

Examples

```
File.chgrp("hello.txt", 10)
#=> :ok

File.chgrp("non_existing.txt", 10)
#=> {:error, :enoent}
```

chgrp!(path, gid)

```
@spec chgrp!(Path.t(), non_neg_integer()) :: :ok
```

Same as `chgrp/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

File.chgrp!("hello.txt", 10) #=> :ok

File.chgrp!("non_existing.txt", 10) ** (File.Error) could not change group for "non_existing.txt": no such file or directory

chmod(path, mode)

```
@spec chmod(Path.t(), non_neg_integer()) :: :ok | {:error,
posix() | :badarg}
```

Changes the `mode` for a given `file`.

Returns `:ok` on success, or `{:error, reason}` on failure.

Permissions

File permissions are specified by adding together the following octal modes:

- `0o400` - read permission: owner
- `0o200` - write permission: owner
- `0o100` - execute permission: owner
- `0o040` - read permission: group
- `0o020` - write permission: group
- `0o010` - execute permission: group
- `0o004` - read permission: other
- `0o002` - write permission: other
- `0o001` - execute permission: other

For example, setting the mode `0o755` gives it write, read and execute permission to the owner and both read and execute permission to group and others.

Examples

```
File.chmod("hello.txt", 0o755)
#=> :ok

File.chmod("non_existing.txt", 0o755)
#=> {:error, :enoent}
```

chmod!(path, mode)

```
@spec chmod!(Path.t(), non_neg_integer()) :: :ok
```

Same as `chmod/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.chmod!("hello.txt", 0o755)
#=> :ok

File.chmod!("non_existing.txt", 0o755)
** (File.Error) could not change mode for
"non_existing.txt": no such file or directory
```

chown(path, uid)

```
@spec chown(Path.t(), non_neg_integer()) :: :ok | {:error,
posix() | :badarg}
```

Changes the owner given by the user ID `uid` for a given `file`. Returns `:ok` on success, or `{:error, reason}` on failure.

Examples

```
File.chown("hello.txt", 15)
#=> :ok

File.chown("secret.txt", 15)
#=> {:error, :eperm}
```

chown!(path, uid)

```
@spec chown!(Path.t(), non_neg_integer()) :: :ok
```

Same as `chown/2`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.chown!("hello.txt", 15)
#=> :ok

File.chown!("secret.txt", 15)
** (File.Error) could not change owner for "secret.txt":
not owner
```

close(io_device)

```
@spec close(io_device()) :: :ok | {:error, posix() | :badarg
| :terminated}
```

Closes the file referenced by `io_device`. It mostly returns `:ok`, except for some severe errors such as out of memory.

Note that if the option `:delayed_write` was used when opening the file, `close/1` might return an old write error and not even try to close the file. See `open/2` for more information.

Examples

```
{:ok, file} = File.open("hello.txt")
File.close(file)
#=> :ok

File.close(:not_an_io_device)
#=> {:error, :badarg}
```

copy(source, destination, bytes_count \\ :infinity)

```
@spec copy(Path.t() | io_device(), Path.t() | io_device(),
pos_integer() | :infinity) ::
  {:ok, non_neg_integer()} | {:error, posix() | :badarg |
:terminated}
```

Copies the contents of `source` to `destination`.

Both parameters can be a filename or an IO device opened with `open/2`. `bytes_count` specifies the number of bytes to copy, the default being `:infinity`.

If file `destination` already exists, it is overwritten by the contents in `source`.

Returns `{:ok, bytes_copied}` if successful, `{:error, reason}` otherwise.

Compared to the `cp/3`, this function is more low-level, allowing a copy from device to device limited by a number of bytes. On the other hand, `cp/3` performs more extensive checks on both source and destination and it also preserves the file mode after copy.

Typical error reasons are the same as in `open/2`, `read/1` and `write/3`.

Examples

```
File.copy("hello.txt", "hello_copy.txt")
#=> {:ok, 6}

File.copy("non_existing.txt", "copy.txt")
#=> {:error, :enoent}
```

copy!(source, destination, bytes_count \\ :infinity)

```
@spec copy!(Path.t() | io device(), Path.t() | io device(),
pos integer() | :infinity) ::
  non neg integer()
```

The same as `copy/3` but raises a `File.CopyError` exception if it fails. Returns the `bytes_copied` otherwise.

Examples

```
File.copy!("hello.txt", "hello_copy.txt")
#=> 6

File.copy!("non_existing.txt", "copy.txt")
** (File.CopyError) could not copy from
"non_existing.txt" to "copy.txt": no such file or
directory
```

cp(source_file, destination_file, options \\ [])

```
@spec cp(Path.t(), Path.t(), [{:on_conflict,
on conflict callback()}]) ::
  :ok | {:error, posix() | :badarg | :terminated}
```

Copies the contents of `source_file` to `destination_file` preserving its modes.

`source_file` must be a file or a symbolic link to one. `destination_file` must be a path to a non-existent file. If either is a directory, `{:error, :eisdir}` will be returned.

The function returns `:ok` in case of success. Otherwise, it returns `{:error, reason}`.

If you want to copy contents from an IO device to another device or do a straight copy from a source to a destination without preserving modes, check `copy/3` instead.

Note: The command `cp` in Unix-like systems behaves differently depending on whether the destination is an existing directory or not. We have chosen to explicitly disallow copying to a destination which is a directory, and an error will be returned if tried.

Options

- `:on_conflict` - (since v1.14.0) Invoked when a file already exists in the destination. The function receives arguments for `source_file` and `destination_file`. It should return `true` if the existing file should be overwritten, `false` if otherwise. The default callback returns `true`. On earlier versions, this callback could be given as third argument, but such behavior is now deprecated.

Examples

```
File.cp("hello.txt", "hello_copy.txt")
#=> :ok

File.cp("hello.txt", "hello_copy.txt", on_conflict: fn
source, destination ->
  IO.gets("Overwriting #{destination} by #{source}. Type
y to confirm. ") == "y\n"
end)
#=> :ok

File.cp("non_existing.txt", "copy.txt")
#=> {:error, :enoent}
```

cp!(source_file, destination_file, options \\ [])

```
@spec cp!(Path.t(), Path.t(), [{:on_conflict,
on_conflict_callback()}]) :: :ok
```

The same as `cp/3`, but raises a `File.CopyError` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.cp!("hello.txt", "hello_copy.txt")
#=> :ok

File.cp!("hello.txt", "hello_copy.txt", on_conflict: fn
source, destination ->
  IO.gets("Overwriting #{destination} by #{source}. Type
y to confirm. ") == "y\n"
end)
#=> :ok

File.cp!("non_existing.txt", "copy.txt")
** (File.CopyError) could not copy from
"non_existing.txt" to "copy.txt": no such file or
directory
```

cp_r(source, destination, options \\ [])

```
@spec cp_r(Path.t(), Path.t(),
  on_conflict: on_conflict_callback(),
  dereference_symlinks: boolean(),
  preserve_directory_permissions: boolean()
) :: {:ok, [binary()]} | {:error, posix() | :badarg |
:terminated, binary()}
```

Copies the contents in `source` to `destination` recursively, maintaining the source directory structure and regular file modes.

If `source` is a file or a symbolic link to it, `destination` must be a path to an existent file, a symbolic link to one, or a path to a non-existent file.

If `source` is a directory, or a symbolic link to it, then `destination` must be an existent `directory` or a symbolic link to one, or a path to a non-existent directory.

If the source is a file, it copies `source` to `destination`. If the `source` is a directory, it copies the contents inside source into the `destination` directory.

For regular files, their respective file modes are preserved in the destination. Directory modes are preserved only when `:preserve_directory_permissions` is `true`. If a file already exists in the destination, it invokes the optional `:on_conflict` callback given as an option. See "Options" for more information.

This function may fail while copying files, in such cases, it will leave the destination directory in a dirty state, where file which have already been copied won't be removed.

The function returns `{:ok, files_and_directories}` in case of success, `files_and_directories` lists all files and directories copied in no specific order. It returns `{:error, reason, file}` otherwise.

Note: The command `cp` in Unix-like systems behaves differently depending on whether `destination` is an existing directory or not. We have chosen to explicitly disallow this behavior. If `source` is a `file` and `destination` is a directory, `{:error, :eisdir}` will be returned.

Special files such as device files, sockets, and named pipes are not copied.

Typical error reasons are:

- `:enoent` - `source` does not exist
- `:eisdir` - `source` is a file and `destination` is a directory

- `:einval` - `destination` is the same as or a subdirectory of `source`

Options

- `:on_conflict` - (since v1.14.0) Invoked when a file already exists in the destination. The function receives arguments for `source` and `destination`. It should return `true` if the existing file should be overwritten, `false` if otherwise. The default callback returns `true`. On earlier versions, this callback could be given as third argument, but such behavior is now deprecated.

- `:dereference_symlinks` - (since v1.14.0) By default, this function will copy symlinks by creating symlinks that point to the same location. This option forces symlinks to be dereferenced and have their contents copied instead when set to `true`. If the dereferenced files do not exist, than the operation fails. The default is `false`.

- `:preserve_directory_permissions` - (since v1.20.0) when `true`, the permissions of source directories are copied to the destination directories after their contents are written. This is useful when source directories are read-only or have restricted permissions that must be preserved. The default is `false`.

Examples

```
# Copies file "a.txt" to "b.txt"
File.cp_r("a.txt", "b.txt")
#=> {:ok, ["b.txt"]}

# Copies all files in "samples" to "tmp"
File.cp_r("samples", "tmp")
#=> {:ok, ["z.txt", "y.txt", "x.txt]}
```

```
# Same as before, but asks the user how to proceed in
case of conflicts
File.cp_r("samples", "tmp", on_conflict: fn source,
destination ->
  IO.gets("Overwriting #{destination} by #{source}. Type
y to confirm. ") == "y\n"
end)
#=> {:ok, ["z.txt", "y.txt", "x.txt]}

File.cp_r("non_existing.txt", "copy.txt")
#=> {:error, :enoent, "non_existing.txt"}

# Copying into a subdirectory of source is not allowed
File.cp_r("src", "src/dest")
#=> {:error, :einval, "src/dest"}
```

cp_r!(source, destination, options \\ [])

```
@spec cp_r!(Path.t(), Path.t(),
  on_conflict: on conflict callback(),
  dereference_symlinks: boolean(),
  preserve_directory_permissions: boolean()
) :: [binary()]
```

The same as `cp_r/3`, but raises a `File.CopyError` exception if it fails. Returns the list of copied files otherwise.

Examples

```
File.cp_r!("a.txt", "b.txt")
#=> ["b.txt"]
```

```
File.cp_r!("samples", "tmp")
#=> ["z.txt", "y.txt", "x.txt]

File.cp_r!("non_existing.txt", "copy.txt")
** (File.CopyError) could not copy recursively from
"non_existing.txt" to "copy.txt". non_existing.txt: no
such file or directory
```

cwd()

```
@spec cwd() :: {:ok, binary()} | {:error, posix() | :badarg}
```

Gets the current working directory.

In rare circumstances, this function can fail on Unix-like systems. It may happen if read permissions do not exist for the parent directories of the current directory. For this reason, returns `{:ok, cwd}` in case of success, `{:error, reason}` otherwise.

Examples

```
File.cwd()
#=> {:ok, "/Users/user/elixir/elixir_lang"}

# Missing read permission for one of the parents of the
current directory
File.cwd()
#=> {:error, :eacces}
```

cwd!()

```
@spec cwd!() :: binary()
```

The same as `cwd/0`, but raises a `File.Error` exception if it fails.

Examples

```
File.cwd!()
#=> "/Users/user/elixir/elixir_lang"
```

dir?(path, opts \\ [])

```
@spec dir?(Path.t(), [dir_option]) :: boolean() when
dir_option: :raw
```

Returns `true` if the given path is a directory.

This function follows symbolic links, so if a symbolic link points to a directory, `true` is returned.

Options

The supported options are:

- `:raw` - a single atom to bypass the file server and only check for the file locally

Examples

```elixir
File.dir?("./test")
#=> true

File.dir?("test")
#=> true

File.dir?("/usr/bin")
#=> true

File.dir?("~/Downloads")
#=> false

"~/Downloads" |> Path.expand() |> File.dir?()
#=> true
```

exists?(path, opts \\ [])

```
@spec exists?(Path.t(), [exists_option]) :: boolean() when exists_option: :raw
```

Returns `true` if the given path exists.

It can be a regular file, directory, socket, symbolic link, named pipe, or device file. Returns `false` for symbolic links pointing to non-existing targets.

Options

The supported options are:

- `:raw` - a single atom to bypass the file server and only check for the file locally

Examples

```
File.exists?("test/")
#=> true

File.exists?("missing.txt")
#=> false

File.exists?("/dev/null")
#=> true
```

ln(existing, new)

(since 1.5.0)

```
@spec ln(Path.t(), Path.t()) :: :ok | {:error, posix() | :badarg}
```

Creates a hard link `new` to the file `existing`.

Returns `:ok` if successful, `{:error, reason}` otherwise. If the operating system does not support hard links, returns `{:error, :enotsup}`.

Examples

```
File.ln("hello.txt", "hard_link_to_hello")
#=> :ok

File.ln("non_existing.txt", "link")
#=> {:error, :enoent}
```

ln!(existing, new)

(since 1.5.0)

```
@spec ln!(Path.t(), Path.t()) :: :ok
```

Same as `ln/2` but raises a `File.LinkError` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.ln!("hello.txt", "hard_link_to_hello")
#=> :ok

File.ln!("non_existing.txt", "link")
** (File.LinkError) could not create hard link from
"non_existing.txt" to "link": no such file or directory
```

ln_s(existing, new)

(since 1.5.0)

```
@spec ln_s(Path.t(), Path.t()) :: :ok | {:error, posix() |
:badarg}
```

Creates a symbolic link `new` to the file or directory `existing`.

Returns `:ok` if successful, `{:error, reason}` otherwise. If the operating system does not support symlinks, returns `{:error, :enotsup}`.

Creates a symlink even if the `existing` target actually doesn't exist

Examples

```
File.ln_s("hello.txt", "link_to_hello")
#=> :ok

File.ln_s("non_existing.txt", "link")
#=> :ok

# Returns error if `new` file exists
File.ln_s("non_existing.txt", "existed_link")
#=> {:error, :eexist}
```

ln_s!(existing, new)

```
@spec ln_s!(Path.t(), Path.t()) :: :ok
```

Same as `ln_s/2` but raises a `File.LinkError` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.ln_s!("hello.txt", "link_to_hello")
#=> :ok

# Raises if `new` file exists
File.ln_s!("non_existing.txt", "existed_link")
** (File.LinkError) could not create symlink from
"non_existing.txt" to "existed_link": file already exists
```

ls(path \\ ".")

```
@spec ls(Path.t()) ::
  {:ok, [binary()]} | {:error, posix() | :badarg |
{:no_translation, binary()}}
```

Returns the list of files in the given directory.

Hidden files are not ignored and the results are *not* sorted.

Since directories are considered files by the file system, they are also included in the returned value.

Returns `{:ok, files}` in case of success, `{:error, reason}` otherwise.

Examples

```
File.ls("bin")
#=> {:ok, ["iex", "elixir"]}

File.ls("non_existing_dir")
#=> {:error, :enoent}
```

ls!(path \\ ".")

```
@spec ls!(Path.t()) :: [binary()]
```

The same as `ls/1` but raises a `File.Error` exception in case of an error.

Examples

```
File.ls!("bin")
#=> ["iex", "elixir"]

File.ls!("non_existing_dir")
** (File.Error) could not list directory
"non_existing_dir": no such file or directory
```

lstat(path, opts \\ [])

```
@spec lstat(Path.t(), stat_options()) ::
  {:ok, File.Stat.t()} | {:error, posix() | :badarg}
```

Returns information about the `path`. If the file is a symlink, sets the `type` to `:symlink` and returns a `File.Stat` struct for the link. For any other file, returns exactly the same values as `stat/2`.

For more details, see `:file.read_link_info/2`.

Options

The accepted options are:

- `:time` - configures how the file timestamps are returned

The values for `:time` can be:

- `:universal` - returns a `{date, time}` tuple in UTC (default)
- `:local` - returns a `{date, time}` tuple using the machine time
- `:posix` - returns the time as integer seconds since epoch

Note: Since file times are stored in POSIX time format on most operating systems, it is faster to retrieve file information with the `time: :posix` option.

Examples

```
File.lstat("link_to_hello")
#=> {:ok, %File.Stat{type: :symlink, ...}}

File.lstat("non_existing.txt", time: :posix)
#=> {:error, :enoent}
```

lstat!(path, opts \\ [])

```
@spec lstat!(Path.t(), stat_options()) :: File.Stat.t()
```

Same as `lstat/2` but returns the `File.Stat` struct directly, or raises a `File.Error` exception if an error is returned.

Examples

```
File.lstat!("link_to_hello")
#=> %File.Stat{type: :symlink, ...}

File.lstat!("non_existing.txt", time: :posix)
** (File.Error) could not read file stats
"non_existing.txt": no such file or directory
```

mkdir(path)

```
@spec mkdir(Path.t()) :: :ok | {:error, posix() | :badarg}
```

Tries to create the directory `path`.

Missing parent directories are not created. Returns `:ok` if successful, or `{:error, reason}` if an error occurs.

Typical error reasons are:

- `:eacces` - missing search or write permissions for the parent directories of `path`
- `:eexist` - there is already a file or directory named `path`
- `:enoent` - a component of `path` does not exist
- `:enospc` - there is no space left on the device
- `:enotdir` - a component of `path` is not a directory; on some platforms, `:enoent` is returned instead

Examples

```
File.mkdir("test/unit")
#=> :ok

File.mkdir("non/existing")
#=> {:error, :enoent}
```

mkdir!(path)

```
@spec mkdir!(Path.t()) :: :ok
```

Same as `mkdir/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.mkdir!("test/unit")
#=> :ok

File.mkdir!("non/existing")
** (File.Error) could not make directory "non/existing":
no such file or directory
```

mkdir_p(path)

```
@spec mkdir_p(Path.t()) :: :ok | {:error, posix() | :badarg}
```

Tries to create the directory `path`.

Missing parent directories are created. Returns `:ok` if successful, or `{:error, reason}` if an error occurs.

Typical error reasons are:

- `:eacces` - missing search or write permissions for the parent directories of `path`
- `:enospc` - there is no space left on the device
- `:enotdir` - a component of `path` is not a directory
- `:eperm` - missed required permissions

Examples

```
File.mkdir_p("non/existing/parents")
#=> :ok
```

```
File.mkdir_p("/usr/sbin/temp")
#=> {:error, :eperm}
```

mkdir_p!(path)

```
@spec mkdir_p!(Path.t()) :: :ok
```

Same as `mkdir_p/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.mkdir_p!("non/existing/parents")
#=> :ok

File.mkdir_p!("/usr/sbin/temp")
** (File.Error) could not make directory (with -p)
"/usr/sbin/temp": not owner
```

open(path, modes_or_function \\ [])

```
@spec open(Path.t(), [mode() | :ram]) ::
  {:ok, io device() | file descriptor()}
  | {:error, posix() | :badarg | :system_limit}
@spec open(Path.t(), (io device() | file descriptor() ->
res)) ::
  {:ok, res} | {:error, posix() | :badarg | :system_limit}
when res: var
```

Opens the given `path`.

`modes_or_function` can either be a list of modes or a function. If it's a list, it's considered to be a list of modes (that are documented below). If it's a function, then it's equivalent to calling `open(path, [], modes_or_function)`. See the documentation for `open/3` for more information on this function.

The allowed modes:

- `:binary` - opens the file in binary mode, disabling special handling of Unicode sequences (default mode).
- `:read` - the file, which must exist, is opened for reading.
- `:write` - the file is opened for writing. It is created if it does not exist.

  If the file does exist, and if write is not combined with read, the file will be truncated.
- `:append` - the file will be opened for writing, and it will be created if it does not exist. Every write operation to a file opened with append will take place at the end of the file.
- `:exclusive` - the file, when opened for writing, is created if it does not exist. If the file exists, open will return `{:error, :eexist}`.
- `:charlist` - when this term is given, read operations on the file will return charlists rather than binaries.
- `:compressed` - makes it possible to read or write gzip compressed files.

  The compressed option must be combined with either read or write, but not both. Note that the file size obtained with `stat/1` will most

probably not match the number of bytes that can be read from a compressed file.

- `:utf8` - this option denotes how data is actually stored in the disk file and makes the file perform automatic translation of characters to and from UTF-8.

  If data is sent to a file in a format that cannot be converted to the UTF-8 or if data is read by a function that returns data in a format that cannot cope with the character range of the data, an error occurs and the file will be closed.

- `:delayed_write`, `:raw`, `:ram`, `:read_ahead`, `:sync`, `{:encoding, ...}`, `{:read_ahead, pos_integer}`, `{:delayed_write, non_neg_integer, non_neg_integer}` - for more information about these options see `:file.open/2`.

This function returns:

- `{:ok, io_device | file_descriptor}` - the file has been opened in the requested mode. We explore the differences between these two results in the following section

- `{:error, reason}` - the file could not be opened due to `reason`.

IO devices

By default, this function returns an IO device. An `io_device` is a process which handles the file and you can interact with it using the functions in the `IO` module. By default, a file is opened in `:binary` mode, which requires the functions `IO.binread/2` and `IO.binwrite/2` to interact with the file. A developer may pass `:utf8` as a mode when opening the file and then all other functions from `IO` are available, since they work directly with Unicode data.

Given the IO device is a file, if the owner process terminates, the file is closed and the process itself terminates too. If any process to which the `io_device` is linked terminates, the file will be closed and the process itself will be terminated.

File descriptors

When the `:raw` or `:ram` modes are given, this function returns a low-level file descriptors. This avoids creating a process but requires using the functions in the `:file` module to interact with it.

Examples

```
{:ok, file} = File.open("foo.tar.gz", [:read,
:compressed])
IO.read(file, :line)
File.close(file)
```

open(path, modes, function)

```
@spec open(Path.t(), [mode() | :ram], (io device() |
file descriptor() -> res)) ::
  {:ok, res} | {:error, posix() | :badarg | :system_limit}
when res: var
```

Similar to `open/2` but expects a function as its last argument.

The file is opened, given to the function as an argument and automatically closed after the function returns, regardless if there was an error when executing the function.

Returns `{:ok, function_result}` in case of success, `{:error, reason}` otherwise.

This function expects the file to be closed with success, which is usually the case unless the `:delayed_write` option is given. For this reason, we do not recommend passing `:delayed_write` to this function.

See `open/2` for the list of available `modes`.

Examples

```
File.open("file.txt", [:read, :write], fn file ->
  IO.read(file, :line)
end)
#=> {:ok, "file content"}
```

open!(path, modes_or_function \\ [])

```
@spec open!(Path.t(), [mode() | :ram]) :: io device() |
file descriptor()
@spec open!(Path.t(), (io device() | file descriptor() ->
res)) :: res when res: var
```

Similar to `open/2` but raises a `File.Error` exception if the file could not be opened. Returns the IO device otherwise.

See `open/2` for the list of available modes.

Examples

```
File.open!("file.txt", fn file ->
  IO.read(file, :line)
```

```
  end)
  #=> "file content"
```

open!(path, modes, function)

```
@spec open!(Path.t(), [mode() | :ram], (io device() |
file descriptor() -> res)) ::
  res
when res: var
```

Similar to `open/3` but raises a `File.Error` exception if the file could not be opened.

If it succeeds opening the file, it returns the `function` result on the IO device.

See `open/2` for the list of available `modes`.

Examples

```
  File.open!("file.txt", [:read, :write], fn file ->
    IO.read(file, :line)
  end)
  #=> "file content"
```

read(path, opts \\ [])

```
@spec read(Path.t(), [exists_option]) ::
  {:ok, binary()} | {:error, posix() | :badarg | :terminated
```

```
| :system_limit}
when exists_option: :raw
```

Returns `{:ok, binary}`, where `binary` is a binary data object that contains the contents of `path`, or `{:error, reason}` if an error occurs.

Typical error reasons:

- `:enoent` - the file does not exist
- `:eacces` - missing permission for reading the file, or for searching one of the parent directories
- `:eisdir` - the named file is a directory
- `:enotdir` - a component of the file name is not a directory; on some platforms, `:enoent` is returned instead
- `:enomem` - there is not enough memory for the contents of the file

You can use `:file.format_error/1` to get a descriptive string of the error.

Options (since v1.20)

The supported options are:

- `:raw` - a single atom to bypass the file server and only check for the file locally

Examples

```
File.read("hello.txt")
#=> {:ok, "world"}

File.read("non_existing.txt")
#=> {:error, :enoent}
```

read!(path, opts \\ [])

```
@spec read!(Path.t(), [exists_option]) :: binary() when
exists_option: :raw
```

Returns a binary with the contents of the given filename, or raises a `File.Error` exception if an error occurs.

Options (since v1.20)

The supported options are:

- `:raw` - a single atom to bypass the file server and only check for the file locally

Examples

```
File.read!("hello.txt")
#=> "world"

File.read!("non_existing.txt")
** (File.Error) could not read file "non_existing.txt":
no such file or directory
```

read_link(path)

(since 1.5.0)

```
@spec read_link(Path.t()) :: {:ok, binary()} | {:error,
posix() | :badarg}
```

Reads the symbolic link at `path`.

If `path` exists and is a symlink, returns `{:ok, target}`, otherwise returns `{:error, reason}`.

For more details, see `:file.read_link/1`.

Typical error reasons are:

- `:einval` - path is not a symbolic link
- `:enoent` - path does not exist
- `:enotsup` - symbolic links are not supported on the current platform

Examples

```
File.read_link("link_to_hello")
#=> {:ok, "hello.txt"}

File.read_link("hello.txt")
#=> {:error, :einval}
```

read_link!(path)

(since 1.5.0)

```
@spec read_link!(Path.t()) :: binary()
```

Same as `read_link/1` but returns the target directly, or raises a `File.Error` exception if an error is returned.

Examples

```
File.read_link!("link_to_hello")
#=> "hello.txt"

File.read_link!("hello.txt")
** (File.Error) could not read link "hello.txt": invalid
argument
```

regular?(path, opts \\ [])

```
@spec regular?(Path.t(), [regular_option]) :: boolean() when
regular_option: :raw
```

Returns `true` if the path is a regular file.

This function follows symbolic links, so if a symbolic link points to a regular file, `true` is returned.

Options

The supported options are:

- `:raw` - a single atom to bypass the file server and only check for the file locally

Examples

```
File.regular?(__ENV__.file)
#=> true
```

rename(source, destination)

(since 1.1.0)

```
@spec rename(Path.t(), Path.t()) :: :ok | {:error, posix() |
:badarg}
```

Renames the `source` file to `destination` file. It can be used to move files (and directories) between directories. If moving a file, you must fully specify the `destination` filename, it is not sufficient to simply specify its directory.

Returns `:ok` in case of success, `{:error, reason}` otherwise.

Note: The command `mv` in Unix-like systems behaves differently depending on whether `source` is a file and the `destination` is an existing directory. We have chosen to explicitly disallow this behavior.

Examples

```
# Rename file "a.txt" to "b.txt"
File.rename("a.txt", "b.txt")
#=> :ok

# Rename directory "samples" to "tmp"
File.rename("samples", "tmp")
#=> :ok

File.rename("non_existing.txt", "existing.txt")
#=> {:error, :enoent}
```

rename!(source, destination)

(since 1.9.0)

```
@spec rename!(Path.t(), Path.t()) :: :ok
```

The same as `rename/2` but raises a `File.RenameError` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.rename!("samples", "tmp")
#=> :ok

File.rename!("non_existing.txt", "existing.txt")
** (File.RenameError) could not rename from
"non_existing.txt" to "existing.txt": no such file or
directory
```

rm(path)

```
@spec rm(Path.t()) :: :ok | {:error, posix() | :badarg}
```

Tries to delete the file `path`.

Returns `:ok` if successful, or `{:error, reason}` if an error occurs.

Note the file is deleted even if in read-only mode.

Typical error reasons are:

- `:enoent` - the file does not exist
- `:eacces` - missing permission for the file or one of its parents
- `:eperm` - the file is a directory and user is not super-user
- `:enotdir` - a component of the file name is not a directory; on some platforms, `:enoent` is returned instead
- `:einval` - filename had an improper type, such as tuple

Examples

```
File.rm("file.txt")
#=> :ok

File.rm("tmp_dir/")
#=> {:error, :eperm}
```

rm!(path)

```
@spec rm!(Path.t()) :: :ok
```

Same as `rm/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.rm!("file.txt")
#=> :ok

File.rm!("non_existing/")
** (File.Error) could not remove file "non_existing/": no
such file or directory
```

rm_rf(path)

```
@spec rm_rf(Path.t()) :: {:ok, [binary()]} | {:error, posix()
| :badarg, binary()}
```

Removes files and directories recursively at the given `path`. Symlinks are not followed but simply removed, non-existing files are simply ignored (i.e. doesn't make this function fail).

Returns `{:ok, files_and_directories}` with all files and directories removed in no specific order, `{:error, reason, file}` otherwise.

Examples

```
File.rm_rf("samples")
#=> {:ok, ["samples", "samples/1.txt"]}

File.rm_rf("unknown")
#=> {:ok, []}

File.rm_rf("/tmp")
#=> {:error, :eperm, "/tmp"}
```

rm_rf!(path)

```
@spec rm_rf!(Path.t()) :: [binary()]
```

Same as `rm_rf/1` but raises a `File.Error` exception in case of failures, otherwise returns the list of files or directories removed.

Examples

```
File.rm_rf!("samples")
#=> ["samples", "samples/1.txt"]

File.rm_rf!("unknown")
#=> []

File.rm_rf!("/tmp")
** (File.Error) could not remove files and directories
recursively from "/tmp": not owner
```

rmdir(path)

```
@spec rmdir(Path.t()) :: :ok | {:error, posix() | :badarg}
```

Tries to delete the dir at `path`.

Returns `:ok` if successful, or `{:error, reason}` if an error occurs. It returns `{:error, :eexist}` if the directory is not empty.

Examples

```
File.rmdir("tmp_dir")
#=> :ok

File.rmdir("non_empty_dir")
#=> {:error, :eexist}

File.rmdir("file.txt")
#=> {:error, :enotdir}
```

rmdir!(path)

```
@spec rmdir!(Path.t()) :: :ok
```

Same as `rmdir/1`, but raises a `File.Error` exception in case of failure. Otherwise `:ok`.

Examples

```
File.rmdir!("tmp_dir")
#=> :ok

File.rmdir!("non_empty_dir")
** (File.Error) could not remove directory
"non_empty_dir": directory is not empty

File.rmdir!("file.txt")
** (File.Error) could not remove directory "file.txt":
not a directory
```

stat(path, opts \\ [])

```
@spec stat(Path.t(), stat_options()) ::
  {:ok, File.Stat.t()} | {:error, posix() | :badarg}
```

Returns information about the `path`. If it exists, it returns a `{:ok, info}` tuple, where info is a `File.Stat` struct. Returns `{:error, reason}` with the same reasons as `read/1` if a failure occurs.

Options

The accepted options are:

- `:time` - configures how the file timestamps are returned

The values for `:time` can be:

- `:universal` - returns a `{date, time}` tuple in UTC (default)
- `:local` - returns a `{date, time}` tuple using the same time zone as the machine
- `:posix` - returns the time as integer seconds since epoch

Note: Since file times are stored in POSIX time format on most operating systems, it is faster to retrieve file information with the `time: :posix` option.

Examples

```
File.stat("hello.txt")
#=> {:ok, %File.Stat{...}}

File.stat("non_existing.txt", time: :posix)
#=> {:error, :enoent}
```

stat!(path, opts \\ [])

```
@spec stat!(Path.t(), stat_options()) :: File.Stat.t()
```

Same as `stat/2` but returns the `File.Stat` directly, or raises a `File.Error` exception if an error is returned.

Examples

```
File.stat!("hello.txt")
#=> %File.Stat{...}

File.stat!("non_existing.txt", time: :posix)
** (File.Error) could not read file stats
"non_existing.txt": no such file or directory
```

stream!(path, line_or_bytes_modes \\ [])

```
@spec stream!(Path.t(), :line | pos_integer() |
[stream_mode()]) :: File.Stream.t()
```

Shortcut for `File.stream!/3`.

stream!(path, line_or_bytes, modes)

```
@spec stream!(Path.t(), :line | pos_integer(),
[stream_mode()]) :: File.Stream.t()
```

Returns a `File.Stream` for the given `path` with the given `modes`.

The stream implements both `Enumerable` and `Collectable` protocols, which means it can be used both for read and write.

The `line_or_bytes` argument configures how the file is read when streaming, by `:line` (default) or by a given number of bytes. When using the `:line` option, CRLF line breaks (`"\r\n"`) are normalized to LF (`"\n"`).

Similar to other file operations, a stream can be created in one node and forwarded to another node. Once the stream is opened in another node, a request will be sent to the creator node to spawn a process for file streaming.

Operating the stream can fail on open for the same reasons as `File.open!/2`. Note that the file is automatically opened each time streaming begins. There is no need to pass `:read` and `:write` modes, as those are automatically set by Elixir.

Raw files

Since Elixir controls when the streamed file is opened, the underlying device cannot be shared and as such it is convenient to open the file in raw mode for performance reasons. Therefore, Elixir **will** open streams in `:raw` mode with the `:read_ahead` option if the stream is open in the same node as it is created and no encoding has been specified. This means any data streamed into the file must be converted to `iodata/0` type. If you pass, for example, `[encoding: :utf8]` or `[encoding: {:utf16, :little}]` in the modes parameter, the underlying stream will use `IO.write/2` and the `String.Chars` protocol to convert the data. See `IO.binwrite/2` and `IO.write/2` .

One may also consider passing the `:delayed_write` option if the stream is meant to be written to under a tight loop.

Byte order marks and read offset

If you pass `:trim_bom` in the modes parameter, the stream will trim UTF-8, UTF-16 and UTF-32 byte order marks when reading from file.

Note that this function does not try to discover the file encoding based on BOM. From Elixir v1.16.0, you may also pass a `:read_offset` that

is skipped whenever enumerating the stream (if both `:read_offset` and `:trim_bom` are given, the offset is skipped after the BOM).

See `Stream.run/1` for an example of streaming into a file.

Examples

```
# Read a utf8 text file which may include BOM
File.stream!("./test/test.txt", [:trim_bom, encoding:
:utf8])
#=> %File.Stream{path: "./test/test.txt", ...}

# Read in 2048 byte chunks rather than lines
File.stream!("./test/test.data", 2048)
#=> %File.Stream{path: "./test/test.data", ...}
```

touch(path, time \\ System.os_time(:second))

```
@spec touch(Path.t(), erlang_time() | posix_time()) ::
  :ok | {:error, posix() | :badarg | :terminated |
:system_limit}
```

Updates modification time (mtime) and access time (atime) of the given file.

The file is created if it doesn't exist. Requires datetime in UTC (as returned by `:erlang.universaltime()`) or an integer representing the POSIX timestamp (as returned by `System.os_time(:second)`).

In Unix-like systems, changing the modification time may require you to be either `root` or the owner of the file. Having write access may not be enough. In those cases, touching the file the first time (to create it) will succeed, but touching an existing file with fail with `{:error, :eperm}`.

Examples

```
File.touch("/tmp/a.txt", {{2018, 1, 30}, {13, 59, 59}})
#=> :ok
File.touch("/fakedir/b.txt", {{2018, 1, 30}, {13, 59,
59}})
{:error, :enoent}

File.touch("/tmp/a.txt", 1_544_519_753)
#=> :ok
```

touch!(path, time \\ System.os_time(:second))

```
@spec touch!(Path.t(), erlang_time() | posix_time()) :: :ok
```

Same as `touch/2` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

The file is created if it doesn't exist. Requires datetime in UTC (as returned by `:erlang.universaltime()`) or an integer representing the POSIX timestamp (as returned by `System.os_time(:second)`).

Examples

```
File.touch!("/tmp/a.txt", {{2018, 1, 30}, {13, 59, 59}})
#=> :ok
File.touch!("/fakedir/b.txt", {{2018, 1, 30}, {13, 59,
59}})
** (File.Error) could not touch "/fakedir/b.txt": no such
file or directory
```

```
File.touch!("/tmp/a.txt", 1_544_519_753)
```

write(path, content, modes \\ [])

```
@spec write(Path.t(), iodata(), [mode()]) ::
  :ok | {:error, posix() | :badarg | :terminated |
:system_limit}
```

Writes `content` to the file `path`.

The file is created if it does not exist. If it exists, the previous contents are overwritten. Returns `:ok` if successful, or `{:error, reason}` if an error occurs.

`content` must be `iodata` (a list of bytes or a binary). Setting the encoding for this function has no effect.

Warning: Every time this function is invoked, a file descriptor is opened and a new process is spawned to write to the file. For this reason, if you are doing multiple writes in a loop, opening the file via `File.open/2` and using the functions in `IO` to write to the file will yield much better performance than calling this function multiple times.

Typical error reasons are:

- `:enoent` - a component of the file name does not exist
- `:enotdir` - a component of the file name is not a directory; on some platforms, `:enoent` is returned instead
- `:enospc` - there is no space left on the device
- `:eacces` - missing permission for writing the file or searching one of the parent directories
- `:eisdir` - the named file is a directory

Check `File.open/2` for the list of available `modes`.

Examples

```
File.write("hello.txt", "world!")
#=> :ok

File.write("temp", "world!")
#=> {:error, :eisdir}
```

write!(path, content, modes \\ [])

```
@spec write!(Path.t(), iodata(), [mode()]) :: :ok
```

Same as `write/3` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.write!("hello.txt", "world!")
#=> :ok

File.write!("temp", "world!")
** (File.Error) could not write to file "temp": illegal
operation on a directory
```

write_stat(path, stat, opts \\ [])

```
@spec write_stat(Path.t(), File.Stat.t(), stat_options()) ::
  :ok | {:error, posix() | :badarg}
```

Writes the given `File.Stat` back to the file system at the given path. Returns `:ok` or `{:error, reason}`.

Examples

```
File.write_stat("hello.txt", new_stat)
#=> :ok

File.write_stat("non_existing.txt", new_stat)
#=> {:error, :enoent}
```

write_stat!(path, stat, opts \\ [])

```
@spec write_stat!(Path.t(), File.Stat.t(), stat_options()) ::
:ok
```

Same as `write_stat/3` but raises a `File.Error` exception if it fails. Returns `:ok` otherwise.

Examples

```
File.write_stat!("hello.txt", new_stat)
#=> :ok

File.write_stat!("non_existing.txt", new_stat)
** (File.Error) could not write file stats
"non_existing.txt": no such file or directory
```

File.Stat

A struct that holds file information.

In Erlang, this struct is represented by a `:file_info` record. Therefore this module also provides functions for converting between the Erlang record and the Elixir struct.

Its fields are:

- `size` - size of file in bytes.
- `type` - `:device | :directory | :regular | :other | :symlink`; the type of the file.
- `access` - `:read | :write | :read_write | :none`; the current system access to the file.
- `atime` - the last time the file was read.
- `mtime` - the last time the file was written.
- `ctime` - the interpretation of this time field depends on the operating system. On Unix-like operating systems, it is the last time the file or the inode was changed. In Windows, it is the time of creation.
- `mode` - the file permissions.
- `links` - the number of links to this file. This is always 1 for file systems which have no concept of links.
- `major_device` - identifies the file system where the file is located. In Windows, the number indicates a drive as follows: 0 means A:, 1

means B:, and so on.

- `minor_device` - only valid for character devices on Unix-like systems. In all other cases, this field is zero.
- `inode` - gives the inode number. On non-Unix-like file systems, this field will be zero.
- `uid` - indicates the owner of the file. Will be zero for non-Unix-like file systems.
- `gid` - indicates the group that owns the file. Will be zero for non-Unix-like file systems.

The time type returned in `atime`, `mtime`, and `ctime` is dependent on the time type set in options. `{:time, type}` where type can be `:local`, `:universal`, or `:posix`. Default is `:universal`.

Summary

Types

t()

Functions

from_record(file_info)

Converts a `:file_info` record into a `File.Stat`.

to_record(stat)

Converts a `File.Stat` struct to a `:file_info` record.

Types

t()

```
@type t() :: %File.Stat{
  access: :read | :write | :read_write | :none | :undefined,
  atime: :calendar.datetime() | integer() | :undefined,
  ctime: :calendar.datetime() | integer() | :undefined,
  gid: non_neg_integer() | :undefined,
  inode: non_neg_integer() | :undefined,
  links: non_neg_integer() | :undefined,
  major_device: non_neg_integer() | :undefined,
  minor_device: non_neg_integer() | :undefined,
  mode: non_neg_integer() | :undefined,
  mtime: :calendar.datetime() | integer() | :undefined,
  size: non_neg_integer() | :undefined,
  type: :device | :directory | :regular | :other | :symlink |
:undefined,
  uid: non_neg_integer() | :undefined
}
```

Functions

from_record(file_info)

```
@spec from_record(:file.file_info()) :: t()
```

Converts a `:file_info` record into a `File.Stat`.

to_record(stat)

```
@spec to_record(t()) :: :file.file_info()
```

Converts a `File.Stat` struct to a `:file_info` record.

File.Stream

Defines a `File.Stream` struct returned by `File.stream!/3`.

The following fields are public:

- `path` - the file path
- `modes` - the file modes
- `raw` - a boolean indicating if bin functions should be used
- `line_or_bytes` - if reading should read lines or a given number of bytes
- `node` - the node the file belongs to

Summary

Types

t()

Types

t()

```
@type t() :: %File.Stream{
  line_or_bytes: term(),
  modes: term(),
  node: term(),
  path: term(),
```

```
  raw: term()
}
```

IO

Functions handling input/output (IO).

Many functions in this module expect an IO device as an argument. An IO device must be a PID or an atom representing a process. For convenience, Elixir provides `:stdio` and `:stderr` as shortcuts to Erlang's `:standard_io` and `:standard_error`.

The majority of the functions expect chardata. In case another type is given, functions will convert those types to string via the `String.Chars` protocol (as shown in typespecs). For more information on chardata, see the "IO data" section below.

The functions of this module use UNIX-style naming where possible.

IO devices

An IO device may be an atom or a PID. In case it is an atom, the atom must be the name of a registered process. In addition, Elixir provides two shortcuts:

- `:stdio` - a shortcut for `:standard_io`, which maps to the current `Process.group_leader/0` in Erlang
- `:stderr` - a shortcut for the named process `:standard_error` provided in Erlang

IO devices maintain their position, which means subsequent calls to any reading or writing functions will start from the place where the device was last accessed. The position of files can be changed using the `:file.position/2` function.

IO data

IO data is a data type that can be used as a more efficient alternative to binaries in certain situations.

A term of type **IO data** is a binary or a list containing bytes (integers within the `0..255` range) or nested IO data. The type is recursive. Let's see an example of one of the possible IO data representing the binary `"hello"`:

```
[?h, "el", ["l", [?o]]]
```

The built-in `iodata/0` type is defined in terms of `iolist/0`. An IO list is the same as IO data but it doesn't allow for a binary at the top level (but binaries are still allowed in the list itself).

Use cases for IO data

IO data exists because often you need to do many append operations on smaller chunks of binaries in order to create a bigger binary. However, in Erlang and Elixir concatenating binaries will copy the concatenated binaries into a new binary.

```
def email(username, domain) do
  username <> "@" <> domain
end
```

In this function, creating the email address will copy the `username` and `domain` binaries. Now imagine you want to use the resulting email inside another binary:

```
def welcome_message(name, username, domain) do
  "Welcome #{name}, your email is: #{email(username,
domain)}"
end

IO.puts(welcome_message("Meg", "meg", "example.com"))
#=> "Welcome Meg, your email is: meg@example.com"
```

Every time you concatenate binaries or use interpolation (`#{}`) you are making copies of those binaries. However, in many cases you don't need the complete binary while you create it, but only at the end to print it out or send it somewhere. In such cases, you can construct the binary by creating IO data:

```
def email(username, domain) do
  [username, ?@, domain]
end

def welcome_message(name, username, domain) do
  ["Welcome ", name, ", your email is: ", email(username,
domain)]
end

IO.puts(welcome_message("Meg", "meg", "example.com"))
#=> "Welcome Meg, your email is: meg@example.com"
```

Building IO data is cheaper than concatenating binaries. Concatenating multiple pieces of IO data just means putting them together inside a list since IO data can be arbitrarily nested, and that's a cheap and efficient operation. Most of the IO-based APIs, such as `:gen_tcp` and `IO`, receive IO data and write it to the socket directly without converting it to binary.

One drawback of IO data is that you can't do things like pattern match on the first part of a piece of IO data like you can with a binary, because you usually don't know the shape of the IO data. In those cases, you may need to convert it to a binary by calling `iodata_to_binary/1`, which is reasonably efficient since it's implemented natively in C. Other functionality, like computing the length of IO data, can be computed directly on the iodata by calling `iodata_length/1`.

Chardata

Erlang and Elixir also have the idea of `chardata/0`. Chardata is very similar to IO data: the only difference is that integers in IO data represent bytes while integers in chardata represent Unicode code points. Bytes (`byte/0`) are integers within the `0..255` range, while Unicode code points (`char/0`) are integers within the `0..0x10FFFF` range. The `IO` module provides the `chardata_to_string/1` function for chardata as the "counter-part" of the `iodata_to_binary/1` function for IO data.

If you try to use `iodata_to_binary/1` on chardata, it will result in an argument error. For example, let's try to put a code point that is not representable with one byte, like `?π`, inside IO data:

```
IO.iodata_to_binary(["The symbol for pi is: ", ?π])
#=> ** (ArgumentError) argument error
```

If we use chardata instead, it will work as expected:

```
iex> IO.chardata_to_string(["The symbol for pi is: ", ?
π])
"The symbol for pi is: π"
```

Summary

Types

chardata()
device()
inspect_opts()
nodata()
warn_stacktrace_opts()

Stacktrace information as keyword options for `warn/2`.

Functions

binread(device \\ :stdio, line_or_chars)

Reads from the IO `device`. The operation is Unicode unsafe.

binstream()

Returns a raw, line-based `IO.Stream` on `:stdio`. The operation is Unicode unsafe.

binstream(device \\ :stdio, line_or_bytes)

Converts the IO `device` into an `IO.Stream`. The operation is Unicode unsafe.

binwrite(device \\ :stdio, iodata)

Writes `iodata` to the given `device`.

chardata_to_string(chardata)

Converts chardata into a string.

getn(prompt, count \\ 1)

Gets a number of bytes from IO device `:stdio`.

getn(device, prompt, count)

Gets a number of bytes from the IO `device`.

gets(device \\ :stdio, prompt)

Reads a line from the IO `device`.

inspect(item, opts \\ [])

Inspects and writes the given `item` to the standard output.

inspect(device, item, opts)

Inspects `item` according to the given options using the IO `device`.

iodata_empty?(arg1)

Checks if an IO data (the length is zero).

iodata_length(iodata)

Returns the size of an IO data.

iodata_to_binary(iodata)

Converts IO data into a binary

puts(device \\ :stdio, item)

Writes `item` to the given `device`, similar to `write/2`, but adds a newline at the end.

read(device \\ :stdio, line_or_chars)

Reads from the IO `device`.

stream()

Returns a line-based `IO.Stream` on `:stdio`.

stream(device \\ :stdio, line_or_codepoints)

Converts the IO `device` into an `IO.Stream`.

warn(message)

Writes a `message` to stderr, along with the current stacktrace.

warn(message, stacktrace_info)

Writes a `message` to stderr, along with the given `stacktrace_info`.

write(device \\ :stdio, chardata)

Writes `chardata` to the given `device`.

Types

chardata()

```
@type chardata() ::
  String.t() | maybe_improper_list(char() | chardata(),
String.t() | [])
```

device()

```
@type device() :: atom() | pid()
```

inspect_opts()

```
@type inspect_opts() :: [Inspect.Opts.new opt() | {:label,
term()}]
```

nodata()

```
@type nodata() :: {:error, term()} | :eof
```

warn_stacktrace_opts()

```
@type warn_stacktrace_opts() :: [
  file: String.t(),
  line: pos integer(),
  column: pos integer(),
  module: module(),
  function: {atom(), arity()}
]
```

Stacktrace information as keyword options for `warn/2`.

At least `:file` is required. Other options are optional and used to provide more precise location information.

Functions

binread(device \\ :stdio, line_or_chars)

```
@spec binread(device(), :eof | :line | non_neg_integer()) ::
iodata() | nodata()
```

Reads from the IO `device`. The operation is Unicode unsafe.

The `device` is iterated as specified by the `line_or_chars` argument:

- if `line_or_chars` is an integer, it represents a number of bytes. The device is iterated by that number of bytes. This should be the preferred mode for reading non-textual inputs.
- if `line_or_chars` is `:line`, the device is iterated line by line. CRLF newlines ("\r\n") are automatically normalized to "\n".
- if `line_or_chars` is `:eof` (since v1.13), the device is iterated until `:eof`. If the device is already at the end, it returns `:eof` itself.

It returns:

- `data` - the output bytes
- `:eof` - end of file was encountered
- `{:error, reason}` - other (rare) error condition; for instance, `{:error, :estale}` if reading from an NFS volume

Note: do not use this function on IO devices in Unicode mode as it will return the wrong result.

binstream()

(since 1.12.0)

```
@spec binstream() :: Enumerable.t(binary())
```

Returns a raw, line-based `IO.Stream` on `:stdio`. The operation is Unicode unsafe.

This is equivalent to:

```
IO.binstream(:stdio, :line)
```

binstream(device \\ :stdio, line_or_bytes)

```
@spec binstream(device(), :line | pos_integer()) ::
Enumerable.t()
```

Converts the IO `device` into an `IO.Stream`. The operation is Unicode unsafe.

An `IO.Stream` implements both `Enumerable` and `Collectable`, allowing it to be used for both read and write.

The `device` is iterated by the given number of bytes or line by line if `:line` is given. In case `:line` is given, "\r\n" is automatically normalized to "\n". Passing the number of bytes should be the preferred mode for reading non-textual inputs.

Note that an IO stream has side effects and every time you go over the stream you may get different results.

This reads from the IO device as a raw binary. Therefore, do not use this function on IO devices in Unicode mode as it will return the wrong result.

`binstream/0` has been introduced in Elixir v1.12.0, while `binstream/2` has been available since v1.0.0.

binwrite(device \\ :stdio, iodata)

```
@spec binwrite(device(), iodata()) :: :ok
```

Writes `iodata` to the given `device`.

This operation is meant to be used with "raw" devices that are started without an encoding. The given `iodata` is written as is to the device, without conversion. For more information on IO data, see the "IO data" section in the module documentation.

Use `write/2` for devices with encoding.

Important: do **not** use this function on IO devices in Unicode mode as it will write the wrong data. In particular, the standard IO device is set to Unicode by default, so writing to stdio with this function will likely result in the wrong data being sent down the wire.

chardata_to_string(chardata)

```
@spec chardata_to_string(chardata()) :: String.t()
```

Converts chardata into a string.

For more information about chardata, see the "Chardata" section in the module documentation.

In case the conversion fails, it raises an `UnicodeConversionError`. If a string is given, it returns the string itself.

Examples

```
iex> IO.chardata_to_string([0x00E6, 0x00DF])
"æß"

iex> IO.chardata_to_string([0x0061, "bc"])
"abc"

iex> IO.chardata_to_string("string")
"string"
```

getn(prompt, count \\ 1)

```
@spec getn(
  device() | chardata() | String.Chars.t(),
  pos_integer() | :eof | chardata() | String.Chars.t()
) :: chardata() | nodata()
```

Gets a number of bytes from IO device `:stdio`.

If `:stdio` is a Unicode device, `count` implies the number of Unicode code points to be retrieved. Otherwise, `count` is the number of raw bytes to be retrieved.

See `IO.getn/3` for a description of return values.

getn(device, prompt, count)

```
@spec getn(device(), chardata() | String.Chars.t(),
pos_integer() | :eof) ::
  chardata() | nodata()
```

Gets a number of bytes from the IO `device`.

If the IO `device` is a Unicode device, `count` implies the number of Unicode code points to be retrieved. Otherwise, `count` is the number of raw bytes to be retrieved.

It returns:

- `data` - the input characters
- `:eof` - end of file was encountered
- `{:error, reason}` - other (rare) error condition; for instance, `{:error, :estale}` if reading from an NFS volume

gets(device \\ :stdio, prompt)

```
@spec gets(device(), chardata() | String.Chars.t()) ::
chardata() | nodata()
```

Reads a line from the IO `device`.

It returns:

- `data` - the characters in the line terminated by a line-feed (LF) or end of file (EOF)
- `:eof` - end of file was encountered
- `{:error, reason}` - other (rare) error condition; for instance, `{:error, :estale}` if reading from an NFS volume

Trivia: `gets` is shorthand for `get string`.

Examples

To display "What is your name?" as a prompt and await user input:

```
IO.gets("What is your name?\n")
```

inspect(item, opts \\ [])

`@spec inspect(item, inspect_opts()) :: item when item: var`

Inspects and writes the given `item` to the standard output.

It's important to note that it returns the given `item` unchanged. This makes it possible to "spy" on values by inserting an `IO.inspect/2` call almost anywhere in your code, for example, in the middle of a pipeline.

It enables pretty printing by default with width of 80 characters. The width can be changed by explicitly passing the `:width` option.

The output can be decorated with a label, by providing the `:label` option to easily distinguish it from other `IO.inspect/2` calls. The label will be printed before the inspected `item`.

See `Inspect.Opts` for a full list of remaining formatting options. To print to other IO devices, see `IO.inspect/3`

Examples

The following code:

```
IO.inspect(<<0, 1, 2>>, width: 40)
```

Prints:

```
<<0, 1, 2>>
```

You can use the `:label` option to decorate the output:

```
IO.inspect(1..100, label: "a wonderful range")
```

Prints:

```
a wonderful range: 1..100
```

Inspect truncates large inputs by default. The `:printable_limit` controls the limit for strings and other string-like constructs (such as charlists):

```
"abc"
|> String.duplicate(9001)
|> IO.inspect(printable_limit: :infinity)
```

For containers such as lists, maps, and tuples, the number of entries is managed by the `:limit` option:

```
1..100
|> Enum.map(& {&1, &1})
|> Enum.into(%{})
|> IO.inspect(limit: :infinity)
```

inspect(device, item, opts)

```
@spec inspect(device(), item, inspect_opts()) :: item when
item: var
```

Inspects `item` according to the given options using the IO `device`.

See `inspect/2` for a full list of options.

iodata_empty?(arg1)

(since 1.20.0)

```
@spec iodata_empty?(iodata()) :: boolean()
```

Checks if an IO data (the length is zero).

For more information about IO data, see the "IO data" section in the module documentation.

Examples

```
iex> IO.iodata_empty?([])
true
iex> IO.iodata_empty?([""])
true
iex> IO.iodata_empty?([1, 2 | <<3, 4>>])
false
```

iodata_length(iodata)

```
@spec iodata_length(iodata()) :: non_neg_integer()
```

Returns the size of an IO data.

For more information about IO data, see the "IO data" section in the module documentation.

Inlined by the compiler.

Examples

```
iex> IO.iodata_length([1, 2 | <<3, 4>>])
4
```

iodata_to_binary(iodata)

```
@spec iodata_to_binary(iodata()) :: binary()
```

Converts IO data into a binary

The operation is Unicode unsafe.

Note that this function treats integers in the given IO data as raw bytes and does not perform any kind of encoding conversion. If you want to convert from a charlist to a UTF-8-encoded string, use `chardata_to_string/1` instead. For more information about IO data and chardata, see the "IO data" section in the module documentation.

If this function receives a binary, the same binary is returned.

Inlined by the compiler.

Examples

```
iex> bin1 = <<1, 2, 3>>
iex> bin2 = <<4, 5>>
iex> bin3 = <<6>>
```

```
iex> IO.iodata_to_binary([bin1, 1, [2, 3, bin2], 4 |
bin3])
<<1, 2, 3, 1, 2, 3, 4, 5, 4, 6>>

iex> bin = <<1, 2, 3>>
iex> IO.iodata_to_binary(bin)
<<1, 2, 3>>
```

puts(device \\ :stdio, item)

```
@spec puts(device(), chardata() | String.Chars.t()) :: :ok
```

Writes `item` to the given `device`, similar to `write/2`, but adds a newline at the end.

By default, the `device` is the standard output. It returns `:ok` if it succeeds.

Trivia: `puts` is shorthand for `put string`.

Examples

```
IO.puts("Hello World!")
#=> Hello World!

IO.puts(:stderr, "error")
#=> error
```

read(device \\ :stdio, line_or_chars)

```
@spec read(device(), :eof | :line | non_neg_integer()) ::
chardata() | nodata()
```

Reads from the IO `device`.

The `device` is iterated as specified by the `line_or_chars` argument:

- if `line_or_chars` is an integer, it represents a number of bytes. The device is iterated by that number of bytes. This should be the preferred mode for reading non-textual inputs.
- if `line_or_chars` is `:line`, the device is iterated line by line. CRLF newlines ("\r\n") are automatically normalized to "\n".
- if `line_or_chars` is `:eof` (since v1.13), the device is iterated until `:eof`. If the device is already at the end, it returns `:eof` itself.

It returns:

- `data` - the output characters
- `:eof` - end of file was encountered
- `{:error, reason}` - other (rare) error condition; for instance, `{:error, :estale}` if reading from an NFS volume

stream()

(since 1.12.0)

```
@spec stream() :: Enumerable.t(String.t())
```

Returns a line-based `IO.Stream` on `:stdio`.

This is equivalent to:

```
IO.stream(:stdio, :line)
```

stream(device \\ :stdio, line_or_codepoints)

```
@spec stream(device(), :line | pos_integer()) ::
Enumerable.t()
```

Converts the IO `device` into an `IO.Stream`.

An `IO.Stream` implements both `Enumerable` and `Collectable`, allowing it to be used for both read and write.

The `device` is iterated by the given number of characters or line by line if `:line` is given. In case `:line` is given, "\r\n" is automatically normalized to "\n".

This reads from the IO as UTF-8. Check out `IO.binstream/2` to handle the IO as a raw binary.

Note that an IO stream has side effects and every time you go over the stream you may get different results.

`stream/0` has been introduced in Elixir v1.12.0, while `stream/2` has been available since v1.0.0.

Examples

Here is an example on how we mimic an echo server from the command line:

```
Enum.each(IO.stream(:stdio, :line), &IO.write(&1))
```

Another example where you might want to collect a user input every new line and break on an empty line, followed by removing redundant new line characters (`"\n"`):

```
IO.stream(:stdio, :line)
|> Enum.take_while(&(&1 != "\n"))
|> Enum.map(&String.replace(&1, "\n", ""))
```

warn(message)

```
@spec warn(chardata() | String.Chars.t()) :: :ok
```

Writes a `message` to stderr, along with the current stacktrace.

It returns `:ok` if it succeeds.

Do not call this function at the tail of another function. Due to tail call optimization, a stacktrace entry would not be added and the stacktrace would be incorrectly trimmed. Therefore make sure at least one expression (or an atom such as `:ok`) follows the `IO.warn/1` call.

Examples

```
IO.warn("variable bar is unused")
#=> warning: variable bar is unused
#=>   (iex) evaluator.ex:108: IEx.Evaluator.eval/4
```

warn(message, stacktrace_info)

```
@spec warn(
  chardata() | String.Chars.t(),
  Exception.stacktrace() | warn_stacktrace_opts() |
Macro.Env.t()
) :: :ok
```

Writes a `message` to stderr, along with the given `stacktrace_info`.

The `stacktrace_info` must be one of:

- a `__STACKTRACE__`, where all entries in the stacktrace will be included in the error message
- a `Macro.Env` structure (since v1.14.0), where a single stacktrace entry from the compilation environment will be used
- a keyword list with at least the `:file` option representing a single stacktrace entry (since v1.14.0). The `:line`, `:column`, `:module`, and `:function` options are also supported

This function notifies the compiler a warning was printed and emits a compiler diagnostic (`Code.diagnostic/1`). The diagnostic will include precise file and location information if a `Macro.Env` is given or those values have been passed as keyword list, but not for stacktraces, as they are often imprecise.

It returns `:ok` if it succeeds.

Examples

```
IO.warn("variable bar is unused", module: MyApp,
function: {:main, 1}, line: 4, file: "my_app.ex")
```

```
#=> warning: variable bar is unused
#=>   my_app.ex:4: MyApp.main/1
```

write(device \\ :stdio, chardata)

```
@spec write(device(), chardata() | String.Chars.t()) :: :ok
```

Writes `chardata` to the given `device`.

By default, the `device` is the standard output.

Examples

```
IO.write("sample")
#=> sample

IO.write(:stderr, "error")
#=> error
```

IO.ANSI

Functionality to render ANSI escape sequences.

ANSI escape sequences are characters embedded in text used to control formatting, color, and other output options on video text terminals.

ANSI escapes are typically enabled on all Unix terminals. They are also available on Windows consoles from Windows 10, although it must be explicitly enabled for the current user in the registry by running the following command:

```
reg add HKCU\Console /v VirtualTerminalLevel /t REG_DWORD
/d 1
```

After running the command above, you must restart your current console.

Examples

Because the ANSI escape sequences are embedded in text, the normal usage of these functions is to concatenate their output with text.

```
formatted_text = IO.ANSI.blue_background() <> "Example"
<> IO.ANSI.reset()
IO.puts(formatted_text)
```

A higher level and more convenient API is also available via `IO.ANSI.format/1`, where you use atoms to represent each ANSI escape sequence and by default checks if ANSI is enabled:

```
IO.puts(IO.ANSI.format([:blue_background, "Example"]))
```

In case ANSI is disabled, the ANSI escape sequences are simply discarded.

Summary

Types

ansicode()
ansidata()
ansilist()

Functions

black()

Sets foreground color to black.

black_background()

Sets background color to black.

blink_off()

Blink: off.

blink_rapid()

Blink: rapid. MS-DOS ANSI.SYS; 150 per minute or more; not widely supported.

blink_slow()

Blink: slow. Less than 150 per minute.

blue()

Sets foreground color to blue.

blue_background()

Sets background color to blue.

bright()

Bright (increased intensity) or bold.

clear()

Clears screen.

clear_line()

Clears line.

color(code)

Sets foreground color.

color(r, g, b)

Sets the foreground color from individual RGB values.

color_background(code)

Sets background color.

color_background(r, g, b)

Sets the background color from individual RGB values.

conceal()

Conceal. Not widely supported.

crossed_out()

Crossed-out. Characters legible, but marked for deletion. Not widely supported.

cursor(line, column)

Sends cursor to the absolute position specified by `line` and `column`.

cursor_down(lines \\ 1)

Sends cursor `lines` down.

cursor_left(columns \\ 1)

Sends cursor `columns` to the left.

cursor_right(columns \\ 1)

Sends cursor `columns` to the right.

cursor_up(lines \\ 1)

Sends cursor `lines` up.

cyan()

Sets foreground color to cyan.

cyan_background()

Sets background color to cyan.

default_background()

Default background color.

default_color()

Default text color.

enabled?()

Checks if ANSI coloring is supported and enabled on this machine.

encircled()

Encircled.

faint()

Faint (decreased intensity). Not widely supported.

font_1()

Sets alternative font 1.

font_2()

Sets alternative font 2.

font_3()

Sets alternative font 3.

font_4()

Sets alternative font 4.

font_5()

Sets alternative font 5.

font_6()

Sets alternative font 6.

font_7()

Sets alternative font 7.

font_8()

Sets alternative font 8.

font_9()

Sets alternative font 9.

format(ansidata, emit? \\ enabled?())

Formats a chardata-like argument by converting named ANSI sequences into actual ANSI codes.

format_fragment(ansidata, emit? \\ enabled?())

Formats a chardata-like argument by converting named ANSI sequences into actual ANSI codes.

framed()

Framed.

green()

Sets foreground color to green.

green_background()

Sets background color to green.

home()

Sends cursor home.

inverse()

Image: negative. Swap foreground and background.
inverse_off()

Image: positive. Normal foreground and background.

italic()

Italic: on. Not widely supported. Sometimes treated as inverse.

light_black()

Sets foreground color to light black.

light_black_background()

Sets background color to light black.

light_blue()

Sets foreground color to light blue.

light_blue_background()

Sets background color to light blue.

light_cyan()

Sets foreground color to light cyan.

light_cyan_background()

Sets background color to light cyan.

light_green()

Sets foreground color to light green.

light_green_background()

Sets background color to light green.

light_magenta()

Sets foreground color to light magenta.

light_magenta_background()

Sets background color to light magenta.

light_red()

Sets foreground color to light red.

light_red_background()

Sets background color to light red.

light_white()

Sets foreground color to light white.

light_white_background()

Sets background color to light white.

light_yellow()

Sets foreground color to light yellow.

light_yellow_background()

Sets background color to light yellow.

magenta()

Sets foreground color to magenta.

magenta_background()

Sets background color to magenta.

no_underline()

Underline: none.

normal()

Normal color or intensity.

not_crossed_out()

Not crossed-out.

not_framed_encircled()

Not framed or encircled.

not_italic()

Not italic.

not_overlined()

Not overlined.

overlined()

Overlined.

primary_font()

Sets primary (default) font.

red()

Sets foreground color to red.

red_background()

Sets background color to red.

reset()

Resets all attributes.

reveal()

Reveal: Not concealed.

reverse()

Image: negative. Swap foreground and background.

reverse_off()

Image: positive. Normal foreground and background.

syntax_colors()

Syntax colors to be used by `Inspect`.

underline()

Underline: single.

white()

Sets foreground color to white.

white_background()

Sets background color to white.

yellow()

Sets foreground color to yellow.

yellow_background()

Sets background color to yellow.

Types

ansicode()

```
@type ansicode() :: atom()
```

ansidata()

```
@type ansidata() :: ansilist() | ansicode() | binary()
```

ansilist()

```
@type ansilist() ::
  maybe_improper_list(
    char() | ansicode() | binary() | ansilist(),
    binary() | ansicode() | []
  )
```

Functions

black()

```
@spec black() :: String.t()
```

Sets foreground color to black.

black_background()

```
@spec black_background() :: String.t()
```

Sets background color to black.

blink_off()

```
@spec blink_off() :: String.t()
```

Blink: off.

blink_rapid()

```
@spec blink_rapid() :: String.t()
```

Blink: rapid. MS-DOS ANSI.SYS; 150 per minute or more; not widely supported.

blink_slow()

```
@spec blink_slow() :: String.t()
```

Blink: slow. Less than 150 per minute.

blue()

```
@spec blue() :: String.t()
```

Sets foreground color to blue.

blue_background()

```
@spec blue_background() :: String.t()
```

Sets background color to blue.

bright()

```
@spec bright() :: String.t()
```

Bright (increased intensity) or bold.

clear()

```
@spec clear() :: String.t()
```

Clears screen.

clear_line()

```
@spec clear_line() :: String.t()
```

Clears line.

color(code)

```
@spec color(0..255) :: String.t()
```

Sets foreground color.

color(r, g, b)

```
@spec color(0..5, 0..5, 0..5) :: String.t()
```

Sets the foreground color from individual RGB values.

Valid values for each color are in the range 0 to 5.

color_background(code)

```
@spec color_background(0..255) :: String.t()
```

Sets background color.

color_background(r, g, b)

```
@spec color_background(0..5, 0..5, 0..5) :: String.t()
```

Sets the background color from individual RGB values.

Valid values for each color are in the range 0 to 5.

conceal()

```
@spec conceal() :: String.t()
```

Conceal. Not widely supported.

crossed_out()

```
@spec crossed_out() :: String.t()
```

Crossed-out. Characters legible, but marked for deletion. Not widely supported.

cursor(line, column)

```
@spec cursor(non_neg_integer(), non_neg_integer()) ::
String.t()
```

Sends cursor to the absolute position specified by `line` and `column`.

Line `0` and column `0` would mean the top left corner.

cursor_down(lines \\ 1)

```
@spec cursor_down(pos integer()) :: String.t()
```

Sends cursor `lines` down.

cursor_left(columns \\ 1)

```
@spec cursor_left(pos integer()) :: String.t()
```

Sends cursor `columns` to the left.

cursor_right(columns \\ 1)

```
@spec cursor_right(pos integer()) :: String.t()
```

Sends cursor `columns` to the right.

cursor_up(lines \\ 1)

```
@spec cursor_up(pos integer()) :: String.t()
```

Sends cursor `lines` up.

cyan()

```
@spec cyan() :: String.t()
```

Sets foreground color to cyan.

cyan_background()

```
@spec cyan_background() :: String.t()
```

Sets background color to cyan.

default_background()

```
@spec default_background() :: String.t()
```

Default background color.

default_color()

```
@spec default_color() :: String.t()
```

Default text color.

enabled?()

```
@spec enabled?() :: boolean()
```

Checks if ANSI coloring is supported and enabled on this machine.

This function simply reads the configuration value for `:ansi_enabled` in the `:elixir` application. The value is by default `false` unless Elixir can detect during startup that both `stdout` and `stderr` are terminals.

encircled()

```
@spec encircled() :: String.t()
```

Encircled.

faint()

```
@spec faint() :: String.t()
```

Faint (decreased intensity). Not widely supported.

font_1()

```
@spec font_1() :: String.t()
```

Sets alternative font 1.

font_2()

```
@spec font_2() :: String.t()
```

Sets alternative font 2.

font_3()

```
@spec font_3() :: String.t()
```

Sets alternative font 3.

font_4()

```
@spec font_4() :: String.t()
```

Sets alternative font 4.

font_5()

```
@spec font_5() :: String.t()
```

Sets alternative font 5.

font_6()

```
@spec font_6() :: String.t()
```

Sets alternative font 6.

font_7()

```
@spec font_7() :: String.t()
```

Sets alternative font 7.

font_8()

```
@spec font_8() :: String.t()
```

Sets alternative font 8.

font_9()

```
@spec font_9() :: String.t()
```

Sets alternative font 9.

format(ansidata, emit? \\ enabled?())

```
@spec format(ansidata(), boolean()) :: IO.chardata()
```

Formats a chardata-like argument by converting named ANSI sequences into actual ANSI codes.

The named sequences are represented by atoms.

It will also append an `IO.ANSI.reset/0` to the chardata when a conversion is performed. If you don't want this behavior, use `format_fragment/2`.

An optional boolean parameter can be passed to enable or disable emitting actual ANSI codes. When `false`, no ANSI codes will be emitted. By default checks if ANSI is enabled using the `enabled?/0` function.

An `ArgumentError` will be raised if an invalid ANSI code is provided.

Examples

```
iex> IO.ANSI.format(["Hello, ", :red, :bright, "world!"],
true)
[[[[[[], "Hello, "] | "\e[31m"] | "\e[1m"], "world!"] |
"\e[0m"]
```

format_fragment(ansidata, emit? \\ enabled?())

```
@spec format_fragment(ansidata(), boolean()) :: IO.chardata()
```

Formats a chardata-like argument by converting named ANSI sequences into actual ANSI codes.

The named sequences are represented by atoms.

An optional boolean parameter can be passed to enable or disable emitting actual ANSI codes. When `false`, no ANSI codes will be emitted. By default checks if ANSI is enabled using the `enabled?/0` function.

Examples

```
iex> IO.ANSI.format_fragment([:bright, ~c"Word"], true)
[[[[[[] | "\e[1m"], 87], 111], 114], 100]
```

framed()

```
@spec framed() :: String.t()
```

Framed.

green()

```
@spec green() :: String.t()
```

Sets foreground color to green.

green_background()

```
@spec green_background() :: String.t()
```

Sets background color to green.

home()

```
@spec home() :: String.t()
```

Sends cursor home.

inverse()

```
@spec inverse() :: String.t()
```

Image: negative. Swap foreground and background.

inverse_off()

```
@spec inverse_off() :: String.t()
```

Image: positive. Normal foreground and background.

italic()

```
@spec italic() :: String.t()
```

Italic: on. Not widely supported. Sometimes treated as inverse.

light_black()

```
@spec light_black() :: String.t()
```

Sets foreground color to light black.

light_black_background()

```
@spec light_black_background() :: String.t()
```

Sets background color to light black.

light_blue()

```
@spec light_blue() :: String.t()
```

Sets foreground color to light blue.

light_blue_background()

```
@spec light_blue_background() :: String.t()
```

Sets background color to light blue.

light_cyan()

```
@spec light_cyan() :: String.t()
```

Sets foreground color to light cyan.

light_cyan_background()

```
@spec light_cyan_background() :: String.t()
```

Sets background color to light cyan.

light_green()

```
@spec light_green() :: String.t()
```

Sets foreground color to light green.

light_green_background()

```
@spec light_green_background() :: String.t()
```

Sets background color to light green.

light_magenta()

```
@spec light_magenta() :: String.t()
```

Sets foreground color to light magenta.

light_magenta_background()

```
@spec light_magenta_background() :: String.t()
```

Sets background color to light magenta.

light_red()

```
@spec light_red() :: String.t()
```

Sets foreground color to light red.

light_red_background()

```
@spec light_red_background() :: String.t()
```

Sets background color to light red.

light_white()

```
@spec light_white() :: String.t()
```

Sets foreground color to light white.

light_white_background()

```
@spec light_white_background() :: String.t()
```

Sets background color to light white.

light_yellow()

```
@spec light_yellow() :: String.t()
```

Sets foreground color to light yellow.

light_yellow_background()

```
@spec light_yellow_background() :: String.t()
```

Sets background color to light yellow.

magenta()

```
@spec magenta() :: String.t()
```

Sets foreground color to magenta.

magenta_background()

```
@spec magenta_background() :: String.t()
```

Sets background color to magenta.

no_underline()

```
@spec no_underline() :: String.t()
```

Underline: none.

normal()

```
@spec normal() :: String.t()
```

Normal color or intensity.

not_crossed_out()

```
@spec not_crossed_out() :: String.t()
```

Not crossed-out.

not_framed_encircled()

```
@spec not_framed_encircled() :: String.t()
```

Not framed or encircled.

not_italic()

```
@spec not_italic() :: String.t()
```

Not italic.

not_overlined()

```
@spec not_overlined() :: String.t()
```

Not overlined.

overlined()

```
@spec overlined() :: String.t()
```

Overlined.

primary_font()

```
@spec primary_font() :: String.t()
```

Sets primary (default) font.

red()

```
@spec red() :: String.t()
```

Sets foreground color to red.

red_background()

```
@spec red_background() :: String.t()
```

Sets background color to red.

reset()

```
@spec reset() :: String.t()
```

Resets all attributes.

reveal()

```
@spec reveal() :: String.t()
```

Reveal: Not concealed.

reverse()

```
@spec reverse() :: String.t()
```

Image: negative. Swap foreground and background.

reverse_off()

```
@spec reverse_off() :: String.t()
```

Image: positive. Normal foreground and background.

syntax_colors()

(since 1.14.0)

```
@spec syntax_colors() :: Keyword.t(ansidata())
```

Syntax colors to be used by `Inspect`.

Those colors are used throughout Elixir's standard library, such as `dbg/2` and `IEx`.

The colors can be changed by setting the `:ansi_syntax_colors` in the `:elixir` application configuration. Configuration for most built-in data types are supported: `:atom`, `:binary`, `:boolean`, `:charlist`, `:list`, `:map`, `:nil`, `:number`, `:string`, and `:tuple`. The default is:

```
[
  atom: :cyan
  boolean: :magenta,
  charlist: :yellow,
  nil: :magenta,
  number: :yellow,
  string: :green
]
```

underline()

```
@spec underline() :: String.t()
```

Underline: single.

white()

```
@spec white() :: String.t()
```

Sets foreground color to white.

white_background()

```
@spec white_background() :: String.t()
```

Sets background color to white.

yellow()

```
@spec yellow() :: String.t()
```

Sets foreground color to yellow.

yellow_background()

```
@spec yellow_background() :: String.t()
```

Sets background color to yellow.

IO.Stream

Defines an `IO.Stream` struct returned by `IO.stream/2` and `IO.binstream/2`.

The following fields are public:

- `device` - the IO device
- `raw` - a boolean indicating if bin functions should be used
- `line_or_bytes` - if reading should read lines or a given number of bytes

It is worth noting that an IO stream has side effects and every time you go over the stream you may get different results.

Summary

Types

t()

Types

t()

```
@type t() :: %IO.Stream{
  device: IO.device(),
  line_or_bytes: :line | non_neg_integer(),
  raw: boolean()
}
```

OptionParser

Functions for parsing command line arguments.

When calling a command, it's possible to pass command line options to modify what the command does. In this documentation, those are called "switches", in other situations they may be called "flags" or simply "options". A switch can be given a value, also called an "argument".

The main function in this module is `parse/2`, which parses a list of command line options and arguments into a keyword list:

```
iex> OptionParser.parse(["--debug"], strict: [debug:
:boolean])
{[debug: true], [], []}
```

`OptionParser` provides some conveniences out of the box, such as aliases and automatic handling of negation switches.

The `parse_head/2` function is an alternative to `parse/2` which stops parsing as soon as it finds a value that is not a switch nor a value for a previous switch.

This module also provides low-level functions, such as `next/2`, for parsing switches manually, as well as `split/1` and `to_argv/1` for parsing from and converting switches to strings.

Summary

Types

argv()
errors()
options()
parsed()

Functions

next(argv, opts \\ [])

Low-level function that parses one option.

parse(argv, opts \\ [])

Parses `argv` into a keyword list.

parse!(argv, opts \\ [])

The same as `parse/2` but raises an `OptionParser.ParseError` exception if any invalid options are given.

parse_head(argv, opts \\ [])

Similar to `parse/2` but only parses the head of `argv`; as soon as it finds a non-switch, it stops parsing.

parse_head!(argv, opts \\ [])

The same as `parse_head/2` but raises an `OptionParser.ParseError` exception if any invalid options are given.

split(string)

Splits a string into `argv/0` chunks.

to_argv(enum, options \\ [])

Receives a key-value enumerable and converts it to `argv/0`.

Types

argv()

```
@type argv() :: [String.t()]
```

errors()

```
@type errors() :: [{String.t(), String.t() | nil}]
```

options()

```
@type options() :: [
  switches: keyword(),
  strict: keyword(),
  aliases: keyword(),
  allow_nonexistent_atoms: boolean(),
  return_separator: boolean()
]
```

parsed()

```
@type parsed() :: keyword()
```

Functions

next(argv, opts \\ [])

```
@spec next(argv(), options()) ::
  {:ok, key :: atom(), value :: term(), argv()}
  | {:invalid, String.t(), String.t() | nil, argv()}
  | {:undefined, String.t(), String.t() | nil, argv()}
  | {:error, argv()}
```

Low-level function that parses one option.

It accepts the same options as `parse/2` and `parse_head/2` as both functions are built on top of this function. This function may return:

- `{:ok, key, value, rest}` - the option `key` with `value` was successfully parsed
- `{:invalid, key, value, rest}` - the option `key` is invalid with `value` (returned when the value cannot be parsed according to the switch type)
- `{:undefined, key, value, rest}` - the option `key` is undefined (returned in strict mode when the switch is unknown or on nonexistent atoms)
- `{:error, rest}` - there are no switches at the head of the given `argv`

parse(argv, opts \\ [])

```
@spec parse(argv(), options()) :: {parsed(), argv(),
errors()}
```

Parses `argv` into a keyword list.

It returns a three-element tuple with the form `{parsed, args, invalid}`, where:

- `parsed` is a keyword list of parsed switches with `{switch_name, value}` tuples in it; `switch_name` is the atom representing the switch name while `value` is the value for that switch parsed according to `opts` (see the "Examples" section for more information)
- `args` is a list of the remaining arguments in `argv` as strings
- `invalid` is a list of invalid options as `{option_name, value}` where `option_name` is the raw option and `value` is `nil` if the option wasn't expected or the string value if the value didn't have the expected type for the corresponding option

Elixir converts switches to underscored atoms, so `--source-path` becomes `:source_path`. This is done to better suit Elixir conventions. However, this means that switches can't contain underscores and switches that do contain underscores are always returned in the list of invalid switches.

When parsing, it is common to list switches and their expected types:

```
iex> OptionParser.parse(["--debug"], strict: [debug:
:boolean])
{[debug: true], [], []}

iex> OptionParser.parse(["--source", "lib"], strict:
[source: :string])
{[source: "lib"], [], []}
```

```
iex> OptionParser.parse(
...>   ["--source-path", "lib", "test/enum_test.exs", "--
verbose"],
...>   strict: [source_path: :string, verbose: :boolean]
...> )
{[source_path: "lib", verbose: true],
["test/enum_test.exs"], []}
```

We will explore the valid switches and operation modes of option parser below.

Options

The following options are supported:

- `:switches` or `:strict` - see the "Switch definitions" section below
- `:allow_nonexistent_atoms` - see the "Parsing unknown switches" section below
- `:aliases` - see the "Aliases" section below
- `:return_separator` - see the "Return separator" section below

Switch definitions

Switches can be specified via one of two options:

- `:strict` - defines strict switches and their types. Any switch in `argv` that is not specified in the list is returned in the invalid options list. This is the preferred way to parse options.

- `:switches` - defines switches and their types. This function still attempts to parse switches that are not in this list.

Both these options accept a keyword list where the key is an atom defining the name of the switch and value is the `type` of the switch (see the "Types" section below for more information).

Note that you should only supply the `:switches` or the `:strict` option. If you supply both, an `ArgumentError` exception will be raised.

Types

Switches parsed by `OptionParser` may take zero or one arguments.

The following switches types take no arguments:

- `:boolean` - sets the value to `true` when given (see also the "Negation switches" section below)
- `:count` - counts the number of times the switch is given

The following switches take one argument:

- `:integer` - parses the value as an integer
- `:float` - parses the value as a float
- `:string` - parses the value as a string
- `:regex` - parses the value as a regular expression with Unicode support

If a switch can't be parsed according to the given type, it is returned in the invalid options list.

Modifiers

Switches can be specified with modifiers, which change how they behave. The following modifiers are supported:

- `:keep` - keeps duplicate elements instead of overriding them; works with all types except `:count`. Specifying `switch_name:`

`:keep` assumes the type of `:switch_name` will be `:string`.

To use `:keep` with a type other than `:string`, use a list as the type for the switch. For example: `[foo: [:integer, :keep]]`.

Negation switches

In case a switch `SWITCH` is specified to have type `:boolean`, it may be passed as `--no-SWITCH` as well which will set the option to `false`:

```
iex> OptionParser.parse(["--no-op", "path/to/file"],
switches: [op: :boolean])
{[op: false], ["path/to/file"], []}
```

Parsing unknown switches

When the `:switches` option is given, `OptionParser` will attempt to parse unknown switches.

Switches without an argument will be set to `true`:

```
iex> OptionParser.parse(["--debug"], switches: [key:
:string])
{[debug: true], [], []}
```

Even though we haven't specified `--debug` in the list of switches, it is part of the returned options. The same happens for switches followed by another switch:

```
iex> OptionParser.parse(["--debug", "--ok"], switches:
[])
```

```
{[debug: true, ok: true], [], []}
```

Switches followed by a value will be assigned the value, as a string:

```
iex> OptionParser.parse(["--debug", "value"], switches:
[key: :string])
{[debug: "value"], [], []}
```

Since we cannot assert the type of the switch value, it is preferred to use the `:strict` option that accepts only known switches and always verify their types.

If you do want to parse unknown switches, remember that Elixir converts switches to atoms. Since atoms are not garbage-collected, to avoid creating new ones, OptionParser by default only parses switches that translate to existing atoms. The code below discards the `--option-parser-example` switch because the `:option_parser_example` atom is never used anywhere:

```
iex> OptionParser.parse(["--option-parser-example"],
switches: [])
{[], [], []}
```

If a switch corresponds to an existing Elixir atom, whether from your code, a dependency or from Elixir itself, it will be accepted. However, it is best to not rely on external code, and always define the atoms you want to parse in the same module that calls `OptionParser` itself, as direct arguments to the `:switches` or `:strict` options.

If you would like to parse all switches, regardless if they exist or not, you can force creation of atoms by passing `allow_nonexistent_atoms: true` as option. Use this option with care. It is only useful when you are

building command-line applications that receive dynamically-named arguments and must be avoided in long-running systems.

Aliases

A set of aliases can be specified in the `:aliases` option:

```
iex> OptionParser.parse(["-d"], aliases: [d: :debug],
strict: [debug: :boolean])
{[debug: true], [], []}
```

Examples

Here are some examples of working with different types and modifiers:

```
iex> OptionParser.parse(["--unlock", "path/to/file"],
strict: [unlock: :boolean])
{[unlock: true], ["path/to/file"], []}

iex> OptionParser.parse(
...>   ["--unlock", "--limit", "0", "path/to/file"],
...>   strict: [unlock: :boolean, limit: :integer]
...> )
{[unlock: true, limit: 0], ["path/to/file"], []}

iex> OptionParser.parse(["--limit", "3"], strict: [limit:
:integer])
{[limit: 3], [], []}

iex> OptionParser.parse(["--limit", "xyz"], strict:
[limit: :integer])
{[], [], [{"--limit", "xyz"}]}
```

```
iex> OptionParser.parse(["--verbose"], switches:
[verbose: :count])
{[verbose: 1], [], []}

iex> OptionParser.parse(["-v", "-v"], aliases: [v:
:verbose], strict: [verbose: :count])
{[verbose: 2], [], []}

iex> OptionParser.parse(["--unknown", "xyz"], strict: [])
{[], ["xyz"], [{"--unknown", nil}]}

iex> OptionParser.parse(
...>   ["--limit", "3", "--unknown", "xyz"],
...>   switches: [limit: :integer]
...> )
{[limit: 3, unknown: "xyz"], [], []}

iex> OptionParser.parse(
...>   ["--unlock", "path/to/file", "--unlock",
"path/to/another/file"],
...>   strict: [unlock: :keep]
...> )
{[unlock: "path/to/file", unlock:
"path/to/another/file"], [], []}
```

Return separator

The separator `--` implies options should no longer be processed. By default, the separator is not returned as parts of the arguments, but that can be changed via the `:return_separator` option:

```
iex> OptionParser.parse(["--", "lib"], return_separator:
true, strict: [])
```

```
{[], ["--", "lib"], []}

iex> OptionParser.parse(["--no-halt", "--", "lib"],
return_separator: true, switches: [halt: :boolean])
{[halt: false], ["--", "lib"], []}

iex> OptionParser.parse(["script.exs", "--no-halt", "--",
"foo"], return_separator: true, switches: [halt:
:boolean])
{[{:halt, false}], ["script.exs", "--", "foo"], []}
```

parse!(argv, opts \\ [])

```
@spec parse!(argv(), options()) :: {parsed(), argv()}
```

The same as `parse/2` but raises an `OptionParser.ParseError` exception if any invalid options are given.

If there are no errors, returns a `{parsed, rest}` tuple where:

- `parsed` is the list of parsed switches (same as in `parse/2`)
- `rest` is the list of arguments (same as in `parse/2`)

Examples

```
iex> OptionParser.parse!(["--debug", "path/to/file"],
strict: [debug: :boolean])
{[debug: true], ["path/to/file"]}

iex> OptionParser.parse!(["--limit", "xyz"], strict:
[limit: :integer])
** (OptionParser.ParseError) 1 error found!
```

```
--limit : Expected type integer, got "xyz"...

iex> OptionParser.parse!(["--unknown", "xyz"], strict:
[])
** (OptionParser.ParseError) 1 error found!
--unknown : Unknown option...

iex> OptionParser.parse!(
...>   ["-l", "xyz", "-f", "bar"],
...>   switches: [limit: :integer, foo: :integer],
...>   aliases: [l: :limit, f: :foo]
...> )
** (OptionParser.ParseError) 2 errors found!
-l : Expected type integer, got "xyz"
-f : Expected type integer, got "bar"...
```

parse_head(argv, opts \\ [])

```
@spec parse_head(argv(), options()) :: {parsed(), argv(),
errors()}
```

Similar to `parse/2` but only parses the head of `argv`; as soon as it finds a non-switch, it stops parsing.

See `parse/2` for more information.

Example

```
iex> OptionParser.parse_head(
...>   ["--source", "lib", "test/enum_test.exs", "--
verbose"],
...>   switches: [source: :string, verbose: :boolean]
...> )
```

```
{[source: "lib"], ["test/enum_test.exs", "--verbose"],
[]}

iex> OptionParser.parse_head(
...>   ["--verbose", "--source", "lib",
"test/enum_test.exs", "--unlock"],
...>   switches: [source: :string, verbose: :boolean,
unlock: :boolean]
...> )
{[verbose: true, source: "lib"], ["test/enum_test.exs",
"--unlock"], []}
```

parse_head!(argv, opts \\ [])

```
@spec parse_head!(argv(), options()) :: {parsed(), argv()}
```

The same as `parse_head/2` but raises an `OptionParser.ParseError` exception if any invalid options are given.

If there are no errors, returns a `{parsed, rest}` tuple where:

- `parsed` is the list of parsed switches (same as in `parse_head/2`)
- `rest` is the list of arguments (same as in `parse_head/2`)

Examples

```
iex> OptionParser.parse_head!(
...>   ["--source", "lib", "path/to/file", "--verbose"],
...>   switches: [source: :string, verbose: :boolean]
...> )
{[source: "lib"], ["path/to/file", "--verbose"]}
```

```
iex> OptionParser.parse_head!(
...>   ["--number", "lib", "test/enum_test.exs", "--
verbose"],
...>   strict: [number: :integer]
...> )
** (OptionParser.ParseError) 1 error found!
--number : Expected type integer, got "lib"...

iex> OptionParser.parse_head!(
...>   ["--verbose", "--source", "lib",
"test/enum_test.exs", "--unlock"],
...>   strict: [verbose: :integer, source: :integer]
...> )
** (OptionParser.ParseError) 2 errors found!
--verbose : Missing argument of type integer
--source : Expected type integer, got "lib"...
```

split(string)

```
@spec split(String.t()) :: argv()
```

Splits a string into `argv/0` chunks.

This function splits the given `string` into a list of strings in a similar way to many shells.

Examples

```
iex> OptionParser.split("foo bar")
["foo", "bar"]
```

```
iex> OptionParser.split("foo \"bar baz\"")
["foo", "bar baz"]
```

to_argv(enum, options \\ [])

```
@spec to_argv(Enumerable.t(), options()) :: argv()
```

Receives a key-value enumerable and converts it to `argv/0`.

Keys must be atoms. Keys with `nil` value are discarded, boolean values are converted to `--key` or `--no-key` (if the value is `true` or `false`, respectively), and all other values are converted using `to_string/1`.

It is advised to pass to `to_argv/2` the same set of `options` given to `parse/2`. Some switches can only be reconstructed correctly with the `:switches` information in hand.

Examples

```
iex> OptionParser.to_argv(foo_bar: "baz")
["--foo-bar", "baz"]
iex> OptionParser.to_argv(bool: true, bool: false,
discarded: nil)
["--bool", "--no-bool"]
```

Some switches will output different values based on the switches types:

```
iex> OptionParser.to_argv([number: 2], switches: [])
["--number", "2"]
iex> OptionParser.to_argv([number: 2], switches: [number:
```

```
:count])
["--number", "--number"]
```

Path

This module provides conveniences for manipulating or retrieving file system paths.

The functions in this module may receive chardata as arguments and will always return a string encoded in UTF-8. Chardata is a string or a list of characters and strings, see `IO.chardata/0`. If a binary is given, in whatever encoding, its encoding will be kept.

The majority of the functions in this module do not interact with the file system, except for a few functions that require it (like `wildcard/2` and `expand/1`).

Summary

Types

relative_to_opts()
t()

A path.

Functions

absname(path)

Converts the given path to an absolute one.

absname(path, relative_to)

Builds a path from `relative_to` to `path`.

basename(path)

Returns the last component of the path or the path itself if it does not contain any directory separators.

basename(path, extension)

Returns the last component of `path` with the `extension` stripped.

dirname(path)

Returns the directory component of `path`.

expand(path)

Converts the path to an absolute one, expanding any `.` and `..` components and a leading `~`.

expand(path, relative_to)

Expands the path relative to the path given as the second argument expanding any `.` and `..` characters.

extname(path)

Returns the extension of the last component of `path`.

join(list)

Joins a list of paths.

join(left, right)

Joins two paths.

relative(name)

Forces the path to be a relative path.

relative_to(path, cwd, opts \\ [])

Returns the direct relative path from `path` in relation to `cwd`.

relative_to_cwd(path, opts \\ [])

Convenience to get the path relative to the current working directory.

rootname(path)

Returns the `path` with the `extension` stripped.

rootname(path, extension)

Returns the `path` with the `extension` stripped.

safe_relative(path, relative_to \\ File.cwd!())

Returns a relative path that is protected from directory-traversal attacks.

safe_relative_to(path, cwd) deprecated

Returns a relative path that is protected from directory-traversal attacks.

split(path)

Splits the path into a list at the path separator.

type(name)

Returns the path type.

wildcard(glob, opts \\ [])

Traverses paths according to the given `glob` expression and returns a list of matches.

Types

relative_to_opts()

```
@type relative_to_opts() :: [{:force, boolean()}]
```

t()

```
@type t() :: IO.chardata()
```

A path.

Functions

absname(path)

```
@spec absname(t()) :: binary()
```

Converts the given path to an absolute one.

Unlike `expand/1`, no attempt is made to resolve `..`, `.`, or `~`.

Examples

Unix-like operating systems

```
Path.absname("foo")
#=> "/usr/local/foo"
```

```
Path.absname("../x")
#=> "/usr/local/../x"
```

Windows

```
Path.absname("foo")
#=> "D:/usr/local/foo"

Path.absname("../x")
#=> "D:/usr/local/../x"
```

absname(path, relative_to)

```
@spec absname(t(), t() | (-> t())) :: binary()
```

Builds a path from `relative_to` to `path`.

If `path` is already an absolute path, `relative_to` is ignored. See also `relative_to/3`. `relative_to` is either a path or an anonymous function, which is invoked only when necessary, that returns a path.

Unlike `expand/2`, no attempt is made to resolve `..`, `.` or `~`.

Examples

```
iex> Path.absname("foo", "bar")
"bar/foo"

iex> Path.absname("../x", "bar")
"bar/../x"
```

```
iex> Path.absname("foo", fn -> "lazy" end)
"lazy/foo"
```

basename(path)

```
@spec basename(t()) :: binary()
```

Returns the last component of the path or the path itself if it does not contain any directory separators.

Examples

```
iex> Path.basename("foo")
"foo"

iex> Path.basename("foo/bar")
"bar"

iex> Path.basename("lib/module/submodule.ex")
"submodule.ex"

iex> Path.basename("/")
""
```

basename(path, extension)

```
@spec basename(t(), t()) :: binary()
```

Returns the last component of `path` with the `extension` stripped.

This function should be used to remove a specific extension which may or may not be there.

Examples

```
iex> Path.basename("~/foo/bar.ex", ".ex")
"bar"

iex> Path.basename("~/foo/bar.exs", ".ex")
"bar.exs"

iex> Path.basename("~/foo/bar.old.ex", ".ex")
"bar.old"
```

dirname(path)

```
@spec dirname(t()) :: binary()
```

Returns the directory component of `path`.

Examples

```
iex> Path.dirname("/foo/bar.ex")
"/foo"

iex> Path.dirname("/foo/bar/baz.ex")
"/foo/bar"

iex> Path.dirname("/foo/bar/")
"/foo/bar"
```

```
iex> Path.dirname("bar.ex")
"."
```

expand(path)

```
@spec expand(t()) :: binary()
```

Converts the path to an absolute one, expanding any `.` and `..` components and a leading `~`.

If a relative path is provided it is expanded relatively to the current working directory.

Examples

```
Path.expand("/foo/bar/../baz")
#=> "/foo/baz"

Path.expand("foo/bar/../baz")
#=> "$PWD/foo/baz"
```

expand(path, relative_to)

```
@spec expand(t(), t()) :: binary()
```

Expands the path relative to the path given as the second argument expanding any `.` and `..` characters.

If the path is already an absolute path, `relative_to` is ignored.

Note that this function treats a `path` with a leading `~` as an absolute one.

The second argument is first expanded to an absolute path.

Examples

```
# Assuming that the absolute path to baz is /quux/baz
Path.expand("foo/bar/../bar", "baz")
#=> "/quux/baz/foo/bar"

Path.expand("foo/bar/../bar", "/baz")
#=> "/baz/foo/bar"

Path.expand("/foo/bar/../bar", "/baz")
#=> "/foo/bar"
```

extname(path)

```
@spec extname(t()) :: binary()
```

Returns the extension of the last component of `path`.

For filenames starting with a dot and without an extension, it returns an empty string.

See `basename/1` and `rootname/1` for related functions to extract information from paths.

Examples

```
iex> Path.extname("foo.erl")
".erl"

iex> Path.extname("~/foo/bar")
""

iex> Path.extname(".gitignore")
""
```

join(list)

```
@spec join([t(), ...]) :: binary()
```

Joins a list of paths.

This function should be used to convert a list of paths to a path. Note that any trailing slash is removed when joining.

Raises an error if the given list of paths is empty.

Examples

```
iex> Path.join(["~", "foo"])
"~/foo"

iex> Path.join(["foo"])
"foo"

iex> Path.join(["/", "foo", "bar/"])
"/foo/bar"
```

join(left, right)

```
@spec join(t(), t()) :: binary()
```

Joins two paths.

The right path will always be expanded to its relative format and any trailing slash will be removed when joining.

Examples

```
iex> Path.join("foo", "bar")
"foo/bar"

iex> Path.join("/foo", "/bar/")
"/foo/bar"
```

The functions in this module support chardata, so giving a list will treat it as a single entity:

```
iex> Path.join("foo", ["bar", "fiz"])
"foo/barfiz"

iex> Path.join(["foo", "bar"], "fiz")
"foobar/fiz"
```

Use `join/1` if you need to join a list of paths instead.

relative(name)

```
@spec relative(t()) :: binary()
```

Forces the path to be a relative path.

If an absolute path is given, it is stripped from its root component.

Examples

Unix-like operating systems

```
Path.relative("/usr/local/bin")   #=> "usr/local/bin"
Path.relative("usr/local/bin")    #=> "usr/local/bin"
Path.relative("../usr/local/bin") #=> "../usr/local/bin"
```

Windows

```
Path.relative("D:/usr/local/bin") #=> "usr/local/bin"
Path.relative("usr/local/bin")    #=> "usr/local/bin"
Path.relative("D:bar.ex")         #=> "bar.ex"
Path.relative("/bar/foo.ex")      #=> "bar/foo.ex"
```

relative_to(path, cwd, opts \\ [])

```
@spec relative_to(t(), t(), relative_to_opts()) :: binary()
```

Returns the direct relative path from `path` in relation to `cwd`.

In other words, this function attempts to return a path such that `Path.expand(result, cwd)` points to `path`. This function aims to return a relative path whenever possible, but that's not guaranteed:

- If both paths are relative, a relative path is always returned

- If both paths are absolute, a relative path may be returned if they share a common prefix. You can pass the `:force` option to force this function to traverse up, but even then a relative path is not guaranteed (for example, if the absolute paths belong to different drives on Windows)

- If a mixture of paths are given, the result will always match the given `path` (the first argument)

This function expands `.` and `..` entries without traversing the file system, so it assumes no symlinks between the paths. See `safe_relative_to/2` for a safer alternative.

Options

- `:force` - (boolean since v1.16.0) if `true` forces a relative path to be returned by traversing the path up. Except if the paths are in different volumes on Windows. Defaults to `false`.

Examples

With relative `cwd`

If both paths are relative, a minimum path is computed:

```
Path.relative_to("tmp/foo/bar", "tmp")      #=> "foo/bar"
Path.relative_to("tmp/foo/bar", "tmp/foo")  #=> "bar"
Path.relative_to("tmp/foo/bar", "tmp/bat")  #=>
"../foo/bar"
```

If an absolute path is given with relative `cwd`, it is returned as:

```
Path.relative_to("/usr/foo/bar", "tmp/bat")  #=>
"/usr/foo/bar"
```

With absolute `cwd`

If both paths are absolute, a relative is computed if possible, without traversing up:

```
Path.relative_to("/usr/local/foo", "/usr/local")      #=>
"foo"
Path.relative_to("/usr/local/foo", "/")               #=>
"usr/local/foo"
Path.relative_to("/usr/local/foo", "/etc")            #=>
"/usr/local/foo"
Path.relative_to("/usr/local/foo", "/usr/local/foo")  #=>
"."
Path.relative_to("/usr/local/../foo", "/usr/foo")     #=>
"."
Path.relative_to("/usr/local/../foo/bar", "/usr/foo") #=>
"bar"
```

If `:force` is set to `true` paths are traversed up:

```
Path.relative_to("/usr", "/usr/local", force: true)
#=> ".."
Path.relative_to("/usr/foo", "/usr/local", force: true)
#=> "../foo"
Path.relative_to("/usr/../foo/bar", "/etc/foo", force:
true) #=> "../../foo/bar"
```

If a relative path is given, it is assumed to be relative to the given path, so the path is returned with "." and ".." expanded:

```
Path.relative_to(".", "/usr/local")          #=> "."
Path.relative_to("foo", "/usr/local")        #=> "foo"
Path.relative_to("foo/../bar", "/usr/local") #=> "bar"
Path.relative_to("foo/..", "/usr/local")     #=> "."
Path.relative_to("../foo", "/usr/local")     #=> "../foo"
```

relative_to_cwd(path, opts \\ [])

```
@spec relative_to_cwd(t(), relative to opts()) :: binary()
```

Convenience to get the path relative to the current working directory.

If, for some reason, the current working directory cannot be retrieved, this function returns the given `path`.

Check `relative_to/3` for the supported options.

rootname(path)

```
@spec rootname(t()) :: binary()
```

Returns the `path` with the `extension` stripped.

Examples

```
iex> Path.rootname("/foo/bar")
"/foo/bar"
```

```
iex> Path.rootname("/foo/bar.ex")
"/foo/bar"
```

rootname(path, extension)

```
@spec rootname(t(), t()) :: binary()
```

Returns the `path` with the `extension` stripped.

This function should be used to remove a specific extension which may or may not be there.

Examples

```
iex> Path.rootname("/foo/bar.erl", ".erl")
"/foo/bar"

iex> Path.rootname("/foo/bar.erl", ".ex")
"/foo/bar.erl"
```

safe_relative(path, relative_to \\ File.cwd!())

(since 1.14.0)

```
@spec safe_relative(t(), t()) :: {:ok, binary()} | :error
```

Returns a relative path that is protected from directory-traversal attacks.

The given relative path is sanitized by eliminating `..` and `.` components.

This function checks that, after expanding those components, the path is still "safe". Paths are considered unsafe if either of these is true:

- The path is not relative, such as `"/foo/bar"`.
- A `..` component would make it so that the path would traverse up above the root of `relative_to`.
- A symbolic link in the path points to something above the root of `relative_to`.

Examples

```
iex> Path.safe_relative("foo")
{:ok, "foo"}

iex> Path.safe_relative("deps/my_dep/app.beam")
{:ok, "deps/my_dep/app.beam"}

iex>
Path.safe_relative("deps/my_dep/./build/../app.beam",
File.cwd!())
{:ok, "deps/my_dep/app.beam"}

iex> Path.safe_relative("my_dep/../..")
:error

iex> Path.safe_relative("/usr/local", File.cwd!())
:error
```

safe_relative_to(path, cwd)

(since 1.14.0)

This function is deprecated. Use safe_relative/2 instead.

```
@spec safe_relative_to(t(), t()) :: {:ok, binary()} | :error
```

Returns a relative path that is protected from directory-traversal attacks.

See `safe_relative/2` for a non-deprecated version of this API.

split(path)

```
@spec split(t()) :: [binary()]
```

Splits the path into a list at the path separator.

If an empty string is given, returns an empty list.

On Windows, path is split on both `"\"` and `"/"` separators and the driver letter, if there is one, is always returned in lowercase.

Examples

```
iex> Path.split("")
[]

iex> Path.split("foo")
["foo"]

iex> Path.split("/foo/bar")
["/", "foo", "bar"]
```

type(name)

```
@spec type(t()) :: :absolute | :relative | :volumerelative
```

Returns the path type.

Examples

Unix-like operating systems

```
Path.type("/")                #=> :absolute
Path.type("/usr/local/bin")   #=> :absolute
Path.type("usr/local/bin")    #=> :relative
Path.type("../usr/local/bin") #=> :relative
Path.type("~/file")           #=> :relative
```

Windows

```
Path.type("D:/usr/local/bin") #=> :absolute
Path.type("usr/local/bin")    #=> :relative
Path.type("D:bar.ex")         #=> :volumerelative
Path.type("/bar/foo.ex")      #=> :volumerelative
```

wildcard(glob, opts \\ [])

```
@spec wildcard(t(), [{:match_dot, boolean()}]) :: [binary()]
```

Traverses paths according to the given `glob` expression and returns a list of matches.

The wildcard looks like an ordinary path, except that the following "wildcard characters" are interpreted in a special way:

- `?` - matches one character.
- `*` - matches any number of characters up to the end of the filename, the next dot, or the next slash.
- `**` - two adjacent `*`'s used as a single pattern will match all files and zero or more directories and subdirectories.
- `[char1,char2,...]` - matches any of the characters listed; two characters separated by a hyphen will match a range of characters. Do not add spaces before and after the comma as it would then match paths containing the space character itself.
- `{item1,item2,...}` - matches one of the alternatives. Do not add spaces before and after the comma as it would then match paths containing the space character itself.

Other characters represent themselves. Only paths that have exactly the same character in the same position will match. Note that matching is case-sensitive: `"a"` will not match `"A"`.

Directory separators must always be written as `/`, even on Windows. You may call `Path.expand/1` to normalize the path before invoking this function.

A character preceded by `\\` loses its special meaning. Note that `\\` must be written as `\\\\` in a string literal. For example, `"\\\\?*"` will match any filename starting with `?.`.

By default, the patterns `*` and `?` do not match files starting with a dot `.`. See the `:match_dot` option in the "Options" section below.

Options

- `:match_dot` - (boolean) if `false`, the special wildcard characters `*` and `?` will not match files starting with a dot (`.`). If `true`, files starting with a `.` will not be treated specially. Defaults to `false`.

Examples

Imagine you have a directory called `projects` with three Elixir projects inside of it: `elixir`, `ex_doc`, and `plug`. You can find all `.beam` files inside the `ebin` directory of each project as follows:

```
Path.wildcard("projects/*/ebin/**/*.beam")
```

If you want to search for both `.beam` and `.app` files, you could do:

```
Path.wildcard("projects/*/ebin/**/*.{beam,app}")
```

Port

Functions for interacting with the external world through ports.

Ports provide a mechanism to start operating system processes external to the Erlang VM and communicate with them via message passing.

Example

```
iex> port = Port.open({:spawn, "cat"}, [:binary])
iex> send(port, {self(), {:command, "hello"}})
iex> send(port, {self(), {:command, "world"}})
iex> flush()
{#Port<0.1444>, {:data, "hello"}}
{#Port<0.1444>, {:data, "world"}}
iex> send(port, {self(), :close})
:ok
iex> flush()
{#Port<0.1444>, :closed}
:ok
```

In the example above, we have created a new port that executes the program `cat`. `cat` is a program available on Unix-like operating systems that receives data from multiple inputs and concatenates them in the output.

After the port was created, we sent it two commands in the form of messages using `send/2`. The first command has the binary payload of "hello" and the second has "world".

After sending those two messages, we invoked the IEx helper `flush()`, which printed all messages received from the port, in this case we got

"hello" and "world" back. Note that the messages are in binary because we passed the `:binary` option when opening the port in `Port.open/2`. Without such option, it would have yielded a list of bytes.

Once everything was done, we closed the port.

Elixir provides many conveniences for working with ports and some drawbacks. We will explore those below.

Message and function APIs

There are two APIs for working with ports. It can be either asynchronous via message passing, as in the example above, or by calling the functions on this module.

The messages supported by ports and their counterpart function APIs are listed below:

- `{pid, {:command, binary}}` - sends the given data to the port. See `command/3`.

- `{pid, :close}` - closes the port. Unless the port is already closed, the port will reply with `{port, :closed}` message once it has flushed its buffers and effectively closed. See `close/1`.

- `{pid, {:connect, new_pid}}` - sets the `new_pid` as the new owner of the port. Once a port is opened, the port is linked and connected to the caller process and communication to the port only happens through the connected process. This message makes `new_pid` the new connected processes. Unless the port is dead, the port will reply to the old owner with `{port, :connected}`. See `connect/2`.

On its turn, the port will send the connected process the following messages:

- `{port, {:data, data}}` - data sent by the port
- `{port, :closed}` - reply to the `{pid, :close}` message
- `{port, :connected}` - reply to the `{pid, {:connect, new_pid}}` message
- `{:EXIT, port, reason}` - exit signals in case the port crashes. If reason is not `:normal`, this message will only be received if the owner process is trapping exits

Open mechanisms

The port can be opened through four main mechanisms.

As a short summary, prefer to use the `:spawn` and `:spawn_executable` options mentioned below. The other two options, `:spawn_driver` and `:fd` are for advanced usage within the VM. Also consider using `System.cmd/3` if all you want is to execute a program and retrieve its return value.

> **Windows argument splitting and untrusted arguments**
>
> On Unix systems, arguments are passed to a new operating system process as an array of strings but on Windows it is up to the child process to parse them and some Windows programs may apply their own rules, which are inconsistent with the standard C runtime `argv` parsing
>
> This is particularly troublesome when invoking `.bat` or `.com` files as these run implicitly through `cmd.exe`, whose argument parsing is vulnerable to malicious input and can be used to run arbitrary shell commands.
>
> Therefore, if you are running on Windows and you execute batch files or `.com` applications, you must not pass untrusted input as arguments to the program. You may avoid accidentally executing them by explicitly passing the extension of the program you want to run, such as `.exe`, and double check the program is indeed not a batch file or `.com` application.
>
> This affects both `spawn` and `spawn_executable`.

spawn

The `:spawn` tuple receives a binary that is going to be executed as a full invocation. For example, we can use it to invoke "echo hello" directly:

```
iex> port = Port.open({:spawn, "echo hello"}, [:binary])
iex> flush()
{#Port<0.1444>, {:data, "hello\n"}}
```

`:spawn` will retrieve the program name from the argument and traverse your operating system `$PATH` environment variable looking for a matching program.

Although the above is handy, it means it is impossible to invoke an executable that has whitespaces on its name or in any of its arguments.

For those reasons, most times it is preferable to execute `:spawn_executable`.

spawn_executable

Spawn executable is a more restricted and explicit version of spawn. It expects full file paths to the executable you want to execute. If they are in your `$PATH`, they can be retrieved by calling `System.find_executable/1`:

```
iex> path = System.find_executable("echo")
iex> port = Port.open({:spawn_executable, path},
[:binary, args: ["hello world"]])
iex> flush()
{#Port<0.1380>, {:data, "hello world\n"}}
```

When using `:spawn_executable`, the list of arguments can be passed via the `:args` option as done above. For the full list of options, see the documentation for the Erlang function `:erlang.open_port/2`.

fd

The `:fd` name option allows developers to access `in` and `out` file descriptors used by the Erlang VM. You would use those only if you are reimplementing core part of the Runtime System, such as the `:user` and `:shell` processes.

Orphan operating system processes

A port can be closed via the `close/1` function or by sending a `{pid, :close}` message. However, if the VM crashes, a long-running program started by the port will have its stdin and stdout channels closed but **it won't be automatically terminated**.

While some Unix command line tools will exit once its parent process terminates, not all command line applications will do so. You can easily check this by starting the port and then shutting down the VM and inspecting your operating system to see if the port process is still running.

We do not always have control over how third-party software terminates. If necessary, one workaround is to wrap the child application in a script that checks whether stdin has been closed. Here is such a script that has been verified to work on bash shells:

```
#!/usr/bin/env bash

# Start the program in the background
exec "$@" &
pid1=$!

# Silence warnings from here on
exec >/dev/null 2>&1

# Read from stdin in the background and
# kill running program when stdin closes
exec 0<&0 $(
  while read; do :; done
  kill -KILL $pid1
) &
pid2=$!

# Clean up
wait $pid1
ret=$?
kill -KILL $pid2
exit $ret
```

Note the program above hijacks stdin, so you won't be able to communicate with the underlying software via stdin (on the positive side, software that reads from stdin typically terminates when stdin closes).

Now instead of:

```
Port.open(
  {:spawn_executable, "/path/to/program"},
  args: ["a", "b", "c"]
)
```

You may invoke:

```
Port.open(
  {:spawn_executable, "/path/to/wrapper"},
  args: ["/path/to/program", "a", "b", "c"]
)
```

Summary

Types

name()

Functions

close(port)

Closes the `port`.

command(port, data, options \\ [])

Sends `data` to the port driver `port`.

connect(port, pid)

Associates the `port` identifier with a `pid`.

demonitor(monitor_ref, options \\ [])

Demonitors the monitor identified by the given `reference`.

info(port)

Returns information about the `port` (or `nil` if the port is closed).

info(port, spec)

Returns information about a specific field within the `port` (or `nil` if the port is closed).

list()

Returns a list of all ports in the current node.

monitor(port)

Starts monitoring the given `port` from the calling process.

open(name, options)

Opens a port given a tuple `name` and a list of `options`.

Types

name()

```
@type name() ::
  {:spawn, charlist() | binary()}
  | {:spawn_driver, charlist() | binary()}
  | {:spawn_executable, :file.name_all()}
  | {:fd, non_neg_integer(), non_neg_integer()}
```

Functions

close(port)

```
@spec close(port()) :: true
```

Closes the `port`.

For more information, see `:erlang.port_close/1`.

Inlined by the compiler.

command(port, data, options \\ [])

```
@spec command(port(), iodata(), [:force | :nosuspend]) ::
boolean()
```

Sends `data` to the port driver `port`.

For more information, see `:erlang.port_command/3`.

Inlined by the compiler.

connect(port, pid)

```
@spec connect(port(), pid()) :: true
```

Associates the `port` identifier with a `pid`.

For more information, see `:erlang.port_connect/2`.

Inlined by the compiler.

demonitor(monitor_ref, options \\ [])

(since 1.6.0)

```
@spec demonitor(reference(), options :: [:flush | :info]) ::
boolean()
```

Demonitors the monitor identified by the given `reference`.

If `monitor_ref` is a reference which the calling process obtained by calling `monitor/1`, that monitoring is turned off. If the monitoring is already turned off, nothing happens.

See `:erlang.demonitor/2` for more information.

Inlined by the compiler.

info(port)

```
@spec info(port()) :: keyword() | nil
```

Returns information about the `port` (or `nil` if the port is closed).

For more information, see `:erlang.port_info/1`.

info(port, spec)

```
@spec info(port(), atom()) :: {atom(), term()} | nil
```

Returns information about a specific field within the `port` (or `nil` if the port is closed).

For more information, see `:erlang.port_info/2`.

list()

```
@spec list() :: [port()]
```

Returns a list of all ports in the current node.

Inlined by the compiler.

monitor(port)

(since 1.6.0)

```
@spec monitor(port() | {name, node()} | name) :: reference()
when name: atom()
```

Starts monitoring the given `port` from the calling process.

Once the monitored port process dies, a message is delivered to the monitoring process in the shape of:

```
{:DOWN, ref, :port, object, reason}
```

where:

- `ref` is a monitor reference returned by this function;
- `object` is either the `port` being monitored (when monitoring by port ID) or `{name, node}` (when monitoring by a port name);
- `reason` is the exit reason.

See `:erlang.monitor/2` for more information.

Inlined by the compiler.

open(name, options)

```
@spec open(name(), list()) :: port()
```

Opens a port given a tuple `name` and a list of `options`.

The module documentation above contains documentation and examples for the supported `name` values, summarized below:

- `{:spawn, command}` - runs an external program. `command` must contain the program name and optionally a list of arguments separated by space. If passing programs or arguments with space in their name, use the next option.
- `{:spawn_executable, filename}` - runs the executable given by the absolute file name `filename`. Arguments can be passed via the `:args` option.
- `{:spawn_driver, command}` - spawns so-called port drivers.
- `{:fd, fd_in, fd_out}` - accesses file descriptors, `fd_in` and `fd_out` opened by the VM.

For more information and the list of options, see `:erlang.open_port/2`.

Inlined by the compiler.

StringIO

Controls an IO device process that wraps a string.

A `StringIO` IO device can be passed as a "device" to most of the functions in the `IO` module.

Examples

```
iex> {:ok, pid} = StringIO.open("foo")
iex> IO.read(pid, 2)
"fo"
```

Summary

Types

open_opts()

Functions

close(pid)

Stops the IO device and returns the remaining input/output buffers.

contents(pid)

Returns the current input/output buffers for the given IO device.

flush(pid)

Flushes the output buffer and returns its current contents.
open(string, options_or_function \\ [])

Creates an IO device.

open(string, options, function)

Creates an IO device.

Types

open_opts()

```
@type open_opts() :: [capture_prompt: boolean(), encoding:
:unicode | :latin1]
```

Functions

close(pid)

```
@spec close(pid()) :: {:ok, {binary(), binary()}}
```

Stops the IO device and returns the remaining input/output buffers.

Examples

```
iex> {:ok, pid} = StringIO.open("in")
iex> IO.write(pid, "out")
iex> StringIO.close(pid)
{:ok, {"in", "out"}}
```

contents(pid)

```
@spec contents(pid()) :: {binary(), binary()}
```

Returns the current input/output buffers for the given IO device.

Examples

```
iex> {:ok, pid} = StringIO.open("in")
iex> IO.write(pid, "out")
iex> StringIO.contents(pid)
{"in", "out"}
```

flush(pid)

```
@spec flush(pid()) :: binary()
```

Flushes the output buffer and returns its current contents.

Examples

```
iex> {:ok, pid} = StringIO.open("in")
iex> IO.write(pid, "out")
iex> StringIO.flush(pid)
"out"
iex> StringIO.contents(pid)
{"in", ""}
```

open(string, options_or_function \\ [])

```
@spec open(binary(), open_opts()) :: {:ok, pid()}
@spec open(binary(), (pid() -> res)) :: {:ok, res} when res:
var
```

Creates an IO device.

`string` will be the initial input of the newly created device.

`options_or_function` can be a keyword list of options or a function.

If options are provided, the result will be `{:ok, pid}`, returning the IO device created. The option `:capture_prompt`, when set to `true`, causes prompts (which are specified as arguments to `IO.get*` functions) to be included in the device's output. See `options/3` for the list of supported options.

If a function is provided, the device will be created and sent to the function. When the function returns, the device will be closed. The final result will be a tuple with `:ok` and the result of the function.

Examples

```
iex> {:ok, pid} = StringIO.open("foo")
iex> IO.gets(pid, ">")
"foo"
iex> StringIO.contents(pid)
{"", ""}

iex> {:ok, pid} = StringIO.open("foo", capture_prompt:
true)
iex> IO.gets(pid, ">")
```

```
"foo"
iex> StringIO.contents(pid)
{"", ">"}

iex> StringIO.open("foo", fn pid ->
...>   input = IO.gets(pid, ">")
...>   IO.write(pid, "The input was #{input}")
...>   StringIO.contents(pid)
...> end)
{:ok, {"", "The input was foo"}}
```

open(string, options, function)

(since 1.7.0)

```
@spec open(binary(), open_opts(), (pid() -> res)) :: {:ok, res} when res: var
```

Creates an IO device.

`string` will be the initial input of the newly created device.

The device will be created and sent to the function given. When the function returns, the device will be closed. The final result will be a tuple with `:ok` and the result of the function.

Options

- `:capture_prompt` - if set to `true`, prompts (specified as arguments to `IO.get*` functions) are captured in the output. Defaults to `false`.
- `:encoding` (since v1.10.0) - encoding of the IO device. Allowed values are `:unicode` (default) and `:latin1`.

Examples

```
iex> StringIO.open("foo", [], fn pid ->
...>   input = IO.gets(pid, ">")
...>   IO.write(pid, "The input was #{input}")
...>   StringIO.contents(pid)
...> end)
{:ok, {"", "The input was foo"}}

iex> StringIO.open("foo", [capture_prompt: true], fn pid
->
...>   input = IO.gets(pid, ">")
...>   IO.write(pid, "The input was #{input}")
...>   StringIO.contents(pid)
...> end)
{:ok, {"", ">The input was foo"}}
```

System

The `System` module provides functions that interact directly with the VM or the host system.

Time

The `System` module also provides functions that work with time, returning different times kept by the system with support for different time units.

One of the complexities in relying on system times is that they may be adjusted. For example, when you enter and leave daylight saving time, the system clock will be adjusted, often adding or removing one hour. We call such changes "time warps". In order to understand how such changes may be harmful, imagine the following code:

```
## DO NOT DO THIS
prev = System.os_time()
# ... execute some code ...
next = System.os_time()
diff = next - prev
```

If, while the code is executing, the system clock changes, some code that executed in 1 second may be reported as taking over 1 hour! To address such concerns, the VM provides a monotonic time via `System.monotonic_time/0` which never decreases and does not leap:

```
## DO THIS
prev = System.monotonic_time()
# ... execute some code ...
```

```
next = System.monotonic_time()
diff = next - prev
```

Generally speaking, the VM provides three time measurements:

- `os_time/0` - the time reported by the operating system (OS). This time may be adjusted forwards or backwards in time with no limitation;
- `system_time/0` - the VM view of the `os_time/0`. The system time and operating system time may not match in case of time warps although the VM works towards aligning them. This time is not monotonic (i.e., it may decrease) as its behavior is configured by the VM time warp mode;
- `monotonic_time/0` - a monotonically increasing time provided by the Erlang VM. This is not strictly monotonically increasing. Multiple sequential calls of the function may return the same value.

The time functions in this module work in the `:native` unit (unless specified otherwise), which is operating system dependent. Most of the time, all calculations are done in the `:native` unit, to avoid loss of precision, with `convert_time_unit/3` being invoked at the end to convert to a specific time unit like `:millisecond` or `:microsecond`. See the `time_unit/0` type for more information.

For a more complete rundown on the VM support for different times, see the chapter on time and time correction in the Erlang docs.

Summary

Types

cmd_opts()
shell_opts()
signal()
time_unit()

The time unit to be passed to functions like `monotonic_time/1` and others.

Functions

argv()

Lists command line arguments.

argv(args)

Modifies command line arguments.

at_exit(fun)

Registers a program exit handler function.

build_info()

Elixir build information.

cmd(command, args, opts \\ [])

Executes the given `command` with `args`.

compiled_endianness()

Returns the endianness the system was compiled with.

convert_time_unit(time, from_unit, to_unit)

Converts `time` from time unit `from_unit` to time unit `to_unit`.

cwd() deprecated

Current working directory.
cwd!() deprecated

Current working directory, exception on error.

delete_env(varname)

Deletes an environment variable.

endianness()

Returns the endianness.

fetch_env(varname)

Returns the value of the given environment variable or `:error` if not found.

fetch_env!(varname)

Returns the value of the given environment variable or raises if not found.

find_executable(program)

Locates an executable on the system.

get_env()

Returns all system environment variables.

get_env(varname, default \\ nil)

Returns the value of the given environment variable.

get_pid() deprecated

Erlang VM process identifier.

halt(status \\ 0)

Immediately halts the Erlang runtime system.

monotonic_time()

Returns the current monotonic time in the `:native` time unit.

monotonic_time(unit)

Returns the current monotonic time in the given time unit.

no_halt()

Checks if the system will halt or not at the end of ARGV processing.

no_halt(boolean)

Marks if the system should halt or not at the end of ARGV processing.

os_time()

Returns the current operating system (OS) time.

os_time(unit)

Returns the current operating system (OS) time in the given time `unit`.

otp_release()

Returns the Erlang/OTP release number.

pid()

Returns the operating system PID for the current Erlang runtime system instance.

put_env(enum)

Sets multiple environment variables.

put_env(varname, value)

Sets an environment variable value.

restart()

Restarts all applications in the Erlang runtime system.

schedulers()

Returns the number of schedulers in the VM.

schedulers_online()

Returns the number of schedulers online in the VM.

shell(command, opts \\ [])

Executes the given `command` in the OS shell.

stacktrace() deprecated

Deprecated mechanism to retrieve the last exception stacktrace.

stop(status \\ 0)

Asynchronously and carefully stops the Erlang runtime system.

system_time()

Returns the current system time in the `:native` time unit.

system_time(unit)

Returns the current system time in the given time unit.

time_offset()

Returns the current time offset between the Erlang VM monotonic time and the Erlang VM system time.

time_offset(unit)

Returns the current time offset between the Erlang VM monotonic time and the Erlang VM system time.

tmp_dir()

Writable temporary directory.

tmp_dir!()

Writable temporary directory, exception on error.

trap_signal(signal, id \\ make_ref(), fun)

Traps the given `signal` to execute the `fun`.

unique_integer(modifiers \\ [])

Generates and returns an integer that is unique in the current runtime instance.

untrap_signal(signal, id)

Removes a previously registered `signal` with `id`.

user_home()

User home directory.

user_home!()

User home directory, exception on error.

version()

Elixir version information.

Types

cmd_opts()

```
@type cmd_opts() :: [
  into: Collectable.t(),
  lines: pos_integer(),
  cd: Path.t(),
  env: [{binary(), binary() | nil}],
  arg0: binary(),
  stderr_to_stdout: boolean(),
  use_stdio: boolean(),
  parallelism: boolean()
]
```

shell_opts()

```
@type shell_opts() :: [
  into: Collectable.t(),
  lines: pos_integer(),
  cd: Path.t(),
  env: [{binary(), binary() | nil}],
  stderr_to_stdout: boolean(),
  use_stdio: boolean(),
  parallelism: boolean(),
  close_stdin: boolean()
]
```

signal()

```
@type signal() ::
  :sigabrt
  | :sigalrm
  | :sigchld
  | :sighup
  | :sigquit
  | :sigstop
  | :sigterm
  | :sigtstp
  | :sigusr1
  | :sigusr2
```

time_unit()

```
@type time_unit() ::
  :second | :millisecond | :microsecond | :nanosecond |
pos_integer()
```

The time unit to be passed to functions like `monotonic_time/1` and others.

The `:second`, `:millisecond`, `:microsecond` and `:nanosecond` time units controls the return value of the functions that accept a time unit.

A time unit can also be a strictly positive integer. In this case, it represents the "parts per second": the time will be returned in `1 / parts_per_second` seconds. For example, using the `:millisecond` time unit is equivalent to using `1000` as the time unit (as the time will be returned in 1/1000 seconds - milliseconds).

Functions

argv()

```
@spec argv() :: [String.t()]
```

Lists command line arguments.

Returns the list of command line arguments passed to the program.

argv(args)

```
@spec argv([String.t()]) :: :ok
```

Modifies command line arguments.

Changes the list of command line arguments. Use it with caution, as it destroys any previous argv information.

at_exit(fun)

```
@spec at_exit((non_neg_integer() -> any())) :: :ok
```

Registers a program exit handler function.

Registers a function that will be invoked at the end of an Elixir script. A script is typically started via the command line via the `elixir` and `mix` executables.

The handler always executes in a different process from the one it was registered in. As a consequence, any resources managed by the calling process (ETS tables, open files, and others) won't be available by the time the handler function is invoked.

The function must receive the exit status code as an argument.

If the VM terminates programmatically, via `System.stop/1`, `System.halt/1`, or exit signals, the `at_exit/1` callbacks are not guaranteed to be executed.

build_info()

```
@spec build_info() :: %{
  build: String.t(),
  date: String.t(),
  revision: String.t(),
  version: String.t(),
  otp_release: String.t()
}
```

Elixir build information.

Returns a map with the Elixir version, the Erlang/OTP release it was compiled with, a short Git revision hash and the date and time it was built.

Every value in the map is a string, and these are:

- `:build` - the Elixir version, short Git revision hash and Erlang/OTP release it was compiled with
- `:date` - a string representation of the ISO8601 date and time it was built
- `:otp_release` - OTP release it was compiled with

- `:revision` - short Git revision hash. If Git was not available at building time, it is set to `""`
- `:version` - the Elixir version

One should not rely on the specific formats returned by each of those fields. Instead one should use specialized functions, such as `version/0` to retrieve the Elixir version and `otp_release/0` to retrieve the Erlang/OTP release.

Examples

```
iex> System.build_info()
%{
  build: "1.9.0-dev (772a00a0c) (compiled with Erlang/OTP
21)",
  date: "2018-12-24T01:09:21Z",
  otp_release: "21",
  revision: "772a00a0c",
  version: "1.9.0-dev"
}
```

cmd(command, args, opts \\ [])

```
@spec cmd(binary(), [binary()], cmd_opts()) ::
  {Collectable.t(), exit_status :: non_neg_integer()}
```

Executes the given `command` with `args`.

`command` is expected to be an executable available in PATH unless an absolute path is given.

`args` must be a list of binaries which the executable will receive as its arguments as is. This means that:

- environment variables will not be interpolated
- wildcard expansion will not happen (unless `Path.wildcard/2` is used explicitly)
- arguments do not need to be escaped or quoted for shell safety

This function returns a tuple containing the collected result and the command exit status.

Internally, this function uses a `Port` for interacting with the outside world. However, if you plan to run a long-running program, ports guarantee stdin/stdout devices will be closed but it does not automatically terminate the program. The documentation for the `Port` module describes this problem and possible solutions under the "Orphan operating system processes" section.

> **Windows argument splitting and untrusted arguments**
>
> On Unix systems, arguments are passed to a new operating system process as an array of strings but on Windows it is up to the child process to parse them and some Windows programs may apply their own rules, which are inconsistent with the standard C runtime `argv` parsing
>
> This is particularly troublesome when invoking `.bat` or `.com` files as these run implicitly through `cmd.exe`, whose argument parsing is vulnerable to malicious input and can be used to run arbitrary shell commands.
>
> Therefore, if you are running on Windows and you execute batch files or `.com` applications, you must not pass untrusted input as arguments to the program. You may avoid accidentally executing them by explicitly passing the extension of the program you want to run, such as `.exe`, and double check the program is indeed not a batch file or `.com` application.

Options

- `:into` - injects the result into the given collectable, defaults to `""`

- `:lines` - (since v1.15.0) reads the output by lines instead of in bytes. It expects a number of maximum bytes to buffer internally (1024 is a reasonable default). The collectable will be called with each finished line (regardless of buffer size) and without the EOL character

- `:cd` - the directory to run the command in

- `:env` - an enumerable of tuples containing environment key-value as binary. The child process inherits all environment variables from its parent process, the Elixir application, except those overwritten or cleared using this option. Specify a value of `nil` to clear (unset) an environment variable, which is useful for preventing credentials passed to the application from leaking into child processes

- `:arg0` - sets the command arg0

- `:stderr_to_stdout` - redirects stderr to stdout when `true`, no effect if `use_stdio` is `false`.

- `:use_stdio` - `true` by default, setting it to false allows direct interaction with the terminal from the callee

- `:parallelism` - when `true`, the VM will schedule port tasks to improve parallelism in the system. If set to `false`, the VM will try to perform commands immediately, improving latency at the expense of parallelism. The default is `false`, and can be set on system startup by passing the `+spp` flag to `--erl`. Use `:erlang.system_info(:port_parallelism)` to check if enabled.

Error reasons

If invalid arguments are given, `ArgumentError` is raised by `System.cmd/3`. `System.cmd/3` also expects a strict set of options and

will raise if unknown or invalid options are given.

Furthermore, `System.cmd/3` may fail with one of the POSIX reasons detailed below:

- `:system_limit` - all available ports in the Erlang emulator are in use
- `:enomem` - there was not enough memory to create the port
- `:eagain` - there are no more available operating system processes
- `:enametoolong` - the external command given was too long
- `:emfile` - there are no more available file descriptors (for the operating system process that the Erlang emulator runs in)
- `:enfile` - the file table is full (for the entire operating system)
- `:eacces` - the command does not point to an executable file
- `:enoent` - the command does not point to an existing file

Shell commands

If you desire to execute a trusted command inside a shell, with pipes, redirecting and so on, please check `shell/2`.

Examples

```
iex> System.cmd("echo", ["hello"])
{"hello\n", 0}

iex> System.cmd("echo", ["hello"], env: [{"MIX_ENV",
```

```
"test"}])
{"hello\n", 0}
```

If you want to stream the output to Standard IO as it arrives:

```
iex> System.cmd("echo", ["hello"], into: IO.stream())
hello
{%IO.Stream{}, 0}
```

If you want to read lines:

```
iex> System.cmd("echo", ["hello\nworld"], into: [],
lines: 1024)
{["hello", "world"], 0}
```

compiled_endianness()

```
@spec compiled_endianness() :: :little | :big
```

Returns the endianness the system was compiled with.

convert_time_unit(time, from_unit, to_unit)

```
@spec convert_time_unit(integer(), time_unit() | :native,
time_unit() | :native) ::
  integer()
```

Converts `time` from time unit `from_unit` to time unit `to_unit`.

The result is rounded via the floor function.

`convert_time_unit/3` accepts an additional time unit (other than the ones in the `time_unit/0` type) called `:native`. `:native` is the time unit used by the Erlang runtime system. It's determined when the runtime starts and stays the same until the runtime is stopped, but could differ the next time the runtime is started on the same machine. For this reason, you should use this function to convert `:native` time units to a predictable unit before you display them to humans.

To determine how many seconds the `:native` unit represents in your current runtime, you can call this function to convert 1 second to the `:native` time unit: `System.convert_time_unit(1, :second, :native)`.

cwd()

This function is deprecated. Use File.cwd/0 instead.

```
@spec cwd() :: String.t() | nil
```

Current working directory.

Returns the current working directory or `nil` if one is not available.

cwd!()

This function is deprecated. Use File.cwd!/0 instead.

```
@spec cwd!() :: String.t()
```

Current working directory, exception on error.

Returns the current working directory or raises `RuntimeError`.

delete_env(varname)

```
@spec delete_env(String.t()) :: :ok
```

Deletes an environment variable.

Removes the variable `varname` from the environment.

endianness()

```
@spec endianness() :: :little | :big
```

Returns the endianness.

fetch_env(varname)

(since 1.9.0)

```
@spec fetch_env(String.t()) :: {:ok, String.t()} | :error
```

Returns the value of the given environment variable or `:error` if not found.

If the environment variable `varname` is set, then `{:ok, value}` is returned where `value` is a string. If `varname` is not set, `:error` is

returned.

Examples

```
iex> System.fetch_env("PORT")
{:ok, "4000"}

iex> System.fetch_env("NOT_SET")
:error
```

fetch_env!(varname)

(since 1.9.0)

```
@spec fetch_env!(String.t()) :: String.t()
```

Returns the value of the given environment variable or raises if not found.

Same as `get_env/1` but raises instead of returning `nil` when the variable is not set.

Examples

```
iex> System.fetch_env!("PORT")
"4000"

iex> System.fetch_env!("NOT_SET")
** (System.EnvError) could not fetch environment variable
"NOT_SET" because it is not set
```

find_executable(program)

```
@spec find_executable(binary()) :: binary() | nil
```

Locates an executable on the system.

This function looks up an executable program given its name using the environment variable PATH on Windows and Unix-like operating systems. It also considers the proper executable extension for each operating system, so for Windows it will try to lookup files with `.com`, `.cmd` or similar extensions.

get_env()

```
@spec get_env() :: %{optional(String.t()) => String.t()}
```

Returns all system environment variables.

The returned value is a map containing name-value pairs. Variable names and their values are strings.

get_env(varname, default \\ nil)

(since 1.9.0)

```
@spec get_env(String.t(), String.t()) :: String.t()
@spec get_env(String.t(), nil) :: String.t() | nil
```

Returns the value of the given environment variable.

The returned value of the environment variable `varname` is a string. If the environment variable is not set, returns the string specified in `default` or `nil` if none is specified.

Examples

```
iex> System.get_env("PORT")
"4000"

iex> System.get_env("NOT_SET")
nil

iex> System.get_env("NOT_SET", "4001")
"4001"
```

get_pid()

This function is deprecated. Use System.pid/0 instead.

```
@spec get_pid() :: binary()
```

Erlang VM process identifier.

Returns the process identifier of the current Erlang emulator in the format most commonly used by the operating system environment.

For more information, see `:os.getpid/0`.

halt(status \\ 0)

```
@spec halt(non_neg_integer() | binary() | :abort) ::
no_return()
```

Immediately halts the Erlang runtime system.

Terminates the Erlang runtime system without properly shutting down applications and ports. Please see `stop/1` for a careful shutdown of the system.

`status` must be a non-negative integer, the atom `:abort` or a binary.

- If an integer, the runtime system exits with the integer value which is returned to the operating system.
- If `:abort`, the runtime system aborts producing a core dump, if that is enabled in the operating system.
- If a string, an Erlang crash dump is produced with status as slogan, and then the runtime system exits with status code 1.

Note that on many platforms, only the status codes 0-255 are supported by the operating system.

For more information, see `:erlang.halt/1`.

Examples

```
System.halt(0)
System.halt(1)
System.halt(:abort)
```

monotonic_time()

```
@spec monotonic_time() :: integer()
```

Returns the current monotonic time in the `:native` time unit.

This time is monotonically increasing and starts in an unspecified point in time. This is not strictly monotonically increasing. Multiple sequential calls of the function may return the same value.

Inlined by the compiler.

monotonic_time(unit)

```
@spec monotonic_time(time_unit() | :native) :: integer()
```

Returns the current monotonic time in the given time unit.

This time is monotonically increasing and starts in an unspecified point in time.

no_halt()

(since 1.9.0)

```
@spec no_halt() :: boolean()
```

Checks if the system will halt or not at the end of ARGV processing.

no_halt(boolean)

(since 1.9.0)

```
@spec no_halt(boolean()) :: :ok
```

Marks if the system should halt or not at the end of ARGV processing.

os_time()

(since 1.3.0)

```
@spec os_time() :: integer()
```

Returns the current operating system (OS) time.

The result is returned in the `:native` time unit.

This time may be adjusted forwards or backwards in time with no limitation and is not monotonic.

Inlined by the compiler.

os_time(unit)

(since 1.3.0)

```
@spec os_time(time_unit() | :native) :: integer()
```

Returns the current operating system (OS) time in the given time `unit`.

This time may be adjusted forwards or backwards in time with no limitation and is not monotonic.

otp_release()

(since 1.3.0)

```
@spec otp_release() :: String.t()
```

Returns the Erlang/OTP release number.

pid()

(since 1.9.0)

```
@spec pid() :: String.t()
```

Returns the operating system PID for the current Erlang runtime system instance.

Returns a string containing the (usually) numerical identifier for a process. On Unix-like operating systems, this is typically the return value of the `getpid()` system call. On Windows, the process ID as returned by the `GetCurrentProcessId()` system call is used.

Examples

```
System.pid()
```

put_env(enum)

```
@spec put_env(Enumerable.t()) :: :ok
```

Sets multiple environment variables.

Sets a new value for each environment variable corresponding to each `{key, value}` pair in `enum`. Keys and non-nil values are automatically converted to charlists. `nil` values erase the given keys.

Overall, this is a convenience wrapper around `put_env/2` and `delete_env/2` with support for different key and value formats.

put_env(varname, value)

```
@spec put_env(binary(), binary()) :: :ok
```

Sets an environment variable value.

Sets a new `value` for the environment variable `varname`.

restart()

(since 1.9.0)

```
@spec restart() :: :ok
```

Restarts all applications in the Erlang runtime system.

All applications are taken down smoothly, all code is unloaded, and all ports are closed before the system starts all applications once again.

Examples

```
System.restart()
```

schedulers()

(since 1.3.0)

```
@spec schedulers() :: pos_integer()
```

Returns the number of schedulers in the VM.

schedulers_online()

(since 1.3.0)

```
@spec schedulers_online() :: pos_integer()
```

Returns the number of schedulers online in the VM.

shell(command, opts \\ [])

(since 1.12.0)

```
@spec shell(binary(), shell_opts()) ::
  {Collectable.t(), exit_status :: non_neg_integer()}
```

Executes the given `command` in the OS shell.

It uses `sh` for Unix-like systems and `cmd` for Windows.

> **Watch out**
>
> Use this function with care. In particular, **never pass untrusted user input to this function**, as the user would be able to perform "command injection attacks" by executing any code directly on the machine. Generally speaking, prefer to use `cmd/3` over this function.

Examples

```
iex> System.shell("echo hello")
{"hello\n", 0}
```

If you want to stream the output to Standard IO as it arrives:

```
iex> System.shell("echo hello", into: IO.stream())
hello
{%IO.Stream{}, 0}
```

Options

It accepts the same options as `cmd/3` (except for `arg0`). It also accepts the following exclusive options:

- `:close_stdin` (since v1.14.1) - if the stdin should be closed on Unix systems, forcing any command that waits on stdin to immediately terminate. Defaults to `false`.

stacktrace()

This function is deprecated. Use __STACKTRACE__ instead.

Deprecated mechanism to retrieve the last exception stacktrace.

It always return an empty list.

stop(status \\ 0)

(since 1.5.0)

```
@spec stop(non_neg_integer() | binary()) :: :ok
```

Asynchronously and carefully stops the Erlang runtime system.

All applications are taken down smoothly, all code is unloaded, and all ports are closed before the system terminates by calling `halt/1`.

`status` must be a non-negative integer or a binary.

- If an integer, the runtime system exits with the integer value which is returned to the operating system. On many platforms, only the status codes 0-255 are supported by the operating system.
- If a binary, an Erlang crash dump is produced with status as slogan, and then the runtime system exits with status code 1.

Note this function is asynchronous and the current process will continue executing after this function is invoked. In case you want to block the current process until the system effectively shuts down, you can invoke `Process.sleep(:infinity)`.

Examples

```
System.stop(0)
System.stop(1)
```

system_time()

```
@spec system_time() :: integer()
```

Returns the current system time in the `:native` time unit.

It is the VM view of the `os_time/0`. They may not match in case of time warps although the VM works towards aligning them. This time is not monotonic.

Inlined by the compiler.

system_time(unit)

```
@spec system_time(time_unit() | :native) :: integer()
```

Returns the current system time in the given time unit.

It is the VM view of the `os_time/0`. They may not match in case of time warps although the VM works towards aligning them. This time is not monotonic.

time_offset()

```
@spec time_offset() :: integer()
```

Returns the current time offset between the Erlang VM monotonic time and the Erlang VM system time.

The result is returned in the `:native` time unit.

See `time_offset/1` for more information.

Inlined by the compiler.

time_offset(unit)

```
@spec time_offset(time_unit() | :native) :: integer()
```

Returns the current time offset between the Erlang VM monotonic time and the Erlang VM system time.

The result is returned in the given time unit `unit`. The returned offset, added to an Erlang monotonic time (for instance, one obtained with `monotonic_time/1`), gives the Erlang system time that corresponds to that monotonic time.

tmp_dir()

```
@spec tmp_dir() :: String.t() | nil
```

Writable temporary directory.

Returns a writable temporary directory. Searches for directories in the following order:

1. the directory named by the TMPDIR environment variable
2. the directory named by the TEMP environment variable
3. the directory named by the TMP environment variable
4. `C:\TMP` on Windows or `/tmp` on Unix-like operating systems
5. as a last resort, the current working directory

Returns `nil` if none of the above are writable.

tmp_dir!()

```
@spec tmp_dir!() :: String.t()
```

Writable temporary directory, exception on error.

Same as `tmp_dir/0` but raises `RuntimeError` instead of returning `nil` if no temp dir is set.

trap_signal(signal, id \\ make_ref(), fun)

(since 1.12.0)

```
@spec trap_signal(signal(), id, (-> :ok)) ::
  {:ok, id} | {:error, :already_registered} | {:error,
:not_sup}
when id: term()
```

Traps the given `signal` to execute the `fun`.

> **Avoid setting traps in libraries**
>
> Trapping signals may have strong implications on how a system shuts down and behaves in production and therefore it is extremely discouraged for libraries to set their own traps. Instead, they should redirect users to configure them themselves. The only cases where it is acceptable for libraries to set their own traps is when using Elixir in script mode, such as in `.exs` files and via Mix tasks.

An optional `id` that uniquely identifies the function can be given, otherwise a unique one is automatically generated. If a previously registered `id` is given, this function returns an error tuple. The `id` can be used to remove a registered signal by calling `untrap_signal/2`.

The given `fun` receives no arguments and it must return `:ok`.

It returns `{:ok, id}` in case of success, `{:error, :already_registered}` in case the id has already been registered for the given signal, or `{:error, :not_sup}` in case trapping exists is not supported by the current OS.

The first time a signal is trapped, it will override the default behavior from the operating system. If the same signal is trapped multiple times, subsequent functions given to `trap_signal` will execute *first*. In other words, you can consider each function is prepended to the signal handler.

By default, the Erlang VM register traps to the three signals:

- `:sigstop` - gracefully shuts down the VM with `stop/0`
- `:sigquit` - halts the VM via `halt/0`
- `:sigusr1` - halts the VM via status code of 1

Therefore, if you add traps to the signals above, the default behavior above will be executed after all user signals.

Implementation notes

All signals run from a single process. Therefore, blocking the `fun` will block subsequent traps. It is also not possible to add or remove traps from within a trap itself.

Internally, this functionality is built on top of `:os.set_signal/2`. When you register a trap, Elixir automatically sets it to `:handle` and it reverts it back to `:default` once all traps are removed (except for `:sigquit`, `:sigterm`, and `:sigusr1` which are always handled). If you or a library call `:os.set_signal/2` directly, it may disable Elixir traps (or Elixir may override your configuration).

unique_integer(modifiers \\ [])

```
@spec unique_integer([:positive | :monotonic]) :: integer()
```

Generates and returns an integer that is unique in the current runtime instance.

"Unique" means that this function, called with the same list of `modifiers`, will never return the same integer more than once on the current runtime instance.

If `modifiers` is `[]`, then a unique integer (that can be positive or negative) is returned. Other modifiers can be passed to change the properties of the returned integer:

- `:positive` - the returned integer is guaranteed to be positive.
- `:monotonic` - the returned integer is monotonically increasing. This means that, on the same runtime instance (but even on different processes), integers returned using the `:monotonic` modifier will always be strictly less than integers returned by successive calls with the `:monotonic` modifier.

All modifiers listed above can be combined; repeated modifiers in `modifiers` will be ignored.

Inlined by the compiler.

untrap_signal(signal, id)

(since 1.12.0)

```
@spec untrap_signal(signal(), id) :: :ok | {:error,
:not_found} when id: term()
```

Removes a previously registered `signal` with `id`.

user_home()

```
@spec user_home() :: String.t() | nil
```

User home directory.

Returns the user home directory (platform independent).

user_home!()

```
@spec user_home!() :: String.t()
```

User home directory, exception on error.

Same as `user_home/0` but raises `RuntimeError` instead of returning `nil` if no user home is set.

version()

```
@spec version() :: String.t()
```

Elixir version information.

Returns Elixir's version as binary.

Calendar behaviour

This module defines the responsibilities for working with calendars, dates, times and datetimes in Elixir.

It defines types and the minimal implementation for a calendar behaviour in Elixir. The goal of the calendar features in Elixir is to provide a base for interoperability rather than a full-featured datetime API.

For the actual date, time and datetime structs, see `Date`, `Time`, `NaiveDateTime`, and `DateTime`.

Types for year, month, day, and more are *overspecified*. For example, the `month/0` type is specified as an integer instead of `1..12`. This is because different calendars may have a different number of days per month.

Summary

Types

calendar()

A calendar implementation.

date()

Any map or struct that contains the date fields.

datetime()

Any map or struct that contains the datetime fields.
day()
day_fraction()

The internal time format is used when converting between calendars.

day_of_era()

A tuple representing the `day` and the `era`.

day_of_week()
era()
hour()
iso_days()

The internal date format that is used when converting between calendars.

microsecond()

Microseconds with stored precision.

minute()
month()
naive_datetime()

Any map or struct that contains the naive datetime fields.

second()
std_offset()

The time zone standard offset in ISO seconds (typically not zero in summer times).

strftime_opts()

Options for formatting dates and times with `strftime/3`.

time()

Any map or struct that contains the time fields.
time_zone()

The time zone ID according to the IANA tz database (for example, `Europe/Zurich`).

time_zone_database()

Specifies the time zone database for calendar operations.

utc_offset()

The time zone UTC offset in ISO seconds for standard time.

week()
year()
zone_abbr()

The time zone abbreviation (for example, `CET` or `CEST` or `BST`).

Callbacks

date_to_string(year, month, day)

Converts the date into a string according to the calendar.

datetime_to_string(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset)

Converts the datetime (with time zone) into a string according to the calendar.

day_of_era(year, month, day)

Calculates the day and era from the given `year`, `month`, and `day`.

day_of_week(year, month, day, starting_on)

Calculates the day of the week from the given `year`, `month`, and `day`.

day_of_year(year, month, day)

Calculates the day of the year from the given `year`, `month`, and `day`.

day_rollover_relative_to_midnight_utc()

Define the rollover moment for the calendar.

days_in_month(year, month)

Returns how many days there are in the given month of the given year.

iso_days_to_beginning_of_day(iso_days)

Converts the given `iso_days/0` to the first moment of the day.

iso_days_to_end_of_day(iso_days)

Converts the given `iso_days/0` to the last moment of the day.

leap_year?(year)

Returns `true` if the given year is a leap year.

months_in_year(year)

Returns how many months there are in the given year.

naive_datetime_from_iso_days(iso_days)

Converts `iso_days/0` to the calendar's datetime format.

naive_datetime_to_iso_days(year, month, day, hour, minute, second, microsecond)

Converts the datetime (without time zone) into the `iso_days/0` format.

naive_datetime_to_string(year, month, day, hour, minute, second, microsecond)

Converts the naive datetime (without time zone) into a string according to the calendar.

parse_date(t)

Parses the string representation for a date returned by `date_to_string/3` into a date tuple.

parse_naive_datetime(t)

Parses the string representation for a naive datetime returned by `naive_datetime_to_string/7` into a naive datetime tuple.

parse_time(t)

Parses the string representation for a time returned by `time_to_string/4` into a time tuple.

parse_utc_datetime(t)

Parses the string representation for a datetime returned by `datetime_to_string/11` into a datetime tuple.

quarter_of_year(year, month, day)

Calculates the quarter of the year from the given `year`, `month`, and `day`.

shift_date(year, month, day, t)

Shifts date by given duration according to its calendar.

shift_naive_datetime(year, month, day, hour, minute, second, microsecond, t)

Shifts naive datetime by given duration according to its calendar.
shift_time(hour, minute, second, microsecond, t)

Shifts time by given duration according to its calendar.

time_from_day_fraction(day_fraction)

Converts `day_fraction/0` to the calendar's time format.

time_to_day_fraction(hour, minute, second, microsecond)

Converts the given time to the `day_fraction/0` format.

time_to_string(hour, minute, second, microsecond)

Converts the time into a string according to the calendar.

valid_date?(year, month, day)

Should return `true` if the given date describes a proper date in the calendar.

valid_time?(hour, minute, second, microsecond)

Should return `true` if the given time describes a proper time in the calendar.

year_of_era(year, month, day)

Calculates the year and era from the given `year`.

Functions

compatible_calendars?(calendar, calendar)

Returns `true` if two calendars have the same moment of starting a new day, `false` otherwise.

get_time_zone_database()

Gets the current time zone database.

put_time_zone_database(database)

Sets the current time zone database.

strftime(date_or_time_or_datetime, string_format, user_options \\ [])

Formats the given date, time, or datetime into a string.

truncate(microsecond_tuple, atom)

Returns a microsecond tuple truncated to a given precision (`:microsecond` , `:millisecond` , or `:second`).

Types

calendar()

```
@type calendar() :: module()
```

A calendar implementation.

date()

```
@type date() :: %{
  optional(any()) => any(),
  calendar: calendar(),
  year: year(),
  month: month(),
```

```
  day: day()
}
```

Any map or struct that contains the date fields.

datetime()

```
@type datetime() :: %{
  optional(any()) => any(),
  calendar: calendar(),
  year: year(),
  month: month(),
  day: day(),
  hour: hour(),
  minute: minute(),
  second: second(),
  microsecond: microsecond(),
  time_zone: time zone(),
  zone_abbr: zone abbr(),
  utc_offset: utc offset(),
  std_offset: std offset()
}
```

Any map or struct that contains the datetime fields.

day()

```
@type day() :: pos integer()
```

day_fraction()

```
@type day_fraction() ::
  {parts_in_day :: non neg integer(), parts_per_day ::
pos integer()}
```

The internal time format is used when converting between calendars.

It represents time as a fraction of a day (starting from midnight). `parts_in_day` specifies how much of the day is already passed, while `parts_per_day` signifies how many parts are there in a day.

day_of_era()

```
@type day_of_era() :: {day :: non neg integer(), era()}
```

A tuple representing the `day` and the `era`.

day_of_week()

```
@type day_of_week() :: non neg integer()
```

era()

```
@type era() :: non neg integer()
```

hour()

```
@type hour() :: non neg integer()
```

iso_days()

```
@type iso_days() :: {days :: integer(), day fraction()}
```

The internal date format that is used when converting between calendars.

This is the number of days including the fractional part that has passed of the last day since `0000-01-01+00:00T00:00.000000` in ISO 8601 notation (also known as *midnight 1 January BC 1* of the proleptic Gregorian calendar).

microsecond()

```
@type microsecond() :: {value :: non neg integer(), precision
:: non neg integer()}
```

Microseconds with stored precision.

`value` always represents the total value in microseconds.

The `precision` represents the number of digits that must be used when representing the microseconds to external format. If the precision is `0`, it means microseconds must be skipped. If the precision is `6`, it means that `value` represents exactly the number of microseconds to be used.

Examples

- `{0, 0}` means no microseconds.
- `{1, 6}` means 1µs.
- `{1000, 6}` means 1000µs (which is 1ms but measured at the microsecond precision).

- `{1000, 3}` means 1ms (which is measured at the millisecond precision).

minute()

```
@type minute() :: non_neg_integer()
```

month()

```
@type month() :: pos_integer()
```

naive_datetime()

```
@type naive_datetime() :: %{
  optional(any()) => any(),
  calendar: calendar(),
  year: year(),
  month: month(),
  day: day(),
  hour: hour(),
  minute: minute(),
  second: second(),
  microsecond: microsecond()
}
```

Any map or struct that contains the naive datetime fields.

second()

```
@type second() :: non_neg_integer()
```

std_offset()

```
@type std_offset() :: integer()
```

The time zone standard offset in ISO seconds (typically not zero in summer times).

It must be added to `utc_offset/0` to get the total offset from UTC used for "wall time".

strftime_opts()

```
@type strftime_opts() :: [
  preferred_datetime: String.t(),
  preferred_date: String.t(),
  preferred_time: String.t(),
  am_pm_names: (:am | :pm -> String.t()) | (:am | :pm, map()
-> String.t()),
  month_names:
    (pos_integer() -> String.t()) | (pos_integer(), map() ->
String.t()),
  abbreviated_month_names:
    (pos_integer() -> String.t()) | (pos_integer(), map() ->
String.t()),
  day_of_week_names:
```

```
    (pos integer() -> String.t()) | (pos integer(), map() ->
String.t()),
  abbreviated_day_of_week_names:
    (pos integer() -> String.t()) | (pos integer(), map() ->
String.t())
]
```

Options for formatting dates and times with `strftime/3`.

time()

```
@type time() :: %{
  optional(any()) => any(),
  hour: hour(),
  minute: minute(),
  second: second(),
  microsecond: microsecond()
}
```

Any map or struct that contains the time fields.

time_zone()

```
@type time_zone() :: String.t()
```

The time zone ID according to the IANA tz database (for example, `Europe/Zurich`).

time_zone_database()

```
@type time_zone_database() :: module()
```

Specifies the time zone database for calendar operations.

Many functions in the `DateTime` module require a time zone database. By default, this module uses the default time zone database returned by `Calendar.get_time_zone_database/0`, which defaults to `Calendar.UTCOnlyTimeZoneDatabase`. This database only handles `Etc/UTC` datetimes and returns `{:error, :utc_only_time_zone_database}` for any other time zone.

Other time zone databases (including ones provided by packages) can be configured as default either via configuration:

```
config :elixir, :time_zone_database,
CustomTimeZoneDatabase
```

or by calling `Calendar.put_time_zone_database/1`.

See `Calendar.TimeZoneDatabase` for more information on custom time zone databases.

utc_offset()

```
@type utc_offset() :: integer()
```

The time zone UTC offset in ISO seconds for standard time.

See also `std_offset/0`.

week()

```
@type week() :: pos_integer()
```

year()

```
@type year() :: integer()
```

zone_abbr()

```
@type zone_abbr() :: String.t()
```

The time zone abbreviation (for example, `CET` or `CEST` or `BST`).

Callbacks

date_to_string(year, month, day)

```
@callback date_to_string(year(), month(), day()) ::
String.t()
```

Converts the date into a string according to the calendar.

datetime_to_string(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset)

```
@callback datetime_to_string(
  year(),
  month(),
  day(),
```

```
  hour(),
  minute(),
  second(),
  microsecond(),
  time_zone(),
  zone_abbr(),
  utc_offset(),
  std_offset()
) :: String.t()
```

Converts the datetime (with time zone) into a string according to the calendar.

day_of_era(year, month, day)

```
@callback day_of_era(year(), month(), day()) :: day_of_era()
```

Calculates the day and era from the given `year`, `month`, and `day`.

day_of_week(year, month, day, starting_on)

```
@callback day_of_week(year(), month(), day(), starting_on ::
:default | atom()) ::
  {day_of_week(), first_day_of_week :: non_neg_integer(),
   last_day_of_week :: non_neg_integer()}
```

Calculates the day of the week from the given `year`, `month`, and `day`.

`starting_on` represents the starting day of the week. All calendars must support at least the `:default` value. They may also support other values representing their days of the week.

The value of `day_of_week` is an ordinal number meaning that a value of `1` is defined to mean "first day of the week". It is specifically not defined to mean `1` is `Monday`.

It is a requirement that `first_day_of_week` is less than `last_day_of_week` and that `day_of_week` must be within that range. Therefore it can be said that `day_of_week in first_day_of_week..last_day_of_week//1` must be `true` for all values of `day_of_week`.

day_of_year(year, month, day)

```
@callback day_of_year(year(), month(), day()) ::
non_neg_integer()
```

Calculates the day of the year from the given `year`, `month`, and `day`.

day_rollover_relative_to_midnight_utc()

```
@callback day_rollover_relative_to_midnight_utc() ::
day_fraction()
```

Define the rollover moment for the calendar.

This is the moment, in your calendar, when the current day ends and the next day starts.

The result of this function is used to check if two calendars roll over at the same time of day. If they do not, we can only convert datetimes and times between them. If they do, this means that we can also convert dates as well as naive datetimes between them.

This day fraction should be in its most simplified form possible, to make comparisons fast.

Examples

- If in your calendar a new day starts at midnight, return `{0, 1}`.
- If in your calendar a new day starts at sunrise, return `{1, 4}`.
- If in your calendar a new day starts at noon, return `{1, 2}`.
- If in your calendar a new day starts at sunset, return `{3, 4}`.

days_in_month(year, month)

```
@callback days_in_month(year(), month()) :: day()
```

Returns how many days there are in the given month of the given year.

iso_days_to_beginning_of_day(iso_days)

(since 1.15.0)

```
@callback iso_days_to_beginning_of_day(iso_days()) ::
iso_days()
```

Converts the given `iso_days/0` to the first moment of the day.

iso_days_to_end_of_day(iso_days)

(since 1.15.0)

```
@callback iso_days_to_end_of_day(iso_days()) :: iso_days()
```

Converts the given `iso_days/0` to the last moment of the day.

leap_year?(year)

```
@callback leap_year?(year()) :: boolean()
```

Returns `true` if the given year is a leap year.

A leap year is a year of a longer length than normal. The exact meaning is up to the calendar. A calendar must return `false` if it does not support the concept of leap years.

months_in_year(year)

```
@callback months_in_year(year()) :: month()
```

Returns how many months there are in the given year.

naive_datetime_from_iso_days(iso_days)

```
@callback naive_datetime_from_iso_days(iso_days()) ::
  {year(), month(), day(), hour(), minute(), second(),
microsecond()}
```

Converts `iso_days/0` to the calendar's datetime format.

naive_datetime_to_iso_days(year, month, day, hour, minute, second, microsecond)

```
@callback naive_datetime_to_iso_days(
  year(),
  month(),
  day(),
  hour(),
  minute(),
  second(),
  microsecond()
) ::
  iso_days()
```

Converts the datetime (without time zone) into the `iso_days/0` format.

naive_datetime_to_string(year, month, day, hour, minute, second, microsecond)

```
@callback naive_datetime_to_string(
  year(),
  month(),
  day(),
  hour(),
  minute(),
  second(),
  microsecond()
) ::
  String.t()
```

Converts the naive datetime (without time zone) into a string according to the calendar.

parse_date(t)

(since 1.10.0)

```
@callback parse_date(String.t()) :: {:ok, {year(), month(),
day()}} | {:error, atom()}
```

Parses the string representation for a date returned by `date_to_string/3` into a date tuple.

parse_naive_datetime(t)

(since 1.10.0)

```
@callback parse_naive_datetime(String.t()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
microsecond()}}
  | {:error, atom()}
```

Parses the string representation for a naive datetime returned by `naive_datetime_to_string/7` into a naive datetime tuple.

The given string may contain a timezone offset but it is ignored.

parse_time(t)

(since 1.10.0)

```
@callback parse_time(String.t()) ::
  {:ok, {hour(), minute(), second(), microsecond()}} |
{:error, atom()}
```

Parses the string representation for a time returned by `time_to_string/4` into a time tuple.

parse_utc_datetime(t)

(since 1.10.0)

```
@callback parse_utc_datetime(String.t()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
microsecond()},
   utc_offset()}
  | {:error, atom()}
```

Parses the string representation for a datetime returned by `datetime_to_string/11` into a datetime tuple.

The returned datetime must be in UTC. The original `utc_offset` it was written in must be returned in the result.

quarter_of_year(year, month, day)

```
@callback quarter_of_year(year(), month(), day()) ::
non_neg_integer()
```

Calculates the quarter of the year from the given `year`, `month`, and `day`.

shift_date(year, month, day, t)

(since 1.17.0)

```
@callback shift_date(year(), month(), day(), Duration.t()) ::
{year(), month(), day()}
```

Shifts date by given duration according to its calendar.

shift_naive_datetime(year, month, day, hour, minute, second, microsecond, t)

(since 1.17.0)

```
@callback shift_naive_datetime(
  year(),
  month(),
  day(),
  hour(),
  minute(),
  second(),
  microsecond(),
  Duration.t()
) :: {year(), month(), day(), hour(), minute(), second(),
microsecond()}
```

Shifts naive datetime by given duration according to its calendar.

shift_time(hour, minute, second, microsecond, t)

(since 1.17.0)

```
@callback shift_time(hour(), minute(), second(),
microsecond(), Duration.t()) ::
  {hour(), minute(), second(), microsecond()}
```

Shifts time by given duration according to its calendar.

time_from_day_fraction(day_fraction)

```
@callback time_from_day_fraction(day_fraction()) ::
  {hour(), minute(), second(), microsecond()}
```

Converts `day_fraction/0` to the calendar's time format.

time_to_day_fraction(hour, minute, second, microsecond)

```
@callback time_to_day_fraction(hour(), minute(), second(),
microsecond()) ::
  day_fraction()
```

Converts the given time to the `day_fraction/0` format.

time_to_string(hour, minute, second, microsecond)

```
@callback time_to_string(hour(), minute(), second(),
microsecond()) :: String.t()
```

Converts the time into a string according to the calendar.

valid_date?(year, month, day)

```
@callback valid_date?(year(), month(), day()) :: boolean()
```

Should return `true` if the given date describes a proper date in the calendar.

valid_time?(hour, minute, second, microsecond)

```
@callback valid_time?(hour(), minute(), second(),
microsecond()) :: boolean()
```

Should return `true` if the given time describes a proper time in the calendar.

year_of_era(year, month, day)

```
@callback year_of_era(year(), month(), day()) :: {year(), era()}
```

Calculates the year and era from the given `year`.

Functions

compatible_calendars?(calendar, calendar)

(since 1.5.0)

```
@spec compatible_calendars?(calendar(), calendar()) :: boolean()
```

Returns `true` if two calendars have the same moment of starting a new day, `false` otherwise.

If two calendars are not compatible, we can only convert datetimes and times between them. If they are compatible, this means that we can also convert dates as well as naive datetimes between them.

get_time_zone_database()

(since 1.8.0)

```
@spec get_time_zone_database() :: time zone database()
```

Gets the current time zone database.

put_time_zone_database(database)

(since 1.8.0)

```
@spec put_time_zone_database(time zone database()) :: :ok
```

Sets the current time zone database.

strftime(date_or_time_or_datetime, string_format, user_options \\ [])

(since 1.11.0)

```
@spec strftime(map(), String.t(), strftime opts()) ::
String.t()
```

Formats the given date, time, or datetime into a string.

The datetime can be any of the `Calendar` types (`Time`, `Date`, `NaiveDateTime`, and `DateTime`) or any map, as long as they contain all of the relevant fields necessary for formatting. For example, if you use `%Y` to format the year, the datetime must have the `:year` field. Therefore, if you pass a `Time`, or a map without the `:year` field to a format that expects `%Y`, an error will be raised.

Examples of common usage:

```
iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%y-
%m-%d %I:%M:%S %p")
"19-08-26 01:52:06 PM"

iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%a,
```

```
%B %d %Y")
"Mon, August 26 2019"
```

User Options

- `:preferred_datetime` - a string for the preferred format to show datetimes, it can't contain the `%c` format and defaults to `"%Y-%m-%d %H:%M:%S"` if the option is not received
- `:preferred_date` - a string for the preferred format to show dates, it can't contain the `%x` format and defaults to `"%Y-%m-%d"` if the option is not received
- `:preferred_time` - a string for the preferred format to show times, it can't contain the `%X` format and defaults to `"%H:%M:%S"` if the option is not received
- `:am_pm_names` - a function that receives either `:am` or `:pm` (and also the datetime if the function is arity/2) and returns the name of the period of the day, if the option is not received it defaults to a function that returns `"am"` and `"pm"`, respectively
- `:month_names` - a function that receives a number (and also the datetime if the function is arity/2) and returns the name of the corresponding month, if the option is not received it defaults to a function that returns the month names in English
- `:abbreviated_month_names` - a function that receives a number (and also the datetime if the function is arity/2) and returns the abbreviated name of the corresponding month, if the option is not received it defaults to a function that returns the abbreviated month names in English

- `:day_of_week_names` - a function that receives a number and (and also the datetime if the function is arity/2) returns the name of the corresponding day of week, if the option is not received it defaults to a function that returns the day of week names in English

- `:abbreviated_day_of_week_names` - a function that receives a number (and also the datetime if the function is arity/2) and returns the abbreviated name of the corresponding day of week, if the option is not received it defaults to a function that returns the abbreviated day of week names in English

Formatting syntax

The formatting syntax for the `string_format` argument is a sequence of characters in the following format:

```
%<padding><width><format>
```

where:

- `%`: indicates the start of a formatted section
- `<padding>`: set the padding (see below)
- `<width>`: a number indicating the minimum size of the formatted section (maximum 1024)
- `<format>`: the format itself (see below)

Accepted padding options

- `-` : no padding, removes all padding from the format
- `_` : pad with spaces
- `0`: pad with zeroes

Accepted string formats

The accepted formats for `string_format` are:

Format	Description	Examples (in ISO)
a	Abbreviated name of day	Mon
A	Full name of day	Monday
b	Abbreviated month name	Jan
B	Full month name	January
c	Preferred date+time representation	2018-10-17 12:34:56
d	Day of the month	01, 31
f	Microseconds (uses its precision for width and padding)	000000, 999999, 0123
H	Hour using a 24-hour clock	00, 23
I	Hour using a 12-hour clock	01, 12
j	Day of the year	001, 366
m	Month	01, 12
M	Minute	00, 59
p	"AM" or "PM" (noon is "PM", midnight as "AM")	AM, PM
P	"am" or "pm" (noon is "pm", midnight as "am")	am, pm
q	Quarter	1, 2, 3, 4
s	Number of seconds since the Epoch, 1970-01-01 00:00:00+0000 (UTC)	1565888877
S	Second	00, 59, 60
u	Day of the week	1 (Monday), 7 (Sunday)
x	Preferred date (without time) representation	2018-10-17
X	Preferred time (without date) representation	12:34:56
y	Year as 2-digits	01, 01, 86, 18
Y	Year	-0001, 0001, 1986

Format	Description	Examples (in ISO)
z	+hhmm/-hhmm time zone offset from UTC (empty string if naive)	+0300, -0530
Z	Time zone abbreviation (empty string if naive)	CET, BRST
%	Literal "%" character	%

Any other character will be interpreted as an invalid format and raise an error.

`%f` Microseconds

`%f` does not support width and padding modifiers. It will be formatted by truncating the microseconds to the precision of the `microseconds` field of the struct, with a minimum precision of 1.

Examples

Without user options:

```
iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%y-
%m-%d %I:%M:%S %p")
"19-08-26 01:52:06 PM"

iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%a,
%B %d %Y")
"Mon, August 26 2019"

iex> Calendar.strftime(~U[2020-04-02 13:52:06.0Z], "%B %-
d, %Y")
"April 2, 2020"
```

```
iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%c")
"2019-08-26 13:52:06"
```

With user options:

```
iex> Calendar.strftime(~U[2019-08-26 13:52:06.0Z], "%c",
preferred_datetime: "%H:%M:%S %d-%m-%y")
"13:52:06 26-08-19"

iex> Calendar.strftime(
...>  ~U[2019-08-26 13:52:06.0Z],
...>  "%A",
...>  day_of_week_names: fn day_of_week ->
...>    {"segunda-feira", "terça-feira", "quarta-feira",
"quinta-feira",
...>    "sexta-feira", "sábado", "domingo"}
...>    |> elem(day_of_week - 1)
...>  end
...>)
"segunda-feira"

iex> Calendar.strftime(
...>  ~U[2019-08-26 13:52:06.0Z],
...>  "%B",
...>  month_names: fn month ->
...>    {"січень", "лютий", "березень", "квітень",
"травень", "червень",
...>    "липень", "серпень", "вересень", "жовтень",
"листопад", "грудень"}
...>    |> elem(month - 1)
...>  end
...>)
"серпень"
```

Microsecond formatting:

```
iex> Calendar.strftime(~U[2019-08-26 13:52:06Z], "%y-%m-
%d %H:%M:%S.%f")
"19-08-26 13:52:06.0"

iex> Calendar.strftime(~U[2019-08-26 13:52:06.048Z], "%y-
%m-%d %H:%M:%S.%f")
"19-08-26 13:52:06.048"

iex> Calendar.strftime(~U[2019-08-26 13:52:06.048531Z],
"%y-%m-%d %H:%M:%S.%f")
"19-08-26 13:52:06.048531"
```

truncate(microsecond_tuple, atom)

(since 1.6.0)

```
@spec truncate(microsecond(), :microsecond | :millisecond |
:second) :: microsecond()
```

Returns a microsecond tuple truncated to a given precision (`:microsecond`, `:millisecond`, or `:second`).

Calendar.ISO

The default calendar implementation, a Gregorian calendar following ISO 8601.

This calendar implements a proleptic Gregorian calendar and is therefore compatible with the calendar used in most countries today. The proleptic means the Gregorian rules for leap years are applied for all time, consequently the dates give different results before the year 1583 from when the Gregorian calendar was adopted.

ISO 8601 compliance

The ISO 8601 specification is feature-rich, but allows applications to selectively implement most parts of it. The choices Elixir makes are catalogued below.

Features

The standard library supports a minimal set of possible ISO 8601 features. Specifically, the parser only supports calendar dates and does not support ordinal and week formats. Additionally, it supports parsing ISO 8601 formatted durations, including negative time units and fractional seconds.

By default Elixir only parses extended-formatted date/times. You can opt-in to parse basic-formatted date/times.

`NaiveDateTime.to_iso8601/2` and `DateTime.to_iso8601/2` allow you to produce either basic or extended formatted strings, and `Calendar.strftime/2` allows you to format datetimes however else you desire.

Elixir does not support reduced accuracy formats (for example, a date without the day component) nor decimal precisions in the lowest component (such as `10:01:25,5`).

Examples

Elixir expects the extended format by default when parsing:

```
iex> Calendar.ISO.parse_naive_datetime("2015-01-
23T23:50:07")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_naive_datetime("20150123T235007")
{:error, :invalid_format}
```

Parsing can be restricted to basic if desired:

```
iex>
Calendar.ISO.parse_naive_datetime("20150123T235007Z",
:basic)
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}
iex>
Calendar.ISO.parse_naive_datetime("20150123T235007Z",
:extended)
{:error, :invalid_format}
```

Only calendar dates are supported in parsing; ordinal and week dates are not.

```
iex> Calendar.ISO.parse_date("2015-04-15")
{:ok, {2015, 4, 15}}
iex> Calendar.ISO.parse_date("2015-105")
{:error, :invalid_format}
iex> Calendar.ISO.parse_date("2015-W16")
```

```
{:error, :invalid_format}
iex> Calendar.ISO.parse_date("2015-W016-3")
{:error, :invalid_format}
```

Years, months, days, hours, minutes, and seconds must be fully specified:

```
iex> Calendar.ISO.parse_date("2015-04-15")
{:ok, {2015, 4, 15}}
iex> Calendar.ISO.parse_date("2015-04")
{:error, :invalid_format}
iex> Calendar.ISO.parse_date("2015")
{:error, :invalid_format}

iex> Calendar.ISO.parse_time("23:50:07.0123456")
{:ok, {23, 50, 7, {12345, 6}}}
iex> Calendar.ISO.parse_time("23:50:07")
{:ok, {23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_time("23:50")
{:error, :invalid_format}
iex> Calendar.ISO.parse_time("23")
{:error, :invalid_format}
```

Extensions

The parser and formatter adopt one ISO 8601 extension: extended year notation.

This allows dates to be prefixed with a `+` or `-` sign, extending the range of expressible years from the default (`0000..9999`) to `-9999..9999`. Elixir still restricts years in this format to four digits.

Examples

```
iex> Calendar.ISO.parse_date("-2015-01-23")
{:ok, {-2015, 1, 23}}
iex> Calendar.ISO.parse_date("+2015-01-23")
{:ok, {2015, 1, 23}}

iex> Calendar.ISO.parse_naive_datetime("-2015-01-23
23:50:07")
{:ok, {-2015, 1, 23, 23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_naive_datetime("+2015-01-23
23:50:07")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}

iex> Calendar.ISO.parse_utc_datetime("-2015-01-23
23:50:07Z")
{:ok, {-2015, 1, 23, 23, 50, 7, {0, 0}}, 0}
iex> Calendar.ISO.parse_utc_datetime("+2015-01-23
23:50:07Z")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}, 0}
```

Additions

ISO 8601 does not allow a whitespace instead of `T` as a separator between date and times, both when parsing and formatting. This is a common enough representation, Elixir allows it during parsing.

The formatting of dates in `NaiveDateTime.to_iso8601/1` and `DateTime.to_iso8601/1` do produce specification-compliant string representations using the `T` separator.

Examples

```
iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07.0123456")
```

```
{:ok, {2015, 1, 23, 23, 50, 7, {12345, 6}}}
iex> Calendar.ISO.parse_naive_datetime("2015-01-
23T23:50:07.0123456")
{:ok, {2015, 1, 23, 23, 50, 7, {12345, 6}}}

iex> Calendar.ISO.parse_utc_datetime("2015-01-23
23:50:07.0123456Z")
{:ok, {2015, 1, 23, 23, 50, 7, {12345, 6}}, 0}
iex> Calendar.ISO.parse_utc_datetime("2015-01-
23T23:50:07.0123456Z")
{:ok, {2015, 1, 23, 23, 50, 7, {12345, 6}}, 0}
```

Summary

Types

bce()

"Before the Current Era" or "Before the Common Era" (BCE), for those years less than `1`.

ce()

The "Current Era" or the "Common Era" (CE) which starts in year `1`.

day()
day_of_week()

Integer that represents the day of the week, where 1 is Monday and 7 is Sunday.

day_of_year()
era()

The calendar era.
format()
hour()
microsecond()

Microseconds with stored precision.

minute()
month()
quarter_of_year()
second()
utc_offset()
weekday()
year()
year_of_era()

Functions

date_to_iodata(year, month, day, format \\ :extended)

Converts the given date into a iodata.

date_to_string(year, month, day, format \\ :extended)

Converts the given date into a string.

datetime_to_iodata(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset, format \\ :extended)

Converts the given datetime into a iodata.

datetime_to_string(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset, format \\ :extended)

Converts the datetime (with time zone) into a string.

day_of_era(year, month, day)

Calculates the day and era from the given `year`, `month`, and `day`.

day_of_week(year, month, day, starting_on)

Calculates the day of the week from the given `year`, `month`, and `day`.

day_of_year(year, month, day)

Calculates the day of the year from the given `year`, `month`, and `day`.

day_rollover_relative_to_midnight_utc()

See `Calendar.day_rollover_relative_to_midnight_utc/0` for documentation.

days_in_month(year, month)

Returns how many days there are in the given year-month.

iso_days_to_beginning_of_day(arg)

Converts the `Calendar.iso_days/0` to the first moment of the day.

iso_days_to_end_of_day(arg)

Converts the `Calendar.iso_days/0` to the last moment of the day.

leap_year?(year)

Returns if the given year is a leap year.

months_in_year(year)

Returns how many months there are in the given year.

naive_datetime_from_iso_days(arg)

Converts the `Calendar.iso_days/0` format to the datetime format specified by this calendar.

naive_datetime_to_iodata(year, month, day, hour, minute, second, microsecond, format \\ :extended)

Converts the given naive_datetime into a iodata.

naive_datetime_to_iso_days(year, month, day, hour, minute, second, microsecond)

Returns the `Calendar.iso_days/0` format of the specified date.

naive_datetime_to_string(year, month, day, hour, minute, second, microsecond, format \\ :extended)

Converts the datetime (without time zone) into a string.

parse_date(string)

Parses a date `string` in the `:extended` format.

parse_date(string, format)

Parses a date `string` according to a given `format`.

parse_duration(arg1)

Parses an ISO 8601 formatted duration string to a list of `Duration` compabitble unit pairs.

parse_naive_datetime(string)

Parses a naive datetime `string` in the `:extended` format.

parse_naive_datetime(string, format)

Parses a naive datetime `string` according to a given `format`.

parse_time(string)

Parses a time `string` in the `:extended` format.

parse_time(string, format)

Parses a time `string` according to a given `format`.

parse_utc_datetime(string)

Parses a UTC datetime `string` in the `:extended` format.

parse_utc_datetime(string, format)

Parses a UTC datetime `string` according to a given `format`.

quarter_of_year(year, month, day)

Calculates the quarter of the year from the given `year`, `month`, and `day`.

shift_date(year, month, day, duration)

Shifts Date by Duration according to its calendar.

shift_naive_datetime(year, month, day, hour, minute, second, microsecond, duration)

Shifts NaiveDateTime by Duration according to its calendar.

shift_time(hour, minute, second, microsecond, duration)

Shifts Time by Duration units according to its calendar.

time_from_day_fraction(arg)

Converts a day fraction to this Calendar's representation of time.

time_to_day_fraction(hour, minute, second, arg)

Returns the normalized day fraction of the specified time.

time_to_iodata(hour, minute, second, microsecond, format \\ :extended)

Converts the given time into a iodata.

time_to_string(hour, minute, second, microsecond, format \\ :extended)

Converts the given time into a string.

time_unit_to_precision(int)

Converts a `System.time_unit/0` to precision.

valid_date?(year, month, day)

Determines if the date given is valid according to the proleptic Gregorian calendar.

valid_time?(hour, minute, second, microsecond)

Determines if the date given is valid according to the proleptic Gregorian calendar.

year_of_era(year)

Calculates the year and era from the given `year`.

year_of_era(year, month, day)

Calendar callback to compute the year and era from the given `year`, `month` and `day`.

Types

bce()

```
@type bce() :: 0
```

"Before the Current Era" or "Before the Common Era" (BCE), for those years less than `1`.

ce()

```
@type ce() :: 1
```

The "Current Era" or the "Common Era" (CE) which starts in year `1`.

day()

```
@type day() :: 1..31
```

day_of_week()

```
@type day_of_week() :: 1..7
```

Integer that represents the day of the week, where 1 is Monday and 7 is Sunday.

day_of_year()

```
@type day_of_year() :: 1..366
```

era()

```
@type era() :: bce() | ce()
```

The calendar era.

The ISO calendar has two eras:

- CE - which starts in year `1` and is defined as era `1`.
- BCE - for those years less than `1` and is defined as era `0`.

format()

```
@type format() :: :basic | :extended
```

hour()

```
@type hour() :: 0..23
```

microsecond()

```
@type microsecond() :: {0..999_999, 0..6}
```

Microseconds with stored precision.

The precision represents the number of digits that must be used when representing the microseconds to external format. If the precision is 0, it

means microseconds must be skipped.

minute()

```
@type minute() :: 0..59
```

month()

```
@type month() :: 1..12
```

quarter_of_year()

```
@type quarter_of_year() :: 1..4
```

second()

```
@type second() :: 0..59
```

utc_offset()

```
@type utc_offset() :: integer()
```

weekday()

```
@type weekday() ::
  :monday | :tuesday | :wednesday | :thursday | :friday |
:saturday | :sunday
```

year()

```
@type year() :: -9999..9999
```

year_of_era()

```
@type year_of_era() :: {1..10000, era()}
```

Functions

date_to_iodata(year, month, day, format \\ :extended)

(since 1.19.0)

```
@spec date_to_iodata(year(), month(), day(), :basic |
:extended) :: iodata()
```

Converts the given date into a iodata.

See `date_to_string/4` for more information.

Examples

```
iex> data = Calendar.ISO.date_to_iodata(2015, 2, 28)
iex> IO.iodata_to_binary(data)
"2015-02-28"
```

date_to_string(year, month, day, format \\ :extended)

(since 1.4.0)

```
@spec date_to_string(year(), month(), day(), :basic |
:extended) :: String.t()
```

Converts the given date into a string.

By default, returns dates formatted in the "extended" format, for human readability. It also supports the "basic" format by passing the `:basic` option.

Examples

```
iex> Calendar.ISO.date_to_string(2015, 2, 28)
"2015-02-28"
iex> Calendar.ISO.date_to_string(2017, 8, 1)
"2017-08-01"
iex> Calendar.ISO.date_to_string(-99, 1, 31)
"-0099-01-31"

iex> Calendar.ISO.date_to_string(2015, 2, 28, :basic)
"20150228"
iex> Calendar.ISO.date_to_string(-99, 1, 31, :basic)
"-00990131"
```

datetime_to_iodata(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset, format \\ :extended)

(since 1.19.0)

```
@spec datetime_to_iodata(
  year(),
  month(),
  day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  Calendar.time_zone(),
  Calendar.zone_abbr(),
  Calendar.utc_offset(),
  Calendar.std_offset(),
  :basic | :extended
) :: iodata()
```

Converts the given datetime into a iodata.

See `datetime_to_iodata/12` for more information.

Examples

```
iex> time_zone = "Etc/UTC"
iex> data = Calendar.ISO.datetime_to_iodata(2017, 8, 1,
1, 2, 3, {4, 5}, time_zone, "UTC", 0, 0)
iex> IO.iodata_to_binary(data)
"2017-08-01 01:02:03.00000Z"
```

datetime_to_string(year, month, day, hour, minute, second, microsecond, time_zone, zone_abbr, utc_offset, std_offset, format \\ :extended)

(since 1.4.0)

```
@spec datetime_to_string(
  year(),
  month(),
  day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  Calendar.time_zone(),
  Calendar.zone_abbr(),
  Calendar.utc_offset(),
  Calendar.std_offset(),
  :basic | :extended
) :: String.t()
```

Converts the datetime (with time zone) into a string.

By default, returns datetimes formatted in the "extended" format, for human readability. It also supports the "basic" format by passing the `:basic` option.

Examples

```
iex> time_zone = "Etc/UTC"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "UTC", 0, 0)
"2017-08-01 01:02:03.00000Z"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "UTC", 3600, 0)
```

```
"2017-08-01 01:02:03.00000+01:00"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "UTC", 3600, 3600)
"2017-08-01 01:02:03.00000+02:00"

iex> time_zone = "Europe/Berlin"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "CET", 3600, 0)
"2017-08-01 01:02:03.00000+01:00 CET Europe/Berlin"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "CDT", 3600, 3600)
"2017-08-01 01:02:03.00000+02:00 CDT Europe/Berlin"

iex> time_zone = "America/Los_Angeles"
iex> Calendar.ISO.datetime_to_string(2015, 2, 28, 1, 2,
3, {4, 5}, time_zone, "PST", -28800, 0)
"2015-02-28 01:02:03.00000-08:00 PST America/Los_Angeles"
iex> Calendar.ISO.datetime_to_string(2015, 2, 28, 1, 2,
3, {4, 5}, time_zone, "PDT", -28800, 3600)
"2015-02-28 01:02:03.00000-07:00 PDT America/Los_Angeles"

iex> time_zone = "Europe/Berlin"
iex> Calendar.ISO.datetime_to_string(2017, 8, 1, 1, 2, 3,
{4, 5}, time_zone, "CET", 3600, 0, :basic)
"20170801 010203.00000+0100 CET Europe/Berlin"
```

day_of_era(year, month, day)

(since 1.8.0)

```
@spec day_of_era(year(), month(), day()) ::
Calendar.day_of_era()
```

Calculates the day and era from the given `year`, `month`, and `day`.

Examples

```
iex> Calendar.ISO.day_of_era(0, 1, 1)
{366, 0}
iex> Calendar.ISO.day_of_era(1, 1, 1)
{1, 1}
iex> Calendar.ISO.day_of_era(0, 12, 31)
{1, 0}
iex> Calendar.ISO.day_of_era(0, 12, 30)
{2, 0}
iex> Calendar.ISO.day_of_era(-1, 12, 31)
{367, 0}
```

day_of_week(year, month, day, starting_on)

(since 1.11.0)

```
@spec day_of_week(year(), month(), day(), :default |
weekday()) ::
  {day of week(), 1, 7}
```

Calculates the day of the week from the given `year`, `month`, and `day`.

It is an integer from 1 to 7, where 1 is the given `starting_on` weekday. For example, if `starting_on` is set to `:monday`, then 1 is Monday and 7 is Sunday.

`starting_on` can also be `:default`, which is equivalent to `:monday`.

Examples

```
iex> Calendar.ISO.day_of_week(2016, 10, 31, :monday)
{1, 1, 7}
```

```
iex> Calendar.ISO.day_of_week(2016, 11, 1, :monday)
{2, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 2, :monday)
{3, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 3, :monday)
{4, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 4, :monday)
{5, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 5, :monday)
{6, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 6, :monday)
{7, 1, 7}
iex> Calendar.ISO.day_of_week(-99, 1, 31, :monday)
{4, 1, 7}

iex> Calendar.ISO.day_of_week(2016, 10, 31, :sunday)
{2, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 1, :sunday)
{3, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 2, :sunday)
{4, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 3, :sunday)
{5, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 4, :sunday)
{6, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 5, :sunday)
{7, 1, 7}
iex> Calendar.ISO.day_of_week(2016, 11, 6, :sunday)
{1, 1, 7}
iex> Calendar.ISO.day_of_week(-99, 1, 31, :sunday)
{5, 1, 7}

iex> Calendar.ISO.day_of_week(2016, 10, 31, :saturday)
{3, 1, 7}
```

day_of_year(year, month, day)

(since 1.8.0)

```
@spec day_of_year(year(), month(), day()) :: day_of_year()
```

Calculates the day of the year from the given `year`, `month`, and `day`.

It is an integer from 1 to 366.

Examples

```
iex> Calendar.ISO.day_of_year(2016, 1, 31)
31
iex> Calendar.ISO.day_of_year(-99, 2, 1)
32
iex> Calendar.ISO.day_of_year(2018, 2, 28)
59
```

day_rollover_relative_to_midnight_utc()

(since 1.5.0)

```
@spec day_rollover_relative_to_midnight_utc() :: {0, 1}
```

See `Calendar.day_rollover_relative_to_midnight_utc/0` for documentation.

days_in_month(year, month)

(since 1.4.0)

```
@spec days_in_month(year(), month()) :: 28..31
```

Returns how many days there are in the given year-month.

Examples

```
iex> Calendar.ISO.days_in_month(1900, 1)
31
iex> Calendar.ISO.days_in_month(1900, 2)
28
iex> Calendar.ISO.days_in_month(2000, 2)
29
iex> Calendar.ISO.days_in_month(2001, 2)
28
iex> Calendar.ISO.days_in_month(2004, 2)
29
iex> Calendar.ISO.days_in_month(2004, 4)
30
iex> Calendar.ISO.days_in_month(-1, 5)
31
```

iso_days_to_beginning_of_day(arg)

(since 1.15.0)

```
@spec iso_days_to_beginning_of_day(Calendar.iso_days()) ::
Calendar.iso_days()
```

Converts the `Calendar.iso_days/0` to the first moment of the day.

Examples

```
iex> Calendar.ISO.iso_days_to_beginning_of_day({0, {0,
86_400_000_000}})
{0, {0, 86400000000}}
iex> Calendar.ISO.iso_days_to_beginning_of_day({730_485,
{43_200_000_000, 86_400_000_000}})
{730485, {0, 86400000000}}
iex> Calendar.ISO.iso_days_to_beginning_of_day({730_485,
{46_800_000_000, 86_400_000_000}})
{730485, {0, 86400000000}}
```

iso_days_to_end_of_day(arg)

(since 1.15.0)

```
@spec iso_days_to_end_of_day(Calendar.iso_days()) ::
Calendar.iso_days()
```

Converts the `Calendar.iso_days/0` to the last moment of the day.

Examples

```
iex> Calendar.ISO.iso_days_to_end_of_day({0, {0,
86_400_000_000}})
{0, {86399999999, 86400000000}}
iex> Calendar.ISO.iso_days_to_end_of_day({730_485,
{43_200_000_000, 86_400_000_000}})
{730485, {86399999999, 86400000000}}
iex> Calendar.ISO.iso_days_to_end_of_day({730_485,
{46_800_000_000, 86_400_000_000}})
{730485, {86399999999, 86400000000}}
```

leap_year?(year)

(since 1.3.0)

```
@spec leap_year?(year()) :: boolean()
```

Returns if the given year is a leap year.

Examples

```
iex> Calendar.ISO.leap_year?(2000)
true
iex> Calendar.ISO.leap_year?(2001)
false
iex> Calendar.ISO.leap_year?(2004)
true
iex> Calendar.ISO.leap_year?(1900)
false
iex> Calendar.ISO.leap_year?(-4)
true
```

months_in_year(year)

(since 1.7.0)

```
@spec months_in_year(year()) :: 12
```

Returns how many months there are in the given year.

Example

```
iex> Calendar.ISO.months_in_year(2004)
12
```

naive_datetime_from_iso_days(arg)

(since 1.5.0)

```
@spec naive_datetime_from_iso_days(Calendar.iso_days()) ::
  {Calendar.year(), Calendar.month(), Calendar.day(),
Calendar.hour(),
   Calendar.minute(), Calendar.second(),
Calendar.microsecond()}
```

Converts the `Calendar.iso_days/0` format to the datetime format specified by this calendar.

Examples

```
iex> Calendar.ISO.naive_datetime_from_iso_days({0, {0,
86_400}})
{0, 1, 1, 0, 0, 0, {0, 6}}
iex> Calendar.ISO.naive_datetime_from_iso_days({730_485,
{0, 86_400}})
{2000, 1, 1, 0, 0, 0, {0, 6}}
iex> Calendar.ISO.naive_datetime_from_iso_days({730_485,
{43_200, 86_400}})
{2000, 1, 1, 12, 0, 0, {0, 6}}
iex> Calendar.ISO.naive_datetime_from_iso_days({-365, {0,
86_400_000_000}})
{-1, 1, 1, 0, 0, 0, {0, 6}}
```

naive_datetime_to_iodata(year, month, day, hour, minute, second, microsecond, format \\ :extended)

(since 1.19.0)

```
@spec naive_datetime_to_iodata(
  year(),
  month(),
  day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  :basic | :extended
) :: iodata()
```

Converts the given naive_datetime into a iodata.

See `naive_datetime_to_iodata/8` for more information.

Examples

```
iex> data = Calendar.ISO.naive_datetime_to_iodata(2015,
2, 28, 1, 2, 3, {4, 6}, :basic)
iex> IO.iodata_to_binary(data)
"20150228 010203.000004"

iex> data = Calendar.ISO.naive_datetime_to_iodata(2015,
2, 28, 1, 2, 3, {4, 6}, :extended)
iex> IO.iodata_to_binary(data)
"2015-02-28 01:02:03.000004"
```

naive_datetime_to_iso_days(year, month, day, hour, minute, second, microsecond)

(since 1.5.0)

```
@spec naive_datetime_to_iso_days(
  Calendar.year(),
  Calendar.month(),
  Calendar.day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond()
) :: Calendar.iso_days()
```

Returns the `Calendar.iso_days/0` format of the specified date.

Examples

```
iex> Calendar.ISO.naive_datetime_to_iso_days(0, 1, 1, 0, 0, 0, {0, 6})
{0, {0, 86400000000}}
iex> Calendar.ISO.naive_datetime_to_iso_days(2000, 1, 1, 12, 0, 0, {0, 6})
{730485, {43200000000, 86400000000}}
iex> Calendar.ISO.naive_datetime_to_iso_days(2000, 1, 1, 13, 0, 0, {0, 6})
{730485, {46800000000, 86400000000}}
iex> Calendar.ISO.naive_datetime_to_iso_days(-1, 1, 1, 0, 0, 0, {0, 6})
{-365, {0, 86400000000}}
```

naive_datetime_to_string(year, month, day, hour, minute, second, microsecond, format \\ :extended)

(since 1.4.0)

```
@spec naive_datetime_to_string(
  year(),
  month(),
  day(),
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  :basic | :extended
) :: String.t()
```

Converts the datetime (without time zone) into a string.

By default, returns datetimes formatted in the "extended" format, for human readability. It also supports the "basic" format by passing the `:basic` option.

Examples

```
iex> Calendar.ISO.naive_datetime_to_string(2015, 2, 28, 1, 2, 3, {4, 6})
"2015-02-28 01:02:03.000004"
iex> Calendar.ISO.naive_datetime_to_string(2017, 8, 1, 1, 2, 3, {4, 5})
"2017-08-01 01:02:03.00000"

iex> Calendar.ISO.naive_datetime_to_string(2015, 2, 28,
```

```
  1, 2, 3, {4, 6}, :basic)
  "20150228 010203.000004"
```

parse_date(string)

(since 1.10.0)

```
@spec parse_date(String.t()) :: {:ok, {year(), month(),
day()}} | {:error, atom()}
```

Parses a date `string` in the `:extended` format.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_date("2015-01-23")
{:ok, {2015, 1, 23}}

iex> Calendar.ISO.parse_date("2015:01:23")
{:error, :invalid_format}
iex> Calendar.ISO.parse_date("2015-01-32")
{:error, :invalid_date}
```

parse_date(string, format)

(since 1.12.0)

```
@spec parse_date(String.t(), format()) ::
  {:ok, {year(), month(), day()}} | {:error, atom()}
```

Parses a date `string` according to a given `format`.

The `format` can either be `:basic` or `:extended`.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_date("20150123", :basic)
{:ok, {2015, 1, 23}}
iex> Calendar.ISO.parse_date("20150123", :extended)
{:error, :invalid_format}
```

parse_duration(arg1)

(since 1.17.0)

```
@spec parse_duration(String.t()) :: {:ok,
[Duration.unit_pair()]} | {:error, atom()}
```

Parses an ISO 8601 formatted duration string to a list of `Duration` compabitble unit pairs.

See `Duration.from_iso8601/1`.

parse_naive_datetime(string)

(since 1.10.0)

```
@spec parse_naive_datetime(String.t()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
```

```
microsecond()}}
  | {:error, atom()}
```

Parses a naive datetime `string` in the `:extended` format.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07Z")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07-02:30")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}

iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07.0")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 1}}}
iex> Calendar.ISO.parse_naive_datetime("2015-01-23
23:50:07,0123456")
{:ok, {2015, 1, 23, 23, 50, 7, {12345, 6}}}
```

parse_naive_datetime(string, format)

(since 1.12.0)

```
@spec parse_naive_datetime(String.t(), format()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
```

```
microsecond()}}
  | {:error, atom()}
```

Parses a naive datetime `string` according to a given `format`.

The `format` can either be `:basic` or `:extended`.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_naive_datetime("20150123 235007",
:basic)
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_naive_datetime("20150123 235007",
:extended)
{:error, :invalid_format}
```

parse_time(string)

(since 1.10.0)

```
@spec parse_time(String.t()) ::
  {:ok, {hour(), minute(), second(), microsecond()}} |
{:error, atom()}
```

Parses a time `string` in the `:extended` format.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_time("23:50:07")
{:ok, {23, 50, 7, {0, 0}}}

iex> Calendar.ISO.parse_time("23:50:07Z")
{:ok, {23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_time("T23:50:07Z")
{:ok, {23, 50, 7, {0, 0}}}
```

parse_time(string, format)

(since 1.12.0)

```
@spec parse_time(String.t(), format()) ::
  {:ok, {hour(), minute(), second(), microsecond()}} |
{:error, atom()}
```

Parses a time `string` according to a given `format`.

The `format` can either be `:basic` or `:extended`.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_time("235007", :basic)
{:ok, {23, 50, 7, {0, 0}}}
iex> Calendar.ISO.parse_time("235007", :extended)
{:error, :invalid_format}
```

parse_utc_datetime(string)

(since 1.10.0)

```
@spec parse_utc_datetime(String.t()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
microsecond()},
   utc_offset()}
  | {:error, atom()}
```

Parses a UTC datetime `string` in the `:extended` format.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_utc_datetime("2015-01-23
23:50:07Z")
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}, 0}

iex> Calendar.ISO.parse_utc_datetime("2015-01-23
23:50:07+02:30")
{:ok, {2015, 1, 23, 21, 20, 7, {0, 0}}, 9000}

iex> Calendar.ISO.parse_utc_datetime("2015-01-23
23:50:07")
{:error, :missing_offset}
```

parse_utc_datetime(string, format)

(since 1.12.0)

```
@spec parse_utc_datetime(String.t(), format()) ::
  {:ok, {year(), month(), day(), hour(), minute(), second(),
microsecond()},
```

```
   utc_offset()}
  | {:error, atom()}
```

Parses a UTC datetime `string` according to a given `format`.

The `format` can either be `:basic` or `:extended`.

For more information on supported strings, see how this module implements ISO 8601.

Examples

```
iex> Calendar.ISO.parse_utc_datetime("20150123 235007Z",
:basic)
{:ok, {2015, 1, 23, 23, 50, 7, {0, 0}}, 0}
iex> Calendar.ISO.parse_utc_datetime("20150123 235007Z",
:extended)
{:error, :invalid_format}
```

quarter_of_year(year, month, day)

(since 1.8.0)

```
@spec quarter_of_year(year(), month(), day()) ::
quarter_of_year()
```

Calculates the quarter of the year from the given `year`, `month`, and `day`.

It is an integer from 1 to 4.

Examples

```
iex> Calendar.ISO.quarter_of_year(2016, 1, 31)
1
iex> Calendar.ISO.quarter_of_year(2016, 4, 3)
2
iex> Calendar.ISO.quarter_of_year(-99, 9, 31)
3
iex> Calendar.ISO.quarter_of_year(2018, 12, 28)
4
```

shift_date(year, month, day, duration)

```
@spec shift_date(year(), month(), day(), Duration.t()) ::
{year(), month(), day()}
```

Shifts Date by Duration according to its calendar.

Examples

```
iex> Calendar.ISO.shift_date(2016, 1, 3, Duration.new!
(month: 2))
{2016, 3, 3}
iex> Calendar.ISO.shift_date(2016, 2, 29, Duration.new!
(month: 1))
{2016, 3, 29}
iex> Calendar.ISO.shift_date(2016, 1, 31, Duration.new!
(month: 1))
{2016, 2, 29}
iex> Calendar.ISO.shift_date(2016, 1, 31, Duration.new!
(year: 4, day: 1))
{2020, 2, 1}
```

shift_naive_datetime(year, month, day, hour, minute, second, microsecond, duration)

```
@spec shift_naive_datetime(
  year(),
  month(),
  day(),
  hour(),
  minute(),
  second(),
  microsecond(),
  Duration.t()
) :: {year(), month(), day(), hour(), minute(), second(),
microsecond()}
```

Shifts NaiveDateTime by Duration according to its calendar.

Examples

```
iex> Calendar.ISO.shift_naive_datetime(2016, 1, 3, 0, 0,
0, {0, 0}, Duration.new!(hour: 1))
{2016, 1, 3, 1, 0, 0, {0, 0}}
iex> Calendar.ISO.shift_naive_datetime(2016, 1, 3, 0, 0,
0, {0, 0}, Duration.new!(hour: 30))
{2016, 1, 4, 6, 0, 0, {0, 0}}
iex> Calendar.ISO.shift_naive_datetime(2016, 1, 3, 0, 0,
0, {0, 0}, Duration.new!(microsecond: {100, 6}))
{2016, 1, 3, 0, 0, 0, {100, 6}}
```

shift_time(hour, minute, second, microsecond, duration)

```
@spec shift_time(hour(), minute(), second(), microsecond(),
Duration.t()) ::
  {hour(), minute(), second(), microsecond()}
```

Shifts Time by Duration units according to its calendar.

Examples

```
iex> Calendar.ISO.shift_time(13, 0, 0, {0, 0},
Duration.new!(hour: 2))
{15, 0, 0, {0, 0}}
iex> Calendar.ISO.shift_time(13, 0, 0, {0, 0},
Duration.new!(microsecond: {100, 6}))
{13, 0, 0, {100, 6}}
```

time_from_day_fraction(arg)

(since 1.5.0)

```
@spec time_from_day_fraction(Calendar.day_fraction()) ::
  {hour(), minute(), second(), microsecond()}
```

Converts a day fraction to this Calendar's representation of time.

Examples

```
iex> Calendar.ISO.time_from_day_fraction({1, 2})
{12, 0, 0, {0, 6}}
iex> Calendar.ISO.time_from_day_fraction({13, 24})
{13, 0, 0, {0, 6}}
```

time_to_day_fraction(hour, minute, second, arg)

(since 1.5.0)

```
@spec time_to_day_fraction(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond()
) :: Calendar.day_fraction()
```

Returns the normalized day fraction of the specified time.

Examples

```
iex> Calendar.ISO.time_to_day_fraction(0, 0, 0, {0, 6})
{0, 86400000000}
iex> Calendar.ISO.time_to_day_fraction(12, 34, 56, {123,
6})
{45296000123, 86400000000}
```

time_to_iodata(hour, minute, second, microsecond, format \\ :extended)

(since 1.19.0)

```
@spec time_to_iodata(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  :basic | :extended
) :: iodata()
```

Converts the given time into a iodata.

See `time_to_string/5` for more information.

Examples

```
iex> data = Calendar.ISO.time_to_iodata(2, 2, 2, {2, 6})
iex> IO.iodata_to_binary(data)
"02:02:02.000002"
```

time_to_string(hour, minute, second, microsecond, format \\ :extended)

(since 1.5.0)

```
@spec time_to_string(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond(),
  :basic | :extended
) :: String.t()
```

Converts the given time into a string.

By default, returns times formatted in the "extended" format, for human readability. It also supports the "basic" format by passing the `:basic` option.

Examples

```
iex> Calendar.ISO.time_to_string(2, 2, 2, {2, 6})
"02:02:02.000002"
iex> Calendar.ISO.time_to_string(2, 2, 2, {2, 2})
```

```
"02:02:02.00"
iex> Calendar.ISO.time_to_string(2, 2, 2, {2, 0})
"02:02:02"

iex> Calendar.ISO.time_to_string(2, 2, 2, {2, 6}, :basic)
"020202.000002"
iex> Calendar.ISO.time_to_string(2, 2, 2, {2, 6},
:extended)
"02:02:02.000002"
```

time_unit_to_precision(int)

(since 1.15.0)

```
@spec time_unit_to_precision(System.time_unit()) :: 0..6
```

Converts a `System.time_unit/0` to precision.

Integer-based time units always get maximum precision.

Examples

```
iex> Calendar.ISO.time_unit_to_precision(:nanosecond)
6

iex> Calendar.ISO.time_unit_to_precision(:second)
0

iex> Calendar.ISO.time_unit_to_precision(1)
6
```

valid_date?(year, month, day)

(since 1.5.0)

```
@spec valid_date?(year(), month(), day()) :: boolean()
```

Determines if the date given is valid according to the proleptic Gregorian calendar.

Examples

```
iex> Calendar.ISO.valid_date?(2015, 2, 28)
true
iex> Calendar.ISO.valid_date?(2015, 2, 30)
false
iex> Calendar.ISO.valid_date?(-1, 12, 31)
true
iex> Calendar.ISO.valid_date?(-1, 12, 32)
false
```

valid_time?(hour, minute, second, microsecond)

(since 1.5.0)

```
@spec valid_time?(
  Calendar.hour(),
  Calendar.minute(),
  Calendar.second(),
  Calendar.microsecond()
) ::
  boolean()
```

Determines if the date given is valid according to the proleptic Gregorian calendar.

Leap seconds are not supported by the built-in Calendar.ISO.

Examples

```
iex> Calendar.ISO.valid_time?(10, 50, 25, {3006, 6})
true
iex> Calendar.ISO.valid_time?(23, 59, 60, {0, 0})
false
iex> Calendar.ISO.valid_time?(24, 0, 0, {0, 0})
false
```

year_of_era(year)

(since 1.8.0)

```
@spec year_of_era(year()) :: {1..10000, era()}
```

Calculates the year and era from the given `year`.

The ISO calendar has two eras: the "current era" (CE) which starts in year `1` and is defined as era `1`. And "before the current era" (BCE) for those years less than `1`, defined as era `0`.

Examples

```
iex> Calendar.ISO.year_of_era(1)
{1, 1}
iex> Calendar.ISO.year_of_era(2018)
{2018, 1}
iex> Calendar.ISO.year_of_era(0)
```

```
{1, 0}
iex> Calendar.ISO.year_of_era(-1)
{2, 0}
```

year_of_era(year, month, day)

(since 1.13.0)

```
@spec year_of_era(year(), month(), day()) :: {1..10000, era()}
```

Calendar callback to compute the year and era from the given `year`, `month` and `day`.

In the ISO calendar, the new year coincides with the new era, so the `month` and `day` arguments are discarded. If you only have the year available, you can `year_of_era/1` instead.

Examples

```
iex> Calendar.ISO.year_of_era(1, 1, 1)
{1, 1}
iex> Calendar.ISO.year_of_era(2018, 12, 1)
{2018, 1}
iex> Calendar.ISO.year_of_era(0, 1, 1)
{1, 0}
iex> Calendar.ISO.year_of_era(-1, 12, 1)
{2, 0}
```

Calendar.TimeZoneDatabase behaviour

This module defines a behaviour for providing time zone data.

IANA provides time zone data that includes data about different UTC offsets and standard offsets for time zones.

Summary

Types

time_zone_period()

A period where a certain combination of UTC offset, standard offset, and zone abbreviation is in effect.

time_zone_period_limit()

Limit for when a certain time zone period begins or ends.

Callbacks

time_zone_period_from_utc_iso_days(iso_days, time_zone)

Time zone period for a point in time in UTC for a specific time zone.

time_zone_periods_from_wall_datetime(naive_datetime, time_zone)

Possible time zone periods for a certain time zone and wall clock date and time.

Types

time_zone_period()

```
@type time_zone_period() :: %{
  optional(any()) => any(),
  utc_offset: Calendar.utc_offset(),
  std_offset: Calendar.std_offset(),
  zone_abbr: Calendar.zone_abbr()
}
```

A period where a certain combination of UTC offset, standard offset, and zone abbreviation is in effect.

For example, one period could be the summer of 2018 in the `Europe/London` timezone, where summer time/daylight saving time is in effect and lasts from spring to autumn. In autumn, the `std_offset` changes along with the `zone_abbr` so a different period is needed during winter.

time_zone_period_limit()

```
@type time_zone_period_limit() :: Calendar.naive_datetime()
```

Limit for when a certain time zone period begins or ends.

A beginning is inclusive. An ending is exclusive. For example, if a period is from `2015-03-29 01:00:00` and until `2015-10-25 01:00:00`, the period includes and begins from the beginning of `2015-03-29 01:00:00` and lasts until just before `2015-10-25 01:00:00`.

A beginning or end for certain periods are infinite, such as the latest period for time zones without DST or plans to change. However, for the purpose of this behaviour, they are only used for gaps in wall time where the needed period limits are at a certain time.

Callbacks

time_zone_period_from_utc_iso_days(iso_days, time_zone)

(since 1.8.0)

```
@callback
time_zone_period_from_utc_iso_days(Calendar.iso_days(),
Calendar.time_zone()) ::
  {:ok, time_zone_period()}
  | {:error, :time_zone_not_found |
:utc_only_time_zone_database}
```

Time zone period for a point in time in UTC for a specific time zone.

Takes a time zone name and a point in time for UTC and returns a `time_zone_period` for that point in time.

time_zone_periods_from_wall_datetime(naive_datetime, time_zone)

(since 1.8.0)

```
@callback time_zone_periods_from_wall_datetime(
  Calendar.naive_datetime(),
  Calendar.time_zone()
) ::
  {:ok, time_zone_period()}
  | {:ambiguous, time_zone_period(), time_zone_period()}
  | {:gap, {time_zone_period(), time_zone_period_limit()},
```

```
     {time_zone_period(), time_zone_period_limit()}}
  | {:error, :time_zone_not_found |
:utc_only_time_zone_database}
```

Possible time zone periods for a certain time zone and wall clock date and time.

When the provided naive datetime is ambiguous, return a tuple with `:ambiguous` and the two possible periods. The periods in the tuple must be sorted with the first element being the one that begins first.

When the provided naive datetime is in a gap, such as during the "spring forward" when going from winter time to summer time, return a tuple with `:gap` and two periods with limits in a nested tuple. The first nested two-tuple is the period before the gap and a naive datetime with a limit for when the period ends (wall time). The second nested two-tuple is the period just after the gap and a datetime (wall time) for when the period begins just after the gap.

If there is only a single possible period for the provided `datetime`, then return a tuple with `:ok` and the `time_zone_period`.

Calendar.UTCOnlyTimeZoneDatabase

Built-in time zone database that works only in the `Etc/UTC` timezone.

For all other time zones, it returns `{:error, :utc_only_time_zone_database}`.

Agent

Agents are a simple abstraction around state.

Often in Elixir there is a need to share or store state that must be accessed from different processes or by the same process at different points in time.

The `Agent` module provides a basic server implementation that allows state to be retrieved and updated via a simple API.

Examples

For example, the following agent implements a counter:

```
defmodule Counter do
  use Agent

  def start_link(initial_value) do
    Agent.start_link(fn -> initial_value end, name:
__MODULE__)
  end

  def value do
    Agent.get(__MODULE__, & &1)
  end

  def increment do
    Agent.update(__MODULE__, &(&1 + 1))
  end
end
```

Usage would be:

```
Counter.start_link(0)
#=> {:ok, #PID<0.123.0>}

Counter.value()
#=> 0

Counter.increment()
#=> :ok

Counter.increment()
#=> :ok

Counter.value()
#=> 2
```

Thanks to the agent server process, the counter can be safely incremented concurrently.

> `use Agent`
>
> When you `use Agent`, the `Agent` module will define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

Agents provide a segregation between the client and server APIs (similar to `GenServer`s). In particular, the functions passed as arguments to the calls to `Agent` functions are invoked inside the agent (the server). This distinction is important because you may want to avoid expensive operations inside the agent, as they will effectively block the agent until the request is fulfilled.

Consider these two examples:

```
# Compute in the agent/server
def get_something(agent) do
  Agent.get(agent, fn state ->
do_something_expensive(state) end)
end

# Compute in the agent/client
def get_something(agent) do
  Agent.get(agent, & &1) |> do_something_expensive()
end
```

The first function blocks the agent. The second function copies all the state to the client and then executes the operation in the client. One aspect to consider is whether the data is large enough to require processing in the server, at least initially, or small enough to be sent to the client cheaply. Another factor is whether the data needs to be processed atomically: getting the state and calling `do_something_expensive(state)` outside of the agent means that the agent's state can be updated in the meantime. This is specially important in case of updates as computing the new state in the client rather than in the server can lead to race conditions if multiple clients are trying to update the same state to different values.

How to supervise

An `Agent` is most commonly started under a supervision tree. When we invoke `use Agent`, it automatically defines a `child_spec/1` function that allows us to start the agent directly under a supervisor. To start an agent under a supervisor with an initial counter of 0, one may do:

```
children = [
  {Counter, 0}
]
```

```
Supervisor.start_link(children, strategy: :one_for_all)
```

While one could also simply pass the `Counter` as a child to the supervisor, such as:

```
children = [
  Counter # Same as {Counter, []}
]

Supervisor.start_link(children, strategy: :one_for_all)
```

The definition above wouldn't work for this particular example, as it would attempt to start the counter with an initial value of an empty list. However, this may be a viable option in your own agents. A common approach is to use a keyword list, as that would allow setting the initial value and giving a name to the counter process, for example:

```
def start_link(opts) do
  {initial_value, opts} = Keyword.pop(opts,
:initial_value, 0)
  Agent.start_link(fn -> initial_value end, opts)
end
```

and then you can use `Counter`, `{Counter, name: :my_counter}` or even `{Counter, initial_value: 0, name: :my_counter}` as a child specification.

`use Agent` also accepts a list of options which configures the child specification and therefore how it runs under a supervisor. The generated `child_spec/1` can be customized with the following options:

- `:id` - the child specification identifier, defaults to the current module
- `:restart` - when the child should be restarted, defaults to `:permanent`
- `:shutdown` - how to shut down the child, either immediately or by giving it time to shut down

For example:

```
use Agent, restart: :transient, shutdown: 10_000
```

See the "Child specification" section in the `Supervisor` module for more detailed information. The `@doc` annotation immediately preceding `use Agent` will be attached to the generated `child_spec/1` function.

Name registration

An agent is bound to the same name registration rules as GenServers. Read more about it in the `GenServer` documentation.

A word on distributed agents

It is important to consider the limitations of distributed agents. Agents provide two APIs, one that works with anonymous functions and another that expects an explicit module, function, and arguments.

In a distributed setup with multiple nodes, the API that accepts anonymous functions only works if the caller (client) and the agent have the same version of the caller module.

Keep in mind this issue also shows up when performing "rolling upgrades" with agents. By rolling upgrades we mean the following situation: you wish to deploy a new version of your software by *shutting*

down some of your nodes and replacing them with nodes running a new version of the software. In this setup, part of your environment will have one version of a given module and the other part another version (the newer one) of the same module.

The best solution is to simply use the explicit module, function, and arguments APIs when working with distributed agents.

Hot code swapping

An agent can have its code hot swapped live by simply passing a module, function, and arguments tuple to the update instruction. For example, imagine you have an agent named `:sample` and you want to convert its inner state from a keyword list to a map. It can be done with the following instruction:

```
{:update, :sample, {:advanced, {Enum, :into, [%{}]}}}
```

The agent's state will be added to the given list of arguments (`[%{}]`) as the first argument.

Summary

Types

agent()

The agent reference

name()

The agent name

on_start()

Return values of `start*` functions

state()

The agent state

Functions

cast(agent, fun)

Performs a cast (*fire and forget*) operation on the agent state.

cast(agent, module, fun, args)

Performs a cast (*fire and forget*) operation on the agent state.

child_spec(arg)

Returns a specification to start an agent under a supervisor.

get(agent, fun, timeout \\ 5000)

Gets an agent value via the given anonymous function.

get(agent, module, fun, args, timeout \\ 5000)

Gets an agent value via the given function.

get_and_update(agent, fun, timeout \\ 5000)

Gets and updates the agent state in one operation via the given anonymous function.

get_and_update(agent, module, fun, args, timeout \\ 5000)

Gets and updates the agent state in one operation via the given function.

start(fun, options \\ [])

Starts an agent process without links (outside of a supervision tree).

start(module, fun, args, options \\ [])

Starts an agent without links with the given module, function, and arguments.

start_link(fun, options \\ [])

Starts an agent linked to the current process with the given function.

start_link(module, fun, args, options \\ [])

Starts an agent linked to the current process.

stop(agent, reason \\ :normal, timeout \\ :infinity)

Synchronously stops the agent with the given `reason`.

update(agent, fun, timeout \\ 5000)

Updates the agent state via the given anonymous function.

update(agent, module, fun, args, timeout \\ 5000)

Updates the agent state via the given function.

Types

agent()

```
@type agent() :: pid() | {atom(), node()} | name()
```

The agent reference

name()

```
@type name() :: atom() | {:global, term()} | {:via, module(),
term()}
```

The agent name

on_start()

```
@type on_start() :: {:ok, pid()} | {:error,
{:already_started, pid()} | term()}
```

Return values of `start*` functions

state()

```
@type state() :: term()
```

The agent state

Functions

cast(agent, fun)

```
@spec cast(agent(), (state() -> state())) :: :ok
```

Performs a cast (*fire and forget*) operation on the agent state.

The function `fun` is sent to the `agent` which invokes the function passing the agent state. The return value of `fun` becomes the new state of the agent.

Note that `cast` returns `:ok` immediately, regardless of whether `agent` (or the node it should live on) exists.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.cast(pid, fn state -> state + 1 end)
:ok
iex> Agent.get(pid, fn state -> state end)
43
```

cast(agent, module, fun, args)

```
@spec cast(agent(), module(), atom(), [term()]) :: :ok
```

Performs a cast (*fire and forget*) operation on the agent state.

Same as `cast/2` but a module, function, and arguments are expected instead of an anonymous function. The state is added as first argument to the given list of arguments.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.cast(pid, Kernel, :+, [12])
:ok
```

```
iex> Agent.get(pid, fn state -> state end)
54
```

child_spec(arg)

(since 1.5.0)

Returns a specification to start an agent under a supervisor.

See the "Child specification" section in the `Supervisor` module for more detailed information.

get(agent, fun, timeout \\ 5000)

```
@spec get(agent(), (state() -> a), timeout()) :: a when a: var
```

Gets an agent value via the given anonymous function.

The function `fun` is sent to the `agent` which invokes the function passing the agent state. The result of the function invocation is returned from this function.

`timeout` is an integer greater than zero which specifies how many milliseconds are allowed before the agent executes the function and returns the result value, or the atom `:infinity` to wait indefinitely. If no result is received within the specified time, the function call fails and the caller exits.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.get(pid, fn state -> state end)
42
```

get(agent, module, fun, args, timeout \\ 5000)

```
@spec get(agent(), module(), atom(), [term()], timeout()) ::
term()
```

Gets an agent value via the given function.

Same as `get/3` but a module, function, and arguments are expected instead of an anonymous function. The state is added as first argument to the given list of arguments.

get_and_update(agent, fun, timeout \\ 5000)

```
@spec get_and_update(agent(), (state() -> {a, state()}),
timeout()) :: a when a: var
```

Gets and updates the agent state in one operation via the given anonymous function.

The function `fun` is sent to the `agent` which invokes the function passing the agent state. The function must return a tuple with two elements, the first being the value to return (that is, the "get" value) and the second one being the new state of the agent.

`timeout` is an integer greater than zero which specifies how many milliseconds are allowed before the agent executes the function and returns the result value, or the atom `:infinity` to wait indefinitely. If no

result is received within the specified time, the function call fails and the caller exits.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.get_and_update(pid, fn state -> {state, state
+ 1} end)
42
iex> Agent.get(pid, fn state -> state end)
43
```

get_and_update(agent, module, fun, args, timeout \\ 5000)

```
@spec get_and_update(agent(), module(), atom(), [term()],
timeout()) :: term()
```

Gets and updates the agent state in one operation via the given function.

Same as `get_and_update/3` but a module, function, and arguments are expected instead of an anonymous function. The state is added as first argument to the given list of arguments.

start(fun, options \\ [])

```
@spec start((-> term()), GenServer.options()) :: on_start()
```

Starts an agent process without links (outside of a supervision tree).

See `start_link/2` for more information.

Examples

```
iex> {:ok, pid} = Agent.start(fn -> 42 end)
iex> Agent.get(pid, fn state -> state end)
42
```

start(module, fun, args, options \\ [])

```
@spec start(module(), atom(), [term()], GenServer.options())
:: on_start()
```

Starts an agent without links with the given module, function, and arguments.

See `start_link/4` for more information.

start_link(fun, options \\ [])

```
@spec start_link((-> term()), GenServer.options()) ::
on_start()
```

Starts an agent linked to the current process with the given function.

This is often used to start the agent as part of a supervision tree.

Once the agent is spawned, the given function `fun` is invoked in the server process, and should return the initial agent state. Note that `start_link/2` does not return until the given function has returned.

Options

The `:name` option is used for registration as described in the module documentation.

If the `:timeout` option is present, the agent is allowed to spend at most the given number of milliseconds on initialization or it will be terminated and the start function will return `{:error, :timeout}`.

If the `:debug` option is present, the corresponding function in the `:sys` module will be invoked.

If the `:spawn_opt` option is present, its value will be passed as options to the underlying process as in `Process.spawn/4`.

Return values

If the server is successfully created and initialized, the function returns `{:ok, pid}`, where `pid` is the PID of the server. If an agent with the specified name already exists, the function returns `{:error, {:already_started, pid}}` with the PID of that process.

If the given function callback fails, the function returns `{:error, reason}`.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.get(pid, fn state -> state end)
42

iex> {:error, {exception, _stacktrace}} = Agent.start(fn
-> raise "oops" end)
```

```
iex> exception
%RuntimeError{message: "oops"}
```

start_link(module, fun, args, options \\ [])

```
@spec start_link(module(), atom(), [term()],
GenServer.options()) :: on_start()
```

Starts an agent linked to the current process.

Same as `start_link/2` but a module, function, and arguments are expected instead of an anonymous function; `fun` in `module` will be called with the given arguments `args` to initialize the state.

stop(agent, reason \\ :normal, timeout \\ :infinity)

```
@spec stop(agent(), reason :: term(), timeout()) :: :ok
```

Synchronously stops the agent with the given `reason`.

It returns `:ok` if the agent terminates with the given reason. If the agent terminates with another reason, the call will exit.

This function keeps OTP semantics regarding error reporting. If the reason is any other than `:normal`, `:shutdown` or `{:shutdown, _}`, an error report will be logged.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.stop(pid)
:ok
```

update(agent, fun, timeout \\ 5000)

```
@spec update(agent(), (state() -> state()), timeout()) :: :ok
```

Updates the agent state via the given anonymous function.

The function `fun` is sent to the `agent` which invokes the function passing the agent state. The return value of `fun` becomes the new state of the agent.

This function always returns `:ok`.

`timeout` is an integer greater than zero which specifies how many milliseconds are allowed before the agent executes the function and returns the result value, or the atom `:infinity` to wait indefinitely. If no result is received within the specified time, the function call fails and the caller exits.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.update(pid, fn state -> state + 1 end)
:ok
iex> Agent.get(pid, fn state -> state end)
43
```

update(agent, module, fun, args, timeout \\ 5000)

```
@spec update(agent(), module(), atom(), [term()], timeout())
:: :ok
```

Updates the agent state via the given function.

Same as `update/3` but a module, function, and arguments are expected instead of an anonymous function. The state is added as first argument to the given list of arguments.

Examples

```
iex> {:ok, pid} = Agent.start_link(fn -> 42 end)
iex> Agent.update(pid, Kernel, :+, [12])
:ok
iex> Agent.get(pid, fn state -> state end)
54
```

Application behaviour

A module for working with applications and defining application callbacks.

Applications are the idiomatic way to package software in Erlang/OTP. To get the idea, they are similar to the "library" concept common in other programming languages, but with some additional characteristics.

An application is a component implementing some specific functionality, with a standardized directory structure, configuration, and life cycle. Applications are *loaded*, *started*, and *stopped*. Each application also has its own environment, which provides a unified API for configuring each application.

Developers typically interact with the application environment and its callback module. Therefore those will be the topics we will cover first before jumping into details about the application resource file and life cycle.

The application environment

Each application has its own environment. The environment is a keyword list that maps atoms to terms. Note that this environment is unrelated to the operating system environment.

By default, the environment of an application is an empty list. In a Mix project's `mix.exs` file, you can set the `:env` key in `application/0`:

```
def application do
  [env: [db_host: "localhost"]]
end
```

Now, in your application, you can read this environment by using functions such as `fetch_env!/2` and friends:

```
defmodule MyApp.DBClient do
  def start_link() do
    SomeLib.DBClient.start_link(host: db_host())
  end

  defp db_host do
    Application.fetch_env!(:my_app, :db_host)
  end
end
```

In Mix projects, the environment of the application and its dependencies can be overridden via the `config/config.exs` and `config/runtime.exs` files. The former is loaded at build-time, before your code compiles, and the latter at runtime, just before your app starts. For example, someone using your application can override its `:db_host` environment variable as follows:

```
import Config
config :my_app, :db_host, "db.local"
```

See the "Configuration" section in the `Mix` module for more information. You can also change the application environment dynamically by using functions such as `put_env/3` and `delete_env/2`.

> **Application environment in libraries**
>
> If you are writing a library to be used by other developers, it is generally recommended to avoid the application environment, as the application environment is effectively a global storage. For more information, read about this anti-pattern.

> **Reading the environment of other applications**
>
> Each application is responsible for its own environment. Do not use the functions in this module for directly accessing or modifying the environment of other applications. Whenever you change the application environment, Elixir's build tool will only recompile the files that belong to that application. So if you read the application environment of another application, there is a chance you will be depending on outdated configuration, as your file won't be recompiled as it changes.

Compile-time environment

In the previous example, we read the application environment at runtime:

```
defmodule MyApp.DBClient do
  def start_link() do
    SomeLib.DBClient.start_link(host: db_host())
  end

  defp db_host do
    Application.fetch_env!(:my_app, :db_host)
  end
end
```

In other words, the environment key `:db_host` for application `:my_app` will only be read when `MyApp.DBClient` effectively starts. While reading the application environment at runtime is the preferred approach, in some rare occasions you may want to use the application environment to configure the compilation of a certain project. However, if you try to access `Application.fetch_env!/2` outside of a function:

```
defmodule MyApp.DBClient do
  @db_host Application.fetch_env!(:my_app, :db_host)
```

```
  def start_link() do
    SomeLib.DBClient.start_link(host: @db_host)
  end
end
```

You might see warnings and errors:

```
warning: Application.fetch_env!/2 is discouraged in the
module body,
use Application.compile_env/3 instead
  iex:3: MyApp.DBClient

** (ArgumentError) could not fetch application
environment :db_host
for application :my_app because the application was not
loaded nor
configured
```

This happens because, when defining modules, the application environment is not yet available. Luckily, the warning tells us how to solve this issue, by using `Application.compile_env/3` instead:

```
defmodule MyApp.DBClient do
  @db_host Application.compile_env(:my_app, :db_host,
"db.local")

  def start_link() do
    SomeLib.DBClient.start_link(host: @db_host)
  end
end
```

The difference here is that `compile_env` expects the default value to be given as an argument, instead of using the `def application` function of your `mix.exs`. Furthermore, by using `compile_env/3`, tools like Mix will store the values used during compilation and compare the compilation values with the runtime values whenever your system starts, raising an error in case they differ.

In any case, compile-time environments should be avoided. Whenever possible, reading the application environment at runtime should be the first choice.

The application callback module

Applications can be loaded, started, and stopped. Generally, build tools like Mix take care of starting an application and all of its dependencies for you, but you can also do it manually by calling:

```
{:ok, _} = Application.ensure_all_started(:some_app)
```

When an application starts, developers may configure a callback module that executes custom code. Developers use this callback to start the application supervision tree.

The first step to do so is to add a `:mod` key to the `application/0` definition in your `mix.exs` file. It expects a tuple, with the application callback module and start argument (commonly an empty list):

```
def application do
  [mod: {MyApp, []}]
end
```

The `MyApp` module given to `:mod` needs to implement the `Application` behaviour. This can be done by putting `use Application` in that

module and implementing the `start/2` callback, for example:

```
defmodule MyApp do
  use Application

  def start(_type, _args) do
    children = []
    Supervisor.start_link(children, strategy:
:one_for_one)
  end
end
```

> `use Application`
>
> When you `use Application`, the `Application` module will set `@behaviour Application` and define an overridable definition for the `stop/1` function, which is required by Erlang/OTP.

The `start/2` callback has to spawn and link a supervisor and return `{:ok, pid}` or `{:ok, pid, state}`, where `pid` is the PID of the supervisor, and `state` is an optional application state. `args` is the second element of the tuple given to the `:mod` option.

The `type` argument passed to `start/2` is usually `:normal` unless in a distributed setup where application takeovers and failovers are configured. Distributed applications are beyond the scope of this documentation.

When an application is shutting down, its `stop/1` callback is called after the supervision tree has been stopped by the runtime. This callback allows the application to do any final cleanup. The argument is the state returned by `start/2`, if it did, or `[]` otherwise. The return value of `stop/1` is ignored.

By using `Application`, modules get a default implementation of `stop/1` that ignores its argument and returns `:ok`, but it can be overridden.

Application callback modules may also implement the optional callback `prep_stop/1`. If present, `prep_stop/1` is invoked before the supervision tree is terminated. Its argument is the state returned by `start/2`, if it did, or `[]` otherwise, and its return value is passed to `stop/1`.

The application resource file

In the sections above, we have configured an application in the `application/0` section of the `mix.exs` file. Ultimately, Mix will use this configuration to create an *application resource file*, which is a file called `APP_NAME.app`. For example, the application resource file of the OTP application `ex_unit` is called `ex_unit.app`.

You can learn more about the generation of application resource files in the documentation of `Mix.Tasks.Compile.App`, available as well by running `mix help compile.app`.

The application life cycle

Loading applications

Applications are *loaded*, which means that the runtime finds and processes their resource files:

```
Application.load(:ex_unit)
#=> :ok
```

When an application is loaded, the environment specified in its resource file is merged with any overrides from config files.

Loading an application *does not* load its modules.

In practice, you rarely load applications by hand because that is part of the start process, explained next.

Starting applications

Applications are also *started*:

```
Application.start(:ex_unit)
#=> :ok
```

Once your application is compiled, running your system is a matter of starting your current application and its dependencies. Differently from other languages, Elixir does not have a `main` procedure that is responsible for starting your system. Instead, you start one or more applications, each with their own initialization and termination logic.

When an application is started, the `Application.load/1` is automatically invoked if it hasn't been done yet. Then, it checks if the dependencies listed in the `applications` key of the resource file are already started. Having at least one dependency not started is an error condition. Functions like `ensure_all_started/1` take care of starting an application and all of its dependencies for you.

If the application does not have a callback module configured, starting is done at this point. Otherwise, its `start/2` callback is invoked. The PID of the top-level supervisor returned by this function is stored by the runtime for later use, and the returned application state is saved too, if any.

Stopping applications

Started applications are, finally, *stopped*:

```
Application.stop(:ex_unit)
#=> :ok
```

Stopping an application without a callback module defined, is in practice a no-op, except for some system tracing.

Stopping an application with a callback module has three steps:

1. If present, invoke the optional callback `prep_stop/1`.
2. Terminate the top-level supervisor.
3. Invoke the required callback `stop/1`.

The arguments passed to the callbacks are related to the state optionally returned by `start/2`, and are documented in the section about the callback module above.

It is important to highlight that step 2 is a blocking one. Termination of a supervisor triggers a recursive chain of children terminations, therefore orderly shutting down all descendant processes. The `stop/1` callback is invoked only after termination of the whole supervision tree.

Shutting down a live system cleanly can be done by calling `System.stop/1`. It will shut down every application in the reverse order they were started.

By default, a SIGTERM from the operating system will automatically translate to `System.stop/0`. You can also have more explicit control over operating system signals via the `:os.set_signal/2` function.

Tooling

The Mix build tool automates most of the application management tasks. For example, `mix test` automatically starts your application dependencies and your application itself before your test runs. `mix run --no-halt` boots your current project and can be used to start a long running system. See `mix help run`.

Developers can also use `mix release` to build **releases**. Releases are able to package all of your source code as well as the Erlang VM into a single directory. Releases also give you explicit control over how each application is started and in which order. They also provide a more streamlined mechanism for starting and stopping systems, debugging, logging, as well as system monitoring.

Finally, Elixir provides tools such as escripts and archives, which are different mechanisms for packaging your application. Those are typically used when tools must be shared between developers and not as deployment options. See `mix help archive.build` and `mix help escript.build` for more detail.

Further information

For further details on applications please check the documentation of the `:application` Erlang module, and the Applications section of the OTP Design Principles User's Guide.

Summary

Types

app()
application_key()
key()

restart_type()

Specifies the type of the application

start_type()
state()
value()

Callbacks

config_change(changed, new, removed)

Callback invoked after code upgrade, if the application environment has changed.

prep_stop(state)

Called before stopping the application.

start(start_type, start_args)

Called when an application is started.

start_phase(phase, start_type, phase_args)

Starts an application in synchronous phases.

stop(state)

Called after an application has been stopped.

Functions

app_dir(app)

Gets the directory for app.

app_dir(app, path)

Returns the given path inside `app_dir/1`.

compile_env(app, key_or_path, default \\ nil)

Reads the application environment at compilation time.

compile_env(env, app, key_or_path, default)

Reads the application environment at compilation time from a macro.

compile_env!(app, key_or_path)

Reads the application environment at compilation time or raises.

compile_env!(env, app, key_or_path)

Reads the application environment at compilation time from a macro or raises.

delete_env(app, key, opts \\ [])

Deletes the `key` from the given `app` environment.

ensure_all_started(app_or_apps, type_or_opts \\ [])

Ensures the given `app` or `apps` and their child applications are started.

ensure_loaded(app)

Ensures the given `app` is loaded.

ensure_started(app, type \\ :temporary)

Ensures the given `app` is started with `restart_type/0`.

fetch_env(app, key)

Returns the value for `key` in `app`'s environment in a tuple.

fetch_env!(app, key)

Returns the value for `key` in `app`'s environment.

format_error(reason)

Formats the error reason returned by `start/2`, `ensure_started/2`, `stop/1`, `load/1` and `unload/1`, returns a string.

get_all_env(app)

Returns all key-value pairs for `app`.

get_application(module)

Gets the application for the given module.

get_env(app, key, default \\ nil)

Returns the value for `key` in `app`'s environment.

load(app)

Loads the given `app`.

loaded_applications()

Returns a list with information about the applications which have been loaded.

put_all_env(config, opts \\ [])

Puts the environment for multiple applications at the same time.

put_env(app, key, value, opts \\ [])

Puts the `value` in `key` for the given `app`.

spec(app)

Returns the spec for `app`.

spec(app, key)

Returns the value for `key` in `app`'s specification.

start(app, type \\ :temporary)

Starts the given `app` with `restart_type/0`.

started_applications(timeout \\ 5000)

Returns a list with information about the applications which are currently running.

stop(app)

Stops the given `app`.

unload(app)

Unloads the given `app`.

Types

app()

```
@type app() :: atom()
```

application_key()

```
@type application_key() ::
  :start_phases
  | :mod
  | :applications
  | :optional_applications
  | :included_applications
  | :registered
  | :maxT
  | :maxP
  | :modules
  | :vsn
  | :id
  | :description
```

key()

```
@type key() :: atom()
```

restart_type()

```
@type restart_type() :: :permanent | :transient | :temporary
```

Specifies the type of the application:

- `:permanent` - if `app` terminates, all other applications and the entire node are also terminated.
- `:transient` - if `app` terminates with `:normal` reason, it is reported but no other applications are terminated. If a transient application terminates abnormally, all other applications and the entire node are also terminated.

- `:temporary` - if `app` terminates, it is reported but no other applications are terminated (the default).

Note that it is always possible to stop an application explicitly by calling `stop/1`. Regardless of the type of the application, no other applications will be affected.

Note also that the `:transient` type is of little practical use, since when a supervision tree terminates, the reason is set to `:shutdown`, not `:normal`.

start_type()

```
@type start_type() :: :normal | {:takeover, node()} |
{:failover, node()}
```

state()

```
@type state() :: term()
```

value()

```
@type value() :: term()
```

Callbacks

config_change(changed, new, removed)

(optional)

```
@callback config_change(changed, new, removed) :: :ok
when changed: keyword(), new: keyword(), removed: [atom()]
```

Callback invoked after code upgrade, if the application environment has changed.

`changed` is a keyword list of keys and their changed values in the application environment. `new` is a keyword list with all new keys and their values. `removed` is a list with all removed keys.

prep_stop(state)

(optional)

```
@callback prep_stop(state()) :: state()
```

Called before stopping the application.

This function is called before the top-level supervisor is terminated. It receives the state returned by `start/2`, if it did, or `[]` otherwise. The return value is later passed to `stop/1`.

start(start_type, start_args)

```
@callback start(start_type(), start_args :: term()) ::
  {:ok, pid()} | {:ok, pid(), state()} | {:error, reason ::
term()}
```

Called when an application is started.

This function is called when an application is started using `Application.start/2` (and functions on top of that, such as `Application.ensure_started/2`). This function should start the top-

level process of the application (which should be the top supervisor of the application's supervision tree if the application follows the OTP design principles around supervision).

`start_type` defines how the application is started:

- `:normal` - used if the startup is a normal startup or if the application is distributed and is started on the current node because of a failover from another node and the application specification key `:start_phases` is `:undefined`.
- `{:takeover, node}` - used if the application is distributed and is started on the current node because of a failover on the node `node`.
- `{:failover, node}` - used if the application is distributed and is started on the current node because of a failover on node `node`, and the application specification key `:start_phases` is not `:undefined`.

`start_args` are the arguments passed to the application in the `:mod` specification key (for example, `mod: {MyApp, [:my_args]}`).

This function should either return `{:ok, pid}` or `{:ok, pid, state}` if startup is successful. `pid` should be the PID of the top supervisor. `state` can be an arbitrary term, and if omitted will default to `[]`; if the application is later stopped, `state` is passed to the `stop/1` callback (see the documentation for the `stop/1` callback for more information).

`use Application` provides no default implementation for the `start/2` callback.

start_phase(phase, start_type, phase_args)

(optional)

```
@callback start_phase(phase :: term(), start_type(),
phase_args :: term()) ::
  :ok | {:error, reason :: term()}
```

Starts an application in synchronous phases.

This function is called after `start/2` finishes but before `Application.start/2` returns. It will be called once for every start phase defined in the application's (and any included applications') specification, in the order they are listed in.

stop(state)

```
@callback stop(state()) :: term()
```

Called after an application has been stopped.

This function is called after an application has been stopped, i.e., after its supervision tree has been stopped. It should do the opposite of what the `start/2` callback did, and should perform any necessary cleanup. The return value of this callback is ignored.

`state` is the state returned by `start/2`, if it did, or `[]` otherwise. If the optional callback `prep_stop/1` is present, `state` is its return value instead.

`use Application` defines a default implementation of this function which does nothing and just returns `:ok`.

Functions

app_dir(app)

```
@spec app_dir(app()) :: String.t()
```

Gets the directory for app.

This information is returned based on the code path. Here is an example:

```
File.mkdir_p!("foo/ebin")
Code.prepend_path("foo/ebin")
Application.app_dir(:foo)
#=> "foo"
```

Even though the directory is empty and there is no `.app` file it is considered the application directory based on the name "foo/ebin". The name may contain a dash `-` which is considered to be the app version and it is removed for the lookup purposes:

```
File.mkdir_p!("bar-123/ebin")
Code.prepend_path("bar-123/ebin")
Application.app_dir(:bar)
#=> "bar-123"
```

For more information on code paths, check the `Code` module in Elixir and also Erlang's `:code` module.

app_dir(app, path)

```
@spec app_dir(app(), String.t() | [String.t()]) :: String.t()
```

Returns the given path inside `app_dir/1`.

If `path` is a string, then it will be used as the path inside `app_dir/1`. If `path` is a list of strings, it will be joined (see `Path.join/1`) and the

result will be used as the path inside `app_dir/1`.

Examples

```
File.mkdir_p!("foo/ebin")
Code.prepend_path("foo/ebin")

Application.app_dir(:foo, "my_path")
#=> "foo/my_path"

Application.app_dir(:foo, ["my", "nested", "path"])
#=> "foo/my/nested/path"
```

compile_env(app, key_or_path, default \\ nil)

(since 1.10.0) (macro)

```
@spec compile_env(app(), key() | list(), value()) :: value()
```

Reads the application environment at compilation time.

Similar to `get_env/3`, except it must be used to read values at compile time. This allows Elixir to track when configuration values change between compile time and runtime.

The first argument is the application name. The second argument `key_or_path` is either an atom key or a path to traverse in search of the configuration, starting with an atom key.

For example, imagine the following configuration:

```
config :my_app, :key, [foo: [bar: :baz]]
```

We can access it during compile time as:

```
Application.compile_env(:my_app, :key)
#=> [foo: [bar: :baz]]

Application.compile_env(:my_app, [:key, :foo])
#=> [bar: :baz]

Application.compile_env(:my_app, [:key, :foo, :bar])
#=> :baz
```

A default value can also be given as third argument. If any of the keys in the path along the way is missing, the default value is used:

```
Application.compile_env(:my_app, [:unknown, :foo, :bar],
:default)
#=> :default

Application.compile_env(:my_app, [:key, :unknown, :bar],
:default)
#=> :default

Application.compile_env(:my_app, [:key, :foo, :unknown],
:default)
#=> :default
```

Giving a path is useful to let Elixir know that only certain paths in a large configuration are compile time dependent.

compile_env(env, app, key_or_path, default)

(since 1.14.0)

```
@spec compile_env(Macro.Env.t(), app(), key() | list(),
value()) :: value()
```

Reads the application environment at compilation time from a macro.

Typically, developers will use `compile_env/3`. This function must only be invoked from macros which aim to read the compilation environment dynamically.

It expects a `Macro.Env` as first argument, where the `Macro.Env` is typically the `__CALLER__` in a macro. It raises if `Macro.Env` comes from a function.

compile_env!(app, key_or_path)

(since 1.10.0) (macro)

```
@spec compile_env!(app(), key() | list()) :: value()
```

Reads the application environment at compilation time or raises.

This is the same as `compile_env/3` but it raises an `ArgumentError` if the configuration is not available.

compile_env!(env, app, key_or_path)

(since 1.14.0)

```
@spec compile_env!(Macro.Env.t(), app(), key() | list()) ::
value()
```

Reads the application environment at compilation time from a macro or raises.

Typically, developers will use `compile_env!/2`. This function must only be invoked from macros which aim to read the compilation environment dynamically.

It expects a `Macro.Env` as first argument, where the `Macro.Env` is typically the `__CALLER__` in a macro. It raises if `Macro.Env` comes from a function.

delete_env(app, key, opts \\ [])

```
@spec delete_env(app(), key(), timeout: timeout(),
persistent: boolean()) :: :ok
```

Deletes the `key` from the given `app` environment.

It receives the same options as `put_env/4`. Returns `:ok`.

ensure_all_started(app_or_apps, type_or_opts \\ [])

```
@spec ensure_all_started(app() | [app()],
  type: restart_type(),
  mode: :serial | :concurrent
) ::
  {:ok, [app()]} | {:error, term()}
@spec ensure_all_started(app() | [app()], restart_type()) ::
  {:ok, [app()]} | {:error, term()}
```

Ensures the given `app` or `apps` and their child applications are started.

The second argument is either the `restart_type/0` (for consistency with `start/2`) or a keyword list.

Options

- `:type` - if the application should be started `:temporary` (default), `:permanent`, or `:transient`. See `restart_type/0` for more information.
- `:mode` - (since v1.15.0) if the applications should be started serially (`:serial`, default) or concurrently (`:concurrent`).

ensure_loaded(app)

(since 1.10.0)

```
@spec ensure_loaded(app()) :: :ok | {:error, term()}
```

Ensures the given `app` is loaded.

Same as `load/1` but returns `:ok` if the application was already loaded.

ensure_started(app, type \\ :temporary)

```
@spec ensure_started(app(), restart_type()) :: :ok | {:error,
term()}
```

Ensures the given `app` is started with `restart_type/0`.

Same as `start/2` but returns `:ok` if the application was already started.

fetch_env(app, key)

```
@spec fetch_env(app(), key()) :: {:ok, value()} | :error
```

Returns the value for `key` in `app`'s environment in a tuple.

If the configuration parameter does not exist, the function returns `:error`.

> **Warning**
>
> You must use this function to read only your own application environment. Do not read the environment of other applications.

Application environment in info

If you are writing a library to be used by other developers, it is generally recommended to avoid the application environment, as the application environment is effectively a global storage. For more information, read our library guidelines.

fetch_env!(app, key)

```
@spec fetch_env!(app(), key()) :: value()
```

Returns the value for `key` in `app`'s environment.

If the configuration parameter does not exist, raises `ArgumentError`.

> **Warning**
>
> You must use this function to read only your own application environment. Do not read the environment of other applications.

Application environment in info

If you are writing a library to be used by other developers, it is generally recommended to avoid the application environment, as the application environment is effectively a global storage. For more information, read our library guidelines.

format_error(reason)

```
@spec format_error(any()) :: String.t()
```

Formats the error reason returned by `start/2`, `ensure_started/2`, `stop/1`, `load/1` and `unload/1`, returns a string.

get_all_env(app)

```
@spec get_all_env(app()) :: [{key(), value()}]
```

Returns all key-value pairs for `app`.

get_application(module)

```
@spec get_application(module()) :: app() | nil
```

Gets the application for the given module.

The application is located by analyzing the spec of all loaded applications. Returns `nil` if the module is not listed in any application spec.

get_env(app, key, default \\ nil)

```
@spec get_env(app(), key(), value()) :: value()
```

Returns the value for `key` in `app`'s environment.

If the configuration parameter does not exist, the function returns the `default` value.

> **Warning**
>
> You must use this function to read only your own application environment. Do not read the environment of other applications.

Examples

`get_env/3` is commonly used to read the configuration of your OTP applications. Since Mix configurations are commonly used to configure applications (including your dependencies), we will use this as a point of illustration.

Consider a new application `:my_app`. `:my_app` contains a database engine which supports a pool of databases. The database engine needs to know the configuration for each of those databases, and that configuration is supplied by key-value pairs in environment of `:my_app`. For example, your `config/runtime.exs` file might have:

```
config :my_app, Databases.RepoOne,
  # A database configuration
  ip: "localhost",
  port: 5433

config :my_app, Databases.RepoTwo,
```

```
  # Another database configuration (for the same OTP app)
  ip: "localhost",
  port: 20_717

config :my_app, my_app_databases: [Databases.RepoOne,
Databases.RepoTwo]
```

Our database engine used by `:my_app` needs to know what databases exist, and what the database configurations are. The database engine can make a call to `Application.get_env(:my_app, :my_app_databases, [])` to retrieve the list of databases (specified by module names).

The engine can then traverse each repository in the list and call `Application.get_env(:my_app, Databases.RepoOne)` and so forth to retrieve the configuration of each one. In this case, each configuration will be a keyword list, so you can use the functions in the `Keyword` module or even the `Access` module to traverse it, for example:

```
config = Application.get_env(:my_app, Databases.RepoOne)
config[:ip]
```

The sample `config/runtime.exs` above could be used both for `:my_app` to configure itself but also to allow any application that depends on `:my_app` to configure how it works. However, one should keep in mind the caveats described in the `Application` module documentation: the application environment is global state which should be avoided if possible.

load(app)

```
@spec load(app()) :: :ok | {:error, term()}
```

Loads the given `app`.

In order to be loaded, an `.app` file must be in the load paths. All `:included_applications` will also be loaded.

Loading the application does not start it nor load its modules, but it does load its environment.

loaded_applications()

```
@spec loaded_applications() :: [{app(), description ::
charlist(), vsn :: charlist()}]
```

Returns a list with information about the applications which have been loaded.

put_all_env(config, opts \\ [])

(since 1.9.0)

```
@spec put_all_env([{app(), [{key(), value()}]}],
  timeout: timeout(),
  persistent: boolean()
) :: :ok
```

Puts the environment for multiple applications at the same time.

The given config should not:

- have the same application listed more than once
- have the same key inside the same application listed more than once

If those conditions are not met, this function will raise.

This function receives the same options as `put_env/4`. Returns `:ok`.

Examples

```
Application.put_all_env(
  my_app: [
    key: :value,
    another_key: :another_value
  ],
  another_app: [
    key: :value
  ]
)
```

put_env(app, key, value, opts \\ [])

```
@spec put_env(app(), key(), value(), timeout: timeout(),
persistent: boolean()) :: :ok
```

Puts the `value` in `key` for the given `app`.

> **Compile environment**
>
> Do not use this function to change environment variables read via `Application.compile_env/2`. The compile environment must be exclusively set before compilation, in your config files.

Options

- `:timeout` - the timeout for the change (defaults to `5_000` milliseconds)

- `:persistent` - persists the given value on application load and reloads

If `put_env/4` is called before the application is loaded, the application environment values specified in the `.app` file will override the ones previously set.

The `:persistent` option can be set to `true` when there is a need to guarantee parameters set with this function will not be overridden by the ones defined in the application resource file on load. This means persistent values will stick after the application is loaded and also on application reload.

spec(app)

```
@spec spec(app()) :: [{application key(), value()}] | nil
```

Returns the spec for `app`.

The following keys are returned:

- `:description`
- `:id`
- `:vsn`
- `:modules`
- `:maxP`
- `:maxT`
- `:registered`
- `:included_applications`
- `:optional_applications`
- `:applications`
- `:mod`
- `:start_phases`

For a description of all fields, see Erlang's application specification.

Note the environment is not returned as it can be accessed via `fetch_env/2`. Returns `nil` if the application is not loaded.

spec(app, key)

```
@spec spec(app(), application_key()) :: value() | nil
```

Returns the value for `key` in `app`'s specification.

See `spec/1` for the supported keys. If the given specification parameter does not exist, this function will raise. Returns `nil` if the application is not loaded.

start(app, type \\ :temporary)

```
@spec start(app(), restart_type()) :: :ok | {:error, term()}
```

Starts the given `app` with `restart_type/0`.

If the `app` is not loaded, the application will first be loaded using `load/1`. Any included application, defined in the `:included_applications` key of the `.app` file will also be loaded, but they won't be started.

Furthermore, all applications listed in the `:applications` key must be explicitly started before this application is. If not, `{:error, {:not_started, app}}` is returned, where `app` is the name of the missing application.

In case you want to automatically load **and start** all of `app`'s dependencies, see `ensure_all_started/2`.

started_applications(timeout \\ 5000)

```
@spec started_applications(timeout()) :: [
  {app(), description :: charlist(), vsn :: charlist()}
]
```

Returns a list with information about the applications which are currently running.

stop(app)

```
@spec stop(app()) :: :ok | {:error, term()}
```

Stops the given `app`.

When stopped, the application is still loaded.

unload(app)

```
@spec unload(app()) :: :ok | {:error, term()}
```

Unloads the given `app`.

It will also unload all `:included_applications`. Note that the function does not purge the application modules.

Config

A simple keyword-based configuration API.

Example

This module is most commonly used to define application configuration, typically in `config/config.exs`:

```
import Config

config :some_app,
  key1: "value1",
  key2: "value2"

import_config "#{config_env()}.exs"
```

`import Config` will import the functions `config/2`, `config/3` `config_env/0`, `config_target/0`, and `import_config/1` to help you manage your configuration.

`config/2` and `config/3` are used to define key-value configuration for a given application. Once Mix starts, it will automatically evaluate the configuration file and persist the configuration above into `:some_app`'s application environment, which can be accessed in as follows:

```
"value1" = Application.fetch_env!(:some_app, :key1)
```

Finally, the line `import_config "#{config_env()}.exs"` will import other config files based on the current configuration environment, such

as `config/dev.exs` and `config/test.exs`.

`Config` also provides a low-level API for evaluating and reading configuration, under the `Config.Reader` module.

> **Avoid application environment in libraries**
>
> If you are writing a library to be used by other developers, it is generally recommended to avoid the application environment, as the application environment is effectively a global storage. Also note that the `config/config.exs` of a library is not evaluated when the library is used as a dependency, as configuration is always meant to configure the current project. For more information, see "Using application configuration for libraries".

Migrating from `use Mix.Config`

The `Config` module in Elixir was introduced in v1.9 as a replacement to `use Mix.Config`, which was specific to Mix and has been deprecated.

You can leverage `Config` instead of `use Mix.Config` in three steps. The first step is to replace `use Mix.Config` at the top of your config files by `import Config`.

The second is to make sure your `import config/1` calls do not have a wildcard character. If so, you need to perform the wildcard lookup manually. For example, if you did:

```
import_config "../apps/*/config/config.exs"
```

It has to be replaced by:

```
for config <- "../apps/*/config/config.exs" |>
Path.expand(__DIR__) |> Path.wildcard() do
```

```
  import_config config
end
```

The last step is to replace all `Mix.env()` calls in the config files with `config_env()`.

Keep in mind you must also avoid using `Mix.env()` inside your project files. To check the environment at *runtime*, you may add a configuration key:

```
# config.exs
...
config :my_app, env: config_env()
```

Then, in other scripts and modules, you may get the environment with `Application.fetch_env!/2`:

```
# router.exs
...
if Application.fetch_env!(:my_app, :env) == :prod do
  ...
end
```

The only places where you may access functions from the `Mix` module are the `mix.exs` file and inside custom Mix tasks, which are always within the `Mix.Tasks` namespace.

`config/runtime.exs`

For runtime configuration, you can use the `config/runtime.exs` file. It is executed right before applications start in both Mix and releases

(assembled with `mix release`).

Summary

Types

config_opts()

Functions

config(root_key, opts)

Configures the given `root_key`.

config(root_key, key, opts)

Configures the given `key` for the given `root_key`.

config_env()

Returns the environment this configuration file is executed on.

config_target()

Returns the target this configuration file is executed on.

import_config(file)

Imports configuration from the given file.

read_config(root_key)

Reads the configuration for the given root key.

Types

config_opts()

```
@type config_opts() :: [imports: [Path.t()] | :disabled, env:
atom(), target: atom()]
```

Functions

config(root_key, opts)

(since 1.9.0)

Configures the given `root_key`.

Keyword lists are always deep-merged.

Examples

The given `opts` are merged into the existing configuration for the given `root_key`. Conflicting keys are overridden by the ones specified in `opts`, unless they are keywords, which are deep merged recursively. For example, the application configuration below

```
config :logger,
  level: :warn,

config :logger,
  level: :info,
  truncate: 1024
```

will have a final configuration for `:logger` of:

```
[level: :info, truncate: 1024]
```

config(root_key, key, opts)

(since 1.9.0)

Configures the given `key` for the given `root_key`.

Keyword lists are always deep merged.

Examples

The given `opts` are merged into the existing values for `key` in the given `root_key`. Conflicting keys are overridden by the ones specified in `opts`, unless they are keywords, which are deep merged recursively. For example, the application configuration below

```
config :ecto, Repo,
  log_level: :warn,
  adapter: Ecto.Adapters.Postgres,
  metadata: [read_only: true]

config :ecto, Repo,
  log_level: :info,
  pool_size: 10,
  metadata: [replica: true]
```

will have a final value of the configuration for the `Repo` key in the `:ecto` application of:

```
Application.get_env(:ecto, Repo)
#=> [
```

```
#=>   log_level: :info,
#=>   pool_size: 10,
#=>   adapter: Ecto.Adapters.Postgres,
#=>   metadata: [read_only: true, replica: true]
#=> ]
```

config_env()

(since 1.11.0) (macro)

Returns the environment this configuration file is executed on.

In Mix projects this function returns the environment this configuration file is executed on. In releases, returns the `MIX_ENV` specified when running `mix release`.

This is most often used to execute conditional code:

```
if config_env() == :prod do
  config :my_app, :debug, false
end
```

config_target()

(since 1.11.0) (macro)

Returns the target this configuration file is executed on.

This is most often used to execute conditional code:

```
if config_target() == :host do
  config :my_app, :debug, false
```

```
end
```

import_config(file)

(since 1.9.0) (macro)

Imports configuration from the given file.

In case the file doesn't exist, an error is raised.

If file is a relative, it will be expanded relatively to the directory the current configuration file is in.

Examples

This is often used to emulate configuration across environments:

```
import_config "#{config_env()}.exs"
```

Note, however, some configuration files, such as `config/runtime.exs` does not support imports, as they are meant to be copied across systems.

read_config(root_key)

(since 1.18.0)

Reads the configuration for the given root key.

This function only reads the configuration from a previous `config/2` or `config/3` call. If `root_key` points to an application, it does not read its actual application environment. Its main use case is to make it easier to access and share configuration values across files.

If the `root_key` was not configured, it returns `nil`.

Examples

```
# In config/config.exs
config :my_app, foo: :bar

# In config/dev.exs
config :another_app, foo: read_config(:my_app)[:foo] ||
raise "missing parent configuration"
```

Config.Provider behaviour

Specifies a provider API that loads configuration during boot.

Config providers are typically used during releases to load external configuration while the system boots. This is done by starting the VM with the minimum amount of applications running, then invoking all of the providers, and then restarting the system. This requires a mutable configuration file on disk, as the results of the providers are written to the file system. For more information on runtime configuration, see `mix release`.

Multiple config files

One common use of config providers is to specify multiple configuration files in a release. Elixir ships with one provider, called `Config.Reader`, which is capable of handling Elixir's built-in config files.

For example, imagine you want to list some basic configuration on Mix's built-in `config/runtime.exs` file, but you also want to support additional configuration files. To do so, you can add this inside the `def project` portion of your `mix.exs`:

```
releases: [
  demo: [
    config_providers: [
      {Config.Reader, {:system, "RELEASE_ROOT",
"/extra_config.exs"}}
    ]
  ]
]
```

You can place this `extra_config.exs` file in your release in multiple ways:

1. If it is available on the host when assembling the release, you can place it on "rel/overlays/extra_config.exs" and it will be automatically copied to the release root

2. If it is available on the target during deployment, you can simply copy it to the release root as a step in your deployment

Now once the system boots, it will load both `config/runtime.exs` and `extra_config.exs` early in the boot process. You can learn more options on `Config.Reader`.

Custom config provider

You can also implement custom config providers, similar to how `Config.Reader` works. For example, imagine you need to load some configuration from a JSON file and load that into the system. Said configuration provider would look like:

```
defmodule JSONConfigProvider do
  @behaviour Config.Provider

  # Let's pass the path to the JSON file as config
  @impl true
  def init(path) when is_binary(path), do: path

  @impl true
  def load(config, path) do
    # We need to start any app we may depend on.
    {:ok, _} = Application.ensure_all_started(:jason)

    json = path |> File.read!() |> Jason.decode!()
```

```
    Config.Reader.merge(
      config,
      my_app: [
        some_value: json["my_app_some_value"],
        another_value: json["my_app_another_value"],
      ]
    )
  end
end
```

Then, when specifying your release, you can specify the provider in the release configuration:

```
releases: [
  demo: [
    config_providers: [
      {JSONConfigProvider, "/etc/config.json"}
    ]
  ]
]
```

Summary

Types

config()
config_path()

A path pointing to a configuration file.

init_opts()

Options for `init/3`.
state()

Callbacks

init(term)

Invoked when initializing a config provider.

load(config, state)

Loads configuration (typically during system boot).

Functions

resolve_config_path!(path)

Resolves a `config_path/0` to an actual path.

validate_config_path!(path)

Validates a `config_path/0`.

Types

config()

```
@type config() :: keyword()
```

config_path()

```
@type config_path() :: {:system, binary(), binary()} |
binary()
```

A path pointing to a configuration file.

Since configuration files are often accessed on target machines, it can be expressed either as:

- a binary representing an absolute path
- a `{:system, system_var, path}` tuple where the config is the concatenation of the environment variable `system_var` with the given `path`

init_opts()

```
@type init_opts() :: [
  extra_config: config(),
  prune_runtime_sys_config_after_boot: boolean(),
  reboot_system_after_config: boolean(),
  validate_compile_env: [{atom(), [atom()], term()}]
]
```

Options for `init/3`.

state()

```
@type state() :: term()
```

Callbacks

init(term)

```
@callback init(term()) :: state()
```

Invoked when initializing a config provider.

A config provider is typically initialized on the machine where the system is assembled and not on the target machine. The `init/1` callback is useful to verify the arguments given to the provider and prepare the state that will be given to `load/2`.

Furthermore, because the state returned by `init/1` can be written to text-based config files, it should be restricted only to simple data types, such as integers, strings, atoms, tuples, maps, and lists. Entries such as PIDs, references, and functions cannot be serialized.

load(config, state)

```
@callback load(config(), state()) :: config()
```

Loads configuration (typically during system boot).

It receives the current `config` and the `state` returned by `init/1`. Then, you typically read the extra configuration from an external source and merge it into the received `config`. Merging should be done with `Config.Reader.merge/2`, as it performs deep merge. It should return the updated config.

Note that `load/2` is typically invoked very early in the boot process, therefore if you need to use an application in the provider, it is your responsibility to start it.

Functions

resolve_config_path!(path)

(since 1.9.0)

```
@spec resolve_config_path!(config_path()) :: binary()
```

Resolves a `config_path/0` to an actual path.

validate_config_path!(path)

(since 1.9.0)

```
@spec validate_config_path!(config_path()) :: :ok
```

Validates a `config_path/0`.

Config.Reader

API for reading config files defined with `Config`.

As a provider

`Config.Reader` can also be used as a `Config.Provider`. A config provider is used during releases to customize how applications are configured. When used as a provider, it expects a single argument: the configuration path (as outlined in `Config.Provider.config_path/0`) for the file to be read and loaded during the system boot.

For example, if you expect the target system to have a config file in an absolute path, you can add this inside the `def project` portion of your `mix.exs`:

```
releases: [
  demo: [
    config_providers: [
      {Config.Reader, "/etc/config.exs"}
    ]
  ]
]
```

Or if you want to read a custom path inside the release:

```
config_providers: [{Config.Reader, {:system,
"RELEASE_ROOT", "/config.exs"}}]
```

You can also pass a keyword list of options to the reader, where the `:path` is a required key:

```
config_providers: [
  {Config.Reader,
   path: "/etc/config.exs",
   env: :prod,
   imports: :disabled}
]
```

Remember Mix already loads `config/runtime.exs` by default. For more examples and scenarios, see the `Config.Provider` module.

Summary

Types

config_opts()

Functions

eval!(file, contents, opts \\ [])

Evaluates the configuration `contents` for the given `file`.

merge(config1, config2)

Merges two configurations.

read!(file, opts \\ [])

Reads the configuration file.

read_imports!(file, opts \\ [])

Reads the given configuration file and returns the configuration with its imports.

Types

config_opts()

```
@type config_opts() :: [imports: [Path.t()] | :disabled, env: atom(), target: atom()]
```

Functions

eval!(file, contents, opts \\ [])

(since 1.11.0)

```
@spec eval!(Path.t(), binary(), config_opts()) :: keyword()
```

Evaluates the configuration `contents` for the given `file`.

Accepts the same options as `read!/2`.

merge(config1, config2)

(since 1.9.0)

```
@spec merge(keyword(), keyword()) :: keyword()
```

Merges two configurations.

The configurations are merged together with the values in the second one having higher preference than the first in case of conflicts. In case both values are set to keyword lists, it deep merges them.

Examples

```
iex> Config.Reader.merge([app: [k: :v1]], [app: [k:
:v2]])
[app: [k: :v2]]

iex> Config.Reader.merge([app: [k: [v1: 1, v2: 2]]],
[app: [k: [v2: :a, v3: :b]]])
[app: [k: [v1: 1, v2: :a, v3: :b]]]

iex> Config.Reader.merge([app1: []], [app2: []])
[app1: [], app2: []]
```

read!(file, opts \\ [])

(since 1.9.0)

```
@spec read!(Path.t(), config_opts()) :: keyword()
```

Reads the configuration file.

Options

- `:imports` - a list of already imported paths or `:disabled` to disable imports
- `:env` - the environment the configuration file runs on. See `Config.config_env/0` for sample usage

- `:target` - the target the configuration file runs on. See `Config.config_target/0` for sample usage

read_imports!(file, opts \\ [])

(since 1.9.0)

```
@spec read_imports!(Path.t(), config_opts()) :: {keyword(), [Path.t()]}
```

Reads the given configuration file and returns the configuration with its imports.

Accepts the same options as `read!/2`. Although note the `:imports` option cannot be disabled in `read_imports!/2`.

DynamicSupervisor behaviour

A supervisor optimized to only start children dynamically.

The `Supervisor` module was designed to handle mostly static children that are started in the given order when the supervisor starts. A `DynamicSupervisor` starts with no children. Instead, children are started on demand via `start_child/2` and there is no ordering between children. This allows the `DynamicSupervisor` to hold millions of children by using efficient data structures and to execute certain operations, such as shutting down, concurrently.

Examples

A dynamic supervisor is started with no children and often with a name:

```
children = [
  {DynamicSupervisor, name: MyApp.DynamicSupervisor,
strategy: :one_for_one}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

The options given in the child specification are documented in `start_link/1`.

Once the dynamic supervisor is running, we can use it to start children on demand. Given this sample `GenServer`:

```
defmodule Counter do
  use GenServer
```

```
  def start_link(initial) do
    GenServer.start_link(__MODULE__, initial)
  end

  def inc(pid) do
    GenServer.call(pid, :inc)
  end

  def init(initial) do
    {:ok, initial}
  end

  def handle_call(:inc, _, count) do
    {:reply, count, count + 1}
  end
end
```

We can use `start_child/2` with a child specification to start a `Counter` server:

```
{:ok, counter1} =
DynamicSupervisor.start_child(MyApp.DynamicSupervisor,
{Counter, 0})
Counter.inc(counter1)
#=> 0

{:ok, counter2} =
DynamicSupervisor.start_child(MyApp.DynamicSupervisor,
{Counter, 10})
Counter.inc(counter2)
#=> 10
```

```
DynamicSupervisor.count_children(MyApp.DynamicSupervisor)
#=> %{active: 2, specs: 2, supervisors: 0, workers: 2}
```

Scalability and partitioning

The `DynamicSupervisor` is a single process responsible for starting other processes. In some applications, the `DynamicSupervisor` may become a bottleneck. To address this, you can start multiple instances of the `DynamicSupervisor` and then pick a "random" instance to start the child on.

Instead of:

```
children = [
  {DynamicSupervisor, name: MyApp.DynamicSupervisor}
]
```

and:

```
DynamicSupervisor.start_child(MyApp.DynamicSupervisor,
{Counter, 0})
```

You can do this:

```
children = [
  {PartitionSupervisor,
   child_spec: DynamicSupervisor,
   name: MyApp.DynamicSupervisors}
]
```

and then:

```
DynamicSupervisor.start_child(
  {:via, PartitionSupervisor, {MyApp.DynamicSupervisors,
self()}},
  {Counter, 0}
)
```

In the code above, we start a partition supervisor that will by default start a dynamic supervisor for each core in your machine. Then, instead of calling the `DynamicSupervisor` by name, you call it through the partition supervisor, using `self()` as the routing key. This means each process will be assigned one of the existing dynamic supervisors. Read the `PartitionSupervisor` docs for more information.

Module-based supervisors

Similar to `Supervisor`, dynamic supervisors also support module-based supervisors.

```
defmodule MyApp.DynamicSupervisor do
  # Automatically defines child_spec/1
  use DynamicSupervisor

  def start_link(init_arg) do
    DynamicSupervisor.start_link(__MODULE__, init_arg,
name: __MODULE__)
  end

  @impl true
  def init(_init_arg) do
    DynamicSupervisor.init(strategy: :one_for_one)
  end
end
```

See the `Supervisor` docs for a discussion of when you may want to use module-based supervisors. A `@doc` annotation immediately preceding `use DynamicSupervisor` will be attached to the generated `child_spec/1` function.

```
use DynamicSupervisor
```

When you `use DynamicSupervisor`, the `DynamicSupervisor` module will set `@behaviour DynamicSupervisor` and define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

Name registration

A supervisor is bound to the same name registration rules as a `GenServer`. Read more about these rules in the documentation for `GenServer`.

Summary

Types

init_option()

Options given to `start_link/1` and `init/1` functions

on_start_child()

Return values of `start_child` functions.

strategy()

Supported strategies

sup_flags()

The supervisor flags returned on init

Callbacks

init(init_arg)

Callback invoked to start the supervisor and during hot code upgrades.

Functions

child_spec(options)

Returns a specification to start a dynamic supervisor under a supervisor.

count_children(supervisor)

Returns a map containing count values for the supervisor.

init(options)

Receives a set of `options` that initializes a dynamic supervisor.

start_child(supervisor, child_spec)

Dynamically adds a child specification to `supervisor` and starts that child.

start_link(options)

Starts a supervisor with the given options.

start_link(module, init_arg, opts \\ [])

Starts a module-based supervisor process with the given `module` and `init_arg`.

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

Synchronously stops the given supervisor with the given `reason`.

terminate_child(supervisor, pid)

Terminates the given child identified by `pid`.

which_children(supervisor)

Returns a list with information about all children of the given supervisor.

Types

init_option()

```
@type init_option() ::
  {:strategy, strategy()}
  | {:max_restarts, non_neg_integer()}
  | {:max_seconds, pos_integer()}
  | {:max_children, non_neg_integer() | :infinity}
  | {:extra_arguments, [term()]}
```

Options given to `start_link/1` and `init/1` functions

on_start_child()

```
@type on_start_child() ::
  {:ok, pid()}
  | {:ok, pid(), info :: term()}
  | :ignore
```

```
  | {:error, {:already_started, pid()} | :max_children |
term()}
```

Return values of `start_child` functions.

Unlike `Supervisor`, this module ignores the child spec ids, so `{:error, {:already_started, pid}}` is not returned for child specs given with the same id. `{:error, {:already_started, pid}}` is returned however if a duplicate name is used when using name registration.

strategy()

```
@type strategy() :: :one_for_one
```

Supported strategies

sup_flags()

```
@type sup_flags() :: %{
  strategy: strategy(),
  intensity: non_neg_integer(),
  period: pos_integer(),
  max_children: non_neg_integer() | :infinity,
  extra_arguments: [term()]
}
```

The supervisor flags returned on init

Callbacks

init(init_arg)

```
@callback init(init_arg :: term()) :: {:ok, sup_flags()} |
:ignore
```

Callback invoked to start the supervisor and during hot code upgrades.

Developers typically invoke `DynamicSupervisor.init/1` at the end of their init callback to return the proper supervision flags.

Functions

child_spec(options)

(since 1.6.1)

```
@spec child_spec([init_option() | GenServer.option()]) ::
Supervisor.child_spec()
```

Returns a specification to start a dynamic supervisor under a supervisor.

It accepts the same options as `start_link/1`.

See `Supervisor` for more information about child specifications.

count_children(supervisor)

(since 1.6.0)

```
@spec count_children(Supervisor.supervisor()) :: %{
  specs: non_neg_integer(),
  active: non_neg_integer(),
  supervisors: non_neg_integer(),
  workers: non_neg_integer()
}
```

Returns a map containing count values for the supervisor.

The map contains the following keys:

- `:specs` - the number of children processes
- `:active` - the count of all actively running child processes managed by this supervisor
- `:supervisors` - the count of all supervisors whether or not the child process is still alive
- `:workers` - the count of all workers, whether or not the child process is still alive

init(options)

(since 1.6.0)

```
@spec init([init_option()]) :: {:ok, sup_flags()}
```

Receives a set of `options` that initializes a dynamic supervisor.

This is typically invoked at the end of the `init/1` callback of module-based supervisors. See the "Module-based supervisors" section in the module documentation for more information.

It accepts the same `options` as `start_link/1` (except for `:name`) and it returns a tuple containing the supervisor options.

Examples

```
def init(_arg) do
  DynamicSupervisor.init(max_children: 1000)
```

```
  end
```

start_child(supervisor, child_spec)

(since 1.6.0)

```
@spec start_child(
  Supervisor.supervisor(),
  Supervisor.child_spec()
  | {module(), term()}
  | module()
  | (old_erlang_child_spec :: :supervisor.child_spec())
) :: on_start_child()
```

Dynamically adds a child specification to `supervisor` and starts that child.

`child_spec` should be a valid child specification. The child process will be started as defined in the child specification. Note that while the `:id` field is still required in the spec, the value is ignored and therefore does not need to be unique. Unlike `Supervisor`, this module does not return `{:error, {:already_started, pid}}` for child specs given with the same id. `{:error, {:already_started, pid}}` is returned however if a duplicate name is used when using name registration.

This function will block the `DynamicSupervisor` until the child initializes. When starting too many processes dynamically, you may want to use a `PartitionSupervisor` to split the work across multiple processes.

If the child process start function returns `{:ok, child}` or `{:ok, child, info}`, then child specification and PID are added to the supervisor and this function returns the same value.

If the child process start function returns `:ignore`, then no child is added to the supervision tree and this function returns `:ignore` too.

If the child process start function returns an error tuple or an erroneous value, or if it fails, the child specification is discarded and this function returns `{:error, error}` where `error` is the error or erroneous value returned from child process start function, or failure reason if it fails.

If the supervisor already has N children in a way that N exceeds the amount of `:max_children` set on the supervisor initialization (see `init/1`), then this function returns `{:error, :max_children}`.

start_link(options)

(since 1.6.0)

```
@spec start_link([init_option() | GenServer.option()]) ::
Supervisor.on_start()
```

Starts a supervisor with the given options.

This function is typically not invoked directly, instead it is invoked when using a `DynamicSupervisor` as a child of another supervisor:

```
children = [
  {DynamicSupervisor, name: MySupervisor}
]
```

If the supervisor is successfully spawned, this function returns `{:ok, pid}`, where `pid` is the PID of the supervisor. If the supervisor is given a name and a process with the specified name already exists, the function returns `{:error, {:already_started, pid}}`, where `pid` is the PID of that process.

Note that a supervisor started with this function is linked to the parent process and exits not only on crashes but also if the parent process exits with `:normal` reason.

Options

- `:name` - registers the supervisor under the given name. The supported values are described under the "Name registration" section in the `GenServer` module docs.

- `:strategy` - the restart strategy option. The only supported value is `:one_for_one` which means that no other child is terminated if a child process terminates. You can learn more about strategies in the `Supervisor` module docs.

- `:max_restarts` - the maximum number of restarts allowed in a time frame. Defaults to `3`.

- `:max_seconds` - the time frame in which `:max_restarts` applies. Defaults to `5`.

- `:max_children` - the maximum amount of children to be running under this supervisor at the same time. When `:max_children` is exceeded, `start_child/2` returns `{:error, :max_children}`. Defaults to `:infinity`.

- `:extra_arguments` - arguments that are prepended to the arguments specified in the child spec given to `start_child/2`. Defaults to an empty list.

- Any of the standard GenServer options

start_link(module, init_arg, opts \\ [])

(since 1.6.0)

```
@spec start_link(module(), term(), [GenServer.option()]) ::
Supervisor.on_start()
```

Starts a module-based supervisor process with the given `module` and `init_arg`.

To start the supervisor, the `init/1` callback will be invoked in the given `module`, with `init_arg` as its argument. The `init/1` callback must return a supervisor specification which can be created with the help of the `init/1` function.

If the `init/1` callback returns `:ignore`, this function returns `:ignore` as well and the supervisor terminates with reason `:normal`. If it fails or returns an incorrect value, this function returns `{:error, term}` where `term` is a term with information about the error, and the supervisor terminates with reason `term`.

The `:name` option can also be given in order to register a supervisor name, the supported values are described in the "Name registration" section in the `GenServer` module docs.

If the supervisor is successfully spawned, this function returns `{:ok, pid}`, where `pid` is the PID of the supervisor. If the supervisor is given a name and a process with the specified name already exists, the function returns `{:error, {:already_started, pid}}`, where `pid` is the PID of that process.

Note that a supervisor started with this function is linked to the parent process and exits not only on crashes but also if the parent process exits with `:normal` reason.

Options

This function accepts any regular `GenServer options`. Options specific to `DynamicSupervisor` must be returned from the `init/1` callback.

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

(since 1.7.0)

```
@spec stop(Supervisor.supervisor(), reason :: term(),
timeout()) :: :ok
```

Synchronously stops the given supervisor with the given `reason`.

It returns `:ok` if the supervisor terminates with the given reason. If it terminates with another reason, the call exits.

This function keeps OTP semantics regarding error reporting. If the reason is any other than `:normal`, `:shutdown` or `{:shutdown, _}`, an error report is logged.

terminate_child(supervisor, pid)

(since 1.6.0)

```
@spec terminate_child(Supervisor.supervisor(), pid()) :: :ok
| {:error, :not_found}
```

Terminates the given child identified by `pid`.

This function will block the `DynamicSupervisor` until the child terminates, which may take an arbitrary amount of time if the child is trapping exits and implements its own terminate callback. For this reason, it is often better to ask the child process itself to terminate, often by declaring in its child spec it has a restart strategy of `:transient` (or

`:temporary`) and then sending it a message to stop with reason `:shutdown`.

If successful, this function returns `:ok`. If there is no process with the given PID, this function returns `{:error, :not_found}`.

which_children(supervisor)

(since 1.6.0)

```
@spec which_children(Supervisor.supervisor()) :: [
  {:undefined, pid() | :restarting, :worker | :supervisor,
   [module()] | :dynamic}
]
```

Returns a list with information about all children of the given supervisor.

Note that calling this function when supervising a large number of children under low memory conditions can bring the system down due to an out of memory error.

This function returns a list of tuples containing:

- `id` - it is always `:undefined` for dynamic supervisors
- `child` - the PID of the corresponding child process or the atom `:restarting` if the process is about to be restarted
- `type` - `:worker` or `:supervisor` as defined in the child specification
- `modules` - as defined in the child specification

GenServer behaviour

A behaviour module for implementing the server of a client-server relation.

A GenServer is a process like any other Elixir process and it can be used to keep state, execute code asynchronously and so on. The advantage of using a generic server process (GenServer) implemented using this module is that it will have a standard set of interface functions and include functionality for tracing and error reporting. It will also fit into a supervision tree.

```
graph BT
    C(Client #3) ~~~ B(Client #2) ~~~ A(Client #1)
    A & B & C -->|request| GenServer
    GenServer -.->|reply| A & B & C
```

Example

The GenServer behaviour abstracts the common client-server interaction. Developers are only required to implement the callbacks and functionality they are interested in.

Let's start with a code example and then explore the available callbacks. Imagine we want to implement a service with a GenServer that works like a stack, allowing us to push and pop elements. We'll customize a generic GenServer with our own module by implementing three callbacks.

`init/1` transforms our initial argument to the initial state for the GenServer. `handle_call/3` fires when the server receives a synchronous `pop` message, popping an element from the stack and

returning it to the user. `handle_cast/2` will fire when the server receives an asynchronous `push` message, pushing an element onto the stack:

```
defmodule Stack do
  use GenServer

  # Callbacks

  @impl true
  def init(elements) do
    initial_state = String.split(elements, ",", trim:
true)
    {:ok, initial_state}
  end

  @impl true
  def handle_call(:pop, _from, state) do
    [to_caller | new_state] = state
    {:reply, to_caller, new_state}
  end

  @impl true
  def handle_cast({:push, element}, state) do
    new_state = [element | state]
    {:noreply, new_state}
  end
end
```

We leave the process machinery of startup, message passing, and the message loop to the GenServer behaviour and focus only on the stack implementation. We can now use the GenServer API to interact with the service by creating a process and sending it messages:

```elixir
# Start the server
{:ok, pid} = GenServer.start_link(Stack, "hello,world")

# This is the client
GenServer.call(pid, :pop)
#=> "hello"

GenServer.cast(pid, {:push, "elixir"})
#=> :ok

GenServer.call(pid, :pop)
#=> "elixir"
```

We start our `Stack` by calling `start_link/2`, passing the module with the server implementation and its initial argument with a comma-separated list of elements. The GenServer behaviour calls the `init/1` callback to establish the initial GenServer state. From this point on, the GenServer has control so we interact with it by sending two types of messages on the client. **call** messages expect a reply from the server (and are therefore synchronous) while **cast** messages do not.

Each call to `GenServer.call/3` results in a message that must be handled by the `handle_call/3` callback in the GenServer. A `cast/2` message must be handled by `handle_cast/2`. `GenServer` supports 8 callbacks, but only `init/1` is required.

> `use GenServer`
>
> When you `use GenServer`, the `GenServer` module will set `@behaviour GenServer` and define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

Client / Server APIs

Although in the example above we have used `GenServer.start_link/3` and friends to directly start and communicate with the server, most of the time we don't call the `GenServer` functions directly. Instead, we wrap the calls in new functions representing the public API of the server. These thin wrappers are called the **client API**.

Here is a better implementation of our Stack module:

```
defmodule Stack do
  use GenServer

  # Client

  def start_link(default) when is_binary(default) do
    GenServer.start_link(__MODULE__, default)
  end

  def push(pid, element) do
    GenServer.cast(pid, {:push, element})
  end

  def pop(pid) do
    GenServer.call(pid, :pop)
  end

  # Server (callbacks)

  @impl true
  def init(elements) do
    initial_state = String.split(elements, ",", trim:
true)
    {:ok, initial_state}
  end

  @impl true
```

```
  def handle_call(:pop, _from, state) do
    [to_caller | new_state] = state
    {:reply, to_caller, new_state}
  end

  @impl true
  def handle_cast({:push, element}, state) do
    new_state = [element | state]
    {:noreply, new_state}
  end
end
```

In practice, it is common to have both server and client functions in the same module. If the server and/or client implementations are growing complex, you may want to have them in different modules.

The following diagram summarizes the interactions between client and server. Both Client and Server are processes and communication happens via messages (continuous line). The Server <-> Module interaction happens when the GenServer process calls your code (dotted lines):

```
sequenceDiagram
    participant C as Client (Process)
    participant S as Server (Process)
    participant M as Module (Code)

    note right of C: Typically started by a supervisor
    C->>+S: GenServer.start_link(module, arg, options)
    S-->>+M: init(arg)
    M-->>-S: {:ok, state} | :ignore | {:error, reason}
    S->>-C: {:ok, pid} | :ignore | {:error, reason}

    note right of C: call is synchronous
    C->>+S: GenServer.call(pid, message)
```

```
    S-->>+M: handle_call(message, from, state)
    M-->>-S: {:reply, reply, state} | {:stop, reason,
reply, state}
    S->>-C: reply

    note right of C: cast is asynchronous
    C-)S: GenServer.cast(pid, message)
    S-->>+M: handle_cast(message, state)
    M-->>-S: {:noreply, state} | {:stop, reason, state}

    note right of C: send is asynchronous
    C-)S: Kernel.send(pid, message)
    S-->>+M: handle_info(message, state)
    M-->>-S: {:noreply, state} | {:stop, reason, state}
```

How to supervise

A `GenServer` is most commonly started under a supervision tree. When we invoke `use GenServer`, it automatically defines a `child_spec/1` function that allows us to start the `Stack` directly under a supervisor. To start a default stack of `["hello", "world"]` under a supervisor, we can do:

```elixir
children = [
  {Stack, "hello,world"}
]

Supervisor.start_link(children, strategy: :one_for_all)
```

Note that specifying a module `MyServer` would be the same as specifying the tuple `{MyServer, []}`.

`use GenServer` also accepts a list of options which configures the child specification and therefore how it runs under a supervisor. The generated `child_spec/1` can be customized with the following options:

- `:id` - the child specification identifier, defaults to the current module
- `:restart` - when the child should be restarted, defaults to `:permanent`
- `:shutdown` - how to shut down the child, either immediately or by giving it time to shut down, defaults to `5_000`

For example:

```
use GenServer, restart: :transient, shutdown: 10_000
```

See the "Child specification" section in the `Supervisor` module for more detailed information. The `@doc` annotation immediately preceding `use GenServer` will be attached to the generated `child_spec/1` function.

When stopping the GenServer, for example by returning a `{:stop, reason, new_state}` tuple from a callback, the exit reason is used by the supervisor to determine whether the GenServer needs to be restarted. See the "Exit reasons and restarts" section in the `Supervisor` module.

Name registration

Both `start_link/3` and `start/3` support the `GenServer` to register a name on start via the `:name` option. Registered names are also automatically cleaned up on termination. The supported values are:

- `nil` (default) - the GenServer is not registered with a name.

- an atom - the GenServer is registered locally (to the current node) with the given name using `Process.register/2`.

- `{:global, term}` - the GenServer is registered globally with the given term using the functions in the `:global` module.

- `{:via, module, term}` - the GenServer is registered with the given mechanism and name. The `:via` option expects a module that exports `register_name/2`, `unregister_name/1`, `whereis_name/1` and `send/2`. One such example is the `:global` module which uses these functions for keeping the list of names of processes and their associated PIDs that are available globally for a network of Elixir nodes. Elixir also ships with a local, decentralized and scalable registry called `Registry` for locally storing names that are generated dynamically.

For example, we could start and register our `Stack` server locally as follows:

```
# Start the server and register it locally with name
MyStack
{:ok, _} = GenServer.start_link(Stack, "hello", name:
MyStack)

# Now messages can be sent directly to MyStack
GenServer.call(MyStack, :pop)
#=> "hello"
```

Once the server is started, the remaining functions in this module (`call/3`, `cast/2`, and friends) will also accept an atom, or any `{:global, ...}` or `{:via, ...}` tuples. In general, the following formats are supported:

- a PID

- an atom if the server is locally registered
- `{atom, node}` if the server is locally registered at another node
- `{:global, term}` if the server is globally registered
- `{:via, module, name}` if the server is registered through an alternative registry

If there is an interest to register dynamic names locally, do not use atoms, as atoms are never garbage-collected and therefore dynamically generated atoms won't be garbage-collected. For such cases, you can set up your own local registry by using the `Registry` module.

For example:

```
{:ok, _} = Registry.start_link(keys: :unique, name:
:stacks)
name = {:via, Registry, {:stacks, "stack 1"}}
{:ok, _pid} = GenServer.start_link(Stack, "hello", name:
name)
GenServer.whereis(name)
#=> #PID<0.150.0>
```

Receiving "regular" messages

The goal of a `GenServer` is to abstract the "receive" loop for developers, automatically handling system messages, supporting code change, synchronous calls and more. Therefore, you should never call your own "receive" inside the GenServer callbacks as doing so will cause the GenServer to misbehave.

Besides the synchronous and asynchronous communication provided by `call/3` and `cast/2`, "regular" messages sent by functions such as `send/2`, `Process.send_after/4` and similar, can be handled inside the `handle_info/2` callback.

handle_info/2 can be used in many situations, such as handling monitor DOWN messages sent by Process.monitor/1. Another use case for handle_info/2 is to perform periodic work, with the help of Process.send_after/4:

```
defmodule MyApp.Periodically do
  use GenServer

  def start_link(_) do
    GenServer.start_link(__MODULE__, %{})
  end

  @impl true
  def init(state) do
    # Schedule work to be performed on start
    schedule_work()

    {:ok, state}
  end

  @impl true
  def handle_info(:work, state) do
    # Do the desired work here
    # ...

    # Reschedule once more
    schedule_work()

    {:noreply, state}
  end

  defp schedule_work do
    # We schedule the work to happen in 2 hours (written
in milliseconds).
    # Alternatively, one might write :timer.hours(2)
```

```
    Process.send_after(self(), :work, 2 * 60 * 60 * 1000)
  end
end
```

Timeouts

The return value of `init/1` or any of the `handle_*` callbacks may include a timeout value in milliseconds; if not, `:infinity` is assumed. The timeout can be used to detect a lull in incoming messages.

The `timeout()` value is used as follows:

- If the process has any message already waiting when the `timeout()` value is returned, the timeout is ignored and the waiting message is handled as usual. This means that even a timeout of `0` milliseconds is not guaranteed to execute (if you want to take another action immediately and unconditionally, use a `:continue` instruction instead).
- If any message arrives before the specified number of milliseconds elapse, the timeout is cleared and that message is handled as usual.
- Otherwise, when the specified number of milliseconds have elapsed with no message arriving, `handle_info/2` is called with `:timeout` as the first argument.

For example:

```
defmodule Counter do
  use GenServer

  @timeout to_timeout(second: 5)
```

```
  @impl true
  def init(count) do
    {:ok, count, @timeout}
  end

  @impl true
  def handle_call(:increment, _from, count) do
    new_count = count + 1
    {:reply, new_count, new_count, @timeout}
  end

  @impl true
  def handle_info(:timeout, count) do
    {:stop, :normal, count}
  end
end
```

A `Counter` server will exit with `:normal` if there are no messages in 5 seconds after the initialization or after the last `:increment` call:

```
{:ok, counter_pid} = GenServer.start(Counter, 50)
GenServer.call(counter_pid, :increment)
#=> 51

# After 5 seconds
Process.alive?(counter_pid)
#=> false
```

When (not) to use a GenServer

So far, we have learned that a `GenServer` can be used as a supervised process that handles sync and async calls. It can also handle system

messages, such as periodic messages and monitoring events. GenServer processes may also be named.

A GenServer, or a process in general, must be used to model runtime characteristics of your system. A GenServer must never be used for code organization purposes.

In Elixir, code organization is done by modules and functions, processes are not necessary. For example, imagine you are implementing a calculator and you decide to put all the calculator operations behind a GenServer:

```
def add(a, b) do
  GenServer.call(__MODULE__, {:add, a, b})
end

def subtract(a, b) do
  GenServer.call(__MODULE__, {:subtract, a, b})
end

def handle_call({:add, a, b}, _from, state) do
  {:reply, a + b, state}
end

def handle_call({:subtract, a, b}, _from, state) do
  {:reply, a - b, state}
end
```

This is an anti-pattern not only because it convolutes the calculator logic but also because you put the calculator logic behind a single process that will potentially become a bottleneck in your system, especially as the number of calls grow. Instead just define the functions directly:

```elixir
def add(a, b) do
  a + b
end

def subtract(a, b) do
  a - b
end
```

If you don't need a process, then you don't need a process. Use processes only to model runtime properties, such as mutable state, concurrency and failures, never for code organization.

Debugging with the :sys module

GenServers, as special processes, can be debugged using the `:sys` module. Through various hooks, this module allows developers to introspect the state of the process and trace system events that happen during its execution, such as received messages, sent replies and state changes.

Let's explore the basic functions from the `:sys` module used for debugging:

- `:sys.get_state/2` - allows retrieval of the state of the process. In the case of a GenServer process, it will be the callback module state, as passed into the callback functions as last argument.
- `:sys.get_status/2` - allows retrieval of the status of the process. This status includes the process dictionary, if the process is running or is suspended, the parent PID, the debugger state, and the state of the behaviour module, which includes the callback module state (as returned by `:sys.get_state/2`). It's possible to change how this status is represented by defining the optional `GenServer.format_status/1` callback.

- `:sys.trace/3` - prints all the system events to `:stdio`.
- `:sys.statistics/3` - manages collection of process statistics.
- `:sys.no_debug/2` - turns off all debug handlers for the given process. It is very important to switch off debugging once we're done. Excessive debug handlers or those that should be turned off, but weren't, can seriously damage the performance of the system.
- `:sys.suspend/2` - allows to suspend a process so that it only replies to system messages but no other messages. A suspended process can be reactivated via `:sys.resume/2`.

Let's see how we could use those functions for debugging the stack server we defined earlier.

```
iex> {:ok, pid} = Stack.start_link("")
iex> :sys.statistics(pid, true) # turn on collecting
process statistics
iex> :sys.trace(pid, true) # turn on event printing
iex> Stack.push(pid, 1)
*DBG* <0.122.0> got cast {push,1}
*DBG* <0.122.0> new state [1]
:ok

iex> :sys.get_state(pid)
[1]

iex> Stack.pop(pid)
*DBG* <0.122.0> got call pop from <0.80.0>
*DBG* <0.122.0> sent 1 to <0.80.0>, new state []
1

iex> :sys.statistics(pid, :get)
{:ok,
 [
   start_time: {{2016, 7, 16}, {12, 29, 41}},
   current_time: {{2016, 7, 16}, {12, 29, 50}},
```

```
   reductions: 117,
   messages_in: 2,
   messages_out: 0
 ]}

iex> :sys.no_debug(pid) # turn off all debug handlers
:ok

iex> :sys.get_status(pid)
{:status, #PID<0.122.0>, {:module, :gen_server},
 [
   [
     "$initial_call": {Stack, :init, 1},            #
process dictionary
     "$ancestors": [#PID<0.80.0>, #PID<0.51.0>]
   ],
   :running,                                        #
:running | :suspended
   #PID<0.80.0>,                                    #
parent
   [],                                              #
debugger state
   [
     header: 'Status for generic server <0.122.0>', #
module status
     data: [
       {'Status', :running},
       {'Parent', #PID<0.80.0>},
       {'Logged events', []}
     ],
     data: [{'State', [1]}]
   ]
 ]}
```

Learn more

If you wish to find out more about GenServers, the Elixir Getting Started guide provides a tutorial-like introduction. The documentation and links in Erlang can also provide extra insight.

- GenServer - Elixir's Getting Started Guide
- `:gen_server` module documentation
- gen_server Behaviour - OTP Design Principles
- Clients and Servers - Learn You Some Erlang for Great Good!

Summary

Types

debug()

Debug options supported by the `start*` functions

from()

Tuple describing the client of a call request.

name()

The GenServer name

on_start()

Return values of `start*` functions

option()

Option values used by the `start*` functions

options()

Options used by the `start*` functions

server()

The server reference.

Callbacks

code_change(old_vsn, state, extra)

Invoked to change the state of the `GenServer` when a different version of a module is loaded (hot code swapping) and the state's term structure should be changed.

format_status(status)

This function is called by a `GenServer` process in the following situations

format_status(reason, pdict_and_state) deprecated
handle_call(request, from, state)

Invoked to handle synchronous `call/3` messages. `call/3` will block until a reply is received (unless the call times out or nodes are disconnected).

handle_cast(request, state)

Invoked to handle asynchronous `cast/2` messages.

handle_continue(continue_arg, state)

Invoked to handle continue instructions.

handle_info(msg, state)

Invoked to handle all other messages.

init(init_arg)

Invoked when the server is started. `start_link/3` or `start/3` will block until it returns.

terminate(reason, state)

Invoked when the server is about to exit. It should do any cleanup required.

Functions

abcast(nodes \\ [node() | Node.list()], name, request)

Casts all servers locally registered as `name` at the specified nodes.

call(server, request, timeout \\ 5000)

Makes a synchronous call to the `server` and waits for its reply.

cast(server, request)

Casts a request to the `server` without waiting for a response.

multi_call(nodes \\ [node() | Node.list()], name, request, timeout \\ :infinity)

Calls all servers locally registered as `name` at the specified `nodes`.

reply(client, reply)

Replies to a client.

start(module, init_arg, options \\ [])

Starts a `GenServer` process without links (outside of a supervision tree).

start_link(module, init_arg, options \\ [])

Starts a `GenServer` process linked to the current process.
stop(server, reason \\ :normal, timeout \\ :infinity)

Synchronously stops the server with the given `reason`.

whereis(server)

Returns the `pid` or `{name, node}` of a GenServer process, `nil` otherwise.

Types

debug()

```
@type debug() :: [:trace | :log | :statistics |
{:log_to_file, Path.t()}]
```

Debug options supported by the `start*` functions

from()

```
@type from() :: {pid(), tag :: term()}
```

Tuple describing the client of a call request.

`pid` is the PID of the caller and `tag` is a unique term used to identify the call.

name()

```
@type name() :: nil | atom() | {:global, term()} | {:via,
module(), term()}
```

The GenServer name

on_start()

```
@type on_start() ::
  {:ok, pid()} | :ignore | {:error, {:already_started, pid()}
| term()}
```

Return values of `start*` functions

option()

```
@type option() ::
  {:debug, debug()}
  | {:name, name()}
  | {:timeout, timeout()}
  | {:spawn_opt, [Process.spawn_opt()]}
  | {:hibernate_after, timeout()}
```

Option values used by the `start*` functions

options()

```
@type options() :: [option()]
```

Options used by the `start*` functions

server()

```
@type server() :: pid() | name() | {atom(), node()}
```

The server reference.

This is either a plain PID or a value representing a registered name. See the "Name registration" section of this document for more information.

Callbacks

code_change(old_vsn, state, extra)

(optional)

```
@callback code_change(old_vsn, state :: term(), extra ::
term()) ::
  {:ok, new_state :: term()} | {:error, reason :: term()}
when old_vsn: term() | {:down, term()}
```

Invoked to change the state of the `GenServer` when a different version of a module is loaded (hot code swapping) and the state's term structure should be changed.

`old_vsn` is the previous version of the module (defined by the `@vsn` attribute) when upgrading. When downgrading the previous version is wrapped in a 2-tuple with first element `:down`. `state` is the current state of the `GenServer` and `extra` is any extra data required to change the state.

Returning `{:ok, new_state}` changes the state to `new_state` and the code change is successful.

Returning `{:error, reason}` fails the code change with reason `reason` and the state remains as the previous state.

If `code_change/3` raises the code change fails and the loop will continue with its previous state. Therefore this callback does not usually contain side effects.

This callback is optional.

format_status(status)

(since 1.17.0) (optional)

```
@callback format_status(status ::
:gen_server.format_status()) ::
  new_status :: :gen_server.format_status()
```

This function is called by a `GenServer` process in the following situations:

- `:sys.get_status/1,2` is invoked to get the `GenServer` status.
- The `GenServer` process terminates abnormally and logs an error.

This callback is used to limit the status of the process returned by `:sys.get_status/1,2` or sent to logger.

The callback gets a map `status` describing the current status and shall return a map `new_status` with the same keys, but it may transform some values.

Two possible use cases for this callback is to remove sensitive information from the state to prevent it from being printed in log files, or to compact large irrelevant status items that would only clutter the logs.

Example

```elixir
  @impl GenServer
  def format_status(status) do
    Map.new(status, fn
      {:state, state} -> {:state, Map.delete(state,
  :private_key)}
      {:message, {:password, _}} -> {:message, {:password,
  "redacted"}}
      key_value -> key_value
    end)
  end
```

format_status(reason, pdict_and_state)

(optional)

This callback is deprecated. Use format_status/1 callback instead.

```elixir
@callback format_status(reason, pdict_and_state :: list()) ::
term()
when reason: :normal | :terminate
```

handle_call(request, from, state)

(optional)

```elixir
@callback handle_call(request :: term(), from(), state ::
term()) ::
  {:reply, reply, new_state}
  | {:reply, reply, new_state,
     timeout() | :hibernate | {:continue, continue_arg ::
term()}}
```

```
  | {:noreply, new_state}
  | {:noreply, new_state,
     timeout() | :hibernate | {:continue, continue_arg ::
term()}}
  | {:stop, reason, reply, new_state}
  | {:stop, reason, new_state}
when reply: term(), new_state: term(), reason: term()
```

Invoked to handle synchronous `call/3` messages. `call/3` will block until a reply is received (unless the call times out or nodes are disconnected).

`request` is the request message sent by a `call/3`, `from` is a 2-tuple containing the caller's PID and a term that uniquely identifies the call, and `state` is the current state of the `GenServer`.

Returning `{:reply, reply, new_state}` sends the response `reply` to the caller and continues the loop with new state `new_state`.

Returning `{:reply, reply, new_state, timeout}` is similar to `{:reply, reply, new_state}` except that it also sets a timeout. See the "Timeouts" section in the module documentation for more information.

Returning `{:reply, reply, new_state, :hibernate}` is similar to `{:reply, reply, new_state}` except the process is hibernated and will continue the loop once a message is in its message queue. However, if a message is already in the message queue, the process will continue the loop immediately. Hibernating a `GenServer` causes garbage collection and leaves a continuous heap that minimises the memory used by the process.

Hibernating should not be used aggressively as too much time could be spent garbage collecting, which would delay the processing of incoming messages. Normally it should only be used when you are not expecting

new messages to immediately arrive and minimising the memory of the process is shown to be beneficial.

Returning `{:reply, reply, new_state, {:continue, continue_arg}}` is similar to `{:reply, reply, new_state}` except that `handle_continue/2` will be invoked immediately after with `continue_arg` as the first argument and `state` as the second one.

Returning `{:noreply, new_state}` does not send a response to the caller and continues the loop with new state `new_state`. The response must be sent with `reply/2`.

There are three main use cases for not replying using the return value:

- To reply before returning from the callback because the response is known before calling a slow function.
- To reply after returning from the callback because the response is not yet available.
- To reply from another process, such as a task.

When replying from another process the `GenServer` should exit if the other process exits without replying as the caller will be blocking awaiting a reply.

Returning `{:noreply, new_state, timeout | :hibernate | {:continue, continue_arg}}` is similar to `{:noreply, new_state}` except a timeout, hibernation or continue occurs as with a `:reply` tuple.

Returning `{:stop, reason, reply, new_state}` stops the loop and `terminate/2` is called with reason `reason` and state `new_state`. Then, the `reply` is sent as the response to call and the process exits with reason `reason`.

Returning `{:stop, reason, new_state}` is similar to `{:stop, reason, reply, new_state}` except a reply is not sent.

This callback is optional. If one is not implemented, the server will fail if a call is performed against it.

handle_cast(request, state)

(optional)

```
@callback handle_cast(request :: term(), state :: term()) ::
  {:noreply, new_state}
  | {:noreply, new_state,
     timeout() | :hibernate | {:continue, continue_arg ::
term()}}
  | {:stop, reason :: term(), new_state}
when new_state: term()
```

Invoked to handle asynchronous `cast/2` messages.

`request` is the request message sent by a `cast/2` and `state` is the current state of the `GenServer`.

Returning `{:noreply, new_state}` continues the loop with new state `new_state`.

Returning `{:noreply, new_state, timeout}` is similar to `{:noreply, new_state}` except that it also sets a timeout. See the "Timeouts" section in the module documentation for more information.

Returning `{:noreply, new_state, :hibernate}` is similar to `{:noreply, new_state}` except the process is hibernated before continuing the loop. See `handle_call/3` for more information.

Returning `{:noreply, new_state, {:continue, continue_arg}}` is similar to `{:noreply, new_state}` except `handle_continue/2` will be invoked immediately after with `continue_arg` as the first argument and `state` as the second one.

Returning `{:stop, reason, new_state}` stops the loop and `terminate/2` is called with the reason `reason` and state `new_state`. The process exits with reason `reason`.

This callback is optional. If one is not implemented, the server will fail if a cast is performed against it.

handle_continue(continue_arg, state)

(optional)

```
@callback handle_continue(continue_arg, state :: term()) ::
  {:noreply, new_state}
  | {:noreply, new_state, timeout() | :hibernate |
{:continue, continue_arg}}
  | {:stop, reason :: term(), new_state}
when new_state: term(), continue_arg: term()
```

Invoked to handle continue instructions.

It is useful for performing work after initialization or for splitting the work in a callback in multiple steps, updating the process state along the way.

Return values are the same as `handle_cast/2`.

This callback is optional. If one is not implemented, the server will fail if a continue instruction is used.

handle_info(msg, state)

(optional)

```
@callback handle_info(msg :: :timeout | term(), state ::
term()) ::
  {:noreply, new_state}
  | {:noreply, new_state,
     timeout() | :hibernate | {:continue, continue_arg ::
term()}}
  | {:stop, reason :: term(), new_state}
when new_state: term()
```

Invoked to handle all other messages.

`msg` is the message and `state` is the current state of the `GenServer`. When a timeout occurs the message is `:timeout`.

Return values are the same as `handle_cast/2`.

This callback is optional. If one is not implemented, the received message will be logged.

init(init_arg)

```
@callback init(init_arg :: term()) ::
  {:ok, state}
  | {:ok, state, timeout() | :hibernate | {:continue,
continue_arg :: term()}}
  | :ignore
  | {:stop, reason :: term()}
when state: term()
```

Invoked when the server is started. `start_link/3` or `start/3` will block until it returns.

`init_arg` is the argument term (second argument) passed to `start_link/3`.

Returning `{:ok, state}` will cause `start_link/3` to return `{:ok, pid}` and the process to enter its loop.

Returning `{:ok, state, timeout}` is similar to `{:ok, state}`, except that it also sets a timeout. See the "Timeouts" section in the module documentation for more information.

Returning `{:ok, state, :hibernate}` is similar to `{:ok, state}` except the process is hibernated before entering the loop. See `handle_call/3` for more information on hibernation.

Returning `{:ok, state, {:continue, continue_arg}}` is similar to `{:ok, state}` except that immediately after entering the loop, the `handle_continue/2` callback will be invoked with `continue_arg` as the first argument and `state` as the second one.

Returning `:ignore` will cause `start_link/3` to return `:ignore` and the process will exit normally without entering the loop or calling `terminate/2`. If used when part of a supervision tree the parent supervisor will not fail to start nor immediately try to restart the `GenServer`. The remainder of the supervision tree will be started and so the `GenServer` should not be required by other processes. It can be started later with `Supervisor.restart_child/2` as the child specification is saved in the parent supervisor. The main use cases for this are:

- The `GenServer` is disabled by configuration but might be enabled later.
- An error occurred and it will be handled by a different mechanism than the `Supervisor`. Likely this approach involves calling `Supervisor.restart_child/2` after a delay to attempt a restart.

Returning `{:error, reason}` will cause `start_link/3` to return `{:error, reason}`.

Returning `{:stop, reason}` will the process to exit with reason `reason`, without entering the loop or calling `terminate/2`. `start_link/3` will return `{:error, reason}`, but only if the caller is trapping exits.

terminate(reason, state)

(optional)

```
@callback terminate(reason, state :: term()) :: term()
when reason: :normal | :shutdown | {:shutdown, term()} |
term()
```

Invoked when the server is about to exit. It should do any cleanup required.

`reason` is exit reason and `state` is the current state of the `GenServer`. The return value is ignored.

`terminate/2` is useful for cleanup that requires access to the `GenServer`'s state. However, it is **not guaranteed** that `terminate/2` is called when a `GenServer` exits. Therefore, important cleanup should be done using process links and/or monitors. A monitoring process will receive the same exit `reason` that would be passed to `terminate/2`.

`terminate/2` is called if:

- the `GenServer` traps exits (using `Process.flag/2`) *and* the parent process (the one which called `start_link/1`) sends an exit signal
- a callback (except `init/1`) does one of the following:
    - returns a `:stop` tuple
    - raises (via `raise/2`) or exits (via `exit/1`)

- returns an invalid value

If part of a supervision tree, a `GenServer` will receive an exit signal from its parent process (its supervisor) when the tree is shutting down. The exit signal is based on the shutdown strategy in the child's specification, where this value can be:

- `:brutal_kill`: the `GenServer` is killed and so `terminate/2` is not called.
- a timeout value, where the supervisor will send the exit signal `:shutdown` and the `GenServer` will have the duration of the timeout to terminate. If after duration of this timeout the process is still alive, it will be killed immediately.

For a more in-depth explanation, please read the "Shutdown values (:shutdown)" section in the `Supervisor` module.

If the `GenServer` receives an exit signal (that is not `:normal`) from any process when it is not trapping exits it will exit abruptly with the same reason and so not call `terminate/2`. Note that a process does *NOT* trap exits by default and an exit signal is sent when a linked process exits or its node is disconnected.

`terminate/2` is only called after the `GenServer` finishes processing all messages which arrived in its mailbox prior to the exit signal. If it receives a `:kill` signal before it finishes processing those, `terminate/2` will not be called. If `terminate/2` is called, any messages received after the exit signal will still be in the mailbox.

There is no cleanup needed when the `GenServer` controls a `port` (for example, `:gen_tcp.socket`) or `File.io_device/0`, because these will be closed on receiving a `GenServer`'s exit signal and do not need to be closed manually in `terminate/2`.

If `reason` is neither `:normal`, `:shutdown`, nor `{:shutdown, term}` an error is logged.

This callback is optional.

Functions

abcast(nodes \\ [node() | Node.list()], name, request)

```
@spec abcast([node()], name :: atom(), term()) :: :abcast
```

Casts all servers locally registered as `name` at the specified nodes.

This function returns immediately and ignores nodes that do not exist, or where the server name does not exist.

See `multi_call/4` for more information.

call(server, request, timeout \\ 5000)

```
@spec call(server(), term(), timeout()) :: term()
```

Makes a synchronous call to the `server` and waits for its reply.

The client sends the given `request` to the server and waits until a reply arrives or a timeout occurs. `handle_call/3` will be called on the server to handle the request.

`server` can be a PID or any of the other values described in the "Name registration" section of the documentation for this module.

Timeouts

`timeout` is an integer greater than zero which specifies how many milliseconds to wait for a reply, or the atom `:infinity` to wait indefinitely. The default value is `5000`. If no reply is received within the specified time, the function call fails and the caller exits. If the caller catches the failure and continues running, and the server is just late with the reply, it may arrive at any time later into the caller's message queue. The caller must in this case be prepared for this and discard any such garbage messages that are two-element tuples with a reference as the first element.

cast(server, request)

```
@spec cast(server(), term()) :: :ok
```

Casts a request to the `server` without waiting for a response.

This function always returns `:ok` regardless of whether the destination `server` (or node) exists. Therefore it is unknown whether the destination `server` successfully handled the request.

`server` can be any of the values described in the "Name registration" section of the documentation for this module.

multi_call(nodes \\ [node() | Node.list()], name, request, timeout \\ :infinity)

```
@spec multi_call([node()], name :: atom(), term(), timeout())
::
  {replies :: [{node(), term()}], bad_nodes :: [node()]}
```

Calls all servers locally registered as `name` at the specified `nodes`.

First, the `request` is sent to every node in `nodes`; then, the caller waits for the replies. This function returns a two-element tuple `{replies, bad_nodes}` where:

- `replies` - is a list of `{node, reply}` tuples where `node` is the node that replied and `reply` is its reply
- `bad_nodes` - is a list of nodes that either did not exist or where a server with the given `name` did not exist or did not reply

`nodes` is a list of node names to which the request is sent. The default value is the list of all known nodes (including this node).

Examples

Assuming the `Stack` GenServer mentioned in the docs for the `GenServer` module is registered as `Stack` in the `:"foo@my-machine"` and `:"bar@my-machine"` nodes:

```
GenServer.multi_call(Stack, :pop)
#=> {[{:"foo@my-machine", :hello}, {:"bar@my-machine",
:world}], []}
```

reply(client, reply)

```
@spec reply(from(), term()) :: :ok
```

Replies to a client.

This function can be used to explicitly send a reply to a client that called `call/3` or `multi_call/4` when the reply cannot be specified in the

return value of `handle_call/3`.

`client` must be the `from` argument (the second argument) accepted by `handle_call/3` callbacks. `reply` is an arbitrary term which will be given back to the client as the return value of the call.

Note that `reply/2` can be called from any process, not just the GenServer that originally received the call (as long as that GenServer communicated the `from` argument somehow).

This function always returns `:ok`.

Examples

```
def handle_call(:reply_in_one_second, from, state) do
  Process.send_after(self(), {:reply, from}, 1_000)
  {:noreply, state}
end

def handle_info({:reply, from}, state) do
  GenServer.reply(from, :one_second_has_passed)
  {:noreply, state}
end
```

start(module, init_arg, options \\ [])

```
@spec start(module(), term(), options()) :: on_start()
```

Starts a `GenServer` process without links (outside of a supervision tree).

See `start_link/3` for more information.

start_link(module, init_arg, options \\ [])

```
@spec start_link(module(), term(), options()) :: on_start()
```

Starts a `GenServer` process linked to the current process.

This is often used to start the `GenServer` as part of a supervision tree.

Once the server is started, the `init/1` function of the given `module` is called with `init_arg` as its argument to initialize the server. To ensure a synchronized start-up procedure, this function does not return until `init/1` has returned.

Note that a `GenServer` started with `start_link/3` is linked to the parent process and will exit in case of crashes from the parent. The GenServer will also exit due to the `:normal` reasons in case it is configured to trap exits in the `init/1` callback.

Options

- `:name` - used for name registration as described in the "Name registration" section in the documentation for `GenServer`
- `:timeout` - if present, the server is allowed to spend the given number of milliseconds initializing or it will be terminated and the start function will return `{:error, :timeout}`
- `:debug` - if present, the corresponding function in the `:sys` module is invoked
- `:spawn_opt` - if present, its value is passed as options to the underlying process as in `Process.spawn/4`

- `:hibernate_after` - if present, the GenServer process awaits any message for the given number of milliseconds and if no message is received, the process goes into hibernation automatically (by calling `:proc_lib.hibernate/3`).

Return values

If the server is successfully created and initialized, this function returns `{:ok, pid}`, where `pid` is the PID of the server. If a process with the specified server name already exists, this function returns `{:error, {:already_started, pid}}` with the PID of that process.

If the `init/1` callback fails with `reason`, this function returns `{:error, reason}`. Otherwise, if it returns `{:stop, reason}` or `:ignore`, the process is terminated and this function returns `{:error, reason}` or `:ignore`, respectively.

stop(server, reason \\ :normal, timeout \\ :infinity)

```
@spec stop(server(), reason :: term(), timeout()) :: :ok
```

Synchronously stops the server with the given `reason`.

The `terminate/2` callback of the given `server` will be invoked before exiting. This function returns `:ok` if the server terminates with the given reason; if it terminates with another reason, the call exits.

This function keeps OTP semantics regarding error reporting. If the reason is any other than `:normal`, `:shutdown` or `{:shutdown, _}`, an error report is logged.

whereis(server)

```
@spec whereis(server()) :: pid() | {atom(), node()} | nil
```

Returns the `pid` or `{name, node}` of a GenServer process, `nil` otherwise.

To be precise, `nil` is returned whenever a `pid` or `{name, node}` cannot be returned. Note there is no guarantee the returned `pid` or `{name, node}` is alive, as a process could terminate immediately after it is looked up.

Examples

For example, to lookup a server process, monitor it and send a cast to it:

```
process = GenServer.whereis(server)
monitor = Process.monitor(process)
GenServer.cast(process, :hello)
```

Node

Functions related to VM nodes.

Some of the functions in this module are inlined by the compiler, similar to functions in the `Kernel` module and they are explicitly marked in their docs as "inlined by the compiler". For more information about inlined functions, check out the `Kernel` module.

Summary

Types

state()
t()

Functions

alive?()

Returns `true` if the local node is alive.

connect(node)

Establishes a connection to `node`.

disconnect(node)

Forces the disconnection of a node.

get_cookie()

Returns the magic cookie of the local node.

list()

Returns a list of all visible nodes in the system, excluding the local node.

list(args)

Returns a list of nodes according to argument given.

monitor(node, flag)

Monitors the status of the node.

monitor(node, flag, options)

Behaves as `monitor/2` except that it allows an extra option to be given, namely `:allow_passive_connect`.

ping(node)

Tries to set up a connection to node.

self()

Returns the current node.

set_cookie(node \\ Node.self(), cookie)

Sets the magic cookie of `node` to the atom `cookie`.

spawn(node, fun)

Returns the PID of a new process started by the application of `fun` on `node`. If `node` does not exist, a useless PID is returned.

spawn(node, fun, opts)

Returns the PID of a new process started by the application of `fun` on `node`.

spawn(node, module, fun, args)

Returns the PID of a new process started by the application of `module.function(args)` on `node`.

spawn(node, module, fun, args, opts)

Returns the PID of a new process started by the application of `module.function(args)` on `node`.

spawn_link(node, fun)

Returns the PID of a new linked process started by the application of `fun` on `node`.

spawn_link(node, module, fun, args)

Returns the PID of a new linked process started by the application of `module.function(args)` on `node`.

spawn_monitor(node, fun)

Spawns the given function on a node, monitors it and returns its PID and monitoring reference.

spawn_monitor(node, module, fun, args)

Spawns the given module and function passing the given args on a node, monitors it and returns its PID and monitoring reference.

start(name, opts \\ [])

Turns a non-distributed node into a distributed node.

stop()

Turns a distributed node into a non-distributed node.

Types

state()

```
@type state() :: :visible | :hidden | :connected | :this |
:known
```

t()

```
@type t() :: node()
```

Functions

alive?()

```
@spec alive?() :: boolean()
```

Returns `true` if the local node is alive.

That is, if the node can be part of a distributed system.

connect(node)

```
@spec connect(t()) :: boolean() | :ignored
```

Establishes a connection to `node`.

Returns `true` if successful, `false` if not, and the atom `:ignored` if the local node is not alive.

For more information, see `:net_kernel.connect_node/1`.

disconnect(node)

```
@spec disconnect(t()) :: boolean() | :ignored
```

Forces the disconnection of a node.

This will appear to the `node` as if the local node has crashed. This function is mainly used in the Erlang network authentication protocols. Returns `true` if disconnection succeeds, otherwise `false`. If the local node is not alive, the function returns `:ignored`.

For more information, see `:erlang.disconnect_node/1`.

get_cookie()

```
@spec get_cookie() :: atom()
```

Returns the magic cookie of the local node.

Returns the cookie if the node is alive, otherwise `:nocookie`.

list()

```
@spec list() :: [t()]
```

Returns a list of all visible nodes in the system, excluding the local node.

Same as `list(:visible)`.

Inlined by the compiler.

list(args)

```
@spec list(state() | [state()]) :: [t()]
```

Returns a list of nodes according to argument given.

The result returned when the argument is a list, is the list of nodes satisfying the disjunction(s) of the list elements.

For more information, see `:erlang.nodes/1`.

Inlined by the compiler.

monitor(node, flag)

```
@spec monitor(t(), boolean()) :: true
```

Monitors the status of the node.

If `flag` is `true`, monitoring is turned on. If `flag` is `false`, monitoring is turned off.

For more information, see `:erlang.monitor_node/2`.

For monitoring status changes of all nodes, see `:net_kernel.monitor_nodes/2`.

monitor(node, flag, options)

```
@spec monitor(t(), boolean(), [:allow_passive_connect]) ::
true
```

Behaves as `monitor/2` except that it allows an extra option to be given, namely `:allow_passive_connect`.

For more information, see `:erlang.monitor_node/3`.

For monitoring status changes of all nodes, see `:net_kernel.monitor_nodes/2`.

ping(node)

```
@spec ping(t()) :: :pong | :pang
```

Tries to set up a connection to node.

Returns `:pang` if it fails, or `:pong` if it is successful.

Examples

```
iex> Node.ping(:unknown_node)
:pang
```

self()

```
@spec self() :: t()
```

Returns the current node.

It returns the same as the built-in `node()`.

set_cookie(node \\ Node.self(), cookie)

```
@spec set_cookie(t(), atom()) :: true
```

Sets the magic cookie of `node` to the atom `cookie`.

The default node is `Node.self/0`, the local node. If `node` is the local node, the function also sets the cookie of all other unknown nodes to `cookie`.

This function will raise `FunctionClauseError` if the given `node` is not alive.

spawn(node, fun)

```
@spec spawn(t(), (-> any())) :: pid()
```

Returns the PID of a new process started by the application of `fun` on `node`. If `node` does not exist, a useless PID is returned.

For the list of available options, see `:erlang.spawn/2`.

Inlined by the compiler.

spawn(node, fun, opts)

```
@spec spawn(t(), (-> any()), Process.spawn_opts()) :: pid() |
{pid(), reference()}
```

Returns the PID of a new process started by the application of `fun` on `node`.

If `node` does not exist, a useless PID is returned.

For the list of available options, see `:erlang.spawn_opt/3`.

Inlined by the compiler.

spawn(node, module, fun, args)

```
@spec spawn(t(), module(), atom(), [any()]) :: pid()
```

Returns the PID of a new process started by the application of `module.function(args)` on `node`.

If `node` does not exist, a useless PID is returned.

For the list of available options, see `:erlang.spawn/4`.

Inlined by the compiler.

spawn(node, module, fun, args, opts)

```
@spec spawn(t(), module(), atom(), [any()],
Process.spawn_opts()) ::
  pid() | {pid(), reference()}
```

Returns the PID of a new process started by the application of `module.function(args)` on `node`.

If `node` does not exist, a useless PID is returned.

For the list of available options, see `:erlang.spawn_opt/5`.

Inlined by the compiler.

spawn_link(node, fun)

```
@spec spawn_link(t(), (-> any())) :: pid()
```

Returns the PID of a new linked process started by the application of `fun` on `node`.

A link is created between the calling process and the new process, atomically. If `node` does not exist, a useless PID is returned (and due to the link, an exit signal with exit reason `:noconnection` will be received).

Inlined by the compiler.

spawn_link(node, module, fun, args)

```
@spec spawn_link(t(), module(), atom(), [any()]) :: pid()
```

Returns the PID of a new linked process started by the application of `module.function(args)` on `node`.

A link is created between the calling process and the new process, atomically. If `node` does not exist, a useless PID is returned (and due to the link, an exit signal with exit reason `:noconnection` will be received).

Inlined by the compiler.

spawn_monitor(node, fun)

(since 1.14.0)

```
@spec spawn_monitor(t(), (-> any())) :: {pid(), reference()}
```

Spawns the given function on a node, monitors it and returns its PID and monitoring reference.

Inlined by the compiler.

spawn_monitor(node, module, fun, args)

(since 1.14.0)

```
@spec spawn_monitor(t(), module(), atom(), [any()]) ::
{pid(), reference()}
```

Spawns the given module and function passing the given args on a node, monitors it and returns its PID and monitoring reference.

Inlined by the compiler.

start(name, opts \\ [])

```
@spec start(node(),
  name_domain: :shortnames | :longnames,
  net_ticktime: pos integer(),
  net_tickintensity: 4..1000,
  dist_listen: boolean(),
  hidden: boolean()
) :: {:ok, pid()} | {:error, term()}
```

Turns a non-distributed node into a distributed node.

This functionality starts the `:net_kernel` and other related processes.

This function is rarely invoked in practice. Instead, nodes are named and started via the command line by using the `--sname` and `--name` flags. If you need to use this function to dynamically name a node, please make sure the `epmd` operating system process is running by calling `epmd -daemon`, such as `System.cmd("epmd", ["-daemon"])`.

Invoking this function when the distribution has already been started, either via the command line interface or dynamically, will return an error.

Examples

```
{:ok, pid} = Node.start(:example, name_domain:
:shortnames, hidden: true)
```

Options

Currently supported options are:

- `:name_domain` - determines the host name part of the node name. If `:longnames`, fully qualified domain names will be used which also is the default. If `:shortnames`, only the short name of the host will be used.
- `:net_ticktime` - The tick time to use in seconds. Defaults to the value of the `net_ticktime` configuration under Erlang's `kernel` application. See the `kernel` documentation for more information.
- `net_tickintensity` - The tick intensity to use. Defaults to the value of the `net_tickintensity` configuration under Erlang's `kernel` application. See the `kernel` documentation for more information.
- `:dist_listen` - Enable or disable listening for incoming connections. Defaults to the value given to the `--erl` flag, otherwise it defaults to `true`. Note that `dist_listen: false` implies `hidden: true`.
- `:hidden` - Enable or disable hidden node. Defaults to `true` if the `--hidden` flag is given to `elixir`'s CLI (or via the `--erl` flag), otherwise it defaults to `false`.

If `name` is set to `:undefined`, the distribution will be started to request a dynamic node name from the first node it connects to. Setting `name` to `:undefined` also implies options `dist_listen: false, hidden: true`.

stop()

```
@spec stop() :: :ok | {:error, :not_allowed | :not_found}
```

Turns a distributed node into a non-distributed node.

For other nodes in the network, this is the same as the node going down. Only possible when the node was started with `Node.start/2`, otherwise returns `{:error, :not_allowed}`. Returns `{:error, :not_found}` if the local node is not alive.

PartitionSupervisor

A supervisor that starts multiple partitions of the same child.

Certain processes may become bottlenecks in large systems. If those processes can have their state trivially partitioned, in a way there is no dependency between them, then they can use the `PartitionSupervisor` to create multiple isolated and independent partitions.

Once the `PartitionSupervisor` starts, you can dispatch to its children using `{:via, PartitionSupervisor, {name, key}}`, where `name` is the name of the `PartitionSupervisor` and key is used for routing.

This module was introduced in Elixir v1.14.0.

Simple Example

Let's start with an example which is not useful per se, but shows how the partitions are started and how messages are routed to them.

Here's a toy GenServer that simply collects the messages it's given. It prints them for easy illustration.

```
defmodule Collector do
  use GenServer

  def start_link(args) do
    GenServer.start_link(__MODULE__, args)
  end

  def init(args) do
    IO.inspect([__MODULE__, " got args ", args, " in ",
self()])
```

```
    {:ok, _initial_state = []}
  end

  def collect(server, msg) do
    GenServer.call(server, {:collect, msg})
  end

  def handle_call({:collect, msg}, _from, state) do
    new_state = [msg | state]
    IO.inspect(["current messages:", new_state, " in
process", self()])
    {:reply, :ok, new_state}
  end
end
```

To run multiple of these, we can start them under a [PartitionSupervisor] by placing this in our supervision tree:

```
{PartitionSupervisor,
  child_spec: Collector.child_spec([some: :arg]),
  name: MyApp.PartitionSupervisor
}
```

We can send messages to them using a "via tuple":

```
# The key is used to route our message to a particular
instance.
key = 1
Collector.collect({:via, PartitionSupervisor,
{MyApp.PartitionSupervisor, key}}, :hi)
# ["current messages:", [:hi], " in process",
#PID<0.602.0>]
:ok
```

```
Collector.collect({:via, PartitionSupervisor,
{MyApp.PartitionSupervisor, key}}, :ho)
# ["current messages:", [:ho, :hi], " in process",
#PID<0.602.0>]
:ok

# With a different key, the message will be routed to a
different instance.
key = 2
Collector.collect({:via, PartitionSupervisor,
{MyApp.PartitionSupervisor, key}}, :a)
# ["current messages:", [:a], " in process",
#PID<0.603.0>]
:ok
Collector.collect({:via, PartitionSupervisor,
{MyApp.PartitionSupervisor, key}}, :b)
# ["current messages:", [:b, :a], " in process",
#PID<0.603.0>]
:ok
```

Now let's move on to a useful example.

`DynamicSupervisor` Example

The `DynamicSupervisor` is a single process responsible for starting other processes. In some applications, the `DynamicSupervisor` may become a bottleneck. To address this, you can start multiple instances of the `DynamicSupervisor` through a `PartitionSupervisor`, and then pick a "random" instance to start the child on.

Instead of starting a single `DynamicSupervisor`:

```
children = [
  {DynamicSupervisor, name: MyApp.DynamicSupervisor}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

and starting children on that dynamic supervisor directly:

```
DynamicSupervisor.start_child(MyApp.DynamicSupervisor,
{Agent, fn -> %{} end})
```

You can start the dynamic supervisors under a `PartitionSupervisor`:

```
children = [
  {PartitionSupervisor,
   child_spec: DynamicSupervisor,
   name: MyApp.DynamicSupervisors}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

and then:

```
DynamicSupervisor.start_child(
  {:via, PartitionSupervisor, {MyApp.DynamicSupervisors,
self()}},
  {Agent, fn -> %{} end}
)
```

In the code above, we start a partition supervisor that will by default start a dynamic supervisor for each core in your machine. Then, instead of

calling the `DynamicSupervisor` by name, you call it through the partition supervisor using the `{:via, PartitionSupervisor, {name, key}}` format. We picked `self()` as the routing key, which means each process will be assigned one of the existing dynamic supervisors. See `start_link/1` to see all options supported by the `PartitionSupervisor`.

Implementation notes

The `PartitionSupervisor` uses either an ETS table or a `Registry` to manage all of the partitions. Under the hood, the `PartitionSupervisor` generates a child spec for each partition and then acts as a regular supervisor. The ID of each child spec is the partition number.

For routing, two strategies are used. If `key` is an integer, it is routed using `rem(abs(key), partitions)` where `partitions` is the number of partitions. Otherwise it uses `:erlang.phash2(key, partitions)`. The particular routing may change in the future, and therefore must not be relied on. If you want to retrieve a particular PID for a certain key, you can use `GenServer.whereis({:via, PartitionSupervisor, {name, key}})`.

Summary

Types

name()

The name of the `PartitionSupervisor`.

partition()

The "identifier" of a partition.
start_link_option()

The possible options to give to `start_link/0`.

Functions

count_children(supervisor)

Returns a map containing count values for the supervisor.

partitions(name)

Returns the number of partitions for the partition supervisor.

resize!(name, partitions)

Resizes the number of partitions in the PartitionSupervisor.

start_link(opts)

Starts a partition supervisor with the given options.

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

Synchronously stops the given partition supervisor with the given `reason`.

which_children(name)

Returns a list with information about all children of the given supervisor.

Types

name()

(since 1.14.0)

```
@type name() :: atom() | {:via, module(), term()}
```

The name of the `PartitionSupervisor`.

partition()

(since 1.19.0)

```
@type partition() :: non_neg_integer()
```

The "identifier" of a partition.

start_link_option()

(since 1.19.0)

```
@type start_link_option() ::
  {:name, name()}
  | {:child_spec, Supervisor.child_spec() |
Supervisor.module_spec()}
  | {:partitions, pos_integer()}
  | {:strategy, Supervisor.strategy()}
  | {:max_restarts, non_neg_integer()}
  | {:max_seconds, non_neg_integer()}
  | {:with_arguments,
     (args :: [term()], partition() -> updated_args ::
[term()])}
```

The possible options to give to `start_link/0`.

Functions

count_children(supervisor)

(since 1.14.0)

```
@spec count_children(name()) :: %{
  specs: non_neg_integer(),
  active: non_neg_integer(),
  supervisors: non_neg_integer(),
  workers: non_neg_integer()
}
```

Returns a map containing count values for the supervisor.

The map contains the following keys:

- `:specs` - the number of partitions (children processes)
- `:active` - the count of all actively running child processes managed by this supervisor
- `:supervisors` - the count of all supervisors whether or not the child process is still alive
- `:workers` - the count of all workers, whether or not the child process is still alive

partitions(name)

(since 1.14.0)

```
@spec partitions(name()) :: non_neg_integer()
```

Returns the number of partitions for the partition supervisor.

resize!(name, partitions)

(since 1.18.0)

```
@spec resize!(name(), non_neg_integer()) :: non_neg_integer()
```

Resizes the number of partitions in the PartitionSupervisor.

This is done by starting or stopping a given number of partitions in the supervisor. All of the child specifications are kept in the `PartitionSupervisor` itself.

The final number of partitions cannot be less than zero and cannot be more than the number of partitions the supervisor started with.

start_link(opts)

(since 1.14.0)

```
@spec start_link([start_link_option()]) ::
Supervisor.on_start()
```

Starts a partition supervisor with the given options.

This function is typically not invoked directly, instead it is invoked when using a `PartitionSupervisor` as a child of another supervisor:

```
children = [
  {PartitionSupervisor, child_spec: SomeChild, name:
MyPartitionSupervisor}
]
```

If the supervisor is successfully spawned, this function returns `{:ok, pid}`, where `pid` is the PID of the supervisor. If the given name for the partition supervisor is already assigned to a process, the function returns

`{:error, {:already_started, pid}}`, where `pid` is the PID of that process.

Note that a supervisor started with this function is linked to the parent process and exits not only on crashes but also if the parent process exits with `:normal` reason.

Options

See `start_link_option/0` for the type of each option.

- `:name` - an atom or via tuple representing the name of the partition supervisor. *Required*.
- `:child_spec` - the child spec to be used when starting the partitions. *Required*.
- `:partitions` - the number of partitions. Defaults to `System.schedulers_online/0` (typically the number of cores).
- `:strategy` - the restart strategy option. You can learn more about strategies in the `Supervisor` module docs. Defaults to `:one_for_one`.
- `:max_restarts` - the maximum number of restarts allowed in a time frame. Defaults to `3`.
- `:max_seconds` - the time frame in which `:max_restarts` applies. Defaults to `5`.
- `:with_arguments` - a two-argument anonymous function that allows the partition to be given to the child starting function. It takes the list of arguments passed to the child start function and the

partition itself, and must return possibly-updated arguments to give to the child start function. See the `:with_arguments` section below.

`:with_arguments`

Sometimes you want each partition to know their partition assigned number. This can be done with the `:with_arguments` option. This function receives the value of the `:child_spec` option and an integer for the partition number. It must return a new list of arguments that will be used to start the partition process.

For example, most processes are started by calling `start_link(opts)`, where `opts` is a keyword list. You could inject the partition into the options given to the child:

```
with_arguments: fn [opts], partition ->
  [Keyword.put(opts, :partition, partition)]
end
```

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

(since 1.14.0)

```
@spec stop(name(), reason :: term(), timeout()) :: :ok
```

Synchronously stops the given partition supervisor with the given `reason`.

It returns `:ok` if the supervisor terminates with the given reason. If it terminates with another reason, the call exits.

This function keeps OTP semantics regarding error reporting. If the reason is any other than `:normal`, `:shutdown` or `{:shutdown, _}`, an error report is logged.

which_children(name)

(since 1.14.0)

```
@spec which_children(name()) :: [
  {integer(), pid() | :restarting, :worker | :supervisor,
[module()] | :dynamic}
]
```

Returns a list with information about all children of the given supervisor.

This function returns a list of tuples containing:

- `id` - the partition number
- `child` - the PID of the corresponding child process or the atom `:restarting` if the process is about to be restarted
- `type` - `:worker` or `:supervisor` as defined in the child specification
- `modules` - as defined in the child specification

Process

Conveniences for working with processes and the process dictionary.

Besides the functions available in this module, the `Kernel` module exposes and auto-imports some basic functionality related to processes available through the following functions:

- `Kernel.spawn/1` and `Kernel.spawn/3`
- `Kernel.spawn_link/1` and `Kernel.spawn_link/3`
- `Kernel.spawn_monitor/1` and `Kernel.spawn_monitor/3`
- `Kernel.self/0`
- `Kernel.send/2`

While this module provides low-level conveniences to work with processes, developers typically use abstractions such as `Agent`, `GenServer`, `Registry`, `Supervisor` and `Task` for building their systems and resort to this module for gathering information, trapping exits, links and monitoring.

Aliases

Aliases are a feature introduced in Erlang/OTP 24. An alias is a way to refer to a PID in order to send messages to it. The advantage of using aliases is that they can be deactivated even if the aliased process is still running. If you send a message to a deactivated alias, nothing will happen. This makes request/response scenarios easier to implement.

You can use `alias/0` or `alias/1` to set an alias, and then you can send messages to that alias like you do with PIDs using `send/2`. To deactivate an alias, you can use `unalias/1`. If you send a message to a deactivated alias, nothing will happen.

For example, you could have a process that listens for `:ping` messages:

```
def server do
  receive do
    {:ping, source_alias} ->
      send(source_alias, :pong)
      server()
  end
end
```

Now, another process might ping this process:

```
server = spawn(&server/0)

source_alias = Process.alias()
send(server, {:ping, source_alias})

receive do
  :pong -> :pong
end
#=> :pong
```

If now you deactivate the `source_alias` and ping the server again, you won't receive any response since the server will `send/2` the `:pong` response to a deactivated alias.

```
Process.unalias(source_alias)
send(server, {:ping, source_alias})

receive do
  :pong -> :pong
after
  1000 -> :timeout
```

```
end
#=> :timeout
```

See also the Process Aliases section of the *Erlang reference manual*.

Summary

Types

alias()

An alias returned by `alias/0` or `alias/1`.

alias_opt()
dest()

A process destination.

process_info_item()
process_info_result_item()
spawn_opt()
spawn_opts()

Functions

alias()

Creates a process alias.

alias(options)

Creates a process alias.

alive?(pid)

Tells whether the given process is alive on the local node.

cancel_timer(timer_ref, options \\ [])

Cancels a timer returned by `send_after/3`.

delete(key)

Deletes the given `key` from the process dictionary.

demonitor(monitor_ref, options \\ [])

Demonitors the monitor identified by the given `reference`.

exit(pid, reason)

Sends an exit signal with the given `reason` to `pid`.

flag(flag, value)

Sets the given `flag` to `value` for the calling process.

flag(pid, flag, value)

Sets the given `flag` to `value` for the given process `pid`.

get()

Returns all key-value pairs in the process dictionary.

get(key, default \\ nil)

Returns the value for the given `key` in the process dictionary, or `default` if `key` is not set.

get_keys()

Returns all keys in the process dictionary.

get_keys(value)

Returns all keys in the process dictionary that have the given `value`.

get_label(pid \\ self())

Returns the label set for the process `pid` as set with `set_label/1` or `:proc_lib.set_label/1`.

group_leader()

Returns the PID of the group leader for the calling process.

group_leader(pid, leader)

Sets the group leader of the given `pid` to `leader`.

hibernate(mod, fun_name, args)

Puts the calling process into a "hibernation" state.

info(pid)

Returns information about the process identified by `pid`, or returns `nil` if the process is not alive.

info(pid, spec)

Returns information about the process identified by `pid`, or returns `nil` if the process is not alive.

link(pid_or_port)

Creates a link between the calling process and the given item (process or port).

list()

Returns a list of PIDs corresponding to all the processes currently existing on the local node.

monitor(item)

Starts monitoring the given `item` from the calling process.

monitor(item, options)

Starts monitoring the given `item` from the calling process.

put(key, value)

Stores the given `key-value` pair in the process dictionary.

read_timer(timer_ref)

Reads a timer created by `send_after/3`.

register(pid_or_port, name)

Registers the given `pid_or_port` under the given `name` on the local node.

registered()

Returns a list of names which have been registered using `register/2`.

send(dest, msg, options)

Sends a message to the given `dest`.

send_after(dest, msg, time, opts \\ [])

Sends `msg` to `dest` after `time` milliseconds.

set_label(label)

Add a descriptive term to the current process.

sleep(timeout)

Sleeps the current process for the given `timeout`.

spawn(fun, opts)

Spawns the given function according to the given options.

spawn(mod, fun, args, opts)

Spawns the given function `fun` from module `mod`, passing the given `args` according to the given options.

unalias(alias)

Explicitly deactivates a process alias.

unlink(pid_or_port)

Removes the link between the calling process and the given item (process or port).

unregister(name)

Removes the registered `name`, associated with a PID or a port identifier.

whereis(name)

Returns the PID or port identifier registered under `name` or `nil` if the name is not registered.

Types

alias()

```
@type alias() :: reference()
```

An alias returned by `alias/0` or `alias/1`.

See the module documentation for more information about aliases.

alias_opt()

```
@type alias_opt() :: :explicit_unalias | :reply
```

dest()

```
@type dest() ::
  pid()
  | port()
  | (registered_name :: atom())
  | {registered_name :: atom(), node()}
```

A process destination.

A remote or local PID, a local port, a locally registered name, or a tuple in the form of `{registered_name, node}` for a registered name at another node.

process_info_item()

```
@type process_info_item() :: atom() | {:dictionary, term()}
```

process_info_result_item()

```
@type process_info_result_item() :: {process info item(), term()}
```

spawn_opt()

```
@type spawn_opt() ::
  :link
  | :monitor
  | {:monitor, :erlang.monitor option()}
  | {:priority, :low | :normal | :high}
  | {:fullsweep_after, non neg integer()}
  | {:min_heap_size, non neg integer()}
  | {:min_bin_vheap_size, non neg integer()}
  | {:max_heap_size, heap_size()}
  | {:message_queue_data, :off_heap | :on_heap}
```

spawn_opts()

```
@type spawn_opts() :: [spawn opt()]
```

Functions

alias()

(since 1.15.0)

```
@spec alias() :: alias()
```

Creates a process alias.

This is the same as calling `alias/1` as `alias([:explicit_unalias])`. See also `:erlang.alias/0`.

Inlined by the compiler.

Examples

```
alias = Process.alias()
```

alias(options)

(since 1.15.0)

```
@spec alias([alias_opt()]) :: alias()
```

Creates a process alias.

See the module documentation for more information about aliases. See also `:erlang.alias/1`.

Inlined by the compiler.

Examples

```
alias = Process.alias([:reply])
```

alive?(pid)

```
@spec alive?(pid()) :: boolean()
```

Tells whether the given process is alive on the local node.

If the process identified by `pid` is alive (that is, it's not exiting and has not exited yet) then this function returns `true`. Otherwise, it returns `false`.

`pid` must refer to a process running on the local node or `ArgumentError` is raised. To check whether a process on any node is alive you can use the `:erpc` module.

```
:erpc.call(node(pid), Process, :alive?, [pid])
```

Inlined by the compiler.

cancel_timer(timer_ref, options \\ [])

```
@spec cancel_timer(reference(), options) :: non_neg_integer()
| false | :ok
when options: [async: boolean(), info: boolean()]
```

Cancels a timer returned by `send_after/3`.

When the result is an integer, it represents the time in milliseconds left until the timer would have expired.

When the result is `false`, a timer corresponding to `timer_ref` could not be found. This can happen either because the timer expired, because it has already been canceled, or because `timer_ref` never corresponded to a timer.

Even if the timer had expired and the message was sent, this function does not tell you if the timeout message has arrived at its destination yet.

Inlined by the compiler.

Options

- `:async` - (boolean) when `false`, the request for cancellation is synchronous. When `true`, the request for cancellation is asynchronous, meaning that the request to cancel the timer is issued and `:ok` is returned right away. Defaults to `false`.

- `:info` - (boolean) whether to return information about the timer being cancelled. When the `:async` option is `false` and `:info` is `true`, then either an integer or `false` (like described above) is returned. If `:async` is `false` and `:info` is `false`, `:ok` is returned. If `:async` is `true` and `:info` is `true`, a message in the form `{:cancel_timer, timer_ref, result}` (where `result` is an integer or `false` like described above) is sent to the caller of this function when the cancellation has been performed. If `:async` is `true` and `:info` is `false`, no message is sent. Defaults to `true`.

delete(key)

```
@spec delete(term()) :: term() | nil
```

Deletes the given `key` from the process dictionary.

Returns the value that was under `key` in the process dictionary, or `nil` if `key` was not stored in the process dictionary.

Examples

```
iex> Process.put(:comments, ["comment", "other comment"])
iex> Process.delete(:comments)
["comment", "other comment"]
```

```
iex> Process.delete(:comments)
nil
```

demonitor(monitor_ref, options \\ [])

```
@spec demonitor(reference(), options :: [:flush | :info]) ::
boolean()
```

Demonitors the monitor identified by the given `reference`.

If `monitor_ref` is a reference which the calling process obtained by calling `monitor/1`, that monitoring is turned off. If the monitoring is already turned off, nothing happens.

See `:erlang.demonitor/2` for more information.

Inlined by the compiler.

Examples

```
pid = spawn(fn -> 1 + 2 end)
ref = Process.monitor(pid)
Process.demonitor(ref)
#=> true
```

exit(pid, reason)

```
@spec exit(pid(), term()) :: true
```

Sends an exit signal with the given `reason` to `pid`.

Exit behavior differs based on the value of `reason`:

- If `:normal`, `pid` will not exit unless it is the calling process, in which case it will exit with the reason `:normal`. If it is trapping exits, the exit signal is transformed into a message `{:EXIT, from, :normal}` and delivered to its message queue.
- If `:kill`, which occurs when `Process.exit(pid, :kill)` is called, an untrappable exit signal is sent to `pid` which will unconditionally exit with reason `:killed`.
- If any other term and `pid` is not trapping exits, `pid` will exit with the given `reason`.
- If any other term and `pid` is trapping exits, the exit signal is transformed into a message `{:EXIT, from, reason}` and delivered to its message queue.

Inlined by the compiler.

> **Differences to `Kernel.exit/1`**
>
> The functions `Kernel.exit/1` and `Process.exit/2` are named similarly but provide very different functionalities. The `Kernel:exit/1` function should be used when the intent is to stop the current process while `Process.exit/2` should be used when the intent is to send an exit signal to another process. Note also that `Kernel.exit/1` can be caught with `try/1` while `Process.exit/2` can only be handled by trapping exits and when the signal is different than `:kill`.

Examples

```
Process.exit(pid, :kill)
#=> true
```

flag(flag, value)

```
@spec flag(:error_handler, module()) :: module()
@spec flag(:max_heap_size, heap_size()) :: heap_size()
@spec flag(:message_queue_data, :off_heap | :on_heap) ::
:off_heap | :on_heap
@spec flag(:min_bin_vheap_size, non_neg_integer()) ::
non_neg_integer()
@spec flag(:min_heap_size, non_neg_integer()) ::
non_neg_integer()
@spec flag(:priority, priority_level()) :: priority_level()
@spec flag(:save_calls, 0..10000) :: 0..10000
@spec flag(:sensitive, boolean()) :: boolean()
@spec flag(:trap_exit, boolean()) :: boolean()
```

Sets the given `flag` to `value` for the calling process.

Returns the old value of `flag`.

See `:erlang.process_flag/2` for more information.

Inlined by the compiler.

flag(pid, flag, value)

```
@spec flag(pid(), :save_calls, 0..10000) :: 0..10000
```

Sets the given `flag` to `value` for the given process `pid`.

Returns the old value of `flag`.

It raises `ArgumentError` if `pid` is not a local process.

The allowed values for `flag` are only a subset of those allowed in `flag/2`, namely `:save_calls`.

See `:erlang.process_flag/3` for more information.

Inlined by the compiler.

get()

```
@spec get() :: [{term(), term()}]
```

Returns all key-value pairs in the process dictionary.

Inlined by the compiler.

get(key, default \\ nil)

```
@spec get(term(), default :: term()) :: term()
```

Returns the value for the given `key` in the process dictionary, or `default` if `key` is not set.

Examples

```
# Assuming :locale was not set
iex> Process.get(:locale, "pt")
"pt"
```

```
iex> Process.put(:locale, "fr")
nil
iex> Process.get(:locale, "pt")
"fr"
```

get_keys()

```
@spec get_keys() :: [term()]
```

Returns all keys in the process dictionary.

Inlined by the compiler.

Examples

```
# Assuming :locale was not set
iex> :locale in Process.get_keys()
false
iex> Process.put(:locale, "pt")
nil
iex> :locale in Process.get_keys()
true
```

get_keys(value)

```
@spec get_keys(term()) :: [term()]
```

Returns all keys in the process dictionary that have the given `value`.

Inlined by the compiler.

get_label(pid \\ self())

(since 1.20.0)

```
@spec get_label(pid()) :: term()
```

Returns the label set for the process `pid` as set with `set_label/1` or `:proc_lib.set_label/1`.

Defaults to the current process when `pid` is not passed.

Examples

```
Process.set_label({:any, "term"})
Process.get_label()
#=> {:any, "term"}
```

Returns `nil` when not set:

```
Process.get_label(pid)
#=> nil
```

group_leader()

```
@spec group_leader() :: pid()
```

Returns the PID of the group leader for the calling process.

Inlined by the compiler.

Examples

```
Process.group_leader()
#=> #PID<0.53.0>
```

group_leader(pid, leader)

```
@spec group_leader(pid(), leader :: pid()) :: true
```

Sets the group leader of the given `pid` to `leader`.

Typically, this is used when a process started from a certain shell should have a group leader other than `:init`.

Inlined by the compiler.

hibernate(mod, fun_name, args)

```
@spec hibernate(module(), atom(), list()) :: no_return()
```

Puts the calling process into a "hibernation" state.

The calling process is put into a waiting state where its memory allocation has been reduced as much as possible, which is useful if the process does not expect to receive any messages in the near future.

See `:erlang.hibernate/3` for more information.

Inlined by the compiler.

info(pid)

```
@spec info(pid()) :: keyword() | nil
```

Returns information about the process identified by `pid`, or returns `nil` if the process is not alive.

Use this only for debugging information.

See `:erlang.process_info/1` for more information.

info(pid, spec)

```
@spec info(pid(), process_info_item()) ::
process_info_result_item() | nil
@spec info(pid(), [process_info_item()]) ::
[process_info_result_item()] | nil
```

Returns information about the process identified by `pid`, or returns `nil` if the process is not alive.

See `:erlang.process_info/2` for more information.

link(pid_or_port)

```
@spec link(pid() | port()) :: true
```

Creates a link between the calling process and the given item (process or port).

Links are bidirectional. Linked processes can be unlinked by using `unlink/1`.

If such a link exists already, this function does nothing since there can only be one link between two given processes. If a process tries to create a link to itself, nothing will happen.

When two processes are linked, each one receives exit signals from the other (see also `exit/2`). Let's assume `pid1` and `pid2` are linked. If `pid2` exits with a reason other than `:normal` (which is also the exit reason used when a process finishes its job) and `pid1` is not trapping exits (see `flag/2`), then `pid1` will exit with the same reason as `pid2` and in turn emit an exit signal to all its other linked processes. The behavior when `pid1` is trapping exits is described in `exit/2`.

See `:erlang.link/1` for more information.

Inlined by the compiler.

list()

```
@spec list() :: [pid()]
```

Returns a list of PIDs corresponding to all the processes currently existing on the local node.

Note that if a process is exiting, it is considered to exist but not be alive. This means that for such process, `alive?/1` will return `false` but its PID will be part of the list of PIDs returned by this function.

See `:erlang.processes/0` for more information.

Inlined by the compiler.

Examples

```
Process.list()
#=> [#PID<0.0.0>, #PID<0.1.0>, #PID<0.2.0>, #PID<0.3.0>,
...]
```

monitor(item)

```
@spec monitor(pid() | {name, node()} | name) :: reference()
when name: atom()
```

Starts monitoring the given `item` from the calling process.

Once the monitored process dies, a message is delivered to the monitoring process in the shape of:

```
{:DOWN, ref, :process, object, reason}
```

where:

- `ref` is a monitor reference returned by this function;
- `object` is either a `pid` of the monitored process (if monitoring a PID) or `{name, node}` (if monitoring a remote or local name);
- `reason` is the exit reason.

If the process is already dead when calling `Process.monitor/1`, a `:DOWN` message is delivered immediately.

See "Links and monitors" for an example. See `:erlang.monitor/2` for more information.

Inlined by the compiler.

Examples

```
pid = spawn(fn -> 1 + 2 end)
#=> #PID<0.118.0>
Process.monitor(pid)
#=> #Reference<0.906660723.3006791681.40191>
Process.exit(pid, :kill)
#=> true
receive do
  msg -> msg
end
#=> {:DOWN, #Reference<0.906660723.3006791681.40191>,
:process, #PID<0.118.0>, :noproc}
```

monitor(item, options)

(since 1.15.0)

```
@spec monitor(pid() | {name, node()} | name,
[:erlang.monitor_option()]) ::
  reference()
when name: atom()
```

Starts monitoring the given `item` from the calling process.

This function is similar to `monitor/1`, but accepts options to customize how `item` is monitored. See `:erlang.monitor/3` for documentation on those options.

Inlined by the compiler.

Examples

```
pid =
  spawn(fn ->
    receive do
      {:ping, source_alias} -> send(source_alias, :pong)
    end
  end)
#=> #PID<0.118.0>

ref_and_alias = Process.monitor(pid, alias:
:reply_demonitor)
#=> #Reference<0.906660723.3006791681.40191>

send(pid, {:ping, ref_and_alias})

receive do: (msg -> msg)
#=> :pong

ref_and_alias = Process.monitor(pid, alias:
:reply_demonitor)
#=> #Reference<0.906660723.3006791681.40191>

send(pid, {:ping, ref_and_alias})

receive do: (msg -> msg)
#=> {:DOWN, #Reference<0.906660723.3006791681.40191>,
:process, #PID<0.118.0>, :noproc}
```

put(key, value)

```
@spec put(term(), term()) :: term() | nil
```

Stores the given `key-value` pair in the process dictionary.

The return value of this function is the value that was previously stored under `key`, or `nil` in case no value was stored under it.

Examples

```
# Assuming :locale was not set
iex> Process.put(:locale, "en")
nil
iex> Process.put(:locale, "fr")
"en"
```

read_timer(timer_ref)

```
@spec read_timer(reference()) :: non_neg_integer() | false
```

Reads a timer created by `send_after/3`.

When the result is an integer, it represents the time in milliseconds left until the timer will expire.

When the result is `false`, a timer corresponding to `timer_ref` could not be found. This can be either because the timer expired, because it has already been canceled, or because `timer_ref` never corresponded to a timer.

Even if the timer had expired and the message was sent, this function does not tell you if the timeout message has arrived at its destination yet.

Inlined by the compiler.

register(pid_or_port, name)

```
@spec register(pid() | port(), atom()) :: true
```

Registers the given `pid_or_port` under the given `name` on the local node.

`name` must be an atom and can then be used instead of the PID/port identifier when sending messages with `Kernel.send/2`.

`register/2` will fail with `ArgumentError` in any of the following cases:

- the PID/Port is not existing locally and alive
- the name is already registered
- the `pid_or_port` is already registered under a different `name`

The following names are reserved and cannot be assigned to processes nor ports:

- `nil`
- `false`
- `true`
- `:undefined`

Examples

```
Process.register(self(), :test)
#=> true
send(:test, :hello)
#=> :hello
send(:wrong_name, :hello)
** (ArgumentError) argument error
```

registered()

```
@spec registered() :: [atom()]
```

Returns a list of names which have been registered using `register/2`.

Inlined by the compiler.

Examples

```
Process.register(self(), :test)
Process.registered()
#=> [:test, :elixir_config, :inet_db, ...]
```

send(dest, msg, options)

```
@spec send(dest, msg, [option]) :: :ok | :noconnect |
:nosuspend
when dest: dest(), msg: any(), option: :noconnect |
:nosuspend
```

Sends a message to the given `dest`.

`dest` may be a remote or local PID, a local port, a locally registered name, or a tuple in the form of `{registered_name, node}` for a registered name at another node.

Inlined by the compiler.

Options

- `:noconnect` - when used, if sending the message would require an auto-connection to another node the message is not sent and `:noconnect` is returned.

- `:nosuspend` - when used, if sending the message would cause the sender to be suspended the message is not sent and `:nosuspend` is returned.

Otherwise the message is sent and `:ok` is returned.

Examples

```
iex> Process.send({:name, :node_that_does_not_exist},
:hi, [:noconnect])
:noconnect
```

send_after(dest, msg, time, opts \\ [])

```
@spec send_after(pid() | atom(), term(), non_neg_integer(),
[option]) :: reference()
when option: {:abs, boolean()}
```

Sends `msg` to `dest` after `time` milliseconds.

If `dest` is a PID, it must be the PID of a local process, dead or alive. If `dest` is an atom, it must be the name of a registered process which is looked up at the time of delivery. No error is produced if the name does not refer to a process.

The message is not sent immediately. Therefore, `dest` can receive other messages in-between even when `time` is `0`.

This function returns a timer reference, which can be read with `read_timer/1` or canceled with `cancel_timer/1`.

The timer will be automatically canceled if the given `dest` is a PID which is not alive or when the given PID exits. Note that timers will not

be automatically canceled when `dest` is an atom (as the atom resolution is done on delivery).

Options

- `:abs` - (boolean) when `false`, `time` is treated as relative to the current monotonic time. When `true`, `time` is the absolute value of the Erlang monotonic time at which `msg` should be delivered to `dest`. To read more about Erlang monotonic time and other time-related concepts, look at the documentation for the `System` module. Defaults to `false`.

Examples

```
timer_ref = Process.send_after(pid, :hi, 1000)
```

set_label(label)

(since 1.17.0)

```
@spec set_label(term()) :: :ok
```

Add a descriptive term to the current process.

The term does not need to be unique, and will be shown in Observer and in crash logs. This label may be useful for identifying a process as one of multiple in a given role, such as `:queue_worker` or `{:live_chat, user_id}`.

Examples

```
Process.set_label(:worker)
#=> :ok

Process.set_label({:any, "term"})
#=> :ok
```

sleep(timeout)

```
@spec sleep(timeout()) :: :ok
```

Sleeps the current process for the given `timeout`.

`timeout` is either the number of milliseconds to sleep as an integer or the atom `:infinity`. When `:infinity` is given, the current process will sleep forever, and not consume or reply to messages.

> **Sleeping limit**
>
> Before Elixir v1.18, `sleep/1` did not accept integer timeout values greater than `16#ffffffff`, that is, `2^32-1`. Since Elixir v1.18, arbitrarily-high integer values are accepted.

Use this function with extreme care. For almost all situations where you would use `sleep/1` in Elixir, there is likely a more correct, faster and precise way of achieving the same with message passing.

For example, if you are waiting for a process to perform some action, it is better to communicate the progress of such action with messages.

In other words, **do not**:

```
Task.start_link(fn ->
  do_something()
```

```
  ...
end)

# Wait until work is done
Process.sleep(2000)
```

But **do**:

```
parent = self()

Task.start_link(fn ->
  do_something()
  send(parent, :work_is_done)
  ...
end)

receive do
  :work_is_done -> :ok
after
  # Optional timeout
  30_000 -> :timeout
end
```

For cases like the one above, `Task.async/1` and `Task.await/2` are preferred.

Similarly, if you are waiting for a process to terminate, monitor that process instead of sleeping. **Do not**:

```
Task.start_link(fn ->
  ...
end)
```

```
# Wait until task terminates
Process.sleep(2000)
```

Instead **do**:

```
{:ok, pid} =
  Task.start_link(fn ->
    ...
  end)

ref = Process.monitor(pid)

receive do
  {:DOWN, ^ref, _, _, _} -> :task_is_down
after
  # Optional timeout
  30_000 -> :timeout
end
```

spawn(fun, opts)

```
@spec spawn((-> any()), spawn_opts()) :: pid() | {pid(), reference()}
```

Spawns the given function according to the given options.

The result depends on the given options. In particular, if `:monitor` is given as an option, it will return a tuple containing the PID and the monitoring reference, otherwise just the spawned process PID.

More options are available; for the comprehensive list of available options check `:erlang.spawn_opt/4`.

Inlined by the compiler.

Examples

```
Process.spawn(fn -> 1 + 2 end, [:monitor])
#=> {#PID<0.93.0>,
#Reference<0.18808174.1939079169.202418>}
Process.spawn(fn -> 1 + 2 end, [:link])
#=> #PID<0.95.0>
```

spawn(mod, fun, args, opts)

```
@spec spawn(module(), atom(), list(), spawn_opts()) :: pid()
| {pid(), reference()}
```

Spawns the given function `fun` from module `mod`, passing the given `args` according to the given options.

The result depends on the given options. In particular, if `:monitor` is given as an option, it will return a tuple containing the PID and the monitoring reference, otherwise just the spawned process PID.

It also accepts extra options, for the list of available options check `:erlang.spawn_opt/4`.

Inlined by the compiler.

unalias(alias)

(since 1.15.0)

```
@spec unalias(alias()) :: boolean()
```

Explicitly deactivates a process alias.

Returns `true` if `alias` was a currently-active alias for current processes, or `false` otherwise.

See the module documentation for more information about aliases. See also `:erlang.unalias/1`.

Inlined by the compiler.

Examples

```
alias = Process.alias()
Process.unalias(alias)
#=> true
```

unlink(pid_or_port)

```
@spec unlink(pid() | port()) :: true
```

Removes the link between the calling process and the given item (process or port).

If there is no such link, this function does nothing. If `pid_or_port` does not exist, this function does not produce any errors and simply does nothing.

The return value of this function is always `true`.

See `:erlang.unlink/1` for more information.

Inlined by the compiler.

unregister(name)

```
@spec unregister(atom()) :: true
```

Removes the registered `name`, associated with a PID or a port identifier.

Fails with `ArgumentError` if the name is not registered to any PID or port.

Inlined by the compiler.

Examples

```
Process.register(self(), :test)
#=> true
Process.unregister(:test)
#=> true
Process.unregister(:wrong_name)
** (ArgumentError) argument error
```

whereis(name)

```
@spec whereis(atom()) :: pid() | port() | nil
```

Returns the PID or port identifier registered under `name` or `nil` if the name is not registered.

See `:erlang.whereis/1` for more information.

Examples

```
Process.register(self(), :test)
Process.whereis(:test)
#=> #PID<0.84.0>
Process.whereis(:wrong_name)
#=> nil
```

Registry

A local, decentralized and scalable key-value process storage.

It allows developers to lookup one or more processes with a given key. If the registry has `:unique` keys, a key points to 0 or 1 process. If the registry allows `:duplicate` keys, a single key may point to any number of processes. In both cases, different keys could identify the same process.

Each entry in the registry is associated to the process that has registered the key. If the process crashes, the keys associated to that process are automatically removed. All key comparisons in the registry are done using the match operation (`===/2`).

The registry can be used for different purposes, such as name lookups (using the `:via` option), storing properties, custom dispatching rules, or a pubsub implementation. We explore some of those use cases below.

The registry may also be transparently partitioned, which provides more scalable behavior for running registries on highly concurrent environments with thousands or millions of entries.

Using in `:via`

Once the registry is started with a given name using `Registry.start_link/1` , it can be used to register and access named processes using the `{:via, Registry, {registry, key}}` tuple:

```
{:ok, _} = Registry.start_link(keys: :unique, name:
MyApp.Registry)
name = {:via, Registry, {MyApp.Registry, "agent"}}
```

```
{:ok, _} = Agent.start_link(fn -> 0 end, name: name)
Agent.get(name, & &1)
#=> 0
Agent.update(name, &(&1 + 1))
Agent.get(name, & &1)
#=> 1
```

In the previous example, we were not interested in associating a value to the process:

```
Registry.lookup(MyApp.Registry, "agent")
#=> [{self(), nil}]
```

However, in some cases it may be desired to associate a value to the process using the alternate `{:via, Registry, {registry, key, value}}` tuple:

```
{:ok, _} = Registry.start_link(keys: :unique, name:
MyApp.Registry)
name = {:via, Registry, {MyApp.Registry, "agent",
:hello}}
{:ok, agent_pid} = Agent.start_link(fn -> 0 end, name:
name)

Registry.lookup(MyApp.Registry, "agent")
#=> [{agent_pid, :hello}]

name_without_meta = {:via, Registry, {MyApp.Registry,
"agent"}}
Agent.update(name_without_meta, fn x -> x + 1 end)
Agent.get(name_without_meta, & &1)
#=> 1
```

> **With *and* without metadata**
>
> When using the version of `:via` tuples with *metadata*, you can still use the version **without** metadata to look up the process.

To this point, we have been starting `Registry` using `start_link/1`. Typically the registry is started as part of a supervision tree though:

```
{Registry, keys: :unique, name: MyApp.Registry}
```

Only registries with unique keys can be used in `:via`. If the name is already taken, the case-specific `start_link` function (`Agent.start_link/2` in the example above) will return `{:error, {:already_started, current_pid}}`.

Using as a dispatcher

`Registry` has a dispatch mechanism that allows developers to implement custom dispatch logic triggered from the caller. For example, let's say we have a duplicate registry started as so:

```
{:ok, _} = Registry.start_link(keys: :duplicate, name:
Registry.DispatcherTest)
```

By calling `register/3`, different processes can register under a given key and associate any value under that key. In this case, let's register the current process under the key `"hello"` and attach the `{IO, :inspect}` tuple to it:

```
{:ok, _} = Registry.register(Registry.DispatcherTest,
```

```
"hello", {IO, :inspect})
```

Now, an entity interested in dispatching events for a given key may call `dispatch/3` passing in the key and a callback. This callback will be invoked with a list of all the values registered under the requested key, alongside the PID of the process that registered each value, in the form of `{pid, value}` tuples. In our example, `value` will be the `{module, function}` tuple in the code above:

```
Registry.dispatch(Registry.DispatcherTest, "hello", fn
entries ->
  for {pid, {module, function}} <- entries, do:
apply(module, function, [pid])
end)
# Prints #PID<...> where the PID is for the process that
called register/3 above
#=> :ok
```

Dispatching happens in the process that calls `dispatch/3` either serially or concurrently in case of multiple partitions (via spawned tasks). The registered processes are not involved in dispatching unless involving them is done explicitly (for example, by sending them a message in the callback).

Furthermore, if there is a failure when dispatching, due to a bad registration, dispatching will always fail and the registered process will not be notified. Therefore let's make sure we at least wrap and report those errors:

```
require Logger

Registry.dispatch(Registry.DispatcherTest, "hello", fn
entries ->
```

```
  for {pid, {module, function}} <- entries do
    try do
      apply(module, function, [pid])
    catch
      kind, reason ->
        formatted = Exception.format(kind, reason,
__STACKTRACE__)
        Logger.error("Registry.dispatch/3 failed with #
{formatted}")
    end
  end
end)
# Prints #PID<...>
#=> :ok
```

You could also replace the whole `apply` system by explicitly sending messages. That's the example we will see next.

Using as a PubSub

Registries can also be used to implement a local, non-distributed, scalable PubSub by relying on the `dispatch/3` function, similarly to the previous section: in this case, however, we will send messages to each associated process, instead of invoking a given module-function.

In this example, we will also set the number of partitions to the number of schedulers online, which will make the registry more performant on highly concurrent environments:

```
{:ok, _} =
  Registry.start_link(
    keys: :duplicate,
    name: Registry.PubSubTest,
    partitions: System.schedulers_online()
```

```
  )

{:ok, _} = Registry.register(Registry.PubSubTest,
"hello", [])

Registry.dispatch(Registry.PubSubTest, "hello", fn
entries ->
  for {pid, _} <- entries, do: send(pid, {:broadcast,
"world"})
end)
#=> :ok
```

The example above broadcasted the message `{:broadcast, "world"}` to all processes registered under the "topic" (or "key" as we called it until now) `"hello"`.

The third argument given to `register/3` is a value associated to the current process. While in the previous section we used it when dispatching, in this particular example we are not interested in it, so we have set it to an empty list. You could store a more meaningful value if necessary.

Registrations

Looking up, dispatching and registering are efficient and immediate at the cost of delayed unsubscription. For example, if a process crashes, its keys are automatically removed from the registry but the change may not propagate immediately. This means certain operations may return processes that are already dead. When such may happen, it will be explicitly stated in the function documentation.

However, keep in mind those cases are typically not an issue. After all, a process referenced by a PID may crash at any time, including between getting the value from the registry and sending it a message. Many parts

of the standard library are designed to cope with that, such as `Process.monitor/1` which will deliver the `:DOWN` message immediately if the monitored process is already dead and `send/2` which acts as a no-op for dead processes.

ETS

Note that the registry uses one ETS table plus two ETS tables per partition.

Summary

Types

body()

A pattern used to representing the output format part of a match spec

dispatch_opts()

Options used for `dispatch/4`.

guard()

A guard to be evaluated when matching on objects in a registry

guards()

A list of guards to be evaluated when matching on objects in a registry

key()

The type of keys allowed on registration

keys()

The type of the registry

listener_message()

The message that the registry sends to listeners when a process registers or unregisters.

match_pattern()

A pattern to match on objects in a registry

meta_key()

The type of registry metadata keys

meta_value()

The type of registry metadata values

registry()

The registry identifier

spec()

A full match spec used when selecting objects in the registry

start_option()

Options used for `child_spec/1` and `start_link/1`

value()

The type of values allowed on registration

Functions

child_spec(options)

Returns a specification to start a registry under a supervisor.

count(registry)

Returns the number of registered keys in a registry. It runs in constant time.

count_match(registry, key, pattern, guards \\ [])

Returns the number of `{pid, value}` pairs under the given `key` in `registry` that match `pattern`.

count_select(registry, spec)

Works like `select/2`, but only returns the number of matching records.

delete_meta(registry, key)

Deletes registry metadata for the given `key` in `registry`.

dispatch(registry, key, mfa_or_fun, opts \\ [])

Invokes the callback with all entries under `key` in each partition for the given `registry`.

keys(registry, pid)

Returns the known keys for the given `pid` in `registry` in no particular order.

lock(registry, lock_key, function)

Out-of-band locking of the given `lock_key` for the duration of `function`.

lookup(registry, key)

Finds the `{pid, value}` pair for the given `key` in `registry` in no particular order.

match(registry, key, pattern, guards \\ [])

Returns `{pid, value}` pairs under the given `key` in `registry` that match `pattern`.

meta(registry, key)

Reads registry metadata given on `start_link/1`.

put_meta(registry, key, value)

Stores registry metadata.

register(registry, key, value)

Registers the current process under the given `key` in `registry`.

select(registry, spec)

Select key, pid, and values registered using full match specs.

start_link(options)

Starts the registry as a supervisor process.

unregister(registry, key)

Unregisters all entries for the given `key` associated to the current process in `registry`.

unregister_match(registry, key, pattern, guards \\ [])

Unregisters entries for keys matching a pattern associated to the current process in `registry`.

update_value(registry, key, callback)

Updates the value for `key` for the current process in the unique `registry`.

values(registry, key, pid)

Reads the values for the given `key` for `pid` in `registry`.

Types

body()

```
@type body() :: [term()]
```

A pattern used to representing the output format part of a match spec

dispatch_opts()

```
@type dispatch_opts() :: [{:parallel, boolean()}]
```

Options used for `dispatch/4`.

guard()

```
@type guard() :: atom() | tuple()
```

A guard to be evaluated when matching on objects in a registry

guards()

```
@type guards() :: [guard()]
```

A list of guards to be evaluated when matching on objects in a registry

key()

```
@type key() :: term()
```

The type of keys allowed on registration

keys()

```
@type keys() :: :unique | :duplicate | {:duplicate, :key} |
{:duplicate, :pid}
```

The type of the registry

listener_message()

(since 1.15.0)

```
@type listener_message() ::
  {:register, registry(), key(), registry_partition :: pid(),
value()}
  | {:unregister, registry(), key(), registry_partition ::
pid()}
```

The message that the registry sends to listeners when a process registers or unregisters.

See the `:listeners` option in `start_link/1`.

match_pattern()

```
@type match_pattern() :: atom() | term()
```

A pattern to match on objects in a registry

meta_key()

```
@type meta_key() :: atom() | tuple()
```

The type of registry metadata keys

meta_value()

```
@type meta_value() :: term()
```

The type of registry metadata values

registry()

```
@type registry() :: atom()
```

The registry identifier

spec()

```
@type spec() :: [{match_pattern(), guards(), body()}]
```

A full match spec used when selecting objects in the registry

start_option()

```
@type start_option() ::
  {:keys, keys()}
  | {:name, registry()}
  | {:partitions, pos_integer()}
  | {:listeners, [atom()]}
  | {:meta, [{meta_key(), meta_value()}]}
```

Options used for `child_spec/1` and `start_link/1`

value()

```
@type value() :: term()
```

The type of values allowed on registration

Functions

child_spec(options)

(since 1.5.0)

```
@spec child_spec([start_option()]) :: Supervisor.child_spec()
```

Returns a specification to start a registry under a supervisor.

See `Supervisor`.

count(registry)

(since 1.7.0)

```
@spec count(registry()) :: non_neg_integer()
```

Returns the number of registered keys in a registry. It runs in constant time.

Examples

In the example below we register the current process and ask for the number of keys in the registry:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueCountTest)
iex> Registry.count(Registry.UniqueCountTest)
0
iex> {:ok, _} =
Registry.register(Registry.UniqueCountTest, "hello",
:world)
iex> {:ok, _} =
Registry.register(Registry.UniqueCountTest, "world",
:world)
iex> Registry.count(Registry.UniqueCountTest)
2
```

The same applies to duplicate registries:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateCountTest)
iex> Registry.count(Registry.DuplicateCountTest)
0
iex> {:ok, _} =
Registry.register(Registry.DuplicateCountTest, "hello",
:world)
iex> {:ok, _} =
Registry.register(Registry.DuplicateCountTest, "hello",
:world)
iex> Registry.count(Registry.DuplicateCountTest)
2
```

count_match(registry, key, pattern, guards \\ [])

(since 1.7.0)

```
@spec count_match(registry(), key(), match_pattern(),
guards()) :: non_neg_integer()
```

Returns the number of `{pid, value}` pairs under the given `key` in `registry` that match `pattern`.

Pattern must be an atom or a tuple that will match the structure of the value stored in the registry. The atom `:_` can be used to ignore a given value or tuple element, while the atom `:"$1"` can be used to temporarily assign part of pattern to a variable for a subsequent comparison.

Optionally, it is possible to pass a list of guard conditions for more precise matching. Each guard is a tuple, which describes checks that should be passed by assigned part of pattern. For example the `$1 > 1` guard condition would be expressed as the `{:>, :"$1", 1}` tuple. Please note that guard conditions will work only for assigned variables

like `:"$1"`, `:"$2"`, and so forth. Avoid usage of special match variables `:"$_"` and `:"$$"`, because it might not work as expected.

Zero will be returned if there is no match.

For unique registries, a single partition lookup is necessary. For duplicate registries, all partitions must be looked up.

Examples

In the example below we register the current process under the same key in a duplicate registry but with different values:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.CountMatchTest)
iex> {:ok, _} =
Registry.register(Registry.CountMatchTest, "hello", {1,
:atom, 1})
iex> {:ok, _} =
Registry.register(Registry.CountMatchTest, "hello", {2,
:atom, 2})
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {1, :_, :_})
1
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {2, :_, :_})
1
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {:_, :atom, :_})
2
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {:"$1", :_, :"$1"})
2
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {:_, :_, :"$1"}, [{:>, :"$1", 1}])
```

```
1
iex> Registry.count_match(Registry.CountMatchTest,
"hello", {:_, :"$1", :_}, [{:is_atom, :"$1"}])
2
```

count_select(registry, spec)

(since 1.14.0)

```
@spec count_select(registry(), spec()) :: non_neg_integer()
```

Works like `select/2`, but only returns the number of matching records.

Examples

In the example below we register the current process under different keys in a unique registry but with the same value:

```
iex> Registry.start_link(keys: :unique, name:
Registry.CountSelectTest)
iex> {:ok, _} =
Registry.register(Registry.CountSelectTest, "hello",
:value)
iex> {:ok, _} =
Registry.register(Registry.CountSelectTest, "world",
:value)
iex> Registry.count_select(Registry.CountSelectTest,
[{{:_, :_, :value}, [], [true]}])
2
```

delete_meta(registry, key)

(since 1.11.0)

```
@spec delete_meta(registry(), meta_key()) :: :ok
```

Deletes registry metadata for the given `key` in `registry`.

Examples

```
iex> Registry.start_link(keys: :unique, name:
Registry.DeleteMetaTest)
iex> Registry.put_meta(Registry.DeleteMetaTest,
:custom_key, "custom_value")
:ok
iex> Registry.meta(Registry.DeleteMetaTest, :custom_key)
{:ok, "custom_value"}
iex> Registry.delete_meta(Registry.DeleteMetaTest,
:custom_key)
:ok
iex> Registry.meta(Registry.DeleteMetaTest, :custom_key)
:error
```

dispatch(registry, key, mfa_or_fun, opts \\ [])

(since 1.4.0)

```
@spec dispatch(registry(), key(), dispatcher,
dispatch_opts()) :: :ok
when dispatcher:
       (entries :: [{pid(), value()}] -> term()) | {module(),
atom(), [term()]}
```

Invokes the callback with all entries under `key` in each partition for the given `registry`.

The list of `entries` is a non-empty list of two-element tuples where the first element is the PID and the second element is the value associated to the PID. If there are no entries for the given key, the callback is never invoked.

If the registry is partitioned, the callback is invoked multiple times per partition. If the registry is partitioned and `parallel: true` is given as an option, the dispatching happens in parallel. In both cases, the callback is only invoked if there are entries for that partition.

For `{:duplicate, :key}` registries, all entries for a given key are in a single partition, so the `:parallel` option has no effect.

See the module documentation for examples of using the `dispatch/3` function for building custom dispatching or a pubsub system.

Options

- `:parallel` - if `true`, the dispatching is done in parallel across all partitions. Only meaningful for `{:duplicate, :pid}` registries. Defaults to `false`.

keys(registry, pid)

(since 1.4.0)

```
@spec keys(registry(), pid()) :: [key()]
```

Returns the known keys for the given `pid` in `registry` in no particular order.

If the registry is unique, the keys are unique. Otherwise they may contain duplicates if the process was registered under the same key multiple times. The list will be empty if the process is dead or it has no keys in this registry.

Examples

Registering under a unique registry does not allow multiple entries:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueKeysTest)
iex> Registry.keys(Registry.UniqueKeysTest, self())
[]
iex> {:ok, _} =
Registry.register(Registry.UniqueKeysTest, "hello",
:world)
iex> Registry.register(Registry.UniqueKeysTest, "hello",
:later) # registry is :unique
{:error, {:already_registered, self()}}
iex> Registry.keys(Registry.UniqueKeysTest, self())
["hello"]
```

Such is possible for duplicate registries though:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateKeysTest)
iex> Registry.keys(Registry.DuplicateKeysTest, self())
[]
iex> {:ok, _} =
Registry.register(Registry.DuplicateKeysTest, "hello",
:world)
iex> {:ok, _} =
Registry.register(Registry.DuplicateKeysTest, "hello",
:world)
iex> Registry.keys(Registry.DuplicateKeysTest, self())
["hello", "hello"]
```

lock(registry, lock_key, function)

(since 1.18.0)

Out-of-band locking of the given `lock_key` for the duration of `function`.

Only one function can execute under the same `lock_key` at a given time. The given function always runs in the caller process.

The `lock_key` has its own namespace and therefore does not clash or overlap with the regular registry keys. In other words, locking works out-of-band from the regular Registry operations. See the "Use cases" section below.

Locking behaves the same regardless of the registry type.

Use cases

The Registry is safe and concurrent out-of-the-box. You are not required to use this function when interacting with the Registry. Furthermore, `Registry` with `:unique` keys can already act as a process-lock for any given key. For example, you can ensure only one process runs at a given time for a given `:key` by doing:

```
name = {:via, Registry, {MyApp.Registry, :key, :value}}

# Do not attempt to start if we are already running
if pid = GenServer.whereis(name) do
  pid
else
  case GenServer.start_link(__MODULE__, :ok, name: name)
do
    {:ok, pid} -> pid
```

```
      {:error, {:already_started, pid}} -> pid
    end
  end
```

Process locking gives you plenty of flexibility and fault isolation and is enough for most cases.

This function is useful only when spawning processes is not an option, for example, when copying the data to another process could be too expensive. Or when the work must be done within the current process for other reasons. In such cases, this function provides a scalable mechanism for managing locks on top of the registry's infrastructure.

Examples

```
iex> Registry.start_link(keys: :unique, name:
Registry.LockTest)
iex> Registry.lock(Registry.LockTest, :hello, fn -> :ok
end)
:ok
iex> Registry.lock(Registry.LockTest, :world, fn ->
self() end)
self()
```

lookup(registry, key)

(since 1.4.0)

```
@spec lookup(registry(), key()) :: [{pid(), value()}]
```

Finds the `{pid, value}` pair for the given `key` in `registry` in no particular order.

An empty list if there is no match.

For unique registries, a single partition lookup is necessary. For duplicate registries, all partitions must be looked up.

Examples

In the example below we register the current process and look it up both from itself and other processes:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueLookupTest)
iex> Registry.lookup(Registry.UniqueLookupTest, "hello")
[]
iex> {:ok, _} =
Registry.register(Registry.UniqueLookupTest, "hello",
:world)
iex> Registry.lookup(Registry.UniqueLookupTest, "hello")
[{self(), :world}]
iex> Task.async(fn ->
Registry.lookup(Registry.UniqueLookupTest, "hello") end)
|> Task.await()
[{self(), :world}]
```

The same applies to duplicate registries:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateLookupTest)
iex> Registry.lookup(Registry.DuplicateLookupTest,
"hello")
[]
iex> {:ok, _} =
Registry.register(Registry.DuplicateLookupTest, "hello",
:world)
```

```
iex> Registry.lookup(Registry.DuplicateLookupTest,
"hello")
[{self(), :world}]
iex> {:ok, _} =
Registry.register(Registry.DuplicateLookupTest, "hello",
:another)
iex>
Enum.sort(Registry.lookup(Registry.DuplicateLookupTest,
"hello"))
[{self(), :another}, {self(), :world}]
```

match(registry, key, pattern, guards \\ [])

(since 1.4.0)

```
@spec match(registry(), key(), match_pattern(), guards()) ::
[{pid(), term()}]
```

Returns `{pid, value}` pairs under the given `key` in `registry` that match `pattern`.

Pattern must be an atom or a tuple that will match the structure of the value stored in the registry. The atom `:_` can be used to ignore a given value or tuple element, while the atom `:"$1"` can be used to temporarily assign part of pattern to a variable for a subsequent comparison.

Optionally, it is possible to pass a list of guard conditions for more precise matching. Each guard is a tuple, which describes checks that should be passed by assigned part of pattern. For example the `$1 > 1` guard condition would be expressed as the `{:>, :"$1", 1}` tuple. Please note that guard conditions will work only for assigned variables like `:"$1"`, `:"$2"`, and so forth. Avoid usage of special match variables `:"$_"` and `:"$$"`, because it might not work as expected.

An empty list will be returned if there is no match.

For unique registries, a single partition lookup is necessary. For duplicate registries, all partitions must be looked up.

Examples

In the example below we register the current process under the same key in a duplicate registry but with different values:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.MatchTest)
iex> {:ok, _} = Registry.register(Registry.MatchTest,
"hello", {1, :atom, 1})
iex> {:ok, _} = Registry.register(Registry.MatchTest,
"hello", {2, :atom, 2})
iex> Registry.match(Registry.MatchTest, "hello", {1, :_,
:_})
[{self(), {1, :atom, 1}}]
iex> Registry.match(Registry.MatchTest, "hello", {2, :_,
:_})
[{self(), {2, :atom, 2}}]
iex> Registry.match(Registry.MatchTest, "hello", {:_,
:atom, :_}) |> Enum.sort()
[{self(), {1, :atom, 1}}, {self(), {2, :atom, 2}}]
iex> Registry.match(Registry.MatchTest, "hello", {:"$1",
:_, :"$1"}) |> Enum.sort()
[{self(), {1, :atom, 1}}, {self(), {2, :atom, 2}}]
iex> guards = [{:>, :"$1", 1}]
iex> Registry.match(Registry.MatchTest, "hello", {:_, :_,
:"$1"}, guards)
[{self(), {2, :atom, 2}}]
iex> guards = [{:is_atom, :"$1"}]
iex> Registry.match(Registry.MatchTest, "hello", {:_,
```

```
  :"$1", :_}, guards) |> Enum.sort()
  [{self(), {1, :atom, 1}}, {self(), {2, :atom, 2}}]
```

meta(registry, key)

(since 1.4.0)

```
@spec meta(registry(), meta key()) :: {:ok, meta value()} |
:error
```

Reads registry metadata given on `start_link/1`.

Atoms and tuples are allowed as keys.

Examples

```
iex> Registry.start_link(keys: :unique, name:
Registry.MetaTest, meta: [custom_key: "custom_value"])
iex> Registry.meta(Registry.MetaTest, :custom_key)
{:ok, "custom_value"}
iex> Registry.meta(Registry.MetaTest, :unknown_key)
:error
```

put_meta(registry, key, value)

(since 1.4.0)

```
@spec put_meta(registry(), meta key(), meta value()) :: :ok
```

Stores registry metadata.

Atoms and tuples are allowed as keys.

Examples

```
iex> Registry.start_link(keys: :unique, name:
Registry.PutMetaTest)
iex> Registry.put_meta(Registry.PutMetaTest, :custom_key,
"custom_value")
:ok
iex> Registry.meta(Registry.PutMetaTest, :custom_key)
{:ok, "custom_value"}
iex> Registry.put_meta(Registry.PutMetaTest, {:tuple,
:key}, "tuple_value")
:ok
iex> Registry.meta(Registry.PutMetaTest, {:tuple, :key})
{:ok, "tuple_value"}
```

register(registry, key, value)

(since 1.4.0)

```
@spec register(registry(), key(), value()) ::
  {:ok, pid()} | {:error, {:already_registered, pid()}}
```

Registers the current process under the given `key` in `registry`.

A value to be associated with this registration must also be given. This value will be retrieved whenever dispatching or doing a key lookup.

This function returns `{:ok, owner}` or `{:error, reason}`. The `owner` is the PID in the registry partition responsible for the PID. The owner is automatically linked to the caller.

If the registry has unique keys, it will return `{:ok, owner}` unless the key is already associated to a PID, in which case it returns `{:error, {:already_registered, pid}}`.

If the registry has duplicate keys, multiple registrations from the current process under the same key are allowed.

If the registry has listeners specified via the `:listeners` option in `start_link/1`, those listeners will be notified of the registration and will receive a message of type `listener_message/0`.

Examples

Registering under a unique registry does not allow multiple entries:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueRegisterTest)
iex> {:ok, _} =
Registry.register(Registry.UniqueRegisterTest, "hello",
:world)
iex> Registry.register(Registry.UniqueRegisterTest,
"hello", :later)
{:error, {:already_registered, self()}}
iex> Registry.keys(Registry.UniqueRegisterTest, self())
["hello"]
```

Such is possible for duplicate registries though:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateRegisterTest)
iex> {:ok, _} =
Registry.register(Registry.DuplicateRegisterTest,
"hello", :world)
iex> {:ok, _} =
Registry.register(Registry.DuplicateRegisterTest,
"hello", :world)
iex> Registry.keys(Registry.DuplicateRegisterTest,
```

```
  self())
  ["hello", "hello"]
```

select(registry, spec)

(since 1.9.0)

```
@spec select(registry(), spec()) :: [term()]
```

Select key, pid, and values registered using full match specs.

The `spec` consists of a list of three part tuples, in the shape of `[{match_pattern, guards, body}]`.

The first part, the match pattern, must be a tuple that will match the structure of the the data stored in the registry, which is `{key, pid, value}`. The atom `:_` can be used to ignore a given value or tuple element, while the atom `:"$1"` can be used to temporarily assign part of pattern to a variable for a subsequent comparison. This can be combined like `{:"$1", :_, :_}`.

The second part, the guards, is a list of conditions that allow filtering the results. Each guard is a tuple, which describes checks that should be passed by assigned part of pattern. For example the `$1 > 1` guard condition would be expressed as the `{:>, :"$1", 1}` tuple. Please note that guard conditions will work only for assigned variables like `:"$1"`, `:"$2"`, and so forth.

The third part, the body, is a list of shapes of the returned entries. Like guards, you have access to assigned variables like `:"$1"`, which you can combine with hard-coded values to freely shape entries Note that tuples have to be wrapped in an additional tuple. To get a result format like `%{key: key, pid: pid, value: value}`, assuming you bound those variables in order in the match part, you would provide a body like `[%`

`{key: :"$1", pid: :"$2", value: :"$3"}]`. Like guards, you can use some operations like `:element` to modify the output format.

Do not use special match variables `:"$_"` and `:"$$"`, because they might not work as expected. In particular, `{:duplicate, :key}` registries use a different internal ETS layout, so match specs that reference the underlying entry structure via `:"$_"` will return different results. Use named variables like `:"$1"`, `:"$2"`, `:"$3"` instead.

Note that for large registries with many partitions this will be costly as it builds the result by concatenating all the partitions.

Examples

This example shows how to get everything from the registry:

```
iex> Registry.start_link(keys: :unique, name:
Registry.SelectAllTest)
iex> {:ok, _} = Registry.register(Registry.SelectAllTest,
"hello", :value)
iex> {:ok, _} = Registry.register(Registry.SelectAllTest,
"world", :value)
iex> Registry.select(Registry.SelectAllTest, [{{:"$1",
:"$2", :"$3"}, [], [{{:"$1", :"$2", :"$3"}}]}]) |>
Enum.sort()
[{"hello", self(), :value}, {"world", self(), :value}]
```

If you want to get keys, you can pass a separate selector:

```
iex> Registry.start_link(keys: :unique, name:
Registry.SelectKeysTest)
iex> {:ok, _} =
Registry.register(Registry.SelectKeysTest, "hello",
```

```
:value)
iex> {:ok, _} =
Registry.register(Registry.SelectKeysTest, "world",
:value)
iex> Registry.select(Registry.SelectKeysTest, [{{:"$1",
:_, :_}, [], [:"$1"]}]) |> Enum.sort()
["hello", "world"]
```

start_link(options)

(since 1.5.0)

```
@spec start_link([start option()]) :: {:ok, pid()} | {:error,
term()}
```

Starts the registry as a supervisor process.

Manually it can be started as:

```
Registry.start_link(keys: :unique, name: MyApp.Registry)
```

In your supervisor tree, you would write:

```
Supervisor.start_link([
  {Registry, keys: :unique, name: MyApp.Registry}
], strategy: :one_for_one)
```

For intensive workloads, the registry may also be partitioned (by specifying the `:partitions` option). If partitioning is required then a good default is to set the number of partitions to the number of schedulers available:

```
Registry.start_link(
  keys: :unique,
  name: MyApp.Registry,
  partitions: System.schedulers_online()
)
```

or:

```
Supervisor.start_link([
  {Registry, keys: :unique, name: MyApp.Registry,
partitions: System.schedulers_online()}
], strategy: :one_for_one)
```

For `:duplicate` registries with many different keys (e.g., many topics with few subscribers each), you can optimize key-based lookups by partitioning by key:

```
Registry.start_link(
  keys: {:duplicate, :key},
  name: MyApp.TopicRegistry,
  partitions: System.schedulers_online()
)
```

This allows key-based lookups to check only a single partition instead of searching all partitions. Use the default `:pid` partitioning when you have fewer keys with many entries each (e.g., one topic with many subscribers).

Options

The registry requires the following keys:

- `:keys` - chooses if keys are `:unique`, `:duplicate`, `{:duplicate, :key}`, or `{:duplicate, :pid}`
- `:name` - the name of the registry and its tables

The following keys are optional:

- `:partitions` - the number of partitions in the registry. Defaults to `1`.
- `:listeners` - a list of named processes which are notified of register and unregister events. The registered process must be monitored by the listener if the listener wants to be notified if the registered process crashes. Messages sent to listeners are of type `listener_message/0`.
- `:meta` - a keyword list of metadata to be attached to the registry.

For `:duplicate` registries, you can specify the partitioning strategy directly in the `:keys` option:

- `:duplicate` or `{:duplicate, :pid}` - Use `:pid` partitioning (default) when you have keys with many entries (e.g., one topic with many subscribers). This is the traditional behavior and groups all entries from the same process together.

- `{:duplicate, :key}` - Use `:key` partitioning when entries are spread across many different keys (e.g., many topics with few subscribers each). This makes key-based lookups more efficient as they only need to check a single partition instead of all partitions. This option uses a different internal ETS table type (`ordered_set` instead of `duplicate_bag`), which means match specs passed to `select/2` and `count_select/2` that reference `:"$_"` may behave differently. Use named match variables (`:"$1"`, `:"$2"`, etc.) instead.

unregister(registry, key)

(since 1.4.0)

```
@spec unregister(registry(), key()) :: :ok
```

Unregisters all entries for the given `key` associated to the current process in `registry`.

Always returns `:ok` and automatically unlinks the current process from the owner if there are no more keys associated to the current process. See also `register/3` to read more about the "owner".

If the registry has listeners specified via the `:listeners` option in `start_link/1`, those listeners will be notified of the unregistration and will receive a message of type `listener_message/0`.

Examples

For unique registries:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueUnregisterTest)
iex> Registry.register(Registry.UniqueUnregisterTest,
"hello", :world)
iex> Registry.keys(Registry.UniqueUnregisterTest, self())
["hello"]
iex> Registry.unregister(Registry.UniqueUnregisterTest,
"hello")
:ok
iex> Registry.keys(Registry.UniqueUnregisterTest, self())
[]
```

For duplicate registries:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateUnregisterTest)
iex> Registry.register(Registry.DuplicateUnregisterTest,
"hello", :world)
iex> Registry.register(Registry.DuplicateUnregisterTest,
"hello", :world)
iex> Registry.keys(Registry.DuplicateUnregisterTest,
self())
["hello", "hello"]
iex>
Registry.unregister(Registry.DuplicateUnregisterTest,
"hello")
:ok
iex> Registry.keys(Registry.DuplicateUnregisterTest,
self())
[]
```

unregister_match(registry, key, pattern, guards \\ [])

(since 1.5.0)

```
@spec unregister_match(registry(), key(), match_pattern(),
guards()) :: :ok
```

Unregisters entries for keys matching a pattern associated to the current process in `registry`.

Examples

For unique registries it can be used to conditionally unregister a key on the basis of whether or not it matches a particular value.

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueUnregisterMatchTest)
iex>
Registry.register(Registry.UniqueUnregisterMatchTest,
"hello", :world)
iex> Registry.keys(Registry.UniqueUnregisterMatchTest,
self())
["hello"]
iex>
Registry.unregister_match(Registry.UniqueUnregisterMatchT
est, "hello", :foo)
:ok
iex> Registry.keys(Registry.UniqueUnregisterMatchTest,
self())
["hello"]
iex>
Registry.unregister_match(Registry.UniqueUnregisterMatchT
est, "hello", :world)
:ok
iex> Registry.keys(Registry.UniqueUnregisterMatchTest,
self())
[]
```

For duplicate registries:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateUnregisterMatchTest)
iex>
Registry.register(Registry.DuplicateUnregisterMatchTest,
"hello", :world_a)
iex>
Registry.register(Registry.DuplicateUnregisterMatchTest,
"hello", :world_b)
iex>
```

```
Registry.register(Registry.DuplicateUnregisterMatchTest,
"hello", :world_c)
iex> Registry.keys(Registry.DuplicateUnregisterMatchTest,
self())
["hello", "hello", "hello"]
iex>
Registry.unregister_match(Registry.DuplicateUnregisterMat
chTest, "hello", :world_a)
:ok
iex> Registry.keys(Registry.DuplicateUnregisterMatchTest,
self())
["hello", "hello"]
iex>
Registry.lookup(Registry.DuplicateUnregisterMatchTest,
"hello")
[{self(), :world_b}, {self(), :world_c}]
```

update_value(registry, key, callback)

(since 1.4.0)

```
@spec update_value(registry(), key(), (value() -> value()))
::
  {new_value :: term(), old_value :: term()} | :error
```

Updates the value for `key` for the current process in the unique `registry`.

Returns a `{new_value, old_value}` tuple or `:error` if there is no such key assigned to the current process.

If a non-unique registry is given, an error is raised.

Examples

```
iex> Registry.start_link(keys: :unique, name:
Registry.UpdateTest)
iex> {:ok, _} = Registry.register(Registry.UpdateTest,
"hello", 1)
iex> Registry.lookup(Registry.UpdateTest, "hello")
[{self(), 1}]
iex> Registry.update_value(Registry.UpdateTest, "hello",
&(&1 + 1))
{2, 1}
iex> Registry.lookup(Registry.UpdateTest, "hello")
[{self(), 2}]
```

values(registry, key, pid)

(since 1.12.0)

```
@spec values(registry(), key(), pid()) :: [value()]
```

Reads the values for the given `key` for `pid` in `registry`.

For unique registries, it is either an empty list or a list with a single element. For duplicate registries, it is a list with zero, one, or multiple elements.

Examples

In the example below we register the current process and look it up both from itself and other processes:

```
iex> Registry.start_link(keys: :unique, name:
Registry.UniqueValuesTest)
iex> Registry.values(Registry.UniqueValuesTest, "hello",
self())
[]
```

```
iex> {:ok, _} =
Registry.register(Registry.UniqueValuesTest, "hello",
:world)
iex> Registry.values(Registry.UniqueValuesTest, "hello",
self())
[:world]
iex> Task.async(fn ->
Registry.values(Registry.UniqueValuesTest, "hello",
self()) end) |> Task.await()
[]
iex> parent = self()
iex> Task.async(fn ->
Registry.values(Registry.UniqueValuesTest, "hello",
parent) end) |> Task.await()
[:world]
```

The same applies to duplicate registries:

```
iex> Registry.start_link(keys: :duplicate, name:
Registry.DuplicateValuesTest)
iex> Registry.values(Registry.DuplicateValuesTest,
"hello", self())
[]
iex> {:ok, _} =
Registry.register(Registry.DuplicateValuesTest, "hello",
:world)
iex> Registry.values(Registry.DuplicateValuesTest,
"hello", self())
[:world]
iex> {:ok, _} =
Registry.register(Registry.DuplicateValuesTest, "hello",
:another)
iex>
Enum.sort(Registry.values(Registry.DuplicateValuesTest,
```

```
"hello", self()))
[:another, :world]
```

Supervisor behaviour

A behaviour module for implementing supervisors.

A supervisor is a process which supervises other processes, which we refer to as *child processes*. Supervisors are used to build a hierarchical process structure called a *supervision tree*. Supervision trees provide fault-tolerance and encapsulate how our applications start and shutdown.

A supervisor may be started directly with a list of child specifications via `start_link/2` or you may define a module-based supervisor that implements the required callbacks. The sections below use `start_link/2` to start supervisors in most examples, but it also includes a specific section on module-based ones.

Examples

In order to start a supervisor, we need to first define a child process that will be supervised. As an example, we will define a `GenServer`, a generic server, that keeps a counter. Other processes can then send messages to this process to read the counter and bump its value.

> **Disclaimer**
>
> In practice you would not define a counter as a GenServer. Instead, if you need a counter, you would pass it around as inputs and outputs to the functions that need it. The reason we picked a counter in this example is due to its simplicity, as it allows us to focus on how supervisors work.

```
defmodule Counter do
  use GenServer
```

```
  def start_link(arg) when is_integer(arg) do
    GenServer.start_link(__MODULE__, arg, name:
__MODULE__)
  end

  ## Callbacks

  @impl true
  def init(counter) do
    {:ok, counter}
  end

  @impl true
  def handle_call(:get, _from, counter) do
    {:reply, counter, counter}
  end

  def handle_call({:bump, value}, _from, counter) do
    {:reply, counter, counter + value}
  end
end
```

The `Counter` receives an argument on `start_link`. This argument is passed to the `init/1` callback which becomes the initial value of the counter. Our counter handles two operations (known as calls): `:get`, to get the current counter value, and `:bump`, that bumps the counter by the given `value` and returns the old counter.

We can now start a supervisor that will start and supervise our counter process. The first step is to define a list of **child specifications** that control how each child behaves. Each child specification is a map, as shown below:

```
children = [
  # The Counter is a child started via
Counter.start_link(0)
  %{
    id: Counter,
    start: {Counter, :start_link, [0]}
  }
]

# Now we start the supervisor with the children and a
strategy
{:ok, pid} = Supervisor.start_link(children, strategy:
:one_for_one)

# After started, we can query the supervisor for
information
Supervisor.count_children(pid)
#=> %{active: 1, specs: 1, supervisors: 0, workers: 1}
```

Note that when starting the GenServer, we are registering it with name `Counter` via the `name: __MODULE__` option. This allows us to call it directly and get its value:

```
GenServer.call(Counter, :get)
#=> 0

GenServer.call(Counter, {:bump, 3})
#=> 0

GenServer.call(Counter, :get)
#=> 3
```

However, there is a bug in our counter server. If we call `:bump` with a non-numeric value, it is going to crash:

```
GenServer.call(Counter, {:bump, "oops"})
** (exit) exited in: GenServer.call(Counter, {:bump,
"oops"}, 5000)
```

Luckily, since the server is being supervised by a supervisor, the supervisor will automatically start a new one, reset back to its initial value of `0`:

```
GenServer.call(Counter, :get)
#=> 0
```

Supervisors support different strategies; in the example above, we have chosen `:one_for_one`. Furthermore, each supervisor can have many workers and/or supervisors as children, with each one having its own configuration (as outlined in the "Child specification" section).

The rest of this document will cover how child processes are specified, how they can be started and stopped, different supervision strategies and more.

Child specification

The child specification describes how the supervisor starts, shuts down, and restarts child processes.

The child specification is a map containing up to 6 elements. The first two keys in the following list are required, and the remaining ones are optional:

- `:id` - any term used to identify the child specification internally by the supervisor; defaults to the given module. This key is required. For supervisors, in the case of conflicting `:id` values, the supervisor will refuse to initialize and require explicit IDs. This is not the case for dynamic supervisors though.

- `:start` - a tuple with the module-function-args to be invoked to start the child process. This key is required.

- `:restart` - an atom that defines when a terminated child process should be restarted (see the "Restart values" section below). This key is optional and defaults to `:permanent`.

- `:shutdown` - an integer or atom that defines how a child process should be terminated (see the "Shutdown values" section below). This key is optional and defaults to `5_000` if the type is `:worker` or `:infinity` if the type is `:supervisor`.

- `:type` - specifies that the child process is a `:worker` or a `:supervisor`. This key is optional and defaults to `:worker`.

- `:modules` - a list of modules used by hot code upgrade mechanisms to determine which processes are using certain modules. It is typically set to the callback module of behaviours like `GenServer`, `Supervisor`, and such. It is set automatically based on the `:start` value and it is rarely changed in practice.

- `:significant` - a boolean indicating if the child process should be considered significant with regard to automatic shutdown. Only `:transient` and `:temporary` child processes can be marked as significant. This key is optional and defaults to `false`. See section "Automatic shutdown" below for more details.

Let's understand what the `:shutdown` and `:restart` options control.

Shutdown values (:shutdown)

The following shutdown values are supported in the `:shutdown` option:

- `:brutal_kill` - the child process is unconditionally and immediately terminated using `Process.exit(child, :kill)`.
- any integer >= 0 - the amount of time in milliseconds that the supervisor will wait for its children to terminate after emitting a `Process.exit(child, :shutdown)` signal. If the child process is not trapping exits, the initial `:shutdown` signal will terminate the child process immediately. If the child process is trapping exits, it has the given amount of time to terminate. If it doesn't terminate within the specified time, the child process is unconditionally terminated by the supervisor via `Process.exit(child, :kill)`.
- `:infinity` - works as an integer except the supervisor will wait indefinitely for the child to terminate. If the child process is a supervisor, the recommended value is `:infinity` to give the supervisor and its children enough time to shut down. This option can be used with regular workers but doing so is discouraged and requires extreme care. If not used carefully, the child process will never terminate, preventing your application from terminating as well.

Restart values (:restart)

The `:restart` option controls what the supervisor should consider to be a successful termination or not. If the termination is successful, the supervisor won't restart the child. If the child process crashed, the supervisor will start a new one.

The following restart values are supported in the `:restart` option:

- `:permanent` - the child process is always restarted.

- `:temporary` - the child process is never restarted, regardless of the supervision strategy: any termination (even abnormal) is considered successful.
- `:transient` - the child process is restarted only if it terminates abnormally, i.e., with an exit reason other than `:normal`, `:shutdown`, or `{:shutdown, term}`.

For a more complete understanding of the exit reasons and their impact, see the "Exit reasons and restarts" section.

`child_spec/1` function

When starting a supervisor, we may pass a list of child specifications. Those specifications are maps that tell how the supervisor should start, stop and restart each of its children:

```
%{
  id: Counter,
  start: {Counter, :start_link, [0]}
}
```

The map above defines a child with `:id` of `Counter` that is started by calling `Counter.start_link(0)`.

However, defining the child specification for each child as a map can be quite error prone, as we may change the `Counter` implementation and forget to update its specification. That's why Elixir allows you to pass a tuple with the module name and the `start_link` argument instead of the specification:

```
children = [
  {Counter, 0}
```

```
]
```

The supervisor will then invoke `Counter.child_spec(0)` to retrieve a child specification. Now the `Counter` module is responsible for building its own specification, for example, we could write:

```
def child_spec(arg) do
  %{
    id: Counter,
    start: {Counter, :start_link, [arg]}
  }
end
```

Then the supervisor will call `Counter.start_link(arg)` to start the child process. This flow is summarized in the diagram below. Caller is a process which spawns the Supervisor process. The Supervisor then proceeds to call your code (Module) to spawn its child process:

```
sequenceDiagram
    participant C as Caller (Process)
    participant S as Supervisor (Process)
    participant M as Module (Code)

    note right of C: child is a {module, arg}
specification
    C->>+S: Supervisor.start_link([child])
    S-->>+M: module.child_spec(arg)
    M-->>-S: %{id: term, start: {module, :start_link,
[arg]}}
    S-->>+M: module.start_link(arg)
    M->>M: Spawns child process (child_pid)
    M-->>-S: {:ok, child_pid} | :ignore | {:error,
```

```
reason}
    S->>-C: {:ok, supervisor_pid} | {:error, reason}
```

Luckily for us, `use GenServer` already defines a `Counter.child_spec/1` exactly like above, so you don't need to write the definition above yourself. If you want to customize the automatically generated `child_spec/1` function, you can pass the options directly to `use GenServer`:

```
use GenServer, restart: :transient
```

Finally, note it is also possible to simply pass the `Counter` module as a child:

```
children = [
  Counter
]
```

When only the module name is given, it is equivalent to `{Counter, []}`, which in our case would be invalid, which is why we always pass the initial counter explicitly.

By replacing the child specification with `{Counter, 0}`, we keep it encapsulated in the `Counter` module. We could now share our `Counter` implementation with other developers and they can add it directly to their supervision tree without worrying about the low-level details of the counter.

Overall, a child specification can be one of the following:

- a map representing the child specification itself - as outlined in the "Child specification" section

- a tuple with a module as first element and the start argument as second - such as `{Counter, 0}`. In this case, `Counter.child_spec(0)` is called to retrieve the child specification
- a module - such as `Counter`. In this case, `Counter.child_spec([])` would be called, which is invalid for the counter, but it is useful in many other cases, especially when you want to pass a list of options to the child process

If you need to convert a `{module, arg}` tuple or a module child specification to a child specification or modify a child specification itself, you can use the `Supervisor.child_spec/2` function. For example, to run the counter with a different `:id` and a `:shutdown` value of 10 seconds (10_000 milliseconds):

```
children = [
  Supervisor.child_spec({Counter, 0}, id: MyCounter,
shutdown: 10_000)
]
```

Supervisor strategies and options

So far we have started the supervisor passing a single child as a tuple as well as a strategy called `:one_for_one`:

```
children = [
  {Counter, 0}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

The first argument given to `start_link/2` is a list of child specifications as defined in the "child_spec/1" section above.

The second argument is a keyword list of options:

- `:strategy` - the supervision strategy option. It can be either `:one_for_one`, `:rest_for_one` or `:one_for_all`. Required. See the "Strategies" section.
- `:max_restarts` - the maximum number of restarts allowed in a time frame. Defaults to `3`.
- `:max_seconds` - the time frame in which `:max_restarts` applies. Defaults to `5`.
- `:auto_shutdown` - the automatic shutdown option. It can be `:never`, `:any_significant`, or `:all_significant`. Optional. See the "Automatic shutdown" section.
- `:name` - a name to register the supervisor process. Supported values are explained in the "Name registration" section in the documentation for `GenServer`. Optional.

Strategies

Supervisors support different supervision strategies (through the `:strategy` option, as seen above):

- `:one_for_one` - if a child process terminates, only that process is restarted.
- `:one_for_all` - if a child process terminates, all other child processes are terminated and then all child processes (including the terminated one) are restarted.

- `:rest_for_one` - if a child process terminates, the terminated child process and the rest of the children started after it, are terminated and restarted.

In the above, process termination refers to unsuccessful termination, which is determined by the `:restart` option.

To efficiently supervise children started dynamically, see `DynamicSupervisor`.

Automatic shutdown

Supervisors have the ability to automatically shut themselves down when child processes marked as `:significant` exit.

Supervisors support different automatic shutdown options (through the `:auto_shutdown` option, as seen above):

- `:never` - this is the default, automatic shutdown is disabled.
- `:any_significant` - if any significant child process exits, the supervisor will automatically shut down its children, then itself.
- `:all_significant` - when all significant child processes have exited, the supervisor will automatically shut down its children, then itself.

Only `:transient` and `:temporary` child processes can be marked as significant, and this configuration affects the behavior. Significant `:transient` child processes must exit normally for automatic shutdown to be considered, where `:temporary` child processes may exit for any reason.

Name registration

A supervisor is bound to the same name registration rules as a `GenServer`. Read more about these rules in the documentation for `GenServer`.

Module-based supervisors

In the example so far, the supervisor was started by passing the supervision structure to `start_link/2`. However, supervisors can also be created by explicitly defining a supervision module:

```
defmodule MyApp.Supervisor do
  # Automatically defines child_spec/1
  use Supervisor

  def start_link(init_arg) do
    Supervisor.start_link(__MODULE__, init_arg, name:
__MODULE__)
  end

  @impl true
  def init(_init_arg) do
    children = [
      {Counter, 0}
    ]

    Supervisor.init(children, strategy: :one_for_one)
  end
end
```

The difference between the two approaches is that a module-based supervisor gives you more direct control over how the supervisor is initialized. Instead of calling `Supervisor.start_link/2` with a list of child specifications that are implicitly initialized for us, we must

explicitly initialize the children by calling `Supervisor.init/2` inside its `init/1` callback. `Supervisor.init/2` accepts the same `:strategy`, `:max_restarts`, and `:max_seconds` options as `start_link/2`.

> `use Supervisor`
>
> When you `use Supervisor`, the `Supervisor` module will set `@behaviour Supervisor` and define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

`use Supervisor` also defines a `child_spec/1` function which allows us to run `MyApp.Supervisor` as a child of another supervisor or at the top of your supervision tree as:

```
children = [
  MyApp.Supervisor
]

Supervisor.start_link(children, strategy: :one_for_one)
```

A general guideline is to use the supervisor without a callback module only at the top of your supervision tree, generally in the `Application.start/2` callback. We recommend using module-based supervisors for any other supervisor in your application, so they can run as a child of another supervisor in the tree. The `child_spec/1` generated automatically by `Supervisor` can be customized with the following options:

- `:id` - the child specification identifier, defaults to the current module
- `:restart` - when the supervisor should be restarted, defaults to `:permanent`

The `@doc` annotation immediately preceding `use Supervisor` will be attached to the generated `child_spec/1` function.

Start and shutdown

When the supervisor starts, it traverses all child specifications and then starts each child in the order they are defined. This is done by calling the function defined under the `:start` key in the child specification and typically defaults to `start_link/1`.

The `start_link/1` (or a custom) is then called for each child process. The `start_link/1` function must return `{:ok, pid}` where `pid` is the process identifier of a new process that is linked to the supervisor. The child process usually starts its work by executing the `init/1` callback. Generally speaking, the `init` callback is where we initialize and configure the child process.

The shutdown process happens in reverse order.

When a supervisor shuts down, it terminates all children in the opposite order they are listed. The termination happens by sending a shutdown exit signal, via `Process.exit(child_pid, :shutdown)`, to the child process and then awaiting for a time interval for the child process to terminate. This interval defaults to 5000 milliseconds. If the child process does not terminate in this interval, the supervisor abruptly terminates the child with reason `:kill`. The shutdown time can be configured in the child specification which is fully detailed in the next section.

If the child process is not trapping exits, it will shutdown immediately when it receives the first exit signal. If the child process is trapping exits, then the `terminate` callback is invoked, and the child process must terminate in a reasonable time interval before being abruptly terminated by the supervisor.

In other words, if it is important that a process cleans after itself when your application or the supervision tree is shutting down, then this process must trap exits and its child specification should specify the proper `:shutdown` value, ensuring it terminates within a reasonable interval.

Exit reasons and restarts

A supervisor restarts a child process depending on its `:restart` configuration. For example, when `:restart` is set to `:transient`, the supervisor does not restart the child in case it exits with reason `:normal`, `:shutdown` or `{:shutdown, term}`.

Those exits also impact logging. By default, behaviours such as GenServers do not emit error logs when the exit reason is `:normal`, `:shutdown` or `{:shutdown, term}`.

So one may ask: which exit reason should I choose? There are three options:

- `:normal` - in such cases, the exit won't be logged, there is no restart in transient mode, and linked processes do not exit
- `:shutdown` or `{:shutdown, term}` - in such cases, the exit won't be logged, there is no restart in transient mode, and linked processes exit with the same reason unless they're trapping exits
- any other term - in such cases, the exit will be logged, there are restarts in transient mode, and linked processes exit with the same reason unless they're trapping exits

Generally speaking, if you are exiting for expected reasons, you want to use `:shutdown` or `{:shutdown, term}`.

Note that the supervisor that reaches maximum restart intensity will exit with `:shutdown` reason. In this case the supervisor will only be restarted if its child specification was defined with the `:restart` option set to `:permanent` (the default).

Summary

Types

auto_shutdown()

Supported automatic shutdown options.

child()

A child process.

child_spec()

The supervisor child specification.

child_spec_overrides()

Options for overriding child specification fields.

init_option()

Options given to `start_link/2` and `init/2`.

module_spec()

A module-based child spec.

name()

The supervisor name.

on_start()

Return values of `start_link/2` and `start_link/3`.

on_start_child()

Return values of `start_child/2`.

option()

Option values used by the `start_link/2` and `start_link/3` functions.

restart()

Supported restart options.

shutdown()

Supported shutdown options.

strategy()

Supported strategies.

sup_flags()

The supervisor flags returned on init.

supervisor()

The supervisor reference.

type()

Type of a supervised child.

Callbacks

init(init_arg)

Callback invoked to start the supervisor and during hot code upgrades.

Functions

child_spec(module_or_map, overrides)

Builds and overrides a child specification.

count_children(supervisor)

Returns a map containing count values for the given supervisor.

delete_child(supervisor, child_id)

Deletes the child specification identified by `child_id`.

init(children, options)

Receives a list of child specifications to initialize and a set of `options`.

restart_child(supervisor, child_id)

Restarts a child process identified by `child_id`.

start_child(supervisor, child_spec)

Adds a child specification to `supervisor` and starts that child.

start_link(children, options)

Starts a supervisor with the given children.

start_link(module, init_arg, options \\ [])

Starts a module-based supervisor process with the given `module` and `init_arg`.

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

Synchronously stops the given supervisor with the given `reason`.

terminate_child(supervisor, child_id)

Terminates the given child identified by `child_id`.

which_children(supervisor)

Returns a list with information about all children of the given supervisor.

Types

auto_shutdown()

```
@type auto_shutdown() :: :never | :any_significant | :all_significant
```

Supported automatic shutdown options.

child()

```
@type child() :: pid() | :undefined
```

A child process.

It can be a PID when the child process was started, or `:undefined` when the child was created by a dynamic supervisor.

child_spec()

```
@type child_spec() :: %{
  :id => atom() | term(),
  :start => {module(), function_name :: atom(), args ::
[term()]},
  optional(:restart) => restart(),
  optional(:shutdown) => shutdown(),
  optional(:type) => type(),
  optional(:modules) => [module()] | :dynamic,
  optional(:significant) => boolean()
}
```

The supervisor child specification.

It defines how the supervisor should start, stop and restart each of its children.

child_spec_overrides()

```
@type child_spec_overrides() :: [
  id: atom() | term(),
  start: {module(), atom(), [term()]},
  restart: restart(),
  shutdown: shutdown(),
  type: type(),
  modules: [module()] | :dynamic,
  significant: boolean()
]
```

Options for overriding child specification fields.

init_option()

```
@type init_option() ::
  {:strategy, strategy()}
  | {:max_restarts, non_neg_integer()}
  | {:max_seconds, pos_integer()}
  | {:auto_shutdown, auto_shutdown()}
```

Options given to `start_link/2` and `init/2`.

module_spec()

(since 1.16.0)

```
@type module_spec() :: {module(), args :: term()} | module()
```

A module-based child spec.

This is a form of child spec that you can pass to functions such as `child_spec/2`, `start_child/2`, and `start_link/2`, in addition to the normalized `child_spec/0`.

A module-based child spec can be:

- a **module** — the supervisor calls `module.child_spec([])` to retrieve the child specification
- a **two-element tuple** in the shape of `{module, arg}` — the supervisor calls `module.child_spec(arg)` to retrieve the child specification

name()

```
@type name() :: atom() | {:global, term()} | {:via, module(),
term()}
```

The supervisor name.

on_start()

```
@type on_start() ::
  {:ok, pid()}
  | :ignore
  | {:error, {:already_started, pid()} | {:shutdown, term()}
| term()}
```

Return values of `start_link/2` and `start_link/3`.

See the documentation for those functions for more information.

on_start_child()

```
@type on_start_child() ::
  {:ok, child()}
  | {:ok, child(), info :: term()}
  | {:error, {:already_started, child()} | :already_present |
term()}
```

Return values of `start_child/2`.

See the documentation for those functions for more information.

option()

```
@type option() :: {:name, name()}
```

Option values used by the `start_link/2` and `start_link/3` functions.

restart()

```
@type restart() :: :permanent | :transient | :temporary
```

Supported restart options.

shutdown()

```
@type shutdown() :: timeout() | :brutal_kill
```

Supported shutdown options.

strategy()

```
@type strategy() :: :one_for_one | :one_for_all |
:rest_for_one
```

Supported strategies.

sup_flags()

```
@type sup_flags() :: %{
  strategy: strategy(),
  intensity: non_neg_integer(),
  period: pos_integer(),
  auto_shutdown: auto_shutdown()
}
```

The supervisor flags returned on init.

supervisor()

```
@type supervisor() :: pid() | name() | {atom(), node()}
```

The supervisor reference.

type()

```
@type type() :: :worker | :supervisor
```

Type of a supervised child.

Whether the supervised child is a worker or a supervisor.

Callbacks

init(init_arg)

```
@callback init(init_arg :: term()) ::
  {:ok,
   {sup_flags(),
    [child_spec() | (old_erlang_child_spec ::
:supervisor.child_spec())]}}
  | :ignore
```

Callback invoked to start the supervisor and during hot code upgrades.

Developers typically invoke `Supervisor.init/2` at the end of their init callback to return the proper supervision flags.

See `start_link/2` for more information.

Functions

child_spec(module_or_map, overrides)

```
@spec child_spec(child_spec() | module_spec(),
child_spec_overrides()) :: child_spec()
```

Builds and overrides a child specification.

Similar to `start_link/2` and `init/2`, it expects a module, `{module, arg}`, or a child specification.

If a two-element tuple in the shape of `{module, arg}` is given, the child specification is retrieved by calling `module.child_spec(arg)`.

If a module is given, the child specification is retrieved by calling `module.child_spec([])`.

After the child specification is retrieved, the fields on `overrides` are directly applied to the child spec. If `overrides` has keys that do not map

to any child specification field, an error is raised.

See the "Child specification" section in the module documentation for all of the available keys for overriding.

Examples

This function is often used to set an `:id` option when the same module needs to be started multiple times in the supervision tree:

```
Supervisor.child_spec({Agent, fn -> :ok end}, id: {Agent, 1})
#=> %{id: {Agent, 1},
#=>   start: {Agent, :start_link, [fn -> :ok end]}}
```

count_children(supervisor)

```
@spec count_children(supervisor()) :: %{
  specs: non_neg_integer(),
  active: non_neg_integer(),
  supervisors: non_neg_integer(),
  workers: non_neg_integer()
}
```

Returns a map containing count values for the given supervisor.

The map contains the following keys:

- `:specs` - the total count of children, dead or alive
- `:active` - the count of all actively running child processes managed by this supervisor

- `:supervisors` - the count of all supervisors whether or not these child supervisors are still alive
- `:workers` - the count of all workers, whether or not these child workers are still alive

delete_child(supervisor, child_id)

```
@spec delete_child(supervisor(), term()) :: :ok | {:error, error}
when error: :not_found | :running | :restarting
```

Deletes the child specification identified by `child_id`.

The corresponding child process must not be running; use `terminate_child/2` to terminate it if it's running.

If successful, this function returns `:ok`. This function may return an error with an appropriate error tuple if the `child_id` is not found, or if the current process is running or being restarted.

init(children, options)

(since 1.5.0)

```
@spec init(
  [
    child_spec()
    | module_spec()
    | (old_erlang_child_spec :: :supervisor.child_spec())
  ],
  [
    init_option()
```

```
  ]
) ::
  {:ok,
   {sup_flags(),
    [child_spec() | (old_erlang_child_spec ::
:supervisor.child_spec())]}}
```

Receives a list of child specifications to initialize and a set of `options`.

This is typically invoked at the end of the `init/1` callback of module-based supervisors. See the sections "Supervisor strategies and options" and "Module-based supervisors" in the module documentation for more information.

This function returns a tuple containing the supervisor flags and child specifications.

Examples

```elixir
def init(_init_arg) do
  children = [
    {Counter, 0}
  ]

  Supervisor.init(children, strategy: :one_for_one)
end
```

Options

- `:strategy` - the supervision strategy option. It can be either `:one_for_one`, `:rest_for_one`, or `:one_for_all`

- `:max_restarts` - the maximum number of restarts allowed in a time frame. Defaults to `3`.

- `:max_seconds` - the time frame in seconds in which `:max_restarts` applies. Defaults to `5`.

- `:auto_shutdown` - the automatic shutdown option. It can be either `:never`, `:any_significant`, or `:all_significant`

The `:strategy` option is required and by default a maximum of 3 restarts is allowed within 5 seconds. Check the `Supervisor` module for a detailed description of the available strategies.

restart_child(supervisor, child_id)

```
@spec restart_child(supervisor(), term()) ::
  {:ok, child()} | {:ok, child(), term()} | {:error, error}
when error: :not_found | :running | :restarting | term()
```

Restarts a child process identified by `child_id`.

The child specification must exist and the corresponding child process must not be running.

Note that for temporary children, the child specification is automatically deleted when the child terminates, and thus it is not possible to restart such children.

If the child process start function returns `{:ok, child}` or `{:ok, child, info}`, the PID is added to the supervisor and this function returns the same value.

If the child process start function returns `:ignore`, the PID remains set to `:undefined` and this function returns `{:ok, :undefined}`.

This function may return an error with an appropriate error tuple if the `child_id` is not found, or if the current process is running or being restarted.

If the child process start function returns an error tuple or an erroneous value, or if it fails, this function returns `{:error, error}`.

start_child(supervisor, child_spec)

```
@spec start_child(
  supervisor(),
  child_spec()
  | module_spec()
  | (old_erlang_child_spec :: :supervisor.child_spec())
) :: on_start_child()
```

Adds a child specification to `supervisor` and starts that child.

`child_spec` should be a valid child specification. The child process will be started as defined in the child specification.

If a child specification with the specified ID already exists, `child_spec` is discarded and this function returns an error with `:already_started` or `:already_present` if the corresponding child process is running or not, respectively.

If the child process start function returns `{:ok, child}` or `{:ok, child, info}`, then child specification and PID are added to the supervisor and this function returns the same value.

If the child process start function returns `:ignore`, the child specification is added to the supervisor, the PID is set to `:undefined` and this function returns `{:ok, :undefined}`.

If the child process start function returns an error tuple or an erroneous value, or if it fails, the child specification is discarded and this function returns `{:error, error}` where `error` is a term containing information about the error and child specification.

> **Order Among Children**
>
> The child specification is **appended** to the children of `supervisor`. This guarantees that semantics of things such as the `:rest_for_one` strategy are preserved correctly.

start_link(children, options)

```
@spec start_link(
  [
    child_spec()
    | module_spec()
    | (old_erlang_child_spec :: :supervisor.child_spec())
  ],
  [
    option() | init_option()
  ]
) ::
  {:ok, pid()}
  | {:error, {:already_started, pid()} | {:shutdown, term()}
| term()}
@spec start_link(module(), term()) :: on_start()
```

Starts a supervisor with the given children.

`children` is a list of the following forms:

- a child specification (see `child_spec/0`)

- a module, where the supervisor calls `module.child_spec([])` to retrieve the child specification (see `module_spec/0`)
- a `{module, arg}` tuple, where the supervisor calls `module.child_spec(arg)` to retrieve the child specification (see `module_spec/0`)
- a (old) Erlang-style child specification (see `:supervisor.child_spec()`)

A strategy is required to be provided through the `:strategy` option. See "Supervisor strategies and options" for examples and other options.

The options can also be used to register a supervisor name. The supported values are described under the "Name registration" section in the `GenServer` module docs.

If the supervisor and all child processes are successfully spawned (if the start function of each child process returns `{:ok, child}`, `{:ok, child, info}`, or `:ignore`), this function returns `{:ok, pid}`, where `pid` is the PID of the supervisor.

If the supervisor is given a name and a process with the specified name already exists, the function returns `{:error, {:already_started, pid}}`, where `pid` is the PID of that process.

If the start function of any of the child processes fails or returns an error tuple or an erroneous value, the supervisor first terminates with reason `:shutdown` all the child processes that have already been started, and then terminates itself. If the caller process is trapping exits, this function then returns `{:error, {:shutdown, {:failed_to_start_child, id, reason}}}`.

Note that a supervisor started with this function is linked to the parent process and exits not only on crashes but also if the parent process exits

with `:normal` reason.

start_link(module, init_arg, options \\ [])

```
@spec start_link(module(), term(), [option()]) :: on_start()
```

Starts a module-based supervisor process with the given `module` and `init_arg`.

To start the supervisor, the `init/1` callback will be invoked in the given `module`, with `init_arg` as its argument. The `init/1` callback must return a supervisor specification which can be created with the help of the `init/2` function.

If the `init/1` callback returns `:ignore`, this function returns `:ignore` as well and the supervisor terminates with reason `:normal`.

If the `init/1` callback fails or returns an incorrect value, the supervisor will shutdown with reason `term`, where `term` contains information about the error. If the caller process is trapping exits, this function then returns `{:error, term()}`.

If the start function of any of the child processes returned by `init/1` fail or return an error tuple or an erroneous value, the supervisor first terminates with reason `:shutdown` all the child processes that have already been started, and then terminates itself. If the caller process is trapping exits, this function then returns `{:error, {:shutdown, {:failed_to_start_child, id, reason}}}`.

The `:name` option can also be given in order to register a supervisor name, the supported values are described in the "Name registration" section in the `GenServer` module docs.

stop(supervisor, reason \\ :normal, timeout \\ :infinity)

```
@spec stop(supervisor(), reason :: term(), timeout()) :: :ok
```

Synchronously stops the given supervisor with the given `reason`.

It returns `:ok` if the supervisor terminates with the given reason. If it terminates with another reason, the call exits.

This function keeps OTP semantics regarding error reporting. If the reason is any other than `:normal`, `:shutdown` or `{:shutdown, _}`, an error report is logged.

terminate_child(supervisor, child_id)

```
@spec terminate_child(supervisor(), term()) :: :ok | {:error,
:not_found}
```

Terminates the given child identified by `child_id`.

The process is terminated, if there's one. The child specification is kept unless the child is temporary.

A non-temporary child process may later be restarted by the supervisor. The child process can also be restarted explicitly by calling `restart_child/2`. Use `delete_child/2` to remove the child specification.

If successful, this function returns `:ok`. If there is no child specification for the given child ID, this function returns `{:error, :not_found}`.

which_children(supervisor)

```
@spec which_children(supervisor()) :: [
  {term() | :undefined, child() | :restarting, :worker |
:supervisor,
   [module()] | :dynamic}
]
```

Returns a list with information about all children of the given supervisor.

Note that calling this function when supervising a large number of children under low memory conditions can bring the system down due to an out of memory error.

This function returns a list of `{id, child, type, modules}` tuples, where:

- `id` - as defined in the child specification
- `child` - the PID of the corresponding child process, `:restarting` if the process is about to be restarted, or `:undefined` if there is no such process
- `type` - `:worker` or `:supervisor`, as specified by the child specification
- `modules` - as specified by the child specification

Task

Conveniences for spawning and awaiting tasks.

Tasks are processes meant to execute one particular action throughout their lifetime, often with little or no communication with other processes. The most common use case for tasks is to convert sequential code into concurrent code by computing a value asynchronously:

```
task = Task.async(fn -> do_some_work() end)
res = do_some_other_work()
res + Task.await(task)
```

Tasks spawned with `async` can be awaited on by their caller process (and only their caller) as shown in the example above. They are implemented by spawning a process that sends a message to the caller once the given computation is performed.

Compared to plain processes, started with `spawn/1`, tasks include monitoring metadata and logging in case of errors.

Besides `async/1` and `await/2`, tasks can also be started as part of a supervision tree and dynamically spawned on remote nodes. We will explore these scenarios next.

async and await

One of the common uses of tasks is to convert sequential code into concurrent code with `Task.async/1` while keeping its semantics. When invoked, a new process will be created, linked and monitored by the caller. Once the task action finishes, a message will be sent to the caller with the result.

`Task.await/2` is used to read the message sent by the task.

There are two important things to consider when using `async`:

1. If you are using async tasks, you **must await** a reply as they are *always* sent. If you are not expecting a reply, consider using `Task.start_link/1` as detailed below.

2. Async tasks link the caller and the spawned process. This means that, if the caller crashes, the task will crash too and vice-versa. This is on purpose: if the process meant to receive the result no longer exists, there is no purpose in completing the computation. If this is not desired, you will want to use supervised tasks, described in a subsequent section.

Tasks are processes

Tasks are processes and so data will need to be completely copied to them. Take the following code as an example:

```
large_data = fetch_large_data()
task = Task.async(fn -> do_some_work(large_data) end)
res = do_some_other_work()
res + Task.await(task)
```

The code above copies over all of `large_data`, which can be resource intensive depending on the size of the data. There are two ways to address this.

First, if you need to access only part of `large_data`, consider extracting it before the task:

```
large_data = fetch_large_data()
subset_data = large_data.some_field
```

```
task = Task.async(fn -> do_some_work(subset_data) end)
```

Alternatively, if you can move the data loading altogether to the task, it may be even better:

```
task = Task.async(fn ->
  large_data = fetch_large_data()
  do_some_work(large_data)
end)
```

Dynamically supervised tasks

The `Task.Supervisor` module allows developers to dynamically create multiple supervised tasks.

A short example is:

```
{:ok, pid} = Task.Supervisor.start_link()

task =
  Task.Supervisor.async(pid, fn ->
    # Do something
  end)

Task.await(task)
```

However, in the majority of cases, you want to add the task supervisor to your supervision tree:

```
Supervisor.start_link([
  {Task.Supervisor, name: MyApp.TaskSupervisor}
```

```
], strategy: :one_for_one)
```

And now you can use async/await by passing the name of the supervisor instead of the PID:

```
Task.Supervisor.async(MyApp.TaskSupervisor, fn ->
  # Do something
end)
|> Task.await()
```

We encourage developers to rely on supervised tasks as much as possible. Supervised tasks improve the visibility of how many tasks are running at a given moment and enable a variety of patterns that give you explicit control on how to handle the results, errors, and timeouts. Here is a summary:

- Using `Task.Supervisor.start_child/2` allows you to start a fire-and-forget task when you don't care about its results or if it completes successfully or not.
- Using `Task.Supervisor.async/2` + `Task.await/2` allows you to execute tasks concurrently and retrieve its result. If the task fails, the caller will also fail.
- Using `Task.Supervisor.async_nolink/2` + `Task.yield/2` + `Task.shutdown/2` allows you to execute tasks concurrently and retrieve their results or the reason they failed within a given time frame. If the task fails, the caller won't fail. You will receive the error reason either on `yield` or `shutdown`.

Furthermore, the supervisor guarantees all tasks terminate within a configurable shutdown period when your application shuts down. See the `Task.Supervisor` module for details on the supported operations.

Distributed tasks

With `Task.Supervisor`, it is easy to dynamically start tasks across nodes:

```
# First on the remote node named :remote@local
Task.Supervisor.start_link(name: MyApp.DistSupervisor)

# Then on the local client node
supervisor = {MyApp.DistSupervisor, :remote@local}
Task.Supervisor.async(supervisor, MyMod, :my_fun, [arg1,
arg2, arg3])
```

Note that, as above, when working with distributed tasks, one should use the `Task.Supervisor.async/5` function that expects explicit module, function, and arguments, instead of `Task.Supervisor.async/3` that works with anonymous functions. That's because anonymous functions expect the same module version to exist on all involved nodes. Check the `Agent` module documentation for more information on distributed processes as the limitations described there apply to the whole ecosystem.

Statically supervised tasks

The `Task` module implements the `child_spec/1` function, which allows it to be started directly under a regular `Supervisor` - instead of a `Task.Supervisor` - by passing a tuple with a function to run:

```
Supervisor.start_link([
  {Task, fn -> :some_work end}
], strategy: :one_for_one)
```

This is often useful when you need to execute code concurrently while setting up your supervision tree. For example: to warm up caches, log the initialization status, and such.

If you don't want to put the Task code directly under the `Supervisor`, you can wrap the `Task` in its own module, similar to how you would do with a `GenServer` or an `Agent`:

```
defmodule MyTask do
  use Task

  def start_link(arg) do
    Task.start_link(__MODULE__, :run, [arg])
  end

  def run(arg) do
    # ...
  end
end
```

And then passing it to the supervisor:

```
Supervisor.start_link([
  {MyTask, arg}
], strategy: :one_for_one)
```

Since these tasks are supervised and not directly linked to the caller, they cannot be awaited on. By default, the functions `Task.start/1` and `Task.start_link/1` are for fire-and-forget tasks, where you don't care about the results or if it completes successfully or not.

Keep in mind the Supervisor will not wait for the task to finish running before starting the next child or returning. If you need synchronous

initialization, then either use an `Agent` or a `GenServer`.

> `use Task`
>
> When you `use Task`, the `Task` module will define a `child_spec/1` function, so your module can be used as a child in a supervision tree.

The generated `child_spec/1` can be customized with the following options:

- `:id` - the child specification identifier, defaults to the current module
- `:restart` - when the child should be restarted, defaults to `:temporary`
- `:shutdown` - how to shut down the child, either immediately or by giving it time to shut down

Opposite to `GenServer`, `Agent` and `Supervisor`, a Task has a default `:restart` of `:temporary`. This means the task will not be restarted even if it crashes. If you desire the task to be restarted for non-successful exits, do:

```
use Task, restart: :transient
```

If you want the task to always be restarted:

```
use Task, restart: :permanent
```

See the "Child specification" section in the `Supervisor` module for more detailed information. The `@doc` annotation immediately preceding `use Task` will be attached to the generated `child_spec/1` function.

Ancestor and Caller Tracking

Whenever you start a new process, Elixir annotates the process with the parent through the `$ancestors` key in the process dictionary. This is often used to track the hierarchy inside a supervision tree.

For example, we recommend developers to always start tasks under a supervisor. This provides more visibility and allows you to control how those tasks are terminated when a node shuts down. That might look something like `Task.Supervisor.start_child(MySupervisor, task_function)`. This means that, although your code is the one invoking the task, the actual ancestor of the task is the supervisor, as the supervisor is the one effectively starting it.

To track the relationship between your code and the task, we use the `$callers` key in the process dictionary. Therefore, assuming the `Task.Supervisor` call above, we have:

```
[your code] -- calls --> [supervisor] ---- spawns -->
[task]
```

Which means we store the following relationships:

```
[your code]              [supervisor] <-- ancestor --
[task]
    ^                                                |
    |---------------------- caller ----------------------|
```

The list of callers of the current process can be retrieved from the Process dictionary with `Process.get(:"$callers")`. This will return either `nil` or a list `[pid_n, ..., pid2, pid1]` with at least one entry

where `pid_n` is the PID that called the current process, `pid2` called `pid_n`, and `pid2` was called by `pid1`.

If a task crashes, the callers field is included as part of the log message metadata under the `:callers` key.

Summary

Types

async_stream_option()

Options given to `async_stream` functions.

ref()

The task opaque reference.

t()

The Task type.

Functions

%Task{}

The Task struct.

async(fun)

Starts a task that must be awaited on.

async(module, function_name, args)

Starts a task that must be awaited on.

async_stream(enumerable, fun, options \\ [])

Returns a stream that runs the given function `fun` concurrently on each element in `enumerable`.

async_stream(enumerable, module, function_name, args, options \\ [])

Returns a stream where the given function (`module` and `function_name`) is mapped concurrently on each element in `enumerable`.

await(task, timeout \\ 5000)

Awaits a task reply and returns it.

await_many(tasks, timeout \\ 5000)

Awaits replies from multiple tasks and returns them.

child_spec(arg)

Returns a specification to start a task under a supervisor.

completed(result)

Starts a task that immediately completes with the given `result`.

ignore(task)

Ignores an existing task.

shutdown(task, shutdown \\ 5000)

Unlinks and shuts down the task, and then checks for a reply.

start(fun)

Starts a task.

start(module, function_name, args)

Starts a task.

start_link(fun)

Starts a task as part of a supervision tree with the given `fun`.

start_link(module, function, args)

Starts a task as part of a supervision tree with the given `module`, `function`, and `args`.

yield(task, timeout \\ 5000)

Temporarily blocks the caller process waiting for a task reply.

yield_many(tasks, opts \\ [])

Yields to multiple tasks in the given time interval.

Types

async_stream_option()

(since 1.17.0)

```
@type async_stream_option() ::
  {:max_concurrency, pos_integer()}
  | {:ordered, boolean()}
  | {:timeout, timeout()}
  | {:on_timeout, :exit | :kill_task}
  | {:zip_input_on_exit, boolean()}
```

Options given to `async_stream` functions.

ref()

```
@opaque ref()
```

The task opaque reference.

t()

```
@type t() :: %Task{mfa: mfa(), owner: pid(), pid: pid() |
nil, ref: ref()}
```

The Task type.

See `%Task{}` for information about each field of the structure.

Functions

%Task{}

(struct)

The Task struct.

It contains these fields:

- `:mfa` - a three-element tuple containing the module, function name, and arity invoked to start the task in `async/1` and `async/3`
- `:owner` - the PID of the process that started the task
- `:pid` - the PID of the task process; `nil` if there is no process specifically assigned for the task

- `:ref` - an opaque term used as the task monitor reference

async(fun)

```
@spec async((-> any())) :: t()
```

Starts a task that must be awaited on.

`fun` must be a zero-arity anonymous function. This function spawns a process that is linked to and monitored by the caller process. A `Task` struct is returned containing the relevant information.

If you start an `async`, you **must await**. This is either done by calling `Task.await/2` or `Task.yield/2` followed by `Task.shutdown/2` on the returned task. Alternatively, if you spawn a task inside a `GenServer`, then the `GenServer` will automatically await for you and call `GenServer.handle_info/2` with the task response and associated `:DOWN` message.

Read the `Task` module documentation for more information about the general usage of async tasks.

Linking

This function spawns a process that is linked to and monitored by the caller process. The linking part is important because it aborts the task if the parent process dies. It also guarantees the code before async/await has the same properties after you add the async call. For example, imagine you have this:

```
x = heavy_function()
y = some_function()
```

```
x + y
```

Now you want to make the `heavy_function()` async:

```
x = Task.async(&heavy_function/0)
y = some_function()
Task.await(x) + y
```

As before, if `heavy_function/0` fails, the whole computation will fail, including the caller process. If you don't want the task to fail then you must change the `heavy_fun/0` code in the same way you would achieve it if you didn't have the async call. For example, to either return `{:ok, val} | :error` results or, in more extreme cases, by using `try/rescue`. In other words, an asynchronous task should be thought of as an extension of the caller process rather than a mechanism to isolate it from all errors.

If you don't want to link the caller to the task, then you must use a supervised task with `Task.Supervisor` and call `Task.Supervisor.async_nolink/2`.

In any case, avoid any of the following:

- Setting `:trap_exit` to `true` - trapping exits should be used only in special circumstances as it would make your process immune to not only exits from the task but from any other processes.

  Moreover, even when trapping exits, calling `await` will still exit if the task has terminated without sending its result back.

- Unlinking the task process started with `async`/`await`. If you unlink the processes and the task does not belong to any supervisor, you may leave dangling tasks in case the caller process dies.

Metadata

The task created with this function stores `:erlang.apply/2` in its `:mfa` metadata field, which is used internally to apply the anonymous function. Use `async/3` if you want another function to be used as metadata.

async(module, function_name, args)

```
@spec async(module(), atom(), [term()]) :: t()
```

Starts a task that must be awaited on.

Similar to `async/1` except the function to be started is specified by the given `module`, `function_name`, and `args`. The `module`, `function_name`, and its arity are stored as a tuple in the `:mfa` field for reflection purposes.

async_stream(enumerable, fun, options \\ [])

(since 1.4.0)

```
@spec async_stream(Enumerable.t(), (term() -> term()),
[async_stream_option()]) ::
  Enumerable.t()
```

Returns a stream that runs the given function `fun` concurrently on each element in `enumerable`.

Works the same as `async_stream/5` but with an anonymous function instead of a module-function-arguments tuple. `fun` must be a one-arity anonymous function.

Each `enumerable` element is passed as argument to the given function `fun` and processed by its own task. The tasks will be linked to the caller process, similarly to `async/1`.

Example

Count the code points in each string asynchronously, then add the counts together using reduce.

```
iex> strings = ["long string", "longer string", "there
are many of these"]
iex> stream = Task.async_stream(strings, fn text -> text
|> String.codepoints() |> Enum.count() end)
iex> Enum.sum_by(stream, fn {:ok, num} -> num end)
47
```

See `async_stream/5` for discussion, options, and more examples.

async_stream(enumerable, module, function_name, args, options \\ [])

(since 1.4.0)

```
@spec async_stream(Enumerable.t(), module(), atom(),
[term()], [async_stream_option()]) ::
  Enumerable.t()
```

Returns a stream where the given function (`module` and `function_name`) is mapped concurrently on each element in `enumerable`.

Each element of `enumerable` will be prepended to the given `args` and processed by its own task. Those tasks will be linked to an intermediate process that is then linked to the caller process. This means a failure in a

task terminates the caller process and a failure in the caller process terminates all tasks.

When streamed, each task will emit `{:ok, value}` upon successful completion or `{:exit, reason}` if the caller is trapping exits. It's possible to have `{:exit, {element, reason}}` for exits using the `:zip_input_on_exit` option. The order of results depends on the value of the `:ordered` option.

The level of concurrency and the time tasks are allowed to run can be controlled via options (see the "Options" section below).

Consider using `Task.Supervisor.async_stream/6` to start tasks under a supervisor. If you find yourself trapping exits to ensure errors in the tasks do not terminate the caller process, consider using `Task.Supervisor.async_stream_nolink/6` to start tasks that are not linked to the caller process.

Options

- `:max_concurrency` - sets the maximum number of tasks to run at the same time. Defaults to `System.schedulers_online/0`.

- `:ordered` - whether the results should be returned in the same order as the input stream. When the output is ordered, Elixir may need to buffer results to emit them in the original order. Setting this option to false disables the need to buffer at the cost of removing ordering. This is also useful when you're using the tasks only for the side effects. Note that regardless of what `:ordered` is set to, the tasks will process asynchronously. If you need to process elements in order, consider using `Enum.map/2` or `Enum.each/2` instead. Defaults to `true`.

- `:timeout` - the maximum amount of time (in milliseconds or `:infinity`) each task is allowed to execute for. Defaults to `5000`.
- `:on_timeout` - what to do when a task times out. The possible values are:
  - `:exit` (default) - the caller (the process that spawned the tasks) exits.
  - `:kill_task` - the task that timed out is killed. The value emitted for that task is `{:exit, :timeout}`.
- `:zip_input_on_exit` - (since v1.14.0) adds the original input to `:exit` tuples. The value emitted for that task is `{:exit, {input, reason}}`, where `input` is the collection element that caused an exit during processing. Defaults to `false`.

Example

Let's build a stream and then enumerate it:

```
stream = Task.async_stream(collection, Mod,
:expensive_fun, [])
Enum.to_list(stream)
```

The concurrency can be increased or decreased using the `:max_concurrency` option. For example, if the tasks are IO heavy, the value can be increased:

```
max_concurrency = System.schedulers_online() * 2
stream = Task.async_stream(collection, Mod,
:expensive_fun, [], max_concurrency: max_concurrency)
Enum.to_list(stream)
```

If you do not care about the results of the computation, you can run the stream with `Stream.run/1`. Also set `ordered: false`, as you don't care about the order of the results either:

```
stream = Task.async_stream(collection, Mod,
:expensive_fun, [], ordered: false)
Stream.run(stream)
```

First async tasks to complete

You can also use `async_stream/3` to execute M tasks and find the N tasks to complete. For example:

```
[
  &heavy_call_1/0,
  &heavy_call_2/0,
  &heavy_call_3/0
]
|> Task.async_stream(fn fun -> fun.() end, ordered:
false, max_concurrency: 3)
|> Stream.filter(&match?({:ok, _}, &1))
|> Enum.take(2)
```

In the example above, we are executing three tasks and waiting for the first 2 to complete. We use `Stream.filter/2` to restrict ourselves only to successfully completed tasks, and then use `Enum.take/2` to retrieve N items. Note it is important to set both `ordered: false` and `max_concurrency: M`, where M is the number of tasks, to make sure all calls execute concurrently.

Attention: unbound async + take

If you want to potentially process a high number of items and keep only part of the results, you may end-up processing more items than desired. Let's see an example:

```
1..100
|> Task.async_stream(fn i ->
  Process.sleep(100)
  IO.puts(to_string(i))
end)
|> Enum.take(10)
```

Running the example above in a machine with 8 cores will process 16 items, even though you want only 10 elements, since `async_stream/3` process items concurrently. That's because it will process 8 elements at once. Then all 8 elements complete at roughly the same time, causing 8 elements to be kicked off for processing. Out of these extra 8, only 2 will be used, and the rest will be terminated.

Depending on the problem, you can filter or limit the number of elements upfront:

```
1..100
|> Stream.take(10)
|> Task.async_stream(fn i ->
  Process.sleep(100)
  IO.puts(to_string(i))
end)
|> Enum.to_list()
```

In other cases, you likely want to tweak `:max_concurrency` to limit how many elements may be over processed at the cost of reducing concurrency. You can also set the number of elements to take to be a

multiple of `:max_concurrency`. For instance, setting `max_concurrency: 5` in the example above.

await(task, timeout \\ 5000)

```
@spec await(t(), timeout()) :: term()
```

Awaits a task reply and returns it.

In case the task process dies, the caller process will exit with the same reason as the task.

A timeout, in milliseconds or `:infinity`, can be given with a default value of `5000`. If the timeout is exceeded, then the caller process will exit. If the task process is linked to the caller process which is the case when a task is started with `async`, then the task process will also exit. If the task process is trapping exits or not linked to the caller process, then it will continue to run.

This function assumes the task's monitor is still active or the monitor's `:DOWN` message is in the message queue. If it has been demonitored, or the message already received, this function will wait for the duration of the timeout awaiting the message.

This function can only be called once for any given task. If you want to be able to check multiple times if a long-running task has finished its computation, use `yield/2` instead.

Examples

```
iex> task = Task.async(fn -> 1 + 1 end)
iex> Task.await(task)
```

Compatibility with OTP behaviours

It is not recommended to `await` a long-running task inside an OTP behaviour such as `GenServer`. Instead, you should match on the message coming from a task inside your `GenServer.handle_info/2` callback.

A GenServer will receive two messages on `handle_info/2`:

- `{ref, result}` - the reply message where `ref` is the monitor reference returned by the `task.ref` and `result` is the task result
- `{:DOWN, ref, :process, pid, reason}` - since all tasks are also monitored, you will also receive the `:DOWN` message delivered by `Process.monitor/1`. If you receive the `:DOWN` message without a a reply, it means the task crashed

Another consideration to have in mind is that tasks started by `Task.async/1` are always linked to their callers and you may not want the GenServer to crash if the task crashes. Therefore, it is preferable to instead use `Task.Supervisor.async_nolink/3` inside OTP behaviours. For completeness, here is an example of a GenServer that start tasks and handles their results:

```
defmodule GenServerTaskExample do
  use GenServer

  def start_link(opts) do
    GenServer.start_link(__MODULE__, :ok, opts)
  end
```

```
  def init(_opts) do
    # We will keep all running tasks in a map
    {:ok, %{tasks: %{}}}
  end

  # Imagine we invoke a task from the GenServer to access
a URL...
  def handle_call(:some_message, _from, state) do
    url = ...
    task =
Task.Supervisor.async_nolink(MyApp.TaskSupervisor, fn ->
fetch_url(url) end)

    # After we start the task, we store its reference and
the url it is fetching
    state = put_in(state.tasks[task.ref], url)

    {:reply, :ok, state}
  end

  # If the task succeeds...
  def handle_info({ref, result}, state) do
    # The task succeed so we can demonitor its reference
    Process.demonitor(ref, [:flush])

    {url, state} = pop_in(state.tasks[ref])
    IO.puts("Got #{inspect(result)} for URL #{inspect
url}")
    {:noreply, state}
  end

  # If the task fails...
  def handle_info({:DOWN, ref, _, _, reason}, state) do
    {url, state} = pop_in(state.tasks[ref])
    IO.puts("URL #{inspect url} failed with reason #
{inspect(reason)}")
```

```
    {:noreply, state}
  end
end
```

With the server defined, you will want to start the task supervisor above and the GenServer in your supervision tree:

```
children = [
  {Task.Supervisor, name: MyApp.TaskSupervisor},
  {GenServerTaskExample, name:
MyApp.GenServerTaskExample}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

await_many(tasks, timeout \\ 5000)

(since 1.11.0)

```
@spec await_many([t()], timeout()) :: [term()]
```

Awaits replies from multiple tasks and returns them.

This function receives a list of tasks and waits for their replies in the given time interval. It returns a list of the results, in the same order as the tasks supplied in the `tasks` input argument.

If any of the task processes dies, the caller process will exit with the same reason as that task.

A timeout, in milliseconds or `:infinity`, can be given with a default value of `5000`. If the timeout is exceeded, then the caller process will exit. Any task processes that are linked to the caller process (which is the case when a task is started with `async`) will also exit. Any task

processes that are trapping exits or not linked to the caller process will continue to run.

This function assumes the tasks' monitors are still active or the monitor's `:DOWN` message is in the message queue. If any tasks have been demonitored, or the message already received, this function will wait for the duration of the timeout.

This function can only be called once for any given task. If you want to be able to check multiple times if a long-running task has finished its computation, use `yield_many/2` instead.

Compatibility with OTP behaviours

It is not recommended to `await` long-running tasks inside an OTP behaviour such as `GenServer`. See `await/2` for more information.

Examples

```
iex> tasks = [
...>   Task.async(fn -> 1 + 1 end),
...>   Task.async(fn -> 2 + 3 end)
...> ]
iex> Task.await_many(tasks)
[2, 5]
```

child_spec(arg)

(since 1.5.0)

```
@spec child_spec(term()) :: Supervisor.child_spec()
```

Returns a specification to start a task under a supervisor.

`arg` is passed as the argument to `Task.start_link/1` in the `:start` field of the spec.

For more information, see the `Supervisor` module, the `Supervisor.child_spec/2` function and the `Supervisor.child_spec/0` type.

completed(result)

(since 1.13.0)

```
@spec completed(any()) :: t()
```

Starts a task that immediately completes with the given `result`.

Unlike `async/1`, this task does not spawn a linked process. It can be awaited or yielded like any other task.

Usage

In some cases, it is useful to create a "completed" task that represents a task that has already run and generated a result. For example, when processing data you may be able to determine that certain inputs are invalid before dispatching them for further processing:

```
def process(data) do
  tasks =
    for entry <- data do
      if invalid_input?(entry) do
        Task.completed({:error, :invalid_input})
      else
        Task.async(fn -> further_process(entry) end)
      end
    end
```

```
    Task.await_many(tasks)
  end
```

In many cases, `Task.completed/1` may be avoided in favor of returning the result directly. You should generally only require this variant when working with mixed asynchrony, when a group of inputs will be handled partially synchronously and partially asynchronously.

ignore(task)

(since 1.13.0)

```
@spec ignore(t()) :: {:ok, term()} | {:exit, term()} | nil
```

Ignores an existing task.

This means the task will continue running, but it will be unlinked and you can no longer yield, await or shut it down.

Returns `{:ok, reply}` if the reply is received before ignoring the task, `{:exit, reason}` if the task died before ignoring it, otherwise `nil`.

Important: avoid using `Task.async/1,3` and then immediately ignoring the task. If you want to start tasks you don't care about their results, use `Task.Supervisor.start_child/2` instead.

shutdown(task, shutdown \\ 5000)

```
@spec shutdown(t(), timeout() | :brutal_kill) :: {:ok,
term()} | {:exit, term()} | nil
```

Unlinks and shuts down the task, and then checks for a reply.

Returns `{:ok, reply}` if the reply is received while shutting down the task, `{:exit, reason}` if the task died, otherwise `nil`. Once shut down, you can no longer await or yield it.

The second argument is either a timeout or `:brutal_kill`. In case of a timeout, a `:shutdown` exit signal is sent to the task process and if it does not exit within the timeout, it is killed. With `:brutal_kill` the task is killed straight away. In case the task terminates abnormally (possibly killed by another process), this function will exit with the same reason.

It is not required to call this function when terminating the caller, unless exiting with reason `:normal` or if the task is trapping exits. If the caller is exiting with a reason other than `:normal` and the task is not trapping exits, the caller's exit signal will stop the task. The caller can exit with reason `:shutdown` to shut down all of its linked processes, including tasks, that are not trapping exits without generating any log messages.

If there is no process linked to the task, such as tasks started by `Task.completed/1`, we check for a response or error accordingly, but without shutting a process down.

If a task's monitor has already been demonitored or received and there is not a response waiting in the message queue this function will return `{:exit, :noproc}` as the result or exit reason can not be determined.

start(fun)

```
@spec start((-> any())) :: {:ok, pid()}
```

Starts a task.

`fun` must be a zero-arity anonymous function.

This should only used when the task is used for side-effects (like I/O) and you have no interest on its results nor if it completes successfully.

If the current node is shutdown, the node will terminate even if the task was not completed. For this reason, we recommend to use `Task.Supervisor.start_child/2` instead, which allows you to control the shutdown time via the `:shutdown` option.

start(module, function_name, args)

```
@spec start(module(), atom(), [term()]) :: {:ok, pid()}
```

Starts a task.

This should only used when the task is used for side-effects (like I/O) and you have no interest on its results nor if it completes successfully.

If the current node is shutdown, the node will terminate even if the task was not completed. For this reason, we recommend to use `Task.Supervisor.start_child/2` instead, which allows you to control the shutdown time via the `:shutdown` option.

start_link(fun)

```
@spec start_link((-> any())) :: {:ok, pid()}
```

Starts a task as part of a supervision tree with the given `fun`.

`fun` must be a zero-arity anonymous function.

This is used to start a statically supervised task under a supervision tree.

start_link(module, function, args)

```
@spec start_link(module(), atom(), [term()]) :: {:ok, pid()}
```

Starts a task as part of a supervision tree with the given `module`, `function`, and `args`.

This is used to start a statically supervised task under a supervision tree.

yield(task, timeout \\ 5000)

```
@spec yield(t(), timeout()) :: {:ok, term()} | {:exit, term()} | nil
```

Temporarily blocks the caller process waiting for a task reply.

Returns `{:ok, reply}` if the reply is received, `nil` if no reply has arrived, or `{:exit, reason}` if the task has already exited. Keep in mind that normally a task failure also causes the process owning the task to exit. Therefore this function can return `{:exit, reason}` if at least one of the conditions below apply:

- the task process exited with the reason `:normal`
- the task isn't linked to the caller (the task was started with `Task.Supervisor.async_nolink/2` or `Task.Supervisor.async_nolink/4`)
- the caller is trapping exits

A timeout, in milliseconds or `:infinity`, can be given with a default value of `5000`. If the time runs out before a message from the task is received, this function will return `nil` and the monitor will remain

active. Therefore `yield/2` can be called multiple times on the same task.

This function assumes the task's monitor is still active or the monitor's `:DOWN` message is in the message queue. If it has been demonitored or the message already received, this function will wait for the duration of the timeout awaiting the message.

If you intend to shut the task down if it has not responded within `timeout` milliseconds, you should chain this together with `shutdown/1`, like so:

```
case Task.yield(task, timeout) || Task.shutdown(task) do
  {:ok, result} ->
    result

  nil ->
    Logger.warning("Failed to get a result in #
{timeout}ms")
    nil
end
```

If you intend to check on the task but leave it running after the timeout, you can chain this together with `ignore/1`, like so:

```
case Task.yield(task, timeout) || Task.ignore(task) do
  {:ok, result} ->
    result

  nil ->
    Logger.warning("Failed to get a result in #
{timeout}ms")
```

```
      nil
  end
```

That ensures that if the task completes after the `timeout` but before `shutdown/1` has been called, you will still get the result, since `shutdown/1` is designed to handle this case and return the result.

yield_many(tasks, opts \\ [])

```
@spec yield_many([t()], timeout()) :: [{t(), {:ok, term()} |
{:exit, term()} | nil}]
@spec yield_many([t()],
  limit: pos_integer(),
  timeout: timeout(),
  on_timeout: :nothing | :ignore | :kill_task
) :: [{t(), {:ok, term()} | {:exit, term()} | nil}]
```

Yields to multiple tasks in the given time interval.

This function receives a list of tasks and waits for their replies in the given time interval. It returns a list of two-element tuples, with the task as the first element and the yielded result as the second. The tasks in the returned list will be in the same order as the tasks supplied in the `tasks` input argument.

Similarly to `yield/2`, each task's result will be

- `{:ok, term}` if the task has successfully reported its result back in the given time interval
- `{:exit, reason}` if the task has died
- `nil` if the task keeps running, either because a limit has been reached or past the timeout

Check `yield/2` for more information.

Example

`Task.yield_many/2` allows developers to spawn multiple tasks and retrieve the results received in a given time frame. If we combine it with `Task.shutdown/2` (or `Task.ignore/1`), it allows us to gather those results and cancel (or ignore) the tasks that have not replied in time.

Let's see an example.

```
tasks =
  for i <- 1..10 do
    Task.async(fn ->
      Process.sleep(i * 1000)
      i
    end)
  end

tasks_with_results = Task.yield_many(tasks, timeout:
5000)

results =
  Enum.map(tasks_with_results, fn {task, res} ->
    # Shut down the tasks that did not reply nor exit
    res || Task.shutdown(task, :brutal_kill)
  end)

# Here we are matching only on {:ok, value} and
# ignoring {:exit, _} (crashed tasks) and `nil` (no
replies)
for {:ok, value} <- results do
```

```
  IO.inspect(value)
end
```

In the example above, we create tasks that sleep from 1 up to 10 seconds and return the number of seconds they slept for. If you execute the code all at once, you should see 1 up to 5 printed, as those were the tasks that have replied in the given time. All other tasks will have been shut down using the `Task.shutdown/2` call.

As a convenience, you can achieve a similar behavior to above by specifying the `:on_timeout` option to be `:kill_task` (or `:ignore`). See `Task.await_many/2` if you would rather exit the caller process on timeout.

Options

The second argument is either a timeout or options, which defaults to this:

- `:limit` - the maximum amount of tasks to wait for. If the limit is reached before the timeout, this function returns immediately without triggering the `:on_timeout` behaviour
- `:timeout` - the maximum amount of time (in milliseconds or `:infinity`) each task is allowed to execute for. Defaults to `5000`.
- `:on_timeout` - what to do when a task times out. The possible values are:
  - `:nothing` - do nothing (default). The tasks can still be awaited on, yielded on, ignored, or shut down later.
  - `:ignore` - the results of the task will be ignored.
  - `:kill_task` - the task that timed out is killed.

Task.Supervisor

A task supervisor.

This module defines a supervisor which can be used to dynamically supervise tasks.

A task supervisor is started with no children, often under a supervisor and a name:

```
children = [
  {Task.Supervisor, name: MyApp.TaskSupervisor}
]

Supervisor.start_link(children, strategy: :one_for_one)
```

The options given in the child specification are documented in `start_link/1`.

Once started, you can start tasks directly under the supervisor, for example:

```
task = Task.Supervisor.async(MyApp.TaskSupervisor, fn ->
  :do_some_work
end)
```

See the `Task` module for more examples.

Scalability and partitioning

The `Task.Supervisor` is a single process responsible for starting other processes. In some applications, the `Task.Supervisor` may become a bottleneck. To address this, you can start multiple instances of the `Task.Supervisor` and then pick a random instance to start the task on.

Instead of:

```
children = [
  {Task.Supervisor, name: MyApp.TaskSupervisor}
]
```

and:

```
Task.Supervisor.async(MyApp.TaskSupervisor, fn ->
:do_some_work end)
```

You can do this:

```
children = [
  {PartitionSupervisor,
   child_spec: Task.Supervisor,
   name: MyApp.TaskSupervisors}
]
```

and then:

```
Task.Supervisor.async(
  {:via, PartitionSupervisor, {MyApp.TaskSupervisors,
self()}},
  fn -> :do_some_work end
)
```

In the code above, we start a partition supervisor that will by default start a dynamic supervisor for each core in your machine. Then, instead of calling the `Task.Supervisor` by name, you call it through the partition supervisor using the `{:via, PartitionSupervisor, {name, key}}` format, where `name` is the name of the partition supervisor and `key` is the routing key. We picked `self()` as the routing key, which means each process will be assigned one of the existing task supervisors. Read the `PartitionSupervisor` docs for more information.

Name registration

A `Task.Supervisor` is bound to the same name registration rules as a `GenServer`. Read more about them in the `GenServer` docs.

Summary

Types

async_opts()

Options for `async/3`, `async/5`, `async_nolink/3`, and `async_nolink/5` functions.

async_stream_option()

Options given to `async_stream` and `async_stream_nolink` functions.

option()

Option values used by `start_link`

start_child_opts()

Functions

async(supervisor, fun, options \\ [])

Starts a task that can be awaited on.

async(supervisor, module, fun, args, options \\ [])

Starts a task that can be awaited on.

async_nolink(supervisor, fun, options \\ [])

Starts a task that can be awaited on.

async_nolink(supervisor, module, fun, args, options \\ [])

Starts a task that can be awaited on.

async_stream(supervisor, enumerable, fun, options \\ [])

Returns a stream that runs the given function `fun` concurrently on each element in `enumerable`.

async_stream(supervisor, enumerable, module, function, args, options \\ [])

Returns a stream where the given function (`module` and `function`) is mapped concurrently on each element in `enumerable`.

async_stream_nolink(supervisor, enumerable, fun, options \\ [])

Returns a stream that runs the given `function` concurrently on each element in `enumerable`.

async_stream_nolink(supervisor, enumerable, module, function, args, options \\ [])

Returns a stream where the given function (`module` and `function`) is mapped concurrently on each element in `enumerable`.

children(supervisor)

Returns all children PIDs except those that are restarting.

start_child(supervisor, fun, options \\ [])

Starts a task as a child of the given `supervisor`.

start_child(supervisor, module, fun, args, options \\ [])

Starts a task as a child of the given `supervisor`.

start_link(options \\ [])

Starts a new supervisor.

terminate_child(supervisor, pid)

Terminates the child with the given `pid`.

Types

async_opts()

```
@type async_opts() :: [{:shutdown, :brutal_kill | timeout()}]
```

Options for `async/3`, `async/5`, `async_nolink/3`, and `async_nolink/5` functions.

async_stream_option()

(since 1.17.0)

```
@type async_stream_option() ::
  Task.async_stream_option() | {:shutdown,
Supervisor.shutdown()}
```

Options given to `async_stream` and `async_stream_nolink` functions.

option()

```
@type option() :: GenServer.option() |
DynamicSupervisor.init_option()
```

Option values used by `start_link`

start_child_opts()

```
@type start_child_opts() :: [
  restart: :temporary | :transient | :permanent,
  shutdown: :brutal_kill | timeout()
]
```

Functions

async(supervisor, fun, options \\ [])

```
@spec async(Supervisor.supervisor(), (-> any()),
async_opts()) :: Task.t()
```

Starts a task that can be awaited on.

The `supervisor` must be a reference as defined in `Supervisor`. The task will still be linked to the caller, see `Task.async/1` for more information and `async_nolink/3` for a non-linked variant.

Raises an error if `supervisor` has reached the maximum number of children.

Options

- `:shutdown - :brutal_kill` if the tasks must be killed directly on shutdown or an integer indicating the timeout value, defaults to 5000 milliseconds. The tasks must trap exits for the timeout to have an effect.

async(supervisor, module, fun, args, options \\ [])

```
@spec async(Supervisor.supervisor(), module(), atom(),
[term()], async_opts()) ::
  Task.t()
```

Starts a task that can be awaited on.

The `supervisor` must be a reference as defined in `Supervisor`. The task will still be linked to the caller, see `Task.async/1` for more information and `async_nolink/3` for a non-linked variant.

Raises an error if `supervisor` has reached the maximum number of children.

Options

- `:shutdown - :brutal_kill` if the tasks must be killed directly on shutdown or an integer indicating the timeout value, defaults to

5000 milliseconds. The tasks must trap exits for the timeout to have an effect.

async_nolink(supervisor, fun, options \\ [])

```
@spec async_nolink(Supervisor.supervisor(), (-> any()),
async_opts()) :: Task.t()
```

Starts a task that can be awaited on.

The `supervisor` must be a reference as defined in `Supervisor`. The task won't be linked to the caller, see `Task.async/1` for more information.

Raises an error if `supervisor` has reached the maximum number of children.

Note this function requires the task supervisor to have `:temporary` as the `:restart` option (the default), as `async_nolink/3` keeps a direct reference to the task which is lost if the task is restarted.

Options

- `:shutdown` - `:brutal_kill` if the tasks must be killed directly on shutdown or an integer indicating the timeout value, defaults to 5000 milliseconds. The tasks must trap exits for the timeout to have an effect.

Compatibility with OTP behaviours

If you create a task using `async_nolink` inside an OTP behaviour like `GenServer`, you should match on the message coming from the task inside your `GenServer.handle_info/2` callback.

The reply sent by the task will be in the format `{ref, result}`, where `ref` is the monitor reference held by the task struct and `result` is the return value of the task function.

Keep in mind that, regardless of how the task created with `async_nolink` terminates, the caller's process will always receive a `:DOWN` message with the same `ref` value that is held by the task struct. If the task terminates normally, the reason in the `:DOWN` message will be `:normal`.

Examples

Typically, you use `async_nolink/3` when there is a reasonable expectation that the task may fail, and you don't want it to take down the caller. Let's see an example where a `GenServer` is meant to run a single task and track its status:

```
defmodule MyApp.Server do
  use GenServer

  # ...

  def start_task do
    GenServer.call(__MODULE__, :start_task)
  end

  # In this case the task is already running, so we just
return :ok.
  def handle_call(:start_task, _from, %{ref: ref} =
state) when is_reference(ref) do
    {:reply, :ok, state}
  end

  # The task is not running yet, so let's start it.
```

```
  def handle_call(:start_task, _from, %{ref: nil} =
state) do
    task =
      Task.Supervisor.async_nolink(MyApp.TaskSupervisor,
fn ->
        ...
      end)

    # We return :ok and the server will continue running
    {:reply, :ok, %{state | ref: task.ref}}
  end

  # The task completed successfully
  def handle_info({ref, answer}, %{ref: ref} = state) do
    # We don't care about the DOWN message now, so let's
demonitor and flush it
    Process.demonitor(ref, [:flush])
    # Do something with the result and then return
    {:noreply, %{state | ref: nil}}
  end

  # The task failed
  def handle_info({:DOWN, ref, :process, _pid, _reason},
%{ref: ref} = state) do
    # Log and possibly restart the task...
    {:noreply, %{state | ref: nil}}
  end
end
```

async_nolink(supervisor, module, fun, args, options \\ [])

```
@spec async_nolink(Supervisor.supervisor(), module(), atom(),
[term()], async_opts()) ::
  Task.t()
```

Starts a task that can be awaited on.

The `supervisor` must be a reference as defined in `Supervisor`. The task won't be linked to the caller, see `Task.async/1` for more information.

Raises an error if `supervisor` has reached the maximum number of children.

Note this function requires the task supervisor to have `:temporary` as the `:restart` option (the default), as `async_nolink/5` keeps a direct reference to the task which is lost if the task is restarted.

async_stream(supervisor, enumerable, fun, options \\ [])

(since 1.4.0)

```
@spec async_stream(
  Supervisor.supervisor(),
  Enumerable.t(),
  (term() -> term()),
  [async_stream_option()]
) :: Enumerable.t()
```

Returns a stream that runs the given function `fun` concurrently on each element in `enumerable`.

Each element in `enumerable` is passed as argument to the given function `fun` and processed by its own task. The tasks will be spawned under the given `supervisor` and linked to the caller process, similarly to `async/3`.

See `async_stream/6` for discussion, options, and examples.

async_stream(supervisor, enumerable, module, function, args, options \\ [])

(since 1.4.0)

```
@spec async_stream(
  Supervisor.supervisor(),
  Enumerable.t(),
  module(),
  atom(),
  [term()],
  [async_stream_option()]
) :: Enumerable.t()
```

Returns a stream where the given function (`module` and `function`) is mapped concurrently on each element in `enumerable`.

Each element will be prepended to the given `args` and processed by its own task. The tasks will be spawned under the given `supervisor` and linked to the caller process, similarly to `async/5`.

When streamed, each task will emit `{:ok, value}` upon successful completion or `{:exit, reason}` if the caller is trapping exits. The order of results depends on the value of the `:ordered` option.

The level of concurrency and the time tasks are allowed to run can be controlled via options (see the "Options" section below).

If you find yourself trapping exits to handle exits inside the async stream, consider using `async_stream_nolink/6` to start tasks that are not linked to the calling process.

Options

- `:max_concurrency` - sets the maximum number of tasks to run at the same time. Defaults to `System.schedulers_online/0`.

- `:ordered` - whether the results should be returned in the same order as the input stream. This option is useful when you have large streams and don't want to buffer results before they are delivered. This is also useful when you're using the tasks for side effects. Defaults to `true`.

- `:timeout` - the maximum amount of time to wait (in milliseconds) without receiving a task reply (across all running tasks). Defaults to `5000`.

- `:on_timeout` - what do to when a task times out. The possible values are:

    - `:exit` (default) - the process that spawned the tasks exits.
    - `:kill_task` - the task that timed out is killed. The value emitted for that task is `{:exit, :timeout}`.

- `:zip_input_on_exit` - (since v1.14.0) adds the original input to `:exit` tuples. The value emitted for that task is `{:exit, {input, reason}}`, where `input` is the collection element that caused an exited during processing. Defaults to `false`.

- `:shutdown` - `:brutal_kill` if the tasks must be killed directly on shutdown or an integer indicating the timeout value. Defaults to `5000` milliseconds. The tasks must trap exits for the timeout to have an effect.

Examples

Let's build a stream and then enumerate it:

```
  stream = Task.Supervisor.async_stream(MySupervisor,
  collection, Mod, :expensive_fun, [])
  Enum.to_list(stream)
```

async_stream_nolink(supervisor, enumerable, fun, options \\ [])

(since 1.4.0)

```
@spec async_stream_nolink(
  Supervisor.supervisor(),
  Enumerable.t(),
  (term() -> term()),
  [async_stream_option()]
) :: Enumerable.t()
```

Returns a stream that runs the given `function` concurrently on each element in `enumerable`.

Each element in `enumerable` is passed as argument to the given function `fun` and processed by its own task. The tasks will be spawned under the given `supervisor` and will not be linked to the caller process, similarly to `async_nolink/3`.

See `async_stream/6` for discussion and examples.

Error handling and cleanup

Even if tasks are not linked to the caller, there is no risk of leaving dangling tasks running after the stream halts.

Consider the following example:

```
Task.Supervisor.async_stream_nolink(MySupervisor,
collection, fun, on_timeout: :kill_task, ordered: false)
|> Enum.each(fn
  {:ok, _} -> :ok
  {:exit, reason} -> raise "Task exited: #
{Exception.format_exit(reason)}"
end)
```

If one task raises or times out:

1. the second clause gets called
2. an exception is raised
3. the stream halts
4. all ongoing tasks will be shut down

Here is another example:

```
Task.Supervisor.async_stream_nolink(MySupervisor,
collection, fun, on_timeout: :kill_task, ordered: false)
|> Stream.filter(&match?({:ok, _}, &1))
|> Enum.take(3)
```

This will return the three first tasks to succeed, ignoring timeouts and errors, and shut down every ongoing task.

Just running the stream with `Stream.run/1` on the other hand would ignore errors and process the whole stream.

async_stream_nolink(supervisor, enumerable, module, function, args, options \\ [])

(since 1.4.0)

```
@spec async_stream_nolink(
  Supervisor.supervisor(),
```

```
  Enumerable.t(),
  module(),
  atom(),
  [term()],
  [async_stream_option()]
) :: Enumerable.t()
```

Returns a stream where the given function (`module` and `function`) is mapped concurrently on each element in `enumerable`.

Each element in `enumerable` will be prepended to the given `args` and processed by its own task. The tasks will be spawned under the given `supervisor` and will not be linked to the caller process, similarly to `async_nolink/5`.

See `async_stream/6` for discussion, options, and examples.

children(supervisor)

```
@spec children(Supervisor.supervisor()) :: [pid()]
```

Returns all children PIDs except those that are restarting.

Note that calling this function when supervising a large number of children under low memory conditions can bring the system down due to an out of memory error.

start_child(supervisor, fun, options \\ [])

```
@spec start_child(Supervisor.supervisor(), (-> any()),
start_child_opts()) ::
  DynamicSupervisor.on_start_child()
```

Starts a task as a child of the given `supervisor`.

```
Task.Supervisor.start_child(MyTaskSupervisor, fn ->
  IO.puts("I am running in a task")
end)
```

Note that the spawned process is not linked to the caller, but only to the supervisor. This command is useful in case the task needs to perform side-effects (like I/O) and you have no interest in its results nor if it completes successfully.

Options

- `:restart` - the restart strategy, may be `:temporary` (the default), `:transient` or `:permanent`. `:temporary` means the task is never restarted, `:transient` means it is restarted if the exit is not `:normal`, `:shutdown` or `{:shutdown, reason}`. A `:permanent` restart strategy means it is always restarted.

- `:shutdown` - `:brutal_kill` if the task must be killed directly on shutdown or an integer indicating the timeout value, defaults to 5000 milliseconds. The task must trap exits for the timeout to have an effect.

start_child(supervisor, module, fun, args, options \\ [])

```
@spec start_child(
  Supervisor.supervisor(),
  module(),
  atom(),
  [term()],
  start_child_opts()
```

```
) ::
  DynamicSupervisor.on_start_child()
```

Starts a task as a child of the given `supervisor`.

Similar to `start_child/3` except the task is specified by the given `module`, `fun` and `args`.

start_link(options \\ [])

```
@spec start_link([option()]) :: Supervisor.on_start()
```

Starts a new supervisor.

Examples

A task supervisor is typically started under a supervision tree using the tuple format:

```
{Task.Supervisor, name: MyApp.TaskSupervisor}
```

You can also start it by calling `start_link/1` directly:

```
Task.Supervisor.start_link(name: MyApp.TaskSupervisor)
```

But this is recommended only for scripting and should be avoided in production code. Generally speaking, processes should always be started inside supervision trees.

Options

- `:name` - used to register a supervisor name, the supported values are described under the `Name Registration` section in the `GenServer` module docs;

- `:max_restarts`, `:max_seconds`, and `:max_children` - as specified in `DynamicSupervisor`;

This function could also receive `:restart` and `:shutdown` as options but those two options have been deprecated and it is now preferred to give them directly to `start_child`.

terminate_child(supervisor, pid)

```
@spec terminate_child(Supervisor.supervisor(), pid()) :: :ok
| {:error, :not_found}
```

Terminates the child with the given `pid`.

Collectable protocol

A protocol to traverse data structures.

The `Enum.into/2` function uses this protocol to insert an enumerable into a collection:

```
iex> Enum.into([a: 1, b: 2], %{})
%{a: 1, b: 2}
```

Why Collectable?

The `Enumerable` protocol is useful to take values out of a collection. In order to support a wide range of values, the functions provided by the `Enumerable` protocol do not keep shape. For example, passing a map to `Enum.map/2` always returns a list.

This design is intentional. `Enumerable` was designed to support infinite collections, resources and other structures with fixed shape. For example, it doesn't make sense to insert values into a `Range`, as it has a fixed shape where only the range limits and step are stored.

The `Collectable` module was designed to fill the gap left by the `Enumerable` protocol. `Collectable.into/1` can be seen as the opposite of `Enumerable.reduce/3`. If the functions in `Enumerable` are about taking values out, then `Collectable.into/1` is about collecting those values into a structure.

Examples

To show how to manually use the `Collectable` protocol, let's play with a simplified implementation for `MapSet`.

```
iex> {initial_acc, collector_fun} =
Collectable.into(MapSet.new())
iex> updated_acc = Enum.reduce([1, 2, 3], initial_acc, fn
elem, acc ->
...>   collector_fun.(acc, {:cont, elem})
...> end)
iex> collector_fun.(updated_acc, :done)
MapSet.new([1, 2, 3])
```

To show how the protocol can be implemented, we can again look at the simplified implementation for `MapSet`. In this implementation "collecting" elements simply means inserting them in the set through `MapSet.put/2`.

```
defimpl Collectable, for: MapSet do
  def into(map_set) do
    collector_fun = fn
      map_set_acc, {:cont, elem} ->
        MapSet.put(map_set_acc, elem)

      map_set_acc, :done ->
        map_set_acc

      _map_set_acc, :halt ->
        :ok
    end

    initial_acc = map_set

    {initial_acc, collector_fun}
```

```
  end
end
```

So now we can call `Enum.into/2`:

```
iex> Enum.into([1, 2, 3], MapSet.new())
MapSet.new([1, 2, 3])
```

Halting

The `:halt` flag will be given whenever the collection won't terminate correctly and must be used to clean up existing resources (such as sockets, file handles, etc).

Note it is not guaranteed that the accumulator given to halt will be the latest version of the accumulator returned by a previous call with `{:cont, elem}`. Therefore, you must track the collected results within the resource you intend to halt.

This is by design: ensuring halt is always called with the latest accumulator would make pure collectables (the ones that do not implement halt) expensive. However, given the collectables that must implement halt already need to track state, the burden of tracking the accumulator across invocations is put on them.

Summary

Types

command()
t()

All the types that implement this protocol.

Functions

into(collectable)

Returns an initial accumulator and a "collector" function.

Types

command()

```
@type command() :: {:cont, term()} | :done | :halt
```

t()

```
@type t() :: term()
```

All the types that implement this protocol.

Functions

into(collectable)

```
@spec into(t()) ::
  {initial_acc :: term(), collector :: (term(), command() ->
t() | term())}
```

Returns an initial accumulator and a "collector" function.

Receives a `collectable` which can be used as the initial accumulator that will be passed to the function.

The collector function receives a term and a command and injects the term into the collectable accumulator on every `{:cont, term}` command.

`:done` is passed as a command when no further values will be injected. This is useful when there's a need to close resources or normalizing values. A collectable must be returned when the command is `:done`.

If injection is suddenly interrupted, `:halt` is passed and the function can return any value as it won't be used.

For examples on how to use the `Collectable` protocol and `into/1` see the module documentation.

Enumerable protocol

Enumerable protocol used by `Enum` and `Stream` modules.

When you invoke a function in the `Enum` module, the first argument is usually a collection that must implement this protocol. For example, the expression `Enum.map([1, 2, 3], &(&1 * 2))` invokes `Enumerable.reduce/3` to perform the reducing operation that builds a mapped list by calling the mapping function `&(&1 * 2)` on every element in the collection and consuming the element with an accumulated list.

Internally, `Enum.map/2` is implemented as follows:

```
def map(enumerable, fun) do
  reducer = fn x, acc -> {:cont, [fun.(x) | acc]} end
  Enumerable.reduce(enumerable, {:cont, []}, reducer) |>
elem(1) |> :lists.reverse()
end
```

Note that the user-supplied function is wrapped into a `reducer/0` function. The `reducer/0` function must return a tagged tuple after each step, as described in the `acc/0` type. At the end, `Enumerable.reduce/3` returns `result/0`.

This protocol uses tagged tuples to exchange information between the reducer function and the data type that implements the protocol. This allows enumeration of resources, such as files, to be done efficiently while also guaranteeing the resource will be closed at the end of the enumeration. This protocol also allows suspension of the enumeration,

which is useful when interleaving between many enumerables is required (as in the `zip/1` and `zip/2` functions).

This protocol requires four functions to be implemented, `reduce/3`, `count/1`, `member?/2`, and `slice/1`. The core of the protocol is the `reduce/3` function. All other functions exist as optimizations paths for data structures that can implement certain properties in better than linear time.

Default implementation for lists

Sometimes you may want to implement this protocol for a list contained in struct. This can be done by delegating to the `Enumerable.List` module in the `reduce/3` implementation and providing a straight-forward implementation for the remaining ones:

```
defimpl Enumerable, for: CustomStruct do
  def count(struct), do: {:ok, length(struct.items)}
  def member?(struct, value), do: {:ok, value in
struct.items}
  def slice(struct), do: {:error, __MODULE__}
  def reduce(struct, acc, fun), do:
Enumerable.List.reduce(struct.items, acc, fun)
end
```

Summary

Types

acc()

The accumulator value for each step.

continuation()

A partially applied reduce function.

reducer()

The reducer function.

result()

The result of the reduce operation.

slicing_fun()

A slicing function that receives the initial position, the number of elements in the slice, and the step.

t()

All the types that implement this protocol.

t(_element)

An enumerable of elements of type `element`.

to_list_fun()

Receives an enumerable and returns a list.

Functions

count(enumerable)

Retrieves the number of elements in the `enumerable`.

member?(enumerable, element)

Checks if an `element` exists within the `enumerable`.

reduce(enumerable, acc, fun)

Reduces the `enumerable` into an element.

slice(enumerable)

Returns a function that slices the data structure contiguously.

Types

acc()

```
@type acc() :: {:cont, term()} | {:halt, term()} | {:suspend, term()}
```

The accumulator value for each step.

It must be a tagged tuple with one of the following "tags":

- `:cont` - the enumeration should continue
- `:halt` - the enumeration should halt immediately
- `:suspend` - the enumeration should be suspended immediately

Depending on the accumulator value, the result returned by `Enumerable.reduce/3` will change. Please check the `result/0` type documentation for more information.

In case a `reducer/0` function returns a `:suspend` accumulator, it must be explicitly handled by the caller and never leak.

continuation()

```
@type continuation() :: (acc() -> result())
```

A partially applied reduce function.

The continuation is the closure returned as a result when the enumeration is suspended. When invoked, it expects a new accumulator and it returns the result.

A continuation can be trivially implemented as long as the reduce function is defined in a tail recursive fashion. If the function is tail recursive, all the state is passed as arguments, so the continuation is the reducing function partially applied.

reducer()

```
@type reducer() :: (element :: term(), element_acc :: term()
-> acc())
```

The reducer function.

Should be called with the `enumerable` element and the accumulator contents.

Returns the accumulator for the next enumeration step.

result()

```
@type result() ::
  {:done, term()} | {:halted, term()} | {:suspended, term(),
continuation()}
```

The result of the reduce operation.

It may be *done* when the enumeration is finished by reaching its end, or *halted*/*suspended* when the enumeration was halted or suspended by the tagged accumulator.

In case the tagged `:halt` accumulator is given, the `:halted` tuple with the accumulator must be returned. Functions like `Enum.take_while/2` use `:halt` underneath and can be used to test halting enumerables.

In case the tagged `:suspend` accumulator is given, the caller must return the `:suspended` tuple with the accumulator and a continuation. The caller is then responsible of managing the continuation and the caller must always call the continuation, eventually halting or continuing until the end. `Enum.zip/2` uses suspension, so it can be used to test whether your implementation handles suspension correctly. You can also use `Stream.zip/2` with `Enum.take_while/2` to test the combination of `:suspend` with `:halt`.

slicing_fun()

```
@type slicing_fun() :: (start :: non_neg_integer(),
                        length :: pos_integer(),
                        step :: pos_integer() ->
                          [term()])
```

A slicing function that receives the initial position, the number of elements in the slice, and the step.

The `start` position is a number `>= 0` and guaranteed to exist in the `enumerable`. The length is a number `>= 1` in a way that `start + length * step <= count`, where `count` is the maximum amount of elements in the enumerable.

The function should return a non empty list where the amount of elements is equal to `length`.

t()

```
@type t() :: term()
```

All the types that implement this protocol.

t(_element)

(since 1.14.0)

```
@type t(_element) :: t()
```

An enumerable of elements of type `element`.

This type is equivalent to `t/0` but is especially useful for documentation.

For example, imagine you define a function that expects an enumerable of integers and returns an enumerable of strings:

```
@spec integers_to_strings(Enumerable.t(integer())) ::
Enumerable.t(String.t())
def integers_to_strings(integers) do
  Stream.map(integers, &Integer.to_string/1)
end
```

to_list_fun()

```
@type to_list_fun() :: (t() -> [term()])
```

Receives an enumerable and returns a list.

Functions

count(enumerable)

```
@spec count(t()) :: {:ok, non neg integer()} | {:error,
module()}
```

Retrieves the number of elements in the `enumerable`.

It should return `{:ok, count}` if you can count the number of elements in `enumerable` in a faster way than fully traversing it.

Otherwise it should return `{:error, __MODULE__}` and a default algorithm built on top of `reduce/3` that runs in linear time will be used.

member?(enumerable, element)

```
@spec member?(t(), term()) :: {:ok, boolean()} | {:error,
module()}
```

Checks if an `element` exists within the `enumerable`.

It should return `{:ok, boolean}` if you can check the membership of a given element in `enumerable` with `===/2` without traversing the whole of it.

Otherwise it should return `{:error, __MODULE__}` and a default algorithm built on top of `reduce/3` that runs in linear time will be used.

When called outside guards, the `in` and `not in` operators work by using this function.

reduce(enumerable, acc, fun)

```
@spec reduce(t(), acc(), reducer()) :: result()
```

Reduces the `enumerable` into an element.

Most of the operations in `Enum` are implemented in terms of reduce. This function should apply the given `reducer/0` function to each element in the `enumerable` and proceed as expected by the returned accumulator.

See the documentation of the types `result/0` and `acc/0` for more information.

Examples

As an example, here is the implementation of `reduce` for lists:

```
def reduce(_list, {:halt, acc}, _fun), do: {:halted, acc}
def reduce(list, {:suspend, acc}, fun), do: {:suspended,
acc, &reduce(list, &1, fun)}
def reduce([], {:cont, acc}, _fun), do: {:done, acc}
def reduce([head | tail], {:cont, acc}, fun), do:
reduce(tail, fun.(head, acc), fun)
```

slice(enumerable)

```
@spec slice(t()) ::
  {:ok, size :: non_neg_integer(), slicing_fun() |
to_list_fun()}
  | {:error, module()}
```

Returns a function that slices the data structure contiguously.

It should return either:

- `{:ok, size, slicing_fun}` - if the `enumerable` has a known bound and can access a position in the `enumerable` without traversing all previous elements. The `slicing_fun` will receive a `start` position, the `amount` of elements to fetch, and a `step`.

- `{:ok, size, to_list_fun}` - if the `enumerable` has a known bound and can access a position in the `enumerable` by first converting it to a list via `to_list_fun`.

- `{:error, __MODULE__}` - the enumerable cannot be sliced efficiently and a default algorithm built on top of `reduce/3` that runs in linear time will be used.

Differences to `count/1`

The `size` value returned by this function is used for boundary checks, therefore it is extremely important that this function only returns `:ok` if retrieving the `size` of the `enumerable` is cheap, fast, and takes constant time. Otherwise the simplest of operations, such as `Enum.at(enumerable, 0)`, will become too expensive.

On the other hand, the `count/1` function in this protocol should be implemented whenever you can count the number of elements in the collection without traversing it.

Inspect protocol

The `Inspect` protocol converts an Elixir data structure into an algebra document.

This is typically done when you want to customize how your own structs are inspected in logs and the terminal.

This documentation refers to implementing the `Inspect` protocol for your own data structures. To learn more about using inspect, see `Kernel.inspect/2` and `IO.inspect/2`.

Inspect representation

There are typically three choices of inspect representation. In order to understand them, let's imagine we have the following `User` struct:

```
defmodule User do
  defstruct [:id, :name, :address]
end
```

Our choices are:

1. Print the struct using Elixir's struct syntax, for example: `%User{address: "Earth", id: 13, name: "Jane"}`. This is the default representation and best choice if all struct fields are public.

2. Print using the `#User<...>` notation, for example: `#User<id: 13, name: "Jane", ...>`. This notation does not emit valid Elixir code and is typically used when the struct has private fields (for example, you may want to hide the field `:address` to redact person identifiable information).

3. Print the struct using the expression syntax, for example: `User.new(13, "Jane", "Earth")`. This assumes there is a `User.new/3` function. This option is mostly used as an alternative to option 2 for representing custom data structures, such as `MapSet`, `Date.Range`, and others.

You can implement the Inspect protocol for your own structs while adhering to the conventions above. Option 1 is the default representation and you can quickly achieve option 2 by deriving the `Inspect` protocol. For option 3, you need your custom implementation.

Deriving

The `Inspect` protocol can be derived to customize the order of fields (the default is alphabetical) and hide certain fields from structs, so they don't show up in logs, inspects and similar. The latter is especially useful for fields containing private information.

The supported options are:

- `:only` - only include the given fields when inspecting.
- `:except` - remove the given fields when inspecting.
- `:optional` - (since v1.14.0) a list of fields that should not be included when they match their default value. This can be used to simplify the struct representation at the cost of hiding information. Since v1.19.0, the `:all` atom can be passed to mark all fields as optional.

Whenever `:only` or `:except` are used to restrict fields, the struct will be printed using the `#User<...>` notation, as the struct can no longer be copy and pasted as valid Elixir code. Let's see an example:

```elixir
defmodule User do
  @derive {Inspect, only: [:id, :name]}
  defstruct [:id, :name, :address]
end

inspect(%User{id: 1, name: "Jane", address: "Earth"})
#=> #User<id: 1, name: "Jane", ...>
```

If you use only the `:optional` option, the struct will still be printed as a valid struct.

```elixir
defmodule Point do
  @derive {Inspect, optional: [:z]}
  defstruct [x: 0, y: 0, z: 0]
end

inspect(%Point{x: 1})
%Point{x: 1, y: 0}
```

Custom implementation

You can also define your custom protocol implementation by defining the `inspect/2` function. The function receives the entity to be inspected followed by the inspecting options, represented by the struct `Inspect.Opts` and it must return an algebra document alongside the updated options (or, optionally, just the algebra document). Building of the algebra document is done with `Inspect.Algebra`.

Many times, inspecting a structure can be implemented in function of existing entities. For example, here is `MapSet`'s `inspect/2` implementation:

```
defimpl Inspect, for: MapSet do
  import Inspect.Algebra

  def inspect(map_set, opts) do
    {doc, opts} =
to_doc_with_opts(MapSet.to_list(map_set), opts)
    {concat(["MapSet.new(", doc, ")"]), opts}
  end
end
```

First `to_doc_with_opts/2` is used to convert another data structure into its algebra document and then `concat/1` concatenates algebra documents together.

In the example above it is concatenating the string `"MapSet.new("`, the document returned by `to_doc_with_opts/2`, and the final string `")"`. Therefore, the MapSet with the numbers 1, 2, and 3 will be printed as:

```
iex> MapSet.new([1, 2, 3], fn x -> x * 2 end)
MapSet.new([2, 4, 6])
```

In other words, `MapSet`'s inspect representation returns an expression that, when evaluated, builds the `MapSet` itself.

Error handling

In case there is an error while your structure is being inspected, Elixir will raise an `ArgumentError` error and will automatically fall back to a raw representation for printing the structure. Furthermore, you must be careful when debugging your own Inspect implementation, as calls to `IO.inspect/2` or `dbg/1` may trigger an infinite loop (as in order to inspect/debug the data structure, you must call `inspect` itself).

Here are some tips:

- For debugging, use `IO.inspect/2` with the `structs: false` option, which disables custom printing and avoids calling the Inspect implementation recursively
- To access the underlying error on your custom `Inspect` implementation, you may invoke the protocol directly. For example, we could invoke the `Inspect.MapSet` implementation above as:

```
Inspect.MapSet.inspect(MapSet.new(), %Inspect.Opts{})
```

  Note that, from Elixir v1.19, the inspect protocol was augmented to allow a two-element tuple with the document and the updated options to be returned from the protocol.

Summary

Types

t()

All the types that implement this protocol.

Functions

inspect(term, opts)

Converts `term` into an algebra document.

Types

t()

```
@type t() :: term()
```

All the types that implement this protocol.

Functions

inspect(term, opts)

```
@spec inspect(t(), Inspect.Opts.t()) ::
  Inspect.Algebra.t() | {Inspect.Algebra.t(),
Inspect.Opts.t()}
```

Converts `term` into an algebra document.

This function shouldn't be invoked directly, unless when implementing a custom `inspect_fun` to be given to `Inspect.Opts`. Everywhere else, `Inspect.Algebra.to_doc/2` should be preferred as it handles structs and exceptions.

Inspect.Algebra

A set of functions for creating and manipulating algebra documents.

This module implements the functionality described in "Strictly Pretty" (2000) by Christian Lindig with small additions, like support for binary nodes and a break mode that maximises use of horizontal space.

```
iex> Inspect.Algebra.line()
:doc_line

iex> "foo"
"foo"
```

With the functions in this module, we can concatenate different elements together and render them:

```
iex> doc =
Inspect.Algebra.concat(Inspect.Algebra.empty(), "foo")
iex> Inspect.Algebra.format(doc, 80)
"foo"
```

The functions `nest/2`, `space/2` and `line/2` help you put the document together into a rigid structure. However, the document algebra gets interesting when using functions like `glue/3` and `group/1`. A glue inserts a break between two documents. A group indicates a document that must fit the current line, otherwise breaks are rendered as new lines. Let's glue two docs together with a break, group it and then render it:

```
iex> doc = Inspect.Algebra.glue("a", " ", "b")
iex> doc = Inspect.Algebra.group(doc)
```

```
iex> Inspect.Algebra.format(doc, 80)
"a b"
```

Note that the break was represented as is, because we haven't reached a line limit. Once we do, it is replaced by a newline:

```
iex> doc = Inspect.Algebra.glue(String.duplicate("a",
20), " ", "b")
iex> doc = Inspect.Algebra.group(doc)
iex> Inspect.Algebra.format(doc, 10)
"aaaaaaaaaaaaaaaaaaaa\nb"
```

This module uses the byte size to compute how much space there is left. If your document contains strings, then those need to be wrapped in `string/1`, which then relies on `String.length/1` to precompute the document size.

Finally, this module also contains Elixir related functions, a bit tied to Elixir formatting, such as `to_doc/2`.

Implementation details

The implementation of `Inspect.Algebra` is based on the Strictly Pretty paper by Lindig which builds on top of previous pretty printing algorithms but is tailored to strict languages, such as Elixir. The core idea in the paper is the use of explicit document groups which are rendered as flat (breaks as spaces) or as break (breaks as newlines).

This implementation provides two types of breaks: `:strict` and `:flex`. When a group does not fit, all strict breaks are treated as newlines. Flex breaks, however, are re-evaluated on every occurrence and may still be rendered flat. See `break/1` and `flex_break/1` for more information.

This implementation also adds `force_unfit/1` and optimistic/pessimistic groups which give more control over the document fitting.

Summary

Guards

is_doc(doc)

Types

container_opts()

Options for container documents.

t()

Functions

break(string \\ " ")

Returns a break document based on the given `string`.

collapse_lines(max)

Collapse any new lines and whitespace following this node, emitting up to `max` new lines.

color(doc, color)

Colors a document with the given color (preceding the document itself).

color(doc, key, opts) deprecated
color_doc(doc, color_key, opts)

Colors a document if the `color_key` has a color in the options.

concat(docs)

Concatenates a list of documents returning a new document.

concat(doc1, doc2)

Concatenates two document entities returning a new document.

container_doc(left, collection, right, inspect_opts, fun, opts \\ [])

Wraps `collection` in `left` and `right` according to limit and contents and returns only the container document.

container_doc_with_opts(left, collection, right, inspect_opts, fun, opts \\ [])

Wraps `collection` in `left` and `right` according to limit and contents.

empty()

Returns a document entity used to represent nothingness.

flex_break(string \\ " ")

Returns a flex break document based on the given `string`.

flex_glue(doc1, break_string \\ " ", doc2)

Glues two documents (`doc1` and `doc2`) inserting a `flex_break/1` given by `break_string` between them.

fold(docs, folder_fun)

Folds a list of documents into a document using the given folder function.

fold_doc(docs, folder_fun) deprecated

force_unfit(doc)

Forces the current group to be unfit.

format(doc, width)

Formats a given document for a given width.

glue(doc1, break_string \\ " ", doc2)

Glues two documents (`doc1` and `doc2`) inserting the given break `break_string` between them.

group(doc, mode \\ :normal)

Returns a group containing the specified document `doc` .

line()

A mandatory linebreak.

line(doc1, doc2)

Inserts a mandatory linebreak between two documents.

nest(doc, level, mode \\ :always)

Nests the given document at the given `level` .

next_break_fits(doc, mode \\ :enabled) deprecated

Considers the next break as fit.

no_limit(doc)

Disable any rendering limit while rendering the given document.

space(doc1, doc2)

Inserts a mandatory single space between two documents.

string(string)

Creates a document represented by string.

to_doc(term, opts)

Converts an Elixir term to an algebra document according to the `Inspect` protocol.

to_doc_with_opts(term, opts)

Converts an Elixir term to an algebra document according to the `Inspect` protocol, alongside the updated options.

Guards

is_doc(doc)

(macro)

Types

container_opts()

```
@type container_opts() :: [separator: String.t(), break:
:strict | :flex | :maybe]
```

Options for container documents.

t()

```
@type t() ::
  binary()
  | doc_nil()
  | doc_cons()
  | doc_line()
  | doc_break()
  | doc_collapse()
  | doc_color()
  | doc_fits()
  | doc_force()
  | doc_group()
  | doc_nest()
  | doc_string()
  | doc_limit()
```

Functions

break(string \\ " ")

```
@spec break(binary()) :: doc_break()
```

Returns a break document based on the given `string`.

This break can be rendered as a linebreak or as the given `string`, depending on the `mode` of the chosen layout.

Examples

Let's create a document by concatenating two strings with a break between them:

```
iex> doc = Inspect.Algebra.concat(["a",
Inspect.Algebra.break("\t"), "b"])
```

```
iex> Inspect.Algebra.format(doc, 80)
"a\tb"
```

Note that the break was represented with the given string, because we didn't reach a line limit. Once we do, it is replaced by a newline:

```
iex> break = Inspect.Algebra.break("\t")
iex> doc = Inspect.Algebra.concat([String.duplicate("a",
20), break, "b"])
iex> doc = Inspect.Algebra.group(doc)
iex> Inspect.Algebra.format(doc, 10)
"aaaaaaaaaaaaaaaaaaaa\nb"
```

collapse_lines(max)

(since 1.6.0)

```
@spec collapse_lines(pos_integer()) :: doc_collapse()
```

Collapse any new lines and whitespace following this node, emitting up to `max` new lines.

color(doc, color)

(since 1.18.0)

```
@spec color(t(), binary()) :: t()
```

Colors a document with the given color (preceding the document itself).

color(doc, key, opts)

This function is deprecated. Use color_doc/3 instead.

color_doc(doc, color_key, opts)

(since 1.18.0)

```
@spec color_doc(t(), Inspect.Opts.color_key(),
Inspect.Opts.t()) :: t()
```

Colors a document if the `color_key` has a color in the options.

concat(docs)

```
@spec concat([t()]) :: t()
```

Concatenates a list of documents returning a new document.

Examples

```
iex> doc = Inspect.Algebra.concat(["a", "b", "c"])
iex> Inspect.Algebra.format(doc, 80)
"abc"
```

concat(doc1, doc2)

```
@spec concat(t(), t()) :: t()
```

Concatenates two document entities returning a new document.

Examples

```
iex> doc = Inspect.Algebra.concat("hello", "world")
iex> Inspect.Algebra.format(doc, 80)
"helloworld"
```

container_doc(left, collection, right, inspect_opts, fun, opts \\ [])

(since 1.6.0)

```
@spec container_doc(
  t(),
  [term()],
  t(),
  Inspect.Opts.t(),
  (term(), Inspect.Opts.t() -> t()),
  container_opts()
) :: t()
```

Wraps `collection` in `left` and `right` according to limit and contents and returns only the container document.

In practice, one must prefer to use `container_doc_with_opts/6` over this function, as `container_doc_with_opts/6` returns the updated options from inspection.

container_doc_with_opts(left, collection, right, inspect_opts, fun, opts \\ [])

(since 1.19.0)

```
@spec container_doc_with_opts(
  t(),
  [term()],
  t(),
  Inspect.Opts.t(),
  (term(), Inspect.Opts.t() -> t()),
  container_opts()
) :: {t(), Inspect.Opts.t()}
```

Wraps `collection` in `left` and `right` according to limit and contents.

It uses the given `left` and `right` documents as surrounding and the separator document `separator` to separate items in `docs`. If all entries in the collection are simple documents (texts or strings), then this function attempts to put as much as possible on the same line. If they are not simple, only one entry is shown per line if they do not fit.

The limit in the given `inspect_opts` is respected and when reached this function stops processing and outputs `"..."` instead.

It returns a tuple with the algebra document and the updated options.

Options

- `:separator` - the separator used between each doc
- `:break` - If `:strict`, always break between each element. If `:flex`, breaks only when necessary. If `:maybe`, chooses `:flex` only if all elements are text-based, otherwise is `:strict`

Examples

```
iex> inspect_opts = %Inspect.Opts{limit: :infinity}
iex> fun = fn i, _opts -> to_string(i) end
iex> {doc, _opts} =
Inspect.Algebra.container_doc_with_opts("[",
Enum.to_list(1..5), "]", inspect_opts, fun)
iex> Inspect.Algebra.format(doc, 5) |>
IO.iodata_to_binary()
"[1,\n 2,\n 3,\n 4,\n 5]"

iex> inspect_opts = %Inspect.Opts{limit: 3}
iex> fun = fn i, _opts -> to_string(i) end
iex> {doc, _opts} =
Inspect.Algebra.container_doc_with_opts("[",
Enum.to_list(1..5), "]", inspect_opts, fun)
iex> Inspect.Algebra.format(doc, 20) |>
IO.iodata_to_binary()
"[1, 2, 3, ...]"

iex> inspect_opts = %Inspect.Opts{limit: 3}
iex> fun = fn i, _opts -> to_string(i) end
iex> opts = [separator: "!"]
iex> {doc, _opts} =
Inspect.Algebra.container_doc_with_opts("[",
Enum.to_list(1..5), "]", inspect_opts, fun, opts)
iex> Inspect.Algebra.format(doc, 20) |>
IO.iodata_to_binary()
"[1! 2! 3! ...]"
```

empty()

```
@spec empty() :: doc_nil()
```

Returns a document entity used to represent nothingness.

Examples

```
iex> Inspect.Algebra.empty()
[]
```

flex_break(string \\ " ")

(since 1.6.0)

```
@spec flex_break(binary()) :: doc_break()
```

Returns a flex break document based on the given `string`.

A flex break still causes a group to break, like `break/1`, but it is re-evaluated when the documented is rendered.

For example, take a group document represented as `[1, 2, 3]` where the space after every comma is a break. When the document above does not fit a single line, all breaks are enabled, causing the document to be rendered as:

```
[1,
 2,
 3]
```

However, if flex breaks are used, then each break is re-evaluated when rendered, so the document could be possible rendered as:

```
[1, 2,
 3]
```

Hence the name "flex". they are more flexible when it comes to the document fitting. On the other hand, they are more expensive since each break needs to be re-evaluated.

This function is used by `container_doc/6` and friends to the maximum number of entries on the same line.

flex_glue(doc1, break_string \\ " ", doc2)

(since 1.6.0)

```
@spec flex_glue(t(), binary(), t()) :: t()
```

Glues two documents (`doc1` and `doc2`) inserting a `flex_break/1` given by `break_string` between them.

This function is used by `container_doc/6` and friends to the maximum number of entries on the same line.

fold(docs, folder_fun)

(since 1.18.0)

```
@spec fold([t()], (t(), t() -> t())) :: t()
```

Folds a list of documents into a document using the given folder function.

The list of documents is folded "from the right"; in that, this function is similar to `List.foldr/3`, except that it doesn't expect an initial accumulator and uses the last element of `docs` as the initial accumulator.

Examples

```
iex> docs = ["A", "B", "C"]
iex> docs =
...>   Inspect.Algebra.fold(docs, fn doc, acc ->
...>     Inspect.Algebra.concat([doc, "!", acc])
...>   end)
iex> Inspect.Algebra.format(docs, 80)
"A!B!C"
```

fold_doc(docs, folder_fun)

This function is deprecated. Use fold/2 instead.

force_unfit(doc)

(since 1.6.0)

```
@spec force_unfit(t()) :: doc_force()
```

Forces the current group to be unfit.

format(doc, width)

```
@spec format(t(), non_neg_integer() | :infinity) :: iodata()
```

Formats a given document for a given width.

Takes the maximum width and a document to print as its arguments and returns an IO data representation of the best layout for the document to

fit in the given width.

The document starts flat (without breaks) until a group is found.

Examples

```
iex> doc = Inspect.Algebra.glue("hello", " ", "world")
iex> doc = Inspect.Algebra.group(doc)
iex> doc |> Inspect.Algebra.format(30) |>
IO.iodata_to_binary()
"hello world"
iex> doc |> Inspect.Algebra.format(10) |>
IO.iodata_to_binary()
"hello\nworld"
```

glue(doc1, break_string \\ " ", doc2)

```
@spec glue(t(), binary(), t()) :: t()
```

Glues two documents (`doc1` and `doc2`) inserting the given break `break_string` between them.

For more information on how the break is inserted, see `break/1` .

Examples

```
iex> doc = Inspect.Algebra.glue("hello", "world")
iex> Inspect.Algebra.format(doc, 80)
"hello world"

iex> doc = Inspect.Algebra.glue("hello", "\t", "world")
```

```
iex> Inspect.Algebra.format(doc, 80)
"hello\tworld"
```

group(doc, mode \\ :normal)

```
@spec group(t(), :normal | :optimistic | :pessimistic) ::
doc_group()
```

Returns a group containing the specified document `doc`.

Documents in a group are attempted to be rendered together to the best of the renderer ability. If there are `break/1`s in the group and the group does not fit the given width, the breaks are converted into lines. Otherwise the breaks are rendered as text based on their string contents.

There are three types of groups, described next.

Group modes

- `:normal` - the group fits if it fits within the given width
- `:optimistic` - the group fits if it fits within the given width. However, when nested within another group, the parent group will assume this group fits as long as it has a single break, even if the optimistic group has a `force_unfit/1` document within it. Overall, this has an effect similar to swapping the order groups break. For example, if you have a `parent_group(child_group)` and they do not fit, the parent converts breaks into newlines first, allowing the child to compute if it fits. However, if the child group is optimistic and it has breaks, then the parent assumes it fits, leaving the overall fitting decision to the child

- `:pessimistic` - the group fits if it fits within the given width. However it disables any optimistic group within it

Examples

```
iex> doc =
...>   Inspect.Algebra.group(
...>     Inspect.Algebra.concat(
...>       Inspect.Algebra.group(
...>         Inspect.Algebra.concat(
...>           "Hello,",
...>           Inspect.Algebra.concat(
...>             Inspect.Algebra.break(),
...>             "A"
...>           )
...>         )
...>       ),
...>       Inspect.Algebra.concat(
...>         Inspect.Algebra.break(),
...>         "B"
...>       )
...>     )
...>   )
iex> Inspect.Algebra.format(doc, 80)
"Hello, A B"
iex> Inspect.Algebra.format(doc, 6)
"Hello,\nA\nB"
```

Mode examples

The different groups modes are used by Elixir's code formatter to avoid breaking code at some specific locations. For example, consider this code:

```
some_function_call(%{..., key: value, ...})
```

Now imagine that this code does not fit its line. The code formatter introduces breaks inside `(` and `)` and inside `%{` and `}`, each within their own group. Therefore the document would break as:

```
some_function_call(
  %{
    ...,
    key: value,
    ...
  }
)
```

To address this, the formatter marks the inner group as optimistic. This means the first group, which is `(...)` will consider the document fits and avoids adding breaks around the parens. So overall the code is formatted as:

```
some_function_call(%{
  ...,
  key: value,
  ...
})
```

line()

(since 1.6.0)

```
@spec line() :: t()
```

A mandatory linebreak.

A group with linebreaks will fit if all lines in the group fit.

Examples

```
iex> doc =
...>   Inspect.Algebra.concat(
...>     Inspect.Algebra.concat(
...>       "Hughes",
...>       Inspect.Algebra.line()
...>     ),
...>     "Wadler"
...>   )
iex> Inspect.Algebra.format(doc, 80)
"Hughes\nWadler"
```

line(doc1, doc2)

```
@spec line(t(), t()) :: t()
```

Inserts a mandatory linebreak between two documents.

See `line/0`.

Examples

```
iex> doc = Inspect.Algebra.line("Hughes", "Wadler")
iex> Inspect.Algebra.format(doc, 80)
"Hughes\nWadler"
```

nest(doc, level, mode \\ :always)

```
@spec nest(t(), non_neg_integer() | :cursor | :reset, :always
| :break) ::
  doc_nest() | t()
```

Nests the given document at the given `level`.

If `level` is an integer, that's the indentation appended to line breaks whenever they occur. If the level is `:cursor`, the current position of the "cursor" in the document becomes the nesting. If the level is `:reset`, it is set back to 0.

`mode` can be `:always`, which means nesting always happen, or `:break`, which means nesting only happens inside a group that has been broken.

Examples

```
iex> doc =
Inspect.Algebra.nest(Inspect.Algebra.glue("hello",
"world"), 5)
iex> doc = Inspect.Algebra.group(doc)
iex> Inspect.Algebra.format(doc, 5)
"hello\n     world"
```

next_break_fits(doc, mode \\ :enabled)

This function is deprecated. Pass the optimistic/pessimistic type to

group/2 instead.

```
@spec next_break_fits(t(), :enabled | :disabled) ::
doc_fits()
```

Considers the next break as fit.

no_limit(doc)

(since 1.14.0)

```
@spec no_limit(t()) :: t()
```

Disable any rendering limit while rendering the given document.

Examples

```
iex> doc = Inspect.Algebra.glue("hello", "world") |>
Inspect.Algebra.group()
iex> Inspect.Algebra.format(doc, 10)
"hello\nworld"
iex> doc = Inspect.Algebra.no_limit(doc)
iex> Inspect.Algebra.format(doc, 10)
"hello world"
```

space(doc1, doc2)

```
@spec space(t(), t()) :: t()
```

Inserts a mandatory single space between two documents.

Examples

```
iex> doc = Inspect.Algebra.space("Hughes", "Wadler")
iex> Inspect.Algebra.format(doc, 5)
"Hughes Wadler"
```

string(string)

(since 1.6.0)

```
@spec string(String.t()) :: doc_string()
```

Creates a document represented by string.

While `Inspect.Algebra` accepts binaries as documents, those are counted by binary size. On the other hand, `string` documents are measured in terms of graphemes towards the document size.

Examples

The following document has 10 bytes and therefore it does not format to width 9 without breaks:

```
iex> doc = Inspect.Algebra.glue("olá", " ", "mundo")
iex> doc = Inspect.Algebra.group(doc)
iex> Inspect.Algebra.format(doc, 9)
"olá\nmundo"
```

However, if we use `string`, then the string length is used, instead of byte size, correctly fitting:

```
iex> string = Inspect.Algebra.string("olá")
iex> doc = Inspect.Algebra.glue(string, " ", "mundo")
iex> doc = Inspect.Algebra.group(doc)
iex> Inspect.Algebra.format(doc, 9)
"olá mundo"
```

to_doc(term, opts)

```
@spec to_doc(any(), Inspect.Opts.t()) :: t()
```

Converts an Elixir term to an algebra document according to the `Inspect` protocol.

In practice, one must prefer to use `to_doc_with_opts/2` over this function, as `to_doc_with_opts/2` returns the updated options from inspection.

to_doc_with_opts(term, opts)

(since 1.19.0)

```
@spec to_doc_with_opts(any(), Inspect.Opts.t()) :: {t(),
Inspect.Opts.t()}
```

Converts an Elixir term to an algebra document according to the `Inspect` protocol, alongside the updated options.

This function is used when implementing the inspect protocol for a given type and you must convert nested terms to documents too.

Inspect.Opts

Defines the options used by the `Inspect` protocol.

The following fields are available:

- `:base` - prints integers and binaries as `:binary`, `:octal`, `:decimal`, or `:hex`. Defaults to `:decimal`.

- `:binaries` - when `:as_binaries` all binaries will be printed in bit syntax.

  When `:as_strings` all binaries will be printed as strings, non-printable bytes will be escaped.

  When the default `:infer`, the binary will be printed as a string if `:base` is `:decimal` and if it is printable, otherwise in bit syntax. See `String.printable?/1` to learn when a string is printable.

- `:charlists` - when `:as_charlists` all charlists will be printed as charlists, non-printable code points will be escaped. Other lists will be printed as lists.

  When `:as_lists` all lists will be printed as lists.

  When the default `:infer`, the list will be printed as a charlist if it is printable, otherwise as list. See `List.ascii_printable?/1` to learn when a charlist is printable.

- `:custom_options` (since v1.9.0) - a keyword list storing custom user-defined options. Useful when implementing the `Inspect` protocol for nested structs to pass the custom options through.

It supports some pre-defined keys:

  - `:sort_maps` (since v1.14.4) - if set to `true`, sorts key-value pairs in maps. This can be helpful to make map inspection deterministic for testing, given maps key order is random.

- `:inspect_fun` (since v1.9.0) - a function to build algebra documents. Defaults to `Inspect.Opts.default_inspect_fun/0`.

- `:limit` - limits the number of items that are inspected for tuples, bitstrings, maps, lists and any other collection of items, with the exception of printable strings and printable charlists which use the `:printable_limit` option. It accepts a positive integer or `:infinity`. It defaults to `200` since `Elixir v1.20.0`, as it has better defaults to deal with nested collections.

- `:pretty` - if set to `true` enables pretty printing. Defaults to `false`.

- `:printable_limit` - limits the number of characters that are inspected on printable strings and printable charlists. You can use `String.printable?/1` and `List.ascii_printable?/1` to check if a given string or charlist is printable. If you don't want to limit the number of characters to a particular number, use `:infinity`. It accepts a positive integer or `:infinity`. Defaults to `4096`.

- `:safe` - when `false`, failures while inspecting structs will be raised as errors instead of being wrapped in the `Inspect.Error` exception. This is useful when debugging failures and crashes for custom inspect implementations. Defaults to `true`.

- `:structs` - when `false`, structs are not formatted by the inspect protocol, they are instead printed as maps. Defaults to `true`.

- `:syntax_colors` - when set to a keyword list of colors the output is colorized. The keys are types and the values are the colors to use for each type (for example, `[number: :red, atom: :blue]`). Types can include `:atom`, `:binary`, `:boolean`, `:list`, `:map`, `:number`, `:regex`, `:string`, `:tuple`, or some types to represent AST like `:variable`, `:call`, and `:operator`. Custom data types may provide their own options. Colors can be any `IO.ANSI.ansidata/0` as accepted by `IO.ANSI.format/1`. A default list of colors can be retrieved from `IO.ANSI.syntax_colors/0`.

- `:width` - number of characters per line used when pretty is `true` or when printing to IO devices. Set to `0` to force each item to be printed on its own line. If you don't want to limit the number of items to a particular number, use `:infinity`. Defaults to `80`.

Summary

Types

color_key()
new_opt()

Options for building an `Inspect.Opts` struct with `new/1`.

t()

Functions

default_inspect_fun()

Returns the default inspect function.

default_inspect_fun(fun)

Sets the default inspect function.
new(opts)

Builds an `Inspect.Opts` struct.

Types

color_key()

```
@type color_key() :: atom()
```

new_opt()

```
@type new_opt() ::
  {:base, :decimal | :binary | :hex | :octal}
  | {:binaries, :infer | :as_binaries | :as_strings}
  | {:charlists, :infer | :as_lists | :as_charlists}
  | {:custom_options, keyword()}
  | {:inspect_fun, (any(), t() -> Inspect.Algebra.t())}
  | {:limit, non_neg_integer() | :infinity}
  | {:pretty, boolean()}
  | {:printable_limit, non_neg_integer() | :infinity}
  | {:safe, boolean()}
  | {:structs, boolean()}
  | {:syntax_colors, [{color_key(), IO.ANSI.ansidata()}]}
  | {:width, non_neg_integer() | :infinity}
```

Options for building an `Inspect.Opts` struct with `new/1`.

t()

```
@type t() :: %Inspect.Opts{
  base: :decimal | :binary | :hex | :octal,
  binaries: :infer | :as_binaries | :as_strings,
  char_lists: term(),
  charlists: :infer | :as_lists | :as_charlists,
  custom_options: keyword(),
  inspect_fun: (any(), t() -> Inspect.Algebra.t()),
  limit: non_neg_integer() | :infinity,
  pretty: boolean(),
  printable_limit: non_neg_integer() | :infinity,
  safe: boolean(),
  structs: boolean(),
  syntax_colors: [{color_key(), IO.ANSI.ansidata()}],
  width: non_neg_integer() | :infinity
}
```

Functions

default_inspect_fun()

(since 1.13.0)

```
@spec default_inspect_fun() :: (term(), t() ->
Inspect.Algebra.t())
```

Returns the default inspect function.

default_inspect_fun(fun)

(since 1.13.0)

```
@spec default_inspect_fun((term(), t() ->
Inspect.Algebra.t())) :: :ok
```

Sets the default inspect function.

Set this option with care as it will change how all values in the system are inspected. The main use of this functionality is to provide an entry point to filter inspected values, in order for entities to comply with rules and legislations on data security and data privacy.

It is **extremely discouraged** for libraries to set their own function as this must be controlled by applications. Libraries should instead define their own structs with custom inspect implementations. If a library must change the default inspect function, then it is best to ask users of your library to explicitly call `default_inspect_fun/1` with your function of choice.

The default is `Inspect.inspect/2`.

Examples

```
previous_fun = Inspect.Opts.default_inspect_fun()

Inspect.Opts.default_inspect_fun(fn
  %{address: _} = map, opts ->
    previous_fun.(%{map | address: "[REDACTED]"}, opts)

  value, opts ->
    previous_fun.(value, opts)
end)
```

new(opts)

(since 1.13.0)

```
@spec new([new opt()]) :: t()
```

Builds an `Inspect.Opts` struct.

JSON.Encoder protocol

A protocol for custom JSON encoding of data structures.

If you have a struct, you can derive the implementation of this protocol by specifying which fields should be encoded to JSON:

```
@derive {JSON.Encoder, only: [...]}
defstruct ...
```

Additionally, you can exclude specific fields using the `:except` option or encode all fields by omitting both options entirely, but these should be used with caution:

```
@derive {JSON.Encoder, except: [...]}
defstruct ...

@derive JSON.Encoder
defstruct ...
```

> **Leaking Private Information**
>
> Prefer using `:only` to avoid accidentally leaking private information when new fields are added. Other approaches should be used with auction.

You can also use `Protocol.derive/3` if you don't own the struct that you want to encode to JSON:

```
Protocol.derive(JSON.Encoder, NameOfTheStruct, only:
[...])
```

```
Protocol.derive(JSON.Encoder, NameOfTheStruct, except:
[...])
Protocol.derive(JSON.Encoder, NameOfTheStruct)
```

Summary

Types

t()

All the types that implement this protocol.

Functions

encode(term, encoder)

A function invoked to encode the given term to `iodata/0`.

Types

t()

```
@type t() :: term()
```

All the types that implement this protocol.

Functions

encode(term, encoder)

A function invoked to encode the given term to `iodata/0`.

List.Chars protocol

The `List.Chars` protocol is responsible for converting a structure to a charlist (only if applicable).

The only function that must be implemented is `to_charlist/1` which does the conversion.

The `to_charlist/1` function automatically imported by `Kernel` invokes this protocol.

Summary

Types

t()

All the types that implement this protocol.

Functions

to_charlist(term)

Converts `term` to a charlist.

Types

t()

```
@type t() :: term()
```

All the types that implement this protocol.

Functions

to_charlist(term)

```
@spec to_charlist(t()) :: charlist()
```

Converts `term` to a charlist.

Protocol behaviour

Reference and functions for working with protocols.

A protocol specifies an API that should be defined by its implementations. A protocol is defined with `Kernel.defprotocol/2` and its implementations with `Kernel.defimpl/3`.

Example

In Elixir, we have two nouns for checking how many items there are in a data structure: `length` and `size`. `length` means the information must be computed. For example, `length(list)` needs to traverse the whole list to calculate its length. On the other hand, `tuple_size(tuple)` and `byte_size(binary)` do not depend on the tuple and binary size as the size information is precomputed in the data structure.

Although Elixir includes specific functions such as `tuple_size`, `byte_size` and `map_size`, sometimes we want to be able to retrieve the size of a data structure regardless of its type. In Elixir we can write polymorphic code, i.e. code that works with different shapes/types, by using protocols. A size protocol could be implemented as follows:

```
defprotocol Size do
  @doc "Calculates the size (and not the length!) of a
data structure"
  def size(data)
end
```

Now that the protocol can be implemented for every data structure the protocol may have a compliant implementation for:

```elixir
defimpl Size, for: BitString do
  def size(binary), do: byte_size(binary)
end

defimpl Size, for: Map do
  def size(map), do: map_size(map)
end

defimpl Size, for: Tuple do
  def size(tuple), do: tuple_size(tuple)
end
```

Finally, we can use the `Size` protocol to call the correct implementation:

```elixir
Size.size({1, 2})
# => 2
Size.size(%{key: :value})
# => 1
```

Note that we didn't implement it for lists as we don't have the `size` information on lists, rather its value needs to be computed with `length`.

The data structure you are implementing the protocol for must be the first argument to all functions defined in the protocol.

It is possible to implement protocols for all Elixir types:

- Structs (see the "Protocols and Structs" section below)
- `Tuple`
- `Atom`
- `List`
- `BitString`
- `Integer`

- Float
- Function
- PID
- Map
- Port
- Reference
- Any (see the "Fallback to Any" section below)

Protocols and Structs

The real benefit of protocols comes when mixed with structs. For instance, Elixir ships with many data types implemented as structs, like MapSet. We can implement the Size protocol for those types as well:

```
defimpl Size, for: MapSet do
  def size(map_set), do: MapSet.size(map_set)
end
```

When implementing a protocol for a struct, the :for option can be omitted if the defimpl/3 call is inside the module that defines the struct:

```
defmodule User do
  defstruct [:email, :name]

  defimpl Size do
    # two fields
    def size(%User{}), do: 2
  end
end
```

If a protocol implementation is not found for a given type, invoking the protocol will raise unless it is configured to fall back to `Any`. Conveniences for building implementations on top of existing ones are also available, look at `defstruct/1` for more information about deriving protocols.

Fallback to `Any`

In some cases, it may be convenient to provide a default implementation for all types. This can be achieved by setting the `@fallback_to_any` attribute to `true` in the protocol definition:

```
defprotocol Size do
  @fallback_to_any true
  def size(data)
end
```

The `Size` protocol can now be implemented for `Any`:

```
defimpl Size, for: Any do
  def size(_), do: 0
end
```

Although the implementation above is arguably not a reasonable one. For example, it makes no sense to say a PID or an integer have a size of `0`. That's one of the reasons why `@fallback_to_any` is an opt-in behavior. For the majority of protocols, raising an error when a protocol is not implemented is the proper behavior.

Multiple implementations

Protocols can also be implemented for multiple types at once:

```elixir
defprotocol Reversible do
  def reverse(term)
end

defimpl Reversible, for: [Map, List] do
  def reverse(term), do: Enum.reverse(term)
end
```

Inside `defimpl/3`, you can use `@protocol` to access the protocol being implemented and `@for` to access the module it is being defined for.

Types

Defining a protocol automatically defines a zero-arity type named `t`, which can be used as follows:

```elixir
@spec print_size(Size.t()) :: :ok
def print_size(data) do
  result =
    case Size.size(data) do
      0 -> "data has no items"
      1 -> "data has one item"
      n -> "data has #{n} items"
    end

  IO.puts(result)
end
```

The `@spec` above expresses that all types allowed to implement the given protocol are valid argument types for the given function.

Configuration

The following module attributes are available to configure a protocol:

- `@fallback_to_any` - when true, enables protocol dispatch to fallback to any
- `@undefined_impl_description` - a string with additional description to be used on `Protocol.UndefinedError` when looking up the implementation fails. This option is only applied if `@fallback_to_any` is not set to true

Consolidation

In order to speed up protocol dispatching, whenever all protocol implementations are known up-front, typically after all Elixir code in a project is compiled, Elixir provides a feature called *protocol consolidation*. Consolidation directly links protocols to their implementations in a way that invoking a function from a consolidated protocol is equivalent to invoking two remote functions - one to identify the correct implementation, and another to call the implementation.

Protocol consolidation is applied by default to all Mix projects during compilation. This may be an issue during test. For instance, if you want to implement a protocol during test, the implementation will have no effect, as the protocol has already been consolidated. One possible solution is to include compilation directories that are specific to your test environment in your mix.exs:

```
def project do
  ...
  elixirc_paths: elixirc_paths(Mix.env())
  ...
end
```

```
defp elixirc_paths(:test), do: ["lib", "test/support"]
defp elixirc_paths(_), do: ["lib"]
```

And then you can define the implementations specific to the test environment inside `test/support/some_file.ex`.

Another approach is to disable protocol consolidation during tests in your mix.exs:

```
def project do
  ...
  consolidate_protocols: Mix.env() != :test
  ...
end
```

If you are using `Mix.install/2`, you can do by passing the `consolidate_protocols` option:

```
Mix.install(
  deps,
  consolidate_protocols: false
)
```

Although doing so is not recommended as it may affect the performance of your code.

Finally, note all protocols are compiled with `debug_info` set to `true`, regardless of the option set by the `elixirc` compiler. The debug info is used for consolidation and it is removed after consolidation unless globally set.

Summary

Callbacks

__deriving__(module, term)

An optional callback to be implemented by protocol authors for custom deriving.

__protocol__(atom)

A function available in all protocol definitions that returns protocol metadata.

impl_for(term)

A function available in all protocol definitions that returns the implementation for the given `term` or nil.

impl_for!(term)

A function available in all protocol definitions that returns the implementation for the given `term` or raises.

Functions

assert_impl!(protocol, base)

Checks if the given module is loaded and is an implementation of the given protocol.

assert_protocol!(module)

Checks if the given module is loaded and is protocol.

consolidate(protocol, types)

Receives a protocol and a list of implementations and consolidates the given protocol.

consolidated?(protocol)

Returns `true` if the protocol was consolidated.

derive(protocol, module, options \\ [])

Derives the `protocol` for `module` with the given options.

extract_impls(protocol, paths)

Extracts all types implemented for the given protocol from the given paths.

extract_protocols(paths)

Extracts all protocols from the given paths.

Callbacks

__deriving__(module, term)

(optional)

```
@macrocallback __deriving__(module(), term()) :: Macro.t()
```

An optional callback to be implemented by protocol authors for custom deriving.

It must return a quoted expression that implements the protocol for the given module.

See `Protocol.derive/3` for an example.

__protocol__(atom)

```
@callback __protocol__(:consolidated?) :: boolean()
@callback __protocol__(:functions) :: [{atom(), arity()}]
@callback __protocol__(:impls) :: {:consolidated, [module()]}
| :not_consolidated
@callback __protocol__(:module) :: module()
```

A function available in all protocol definitions that returns protocol metadata.

impl_for(term)

```
@callback impl_for(term()) :: module() | nil
```

A function available in all protocol definitions that returns the implementation for the given `term` or nil.

If `@fallback_to_any` is true, `nil` is never returned.

impl_for!(term)

```
@callback impl_for!(term()) :: module()
```

A function available in all protocol definitions that returns the implementation for the given `term` or raises.

If `@fallback_to_any` is true, it never raises.

Functions

assert_impl!(protocol, base)

```
@spec assert_impl!(module(), module()) :: :ok
```

Checks if the given module is loaded and is an implementation of the given protocol.

Returns `:ok` if so, otherwise raises `ArgumentError`.

assert_protocol!(module)

```
@spec assert_protocol!(module()) :: :ok
```

Checks if the given module is loaded and is protocol.

Returns `:ok` if so, otherwise raises `ArgumentError`.

consolidate(protocol, types)

```
@spec consolidate(module(), [module()]) ::
  {:ok, binary()} | {:error, :not_a_protocol} | {:error,
:no_beam_info}
```

Receives a protocol and a list of implementations and consolidates the given protocol.

Consolidation happens by changing the protocol `impl_for` in the abstract format to have fast lookup rules. Usually the list of implementations to use during consolidation are retrieved with the help of `extract_impls/2`.

It returns the updated version of the protocol bytecode. If the first element of the tuple is `:ok`, it means the protocol was consolidated.

A given bytecode or protocol implementation can be checked to be consolidated or not by analyzing the protocol attribute:

```
Protocol.consolidated?(Enumerable)
```

This function does not load the protocol at any point nor loads the new bytecode for the compiled module. However, each implementation must be available and it will be loaded.

consolidated?(protocol)

```
@spec consolidated?(module()) :: boolean()
```

Returns `true` if the protocol was consolidated.

derive(protocol, module, options \\ [])

(macro)

Derives the `protocol` for `module` with the given options.

Every time you derive a protocol, Elixir will verify if the protocol has implemented the `Protocol.__deriving__/2` callback. If so, the callback will be invoked and it should define the implementation

module. Otherwise an implementation that simply points to the `Any` implementation is automatically derived.

Examples

```
defprotocol Derivable do
  @impl true
  defmacro __deriving__(module, options) do
    # If you need to load struct metadata, you may call:
    # struct_info = Macro.struct_info!(module,
__CALLER__)

    quote do
      defimpl Derivable, for: unquote(module) do
        def ok(arg) do
          {:ok, arg, unquote(options)}
        end
      end
    end
  end

  def ok(arg)
end
```

Once the protocol is defined, there are two ways it can be derived. The first is by using the `@derive` module attribute by the time you define the struct:

```
defmodule ImplStruct do
  @derive [Derivable]
  defstruct a: 0, b: 0
end
```

```
Derivable.ok(%ImplStruct{})
#=> {:ok, %ImplStruct{a: 0, b: 0}, []}
```

If the struct has already been defined, you can call this macro:

```
require Protocol
Protocol.derive(Derivable, ImplStruct, :oops)
Derivable.ok(%ImplStruct{a: 1, b: 1})
#=> {:ok, %ImplStruct{a: 1, b: 1}, :oops}
```

extract_impls(protocol, paths)

```
@spec extract_impls(module(), [charlist() | String.t() |
{charlist(), [charlist()]}]) ::
  [atom()]
```

Extracts all types implemented for the given protocol from the given paths.

The paths can be either a charlist or a string. Internally they are worked on as charlists, so passing them as lists avoid extra conversion.

Does not load any of the implementations.

Examples

```
# Get Elixir's ebin directory path and retrieve all
protocols
iex> path = Application.app_dir(:elixir, "ebin")
iex> mods = Protocol.extract_impls(Enumerable, [path])
```

```
iex> List in mods
true
```

extract_protocols(paths)

```
@spec extract_protocols([charlist() | String.t() |
{charlist(), [charlist()]}]) :: [
  atom()
]
```

Extracts all protocols from the given paths.

The paths can be either a charlist or a string. Internally they are worked on as charlists, so passing them as lists avoid extra conversion.

Does not load any of the protocols.

Examples

```
# Get Elixir's ebin directory path and retrieve all
protocols
iex> path = Application.app_dir(:elixir, "ebin")
iex> mods = Protocol.extract_protocols([path])
iex> Enumerable in mods
true
```

String.Chars protocol

The `String.Chars` protocol is responsible for converting a structure to a binary (only if applicable).

The only function required to be implemented is `to_string/1`, which does the conversion.

The `to_string/1` function automatically imported by `Kernel` invokes this protocol. String interpolation also invokes `to_string/1` in its arguments. For example, `"foo#{bar}"` is the same as `"foo" <> to_string(bar)`.

Summary

Types

t()

All the types that implement this protocol.

Functions

to_string(term)

Converts `term` to a string.

Types

t()

```
@type t() :: term()
```

All the types that implement this protocol.

Functions

to_string(term)

```
@spec to_string(t()) :: String.t()
```

Converts `term` to a string.

Code

Utilities for managing code compilation, code evaluation, and code loading.

This module complements Erlang's `:code` module to add behavior which is specific to Elixir. For functions to manipulate Elixir's AST (rather than evaluating it), see the `Macro` module.

Working with files

This module contains three functions for compiling and evaluating files. Here is a summary of them and their behavior:

- `require_file/2` - compiles a file and tracks its name. It does not compile the file again if it has been previously required.
- `compile_file/2` - compiles a file without tracking its name. Compiles the file multiple times when invoked multiple times.
- `eval_file/2` - evaluates the file contents without tracking its name. It returns the result of the last expression in the file, instead of the modules defined in it. Evaluated files do not trigger the compilation tracers described in the next section.

In a nutshell, the first must be used when you want to keep track of the files handled by the system, to avoid the same file from being compiled multiple times. This is common in scripts.

`compile_file/2` must be used when you are interested in the modules defined in a file, without tracking. `eval_file/2` should be used when you are interested in the result of evaluating the file rather than the modules it defines.

The functions above work with Elixir source. If you want to work with modules compiled to bytecode, which have the `.beam` extension and are typically found below the _build directory of a Mix project, see the functions in Erlang's `:code` module.

Code loading on the Erlang VM

Erlang has two modes to load code: interactive and embedded.

By default, the Erlang VM runs in interactive mode, where modules are loaded as needed. In embedded mode the opposite happens, as all modules need to be loaded upfront or explicitly.

You can use `ensure_loaded/1` (as well as `ensure_loaded?/1` and `ensure_loaded!/1`) to check if a module is loaded before using it and act.

`ensure_compiled/1` and `ensure_compiled!/1`

Elixir also includes `ensure_compiled/1` and `ensure_compiled!/1` functions that are a superset of `ensure_loaded/1`.

Since Elixir's compilation happens in parallel, in some situations you may need to use a module that was not yet compiled, therefore it can't even be loaded.

When invoked, `ensure_compiled/1` and `ensure_compiled!/1` halt the compilation of the caller until the module becomes available. Note that the distinction between `ensure_compiled/1` and `ensure_compiled!/1` is important: if you are using `ensure_compiled!/1`, you are indicating to the compiler that you can only continue if said module is available.

If you are using `Code.ensure_compiled/1`, you are implying you may continue without the module and therefore Elixir may return `{:error,`

`:unavailable}` for cases where the module is not yet available (but may be available later on).

For those reasons, developers must typically use `Code.ensure_compiled!/1`. In particular, do not do this:

```
case Code.ensure_compiled(module) do
  {:module, _} -> module
  {:error, _} -> raise ...
end
```

Finally, note you only need `ensure_compiled!/1` to check for modules being defined within the same project. It does not apply to modules from dependencies as dependencies are always compiled upfront.

In most cases, `ensure_loaded/1` is enough. `ensure_compiled!/1` must be used in rare cases, usually involving macros that need to invoke a module for callback information. The use of `ensure_compiled/1` is even less likely.

Compilation tracers

Elixir supports compilation tracers, which allow modules to observe constructs handled by the Elixir compiler when compiling files. A tracer is a module that implements the `trace/2` function. The function receives the event name as first argument and `Macro.Env` as second and it must return `:ok`. It is very important for a tracer to do as little work as possible synchronously and dispatch the bulk of the work to a separate process. **Slow tracers will slow down compilation**.

You can configure your list of tracers via `put_compiler_option/2`. The following events are available to tracers:

- `:start` - (since v1.11.0) invoked whenever the compiler starts to trace a new lexical context. A lexical context is started when compiling a new file or when defining a module within a function. Note evaluated code does not start a new lexical context (because they don't track unused aliases, imports, etc) but defining a module inside evaluated code will.

  Note this event may be emitted in parallel, where multiple files/modules invoke `:start` and run at the same time. The value of the `lexical_tracker` of the macro environment, albeit opaque, can be used to uniquely identify the environment.

- `:stop` - (since v1.11.0) invoked whenever the compiler stops tracing a new lexical context, such as a new file.

- `{:import, meta, module, opts}` - traced whenever `module` is imported. `meta` is the import AST metadata and `opts` are the import options.

- `{:imported_function, meta, module, name, arity}` and `{:imported_macro, meta, module, name, arity}` - traced whenever an imported function or macro is invoked. `meta` is the call AST metadata, `module` is the module the import is from, followed by the `name` and `arity` of the imported function/macro. A :remote_function/:remote_macro event may still be emitted for the imported module/name/arity.

- `{:imported_quoted, meta, module, name, [arity]}` - traced whenever an imported function or macro is processed inside a `quote/2`. `meta` is the call AST metadata, `module` is the module the import is from, followed by the `name` and a list of `arities` of the imported function/macro.

- `{:alias, meta, alias, as, opts}` - traced whenever `alias` is aliased to `as`. `meta` is the alias AST metadata and `opts` are the

alias options.

- `{:alias_expansion, meta, as, alias}` traced whenever there is an alias expansion for a previously defined `alias`, i.e. when the user writes `as` which is expanded to `alias`. `meta` is the alias expansion AST metadata.

- `{:alias_reference, meta, module}` - traced whenever there is an alias in the code, i.e. whenever the user writes `MyModule.Foo.Bar` in the code, regardless if it was expanded or not.

- `{:require, meta, module, opts}` - traced whenever `module` is required. `meta` is the require AST metadata and `opts` are the require options. If the `meta` option contains the `:from_macro`, then module was called from within a macro and therefore must be treated as a compile-time dependency.

- `{:struct_expansion, meta, module, keys}` - traced whenever `module`'s struct is expanded. `meta` is the struct AST metadata and `keys` are the keys being used by expansion

- `{:remote_function, meta, module, name, arity}` and `{:remote_macro, meta, module, name, arity}` - traced whenever a remote function or macro is referenced. `meta` is the call AST metadata, `module` is the invoked module, followed by the `name` and `arity`.

- `{:local_function, meta, name, arity}` and `{:local_macro, meta, name, arity}` - traced whenever a local function or macro is referenced. `meta` is the call AST metadata, followed by the `name` and `arity`.

- `{:compile_env, app, path, return}` - traced whenever `Application.compile_env/3` or `Application.compile_env!/2` are called. `app` is an atom, `path` is a list of keys to traverse in the application environment and `return` is either `{:ok, value}` or `:error`.

- `:defmodule` - (since v1.16.2) traced as soon as the definition of a module starts. This is invoked early on in the module life cycle, `Module.open?/1` still returns `false` for such traces

- `{:on_module, bytecode, _ignore}` - (since v1.13.0) traced whenever a module is defined. This is equivalent to the `@after_compile` callback and invoked after any `@after_compile` in the given module. The third element is currently `:none` but it may provide more metadata in the future. It is best to ignore it at the moment. Note that `Module` functions expecting not yet compiled modules (such as `Module.definitions_in/1`) are still available at the time this event is emitted.

The `:tracers` compiler option can be combined with the `:parser_options` compiler option to enrich the metadata of the traced events above.

New events may be added at any time in the future, therefore it is advised for the `trace/2` function to have a "catch-all" clause.

Below is an example tracer that prints all remote function invocations:

```
defmodule MyTracer do
  def trace({:remote_function, _meta, module, name,
arity}, env) do
    IO.puts("#{env.file}:#{env.line} #{inspect(module)}.#
{name}/#{arity}")
    :ok
```

```
    end

    def trace(_event, _env) do
      :ok
    end
  end
```

Summary

Types

binding()

A list with all variables and their values.

diagnostic(severity)

Diagnostics returned by the compiler and code evaluation.

env_eval_opt()

Options for evaluation environment, accepted by `env_for_eval/1`.

eval_opt()

Options for evaluation functions like `eval_string/3`, `eval_quoted/3` and `eval_quoted_with_env/4`.

format_opt()

Options for code formatting functions.

line()

The line. 0 indicates no line.

parser_opts()

Options for parsing functions that convert strings to quoted expressions.

position()

The position of the diagnostic.

quoted_to_algebra_opt()

Options for `quoted_to_algebra/2`.

Functions

append_path(path, opts \\ [])

Appends a path to the Erlang VM code path list.

append_paths(paths, opts \\ [])

Appends a list of `paths` to the Erlang VM code path list.

available_compiler_options()

Returns a list with all available compiler options.

can_await_module_compilation?()

Returns `true` if the current process can await for module compilation.

compile_file(file, relative_to \\ nil)

Compiles the given file.

compile_quoted(quoted, file \\ "nofile")

Compiles the quoted expression.

compile_string(string, file \\ "nofile")

Compiles the given string.

compiler_options()

Gets all compilation options from the code server.

compiler_options(opts)

Stores all given compilation options.

delete_path(path)

Deletes a path from the Erlang VM code path list.

delete_paths(paths)

Deletes a list of paths from the Erlang VM code path list.

ensure_all_loaded(modules)

Ensures the given modules are loaded.

ensure_all_loaded!(modules)

Same as `ensure_all_loaded/1` but raises if any of the modules cannot be loaded.

ensure_compiled(module)

Similar to `ensure_compiled!/1` but indicates you can continue without said module.

ensure_compiled!(module)

Ensures the given module is compiled and loaded.

ensure_loaded(module)

Ensures the given module is loaded.

ensure_loaded!(module)

Same as `ensure_loaded/1` but raises if the module cannot be loaded.

ensure_loaded?(module)

Ensures the given module is loaded.

env_for_eval(env_or_opts)

Returns an environment for evaluation.

eval_file(file, relative_to \\ nil)

Evaluates the given file.

eval_quoted(quoted, binding \\ [], env_or_opts \\ [])

Evaluates the quoted contents.

eval_quoted_with_env(quoted, binding, env, opts \\ [])

Evaluates the given `quoted` contents with `binding` and `env`.

eval_string(string, binding \\ [], opts_or_env \\ [])

Evaluates the contents given by `string`.

fetch_docs(module_or_path)

Returns the docs for the given module or path to `.beam` file.

format_file!(file, opts \\ [])

Formats a file.

format_string!(string, opts \\ [])

Formats the given code `string`.

get_compiler_option(key)

Returns the value of a given compiler option.

loaded?(module)

Returns `true` if the module is loaded.

prepend_path(path, opts \\ [])

Prepends a path to the Erlang VM code path list.

prepend_paths(paths, opts \\ [])

Prepends a list of `paths` to the Erlang VM code path list.

print_diagnostic(diagnostic, opts \\ [])

Prints a diagnostic into the standard error.

purge_compiler_modules()

Purge compiler modules.

put_compiler_option(key, value)

Stores a compilation option.

quoted_to_algebra(quoted, opts \\ [])

Converts a quoted expression to an algebra document using Elixir's formatter rules.

require_file(file, relative_to \\ nil)

Requires the given `file`.

required_files()

Lists all required files.

string_to_quoted(string, opts \\ [])

Converts the given string to its quoted form.

string_to_quoted!(string, opts \\ [])

Converts the given string to its quoted form.

string_to_quoted_with_comments(string, opts \\ [])

Converts the given string to its quoted form and a list of comments.

string_to_quoted_with_comments!(string, opts \\ [])

Converts the given string to its quoted form and a list of comments.

unrequire_files(files)

Removes files from the required files list.

with_diagnostics(opts \\ [], fun)

Executes the given `fun` and capture all diagnostics.

Types

binding()

```
@type binding() :: [{atom() | tuple(), any()}]
```

A list with all variables and their values.

The binding keys are usually atoms, but they may be a tuple for variables defined in a different context.

diagnostic(severity)

```
@type diagnostic(severity) :: %{
  :source => Path.t() | nil,
  :file => Path.t() | nil,
  :severity => severity,
  :message => String.t(),
  :position => position(),
  :stacktrace => Exception.stacktrace(),
  :span => {line :: pos_integer(), column :: pos_integer()} |
nil,
  optional(:details) => term(),
  optional(any()) => any()
}
```

Diagnostics returned by the compiler and code evaluation.

The file and position relate to where the diagnostic should be shown. If there is a file and position, then the diagnostic is precise and you can use the given file and position for generating snippets, IDEs annotations, and so on. An optional span is available with the line and column the diagnostic ends.

Otherwise, a stacktrace may be given, which you can place your own heuristics to provide better reporting.

The source field points to the source file the compiler tracked the error to. For example, a file `lib/foo.ex` may embed `.eex` templates from

`lib/foo/bar.eex`. A syntax error on the EEx template will point to file `lib/foo/bar.eex` but the source is `lib/foo.ex`.

env_eval_opt()

```
@type env_eval_opt() ::
  {:file, binary()} | {:line, pos_integer()} | {:module,
module()}
```

Options for evaluation environment, accepted by `env_for_eval/1`.

eval_opt()

```
@type eval_opt() ::
  {:prune_binding, boolean()} | {:dbg_callback, {module(),
atom(), list()}}
```

Options for evaluation functions like `eval_string/3`, `eval_quoted/3` and `eval_quoted_with_env/4`.

format_opt()

```
@type format_opt() ::
  {:file, binary()}
  | {:line, pos_integer()}
  | {:line_length, pos_integer()}
  | {:locals_without_parens, keyword()}
  | {:force_do_end_blocks, boolean()}
  | {:migrate, boolean()}
```

```
  | {:migrate_bitstring_modifiers, boolean()}
  | {:migrate_call_parens_on_pipe, boolean()}
  | {:migrate_charlists_as_sigils, boolean()}
  | {:migrate_unless, boolean()}
  | {atom(), term()}
```

Options for code formatting functions.

line()

```
@type line() :: non_neg_integer()
```

The line. 0 indicates no line.

parser_opts()

```
@type parser_opts() :: [
  file: binary(),
  line: pos_integer(),
  column: pos_integer(),
  indentation: non_neg_integer(),
  columns: boolean(),
  unescape: boolean(),
  existing_atoms_only: boolean(),
  token_metadata: boolean(),
  literal_encoder: (term(), Macro.metadata() -> term()),
  static_atoms_encoder: (binary(), Macro.metadata() ->
                           {:ok, term()} | {:error,
binary()}),
  emit_warnings: boolean()
]
```

Options for parsing functions that convert strings to quoted expressions.

position()

```
@type position() :: line() | {line :: pos_integer(), column
:: pos_integer()}
```

The position of the diagnostic.

Can be either a line number or a `{line, column}`. Line and columns numbers are one-based. A position of `0` represents unknown.

quoted_to_algebra_opt()

```
@type quoted_to_algebra_opt() ::
  {:line, pos_integer() | nil}
  | {:escape, boolean()}
  | {:locals_without_parens, keyword()}
  | {:comments, [term()]}
```

Options for `quoted_to_algebra/2`.

Functions

append_path(path, opts \\ [])

```
@spec append_path(Path.t(), [{:cache, boolean()}]) :: true |
false
```

Appends a path to the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

The path is expanded with `Path.expand/1` before being appended. It requires the path to exist. Returns a boolean indicating if the path was successfully added.

Examples

```
Code.append_path(".")
#=> true

Code.append_path("/does_not_exist")
#=> false
```

Options

- `:cache` - (since v1.15.0) when true, the code path is cached the first time it is traversed in order to reduce file system operations.

append_paths(paths, opts \\ [])

(since 1.15.0)

```
@spec append_paths([Path.t()], [{:cache, boolean()}]) :: :ok
```

Appends a list of `paths` to the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

All paths are expanded with `Path.expand/1` before being appended. Only existing paths are appended. This function always returns `:ok`, regardless of how many paths were appended. Use `append_path/1` if you need more control.

Examples

```
Code.append_paths([".", "/does_not_exist"])
#=> :ok
```

Options

- `:cache` - when true, the code path is cached the first time it is traversed in order to reduce file system operations.

available_compiler_options()

```
@spec available_compiler_options() :: [atom()]
```

Returns a list with all available compiler options.

For a description of all options, see `put_compiler_option/2`.

Examples

```
Code.available_compiler_options()
#=> [:docs, :debug_info, ...]
```

can_await_module_compilation?()

(since 1.11.0)

```
@spec can_await_module_compilation?() :: boolean()
```

Returns `true` if the current process can await for module compilation.

When compiling Elixir code via `Kernel.ParallelCompiler`, which is used by Mix and `elixirc`, calling a module that has not yet been compiled will block the caller until the module becomes available. Executing Elixir scripts, such as passing a filename to `elixir`, does not await.

compile_file(file, relative_to \\ nil)

(since 1.7.0)

```
@spec compile_file(binary(), nil | binary()) :: [{module(), binary()}]
```

Compiles the given file.

Accepts `relative_to` as an argument to tell where the file is located.

Returns a list of tuples where the first element is the module name and the second one is its bytecode (as a binary). Opposite to `require_file/2`, it does not track the filename of the compiled file.

If you would like to get the result of evaluating file rather than the modules defined in it, see `eval_file/2`.

For compiling many files concurrently, see `Kernel.ParallelCompiler.compile/2`.

compile_quoted(quoted, file \\ "nofile")

```
@spec compile_quoted(Macro.t(), binary()) :: [{module(), binary()}]
```

Compiles the quoted expression.

Returns a list of tuples where the first element is the module name and the second one is its bytecode (as a binary). A `file` can be given as second argument which will be used for reporting warnings and errors.

compile_string(string, file \\ "nofile")

```
@spec compile_string(List.Chars.t(), binary()) :: [{module(), binary()}]
```

Compiles the given string.

Returns a list of tuples where the first element is the module name and the second one is its bytecode (as a binary). A `file` can be given as a second argument which will be used for reporting warnings and errors.

Warning: `string` can be any Elixir code and code can be executed with the same privileges as the Erlang VM: this means that such code could compromise the machine (for example by executing system commands). Don't use `compile_string/2` with untrusted input (such as strings coming from the network).

compiler_options()

```
@spec compiler_options() :: map()
```

Gets all compilation options from the code server.

To get individual options, see `get_compiler_option/1`. For a description of all options, see `put_compiler_option/2`.

Examples

```
Code.compiler_options()
#=> %{debug_info: true, docs: true, ...}
```

compiler_options(opts)

```
@spec compiler_options(Enumerable.t({atom(), term()})) :: %{
  optional(atom()) => term()
}
```

Stores all given compilation options.

Changing the compilation options affect all processes running in a given Erlang VM node. To store individual options and for a description of all options, see `put_compiler_option/2`.

Returns a map with previous values.

Examples

```
Code.compiler_options(infer_signatures: false)
#=> %{infer_signatures: [:elixir]}
```

delete_path(path)

```
@spec delete_path(Path.t()) :: boolean()
```

Deletes a path from the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

The path is expanded with `Path.expand/1` before being deleted. If the path does not exist, this function returns `false`.

Examples

```
Code.prepend_path(".")
Code.delete_path(".")
#=> true

Code.delete_path("/does_not_exist")
#=> false
```

delete_paths(paths)

(since 1.15.0)

```
@spec delete_paths([Path.t()]) :: :ok
```

Deletes a list of paths from the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

The path is expanded with `Path.expand/1` before being deleted. If the path does not exist, this function returns `false`.

ensure_all_loaded(modules)

(since 1.15.0)

```
@spec ensure_all_loaded([module()]) :: :ok | {:error,
[{module(), reason}]}
when reason: :badfile | :nofile | :on_load_failure
```

Ensures the given modules are loaded.

Similar to `ensure_loaded/1`, but accepts a list of modules instead of a single module, and loads all of them.

If all modules load successfully, returns `:ok`. Otherwise, returns `{:error, errors}` where `errors` is a list of tuples made of the module and the reason it failed to load.

Examples

```
iex> Code.ensure_all_loaded([Atom, String])
:ok

iex> Code.ensure_all_loaded([Atom, DoesNotExist])
{:error, [{DoesNotExist, :nofile}]}
```

ensure_all_loaded!(modules)

(since 1.15.0)

```
@spec ensure_all_loaded!([module()]) :: :ok
```

Same as `ensure_all_loaded/1` but raises if any of the modules cannot be loaded.

ensure_compiled(module)

```
@spec ensure_compiled(module()) ::
  {:module, module()}
  | {:error, :embedded | :badfile | :nofile |
:on_load_failure | :unavailable}
```

Similar to `ensure_compiled!/1` but indicates you can continue without said module.

While `ensure_compiled!/1` indicates to the Elixir compiler you can only continue when said module is available, this function indicates you may continue compilation without said module.

If it succeeds in loading the module, it returns `{:module, module}`. If not, returns `{:error, reason}` with the error reason. If the module being checked is currently in a compiler deadlock, this function returns `{:error, :unavailable}`. Unavailable doesn't necessarily mean the module doesn't exist, just that it is not currently available, but it (or may not) become available in the future.

Therefore, if you can only continue if the module is available, use `ensure_compiled!/1` instead. In particular, do not do this:

```
case Code.ensure_compiled(module) do
  {:module, _} -> module
  {:error, _} -> raise ...
end
```

See the module documentation for more information on code loading.

ensure_compiled!(module)

(since 1.12.0)

```
@spec ensure_compiled!(module()) :: module()
```

Ensures the given module is compiled and loaded.

If the module is already loaded, it works as no-op. If the module was not compiled yet, `ensure_compiled!/1` halts the compilation of the caller until the module given to `ensure_compiled!/1` becomes available or all files for the current project have been compiled. If compilation finishes and the module is not available or is in a deadlock, an error is raised.

Given this function halts compilation, use it carefully. In particular, avoid using it to guess which modules are in the system. Overuse of this function can also lead to deadlocks, where two modules check at the same time if the other is compiled. This returns a specific unavailable error code, where we cannot successfully verify a module is available or not.

See the module documentation for more information on code loading.

ensure_loaded(module)

```
@spec ensure_loaded(module()) ::
  {:module, module()}
  | {:error, :embedded | :badfile | :nofile |
:on_load_failure}
```

Ensures the given module is loaded.

If the module is already loaded, this works as no-op. If the module was not yet loaded, it tries to load it.

If it succeeds in loading the module, it returns `{:module, module}`. If not, returns `{:error, reason}` with the error reason.

See the module documentation for more information on code loading.

Examples

```
iex> Code.ensure_loaded(Atom)
{:module, Atom}

iex> Code.ensure_loaded(DoesNotExist)
{:error, :nofile}
```

ensure_loaded!(module)

(since 1.12.0)

```
@spec ensure_loaded!(module()) :: module()
```

Same as `ensure_loaded/1` but raises if the module cannot be loaded.

ensure_loaded?(module)

```
@spec ensure_loaded?(module()) :: boolean()
```

Ensures the given module is loaded.

Similar to `ensure_loaded/1`, but returns `true` if the module is already loaded or was successfully loaded. Returns `false` otherwise.

Examples

```
iex> Code.ensure_loaded?(String)
true
```

env_for_eval(env_or_opts)

(since 1.14.0)

```
@spec env_for_eval(Macro.Env.t() | [env_eval_opt()]) ::
Macro.Env.t()
```

Returns an environment for evaluation.

It accepts either a `Macro.Env`, that is then pruned and prepared, or a list of options. It returns an environment that is ready for evaluation.

Most functions in this module will automatically prepare the given environment for evaluation, so you don't need to explicitly call this function, with the exception of `eval_quoted_with_env/3`, which was designed precisely to be called in a loop, to implement features such as interactive shells or anything else with multiple evaluations.

Options

If an env is not given, the options can be:

- `:file` - the file to be considered in the evaluation
- `:line` - the line on which the script starts
- `:module` - the module to run the environment on

eval_file(file, relative_to \\ nil)

```
@spec eval_file(binary(), nil | binary()) :: {term(),
binding()}
```

Evaluates the given file.

Accepts `relative_to` as an argument to tell where the file is located.

While `require_file/2` and `compile_file/2` return the loaded modules and their bytecode, `eval_file/2` simply evaluates the file contents and returns the evaluation result and its binding (exactly the same return value as `eval_string/3`).

eval_quoted(quoted, binding \\ [], env_or_opts \\ [])

```
@spec eval_quoted(Macro.t(), binding(), Macro.Env.t() |
[eval_opt() | env_eval_opt()]) ::
  {term(), binding()}
```

Evaluates the quoted contents.

Warning: Calling this function inside a macro is considered bad practice as it will attempt to evaluate runtime values at compile time. Macro arguments are typically transformed by unquoting them into the returned quoted expressions (instead of evaluated).

See `eval_string/3` for a description of arguments and return types. It accepts the same options as both `env_for_eval/1` and `eval_quoted_with_env/4`.

Examples

```
iex> contents = quote(do: var!(a) + var!(b))
iex> {result, binding} = Code.eval_quoted(contents, [a:
1, b: 2], file: __ENV__.file, line: __ENV__.line)
iex> result
3
```

```
iex> Enum.sort(binding)
[a: 1, b: 2]
```

For convenience, you can pass `__ENV__/0` as the `opts` argument and all options will be automatically extracted from the current environment:

```
iex> contents = quote(do: var!(a) + var!(b))
iex> {result, binding} = Code.eval_quoted(contents, [a:
1, b: 2], __ENV__)
iex> result
3
iex> Enum.sort(binding)
[a: 1, b: 2]
```

eval_quoted_with_env(quoted, binding, env, opts \\ [])

(since 1.14.0)

```
@spec eval_quoted_with_env(Macro.t(), binding(),
Macro.Env.t(), [eval_opt()]) ::
  {term(), binding(), Macro.Env.t()}
```

Evaluates the given `quoted` contents with `binding` and `env`.

This function is meant to be called in a loop, to implement features such as interactive shells or anything else with multiple evaluations. Therefore, the first time you call this function, you must compute the initial environment with `env_for_eval/1`. The remaining calls must pass the environment that was returned by this function.

Options

- `:prune_binding` - (since v1.14.2) prune binding to keep only variables read or written by the evaluated code. Note that variables used by modules are always pruned, even if later used by the modules. You can submit to the `:on_module` tracer event and access the variables used by the module from its environment.

- `:dbg_callback` - (since v1.20.0) overrides the behaviour of `dbg/2` used in the evaluated code. It must be a `{module, function, args}` tuple, see `dbg/2` for more details.

eval_string(string, binding \\ [], opts_or_env \\ [])

```
@spec eval_string(
  List.Chars.t(),
  binding(),
  Macro.Env.t() | [eval_opt() | env_eval_opt()]
) ::
  {term(), binding()}
```

Evaluates the contents given by `string`.

The `binding` argument is a list of all variables and their values. The `opts` argument is a keyword list of environment options.

Warning: `string` can be any Elixir code and will be executed with the same privileges as the Erlang VM: this means that such code could compromise the machine (for example by executing system commands). Don't use `eval_string/3` with untrusted input (such as strings coming from the network).

Options

It accepts the same options as both `env_for_eval/1` and `eval_quoted_with_env/4`. Additionally, you may also pass an environment as third argument, so the evaluation happens within that environment.

Return

Returns a tuple of the form `{value, binding}`, where `value` is the value returned from evaluating `string`. If an error occurs while evaluating `string`, an exception will be raised.

`binding` is a list with all variable names and their values after evaluating `string`. The binding keys are usually atoms, but they may be a tuple for variables defined in a different context. The names are in no particular order.

Examples

```
iex> {result, binding} = Code.eval_string("a + b", [a: 1,
b: 2], file: __ENV__.file, line: __ENV__.line)
iex> result
3
iex> Enum.sort(binding)
[a: 1, b: 2]

iex> {result, binding} = Code.eval_string("c = a + b",
[a: 1, b: 2], __ENV__)
iex> result
3
iex> Enum.sort(binding)
[a: 1, b: 2, c: 3]

iex> {result, binding} = Code.eval_string("a = a + b",
```

```
[a: 1, b: 2])
iex> result
3
iex> Enum.sort(binding)
[a: 3, b: 2]
```

For convenience, you can pass `__ENV__/0` as the `opts_or_env` argument and all imports, requires and aliases defined in the current environment will be automatically carried over:

```
iex> require Integer, warn: false
iex> {result, binding} = Code.eval_string("if
Integer.is_odd(a), do: a + b", [a: 1, b: 2], __ENV__)
iex> result
3
iex> Enum.sort(binding)
[a: 1, b: 2]
```

fetch_docs(module_or_path)

(since 1.7.0)

```
@spec fetch_docs(module() | String.t()) ::
  {:docs_v1, annotation, beam_language, format, module_doc ::
doc_content,
   metadata, docs :: [doc_element]}
  | {:error,
     :module_not_found
     | :chunk_not_found
     | {:invalid_chunk, binary()}
     | :invalid_beam}
when annotation: :erl_anno.anno(),
     beam_language: :elixir | :erlang | atom(),
```

```
    doc_content: %{optional(binary()) => binary()} | :none |
:hidden,
    doc_element:
      {{kind :: atom(), function_name :: atom(), arity()},
annotation,
       signature, doc_content, metadata},
    format: binary(),
    signature: [binary()],
    metadata: map()
```

Returns the docs for the given module or path to `.beam` file.

When given a module name, it finds its BEAM code and reads the docs from it.

When given a path to a `.beam` file, it will load the docs directly from that file.

It returns the term stored in the documentation chunk in the format defined by EEP 48 or `{:error, reason}` if the chunk is not available.

Examples

```
# Module documentation of an existing module
iex> {:docs_v1, _, :elixir, _, %{"en" => module_doc}, _,
_} = Code.fetch_docs(Atom)
iex> module_doc |> String.split("\n") |> Enum.at(0)
"Atoms are constants whose values are their own name."

# A module that doesn't exist
iex> Code.fetch_docs(ModuleNotGood)
{:error, :module_not_found}
```

format_file!(file, opts \\ [])

(since 1.6.0)

```
@spec format_file!(binary(), [format_opt()]) :: iodata()
```

Formats a file.

See `format_string!/2` for more information on code formatting and available options.

format_string!(string, opts \\ [])

(since 1.6.0)

```
@spec format_string!(binary(), [format_opt()]) :: iodata()
```

Formats the given code `string`.

The formatter receives a string representing Elixir code and returns iodata representing the formatted code according to pre-defined rules.

Options

Regular options (do not change the AST):

- `:file` - the file which contains the string, used for error reporting
- `:line` - the line the string starts, used for error reporting
- `:line_length` - the line length to aim for when formatting the document. Defaults to `98`. This value indicates when an expression should be broken over multiple lines but it is not guaranteed to do so. See the "Line length" section below for more information
- `:locals_without_parens` - a keyword list of name and arity pairs that should be kept without parens whenever possible. The arity

may be the atom `:*`, which implies all arities of that name. The formatter already includes a list of functions and this option augments this list.

- `:force_do_end_blocks` (since v1.9.0) - when `true`, converts all inline usages of `do: ...`, `else: ...` and friends into `do-end` blocks. Defaults to `false`. Note that this option is convergent: once you set it to `true`, **all keywords** will be converted. If you set it to `false` later on, `do-end` blocks won't be converted back to keywords.

Migration options (change the AST), see the "Migration formatting" section below:

- `:migrate` (since v1.18.0) - when `true`, sets all other migration options to `true` by default. Defaults to `false`.

- `:migrate_bitstring_modifiers` (since v1.18.0) - when `true`, removes unnecessary parentheses in known bitstring modifiers, for example `<<foo::binary()>>` becomes `<<foo::binary>>`, or adds parentheses for custom modifiers, where `<<foo::custom_type>>` becomes `<<foo::custom_type()>>`. Defaults to the value of the `:migrate` option. This option changes the AST.

- `:migrate_call_parens_on_pipe` (since v1.19.0) - when `true`, formats calls on the right-hand side of the pipe operator to always include parentheses, for example `foo |> bar` becomes `foo |> bar()` and `foo |> mod.fun` becomes `foo |> mod.fun()`. Parentheses are always added for qualified calls like `foo |> Bar.bar` even when this option is `false`. Defaults to the value of the `:migrate` option. This option changes the AST.

- `:migrate_charlists_as_sigils` (since v1.18.0) - when `true`, formats charlists as `~c` sigils, for example `'foo'` becomes

`~c"foo"`. Defaults to the value of the `:migrate` option. This option changes the AST.

- `:migrate_unless` (since v1.18.0) - when `true`, rewrites `unless` expressions using `if` with a negated condition, for example `unless foo, do:` becomes `if !foo, do:`. Defaults to the value of the `:migrate` option. This option changes the AST.

Design principles

The formatter was designed under three principles.

First, the formatter never changes the semantics of the code by default. This means the input AST and the output AST are almost always equivalent.

The second principle is to provide as little configuration as possible. This eases the formatter adoption by removing contention points while making sure a single style is followed consistently by the community as a whole.

The formatter does not hard code names. The formatter will not behave specially because a function is named `defmodule`, `def`, or the like. This principle mirrors Elixir's goal of being an extensible language where developers can extend the language with new constructs as if they were part of the language. When it is absolutely necessary to change behavior based on the name, this behavior should be configurable, such as the `:locals_without_parens` option.

Running the formatter

The formatter attempts to fit the most it can on a single line and introduces line breaks wherever possible when it cannot.

In some cases, this may lead to undesired formatting. Therefore, **some code generated by the formatter may not be aesthetically pleasing and may require explicit intervention from the developer**. That's why we do not recommend to run the formatter blindly in an existing codebase. Instead you should format and sanity check each formatted file.

For example, the formatter may break a long function definition over multiple clauses:

```
def my_function(
  %User{name: name, age: age, ...},
  arg1,
  arg2
) do
  ...
end
```

While the code above is completely valid, you may prefer to match on the struct variables inside the function body in order to keep the definition on a single line:

```
def my_function(%User{} = user, arg1, arg2) do
  %{name: name, age: age, ...} = user
  ...
end
```

In some situations, you can use the fact the formatter does not generate elegant code as a hint for refactoring. Take this code:

```
def board?(board_id, %User{} = user,
available_permissions, required_permissions) do
  Tracker.OrganizationMembers.user_in_organization?
```

```
(user.id, board.organization_id) and
    required_permissions ==
Enum.to_list(MapSet.intersection(MapSet.new(required_perm
issions), MapSet.new(available_permissions)))
end
```

The code above has very long lines and running the formatter is not going to address this issue. In fact, the formatter may make it more obvious that you have complex expressions:

```
def board?(board_id, %User{} = user,
available_permissions, required_permissions) do
  Tracker.OrganizationMembers.user_in_organization?
(user.id, board.organization_id) and
    required_permissions ==
      Enum.to_list(
        MapSet.intersection(
          MapSet.new(required_permissions),
          MapSet.new(available_permissions)
        )
      )
end
```

Take such cases as a suggestion that your code should be refactored:

```
def board?(board_id, %User{} = user,
available_permissions, required_permissions) do
  Tracker.OrganizationMembers.user_in_organization?
(user.id, board.organization_id) and
    matching_permissions?(required_permissions,
available_permissions)
end
```

```
defp matching_permissions?(required_permissions,
available_permissions) do
  intersection =
    required_permissions
    |> MapSet.new()
    |>
MapSet.intersection(MapSet.new(available_permissions))
    |> Enum.to_list()

  required_permissions == intersection
end
```

To sum it up: since the formatter cannot change the semantics of your code, sometimes it is necessary to tweak or refactor the code to get optimal formatting. To help better understand how to control the formatter, we describe in the next sections the cases where the formatter keeps the user encoding and how to control multiline expressions.

Line length

Another point about the formatter is that the `:line_length` configuration indicates when an expression should be broken over multiple lines but it is not guaranteed to do so. In many cases, it is not possible for the formatter to break your code apart, which means it will go over the line length. For example, if you have a long string:

```
"this is a very long string that will go over the line
length"
```

The formatter doesn't know how to break it apart without changing the code underlying syntax representation, so it is up to you to step in:

```
"this is a very long string " <>
   "that will go over the line length"
```

The string concatenation makes the code fit on a single line and also gives more options to the formatter.

This may also appear in keywords such as do/end blocks and operators, where the `do` keyword may go over the line length because there is no opportunity for the formatter to introduce a line break in a readable way. For example, if you do:

```
case very_long_expression() do
end
```

And only the `do` keyword is beyond the line length, Elixir **will not** emit this:

```
case very_long_expression()
do
end
```

So it prefers to not touch the line at all and leave `do` above the line limit.

Keeping user's formatting

The formatter respects the input format in some cases. Those are listed below:

- Insignificant digits in numbers are kept as is. The formatter, however, always inserts underscores for decimal numbers with more than 5 digits and converts hexadecimal digits to uppercase

- Strings, charlists, atoms and sigils are kept as is. No character is automatically escaped or unescaped. The choice of delimiter is also respected from the input

- Newlines inside blocks are kept as in the input except for:

    1. expressions that take multiple lines will always have an empty line before and after and 2) empty lines are always squeezed together into a single empty line

- The choice between `:do` keyword and `do-end` blocks is left to the user

- Lists, tuples, bitstrings, maps, structs and function calls will be broken into multiple lines if they are followed by a newline in the opening bracket and preceded by a new line in the closing bracket

- Newlines before certain operators (such as the pipeline operators) and before other operators (such as comparison operators)

The behaviors above are not guaranteed. We may remove or add new rules in the future. The goal of documenting them is to provide better understanding on what to expect from the formatter.

Multi-line lists, maps, tuples, and the like

You can force lists, tuples, bitstrings, maps, structs and function calls to have one entry per line by adding a newline after the opening bracket and a new line before the closing bracket lines. For example:

```
[
  foo,
  bar
]
```

If there are no newlines around the brackets, then the formatter will try to fit everything on a single line, such that the snippet below

```
[foo,
 bar]
```

will be formatted as

```
[foo, bar]
```

You can also force function calls and keywords to be rendered on multiple lines by having each entry on its own line:

```
defstruct name: nil,
          age: 0
```

The code above will be kept with one keyword entry per line by the formatter. To avoid that, just squash everything into a single line.

Parens and no parens in function calls

Elixir has two syntaxes for function calls. With parens and no parens. By default, Elixir will add parens to all calls except for:

1. calls that have `do-end` blocks
2. local calls without parens where the name and arity of the local call is also listed under `:locals_without_parens` (except for calls with arity 0, where the compiler always require parens)

The choice of parens and no parens also affects indentation. When a function call with parens doesn't fit on the same line, the formatter introduces a newline around parens and indents the arguments with two spaces:

```
some_call(
  arg1,
  arg2,
  arg3
)
```

On the other hand, function calls without parens are always indented by the function call length itself, like this:

```
some_call arg1,
          arg2,
          arg3
```

If the last argument is a data structure, such as maps and lists, and the beginning of the data structure fits on the same line as the function call, then no indentation happens, this allows code like this:

```
Enum.reduce(some_collection, initial_value, fn element,
acc ->
  # code
end)

some_function_without_parens %{
  foo: :bar,
  baz: :bat
}
```

Code comments

The formatter handles code comments and guarantees a space is always added between the beginning of the comment (#) and the next character.

The formatter also extracts all trailing comments to their previous line. For example, the code below

```
hello #world
```

will be rewritten to

```
# world
hello
```

While the formatter attempts to preserve comments in most situations, that's not always possible, because code comments are handled apart from the code representation (AST). While the formatter can preserve code comments between expressions and function arguments, the formatter cannot currently preserve them around operators. For example, the following code:

```
foo() ||
  # also check for bar
  bar()
```

will move the code comments to before the operator usage:

```
# also check for bar
foo() ||
  bar()
```

In some situations, code comments can be seen as ambiguous by the formatter. For example, the comment in the anonymous function below

```
fn
  arg1 ->
    body1
    # comment

  arg2 ->
    body2
end
```

and in this one

```
fn
  arg1 ->
    body1

  # comment
  arg2 ->
    body2
end
```

are considered equivalent (the nesting is discarded alongside most of user formatting). In such cases, the code formatter will always format to the latter.

Newlines

The formatter converts all newlines in code from `\r\n` to `\n`.

Migration formatting

As part of the Elixir release cycle, deprecations are being introduced, emitting warnings which might require existing code to be changed. In

order to reduce the burden on developers when upgrading Elixir to the next version, the formatter exposes some options, disabled by default, in order to automate this process.

These options should address most of the typical use cases, but given they introduce changes to the AST, there is a non-zero risk for meta-programming heavy projects that relied on a specific AST, or projects that are re-defining functions from the `Kernel`. In such cases, migrations cannot be applied blindly and some extra changes might be needed in order to address the deprecation warnings.

get_compiler_option(key)

(since 1.10.0)

```
@spec get_compiler_option(atom()) :: term()
```

Returns the value of a given compiler option.

For a description of all options, see `put_compiler_option/2`.

Examples

```
Code.get_compiler_option(:debug_info)
#=> true
```

loaded?(module)

(since 1.15.0)

```
@spec loaded?(module()) :: boolean()
```

Returns `true` if the module is loaded.

This function doesn't attempt to load the module. For such behavior, `ensure_loaded?/1` can be used.

Examples

```
iex> Code.loaded?(String)
true

iex> Code.loaded?(NotYetLoaded)
false
```

prepend_path(path, opts \\ [])

```
@spec prepend_path(Path.t(), [{:cache, boolean()}]) ::
boolean()
```

Prepends a path to the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

The path is expanded with `Path.expand/1` before being prepended. It requires the path to exist. Returns a boolean indicating if the path was successfully added.

Examples

```
Code.prepend_path(".")
#=> true
```

```
Code.prepend_path("/does_not_exist")
#=> false
```

Options

- `:cache` - (since v1.15.0) when true, the code path is cached the first time it is traversed in order to reduce file system operations.

prepend_paths(paths, opts \\ [])

(since 1.15.0)

```
@spec prepend_paths([Path.t()], [{:cache, boolean()}]) :: :ok
```

Prepends a list of `paths` to the Erlang VM code path list.

This is the list of directories the Erlang VM uses for finding module code. The list of files is managed per Erlang VM node.

All paths are expanded with `Path.expand/1` before being prepended. Only existing paths are prepended. This function always returns `:ok`, regardless of how many paths were prepended. Use `prepend_path/1` if you need more control.

Examples

```
Code.prepend_paths([".", "/does_not_exist"])
#=> :ok
```

Options

- `:cache` - when true, the code path is cached the first time it is traversed in order to reduce file system operations.

print_diagnostic(diagnostic, opts \\ [])

(since 1.15.0)

```
@spec print_diagnostic(diagnostic(:warning | :error),
[{:snippet, boolean()}]) :: :ok
```

Prints a diagnostic into the standard error.

A diagnostic is either returned by `Kernel.ParallelCompiler` or by `Code.with_diagnostics/2`.

Options

- `:snippet` - whether to read the code snippet in the diagnostic location. As it may impact performance, it is not recommended to be used in runtime. Defaults to `true`.

purge_compiler_modules()

(since 1.7.0)

```
@spec purge_compiler_modules() :: {:ok, non_neg_integer()}
```

Purge compiler modules.

The compiler utilizes temporary modules to compile code. For example, `elixir_compiler_1`, `elixir_compiler_2`, and so on. In case the compiled code stores references to anonymous functions or similar, the Elixir compiler may be unable to reclaim those modules, keeping an

unnecessary amount of code in memory and eventually leading to modules such as `elixir_compiler_12345`.

This function purges all modules currently kept by the compiler, allowing old compiler module names to be reused. If there are any processes running any code from such modules, they will be terminated too.

This function is only meant to be called if you have a long running node that is constantly evaluating code.

It returns `{:ok, number_of_modules_purged}`.

put_compiler_option(key, value)

(since 1.10.0)

```
@spec put_compiler_option(atom(), term()) :: :ok
```

Stores a compilation option.

Changing the compilation options affect all processes running in a given Erlang VM node.

Available options are:

- `:debug_info` - when `true`, retains debug information in the compiled module. This option can also be overridden per module using the `@compile` directive. Defaults to `true`.

  This enables tooling to partially reconstruct the original source code, for instance, to perform static analysis of code. Therefore, disabling `:debug_info` is not recommended as it removes the ability of the Elixir compiler and other tools to provide feedback. If you want to remove the `:debug_info` while deploying, tools like `mix release` already do such by default.

Other environments, such as `mix test`, automatically disables this via the `:test_elixirc_options` project configuration, as there is typically no need to store debug chunks for test files.

- `:docs` - when `true`, retains documentation in the compiled module. Defaults to `true`.

- `:ignore_already_consolidated` (since v1.10.0) - when `true`, does not warn when a protocol has already been consolidated and a new implementation is added. Defaults to `false`.

- `:ignore_module_conflict` - when `true`, does not warn when a module has already been defined. Defaults to `false`.

- `:infer_signatures` (since v1.18.0) - a list of applications of which modules should be using during type inference. When `false`, it disables module-local signature inference used when type checking remote calls to the compiled module. Type checking will be executed regardless of the value of this option. Mix projects will set this option to your dependencies list in dev/prod, and it will disable this option during test (as there is typically no need to infer signature for test files). Outside of Mix projects, it defaults to `[:elixir]`.

- `:module_definition` (since v1.20.0) - stores if the module definition should be `:compiled` (the default) or `:interpreted`. Note this does not affect the `.beam` file written to disk, only how the contents inside `defmodule` are executed. Using the `:interpreted` mode may offer better compilation times for large projects, especially on machines with high core count, however, it comes with some downsides:

    - Errors during compilation may have less precise stacktraces

    - Anonymous functions within `defmodule` can have only up to 20 arguments. If this is an issue, you can use maps or tuples to group the data. Note the functions themselves inside `defmodule`, such as the ones defined inside `def` and friends, can still have up to 255 arguments

- `:no_warn_undefined` (since v1.10.0) - list of modules and `{Mod, fun, arity}` tuples that will not emit warnings that the module or function does not exist at compilation time. Pass atom `:all` to skip warning for all undefined functions. This can be useful when doing dynamic compilation. Defaults to `[]`.

- `:on_undefined_variable` (since v1.15.0) - either `:raise` or `:warn`. When `:raise` (the default), undefined variables will trigger a compilation error. You may be set it to `:warn` if you want undefined variables to emit a warning and expand as to a local call to the zero-arity function of the same name (for example, `node` would be expanded as `node()`). This `:warn` behavior only exists for compatibility reasons when working with old dependencies, its usage is discouraged and it will be removed in future releases.

- `:parser_options` (since v1.10.0) - a keyword list of options to be given to the parser when compiling files. It accepts the same options as `string_to_quoted/2` (except by the options that change the AST itself). This can be used in combination with the tracer to retrieve localized information about events happening during compilation. Defaults to `[columns: true]`. This option only affects code compilation functions, such as `compile_string/2` and `compile_file/2` but not `string_to_quoted/2` and friends, as the latter is used for other purposes beyond compilation.

- `:relative_paths` - when `true`, uses relative paths in quoted nodes, warnings, and errors generated by the compiler. Note

disabling this option won't affect runtime warnings and errors. Defaults to `true`.

- `:tracers` (since v1.10.0) - a list of tracers (modules) to be used during compilation. See the module docs for more information. Defaults to `[]`.

It always returns `:ok`. Raises an error for invalid options.

Examples

```
Code.put_compiler_option(:debug_info, true)
#=> :ok
```

quoted_to_algebra(quoted, opts \\ [])

(since 1.13.0)

```
@spec quoted_to_algebra(Macro.t(), [format_opt() |
quoted_to_algebra_opt()]) ::
  Inspect.Algebra.t()
```

Converts a quoted expression to an algebra document using Elixir's formatter rules.

The algebra document can be converted into a string by calling:

```
doc
|> Inspect.Algebra.format(:infinity)
|> IO.iodata_to_binary()
```

For a high-level function that does the same, see `Macro.to_string/1`.

Formatting considerations

The Elixir AST does not contain metadata for literals like strings, lists, or tuples with two elements, which means that the produced algebra document will not respect all of the user preferences and comments may be misplaced. To get better results, you can use the `:token_metadata`, `:unescape` and `:literal_encoder` options to `string_to_quoted/2` to provide additional information to the formatter:

```
[
  literal_encoder: &{:ok, {:__block__, &2, [&1]}},
  token_metadata: true,
  unescape: false
]
```

This will produce an AST that contains information such as `do` blocks start and end lines or sigil delimiters, and by wrapping literals in blocks they can now hold metadata like line number, string delimiter and escaped sequences, or integer formatting (such as `0x2a` instead of `47`). However, **note this AST is not valid**. If you evaluate it, it won't have the same semantics as the regular Elixir AST due to the `:unescape` and `:literal_encoder` options. However, those options are useful if you're doing source code manipulation, where it's important to preserve user choices and comments placing.

Options

This function accepts all options supported by `format_string!/2` for controlling code formatting, plus these additional options:

- `:comments` - the list of comments associated with the quoted expression. Defaults to `[]`. It is recommended that both `:token_metadata` and `:literal_encoder` options are given to

string_to_quoted_with_comments/2 in order to get proper placement for comments

- :escape - when true, escaped sequences like \n will be escaped into \\n. If the :unescape option was set to false when using string_to_quoted/2, setting this option to false will prevent it from escaping the sequences twice. Defaults to true.

See format_string!/2 for the full list of formatting options including :file, :line, :line_length, :locals_without_parens, :force_do_end_blocks, :syntax_colors, and all migration options like :migrate_charlists_as_sigils.

require_file(file, relative_to \\ nil)

```
@spec require_file(binary(), nil | binary()) :: [{module(), binary()}] | nil
```

Requires the given file.

Accepts relative_to as an argument to tell where the file is located. If the file was already required, require_file/2 doesn't do anything and returns nil.

Note that if require_file/2 is invoked by different processes concurrently, the first process to invoke require_file/2 acquires a lock and the remaining ones will block until the file is available. This means that if require_file/2 is called more than once with a given file, that file will be compiled only once. The first process to call require_file/2 will get the list of loaded modules, others will get nil. The list of required files is managed per Erlang VM node.

See `compile_file/2` if you would like to compile a file without tracking its filenames. Finally, if you would like to get the result of evaluating a file rather than the modules defined in it, see `eval_file/2`.

Examples

If the file has not been required, it returns the list of modules:

```
modules = Code.require_file("eex_test.exs",
"../eex/test")
List.first(modules)
#=> {EExTest.Compiled, <<70, 79, 82, 49, ...>>}
```

If the file has been required, it returns `nil`:

```
Code.require_file("eex_test.exs", "../eex/test")
#=> nil
```

required_files()

(since 1.7.0)

```
@spec required_files() :: [binary()]
```

Lists all required files.

Examples

```
Code.require_file("../eex/test/eex_test.exs")
List.first(Code.required_files()) =~ "eex_test.exs"
#=> true
```

string_to_quoted(string, opts \\ [])

```
@spec string_to_quoted(List.Chars.t(), parser opts()) ::
  {:ok, Macro.t()}
  | {:error, {location :: keyword(), binary() | {binary(),
binary()}, binary()}}
```

Converts the given string to its quoted form.

Returns `{:ok, quoted_form}` if it succeeds, `{:error, {meta, message_info, token}}` otherwise.

Options

- `:file` - the filename to be reported in case of parsing errors. Defaults to `"nofile"`.
- `:line` - the starting line of the string being parsed. Defaults to `1`.
- `:column` - (since v1.11.0) the starting column of the string being parsed. Defaults to `1`.
- `:indentation` - (since v1.19.0) the indentation for the string being parsed. This is useful when the code parsed is embedded within another document. Defaults to `0`.
- `:columns` - when `true`, attach a `:column` key to the quoted metadata. Defaults to `false`.
- `:unescape` (since v1.10.0) - when `false`, preserves escaped sequences. For example, `"null byte\\t\\x00"` will be kept as is instead of being converted to a bitstring literal. Note if you set this option to false, the resulting AST is no longer valid, but it can be

useful to analyze/transform source code, typically in combination with `quoted_to_algebra/2`. Defaults to `true`.

- `:existing_atoms_only` - when `true`, raises an error when non-existing atoms are found by the tokenizer. Defaults to `false`.

- `:token_metadata` (since v1.10.0) - when `true`, includes token-related metadata in the expression AST, such as metadata for `do` and `end` tokens, for closing tokens, end of expressions, as well as delimiters for sigils. See `Macro.metadata/0`. Defaults to `false`.

- `:literal_encoder` (since v1.10.0) - how to encode literals in the AST. It must be a function that receives two arguments, the literal and its metadata, and it must return `{:ok, ast :: Macro.t}` or `{:error, reason :: binary}`. If you return anything than the literal itself as the `term`, then the AST is no longer valid. This option may still useful for textual analysis of the source code.

- `:static_atoms_encoder` - the static atom encoder function, see "The `:static_atoms_encoder` function" section below. Note this option overrides the `:existing_atoms_only` behavior for static atoms but `:existing_atoms_only` is still used for dynamic atoms, such as atoms with interpolations.

- `:emit_warnings` (since v1.16.0) - when `false`, does not emit tokenizing/parsing related warnings. Defaults to `true`.

`Macro.to_string/2`

The opposite of converting a string to its quoted form is `Macro.to_string/2`, which converts a quoted form to a string/binary representation.

The `:static_atoms_encoder` function

When `static_atoms_encoder: &my_encoder/2` is passed as an argument, `my_encoder/2` is called every time the tokenizer needs to create a "static" atom. Static atoms are atoms in the AST that function as aliases, remote calls, local calls, variable names, regular atoms and keyword lists.

The encoder function will receive the atom name (as a binary) and a keyword list with the current line and column. It must return `{:ok, token :: term} | {:error, reason :: binary}`.

The encoder function is supposed to create an atom from the given string. To produce a valid AST, it is required to return `{:ok, term}`, where `term` is an atom. It is possible to return something other than an atom, however, in that case the AST is no longer "valid" in that it cannot be used to compile or evaluate Elixir code. A use case for this is if you want to use the Elixir parser in a user-facing situation, but you don't want to exhaust the atom table.

The atom encoder is not called for *all* atoms that are present in the AST. It won't be invoked for the following atoms:

- operators (`:+`, `:-` , and so on)
- syntax keywords (`fn`, `do`, `else` , and so on)
- atoms containing interpolation (`:"#{1 + 1} is two"`), as these atoms are constructed at runtime
- atoms used to represent single-letter sigils like `:sigil_X` (but multi-letter sigils like `:sigil_XYZ` are encoded).

Examples

```
iex> Code.string_to_quoted("1 + 3")
{:ok, {:+, [line: 1], [1, 3]}}

iex> Code.string_to_quoted("1 \ 3")
{:error, {[line: 1, column: 4], "syntax error before: ",
"\"3\""}}
```

string_to_quoted!(string, opts \\ [])

```
@spec string_to_quoted!(List.Chars.t(), parser opts()) ::
Macro.t()
```

Converts the given string to its quoted form.

It returns the AST if it succeeds, raises an exception otherwise. The exception is a `TokenMissingError` in case a token is missing (usually because the expression is incomplete), `MismatchedDelimiterError` (in case of mismatched opening and closing delimiters) and `SyntaxError` otherwise.

Check `string_to_quoted/2` for options information.

string_to_quoted_with_comments(string, opts \\ [])

(since 1.13.0)

```
@spec string_to_quoted_with_comments(List.Chars.t(),
parser opts()) ::
  {:ok, Macro.t(), [map()]} | {:error, {location ::
keyword(), term(), term()}}
```

Converts the given string to its quoted form and a list of comments.

This function is useful when performing textual changes to the source code, while preserving information like comments and literals position.

Returns `{:ok, quoted_form, comments}` if it succeeds, `{:error, {line, error, token}}` otherwise.

Comments are maps with the following fields:

- `:line` - The line number of the source code
- `:text` - The full text of the comment, including the leading `#`
- `:previous_eol_count` - How many end of lines there are between the comment and the previous AST node or comment
- `:next_eol_count` - How many end of lines there are between the comment and the next AST node or comment

Check `string_to_quoted/2` for options information.

Examples

```
iex> Code.string_to_quoted_with_comments("""
...> :foo
...>
...> # Hello, world!
...>
...>
...> # Some more comments!
...> """)
{:ok, :foo, [
  %{line: 3, column: 1, previous_eol_count: 2,
next_eol_count: 3, text: "# Hello, world!"},
  %{line: 6, column: 1, previous_eol_count: 3,
next_eol_count: 1, text: "# Some more comments!"},
```

```
]}

iex> Code.string_to_quoted_with_comments(":foo # :bar")
{:ok, :foo, [
  %{line: 1, column: 6, previous_eol_count: 0,
next_eol_count: 0, text: "# :bar"}
]}
```

string_to_quoted_with_comments!(string, opts \\ [])

(since 1.13.0)

```
@spec string_to_quoted_with_comments!(List.Chars.t(),
parser opts()) ::
  {Macro.t(), [map()]}
```

Converts the given string to its quoted form and a list of comments.

Returns the AST and a list of comments if it succeeds, raises an exception otherwise. The exception is a `TokenMissingError` in case a token is missing (usually because the expression is incomplete), `SyntaxError` otherwise.

Check `string_to_quoted/2` for options information.

unrequire_files(files)

(since 1.7.0)

```
@spec unrequire_files([binary()]) :: :ok
```

Removes files from the required files list.

The modules defined in the file are not removed; calling this function only removes them from the list, allowing them to be required again.

The list of files is managed per Erlang VM node.

Examples

```
# Require EEx test code
Code.require_file("../eex/test/eex_test.exs")

# Now unrequire all files
Code.unrequire_files(Code.required_files())

# Note that modules are still available
function_exported?(EExTest.Compiled, :before_compile, 0)
#=> true
```

with_diagnostics(opts \\ [], fun)

(since 1.15.0)

```
@spec with_diagnostics([{:log, boolean()}], (-> result)) ::
  {result, [diagnostic(:warning | :error)]}
when result: term()
```

Executes the given `fun` and capture all diagnostics.

Diagnostics are warnings and errors emitted during code evaluation or single-file compilation and by functions such as `IO.warn/2`.

If using `mix compile` or `Kernel.ParallelCompiler`, note they already capture and return diagnostics.

Options

- `:log` - if the diagnostics should be logged as they happen. Defaults to `false`.

> **Rescuing errors**
>
> `with_diagnostics/2` does not automatically handle exceptions. You may capture them by adding a `try/1` in `fun`:
>
> ```
> {result, all_errors_and_warnings} =
>   Code.with_diagnostics(fn ->
>     try do
>       {:ok, Code.compile_quoted(quoted)}
>     rescue
>       err -> {:error, err}
>     end
>   end)
> ```

Code.Fragment

This module provides conveniences for analyzing fragments of textual code and extract available information whenever possible.

This module should be considered experimental.

Summary

Types

container_cursor_to_quoted_opts()

Options for converting code fragments to quoted expressions.

cursor_opts()

Options for cursor context functions.

position()

Functions

container_cursor_to_quoted(fragment, opts \\ [])

Receives a string and returns a quoted expression with the cursor AST position within its parent expression.

cursor_context(fragment, opts \\ [])

Receives a string and returns the cursor context.

lines(string)

Returns the list of lines in the given string, preserving their line endings.
surround_context(fragment, position, options \\ [])

Receives a string and returns the surround context.

Types

container_cursor_to_quoted_opts()

```
@type container_cursor_to_quoted_opts() :: [
  file: String.t(),
  line: pos_integer(),
  column: pos_integer(),
  columns: boolean(),
  token_metadata: boolean(),
  literal_encoder: (term(), Macro.metadata() -> term()),
  trailing_fragment: String.t()
]
```

Options for converting code fragments to quoted expressions.

cursor_opts()

```
@type cursor_opts() :: []
```

Options for cursor context functions.

Currently, these options are not used but reserved for future extensibility.

position()

```
@type position() :: {line :: pos integer(), column ::
pos integer()}
```

Functions

container_cursor_to_quoted(fragment, opts \\ [])

(since 1.13.0)

```
@spec container_cursor_to_quoted(List.Chars.t(),
container cursor to quoted opts()) ::
  {:ok, Macro.t()}
  | {:error, {location :: keyword(), binary() | {binary(),
binary()}, binary()}}
```

Receives a string and returns a quoted expression with the cursor AST position within its parent expression.

This function receives a string with an Elixir code fragment, representing a cursor position, and converts such string to AST with the inclusion of special `__cursor__()` node representing the cursor position within its container (i.e. its parent).

For example, take this code, which would be given as input:

```
max(some_value,
```

This function will return the AST equivalent to:

```
max(some_value, __cursor__())
```

In other words, this function is capable of closing any open brackets and insert the cursor position. Other content at the cursor position which is

not a parent is discarded. For example, if this is given as input:

```
max(some_value, another_val
```

It will return the same AST:

```
max(some_value, __cursor__())
```

Similarly, if only this is given:

```
max(some_va
```

Then it returns:

```
max(__cursor__())
```

Calls without parenthesis are also supported, as we assume the brackets are implicit.

Tuples, lists, maps, and binaries all retain the cursor position:

```
max(some_value, [1, 2,
```

Returns the following AST:

```
max(some_value, [1, 2, __cursor__()])
```

Keyword lists (and do-end blocks) are also retained. The following:

```
if(some_value, do:
if(some_value, do: :token
if(some_value, do: 1 + val
```

all return:

```
if(some_value, do: __cursor__())
```

For multi-line blocks, all previous lines are preserved.

The AST returned by this function is not safe to evaluate but it can be analyzed and expanded.

Examples

Function call:

```
iex>
Code.Fragment.container_cursor_to_quoted("max(some_value,
")
{:ok, {:max, [line: 1], [{:some_value, [line: 1], nil},
{:__cursor__, [line: 1], []}]}}
```

Containers (for example, a list):

```
iex> Code.Fragment.container_cursor_to_quoted("[some,
value")
{:ok, [{:some, [line: 1], nil}, {:__cursor__, [line: 1],
[]}]}
```

If an expression is complete, then the whole expression is discarded and only the parent is returned:

```
iex>
Code.Fragment.container_cursor_to_quoted("if(is_atom(var)
")
{:ok, {:if, [line: 1], [{:__cursor__, [line: 1], []}]}}
```

this means complete expressions themselves return only the cursor:

```
iex>
Code.Fragment.container_cursor_to_quoted("if(is_atom(var)
)")
{:ok, {:__cursor__, [line: 1], []}}
```

Operators are also included from Elixir v1.15:

```
iex> Code.Fragment.container_cursor_to_quoted("foo +")
{:ok, {:+, [line: 1], [{:foo, [line: 1], nil},
{:__cursor__, [line: 1], []}]}}
```

In order to parse the left-side of `->` properly, which appears both in anonymous functions and do-end blocks, the trailing fragment option must be given with the rest of the contents:

```
iex> Code.Fragment.container_cursor_to_quoted("fn x",
trailing_fragment: " -> :ok end")
{:ok, {:fn, [line: 1], [{:->, [line: 1], [[{:__cursor__,
[line: 1], []}], :ok]}]}}
```

Options

- `:file` - the filename to be reported in case of parsing errors. Defaults to `"nofile"`.
- `:line` - the starting line of the string being parsed. Defaults to `1`.
- `:column` - the starting column of the string being parsed. Defaults to `1`.
- `:columns` - when `true`, attach a `:column` key to the quoted metadata. Defaults to `false`.
- `:token_metadata` - when `true`, includes token-related metadata in the expression AST, such as metadata for `do` and `end` tokens, for closing tokens, end of expressions, as well as delimiters for sigils. See `Macro.metadata/0`. Defaults to `false`.
- `:literal_encoder` - a function to encode literals in the AST. See the documentation for `Code.string_to_quoted/2` for more information.
- `:trailing_fragment` (since v1.18.0) - the rest of the contents after the cursor. This is necessary to correctly complete anonymous functions and the left-hand side of `->`
- `:preserve_sigils` (since v1.20.0) - preserve sigil cursor location (see "Tracking sigils" section below)

Tracking sigils

The `:preserve_sigils` option can be used to track cursor positions inside a sigil.

If the sigil is terminated abruptly, the `sigil_*` call will have the cursor as the second argument:

```
iex> Code.Fragment.container_cursor_to_quoted("~r/foo",
preserve_sigils: true)
{:ok,
{:sigil_r, [delimiter: "/", line: 1],
  [{:<<>>, [line: 1], ["foo"]}, {:__cursor__, [line: 1,
column: 7], []}]}}
```

In case the sigil is completed and has zero or more modifiers, the cursor will be nested in the list, with all previous delimiters specified:

```
iex> Code.Fragment.container_cursor_to_quoted("~r/foo/i",
preserve_sigils: true)
{:ok,
 {:sigil_r, [delimiter: "/", line: 1],
  [{:<<>>, [line: 1], ["foo"]}, [105, {:__cursor__,
[line: 1, column: 9], []}]]}}
```

If the cursor is after the sigil, then it is discarded as everything else:

```
iex> Code.Fragment.container_cursor_to_quoted("~r/foo/i
", preserve_sigils: true)
{:ok, {:__cursor__, [line: 1], []}}
```

cursor_context(fragment, opts \\ [])

(since 1.13.0)

```
@spec cursor_context(List.Chars.t(), cursor_opts()) ::
  {:alias, charlist()}
  | {:alias, inside_alias, charlist()}
  | {:block_keyword_or_binary_operator, charlist()}
  | {:dot, inside_dot, charlist()}
```

```
  | {:dot_arity, inside_dot, charlist()}
  | {:dot_call, inside_dot, charlist()}
  | :expr
  | {:local_or_var, charlist()}
  | {:local_arity, charlist()}
  | {:local_call, charlist()}
  | {:anonymous_call, inside_caller}
  | {:capture_arg, charlist()}
  | {:module_attribute, charlist()}
  | {:operator, charlist()}
  | {:operator_arity, charlist()}
  | {:operator_call, charlist()}
  | :none
  | {:sigil, charlist()}
  | {:struct, inside_struct}
  | {:unquoted_atom, charlist()}
when inside_dot:
       {:alias, charlist()}
       | {:alias, inside_alias, charlist()}
       | {:dot, inside_dot, charlist()}
       | {:module_attribute, charlist()}
       | {:unquoted_atom, charlist()}
       | {:var, charlist()}
       | :expr,
     inside_alias: {:local_or_var, charlist()} |
{:module_attribute, charlist()},
     inside_struct:
       charlist()
       | {:alias, inside_alias, charlist()}
       | {:local_or_var, charlist()}
       | {:module_attribute, charlist()}
       | {:dot, inside_dot, charlist()},
     inside_caller: {:var, charlist()} | {:module_attribute,
charlist()}
```

Receives a string and returns the cursor context.

This function receives a string with an Elixir code fragment, representing a cursor position, and based on the string, it provides contextual information about the latest token. The return of this function can then be used to provide tips, suggestions, and autocompletion functionality.

This function performs its analyses on tokens. This means it does not understand how constructs are nested within each other. See the "Limitations" section below.

Consider adding a catch-all clause when handling the return type of this function as new cursor information may be added in future releases.

Examples

```
iex> Code.Fragment.cursor_context("")
:expr

iex> Code.Fragment.cursor_context("hello_wor")
{:local_or_var, ~c"hello_wor"}
```

Return values

- `{:alias, charlist}` - the context is an alias, potentially a nested one, such as `Hello.Wor` or `HelloWor`
- `{:alias, inside_alias, charlist}` - the context is an alias, potentially a nested one, where `inside_alias` is an expression `{:module_attribute, charlist}` or `{:local_or_var, charlist}` and `charlist` is a static part Examples are `__MODULE__.Submodule` or `@hello.Submodule`
- `{:block_keyword_or_binary_operator, charlist}` - may be a block keyword (do, end, after, catch, else, rescue) or a binary

operator

- `{:dot, inside_dot, charlist}` - the context is a dot where `inside_dot` is either a `{:var, charlist}`, `{:alias, charlist}`, `{:module_attribute, charlist}`, `{:unquoted_atom, charlist}` or a `dot` itself. If a var is given, this may either be a remote call or a map field access. Examples are `Hello.wor`, `:hello.wor`, `hello.wor`, `Hello.nested.wor`, `hello.nested.wor`, and `@hello.world`. If `charlist` is empty and `inside_dot` is an alias, then the autocompletion may either be an alias or a remote call.

- `{:dot_arity, inside_dot, charlist}` - the context is a dot arity where `inside_dot` is either a `{:var, charlist}`, `{:alias, charlist}`, `{:module_attribute, charlist}`, `{:unquoted_atom, charlist}` or a `dot` itself. If a var is given, it must be a remote arity. Examples are `Hello.world/`, `:hello.world/`, `hello.world/2`, and `@hello.world/2`

- `{:dot_call, inside_dot, charlist}` - the context is a dot call. This means parentheses or space have been added after the expression. where `inside_dot` is either a `{:var, charlist}`, `{:alias, charlist}`, `{:module_attribute, charlist}`, `{:unquoted_atom, charlist}` or a `dot` itself. If a var is given, it must be a remote call. Examples are `Hello.world(`, `:hello.world(`, `Hello.world`, `hello.world(`, `hello.world`, and `@hello.world(`

- `:expr` - may be any expression. Autocompletion may suggest an alias, local or var

- `{:local_or_var, charlist}` - the context is a variable or a local (import or local) call, such as `hello_wor`

- `{:local_arity, charlist}` - the context is a local (import or local) arity, such as `hello_world/`
- `{:local_call, charlist}` - the context is a local (import or local) call, such as `hello_world(` and `hello_world`
- `{:anonymous_call, inside_caller}` - the context is an anonymous call, such as `fun.(` and `@fun.(`.
- `{:module_attribute, charlist}` - the context is a module attribute, such as `@hello_wor`
- `{:operator, charlist}` - the context is an operator, such as `+` or `==`. Note textual operators, such as `when` do not appear as operators but rather as `:local_or_var`. `@` is never an `:operator` and always a `:module_attribute`
- `{:operator_arity, charlist}` - the context is an operator arity, which is an operator followed by /, such as `+/`, `not/` or `when/`
- `{:operator_call, charlist}` - the context is an operator call, which is an operator followed by space, such as `left +`, `not` or `x when`
- `:none` - no context possible
- `{:sigil, charlist}` - the context is a sigil. It may be either the beginning of a sigil, such as `~` or `~s`, or an operator starting with `~`, such as `~>` and `~>>`
- `{:struct, inside_struct}` - the context is a struct, such as `%`, `%UR` or `%URI`. `inside_struct` can either be a `charlist` in case of a static alias or an expression `{:alias, inside_alias,`

`charlist}, {:module_attribute, charlist}, {:local_or_var, charlist}, {:dot, inside_dot, charlist}`

- `{:unquoted_atom, charlist}` - the context is an unquoted atom. This can be any atom or an atom representing a module

We recommend looking at the test suite of this function for a complete list of examples and their return values.

Limitations

The analysis is based on the current token, by analysing the last line of the input. For example, this code:

```
iex> Code.Fragment.cursor_context("%URI{")
:expr
```

returns `:expr`, which suggests any variable, local function or alias could be used. However, given we are inside a struct, the best suggestion would be a struct field. In such cases, you can use `container_cursor_to_quoted`, which will return the container of the AST the cursor is currently within. You can then analyse this AST to provide completion of field names.

As a consequence of its token-based implementation, this function considers only the last line of the input. This means it will show suggestions inside strings, heredocs, etc, which is intentional as it helps with doctests, references, and more.

lines(string)

(since 1.19.0)

Returns the list of lines in the given string, preserving their line endings.

Only the line endings recognized by the Elixir compiler are considered, namely `\r\n` and `\n`. If you would like the retrieve lines without their line endings, use `String.split(string, ["\r\n", "\n"])`.

Examples

```
iex> Code.Fragment.lines("foo\r\nbar\r\nbaz")
["foo\r\n", "bar\r\n", "baz"]

iex> Code.Fragment.lines("foo\nbar\nbaz")
["foo\n", "bar\n", "baz"]

iex> Code.Fragment.lines("")
[""]
```

surround_context(fragment, position, options \\ [])

(since 1.13.0)

```
@spec surround_context(List.Chars.t(), position(),
cursor opts()) ::
  %{begin: position(), end: position(), context: context} |
:none
when context:
       {:alias, charlist()}
       | {:alias, inside_alias, charlist()}
       | {:dot, inside_dot, charlist()}
       | {:local_or_var, charlist()}
       | {:local_arity, charlist()}
       | {:local_call, charlist()}
       | {:module_attribute, charlist()}
       | {:operator, charlist()}
       | {:sigil, charlist()}
       | {:struct, inside_struct}
```

```
        | {:unquoted_atom, charlist()}
        | {:keyword, charlist()}
        | {:key, charlist()}
        | {:capture_arg, charlist()},
      inside_dot:
        {:alias, charlist()}
        | {:alias, inside_alias, charlist()}
        | {:dot, inside_dot, charlist()}
        | {:module_attribute, charlist()}
        | {:unquoted_atom, charlist()}
        | {:var, charlist()}
        | :expr,
      inside_alias: {:local_or_var, charlist()} |
{:module_attribute, charlist()},
      inside_struct:
        charlist()
        | {:alias, inside_alias, charlist()}
        | {:local_or_var, charlist()}
        | {:module_attribute, charlist()}
        | {:dot, inside_dot, charlist()}
```

Receives a string and returns the surround context.

This function receives a string with an Elixir code fragment and a `position`. It returns a map containing the beginning and ending of the identifier alongside its context, or `:none` if there is nothing with a known context. This is useful to provide mouse-over and highlight functionality in editors.

The difference between `cursor_context/2` and `surround_context/3` is that the former assumes the expression in the code fragment is incomplete. For example, `do` in `cursor_context/2` may be a keyword or a variable or a local call, while `surround_context/3` assumes the expression in the code fragment is complete, therefore `do` would always be a keyword.

The `position` contains both the `line` and `column`, both starting with the index of 1. The column must precede the surrounding expression. For example, the expression `foo`, will return something for the columns 1, 2, and 3, but not 4:

```
foo
^ column 1

foo
 ^ column 2

foo
  ^ column 3

foo
   ^ column 4
```

The returned map contains the column the expression starts and the first column after the expression ends.

Similar to `cursor_context/2`, this function is also token-based and may not be accurate under all circumstances. See the "Return values" and "Limitations" section under `cursor_context/2` for more information.

Examples

```
iex> Code.Fragment.surround_context("foo", {1, 1})
%{begin: {1, 1}, context: {:local_or_var, ~c"foo"}, end:
{1, 4}}
```

Differences to `cursor_context/2`

Because `surround_context/3` attempts to capture complex expressions, it has some differences to `cursor_context/2`:

- `dot_call/dot_arity` and `operator_call/operator_arity` are collapsed into `dot` and `operator` contexts respectively as there aren't any meaningful distinctions between them
- On the other hand, this function still makes a distinction between `local_call/local_arity` and `local_or_var`, since the latter can be a local or variable
- `@` when not followed by any identifier is returned as `{:operator, ~c"@"}` (in contrast to `{:module_attribute, ~c""}` in `cursor_context/2`
- This function never returns empty sigils `{:sigil, ~c""}` or empty structs `{:struct, ~c""}` as context
- This function returns keywords as `{:keyword, ~c"do"}`
- This function never returns `:expr`

We recommend looking at the test suite of this function for a complete list of examples and their return values.

Kernel.ParallelCompiler

A module responsible for compiling and requiring files in parallel.

Summary

Types

compile_opts()

Options for parallel compilation functions.

error()
info()
require_opts()

Options for requiring files in parallel.

warning()

Functions

async(fun) deprecated

Starts a task for parallel compilation.

compile(files, options \\ [])

Compiles the given files.

compile_to_path(files, path, options \\ [])

Compiles the given files and writes resulting BEAM files into path.

pmap(collection, fun)

Perform parallel compilation of `collection` with `fun`.

require(files, options \\ [])

Requires the given files in parallel.

Types

compile_opts()

```
@type compile_opts() :: [
  after_compile: (-> term()),
  each_file: (Path.t() -> term()),
  each_long_compilation: (Path.t() -> term()) | (Path.t(),
pid() -> term()),
  each_long_verification: (module() -> term()) | (module(),
pid() -> term()),
  each_module: (Path.t(), module(), binary() -> term()),
  each_cycle: (-> {:compile, [module()],
[Code.diagnostic(:warning)]}
                  | {:runtime, [module()],
[Code.diagnostic(:warning)]}),
  long_compilation_threshold: pos_integer(),
  long_verification_threshold: pos_integer(),
  verification: boolean(),
  profile: :time,
  dest: Path.t(),
  beam_timestamp: term(),
  return_diagnostics: boolean(),
  max_concurrency: pos_integer(),
  purge_compiler_modules: boolean()
]
```

Options for parallel compilation functions.

error()

```
@type error() :: {file :: Path.t(), Code.position(), message
:: String.t()}
```

info()

```
@type info() :: %{
  runtime_warnings: [Code.diagnostic(:warning)],
  compile_warnings: [Code.diagnostic(:warning)]
}
```

require_opts()

```
@type require_opts() :: [
  each_file: (Path.t() -> term()),
  each_module: (Path.t(), module(), binary() -> term()),
  max_concurrency: pos_integer(),
  return_diagnostics: boolean()
]
```

Options for requiring files in parallel.

warning()

```
@type warning() :: {file :: Path.t(), Code.position(),
message :: String.t()}
```

Functions

async(fun)

This function is deprecated. Use `pmap/2` instead.

Starts a task for parallel compilation.

compile(files, options \\ [])

(since 1.6.0)

```
@spec compile([Path.t()], compile_opts()) ::
  {:ok, [atom()], [warning()] | info()}
  | {:error, [error()] | [Code.diagnostic(:error)],
[warning()] | info()}
```

Compiles the given files.

Those files are compiled in parallel and can automatically detect dependencies between them. Once a dependency is found, the current file stops being compiled until the dependency is resolved.

It must be invoked with `return_diagnostics: true` as option, so it returns `{:ok, modules, warnings_info}` or `{:error, errors, warnings_info}`, where `warnings_info` has the shape:

```
%{
  runtime_warnings: [warning],
  compile_warnings: [warning]
}
```

Options

- `:after_compile` - invoked after all modules are compiled, but before they are verified. If the files are being written to disk, such as in `compile_to_path/3`, this will be invoked after the files are written
- `:each_file` - for each file compiled, invokes the callback passing the file
- `:each_long_compilation` - for each file that takes more than a given timeout (see the `:long_compilation_threshold` option) to compile, invoke this callback passing the file as its argument (and optionally the PID of the process compiling the file)
- `:each_long_verification` (since v1.19.0) - for each file that takes more than a given timeout (see the `:long_verification_threshold` option) to compile, invoke this callback passing the module as its argument (and optionally the PID of the process verifying the module)
- `:each_module` - for each module compiled, invokes the callback passing the file, module and the module bytecode
- `:each_cycle` - after the given files are compiled, invokes this function that should return the following values:
  - `{:compile, modules, warnings}` - to continue compilation with a list of further modules to compile

    - `{:runtime, modules, warnings}` - to stop compilation and verify the list of modules because dependent modules have changed

- `:long_compilation_threshold` - the timeout (in seconds) to check for files taking too long to compile. For each file that exceeds the threshold, the `:each_long_compilation` callback is invoked. Defaults to `10` seconds.

- `:long_verification_threshold` (since v1.19.0) - the timeout (in seconds) to check for modules taking too long to compile. For each module that exceeds the threshold, the `:each_long_verification` callback is invoked. Defaults to `10` seconds.

- `:verification` (since v1.19.0) - if code verification, such as unused functions, deprecation warnings, and type checking should run. Defaults to `true`. We recommend disabling it only for debugging purposes.

- `:profile` - if set to `:time`, measure the compilation time of each compilation cycle, each module type check, and group pass checker

- `:purge_compiler_modules` - if set to `true`, automatically purge compilation modules after compilation (see `Code.purge_compiler_modules/0`)

- `:dest` - the destination directory for the BEAM files. When using `compile/2`, this information is only used to properly annotate the BEAM files before they are loaded into memory. If you want a file to actually be written to `dest`, use `compile_to_path/3` instead.

- `:beam_timestamp` - the modification timestamp to give all BEAM files

- `:return_diagnostics` (since v1.15.0) - returns maps with information instead of a list of warnings and returns diagnostics as maps instead of tuples. This option must be set to true, except for backwards compatibility reasons.

- `:max_concurrency` - the maximum number of files to compile in parallel. Setting this option to 1 will compile files sequentially. Defaults to the number of schedulers online, or at least `2`.

compile_to_path(files, path, options \\ [])

(since 1.6.0)

```
@spec compile_to_path([Path.t()], Path.t(), compile opts())
::
  {:ok, [atom()], [warning()] | info()}
  | {:error, [error()] | [Code.diagnostic(:error)],
[warning()] | info()}
```

Compiles the given files and writes resulting BEAM files into path.

See `compile/2` for more information.

pmap(collection, fun)

(since 1.16.0)

Perform parallel compilation of `collection` with `fun`.

If you have a file that needs to compile other modules in parallel, the spawned processes need to be aware of the compiler environment. This function allows a developer to perform such tasks.

require(files, options \\ [])

(since 1.6.0)

```
@spec require([Path.t()], require opts()) ::
  {:ok, [atom()], [warning()] | info()}
  | {:error, [error()] | [Code.diagnostic(:error)],
[warning()] | info()}
```

Requires the given files in parallel.

Opposite to compile, dependencies are not attempted to be automatically solved between files.

It must be invoked with `return_diagnostics: true` as option, so it returns `{:ok, modules, warnings_info}` or `{:error, errors, warnings_info}`, where `warnings_info` has the shape:

```
%{
  runtime_warnings: [warning],
  compile_warnings: [warning]
}
```

Options

- `:each_file` - for each file compiled, invokes the callback passing the file
- `:each_module` - for each module compiled, invokes the callback passing the file, module and the module bytecode
- `:max_concurrency` - the maximum number of files to compile in parallel. Setting this option to 1 will compile files sequentially. Defaults to the number of schedulers online, or at least `2`.

- `:return_diagnostics` (since v1.15.0) - returns maps with information instead of a list of warnings and returns diagnostics as maps instead of tuples. This option must be set to true, except for backwards compatibility reasons.

Macro

Functions for manipulating AST and implementing macros.

Macros are compile-time constructs that receive Elixir's AST as input and return Elixir's AST as output.

Many of the functions in this module exist precisely to work with Elixir AST, to traverse, query, and transform it.

Let's see a simple example that shows the difference between functions and macros:

```
defmodule Example do
  defmacro macro_inspect(value) do
    IO.inspect(value)
    value
  end

  def fun_inspect(value) do
    IO.inspect(value)
    value
  end
end
```

Now let's give it a try:

```
import Example

macro_inspect(1)
#=> 1
#=> 1
```

```
fun_inspect(1)
#=> 1
#=> 1
```

So far they behave the same, as we are passing an integer as argument. But let's see what happens when we pass an expression:

```
macro_inspect(1 + 2)
#=> {:+, [line: 3], [1, 2]}
#=> 3

fun_inspect(1 + 2)
#=> 3
#=> 3
```

The macro receives the representation of the code given as argument, while a function receives the result of the code given as argument. A macro must return a superset of the code representation. See `input/0` and `output/0` for more information.

To learn more about Elixir's AST and how to build them programmatically, see `quote/2`.

> **Evaluating code**
>
> The functions in this module do not evaluate code. In fact, evaluating code from macros is often an anti-pattern. For code evaluation, see the `Code` module.

Summary

Types

captured_remote_function()

A captured remote function in the format of &Mod.fun/arity

escape_opts()
input()

The inputs of a macro

inspect_atom_opts()
metadata()

A keyword list of AST metadata.

output()

The output of a macro

t()

Abstract Syntax Tree (AST)

Functions

camelize(string)

Converts the given string to CamelCase format.

classify_atom(atom)

Classifies an `atom` based on its possible AST placement.

compile_apply(mod, fun, args, caller)

Applies a `mod`, `function`, and `args` at compile-time in `caller`.

dbg(code, options, env)

Default backend for `Kernel.dbg/2`.

decompose_call(ast)

Decomposes a local or remote call into its remote part (when provided), function name and argument list.

escape(expr, opts \\ [])

Recursively escapes a value so it can be inserted into a syntax tree.

expand(ast, env)

Receives an AST node and expands it until it can no longer be expanded.

expand_literals(ast, env)

Expands all literals in `ast` with the given `env`.

expand_literals(ast, acc, fun)

Expands all literals in `ast` with the given `acc` and `fun`.

expand_once(ast, env)

Receives an AST node and expands it once.

generate_arguments(amount, context)

Generates AST nodes for a given number of required argument variables using `Macro.var/2`.

generate_unique_arguments(amount, context)

Generates AST nodes for a given number of required argument variables using `Macro.unique_var/2`.

inspect_atom(source_format, atom, opts \\ [])

Inspects `atom` according to different source formats.

operator?(name, arity)

Returns `true` if the given name and arity is an operator.

path(ast, fun)

Returns the path to the node in `ast` for which `fun` returns a truthy value.

pipe(expr, call_args, position)

Pipes `expr` into the `call_args` at the given `position`.

postwalk(ast, fun)

This function behaves like `prewalk/2`, but performs a depth-first, post-order traversal of quoted expressions.

postwalk(ast, acc, fun)

This functions behaves like `prewalk/3`, but performs a depth-first, post-order traversal of quoted expressions using an accumulator.

postwalker(ast)

Returns an enumerable that traverses the `ast` in depth-first, post-order traversal.

prewalk(ast, fun)

Performs a depth-first, pre-order traversal of quoted expressions.

prewalk(ast, acc, fun)

Performs a depth-first, pre-order traversal of quoted expressions using an accumulator.
prewalker(ast)

Returns an enumerable that traverses the `ast` in depth-first, pre-order traversal.

quoted_literal?(term)

Returns `true` if the given quoted expression represents a quoted literal.

special_form?(name, arity)

Returns `true` if the given name and arity is a special form.

struct!(module, env) deprecated
struct_info!(module, env)

Extracts the struct information.

to_string(tree)

Converts the given expression AST to a string.

to_string(tree, fun) deprecated

Converts the given expression AST to a string.

traverse(ast, acc, pre, post)

Performs a depth-first traversal of quoted expressions using an accumulator.

underscore(atom_or_string)

Converts the given argument to a string with the underscore-slash format.

unescape_string(string)

Unescapes characters in a string.

unescape_string(string, map)

Unescapes characters in a string according to the given mapping.

unique_var(var, context)

Generates an AST node representing a unique variable given by the atoms `var` and `context`.

unpipe(expr)

Breaks a pipeline expression into a list.

update_meta(quoted, fun)

Applies the given function to the node metadata if it contains one.

validate(expr)

Validates the given expressions are valid quoted expressions.

var(var, context)

Generates an AST node representing the variable given by the atoms `var` and `context`.

Types

captured_remote_function()

```
@type captured_remote_function() :: fun()
```

A captured remote function in the format of &Mod.fun/arity

escape_opts()

```
@type escape_opts() :: [
  unquote: boolean(),
  prune_metadata: boolean(),
  generated: boolean()
]
```

input()

```
@type input() ::
  input_expr() | {input(), input()} | [input()] | atom() |
number() | binary()
```

The inputs of a macro

inspect_atom_opts()

```
@type inspect_atom_opts() :: [{:escape, (binary(), char() ->
binary())}]
```

metadata()

```
@type metadata() :: keyword()
```

A keyword list of AST metadata.

The metadata in Elixir AST is a keyword list of values. Any key can be used and different parts of the compiler may use different keys. For example, the AST received by a macro will always include the `:line` annotation, while the AST emitted by `quote/2` will only have the `:line` annotation if the `:line` option is provided.

The following metadata keys are public:

- `:context` - Defines the context in which the AST was generated. For example, `quote/2` will include the module calling `quote/2` as the context. This is often used to distinguish regular code from code generated by a macro or by `quote/2`.

- `:counter` - The variable counter used for variable hygiene. In terms of the compiler, each variable is identified by the combination of either `name` and `metadata[:counter]`, or `name` and `context`.

- `:from_brackets` - Used to determine whether a call to `Access.get/3` is from bracket syntax.

- `:from_interpolation` - Used to determine whether a call to `Kernel.to_string/1` is from interpolation.

- `:generated` - Whether the code should be considered as generated by the compiler or not. This means the compiler and tools like Dialyzer may not emit certain warnings.

- `:if_undefined` - How to expand a variable that is undefined. Set it to `:apply` if you want a variable to become a nullary call without warning or `:raise`

- `:keep` - Used by `quote/2` with the option `location: :keep` to annotate the file and the line number of the quoted source.

- `:line` - The line number of the AST node. Note line information is discarded from quoted code but can be enabled back via the `:line` option.

The following metadata keys are enabled by `Code.string_to_quoted/2`:

- `:assoc` - contains metadata about the `=>` operator location in a map key-value AST node (when `:token_metadata` is true). This entry appears on map key nodes only

- `:closing` - contains metadata about the closing pair, such as a `}` in a tuple or in a map, or such as the closing `)` in a function call with parens (when `:token_metadata` is true). If the function call has a do-end block attached to it, its metadata is found under the `:do` and `:end` metadata

- `:column` - the column number of the AST node (when `:columns` is true). Note column information is always discarded from quoted code.

- `:delimiter` - contains the opening delimiter for sigils, strings, and charlists as a string (such as `"{"`, `"/"`, `"'"`, and the like)

- `:do` - contains metadata about the `do` location in a function call with `do-end` blocks (when `:token_metadata` is true)

- `:end` - contains metadata about the `end` location in a function call with `do-end` blocks (when `:token_metadata` is true)

- `:end_of_expression` - denotes when the end of expression effectively happens (when `:token_metadata` is true). This is only available for expressions inside "blocks of code", which are either direct children of a `__block__` or the right side of `->`. The last

expression of the block does not have metadata if it is not followed by an end of line character (either a newline or `;`). This entry may appear multiple times in the same metadata if the expression is surround by parens

- `:format` - set to `:keyword` when an atom is defined as a keyword. It may also be set to `:atom` to distinguish `nil`, `false`, and `true`
- `:indentation` - indentation of a sigil heredoc
- `:parens` - denotes a node was surrounded by parens for grouping. This entry may appear multiple times in the same metadata if multiple pairs are used for grouping

The following metadata keys are private:

- `:alias` - Used for alias hygiene.
- `:ambiguous_op` - Used for improved error messages in the compiler.
- `:imports` - Used for import hygiene.
- `:var` - Used for improved error messages on undefined variables.

Do not rely on them as they may change or be fully removed in future versions of the language. They are often used by `quote/2` and the compiler to provide features like hygiene, better error messages, and so forth.

If you introduce custom keys into the AST metadata, please make sure to prefix them with the name of your library or application, so that they will not conflict with keys that could potentially be introduced by the compiler in the future.

output()

```
@type output() ::
  output_expr()
  | {output(), output()}
  | [output()]
  | atom()
  | number()
  | binary()
  | captured remote function()
  | pid()
```

The output of a macro

t()

```
@type t() :: input()
```

Abstract Syntax Tree (AST)

Functions

camelize(string)

```
@spec camelize(String.t()) :: String.t()
```

Converts the given string to CamelCase format.

This function was designed to camelize language identifiers/tokens, that's why it belongs to the `Macro` module. Do not use it as a general mechanism for camelizing strings as it does not support Unicode or characters that are not valid in Elixir identifiers.

Examples

```
iex> Macro.camelize("foo_bar")
"FooBar"

iex> Macro.camelize("foo/bar")
"Foo.Bar"
```

If uppercase characters are present, they are not modified in any way as a mechanism to preserve acronyms:

```
iex> Macro.camelize("API.V1")
"API.V1"
iex> Macro.camelize("API_SPEC")
"API_SPEC"
```

classify_atom(atom)

(since 1.14.0)

```
@spec classify_atom(atom()) :: :alias | :identifier | :quoted
| :unquoted
```

Classifies an `atom` based on its possible AST placement.

It returns one of the following atoms:

- `:alias` - the atom represents an alias
- `:identifier` - the atom can be used as a variable or local function call (as well as be an unquoted atom)
- `:unquoted` - the atom can be used in its unquoted form, includes operators and atoms with `@` in them

- `:quoted` - all other atoms which can only be used in their quoted form

Most operators are going to be `:unquoted`, such as `:+`, with some exceptions returning `:quoted` due to ambiguity, such as `:"::"`. Use `operator?/2` to check if a given atom is an operator.

Examples

```
iex> Macro.classify_atom(:foo)
:identifier
iex> Macro.classify_atom(Foo)
:alias
iex> Macro.classify_atom(:foo@bar)
:unquoted
iex> Macro.classify_atom(:+)
:unquoted
iex> Macro.classify_atom(:Foo)
:unquoted
iex> Macro.classify_atom(:"with spaces")
:quoted
```

compile_apply(mod, fun, args, caller)

(since 1.16.0)

Applies a `mod`, `function`, and `args` at compile-time in `caller`.

This is used when you want to dynamically invoke a function at compile-time and force it to be tracked as a compile-time dependency. For example, this is used by `dbg/1` to force the `dbg_callback` configuration to be a compile-time dependency.

If you want to "invoke" a macro instead, remember macros are by definition compile-time, and you can use `Macro.expand/2`.

dbg(code, options, env)

(since 1.14.0)

```
@spec dbg(t(), t(), Macro.Env.t()) :: t()
```

Default backend for `Kernel.dbg/2`.

This function provides a default backend for `Kernel.dbg/2`. See the `Kernel.dbg/2` documentation for more information.

This function:

- prints information about the given `env`
- prints information about `code` and its returned value (using `opts` to inspect terms)
- returns the value returned by evaluating `code`

You can call this function directly to build `Kernel.dbg/2` backends that fall back to this function.

This function raises if the context of the given `env` is `:match` or `:guard`.

decompose_call(ast)

```
@spec decompose_call(t()) :: {atom(), [t()]} | {t(), atom(), [t()]} | :error
```

Decomposes a local or remote call into its remote part (when provided), function name and argument list.

Returns `:error` when an invalid call syntax is provided.

Examples

```
iex> Macro.decompose_call(quote(do: foo))
{:foo, []}

iex> Macro.decompose_call(quote(do: foo()))
{:foo, []}

iex> Macro.decompose_call(quote(do: foo(1, 2, 3)))
{:foo, [1, 2, 3]}

iex> Macro.decompose_call(quote(do: Elixir.M.foo(1, 2,
3)))
{{:__aliases__, [], [:Elixir, :M]}, :foo, [1, 2, 3]}

iex> Macro.decompose_call(quote(do: 42))
:error

iex> Macro.decompose_call(quote(do: {:foo, [], []}))
:error
```

escape(expr, opts \\ [])

```
@spec escape(term(), escape_opts()) :: t()
```

Recursively escapes a value so it can be inserted into a syntax tree.

Examples

```
iex> Macro.escape(:foo)
:foo

iex> Macro.escape({:a, :b, :c})
{:{}, [], [:a, :b, :c]}

iex> Macro.escape({:unquote, [], [1]}, unquote: true)
1
```

Options

- `:unquote` - when `true`, this function leaves `unquote/1` and `unquote_splicing/1` expressions unescaped, effectively unquoting the contents on escape. This option is useful only when escaping ASTs which may have quoted fragments in them. Note this option will give a special meaning to `quote`/`unquote` nodes, which need to be valid AST before escaping. Defaults to `false`.

- `:prune_metadata` - when `true`, removes most metadata from escaped AST nodes. Note this option changes the semantics of escaped code and it should only be used when escaping ASTs. Defaults to `false`.

- `:generated` - (since v1.19.0) Whether the AST should be considered as generated by the compiler or not. This means the compiler and tools like Dialyzer may not emit certain warnings.

  As an example for `:prune_metadata`, `ExUnit` stores the AST of every assertion, so when an assertion fails we can show code snippets to users. Without this option, each time the test module is compiled, we would get a different MD5 of the module bytecode, because the AST contains metadata, such as counters, specific to the compilation environment. By pruning the metadata, we ensure that

the module is deterministic and reduce the amount of data `ExUnit` needs to keep around. Only the minimal amount of metadata is kept, such as `:line`, `:no_parens` and `:delimiter`.

Comparison to `quote/2`

The `escape/2` function is sometimes confused with `quote/2`, because the above examples behave the same with both. The key difference is best illustrated when the value to escape is stored in a variable.

```
iex> Macro.escape({:a, :b, :c})
{:{}, [], [:a, :b, :c]}
iex> quote do: {:a, :b, :c}
{:{}, [], [:a, :b, :c]}

iex> value = {:a, :b, :c}
iex> Macro.escape(value)
{:{}, [], [:a, :b, :c]}

iex> quote do: value
{:value, [], __MODULE__}

iex> value = {:a, :b, :c}
iex> quote do: unquote(value)
** (ArgumentError) tried to unquote invalid AST: {:a, :b, :c}
Did you forget to escape term using Macro.escape/1?
```

`escape/2` is used to escape *values* (either directly passed or variable bound), while `quote/2` produces syntax trees for expressions.

Dealing with references and other runtime values

Macros work at compile-time and therefore `Macro.escape/1` can only escape values that are valid during compilation, such as numbers, atoms, tuples, maps, binaries, etc.

However, you may have values at compile-time which cannot be escaped, such as `reference`s and `pid`s, since the process or memory address they point to will no longer exist once compilation completes. Attempting to escape said values will raise an exception. This is a common issue when working with NIFs.

Luckily, Elixir v1.19 introduces a mechanism that allows those values to be escaped, as long as they are encapsulated by a struct within a module that defines the `__escape__/1` function. This is possible as long as the reference has a natural text or binary representation that can be serialized during compilation.

Let's imagine we have the following struct:

```
defmodule WrapperStruct do
  defstruct [:ref]

  def new(...), do: %WrapperStruct{ref: ...}

  # efficiently dump to / load from binaries
  def dump_to_binary(%WrapperStruct{ref: ref}), do: ...
  def load_from_binary(binary), do: %WrapperStruct{ref:
...}
end
```

Such a struct could not be used in module attributes or escaped with `Macro.escape/2`:

```
defmodule Foo do
  @my_struct WrapperStruct.new(...)
```

```
  def my_struct, do: @my_struct
end

** (ArgumentError) cannot inject attribute @my_struct
into function/macro because cannot escape #Reference<...>
```

To address this, structs can re-define how they should be escaped by defining a custom `__escape__/1` function which returns the AST. In our example:

```
defmodule WrapperStruct do
  # ...

  def __escape__(struct) do
    # dump to a binary representation at compile-time
    binary = dump_to_binary(struct)
    quote do
      # load from the binary representation at runtime

WrapperStruct.load_from_binary(unquote(Macro.escape(binar
y)))
    end
  end
end
```

Now, our example above will be expanded as:

```
def my_struct, do:
WrapperStruct.load_from_binary(<<...>>)
```

When implementing `__escape__/1`, you must ensure that the quoted expression will evaluate to a struct that represents the one given as

argument.

expand(ast, env)

```
@spec expand(input(), Macro.Env.t()) :: output()
```

Receives an AST node and expands it until it can no longer be expanded.

Note this function does not traverse the AST, only the root node is expanded.

This function uses `expand_once/2` under the hood. Check it out for more information and examples.

expand_literals(ast, env)

(since 1.14.1)

```
@spec expand_literals(input(), Macro.Env.t()) :: output()
```

Expands all literals in `ast` with the given `env`.

This function is mostly used to remove compile-time dependencies from AST nodes. In such cases, the given environment is usually manipulated to represent a function:

```
Macro.expand_literals(ast, %{env | function: {:my_code,
1}})
```

At the moment, the only expandable literal nodes in an AST are aliases, so this function only expands aliases (and it does so anywhere in a literal).

However, be careful when removing compile-time dependencies between modules. If you remove them but you still invoke the module at compile-time, Elixir will be unable to properly recompile modules when they change.

expand_literals(ast, acc, fun)

(since 1.14.1)

```
@spec expand_literals(t(), acc, (t(), acc -> {t(), acc})) ::
{t(), acc}
when acc: term()
```

Expands all literals in `ast` with the given `acc` and `fun`.

`fun` will be invoked with an expandable AST node and `acc` and must return a new node with `acc`. This is a general version of `expand_literals/2` which supports a custom expansion function. Please check `expand_literals/2` for use cases and pitfalls.

expand_once(ast, env)

```
@spec expand_once(input(), Macro.Env.t()) :: output()
```

Receives an AST node and expands it once.

The following contents are expanded:

- Macros (local or remote)
- Aliases are expanded (if possible) and return atoms
- Compilation environment macros (`__CALLER__/0`, `__DIR__/0`, `__ENV__/0` and `__MODULE__/0`)
- Module attributes reader (`@foo`)

If the expression cannot be expanded, it returns the expression itself. This function does not traverse the AST, only the root node is expanded. The expansion happens as if it was expanded by the Elixir compiler and therefore compilation tracers will be invoked and deprecation warnings will be emitted during the expansion.

`expand_once/2` performs the expansion just once. Check `expand/2` to perform expansion until the node can no longer be expanded.

Examples

In the example below, we have a macro that generates a module with a function named `name_length` that returns the length of the module name. The value of this function will be calculated at compilation time and not at runtime.

Consider the implementation below:

```
defmacro defmodule_with_length(name, do: block) do
  length = length(Atom.to_charlist(name))

  quote do
    defmodule unquote(name) do
      def name_length, do: unquote(length)
      unquote(block)
    end
  end
end
```

When invoked like this:

```
defmodule_with_length My.Module do
  def other_function, do: ...
```

```
end
```

The compilation will fail because `My.Module` when quoted is not an atom, but a syntax tree as follows:

```
{:__aliases__, [], [:My, :Module]}
```

That said, we need to expand the aliases node above to an atom, so we can retrieve its length. Expanding the node is not straightforward because we also need to expand the caller aliases. For example:

```
alias MyHelpers, as: My

defmodule_with_length My.Module do
  def other_function, do: ...
end
```

The final module name will be `MyHelpers.Module` and not `My.Module`. With `Macro.expand/2`, such aliases are taken into consideration. Local and remote macros are also expanded. We could rewrite our macro above to use this function as:

```
defmacro defmodule_with_length(name, do: block) do
  expanded = Macro.expand(name, __CALLER__)
  length = length(Atom.to_charlist(expanded))

  quote do
    defmodule unquote(name) do
      def name_length, do: unquote(length)
      unquote(block)
    end
```

```
  end
end
```

generate_arguments(amount, context)

(since 1.5.0)

```
@spec generate_arguments(0, context :: atom()) :: []
@spec generate_arguments(pos_integer(), context) :: [{atom(),
[], context}, ...]
when context: atom()
```

Generates AST nodes for a given number of required argument variables using `Macro.var/2`.

Note the arguments are not unique. If you later on want to access the same variables, you can invoke this function with the same inputs. Use `generate_unique_arguments/2` to generate unique arguments that can't be overridden.

Examples

```
iex> Macro.generate_arguments(2, __MODULE__)
[{:arg1, [], __MODULE__}, {:arg2, [], __MODULE__}]
```

generate_unique_arguments(amount, context)

(since 1.11.3)

```
@spec generate_unique_arguments(0, context :: atom()) :: []
```

```
@spec generate_unique_arguments(pos integer(), context) :: [
  {atom(), [{:counter, integer()}], context},
  ...
]
when context: atom()
```

Generates AST nodes for a given number of required argument variables using `Macro.unique_var/2`.

The second argument is generally the macro caller's module.

Examples

```
[var1, var2] = Macro.generate_unique_arguments(2,
__CALLER__.module)
```

inspect_atom(source_format, atom, opts \\ [])

(since 1.14.0)

```
@spec inspect_atom(:literal | :key | :remote_call, atom(),
inspect atom opts()) ::
  binary()
```

Inspects `atom` according to different source formats.

The atom can be inspected according to the three different formats it appears in the AST: as a literal (`:literal`), as a key (`:key`), or as the function name of a remote call (`:remote_call`).

Options

- `:escape` - a two-arity function used to escape a quoted atom content, if necessary. The function receives the atom content as

string and a quote delimiter character, which should always be escaped. By default the content is escaped such that the inspected sequence would be parsed as the given atom.

Examples

As a literal

Literals include regular atoms, quoted atoms, operators, aliases, and the special `nil`, `true`, and `false` atoms.

```
iex> Macro.inspect_atom(:literal, nil)
"nil"
iex> Macro.inspect_atom(:literal, :foo)
":foo"
iex> Macro.inspect_atom(:literal, :<>)
":<>"
iex> Macro.inspect_atom(:literal, :Foo)
":Foo"
iex> Macro.inspect_atom(:literal, Foo.Bar)
"Foo.Bar"
iex> Macro.inspect_atom(:literal, :"with spaces")
":\"with spaces\""
```

As a key

Inspect an atom as a key of a keyword list or a map.

```
iex> Macro.inspect_atom(:key, :foo)
"foo:"
iex> Macro.inspect_atom(:key, :<>)
"<>:"
iex> Macro.inspect_atom(:key, :Foo)
```

```
"Foo:"
iex> Macro.inspect_atom(:key, :"with spaces")
"\"with spaces\":"
```

As a remote call

Inspect an atom the function name of a remote call.

```
iex> Macro.inspect_atom(:remote_call, :foo)
"foo"
iex> Macro.inspect_atom(:remote_call, :<>)
"<>"
iex> Macro.inspect_atom(:remote_call, :Foo)
"\"Foo\""
iex> Macro.inspect_atom(:remote_call, :"with spaces")
"\"with spaces\""
```

operator?(name, arity)

(since 1.7.0)

```
@spec operator?(name :: atom(), arity()) :: boolean()
```

Returns `true` if the given name and arity is an operator.

Examples

```
iex> Macro.operator?(:not_an_operator, 3)
false
iex> Macro.operator?(:.., 0)
true
iex> Macro.operator?(:+, 1)
```

```
true
iex> Macro.operator?(:++, 2)
true
iex> Macro.operator?(:..//, 3)
true
```

path(ast, fun)

(since 1.14.0)

```
@spec path(t(), (t() -> as boolean(term()))) :: [t()] | nil
```

Returns the path to the node in `ast` for which `fun` returns a truthy value.

The path is a list, starting with the node in which `fun` returns a truthy value, followed by all of its parents.

Returns `nil` if `fun` returns only falsy values.

Computing the path can be an efficient operation when you want to find a particular node in the AST within its context and then assert something about it.

Examples

```
iex> Macro.path(quote(do: [1, 2, 3]), & &1 == 3)
[3, [1, 2, 3]]

iex> Macro.path(quote(do: [1, 2]), & &1 == 5)
nil

iex> Macro.path(quote(do: Foo.bar(3)), & &1 == 3)
[3, quote(do: Foo.bar(3))]
```

```
iex> Macro.path(quote(do: %{foo: [bar: :baz]}), & &1 ==
:baz)
[
  :baz,
  {:bar, :baz},
  [bar: :baz],
  {:foo, [bar: :baz]},
  {:%{}, [], [foo: [bar: :baz]]}
]
```

pipe(expr, call_args, position)

```
@spec pipe(t(), t(), integer()) :: t()
```

Pipes `expr` into the `call_args` at the given `position`.

This function can be used to implement `|>` like functionality. For example, `|>` itself is implemented as:

```
defmacro left |> right do
  Macro.pipe(left, right, 0)
end
```

`expr` is the AST of an expression. `call_args` must be the AST *of a call*, otherwise this function will raise an error. As an example, consider the pipe operator `|>/2`, which uses this function to build pipelines.

Even if the expression is piped into the AST, it doesn't necessarily mean that the AST is valid. For example, you could pipe an argument to `div/2`, effectively turning it into a call to `div/3`, which is a function

that doesn't exist by default. The code will raise unless a `div/3` function is locally defined.

postwalk(ast, fun)

```
@spec postwalk(t(), (t() -> t())) :: t()
```

This function behaves like `prewalk/2`, but performs a depth-first, post-order traversal of quoted expressions.

postwalk(ast, acc, fun)

```
@spec postwalk(t(), any(), (t(), any() -> {t(), any()})) ::
{t(), any()}
```

This functions behaves like `prewalk/3`, but performs a depth-first, post-order traversal of quoted expressions using an accumulator.

postwalker(ast)

(since 1.13.0)

```
@spec postwalker(t()) :: Enumerable.t()
```

Returns an enumerable that traverses the `ast` in depth-first, post-order traversal.

Examples

```
iex> ast = quote do: foo(1, "abc")
iex> Enum.map(Macro.postwalker(ast), & &1)
[1, "abc", {:foo, [], [1, "abc"]}]
```

prewalk(ast, fun)

```
@spec prewalk(t(), (t() -> t())) :: t()
```

Performs a depth-first, pre-order traversal of quoted expressions.

Returns a new AST where each node is the result of invoking `fun` on each corresponding node of `ast`.

Examples

```
iex> ast = quote do: 5 + 3 * 7
iex> {:+, _, [5, {:*, _, [3, 7]}]} = ast
iex> new_ast = Macro.prewalk(ast, fn
...>   {:+, meta, children} -> {:*, meta, children}
...>   {:*, meta, children} -> {:+, meta, children}
...>   other -> other
...> end)
iex> {:*, _, [5, {:+, _, [3, 7]}]} = new_ast
iex> Code.eval_quoted(ast)
{26, []}
iex> Code.eval_quoted(new_ast)
{50, []}
```

prewalk(ast, acc, fun)

```
@spec prewalk(t(), any(), (t(), any() -> {t(), any()})) ::
{t(), any()}
```

Performs a depth-first, pre-order traversal of quoted expressions using an accumulator.

Returns a tuple where the first element is a new AST where each node is the result of invoking `fun` on each corresponding node and the second one is the final accumulator.

Examples

```
iex> ast = quote do: 5 + 3 * 7
iex> {:+, _, [5, {:*, _, [3, 7]}]} = ast
iex> {new_ast, acc} = Macro.prewalk(ast, [], fn
...>   {:+, meta, children}, acc -> {{:*, meta,
children}, [:+ | acc]}
...>   {:*, meta, children}, acc -> {{:+, meta,
children}, [:* | acc]}
...>   other, acc -> {other, acc}
...> end)
iex> {{:*, _, [5, {:+, _, [3, 7]}]}, [:*, :+]} =
{new_ast, acc}
iex> Code.eval_quoted(ast)
{26, []}
iex> Code.eval_quoted(new_ast)
{50, []}
```

prewalker(ast)

(since 1.13.0)

```
@spec prewalker(t()) :: Enumerable.t()
```

Returns an enumerable that traverses the `ast` in depth-first, pre-order traversal.

Examples

```
iex> ast = quote do: foo(1, "abc")
iex> Enum.map(Macro.prewalker(ast), & &1)
[{:foo, [], [1, "abc"]}, 1, "abc"]
```

quoted_literal?(term)

(since 1.7.0)

```
@spec quoted_literal?(t()) :: boolean()
```

Returns `true` if the given quoted expression represents a quoted literal.

Atoms and numbers are always literals. Binaries, lists, tuples, maps, and structs are only literals if all of their terms are also literals.

Examples

```
iex> Macro.quoted_literal?(quote(do: "foo"))
true
iex> Macro.quoted_literal?(quote(do: {"foo", 1}))
true
iex> Macro.quoted_literal?(quote(do: {"foo", 1, :baz}))
true
iex> Macro.quoted_literal?(quote(do: %{foo: "bar"}))
true
iex> Macro.quoted_literal?(quote(do: %URI{path: "/"}))
true
iex> Macro.quoted_literal?(quote(do: URI.parse("/")))
```

```
  false
  iex> Macro.quoted_literal?(quote(do: {foo, var}))
  false
```

special_form?(name, arity)

(since 1.7.0)

```
@spec special_form?(name :: atom(), arity()) :: boolean()
```

Returns `true` if the given name and arity is a special form.

struct!(module, env)

This function is deprecated. Use Macro.struct_info!/2 instead.

struct_info!(module, env)

(since 1.18.0)

```
@spec struct_info!(module(), Macro.Env.t()) :: [
  %{
    :field => atom(),
    optional(:required) => boolean(),
    optional(:default) => term()
  }
]
```

Extracts the struct information.

This is useful when a struct needs to be expanded at compilation time and the struct being expanded may or may not have been compiled (including structs in the defined under the module being compiled). For compiled modules, it will invoke `module.__info__(:struct)`.

Calling this function also adds an export dependency on the given struct.

It will raise `ArgumentError` if the struct is not available.

Compatibility considerations

This function currently returns both `:required` and `:default` entries for each field. While this naming is inconsistent (a required field should not have a default), this is done for backwards compatibility purposes.

In future releases, Elixir may introduce truly required struct fields, the required field will be removed and default will be present only if the field is optional. Your code should prepare for such scenario accordingly.

to_string(tree)

```
@spec to_string(t()) :: String.t()
```

Converts the given expression AST to a string.

This is a convenience function for converting AST into a string, which discards all formatting of the original code and wraps newlines around 98 characters. See `Code.quoted_to_algebra/2` as a lower level function with more control around formatting.

If the AST contains invalid nodes, this function will attempt to inspect them, to aid debugging, although the elements won't be formatted accordingly.

Examples

```
iex> Macro.to_string(quote(do: foo.bar(1, 2, 3)))
"foo.bar(1, 2, 3)"
```

to_string(tree, fun)

This function is deprecated. Use Macro.to_string/1 instead.

```
@spec to_string(t(), (t(), String.t() -> String.t())) ::
String.t()
```

Converts the given expression AST to a string.

The given `fun` is called for every node in the AST with two arguments: the AST of the node being printed and the string representation of that same node. The return value of this function is used as the final string representation for that AST node.

This function discards all formatting of the original code.

Examples

```
Macro.to_string(quote(do: 1 + 2), fn
  1, _string -> "one"
  2, _string -> "two"
  _ast, string -> string
end)
#=> "one + two"
```

traverse(ast, acc, pre, post)

```
@spec traverse(t(), any(), (t(), any() -> {t(), any()}),
(t(), any() -> {t(), any()})) ::
  {t(), any()}
```

Performs a depth-first traversal of quoted expressions using an accumulator.

Returns a tuple where the first element is a new AST and the second one is the final accumulator. The new AST is the result of invoking `pre` on each node of `ast` during the pre-order phase and `post` during the post-order phase.

Examples

```
iex> ast = quote do: 5 + 3 * 7
iex> {:+, _, [5, {:*, _, [3, 7]}]} = ast
iex> {new_ast, acc} =
...>  Macro.traverse(
...>    ast,
...>    [],
...>    fn
...>      {:+, meta, children}, acc -> {{:-, meta,
children}, [:- | acc]}
...>      {:*, meta, children}, acc -> {{:/, meta,
children}, [:/ | acc]}
...>      other, acc -> {other, acc}
...>    end,
...>    fn
...>      {:-, meta, children}, acc -> {{:min, meta,
children}, [:min | acc]}
...>      {:/, meta, children}, acc -> {{:max, meta,
```

```
children}, [:max | acc]}
...>      other, acc -> {other, acc}
...>    end
...>  )
iex> {:min, _, [5, {:max, _, [3, 7]}]} = new_ast
iex> [:min, :max, :/, :-] = acc
iex> Code.eval_quoted(new_ast)
{5, []}
```

underscore(atom_or_string)

```
@spec underscore(atom() | String.t()) :: String.t()
```

Converts the given argument to a string with the underscore-slash format.

The argument must either be an atom, representing an Elixir module, or a string representing a module without the `Elixir.` prefix.

This function was designed to format language identifiers/tokens with the underscore-slash format, that's why it belongs to the `Macro` module. Do not use it as a general mechanism for underscoring strings as it does not support Unicode or characters that are not valid in Elixir identifiers.

Examples

```
iex> Macro.underscore("FooBar")
"foo_bar"

iex> Macro.underscore("Foo.Bar")
"foo/bar"
```

```
iex> Macro.underscore(Foo.Bar)
"foo/bar"
```

In general, `underscore` can be thought of as the reverse of `camelize`, however, in some cases formatting may be lost:

```
iex> Macro.underscore("SAPExample")
"sap_example"

iex> Macro.camelize("sap_example")
"SapExample"

iex> Macro.camelize("hello_10")
"Hello10"

iex> Macro.camelize("foo/bar")
"Foo.Bar"
```

unescape_string(string)

```
@spec unescape_string(String.t()) :: String.t()
```

Unescapes characters in a string.

This is the unescaping behaviour used by default in Elixir single- and double-quoted strings. Check `unescape_string/2` for information on how to customize the escaping map.

In this setup, Elixir will escape the following: `\0`, `\a`, `\b`, `\d`, `\e`, `\f`, `\n`, `\r`, `\s`, `\t` and `\v`. Bytes can be given as hexadecimals via `\xNN` and Unicode code points as `\uNNNN` escapes.

This function is commonly used on sigil implementations (like `~r`, `~s` and others), which receive a raw, unescaped string, and it can be used anywhere that needs to mimic how Elixir parses strings.

Examples

```
iex> Macro.unescape_string("example\\n")
"example\n"
```

In the example above, we pass a string with `\n` escaped and return a version with it unescaped.

unescape_string(string, map)

```
@spec unescape_string(String.t(), (non_neg_integer() ->
non_neg_integer() | false)) ::
  String.t()
```

Unescapes characters in a string according to the given mapping.

Check `unescape_string/1` if you want to use the same mapping as Elixir single- and double-quoted strings.

Mapping function

The mapping function receives an integer representing the code point of the character it wants to unescape. There are also the special atoms `:newline`, `:unicode`, and `:hex`, which control newline, unicode, and escaping respectively.

Here is the default mapping function implemented by Elixir:

```
  def unescape_map(:newline), do: true
  def unescape_map(:unicode), do: true
  def unescape_map(:hex), do: true
  def unescape_map(?0), do: ?0
  def unescape_map(?a), do: ?\a
  def unescape_map(?b), do: ?\b
  def unescape_map(?d), do: ?\d
  def unescape_map(?e), do: ?\e
  def unescape_map(?f), do: ?\f
  def unescape_map(?n), do: ?\n
  def unescape_map(?r), do: ?\r
  def unescape_map(?s), do: ?\s
  def unescape_map(?t), do: ?\t
  def unescape_map(?v), do: ?\v
  def unescape_map(e), do: e
```

If the `unescape_map/1` function returns `false`, the char is not escaped and the backslash is kept in the string.

Examples

Using the `unescape_map/1` function defined above is easy:

```
  Macro.unescape_string("example\\n", &unescape_map(&1))
```

unique_var(var, context)

(since 1.11.3)

```
@spec unique_var(var, context) :: {var, [{:counter,
integer()}], context}
when var: atom(), context: atom()
```

Generates an AST node representing a unique variable given by the atoms `var` and `context`.

Calling this function with the same arguments will generate another variable, with its own unique counter. See `var/2` for an alternative.

The second argument is generally the macro caller's module.

Examples

```
var = Macro.unique_var(:foo, __CALLER__.module)
```

unpipe(expr)

```
@spec unpipe(t()) :: [t()]
```

Breaks a pipeline expression into a list.

The AST for a pipeline (a sequence of applications of `|>/2`) is similar to the AST of a sequence of binary operators or function applications: the top-level expression is the right-most `:|>` (which is the last one to be executed), and its left-hand and right-hand sides are its arguments:

```
quote do: 100 |> div(5) |> div(2)
#=> {:|>, _, [arg1, arg2]}
```

In the example above, the `|>/2` pipe is the right-most pipe; `arg1` is the AST for `100 |> div(5)`, and `arg2` is the AST for `div(2)`.

It's often useful to have the AST for such a pipeline as a list of function applications. This function does exactly that:

```
Macro.unpipe(quote do: 100 |> div(5) |> div(2))
#=> [{100, 0}, {{:div, [], [5]}, 0}, {{:div, [], [2]},
0}]
```

We get a list that follows the pipeline directly: first the `100`, then the `div(5)` (more precisely, its AST), then `div(2)`. The `0` as the second element of the tuples is the position of the previous element in the pipeline inside the current function application: `{{:div, [], [5]}, 0}` means that the previous element (`100`) will be inserted as the 0th (first) argument to the `div/2` function, so that the AST for that function will become `{:div, [], [100, 5]}` (`div(100, 5)`).

update_meta(quoted, fun)

```
@spec update_meta(t(), (keyword() -> keyword())) :: t()
```

Applies the given function to the node metadata if it contains one.

This is often useful when used with `Macro.prewalk/2` to remove information like lines and hygienic counters from the expression for either storage or comparison.

Examples

```
iex> quoted = quote line: 10, do: sample()
{:sample, [line: 10], []}
iex> Macro.update_meta(quoted, &Keyword.delete(&1,
:line))
{:sample, [], []}
```

validate(expr)

```
@spec validate(term()) :: :ok | {:error, term()}
```

Validates the given expressions are valid quoted expressions.

Check the type `Macro.t/0` for a complete specification of a valid quoted expression.

It returns `:ok` if the expression is valid. Otherwise it returns a tuple in the form of `{:error, remainder}` where `remainder` is the invalid part of the quoted expression.

Examples

```
iex> Macro.validate({:two_element, :tuple})
:ok
iex> Macro.validate({:three, :element, :tuple})
{:error, {:three, :element, :tuple}}

iex> Macro.validate([1, 2, 3])
:ok
iex> Macro.validate([1, 2, 3, {4}])
{:error, {4}}
```

var(var, context)

```
@spec var(var, context) :: {var, [], context} when var:
atom(), context: atom()
```

Generates an AST node representing the variable given by the atoms `var` and `context`.

Note this variable is not unique. If you later on want to access this same variable, you can invoke `var/2` again with the same arguments. Use `unique_var/2` to generate a unique variable that can't be overridden.

Examples

In order to build a variable, a context is expected. Most of the times, in order to preserve hygiene, the context must be `__MODULE__/0`:

```
iex> Macro.var(:foo, __MODULE__)
{:foo, [], __MODULE__}
```

However, if there is a need to access the user variable, nil can be given:

```
iex> Macro.var(:foo, nil)
{:foo, [], nil}
```

Macro.Env

A struct that holds compile time environment information.

The current environment can be accessed at any time as `__ENV__/0`. Inside macros, the caller environment can be accessed as `__CALLER__/0`.

The majority of the functions in this module are provided for low-level tools, which need to integrate with the Elixir compiler, such as language servers and embedded languages. For regular usage in Elixir code and macros, you must use the `Macro` module instead. In particular, avoid modifying the `Macro.Env` struct directly and prefer to use high-level constructs, such as a `import`, `aliases`, and so forth to build your own environment. For example, to build a custom environment, you can define a function such as:

```
def make_custom_env do
  import SomeModule, only: [some_function: 2], warn:
false
  alias A.B.C, warn: false
  __ENV__
end
```

Struct fields

The `Macro.Env` struct contains the following fields:

- `context` - the context of the environment; it can be `nil` (default context), `:guard` (inside a guard) or `:match` (inside a match)
- `context_modules` - a list of modules defined in the current context
- `file` - the current absolute file name as a binary

- `function` - a tuple as `{atom, integer}`, where the first element is the function name and the second its arity; returns `nil` if not inside a function
- `line` - the current line as an integer
- `module` - the current module name

The following fields are private to Elixir's macro expansion mechanism and must not be accessed directly:

- `aliases`
- `functions`
- `macro_aliases`
- `macros`
- `lexical_tracker`
- `requires`
- `tracers`
- `versioned_vars`

Summary

Types

context()
context_modules()
define_alias_opts()
define_import_opts()
define_require_opts()
expand_alias_opts()
expand_import_opts()
expand_require_opts()
file()
line()

name_arity()
t()
variable()

Functions

define_alias(env, meta, module, opts \\ [])

Defines the given `as` an alias to `module` in the environment.

define_import(env, meta, module, opts \\ [])

Defines the given `module` as imported in the environment.

define_require(env, meta, module, opts \\ [])

Defines the given `module` as required in the environment.

expand_alias(env, meta, list, opts \\ [])

Expands an alias given by the alias segments.

expand_import(env, meta, name, arity, opts \\ [])

Expands an import given by `name` and `arity`.

expand_require(env, meta, module, name, arity, opts \\ [])

Expands a require given by `module`, `name`, and `arity`.

has_var?(env, var)

Checks if a variable belongs to the environment.

in_guard?(env)

Returns whether the compilation environment is currently inside a guard.

in_match?(env)

Returns whether the compilation environment is currently inside a match clause.

location(env)

Returns a keyword list containing the file and line information as keys.

lookup_alias_as(env, atom)

Returns the names of any aliases for the given module or atom.

lookup_import(env, name_arity)

Returns the modules from which the given `{name, arity}` was imported.

prepend_tracer(env, tracer)

Prepend a tracer to the list of tracers in the environment.

prune_compile_info(env)

Prunes compile information from the environment.

required?(env, module)

Returns `true` if the given module has been required.

stacktrace(env)

Returns the environment stacktrace.

to_guard(env)

Returns an environment in the guard context.

to_match(env)

Returns an environment in the match context.
vars(env)

Returns a list of variables in the current environment.

Types

context()

```
@type context() :: :match | :guard | nil
```

context_modules()

```
@type context_modules() :: [module()]
```

define_alias_opts()

```
@type define_alias_opts() :: [trace: boolean(), as: atom(),
warn: boolean()]
```

define_import_opts()

```
@type define_import_opts() :: [
  trace: boolean(),
  emit_warnings: boolean(),
  info_callback: (atom() -> [{atom(), arity()}]),
```

```
  only: :functions | :macros | [{atom(), arity()}],
  except: [{atom(), arity()}],
  warn: boolean()
]
```

define_require_opts()

```
@type define_require_opts() :: [trace: boolean(), as: atom(),
warn: boolean()]
```

expand_alias_opts()

```
@type expand_alias_opts() :: [{:trace, boolean()}]
```

expand_import_opts()

```
@type expand_import_opts() :: [
  allow_locals: boolean() | (-> function() | false),
  check_deprecations: boolean(),
  trace: boolean()
]
```

expand_require_opts()

```
@type expand_require_opts() :: [check_deprecations:
boolean(), trace: boolean()]
```

file()

```
@type file() :: binary()
```

line()

```
@type line() :: non_neg_integer()
```

name_arity()

```
@type name_arity() :: {atom(), arity()}
```

t()

```
@type t() :: %Macro.Env{
  aliases: aliases(),
  context: context(),
  context_modules: context_modules(),
  file: file(),
  function: name_arity() | nil,
  functions: functions(),
  lexical_tracker: lexical_tracker(),
  line: line(),
  macro_aliases: macro_aliases(),
  macros: macros(),
  module: module(),
  requires: requires(),
```

```
  tracers: tracers(),
  versioned_vars: versioned_vars()
}
```

variable()

```
@type variable() :: {atom(), atom() | term()}
```

Functions

define_alias(env, meta, module, opts \\ [])

(since 1.17.0)

```
@spec define_alias(t(), Macro.metadata(), module(),
define_alias_opts()) ::
  {:ok, t()} | {:error, String.t()}
```

Defines the given `as` an alias to `module` in the environment.

This is used by tools which need to mimic the Elixir compiler. The appropriate `:alias` compiler tracing event will be emitted.

Additional options

It accepts the same options as `Kernel.SpecialForm.alias/2` plus:

- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies

Examples

```
iex> env = __ENV__
iex> Macro.Env.expand_alias(env, [], [:Baz])
:error
iex> {:ok, env} = Macro.Env.define_alias(env, [line: 10],
Foo.Bar, as: Baz)
iex> Macro.Env.expand_alias(env, [], [:Baz])
{:alias, Foo.Bar}
iex> Macro.Env.expand_alias(env, [], [:Baz, :Bat])
{:alias, Foo.Bar.Bat}
```

If no `:as` option is given, the alias will be inferred from the module:

```
iex> env = __ENV__
iex> {:ok, env} = Macro.Env.define_alias(env, [line: 10],
Foo.Bar)
iex> Macro.Env.expand_alias(env, [], [:Bar])
{:alias, Foo.Bar}
```

If it is not possible to infer one, an error is returned:

```
iex> Macro.Env.define_alias(__ENV__, [line: 10],
:an_atom)
{:error,
 "alias cannot be inferred automatically for module:
:an_atom, " <>
   "please use the :as option. Implicit aliasing is only
supported with Elixir modules"}
```

define_import(env, meta, module, opts \\ [])

(since 1.17.0)

```
@spec define_import(t(), Macro.metadata(), module(),
define_import_opts()) ::
  {:ok, t()} | {:error, String.t()}
```

Defines the given `module` as imported in the environment.

It assumes `module` is available. This is used by tools which need to mimic the Elixir compiler. The appropriate `:import` compiler tracing event will be emitted.

Additional options

It accepts the same options as `Kernel.SpecialForm.import/2` plus:

- `:emit_warnings` - emit warnings found when defining imports
- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies
- `:info_callback` - a function to use instead of `Module.__info__/1`. The function will be invoked with `:functions` or `:macros` argument. It has to return a list of `{function, arity}` key value pairs. If it fails, it defaults to using module metadata based on `module_info/1`.

Examples

```
iex> env = __ENV__
iex> Macro.Env.lookup_import(env, {:flatten, 1})
[]
iex> {:ok, env} = Macro.Env.define_import(env, [line:
10], List)
iex> Macro.Env.lookup_import(env, {:flatten, 1})
[{:function, List}]
```

It accepts the same options as `Kernel.SpecialForm.import/2`:

```
iex> env = __ENV__
iex> Macro.Env.lookup_import(env, {:is_odd, 1})
[]
iex> {:ok, env} = Macro.Env.define_import(env, [line:
10], Integer, only: :macros)
iex> Macro.Env.lookup_import(env, {:is_odd, 1})
[{:macro, Integer}]
```

Info callback override

```
iex> env = __ENV__
iex> Macro.Env.lookup_import(env, {:flatten, 1})
[]
iex> {:ok, env} = Macro.Env.define_import(env, [line:
10], SomeModule, [info_callback: fn :functions ->
[{:flatten, 1}]; :macros -> [{:some, 2}]; end])
iex> Macro.Env.lookup_import(env, {:flatten, 1})
[{:function, SomeModule}]
iex> Macro.Env.lookup_import(env, {:some, 2})
[{:macro, SomeModule}]
```

define_require(env, meta, module, opts \\ [])

(since 1.17.0)

```
@spec define_require(t(), Macro.metadata(), module(),
define_require_opts()) ::
  {:ok, t()} | {:error, String.t()}
```

Defines the given `module` as required in the environment.

It does not check or assert the module is available. This is used by tools which need to mimic the Elixir compiler. The appropriate `:require` compiler tracing event will be emitted.

Additional options

It accepts the same options as `Kernel.SpecialForm.require/2` plus:

- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies

Examples

```
iex> env = __ENV__
iex> Macro.Env.required?(env, Integer)
false
iex> {:ok, env} = Macro.Env.define_require(env, [line:
10], Integer)
```

```
iex> Macro.Env.required?(env, Integer)
true
```

If the `:as` option is given, it will also define an alias:

```
iex> env = __ENV__
iex> {:ok, env} = Macro.Env.define_require(env, [line:
10], Foo.Bar, as: Baz)
iex> Macro.Env.expand_alias(env, [], [:Baz])
{:alias, Foo.Bar}
```

expand_alias(env, meta, list, opts \\ [])

(since 1.17.0)

```
@spec expand_alias(t(), keyword(), [atom()],
expand_alias_opts()) ::
  {:alias, atom()} | :error
```

Expands an alias given by the alias segments.

It returns `{:alias, alias}` if the segments is a list of atoms and an alias was found. Returns `:error` otherwise.

This expansion may emit the `:alias_expansion` trace event but it does not emit the `:alias_reference` one.

Options

- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code

expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies

Examples

```
iex> alias List, as: MyList
iex> Macro.Env.expand_alias(__ENV__, [], [:MyList])
{:alias, List}
iex> Macro.Env.expand_alias(__ENV__, [], [:MyList,
:Nested])
{:alias, List.Nested}
```

If there is no alias or the alias starts with `Elixir.` (which disables aliasing), then `:error` is returned:

```
iex> alias List, as: MyList
iex> Macro.Env.expand_alias(__ENV__, [], [:Elixir,
MyList])
:error
iex> Macro.Env.expand_alias(__ENV__, [], [:AnotherList])
:error
```

expand_import(env, meta, name, arity, opts \\ [])

(since 1.17.0)

```
@spec expand_import(t(), keyword(), atom(), arity(),
expand import opts()) ::
  {:macro, module(), (Macro.metadata(), args :: [Macro.t()] -
> Macro.t())}
  | {:function, module(), atom()}
  | {:error, :not_found | {:conflict, module()} |
{:ambiguous, [module()]}}
```

Expands an import given by `name` and `arity`.

If the import points to a macro, it returns a tuple with the module and a function that expands the macro. The function expects the metadata to be attached to the expansion and the arguments of the macro.

If the import points to a function, it returns a tuple with the module and the function name.

If any import is found, the appropriate compiler tracing event will be emitted.

Otherwise returns `{:error, reason}`.

Options

- `:allow_locals` - controls how local macros are resolved. Defaults to `true`.
    - When `false`, does not attempt to capture local macros defined in the current module in `env`
    - When `true`, uses a default resolver that looks for public macros in the current module
    - When a function, it will be invoked to lazily compute a local function (or return false). It has signature `(-> function() | false)`

- `:check_deprecations` - when set to `false`, does not check for deprecations when expanding macros

- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code

expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies

expand_require(env, meta, module, name, arity, opts \\ [])

(since 1.17.0)

```
@spec expand_require(t(), keyword(), module(), atom(),
arity(), expand_require_opts()) ::
  {:macro, module(), (Macro.metadata(), args :: [Macro.t()] -
> Macro.t())}
  | :error
```

Expands a require given by `module`, `name`, and `arity`.

If the require points to a macro and the module has been required, it returns a tuple with the module and a function that expands the macro. The function expects the metadata to be attached to the expansion and the arguments of the macro. The appropriate `:remote_macro` compiler tracing event will be emitted if a macro is found (note a `:remote_function` event is not emitted in `:error` cases).

Otherwise returns `:error`.

Options

- `:check_deprecations` - when set to `false`, does not check for deprecations when expanding macros
- `:trace` - when set to `false`, it disables compilation tracers and lexical tracker. This option must only be used by language servers and other tools that need to introspect code without affecting how it is compiled. Disabling tracer inside macros or regular code expansion is extremely discouraged as it blocks the compiler from accurately tracking dependencies

has_var?(env, var)

(since 1.7.0)

```
@spec has_var?(t(), variable()) :: boolean()
```

Checks if a variable belongs to the environment.

Examples

```
iex> x = 13
iex> x
13
iex> Macro.Env.has_var?(__ENV__, {:x, nil})
true
iex> Macro.Env.has_var?(__ENV__, {:unknown, nil})
false
```

in_guard?(env)

```
@spec in_guard?(t()) :: boolean()
```

Returns whether the compilation environment is currently inside a guard.

in_match?(env)

```
@spec in_match?(t()) :: boolean()
```

Returns whether the compilation environment is currently inside a match clause.

location(env)

```
@spec location(t()) :: keyword()
```

Returns a keyword list containing the file and line information as keys.

lookup_alias_as(env, atom)

(since 1.15.0)

```
@spec lookup_alias_as(t(), atom()) :: [atom()]
```

Returns the names of any aliases for the given module or atom.

Examples

```
iex> alias Foo.Bar
iex> Bar
Foo.Bar
iex> Macro.Env.lookup_alias_as(__ENV__, Foo.Bar)
[Elixir.Bar]
iex> alias Foo.Bar, as: Baz
iex> Baz
Foo.Bar
iex> Macro.Env.lookup_alias_as(__ENV__, Foo.Bar)
[Elixir.Bar, Elixir.Baz]
iex> Macro.Env.lookup_alias_as(__ENV__, Unknown)
[]
```

lookup_import(env, name_arity)

(since 1.13.0)

```
@spec lookup_import(t(), name_arity()) :: [{:function | :macro, module()}]
```

Returns the modules from which the given `{name, arity}` was imported.

It returns a list of two element tuples in the shape of `{:function | :macro, module}`. The elements in the list are in no particular order and the order is not guaranteed.

> **Use only for introspection**
>
> This function does not emit compiler tracing events, which may block the compiler from correctly tracking dependencies. Use this function for reflection purposes but to do not use it to expand imports into qualified calls. Instead, use `expand_import/5`.

Examples

```
iex> Macro.Env.lookup_import(__ENV__, {:duplicate, 2})
[]
iex> import Tuple, only: [duplicate: 2], warn: false
iex> Macro.Env.lookup_import(__ENV__, {:duplicate, 2})
[{:function, Tuple}]
iex> import List, only: [duplicate: 2], warn: false
iex> Macro.Env.lookup_import(__ENV__, {:duplicate, 2})
[{:function, List}, {:function, Tuple}]

iex> Macro.Env.lookup_import(__ENV__, {:def, 1})
[{:macro, Kernel}]
```

prepend_tracer(env, tracer)

(since 1.13.0)

```
@spec prepend_tracer(t(), module()) :: t()
```

Prepend a tracer to the list of tracers in the environment.

Examples

```
Macro.Env.prepend_tracer(__ENV__, MyCustomTracer)
```

prune_compile_info(env)

(since 1.14.0)

```
@spec prune_compile_info(t()) :: t()
```

Prunes compile information from the environment.

This happens when the environment is captured at compilation time, for example, in the module body, and then used to evaluate code after the module has been defined.

required?(env, module)

(since 1.13.0)

```
@spec required?(t(), module()) :: boolean()
```

Returns `true` if the given module has been required.

Examples

```
iex> Macro.Env.required?(__ENV__, Integer)
false
iex> require Integer, warn: false
iex> Macro.Env.required?(__ENV__, Integer)
true

iex> Macro.Env.required?(__ENV__, Kernel)
true
```

stacktrace(env)

```
@spec stacktrace(t()) :: list()
```

Returns the environment stacktrace.

to_guard(env)

(since 1.17.0)

```
@spec to_guard(t()) :: t()
```

Returns an environment in the guard context.

to_match(env)

```
@spec to_match(t()) :: t()
```

Returns an environment in the match context.

vars(env)

(since 1.7.0)

```
@spec vars(t()) :: [variable()]
```

Returns a list of variables in the current environment.

Each variable is identified by a tuple of two elements, where the first element is the variable name as an atom and the second element is its context, which may be an atom or an integer.

Behaviour

This module is deprecated. Use @callback and @macrocallback attributes instead.

Mechanism for handling behaviours.

This module is deprecated. Instead of `defcallback/1` and `defmacrocallback/1`, the `@callback` and `@macrocallback` module attributes can be used respectively. See the documentation for `Module` for more information on these attributes.

Instead of `MyModule.__behaviour__(:callbacks)`, `MyModule.behaviour_info(:callbacks)` can be used. `behaviour_info/1` is documented in `Module`.

Summary

Functions

defcallback(spec) deprecated

Defines a function callback according to the given type specification.

defmacrocallback(spec) deprecated

Defines a macro callback according to the given type specification.

Functions

defcallback(spec)

(macro)

This macro is deprecated. Use the @callback module attribute instead.

Defines a function callback according to the given type specification.

defmacrocallback(spec)

(macro)

This macro is deprecated. Use the @macrocallback module attribute instead.

Defines a macro callback according to the given type specification.

Dict

This module is deprecated. Use Map or Keyword modules instead.

Generic API for dictionaries.

If you need a general dictionary, use the `Map` module. If you need to manipulate keyword lists, use `Keyword`.

To convert maps into keywords and vice-versa, use the `new` function in the respective modules.

Summary

Types

key()
t()
value()

Functions

delete(dict, key) deprecated
drop(dict, keys) deprecated
empty(dict) deprecated
equal?(dict1, dict2) deprecated
fetch(dict, key) deprecated
fetch!(dict, key) deprecated
get(dict, key, default \\ nil) deprecated
get_and_update(dict, key, fun) deprecated

get_lazy(dict, key, fun) deprecated
has_key?(dict, key) deprecated
keys(dict) deprecated
merge(dict1, dict2) deprecated
merge(dict1, dict2, fun) deprecated
pop(dict, key, default \\ nil) deprecated
pop_lazy(dict, key, fun) deprecated
put(dict, key, val) deprecated
put_new(dict, key, val) deprecated
put_new_lazy(dict, key, fun) deprecated
size(dict) deprecated
split(dict, keys) deprecated
take(dict, keys) deprecated
to_list(dict) deprecated
update(dict, key, default, fun) deprecated
update!(dict, key, fun) deprecated
values(dict) deprecated

Types

key()

```
@type key() :: any()
```

t()

```
@type t() :: list() | map()
```

value()

```
@type value() :: any()
```

Functions

delete(dict, key)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec delete(t(), key()) :: t()
```

drop(dict, keys)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec drop(t(), [key()]) :: t()
```

empty(dict)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec empty(t()) :: t()
```

equal?(dict1, dict2)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec equal?(t(), t()) :: boolean()
```

fetch(dict, key)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec fetch(t(), key()) :: value()
```

fetch!(dict, key)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec fetch!(t(), key()) :: value()
```

get(dict, key, default \\ nil)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec get(t(), key(), value()) :: value()
```

get_and_update(dict, key, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec get_and_update(t(), key(), (value() -> {value(),
value()})) :: {value(), t()}
```

get_lazy(dict, key, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec get_lazy(t(), key(), (-> value())) :: value()
```

has_key?(dict, key)

This function is deprecated. Use the Map module for working with

maps or the Keyword module for working with keyword lists.

```
@spec has_key?(t(), key()) :: boolean()
```

keys(dict)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec keys(t()) :: [key()]
```

merge(dict1, dict2)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec merge(t(), t()) :: t()
```

merge(dict1, dict2, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec merge(t(), t(), (key(), value(), value() -> value()))
:: t()
```

pop(dict, key, default \\ nil)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec pop(t(), key(), value()) :: {value(), t()}
```

pop_lazy(dict, key, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec pop_lazy(t(), key(), (-> value())) :: {value(), t()}
```

put(dict, key, val)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec put(t(), key(), value()) :: t()
```

put_new(dict, key, val)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec put_new(t(), key(), value()) :: t()
```

put_new_lazy(dict, key, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec put_new_lazy(t(), key(), (-> value())) :: t()
```

size(dict)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec size(t()) :: non_neg_integer()
```

split(dict, keys)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec split(t(), [key()]) :: {t(), t()}
```

take(dict, keys)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec take(t(), [key()]) :: t()
```

to_list(dict)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec to_list(t()) :: list()
```

update(dict, key, default, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec update(t(), key(), value(), (value() -> value())) ::
t()
```

update!(dict, key, fun)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec update!(t(), key(), (value() -> value())) :: t()
```

values(dict)

This function is deprecated. Use the Map module for working with maps or the Keyword module for working with keyword lists.

```
@spec values(t()) :: [value()]
```

GenEvent behaviour

This behaviour is deprecated. Use Erlang/OTP's :gen_event module instead.

An event manager with event handlers behaviour.

If you are interested in implementing an event manager, please read the "Alternatives" section below. If you have to implement an event handler to integrate with an existing system, such as Elixir's Logger, please use `:gen_event` instead.

Alternatives

There are a few suitable alternatives to replace GenEvent. Each of them can be the most beneficial based on the use case.

Supervisor and GenServers

One alternative to GenEvent is a very minimal solution consisting of using a supervisor and multiple GenServers started under it. The supervisor acts as the "event manager" and the children GenServers act as the "event handlers". This approach has some shortcomings (it provides no back-pressure for example) but can still replace GenEvent for low-profile usages of it. This blog post by José Valim has more detailed information on this approach.

GenStage

If the use case where you were using GenEvent requires more complex logic, GenStage provides a great alternative. GenStage is an external

Elixir library maintained by the Elixir team; it provides a tool to implement systems that exchange events in a demand-driven way with built-in support for back-pressure. See the GenStage documentation for more information.

`:gen_event`

If your use case requires exactly what GenEvent provided, or you have to integrate with an existing `:gen_event`-based system, you can still use the `:gen_event` Erlang module.

Summary

Types

handler()
manager()
name()
on_start()
options()

Callbacks

code_change(old_vsn, state, extra)
handle_call(request, state)
handle_event(event, state)
handle_info(msg, state)
init(args)
terminate(reason, state)

Types

handler()

```
@type handler() :: atom() | {atom(), term()}
```

manager()

```
@type manager() :: pid() | name() | {atom(), node()}
```

name()

```
@type name() :: atom() | {:global, term()} | {:via, module(),
term()}
```

on_start()

```
@type on_start() :: {:ok, pid()} | {:error,
{:already_started, pid()}}
```

options()

```
@type options() :: [{:name, name()}]
```

Callbacks

code_change(old_vsn, state, extra)

```
@callback code_change(old_vsn, state :: term(), extra ::
term()) ::
  {:ok, new_state :: term()}
when old_vsn: term() | {:down, term()}
```

handle_call(request, state)

```
@callback handle_call(request :: term(), state :: term()) ::
  {:ok, reply, new_state}
  | {:ok, reply, new_state, :hibernate}
  | {:remove_handler, reply}
when reply: term(), new_state: term()
```

handle_event(event, state)

```
@callback handle_event(event :: term(), state :: term()) ::
  {:ok, new_state} | {:ok, new_state, :hibernate} |
:remove_handler
when new_state: term()
```

handle_info(msg, state)

```
@callback handle_info(msg :: term(), state :: term()) ::
  {:ok, new_state} | {:ok, new_state, :hibernate} |
```

```
:remove_handler
when new_state: term()
```

init(args)

```
@callback init(args :: term()) ::
  {:ok, state} | {:ok, state, :hibernate} | {:error, reason
:: term()}
when state: term()
```

terminate(reason, state)

```
@callback terminate(reason, state :: term()) :: term()
when reason:
       :stop | {:stop, term()} | :remove_handler | {:error,
term()} | term()
```

HashDict

This module is deprecated. Use Map instead.

Tuple-based HashDict implementation.

This module is deprecated. Use the `Map` module instead.

Summary

Types

t()

Functions

delete(dict, key) deprecated
drop(dict, keys) deprecated
equal?(dict1, dict2) deprecated
fetch(hash_dict, key) deprecated
fetch!(dict, key) deprecated
get(dict, key, default \\ nil) deprecated
get_and_update(dict, key, fun) deprecated
get_lazy(dict, key, fun) deprecated
has_key?(dict, key) deprecated
keys(dict) deprecated
merge(dict1, dict2, fun \\ fn _k, _v1, v2 -> v2 end) deprecated
new() deprecated

Creates a new empty dict.

pop(dict, key, default \\ nil) deprecated
pop_lazy(dict, key, fun) deprecated
put(hash_dict, key, value) deprecated
put_new(dict, key, value) deprecated
put_new_lazy(dict, key, fun) deprecated
size(hash_dict) deprecated
split(dict, keys) deprecated
take(dict, keys) deprecated
to_list(dict) deprecated
update(dict, key, default, fun) deprecated
update!(dict, key, fun) deprecated
values(dict) deprecated

Types

t()

```
@opaque t()
```

Functions

delete(dict, key)

This function is deprecated. Use maps and the Map module instead.

drop(dict, keys)

This function is deprecated. Use maps and the Map module instead.

equal?(dict1, dict2)

This function is deprecated. Use maps and the Map module instead.

fetch(hash_dict, key)

This function is deprecated. Use maps and the Map module instead.

fetch!(dict, key)

This function is deprecated. Use maps and the Map module instead.

get(dict, key, default \\ nil)

This function is deprecated. Use maps and the Map module instead.

get_and_update(dict, key, fun)

This function is deprecated. Use maps and the Map module instead.

get_lazy(dict, key, fun)

This function is deprecated. Use maps and the Map module instead.

has_key?(dict, key)

This function is deprecated. Use maps and the Map module instead.

keys(dict)

This function is deprecated. Use maps and the Map module instead.

merge(dict1, dict2, fun \\ fn _k, _v1, v2 -> v2 end)

This function is deprecated. Use maps and the Map module instead.

new()

This function is deprecated. Use maps and the Map module instead.

```
@spec new() :: Dict.t()
```

Creates a new empty dict.

pop(dict, key, default \\ nil)

This function is deprecated. Use maps and the Map module instead.

pop_lazy(dict, key, fun)

This function is deprecated. Use maps and the Map module instead.

put(hash_dict, key, value)

This function is deprecated. Use maps and the Map module instead.

put_new(dict, key, value)

This function is deprecated. Use maps and the Map module instead.

put_new_lazy(dict, key, fun)

This function is deprecated. Use maps and the Map module instead.

size(hash_dict)

This function is deprecated. Use maps and the Map module instead.

split(dict, keys)

This function is deprecated. Use maps and the Map module instead.

take(dict, keys)

This function is deprecated. Use maps and the Map module instead.

to_list(dict)

This function is deprecated. Use maps and the Map module instead.

update(dict, key, default, fun)

This function is deprecated. Use maps and the Map module instead.

update!(dict, key, fun)

This function is deprecated. Use maps and the Map module instead.

values(dict)

This function is deprecated. Use maps and the Map module instead.

HashSet

This module is deprecated. Use MapSet instead.

Tuple-based HashSet implementation.

This module is deprecated. Use the `MapSet` module instead.

Summary

Types

t()

Functions

delete(set, term) deprecated
difference(set1, set2) deprecated
disjoint?(set1, set2) deprecated
equal?(set1, set2) deprecated
intersection(set1, set2) deprecated
member?(hash_set, term) deprecated
new() deprecated
put(hash_set, term) deprecated
size(hash_set) deprecated
subset?(set1, set2) deprecated
to_list(set) deprecated
union(set1, set2) deprecated

Types

t()

```
@opaque t()
```

Functions

delete(set, term)

This function is deprecated. Use the MapSet module instead.

difference(set1, set2)

This function is deprecated. Use the MapSet module instead.

disjoint?(set1, set2)

This function is deprecated. Use the MapSet module instead.

equal?(set1, set2)

This function is deprecated. Use the MapSet module instead.

intersection(set1, set2)

This function is deprecated. Use the MapSet module instead.

member?(hash_set, term)

This function is deprecated. Use the MapSet module instead.

new()

This function is deprecated. Use the MapSet module instead.

```
@spec new() :: Set.t()
```

put(hash_set, term)

This function is deprecated. Use the MapSet module instead.

size(hash_set)

This function is deprecated. Use the MapSet module instead.

subset?(set1, set2)

This function is deprecated. Use the MapSet module instead.

to_list(set)

This function is deprecated. Use the MapSet module instead.

union(set1, set2)

This function is deprecated. Use the MapSet module instead.

Set

This module is deprecated. Use MapSet instead.

Generic API for sets.

This module is deprecated, use the `MapSet` module instead.

Summary

Types

t()
value()
values()

Functions

delete(set, value) deprecated
difference(set1, set2) deprecated
disjoint?(set1, set2) deprecated
empty(set) deprecated
equal?(set1, set2) deprecated
intersection(set1, set2) deprecated
member?(set, value) deprecated
put(set, value) deprecated
size(set) deprecated
subset?(set1, set2) deprecated
to_list(set) deprecated
union(set1, set2) deprecated

Types

t()

```
@type t() :: map()
```

value()

```
@type value() :: any()
```

values()

```
@type values() :: [value()]
```

Functions

delete(set, value)

This function is deprecated. Use the MapSet module for working with sets.

difference(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

disjoint?(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

empty(set)

This function is deprecated. Use the MapSet module for working with sets.

equal?(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

intersection(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

member?(set, value)

This function is deprecated. Use the MapSet module for working with sets.

put(set, value)

This function is deprecated. Use the MapSet module for working with sets.

size(set)

This function is deprecated. Use the MapSet module for working with sets.

subset?(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

to_list(set)

This function is deprecated. Use the MapSet module for working with sets.

union(set1, set2)

This function is deprecated. Use the MapSet module for working with sets.

Supervisor.Spec

This module is deprecated. Use the new child specifications outlined in the Supervisor module instead.

Outdated functions for building child specifications.

The functions in this module are deprecated and they do not work with the module-based child specs introduced in Elixir v1.5. Please see the `Supervisor` documentation instead.

Convenience functions for defining supervisor specifications.

Example

By using the functions in this module one can specify the children to be used under a supervisor, started with `Supervisor.start_link/2`:

```
import Supervisor.Spec

children = [
  worker(MyWorker, [arg1, arg2, arg3]),
  supervisor(MySupervisor, [arg1])
]

Supervisor.start_link(children, strategy: :one_for_one)
```

Sometimes, it may be handy to define supervisors backed by a module:

```elixir
defmodule MySupervisor do
  use Supervisor

  def start_link(arg) do
    Supervisor.start_link(__MODULE__, arg)
  end

  def init(arg) do
    children = [
      worker(MyWorker, [arg], restart: :temporary)
    ]

    supervise(children, strategy: :simple_one_for_one)
  end
end
```

Note that in this case we don't have to explicitly import `Supervisor.Spec` since `use Supervisor` automatically does so. Defining a module-based supervisor can be useful, for example, to perform initialization tasks in the `Supervisor.init/1` callback.

Supervisor and worker options

In the example above, we defined specs for workers and supervisors. These specs (both for workers as well as supervisors) accept the following options:

- `:id` - a name used to identify the child specification internally by the supervisor; defaults to the given module name for the child worker/supervisor

- `:function` - the function to invoke on the child to start it

- `:restart` - an atom that defines when a terminated child process should be restarted (see the "Restart values" section below)
- `:shutdown` - an atom that defines how a child process should be terminated (see the "Shutdown values" section below)
- `:modules` - it should be a list with one element `[module]`, where module is the name of the callback module only if the child process is a `Supervisor` or `GenServer`; if the child process is a `GenEvent`, `:modules` should be `:dynamic`

Restart values (:restart)

The following restart values are supported in the `:restart` option:

- `:permanent` - the child process is always restarted
- `:temporary` - the child process is never restarted (not even when the supervisor's strategy is `:rest_for_one` or `:one_for_all`)
- `:transient` - the child process is restarted only if it terminates abnormally, i.e., with an exit reason other than `:normal`, `:shutdown` or `{:shutdown, term}`

Note that supervisor that reached maximum restart intensity will exit with `:shutdown` reason. In this case the supervisor will only restart if its child specification was defined with the `:restart` option set to `:permanent` (the default).

Shutdown values (`:shutdown`)

The following shutdown values are supported in the `:shutdown` option:

- `:brutal_kill` - the child process is unconditionally terminated using `Process.exit(child, :kill)`
- `:infinity` - if the child process is a supervisor, this is a mechanism to give the subtree enough time to shut down; it can also be used with workers with care
- a non-negative integer - the amount of time in milliseconds that the supervisor tells the child process to terminate by calling `Process.exit(child, :shutdown)` and then waits for an exit signal back. If no exit signal is received within the specified time, the child process is unconditionally terminated using `Process.exit(child, :kill)`

Summary

Types

child_id()

Supported ID values

modules()

Supported module values

restart()

Supported restart values

shutdown()

Supported shutdown values

spec()

The supervisor specification
strategy()

Supported strategies

worker()

Supported worker values

Functions

supervise(children, options) deprecated

Receives a list of `children` (workers or supervisors) to supervise and a set of `options`.

supervisor(module, args, options \\ []) deprecated

Defines the given `module` as a supervisor which will be started with the given arguments.

worker(module, args, options \\ []) deprecated

Defines the given `module` as a worker which will be started with the given arguments.

Types

child_id()

```
@type child_id() :: term()
```

Supported ID values

modules()

```
@type modules() :: :dynamic | [module()]
```

Supported module values

restart()

```
@type restart() :: :permanent | :transient | :temporary
```

Supported restart values

shutdown()

```
@type shutdown() :: timeout() | :brutal_kill
```

Supported shutdown values

spec()

```
@type spec() ::
  {child_id(), start_fun :: {module(), atom(), [term()]},
restart(), shutdown(),
   worker(), modules()}
```

The supervisor specification

strategy()

```
@type strategy() :: :simple_one_for_one | :one_for_one |
:one_for_all | :rest_for_one
```

Supported strategies

worker()

```
@type worker() :: :worker | :supervisor
```

Supported worker values

Functions

supervise(children, options)

This function is deprecated. Use the new child specifications outlined in the Supervisor module instead.

```
@spec supervise([spec()],
  strategy: strategy(),
  max_restarts: non_neg_integer(),
  max_seconds: pos_integer()
) :: {:ok, tuple()}
```

Receives a list of `children` (workers or supervisors) to supervise and a set of `options`.

Returns a tuple containing the supervisor specification. This tuple can be used as the return value of the `Supervisor.init/1` callback when implementing a module-based supervisor.

Examples

```
supervise(children, strategy: :one_for_one)
```

Options

- `:strategy` - the restart strategy option. It can be either `:one_for_one`, `:rest_for_one`, `:one_for_all`, or `:simple_one_for_one`. You can learn more about strategies in the `Supervisor` module docs.
- `:max_restarts` - the maximum number of restarts allowed in a time frame. Defaults to `3`.
- `:max_seconds` - the time frame in which `:max_restarts` applies. Defaults to `5`.

The `:strategy` option is required and by default a maximum of 3 restarts is allowed within 5 seconds. Check the `Supervisor` module for a detailed description of the available strategies.

supervisor(module, args, options \\ [])

This function is deprecated. Use the new child specifications outlined in the Supervisor module instead.

```
@spec supervisor(module(), [term()],
  restart: restart(),
  shutdown: shutdown(),
  id: term(),
  function: atom(),
  modules: modules()
) :: spec()
```

Defines the given `module` as a supervisor which will be started with the given arguments.

```
supervisor(module, [], restart: :permanent)
```

By default, the function `start_link` is invoked on the given module. Overall, the default values for the options are:

```
[
  id: module,
  function: :start_link,
  restart: :permanent,
  shutdown: :infinity,
  modules: [module]
]
```

See the "Supervisor and worker options" section in the `Supervisor.Spec` module for more information on the available options.

worker(module, args, options \\ [])

This function is deprecated. Use the new child specifications outlined

in the Supervisor module instead.

```
@spec worker(module(), [term()],
  restart: restart(),
  shutdown: shutdown(),
  id: term(),
  function: atom(),
  modules: modules()
) :: spec()
```

Defines the given `module` as a worker which will be started with the given arguments.

```
worker(ExUnit.Runner, [], restart: :permanent)
```

By default, the function `start_link` is invoked on the given module. Overall, the default values for the options are:

```
[
  id: module,
  function: :start_link,
  restart: :permanent,
  shutdown: 5000,
  modules: [module]
]
```

See the "Supervisor and worker options" section in the `Supervisor.Spec` module for more information on the available options.

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Integer</samp><samp class="o">.</samp><samp class="n">to_string</samp><samp class="p" data-group-id="group-1">(</samp><samp class="mf">1.0</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w">

</samp><samp class="gt">** (ArgumentError) errors were found at the given arguments:</samp><samp class="w">

</samp><samp class="n">...</samp></code>

`ArgumentError` exceptions have a single field, `:message` (a `String.t/0`), which is public and can be accessed freely when reading or creating `ArgumentError` exceptions.

ArithmeticError exception

An exception raised on invalid arithmetic operations.

For example, this exception is raised if you divide by `0`:

```
iex> 1 / 0
** (ArithmeticError) bad argument in arithmetic
expression
```

BadArityError exception

An exception raised when a function is called with the wrong number of arguments.

For example:

```
my_function = fn x, y -> x + y end
my_function.(42)
** (BadArityError) #Function<41.39164016/2 in
:erl_eval.expr/6> with arity 2 called with 1 argument
(42)
```

BadBooleanError exception

An exception raised when a boolean is expected, but something else was given.

This exception is raised by `and` and `or` when the first argument is not a boolean:

```
iex> 123 and true
** (BadBooleanError) expected a boolean on left-side of
"and", got:
...
```

BadFunctionError exception

An exception raised when a function is expected, but something else was given.

For example:

```
iex> value = "hello"
iex> value.()
** (BadFunctionError) expected a function, got: "hello"
```

BadMapError exception

An exception raised when a map is expected, but something else was given.

For example:

```
iex> value = "hello"
iex> %{value | key: "value"}
** (BadMapError) expected a map, got:
...
```

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">case</samp><samp class="w"> </samp><samp class="nc">System</samp><samp class="o">.</samp><samp class="n">unique_integer</samp><samp class="p" data-group-id="group-1">(</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w"> </samp><samp class="k" data-group-id="group-2">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="n">bin</samp><samp class="w"> </samp><samp class="ow">when</samp><samp class="w"> </samp><samp class="n">is_binary</samp><samp class="p" data-group-id="group-3">(</samp><samp class="n">bin</samp><samp class="p" data-group-id="group-3">)</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="ss">:oops</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="ss">:ok</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="ss">:neither_this_one</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-2">end</samp><samp class="w">

</samp><samp class="gt">** (CaseClauseError) no case clause matching:
</samp><samp class="w">

</samp><samp class="n">...</samp></code>

The following fields of this exception are public and can be accessed freely:

- `:term` (`term/0`) - the term that did not match any of the clauses

Code.LoadError exception

An exception raised when a file cannot be loaded.

This is typically raised by functions in the `Code` module, for example:

```
Code.require_file("missing_file.exs")
** (Code.LoadError) could not load missing_file.exs.
Reason: enoent
```

The following fields of this exception are public and can be accessed freely:

- `:file` (`String.t/0`) - the file name
- `:reason` (`term/0`) - the reason why the file could not be loaded

CompileError exception

An exception raised when there's an error when compiling code.

For example: `1 = y ** (CompileError) iex:1: undefined variable "y"`

The following fields of this exceptions are public and can be accessed freely:

- `:file` (`Path.t/0` or `nil`) - the file where the error occurred, or `nil` if the error occurred in code that did not come from a file
- `:line` (`non_neg_integer/0`) - the line where the error occurred
- `:description` (`String.t/0`) - a description of the compile error

This is mostly raised by Elixir tooling when compiling and evaluating code.

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">cond</samp><samp class="w"> </samp><samp class="k" data-group-id="group-1">do</samp>
<samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="mi">1</samp><samp class="w"> </samp><samp class="o">+</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="w"> </samp><samp class="o">==
</samp><samp class="w"> </samp><samp class="mi">3</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w">
</samp><samp class="ss">:woah</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="no">nil</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="s">"yeah this won't happen"</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w">

</samp><samp class="gt">** (CondClauseError) no cond clause evaluated to a truthy value</samp></code>

Enum.EmptyError exception

An exception that is raised when something expects a non-empty enumerable but finds an empty one.

For example:

```
iex> Enum.min([])
** (Enum.EmptyError) empty error
```

Enum.OutOfBoundsError exception

An exception that is raised when a function expects an enumerable to have a certain size but finds that it is too small.

For example:

```
iex> Enum.fetch!([1, 2, 3], 5)
** (Enum.OutOfBoundsError) out of bounds error at
position 5 when traversing enumerable [1, 2, 3]
```

Summary

Functions

message(index, enumerable)

Functions

message(index, enumerable)

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">:erlang</samp><samp class="o">.</samp><samp class="n">error</samp><samp class="p" data-group-id="group-1">(</samp><samp class="ss">:some_invalid_error</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w"> </samp><samp class="gt">** (ErlangError) Erlang error: :some_invalid_error</samp></code>

File.CopyError exception

An exception that is raised when copying a file fails.

For example, this exception is raised when trying to copy to file or directory that isn't present: `iex> File.cp_r!("non_existent", "source_dir/subdir") ** (File.CopyError) could not copy recursively from "non_existent" to "source_dir/subdir". non_existent: no such file or directory`

The following fields of this exception are public and can be accessed freely:

- `:source` (`Path.t/0`) - the source path
- `:destination` (`Path.t/0`) - the destination path
- `:reason` (`File.posix/0`) - the reason why the file could not be copied

File.Error exception

An exception that is raised when a file operation fails.

For example, this exception is raised, when trying to read a non existent file: `iex> File.read!("nonexistent_file.txt") ** (File.Error) could not read file "nonexistent_file.txt": no such file or directory`

The following fields of this exception are public and can be accessed freely:

- `:path` (`Path.t/0`) - the path of the file that caused the error
- `:reason` (`File.posix/0`) - the reason for the error

File.LinkError exception

An exception that is raised when linking a file fails.

For example, this exception is raised when trying to link to file that isn't present: `iex> File.ln!("existing.txt", "link.txt") ** (File.LinkError) could not create hard link from "link.txt" to "existing.txt": no such file or directory`

The following fields of this exception are public and can be accessed freely:

- `:existing` (`Path.t/0`) - the existing file to link
- `:new` (`Path.t/0`) - the link destination
- `:reason` (`File.posix/0`) - the reason why the file could not be linked

File.RenameError exception

An exception that is raised when renaming a file fails.

For example, this exception is raised when trying to rename a file that isn't present: iex> File.rename!("source.txt", "target.txt") ** (File.RenameError) could not rename from "source.txt" to "target.txt": no such file or directory The following fields of this exception are public and can be accessed freely:

- `:source` (`Path.t/0`) - the source path
- `:destination` (`Path.t/0`) - the destination path
- `:reason` (`File.posix/0`) - the reason why the file could not be renamed

FunctionClauseError exception

An exception raised when a function call doesn't match any defined clause.

For example:

```
iex> List.duplicate(:ok, -3)
** (FunctionClauseError) no function clause matching in
List.duplicate/2
```

The following fields of this exception are public and can be accessed freely:

- `:module` (`module/0`) - the module name
- `:function` (`atom/0`) - the function name
- `:arity` (`non_neg_integer/0`) - the arity of the function

IO.StreamError exception

Inspect.Error exception

Raised when a struct cannot be inspected.

JSON.DecodeError exception

The exception raised by `JSON.decode!/1`.

Kernel.TypespecError exception

An exception raised when there's an error in a typespec.

For example, if your typespec definition points to an invalid type, you get an exception: `@type my_type :: intger()`

will raise:

```
** (Kernel.TypespecError) type intger/0 undefined
```

The following fields of this exceptions are public and can be accessed freely:

- `:file` (`Path.t/0` or `nil`) - the file where the error occurred, or `nil` if the error occurred in code that did not come from a file
- `:line` (`non_neg_integer/0`) - the line where the error occurred

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="n">map</samp><samp class="w"> </samp><samp class="o">=</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">%{</samp><samp class="ss">name</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="s">"Alice"</samp><samp class="p">,</samp> <samp class="w"> </samp><samp class="ss">age</samp><samp class="p">:</samp><samp class="w"> </samp><samp class="mi">25</samp><samp class="p" data-group-id="group-1">} </samp><samp class="w">

</samp><samp class="gp unselectable">iex> </samp><samp class="nc">Map</samp><samp class="o">.</samp><samp class="n">fetch!</samp><samp class="p" data-group-id="group-2">(</samp><samp class="n">map</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="ss">:first_name</samp><samp class="p" data-group-id="group-2">)</samp><samp class="w">

</samp><samp class="gt">** (KeyError) key :first_name not found in: </samp><samp class="w">

</samp><samp class="n">...</samp></code>

The following fields of this exception are public and can be accessed freely:

- `:term` (`term/0`) - the data structure that was searched
- `:key` (`term/0`) - the key that was not found

MatchError exception

An exception raised when a pattern match (`=/2`) fails.

For example:

```
iex> [_ | _] = []
** (MatchError) no match of right hand side value:
...
```

The following fields of this exception are public and can be accessed freely:

- `:term` (`term/0`) - the term that did not match the pattern

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Code</samp><samp class="o">.</samp><samp class="n">eval_string</samp><samp class="p" data-group-id="group-1">(</samp><samp class="s">"[1, 2, 3}"</samp><samp class="p" data-group-id="group-1">)</samp><samp class="w"> </samp><samp class="gt">** (MismatchedDelimiterError) mismatched delimiter found on nofile:1:9:</samp><samp class="w"> </samp><samp class="n">...</samp></code>

The following fields of this exceptions are public and can be accessed freely:

- `:file` (`Path.t/0` or `nil`) - the file where the error occurred, or `nil` if the error occurred in code that did not come from a file
- `:line` - the line for the opening delimiter
- `:column` - the column for the opening delimiter
- `:end_line` - the line for the mismatched closing delimiter
- `:end_column` - the column for the mismatched closing delimiter
- `:opening_delimiter` - an atom representing the opening delimiter
- `:closing_delimiter` - an atom representing the mismatched closing delimiter
- `:expected_delimiter` - an atom representing the closing delimiter
- `:description` - a description of the mismatched delimiter error

MissingApplicationsError exception

An exception that is raised when an application depends on one or more missing applications.

This exception is used by Mix and other tools. It can also be used by library authors when their library only requires an external application (like a dependency) for a subset of features.

The fields of this exception are public. See `t/0`.

Available since v1.18.0.

Examples

```
unless Application.spec(:plug, :vsn) do
  raise MissingApplicationsError,
    description: "application :plug is required for
testing Plug-related functionality",
    apps: [{:plug, "~> 1.0"}]
end
```

Summary

Types

t()

Types

t()

(since 1.18.0)

```
@type t() :: %MissingApplicationsError{
  __exception__: term(),
  apps: [{Application.app(), Version.requirement()}, ...],
  description: String.t()
}
```

OptionParser.ParseError exception

An exception raised when parsing option fails.

For example, see `OptionParser.parse!/2`.

Protocol.UndefinedError exception

An exception raised when a protocol is not implemented for a given value.

For example:

```
iex> Enum.at("A string!", 0)
** (Protocol.UndefinedError) protocol Enumerable not
implemented for BitString
...
```

The following fields of this exception are public and can be accessed freely:

- `:protocol` (`module/0`) - the protocol that is not implemented
- `:value` (`term/0`) - the value that does not implement the protocol

Regex.CompileError exception

An exception raised when a regular expression could not be compiled.

RuntimeError exception

An exception for a generic runtime error.

This is the exception that `raise/1` raises when you pass it only a string as a message:

```
iex> raise "oops!"
** (RuntimeError) oops!
```

You should use this exceptions sparingly, since most of the time it might be better to define your own exceptions specific to your application or library. Sometimes, however, there are situations in which you don't expect a condition to happen, but you want to give a meaningful error message if it does. In those cases, `RuntimeError` can be a good choice.

Fields

`RuntimeError` exceptions have a single field, `:message` (a `String.t/0`), which is public and can be accessed freely when reading or creating `RuntimeError` exceptions.

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="nc">Code</samp><samp class="o">.</samp><samp class="n">eval_string</samp><samp class="p" data-group-id="group-1">(</samp><samp class="s">"5 + 5h"</samp> <samp class="p" data-group-id="group-1">)</samp><samp class="w"> </samp><samp class="gt">** (SyntaxError) invalid syntax found on nofile:1:5:</samp><samp class="w"> </samp><samp class="n">... </samp></code>

The following fields of this exceptions are public and can be accessed freely:

- `:file` (`Path.t/0` or `nil`) - the file where the error occurred, or `nil` if the error occurred in code that did not come from a file
- `:line` - the line where the error occurred
- `:column` - the column where the error occurred
- `:description` - a description of the syntax error

System.EnvError exception

An exception raised when a system environment variable is not set.

For example, see `System.fetch_env!/1`.

SystemLimitError exception

An exception raised when a system limit has been reached.

For example, this can happen if you try to create an atom that is too large:

```
iex> String.to_atom(String.duplicate("a", 100_000))
** (SystemLimitError) a system limit has been reached
```

TokenMissingError exception

An exception raised when a token is missing when parsing code.

For example: `iex> Code.eval_string("[1, 2, 3") **`
`(TokenMissingError) token missing on nofile:1:9: ...`

The following fields of this exceptions are public and can be accessed freely:

- `:file` (`Path.t/0` or `nil`) - the file where the error occurred, or `nil` if the error occurred in code that did not come from a file
- `:line` - the line for the opening delimiter
- `:column` - the column for the opening delimiter
- `:end_line` - the line for the end of the string
- `:end_column` - the column for the end of the string
- `:opening_delimiter` - an atom representing the opening delimiter
- `:expected_delimiter` - an atom representing the expected delimiter
- `:description` - a description of the missing token error

This is mostly raised by Elixir tooling when compiling and evaluating code.

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">try</samp><samp class="w"> </samp><samp class="k" data-group-id="group-1">do</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="w"> </samp><samp class="ss">:ok</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">rescue</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="w"> </samp><samp class="n">e</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="n">e</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">else</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="w"> </samp><samp class="c1"># :ok -> :ok is missing</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="w"> </samp><samp class="ss">:not_ok</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="ss">:not_ok</samp><samp class="w"> </samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-1">end</samp><samp class="w"> </samp><samp class="gt">** (TryClauseError) no try clause matching:</samp><samp class="w">

</samp><samp class="n">...</samp></code>

The following fields of this exception are public and can be accessed freely:

- `:term` (`term/0`) - the term that did not match any of the clauses

URI.Error exception

An exception raised when an error occurs when a `URI` is invalid.

For example, see `URI.new!/1`.

UndefinedFunctionError exception

An exception raised when a function is invoked that is not defined.

For example:

```
# Let's use apply/3 as otherwise Elixir emits a compile-time warning
iex> apply(String, :non_existing_fun, ["hello"])
** (UndefinedFunctionError) function String.non_existing_fun/1 is undefined or private
```

The following fields of this exception are public and can be accessed freely:

- `:module` (`module/0`) - the module name
- `:function` (`atom/0`) - the function name
- `:arity` (`non_neg_integer/0`) - the arity of the function

UnicodeConversionError exception

An exception raised when converting data to or from Unicode.

For example:

```
iex> String.to_charlist(<<0xFF>>)
** (UnicodeConversionError) invalid encoding starting at
<<255>>
```

Version.InvalidRequirementError exception

An exception raised when a version requirement is invalid.

For example, see `Version.parse_requirement!/1`.

Version.InvalidVersionError exception

An exception raised when a version is invalid.

For example, see `Version.parse!/1`.

<code class="makeup elixir" translate="no"><samp class="gp unselectable">iex> </samp><samp class="k">with</samp><samp class="w"> </samp><samp class="p" data-group-id="group-1">{</samp><samp class="ss">:ok</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">1</samp><samp class="p" data-group-id="group-1">}</samp><samp class="w"> </samp><samp class="o"><-</samp><samp class="w"> </samp><samp class="p" data-group-id="group-2">{</samp><samp class="ss">:ok</samp><samp class="p">,</samp><samp class="w"> </samp><samp class="mi">2</samp><samp class="p" data-group-id="group-2">}</samp><samp class="w"> </samp><samp class="k" data-group-id="group-3">do</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="ss">:woah</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-3">else</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="ss">:error</samp><samp class="w"> </samp><samp class="o">-></samp><samp class="w"> </samp><samp class="ss">:error</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="w">
</samp><samp class="p" data-group-id="group-4">{</samp><samp class="ss">:error</samp><samp class="p">,</samp><samp class="w">
</samp><samp class="bp">_</samp><samp class="p" data-group-id="group-4">}</samp><samp class="w"> </samp><samp class="o">->
</samp><samp class="w"> </samp><samp class="ss">:error</samp><samp class="w">

</samp><samp class="gp unselectable">...> </samp><samp class="k" data-group-id="group-3">end</samp><samp class="w">

</samp><samp class="gt">** (WithClauseError) no with clause matching:
</samp><samp class="w">

</samp><samp class="n">...</samp></code>

The following fields of this exception are public and can be accessed freely:

- `:term` (`term/0`) - the term that did not match any of the clauses